Java™ How to Program



NINTH EDITION

PAUL DEITEL
HARVEY DEITEL

16

Strings, Characters and Regular Expressions



The chief defect of Henry King
Was chewing little bits of string.
—Hilaire Belloc

Vigorous writing is concise. A sentence should contain no unnecessary words, a paragraph no unnecessary sentences.
—William Strunk, Jr.

I have made this letter longer than usual, because I lack the time to make it short.
—Blaise Pascal

Objectives

In this chapter you'll learn:

- To create and manipulate immutable character-string objects of class `String`.
- To create and manipulate mutable character-string objects of class `StringBuilder`.
- To create and manipulate objects of class `Character`.
- To break a `String` object into tokens using `String` method `split`.
- To use regular expressions to validate `String` data entered into an application.

Outline

16.1 Introduction
16.2 Fundamentals of Characters and Strings
16.3 Class `String`
16.3.1 `String` Constructors
16.3.2 `String` Methods `length`, `charAt` and `getChars`
16.3.3 Comparing Strings
16.3.4 Locating Characters and Substrings in Strings
16.3.5 Extracting Substrings from Strings
16.3.6 Concatenating Strings
16.3.7 Miscellaneous `String` Methods
16.3.8 `String` Method `valueOf`
16.4 Class `StringBuilder`
16.4.1 `StringBuilder` Constructors
16.4.2 `StringBuilder` Methods `length`, `capacity`, `setLength` and `ensureCapacity`
16.4.3 `StringBuilder` Methods `charAt`, `setCharAt`, `getChars` and `reverse`
16.4.4 `StringBuilder` `append` Methods
16.4.5 `StringBuilder` Insertion and Deletion Methods
16.5 Class `Character`
16.6 Tokenizing `String`s
16.7 Regular Expressions, Class `Pattern` and Class `Matcher`
16.8 Wrap-Up

Summary | *Self-Review Exercises* | *Answers to Self-Review Exercises* | *Exercises* | *Special Section: Advanced String-Manipulation Exercises* | *Special Section: Challenging String-Manipulation Projects* | *Making a Difference*

16.1 Introduction

This chapter introduces Java's string- and character-processing capabilities. The techniques discussed here are appropriate for validating program input, displaying information to users and other text-based manipulations. They're also appropriate for developing text editors, word processors, page-layout software, computerized typesetting systems and other kinds of text-processing software. We've presented several string-processing capabilities in earlier chapters. This chapter discusses in detail the capabilities of classes `String`, `StringBuilder` and `Character` from the `java.lang` package. These classes provide the foundation for string and character manipulation in Java.

The chapter also discusses regular expressions that provide applications with the capability to validate input. The functionality is located in the `String` class along with classes `Matcher` and `Pattern` located in the `java.util.regex` package.

16.2 Fundamentals of Characters and Strings

Characters are the fundamental building blocks of Java source programs. Every program is composed of a sequence of characters that—when grouped together meaningfully—are interpreted by the Java compiler as a series of instructions used to accomplish a task. A program may contain **character literals**. A character literal is an integer value represented as a character in single quotes. For example, `'z'` represents the integer value of `z`, and `'\n'` represents the integer value of newline. The value of a character literal is the integer value of the character in the **Unicode character set**. Appendix B presents the integer equivalents of the characters in the ASCII character set, which is a subset of Unicode (discussed in Appendix L).

Recall from Section 2.2 that a string is a sequence of characters treated as a single unit. A string may include letters, digits and various **special characters**, such as `+`, `-`, `*`, `/` and `$`.

A string is an object of class `String`. **String literals** (stored in memory as `String` objects) are written as a sequence of characters in double quotation marks, as in:

```
"John Q. Doe"                (a name)
"9999 Main Street"           (a street address)
"Waltham, Massachusetts"     (a city and state)
"(201) 555-1212"             (a telephone number)
```

A string may be assigned to a `String` reference. The declaration

```
String color = "blue";
```

initializes `String` variable `color` to refer to a `String` object that contains the string `"blue"`.



Performance Tip 16.1

To conserve memory, Java treats all string literals with the same contents as a single `String` object that has many references to it.

16.3 Class `String`

Class `String` is used to represent strings in Java. The next several subsections cover many of class `String`'s capabilities.

16.3.1 `String` Constructors

Class `String` provides constructors for initializing `String` objects in a variety of ways. Four of the constructors are demonstrated in the `main` method of Fig. 16.1.

Line 12 instantiates a new `String` using class `String`'s no-argument constructor and assigns its reference to `s1`. The new `String` object contains no characters (i.e., the **empty**

```
1  // Fig. 16.1: StringConstructors.java
2  // String class constructors.
3
4  public class StringConstructors
5  {
6     public static void main( String[] args )
7     {
8        char[] charArray = { 'b', 'i', 'r', 't', 'h', ' ', 'd', 'a', 'y' };
9        String s = new String( "hello" );
10
11       // use String constructors
12       String s1 = new String();
13       String s2 = new String( s );
14       String s3 = new String( charArray );
15       String s4 = new String( charArray, 6, 3 );
16
17       System.out.printf(
18          "s1 = %s\ns2 = %s\ns3 = %s\ns4 = %s\n",
19          s1, s2, s3, s4 ); // display strings
20    } // end main
21 } // end class StringConstructors
```

Fig. 16.1 | `String` class constructors. (Part 1 of 2.)

```
s1 =
s2 = hello
s3 = birth day
s4 = day
```

Fig. 16.1 | `String` class constructors. (Part 2 of 2.)

string, which can also be represented as "") and has a length of 0. Line 13 instantiates a new `String` object using class `String`'s constructor that takes a `String` object as an argument and assigns its reference to `s2`. The new `String` object contains the same sequence of characters as the `String` object `s` that's passed as an argument to the constructor.



Software Engineering Observation 16.1

*It's not necessary to copy an existing `String` object. `String` objects are **immutable**—their character contents cannot be changed after they're created, because class `String` does not provide methods that allow the contents of a `String` object to be modified.*

Line 14 instantiates a new `String` object and assigns its reference to `s3` using class `String`'s constructor that takes a `char` array as an argument. The new `String` object contains a copy of the characters in the array.

Line 15 instantiates a new `String` object and assigns its reference to `s4` using class `String`'s constructor that takes a `char` array and two integers as arguments. The second argument specifies the starting position (the offset) from which characters in the array are accessed. Remember that the first character is at position `0`. The third argument specifies the number of characters (the count) to access in the array. The new `String` object is formed from the accessed characters. If the offset or the count specified as an argument results in accessing an element outside the bounds of the character array, a `StringIndexOutOfBoundsException` is thrown.



Common Programming Error 16.1

Accessing a character outside the bounds of a `String` (i.e., an index less than 0 or an index greater than or equal to the `String`'s length) results in a `StringIndexOutOfBoundsException`.

16.3.2 String Methods length, charAt and getChars

`String` methods **`length`**, **`charAt`** and **`getChars`** return the length of a `String`, obtain the character at a specific location in a `String` and retrieve a set of characters from a `String` as a `char` array, respectively. Figure 16.2 demonstrates each of these methods.

```
1  // Fig. 16.2: StringMiscellaneous.java
2  // This application demonstrates the length, charAt and getChars
3  // methods of the String class.
4
5  public class StringMiscellaneous
6  {
```

Fig. 16.2 | `String` methods `length`, `charAt` and `getChars`. (Part 1 of 2.)

```
7      public static void main( String[] args )
8      {
9         String s1 = "hello there";
10        char[] charArray = new char[ 5 ];
11
12        System.out.printf( "s1: %s", s1 );
13
14        // test length method
15        System.out.printf( "\nLength of s1: %d", s1.length() );
16
17        // loop through characters in s1 with charAt and display reversed
18        System.out.print( "\nThe string reversed is: " );
19
20        for ( int count = s1.length() - 1; count >= 0; count-- )
21           System.out.printf( "%c ", s1.charAt( count ) );
22
23        // copy characters from string into charArray
24        s1.getChars( 0, 5, charArray, 0 );
25        System.out.print( "\nThe character array is: " );
26
27        for ( char character : charArray )
28           System.out.print( character );
29
30        System.out.println();
31     } // end main
32  } // end class StringMiscellaneous
```

```
s1: hello there
Length of s1: 11
The string reversed is: e r e h t   o l l e h
The character array is: hello
```

Fig. 16.2 | `String` methods `length`, `charAt` and `getChars`. (Part 2 of 2.)

Line 15 uses `String` method `length` to determine the number of characters in `String` `s1`. Like arrays, strings know their own length. However, unlike arrays, you access a `String`'s length via class `String`'s `length` method.

Lines 20–21 print the characters of the `String` `s1` in reverse order (and separated by spaces). `String` method `charAt` (line 21) returns the character at a specific position in the `String`. Method `charAt` receives an integer argument that's used as the index and returns the character at that position. Like arrays, the first element of a `String` is at position 0.

Line 24 uses `String` method `getChars` to copy the characters of a `String` into a character array. The first argument is the starting index from which characters are to be copied. The second argument is the index that's one past the last character to be copied from the `String`. The third argument is the character array into which the characters are to be copied. The last argument is the starting index where the copied characters are placed in the target character array. Next, lines 27–28 print the `char` array contents one character at a time.

16.3.3 Comparing Strings

Chapter 19 discusses sorting and searching arrays. Frequently, the information being sorted or searched consists of `String`s that must be compared to place them into order or to

determine whether a string appears in an array (or other collection). Class `String` provides methods for comparing strings, as demonstrated in the next two examples.

To understand what it means for one string to be greater than or less than another, consider the process of alphabetizing a series of last names. No doubt, you'd place "Jones" before "Smith" because the first letter of "Jones" comes before the first letter of "Smith" in the alphabet. But the alphabet is more than just a list of 26 letters—it's an ordered set of characters. Each letter occurs in a specific position within the set. Z is more than just a letter of the alphabet—it's specifically the twenty-sixth letter of the alphabet.

How does the computer know that one letter "comes before" another? All characters are represented in the computer as numeric codes (see Appendix B). When the computer compares `String`s, it actually compares the numeric codes of the characters in the `String`s.

Figure 16.3 demonstrates `String` methods `equals`, `equalsIgnoreCase`, `compareTo` and **`regionMatches`** and using the equality operator == to compare `String` objects.

```
1  // Fig. 16.3: StringCompare.java
2  // String methods equals, equalsIgnoreCase, compareTo and regionMatches.
3
4  public class StringCompare
5  {
6     public static void main( String[] args )
7     {
8        String s1 = new String( "hello" ); // s1 is a copy of "hello"
9        String s2 = "goodbye";
10       String s3 = "Happy Birthday";
11       String s4 = "happy birthday";
12
13       System.out.printf(
14          "s1 = %s\ns2 = %s\ns3 = %s\ns4 = %s\n\n", s1, s2, s3, s4 );
15
16       // test for equality
17       if ( s1.equals( "hello" ) )  // true
18          System.out.println( "s1 equals \"hello\"" );
19       else
20          System.out.println( "s1 does not equal \"hello\"" );
21
22       // test for equality with ==
23       if ( s1 == "hello" )  // false; they are not the same object
24          System.out.println( "s1 is the same object as \"hello\"" );
25       else
26          System.out.println( "s1 is not the same object as \"hello\"" );
27
28       // test for equality (ignore case)
29       if ( s3.equalsIgnoreCase( s4 ) )  // true
30          System.out.printf( "%s equals %s with case ignored\n", s3, s4 );
31       else
32          System.out.println( "s3 does not equal s4" );
33
```

Fig. 16.3 | `String` methods `equals`, `equalsIgnoreCase`, `compareTo` and `regionMatches`. (Part 1 of 2.)

```
34         // test compareTo
35         System.out.printf(
36            "\ns1.compareTo( s2 ) is %d", s1.compareTo( s2 ) );
37         System.out.printf(
38            "\ns2.compareTo( s1 ) is %d", s2.compareTo( s1 ) );
39         System.out.printf(
40            "\ns1.compareTo( s1 ) is %d", s1.compareTo( s1 ) );
41         System.out.printf(
42            "\ns3.compareTo( s4 ) is %d", s3.compareTo( s4 ) );
43         System.out.printf(
44            "\ns4.compareTo( s3 ) is %d\n\n", s4.compareTo( s3 ) );
45
46         // test regionMatches (case sensitive)
47         if ( s3.regionMatches( 0, s4, 0, 5 ) )
48            System.out.println( "First 5 characters of s3 and s4 match" );
49         else
50            System.out.println(
51               "First 5 characters of s3 and s4 do not match" );
52
53         // test regionMatches (ignore case)
54         if ( s3.regionMatches( true, 0, s4, 0, 5 ) )
55            System.out.println(
56               "First 5 characters of s3 and s4 match with case ignored" );
57         else
58            System.out.println(
59               "First 5 characters of s3 and s4 do not match" );
60      } // end main
61   } // end class StringCompare
```

```
s1 = hello
s2 = goodbye
s3 = Happy Birthday
s4 = happy birthday

s1 equals "hello"
s1 is not the same object as "hello"
Happy Birthday equals happy birthday with case ignored

s1.compareTo( s2 ) is 1
s2.compareTo( s1 ) is -1
s1.compareTo( s1 ) is 0
s3.compareTo( s4 ) is -32
s4.compareTo( s3 ) is 32

First 5 characters of s3 and s4 do not match
First 5 characters of s3 and s4 match with case ignored
```

Fig. 16.3 | `String` methods `equals`, `equalsIgnoreCase`, `compareTo` and `regionMatches`. (Part 2 of 2.)

`String` Method `equals`

The condition at line 17 uses method `equals` to compare `String` `s1` and the `String` literal `"hello"` for equality. Method `equals` (a method of class `Object` overridden in `String`) tests any two objects for equality—the strings contained in the two objects are identical.

The method returns `true` if the contents of the objects are equal, and `false` otherwise. The preceding condition is `true` because `String s1` was initialized with the string literal `"hello"`. Method `equals` uses a **lexicographical comparison**—it compares the integer Unicode values (see Appendix L, for more information) that represent each character in each `String`. Thus, if the `String` `"hello"` is compared with the string `"HELLO"`, the result is `false`, because the integer representation of a lowercase letter is different from that of the corresponding uppercase letter.

Comparing `String`s with the == Operator

The condition at line 23 uses the equality operator == to compare `String s1` for equality with the `String` literal `"hello"`. When primitive-type values are compared with ==, the result is `true` if *both values are identical*. When references are compared with ==, the result is `true` if *both references refer to the same object in memory*. To compare the actual contents (or state information) of objects for equality, a method must be invoked. In the case of `String`s, that method is `equals`. The preceding condition evaluates to `false` at line 23 because the reference `s1` was initialized with the statement

```
s1 = new String( "hello" );
```

which creates a new `String` object with a copy of string literal `"hello"` and assigns the new object to variable `s1`. If `s1` had been initialized with the statement

```
s1 = "hello";
```

which directly assigns the string literal `"hello"` to variable `s1`, the condition would be `true`. Remember that Java treats all string literal objects with the same contents as one `String` object to which there can be many references. Thus, lines 8, 17 and 23 all refer to the same `String` object `"hello"` in memory.



Common Programming Error 16.2

Comparing references with == can lead to logic errors, because == compares the references to determine whether they refer to the same object, not whether two objects have the same contents. *When two identical (but separate) objects are compared with ==, the result will be* `false`. *When comparing objects to determine whether they have the same contents, use method* `equals`.

`String` Method `equalsIgnoreCase`

If you're sorting `String`s, you may compare them for equality with method `equalsIgnoreCase`, which ignores whether the letters in each `String` are uppercase or lowercase when performing the comparison. Thus, `"hello"` and `"HELLO"` compare as equal. Line 29 uses `String` method `equalsIgnoreCase` to compare `String s3`—`Happy Birthday`—for equality with `String s4`—`happy birthday`. The result of this comparison is `true` because the comparison ignores case sensitivity.

`String` Method `compareTo`

Lines 35–44 use method `compareTo` to compare `String`s. Method `compareTo` is declared in the `Comparable` interface and implemented in the `String` class. Line 36 compares `String s1` to `String s2`. Method `compareTo` returns 0 if the `String`s are equal, a negative number if the `String` that invokes `compareTo` is less than the `String` that's passed as an argument and a positive number if the `String` that invokes `compareTo` is greater than the

`String` that's passed as an argument. Method `compareTo` uses a lexicographical comparison—it compares the numeric values of corresponding characters in each `String`.

`String` Method `regionMatches`

The condition at line 47 uses `String` method `regionMatches` to compare portions of two `String`s for equality. The first argument is the starting index in the `String` that invokes the method. The second argument is a comparison `String`. The third argument is the starting index in the comparison `String`. The last argument is the number of characters to compare between the two `String`s. The method returns `true` only if the specified number of characters are lexicographically equal.

Finally, the condition at line 54 uses a five-argument version of `String` method `regionMatches` to compare portions of two `String`s for equality. When the first argument is `true`, the method ignores the case of the characters being compared. The remaining arguments are identical to those described for the four-argument `regionMatches` method.

`String` Methods `startsWith` and `endsWith`

The next example (Fig. 16.4) demonstrates `String` methods **`startsWith`** and **`endsWith`**. Method `main` creates array `strings` containing `"started"`, `"starting"`, `"ended"` and `"ending"`. The remainder of method `main` consists of three `for` statements that test the elements of the array to determine whether they start with or end with a particular set of characters.

```
1  // Fig. 16.4: StringStartEnd.java
2  // String methods startsWith and endsWith.
3
4  public class StringStartEnd
5  {
6     public static void main( String[] args )
7     {
8        String[] strings = { "started", "starting", "ended", "ending" };
9
10       // test method startsWith
11       for ( String string : strings )
12       {
13          if ( string.startsWith( "st" ) )
14             System.out.printf( "\"%s\" starts with \"st\"\n", string );
15       } // end for
16
17       System.out.println();
18
19       // test method startsWith starting from position 2 of string
20       for ( String string : strings )
21       {
22          if ( string.startsWith( "art", 2 ) )
23             System.out.printf(
24                "\"%s\" starts with \"art\" at position 2\n", string );
25       } // end for
26
27       System.out.println();
```

Fig. 16.4 | `String` methods `startsWith` and `endsWith`. (Part 1 of 2.)

```
28
29          // test method endsWith
30          for ( String string : strings )
31          {
32             if ( string.endsWith( "ed" ) )
33                System.out.printf( "\"%s\" ends with \"ed\"\n", string );
34          } // end for
35       } // end main
36    } // end class StringStartEnd
```

```
"started" starts with "st"
"starting" starts with "st"

"started" starts with "art" at position 2
"starting" starts with "art" at position 2

"started" ends with "ed"
"ended" ends with "ed"
```

Fig. 16.4 | `String` methods `startsWith` and `endsWith`. (Part 2 of 2.)

Lines 11–15 use the version of method `startsWith` that takes a `String` argument. The condition in the `if` statement (line 13) determines whether each `String` in the array starts with the characters `"st"`. If so, the method returns `true` and the application prints that `String`. Otherwise, the method returns `false` and nothing happens.

Lines 20–25 use the `startsWith` method that takes a `String` and an integer as arguments. The integer specifies the index at which the comparison should begin in the `String`. The condition in the `if` statement (line 22) determines whether each `String` in the array has the characters `"art"` beginning with the third character in each `String`. If so, the method returns `true` and the application prints the `String`.

The third `for` statement (lines 30–34) uses method `endsWith`, which takes a `String` argument. The condition at line 32 determines whether each `String` in the array ends with the characters `"ed"`. If so, the method returns `true` and the application prints the `String`.

16.3.4 Locating Characters and Substrings in Strings

Often it's useful to search a string for a character or set of characters. For example, if you're creating your own word processor, you might want to provide a capability for searching through documents. Figure 16.5 demonstrates the many versions of `String` methods **`indexOf`** and **`lastIndexOf`** that search for a specified character or substring in a `String`.

```
1  // Fig. 16.5: StringIndexMethods.java
2  // String searching methods indexOf and lastIndexOf.
3
4  public class StringIndexMethods
5  {
6     public static void main( String[] args )
7     {
8        String letters = "abcdefghijklmabcdefghijklm";
```

Fig. 16.5 | String-searching methods `indexOf` and `lastIndexOf`. (Part 1 of 2.)

```
9
10        // test indexOf to locate a character in a string
11        System.out.printf(
12           "'c' is located at index %d\n", letters.indexOf( 'c' ) );
13        System.out.printf(
14           "'a' is located at index %d\n", letters.indexOf( 'a', 1 ) );
15        System.out.printf(
16           "'$' is located at index %d\n\n", letters.indexOf( '$' ) );
17
18        // test lastIndexOf to find a character in a string
19        System.out.printf( "Last 'c' is located at index %d\n",
20           letters.lastIndexOf( 'c' ) );
21        System.out.printf( "Last 'a' is located at index %d\n",
22           letters.lastIndexOf( 'a', 25 ) );
23        System.out.printf( "Last '$' is located at index %d\n\n",
24           letters.lastIndexOf( '$' ) );
25
26        // test indexOf to locate a substring in a string
27        System.out.printf( "\"def\" is located at index %d\n",
28           letters.indexOf( "def" ) );
29        System.out.printf( "\"def\" is located at index %d\n",
30           letters.indexOf( "def", 7 ) );
31        System.out.printf( "\"hello\" is located at index %d\n\n",
32           letters.indexOf( "hello" ) );
33
34        // test lastIndexOf to find a substring in a string
35        System.out.printf( "Last \"def\" is located at index %d\n",
36           letters.lastIndexOf( "def" ) );
37        System.out.printf( "Last \"def\" is located at index %d\n",
38           letters.lastIndexOf( "def", 25 ) );
39        System.out.printf( "Last \"hello\" is located at index %d\n",
40           letters.lastIndexOf( "hello" ) );
41     } // end main
42  } // end class StringIndexMethods
```

```
'c' is located at index 2
'a' is located at index 13
'$' is located at index -1

Last 'c' is located at index 15
Last 'a' is located at index 13
Last '$' is located at index -1

"def" is located at index 3
"def" is located at index 16
"hello" is located at index -1

Last "def" is located at index 16
Last "def" is located at index 16
Last "hello" is located at index -1
```

Fig. 16.5 | String-searching methods `indexOf` and `lastIndexOf`. (Part 2 of 2.)

All the searches in this example are performed on the `String` `letters` (initialized with `"abcdefghijklmabcdefghijklm"`). Lines 11–16 use method `indexOf` to locate the first

occurrence of a character in a `String`. If the method finds the character, it returns the character's index in the `String`—otherwise, it returns `-1`. There are two versions of `indexOf` that search for characters in a `String`. The expression in line 12 uses the version of method `indexOf` that takes an integer representation of the character to find. The expression at line 14 uses another version of method `indexOf`, which takes two integer arguments—the character and the starting index at which the search of the `String` should begin.

Lines 19–24 use method `lastIndexOf` to locate the last occurrence of a character in a `String`. The method searches from the end of the `String` toward the beginning. If it finds the character, it returns the character's index in the `String`—otherwise, it returns –1. There are two versions of `lastIndexOf` that search for characters in a `String`. The expression at line 20 uses the version that takes the integer representation of the character. The expression at line 22 uses the version that takes two integer arguments—the integer representation of the character and the index from which to begin searching backward.

Lines 27–40 demonstrate versions of methods `indexOf` and `lastIndexOf` that each take a `String` as the first argument. These versions perform identically to those described earlier except that they search for sequences of characters (or substrings) that are specified by their `String` arguments. If the substring is found, these methods return the index in the `String` of the first character in the substring.

16.3.5 Extracting Substrings from Strings

Class `String` provides two `substring` methods to enable a new `String` object to be created by copying part of an existing `String` object. Each method returns a new `String` object. Both methods are demonstrated in Fig. 16.6.

```
1  // Fig. 16.6: SubString.java
2  // String class substring methods.
3
4  public class SubString
5  {
6     public static void main( String[] args )
7     {
8        String letters = "abcdefghijklmabcdefghijklm";
9
10       // test substring methods
11       System.out.printf( "Substring from index 20 to end is \"%s\"\n",
12          letters.substring( 20 ) );
13       System.out.printf( "%s \"%s\"\n",
14          "Substring from index 3 up to, but not including 6 is",
15          letters.substring( 3, 6 ) );
16    } // end main
17 } // end class SubString
```

```
Substring from index 20 to end is "hijklm"
Substring from index 3 up to, but not including 6 is "def"
```

Fig. 16.6 | `String` class `substring` methods.

The expression `letters.substring(20)` at line 12 uses the `substring` method that takes one integer argument. The argument specifies the starting index in the original

`String` `letters` from which characters are to be copied. The substring returned contains a copy of the characters from the starting index to the end of the `String`. Specifying an index outside the bounds of the `String` causes a **`StringIndexOutOfBoundsException`**.

Line 15 uses the `substring` method that takes two integer arguments—the starting index from which to copy characters in the original `String` and the index one beyond the last character to copy (i.e., copy up to, but not including, that index in the `String`). The substring returned contains a copy of the specified characters from the original `String`. An index outside the bounds of the `String` causes a `StringIndexOutOfBoundsException`.

16.3.6 Concatenating Strings

`String` method **`concat`** (Fig. 16.7) concatenates two `String` objects and returns a new `String` object containing the characters from both original `String`s. The expression `s1.concat( s2 )` at line 13 forms a `String` by appending the characters in `s2` to the characters in `s1`. The original `String`s to which `s1` and `s2` refer are not modified.

```
1  // Fig. 16.7: StringConcatenation.java
2  // String method concat.
3
4  public class StringConcatenation
5  {
6     public static void main( String[] args )
7     {
8        String s1 = "Happy ";
9        String s2 = "Birthday";
10
11       System.out.printf( "s1 = %s\ns2 = %s\n\n",s1, s2 );
12       System.out.printf(
13          "Result of s1.concat( s2 ) = %s\n", s1.concat( s2 ) );
14       System.out.printf( "s1 after concatenation = %s\n", s1 );
15    } // end main
16 } // end class StringConcatenation
```

```
s1 = Happy
s2 = Birthday

Result of s1.concat( s2 ) = Happy Birthday
s1 after concatenation = Happy
```

Fig. 16.7 | `String` method `concat`.

16.3.7 Miscellaneous `String` Methods

Class `String` provides several methods that return modified copies of `String`s or that return character arrays. These methods are demonstrated in the application in Fig. 16.8.

Line 16 uses `String` method `replace` to return a new `String` object in which every occurrence in `s1` of character `'l'` (lowercase el) is replaced with character `'L'`. Method `replace` leaves the original `String` unchanged. If there are no occurrences of the first argument in the `String`, method `replace` returns the original `String`. An overloaded version of method `replace` enables you to replace substrings rather than individual characters.

```
1  // Fig. 16.8: StringMiscellaneous2.java
2  // String methods replace, toLowerCase, toUpperCase, trim and toCharArray.
3
4  public class StringMiscellaneous2
5  {
6     public static void main( String[] args )
7     {
8        String s1 = "hello";
9        String s2 = "GOODBYE";
10       String s3 = "   spaces   ";
11
12       System.out.printf( "s1 = %s\ns2 = %s\ns3 = %s\n\n", s1, s2, s3 );
13
14       // test method replace
15       System.out.printf(
16          "Replace 'l' with 'L' in s1: %s\n\n", s1.replace( 'l', 'L' ) );
17
18       // test toLowerCase and toUpperCase
19       System.out.printf( "s1.toUpperCase() = %s\n", s1.toUpperCase() );
20       System.out.printf( "s2.toLowerCase() = %s\n\n", s2.toLowerCase() );
21
22       // test trim method
23       System.out.printf( "s3 after trim = \"%s\"\n\n", s3.trim() );
24
25       // test toCharArray method
26       char[] charArray = s1.toCharArray();
27       System.out.print( "s1 as a character array = " );
28
29       for ( char character : charArray )
30          System.out.print( character );
31
32       System.out.println();
33    } // end main
34 } // end class StringMiscellaneous2
```

```
s1 = hello
s2 = GOODBYE
s3 =    spaces

Replace 'l' with 'L' in s1: heLLo

s1.toUpperCase() = HELLO
s2.toLowerCase() = goodbye

s3 after trim = "spaces"

s1 as a character array = hello
```

Fig. 16.8 | `String` methods `replace`, `toLowerCase`, `toUpperCase`, `trim` and `toCharArray`.

Line 19 uses `String` method **`toUpperCase`** to generate a new `String` with uppercase letters where corresponding lowercase letters exist in `s1`. The method returns a new `String` object containing the converted `String` and leaves the original `String` unchanged. If there are no characters to convert, method `toUpperCase` returns the original `String`.

Line 20 uses `String` method **`toLowerCase`** to return a new `String` object with lowercase letters where corresponding uppercase letters exist in `s2`. The original `String` remains unchanged. If there are no characters in the original `String` to convert, `toLowerCase` returns the original `String`.

Line 23 uses `String` method **`trim`** to generate a new `String` object that removes all white-space characters that appear at the beginning and/or end of the `String` on which `trim` operates. The method returns a new `String` object containing the `String` without leading or trailing white space. The original `String` remains unchanged. If there are no whitespace characters at the beginning and/or end, `trim` returns the original `String`.

Line 26 uses `String` method **`toCharArray`** to create a new character array containing a copy of the characters in `s1`. Lines 29–30 output each `char` in the array.

16.3.8 String Method valueOf

As we've seen, every object in Java has a `toString` method that enables a program to obtain the object's string representation. Unfortunately, this technique cannot be used with primitive types because they do not have methods. Class `String` provides `static` methods that take an argument of any type and convert it to a `String` object. Figure 16.9 demonstrates the `String` class **`valueOf`** methods.

The expression `String.valueOf(charArray)` at line 18 uses the character array `charArray` to create a new `String` object. The expression `String.valueOf(charArray, 3, 3)` at line 20 uses a portion of the character array `charArray` to create a new `String` object. The second argument specifies the starting index from which the characters are used. The third argument specifies the number of characters to be used.

```
1  // Fig. 16.9: StringValueOf.java
2  // String valueOf methods.
3
4  public class StringValueOf
5  {
6     public static void main( String[] args )
7     {
8        char[] charArray = { 'a', 'b', 'c', 'd', 'e', 'f' };
9        boolean booleanValue = true;
10       char characterValue = 'Z';
11       int integerValue = 7;
12       long longValue = 10000000000L; // L suffix indicates long
13       float floatValue = 2.5f; // f indicates that 2.5 is a float
14       double doubleValue = 33.333; // no suffix, double is default
15       Object objectRef = "hello"; // assign string to an Object reference
16
17       System.out.printf(
18          "char array = %s\n", String.valueOf( charArray ) );
19       System.out.printf( "part of char array = %s\n",
20          String.valueOf( charArray, 3, 3 ) );
21       System.out.printf(
22          "boolean = %s\n", String.valueOf( booleanValue ) );
23       System.out.printf(
24          "char = %s\n", String.valueOf( characterValue ) );
```

Fig. 16.9 | `String` `valueOf` methods. (Part 1 of 2.)

```
25         System.out.printf( "int = %s\n", String.valueOf( integerValue ) );
26         System.out.printf( "long = %s\n", String.valueOf( longValue ) );
27         System.out.printf( "float = %s\n", String.valueOf( floatValue ) );
28         System.out.printf(
29            "double = %s\n", String.valueOf( doubleValue ) );
30         System.out.printf( "Object = %s", String.valueOf( objectRef ) );
31      } // end main
32   } // end class StringValueOf
```

```
char array = abcdef
part of char array = def
boolean = true
char = Z
int = 7
long = 10000000000
float = 2.5
double = 33.333
Object = hello
```

Fig. 16.9 | `String` `valueOf` methods. (Part 2 of 2.)

There are seven other versions of method `valueOf`, which take arguments of type `boolean`, `char`, `int`, `long`, `float`, `double` and `Object`, respectively. These are demonstrated in lines 21–30. The version of `valueOf` that takes an `Object` as an argument can do so because all `Objects` can be converted to `Strings` with method `toString`.

[*Note:* Lines 12–13 use literal values `10000000000L` and `2.5f` as the initial values of `long` variable `longValue` and `float` variable `floatValue`, respectively. By default, Java treats integer literals as type `int` and floating-point literals as type `double`. Appending the letter `L` to the literal `10000000000` and appending letter `f` to the literal `2.5` indicates to the compiler that `10000000000` should be treated as a `long` and `2.5` as a `float`. An uppercase `L` or lowercase `l` can be used to denote a variable of type `long` and an uppercase `F` or lowercase `f` can be used to denote a variable of type `float`.]

16.4 Class `StringBuilder`

We now discuss the features of class **`StringBuilder`** for creating and manipulating *dynamic* string information—that is, *modifiable* strings. Every `StringBuilder` is capable of storing a number of characters specified by its capacity. If a `StringBuilder`'s capacity is exceeded, the capacity expands to accommodate the additional characters.



Performance Tip 16.2

Java can perform certain optimizations involving `String` objects (such as referring to one `String` object from multiple variables) because it knows these objects will not change. `Strings` (not `StringBuilders`) should be used if the data will not change.



Performance Tip 16.3

In programs that frequently perform string concatenation, or other string modifications, it's often more efficient to implement the modifications with class `StringBuilder`.



Software Engineering Observation 16.2

`StringBuilder`s are not thread safe. If multiple threads require access to the same dynamic string information, use class **`StringBuffer`** *in your code. Classes `StringBuilder` and `StringBuffer` provide identical capabilities, but class `StringBuffer` is thread safe. For more details on threading, see Chapter 26.*

16.4.1 StringBuilder Constructors

Class `StringBuilder` provides four constructors. We demonstrate three of these in Fig. 16.10. Line 8 uses the no-argument `StringBuilder` constructor to create a `StringBuilder` with no characters in it and an initial capacity of 16 characters (the default for a `StringBuilder`). Line 9 uses the `StringBuilder` constructor that takes an integer argument to create a `StringBuilder` with no characters in it and the initial capacity specified by the integer argument (i.e., 10). Line 10 uses the `StringBuilder` constructor that takes a `String` argument to create a `StringBuilder` containing the characters in the `String` argument. The initial capacity is the number of characters in the `String` argument plus 16.

Lines 12–14 implicitly use the method `toString` of class `StringBuilder` to output the `StringBuilder`s with the `printf` method. In Section 16.4.4, we discuss how Java uses `StringBuilder` objects to implement the + and += operators for string concatenation.

```
1  // Fig. 16.10: StringBuilderConstructors.java
2  // StringBuilder constructors.
3
4  public class StringBuilderConstructors
5  {
6     public static void main( String[] args )
7     {
8        StringBuilder buffer1 = new StringBuilder();
9        StringBuilder buffer2 = new StringBuilder( 10 );
10       StringBuilder buffer3 = new StringBuilder( "hello" );
11
12       System.out.printf( "buffer1 = \"%s\"\n", buffer1 );
13       System.out.printf( "buffer2 = \"%s\"\n", buffer2 );
14       System.out.printf( "buffer3 = \"%s\"\n", buffer3 );
15    } // end main
16 } // end class StringBuilderConstructors
```

```
buffer1 = ""
buffer2 = ""
buffer3 = "hello"
```

Fig. 16.10 | `StringBuilder` constructors.

16.4.2 StringBuilder Methods length, capacity, setLength and ensureCapacity

Class `StringBuilder` provides methods **`length`** and **`capacity`** to return the number of characters currently in a `StringBuilder` and the number of characters that can be stored in a `StringBuilder` without allocating more memory, respectively. Method **`ensureCapacity`** guarantees that a `StringBuilder` has at least the specified capacity. Method

`setLength` increases or decreases the length of a `StringBuilder`. Figure 16.11 demonstrates these methods.

```
1  // Fig. 16.11: StringBuilderCapLen.java
2  // StringBuilder length, setLength, capacity and ensureCapacity methods.
3
4  public class StringBuilderCapLen
5  {
6     public static void main( String[] args )
7     {
8        StringBuilder buffer = new StringBuilder( "Hello, how are you?" );
9
10       System.out.printf( "buffer = %s\nlength = %d\ncapacity = %d\n\n",
11          buffer.toString(), buffer.length(), buffer.capacity() );
12
13       buffer.ensureCapacity( 75 );
14       System.out.printf( "New capacity = %d\n\n", buffer.capacity() );
15
16       buffer.setLength( 10 );
17       System.out.printf( "New length = %d\nbuffer = %s\n",
18          buffer.length(), buffer.toString() );
19    } // end main
20 } // end class StringBuilderCapLen
```

```
buffer = Hello, how are you?
length = 19
capacity = 35

New capacity = 75

New length = 10
buffer = Hello, how
```

Fig. 16.11 | `StringBuilder` `length`, `setLength`, `capacity` and `ensureCapacity` methods.

The application contains one `StringBuilder` called `buffer`. Line 8 uses the `StringBuilder` constructor that takes a `String` argument to initialize the `StringBuilder` with `"Hello, how are you?"`. Lines 10–11 print the contents, length and capacity of the `StringBuilder`. Note in the output window that the capacity of the `StringBuilder` is initially 35. Recall that the `StringBuilder` constructor that takes a `String` argument initializes the capacity to the length of the string passed as an argument plus 16.

Line 13 uses method `ensureCapacity` to expand the capacity of the `StringBuilder` to a minimum of 75 characters. Actually, if the original capacity is less than the argument, the method ensures a capacity that's the greater of the number specified as an argument and twice the original capacity plus 2. The `StringBuilder`'s current capacity remains unchanged if it's more than the specified capacity.



Performance Tip 16.4

Dynamically increasing the capacity of a `StringBuilder` can take a relatively long time. Executing a large number of these operations can degrade the performance of an application. If a `StringBuilder` is going to increase greatly in size, possibly multiple times, setting its capacity high at the beginning will increase performance.

Line 16 uses method `setLength` to set the length of the `StringBuilder` to 10. If the specified length is less than the current number of characters in the `StringBuilder`, the buffer is truncated to the specified length (i.e., the characters in the `StringBuilder` after the specified length are discarded). If the specified length is greater than the number of characters currently in the `StringBuilder`, `null` characters (characters with the numeric representation 0) are appended until the total number of characters in the `StringBuilder` is equal to the specified length.

16.4.3 `StringBuilder` Methods `charAt`, `setCharAt`, `getChars` and `reverse`

Class `StringBuilder` provides methods **`charAt`**, **`setCharAt`**, **`getChars`** and **`reverse`** to manipulate the characters in a `StringBuilder` (Fig. 16.12). Method `charAt` (line 12) takes an integer argument and returns the character in the `StringBuilder` at that index. Method `getChars` (line 15) copies characters from a `StringBuilder` into the character array passed as an argument. This method takes four arguments—the starting index from which characters should be copied in the `StringBuilder`, the index one past the last character to be copied from the `StringBuilder`, the character array into which the characters are to be copied and the starting location in the character array where the first character should be placed. Method `setCharAt` (lines 21 and 22) takes an integer and a character argument and sets the character at the specified position in the `StringBuilder` to the character argument. Method `reverse` (line 25) reverses the contents of the `StringBuilder`.



Common Programming Error 16.3

Attempting to access a character that's outside the bounds of a `StringBuilder` (i.e., with an index less than 0 or greater than or equal to the `StringBuilder`'s length) results in a `StringIndexOutOfBoundsException`.

```
1  // Fig. 16.12: StringBuilderChars.java
2  // StringBuilder methods charAt, setCharAt, getChars and reverse.
3
4  public class StringBuilderChars
5  {
6     public static void main( String[] args )
7     {
8        StringBuilder buffer = new StringBuilder( "hello there" );
9
10       System.out.printf( "buffer = %s\n", buffer.toString() );
11       System.out.printf( "Character at 0: %s\nCharacter at 4: %s\n\n",
12          buffer.charAt( 0 ), buffer.charAt( 4 ) );
13
14       char[] charArray = new char[ buffer.length() ];
15       buffer.getChars( 0, buffer.length(), charArray, 0 );
16       System.out.print( "The characters are: " );
17
18       for ( char character : charArray )
19          System.out.print( character );
```

Fig. 16.12 | `StringBuilder` methods `charAt`, `setCharAt`, `getChars` and `reverse`. (Part 1 of 2.)

```
20
21         buffer.setCharAt( 0, 'H' );
22         buffer.setCharAt( 6, 'T' );
23         System.out.printf( "\n\nbuffer = %s", buffer.toString() );
24
25         buffer.reverse();
26         System.out.printf( "\n\nbuffer = %s\n", buffer.toString() );
27      } // end main
28   } // end class StringBuilderChars
```

```
buffer = hello there
Character at 0: h
Character at 4: o

The characters are: hello there

buffer = Hello There

buffer = erehT olleH
```

Fig. 16.12 | `StringBuilder` methods `charAt`, `setCharAt`, `getChars` and `reverse`. (Part 2 of 2.)

16.4.4 StringBuilder append Methods

Class `StringBuilder` provides overloaded **append** methods (Fig. 16.13) to allow values of various types to be appended to the end of a `StringBuilder`. Versions are provided for each of the primitive types, and for character arrays, `String`s, `Object`s, and more. (Remember that method `toString` produces a string representation of any `Object`.) Each method takes its argument, converts it to a string and appends it to the `StringBuilder`.

```
1  // Fig. 16.13: StringBuilderAppend.java
2  // StringBuilder append methods.
3
4  public class StringBuilderAppend
5  {
6     public static void main( String[] args )
7     {
8        Object objectRef = "hello";
9        String string = "goodbye";
10       char[] charArray = { 'a', 'b', 'c', 'd', 'e', 'f' };
11       boolean booleanValue = true;
12       char characterValue = 'Z';
13       int integerValue = 7;
14       long longValue = 10000000000L;
15       float floatValue = 2.5f;
16       double doubleValue = 33.333;
17
18       StringBuilder lastBuffer = new StringBuilder( "last buffer" );
19       StringBuilder buffer = new StringBuilder();
20
21       buffer.append( objectRef );
```

Fig. 16.13 | `StringBuilder` `append` methods. (Part 1 of 2.)

```
22          buffer.append( "\n" );
23          buffer.append( string );
24          buffer.append( "\n" );
25          buffer.append( charArray );
26          buffer.append( "\n" );
27          buffer.append( charArray, 0, 3 );
28          buffer.append( "\n" );
29          buffer.append( booleanValue );
30          buffer.append( "\n" );
31          buffer.append( characterValue );
32          buffer.append( "\n" );
33          buffer.append( integerValue );
34          buffer.append( "\n" );
35          buffer.append( longValue );
36          buffer.append( "\n" );
37          buffer.append( floatValue );
38          buffer.append( "\n" );
39          buffer.append( doubleValue );
40          buffer.append( "\n" );
41          buffer.append( lastBuffer );
42
43          System.out.printf( "buffer contains %s\n", buffer.toString() );
44       } // end main
45    } // end StringBuilderAppend
```

```
buffer contains hello
goodbye
abcdef
abc
true
Z
7
10000000000
2.5
33.333
last buffer
```

Fig. 16.13 | `StringBuilder` `append` methods. (Part 2 of 2.)

Actually, a compiler can use `StringBuilder` (or `StringBuffer`) and the `append` methods to implement the `+` and `+=` `String` concatenation operators. For example, assuming the declarations

```
String string1 = "hello";
String string2 = "BC";
int value = 22;
```

the statement

```
String s = string1 + string2 + value;
```

concatenates `"hello"`, `"BC"` and `22`. The concatenation can be performed as follows:

```
String s = new StringBuilder().append( "hello" ).append( "BC" ).
   append( 22 ).toString();
```

First, the preceding statement creates an empty `StringBuilder`, then appends to it the strings `"hello"` and `"BC"` and the integer `22`. Next, `StringBuilder`'s `toString` method converts the `StringBuilder` to a `String` object to be assigned to `String` `s`. The statement

```
s += "!";
```

can be performed as follows (this may differ by compiler):

```
s = new StringBuilder().append( s ).append( "!" ).toString();
```

This creates an empty `StringBuilder`, then appends to it the current contents of `s` followed by `"!"`. Next, `StringBuilder`'s method `toString` (which must be called explicitly here) returns the `StringBuilder`'s contents as a `String`, and the result is assigned to `s`.

16.4.5 StringBuilder Insertion and Deletion Methods

`StringBuilder` provides overloaded **`insert`** methods to insert values of various types at any position in a `StringBuilder`. Versions are provided for the primitive types and for character arrays, `String`s, `Object`s and `CharSequence`s. Each method takes its second argument and inserts it at the index specified by the first argument. If the first argument is less than `0` or greater than the `StringBuilder`'s length, a `StringIndexOutOfBoundsException` occurs. Class `StringBuilder` also provides methods **`delete`** and **`deleteCharAt`** to delete characters at any position in a `StringBuilder`. Method `delete` takes two arguments—the starting index and the index one past the end of the characters to delete. All characters beginning at the starting index up to but not including the ending index are deleted. Method `deleteCharAt` takes one argument—the index of the character to delete. Invalid indices cause both methods to throw a `StringIndexOutOfBoundsException`. Figure 16.14 demonstrates methods `insert`, `delete` and `deleteCharAt`.

```
1  // Fig. 16.14: StringBuilderInsertDelete.java
2  // StringBuilder methods insert, delete and deleteCharAt.
3
4  public class StringBuilderInsertDelete
5  {
6     public static void main( String[] args )
7     {
8        Object objectRef = "hello";
9        String string = "goodbye";
10       char[] charArray = { 'a', 'b', 'c', 'd', 'e', 'f' };
11       boolean booleanValue = true;
12       char characterValue = 'K';
13       int integerValue = 7;
14       long longValue = 10000000;
15       float floatValue = 2.5f; // f suffix indicates that 2.5 is a float
16       double doubleValue = 33.333;
17
18       StringBuilder buffer = new StringBuilder();
19
20       buffer.insert( 0, objectRef );
21       buffer.insert( 0, "  " ); // each of these contains two spaces
22       buffer.insert( 0, string );
```

Fig. 16.14 | `StringBuilder` methods `insert`, `delete` and `deleteCharAt`. (Part 1 of 2.)

```
23          buffer.insert( 0, "  " );
24          buffer.insert( 0, charArray );
25          buffer.insert( 0, "  " );
26          buffer.insert( 0, charArray, 3, 3 );
27          buffer.insert( 0, "  " );
28          buffer.insert( 0, booleanValue );
29          buffer.insert( 0, "  " );
30          buffer.insert( 0, characterValue );
31          buffer.insert( 0, "  " );
32          buffer.insert( 0, integerValue );
33          buffer.insert( 0, "  " );
34          buffer.insert( 0, longValue );
35          buffer.insert( 0, "  " );
36          buffer.insert( 0, floatValue );
37          buffer.insert( 0, "  " );
38          buffer.insert( 0, doubleValue );
39
40          System.out.printf(
41             "buffer after inserts:\n%s\n\n", buffer.toString() );
42
43          buffer.deleteCharAt( 10 ); // delete 5 in 2.5
44          buffer.delete( 2, 6 ); // delete .333 in 33.333
45
46          System.out.printf(
47             "buffer after deletes:\n%s\n", buffer.toString() );
48       } // end main
49    } // end class StringBuilderInsertDelete
```

```
buffer after inserts:
33.333  2.5  10000000  7  K  true  def  abcdef  goodbye  hello

buffer after deletes:
33  2.  10000000  7  K  true  def  abcdef  goodbye  hello
```

Fig. 16.14 | `StringBuilder` methods `insert`, `delete` and `deleteCharAt`. (Part 2 of 2.)

16.5 Class Character

Java provides eight type-wrapper classes—`Boolean`, `Character`, `Double`, `Float`, `Byte`, `Short`, `Integer` and `Long`—that enable primitive-type values to be treated as objects. In this section, we present class `Character`—the type-wrapper class for primitive type `char`.

Most `Character` methods are `static` methods designed for convenience in processing individual `char` values. These methods take at least a character argument and perform either a test or a manipulation of the character. Class `Character` also contains a constructor that receives a `char` argument to initialize a `Character` object. Most of the methods of class `Character` are presented in the next three examples. For more information on class `Character` (and all the type-wrapper classes), see the `java.lang` package in the Java API documentation.

Figure 16.15 demonstrates `static` methods that test characters to determine whether they're a specific character type and the `static` methods that perform case conversions on characters. You can enter any character and apply the methods to the character.

Line 15 uses `Character` method **`isDefined`** to determine whether character `c` is defined in the Unicode character set. If so, the method returns `true`; otherwise, it returns `false`. Line 16 uses `Character` method **`isDigit`** to determine whether character `c` is a defined Unicode digit. If so, the method returns `true`, and otherwise, `false`.

```
1  // Fig. 16.15: StaticCharMethods.java
2  // Character static methods for testing characters and converting case.
3  import java.util.Scanner;
4
5  public class StaticCharMethods
6  {
7     public static void main( String[] args )
8     {
9        Scanner scanner = new Scanner( System.in ); // create scanner
10       System.out.println( "Enter a character and press Enter" );
11       String input = scanner.next();
12       char c = input.charAt( 0 ); // get input character
13
14       // display character info
15       System.out.printf( "is defined: %b\n", Character.isDefined( c ) );
16       System.out.printf( "is digit: %b\n", Character.isDigit( c ) );
17       System.out.printf( "is first character in a Java identifier: %b\n",
18          Character.isJavaIdentifierStart( c ) );
19       System.out.printf( "is part of a Java identifier: %b\n",
20          Character.isJavaIdentifierPart( c ) );
21       System.out.printf( "is letter: %b\n", Character.isLetter( c ) );
22       System.out.printf(
23          "is letter or digit: %b\n", Character.isLetterOrDigit( c ) );
24       System.out.printf(
25          "is lower case: %b\n", Character.isLowerCase( c ) );
26       System.out.printf(
27          "is upper case: %b\n", Character.isUpperCase( c ) );
28       System.out.printf(
29          "to upper case: %s\n", Character.toUpperCase( c ) );
30       System.out.printf(
31          "to lower case: %s\n", Character.toLowerCase( c ) );
32    } // end main
33 } // end class StaticCharMethods
```

```
Enter a character and press Enter
A
is defined: true
is digit: false
is first character in a Java identifier: true
is part of a Java identifier: true
is letter: true
is letter or digit: true
is lower case: false
is upper case: true
to upper case: A
to lower case: a
```

Fig. 16.15 | `Character` `static` methods for testing characters and converting case. (Part 1 of 2.)

```
Enter a character and press Enter
8
is defined: true
is digit: true
is first character in a Java identifier: false
is part of a Java identifier: true
is letter: false
is letter or digit: true
is lower case: false
is upper case: false
to upper case: 8
to lower case: 8
```

```
Enter a character and press Enter
$
is defined: true
is digit: false
is first character in a Java identifier: true
is part of a Java identifier: true
is letter: false
is letter or digit: false
is lower case: false
is upper case: false
to upper case: $
to lower case: $
```

Fig. 16.15 | `Character` `static` methods for testing characters and converting case. (Part 2 of 2.)

Line 18 uses `Character` method **`isJavaIdentifierStart`** to determine whether `c` is a character that can be the first character of an identifier in Java—that is, a letter, an underscore (_) or a dollar sign ($). If so, the method returns `true`, and otherwise, `false`. Line 20 uses `Character` method **`isJavaIdentifierPart`** to determine whether character `c` is a character that can be used in an identifier in Java—that is, a digit, a letter, an underscore (_) or a dollar sign ($). If so, the method returns `true`, and otherwise, `false`.

Line 21 uses `Character` method **`isLetter`** to determine whether character `c` is a letter. If so, the method returns `true`, and otherwise, `false`. Line 23 uses `Character` method **`isLetterOrDigit`** to determine whether character `c` is a letter or a digit. If so, the method returns `true`, and otherwise, `false`.

Line 25 uses `Character` method **`isLowerCase`** to determine whether character `c` is a lowercase letter. If so, the method returns `true`, and otherwise, `false`. Line 27 uses `Character` method **`isUpperCase`** to determine whether character `c` is an uppercase letter. If so, the method returns `true`, and otherwise, `false`.

Line 29 uses `Character` method **`toUpperCase`** to convert the character `c` to its uppercase equivalent. The method returns the converted character if the character has an uppercase equivalent, and otherwise, the method returns its original argument. Line 31 uses `Character` method **`toLowerCase`** to convert the character `c` to its lowercase equivalent. The method returns the converted character if the character has a lowercase equivalent, and otherwise, the method returns its original argument.

Figure 16.16 demonstrates static `Character` methods **`digit`** and **`forDigit`**, which convert characters to digits and digits to characters, respectively, in different number sys-

tems. Common number systems include decimal (base 10), octal (base 8), hexadecimal (base 16) and binary (base 2). The base of a number is also known as its **radix**. For more information on conversions between number systems, see Appendix H.

Line 28 uses method `forDigit` to convert the integer `digit` into a character in the number system specified by the integer `radix` (the base of the number). For example, the decimal integer `13` in base 16 (the `radix`) has the character value `'d'`. Lowercase and uppercase letters represent the same value in number systems. Line 35 uses method `digit` to convert variable `character` into an integer in the number system specified by the integer `radix` (the base of the number). For example, the character `'A'` is the base 16 (the `radix`) representation of the base 10 value 10. The radix must be between 2 and 36, inclusive.

```
1  // Fig. 16.16: StaticCharMethods2.java
2  // Character class static conversion methods.
3  import java.util.Scanner;
4
5  public class StaticCharMethods2
6  {
7     // executes application
8     public static void main( String[] args )
9     {
10       Scanner scanner = new Scanner( System.in );
11
12       // get radix
13       System.out.println( "Please enter a radix:" );
14       int radix = scanner.nextInt();
15
16       // get user choice
17       System.out.printf( "Please choose one:\n1 -- %s\n2 -- %s\n",
18          "Convert digit to character", "Convert character to digit" );
19       int choice = scanner.nextInt();
20
21       // process request
22       switch ( choice )
23       {
24          case 1: // convert digit to character
25             System.out.println( "Enter a digit:" );
26             int digit = scanner.nextInt();
27             System.out.printf( "Convert digit to character: %s\n",
28                Character.forDigit( digit, radix ) );
29             break;
30
31          case 2: // convert character to digit
32             System.out.println( "Enter a character:" );
33             char character = scanner.next().charAt( 0 );
34             System.out.printf( "Convert character to digit: %s\n",
35                Character.digit( character, radix ) );
36             break;
37       } // end switch
38    } // end main
39 } // end class StaticCharMethods2
```

Fig. 16.16 | `Character` class `static` conversion methods. (Part 1 of 2.)

```
Please enter a radix:
16
Please choose one:
1 -- Convert digit to character
2 -- Convert character to digit
2
Enter a character:
A
Convert character to digit: 10
```

```
Please enter a radix:
16
Please choose one:
1 -- Convert digit to character
2 -- Convert character to digit
1
Enter a digit:
13
Convert digit to character: d
```

Fig. 16.16 | `Character` class `static` conversion methods. (Part 2 of 2.)

Figure 16.17 demonstrates the constructor and several non-`static` methods of class `Character`—**`charValue`**, `toString` and `equals`. Lines 7–8 instantiate two `Character` objects by assigning the character constants `'A'` and `'a'`, respectively, to the `Character` variables. Java automatically converts these `char` literals into `Character` objects—a process known as *autoboxing* that we discuss in more detail in Section 20.4. Line 11 uses `Character` method `charValue` to return the `char` value stored in `Character` object `c1`. Line 11 returns a string representation of `Character` object `c2` using method `toString`. The condition in line 13 uses method `equals` to determine whether the object `c1` has the same contents as the object `c2` (i.e., the characters inside each object are equal).

```
1  // Fig. 16.17: OtherCharMethods.java
2  // Character class non-static methods.
3  public class OtherCharMethods
4  {
5     public static void main( String[] args )
6     {
7        Character c1 = 'A';
8        Character c2 = 'a';
9
10       System.out.printf(
11          "c1 = %s\nc2 = %s\n\n", c1.charValue(), c2.toString() );
12
13       if ( c1.equals( c2 ) )
14          System.out.println( "c1 and c2 are equal\n" );
15       else
16          System.out.println( "c1 and c2 are not equal\n" );
17    } // end main
18 } // end class OtherCharMethods
```

Fig. 16.17 | `Character` class non-`static` methods. (Part 1 of 2.)

```
c1 = A
c2 = a

c1 and c2 are not equal
```

Fig. 16.17 | `Character` class non-`static` methods. (Part 2 of 2.)

16.6 Tokenizing `String`s

When you read a sentence, your mind breaks it into **tokens**—individual words and punctuation marks that convey meaning to you. Compilers also perform tokenization. They break up statements into individual pieces like keywords, identifiers, operators and other programming-language elements. We now study class `String`'s **`split`** method, which breaks a `String` into its component tokens. Tokens are separated from one another by **delimiters**, typically white-space characters such as space, tab, newline and carriage return. Other characters can also be used as delimiters to separate tokens. The application in Fig. 16.18 demonstrates `String`'s `split` method.

When the user presses the *Enter* key, the input sentence is stored in variable `sentence`. Line 17 invokes `String` method `split` with the `String` argument `" "`, which returns an array of `String`s. The space character in the argument `String` is the delimiter that method `split` uses to locate the tokens in the `String`. As you'll learn in the next section, the argument to method split can be a regular expression for more complex tokenizing. Line 19 displays the length of the array `tokens`—i.e., the number of tokens in `sentence`. Lines 21–22 output each token on a separate line.

```
1  // Fig. 16.18: TokenTest.java
2  // StringTokenizer object used to tokenize strings.
3  import java.util.Scanner;
4  import java.util.StringTokenizer;
5
6  public class TokenTest
7  {
8     // execute application
9     public static void main( String[] args )
10    {
11       // get sentence
12       Scanner scanner = new Scanner( System.in );
13       System.out.println( "Enter a sentence and press Enter" );
14       String sentence = scanner.nextLine();
15
16       // process user sentence
17       String[] tokens = sentence.split( " " );
18       System.out.printf( "Number of elements: %d\nThe tokens are:\n",
19          tokens.length );
20
21       for ( String token : tokens )
22          System.out.println( token );
23    } // end main
24 } // end class TokenTest
```

Fig. 16.18 | `StringTokenizer` object used to tokenize strings. (Part 1 of 2.)

```
Enter a sentence and press Enter
This is a sentence with seven tokens
Number of elements: 7
The tokens are:
This
is
a
sentence
with
seven
tokens
```

Fig. 16.18 | `StringTokenizer` object used to tokenize strings. (Part 2 of 2.)

16.7 Regular Expressions, Class `Pattern` and Class `Matcher`

A **regular expression** is a `String` that describes a search pattern for matching characters in other `String`s. Such expressions are useful for validating input and ensuring that data is in a particular format. For example, a ZIP code must consist of five digits, and a last name must contain only letters, spaces, apostrophes and hyphens. One application of regular expressions is to facilitate the construction of a compiler. Often, a large and complex regular expression is used to validate the syntax of a program. If the program code does not match the regular expression, the compiler knows that there's a syntax error in the code.

Class `String` provides several methods for performing regular-expression operations, the simplest of which is the matching operation. `String` method **`matches`** receives a `String` that specifies the regular expression and matches the contents of the `String` object on which it's called to the regular expression. The method returns a `boolean` indicating whether the match succeeded.

A regular expression consists of literal characters and special symbols. Figure 16.19 specifies some **predefined character classes** that can be used with regular expressions. A character class is an escape sequence that represents a group of characters. A digit is any numeric character. A **word character** is any letter (uppercase or lowercase), any digit or the underscore character. A white-space character is a space, a tab, a carriage return, a newline or a form feed. Each character class matches a single character in the `String` we're attempting to match with the regular expression.

Regular expressions are not limited to these predefined character classes. The expressions employ various operators and other forms of notation to match complex patterns.

Character	Matches	Character	Matches
`\d`	any digit	`\D`	any nondigit
`\w`	any word character	`\W`	any nonword character
`\s`	any white-space character	`\S`	any nonwhite-space character

Fig. 16.19 | Predefined character classes.

We examine several of these techniques in the application in Figs. 16.20 and 16.21, which validates user input via regular expressions. [*Note:* This application is not designed to match all possible valid user input.]

```
1  // Fig. 16.20: ValidateInput.java
2  // Validate user information using regular expressions.
3
4  public class ValidateInput
5  {
6     // validate first name
7     public static boolean validateFirstName( String firstName )
8     {
9        return firstName.matches( "[A-Z][a-zA-Z]*" );
10    } // end method validateFirstName
11
12    // validate last name
13    public static boolean validateLastName( String lastName )
14    {
15       return lastName.matches( "[a-zA-z]+([ '-][a-zA-Z]+)*" );
16    } // end method validateLastName
17
18    // validate address
19    public static boolean validateAddress( String address )
20    {
21       return address.matches(
22          "\\d+\\s+([a-zA-Z]+|[a-zA-Z]+\\s[a-zA-Z]+)" );
23    } // end method validateAddress
24
25    // validate city
26    public static boolean validateCity( String city )
27    {
28       return city.matches( "([a-zA-Z]+|[a-zA-Z]+\\s[a-zA-Z]+)" );
29    } // end method validateCity
30
31    // validate state
32    public static boolean validateState( String state )
33    {
34       return state.matches( "([a-zA-Z]+|[a-zA-Z]+\\s[a-zA-Z]+)" ) ;
35    } // end method validateState
36
37    // validate zip
38    public static boolean validateZip( String zip )
39    {
40       return zip.matches( "\\d{5}" );
41    } // end method validateZip
42
43    // validate phone
44    public static boolean validatePhone( String phone )
45    {
46       return phone.matches( "[1-9]\\d{2}-[1-9]\\d{2}-\\d{4}" );
47    } // end method validatePhone
48 } // end class ValidateInput
```

Fig. 16.20 | Validating user information using regular expressions.

```
1  // Fig. 16.21: Validate.java
2  // Validate user information using regular expressions.
3  import java.util.Scanner;
4
5  public class Validate
6  {
7     public static void main( String[] args )
8     {
9        // get user input
10       Scanner scanner = new Scanner( System.in );
11       System.out.println( "Please enter first name:" );
12       String firstName = scanner.nextLine();
13       System.out.println( "Please enter last name:" );
14       String lastName = scanner.nextLine();
15       System.out.println( "Please enter address:" );
16       String address = scanner.nextLine();
17       System.out.println( "Please enter city:" );
18       String city = scanner.nextLine();
19       System.out.println( "Please enter state:" );
20       String state = scanner.nextLine();
21       System.out.println( "Please enter zip:" );
22       String zip = scanner.nextLine();
23       System.out.println( "Please enter phone:" );
24       String phone = scanner.nextLine();
25
26       // validate user input and display error message
27       System.out.println( "\nValidate Result:" );
28
29       if ( !ValidateInput.validateFirstName( firstName ) )
30          System.out.println( "Invalid first name" );
31       else if ( !ValidateInput.validateLastName( lastName ) )
32          System.out.println( "Invalid last name" );
33       else if ( !ValidateInput.validateAddress( address ) )
34          System.out.println( "Invalid address" );
35       else if ( !ValidateInput.validateCity( city ) )
36          System.out.println( "Invalid city" );
37       else if ( !ValidateInput.validateState( state ) )
38          System.out.println( "Invalid state" );
39       else if ( !ValidateInput.validateZip( zip ) )
40          System.out.println( "Invalid zip code" );
41       else if ( !ValidateInput.validatePhone( phone ) )
42          System.out.println( "Invalid phone number" );
43       else
44          System.out.println( "Valid input.  Thank you." );
45    } // end main
46 } // end class Validate
```

```
Please enter first name:
Jane
Please enter last name:
Doe
Please enter address:
123 Some Street
```

Fig. 16.21 | Inputs and validates data from user using the `ValidateInput` class. (Part 1 of 2.)

```
Please enter city:
Some City
Please enter state:
SS
Please enter zip:
123
Please enter phone:
123-456-7890

Validate Result:
Invalid zip code
```

```
Please enter first name:
Jane
Please enter last name:
Doe
Please enter address:
123 Some Street
Please enter city:
Some City
Please enter state:
SS
Please enter zip:
12345
Please enter phone:
123-456-7890

Validate Result:
Valid input.  Thank you.
```

Fig. 16.21 | Inputs and validates data from user using the `ValidateInput` class. (Part 2 of 2.)

Figure 16.20 validates user input. Line 9 validates the first name. To match a set of characters that does not have a predefined character class, use square brackets, `[]`. For example, the pattern `"[aeiou]"` matches a single character that's a vowel. Character ranges are represented by placing a dash (-) between two characters. In the example, `"[A-Z]"` matches a single uppercase letter. If the first character in the brackets is `"^"`, the expression accepts any character other than those indicated. However, `"[^Z]"` is not the same as `"[A-Y]"`, which matches uppercase letters A–Y—`"[^Z]"` matches any character other than capital Z, including lowercase letters and nonletters such as the newline character. Ranges in character classes are determined by the letters' integer values. In this example, `"[A-Za-z]"` matches all uppercase and lowercase letters. The range `"[A-z]"` matches all letters and also matches those characters (such as [and \) with an integer value between uppercase Z and lowercase a (for more information on integer values of characters see Appendix B). Like predefined character classes, character classes delimited by square brackets match a single character in the search object.

In line 9, the asterisk after the second character class indicates that any number of letters can be matched. In general, when the regular-expression operator `"*"` appears in a regular expression, the application attempts to match zero or more occurrences of the subexpression immediately preceding the `"*"`. Operator `"+"` attempts to match one or

more occurrences of the subexpression immediately preceding "+". So both "A*" and "A+" will match "AAA" or "A", but only "A*" will match an empty string.

If method validateFirstName returns true (line 29 of Fig. 16.21), the application attempts to validate the last name (line 31) by calling validateLastName (lines 13–16 of Fig. 16.20). The regular expression to validate the last name matches any number of letters split by spaces, apostrophes or hyphens.

Line 33 of Fig. 16.21 calls method validateAddress (lines 19–23 of Fig. 16.20) to validate the address. The first character class matches any digit one or more times (\\d+). Two \ characters are used, because \ normally starts an escape sequence in a string. So \\d in a String represents the regular expression pattern \d. Then we match one or more white-space characters (\\s+). The character "|" matches the expression to its left or to its right. For example, "Hi (John|Jane)" matches both "Hi John" and "Hi Jane". The parentheses are used to group parts of the regular expression. In this example, the left side of | matches a single word, and the right side matches two words separated by any amount of white space. So the address must contain a number followed by one or two words. Therefore, "10 Broadway" and "10 Main Street" are both valid addresses in this example. The city (lines 26–29 of Fig. 16.20) and state (lines 32–35 of Fig. 16.20) methods also match any word of at least one character or, alternatively, any two words of at least one character if the words are separated by a single space, so both Waltham and West Newton would match.

Quantifiers

The asterisk (*) and plus (+) are formally called **quantifiers**. Figure 16.22 lists all the quantifiers. We've already discussed how the asterisk (*) and plus (+) quantifiers work. All quantifiers affect only the subexpression immediately preceding the quantifier. Quantifier question mark (?) matches zero or one occurrences of the expression that it quantifies. A set of braces containing one number ({*n*}) matches exactly *n* occurrences of the expression it quantifies. We demonstrate this quantifier to validate the zip code in Fig. 16.20 at line 40. Including a comma after the number enclosed in braces matches at least *n* occurrences of the quantified expression. The set of braces containing two numbers ({*n*,*m*}), matches between *n* and *m* occurrences of the expression that it qualifies. Quantifiers may be applied to patterns enclosed in parentheses to create more complex regular expressions.

Quantifier	Matches
*	Matches zero or more occurrences of the pattern.
+	Matches one or more occurrences of the pattern.
?	Matches zero or one occurrences of the pattern.
{*n*}	Matches exactly *n* occurrences.
{*n*,}	Matches at least *n* occurrences.
{*n*,*m*}	Matches between *n* and *m* (inclusive) occurrences.

Fig. 16.22 | Quantifiers used in regular expressions.

All of the quantifiers are **greedy**. This means that they'll match as many occurrences as they can as long as the match is still successful. However, if any of these quantifiers is

followed by a question mark (?), the quantifier becomes **reluctant** (sometimes called **lazy**). It then will match as few occurrences as possible as long as the match is still successful.

The zip code (line 40 in Fig. 16.20) matches a digit five times. This regular expression uses the digit character class and a quantifier with the digit 5 between braces. The phone number (line 46 in Fig. 16.20) matches three digits (the first one cannot be zero) followed by a dash followed by three more digits (again the first one cannot be zero) followed by four more digits.

`String` method `matches` checks whether an entire `String` conforms to a regular expression. For example, we want to accept `"Smith"` as a last name, but not `"9@Smith#"`. If only a substring matches the regular expression, method `matches` returns `false`.

Replacing Substrings and Splitting Strings

Sometimes it's useful to replace parts of a string or to split a string into pieces. For this purpose, class `String` provides methods **`replaceAll`**, **`replaceFirst`** and **`split`**. These methods are demonstrated in Fig. 16.23.

```
1  // Fig. 16.23: RegexSubstitution.java
2  // String methods replaceFirst, replaceAll and split.
3  import java.util.Arrays;
4
5  public class RegexSubstitution
6  {
7     public static void main( String[] args )
8     {
9        String firstString = "This sentence ends in 5 stars *****";
10       String secondString = "1, 2, 3, 4, 5, 6, 7, 8";
11
12       System.out.printf( "Original String 1: %s\n", firstString );
13
14       // replace '*' with '^'
15       firstString = firstString.replaceAll( "\\*", "^" );
16
17       System.out.printf( "^ substituted for *: %s\n", firstString );
18
19       // replace 'stars' with 'carets'
20       firstString = firstString.replaceAll( "stars", "carets" );
21
22       System.out.printf(
23          "\"carets\" substituted for \"stars\": %s\n", firstString );
24
25       // replace words with 'word'
26       System.out.printf( "Every word replaced by \"word\": %s\n\n",
27          firstString.replaceAll( "\\w+", "word" ) );
28
29       System.out.printf( "Original String 2: %s\n", secondString );
30
31       // replace first three digits with 'digit'
32       for ( int i = 0; i < 3; i++ )
33          secondString = secondString.replaceFirst( "\\d", "digit" );
34
```

Fig. 16.23 | `String` methods `replaceFirst`, `replaceAll` and `split`. (Part 1 of 2.)

```
35         System.out.printf(
36            "First 3 digits replaced by \"digit\" : %s\n", secondString );
37
38         System.out.print( "String split at commas: " );
39         String[] results = secondString.split( ",\\s*" ); // split on commas
40         System.out.println( Arrays.toString( results ) );
41      } // end main
42   } // end class RegexSubstitution
```

```
Original String 1: This sentence ends in 5 stars *****
^ substituted for *: This sentence ends in 5 stars ^^^^^
"carets" substituted for "stars": This sentence ends in 5 carets ^^^^^
Every word replaced by "word": word word word word word ^^^^^

Original String 2: 1, 2, 3, 4, 5, 6, 7, 8
First 3 digits replaced by "digit" : digit, digit, digit, 4, 5, 6, 7, 8
String split at commas: ["digit", "digit", "digit", "4", "5", "6", "7", "8"]
```

Fig. 16.23 | `String` methods `replaceFirst`, `replaceAll` and `split`. (Part 2 of 2.)

Method `replaceAll` replaces text in a `String` with new text (the second argument) wherever the original `String` matches a regular expression (the first argument). Line 15 replaces every instance of `"*"` in `firstString` with `"^"`. The regular expression (`"\\*"`) precedes character * with two backslashes. Normally, * is a quantifier indicating that a regular expression should match any number of occurrences of a preceding pattern. However, in line 15, we want to find all occurrences of the literal character *—to do this, we must escape character * with character \. Escaping a special regular-expression character with \ instructs the matching engine to find the actual character. Since the expression is stored in a Java `String` and \ is a special character in Java `String`s, we must include an additional \. So the Java `String` `"\\*"` represents the regular-expression pattern * which matches a single * character in the search string. In line 20, every match for the regular expression `"stars"` in `firstString` is replaced with `"carets"`. Line 27 uses `replaceAll` to replace all words in the string with `"word"`.

Method `replaceFirst` (line 33) replaces the first occurrence of a pattern match. Java `String`s are immutable; therefore, method `replaceFirst` returns a new `String` in which the appropriate characters have been replaced. This line takes the original `String` and replaces it with the `String` returned by `replaceFirst`. By iterating three times we replace the first three instances of a digit (`\d`) in `secondString` with the text `"digit"`.

Method `split` divides a `String` into several substrings. The original is broken in any location that matches a specified regular expression. Method `split` returns an array of `String`s containing the substrings between matches for the regular expression. In line 39, we use method `split` to tokenize a `String` of comma-separated integers. The argument is the regular expression that locates the delimiter. In this case, we use the regular expression `",\\s*"` to separate the substrings wherever a comma occurs. By matching any white-space characters, we eliminate extra spaces from the resulting substrings. The commas and white-space characters are not returned as part of the substrings. Again, the Java `String` `",\\s*"` represents the regular expression ,\s*. Line 40 uses `Arrays` method `toString` to display the contents of array `results` in square brackets and separated by commas.

Classes Pattern *and* Matcher

In addition to the regular-expression capabilities of class `String`, Java provides other classes in package `java.util.regex` that help developers manipulate regular expressions. Class **`Pattern`** represents a regular expression. Class **`Matcher`** contains both a regular-expression pattern and a `CharSequence` in which to search for the pattern.

`CharSequence` (package `java.lang`) is an interface that allows read access to a sequence of characters. The interface requires that the methods `charAt`, `length`, `subSequence` and `toString` be declared. Both `String` and `StringBuilder` implement interface `CharSequence`, so an instance of either of these classes can be used with class `Matcher`.



Common Programming Error 16.4

A regular expression can be tested against an object of any class that implements interface `CharSequence`*, but the regular expression must be a* `String`*. Attempting to create a regular expression as a* `StringBuilder` *is an error.*

If a regular expression will be used only once, `static` `Pattern` method **`matches`** can be used. This method takes a `String` that specifies the regular expression and a `CharSequence` on which to perform the match. This method returns a `boolean` indicating whether the search object (the second argument) matches the regular expression.

If a regular expression will be used more than once (in a loop, for example), it's more efficient to use `static` `Pattern` method **`compile`** to create a specific `Pattern` object for that regular expression. This method receives a `String` representing the pattern and returns a new `Pattern` object, which can then be used to call method **`matcher`**. This method receives a `CharSequence` to search and returns a `Matcher` object.

`Matcher` provides method **`matches`**, which performs the same task as `Pattern` method `matches`, but receives no arguments—the search pattern and search object are encapsulated in the `Matcher` object. Class `Matcher` provides other methods, including **`find`**, **`lookingAt`**, **`replaceFirst`** and **`replaceAll`**.

Figure 16.24 presents a simple example that employs regular expressions. This program matches birthdays against a regular expression. The expression matches only birthdays that do not occur in April and that belong to people whose names begin with `"J"`.

Lines 11–12 create a `Pattern` by invoking static `Pattern` method `compile`. The dot character **`.`** in the regular expression (line 12) matches any single character except a newline character.

```
1  // Fig. 16.24: RegexMatches.java
2  // Classes Pattern and Matcher.
3  import java.util.regex.Matcher;
4  import java.util.regex.Pattern;
5
6  public class RegexMatches
7  {
8     public static void main( String[] args )
9     {
10       // create regular expression
11       Pattern expression =
12          Pattern.compile( "J.*\\d[0-35-9]-\\d\\d-\\d\\d" );
```

Fig. 16.24 | Classes `Pattern` and `Matcher`. (Part 1 of 2.)

```
13
14          String string1 = "Jane's Birthday is 05-12-75\n" +
15             "Dave's Birthday is 11-04-68\n" +
16             "John's Birthday is 04-28-73\n" +
17             "Joe's Birthday is 12-17-77";
18
19          // match regular expression to string and print matches
20          Matcher matcher = expression.matcher( string1 );
21
22          while ( matcher.find() )
23             System.out.println( matcher.group() );
24       } // end main
25    } // end class RegexMatches
```

```
Jane's Birthday is 05-12-75
Joe's Birthday is 12-17-77
```

Fig. 16.24 | Classes `Pattern` and `Matcher`. (Part 2 of 2.)

Line 20 creates the `Matcher` object for the compiled regular expression and the matching sequence (`string1`). Lines 22–23 use a `while` loop to iterate through the `String`. Line 22 uses `Matcher` method `find` to attempt to match a piece of the search object to the search pattern. Each call to this method starts at the point where the last call ended, so multiple matches can be found. `Matcher` method `lookingAt` performs the same way, except that it always starts from the beginning of the search object and will always find the first match if there is one.



Common Programming Error 16.5

Method `matches` (from class `String`, `Pattern` or `Matcher`) will return `true` only if the entire search object matches the regular expression. Methods `find` and `lookingAt` (from class `Matcher`) will return `true` if a portion of the search object matches the regular expression.

Line 23 uses `Matcher` method **group**, which returns the `String` from the search object that matches the search pattern. The `String` that's returned is the one that was last matched by a call to `find` or `lookingAt`. The output in Fig. 16.24 shows the two matches that were found in `string1`.

For more information on regular expressions, visit our Regular Expressions Resource Center at `www.deitel.com/regularexpressions/`.

16.8 Wrap-Up

In this chapter, you learned about more `String` methods for selecting portions of `String`s and manipulating `String`s. You learned about the `Character` class and some of the methods it declares to handle `char`s. The chapter also discussed the capabilities of the `StringBuilder` class for creating `String`s. The end of the chapter discussed regular expressions, which provide a powerful capability to search and match portions of `String`s that fit a particular pattern. In the next chapter, you'll learn about file processing, including how persistent data is stored and and retrieved.

Summary

Section 16.2 Fundamentals of Characters and Strings

- A character literal's value (p. 673) is its integer value in Unicode (p. 673). Strings can include letters, digits and special characters such as +, -, *, / and $. A string in Java is an object of class `String`. `String` literals (p. 674) are often referred to as `String` objects and are written in a program in double quotes.

Section 16.3 Class `String`

- `String` objects are immutable (p. 675)—after they're created, their character contents cannot be changed.
- `String` method `length` (p. 675) returns the number of characters in a `String`.
- `String` method `charAt` (p. 675) returns the character at a specific position.
- `String` method `equals` tests for equality. The method returns `true` if the contents of the `String`s are equal, `false` otherwise. Method `equals` uses a lexicographical comparison (p. 679) for `String`s.
- When primitive-type values are compared with ==, the result is `true` if both values are identical. When references are compared with ==, the result is `true` if both refer to the same object.
- Java treats all string literals with the same contents as a single `String` object.
- `String` method `equalsIgnoreCase` performs a case-insensitive string comparison.
- `String` method `compareTo` uses a lexicographical comparison and returns 0 if the `String`s are equal, a negative number if the string that calls `compareTo` is less than the argument `String` and a positive number if the string that calls `compareTo` is greater than than the argument `String`.
- `String` method `regionMatches` (p. 677) compares portions of two strings for equality.
- `String` methods `startsWith` and `endsWith` (p. 680) determine whether a string starts with or ends with the specified characters, respectively.
- `String` method `indexOf` (p. 681) locates the first occurrence of a character or a substring in a string. `String` method `lastIndexOf` (p. 681) locates the last occurrence of a character or a substring in a string.
- `String` method `substring` copies and returns part of an existing string object.
- `String` method `concat` concatenates two string objects and returns a new string object.
- `String` method `replace` returns a new string object that replaces every occurrence in a `String` of its first character argument with its second character argument.
- `String` method `toUpperCase` (p. 685) returns a new string with uppercase letters in the positions where the original string had lowercase letters. `String` method `toLowerCase` (p. 686) returns a new string with lowercase letters in the positions where the original string had uppercase letters.
- `String` method `trim` (p. 686) returns a new string object in which all white-space characters (e.g., spaces, newlines and tabs) have been removed from the beginning and end of a string.
- `String` method `toCharArray` (p. 686) returns a `char` array containing a copy of the string's characters.
- `String` class `static` method `valueOf` returns its argument converted to a string.

Section 16.4 Class `StringBuilder`

- Class `StringBuilder` provides constructors that enable `StringBuilder`s to be initialized with no characters and an initial capacity of 16 characters, with no characters and an initial capacity specified in the integer argument, or with a copy of the characters of the `String` argument and an initial capacity that's the number of characters in the `String` argument plus 16.

- `StringBuilder` method `length` (p. 688) returns the number of characters currently stored in a `StringBuilder`. `StringBuilder` method `capacity` (p. 688) returns the number of characters that can be stored in a `StringBuilder` without allocating more memory.
- `StringBuilder` method `ensureCapacity` (p. 688) ensures that a `StringBuilder` has at least the specified capacity. Method `setLength` increases or decreases the length of a `StringBuilder`.
- `StringBuilder` method `charAt` (p. 690) returns the character at the specified index. Method `setCharAt` (p. 690) sets the character at the specified position. `StringBuilder` method `getChars` (p. 690) copies characters in the `StringBuilder` into the character array passed as an argument.
- `StringBuilder`'s overloaded `append` methods (p. 691) add primitive-type, character array, `String`, `Object` or `CharSequence` (p. 707) values to the end of a `StringBuilder`.
- `StringBuilder`'s overloaded `insert` (p. 693) methods insert primitive-type, character array, `String`, `Object` or `CharSequence` values at any position in a `StringBuilder`.

Section 16.5 Class `Character`

- `Character` method `isDefined` (p. 695) determines whether a character is in the Unicode character set.
- `Character` method `isDigit` (p. 695) determines whether a character is a defined Unicode digit.
- `Character` method `isJavaIdentifierStart` (p. 696) determines whether a character can be used as the first character of a Java identifier. `Character` method `isJavaIdentifierPart` (p. 696) determines if a character can be used in an identifier.
- `Character` method `isLetter` (p. 696) determines if a character is a letter. `Character` method `isLetterOrDigit` (p. 696) determines if a character is a letter or a digit.
- `Character` method `isLowerCase` (p. 696) determines whether a character is a lowercase letter. `Character` method `isUpperCase` (p. 696) determines whether a character is an uppercase letter.
- `Character` method `toUpperCase` (p. 696) converts a character to its uppercase equivalent. `Character` method `toLowerCase` (p. 696) converts a character to its lowercase equivalent.
- `Character` method `digit` (p. 696) converts its character argument into an integer in the number system specified by its integer argument `radix` (p. 697). `Character` method `forDigit` (p. 696) converts its integer argument `digit` into a character in the number system specified by its integer argument `radix`.
- `Character` method `charValue` (p. 698) returns the `char` stored in a `Character` object. `Character` method `toString` returns a `String` representation of a `Character`.

Section 16.6 Tokenizing `Strings`

- Class `String`'s `split` method (p. 705) tokenizes a `String` based on the delimiter (p. 699) specified as an argument and returns an array of `String`s containing the tokens (p. 699).

Section 16.7 Regular Expressions, Class `Pattern` *and Class* `Matcher`

- Regular expressions (p. 700) are sequences of characters and symbols that define a set of strings. They're useful for validating input and ensuring that data is in a particular format.
- `String` method `matches` (p. 700) receives a string that specifies the regular expression and matches the contents of the `String` object on which it's called to the regular expression. The method returns a `boolean` indicating whether the match succeeded.
- A character class is an escape sequence that represents a group of characters. Each character class matches a single character in the string we're attempting to match with the regular expression.
- A word character (`\w`; p. 700) is any letter (uppercase or lowercase), any digit or the underscore character.

- A white-space character (`\s`) is a space, a tab, a carriage return, a newline or a form feed.
- A digit (`\d`) is any numeric character.
- To match a set of characters that does not have a predefined character class (p. 700), use square brackets, `[]`. Ranges can be represented by placing a dash (`-`) between two characters. If the first character in the brackets is `"^"`, the expression accepts any character other than those indicated.
- When the regular expression operator `"*"` appears in a regular expression, the program attempts to match zero or more occurrences of the subexpression immediately preceding the `"*"`.
- Operator `"+"` attempts to match one or more occurrences of the subexpression preceding it.
- The character `"|"` allows a match of the expression to its left or to its right.
- Parentheses ( ) are used to group parts of the regular expression.
- The asterisk (`*`) and plus (`+`) are formally called quantifiers (p. 704).
- A quantifier affects only the subexpression immediately preceding it.
- Quantifier question mark (`?`) matches zero or one occurrences of the expression that it quantifies.
- A set of braces containing one number (`{`*n*`}`) matches exactly *n* occurrences of the expression it quantifies. Including a comma after the number enclosed in braces matches at least *n* occurrences.
- A set of braces containing two numbers (`{`*n*`,`*m*`}`) matches between *n* and *m* occurrences of the expression that it qualifies.
- Quantifiers are greedy (p. 704)—they'll match as many occurrences as they can as long as the match is successful. If a quantifier is followed by a question mark (`?`), the quantifier becomes reluctant (p. 705), matching as few occurrences as possible as long as the match is successful.
- `String` method `replaceAll` (p. 705) replaces text in a string with new text (the second argument) wherever the original string matches a regular expression (the first argument).
- Escaping a special regular-expression character with a `\` instructs the regular-expression matching engine to find the actual character, as opposed to what it represents in a regular expression.
- `String` method `replaceFirst` (p. 705) replaces the first occurrence of a pattern match and returns a new string in which the appropriate characters have been replaced.
- `String` method `split` divides a string into substrings at any location that matches a specified regular expression and returns an array of the substrings.
- Class `Pattern` (p. 707) represents a regular expression.
- Class `Matcher` (p. 707) contains a regular-expression pattern and a `CharSequence` in which to search.
- `CharSequence` is an interface (p. 707) that allows read access to a sequence of characters. Both `String` and `StringBuilder` implement this interface, so they can be used with class `Matcher`.
- If a regular expression will be used only once, static `Pattern` method `matches` (p. 707) takes a string that specifies the regular expression and a `CharSequence` on which to perform the match. This method returns a `boolean` indicating whether the search object matches the regular expression.
- If a regular expression will be used more than once, it's more efficient to use static `Pattern` method `compile` (p. 707) to create a specific `Pattern` object for that regular expression. This method receives a string representing the pattern and returns a new `Pattern` object.
- `Pattern` method `matcher` (p. 707) receives a `CharSequence` to search and returns a `Matcher` object. `Matcher` method `matches` (p. 707) performs the same task as `Pattern` method `matches` but without arguments.
- `Matcher` method `find` (p. 707) attempts to match a piece of the search object to the search pattern. Each call to this method starts at the point where the last call ended, so multiple matches can be found.

- `Matcher` method `lookingAt` (p. 707) performs the same as `find`, except that it always starts from the beginning of the search object and will always find the first match if there is one.
- `Matcher` method `group` (p. 708) returns the string from the search object that matches the search pattern. The string returned is the one that was last matched by a call to `find` or `lookingAt`.

Self-Review Exercises

16.1 State whether each of the following is *true* or *false*. If *false*, explain why.
a) When `String` objects are compared using ==, the result is `true` if the `Strings` contain the same values.
b) A `String` can be modified after it's created.

16.2 For each of the following, write a single statement that performs the indicated task:
a) Compare the string in `s1` to the string in `s2` for equality of contents.
b) Append the string `s2` to the string `s1`, using +=.
c) Determine the length of the string in `s1`.

Answers to Self-Review Exercises

16.1 a) False. String objects are compared using operator == to determine whether they're the same object in memory.
b) False. `String` objects are immutable and cannot be modified after they're created. `StringBuilder` objects can be modified after they're created.

16.2 a) `s1.equals( s2 )`
b) `s1 += s2;`
c) `s1.length()`

Exercises

16.3 *(Comparing `Strings`)* Write an application that uses `String` method `compareTo` to compare two strings input by the user. Output whether the first string is less than, equal to or greater than the second.

16.4 *(Comparing Portions of `Strings`)* Write an application that uses `String` method `regionMatches` to compare two strings input by the user. The application should input the number of characters to be compared and the starting index of the comparison. The application should state whether the strings are equal. Ignore the case of the characters when performing the comparison.

16.5 *(Random Sentences)* Write an application that uses random-number generation to create sentences. Use four arrays of strings called `article`, `noun`, `verb` and `preposition`. Create a sentence by selecting a word at random from each array in the following order: `article`, `noun`, `verb`, `preposition`, `article` and `noun`. As each word is picked, concatenate it to the previous words in the sentence. The words should be separated by spaces. When the final sentence is output, it should start with a capital letter and end with a period. The application should generate and display 20 sentences.

The article array should contain the articles `"the"`, `"a"`, `"one"`, `"some"` and `"any"`; the noun array should contain the nouns `"boy"`, `"girl"`, `"dog"`, `"town"` and `"car"`; the verb array should contain the verbs `"drove"`, `"jumped"`, `"ran"`, `"walked"` and `"skipped"`; the preposition array should contain the prepositions `"to"`, `"from"`, `"over"`, `"under"` and `"on"`.

16.6 *(Project: Limericks)* A limerick is a humorous five-line verse in which the first and second lines rhyme with the fifth, and the third line rhymes with the fourth. Using techniques similar to those developed in Exercise 16.5, write a Java application that produces random limericks. Polishing this application to produce good limericks is a challenging problem, but the result will be worth the effort!

16.7 *(Pig Latin)* Write an application that encodes English-language phrases into pig Latin. Pig Latin is a form of coded language. There are many different ways to form pig Latin phrases. For simplicity, use the following algorithm:

To form a pig Latin phrase from an English-language phrase, tokenize the phrase into words with `String` method `split`. To translate each English word into a pig Latin word, place the first letter of the English word at the end of the word and add the letters "ay." Thus, the word "jump" becomes "umpjay," the word "the" becomes "hetay," and the word "computer" becomes "omputer-cay." Blanks between words remain as blanks. Assume the following: The English phrase consists of words separated by blanks, there are no punctuation marks and all words have two or more letters. Method `printLatinWord` should display each word. Each token is passed to method `printLatinWord` to print the pig Latin word. Enable the user to input the sentence. Keep a running display of all the converted sentences in a text area.

16.8 *(Tokenizing Telephone Numbers)* Write an application that inputs a telephone number as a string in the form `(555) 555-5555`. The application should `String` method `split` to extract the area code as a token, the first three digits of the phone number as a token and the last four digits of the phone number as a token. The seven digits of the phone number should be concatenated into one string. Both the area code and the phone number should be printed. Remember that you'll have to change delimiter characters during the tokenization process.

16.9 *(Displaying a Sentence with Its Words Reversed)* Write an application that inputs a line of text, tokenizes the line with `String` method `split` and outputs the tokens in reverse order. Use space characters as delimiters.

16.10 *(Displaying `Strings` in Uppercase and Lowercase)* Write an application that inputs a line of text and outputs the text twice—once in all uppercase letters and once in all lowercase letters.

16.11 *(Searching `Strings`)* Write an application that inputs a line of text and a search character and uses `String` method `indexOf` to determine the number of occurrences of the character in the text.

16.12 *(Searching `Strings`)* Write an application based on the application in Exercise 16.11 that inputs a line of text and uses `String` method `indexOf` to determine the total number of occurrences of each letter of the alphabet in the text. Uppercase and lowercase letters should be counted together. Store the totals for each letter in an array, and print the values in tabular format after the totals have been determined.

16.13 *(Tokenizing and Comparing `Strings`)* Write an application that reads a line of text, tokenizes the line using space characters as delimiters and outputs only those words beginning with the letter `"b"`.

16.14 *(Tokenizing and Comparing `Strings`)* Write an application that reads a line of text, tokenizes it using space characters as delimiters and outputs only those words ending with the letters `"ED"`.

16.15 *(Converting `int` Values to Characters)* Write an application that inputs an integer code for a character and displays the corresponding character. Modify this application so that it generates all possible three-digit codes in the range from 000 to 255 and attempts to print the corresponding characters.

16.16 *(Defining Your Own `String` Methods)* Write your own versions of `String` search methods `indexOf` and `lastIndexOf`.

16.17 *(Creating Three-Letter `Strings` from a Five-Letter Word)* Write an application that reads a five-letter word from the user and produces every possible three-letter string that can be derived from the letters of that word. For example, the three-letter words produced from the word "bathe" include "ate," "bat," "bet," "tab," "hat," "the" and "tea."

Special Section: Advanced String-Manipulation Exercises

The preceding exercises are keyed to the text and designed to test your understanding of fundamental string-manipulation concepts. This section includes a collection of intermediate and advanced string-manipulation exercises. You should find these problems challenging, yet entertaining. The problems vary considerably in difficulty. Some require an hour or two of application writing and implementation. Others are useful for lab assignments that might require two or three weeks of study and implementation. Some are challenging term projects.

16.18 *(Text Analysis)* The availability of computers with string-manipulation capabilities has resulted in some rather interesting approaches to analyzing the writings of great authors. Much attention has been focused on whether William Shakespeare ever lived. Some scholars believe there's substantial evidence indicating that Christopher Marlowe actually penned the masterpieces attributed to Shakespeare. Researchers have used computers to find similarities in the writings of these two authors. This exercise examines three methods for analyzing texts with a computer.

a) Write an application that reads a line of text from the keyboard and prints a table indicating the number of occurrences of each letter of the alphabet in the text. For example, the phrase

```
To be, or not to be: that is the question:
```

contains one "a," two "b's," no "c's," and so on.

b) Write an application that reads a line of text and prints a table indicating the number of one-letter words, two-letter words, three-letter words, and so on, appearing in the text. For example, Fig. 16.25 shows the counts for the phrase

```
Whether 'tis nobler in the mind to suffer
```

Word length	Occurrences
1	0
2	2
3	1
4	2 (including `'tis`)
5	0
6	2
7	1

Fig. 16.25 | Word-length counts for the string `"Whether 'tis nobler in the mind to suffer"`.

c) Write an application that reads a line of text and prints a table indicating the number of occurrences of each different word in the text. The application should include the words in the table in the same order in which they appear in the text. For example, the lines

```
To be, or not to be: that is the question:
Whether 'tis nobler in the mind to suffer
```

contain the word "to" three times, the word "be" two times, the word "or" once, etc.

16.19 *(Printing Dates in Various Formats)* Dates are printed in several common formats. Two of the more common formats are

```
04/25/1955 and April 25, 1955
```

Write an application that reads a date in the first format and prints it in the second format.

16.20 ***(Check Protection)*** Computers are frequently employed in check-writing systems, such as payroll and accounts payable applications. There are many strange stories about weekly paychecks being printed (by mistake) for amounts in excess of $1 million. Incorrect amounts are printed by computerized check-writing systems because of human error or machine failure. Systems designers build controls into their systems to prevent such erroneous checks from being issued.

Another serious problem is the intentional alteration of a check amount by someone who plans to cash a check fraudulently. To prevent a dollar amount from being altered, some computerized check-writing systems employ a technique called check protection. Checks designed for imprinting by computer contain a fixed number of spaces in which the computer may print an amount. Suppose a paycheck contains eight blank spaces in which the computer is supposed to print the amount of a weekly paycheck. If the amount is large, then all eight of the spaces will be filled. For example,

```
1,230.60 (check amount)
--------
12345678 (position numbers)
```

On the other hand, if the amount is less than $1000, then several of the spaces would ordinarily be left blank. For example,

```
   99.87
--------
12345678
```

contains three blank spaces. If a check is printed with blank spaces, it's easier for someone to alter the amount. To prevent alteration, many check-writing systems insert *leading asterisks* to protect the amount as follows:

```
***99.87
--------
12345678
```

Write an application that inputs a dollar amount to be printed on a check, then prints the amount in check-protected format with leading asterisks if necessary. Assume that nine spaces are available for printing the amount.

16.21 ***(Writing the Word Equivalent of a Check Amount)*** Continuing the discussion in Exercise 16.20, we reiterate the importance of designing check-writing systems to prevent alteration of check amounts. One common security method requires that the amount be written in numbers and spelled out in words as well. Even if someone is able to alter the numerical amount of the check, it's extremely difficult to change the amount in words. Write an application that inputs a numeric check amount that's less than $1000 and writes the word equivalent of the amount. For example, the amount 112.43 should be written as

```
ONE hundred TWELVE and 43/100
```

16.22 ***(Morse Code)*** Perhaps the most famous of all coding schemes is the Morse code, developed by Samuel Morse in 1832 for use with the telegraph system. The Morse code assigns a series of dots and dashes to each letter of the alphabet, each digit, and a few special characters (e.g., period, comma, colon, semicolon). In sound-oriented systems, the dot represents a short sound and the dash a long sound. Other representations of dots and dashes are used with light-oriented systems and signal-flag systems. Separation between words is indicated by a space or, simply, the absence of a dot or dash. In a sound-oriented system, a space is indicated by a short time during which no sound is transmitted. The international version of the Morse code appears in Fig. 16.26.

Write an application that reads an English-language phrase and encodes it into Morse code. Also write an application that reads a phrase in Morse code and converts it into the English-language equivalent. Use one blank between each Morse-coded letter and three blanks between each Morse-coded word.

Character	Code	Character	Code	Character	Code
A	.-	N	-.	*Digits*	
B	-...	O	---	1	.----
C	-.-.	P	.--.	2	..---
D	-..	Q	--.-	3	...--
E	.	R	.-.	4	....-
F	..-.	S	...	5	.....
G	--.	T	-	6	-....
H	....	U	..-	7	--...
I	..	V	...-	8	---..
J	.---	W	.--	9	----.
K	-.-	X	-..-	0	-----
L	.-..	Y	-.--		
M	--	Z	--..		

Fig. 16.26 | Letters and digits as expressed in international Morse code.

16.23 *(Metric Conversions)* Write an application that will assist the user with metric conversions. Your application should allow the user to specify the names of the units as strings (i.e., centimeters, liters, grams, and so on, for the metric system and inches, quarts, pounds, and so on, for the English system) and should respond to simple questions, such as

```
"How many inches are in 2 meters?"
"How many liters are in 10 quarts?"
```

Your application should recognize invalid conversions. For example, the question

```
"How many feet are in 5 kilograms?"
```

is not meaningful because `"feet"` is a unit of length, whereas `"kilograms"` is a unit of mass.

Special Section: Challenging String-Manipulation Projects

16.24 *(Project: A Spelling Checker)* Many popular word-processing software packages have built-in spell checkers. In this project, you're asked to develop your own spell-checker utility. We make suggestions to help get you started. You should then consider adding more capabilities. Use a computerized dictionary (if you have access to one) as a source of words.

Why do we type so many words with incorrect spellings? In some cases, it's because we simply do not know the correct spelling, so we make a best guess. In some cases, it's because we transpose two letters (e.g., "defualt" instead of "default"). Sometimes we double-type a letter accidentally (e.g., "hanndy" instead of "handy"). Sometimes we type a nearby key instead of the one we intended (e.g., "biryhday" instead of "birthday"), and so on.

Design and implement a spell-checker application in Java. Your application should maintain an array `wordList` of strings. Enable the user to enter these strings. [*Note:* In Chapter 17, we introduce file processing. With this capability, you can obtain the words for the spell checker from a computerized dictionary stored in a file.]

Your application should ask a user to enter a word. The application should then look up that word in the `wordList` array. If the word is in the array, your application should print `"Word is spelled correctly."` If the word is not in the array, your application should print `"Word is not`

`spelled correctly`." Then your application should try to locate other words in `wordList` that might be the word the user intended to type. For example, you can try all possible single transpositions of adjacent letters to discover that the word "default" is a direct match to a word in `wordList`. Of course, this implies that your application will check all other single transpositions, such as "edfault," "dfeault," "deafult," "defalut" and "defautl." When you find a new word that matches one in `wordList`, print it in a message, such as

```
Did you mean "default"?
```

Implement other tests, such as replacing each double letter with a single letter, and any other tests you can develop to improve the value of your spell checker.

16.25 ***(Project: A Crossword Puzzle Generator)*** Most people have worked a crossword puzzle, but few have ever attempted to generate one. Generating a crossword puzzle is suggested here as a string-manipulation project requiring substantial sophistication and effort.

There are many issues the programmer must resolve to get even the simplest crossword-puzzle-generator application working. For example, how do you represent the grid of a crossword puzzle inside the computer? Should you use a series of strings or two-dimensional arrays?

The programmer needs a source of words (i.e., a computerized dictionary) that can be directly referenced by the application. In what form should these words be stored to facilitate the complex manipulations required by the application?

If you're really ambitious, you'll want to generate the clues portion of the puzzle, in which the brief hints for each across word and each down word are printed. Merely printing a version of the blank puzzle itself is not a simple problem.

Making a Difference

16.26 ***(Cooking with Healthier Ingredients)*** Obesity in America is increasing at an alarming rate. Check the map from the Centers for Disease Control and Prevention (CDC) at `www.cdc.gov/nccdphp/dnpa/Obesity/trend/maps/index.htm`, which shows obesity trends in the United States over the last 20 years. As obesity increases, so do occurrences of related problems (e.g., heart disease, high blood pressure, high cholesterol, type 2 diabetes). Write a program that helps users choose healthier ingredients when cooking, and helps those allergic to certain foods (e.g., nuts, gluten) find substitutes. The program should read a recipe from a `JTextArea` and suggest healthier replacements for some of the ingredients. For simplicity, your program should assume the recipe has no abbreviations for measures such as teaspoons, cups, and tablespoons, and uses numerical digits for quantities (e.g., 1 egg, 2 cups) rather than spelling them out (one egg, two cups). Some common substitutions are shown in Fig. 16.27. Your program should display a warning such as, "Always consult your physician before making significant changes to your diet."

Your program should take into consideration that replacements are not always one-for-one. For example, if a cake recipe calls for three eggs, it might reasonably use six egg whites instead. Conversion data for measurements and substitutes can be obtained at websites such as:

```
chinesefood.about.com/od/recipeconversionfaqs/f/usmetricrecipes.htm
www.pioneerthinking.com/eggsub.html
www.gourmetsleuth.com/conversions.htm
```

Your program should consider the user's health concerns, such as high cholesterol, high blood pressure, weight loss, gluten allergy, and so on. For high cholesterol, the program should suggest substitutes for eggs and dairy products; if the user wishes to lose weight, low-calorie substitutes for ingredients such as sugar should be suggested.

16.27 ***(Spam Scanner)*** Spam (or junk e-mail) costs U.S. organizations billions of dollars a year in spam-prevention software, equipment, network resources, bandwidth, and lost productivity. Research online some of the most common spam e-mail messages and words, and check your own

Ingredient	Substitution
1 cup sour cream	1 cup yogurt
1 cup milk	1/2 cup evaporated milk and 1/2 cup water
1 teaspoon lemon juice	1/2 teaspoon vinegar
1 cup sugar	1/2 cup honey, 1 cup molasses or 1/4 cup agave nectar
1 cup butter	1 cup margarine or yogurt
1 cup flour	1 cup rye or rice flour
1 cup mayonnaise	1 cup cottage cheese or 1/8 cup mayonnaise and 7/8 cup yogurt
1 egg	2 tablespoons cornstarch, arrowroot flour or potato starch or 2 egg whites or 1/2 of a large banana (mashed)
1 cup milk	1 cup soy milk
1/4 cup oil	1/4 cup applesauce
white bread	whole-grain bread

Fig. 16.27 | Ingredients and substitutions.

junk e-mail folder. Create a list of 30 words and phrases commonly found in spam messages. Write an application in which the user enters an e-mail message in a `JTextArea`. Then, scan the message for each of the 30 keywords or phrases. For each occurrence of one of these within the message, add a point to the message's "spam score." Next, rate the likelihood that the message is spam, based on the number of points it received.

16.28 ***(SMS Language)*** Short Message Service (SMS) is a communications service that allows sending text messages of 160 or fewer characters between mobile phones. With the proliferation of mobile phone use worldwide, SMS is being used in many developing nations for political purposes (e.g., voicing opinions and opposition), reporting news about natural disasters, and so on. For example, check out `comunica.org/radio2.0/archives/87`. Since the length of SMS messages is limited, SMS Language—abbreviations of common words and phrases in mobile text messages, e-mails, instant messages, etc.—is often used. For example, "in my opinion" is "imo" in SMS Language. Research SMS Language online. Write a GUI application in which the user can enter a message using SMS Language, then click a button to translate it into English (or your own language). Also provide a mechanism to translate text written in English (or your own language) into SMS Language. One potential problem is that one SMS abbreviation could expand into a variety of phrases. For example, IMO (as used above) could also stand for "International Maritime Organization," "in memory of," etc.

17

Files, Streams and Object Serialization



I can only assume that a "Do Not File" document is filed in a "Do Not File" file.
—Senator Frank Church
Senate Intelligence Subcommittee Hearing, 1975

Consciousness … does not appear to itself chopped up in bits. … A "river" or a "stream" are the metaphors by which it is most naturally described.
—William James

Objectives

In this chapter you'll learn:

- To create, read, write and update files.
- To retrieve information about files and directories.
- The Java input/output stream class hierarchy.
- The differences between text files and binary files.
- To use classes `Scanner` and `Formatter` to process text files.
- To use classes `FileInputStream` and `FileOutputStream` to read from and write to files.
- To use classes `ObjectInputStream` and `ObjectOutputStream` to read objects from and write objects to files.
- To use a `JFileChooser` dialog.

Outline

17.1 Introduction
17.2 Files and Streams
17.3 Class `File`
17.4 Sequential-Access Text Files
- 17.4.1 Creating a Sequential-Access Text File
- 17.4.2 Reading Data from a Sequential-Access Text File
- 17.4.3 Case Study: A Credit-Inquiry Program
- 17.4.4 Updating Sequential-Access Files

17.5 Object Serialization
- 17.5.1 Creating a Sequential-Access File Using Object Serialization
- 17.5.2 Reading and Deserializing Data from a Sequential-Access File

17.6 Additional `java.io` Classes
- 17.6.1 Interfaces and Classes for Byte-Based Input and Output
- 17.6.2 Interfaces and Classes for Character-Based Input and Output

17.8 Opening Files with `JFileChooser`
17.8 Wrap-Up

Summary | *Self-Review Exercises* | *Answers to Self-Review Exercises* | *Exercises* | *Making a Difference*

17.1 Introduction[1]

Data stored in variables and arrays is temporary—it's lost when a local variable goes out of scope or when the program terminates. For long-term retention of data, even after the programs that create the data terminate, computers use **files**. You use files every day for tasks such as writing a document or creating a spreadsheet. Computers store files on **secondary storage devices** such as hard disks, optical disks, flash drives and magnetic tapes. Data maintained in files is **persistent data**—it exists beyond the duration of program execution. In this chapter, we explain how Java programs create, update and process files.

We begin with a discussion of Java's architecture for handling files programmatically. Next we explain that data can be stored in text files and binary files—and we cover the differences between them. We demonstrate retrieving information about files and directories using class `File`, then devote several sections to the different mechanisms for writing data to and reading data from files. We show how to create and manipulate sequential-access text files. Working with text files allows you to quickly and easily start manipulating files. As you'll learn, however, it's difficult to read data from text files back into object form. Fortunately, many object-oriented languages (including Java) provide ways to write objects to and read objects from files (known as object serialization and deserialization). To demonstrate this, we recreate some of our sequential-access programs that used text files, this time by storing objects in binary files.

17.2 Files and Streams

Java views each file as a sequential **stream of bytes** (Fig. 17.1). Every operating system provides a mechanism to determine the end of a file, such as an **end-of-file marker** or a count of the total bytes in the file that's recorded in a system-maintained administrative data structure. A Java program processing a stream of bytes simply receives an indication from the operating system when it reaches the end of the stream—the program does *not* need to know how the underlying platform represents files or streams. In some cases, the end-

1. The techniques shown in this chapter are based on Java SE 6. Java SE 7 introduces new file-system APIs for interacting with files and directories. On the book's Companion Website (accessible via `www.pearsonhighered.com/deitel`) we've posted a version of this chapter implemented using these Java SE 7 APIs.

of-file indication occurs as an exception. In other cases, the indication is a return value from a method invoked on a stream-processing object.



Fig. 17.1 | Java's view of a file of *n* bytes.

Byte-Based and Character-Based Streams

File streams can be used to input and output data as bytes or characters. **Byte-based streams** input and output data in its binary format. **Character-based streams** input and output data as a sequence of characters. If the value `5` were being stored using a byte-based stream, it would be stored in the binary format of the numeric value `5`, or `101`. If the value `5` were being stored using a character-based stream, it would be stored in the binary format of the character `5`, or `00000000 00110101` (this is the binary representation for the numeric value `53`, which indicates the Unicode® character `5`). The difference between the two forms is that the numeric value can be used as an integer in calculations, whereas the character `5` is simply a character that can be used in a string of text, as in `"Sarah Miller is 15 years old"`. Files that are created using byte-based streams are referred to as **binary files**, while files created using character-based streams are referred to as **text files**. Text files can be read by text editors, while binary files are read by programs that understand the file's specific content and its ordering.

Standard Input, Standard Output and Standard Error Streams

A Java program **opens** a file by creating an object and associating a stream of bytes or characters with it. The object's constructor interacts with the operating system to open the file. Java can also associate streams with different devices. When a Java program begins executing, in fact, it creates three stream objects that are associated with devices—`System.in`, `System.out` and `System.err`. `System.in` (the standard input stream object) normally enables a program to input bytes from the keyboard; object `System.out` (the standard output stream object) normally enables a program to output character data to the screen; and object `System.err` (the standard error stream object) normally enables a program to output character-based error messages to the screen. Each stream can be redirected. For `System.in`, this capability enables the program to read bytes from a different source. For `System.out` and `System.err`, it enables the output to be sent to a different location, such as a file on disk. Class `System` provides methods **`setIn`**, **`setOut`** and **`setErr`** to **redirect** the standard input, output and error streams, respectively.

The `java.io` Package

Java programs perform file processing by using classes from package **`java.io`**. This package includes definitions for stream classes, such as **`FileInputStream`** (for byte-based input from a file), **`FileOutputStream`** (for byte-based output to a file), **`FileReader`** (for character-based input from a file) and **`FileWriter`** (for character-based output to a file), which inherit from classes `InputStream`, `OutputStream`, `Reader` and `Writer`, respectively. Thus, the methods of the these stream classes can also be applied to file streams.

Java contains classes that enable you to perform input and output of objects or variables of primitive data types. The data will still be stored as bytes or characters behind the scenes, allowing you to read or write data in the form of `int`s, `String`s, or other types without having to worry about the details of converting such values to byte format. To perform such input and output, objects of classes **`ObjectInputStream`** and **`ObjectOutputStream`** can be used together with the byte-based file stream classes `FileInputStream` and `FileOutputStream` (these classes will be discussed in more detail shortly). The complete hierarchy of types in package `java.io` can be viewed in the online documentation at

```
download.oracle.com/javase/6/docs/api/java/io/package-tree.html
```

As you can see in the hierarchy, Java offers many classes for performing input/output operations. We use several of these classes in this chapter to implement file-processing programs that create and manipulate sequential-access files. In Chapter 27, we use stream classes extensively to implement networking applications.

In addition to the `java.io` classes, character-based input and output can be performed with classes `Scanner` and **`Formatter`**. Class `Scanner` is used extensively to input data from the keyboard—it can also read data from a file. Class `Formatter` enables formatted data to be output to any text-based stream in a manner similar to method `System.out.printf`. Appendix G presents the details of formatted output with `printf`. All these features can be used to format text files as well.

17.3 Class `File`

This section presents class **`File`**, which is useful for retrieving information about files or directories from disk. Objects of class `File` do not open files or provide any file-processing capabilities. However, `File` objects are used frequently with objects of other `java.io` classes to specify files or directories to manipulate.

Creating `File` Objects

Class `File` provides four constructors. The one with a `String` argument specifies the `name` of a file or directory to associate with the `File` object. The `name` can contain **path information** as well as a file or directory name. A file or directory's path specifies its location on disk. The path includes some or all of the directories leading to the file or directory. An **absolute path** contains all the directories, starting with the **root directory**, that lead to a specific file or directory. Every file or directory on a particular disk drive has the same root directory in its path. A **relative path** normally starts from the directory in which the application began executing and is therefore "relative" to the current directory. The constructor with two `String` arguments specifies an absolute or relative path as the first argument and the file or directory to associate with the `File` object as the second argument. The constructor with `File` and `String` arguments uses an existing `File` object that specifies the parent directory of the file or directory specified by the `String` argument. The fourth constructor uses a `URI` object to locate the file. A **Uniform Resource Identifier (URI)** is a more general form of the **Uniform Resource Locators (URLs)** that are used to locate websites. For example, `http://www.deitel.com/` is the URL for the Deitel & Associates website. URIs for locating files vary across operating systems. On Windows platforms, the URI

```
file://C:/data.txt
```

identifies the file data.txt stored in the root directory of the C: drive. On UNIX/Linux platforms, the URI

```
file:/home/student/data.txt
```

identifies the file data.txt stored in the home directory of the user student.

Figure 17.2 lists some common File methods. The complete list can be viewed at download.oracle.com/javase/6/docs/api/java/io/File.html.

Method	Description
boolean canRead()	Returns true if a file is readable by the current application; false otherwise.
boolean canWrite()	Returns true if a file is writable by the current application; false otherwise.
boolean exists()	Returns true if the file or directory represented by the File object exists; false otherwise.
boolean isFile()	Returns true if the name specified as the argument to the File constructor is a file; false otherwise.
boolean isDirectory()	Returns true if the name specified as the argument to the File constructor is a directory; false otherwise.
boolean isAbsolute()	Returns true if the arguments specified to the File constructor indicate an absolute path to a file or directory; false otherwise.
String getAbsolutePath()	Returns a String with the absolute path of the file or directory.
String getName()	Returns a String with the name of the file or directory.
String getPath()	Returns a String with the path of the file or directory.
String getParent()	Returns a String with the parent directory of the file or directory (i.e., the directory in which the file or directory is located).
long length()	Returns the length of the file, in bytes. If the File object represents a directory, an unspecified value is returned.
long lastModified()	Returns a platform-dependent representation of the time at which the file or directory was last modified. The value returned is useful only for comparison with other values returned by this method.
String[] list()	Returns an array of Strings representing a directory's contents. Returns null if the File object does not represent a directory.

Fig. 17.2 | File methods.

Demonstrating Class File

Figure 17.3 prompts the user to enter the name of a file or directory, then uses class File to output information about the file or directory.

The program begins by prompting the user for a file or directory (line 12). Line 13 inputs the file name or directory name and passes it to method analyzePath (lines 17–50). The method creates a new File object (line 20) and assigns its reference to name. Line 22 invokes File method exists to determine whether the name input by the user exists

```
1  // Fig. 17.3: FileDemonstration.java
2  // File class used to obtain file and directory information.
3  import java.io.File;
4  import java.util.Scanner;
5
6  public class FileDemonstration
7  {
8     public static void main( String[] args )
9     {
10       Scanner input = new Scanner( System.in );
11
12       System.out.print( "Enter file or directory name: " );
13       analyzePath( input.nextLine() );
14    } // end main
15
16    // display information about file user specifies
17    public static void analyzePath( String path )
18    {
19       // create File object based on user input
20       File name = new File( path );
21
22       if ( name.exists() ) // if name exists, output information about it
23       {
24          // display file (or directory) information
25          System.out.printf(
26             "%s%s\n%s\n%s\n%s\n%s%s\n%s%s\n%s%s\n%s%s\n%s%s",
27             name.getName(), " exists",
28             ( name.isFile() ? "is a file" : "is not a file" ),
29             ( name.isDirectory() ? "is a directory" :
30                "is not a directory" ),
31             ( name.isAbsolute() ? "is absolute path" :
32                "is not absolute path" ), "Last modified: ",
33             name.lastModified(), "Length: ", name.length(),
34             "Path: ", name.getPath(), "Absolute path: ",
35             name.getAbsolutePath(), "Parent: ", name.getParent() );
36
37          if ( name.isDirectory() ) // output directory listing
38          {
39             String[] directory = name.list();
40             System.out.println( "\n\nDirectory contents:\n" );
41
42             for ( String directoryName : directory )
43                System.out.println( directoryName );
44          } // end if
45       } // end outer if
46       else // not file or directory, output error message
47       {
48          System.out.printf( "%s %s", path, "does not exist." );
49       } // end else
50    } // end method analyzePath
51 } // end class FileDemonstration
```

Fig. 17.3 | `File` class used to obtain file and directory information. (Part 1 of 2.)

```
Enter file or directory name: E:\Program Files\Java\jdk1.6.0_11\demo\jfc
jfc exists
is not a file
is a directory
is absolute path
Last modified: 1228404395024
Length: 4096
Path: E:\Program Files\Java\jdk1.6.0_11\demo\jfc
Absolute path: E:\Program Files\Java\jdk1.6.0_11\demo\jfc
Parent: E:\Program Files\Java\jdk1.6.0_11\demo

Directory contents:

CodePointIM
FileChooserDemo
Font2DTest
Java2D
Laffy
Metalworks
Notepad
SampleTree
Stylepad
SwingApplet
SwingSet2
SwingSet3
```

```
Enter file or directory name: C:\Program Files\Java\jdk1.6.0_11\demo\jfc
\Java2D\README.txt
README.txt exists
is a file
is not a directory
is absolute path
Last modified: 1228404384270
Length: 7518
Path: E:\Program Files\Java\jdk1.6.0_11\demo\jfc\Java2D\README.txt
Absolute path: E:\Program Files\Java\jdk1.6.0_11\demo\jfc\Java2D\README.txt
Parent: E:\Program Files\Java\jdk1.6.0_11\demo\jfc\Java2D
```

Fig. 17.3 | `File` class used to obtain file and directory information. (Part 2 of 2.)

(either as a file or as a directory) on the disk. If the name does not exist, control proceeds to lines 46–49 and displays a message to the screen containing the name the user typed, followed by "`does not exist`." Otherwise, the `if` statement (lines 22–45) executes. The program outputs the name of the file or directory (line 27), followed by the results of testing the `File` object with `isFile` (line 28), `isDirectory` (line 29) and `isAbsolute` (line 31). Next, the program displays the values returned by `lastModified` (line 33), `length` (line 33), `getPath` (line 34), `getAbsolutePath` (line 35) and `getParent` (line 35). If the `File` object represents a directory (line 37), the program obtains a list of the directory's contents as an array of `String`s by using `File` method `list` (line 39) and displays the list on the screen.

The first output of this program demonstrates a `File` object associated with the `jfc` directory from the JDK. The second output demonstrates a `File` object associated with

the `README.txt` file from the Java 2D example that comes with the JDK. In both cases, we specified an absolute path on our computer.

A **separator character** is used to separate directories and files in the path. On a Windows computer, the separator character is a backslash (\). On a UNIX system, it's a forward slash (/). Java processes both characters identically in a path name. For example, if we were to use the path

```
c:\Program Files\Java\jdk1.6.0_11\demo/jfc
```

which employs each separator character, Java would still process the path properly. When building `String`s that represent path information, use `File.separator` to obtain the local computer's proper separator character rather than explicitly using `/` or `\`. This constant returns a `String` consisting of one character—the proper separator for the system.



Common Programming Error 17.1

Using `\` as a directory separator rather than `\\` in a string literal is a logic error. A single `\` indicates that the `\` followed by the next character represents an escape sequence. Use `\\` to insert a `\` in a string literal.

17.4 Sequential-Access Text Files

Next, we create and manipulate sequential-access files in which records are stored in order by the record-key field. We begin with text files, enabling the reader to quickly create and edit human-readable files. We discuss creating, writing data to, reading data from and updating sequential-access text files. We also include a credit-inquiry program that retrieves specific data from a file.

17.4.1 Creating a Sequential-Access Text File

Java imposes no structure on a file—notions such as records do not exist as part of the Java language. Therefore, you must structure files to meet the requirements of your applications. In the following example, we see how to impose a keyed record structure on a file.

The program in Figs. 17.4, 17.5 and 17.8 creates a simple sequential-access file that might be used in an accounts receivable system to keep track of the amounts owed to a company by its credit clients. For each client, the program obtains from the user an account number and the client's name and balance (i.e., the amount the client owes the company for goods and services received). Each client's data constitutes a "record" for that client. This application uses the account number as the record key—the file will be created and maintained in account-number order. The program assumes that the user enters the records in account-number order. In a comprehensive accounts receivable system (based on sequential-access files), a sorting capability would be provided so that the user could enter the records in any order. The records would then be sorted and written to the file.

Class `AccountRecord`

Class `AccountRecord` (Fig. 17.4) encapsulates the client record information used by the examples in this chapter. `AccountRecord` is declared in package `com.deitel.ch17` (line 3), so that it can be imported into several of this chapter's examples for reuse. (Section 8.14 provides information on compiling and using your own packages.) Class `AccountRecord` contains `private` instance variables `account`, `firstName`, `lastName` and `balance` (lines 7–

10) and *set* and *get* methods for accessing these fields. Though the *set* methods do not validate the data in this example, they should do so in an "industrial-strength" system.

```
1  // Fig. 17.4: AccountRecord.java
2  // AccountRecord class maintains information for one account.
3  package com.deitel.ch17; // packaged for reuse
4
5  public class AccountRecord
6  {
7     private int account;
8     private String firstName;
9     private String lastName;
10    private double balance;
11
12    // no-argument constructor calls other constructor with default values
13    public AccountRecord()
14    {
15       this( 0, "", "", 0.0 ); // call four-argument constructor
16    } // end no-argument AccountRecord constructor
17
18    // initialize a record
19    public AccountRecord( int acct, String first, String last, double bal )
20    {
21       setAccount( acct );
22       setFirstName( first );
23       setLastName( last );
24       setBalance( bal );
25    } // end four-argument AccountRecord constructor
26
27    // set account number
28    public void setAccount( int acct )
29    {
30       account = acct;
31    } // end method setAccount
32
33    // get account number
34    public int getAccount()
35    {
36       return account;
37    } // end method getAccount
38
39    // set first name
40    public void setFirstName( String first )
41    {
42       firstName = first;
43    } // end method setFirstName
44
45    // get first name
46    public String getFirstName()
47    {
48       return firstName;
49    } // end method getFirstName
```

Fig. 17.4 | `AccountRecord` class maintains information for one account. (Part 1 of 2.)

```
50
51    // set last name
52    public void setLastName( String last )
53    {
54       lastName = last;
55    } // end method setLastName
56
57    // get last name
58    public String getLastName()
59    {
60       return lastName;
61    } // end method getLastName
62
63    // set balance
64    public void setBalance( double bal )
65    {
66       balance = bal;
67    } // end method setBalance
68
69    // get balance
70    public double getBalance()
71    {
72       return balance;
73    } // end method getBalance
74 } // end class AccountRecord
```

Fig. 17.4 | `AccountRecord` class maintains information for one account. (Part 2 of 2.)

To compile class `AccountRecord`, open a command window, change directories to this chapter's `fig17_05` directory (which contains `AccountRecord.java`), then type:

```
javac -d .. AccountRecord.java
```

This places `AccountRecord.class` in its package directory structure and places the package in the `ch17` folder that contains all the examples for this chapter. When you compile class `AccountRecord` (or any other classes that will be reused in this chapter), you should place them in a common directory. When you compile or execute classes that use class `AccountRecord` (e.g., `CreateTextFile` in Fig. 17.5), you must specify the command-line argument `-classpath` to both `javac` and `java`, as in

```
javac -classpath .;c:\examples\ch17 CreateTextFile.java
java -classpath .;c:\examples\ch17 CreateTextFile
```

The current directory (specified with `.`) is included in the classpath to ensure that the compiler can locate other classes in the same directory as the class being compiled. The path separator used in the preceding commands must be appropriate for your platform—a semicolon (`;`) on Windows and a colon (`:`) on UNIX/Linux/Mac OS X. The preceding commands assume that the package containing `AccountRecord` is located at in the directory `C:\examples\ch17` on a Windows computer.

Class `CreateTextFile`

Now let's examine class `CreateTextFile` (Fig. 17.5). Line 14 declares `Formatter` variable `output`. As discussed in Section 17.2, a `Formatter` object outputs formatted `String`s, us-

ing the same formatting capabilities as method `System.out.printf`. A `Formatter` object can output to various locations, such as the screen or a file, as is done here. The `Formatter` object is instantiated in line 21 in method `openFile` (lines 17–34). The constructor used in line 21 takes one argument—a `String` containing the name of the file, including its path. If a path is not specified, as is the case here, the JVM assumes that the file is in the directory from which the program was executed. For text files, we use the `.txt` file extension. If the file does not exist, it will be created. If an existing file is opened, its contents are **truncated**—all the data in the file is discarded. At this point the file is open for writing, and the resulting `Formatter` object can be used to write data to the file.

```
1  // Fig. 17.5: CreateTextFile.java
2  // Writing data to a sequential text file with class Formatter.
3  import java.io.FileNotFoundException;
4  import java.lang.SecurityException;
5  import java.util.Formatter;
6  import java.util.FormatterClosedException;
7  import java.util.NoSuchElementException;
8  import java.util.Scanner;
9
10 import com.deitel.ch17.AccountRecord;
11
12 public class CreateTextFile
13 {
14    private Formatter output; // object used to output text to file
15
16    // enable user to open file
17    public void openFile()
18    {
19       try
20       {
21          output = new Formatter( "clients.txt" ); // open the file
22       } // end try
23       catch ( SecurityException securityException )
24       {
25          System.err.println(
26             "You do not have write access to this file." );
27          System.exit( 1 ); // terminate the program
28       } // end catch
29       catch ( FileNotFoundException fileNotFoundException )
30       {
31          System.err.println( "Error opening or creating file." );
32          System.exit( 1 ); // terminate the program
33       } // end catch
34    } // end method openFile
35
36    // add records to file
37    public void addRecords()
38    {
39       // object to be written to file
40       AccountRecord record = new AccountRecord();
```

Fig. 17.5 | Writing data to a sequential text file with class `Formatter`. (Part 1 of 3.)

```
41
42          Scanner input = new Scanner( System.in );
43
44          System.out.printf( "%s\n%s\n%s\n%s\n\n",
45             "To terminate input, type the end-of-file indicator ",
46             "when you are prompted to enter input.",
47             "On UNIX/Linux/Mac OS X type <ctrl> d then press Enter",
48             "On Windows type <ctrl> z then press Enter" );
49
50          System.out.printf( "%s\n%s",
51             "Enter account number (> 0), first name, last name and balance.",
52             "? " );
53
54          while ( input.hasNext() ) // loop until end-of-file indicator
55          {
56             try // output values to file
57             {
58                // retrieve data to be output
59                record.setAccount( input.nextInt() ); // read account number
60                record.setFirstName( input.next() ); // read first name
61                record.setLastName( input.next() ); // read last name
62                record.setBalance( input.nextDouble() ); // read balance
63
64                if ( record.getAccount() > 0 )
65                {
66                   // write new record
67                   output.format( "%d %s %s %.2f\n", record.getAccount(),
68                      record.getFirstName(), record.getLastName(),
69                      record.getBalance() );
70                } // end if
71                else
72                {
73                   System.out.println(
74                      "Account number must be greater than 0." );
75                } // end else
76             } // end try
77             catch ( FormatterClosedException formatterClosedException )
78             {
79                System.err.println( "Error writing to file." );
80                return;
81             } // end catch
82             catch ( NoSuchElementException elementException )
83             {
84                System.err.println( "Invalid input. Please try again." );
85                input.nextLine(); // discard input so user can try again
86             } // end catch
87
88             System.out.printf( "%s %s\n%s", "Enter account number (>0),",
89                "first name, last name and balance.", "? " );
90          } // end while
91       } // end method addRecords
92
```

Fig. 17.5 | Writing data to a sequential text file with class `Formatter`. (Part 2 of 3.)

```
93      // close file
94      public void closeFile()
95      {
96         if ( output != null )
97            output.close();
98      } // end method closeFile
99   } // end class CreateTextFile
```

Fig. 17.5 | Writing data to a sequential text file with class `Formatter`. (Part 3 of 3.)

Lines 23–28 handle the **`SecurityException`**, which occurs if the user does not have permission to write data to the file. Lines 29–33 handle the **`FileNotFoundException`**, which occurs if the file does not exist and a new file cannot be created. This exception may also occur if there's an error opening the file. In both exception handlers we call `static` method **`System.exit`** and pass the value `1`. This method terminates the application. An argument of `0` to method `exit` indicates successful program termination. A nonzero value, such as `1` in this example, normally indicates that an error has occurred. This value is passed to the command window that executed the program. The argument is useful if the program is executed from a **batch file** on Windows systems or a **shell script** on UNIX/Linux/Mac OS X systems. Batch files and shell scripts offer a convenient way of executing several programs in sequence. When the first program ends, the next program begins execution. It's possible to use the argument to method `exit` in a batch file or shell script to determine whether other programs should execute. For more information on batch files or shell scripts, see your operating system's documentation.

Method `addRecords` (lines 37–91) prompts the user to enter the various fields for each record or to enter the end-of-file key sequence when data entry is complete. Figure 17.6 lists the key combinations for entering end-of-file for various computer systems.

Operating system	Key combination
UNIX/Linux/Mac OS X	*<Enter> <Ctrl> d*
Windows	*<Ctrl> z*

Fig. 17.6 | End-of-file key combinations.

Line 40 creates an `AccountRecord` object, which will be used to store the values of the current record entered by the user. Line 42 creates a `Scanner` object to read input from the user at the keyboard. Lines 44–48 and 50–52 prompt the user for input.

Line 54 uses `Scanner` method `hasNext` to determine whether the end-of-file key combination has been entered. The loop executes until `hasNext` encounters end-of-file.

Lines 59–62 read data from the user, storing the record information in the `AccountRecord` object. Each statement throws a **`NoSuchElementException`** (handled in lines 82–86) if the data is in the wrong format (e.g., a `String` when an `int` is expected) or if there's no more data to input. If the account number is greater than 0 (line 64), the record's information is written to `clients.txt` (lines 67–69) using method **`format`**, which can perform identical formatting to the `System.out.printf` method used extensively in earlier chapters. Method `format` outputs a formatted `String` to the output destination of

the `Formatter` object—the file `clients.txt`. The format string `"%d %s %s %.2f\n"` indicates that the current record will be stored as an integer (the account number) followed by a `String` (the first name), another `String` (the last name) and a floating-point value (the balance). Each piece of information is separated from the next by a space, and the double value (the balance) is output with two digits to the right of the decimal point (as indicated by the `.2` in `%.2f`). The data in the text file can be viewed with a text editor or retrieved later by a program designed to read the file (Section 17.4.2).

When lines 67–69 execute, if the `Formatter` object is closed, a **`FormatterClosedException`** will be thrown. This exception is handled in lines 77–81. [*Note:* You can also output data to a text file using class **`java.io.PrintWriter`**, which provides `format` and `printf` methods for outputting formatted data.]

Lines 94–98 declare method `closeFile`, which closes the `Formatter` and the underlying output file. Line 97 closes the object by simply calling method **`close`**. If method `close` is not called explicitly, the operating system normally will close the file when program execution terminates—this is an example of operating-system "housekeeping." However, you should always explicitly close a file when it's no longer needed.

Platform-Specific Line-Separator Characters

Lines 67–69 output a line of text followed by a newline (`\n`). If you use a text editor to open the `clients.txt` file produced, each record might not display on a separate line. For example, in Notepad (Microsoft Windows), users will see one continuous line of text. This occurs because different platforms use different line-separator characters. On UNIX/Linux/Mac OS X, the line separator is a newline (`\n`). On Windows, it's a combination of a carriage return and a line feed—represented as `\r\n`. You can use the **`%n`** format specifier in a format control string to output a platform-specific line separator, thus ensuring that the text file can be opened and viewed correctly in a text editor for the platform on which the file was created. The method `System.out.println` outputs a platform-specific line separator after its argument. Also, regardless of the line separator used in a text file, a Java program can still recognize the lines of text and read them.

Class `CreateTextFileTest`

Figure 17.7 runs the program. Line 8 creates a `CreateTextFile` object, which is then used to open, add records to and close the file (lines 10–12). The sample data for this application is shown in Fig. 17.8. In the sample execution for this program, the user enters information for five accounts, then enters end-of-file to signal that data entry is complete. The sample execution does not show how the data records actually appear in the file. In the next section, to verify that the file has been created successfully, we present a program that reads the file and prints its contents. Because this is a text file, you can also verify the information simply by opening the file in a text editor.

```
1  // Fig. 17.7: CreateTextFileTest.java
2  // Testing the CreateTextFile class.
3
4  public class CreateTextFileTest
5  {
```

Fig. 17.7 | Testing the `CreateTextFile` class. (Part 1 of 2.)

```
6      public static void main( String[] args )
7      {
8         CreateTextFile application = new CreateTextFile();
9
10        application.openFile();
11        application.addRecords();
12        application.closeFile();
13     } // end main
14  } // end class CreateTextFileTest
```

```
To terminate input, type the end-of-file indicator
when you are prompted to enter input.
On UNIX/Linux/Mac OS X type <ctrl> d then press Enter
On Windows type <ctrl> z then press Enter

Enter account number (> 0), first name, last name and balance.
? 100 Bob Jones 24.98
Enter account number (> 0), first name, last name and balance.
? 200 Steve Doe -345.67
Enter account number (> 0), first name, last name and balance.
? 300 Pam White 0.00
Enter account number (> 0), first name, last name and balance.
? 400 Sam Stone -42.16
Enter account number (> 0), first name, last name and balance.
? 500 Sue Rich 224.62
Enter account number (> 0), first name, last name and balance.
? ^Z
```

Fig. 17.7 | Testing the `CreateTextFile` class. (Part 2 of 2.)

Sample data			
100	Bob	Jones	24.98
200	Steve	Doe	-345.67
300	Pam	White	0.00
400	Sam	Stone	-42.16
500	Sue	Rich	224.62

Fig. 17.8 | Sample data for the program in Figs. 17.5–17.7.

17.4.2 Reading Data from a Sequential-Access Text File

Data is stored in files so that it may be retrieved for processing when needed. Section 17.4.1 demonstrated how to create a file for sequential access. This section shows how to read data sequentially from a text file. We demonstrate how class `Scanner` can be used to input data from a file rather than the keyboard.

The application in Figs. 17.9 and 17.10 reads records from the file `"clients.txt"` created by the application of Section 17.4.1 and displays the record contents. Line 13 of Fig. 17.9 declares a `Scanner` that will be used to retrieve input from the file.

```
1  // Fig. 17.9: ReadTextFile.java
2  // This program reads a text file and displays each record.
3  import java.io.File;
4  import java.io.FileNotFoundException;
5  import java.lang.IllegalStateException;
6  import java.util.NoSuchElementException;
7  import java.util.Scanner;
8
9  import com.deitel.ch17.AccountRecord;
10
11 public class ReadTextFile
12 {
13    private Scanner input;
14
15    // enable user to open file
16    public void openFile()
17    {
18       try
19       {
20          input = new Scanner( new File( "clients.txt" ) );
21       } // end try
22       catch ( FileNotFoundException fileNotFoundException )
23       {
24          System.err.println( "Error opening file." );
25          System.exit( 1 );
26       } // end catch
27    } // end method openFile
28
29    // read record from file
30    public void readRecords()
31    {
32       // object to be written to screen
33       AccountRecord record = new AccountRecord();
34
35       System.out.printf( "%-10s%-12s%-12s%10s\n", "Account",
36          "First Name", "Last Name", "Balance" );
37
38       try // read records from file using Scanner object
39       {
40          while ( input.hasNext() )
41          {
42             record.setAccount( input.nextInt() ); // read account number
43             record.setFirstName( input.next() ); // read first name
44             record.setLastName( input.next() ); // read last name
45             record.setBalance( input.nextDouble() ); // read balance
46
47             // display record contents
48             System.out.printf( "%-10d%-12s%-12s%10.2f\n",
49                record.getAccount(), record.getFirstName(),
50                record.getLastName(), record.getBalance() );
51          } // end while
52       } // end try
```

Fig. 17.9 | Sequential file reading using a `Scanner`. (Part 1 of 2.)

```
53          catch ( NoSuchElementException elementException )
54          {
55             System.err.println( "File improperly formed." );
56             input.close();
57             System.exit( 1 );
58          } // end catch
59          catch ( IllegalStateException stateException )
60          {
61             System.err.println( "Error reading from file." );
62             System.exit( 1 );
63          } // end catch
64       } // end method readRecords
65
66       // close file and terminate application
67       public void closeFile()
68       {
69          if ( input != null )
70             input.close(); // close file
71       } // end method closeFile
72    } // end class ReadTextFile
```

Fig. 17.9 | Sequential file reading using a `Scanner`. (Part 2 of 2.)

```
1  // Fig. 17.10: ReadTextFileTest.java
2  // Testing the ReadTextFile class.
3
4  public class ReadTextFileTest
5  {
6     public static void main( String[] args )
7     {
8        ReadTextFile application = new ReadTextFile();
9
10       application.openFile();
11       application.readRecords();
12       application.closeFile();
13    } // end main
14 } // end class ReadTextFileTest
```

```
Account   First Name  Last Name      Balance
100       Bob         Jones            24.98
200       Steve       Doe            -345.67
300       Pam         White            0.00
400       Sam         Stone          -42.16
500       Sue         Rich           224.62
```

Fig. 17.10 | Testing the `ReadTextFile` class.

Method `openFile` (lines 16–27) opens the file for reading by instantiating a `Scanner` object in line 20. We pass a `File` object to the constructor, which specifies that the `Scanner` object will read from the file `"clients.txt"` located in the directory from which the application executes. If the file cannot be found, a `FileNotFoundException` occurs. The exception is handled in lines 22–26.

Method `readRecords` (lines 30–64) reads and displays records from the file. Line 33 creates `AccountRecord` object `record` to store the current record's information. Lines 35–36 display headers for the columns in the application's output. Lines 40–51 read data from the file until the end-of-file marker is reached (in which case, method `hasNext` will return `false` at line 40). Lines 42–45 use `Scanner` methods `nextInt`, `next` and `nextDouble` to input an `int` (the account number), two `String`s (the first and last names) and a `double` value (the balance). Each record is one line of data in the file. The values are stored in object `record`. If the information in the file is not properly formed (e.g., there's a last name where there should be a balance), a `NoSuchElementException` occurs when the record is input. This exception is handled in lines 53–58. If the `Scanner` was closed before the data was input, an **`IllegalStateException`** occurs (handled in lines 59–63). If no exceptions occur, the record's information is displayed on the screen (lines 48–50). Note in the format string in line 48 that the account number, first name and last name are left justified, while the balance is right justified and output with two digits of precision. Each iteration of the loop inputs one line of text from the text file, which represents one record.

Lines 67–71 define method `closeFile`, which closes the `Scanner`. Method `main` is defined in Fig. 17.10 in lines 6–13. Line 8 creates a `ReadTextFile` object, which is then used to open, add records to and close the file (lines 10–12).

17.4.3 Case Study: A Credit-Inquiry Program

To retrieve data sequentially from a file, programs start from the beginning of the file and read all the data consecutively until the desired information is found. It might be necessary to process the file sequentially several times (from the beginning of the file) during the execution of a program. Class `Scanner` does *not* allow repositioning to the beginning of the file. If it's necessary to read the file again, the program must close the file and reopen it.

The program in Figs. 17.11–17.13 allows a credit manager to obtain lists of customers with zero balances (i.e., customers who do not owe any money), customers with credit balances (i.e., customers to whom the company owes money) and customers with debit balances (i.e., customers who owe the company money for goods and services received). A credit balance is a negative amount, a debit balance a positive amount.

`MenuOption` Enumeration

We begin by creating an `enum` type (Fig. 17.11) to define the different menu options the user will have. The options and their values are listed in lines 7–10. Method `getValue` (lines 19–22) retrieves the value of a specific `enum` constant.

```
1  // Fig. 17.11: MenuOption.java
2  // Enumeration for the credit-inquiry program's options.
3
4  public enum MenuOption
5  {
6     // declare contents of enum type
7     ZERO_BALANCE( 1 ),
8     CREDIT_BALANCE( 2 ),
9     DEBIT_BALANCE( 3 ),
10    END( 4 );
```

Fig. 17.11 | Enumeration for the credit-inquiry program's menu options. (Part 1 of 2.)

```
11
12     private final int value; // current menu option
13
14     // constructor
15     MenuOption( int valueOption )
16     {
17        value = valueOption;
18     } // end MenuOptions enum constructor
19
20     // return the value of a constant
21     public int getValue()
22     {
23        return value;
24     } // end method getValue
25  } // end enum MenuOption
```

Fig. 17.11 | Enumeration for the credit-inquiry program's menu options. (Part 2 of 2.)

CreditInquiry Class

Figure 17.12 contains the functionality for the credit-inquiry program, and Fig. 17.13 contains the `main` method that executes the program. The program displays a text menu and allows the credit manager to enter one of three options to obtain credit information. Option 1 (`ZERO_BALANCE`) displays accounts with zero balances. Option 2 (`CREDIT_BALANCE`) displays accounts with credit balances. Option 3 (`DEBIT_BALANCE`) displays accounts with debit balances. Option 4 (`END`) terminates program execution.

```
1  // Fig. 17.12: CreditInquiry.java
2  // This program reads a file sequentially and displays the
3  // contents based on the type of account the user requests
4  // (credit balance, debit balance or zero balance).
5  import java.io.File;
6  import java.io.FileNotFoundException;
7  import java.lang.IllegalStateException;
8  import java.util.NoSuchElementException;
9  import java.util.Scanner;
10
11 import com.deitel.ch17.AccountRecord;
12
13 public class CreditInquiry
14 {
15    private MenuOption accountType;
16    private Scanner input;
17    private final static MenuOption[] choices = { MenuOption.ZERO_BALANCE,
18       MenuOption.CREDIT_BALANCE, MenuOption.DEBIT_BALANCE,
19       MenuOption.END };
20
21    // read records from file and display only records of appropriate type
22    private void readRecords()
23    {
24       // object to store data that will be written to file
25       AccountRecord record = new AccountRecord();
```

Fig. 17.12 | Credit-inquiry program. (Part 1 of 4.)

```
26
27          try // read records
28          {
29             // open file to read from beginning
30             input = new Scanner( new File( "clients.txt" ) );
31
32             while ( input.hasNext() ) // input the values from the file
33             {
34                record.setAccount( input.nextInt() ); // read account number
35                record.setFirstName( input.next() ); // read first name
36                record.setLastName( input.next() ); // read last name
37                record.setBalance( input.nextDouble() ); // read balance
38
39                // if proper acount type, display record
40                if ( shouldDisplay( record.getBalance() ) )
41                   System.out.printf( "%-10d%-12s%-12s%10.2f\n",
42                      record.getAccount(), record.getFirstName(),
43                      record.getLastName(), record.getBalance() );
44             } // end while
45          } // end try
46          catch ( NoSuchElementException elementException )
47          {
48             System.err.println( "File improperly formed." );
49             input.close();
50             System.exit( 1 );
51          } // end catch
52          catch ( IllegalStateException stateException )
53          {
54             System.err.println( "Error reading from file." );
55             System.exit( 1 );
56          } // end catch
57          catch ( FileNotFoundException fileNotFoundException )
58          {
59             System.err.println( "File cannot be found." );
60             System.exit( 1 );
61          } // end catch
62          finally
63          {
64             if ( input != null )
65                input.close(); // close the Scanner and the file
66          } // end finally
67       } // end method readRecords
68
69       // use record type to determine if record should be displayed
70       private boolean shouldDisplay( double balance )
71       {
72          if ( ( accountType == MenuOption.CREDIT_BALANCE )
73             && ( balance < 0 ) )
74             return true;
75
76          else if ( ( accountType == MenuOption.DEBIT_BALANCE )
77             && ( balance > 0 ) )
78             return true;
```

Fig. 17.12 | Credit-inquiry program. (Part 2 of 4.)

```
79
80          else if ( ( accountType == MenuOption.ZERO_BALANCE )
81             && ( balance == 0 ) )
82             return true;
83
84          return false;
85       } // end method shouldDisplay
86
87       // obtain request from user
88       private MenuOption getRequest()
89       {
90          Scanner textIn = new Scanner( System.in );
91          int request = 1;
92
93          // display request options
94          System.out.printf( "\n%s\n%s\n%s\n%s\n%s\n",
95             "Enter request", " 1 - List accounts with zero balances",
96             " 2 - List accounts with credit balances",
97             " 3 - List accounts with debit balances", " 4 - End of run" );
98
99          try // attempt to input menu choice
100         {
101            do // input user request
102            {
103               System.out.print( "\n? " );
104               request = textIn.nextInt();
105            } while ( ( request < 1 ) || ( request > 4 ) );
106         } // end try
107         catch ( NoSuchElementException elementException )
108         {
109            System.err.println( "Invalid input." );
110            System.exit( 1 );
111         } // end catch
112
113         return choices[ request - 1 ]; // return enum value for option
114      } // end method getRequest
115
116      public void processRequests()
117      {
118         // get user's request (e.g., zero, credit or debit balance)
119         accountType = getRequest();
120
121         while ( accountType != MenuOption.END )
122         {
123            switch ( accountType )
124            {
125               case ZERO_BALANCE:
126                  System.out.println( "\nAccounts with zero balances:\n" );
127                  break;
128               case CREDIT_BALANCE:
129                  System.out.println( "\nAccounts with credit balances:\n" );
130                  break;
```

Fig. 17.12 | Credit-inquiry program. (Part 3 of 4.)

```
131             case DEBIT_BALANCE:
132                System.out.println( "\nAccounts with debit balances:\n" );
133                break;
134          } // end switch
135
136          readRecords();
137          accountType = getRequest();
138       } // end while
139    } // end method processRequests
140 } // end class CreditInquiry
```

Fig. 17.12 | Credit-inquiry program. (Part 4 of 4.)

```
1  // Fig. 17.13: CreditInquiryTest.java
2  // This program tests class CreditInquiry.
3
4  public class CreditInquiryTest
5  {
6     public static void main( String[] args )
7     {
8        CreditInquiry application = new CreditInquiry();
9        application.processRequests();
10    } // end main
11 } // end class CreditInquiryTest
```

Fig. 17.13 | Testing the `CreditInquiry` class.

```
Enter request
 1 - List accounts with zero balances
 2 - List accounts with credit balances
 3 - List accounts with debit balances
 4 - End of run

? 1

Accounts with zero balances:
300       Pam         White             0.00

Enter request
 1 - List accounts with zero balances
 2 - List accounts with credit balances
 3 - List accounts with debit balances
 4 - End of run

? 2

Accounts with credit balances:
200       Steve       Doe            -345.67
400       Sam         Stone           -42.16
```

Fig. 17.14 | Sample output of the credit-inquiry program in Fig. 17.13. (Part 1 of 2.)

```
Enter request
 1 - List accounts with zero balances
 2 - List accounts with credit balances
 3 - List accounts with debit balances
 4 - End of run

? 3

Accounts with debit balances:
100       Bob         Jones           24.98
500       Sue         Rich           224.62

? 4
```

Fig. 17.14 | Sample output of the credit-inquiry program in Fig. 17.13. (Part 2 of 2.)

The record information is collected by reading through the file and determining if each record satisfies the criteria for the selected account type. Method `processRequests` (lines 116–139 of Fig. 17.12) calls method `getRequest` to display the menu options (line 119), translates the number typed by the user into a `MenuOption` and stores the result in `MenuOption` variable `accountType`. Lines 121–138 loop until the user specifies that the program should terminate. Lines 123–134 display a header for the current set of records to be output to the screen. Line 136 calls method `readRecords` (lines 22–67), which loops through the file and reads every record.

Line 30 of method `readRecords` opens the file for reading with a `Scanner`. The file will be opened for reading with a new `Scanner` object each time this method is called, so that we can again read from the beginning of the file. Lines 34–37 read a record. Line 40 calls method `shouldDisplay` (lines 70–85) to determine whether the current record satisfies the account type requested. If `shouldDisplay` returns `true`, the program displays the account information. When the end-of-file marker is reached, the loop terminates and line 65 calls the `Scanner`'s `close` method to close the `Scanner` and the file. Notice that this occurs in a `finally` block, which will execute whether or not the file was successfully read. Once all the records have been read, control returns to method `processRequests` and `getRequest` is again called (line 137) to retrieve the user's next menu option. Figure 17.13 contains method `main`, and calls method `processRequests` in line 9.

17.4.4 Updating Sequential-Access Files

The data in many sequential files cannot be modified without the risk of destroying other data in the file. For example, if the name "`White`" needs to be changed to "`Worthington`," the old name cannot simply be overwritten, because the new name requires more space. The record for `White` was written to the file as

```
300 Pam White 0.00
```

If the record is rewritten beginning at the same location in the file using the new name, the record will be

```
300 Pam Worthington 0.00
```

The new record is larger (has more characters) than the original record. The characters beyond the second "`o`" in "`Worthington`" will overwrite the beginning of the next sequential

record in the file. The problem here is that fields in a text file—and hence records—can vary in size. For example, 7, 14, –117, 2074 and 27383 are all `int`s stored in the same number of bytes (4) internally, but they're different-sized fields when displayed on the screen or written to a file as text. Therefore, records in a sequential-access file are not usually updated in place. Instead, the entire file is usually rewritten. To make the preceding name change, the records before `300 Pam White 0.00` would be copied to a new file, the new record (which can be of a different size than the one it replaces) would be written and the records after `300 Pam White 0.00` would be copied to the new file. Rewriting the entire file is uneconomical to update just one record, but reasonable if a substantial number of records need to be updated.

17.5 Object Serialization

In Section 17.4, we demonstrated how to write the individual fields of an `AccountRecord` object into a file as text, and how to read those fields from a file and place their values into an `AccountRecord` object in memory. In the examples, `AccountRecord` was used to aggregate the information for one record. When the instance variables for an `AccountRecord` were output to a disk file, certain information was lost, such as the type of each value. For instance, if the value `"3"` is read from a file, there's no way to tell whether it came from an `int`, a `String` or a `double`. We have only data, not type information, on a disk. If the program that's going to read this data "knows" what object type the data corresponds to, then the data is simply read into objects of that type. For example, in Section 17.4.2, we know that we're inputting an `int` (the account number), followed by two `String`s (the first and last name) and a `double` (the balance). We also know that these values are separated by spaces, with only one record on each line. Sometimes we'll not know exactly how the data is stored in a file. In such cases, we want to read or write an entire object from a file. Java provides such a mechanism, called **object serialization**. A so-called **serialized object** is an object represented as a sequence of bytes that includes the object's data as well as information about the object's type and the types of data stored in the object. After a serialized object has been written into a file, it can be read from the file and **deserialized**—that is, the type information and bytes that represent the object and its data can be used to recreate the object in memory.



Software Engineering Observation 17.1

The serialization mechanism makes exact copies of objects. This makes it a simple way to clone objects without having to override `Object` *method* `clone`.

Classes `ObjectInputStream` *and* `ObjectOutputStream`

Classes `ObjectInputStream` and `ObjectOutputStream`, which respectively implement the **`ObjectInput`** and **`ObjectOutput`** interfaces, enable entire objects to be read from or written to a stream (possibly a file). To use serialization with files, we initialize `ObjectInputStream` and `ObjectOutputStream` objects with stream objects that read from and write to files—objects of classes `FileInputStream` and `FileOutputStream`, respectively. Initializing stream objects with other stream objects in this manner is sometimes called **wrapping**—the new stream object being created wraps the stream object specified as a constructor argument. To wrap a `FileInputStream` in an `ObjectInputStream`, for instance, we pass the `FileInputStream` object to the `ObjectInputStream`'s constructor.

Interfaces `ObjectOutput` *and* `ObjectInput`

The `ObjectOutput` interface contains method **`writeObject`**, which takes an `Object` as an argument and writes its information to an `OutputStream`. A class that implements interface `ObjectOutput` (such as `ObjectOutputStream`) declares this method and ensures that the object being output implements interface `Serializable` (discussed shortly). Correspondingly, the `ObjectInput` interface contains method **`readObject`**, which reads and returns a reference to an `Object` from an `InputStream`. After an object has been read, its reference can be cast to the object's actual type. As you'll see in Chapter 27, applications that communicate via a network, such as the Internet, can also transmit entire objects across the network.

17.5.1 Creating a Sequential-Access File Using Object Serialization

This section and Section 17.5.2 create and manipulate sequential-access files using object serialization. The object serialization we show here is performed with byte-based streams, so the sequential files created and manipulated will be binary files. Recall that binary files typically cannot be viewed in standard text editors. For this reason, we write a separate application that knows how to read and display serialized objects. We begin by creating and writing serialized objects to a sequential-access file. The example is similar to the one in Section 17.4, so we focus only on the new features.

Defining Class `AccountRecordSerializable`

Let's begin by modifying our `AccountRecord` class so that objects of this class can be serialized. Class `AccountRecordSerializable` (Fig. 17.15) implements interface **`Serializable`** (line 7), which allows objects of `AccountRecordSerializable` to be serialized and deserialized with `ObjectOutputStreams` and `ObjectInputStreams`, respectively. Interface `Serializable` is a **tagging interface**. Such an interface does not contain methods. A class that implements `Serializable` is tagged as being a `Serializable` object. This is important, because an `ObjectOutputStream` will *not* output an object unless it *is a* `Serializable` object, which is the case for any object of a class that implements `Serializable`.

```
1  // Fig. 17.15: AccountRecordSerializable.java
2  // AccountRecordSerializable class for serializable objects.
3  package com.deitel.ch17; // packaged for reuse
4
5  import java.io.Serializable;
6
7  public class AccountRecordSerializable implements Serializable
8  {
9     private int account;
10    private String firstName;
11    private String lastName;
12    private double balance;
13
14    // no-argument constructor calls other constructor with default values
15    public AccountRecordSerializable()
16    {
17       this( 0, "", "", 0.0 );
18    } // end no-argument AccountRecordSerializable constructor
```

Fig. 17.15 | `AccountRecordSerializable` class for serializable objects. (Part 1 of 3.)

```
19
20    // four-argument constructor initializes a record
21    public AccountRecordSerializable(
22       int acct, String first, String last, double bal )
23    {
24       setAccount( acct );
25       setFirstName( first );
26       setLastName( last );
27       setBalance( bal );
28    } // end four-argument AccountRecordSerializable constructor
29
30    // set account number
31    public void setAccount( int acct )
32    {
33       account = acct;
34    } // end method setAccount
35
36    // get account number
37    public int getAccount()
38    {
39       return account;
40    } // end method getAccount
41
42    // set first name
43    public void setFirstName( String first )
44    {
45       firstName = first;
46    } // end method setFirstName
47
48    // get first name
49    public String getFirstName()
50    {
51       return firstName;
52    } // end method getFirstName
53
54    // set last name
55    public void setLastName( String last )
56    {
57       lastName = last;
58    } // end method setLastName
59
60    // get last name
61    public String getLastName()
62    {
63       return lastName;
64    } // end method getLastName
65
66    // set balance
67    public void setBalance( double bal )
68    {
69       balance = bal;
70    } // end method setBalance
71
```

Fig. 17.15 | `AccountRecordSerializable` class for serializable objects. (Part 2 of 3.)

```
72      // get balance
73      public double getBalance()
74      {
75         return balance;
76      } // end method getBalance
77   } // end class AccountRecordSerializable
```

Fig. 17.15 | `AccountRecordSerializable` class for serializable objects. (Part 3 of 3.)

In a `Serializable` class, every instance variable must be `Serializable`. Non-`Serializable` instance variables must be declared **`transient`** to indicate that they should be ignored during the serialization process. *By default, all primitive-type variables are serializable.* For reference-type variables, you must check the class's documentation (and possibly its superclasses) to ensure that the type is `Serializable`. For example, `String`s are `Serializable`. By default, arrays are serializable; however, in a reference-type array, the referenced objects might not be. Class `AccountRecordSerializable` contains `private` data members `account`, `firstName`, `lastName` and `balance`—all of which are `Serializable`. This class also provides `public` *get* and *set* methods for accessing the `private` fields.

Writing Serialized Objects to a Sequential-Access File

Now let's discuss the code that creates the sequential-access file (Figs. 17.16–17.17). We concentrate only on new concepts here. As stated in Section 17.2, a program can open a file by creating an object of stream class `FileInputStream` or `FileOutputStream`. In this example, the file is to be opened for output, so the program creates a `FileOutputStream` (line 21 of Fig. 17.16). The `String` argument that's passed to the `FileOutputStream`'s constructor represents the name and path of the file to be opened. Existing files that are opened for output in this manner are truncated. We chose the `.ser` file extension for binary files that contain serialized objects, but this is not required.



Common Programming Error 17.2

It's a logic error to open an existing file for output when, in fact, you wish to preserve the file. Class `FileOutputStream` provides an overloaded constructor that enables you to open a file and append data to the end of the file. This will preserve the file's contents.

```
1  // Fig. 17.16: CreateSequentialFile.java
2  // Writing objects sequentially to a file with class ObjectOutputStream.
3  import java.io.FileOutputStream;
4  import java.io.IOException;
5  import java.io.ObjectOutputStream;
6  import java.util.NoSuchElementException;
7  import java.util.Scanner;
8
9  import com.deitel.ch17.AccountRecordSerializable;
10
11 public class CreateSequentialFile
12 {
13    private ObjectOutputStream output; // outputs data to file
```

Fig. 17.16 | Sequential file created using `ObjectOutputStream`. (Part 1 of 3.)

```
14
15    // allow user to specify file name
16    public void openFile()
17    {
18       try // open file
19       {
20          output = new ObjectOutputStream(
21             new FileOutputStream( "clients.ser" ) );
22       } // end try
23       catch ( IOException ioException )
24       {
25          System.err.println( "Error opening file." );
26       } // end catch
27    } // end method openFile
28
29    // add records to file
30    public void addRecords()
31    {
32       AccountRecordSerializable record; // object to be written to file
33       int accountNumber = 0; // account number for record object
34       String firstName; // first name for record object
35       String lastName; // last name for record object
36       double balance; // balance for record object
37
38       Scanner input = new Scanner( System.in );
39
40       System.out.printf( "%s\n%s\n%s\n%s\n\n",
41          "To terminate input, type the end-of-file indicator ",
42          "when you are prompted to enter input.",
43          "On UNIX/Linux/Mac OS X type <ctrl> d then press Enter",
44          "On Windows type <ctrl> z then press Enter" );
45
46       System.out.printf( "%s\n%s",
47          "Enter account number (> 0), first name, last name and balance.",
48          "? " );
49
50       while ( input.hasNext() ) // loop until end-of-file indicator
51       {
52          try // output values to file
53          {
54             accountNumber = input.nextInt(); // read account number
55             firstName = input.next(); // read first name
56             lastName = input.next(); // read last name
57             balance = input.nextDouble(); // read balance
58
59             if ( accountNumber > 0 )
60             {
61                // create new record
62                record = new AccountRecordSerializable( accountNumber,
63                   firstName, lastName, balance );
64                output.writeObject( record ); // output record
65             } // end if
```

Fig. 17.16 | Sequential file created using `ObjectOutputStream`. (Part 2 of 3.)

```
66                else
67                {
68                   System.out.println(
69                      "Account number must be greater than 0." );
70                } // end else
71             } // end try
72             catch ( IOException ioException )
73             {
74                System.err.println( "Error writing to file." );
75                return;
76             } // end catch
77             catch ( NoSuchElementException elementException )
78             {
79                System.err.println( "Invalid input. Please try again." );
80                input.nextLine(); // discard input so user can try again
81             } // end catch
82
83             System.out.printf( "%s %s\n%s", "Enter account number (>0),",
84                "first name, last name and balance.", "? " );
85          } // end while
86       } // end method addRecords
87
88       // close file and terminate application
89       public void closeFile()
90       {
91          try // close file
92          {
93             if ( output != null )
94                output.close();
95          } // end try
96          catch ( IOException ioException )
97          {
98             System.err.println( "Error closing file." );
99             System.exit( 1 );
100         } // end catch
101      } // end method closeFile
102   } // end class CreateSequentialFile
```

Fig. 17.16 | Sequential file created using `ObjectOutputStream`. (Part 3 of 3.)

```
1  // Fig. 17.17: CreateSequentialFileTest.java
2  // Testing class CreateSequentialFile.
3
4  public class CreateSequentialFileTest
5  {
6     public static void main( String[] args )
7     {
8        CreateSequentialFile application = new CreateSequentialFile();
9
10       application.openFile();
11       application.addRecords();
```

Fig. 17.17 | Testing class `CreateSequentialFile`. (Part 1 of 2.)

```
12         application.closeFile();
13      } // end main
14   } // end class CreateSequentialFileTest
```

```
To terminate input, type the end-of-file indicator
when you are prompted to enter input.
On UNIX/Linux/Mac OS X type <ctrl> d then press Enter
On Windows type <ctrl> z then press Enter

Enter account number (> 0), first name, last name and balance.
? 100 Bob Jones 24.98
Enter account number (> 0), first name, last name and balance.
? 200 Steve Doe -345.67
Enter account number (> 0), first name, last name and balance.
? 300 Pam White 0.00
Enter account number (> 0), first name, last name and balance.
? 400 Sam Stone -42.16
Enter account number (> 0), first name, last name and balance.
? 500 Sue Rich 224.62
Enter account number (> 0), first name, last name and balance.
? ^Z
```

Fig. 17.17 | Testing class `CreateSequentialFile`. (Part 2 of 2.)

Class `FileOutputStream` provides methods for writing `byte` arrays and individual `byte`s to a file, but we wish to write objects to a file. For this reason, we wrap a `FileOutputStream` in an `ObjectOutputStream` by passing the new `FileOutputStream` object to the `ObjectOutputStream`'s constructor (lines 20–21). The `ObjectOutputStream` object uses the `FileOutputStream` object to write objects into the file. Lines 20–21 may throw an **`IOException`** if a problem occurs while opening the file (e.g., when a file is opened for writing on a drive with insufficient space or when a read-only file is opened for writing). If so, the program displays an error message (lines 23–26). If no exception occurs, the file is open, and variable `output` can be used to write objects to it.

This program assumes that data is input correctly and in the proper record-number order. Method `addRecords` (lines 30–86) performs the write operation. Lines 62–63 create an `AccountRecordSerializable` object from the data entered by the user. Line 64 calls `ObjectOutputStream` method `writeObject` to write the `record` object to the output file. Only one statement is required to write the entire object.

Method `closeFile` (lines 89–101) calls `ObjectOutputStream` method **`close`** on `output` to close both the `ObjectOutputStream` and its underlying `FileOutputStream` (line 94). The call to method `close` is contained in a `try` block. Method `close` throws an `IOException` if the file cannot be closed properly. In this case, it's important to notify the user that the information in the file might be corrupted. When using wrapped streams, closing the outermost stream also closes the underlying file.

In the sample execution for the program in Fig. 17.17, we entered information for five accounts—the same information shown in Fig. 17.8. The program does not show how the data records actually appear in the file. Remember that now we're using binary files, which are not humanly readable. To verify that the file has been created successfully, the next section presents a program to read the file's contents.

17.5.2 Reading and Deserializing Data from a Sequential-Access File

The preceding section showed how to create a file for sequential access using object serialization. In this section, we discuss how to read serialized data sequentially from a file.

The program in Figs. 17.18–17.19 reads records from a file created by the program in Section 17.5.1 and displays the contents. The program opens the file for input by creating a `FileInputStream` object (line 21). The name of the file to open is specified as an argument to the `FileInputStream` constructor. In Fig. 17.16, we wrote objects to the file, using an `ObjectOutputStream` object. Data must be read from the file in the same format in which it was written. Therefore, we use an `ObjectInputStream` wrapped around a `FileInputStream` in this program (lines 20–21). If no exceptions occur when opening the file, variable `input` can be used to read objects from the file.

```
1  // Fig. 17.18: ReadSequentialFile.java
2  // Reading a file of objects sequentially with ObjectInputStream
3  // and displaying each record.
4  import java.io.EOFException;
5  import java.io.FileInputStream;
6  import java.io.IOException;
7  import java.io.ObjectInputStream;
8
9  import com.deitel.ch17.AccountRecordSerializable;
10
11 public class ReadSequentialFile
12 {
13    private ObjectInputStream input;
14
15    // enable user to select file to open
16    public void openFile()
17    {
18       try // open file
19       {
20          input = new ObjectInputStream(
21             new FileInputStream( "clients.ser" ) );
22       } // end try
23       catch ( IOException ioException )
24       {
25          System.err.println( "Error opening file." );
26       } // end catch
27    } // end method openFile
28
29    // read record from file
30    public void readRecords()
31    {
32       AccountRecordSerializable record;
33       System.out.printf( "%-10s%-12s%-12s%10s\n", "Account",
34          "First Name", "Last Name", "Balance" );
35
36       try // input the values from the file
37       {
```

Fig. 17.18 | Reading a file of objects sequentially with `ObjectInputStream` and displaying each record. (Part 1 of 2.)

```
38            while ( true )
39            {
40               record = ( AccountRecordSerializable ) input.readObject();
41
42               // display record contents
43               System.out.printf( "%-10d%-12s%-12s%10.2f\n",
44                  record.getAccount(), record.getFirstName(),
45                  record.getLastName(), record.getBalance() );
46            } // end while
47         } // end try
48         catch ( EOFException endOfFileException )
49         {
50            return; // end of file was reached
51         } // end catch
52         catch ( ClassNotFoundException classNotFoundException )
53         {
54            System.err.println( "Unable to create object." );
55         } // end catch
56         catch ( IOException ioException )
57         {
58            System.err.println( "Error during read from file." );
59         } // end catch
60      } // end method readRecords
61
62      // close file and terminate application
63      public void closeFile()
64      {
65         try // close file and exit
66         {
67            if ( input != null )
68               input.close();
69         } // end try
70         catch ( IOException ioException )
71         {
72            System.err.println( "Error closing file." );
73            System.exit( 1 );
74         } // end catch
75      } // end method closeFile
76   } // end class ReadSequentialFile
```

Fig. 17.18 | Reading a file of objects sequentially with `ObjectInputStream` and displaying each record. (Part 2 of 2.)

The program reads records from the file in method `readRecords` (lines 30–60). Line 40 calls `ObjectInputStream` method `readObject` to read an `Object` from the file. To use `AccountRecordSerializable`-specific methods, we downcast the returned `Object` to type `AccountRecordSerializable`. Method `readObject` throws an **`EOFException`** (processed at lines 48–51) if an attempt is made to read beyond the end of the file. Method `readObject` throws a `ClassNotFoundException` if the class for the object being read cannot be located. This may occur if the file is accessed on a computer that does not have the class. Figure 17.19 contains method `main` (lines 6–13), which opens the file, calls method `readRecords` and closes the file.

```
1  // Fig. 17.19: ReadSequentialFileTest.java
2  // Testing class ReadSequentialFile.
3
4  public class ReadSequentialFileTest
5  {
6     public static void main( String[] args )
7     {
8        ReadSequentialFile application = new ReadSequentialFile();
9
10       application.openFile();
11       application.readRecords();
12       application.closeFile();
13    } // end main
14 } // end class ReadSequentialFileTest
```

```
Account   First Name  Last Name      Balance
100       Bob         Jones            24.98
200       Steve       Doe            -345.67
300       Pam         White            0.00
400       Sam         Stone          -42.16
500       Sue         Rich           224.62
```

Fig. 17.19 | Testing class `ReadSequentialFile`.

17.6 Additional `java.io` Classes

This section overviews additional interfaces and classes (from package `java.io`) for byte-based input and output streams and character-based input and output streams.

17.6.1 Interfaces and Classes for Byte-Based Input and Output

`InputStream` and **`OutputStream`** are `abstract` classes that declare methods for performing byte-based input and output, respectively. We used various concrete subclasses `FileInputStream` `InputStream` and `OutputStream` to manipulate files in this chapter.

Pipe Streams

Pipes are synchronized communication channels between threads. We discuss threads in Chapter 26. Java provides **`PipedOutputStream`** (a subclass of `OutputStream`) and **`PipedInputStream`** (a subclass of `InputStream`) to establish pipes between two threads in a program. One thread sends data to another by writing to a `PipedOutputStream`. The target thread reads information from the pipe via a `PipedInputStream`.

Filter Streams

A **`FilterInputStream`** filters an `InputStream`, and a `FilterOutputStream` filters an `OutputStream`. **Filtering** means simply that the filter stream provides additional functionality, such as aggregating data bytes into meaningful primitive-type units. `FilterInputStream` and `FilterOutputStream` are typically extended, so some of their filtering capabilities are provided by their subclasses.

A **`PrintStream`** (a subclass of `FilterOutputStream`) performs text output to the specified stream. Actually, we've been using `PrintStream` output throughout the text to this point—`System.out` and `System.err` are `PrintStream` objects.

Data Streams

Reading data as raw bytes is fast, but crude. Usually, programs read data as aggregates of bytes that form `int`s, `float`s, `double`s and so on. Java programs can use several classes to input and output data in aggregate form.

Interface `DataInput` describes methods for reading primitive types from an input stream. Classes **`DataInputStream`** and `RandomAccessFile` each implement this interface to read sets of bytes and view them as primitive-type values. Interface `DataInput` includes methods such as `readBoolean`, `readByte`, `readChar`, `readDouble`, `readFloat`, `readFully` (for `byte` arrays), `readInt`, `readLong`, `readShort`, `readUnsignedByte`, `readUnsignedShort`, `readUTF` (for reading Unicode characters encoded by Java—we discuss UTF encoding in Appendix L) and `skipBytes`.

Interface `DataOutput` describes a set of methods for writing primitive types to an output stream. Classes **`DataOutputStream`** (a subclass of `FilterOutputStream`) and `RandomAccessFile` each implement this interface to write primitive-type values as bytes. Interface `DataOutput` includes overloaded versions of method `write` (for a `byte` or for a `byte` array) and methods `writeBoolean`, `writeByte`, `writeBytes`, `writeChar`, `writeChars` (for Unicode `String`s), `writeDouble`, `writeFloat`, `writeInt`, `writeLong`, `writeShort` and `writeUTF` (to output text modified for Unicode).

Buffered Streams

Buffering is an I/O-performance-enhancement technique. With a **`BufferedOutputStream`** (a subclass of class `FilterOutputStream`), each output statement does *not* necessarily result in an actual physical transfer of data to the output device (which is a slow operation compared to processor and main memory speeds). Rather, each output operation is directed to a region in memory called a **buffer** that's large enough to hold the data of many output operations. Then, actual transfer to the output device is performed in one large **physical output operation** each time the buffer fills. The output operations directed to the output buffer in memory are often called **logical output operations**. With a `BufferedOutputStream`, a partially filled buffer can be forced out to the device at any time by invoking the stream object's **`flush`** method.

Using buffering can greatly increase the performance of an application. Typical I/O operations are extremely slow compared with the speed of accessing data in computer memory. Buffering reduces the number of I/O operations by first combining smaller outputs together in memory. The number of actual physical I/O operations is small compared with the number of I/O requests issued by the program. Thus, the program that's using buffering is more efficient.



Performance Tip 17.1

Buffered I/O can yield significant performance improvements over unbuffered I/O.

With a **`BufferedInputStream`** (a subclass of class `FilterInputStream`), many "logical" chunks of data from a file are read as one large **physical input operation** into a memory buffer. As a program requests each new chunk of data, it's taken from the buffer. (This procedure is sometimes referred to as a **logical input operation**.) When the buffer is empty, the next actual physical input operation from the input device is performed to read in the next group of "logical" chunks of data. Thus, the number of actual physical input operations is small compared with the number of read requests issued by the program.

Memory-Based **byte** *Array Steams*

Java stream I/O includes capabilities for inputting from `byte` arrays in memory and outputting to `byte` arrays in memory. A `ByteArrayInputStream` (a subclass of `InputStream`) reads from a `byte` array in memory. A `ByteArrayOutputStream` (a subclass of `OutputStream`) outputs to a `byte` array in memory. One use of `byte`-array I/O is *data validation*. A program can input an entire line at a time from the input stream into a `byte` array. Then a validation routine can scrutinize the contents of the `byte` array and correct the data if necessary. Finally, the program can proceed to input from the `byte` array, "knowing" that the input data is in the proper format. Outputting to a `byte` array is a nice way to take advantage of the powerful output-formatting capabilities of Java streams. For example, data can be stored in a `byte` array, using the same formatting that will be displayed at a later time, and the `byte` array can then be output to a file to preserve the formatting.

Sequencing Input from Multiple Streams

A `SequenceInputStream` (a subclass of `InputStream`) logically concatenates several `InputStreams`—the program sees the group as one continuous `InputStream`. When the program reaches the end of one input stream, that stream closes, and the next stream in the sequence opens.

17.6.2 Interfaces and Classes for Character-Based Input and Output

In addition to the byte-based streams, Java provides the **`Reader`** and **`Writer`** `abstract` classes, which are Unicode two-byte, character-based streams. Most of the byte-based streams have corresponding character-based concrete `Reader` or `Writer` classes.

Character-Based Buffering **`Reader`***s and* **`Writer`***s*

Classes **`BufferedReader`** (a subclass of `abstract` class `Reader`) and **`BufferedWriter`** (a subclass of `abstract` class `Writer`) enable buffering for character-based streams. Remember that character-based streams use Unicode characters—such streams can process data in any language that the Unicode character set represents.

Memory-Based **`char`** *Array* **`Reader`***s and* **`Writer`***s*

Classes **`CharArrayReader`** and **`CharArrayWriter`** read and write, respectively, a stream of characters to a `char` array. A **`LineNumberReader`** (a subclass of `BufferedReader`) is a buffered character stream that keeps track of the number of lines read—newlines, returns and carriage-return–line-feed combinations increment the line count. Keeping track of line numbers can be useful if the program needs to inform the reader of an error on a specific line.

Character-Based File, Pipe and String **`Reader`***s and* **`Writer`***s*

An `InputStream` can be converted to a `Reader` via class **`InputStreamReader`**. Similarly, an `OuputStream` can be converted to a `Writer` via class **`OutputStreamWriter`**. Class `FileReader` (a subclass of `InputStreamReader`) and class `FileWriter` (a subclass of `OutputStreamWriter`) read characters from and write characters to a file, respectively. Class **`PipedReader`** and class **`PipedWriter`** implement piped-character streams for transfering data between threads. Class **`StringReader`** and **`StringWriter`** read characters from and write characters to `String`s, respectively. A `PrintWriter` writes characters to a stream.

17.7 Opening Files with JFileChooser

Class **JFileChooser** displays a dialog (known as the JFileChooser dialog) that enables the user to easily select files or directories. To demonstrate this dialog, we enhance the example in Section 17.3, as shown in Figs. 17.20–17.21. The example now contains a graphical user interface, but still displays the same data as before. The constructor calls method analyzePath in line 34. This method then calls method getFile in line 68 to retrieve the File object.

```
1  // Fig. 17.20: FileDemonstration.java
2  // Demonstrating JFileChooser.
3  import java.awt.BorderLayout;
4  import java.awt.event.ActionEvent;
5  import java.awt.event.ActionListener;
6  import java.io.File;
7  import javax.swing.JFileChooser;
8  import javax.swing.JFrame;
9  import javax.swing.JOptionPane;
10 import javax.swing.JScrollPane;
11 import javax.swing.JTextArea;
12 import javax.swing.JTextField;
13
14 public class FileDemonstration extends JFrame
15 {
16    private JTextArea outputArea; // used for output
17    private JScrollPane scrollPane; // used to provide scrolling to output
18
19    // set up GUI
20    public FileDemonstration()
21    {
22       super( "Testing class File" );
23
24       outputArea = new JTextArea();
25
26       // add outputArea to scrollPane
27       scrollPane = new JScrollPane( outputArea );
28
29       add( scrollPane, BorderLayout.CENTER ); // add scrollPane to GUI
30
31       setSize( 400, 400 ); // set GUI size
32       setVisible( true ); // display GUI
33
34       analyzePath(); // create and analyze File object
35    } // end FileDemonstration constructor
36
37    // allow user to specify file or directory name
38    private File getFileOrDirectory()
39    {
40       // display file dialog, so user can choose file or directory to open
41       JFileChooser fileChooser = new JFileChooser();
42       fileChooser.setFileSelectionMode(
43          JFileChooser.FILES_AND_DIRECTORIES );
```

Fig. 17.20 | Demonstrating JFileChooser. (Part 1 of 3.)

```
44
45          int result = fileChooser.showOpenDialog( this );
46
47          // if user clicked Cancel button on dialog, return
48          if ( result == JFileChooser.CANCEL_OPTION )
49             System.exit( 1 );
50
51          File fileName = fileChooser.getSelectedFile(); // get File
52
53          // display error if invalid
54          if ( ( fileName == null ) || ( fileName.getName().equals( "" ) ) )
55          {
56             JOptionPane.showMessageDialog( this, "Invalid Name",
57                "Invalid Name", JOptionPane.ERROR_MESSAGE );
58             System.exit( 1 );
59          } // end if
60
61          return fileName;
62       } // end method getFile
63
64       // display information about file or directory user specifies
65       public void analyzePath()
66       {
67          // create File object based on user input
68          File name = getFileOrDirectory();
69
70          if ( name.exists() ) // if name exists, output information about it
71          {
72             // display file (or directory) information
73             outputArea.setText( String.format(
74                "%s%s\n%s\n%s\n%s\n%s%s\n%s%s\n%s%s\n%s%s\n%s%s",
75                name.getName(), " exists",
76                ( name.isFile() ? "is a file" : "is not a file" ),
77                ( name.isDirectory() ? "is a directory" :
78                   "is not a directory" ),
79                ( name.isAbsolute() ? "is absolute path" :
80                   "is not absolute path" ), "Last modified: ",
81                name.lastModified(), "Length: ", name.length(),
82                "Path: ", name.getPath(), "Absolute path: ",
83                name.getAbsolutePath(), "Parent: ", name.getParent() ) );
84
85             if ( name.isDirectory() ) // output directory listing
86             {
87                String[] directory = name.list();
88                outputArea.append( "\n\nDirectory contents:\n" );
89
90                for ( String directoryName : directory )
91                   outputArea.append( directoryName + "\n" );
92             } // end else
93          } // end outer if
94          else // not file or directory, output error message
95          {
```

Fig. 17.20 | Demonstrating `JFileChooser`. (Part 2 of 3.)

```
            JOptionPane.showMessageDialog( this, name +
               " does not exist.", "ERROR", JOptionPane.ERROR_MESSAGE );
         } // end else
      } // end method analyzePath
   } // end class FileDemonstration
```

Fig. 17.20 | Demonstrating `JFileChooser`. (Part 3 of 3.)

```
// Fig. 17.21: FileDemonstrationTest.java
// Testing class FileDemonstration.
import javax.swing.JFrame;

public class FileDemonstrationTest
{
   public static void main( String[] args )
   {
      FileDemonstration application = new FileDemonstration();
      application.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
   } // end main
} // end class FileDemonstrationTest
```

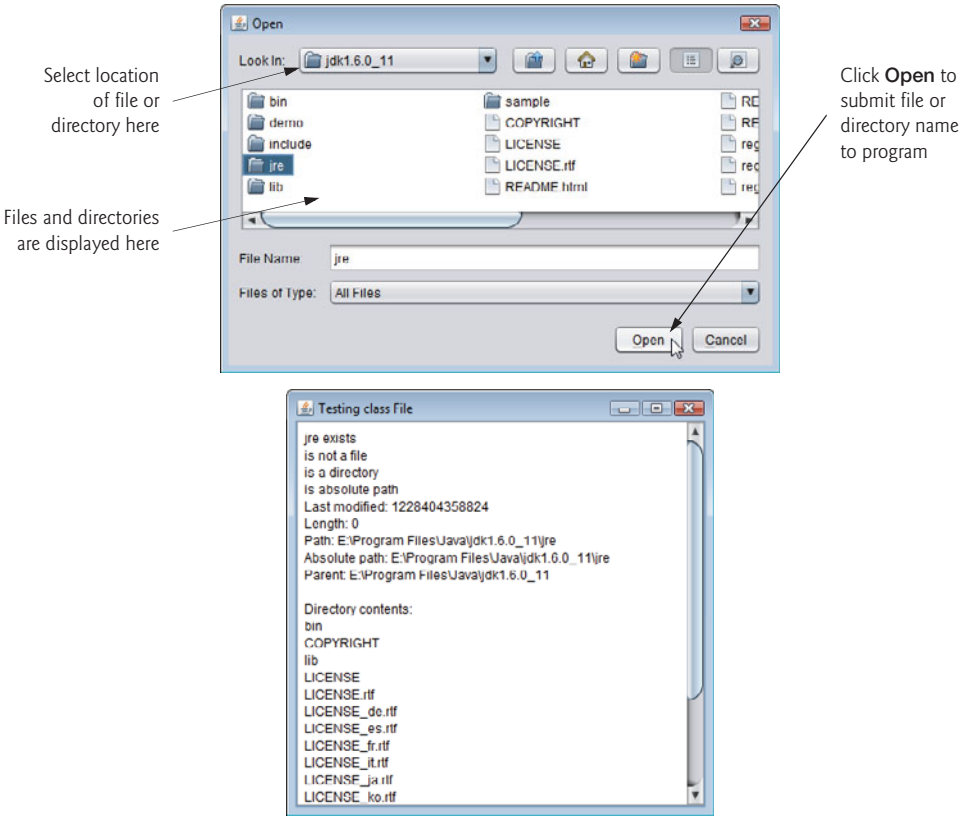


Fig. 17.21 | Testing class `FileDemonstration`.

Method `getFile` is defined in lines 38–62 of Fig. 17.20. Line 41 creates a `JFileChooser` and assigns its reference to `fileChooser`. Lines 42–43 call method **`setFileSelectionMode`** to specify what the user can select from the `fileChooser`. For this program, we use `JFileChooser` `static` constant **`FILES_AND_DIRECTORIES`** to indicate that files and directories can be selected. Other `static` constants include **`FILES_ONLY`** (the default) and **`DIRECTORIES_ONLY`**.

Line 45 calls method **`showOpenDialog`** to display the `JFileChooser` dialog titled **Open**. Argument `this` specifies the `JFileChooser` dialog's parent window, which determines the position of the dialog on the screen. If `null` is passed, the dialog is displayed in the center of the screen—otherwise, the dialog is centered over the application window (specified by the argument `this`). A `JFileChooser` dialog is a *modal dialog* that does not allow the user to interact with any other window in the program until the user closes the `JFileChooser` by clicking the **Open** or **Cancel** button. The user selects the drive, directory or file name, then clicks **Open**. Method `showOpenDialog` returns an integer specifying which button (**Open** or **Cancel**) the user clicked to close the dialog. Line 48 tests whether the user clicked **Cancel** by comparing the result with `static` constant **`CANCEL_OPTION`**. If they're equal, the program terminates. Line 51 retrieves the file the user selected by calling `JFileChooser` method **`getSelectedFile`**. The program then displays information about the selected file or directory.

17.8 Wrap-Up

In this chapter, you learned how to manipulate persistent data. We compared character-based and byte-based streams, and introduced several file-processing classes from the `java.io` package. You used class `File` to retrieve information about a file or directory. You used sequential-access file processing to manipulate records that are stored in order by the record-key field. You learned the differences between text-file processing and object serialization, and used serialization to store and retrieve entire objects. The chapter concluded with a small example of using a `JFileChooser` dialog to allow users to easily select files from a GUI. The next chapter presents recursion—methods that call themselves.

Summary

Section 17.1 Introduction

- Computers use files for long-term retention of large amounts of persistent data (p. 720), even after the programs that created the data terminate.
- Computers store files on secondary storage devices (p. 720) such as hard disks.

Section 17.2 Files and Streams

- Java views each file as a sequential stream of bytes (p. 720).
- Every operating system provides a mechanism to determine the end of a file, such as an end-of-file marker (p. 720) or a count of the total bytes in the file.
- Byte-based streams (p. 721) represent data in binary format.
- Character-based streams (p. 721) represent data as sequences of characters.
- Files that are created using byte-based streams are binary files (p. 721). Files created using character-based streams are text files (p. 721). Text files can be read by text editors, whereas binary files are read by a program that converts the data to a human-readable format.

- Java also can associate streams with different devices. Three stream objects are associated with devices when a Java program begins executing—`System.in`, `System.out` and `System.err`.

Section 17.3 Class `File`

- Class `File` (p. 722) is used to obtain information about files and directories.
- Character-based input and output can be performed with classes `Scanner` and `Formatter`.
- Class `Formatter` (p. 722) enables formatted data to be output to the screen or to a file in a manner similar to `System.out.printf`.
- A file or directory's path (p. 722) specifies its location on disk.
- An absolute path (p. 722) contains all the directories, starting with the root directory (p. 722), that lead to a specific file or directory. Every file or directory on a disk drive has the same root directory in its path.
- A relative path (p. 722) starts from the directory in which the application began executing.
- A separator character (p. 726) is used to separate directories and files in the path.

Section 17.4 Sequential-Access Text Files

- Java imposes no structure on a file. You must structure files to meet your application's needs.
- To retrieve data sequentially from a file, programs normally start from the beginning of the file and read all the data consecutively until the desired information is found.
- Data in many sequential files cannot be modified without the risk of destroying other data in the file. Records in a sequential-access file are usually updated by rewriting the entire file.

Section 17.5 Object Serialization

- Java provides a mechanism called object serialization (p. 742) that enables entire objects to be written to or read from a stream.
- A serialized object (p. 742) is represented as a sequence of bytes that includes the object's data as well as information about the object's type and the types of data it stores.
- After a serialized object has been written into a file, it can be read from the file and deserialized (p. 742) to recreate the object in memory.
- Classes `ObjectInputStream` (p. 722) and `ObjectOutputStream` (p. 722) enable entire objects to be read from or written to a stream (possibly a file).
- Only classes that implement interface `Serializable` (p. 743) can be serialized and deserialized.
- The `ObjectOutput` interface (p. 742) contains method `writeObject` (p. 743), which takes an `Object` as an argument and writes its information to an `OutputStream` (p. 751). A class that implements this interface, such as `ObjectOutputStream`, would ensure that the `Object` is `Serializable`.
- The `ObjectInput` interface (p. 742) contains method `readObject` (p. 743), which reads and returns a reference to an `Object` from an `InputStream`. After an object has been read, its reference can be cast to the object's actual type.

Section 17.6 Additional `java.io` Classes

- `InputStream` and `OutputStream` are `abstract` classes for performing byte-based I/O.
- Pipes (p. 751) are synchronized communication channels between threads. One thread sends data via a `PipedOutputStream` (p. 751). The target thread reads information from the pipe via a `PipedInputStream` (p. 751).
- A filter stream (p. 751) provides additional functionality, such as aggregating data bytes into meaningful primitive-type units. `FilterInputStream` (p. 751) and `FilterOutputStream` are typically extended, so some of their filtering capabilities are provided by their concrete subclasses.

- A `PrintStream` (p. 751) performs text output. `System.out` and `System.err` are `PrintStreams`.
- Interface `DataInput` describes methods for reading primitive types from an input stream. Classes `DataInputStream` (p. 752) and `RandomAccessFile` each implement this interface.
- Interface `DataOutput` describes methods for writing primitive types to an output stream. Classes `DataOutputStream` (p. 752) and `RandomAccessFile` each implement this interface.
- Buffering is an I/O-performance-enhancement technique. Buffering reduces the number of I/O operations by combining smaller outputs together in memory. The number of physical I/O operations is much smaller than the number of I/O requests issued by the program.
- With a `BufferedOutputStream` (p. 752) each output operation is directed to a buffer (p. 752) large enough to hold the data of many output operations. Transfer to the output device is performed in one large physical output operation (p. 752) when the buffer fills. A partially filled buffer can be forced out to the device at any time by invoking the stream object's `flush` method (p. 752).
- With a `BufferedInputStream` (p. 752), many "logical" chunks of data from a file are read as one large physical input operation (p. 752) into a memory buffer. As a program requests data, it's taken from the buffer. When the buffer is empty, the next actual physical input operation is performed.
- A `ByteArrayInputStream` reads from a `byte` array in memory. A `ByteArrayOutputStream` outputs to a `byte` array in memory.
- A `SequenceInputStream` concatenates several `InputStreams`. When the program reaches the end of an input stream, that stream closes, and the next stream in the sequence opens.
- The `Reader` (p. 753) and `Writer` (p. 753) `abstract` classes are Unicode character-based streams. Most byte-based streams have corresponding character-based concrete `Reader` or `Writer` classes.
- Classes `BufferedReader` (p. 753) and `BufferedWriter` (p. 753) buffer character-based streams.
- Classes `CharArrayReader` (p. 753) and `CharArrayWriter` (p. 753) manipulate `char` arrays.
- A `LineNumberReader` (p. 753) is a buffered character stream tracks the number of lines read.
- Classes `FileReader` (p. 753) and `FileWriter` (p. 753) perform character-based file I/O.
- Class `PipedReader` (p. 753) and class `PipedWriter` (p. 753) implement piped-character streams for transfering data between threads.
- Class `StringReader` (p. 753) and `StringWriter` (p. 753) read characters from and write characters to `Strings`, respectively. A `PrintWriter` (p. 732) writes characters to a stream.

Section 17.7 Opening Files with `JFileChooser`

- Class `JFileChooser` (p. 754) is used to display a dialog that enables users of a program to easily select files or directories from a GUI.

Self-Review Exercises

17.1 Determine whether each of the following statements is *true* or *false*. If *false*, explain why.

a) You must explicitly create the stream objects `System.in`, `System.out` and `System.err`.

b) When reading data from a file using class `Scanner`, if you wish to read data in the file multiple times, the file must be closed and reopened to read from the beginning of the file.

c) Method `exists` of class `File` returns `true` if the name specified as the argument to the `File` constructor is a file or directory in the specified path.

d) Binary files are human readable in a text editor.

e) An absolute path contains all the directories, starting with the root directory, that lead to a specific file or directory.

f) Class `Formatter` contains method `printf`, which enables formatted data to be output to the screen or to a file.

17.2 Complete the following tasks, assuming that each applies to the same program:

a) Write a statement that opens file `"oldmast.txt"` for input—use `Scanner` variable `inOldMaster`.

b) Write a statement that opens file `"trans.txt"` for input—use `Scanner` variable `inTransaction`.

c) Write a statement that opens file `"newmast.txt"` for output (and creation)—use `formatter` variable `outNewMaster`.

d) Write the statements needed to read a record from the file `"oldmast.txt"`. Use the data to create an object of class `AccountRecord`—use `Scanner` variable `inOldMaster`. Assume that class `AccountRecord` is the same as the `AccountRecord` class in Fig. 17.4.

e) Write the statements needed to read a record from the file `"trans.txt"`. The record is an object of class `TransactionRecord`—use `Scanner` variable `inTransaction`. Assume that class `TransactionRecord` contains method `setAccount` (which takes an `int`) to set the account number and method `setAmount` (which takes a `double`) to set the amount of the transaction.

f) Write a statement that outputs a record to the file `"newmast.txt"`. The record is an object of type `AccountRecord`—use `Formatter` variable `outNewMaster`.

17.3 Complete the following tasks, assuming that each applies to the same program:

a) Write a statement that opens file `"oldmast.ser"` for input—use `ObjectInputStream` variable `inOldMaster` to wrap a `FileInputStream` object.

b) Write a statement that opens file `"trans.ser"` for input—use `ObjectInputStream` variable `inTransaction` to wrap a `FileInputStream` object.

c) Write a statement that opens file `"newmast.ser"` for output (and creation)—use `ObjectOutputStream` variable `outNewMaster` to wrap a `FileOutputStream`.

d) Write a statement that reads a record from the file `"oldmast.ser"`. The record is an object of class `AccountRecordSerializable`—use `ObjectInputStream` variable `inOldMaster`. Assume class `AccountRecordSerializable` is the same as the `AccountRecordSerializable` class in Fig. 17.15

e) Write a statement that reads a record from the file `"trans.ser"`. The record is an object of class `TransactionRecord`—use `ObjectInputStream` variable `inTransaction`.

f) Write a statement that outputs a record of type `AccountRecordSerializable` to the file `"newmast.ser"`—use `ObjectOutputStream` variable `outNewMaster`.

17.4 Find the error in each block of code and show how to correct it.

a) Assume that `account`, `company` and `amount` are declared.

```
ObjectOutputStream outputStream;
outputStream.writeInt( account );
outputStream.writeChars( company );
outputStream.writeDouble( amount );
```

b) The following statements should read a record from the file `"payables.txt"`. The `Scanner` variable `inPayable` should be used to refer to this file.

```
Scanner inPayable = new Scanner( new File( "payables.txt" ) );
PayablesRecord record = ( PayablesRecord ) inPayable.readObject();
```

Answers to Self-Review Exercises

17.1 a) False. These three streams are created for you when a Java application begins executing.

b) True.

c) True.

d) False. Text files are human readable in a text editor. Binary files might be human readable, but only if the bytes in the file represent ASCII characters

e) True.
f) False. Class `Formatter` contains method `format`, which enables formatted data to be output to the screen or to a file.

17.2 a)
```
Scanner inOldMaster = new Scanner( new File ( "oldmast.txt" ) );
```
b)
```
Scanner inTransaction = new Scanner( new File( "trans.txt" ) );
```
c)
```
Formatter outNewMaster = new Formatter( "newmast.txt" );
```
d)
```
AccountRecord account = new AccountRecord();
account.setAccount( inOldMaster.nextInt() );
account.setFirstName( inOldMaster.next() );
account.setLastName( inOldMaster.next() );
account.setBalance( inOldMaster.nextDouble() );
```
e)
```
TransactionRecord transaction = new Transaction();
transaction.setAccount( inTransaction.nextInt() );
transaction.setAmount( inTransaction.nextDouble() );
```
f)
```
outNewMaster.format( "%d %s %s %.2f\n",
account.getAccount(), account.getFirstName(),
account.getLastName(), account.getBalance() );
```

17.3 a)
```
ObjectInputStream inOldMaster = new ObjectInputStream(
   new FileInputStream( "oldmast.ser" ) );
```
b)
```
ObjectInputStream inTransaction = new ObjectInputStream(
   new FileInputStream( "trans.ser" ) );
```
c)
```
ObjectOutputStream outNewMaster = new ObjectOutputStream(
   new FileOutputStream( "newmast.ser" ) );
```
d)
```
accountRecord = ( AccountRecordSerializable ) inOldMaster.readObject();
```
e)
```
transactionRecord = ( TransactionRecord ) inTransaction.readObject();
```
f)
```
outNewMaster.writeObject( newAccountRecord );
```

17.4 a) Error: The file was not opened before the attempt to output data to the stream.
Correction: Open a file for output by creating a new `ObjectOutputStream` object that wraps a `FileOutputStream` object.

b) Error: This example uses text files with a `Scanner`; there's no object serialization. As a result, method `readObject` cannot be used to read that data from the file. Each piece of data must be read separately, then used to create a `PayablesRecord` object.
Correction: Use `methods` of `inPayable` to read each piece of the `PayablesRecord` object.

Exercises

17.5 ***(File Matching)*** Self-Review Exercise 17.2 asked you to write a series of single statements. Actually, these statements form the core of an important type of file-processing program—namely, a file-matching program. In commercial data processing, it's common to have several files in each application system. In an accounts receivable system, for example, there's generally a master file containing detailed information about each customer, such as the customer's name, address, telephone number, outstanding balance, credit limit, discount terms, contract arrangements and possibly a condensed history of recent purchases and cash payments.

As transactions occur (i.e., sales are made and payments arrive in the mail), information about them is entered into a file. At the end of each business period (a month for some companies, a week for others, and a day in some cases), the file of transactions (called `"trans.txt"`) is applied to the master file (called `"oldmast.txt"`) to update each account's purchase and payment record. During an update, the master file is rewritten as the file `"newmast.txt"`, which is then used at the end of the next business period to begin the updating process again.

File-matching programs must deal with certain problems that do not arise in single-file programs. For example, a match does not always occur. If a customer on the master file has not made any purchases or cash payments in the current business period, no record for this customer will appear on the transaction file. Similarly, a customer who did make some purchases or cash payments could have just moved to this community, and if so, the company may not have had a chance to create a master record for this customer.

Write a complete file-matching accounts receivable program. Use the account number on each file as the record key for matching purposes. Assume that each file is a sequential text file with records stored in increasing account-number order.

a) Define class `TransactionRecord`. Objects of this class contain an account number and amount for the transaction. Provide methods to modify and retrieve these values.

b) Modify class `AccountRecord` in Fig. 17.4 to include method `combine`, which takes a `TransactionRecord` object and combines the balance of the `AccountRecord` object and the amount value of the `TransactionRecord` object.

c) Write a program to create data for testing the program. Use the sample account data in Figs. 17.22 and 17.23. Run the program to create the files `trans.txt` and `oldmast.txt` to be used by your file-matching program.

Master file account number	Name	Balance
100	Alan Jones	348.17
300	Mary Smith	27.19
500	Sam Sharp	0.00
700	Suzy Green	-14.22

Fig. 17.22 | Sample data for master file.

Transaction file account number	Transaction amount
100	27.14
300	62.11
400	100.56
900	82.17

Fig. 17.23 | Sample data for transaction file.

d) Create class `FileMatch` to perform the file-matching functionality. The class should contain methods that read `oldmast.txt` and `trans.txt`. When a match occurs (i.e., records with the same account number appear in both the master file and the transaction file), add the dollar amount in the transaction record to the current balance in the master record, and write the `"newmast.txt"` record. (Assume that purchases are indicated by positive amounts in the transaction file and payments by negative amounts.) When there's a master record for a particular account, but no corresponding transaction record, merely write the master record to `"newmast.txt"`. When there's a transaction record, but no corresponding master record, print to a log file the message `"Unmatched transaction record for account number..."` (fill in the account number from the transaction record). The log file should be a text file named `"log.txt"`.

17.6 ***(File Matching with Multiple Transactions)*** It's possible (and actually common) to have several transaction records with the same record key. This situation occurs, for example, when a customer makes several purchases and cash payments during a business period. Rewrite your accounts receivable file-matching program from Exercise 17.5 to provide for the possibility of handling several transaction records with the same record key. Modify the test data of `CreateData.java` to include the additional transaction records in Fig. 17.24.

Account number	Dollar amount
300	83.89
700	80.78
700	1.53

Fig. 17.24 | Additional transaction records.

17.7 ***(File Matching with Object Serialization)*** Recreate your solution for Exercise 17.6 using object serialization. Use the statements from Exercise 17.3 as your basis for this program. You may want to create applications to read the data stored in the `.ser` files—the code in Section 17.5.2 can be modified for this purpose.

17.8 ***(Telephone-Number Word Generator)*** Standard telephone keypads contain the digits zero through nine. The numbers two through nine each have three letters associated with them (Fig. 17.25). Many people find it difficult to memorize phone numbers, so they use the correspondence between digits and letters to develop seven-letter words that correspond to their phone numbers. For example, a person whose telephone number is 686-2377 might use the correspondence indicated in Fig. 17.25 to develop the seven-letter word "NUMBERS." Every seven-letter word corresponds to exactly one seven-digit telephone number. A restaurant wishing to increase its takeout business could surely do so with the number 825-3688 (i.e., "TAKEOUT").

Digit	Letters	Digit	Letters	Digit	Letters
2	A B C	5	J K L	8	T U V
3	D E F	6	M N O	9	W X Y
4	G H I	7	P R S		

Fig. 17.25 | Telephone keypad digits and letters.

Every seven-letter phone number corresponds to many different seven-letter words, but most of these words represent unrecognizable juxtapositions of letters. It's possible, however, that the owner of a barbershop would be pleased to know that the shop's telephone number, 424-7288, corresponds to "HAIRCUT." A veterinarian with the phone number 738-2273 would be pleased to know that the number corresponds to the letters "PETCARE." An automotive dealership would be pleased to know that the dealership number, 639-2277, corresponds to "NEWCARS."

Write a program that, given a seven-digit number, uses a `PrintStream` object to write to a file every possible seven-letter word combination corresponding to that number. There are 2,187 (3^7) such combinations. Avoid phone numbers with the digits 0 and 1.

17.9 ***(Student Poll)*** Figure 7.8 contains an array of survey responses that's hard coded into the program. Suppose we wish to process survey results that are stored in a file. This exercise requires two

separate programs. First, create an application that prompts the user for survey responses and outputs each response to a file. Use a `Formatter` to create a file called `numbers.txt`. Each integer should be written using method `format`. Then modify the program in Fig. 7.8 to read the survey responses from `numbers.txt`. The responses should be read from the file by using a `Scanner`. Use method `nextInt` to input one integer at a time from the file. The program should continue to read responses until it reaches the end of the file. The results should be output to the text file `"output.txt"`.

17.10 *(Adding Object Serialization to the `MyShape` Drawing Application)* Modify Exercise 14.17 to allow the user to save a drawing into a file or load a prior drawing from a file using object serialization. Add buttons **Load** (to read objects from a file) and **Save** (to write objects to a file). Use an `ObjectOutputStream` to write to the file and an `ObjectInputStream` to read from the file. Write the array of `MyShape` objects using method `writeObject` (class `ObjectOutputStream`), and read the array using method `readObject` (`ObjectInputStream`). The object-serialization mechanism can read or write entire arrays—it's not necessary to manipulate each element of the array of `MyShape` objects individually. It's simply required that all the shapes be `Serializable`. For both the **Load** and **Save** buttons, use a `JFileChooser` to allow the user to select the file in which the shapes will be stored or from which they'll be read. When the user first runs the program, no shapes should be displayed on the screen. The user can display shapes by opening a previously saved file or by drawing new shapes. Once there are shapes on the screen, users can save them to a file using the **Save** button.

Making a Difference

17.11 *(Phishing Scanner)* Phishing is a form of identity theft in which, in an e-mail, a sender posing as a trustworthy source attempts to acquire private information, such as your user names, passwords, credit-card numbers and social security number. Phishing e-mails claiming to be from popular banks, credit-card companies, auction sites, social networks and online payment services may look quite legitimate. These fraudulent messages often provide links to spoofed (fake) websites where you're asked to enter sensitive information.

Visit McAfee® (`www.mcafee.com/us/threat_center/anti_phishing/phishing_top10.html`), Security Extra (`www.securityextra.com/`) and other websites to find lists of the top phishing scams. Also check out the Anti-Phishing Working Group (`www.antiphishing.org/`), and the FBI's Cyber Investigations website (`www.fbi.gov/cyberinvest/cyberhome.htm`), where you'll find information about the latest scams and how to protect yourself.

Create a list of 30 words, phrases and company names commonly found in phishing messages. Assign a point value to each based on your estimate of its likeliness to be in a phishing message (e.g., one point if it's somewhat likely, two points if moderately likely, or three points if highly likely). Write an application that scans a file of text for these terms and phrases. For each occurrence of a keyword or phrase within the text file, add the assigned point value to the total points for that word or phrase. For each keyword or phrase found, output one line with the word or phrase, the number of occurrences and the point total. Then show the point total for the entire message. Does your program assign a high point total to some actual phishing e-mails you've received? Does it assign a high point total to some legitimate e-mails you've received?

18

Recursion



We must learn to explore all the options and possibilities that confront us in a complex and rapidly changing world.
—James William Fulbright

O! thou hast damnable iteration, and art indeed able to corrupt a saint.
—William Shakespeare

It's a poor sort of memory that only works backwards.
—Lewis Carroll

Life can only be understood backwards; but it must be lived forwards.
—Soren Kierkegaard

Objectives

In this chapter you'll learn:

- The concept of recursion.
- How to write and use recursive methods.
- How to determine the base case and recursion step in a recursive algorithm.
- How recursive method calls are handled by the system.
- The differences between recursion and iteration, and when to use each.
- What the geometric shapes called fractals are and how to draw them using recursion.
- What recursive backtracking is and why it's an effective problem-solving technique.

Outline

18.1 Introduction
18.2 Recursion Concepts
18.3 Example Using Recursion: Factorials
18.4 Example Using Recursion: Fibonacci Series
18.5 Recursion and the Method-Call Stack
18.6 Recursion vs. Iteration
18.7 Towers of Hanoi
18.8 Fractals
18.9 Recursive Backtracking
18.10 Wrap-Up

Summary | *Self-Review Exercises* | *Answers to Self-Review Exercises* | *Exercises*

18.1 Introduction

The programs we've discussed so far are generally structured as methods that call one another in a hierarchical manner. For some problems, it's useful to have a method call itself. A method that does so is known as a **recursive method**. A recursive method can call itself either *directly* or *indirectly through another method*. Recursion is an important topic discussed at length in upper-level computer science courses. In this chapter, we consider recursion conceptually, then present several programs containing recursive methods. Figure 18.1 summarizes the recursion examples and exercises in this book.

Chapter	Recursion examples and exercises in this book
18	Factorial Method (Figs. 18.3 and 18.4) Fibonacci Method (Fig. 18.5) Towers of Hanoi (Fig. 18.11) Fractals (Figs. 18.18 and 18.19) What Does This Code Do? (Exercise 18.7, Exercise 18.12 and Exercise 18.13) Find the Error in the Following Code (Exercise 18.8) Raising an Integer to an Integer Power (Exercise 18.9) Visualizing Recursion (Exercise 18.10) Greatest Common Divisor (Exercise 18.11) Determine Whether a String Is a Palindrome (Exercise 18.14) Eight Queens (Exercise 18.15) Print an Array (Exercise 18.16) Print an Array Backward (Exercise 18.17) Minimum Value in an Array (Exercise 18.18) Star Fractal (Exercise 18.19) Maze Traversal Using Recursive Backtracking (Exercise 18.20) Generating Mazes Randomly (Exercise 18.21) Mazes of Any Size (Exercise 18.22) Time Needed to Calculate a Fibonacci Number (Exercise 18.23)
19	Merge Sort (Figs. 19.10 and 19.11) Linear Search (Exercise 19.8) Binary Search (Exercise 19.9) Quicksort (Exercise 19.10)

Fig. 18.1 | Summary of the recursion examples and exercises in this text. (Part 1 of 2.)

Chapter	Recursion examples and exercises in this book
22	Binary-Tree Insert (Fig. 22.17) Preorder Traversal of a Binary Tree (Fig. 22.17) Inorder Traversal of a Binary Tree (Fig. 22.17) Postorder Traversal of a Binary Tree (Fig. 22.17) Print a Linked List Backward (Exercise 22.20) Search a Linked List (Exercise 22.21)

Fig. 18.1 | Summary of the recursion examples and exercises in this text. (Part 2 of 2.)

18.2 Recursion Concepts

Recursive problem-solving approaches have a number of elements in common. When a recursive method is called to solve a problem, it actually is capable of solving only the simplest case(s), or **base case(s)**. If the method is called with a base case, it returns a result. If the method is called with a more complex problem, it typically divides the problem into two conceptual pieces—a piece that the method knows how to do and a piece that it does not know how to do. To make recursion feasible, the latter piece must resemble the original problem, but be a slightly simpler or smaller version of it. Because this new problem looks like the original problem, the method calls a fresh copy of itself to work on the smaller problem—this is referred to as a **recursive call** and is also called the **recursion step**. The recursion step normally includes a `return` statement, because its result will be combined with the portion of the problem the method knew how to solve to form a result that will be passed back to the original caller. This concept of separating the problem into two smaller portions is a form of the divide-and-conquer approach introduced in Chapter 6.

The recursion step executes while the original method call is still active (i.e., it has not finished executing). It can result in many more recursive calls as the method divides each new subproblem into two conceptual pieces. For the recursion to eventually terminate, each time the method calls itself with a simpler version of the original problem, the sequence of smaller and smaller problems must *converge on a base case*. When the method recognizes the base case, it returns a result to the previous copy of the method. A sequence of returns ensues until the original method call returns the final result to the caller.

A recursive method may call another method, which may in turn make a call back to the recursive method. This is known as an **indirect recursive call** or **indirect recursion**. For example, method `A` calls method `B`, which makes a call back to method `A`. This is still recursion, because the second call to method `A` is made while the first call to method `A` is active—that is, the first call to method `A` has not yet finished executing (because it's waiting on method `B` to return a result to it) and has not returned to method `A`'s original caller.

To better understand the concept of recursion, let's look at an example that's quite familiar to computer users—the recursive definition of a directory on a computer. A computer normally stores related files in a directory. A directory can be empty, can contain files and/or can contain other directories (usually referred to as subdirectories). Each of these subdirectories, in turn, may also contain both files and directories. If we want to list each file in a directory (including all the files in the directory's subdirectories), we need to create a method that first lists the initial directory's files, then makes recursive calls to list the files in each of that directory's subdirectories. The base case occurs when a directory is reached

that does not contain any subdirectories. At this point, all the files in the original directory have been listed and no further recursion is necessary.

18.3 Example Using Recursion: Factorials

Let's write a recursive program to perform a popular mathematical calculation. Consider the *factorial* of a positive integer n, written $n!$ (pronounced "n factorial"), which is the product

$$n \cdot (n-1) \cdot (n-2) \cdot \ldots \cdot 1$$

with 1! equal to 1 and 0! defined to be 1. For example, 5! is the product $5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$, which is equal to 120.

The factorial of integer `number` (where `number` ≥ 0) can be calculated iteratively (non-recursively) using a `for` statement as follows:

```
factorial = 1;
for ( int counter = number; counter >= 1; counter-- )
   factorial *= counter;
```

A recursive declaration of the factorial method is arrived at by observing the following relationship:

$$n! = n \cdot (n-1)!$$

For example, 5! is clearly equal to $5 \cdot 4!$, as shown by the following equations:

$$5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$$
$$5! = 5 \cdot (4 \cdot 3 \cdot 2 \cdot 1)$$
$$5! = 5 \cdot (4!)$$

The evaluation of 5! would proceed as shown in Fig. 18.2. Figure 18.2(a) shows how the succession of recursive calls proceeds until 1! (the base case) is evaluated to be 1, which

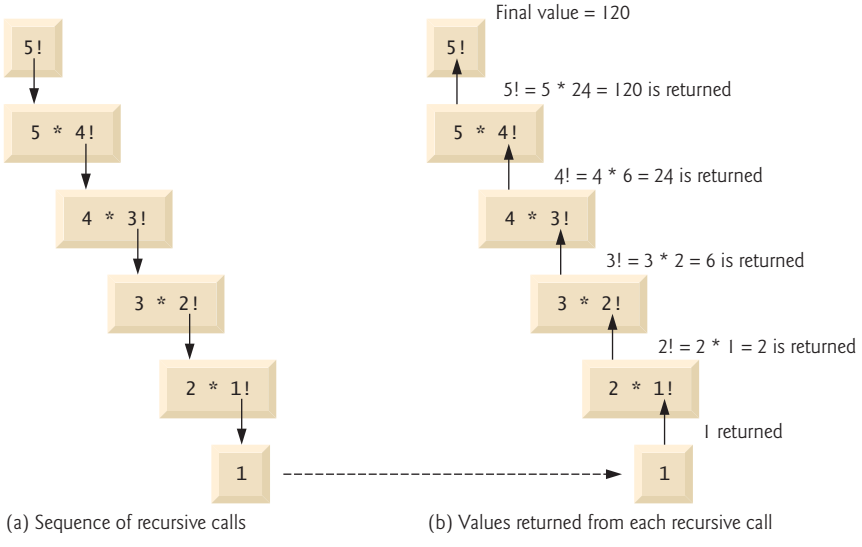


Fig. 18.2 | Recursive evaluation of 5!.

terminates the recursion. Figure 18.2(b) shows the values returned from each recursive call to its caller until the final value is calculated and returned.

Figure 18.3 uses recursion to calculate and print the factorials of the integers from 0–21. The recursive method `factorial` (lines 7–13) first tests to determine whether a terminating condition (line 9) is `true`. If `number` is less than or equal to `1` (the base case), `factorial` returns `1`, no further recursion is necessary and the method returns. If `number` is greater than `1`, line 12 expresses the problem as the product of `number` and a recursive call to `factorial` evaluating the factorial of `number - 1`, which is a slightly smaller problem than the original calculation, `factorial( number )`.

```
1  // Fig. 18.3: FactorialCalculator.java
2  // Recursive factorial method.
3
4  public class FactorialCalculator
5  {
6     // recursive method factorial (assumes its parameter is >= 0)
7     public long factorial( long number )
8     {
9        if ( number <= 1 ) // test for base case
10          return 1; // base cases: 0! = 1 and 1! = 1
11       else // recursion step
12          return number * factorial( number - 1 );
13    } // end method factorial
14
15    // output factorials for values 0-21
16    public static void main( String[] args )
17    {
18       // calculate the factorials of 0 through 21
19       for ( int counter = 0; counter <= 21; counter++ )
20          System.out.printf( "%d! = %d\n", counter, factorial( counter ) );
21    } // end main
22 } // end class FactorialCalculator
```

```
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
...
12! = 479001600 —— 12! causes overflow for int variables
...
20! = 2432902008176640000
21! = -4249290049419214848 —— 21! causes overflow for long variables
```

Fig. 18.3 | Factorial calculations with a recursive method.



Common Programming Error 18.1

Either omitting the base case or writing the recursion step incorrectly so that it does not converge on the base case can cause a logic error known as ***infinite recursion****, where recursive calls are continuously made until memory is exhausted. This error is analogous to the problem of an infinite loop in an iterative (nonrecursive) solution.*

Method `main` (lines 16–21) displays the factorials of 0–21. The call to the `factorial` method occurs in line 20. Method `factorial` receives a parameter of type `long` and returns a result of type `long`. The program's output shows that factorial values become large quickly. We use type `long` (which can represent relatively large integers) so the program can calculate factorials greater than 12!. Unfortunately, the `factorial` method produces large values so quickly that we exceed the largest `long` value when we attempt to calculate 21!, as you can see in the last line of the program's output.

Due to the limitations of integral types, `float` or `double` variables may ultimately be needed to calculate factorials of larger numbers. This points to a weakness in some programming languages—namely, that they aren't easily extended with new types to handle unique application requirements. As we saw in Chapter 9, Java is an *extensible* language that allows us to create arbitrarily large integers if we wish. In fact, package `java.math` provides classes **`BigInteger`** and **`BigDecimal`** explicitly for arbitrary precision calculations that cannot be performed with primitive types. You can learn more about these classes at

```
download.oracle.com/javase/6/docs/api/java/math/BigInteger.html
download.oracle.com/javase/6/docs/api/java/math/BigDecimal.html
```

Reimplementing Class **`FactorialCalculator`** *Using Class* **`BigInteger`**

Figure 18.4 reimplements class `FactorialCalculator` using `BigInteger` variables. To demonstrate larger values than what `long` variables can store, we calculate the factorials of the numbers 0–50. Line 3 imports class `BigInteger` from package `java.math`. The new `factorial` method (lines 8–15) receives a `BigInteger` as an argument and returns a `BigInteger`.

```
1  // Fig. 18.4: FactorialCalculator.java
2  // Recursive factorial method.
3  import java.math.BigInteger;
4
5  public class FactorialCalculator
6  {
7     // recursive method factorial (assumes its parameter is >= 0)
8     public static BigInteger factorial( BigInteger number )
9     {
10       if ( number.compareTo( BigInteger.ONE ) <= 0 ) // test base case
11          return BigInteger.ONE; // base cases: 0! = 1 and 1! = 1
12       else // recursion step
13          return number.multiply(
14             factorial( number.subtract( BigInteger.ONE ) ) );
15    } // end method factorial
16
17    // output factorials for values 0-50
18    public static void main( String[] args )
19    {
20       // calculate the factorials of 0 through 50
21       for ( int counter = 0; counter <= 50; counter++ )
22          System.out.printf( "%d! = %d\n", counter,
23             factorial( BigInteger.valueOf( counter ) ) );
24    } // end main
25 } // end class FactorialCalculator
```

Fig. 18.4 | Factorial calculations with a recursive method. (Part 1 of 2.)

```
0! = 1
1! = 1
2! = 2
3! = 6
...
21! = 51090942171709440000 ── 21! and larger values no longer cause overflow
22! = 1124000727777607680000
...
47! = 258623241511168180642964355153611979969197632389120000000000
48! = 12413915592536072670862289047373375038521486354677760000000000
49! = 608281864034267560872252163321295376887552831379210240000000000
50! = 30414093201713378043612608166064768844377641568960512000000000000
```

Fig. 18.4 | Factorial calculations with a recursive method. (Part 2 of 2.)

Since `BigInteger` is not a primitive type, we can't use the arithmetic, relational and equality operators with `BigInteger`s; instead, we must use `BigInteger` methods to perform these tasks. Line 10 tests for the base case using `BigInteger` method **`compareTo`**. This method compares the `BigInteger` that calls the method to the method's `BigInteger` argument. The method returns `-1` if the `BigInteger` that calls the method is less than the argument, `0` if they're equal or `1` if the `BigInteger` that calls the method is greater than the argument. Line 10 compares the `BigInteger number` with the `BigInteger` constant **`ONE`**, which represents the integer value 1. If `compareTo` returns `-1` or `0`, then number is less than or equal to 1 (the base case) and the method returns the constant `BigInteger.ONE`. Otherwise, lines 13–14 perform the recursion step using `BigInteger` methods **`multiply`** and **`subtract`** to implement the calculations required to multiply `number` by the factorial of `number - 1`). The program's output shows that `BigInteger` easily handles the large values produced by the factorial calculation.

18.4 Example Using Recursion: Fibonacci Series

The **Fibonacci series**,

0, 1, 1, 2, 3, 5, 8, 13, 21, …

begins with 0 and 1 and has the property that each subsequent Fibonacci number is the sum of the previous two. This series occurs in nature and describes a form of spiral. The ratio of successive Fibonacci numbers converges on a constant value of 1.618…, a number that has been called the **golden ratio** or the **golden mean.** Humans tend to find the golden mean aesthetically pleasing. Architects often design windows, rooms and buildings whose length and width are in the ratio of the golden mean. Postcards are often designed with a golden-mean length-to-width ratio.

The Fibonacci series may be defined recursively as follows:

fibonacci(0) = 0
fibonacci(1) = 1
fibonacci(n) = fibonacci(n – 1) + fibonacci(n – 2)

There are *two base cases* for the Fibonacci calculation: `fibonacci(0)` is defined to be 0, and `fibonacci(1)` to be 1. The program in Fig. 18.5 calculates the ith Fibonacci number recursively, using method `fibonacci` (lines 10–18). Method `main` (lines 21–26) tests `fibonacci`,

displaying the Fibonacci values of 0–40. The variable `counter` created in the `for` header (line 23) indicates which Fibonacci number to calculate for each iteration of the loop. Fibonacci numbers tend to become large quickly (though not as quickly as factorials). Therefore, we use type `BigInteger` as the parameter type and the return type of method `fibonacci`.

```
1  // Fig. 18.5: FibonacciCalculator.java
2  // Recursive fibonacci method.
3  import java.math.BigInteger;
4
5  public class FibonacciCalculator
6  {
7     private static BigInteger TWO = BigInteger.valueOf( 2 );
8
9     // recursive declaration of method fibonacci
10    public static BigInteger fibonacci( BigInteger number )
11    {
12       if ( number.equals( BigInteger.ZERO ) ||
13            number.equals( BigInteger.ONE ) ) // base cases
14          return number;
15       else // recursion step
16          return fibonacci( number.subtract( BigInteger.ONE ) ).add(
17             fibonacci( number.subtract( TWO ) ) );
18    } // end method fibonacci
19
20    // displays the fibonacci values from 0-40
21    public static void main( String[] args )
22    {
23       for ( int counter = 0; counter <= 40; counter++ )
24          System.out.printf( "Fibonacci of %d is: %d\n", counter,
25             fibonacci( BigInteger.valueOf( counter ) ) );
26    } // end main
27 } // end class FibonacciCalculator
```

```
Fibonacci of 0 is: 0
Fibonacci of 1 is: 1
Fibonacci of 2 is: 1
Fibonacci of 3 is: 2
Fibonacci of 4 is: 3
Fibonacci of 5 is: 5
Fibonacci of 6 is: 8
Fibonacci of 7 is: 13
Fibonacci of 8 is: 21
Fibonacci of 9 is: 34
Fibonacci of 10 is: 55
...
Fibonacci of 37 is: 24157817
Fibonacci of 38 is: 39088169
Fibonacci of 39 is: 63245986
Fibonacci of 40 is: 102334155
```

Fig. 18.5 | Fibonacci numbers generated with a recursive method.

The call to method `fibonacci` (line 25 of Fig. 18.5) from `main` is not a recursive call, but all subsequent calls to `fibonacci` performed from lines 16–17 of Fig. 18.5 are recur-

sive, because at that point the calls are initiated by method `fibonacci` itself. Each time `fibonacci` is called, it immediately tests for the base cases—`number` equal to 0 or `number` equal to 1 (Fig. 18.5, lines 12–13). We use `BigInteger` constants **`ZERO`** and `ONE` to represent the values 0 and 1, respectively. If the condition in lines 12–13 is `true`, `fibonacci` simply returns `number`, because `fibonacci(0)` is 0 and `fibonacci(1)` is 1. Interestingly, if `number` is greater than 1, the recursion step generates *two* recursive calls (lines 16–17), each for a slightly smaller problem than the original call to `fibonacci`. Lines 16–17 use `BigInteger` methods **`add`** and `subtract` to help implement the recursive step. We also use a constant of type `BigInteger` named `TWO` that we defined at line 7.

Analyzing the Calls to Method `Fibonacci`

Figure 18.6 shows how method `fibonacci` evaluates `fibonacci(3)`. At the bottom of the figure we're left with the values 1, 0 and 1—the results of evaluating the base cases. The first two return values (from left to right), 1 and 0, are returned as the values for the calls `fibonacci(1)` and `fibonacci(0)`. The sum 1 plus 0 is returned as the value of `fibonacci(2)`. This is added to the result (1) of the call to `fibonacci(1)`, producing the value 2. This final value is then returned as the value of `fibonacci(3)`.

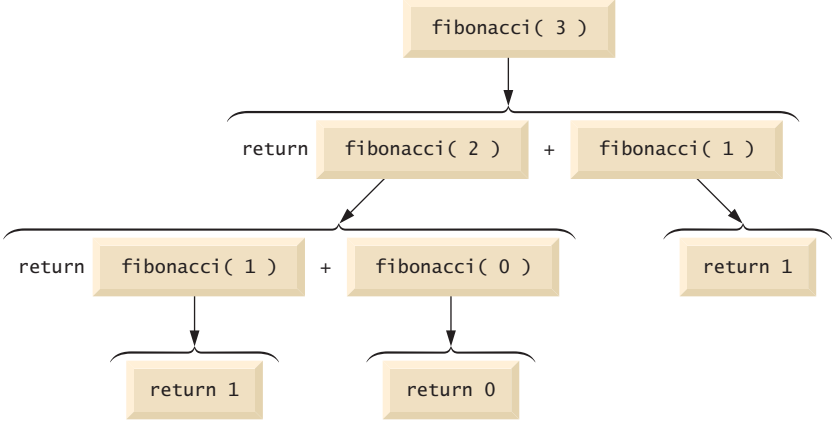


Fig. 18.6 | Set of recursive calls for `fibonacci( 3 )`.

Figure 18.6 raises some interesting issues about *the order in which Java compilers evaluate the operands of operators*. This order is different from that in which operators are applied to their operands—namely, the order dictated by the rules of operator precedence. From Figure 18.6, it appears that while `fibonacci(3)` is being evaluated, two recursive calls will be made—`fibonacci(2)` and `fibonacci(1)`. But in what order will they be made? *The Java language specifies that the order of evaluation of the operands is from left to right.* Thus, the call `fibonacci(2)` is made first and the call `fibonacci(1)` second.

A word of caution is in order about recursive programs like the one we use here to generate Fibonacci numbers. Each invocation of the `fibonacci` method that does not match one of the base cases (0 or 1) results in *two more recursive calls* to the `fibonacci` method. Hence, this set of recursive calls rapidly gets out of hand. Calculating the Fibonacci value of

20 with the program in Fig. 18.5 requires 21,891 calls to the `fibonacci` method; calculating the Fibonacci value of 30 requires 2,692,537 calls! As you try to calculate larger Fibonacci values, you'll notice that each consecutive Fibonacci number you use the application to calculate results in a substantial increase in calculation time and in the number of calls to the `fibonacci` method. For example, the Fibonacci value of 31 requires 4,356,617 calls, and the Fibonacci value of 32 requires 7,049,155 calls! As you can see, the number of calls to `fibonacci` increases quickly—1,664,080 additional calls between Fibonacci values of 30 and 31 and 2,692,538 additional calls between Fibonacci values of 31 and 32! The difference in the number of calls made between Fibonacci values of 31 and 32 is more than 1.5 times the difference in the number of calls for Fibonacci values between 30 and 31. Problems of this nature can humble even the world's most powerful computers. [*Note:* In the field of complexity theory, computer scientists study how hard algorithms work to complete their tasks. Complexity issues are discussed in detail in the upper-level computer science curriculum course generally called "Algorithms." We introduce various complexity issues in Chapter 19, Searching, Sorting and Big O.] In the exercises, you'll be asked to enhance the Fibonacci program of Fig. 18.5 so that it calculates the approximate amount of time required to perform the calculation. For this purpose, you'll call `static System` method `currentTimeMillis`, which takes no arguments and returns the computer's current time in milliseconds.



Performance Tip 18.1

Avoid Fibonacci-style recursive programs, because they result in an exponential "explosion" of method calls.

18.5 Recursion and the Method-Call Stack

In Chapter 6, the stack data structure was introduced in the context of understanding how Java performs method calls. We discussed both the method-call stack (also known as the program-execution stack) and activation records. In this section, we'll use these concepts to demonstrate how the program-execution stack handles recursive method calls.

Let's begin by returning to the Fibonacci example—specifically, calling method `fibonacci` with the value `3`, as in Fig. 18.6. To show the order in which the method calls' activation records are placed on the stack, we've lettered the method calls in Fig. 18.7.

When the first method call (`A`) is made, an activation record containing the value of the local variable `number` (`3`, in this case) is pushed onto the program-execution stack. This stack, including the activation record for method call `A`, is illustrated in part (a) of Fig. 18.8. [*Note:* We use a simplified stack here. An actual program-execution stack and its activation records would be more complex than in Fig. 18.8, containing such information as where the method call is to return to when it has completed execution.]

Within method call `A`, method calls `B` and `E` are made. The original method call has not yet completed, so its activation record remains on the stack. The first method call to be made from within `A` is method call `B`, so the activation record for method call `B` is pushed onto the stack on top of the one for method call `A`. Method call `B` must execute and complete before method call `E` is made. Within method call `B`, method calls `C` and `D` will be made. Method call `C` is made first, and its activation record is pushed onto the stack [part (b) of Fig. 18.8]. Method call `B` has not yet finished, and its activation record is still on the method-call stack. When method call `C` executes, it makes no further method calls, but simply returns the value `1`. When this method returns, its activation record is popped

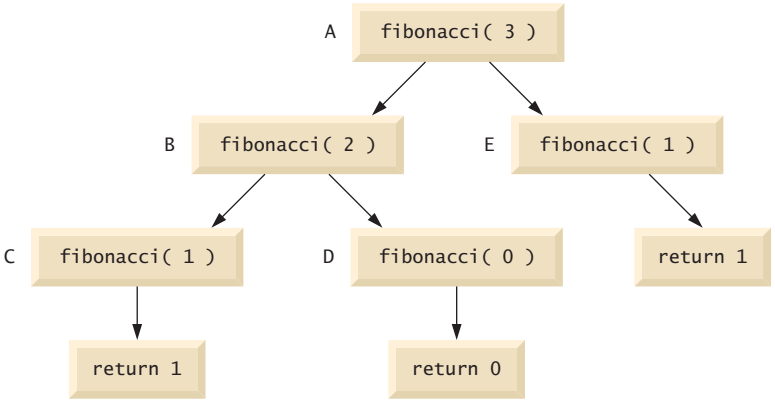


Fig. 18.7 | Method calls made within the call `fibonacci( 3 )`.

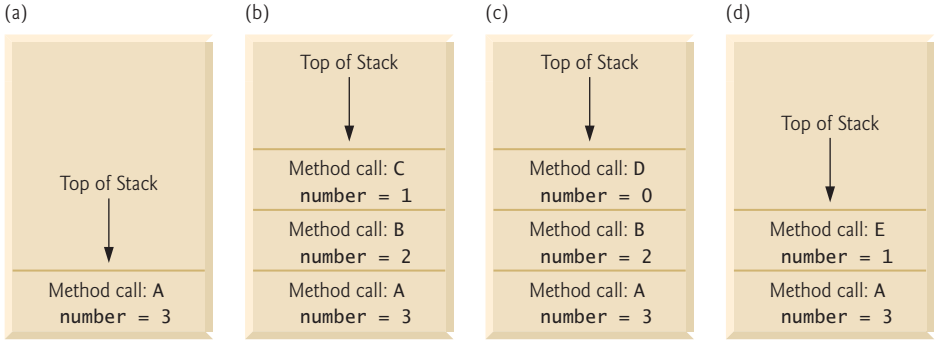


Fig. 18.8 | Method calls on the program-execution stack.

off the top of the stack. The method call at the top of the stack is now `B`, which continues to execute by performing method call `D`. The activation record for method call `D` is pushed onto the stack [part (c) of Fig. 18.8]. Method call `D` completes without making any more method calls and returns the value `0`. The activation record for this method call is then popped off the stack. Now, both method calls made from within method call `B` have returned. Method call `B` continues to execute, returning the value `1`. Method call `B` completes, and its activation record is popped off the stack. At this point, the activation record for method call `A` is at the top of the stack and the method continues its execution. This method makes method call `E`, whose activation record is now pushed onto the stack [part (d) of Fig. 18.8]. Method call `E` completes and returns the value `1`. The activation record for this method call is popped off the stack, and once again method call `A` continues to execute. At this point, method call `A` will not be making any other method calls and can finish its execution, returning the value `2` to `A`'s caller (`fibonacci(3) = 2`). `A`'s activation record is popped off the stack. The executing method is always the one whose activation record is at the top of the stack, and the activation record for that method contains the values of its local variables.

18.6 Recursion vs. Iteration

We've studied methods `factorial` and `fibonacci`, which can easily be implemented either recursively or iteratively. In this section, we compare the two approaches and discuss why you might choose one approach over the other in a particular situation.

Both iteration and recursion are *based on a control statement*: Iteration uses a repetition statement (e.g., `for`, `while` or `do...while`), whereas recursion uses a selection statement (e.g., `if`, `if...else` or `switch`). Both iteration and recursion involve *repetition*: Iteration explicitly uses a repetition statement, whereas recursion achieves repetition through repeated method calls. Iteration and recursion each involve a *termination test*: Iteration terminates when the loop-continuation condition fails, whereas recursion terminates when a base case is reached. Iteration with counter-controlled repetition and recursion both *gradually approach termination*: Iteration keeps modifying a counter until the counter assumes a value that makes the loop-continuation condition fail, whereas recursion keeps producing smaller versions of the original problem until the base case is reached. Both iteration and recursion *can occur infinitely*: An infinite loop occurs with iteration if the loop-continuation test never becomes false, whereas infinite recursion occurs if the recursion step does not reduce the problem each time in a manner that converges on the base case, or if the base case is not tested.

To illustrate the differences between iteration and recursion, let's examine an iterative solution to the factorial problem (Fig. 18.9). A repetition statement is used (lines 12–13) rather than the selection statement of the recursive solution (lines 9–12 of Fig. 18.3). Both solutions use a termination test. In the recursive solution, line 9 tests for the base case. In the iterative solution, line 12 tests the loop-continuation condition—if the test fails, the loop terminates. Finally, instead of producing smaller versions of the original problem, the iterative solution uses a counter that is modified until the loop-continuation condition becomes false.

```
1  // Fig. 18.9: FactorialCalculator.java
2  // Iterative factorial method.
3
4  public class FactorialCalculator
5  {
6     // recursive declaration of method factorial
7     public long factorial( long number )
8     {
9        long result = 1;
10
11       // iterative declaration of method factorial
12       for ( long i = number; i >= 1; i-- )
13          result *= i;
14
15       return result;
16    } // end method factorial
17
18    // output factorials for values 0-10
19    public static void main( String[] args )
20    {
```

Fig. 18.9 | Iterative factorial solution. (Part 1 of 2.)

```
21         // calculate the factorials of 0 through 10
22         for ( int counter = 0; counter <= 10; counter++ )
23            System.out.printf( "%d! = %d\n", counter, factorial( counter ) );
24      } // end main
25   } // end class FactorialCalculator
```

```
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
8! = 40320
9! = 362880
10! = 3628800
```

Fig. 18.9 | Iterative factorial solution. (Part 2 of 2.)

Recursion has many *negatives*. It repeatedly invokes the mechanism, and consequently the *overhead, of method calls*. This repetition can be expensive in terms of both processor time and memory space. Each recursive call causes another copy of the method (actually, only the method's variables, stored in the activation record) to be created—this set of copies *can consume considerable memory space*. Since iteration occurs within a method, repeated method calls and extra memory assignment are avoided.



Software Engineering Observation 18.1

Any problem that can be solved recursively can also be solved iteratively (nonrecursively). A recursive approach is normally preferred over an iterative approach when the recursive approach more naturally mirrors the problem and results in a program that is easier to understand and debug. A recursive approach can often be implemented with fewer lines of code. Another reason to choose a recursive approach is that an iterative one might not be apparent.



Performance Tip 18.2

Avoid using recursion in situations requiring high performance. Recursive calls take time and consume additional memory.



Common Programming Error 18.2

Accidentally having a nonrecursive method call itself either directly or indirectly through another method can cause infinite recursion.

18.7 Towers of Hanoi

Earlier in this chapter we studied methods that can be easily implemented both recursively and iteratively. Now, we present a problem whose recursive solution demonstrates the elegance of recursion, and whose iterative solution may not be as apparent.

The **Towers of Hanoi** is one of the classic problems every budding computer scientist must grapple with. Legend has it that in a temple in the Far East, priests are attempting to

move a stack of golden disks from one diamond peg to another (Fig. 18.10). The initial stack has 64 disks threaded onto one peg and arranged from bottom to top by decreasing size. The priests are attempting to move the stack from one peg to another under the constraints that exactly one disk is moved at a time and at no time may a larger disk be placed above a smaller disk. Three pegs are provided, one being used for temporarily holding disks. Supposedly, the world will end when the priests complete their task, so there's little incentive for us to facilitate their efforts.

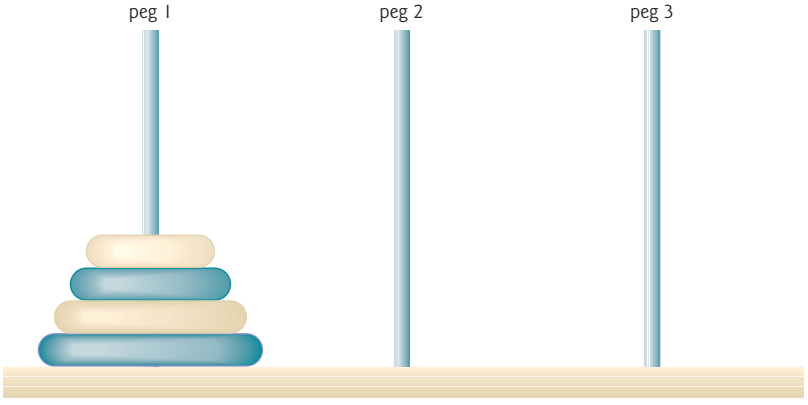


Fig. 18.10 | Towers of Hanoi for the case with four disks.

Let's assume that the priests are attempting to move the disks from peg 1 to peg 3. We wish to develop an algorithm that prints the precise sequence of peg-to-peg disk transfers.

If we try to find an iterative solution, we'll likely find ourselves hopelessly "knotted up" in managing the disks. Instead, attacking this problem recursively quickly yields a solution. Moving n disks can be viewed in terms of moving only $n - 1$ disks (hence the recursion) as follows:

1. Move $n - 1$ disks from peg 1 to peg 2, using peg 3 as a temporary holding area.
2. Move the last disk (the largest) from peg 1 to peg 3.
3. Move $n - 1$ disks from peg 2 to peg 3, using peg 1 as a temporary holding area.

The process ends when the last task involves moving $n = 1$ disk (i.e., the base case). This task is accomplished by moving the disk, without using a temporary holding area.

In Fig. 18.11, method `solveTowers` (lines 6–25) solves the Towers of Hanoi, given the total number of disks (in this case `3`), the starting peg, the ending peg, and the temporary holding peg as parameters. The base case (lines 10–14) occurs when only one disk needs to be moved from the starting peg to the ending peg. In the recursion step (lines 18–24), line 18 moves `disks - 1` disks from the first peg (`sourcePeg`) to the temporary holding peg (`tempPeg`). When all but one of the disks have been moved to the temporary peg, line 21 moves the largest disk from the start peg to the destination peg. Line 24 finishes the rest of the moves by calling the method `solveTowers` to recursively move `disks - 1` disks from the temporary peg (`tempPeg`) to the destination peg (`destinationPeg`), this time using the first peg (`sourcePeg`) as the temporary peg. Line 35 in `main` calls the recursive `solveTowers` method, which outputs the steps to the command prompt.

```
1  // Fig. 18.11: TowersOfHanoi.java
2  // Towers of Hanoi solution with a recursive method.
3  public class TowersOfHanoi
4  {
5     // recursively move disks between towers
6     public static void solveTowers( int disks, int sourcePeg,
7        int destinationPeg, int tempPeg )
8     {
9        // base case -- only one disk to move
10       if ( disks == 1 )
11       {
12          System.out.printf( "\n%d --> %d", sourcePeg, destinationPeg );
13          return;
14       } // end if
15
16       // recursion step -- move (disk - 1) disks from sourcePeg
17       // to tempPeg using destinationPeg
18       solveTowers( disks - 1, sourcePeg, tempPeg, destinationPeg );
19
20       // move last disk from sourcePeg to destinationPeg
21       System.out.printf( "\n%d --> %d", sourcePeg, destinationPeg );
22
23       // move ( disks - 1 ) disks from tempPeg to destinationPeg
24       solveTowers( disks - 1, tempPeg, destinationPeg, sourcePeg );
25    } // end method solveTowers
26
27    public static void main( String[] args )
28    {
29       int startPeg = 1; // value 1 used to indicate startPeg in output
30       int endPeg = 3; // value 3 used to indicate endPeg in output
31       int tempPeg = 2; // value 2 used to indicate tempPeg in output
32       int totalDisks = 3; // number of disks
33
34       // initial nonrecursive call: move all disks.
35       towersOfHanoi.solveTowers( totalDisks, startPeg, endPeg, tempPeg );
36    } // end main
37 } // end class TowersOfHanoi
```

```
1 --> 3
1 --> 2
3 --> 2
1 --> 3
2 --> 1
2 --> 3
1 --> 3
```

Fig. 18.11 | Towers of Hanoi solution with a recursive method.

18.8 Fractals

A **fractal** is a geometric figure that can be generated from a pattern repeated recursively (Fig. 18.12). The figure is modified by applying the pattern to each segment of the original figure. We'll look at a few such approximations in this section. [*Note:* We'll refer to our geometric figures as fractals, even though they're approximations.] Although these figures

had been studied before the 20th century, it was the mathematician Benoit Mandelbrot who in the 1970s introduced the term "fractal," along with the specifics of how a fractal is created and the practical applications of fractals. Mandelbrot's fractal geometry provides mathematical models for many complex forms found in nature, such as mountains, clouds and coastlines. Fractals have many uses in mathematics and science. They can be used to better understand systems or patterns that appear in nature (e.g., ecosystems), in the human body (e.g., in the folds of the brain), or in the universe (e.g., galaxy clusters). Not all fractals resemble objects in nature. Drawing fractals has become a popular art form. Fractals have a **self-similar property**—when subdivided into parts, each resembles a reduced-size copy of the whole. Many fractals yield an exact copy of the original when a portion of the fractal is magnified—such a fractal is said to be **strictly self-similar**. See our Recursion Resource Center (`www.deitel.com/recursion/`) for websites that demonstrate fractals.

As an example, let's look at the strictly self-similar **Koch Curve** fractal (Fig. 18.12). It's formed by removing the middle third of each line in the drawing and replacing it with two lines that form a point, such that if the middle third of the original line remained, an equilateral triangle would be formed. Formulas for creating fractals often involve removing all or part of the previous fractal image. This pattern has already been determined for this fractal—we focus here on how to use those formulas in a recursive solution.

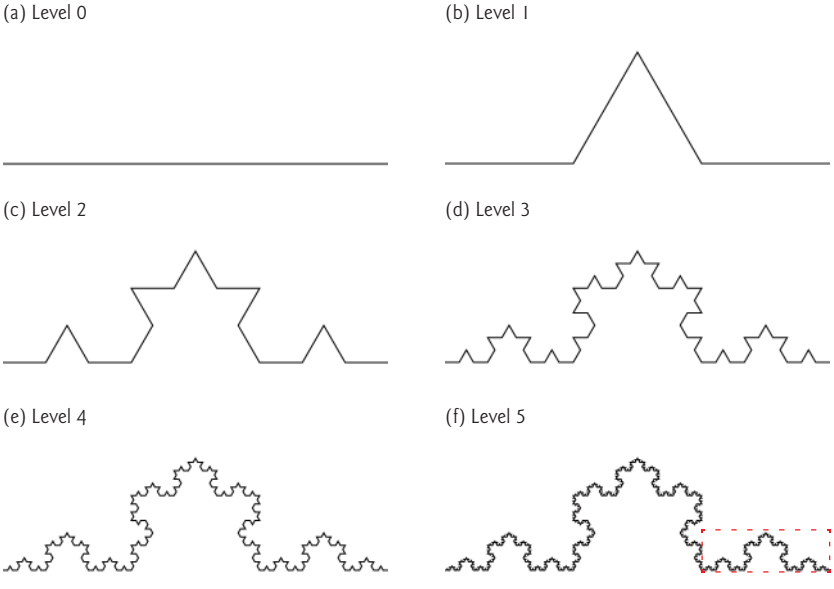


Fig. 18.12 | Koch Curve fractal.

We start with a straight line (Fig. 18.12(a)) and apply the pattern, creating a triangle from the middle third (Fig. 18.12(b)). We then apply the pattern again to each straight line, resulting in Fig. 18.12(c). Each time the pattern is applied, we say that the fractal is at a new **level**, or **depth** (sometimes the term **order** is also used). Fractals can be displayed at many levels—for instance, a fractal at level 3 has had three iterations of the pattern applied (Fig. 18.12(d)). After only a few iterations, this fractal begins to look like a portion

of a snowflake (Fig. 18.12(e and f)). Since this is a strictly self-similar fractal, each portion of it contains an exact copy of the fractal. In Fig. 18.12(f), for example, we've highlighted a portion of the fractal with a dashed red box. If the image in this box were increased in size, it would look exactly like the entire fractal of part (f).

A similar fractal, the **Koch Snowflake**, is the same as the Koch Curve but begins with a triangle rather than a line. The same pattern is applied to each side of the triangle, resulting in an image that looks like an enclosed snowflake. For simplicity, we've chosen to focus on the Koch Curve. To learn more about the Koch Curve and Koch Snowflake, see the links in our Recursion Resource Center (`www.deitel.com/recursion/`).

The "Lo Fractal"

We now demonstrate the use of recursion to draw fractals by writing a program to create a strictly self-similar fractal. We call this the "Lo fractal," named for Sin Han Lo, a Deitel & Associates colleague who created it. The fractal will eventually resemble one-half of a feather (see the outputs in Fig. 18.19). The base case, or fractal level of 0, begins as a line between two points, `A` and `B` (Fig. 18.13). To create the next higher level, we find the midpoint (`C`) of the line. To calculate the location of point `C`, use the following formula:

```
xC = (xA + xB) / 2;
yC = (yA + yB) / 2;
```

[*Note:* The `x` and `y` to the left of each letter refer to the *x*-coordinate and *y*-coordinate of that point, respectively. For instance, `xA` refers to the *x*-coordinate of point `A`, while `yC` refers to the *y*-coordinate of point `C`. In our diagrams we denote the point by its letter, followed by two numbers representing the *x*- and *y*-coordinates.]

To create this fractal, we also must find a point `D` that lies left of segment `AC` and creates an isosceles right triangle `ADC`. To calculate point `D`'s location, use the following formulas:

```
xD = xA + (xC - xA) / 2 - (yC - yA) / 2;
yD = yA + (yC - yA) / 2 + (xC - xA) / 2;
```

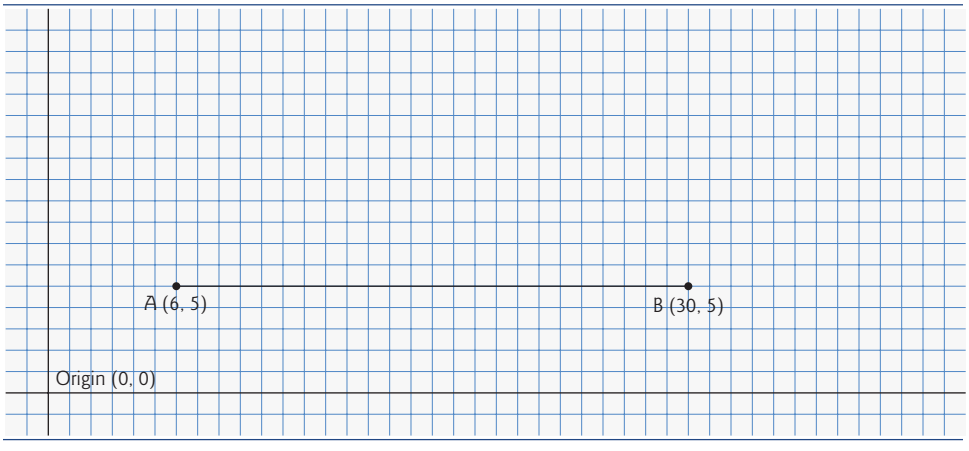


Fig. 18.13 | "Lo fractal" at level 0.

We now move from level 0 to level 1 as follows: First, add points `C` and `D` (as in Fig. 18.14). Then, remove the original line and add segments `DA`, `DC` and `DB`. The

remaining lines will curve at an angle, causing our fractal to look like a feather. For the next level of the fractal, this algorithm is repeated on each of the three lines in level 1. For each line, the formulas above are applied, where the former point `D` is now considered to be point `A`, while the other end of each line is considered to be point `B`. Figure 18.15 contains the line from level 0 (now a dashed line) and the three added lines from level 1. We've changed point `D` to be point `A`, and the original points `A`, `C` and `B` to `B1`, `B2` and `B3`, respectively. The preceding formulas have been used to find the new points `C` and `D` on each line. These points are also numbered 1–3 to keep track of which point is associated with each line. The points `C1` and `D1`, for instance, represent points `C` and `D` associated with the line formed from points `A` to `B1`. To achieve level 2, the three lines in Fig. 18.15 are removed and replaced with new lines from the `C` and `D` points just added. Figure 18.16 shows the new lines (the lines from level 2 are shown as dashed lines for your convenience). Figure 18.17 shows level 2 without the dashed lines from level 1. Once this process has been repeated several times, the fractal created will begin to look like one-half of a feather, as shown in the output of Fig. 18.19. We'll present the code for this application shortly.

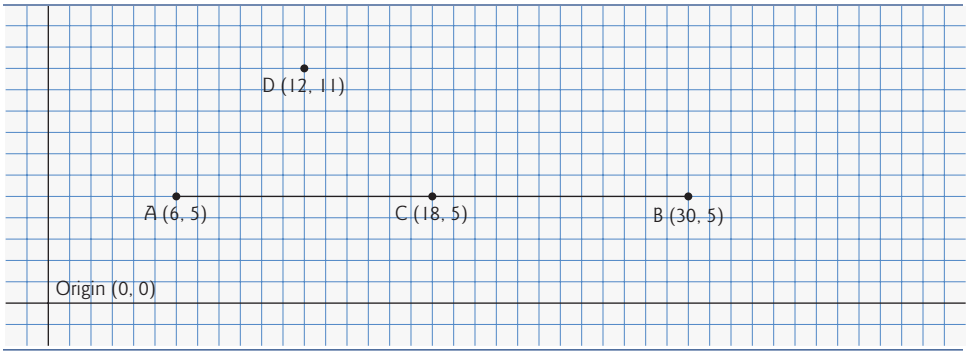


Fig. 18.14 | Determining points C and D for level 1 of the "Lo fractal."

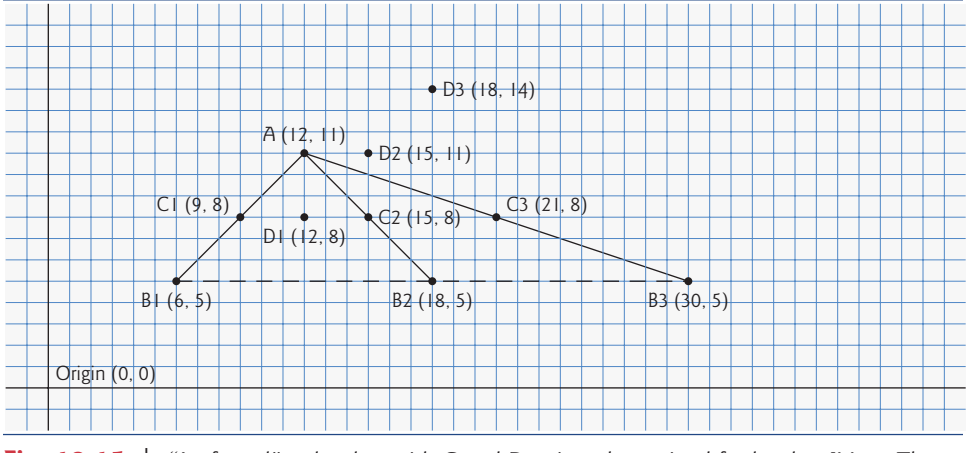


Fig. 18.15 | "Lo fractal" at level 1, with C and D points determined for level 2. [*Note:* The fractal at level 0 is included as a dashed line as a reminder of where the line was located in relation to the current fractal.]

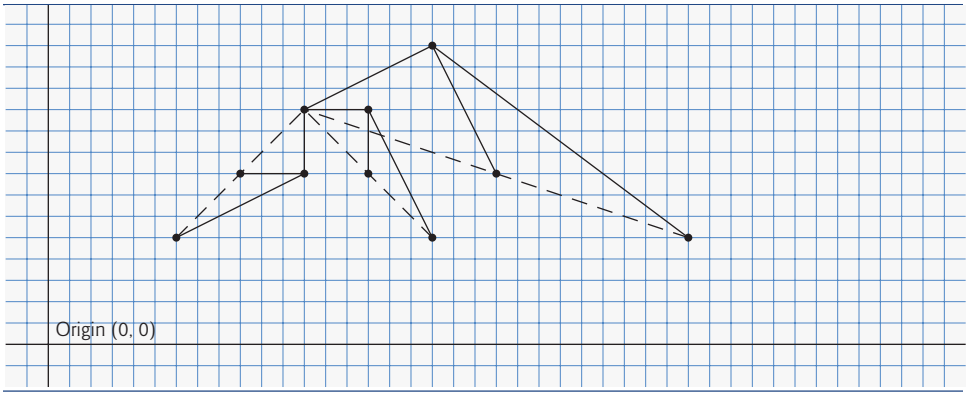


Fig. 18.16 | "Lo fractal" at level 2, with dashed lines from level 1 provided.

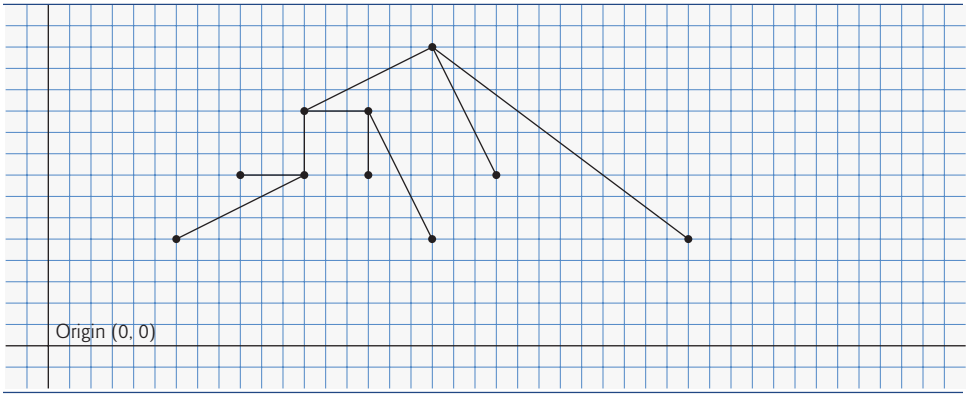


Fig. 18.17 | "Lo fractal" at level 2.

The application in Fig. 18.18 defines the user interface for drawing this fractal (shown at the end of Fig. 18.19). The interface consists of three buttons—one for the user to change the color of the fractal, one to increase the level of recursion and one to decrease the level of recursion. A `JLabel` keeps track of the current level of recursion, which is modified by calling method `setLevel`, to be discussed shortly. Lines 15–16 specify constants `WIDTH` and `HEIGHT` to be `400` and `480`, respectively, for the size of the `JFrame`. The user triggers an `ActionEvent` by clicking the **Color** button. The event handler for this button is registered in lines 37–53. The method `actionPerformed` displays a `JColorChooser`. This dialog returns the selected `Color` object or blue (if the user presses **Cancel** or closes the dialog without pressing **OK**). Line 50 calls the `setColor` method in class `FractalJPanel` to update the color.

```
1  // Fig. 18.18: Fractal.java
2  // Fractal user interface.
3  import java.awt.Color;
4  import java.awt.FlowLayout;
5  import java.awt.event.ActionEvent;
```

Fig. 18.18 | Fractal user interface. (Part 1 of 4.)

```
6  import java.awt.event.ActionListener;
7  import javax.swing.JFrame;
8  import javax.swing.JButton;
9  import javax.swing.JLabel;
10 import javax.swing.JPanel;
11 import javax.swing.JColorChooser;
12
13 public class Fractal extends JFrame
14 {
15    private static final int WIDTH = 400;  // define width of GUI
16    private static final int HEIGHT = 480; // define height of GUI
17    private static final int MIN_LEVEL = 0, MAX_LEVEL = 15;
18
19    private JButton changeColorJButton, increaseLevelJButton,
20       decreaseLevelJButton;
21    private JLabel levelJLabel;
22    private FractalJPanel drawSpace;
23    private JPanel mainJPanel, controlJPanel;
24
25    // set up GUI
26    public Fractal()
27    {
28       super( "Fractal" );
29
30       // set up control panel
31       controlJPanel = new JPanel();
32       controlJPanel.setLayout( new FlowLayout() );
33
34       // set up color button and register listener
35       changeColorJButton = new JButton( "Color" );
36       controlJPanel.add( changeColorJButton );
37       changeColorJButton.addActionListener(
38          new ActionListener() // anonymous inner class
39          {
40             // process changeColorJButton event
41             public void actionPerformed( ActionEvent event )
42             {
43                Color color = JColorChooser.showDialog(
44                   Fractal.this, "Choose a color", Color.BLUE );
45
46                // set default color, if no color is returned
47                if ( color == null )
48                   color = Color.BLUE;
49
50                drawSpace.setColor( color );
51             } // end method actionPerformed
52          }  // end anonymous inner class
53       ); // end addActionListener
54
55       // set up decrease level button to add to control panel and
56       // register listener
57       decreaseLevelJButton = new JButton( "Decrease Level" );
```

Fig. 18.18 | Fractal user interface. (Part 2 of 4.)

```
58          controlJPanel.add( decreaseLevelJButton );
59          decreaseLevelJButton.addActionListener(
60             new ActionListener() // anonymous inner class
61             {
62                // process decreaseLevelJButton event
63                public void actionPerformed( ActionEvent event )
64                {
65                   int level = drawSpace.getLevel();
66                   --level; // decrease level by one
67
68                   // modify level if possible
69                   if ( ( level >= MIN_LEVEL ) ) &&
70                      ( level <= MAX_LEVEL ) )
71                   {
72                      levelJLabel.setText( "Level: " + level );
73                      drawSpace.setLevel( level );
74                      repaint();
75                   } // end if
76                } // end method actionPerformed
77             } // end anonymous inner class
78          ); // end addActionListener
79
80          // set up increase level button to add to control panel
81          // and register listener
82          increaseLevelJButton = new JButton( "Increase Level" );
83          controlJPanel.add( increaseLevelJButton );
84          increaseLevelJButton.addActionListener(
85             new ActionListener() // anonymous inner class
86             {
87                // process increaseLevelJButton event
88                public void actionPerformed( ActionEvent event )
89                {
90                   int level = drawSpace.getLevel();
91                   ++level; // increase level by one
92
93                   // modify level if possible
94                   if ( ( level >= MIN_LEVEL ) ) &&
95                      ( level <= MAX_LEVEL ) )
96                   {
97                      levelJLabel.setText( "Level: " + level );
98                      drawSpace.setLevel( level );
99                      repaint();
100                  } // end if
101               } // end method actionPerformed
102            } // end anonymous inner class
103         ); // end addActionListener
104
105         // set up levelJLabel to add to controlJPanel
106         levelJLabel = new JLabel( "Level: 0" );
107         controlJPanel.add( levelJLabel );
108
109         drawSpace = new FractalJPanel( 0 );
110
```

Fig. 18.18 | Fractal user interface. (Part 3 of 4.)

```
111         // create mainJPanel to contain controlJPanel and drawSpace
112         mainJPanel = new JPanel();
113         mainJPanel.add( controlJPanel );
114         mainJPanel.add( drawSpace );
115
116         add( mainJPanel ); // add JPanel to JFrame
117
118         setSize( WIDTH, HEIGHT ); // set size of JFrame
119         setVisible( true ); // display JFrame
120      } // end Fractal constructor
121
122      public static void main( String[] args )
123      {
124         Fractal demo = new Fractal();
125         demo.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
126      } // end main
127   } // end class Fractal
```

Fig. 18.18 | Fractal user interface. (Part 4 of 4.)

The **Decrease Level** button's event handler is registered in lines 59–78. Method `actionPerformed` retrieves the current level of recursion and decrements it by 1 (lines 65–66). Lines 69–70 ensure that the level is greater than or equal to `MIN_LEVEL` and less than or equal to `MAX_LEVEL`—the fractal is not defined for recursion levels less than `MIN_LEVEL` and you can't see the additional detail above `MAX_LEVEL`. You can go up to any desired level, but around level 10 and higher, the fractal rendering becomes slow due to the details to be drawn. Lines 72–74 reset the level label to reflect the change—the new level is set and the `repaint` method is called to update the image to show the fractal at the new level.

The **Increase Level** `JButton` works like the **Decrease Level** `JButton`, but the level is incremented rather than decremented to show more details of the fractal (lines 90–91). When the application is first executed, the level will be set to 0, which will display a blue line between two points that were specified in the `FractalJPanel` class.

Class `FractalJPanel` (Fig. 18.19) specifies the drawing `JPanel`'s dimensions as `400` by `400` (lines 13–14). The `FractalJPanel` constructor (lines 18–24) takes the current level as a parameter and assigns it to its instance variable `level`. Instance variable `color` is set to blue by default. Lines 22–23 change the `JPanel`'s background color to white (so the fractal is easy to see), and set the drawing `FractalJPanel`'s dimensions.

```
1  // Fig. 18.19: FractalJPanel.java
2  // Drawing the "Lo fractal" using recursion.
3  import java.awt.Graphics;
4  import java.awt.Color;
5  import java.awt.Dimension;
6  import javax.swing.JPanel;
7
8  public class FractalJPanel extends JPanel
9  {
10    private Color color; // stores color used to draw fractal
11    private int level;   // stores current level of fractal
```

Fig. 18.19 | Drawing the "Lo fractal" using recursion. (Part 1 of 4.)

```
12
13      private static final int WIDTH = 400;  // defines width of JPanel
14      private static final int HEIGHT = 400; // defines height of JPanel
15
16      // set the initial fractal level to the value specified
17      // and set up JPanel specifications
18      public FractalJPanel( int currentLevel )
19      {
20         color = Color.BLUE;  // initialize drawing color to blue
21         level = currentLevel; // set initial fractal level
22         setBackground( Color.WHITE );
23         setPreferredSize( new Dimension( WIDTH, HEIGHT ) );
24      } // end FractalJPanel constructor
25
26      // draw fractal recursively
27      public void drawFractal( int level, int xA, int yA, int xB,
28         int yB, Graphics g )
29      {
30         // base case: draw a line connecting two given points
31         if ( level == 0 )
32            g.drawLine( xA, yA, xB, yB );
33         else // recursion step: determine new points, draw next level
34         {
35            // calculate midpoint between (xA, yA) and (xB, yB)
36            int xC = ( xA + xB ) / 2;
37            int yC = ( yA + yB ) / 2;
38
39            // calculate the fourth point (xD, yD) which forms an
40            // isosceles right triangle between (xA, yA) and (xC, yC)
41            // where the right angle is at (xD, yD)
42            int xD = xA + ( xC - xA ) / 2 - ( yC - yA ) / 2;
43            int yD = yA + ( yC - yA ) / 2 + ( xC - xA ) / 2;
44
45            // recursively draw the Fractal
46            drawFractal( level - 1, xD, yD, xA, yA, g );
47            drawFractal( level - 1, xD, yD, xC, yC, g );
48            drawFractal( level - 1, xD, yD, xB, yB, g );
49         } // end else
50      } // end method drawFractal
51
52      // start drawing the fractal
53      public void paintComponent( Graphics g )
54      {
55         super.paintComponent( g );
56
57         // draw fractal pattern
58         g.setColor( color );
59         drawFractal( level, 100, 90, 290, 200, g );
60      } // end method paintComponent
61
62      // set the drawing color to c
63      public void setColor( Color c )
64      {
```

Fig. 18.19 | Drawing the "Lo fractal" using recursion. (Part 2 of 4.)

```
65          color = c;
66       } // end method setColor
67
68       // set the new level of recursion
69       public void setLevel( int currentLevel )
70       {
71          level = currentLevel;
72       } // end method setLevel
73
74       // returns level of recursion
75       public int getLevel()
76       {
77          return level;
78       } // end method getLevel
79    } // end class FractalJPanel
```

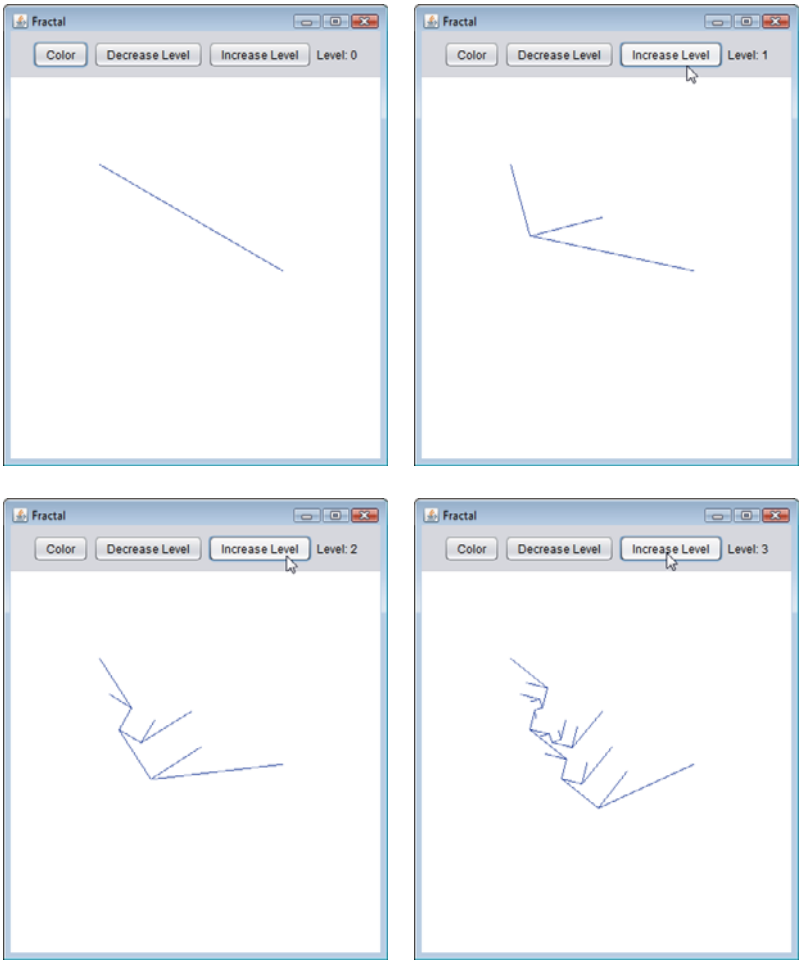


Fig. 18.19 | Drawing the "Lo fractal" using recursion. (Part 3 of 4.)

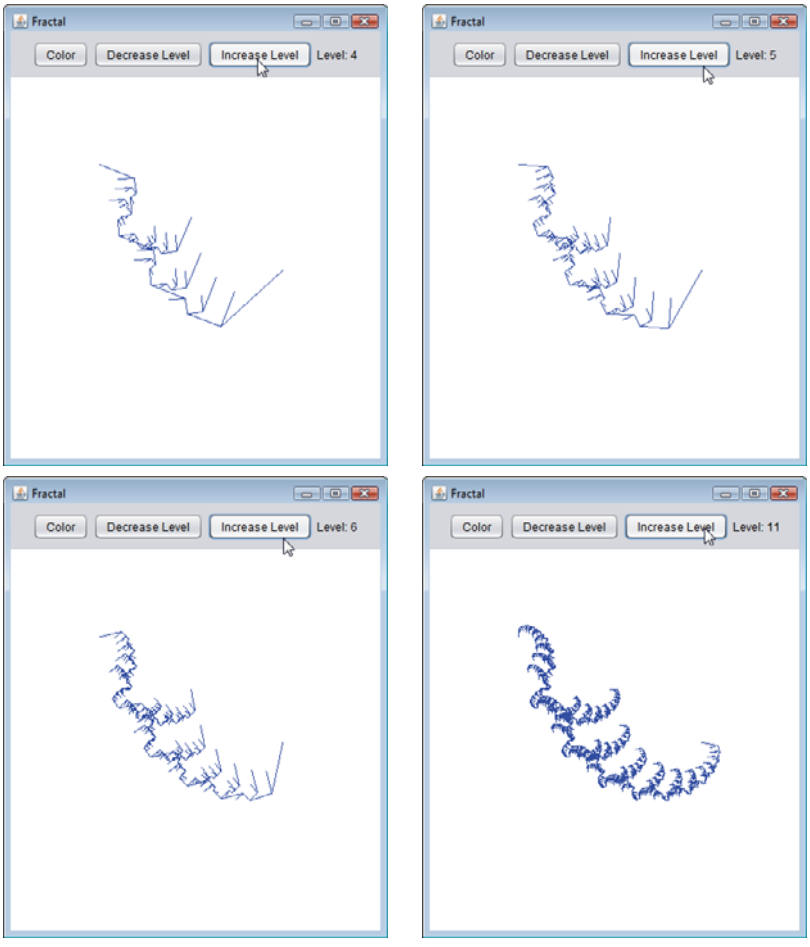


Fig. 18.19 | Drawing the "Lo fractal" using recursion. (Part 4 of 4.)

Lines 27–50 define the recursive method that creates the fractal. This method takes six parameters: the level, four integers that specify the *x*- and *y*-coordinates of two points, and the `Graphics` object `g`. The base case for this method (line 31) occurs when `level` equals 0, at which time a line will be drawn between the two points given as parameters. Lines 36–43 calculate (`xC`, `yC`), the midpoint between (`xA`, `yA`) and (`xB`, `yB`), and (`xD`, `yD`), the point that creates a right isosceles triangle with (`xA`, `yA`) and (`xC`, `yC`). Lines 46–48 make three recursive calls on three different sets of points.

In method `paintComponent`, line 59 makes the first call to method `drawFractal` to start the drawing. This method call is not recursive, but all subsequent calls to `drawFractal` performed from the body of `drawFractal` are. Since the lines will not be drawn until the base case is reached, the distance between two points decreases on each recursive call. As the level of recursion increases, the fractal becomes smoother and more detailed. The shape of this fractal stabilizes as the level approaches 11. Fractals will stabilize at different levels based on their shape and size.

Figure 18.19 shows the development of the fractal from levels 0 to 6. The last image shows the defining shape of the fractal at level 11. If we focus on one of the arms of this fractal, it will be identical to the whole image. This property defines the fractal to be strictly self-similar. See our Recursion Resource Center at `www.deitel.com/recursion/` for further resources on fractals.

18.9 Recursive Backtracking

Our recursive methods all have a similar architecture—if the base case is reached, return a result; if not, make one or more recursive calls. This section explores a more complex recursive technique that finds a path through a maze, returning true if there's a possible solution. The solution involves moving through the maze one step at a time, where moves can be made by going down, right, up or left (diagonal moves are not permitted). From the current location in the maze (starting with the entry point), the following steps are taken: For each possible direction, the move is made in that direction and a recursive call is made to solve the remainder of the maze from the new location. When a dead end is reached (i.e., we cannot take any more steps forward without hitting a wall), we back up to the previous location and try to go in a different direction. If no other direction can be taken, we back up again. This process continues until we find a point in the maze where a move *can* be made in another direction. Once such a location is found, we move in the new direction and continue with another recursive call to solve the rest of the maze.

To back up to the previous location in the maze, our recursive method simply returns false, moving up the method-call chain to the previous recursive call (which references the previous location in the maze). Using recursion to return to an earlier decision point is known as **recursive backtracking**. If one set of recursive calls does not result in a solution to the problem, the program backs up to the previous decision point and makes a different decision, often resulting in another set of recursive calls. In this example, the previous decision point is the previous location in the maze, and the decision to be made is the direction that the next move should take. One direction has led to a dead end, so the search continues with a different direction. The recursive backtracking solution to the maze problem uses recursion to return only partway up the method-call chain, then try a different direction. If the backtracking reaches the entry location of the maze and the paths in all directions have been attempted, the maze does not have a solution.

The exercises ask you to implement recursive backtracking solutions to the maze problem (Exercises 18.20–18.22) and the Eight Queens problem (Exercise 18.15), which attempts to find a way to place eight queens on an empty chessboard so that no queen is "attacking" any other (i.e., no two queens are in the same row, in the same column or along the same diagonal).

18.10 Wrap-Up

In this chapter, you learned how to create recursive methods—i.e., methods that call themselves. You learned that recursive methods typically divide a problem into two conceptual pieces—a piece that the method knows how to do (the base case) and a piece that the method does not know how to do (the recursion step). The recursion step is a slightly smaller version of the original problem and is performed by a recursive method call. You saw some popular recursion examples, including calculating factorials and producing val-

ues in the Fibonacci series. You then learned how recursion works "under the hood," including the order in which recursive method calls are pushed on or popped off the program-execution stack. Next, you compared recursive and iterative (nonrecursive) approaches. You learned how to use recursion to solve more complex problems—the Towers of Hanoi and displaying fractals. The chapter concluded with an introduction to recursive backtracking, a technique for solving problems that involves backing up through recursive calls to try different possible solutions. In the next chapter, you'll learn numerous techniques for sorting lists of data and searching for an item in a list of data, and you'll explore the circumstances under which each searching and sorting technique should be used.

Summary

Section 18.1 Introduction

- A recursive method (p. 766) calls itself directly or indirectly through another method.

Section 18.2 Recursion Concepts

- When a recursive method is called (p. 767) to solve a problem, it can solve only the simplest case(s), or base case(s). If called with a base case (p. 767), the method returns a result.
- If a recursive method is called with a more complex problem than a base case, it typically divides the problem into two conceptual pieces—a piece that the method knows how to do and a piece that it does not know how to do.
- To make recursion feasible, the piece that the method does not know how to do must resemble the original problem, but be a slightly simpler or smaller version of it. Because this new problem looks like the original problem, the method calls a fresh copy of itself to work on the smaller problem—this is called the recursion step (p. 767).
- For recursion to eventually terminate, each time a method calls itself with a simpler version of the original problem, the sequence of smaller and smaller problems must converge on a base case. When, the method recognizes the base case, it returns a result to the previous copy of the method.
- A recursive call may be a call to another method, which in turn makes a call back to the original method. Such a process still results in a recursive call to the original method. This is known as an indirect recursive call or indirect recursion (p. 767).

Section 18.3 Example Using Recursion: Factorials

- Either omitting the base case or writing the recursion step incorrectly so that it does not converge on the base case can cause infinite recursion (p. 769), eventually exhausting memory. This error is analogous to the problem of an infinite loop in an iterative (nonrecursive) solution.

Section 18.4 Example Using Recursion: Fibonacci Series

- The Fibonacci series (p. 771) begins with 0 and 1 and has the property that each subsequent Fibonacci number is the sum of the preceding two.
- The ratio of successive Fibonacci numbers converges on a constant value of 1.618…, a number that has been called the golden ratio or the golden mean (p. 771).
- Some recursive solutions, such as Fibonacci, result in an "explosion" of method calls.

Section 18.5 Recursion and the Method-Call Stack

- The executing method is always the one whose activation record is at the top of the stack, and the activation record for that method contains the values of its local variables.

Section 18.6 Recursion vs. Iteration

- Both iteration and recursion are based on a control statement: Iteration uses a repetition statement, recursion a selection statement.
- Both iteration and recursion involve repetition: Iteration explicitly uses a repetition statement, whereas recursion achieves repetition through repeated method calls.
- Iteration and recursion each involve a termination test: Iteration terminates when the loop-continuation condition fails, recursion when a base case is recognized.
- Iteration with counter-controlled repetition and recursion each gradually approach termination: Iteration keeps modifying a counter until the counter assumes a value that makes the loop-continuation condition fail, whereas recursion keeps producing simpler versions of the original problem until the base case is reached.
- Both iteration and recursion can occur infinitely: An infinite loop occurs with iteration if the loop-continuation test never becomes false, whereas infinite recursion occurs if the recursion step does not reduce the problem each time in a manner that converges on the base case.
- Recursion repeatedly invokes the mechanism, and consequently the overhead, of method calls.
- Any problem that can be solved recursively can also be solved iteratively.
- A recursive approach is normally preferred over an iterative approach when it more naturally mirrors the problem and results in a program that is easier to understand and debug.
- A recursive approach can often be implemented with few lines of code, but a corresponding iterative approach might take a large amount of code. Another reason to choose a recursive solution is that an iterative solution might not be apparent.

Section 18.8 Fractals

- A fractal (p. 779) is a geometric figure that is generated from a pattern repeated recursively an infinite number of times.
- Fractals have a self-similar property (p. 780)—subparts are reduced-size copies of the whole.

Section 18.9 Recursive Backtracking

- In recursive backtracking (p. 790), if one set of recursive calls does not result in a solution to the problem, the program backs up to the previous decision point and makes a different decision, often resulting in another set of recursive calls.

Self-Review Exercises

18.1 State whether each of the following is *true* or *false*. If *false*, explain why.

a) A method that calls itself indirectly is not an example of recursion.
b) Recursion can be efficient in computation because of reduced memory-space usage.
c) When a recursive method is called to solve a problem, it actually is capable of solving only the simplest case(s), or base case(s).
d) To make recursion feasible, the recursion step in a recursive solution must resemble the original problem, but be a slightly larger version of it.

18.2 A _____ is needed to terminate recursion.

a) recursion step
b) `break` statement
c) `void` return type
d) base case

18.3 The first call to invoke a recursive method is _____.

a) not recursive

b) recursive
c) the recursion step
d) none of the above

18.4 Each time a fractal's pattern is applied, the fractal is said to be at a new _____.
a) width
b) height
c) level
d) volume

18.5 Iteration and recursion each involve a _____.
a) repetition statement
b) termination test
c) counter variable
d) none of the above

18.6 Fill in the blanks in each of the following statements:
a) The ratio of successive Fibonacci numbers converges on a constant value of 1.618…, a number that has been called the _____ or the _____.
b) Iteration normally uses a repetition statement, whereas recursion normally uses a(n) _____ statement.
c) Fractals have a(n) _____ property—when subdivided into parts, each is a reduced-size copy of the whole.

Answers to Self-Review Exercises

18.1 a) False. A method that calls itself in this manner is an example of indirect recursion. b) False. Recursion can be inefficient in computation because of multiple method calls and memory-space usage. c) True. d) False. To make recursion feasible, the recursion step in a recursive solution must resemble the original problem, but be a slightly *smaller* version of it.

18.2 d

18.3 a

18.4 c

18.5 b

18.6 a) golden ratio, golden mean. b) selection. c) self-similar.

Exercises

18.7 What does the following code do?

```
1  public int mystery( int a, int b )
2  {
3     if ( b == 1 )
4        return a;
5     else
6        return a + mystery( a, b - 1 );
7  } // end method mystery
```

18.8 Find the error(s) in the following recursive method, and explain how to correct it (them). This method should find the sum of the values from 0 to `n`.

```
1  public int sum( int n )
2  {
3     if ( n == 0 )
4        return 0;
5     else
6        return n + sum( n );
7  } // end method sum
```

18.9 *(Recursive `power` Method)* Write a recursive method `power(base, exponent)` that, when called, returns

$$base^{exponent}$$

For example, `power( 3,4 )` = 3 * 3 * 3 * 3. Assume that `exponent` is an integer greater than or equal to 1. [*Hint:* The recursion step should use the relationship

$$base^{exponent} = base \cdot base^{exponent - 1}$$

and the terminating condition occurs when `exponent` is equal to `1`, because

$$base^{1} = base$$

Incorporate this method into a program that enables the user to enter the `base` and `exponent`.]

18.10 *(Visualizing Recursion)* It's interesting to watch recursion "in action." Modify the factorial method in Fig. 18.3 to print its local variable and recursive-call parameter. For each recursive call, display the outputs on a separate line and add a level of indentation. Do your utmost to make the outputs clear, interesting and meaningful. Your goal here is to design and implement an output format that makes it easier to understand recursion. You may want to add such display capabilities to other recursion examples and exercises throughout the text.

18.11 *(Greatest Common Divisor)* The greatest common divisor of integers `x` and `y` is the largest integer that evenly divides into both `x` and `y`. Write a recursive method `gcd` that returns the greatest common divisor of `x` and `y`. The `gcd` of `x` and `y` is defined recursively as follows: If `y` is equal to `0`, then `gcd( x, y )` is `x`; otherwise, `gcd( x, y )` is `gcd( y, x % y )`, where `%` is the remainder operator. Use this method to replace the one you wrote in the application of Exercise 6.27.

18.12 What does the following program do?

```
1  // Exercise 18.12 Solution: MysteryClass.java
2  public class MysteryClass
3  {
4     public static int mystery( int[] array2, int size )
5     {
6        if ( size == 1 )
7           return array2[ 0 ];
8        else
9           return array2[ size - 1 ] + mystery( array2, size - 1 );
10    } // end method mystery
11
12    public static void main( String[] args )
13    {
14       int[] array = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
15
16       int result = mystery( array, array.length );
17       System.out.printf( "Result is: %d\n", result );
18    } // end method main
19 } // end class MysteryClass
```

18.13 What does the following program do?

```
1  // Exercise 18.13 Solution: SomeClass.java
2  public class SomeClass
3  {
4     public static String someMethod( int[] array2, int x )
5     {
6        if ( x < array2.length )
7           return String.format(
8              "%s%d ", someMethod( array2, x + 1 ), array2[ x ] );
9        else
10          return "";
11    } // end method someMethod
12
13    public static void main( String[] args )
14    {
15       int[] array = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
16       String results = someMethod( array, 0 );
17       System.out.println( results );
18    } // end main
19 } // end class SomeClass
```

18.14 *(Palindromes)* A palindrome is a string that is spelled the same way forward and backward. Some examples of palindromes are "radar," "able was i ere i saw elba" and (if spaces are ignored) "a man a plan a canal panama." Write a recursive method `testPalindrome` that returns `boolean` value `true` if the string stored in the array is a palindrome and `false` otherwise. The method should ignore spaces and punctuation in the string.

18.15 *(Eight Queens)* A puzzler for chess buffs is the Eight Queens problem, which asks: Is it possible to place eight queens on an empty chessboard so that no queen is "attacking" any other (i.e., no two queens are in the same row, in the same column or along the same diagonal)? For instance, if a queen is placed in the upper-left corner of the board, no other queens could be placed in any of the marked squares shown in Fig. 18.20. Solve the problem recursively. [*Hint:* Your solution should begin with the first column and look for a location in that column where a queen can be placed—initially, place the queen in the first row. The solution should then recursively search the remaining columns. In the first few columns, there will be several locations where a queen may be placed. Take the first available location. If a column is reached with no possible location for a queen, the program should return to the previous column, and move the queen in that column to a new row. This continuous backing up and trying new alternatives is an example of recursive backtracking.]

18.16 *(Print an Array)* Write a recursive method `printArray` that displays all the elements in an array of integers, separated by spaces.

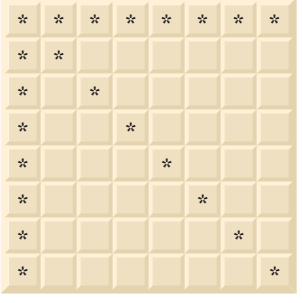


Fig. 18.20 | Squares eliminated by placing a queen in the upper-left corner of a chessboard.

18.17 *(Print an Array Backward)* Write a recursive method `stringReverse` that takes a character array containing a string as an argument and prints the string backward. [*Hint:* Use `String` method `toCharArray`, which takes no arguments, to get a `char` array containing the characters in the `String`.]

18.18 *(Find the Minimum Value in an Array)* Write a recursive method `recursiveMinimum` that determines the smallest element in an array of integers. The method should return when it receives an array of one element.

18.19 *(Fractals)* Repeat the fractal pattern in Section 18.8 to form a star. Begin with five lines (see Fig. 18.21) instead of one, where each line is a different arm of the star. Apply the "Lo fractal" pattern to each arm of the star.

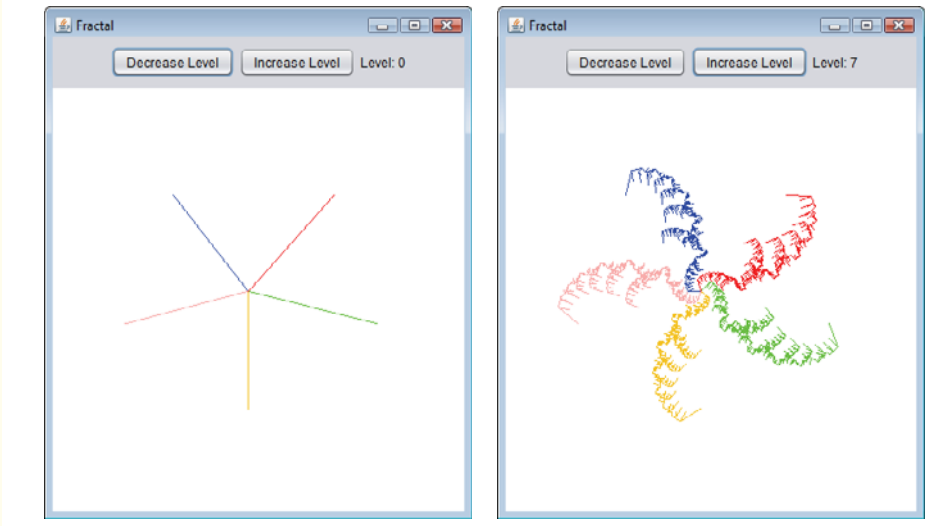


Fig. 18.21 | Sample outputs for Exercise 18.19.

18.20 *(Maze Traversal Using Recursive Backtracking)* The grid of #s and dots (.) in Fig. 18.22 is a two-dimensional array representation of a maze. The #s represent the walls of the maze, and the dots represent locations in the possible paths through the maze. A move can be made only to a location in the array that contains a dot.

```
# # # # # # # # # # # # #
# . . . # . . . . . . . #
. . # . # . # # # # . # .
# # # . # . . . . # . # .
# . . . . # # # . # . . .
# # # # . # . # . # . # .
# . . # . # . # . # . # .
# # . # . # . # . # . # .
# . . . . . . . . # . # .
# # # # # # . # # # . # .
# . . . . . . # . . . # .
# # # # # # # # # # # # #
```

Fig. 18.22 | Two-dimensional array representation of a maze.

Write a recursive method (`mazeTraversal`) to walk through mazes like the one in Fig. 18.22. The method should receive as arguments a 12-by-12 character array representing the maze and the current location in the maze (the first time this method is called, the current location should be the entry point of the maze). As `mazeTraversal` attempts to locate the exit, it should place the character `x` in each square in the path. There's a simple algorithm for walking through a maze that guarantees finding the exit (assuming there's an exit). If there's no exit, you'll arrive at the starting location again. The algorithm is as follows: From the current location in the maze, try to move one space in any of the possible directions (down, right, up or left). If it's possible to move in at least one direction, call `mazeTraversal` recursively, passing the new spot on the maze as the current spot. If it's not possible to go in any direction, "back up" to a previous location in the maze and try a new direction for that location (this is an example of recursive backtracking). Program the method to display the maze after each move so the user can watch as the maze is solved. The final output of the maze should display only the path needed to solve the maze—if going in a particular direction results in a dead end, the `x`'s going in that direction should not be displayed. [*Hint:* To display only the final path, it may be helpful to mark off spots that result in a dead end with another character (such as `'0'`).]

18.21 ***(Generating Mazes Randomly)*** Write a method `mazeGenerator` that takes as an argument a two-dimensional 12-by-12 character array and randomly produces a maze. The method should also provide the starting and ending locations of the maze. Test your method `mazeTraversal` from Exercise 18.20, using several randomly generated mazes.

18.22 ***(Mazes of Any Size)*** Generalize methods `mazeTraversal` and `mazeGenerator` of Exercise 18.20 and Exercise 18.21 to process mazes of any width and height.

18.23 ***(Time to Calculate Fibonacci Numbers)*** Enhance the Fibonacci program of Fig. 18.5 so that it calculates the approximate amount of time required to perform the calculation and the number of calls made to the recursive method. For this purpose, call `static System` method `currentTimeMillis`, which takes no arguments and returns the computer's current time in milliseconds. Call this method twice—once before and once after the call to `fibonacci`. Save each value and calculate the difference in the times to determine how many milliseconds were required to perform the calculation. Then, add a variable to the `FibonacciCalculator` class, and use this variable to determine the number of calls made to method `fibonacci`. Display your results.

19

Searching, Sorting and Big O



With sobs and tears he sorted out
Those of the largest size …
—Lewis Carroll

Attempt the end, and never stand to doubt;
Nothing's so hard, but search will find it out.
—Robert Herrick

'Tis in my memory lock'd,
And you yourself shall keep the key of it.
—William Shakespeare

It is an immutable law in business that words are words, explanations are explanations, promises are promises — but only performance is reality.
—Harold S. Green

Objectives

In this chapter you'll learn:

- To search for a given value in an array using linear search and binary search.
- To sort arrays using the iterative selection and insertion sort algorithms.
- To sort arrays using the recursive merge sort algorithm.
- To determine the efficiency of searching and sorting algorithms.

Outline

19.1 Introduction
19.2 Searching Algorithms
19.2.1 Linear Search
19.2.2 Binary Search
19.3 Sorting Algorithms
19.3.1 Selection Sort
19.3.2 Insertion Sort
19.3.3 Merge Sort
19.4 Wrap-Up

Summary | *Self-Review Exercises* | *Answers to Self-Review Exercises* | *Exercises*

19.1 Introduction

Searching data involves determining whether a value (referred to as the **search key**) is present in the data and, if so, finding its location. Two popular search algorithms are the simple linear search and the faster but more complex binary search. **Sorting** places data in ascending or descending order, based on one or more **sort keys**. A list of names could be sorted alphabetically, bank accounts could be sorted by account number, employee payroll records could be sorted by social security number, and so on. This chapter introduces two simple sorting algorithms, the selection sort and the insertion sort, along with the more efficient but more complex merge sort. The Java Collections API (Chapter 20) provides various built-in searching and sorting capabilities. Figure 19.1 summarizes the searching and sorting algorithms discussed in the examples and exercises of this book.

Chapter	Algorithm	Location
Searching Algorithms:		
19	Linear search	Section 19.2.1
	Binary search	Section 19.2.2
	Recursive linear search	Exercise 19.8
	Recursive binary search	Exercise 19.9
20	`binarySearch` method of class `Collections`	Fig. 20.12
22	Linear search of a `List`	Exercise 22.21
	Binary tree search	Exercise 22.23
Sorting Algorithms:		
19	Selection sort	Section 19.3.1
	Insertion sort	Section 19.3.2
	Recursive merge sort	Section 19.3.3
	Bubble sort	Exercises 19.5 and 19.6
	Bucket sort	Exercise 19.7
	Recursive quicksort	Exercise 19.10
20	`sort` method of class `Collections`	Figs. 20.6–20.9
	`SortedSet` collection	Fig. 20.17
22	Binary tree sort	Section 22.7

Fig. 19.1 | Searching and sorting algorithms covered gin this text.

19.2 Searching Algorithms

Looking up a phone number, finding a website via a search engine and checking the definition of a word in a dictionary all involve searching large amounts of data. The next two sections discuss two common search algorithms—one that is easy to program yet relatively inefficient and one that is relatively efficient but more complex and difficult to program.

19.2.1 Linear Search

The **linear search algorithm** searches each element in an array sequentially. If the search key does not match an element in the array, the algorithm tests each element, and when the end of the array is reached, informs the user that the search key is not present. If the search key is in the array, the algorithm tests each element until it finds one that matches the search key and returns the index of that element.

As an example, consider an array containing the following values

```
34   56   2   10   77   51   93   30   5   52
```

and a program that is searching for 51. Using the linear search algorithm, the program first checks whether 34 matches the search key. It does not, so the algorithm checks whether 56 matches the search key. The program continues moving through the array sequentially, testing 2, then 10, then 77. When the program tests 51, which matches the search key, the program returns the index 5, which is the location of 51 in the array. If, after checking every array element, the program determines that the search key does not match any element in the array, it returns a sentinel value (e.g., `-1`).

Figure 19.2 declares the `LinearArray` class. This class has two `private` instance variables—an array of `int`s named `data`, and a `static` `Random` object to fill the array with randomly generated `int`s. When an object of class `LinearArray` is instantiated, the constructor (lines 13–20) creates and initializes the array `data` with random `int`s in the range 10–99. If there are *duplicate* values in the array, linear search returns the index of the *first* element in the array that matches the search key.

```
1  // Fig. 19.2: LinearArray.java
2  // Class that contains an array of random integers and a method
3  // that will search that array sequentially.
4  import java.util.Random;
5  import java.util.Arrays;
6
7  public class LinearArray
8  {
9     private int[] data; // array of values
10    private static final Random generator = new Random();
11
12    // create array of given size and fill with random numbers
13    public LinearArray( int size )
14    {
15       data = new int[ size ]; // create space for array
16
```

Fig. 19.2 | Class that contains an array of random integers and a method that will search that array sequentially. (Part 1 of 2.)

```
17         // fill array with random ints in range 10-99
18         for ( int i = 0; i < size; i++ )
19            data[ i ] = 10 + generator.nextInt( 90 );
20      } // end LinearArray constructor
21
22      // perform a linear search on the data
23      public int linearSearch( int searchKey )
24      {
25         // loop through array sequentially
26         for ( int index = 0; index < data.length; index++ )
27            if ( data[ index ] == searchKey )
28               return index; // return index of integer
29
30         return -1; // integer was not found
31      } // end method linearSearch
32
33      // method to output values in array
34      public String toString()
35      {
36         return Arrays.toString( data );
37      } // end method toString
38   } // end class LinearArray
```

Fig. 19.2 | Class that contains an array of random integers and a method that will search that array sequentially. (Part 2 of 2.)

Lines 23–31 perform the linear search. The search key is passed to parameter `searchKey`. Lines 26–28 loop through the elements in the array. Line 27 compares each with `searchKey`. If the values are equal, line 28 returns the *index* of the element. If the loop ends without finding the value, line 30 returns `-1`. Lines 34–37 declare method `toString`, which uses `Arrays` `static` method `toString` to return a `String` representation of the array.

Figure 19.3 creates a `LinearArray` object containing an array of `10` `int`s (line 16) and allows the user to search the array. Lines 20–22 prompt the user for and store the search key. The array holds `int`s from `10`–`99` (line 19 of Fig. 19.2). Line 28 calls method `linearSearch` to determine whether `searchInt` is in the array. If it's not, `linearSearch` returns `-1` and the program notifies the user (lines 31–32). If `searchInt` is in the array, `linearSearch` returns the position of the element, which the program outputs in lines 34–35. Lines 38–40 retrieve the next integer from the user.

```
1  // Fig. 19.3: LinearSearchTest.java
2  // Sequentially searching an array for an item.
3  import java.util.Scanner;
4
5  public class LinearSearchTest
6  {
7     public static void main( String[] args )
8     {
9        // create Scanner object to input data
10       Scanner input = new Scanner( System.in );
```

Fig. 19.3 | Sequentially searching an array for an item. (Part 1 of 2.)

```
11
12         int searchInt; // search key
13         int position; // location of search key in array
14
15         // create array and output it
16         LinearArray searchArray = new LinearArray( 10 );
17         System.out.println( searchArray + "\n" ); // print array
18
19         // get input from user
20         System.out.print(
21            "Please enter an integer value (-1 to quit): " );
22         searchInt = input.nextInt(); // read first int from user
23
24         // repeatedly input an integer; -1 terminates the program
25         while ( searchInt != -1 )
26         {
27            // perform linear search
28            position = searchArray.linearSearch( searchInt );
29
30            if ( position == -1 ) // integer was not found
31               System.out.println( "The integer " + searchInt +
32                  " was not found.\n" );
33            else // integer was found
34               System.out.println( "The integer " + searchInt +
35                  " was found in position " + position + ".\n" );
36
37            // get input from user
38            System.out.print(
39               "Please enter an integer value (-1 to quit): " );
40            searchInt = input.nextInt(); // read next int from user
41         } // end while
42      } // end main
43   } // end class LinearSearchTest
```

```
[59, 97, 34, 90, 79, 56, 24, 51, 30, 69]

Please enter an integer value (-1 to quit): 79
The integer 79 was found in position 4.

Please enter an integer value (-1 to quit): 61
The integer 61 was not found.

Please enter an integer value (-1 to quit): 51
The integer 51 was found in position 7.

Please enter an integer value (-1 to quit): -1
```

Fig. 19.3 | Sequentially searching an array for an item. (Part 2 of 2.)

Efficiency of Linear Search

Searching algorithms all accomplish the same goal—finding an element that matches a given search key, if such an element does, in fact, exist. There are, however, a number of things that differentiate search algorithms from one another. *The major difference is the amount of effort they require to complete the search.* One way to describe this effort is with **Big O notation**, which indicates the worst-case run time for an algorithm—that is, how

hard an algorithm may have to work to solve a problem. For searching and sorting algorithms, this depends particularly on how many data elements there are.

O(1) Algorithms

Suppose an algorithm is designed to test whether the first element of an array is equal to the second. If the array has 10 elements, this algorithm requires one comparison. If the array has 1000 elements, it still requires one comparison. In fact, the algorithm is completely independent of the number of elements in the array. This algorithm is said to have a **constant run time**, which is represented in Big O notation as ***O*(1)**. An algorithm that is $O(1)$ does not necessarily require only one comparison. $O(1)$ just means that the number of comparisons is *constant*—it does not grow as the size of the array increases. An algorithm that tests whether the first element of an array is equal to any of the next three elements is still $O(1)$ even though it requires three comparisons.

O(n) Algorithms

An algorithm that tests whether the first array element is equal to *any* of the other array elements will require at most $n - 1$ comparisons, where n is the number of array elements. If the array has 10 elements, this algorithm requires up to nine comparisons. If the array has 1000 elements, it requires up to 999 comparisons. As n grows larger, the n part of the expression "dominates," and subtracting one becomes inconsequential. Big O is designed to highlight these dominant terms and ignore terms that become unimportant as n grows. For this reason, an algorithm that requires a total of $n - 1$ comparisons (such as the one we described earlier) is said to be ***O*(*n*)**. An $O(n)$ algorithm is referred to as having a **linear run time**. $O(n)$ is often pronounced "on the order of n" or simply "order n."

O(n²) Algorithms

Now suppose you have an algorithm that tests whether *any* element of an array is duplicated elsewhere in the array. The first element must be compared with every other element in the array. The second element must be compared with every other element except the first (it was already compared to the first). The third element must be compared with every other element except the first two. In the end, this algorithm will end up making $(n - 1) + (n - 2) + \ldots + 2 + 1$ or $n^2/2 - n/2$ comparisons. As n increases, the n^2 term dominates and the n term becomes inconsequential. Again, Big O notation highlights the n^2 term, leaving $n^2/2$. But, as we'll soon see, constant factors are omitted in Big O notation.

Big O is concerned with how an algorithm's run time grows in relation to the number of items processed. Suppose an algorithm requires n^2 comparisons. With four elements, the algorithm requires 16 comparisons; with eight elements, 64 comparisons. With this algorithm, *doubling* the number of elements *quadruples* the number of comparisons. Consider a similar algorithm requiring $n^2/2$ comparisons. With four elements, the algorithm requires eight comparisons; with eight elements, 32 comparisons. Again, *doubling* the number of elements *quadruples* the number of comparisons. Both of these algorithms grow as the square of n, so Big O ignores the constant and both algorithms are considered to be ***O*(*n*²)**, which is referred to as **quadratic run time** and pronounced "on the order of n-squared" or more simply "order n-squared."

When n is small, $O(n^2)$ algorithms (running on today's computers) will not noticeably affect performance. But as n grows, you'll start to notice the performance degradation. An $O(n^2)$ algorithm running on a million-element array would require a trillion "operations"

(where each could actually require several machine instructions to execute). This could take several hours. A billion-element array would require a quintillion operations, a number so large that the algorithm could take decades! $O(n^2)$ algorithms, unfortunately, are easy to write, as you'll see in this chapter. You'll also see algorithms with more favorable Big O measures. These efficient algorithms often take a bit more cleverness and work to create, but their superior performance can be well worth the extra effort, especially as n gets large and as algorithms are combined into larger programs.

Big O of the Linear Search

The linear search algorithm runs in $O(n)$ time. The worst case in this algorithm is that every element must be checked to determine whether the search item exists in the array. If the size of the array is *doubled*, the number of comparisons that the algorithm must perform is also *doubled*. Linear search can provide outstanding performance if the element matching the search key happens to be at or near the front of the array. But we seek algorithms that perform well, on average, across *all* searches, including those where the element matching the search key is near the end of the array.

Linear search is easy to program, but it can be slow compared to other search algorithms. If a program needs to perform many searches on large arrays, it's better to implement a more efficient algorithm, such as the binary search, which we present next.



Performance Tip 19.1

Sometimes the simplest algorithms perform poorly. Their virtue is that they're easy to program, test and debug. Sometimes more complex algorithms are required to realize maximum performance.

19.2.2 Binary Search

The **binary search algorithm** is more efficient than linear search, but it requires that the array be sorted. The first iteration of this algorithm tests the *middle* element in the array. If this matches the search key, the algorithm ends. Assuming the array is sorted in *ascending* order, then if the search key is *less than* the middle element, it cannot match any element in the second half of the array and the algorithm continues with only the first half of the array (i.e., the first element up to, but not including, the middle element). If the search key is *greater than* the middle element, it cannot match any element in the first half of the array and the algorithm continues with only the second half (i.e., the element *after* the middle element through the last element). Each iteration tests the middle value of the remaining portion of the array. If the search key does not match the element, the algorithm eliminates half of the remaining elements. The algorithm ends by either finding an element that matches the search key or reducing the subarray to zero size.

As an example consider the sorted 15-element array

```
2  3  5  10  27  30  34  51  56  65  77  81  82  93  99
```

and a search key of 65. A program implementing the binary search algorithm would first check whether 51 is the search key (because 51 is the *middle* element of the array). The search key (65) is larger than 51, so 51 is ignored along with the first half of the array (all elements smaller than 51), leaving

```
56  65  77  81  82  93  99
```

Next, the algorithm checks whether 81 (the middle element of the remainder of the array) matches the search key. The search key (65) is smaller than 81, so 81 is discarded along with the elements larger than 81. After just two tests, the algorithm has narrowed the number of values to check to only three (56, 65 and 77). It then checks 65 (which indeed matches the search key) and returns the index of the array element containing 65. This algorithm required just three comparisons to determine whether the search key matched an element of the array. Using a linear search algorithm would have required 10 comparisons. [*Note:* In this example, we've chosen to use an array with 15 elements so that there will always be an obvious middle element in the array. With an even number of elements, the middle of the array lies between two elements. We implement the algorithm to choose the higher of those two elements.]

Binary Search Implementation

Figure 19.4 declares class `BinaryArray`. This class is similar to `LinearArray`—it has two `private` instance variables, a constructor, a search method (`binarySearch`), a `remainingElements` method and a `toString` method. Lines 13–22 declare the constructor. After the array has been initialized with random `int`s from 10–99 (lines 18–19), line 21 calls the `Arrays.sort` method on the array `data`. The **`sort` method** is a `static` method of class `Arrays` that sorts the elements in an array in ascending order by default; an overloaded version of this method allows you to change the sorting order. Recall that the binary search algorithm will work only on a sorted array.

```
1  // Fig. 19.4: BinaryArray.java
2  // Class that contains an array of random integers and a method
3  // that uses binary search to find an integer.
4  import java.util.Random;
5  import java.util.Arrays;
6
7  public class BinaryArray
8  {
9     private int[] data; // array of values
10    private static final Random generator = new Random();
11
12    // create array of given size and fill with random integers
13    public BinaryArray( int size )
14    {
15       data = new int[ size ]; // create space for array
16
17       // fill array with random ints in range 10-99
18       for ( int i = 0; i < size; i++ )
19          data[ i ] = 10 + generator.nextInt( 90 );
20
21       Arrays.sort( data );
22    } // end BinaryArray constructor
23
24    // perform a binary search on the data
25    public int binarySearch( int searchElement )
26    {
```

Fig. 19.4 | Class that contains an array of random integers and a method that uses binary search to find an integer. (Part 1 of 3.)

```
27          int low = 0; // low end of the search area
28          int high = data.length - 1; // high end of the search area
29          int middle = ( low + high + 1 ) / 2; // middle element
30          int location = -1; // return value; -1 if not found
31
32          do // loop to search for element
33          {
34             // print remaining elements of array
35             System.out.print( remainingElements( low, high ) );
36
37             // output spaces for alignment
38             for ( int i = 0; i < middle; i++ )
39                System.out.print( "   " );
40             System.out.println( " * " ); // indicate current middle
41
42             // if the element is found at the middle
43             if ( searchElement == data[ middle ] )
44                location = middle; // location is the current middle
45
46             // middle element is too high
47             else if ( searchElement < data[ middle ] )
48                high = middle - 1; // eliminate the higher half
49             else  // middle element is too low
50                low = middle + 1; // eliminate the lower half
51
52             middle = ( low + high + 1 ) / 2; // recalculate the middle
53          } while ( ( low <= high ) && ( location == -1 ) );
54
55          return location; // return location of search key
56       } // end method binarySearch
57
58       // method to output certain values in array
59       public String remainingElements( int low, int high )
60       {
61          StringBuilder temporary = new StringBuilder();
62
63          // output spaces for alignment
64          for ( int i = 0; i < low; i++ )
65             temporary.append( "   " );
66
67          // output elements left in array
68          for ( int i = low; i <= high; i++ )
69             temporary.append( data[ i ] + " " );
70
71          temporary.append( "\n" );
72          return temporary.toString();
73       } // end method remainingElements
74
75       // method to output values in array
76       public String toString()
77       {
```

Fig. 19.4 | Class that contains an array of random integers and a method that uses binary search to find an integer. (Part 2 of 3.)

```
78          return remainingElements( 0, data.length - 1 );
79       } // end method toString
80    } // end class BinaryArray
```

Fig. 19.4 | Class that contains an array of random integers and a method that uses binary search to find an integer. (Part 3 of 3.)

Lines 25–56 declare method `binarySearch`. The search key is passed into parameter `searchElement` (line 25). Lines 27–29 calculate the `low` end index, `high` end index and `middle` index of the portion of the array that the program is currently searching. At the beginning of the method, the `low` end is `0`, the `high` end is the length of the array minus `1` and the `middle` is the average of these two values. Line 30 initializes the `location` of the element to `-1`—the value that will be returned if the element is not found. Lines 32–53 loop until `low` is greater than `high` (this occurs when the element is not found) or `location` does not equal `-1` (indicating that the search key was found). Line 43 tests whether the value in the `middle` element is equal to `searchElement`. If this is `true`, line 44 assigns `middle` to `location`. Then the loop terminates and `location` is returned to the caller. Each iteration of the loop tests a single value (line 43) and *eliminates half of the remaining values in the array* (line 48 or 50).

Figure 19.5 performs a binary search using the search key the user enters to determine whether it matches an element in the array. The first line of output from this program is the array of `int`s, in increasing order. When the user instructs the program to search for `23`, the program first tests the middle element, which is `42` (as indicated by `*`). The search key is less than `42`, so the program eliminates the second half of the array and tests the middle element from the first half. The search key is smaller than `34`, so the program eliminates the second half of the array, leaving only three elements. Finally, the program checks `23` (which matches the search key) and returns the index `1`.

```
1  // Fig. 19.5: BinarySearchTest.java
2  // Use binary search to locate an item in an array.
3  import java.util.Scanner;
4
5  public class BinarySearchTest
6  {
7     public static void main( String[] args )
8     {
9        // create Scanner object to input data
10       Scanner input = new Scanner( System.in );
11
12       int searchInt; // search key
13       int position; // location of search key in array
14
15       // create array and output it
16       BinaryArray searchArray = new BinaryArray( 15 );
17       System.out.println( searchArray );
18
```

Fig. 19.5 | Use binary search to locate an item in an array (* in the output marks the middle element). (Part 1 of 3.)

```
19         // get input from user
20         System.out.print(
21            "Please enter an integer value (-1 to quit): " );
22         searchInt = input.nextInt(); // read an int from user
23         System.out.println();
24
25         // repeatedly input an integer; -1 terminates the program
26         while ( searchInt != -1 )
27         {
28            // use binary search to try to find integer
29            position = searchArray.binarySearch( searchInt );
30
31            // return value of -1 indicates integer was not found
32            if ( position == -1 )
33               System.out.println( "The integer " + searchInt +
34                  " was not found.\n" );
35            else
36               System.out.println( "The integer " + searchInt +
37                  " was found in position " + position + ".\n" );
38
39            // get input from user
40            System.out.print(
41               "Please enter an integer value (-1 to quit): " );
42            searchInt = input.nextInt(); // read an int from user
43            System.out.println();
44         } // end while
45      } // end main
46   } // end class BinarySearchTest
```

```
13 23 24 34 35 36 38 42 47 51 68 74 75 85 97

Please enter an integer value (-1 to quit): 23

13 23 24 34 35 36 38 42 47 51 68 74 75 85 97
                     *
13 23 24 34 35 36 38
          *
13 23 24
     *
The integer 23 was found in position 1.

Please enter an integer value (-1 to quit): 75

13 23 24 34 35 36 38 42 47 51 68 74 75 85 97
                     *
                       47 51 68 74 75 85 97
                                  *
                                    75 85 97
                                        *
                                    75
                                     *
The integer 75 was found in position 12.
```

Fig. 19.5 | Use binary search to locate an item in an array (* in the output marks the middle element). (Part 2 of 3.)

```
Please enter an integer value (-1 to quit): 52

13 23 24 34 35 36 38 42 47 51 68 74 75 85 97
                       *
                         47 51 68 74 75 85 97
                                  *
                         47 51 68
                             *
                                68
                                 *
The integer 52 was not found.

Please enter an integer value (-1 to quit): -1
```

Fig. 19.5 | Use binary search to locate an item in an array (* in the output marks the middle element). (Part 3 of 3.)

Efficiency of the Binary Search

In the worst-case scenario, searching a *sorted* array of 1023 elements takes only 10 comparisons when using a binary search. Repeatedly dividing 1023 by 2 (because after each comparison we can eliminate half the array) and rounding down (because we also remove the middle element) yields the values 511, 255, 127, 63, 31, 15, 7, 3, 1 and 0. The number 1023 ($2^{10} - 1$) is divided by 2 only 10 times to get the value 0, which indicates that there are no more elements to test. Dividing by 2 is equivalent to one comparison in the binary search algorithm. Thus, an array of 1,048,575 ($2^{20} - 1$) elements takes a maximum of 20 comparisons to find the key, and an array of over one billion elements takes a maximum of 30 comparisons to find the key. This is a tremendous improvement in performance over the linear search. For a one-billion-element array, this is a difference between an average of 500 million comparisons for the linear search and a maximum of only 30 comparisons for the binary search! The maximum number of comparisons needed for the binary search of any sorted array is the exponent of the first power of 2 greater than the number of elements in the array, which is represented as $\log_2 n$. All logarithms grow at roughly the same rate, so in big O notation the base can be omitted. This results in a big O of $O(\log n)$ for a binary search, which is also known as **logarithmic run time**.

19.3 Sorting Algorithms

Sorting data (i.e., placing the data into some particular order, such as ascending or descending) is one of the most important computing applications. A bank sorts all checks by account number so that it can prepare individual bank statements at the end of each month. Telephone companies sort their lists of accounts by last name and, further, by first name to make it easy to find phone numbers. Virtually every organization must sort some data, and often massive amounts of it. Sorting data is an intriguing, computer-intensive problem that has attracted intense research efforts.

An important item to understand about sorting is that the end result—the sorted array—will be the *same* no matter which algorithm you use to sort the array. The choice of algorithm affects only the run time and memory use of the program. The rest of this chapter introduces three common sorting algorithms. The first two—*selection sort* and *insertion sort*—are easy to program but *inefficient*. The last algorithm—merge sort—is

much *faster* than selection sort and insertion sort but *harder to program*. We focus on sorting arrays of primitive-type data, namely `int`s. It's possible to sort arrays of class objects as well. We discuss this in Section 20.7.1.

19.3.1 Selection Sort

Selection sort is a simple, but inefficient, sorting algorithm. Its first iteration selects the *smallest* element in the array and swaps it with the first element. The second iteration selects the *second-smallest* item (which is the smallest item of the remaining elements) and swaps it with the second element. The algorithm continues until the last iteration selects the *second-largest* element and swaps it with the second-to-last index, leaving the largest element in the last index. After the *i*th iteration, the smallest *i* items of the array will be sorted into increasing order in the first *i* elements of the array.

As an example, consider the array

```
34  56  4  10  77  51  93  30  5  52
```

A program that implements selection sort first determines the smallest element (4) of this array, which is contained in index 2. The program swaps 4 with 34, resulting in

```
4  56  34  10  77  51  93  30  5  52
```

The program then determines the smallest value of the remaining elements (all elements except 4), which is 5, contained in index 8. The program swaps 5 with 56, resulting in

```
4  5  34  10  77  51  93  30  56  52
```

On the third iteration, the program determines the next smallest value (10) and swaps it with 34.

```
4  5  10  34  77  51  93  30  56  52
```

The process continues until the array is fully sorted.

```
4  5  10  30  34  51  52  56  77  93
```

After the first iteration, the smallest element is in the first position. After the second iteration, the two smallest elements are in order in the first two positions. After the third iteration, the three smallest elements are in order in the first three positions.

Figure 19.6 declares the `SelectionSort` class. This class has two `private` instance variables—an array of `int`s named `data`, and a `static` `Random` object to generate random integers to fill the array. When an object of class `SelectionSort` is instantiated, the constructor (lines 13–20) creates and initializes the array `data` with random `int`s in the range 10–99.

```
1  // Fig. 19.6: SelectionSort.java
2  // Class that creates an array filled with random integers.
3  // Provides a method to sort the array with selection sort.
4  import java.util.Arrays;
5  import java.util.Random;
6
```

Fig. 19.6 | Class that creates an array filled with random integers. Provides a method to sort the array with selection sort. (Part 1 of 3.)

```
7  public class SelectionSort
8  {
9     private int[] data; // array of values
10    private static final Random generator = new Random();
11
12    // create array of given size and fill with random integers
13    public SelectionSort( int size )
14    {
15       data = new int[ size ]; // create space for array
16
17       // fill array with random ints in range 10-99
18       for ( int i = 0; i < size; i++ )
19          data[ i ] = 10 + generator.nextInt( 90 );
20    } // end SelectionSort constructor
21
22    // sort array using selection sort
23    public void sort()
24    {
25       int smallest; // index of smallest element
26
27       // loop over data.length - 1 elements
28       for ( int i = 0; i < data.length - 1; i++ )
29       {
30          smallest = i; // first index of remaining array
31
32          // loop to find index of smallest element
33          for ( int index = i + 1; index < data.length; index++ )
34             if ( data[ index ] < data[ smallest ] )
35                smallest = index;
36
37          swap( i, smallest ); // swap smallest element into position
38          printPass( i + 1, smallest ); // output pass of algorithm
39       } // end outer for
40    } // end method sort
41
42    // helper method to swap values in two elements
43    public void swap( int first, int second )
44    {
45       int temporary = data[ first ]; // store first in temporary
46       data[ first ] = data[ second ]; // replace first with second
47       data[ second ] = temporary; // put temporary in second
48    } // end method swap
49
50    // print a pass of the algorithm
51    public void printPass( int pass, int index )
52    {
53       System.out.print( String.format( "after pass %2d: ", pass ) );
54
55       // output elements till selected item
56       for ( int i = 0; i < index; i++ )
57          System.out.print( data[ i ] + "  " );
```

Fig. 19.6 | Class that creates an array filled with random integers. Provides a method to sort the array with selection sort. (Part 2 of 3.)

```
58
59          System.out.print( data[ index ] + "* " ); // indicate swap
60
61          // finish outputting array
62          for ( int i = index + 1; i < data.length; i++ )
63             System.out.print( data[ i ] + "   " );
64
65          System.out.print( "\n                " ); // for alignment
66
67          // indicate amount of array that is sorted
68          for ( int j = 0; j < pass; j++ )
69             System.out.print( "--   " );
70          System.out.println( "\n" ); // add endline
71       } // end method printPass
72
73       // method to output values in array
74       public String toString()
75       {
76          return Arrays.toString( data );
77       } // end method toString
78    } // end class SelectionSort
```

Fig. 19.6 | Class that creates an array filled with random integers. Provides a method to sort the array with selection sort. (Part 3 of 3.)

Lines 23–40 declare the `sort` method. Line 25 declares the variable `smallest`, which will store the index of the smallest element in the remaining array. Lines 28–39 loop `data.length - 1` times. Line 30 initializes the index of the smallest element to the current item. Lines 33–35 loop over the remaining elements in the array. For each of these elements, line 34 compares its value to the value of the smallest element. If the current element is smaller than the smallest element, line 35 assigns the current element's index to `smallest`. When this loop finishes, `smallest` will contain the index of the smallest element in the remaining array. Line 36 calls method `swap` (lines 43–48) to place the smallest remaining element in the next ordered spot in the array.

Line 9 of Fig. 19.7 creates a `SelectionSort` object with 10 elements. Line 12 outputs the unsorted object. Line 14 calls method `sort` (lines 22–39 of Fig. 19.6), which sorts the elements using selection sort. Then lines 16–17 output the sorted object. The output uses dashes (lines 67–68) to indicate the portion of the array that is sorted after each pass. An asterisk is placed next to the position of the element that was swapped with the smallest element on that pass. On each pass, the element next to the asterisk (specified at line 58) and the element above the rightmost set of dashes were swapped.

```
1  // Fig. 19.7: SelectionSortTest.java
2  // Testing the selection sort class.
3
4  public class SelectionSortTest
5  {
```

Fig. 19.7 | Testing the selection sort class. (Part 1 of 2.)

```
6      public static void main( String[] args )
7      {
8         // create object to perform selection sort
9         SelectionSort sortArray = new SelectionSort( 10 );
10
11        System.out.println( "Unsorted array:" );
12        System.out.println( sortArray + "\n" ); // print unsorted array
13
14        sortArray.sort(); // sort array
15
16        System.out.println( "Sorted array:" );
17        System.out.println( sortArray ); // print sorted array
18     } // end main
19  } // end class SelectionSortTest
```

```
Unsorted array:
[61, 87, 80, 58, 40, 50, 20, 13, 71, 45]

after pass  1: 13  87  80  58  40  50  20  61* 71  45
               --

after pass  2: 13  20  80  58  40  50  87* 61  71  45
               --  --

after pass  3: 13  20  40  58  80* 50  87  61  71  45
               --  --  --

after pass  4: 13  20  40  45  80  50  87  61  71  58*
               --  --  --  --

after pass  5: 13  20  40  45  50  80* 87  61  71  58
               --  --  --  --  --

after pass  6: 13  20  40  45  50  58  87  61  71  80*
               --  --  --  --  --  --

after pass  6: 13  20  40  45  50  58  87  61  71  80*
               --  --  --  --  --  --

after pass  7: 13  20  40  45  50  58  61  87* 71  80
               --  --  --  --  --  --  --

after pass  8: 13  20  40  45  50  58  61  71  87* 80
               --  --  --  --  --  --  --  --

after pass  9: 13  20  40  45  50  58  61  71  80  87*
               --  --  --  --  --  --  --  --  --

Sorted array:
[13, 20, 40, 45, 50, 58, 61, 71, 80, 87]
```

Fig. 19.7 | Testing the selection sort class. (Part 2 of 2.)

Efficiency of the Selection Sort

The selection sort algorithm runs in $O(n^2)$ time. The `sort` method in lines 23–40 of Fig. 19.6, which implements the selection sort algorithm, contains two `for` loops. The outer `for` loop (lines 28–39) iterates over the first $n - 1$ elements in the array, swapping the smallest remaining item into its sorted position. The inner `for` loop (lines 33–35) iterates over each

item in the remaining array, searching for the smallest element. This loop executes $n - 1$ times during the first iteration of the outer loop, $n - 2$ times during the second iteration, then $n - 3, \ldots, 3, 2, 1$. This inner loop will iterate a total of $n(n - 1)/2$ or $(n^2 - n)/2$. In Big O notation, smaller terms drop out and constants are ignored, leaving a final Big O of $O(n^2)$.

19.3.2 Insertion Sort

Insertion sort is another *simple, but inefficient*, sorting algorithm. The first iteration of this algorithm takes the *second element* in the array and, if it's *less than* the *first element*, *swaps it with the first element*. The second iteration looks at the third element and inserts it into the correct position with respect to the first two, so all three elements are in order. At the *i*th iteration of this algorithm, the first *i* elements in the original array will be sorted.

Consider as an example the following array. [*Note:* This array is identical to the one used in the discussions of selection sort and merge sort.]

```
34   56   4   10   77   51   93   30   5   52
```

A program that implements the insertion sort algorithm will first look at the first two elements of the array, 34 and 56. These are already in order, so the program continues. (If they were out of order, the program would swap them.)

In the next iteration, the program looks at the third value, 4. This value is less than 56, so the program stores 4 in a temporary variable and moves 56 one element to the right. The program then checks and determines that 4 is less than 34, so it moves 34 one element to the right. The program has now reached the beginning of the array, so it places 4 in the zeroth element. The array now is

```
4   34   56   10   77   51   93   30   5   52
```

In the next iteration, the program stores 10 in a temporary variable. Then it compares 10 to 56 and moves 56 one element to the right because it's larger than 10. The program then compares 10 to 34, moving 34 right one element. When the program compares 10 to 4, it observes that 10 is larger than 4 and places 10 in element 1. The array now is

```
4   10   34   56   77   51   93   30   5   52
```

Using this algorithm, at the *i*th iteration, the first *i* elements of the original array are sorted, but they may not be in their final locations—smaller values may be located later in the array.

Figure 19.8 declares the `InsertionSort` class. Lines 23–47 declare the `sort` method. Line 25 declares the variable `insert`, which holds the element you're going to insert while you move the other elements. Lines 28–46 loop over `data.length - 1` items in the array. In each iteration, line 31 stores in `insert` the value of the element that will be inserted into the sorted portion of the array. Line 34 declares and initializes the variable `moveItem`, which keeps track of where to insert the element. Lines 37–42 loop to locate the correct position where the element should be inserted. The loop will terminate either when the program reaches the front of the array or when it reaches an element that is less than the value to be inserted. Line 40 moves an element to the right, and line 41 decrements the position at which to insert the next element. After the loop ends, line 44 inserts the element into place. Figure 19.9 is the same as Fig. 19.7 except that it creates and uses an `InsertionSort` object. The output of this program uses dashes to indicate the portion of the array that is sorted after each pass. An asterisk is placed next to the element that was inserted into place on that pass.

```
1  // Fig. 19.8: InsertionSort.java
2  // Class that creates an array filled with random integers.
3  // Provides a method to sort the array with insertion sort.
4  import java.util.Arrays;
5  import java.util.Random;
6
7  public class InsertionSort
8  {
9     private int[] data; // array of values
10    private static final Random generator = new Random();
11
12    // create array of given size and fill with random integers
13    public InsertionSort( int size )
14    {
15       data = new int[ size ]; // create space for array
16
17       // fill array with random ints in range 10-99
18       for ( int i = 0; i < size; i++ )
19          data[ i ] = 10 + generator.nextInt( 90 );
20    } // end InsertionSort constructor
21
22    // sort array using insertion sort
23    public void sort()
24    {
25       int insert; // temporary variable to hold element to insert
26
27       // loop over data.length - 1 elements
28       for ( int next = 1; next < data.length; next++ )
29       {
30          // store value in current element
31          insert = data[ next ];
32
33          // initialize location to place element
34          int moveItem = next;
35
36          // search for place to put current element
37          while ( moveItem > 0 && data[ moveItem - 1 ] > insert )
38          {
39             // shift element right one slot
40             data[ moveItem ] = data[ moveItem - 1 ];
41             moveItem--;
42          } // end while
43
44          data[ moveItem ] = insert; // place inserted element
45          printPass( next, moveItem ); // output pass of algorithm
46       } // end for
47    } // end method sort
48
49    // print a pass of the algorithm
50    public void printPass( int pass, int index )
51    {
```

Fig. 19.8 | Class that creates an array filled with random integers. Provides a method to sort the array with insertion sort. (Part 1 of 2.)

```
52          System.out.print( String.format( "after pass %2d: ", pass ) );
53
54          // output elements till swapped item
55          for ( int i = 0; i < index; i++ )
56             System.out.print( data[ i ] + "   " );
57
58          System.out.print( data[ index ] + "* " ); // indicate swap
59
60          // finish outputting array
61          for ( int i = index + 1; i < data.length; i++ )
62             System.out.print( data[ i ] + "   " );
63
64          System.out.print( "\n               " ); // for alignment
65
66          // indicate amount of array that is sorted
67          for( int i = 0; i <= pass; i++ )
68             System.out.print( "--   " );
69          System.out.println( "\n" ); // add endline
70       } // end method printPass
71
72       // method to output values in array
73       public String toString()
74       {
75          return Arrays.toString( data );
76       } // end method toString
77    } // end class InsertionSort
```

Fig. 19.8 | Class that creates an array filled with random integers. Provides a method to sort the array with insertion sort. (Part 2 of 2.)

```
1  // Fig. 19.9: InsertionSortTest.java
2  // Testing the insertion sort class.
3
4  public class InsertionSortTest
5  {
6     public static void main( String[] args )
7     {
8        // create object to perform insertion sort
9        InsertionSort sortArray = new InsertionSort( 10 );
10
11       System.out.println( "Unsorted array:" );
12       System.out.println( sortArray + "\n" ); // print unsorted array
13
14       sortArray.sort(); // sort array
15
16       System.out.println( "Sorted array:" );
17       System.out.println( sortArray ); // print sorted array
18    } // end main
19 } // end class InsertionSortTest
```

Fig. 19.9 | Testing the insertion sort class. (Part 1 of 2.)

```
Unsorted array:
[40, 17, 45, 82, 62, 32, 30, 44, 93, 10]

after pass  1: 17* 40  45  82  62  32  30  44  93  10
               --  --

after pass  2: 17  40  45* 82  62  32  30  44  93  10
               --  --  --

after pass  3: 17  40  45  82* 62  32  30  44  93  10
               --  --  --  --

after pass  4: 17  40  45  62* 82  32  30  44  93  10
               --  --  --  --  --

after pass  5: 17  32* 40  45  62  82  30  44  93  10
               --  --  --  --  --  --

after pass  6: 17  30* 32  40  45  62  82  44  93  10
               --  --  --  --  --  --  --

after pass  7: 17  30  32  40  44* 45  62  82  93  10
               --  --  --  --  --  --  --  --

after pass  8: 17  30  32  40  44  45  62  82  93* 10
               --  --  --  --  --  --  --  --  --

after pass  9: 10* 17  30  32  40  44  45  62  82  93
               --  --  --  --  --  --  --  --  --  --

Sorted array:
[10, 17, 30, 32, 40, 44, 45, 62, 82, 93]
```

Fig. 19.9 | Testing the insertion sort class. (Part 2 of 2.)

Efficiency of the Insertion Sort

The insertion sort algorithm also runs in $O(n^2)$ time. Like selection sort, the implementation of insertion sort (lines 23–47 of Fig. 19.8) contains two loops. The `for` loop (lines 28–46) iterates `data.length - 1` times, inserting an element into the appropriate position in the elements sorted so far. For the purposes of this application, `data.length - 1` is equivalent to $n - 1$ (as `data.length` is the size of the array). The `while` loop (lines 37–42) iterates over the preceding elements in the array. In the worst case, this `while` loop will require $n - 1$ comparisons. Each individual loop runs in $O(n)$ time. In Big O notation, nested loops mean that you must multiply the number of comparisons. For each iteration of an outer loop, there will be a certain number of iterations of the inner loop. In this algorithm, for each $O(n)$ iterations of the outer loop, there will be $O(n)$ iterations of the inner loop. Multiplying these values results in a Big O of $O(n^2)$.

19.3.3 Merge Sort

Merge sort is an *efficient* sorting algorithm but is conceptually *more complex* than selection sort and insertion sort. The merge sort algorithm sorts an array by *splitting* it into two equal-sized subarrays, *sorting* each subarray, then *merging* them into one larger array. With an odd number of elements, the algorithm creates the two subarrays such that one has one more element than the other.

The implementation of merge sort in this example is recursive. The base case is an array with one element, which is, of course, sorted, so the merge sort immediately returns in this case. The recursion step splits the array into two approximately equal pieces, recursively sorts them, then merges the two sorted arrays into one larger, sorted array.

Suppose the algorithm has already merged smaller arrays to create sorted arrays A:

```
   4   10   34   56   77
```

and B:

```
   5   30   51   52   93
```

Merge sort combines these two arrays into one larger, sorted array. The smallest element in A is 4 (located in the zeroth index of A). The smallest element in B is 5 (located in the zeroth index of B). In order to determine the smallest element in the larger array, the algorithm compares 4 and 5. The value from A is smaller, so 4 becomes the first element in the merged array. The algorithm continues by comparing 10 (the second element in A) to 5 (the first element in B). The value from B is smaller, so 5 becomes the second element in the larger array. The algorithm continues by comparing 10 to 30, with 10 becoming the third element in the array, and so on.

Lines 22–25 of Fig. 19.10 declare the `sort` method. Line 24 calls method `sortArray` with `0` and `data.length - 1` as the arguments—corresponding to the beginning and ending indices, respectively, of the array to be sorted. These values tell method `sortArray` to operate on the entire array.

Method `sortArray` is declared in lines 28–49. Line 31 tests the base case. If the size of the array is `1`, the array is already sorted, so the method returns immediately. If the size of the array is greater than `1`, the method splits the array in two, recursively calls method `sortArray` to sort the two subarrays, then merges them. Line 43 recursively calls method `sortArray` on the first half of the array, and line 44 recursively calls method `sortArray` on the second half. When these two method calls return, each half of the array has been sorted. Line 47 calls method `merge` (lines 52–91) on the two halves of the array to combine the two sorted arrays into one larger sorted array.

```
1  // Fig. 19.10: MergeSort.java
2  // Class creates an array filled with random integers.
3  // Provides a method to sort the array with merge sort.
4  import java.util.Random;
5
6  public class MergeSort
7  {
8     private int[] data; // array of values
9     private static final Random generator = new Random();
10
11    // create array of given size and fill with random integers
12    public MergeSort( int size )
13    {
14       data = new int[ size ]; // create space for array
15
```

Fig. 19.10 | Class that creates an array filled with random integers. Provides a method to sort the array with merge sort. (Part 1 of 3.)

```
16          // fill array with random ints in range 10-99
17          for ( int i = 0; i < size; i++ )
18             data[ i ] = 10 + generator.nextInt( 90 );
19       } // end MergeSort constructor
20
21       // calls recursive split method to begin merge sorting
22       public void sort()
23       {
24          sortArray( 0, data.length - 1 ); // split entire array
25       } // end method sort
26
27       // splits array, sorts subarrays and merges subarrays into sorted array
28       private void sortArray( int low, int high )
29       {
30          // test base case; size of array equals 1
31          if ( ( high - low ) >= 1 ) // if not base case
32          {
33             int middle1 = ( low + high ) / 2; // calculate middle of array
34             int middle2 = middle1 + 1; // calculate next element over
35
36             // output split step
37             System.out.println( "split:   " + subarray( low, high ) );
38             System.out.println( "         " + subarray( low, middle1 ) );
39             System.out.println( "         " + subarray( middle2, high ) );
40             System.out.println();
41
42             // split array in half; sort each half (recursive calls)
43             sortArray( low, middle1 ); // first half of array
44             sortArray( middle2, high ); // second half of array
45
46             // merge two sorted arrays after split calls return
47             merge ( low, middle1, middle2, high );
48          } // end if
49       } // end method sortArray
50
51       // merge two sorted subarrays into one sorted subarray
52       private void merge( int left, int middle1, int middle2, int right )
53       {
54          int leftIndex = left; // index into left subarray
55          int rightIndex = middle2; // index into right subarray
56          int combinedIndex = left; // index into temporary working array
57          int[] combined = new int[ data.length ]; // working array
58
59          // output two subarrays before merging
60          System.out.println( "merge:   " + subarray( left, middle1 ) );
61          System.out.println( "         " + subarray( middle2, right ) );
62
63          // merge arrays until reaching end of either
64          while ( leftIndex <= middle1 && rightIndex <= right )
65          {
66             // place smaller of two current elements into result
67             // and move to next space in arrays
```

Fig. 19.10 | Class that creates an array filled with random integers. Provides a method to sort the array with merge sort. (Part 2 of 3.)

```
68          if ( data[ leftIndex ] <= data[ rightIndex ] )
69             combined[ combinedIndex++ ] = data[ leftIndex++ ];
70          else
71             combined[ combinedIndex++ ] = data[ rightIndex++ ];
72       } // end while
73
74       // if left array is empty
75       if ( leftIndex == middle2 )
76          // copy in rest of right array
77          while ( rightIndex <= right )
78             combined[ combinedIndex++ ] = data[ rightIndex++ ];
79       else // right array is empty
80          // copy in rest of left array
81          while ( leftIndex <= middle1 )
82             combined[ combinedIndex++ ] = data[ leftIndex++ ];
83
84       // copy values back into original array
85       for ( int i = left; i <= right; i++ )
86          data[ i ] = combined[ i ];
87
88       // output merged array
89       System.out.println( "          " + subarray( left, right ) );
90       System.out.println();
91    } // end method merge
92
93    // method to output certain values in array
94    public String subarray( int low, int high )
95    {
96       StringBuilder temporary = new StringBuilder();
97
98       // output spaces for alignment
99       for ( int i = 0; i < low; i++ )
100         temporary.append( "   " );
101
102      // output elements left in array
103      for ( int i = low; i <= high; i++ )
104         temporary.append( " " + data[ i ] );
105
106      return temporary.toString();
107   } // end method subarray
108
109   // method to output values in array
110   public String toString()
111   {
112      return subarray( 0, data.length - 1 );
113   } // end method toString
114 } // end class MergeSort
```

Fig. 19.10 | Class that creates an array filled with random integers. Provides a method to sort the array with merge sort. (Part 3 of 3.)

Lines 64–72 in the `merge` method loop until the program reaches the end of either subarray. Line 68 tests which element at the beginning of the arrays is smaller. If the element in the left array is smaller, line 69 places it in position in the combined array. If the element

in the right array is smaller, line 71 places it in position in the combined array. When the `while` loop has completed (line 72), one entire subarray is placed in the combined array, but the other subarray still contains data. Line 75 tests whether the left array has reached the end. If so, lines 77–78 fill the combined array with the elements of the right array. If the left array has not reached the end, then the right array must have reached the end, and lines 81–82 fill the combined array with the elements of the left array. Finally, lines 85–86 copy the combined array into the original array. Figure 19.11 creates and uses a `MergeSort` object. The output from this program displays the splits and merges performed by merge sort, showing the progress of the sort at each step of the algorithm. It's well worth your time to step through these outputs to fully understand this elegant sorting algorithm.

```
1  // Figure 16.11: MergeSortTest.java
2  // Testing the merge sort class.
3
4  public class MergeSortTest
5  {
6     public static void main( String[] args )
7     {
8        // create object to perform merge sort
9        MergeSort sortArray = new MergeSort( 10 );
10
11       // print unsorted array
12       System.out.println( "Unsorted:" + sortArray + "\n" );
13
14       sortArray.sort(); // sort array
15
16       // print sorted array
17       System.out.println( "Sorted:  " + sortArray );
18    } // end main
19 } // end class MergeSortTest
```

```
Unsorted: 75 56 85 90 49 26 12 48 40 47

split:    75 56 85 90 49 26 12 48 40 47
          75 56 85 90 49
                         26 12 48 40 47

split:    75 56 85 90 49
          75 56 85
                   90 49

split:    75 56 85
          75 56
                85

split:    75 56
          75
             56

merge:    75
             56
          56 75
```

Fig. 19.11 | Testing the merge sort class. (Part 1 of 2.)

```
merge:    56 75
               85
          56 75 85

split:               90 49
                     90
                        49

merge:               90
                        49
                     49 90

merge:    56 75 85
                     49 90
          49 56 75 85 90

split:                        26 12 48 40 47
                              26 12 48
                                       40 47

split:                        26 12 48
                              26 12
                                    48

split:                        26 12
                              26
                                 12

merge:                        26
                                 12
                              12 26

merge:                        12 26
                                    48
                              12 26 48

split:                                 40 47
                                       40
                                          47

merge:                                 40
                                          47
                                       40 47

merge:                        12 26 48
                                       40 47
                              12 26 40 47 48

merge:    49 56 75 85 90
                              12 26 40 47 48
          12 26 40 47 48 49 56 75 85 90

Sorted:   12 26 40 47 48 49 56 75 85 90
```

Fig. 19.11 | Testing the merge sort class. (Part 2 of 2.)

Efficiency of the Merge Sort

Merge sort is *far more efficient* than either insertion or selection sort. Consider the first (non-recursive) call to `sortArray`. This results in two recursive calls to `sortArray` with subarrays each approximately half the size of the original array, and a single call to method `merge`. This

call to `merge` requires, at worst, $n - 1$ comparisons to fill the original array, which is $O(n)$. (Recall that each element in the array can be chosen by comparing one element from each of the subarrays.) The two calls to `sortArray` result in four more recursive calls to `sortArray`, each with a subarray approximately a quarter the size of the original array along with two calls to method `merge`. These two calls to method `merge` each require, at worst, $n/2 - 1$ comparisons, for a total number of comparisons of $O(n)$. This process continues, each call to `sortArray` generating two additional calls to `sortArray` and a call to `merge`, until the algorithm has *split* the array into one-element subarrays. At each level, $O(n)$ comparisons are required to *merge* the subarrays. Each level splits the size of the arrays in half, so doubling the array size requires one more level. Quadrupling the array size requires two more levels. This pattern is logarithmic and results in $\log_2 n$ levels. This results in a total efficiency of $\boldsymbol{O(n \log n)}$.

Figure 19.12 summarizes the searching and sorting algorithms covered in this chapter with the Big O for each. Figure 19.13 lists the Big O values we've covered in this chapter along with a number of values for n to highlight the differences in the growth rates.

Algorithm	Location	Big O
Searching Algorithms:		
Linear search	Section 19.2.1	$O(n)$
Binary search	Section 19.2.2	$O(\log n)$
Recursive linear search	Exercise 19.8	$O(n)$
Recursive binary search	Exercise 19.9	$O(\log n)$
Sorting Algorithms:		
Selection sort	Section 19.3.1	$O(n^2)$
Insertion sort	Section 19.3.2	$O(n^2)$
Merge sort	Section 19.3.3	$O(n \log n)$
Bubble sort	Exercises 19.5 and 19.6	$O(n^2)$

Fig. 19.12 | Searching and sorting algorithms with Big O values.

n =	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$
1	0	1	0	1
2	1	2	2	4
3	1	3	3	9
4	1	4	4	16
5	1	5	5	25
10	1	10	10	100
100	2	100	200	10,000
1000	3	1000	3000	10^6
1,000,000	6	1,000,000	6,000,000	10^{12}
1,000,000,000	9	1,000,000,000	9,000,000,000	10^{18}

Fig. 19.13 | Number of comparisons for common Big O notations.

19.4 Wrap-Up

This chapter introduced searching and sorting. We discussed two searching algorithms—linear search and binary search—and three sorting algorithms—selection sort, insertion sort and merge sort. We introduced Big O notation, which helps you analyze the efficiency of an algorithm. The next three chapters continue our discussion of dynamic data structures that can grow or shrink at execution time. Chapter 20 presents the Java API's built-in algorithms (such as searching and sorting) and generic collections. Chapter 21 demonstrates how to use Java's generics capabilities to implement generic methods and classes. Chapter 22 discusses the details of implementing generic data structures.

Summary

Section 19.1 Introduction

- Searching (p. 799) involves determining if a search key is in the data and, if so, finding its location.
- Sorting (p. 799) involves arranging data into order.

Section 19.2 Searching Algorithms

- The linear search algorithm (p. 800) searches each element in the array sequentially until it finds the correct element, or until it reaches the end of the array without finding the element.
- A major difference among searching algorithms is the amount of effort they require in order to return a result.
- Big O notation (p. 802) describes an algorithm's efficiency in terms of the work required to solve a problem. For searching and sorting algorithms typically it depends on the number of elements in the data.
- An algorithm that is $O(1)$ does not necessarily require only one comparison (p. 803). It just means that the number of comparisons does not grow as the size of the array increases.
- An $O(n)$ algorithm is referred to as having a linear run time (p. 803).
- Big O is designed to highlight dominant factors and ignore terms that become unimportant with high n values.
- Big O notation is concerned with the growth rate of algorithm run times, so constants are ignored.
- The linear search algorithm runs in $O(n)$ time.
- The worst case in linear search is that every element must be checked to determine whether the search item exists. This occurs if the search key (p. 799) is the last element in the array or is not present.
- Binary search (p. 804) is more efficient than linear search, but it requires that the array be sorted.
- The first iteration of binary search tests the middle element in the array. If this is the search key, the algorithm returns its location. If the search key is less than the middle element, the search continues with the first half of the array. If the search key is greater than the middle element, the search continues with the second half of the array. Each iteration tests the middle value of the remaining array and, if the element is not found, eliminates half of the remaining elements.
- Binary search is a more efficient searching algorithm than linear search because each comparison eliminates from consideration half of the elements in the array.
- Binary search runs in $O(\log n)$ time because each step removes half of the remaining elements; this is also known as logarithmic run time (p. 809).
- If the array size is doubled, binary search requires only one extra comparison.

Section 19.3 Sorting Algorithms

- Selection sort (p. 810) is a simple, but inefficient, sorting algorithm.
- The sort begins by selecting the smallest item and swaps it with the first element. The second iteration selects the second-smallest item (which is the smallest remaining item) and swaps it with the second element. The sort continues until the last iteration selects the second-largest element and swaps it with the second-to-last element, leaving the largest element in the last index. At the *i*th iteration of selection sort, the smallest *i* items of the whole array are sorted into the first *i* indices.
- The selection sort algorithm runs in $O(n^2)$ time (p. 803).
- The first iteration of insertion sort (p. 814) takes the second element in the array and, if it's less than the first element, swaps it with the first element. The second iteration looks at the third element and inserts it in the correct position with respect to the first two elements. After the *i*th iteration of insertion sort, the first *i* elements in the original array are sorted.
- The insertion sort algorithm runs in $O(n^2)$ time.
- Merge sort (p. 817) is a sorting algorithm that is faster, but more complex to implement, than selection sort and insertion sort. The merge sort algorithm sorts an array by splitting the array into two equal-sized subarrays, sorting each subarray recursively and merging the subarrays into one larger array.
- Merge sort's base case is an array with one element. A one-element array is already sorted, so merge sort immediately returns when it's called with a one-element array. The merge part of merge sort takes two sorted arrays and combines them into one larger sorted array.
- Merge sort performs the merge by looking at the first element in each array, which is also the smallest element in the array. Merge sort takes the smallest of these and places it in the first element of the larger array. If there are still elements in the subarray, merge sort looks at the second of these (which is now the smallest element remaining) and compares it to the first element in the other subarray. Merge sort continues this process until the larger array is filled.
- In the worst case, the first call to merge sort has to make $O(n)$ comparisons to fill the n slots in the final array.
- The merging portion of the merge sort algorithm is performed on two subarrays, each of approximately size $n/2$. Creating each of these subarrays requires $n/2 - 1$ comparisons for each subarray, or $O(n)$ comparisons total. This pattern continues as each level works on twice as many arrays, but each is half the size of the previous array.
- Similar to binary search, this halving results in log n levels for a total efficiency of $O(n \log n)$ (p. 823).

Self-Review Exercises

19.1 Fill in the blanks in each of the following statements:

a) A selection sort application would take approximately _____ times as long to run on a 128-element array as on a 32-element array.

b) The efficiency of merge sort is _____.

19.2 What key aspect of both the binary search and the merge sort accounts for the logarithmic portion of their respective Big Os?

19.3 In what sense is the insertion sort superior to the merge sort? In what sense is the merge sort superior to the insertion sort?

19.4 In the text, we say that after the merge sort splits the array into two subarrays, it then sorts these two subarrays and merges them. Why might someone be puzzled by our statement that "it then sorts these two subarrays"?

Answers to Self-Review Exercises

19.1 a) 16, because an $O(n^2)$ algorithm takes 16 times as long to sort four times as much information. b) $O(n \log n)$.

19.2 Both of these algorithms incorporate "halving"—somehow reducing something by half. The binary search eliminates from consideration one-half of the array after each comparison. The merge sort splits the array in half each time it's called.

19.3 The insertion sort is easier to understand and to program than the merge sort. The merge sort is far more efficient [$O(n \log n)$] than the insertion sort [$O(n^2)$].

19.4 In a sense, it does not really sort these two subarrays. It simply keeps splitting the original array in half until it provides a one-element subarray, which is, of course, sorted. It then builds up the original two subarrays by merging these one-element arrays to form larger subarrays, which are then merged, and so on.

Exercises

19.5 ***(Bubble Sort)*** Implement bubble sort—another simple yet inefficient sorting technique. It's called bubble sort or sinking sort because smaller values gradually "bubble" their way to the top of the array (i.e., toward the first element) like air bubbles rising in water, while the larger values sink to the bottom (end) of the array. The technique uses nested loops to make several passes through the array. Each pass compares successive pairs of elements. If a pair is in increasing order (or the values are equal), the bubble sort leaves the values as they are. If a pair is in decreasing order, the bubble sort swaps their values in the array. The first pass compares the first two elements of the array and swaps their values if necessary. It then compares the second and third elements in the array. The end of this pass compares the last two elements in the array and swaps them if necessary. After one pass, the largest element will be in the last index. After two passes, the largest two elements will be in the last two indices. Explain why bubble sort is an $O(n^2)$ algorithm.

19.6 ***(Enhanced Bubble Sort)*** Make the following simple modifications to improve the performance of the bubble sort you developed in Exercise 19.5:

a) After the first pass, the largest number is guaranteed to be in the highest-numbered arry element; after the second pass, the two highest numbers are "in place"; and so on. Instead of making nine comparisons on every pass for a ten-element array, modify the bubble sort to make eight comparisons on the second pass, seven on the third pass, and so on.

b) The data in the array may already be in proper or near-proper order, so why make nine passes if fewer will suffice? Modify the sort to check at the end of each pass whether any swaps have been made. If there were none, the data must already be sorted, so the program should terminate. If swaps have been made, at least one more pass is needed.

19.7 ***(Bucket Sort)*** A bucket sort begins with a one-dimensional array of positive integers to be sorted and a two-dimensional array of integers with rows indexed from 0 to 9 and columns indexed from 0 to $n - 1$, where n is the number of values to be sorted. Each row of the two-dimensional array is referred to as a *bucket*. Write a class named `BucketSort` containing a method called `sort` that operates as follows:

a) Place each value of the one-dimensional array into a row of the bucket array, based on the value's "ones" (rightmost) digit. For example, 97 is placed in row 7, 3 is placed in row 3 and 100 is placed in row 0. This procedure is called a *distribution pass*.

b) Loop through the bucket array row by row, and copy the values back to the original array. This procedure is called a *gathering pass*. The new order of the preceding values in the one-dimensional array is 100, 3 and 97.

c) Repeat this process for each subsequent digit position (tens, hundreds, thousands, etc.). On the second (tens digit) pass, 100 is placed in row 0, 3 is placed in row 0 (because 3

has no tens digit) and 97 is placed in row 9. After the gathering pass, the order of the values in the one-dimensional array is 100, 3 and 97. On the third (hundreds digit) pass, 100 is placed in row 1, 3 is placed in row 0 and 97 is placed in row 0 (after the 3). After this last gathering pass, the original array is in sorted order.

The two-dimensional array of buckets is 10 times the length of the integer array being sorted. This sorting technique provides better performance than a bubble sort, but requires much more memory—the bubble sort requires space for only one additional element of data. This comparison is an example of the space/time trade-off: The bucket sort uses more memory than the bubble sort, but performs better. This version of the bucket sort requires copying all the data back to the original array on each pass. Another possibility is to create a second two-dimensional bucket array and repeatedly swap the data between the two bucket arrays.

19.8 ***(Recursive Linear Search)*** Modify Fig. 19.2 to use recursive method `recursiveLinearSearch` to perform a linear search of the array. The method should receive the search key and starting index as arguments. If the search key is found, return its index in the array; otherwise, return `-1`. Each call to the recursive method should check one index in the array.

19.9 ***(Recursive Binary Search)*** Modify Fig. 19.4 to use recursive method `recursiveBinarySearch` to perform a binary search of the array. The method should receive the search key, starting index and ending index as arguments. If the search key is found, return its index in the array. If the search key is not found, return `-1`.

19.10 ***(Quicksort)*** The recursive sorting technique called quicksort uses the following basic algorithm for a one-dimensional array of values:

a) *Partitioning Step*: Take the first element of the unsorted array and determine its final location in the sorted array (i.e., all values to the left of the element in the array are less than the element, and all values to the right of the element in the array are greater than the element—we show how to do this below). We now have one element in its proper location and two unsorted subarrays.

b) *Recursive Step*: Perform *Step 1* on each unsorted subarray. Each time *Step 1* is performed on a subarray, another element is placed in its final location of the sorted array, and two unsorted subarrays are created. When a subarray consists of one element, that element is in its final location (because a one-element array is already sorted).

The basic algorithm seems simple enough, but how do we determine the final position of the first element of each subarray? As an example, consider the following set of values (the element in bold is the partitioning element—it will be placed in its final location in the sorted array):

37 2 6 4 89 8 10 12 68 45

Starting from the rightmost element of the array, compare each element with **37** until an element less than **37** is found; then swap **37** and that element. The first element less than **37** is 12, so **37** and 12 are swapped. The new array is

12 2 6 4 89 8 10 **37** 68 45

Element 12 is in italics to indicate that it was just swapped with **37**.

Starting from the left of the array, but beginning with the element after 12, compare each element with **37** until an element greater than **37** is found—then swap **37** and that element. The first element greater than **37** is 89, so **37** and 89 are swapped. The new array is

12 2 6 4 **37** 8 10 *89* 68 45

Starting from the right, but beginning with the element before 89, compare each element with **37** until an element less than **37** is found—then swap **37** and that element. The first element less than **37** is 10, so **37** and 10 are swapped. The new array is

12 2 6 4 *10* 8 **37** 89 68 45

Starting from the left, but beginning with the element after 10, compare each element with **37** until an element greater than **37** is found—then swap **37** and that element. There are no more elements greater than **37**, so when we compare **37** with itself, we know that **37** has been placed in its final location in the sorted array. Every value to the left of **37** is smaller than it, and every value to the right of **37** is larger than it.

Once the partition has been applied on the previous array, there are two unsorted subarrays. The subarray with values less than 37 contains 12, 2, 6, 4, 10 and 8. The subarray with values greater than 37 contains 89, 68 and 45. The sort continues recursively, with both subarrays being partitioned in the same manner as the original array.

Based on the preceding discussion, write recursive method `quickSortHelper` to sort a one-dimensional integer array. The method should receive as arguments a starting index and an ending index on the original array being sorted.

20

Generic Collections



I think this is the most extraordinary collection of talent, of human knowledge, that has ever been gathered together at the White House—with the possible exception of when Thomas Jefferson dined alone.
—John F. Kennedy

Journey over all the universe in a map.
—Miguel de Cervantes

Not by age but by capacity is wisdom acquired.
—Titus Maccius Plautus

Objectives

In this chapter you'll learn:

- What collections are.
- To use class `Arrays` for array manipulations.
- The type-wrapper classes that enable programs to process primitive data values as objects.
- To use the prebuilt generic data structures of the collections framework.
- To use iterators to "walk through" a collection.
- To use persistent hash tables manipulated with objects of class `Properties`.
- To use synchronization and modifiability wrappers.

Outline

20.1 Introduction
20.2 Collections Overview
20.3 Type-Wrapper Classes for Primitive Types
20.4 Autoboxing and Auto-Unboxing
20.5 Interface `Collection` and Class `Collections`
20.6 Lists
20.6.1 `ArrayList` and `Iterator`
20.6.2 `LinkedList`
20.7 Collections Methods
20.7.1 Method `sort`
20.7.2 Method `shuffle`
20.7.3 Methods `reverse`, `fill`, `copy`, `max` and `min`
20.7.4 Method `binarySearch`
20.7.5 Methods `addAll`, `frequency` and `disjoint`
20.8 `Stack` Class of Package `java.util`
20.9 Class `PriorityQueue` and `Interface Queue`
20.10 Sets
20.11 Maps
20.12 `Properties` Class
20.13 Synchronized Collections
20.14 Unmodifiable Collections
20.15 Abstract Implementations
20.16 Wrap-Up

Summary | *Self-Review Exercises* | *Answers to Self-Review Exercises* | *Exercises*

20.1 Introduction

In Section 7.14, we introducted the generic `ArrayList` collection—a dynamically resizable array-like data structure that stores references to objects of a type that you specify when you create the `ArrayList`. In this chapter, we continue our discussion of the Java **collections framework**, which contains many other prebuilt generic data structures and various methods for manipulating them.

Because you specify the type to store in a collection at compile time, generic collections provide compile-time type safety that allows the compiler to catch attempts to use invalid types. For example, you cannot store `Employees` in a collection of `Strings`. Some examples of collections are the cards you hold in a card game, your favorite songs stored in your computer, the members of a sports team and the real-estate records in your local registry of deeds (which map book numbers and page numbers to property owners).

We discuss the collections-framework interfaces that declare the capabilities of each collection type, the implementation classes, the methods that process collection objects, and the so-called **iterators** that "walk through" collections. This chapter provides an introduction to the collections framework. For complete details, visit `download.oracle.com/javase/6/docs/technotes/guides/collections/index.html`.

20.2 Collections Overview

A **collection** is a data structure—actually, an object—that can hold references to other objects. Usually, collections contain references to objects that are all of the same type. The collections-framework interfaces declare the operations to be performed generically on various types of collections. Figure 20.1 lists some of the interfaces of the collections framework. Several implementations of these interfaces are provided within the framework. You may also provide implementations specific to your own requirements.

Interface	Description
`Collection`	The root interface in the collections hierarchy from which interfaces `Set`, `Queue` and `List` are derived.
`Set`	A collection that does not contain duplicates.
`List`	An ordered collection that can contain duplicate elements.
`Map`	A collection that associates keys to values and cannot contain duplicate keys.
`Queue`	Typically a first-in, first-out collection that models a waiting line; other orders can be specified.

Fig. 20.1 | Some collections-framework interfaces.

The classes and interfaces of the collections framework are members of package `java.util`. In the next section, we begin our discussion by examining the collections-framework capabilities for array manipulation. In earlier versions of Java, the classes in the collections framework stored and manipulated `Object` references, enabling you to store *any* object in a collection. One inconvenient aspect of this approach occurs when retrieving `Object` references from a collection. A program normally needs to process specific types of objects. As a result, the `Object` references obtained from a collection typically need to be *cast* to an appropriate type to allow the program to process the objects correctly.

In Java SE 5, the collections framework was enhanced with the *generics* capabilities we introduced in Chapter 7 when discussing generic `ArrayLists`. This means that you can specify the *exact type* that will be stored in a collection. You also receive the benefits of compile-time type checking—the compiler ensures that you're using appropriate types with your collection and, if not, issues compile-time error messages. Also, once you specify the type stored in a collection, any reference you retrieve from the collection will have the specified type. This eliminates the need for explicit type casts that can throw `ClassCastExceptions` if the referenced object is *not* of the appropriate type. In addition, the generic collections are *backward compatible* with Java code that was written before generics were introduced.

20.3 Type-Wrapper Classes for Primitive Types

Each primitive type (listed in Appendix) has a corresponding **type-wrapper class** (in package `java.lang`). These classes are called **`Boolean`**, **`Byte`**, **`Character`**, **`Double`**, **`Float`**, **`Integer`**, **`Long`** and **`Short`**. These enable you to manipulate primitive-type values as objects. The data structures that we reuse or develop in Chapters 20–22 manipulate and share *objects*—they cannot manipulate variables of primitive types. However, they can manipulate objects of the type-wrapper classes, because every class ultimately derives from `Object`.

Each of the numeric type-wrapper classes—`Byte`, `Short`, `Integer`, `Long`, `Float` and `Double`—extends class `Number`. Also, the type-wrapper classes are `final` classes, so you cannot extend them.

Primitive types do not have methods, so the methods related to a primitive type are located in the corresponding type-wrapper class (e.g., method `parseInt`, which converts a `String` to an `int` value, is located in class `Integer`). If you need to manipulate a primitive value in your program, first refer to the documentation for the type-wrapper classes—the method you need might already be declared.

20.4 Autoboxing and Auto-Unboxing

Prior to Java SE 5, if you wanted to insert a primitive value into a data structure, you had to create a new object of the corresponding type-wrapper class, then insert it in the collection. Similarly, if you wanted to retrieve an object of a type-wrapper class from a collection and manipulate its primitive value, you had to invoke a method on the object to obtain its corresponding primitive-type value. For example, suppose you wanted to add an `int` to an array that stores only references to `Integer` objects. Prior to Java SE 5, you'd be required to "wrap" an `int` value in an `Integer` object before adding the integer to the array and to "unwrap" the `int` value to retrieve it from the array, as in

```
Integer[] integerArray = new Integer[ 5 ]; // create integerArray

// assign Integer 10 to integerArray[ 0 ]
integerArray[ 0 ] = new Integer( 10 );

// get int value of Integer
int value = integerArray[ 0 ].intValue();
```

Notice that the `int` primitive value 10 is used to initialize an `Integer` object. This achieves the desired result but requires extra code and is cumbersome. We then need to invoke method `intValue` of class `Integer` to obtain the `int` value in the `Integer` object.

Java SE 5 introduced two new conversions—the *boxing conversion* and the *unboxing conversion*—to simplify converting between primitive-type values and type-wrapper objects with no additional coding on the part of the programmer. A **boxing conversion** converts a value of a primitive type to an object of the corresponding type-wrapper class. An **unboxing conversion** converts an object of a type-wrapper class to a value of the corresponding primitive type. These conversions can be performed automatically (called **autoboxing** and **auto-unboxing**). For example, the previous statements can be rewritten as

```
Integer[] integerArray = new Integer[ 5 ]; // create integerArray
integerArray[ 0 ] = 10; // assign Integer 10 to integerArray[ 0 ]
int value = integerArray[ 0 ]; // get int value of Integer
```

In this case, *autoboxing* occurs when assigning an `int` value (`10`) to `integerArray[0]`, because `integerArray` stores references to `Integer` objects, not `int` values. *Auto-unboxing* occurs when assigning `integerArray[0]` to `int` variable `value`, because variable `value` stores an `int` value, not a reference to an `Integer` object. Boxing conversions also occur in conditions, which can evaluate to primitive `boolean` values or `Boolean` objects. Many of the examples in Chapters 20–22 use these conversions to store primitive values in and retrieve them from data structures.

20.5 Interface `Collection` and Class `Collections`

Interface **`Collection`** is the root interface in the collection hierarchy from which interfaces `Set`, `Queue` and `List` are derived. Interface **`Set`** defines a collection that does not contain duplicates. Interface **`Queue`** defines a collection that represents a waiting line—typically, insertions are made at the back of a queue and deletions from the front, though other orders can be specified. We discuss `Queue` and `Set` in Sections 20.9–20.10. Interface `Collection` contains **bulk operations** (i.e., operations performed on an entire collection) for operations such as adding, clearing and comparing objects (or elements) in a collection. A `Collection` can also be converted to an array. In addition, interface `Collection` provides

a method that returns an **`Iterator`** object, which allows a program to walk through the collection and remove elements from it during the iteration. We discuss class `Iterator` in Section 20.6.1. Other methods of interface `Collection` enable a program to determine a collection's size and whether a collection is empty.



Software Engineering Observation 20.1

`Collection` is used commonly as a parameter type in methods to allow polymorphic processing of all objects that implement interface `Collection`.



Software Engineering Observation 20.2

Most collection implementations provide a constructor that takes a `Collection` argument, thereby allowing a new collection to be constructed containing the elements of the specified collection.

Class **`Collections`** provides `static` methods that search, sort and perform other operations on collections. Section 20.7 discusses more about the methods that are available in class `Collections`. We also cover this class's **wrapper methods** that enable you to treat a collection as a *synchronized collection* (Section 20.13) or an *unmodifiable collection* (Section 20.14). Unmodifiable collections are useful when clients of a class need to view a collection's elements, but they should not be allowed to modify the collection by adding and removing elements. Synchronized collections are for use with multithreading (discussed in Chapter 26), which enables programs to perform operations in parallel. When two or more threads of a program *share* a collection, problems might occur. As an analogy, consider a traffic intersection. If all cars were allowed to access the intersection at the same time, collisions might occur. For this reason, traffic lights are provided to control access to the intersection. Similarly, we can *synchronize* access to a collection to ensure that only *one* thread manipulates the collection at a time. The synchronization wrapper methods of class `Collections` return synchronized versions of collections that can be shared among threads in a program.

20.6 Lists

A `List` (sometimes called a **sequence**) is an ordered `Collection` that can contain duplicate elements. Like array indices, `List` indices are zero based (i.e., the first element's index is zero). In addition to the methods inherited from `Collection`, `List` provides methods for manipulating elements via their indices, manipulating a specified range of elements, searching for elements and obtaining a **`ListIterator`** to access the elements.

Interface `List` is implemented by several classes, including **`ArrayList`**, **`LinkedList`** and **`Vector`**. *Autoboxing* occurs when you add primitive-type values to objects of these classes, because they store only *references* to objects. Class `ArrayList` and `Vector` are resizable-array implementations of `List`. Inserting an element between existing elements of an `ArrayList` or `Vector` is an *inefficient* operation—all elements after the new one must be moved out of the way, which could be an expensive operation in a collection with a large number of elements. A `LinkedList` enables efficient insertion (or removal) of elements in the middle of a collection. We discuss the architecture of linked lists in Chapter 22.

`ArrayList` and `Vector` have nearly identical behaviors. `Vector`s are synchronized by default, whereas `ArrayList`s are not. Also, class `Vector` is from Java 1.0, before the collections framework was added to Java. As such, `Vector` has some methods that are not part of

interface `List` and are not implemented in class `ArrayList` but perform identical tasks. For example, `Vector` methods `addElement` and `add` both append an element to a `Vector`, but only method `add` is specified in interface `List` and implemented by `ArrayList`. *Unsynchronized collections provide better performance than synchronized ones.* For this reason, `ArrayList` is typically preferred over `Vector` in programs that do not share a collection among threads. Separately, the Java collections API provides *synchronization wrappers* (Section 20.13) that can be used to add synchronization to the unsynchronized collections, and several powerful synchronized collections are available in the Java concurrency APIs.



Performance Tip 20.1

`ArrayLists` behave like `Vectors` without synchronization and therefore execute faster than `Vectors`, because `ArrayLists` do not have the overhead of thread synchronization.



Software Engineering Observation 20.3

`LinkedLists` can be used to create stacks, queues and deques (double-ended queues, pronounced "decks"). The collections framework provides implementations of some of these data structures.

The following three subsections demonstrate the `List` and `Collection` capabilities. Section 20.6.1 removes elements from an `ArrayList` with an `Iterator`. Section 20.6.2 uses `ListIterator` and several `List`- and `LinkedList`-specific methods.

20.6.1 ArrayList and Iterator

Figure 20.2 uses an `ArrayList` (introduced in Section 7.14) to demonstrate several capabilities of interface `Collection`. The program places two `Color` arrays in `ArrayLists` and uses an `Iterator` to remove elements in the second `ArrayList` collection from the first.

```
1  // Fig. 20.2: CollectionTest.java
2  // Collection interface demonstrated via an ArrayList object.
3  import java.util.List;
4  import java.util.ArrayList;
5  import java.util.Collection;
6  import java.util.Iterator;
7
8  public class CollectionTest
9  {
10    public static void main( String[] args )
11    {
12       // add elements in colors array to list
13       String[] colors = { "MAGENTA", "RED", "WHITE", "BLUE", "CYAN" };
14       List< String > list = new ArrayList< String >();
15
16       for ( String color : colors )
17          list.add( color ); // adds color to end of list
18
19       // add elements in removeColors array to removeList
20       String[] removeColors = { "RED", "WHITE", "BLUE" };
```

Fig. 20.2 | `Collection` interface demonstrated via an `ArrayList` object. (Part 1 of 2.)

```
21         List< String > removeList = new ArrayList< String >();
22
23         for ( String color : removeColors )
24            removeList.add( color );
25
26         // output list contents
27         System.out.println( "ArrayList: " );
28
29         for ( int count = 0; count < list.size(); count++ )
30            System.out.printf( "%s ", list.get( count ) );
31
32         // remove from list the colors contained in removeList
33         removeColors( list, removeList );
34
35         // output list contents
36         System.out.println( "\n\nArrayList after calling removeColors: " );
37
38         for ( String color : list )
39            System.out.printf( "%s ", color );
40      } // end main
41
42      // remove colors specified in collection2 from collection1
43      private static void removeColors( Collection< String > collection1,
44         Collection< String > collection2 )
45      {
46         // get iterator
47         Iterator< String > iterator = collection1.iterator();
48
49         // loop while collection has items
50         while ( iterator.hasNext() )
51         {
52            if ( collection2.contains( iterator.next() ) )
53               iterator.remove(); // remove current Color
54         } // end while
55      } // end method removeColors
56   } // end class CollectionTest
```

```
ArrayList:
MAGENTA RED WHITE BLUE CYAN

ArrayList after calling removeColors:
MAGENTA CYAN
```

Fig. 20.2 | `Collection` interface demonstrated via an `ArrayList` object. (Part 2 of 2.)

Lines 13 and 20 declare and initialize `String` arrays `colors` and `removeColors`. Lines 14 and 21 create `ArrayList<String>` objects and assign their references to `List<String>` variables `list` and `removeList`, respectively. Recall that `ArrayList` is a *generic* class, so we can specify a type argument (`String` in this case) to indicate the type of the elements in each list. We refer to the `ArrayLists` in this example via `List` variables. This makes our code more flexible and easier to modify. If we later decide that `LinkedLists` would be more appropriate, we'll need to modify only lines 14 and 21 where we created the `ArrayList` objects.

Lines 16–17 populate `list` with `String`s stored in array `colors`, and lines 23–24 populate `removeList` with `String`s stored in array `removeColors` using **`List` method `add`**. Lines 29–30 output each element of `list`. Line 29 calls **`List` method `size`** to get the number of elements in the `ArrayList`. Line 30 uses **`List` method `get`** to retrieve individual element values. Lines 29–30 also could have used the enhanced `for` statement (which we'll demonstrate with collections in other examples).

Line 33 calls method `removeColors` (lines 43–55), passing `list` and `removeList` as arguments. Method `removeColors` deletes the `String`s in `removeList` from the `String`s in `list`. Lines 38–39 print `list`'s elements after `removeColors` completes its task.

Method `removeColors` declares two `Collection<String>` parameters (lines 43–44) that allow any two `Collection`s containing strings to be passed as arguments to this method. The method accesses the elements of the first `Collection` (`collection1`) via an `Iterator`. Line 47 calls **`Collection` method `iterator`** to get an `Iterator` for the `Collection`. Interfaces `Collection` and `Iterator` are generic types. The loop-continuation condition (line 50) calls **`Iterator` method `hasNext`** to determine whether the `Collection` contains more elements. Method `hasNext` returns `true` if another element exists and `false` otherwise.

The `if` condition in line 52 calls **`Iterator` method `next`** to obtain a reference to the next element, then uses method **`contains`** of the second `Collection` (`collection2`) to determine whether `collection2` contains the element returned by `next`. If so, line 53 calls **`Iterator` method `remove`** to remove the element from the `Collection` `collection1`.



Common Programming Error 20.1

If a collection is modified by one of its methods after an iterator is created for that collection, the iterator immediately becomes invalid—operations performed with the iterator after this point throw `ConcurrentModificationException`s. For this reason, iterators are said to be "fail fast."

New in Java SE 7: Type Inference with the <> Notation

Lines 14 and 21 specify the type stored in the `ArrayList` (that is, `String`) on the left and right sides of the initialization statements. Java SE 7 supports type inferencing with the <> notation in statements that declare and create generic type variables and objects. For example, line 14 can be written as:

```
List< String > list = new ArrayList<>();
```

In this case, Java uses the type in angle brackets on the left of the declaration (that is, `String`) as the type stored int the `ArrayList` created on the right side of the declaration.

20.6.2 `LinkedList`

Figure 20.3 demonstrates various operations on `LinkedList`s. The program creates two `LinkedList`s of `String`s. The elements of one `List` are added to the other. Then all the `String`s are converted to uppercase, and a range of elements is deleted.

```
1  // Fig. 20.3: ListTest.java
2  // Lists, LinkedLists and ListIterators.
3  import java.util.List;
```

Fig. 20.3 | `List`s, `LinkedList`s and `ListIterator`s. (Part 1 of 3.)

```
4  import java.util.LinkedList;
5  import java.util.ListIterator;
6
7  public class ListTest
8  {
9     public static void main( String[] args )
10    {
11       // add colors elements to list1
12       String[] colors =
13          { "black", "yellow", "green", "blue", "violet", "silver" };
14       List< String > list1 = new LinkedList< String >();
15
16       for ( String color : colors )
17          list1.add( color );
18
19       // add colors2 elements to list2
20       String[] colors2 =
21          { "gold", "white", "brown", "blue", "gray", "silver" };
22       List< String > list2 = new LinkedList< String >();
23
24       for ( String color : colors2 )
25          list2.add( color );
26
27       list1.addAll( list2 ); // concatenate lists
28       list2 = null; // release resources
29       printList( list1 ); // print list1 elements
30
31       convertToUppercaseStrings( list1 ); // convert to uppercase string
32       printList( list1 ); // print list1 elements
33
34       System.out.print( "\nDeleting elements 4 to 6..." );
35       removeItems( list1, 4, 7 ); // remove items 4-6 from list
36       printList( list1 ); // print list1 elements
37       printReversedList( list1 ); // print list in reverse order
38    } // end main
39
40    // output List contents
41    private static void printList( List< String > list )
42    {
43       System.out.println( "\nlist: " );
44
45       for ( String color : list )
46          System.out.printf( "%s ", color );
47
48       System.out.println();
49    } // end method printList
50
51    // locate String objects and convert to uppercase
52    private static void convertToUppercaseStrings( List< String > list )
53    {
54       ListIterator< String > iterator = list.listIterator();
55
```

Fig. 20.3 | `Lists`, `LinkedLists` and `ListIterators`. (Part 2 of 3.)

```
56          while ( iterator.hasNext() )
57          {
58             String color = iterator.next(); // get item
59             iterator.set( color.toUpperCase() ); // convert to upper case
60          } // end while
61       } // end method convertToUppercaseStrings
62
63       // obtain sublist and use clear method to delete sublist items
64       private static void removeItems( List< String > list,
65          int start, int end )
66       {
67          list.subList( start, end ).clear(); // remove items
68       } // end method removeItems
69
70       // print reversed list
71       private static void printReversedList( List< String > list )
72       {
73          ListIterator< String > iterator = list.listIterator( list.size() );
74
75          System.out.println( "\nReversed List:" );
76
77          // print list in reverse order
78          while ( iterator.hasPrevious() )
79             System.out.printf( "%s ", iterator.previous() );
80       } // end method printReversedList
81    } // end class ListTest
```

```
list:
black yellow green blue violet silver gold white brown blue gray silver

list:
BLACK YELLOW GREEN BLUE VIOLET SILVER GOLD WHITE BROWN BLUE GRAY SILVER

Deleting elements 4 to 6...
list:
BLACK YELLOW GREEN BLUE WHITE BROWN BLUE GRAY SILVER

Reversed List:
SILVER GRAY BLUE BROWN WHITE BLUE GREEN YELLOW BLACK
```

Fig. 20.3 | `List`s, `LinkedList`s and `ListIterator`s. (Part 3 of 3.)

Lines 14 and 22 create `LinkedList`s `list1` and `list2` of type `String`. `LinkedList` is a generic class that has one type parameter for which we specify the type argument `String` in this example. Lines 16–17 and 24–25 call `List` method `add` to append elements from arrays `colors` and `colors2` to the end of `list1` and `list2`, respectively.

Line 27 calls **`List` method `addAll`** to append all elements of `list2` to the end of `list1`. Line 28 sets `list2` to `null`, so the `LinkedList` to which `list2` referred can be garbage collected. Line 29 calls method `printList` (lines 41–49) to output `list1`'s contents. Line 31 calls method `convertToUppercaseStrings` (lines 52–61) to convert each `String` element to uppercase, then line 32 calls `printList` again to display the modified `String`s. Line 35 calls method `removeItems` (lines 64–68) to remove the elements starting at index `4` up to, but not including, index `7` of the list. Line 37 calls method `printReversedList` (lines 71–80) to print the list in reverse order.

Method `convertToUppercaseStrings`

Method `convertToUppercaseStrings` (lines 52–61) changes lowercase `String` elements in its `List` argument to uppercase `String`s. Line 54 calls **`List` method `listIterator`** to get the `List`'s **bidirectional iterator** (i.e., one that can traverse a `List` backward or forward). `ListIterator` is also a generic class. In this example, the `ListIterator` references `String` objects, because method `listIterator` is called on a `List` of `String`s. Line 56 calls method `hasNext` to determine whether the `List` contains another element. Line 58 gets the next `String` in the `List`. Line 59 calls **`String` method `toUpperCase`** to get an uppercase version of the `String` and calls **`ListIterator` method `set`** to replace the current `String` to which `iterator` refers with the `String` returned by method `toUpperCase`. Like method `toUpperCase`, **`String` method `toLowerCase`** returns a lowercase version of the `String`.

Method `removeItems`

Method `removeItems` (lines 64–68) removes a range of items from the list. Line 67 calls **`List` method `subList`** to obtain a portion of the `List` (called a **sublist**). This is a so-called **range-view method**, which enables the program to view a portion of the list. The sublist is simply a view into the `List` on which `subList` is called. Method `subList` takes as arguments the beginning and ending index for the sublist. The ending index is not part of the range of the sublist. In this example, line 35 passes `4` for the beginning index and `7` for the ending index to `subList`. The sublist returned is the set of elements with indices `4` through `6`. Next, the program calls **`List` method `clear`** on the sublist to remove the elements of the sublist from the `List`. Any changes made to a sublist are also made to the original `List`.

Method `printReversedList`

Method `printReversedList` (lines 71–80) prints the list backward. Line 73 calls `List` method `listIterator` with the starting position as an argument (in our case, the last element in the list) to get a bidirectional iterator for the list. **`List` method `size`** returns the number of items in the `List`. The `while` condition (line 78) calls **`ListIterator`'s `hasPrevious` method** to determine whether there are more elements while traversing the list backward. Line 79 calls **`ListIterator`'s `previous` method** to get the previous element from the list and outputs it to the standard output stream.

Views into Collections and `Arrays` *Method* `asList`

An important feature of the collections framework is the ability to manipulate the elements of one collection type (such as a set) through a different collection type (such as a list), regardless of the collection's internal implementation. The set of `public` methods through which collections are manipulated is called a **view**.

Class `Arrays` provides `static` method **`asList`** to view an array (sometimes called the **backing array**) as a **`List`** collection. A `List` view allows you to manipulate the array as if it were a list. This is useful for adding the elements in an array to a collection and for sorting array elements. The next example demonstrates how to create a `LinkedList` with a `List` view of an array, because we cannot pass the array to a `LinkedList` constructor. Sorting array elements with a `List` view is demonstrated in Fig. 20.7. Any modifications made through the `List` view change the array, and any modifications made to the array change the `List` view. The only operation permitted on the view returned by `asList` is *set*, which changes the value of the view and the backing array. Any other attempts to change the view (such as adding or removing elements) result in an **`UnsupportedOperationException`**.

Viewing Arrays as `List`s and Converting `List`s to Arrays

Figure 20.4 uses `Arrays` method `asList` to view an array as a `List` and uses **`List` method `toArray`** to get an array from a `LinkedList` collection. The program calls method `asList` to create a `List` view of an array, which is used to initialize a `LinkedList` object, then adds a series of strings to the `LinkedList` and calls method `toArray` to obtain an array containing references to the `String`s.

```
1  // Fig. 20.4: UsingToArray.java
2  // Viewing arrays as Lists and converting Lists to arrays.
3  import java.util.LinkedList;
4  import java.util.Arrays;
5
6  public class UsingToArray
7  {
8     // creates a LinkedList, adds elements and converts to array
9     public static void main( String[] args )
10    {
11       String[] colors = { "black", "blue", "yellow" };
12
13       LinkedList< String > links =
14          new LinkedList< String >( Arrays.asList( colors ) );
15
16       links.addLast( "red" ); // add as last item
17       links.add( "pink" ); // add to the end
18       links.add( 3, "green" ); // add at 3rd index
19       links.addFirst( "cyan" ); // add as first item
20
21       // get LinkedList elements as an array
22       colors = links.toArray( new String[ links.size() ] );
23
24       System.out.println( "colors: " );
25
26       for ( String color : colors )
27          System.out.println( color );
28    } // end main
29 } // end class UsingToArray
```

```
colors:
cyan
black
blue
yellow
green
red
pink
```

Fig. 20.4 | Viewing arrays as `List`s and converting `List`s to arrays.

Lines 13–14 construct a `LinkedList` of `String`s containing the elements of array `colors`. Line 14 uses `Arrays` method `asList` to return a `List` view of the array, then uses that to initialize the `LinkedList` with its constructor that receives a `Collection` as an argument (a `List` *is a* `Collection`). Line 16 calls **`LinkedList` method `addLast`** to add `"red"`

to the end of `links`. Lines 17–18 call **`LinkedList` method `add`** to add `"pink"` as the last element and `"green"` as the element at index 3 (i.e., the fourth element). Method `addLast` (line 16) functions identically to method `add` (line 17). Line 19 calls **`LinkedList` method `addFirst`** to add `"cyan"` as the new first item in the `LinkedList`. The `add` operations are permitted because they operate on the `LinkedList` object, not the view returned by `asList`. [*Note:* When `"cyan"` is added as the first element, `"green"` becomes the fifth element in the `LinkedList`.]

Line 22 calls the `List` interface's `toArray` method to get a `String` array from `links`. The array is a copy of the list's elements—modifying the array's contents does *not* modify the list. The array passed to method `toArray` is of the same type that you'd like method `toArray` to return. If the number of elements in that array is greater than or equal to the number of elements in the `LinkedList`, `toArray` copies the list's elements into its array argument and returns that array. If the `LinkedList` has more elements than the number of elements in the array passed to `toArray`, `toArray` allocates a new array of the same type it receives as an argument, copies the list's elements into the new array and returns the new array.



Common Programming Error 20.2

Passing an array that contains data to `toArray` can cause logic errors. If the number of elements in the array is smaller than the number of elements in the list on which `toArray` is called, a new array is allocated to store the list's elements—without preserving the array argument's elements. *If the number of elements in the array is greater than the number of elements in the list, the elements of the array (starting at index zero) are overwritten with the list's elements. Array elements that are not overwritten retain their values.*

20.7 Collections Methods

Class `Collections` provides several high-performance algorithms for manipulating collection elements. The algorithms (Fig. 20.5) are implemented as `static` methods. The methods `sort`, `binarySearch`, `reverse`, `shuffle`, `fill` and `copy` operate on `Lists`. Methods `min`, `max`, `addAll`, `frequency` and `disjoint` operate on `Collections`.

Method	Description
`sort`	Sorts the elements of a `List`.
`binarySearch`	Locates an object in a `List`.
`reverse`	Reverses the elements of a `List`.
`shuffle`	Randomly orders a `List`'s elements.
`fill`	Sets every `List` element to refer to a specified object.
`copy`	Copies references from one `List` into another.
`min`	Returns the smallest element in a `Collection`.
`max`	Returns the largest element in a `Collection`.
`addAll`	Appends all elements in an array to a `Collection`.
`frequency`	Calculates how many collection elements are equal to the specified element.
`disjoint`	Determines whether two collections have no elements in common.

Fig. 20.5 | `Collections` methods.



Software Engineering Observation 20.4

The collections framework methods are polymorphic. That is, each can operate on objects that implement specific interfaces, regardless of the underlying implementations.

20.7.1 Method sort

Method `sort` sorts the elements of a `List`, which must implement the **`Comparable` interface**. The order is determined by the natural order of the elements' type as implemented by a `compareTo` method. Method `compareTo` is declared in interface `Comparable` and is sometimes called the **natural comparison method**. The `sort` call may specify as a second argument a **`Comparator`** object that determines an alternative ordering of the elements.

Sorting in Ascending Order

Figure 20.6 uses `Collections` method `sort` to order the elements of a `List` in ascending order (line 17). Recall that `List` is a generic type and accepts one type argument that specifies the list element type—line 14 creates `list` as a `List` of `String`s. Lines 15 and 20 each use an implicit call to the `list`'s `toString` method to output the list contents in the format shown in the output.

```
1  // Fig. 20.6: Sort1.java
2  // Collections method sort.
3  import java.util.List;
4  import java.util.Arrays;
5  import java.util.Collections;
6
7  public class Sort1
8  {
9     public static void main( String[] args )
10    {
11       String[] suits = { "Hearts", "Diamonds", "Clubs", "Spades" };
12
13       // Create and display a list containing the suits array elements
14       List< String > list = Arrays.asList( suits ); // create List
15       System.out.printf( "Unsorted array elements: %s\n", list );
16
17       Collections.sort( list ); // sort ArrayList
18
19       // output list
20       System.out.printf( "Sorted array elements: %s\n", list );
21    } // end main
22 } // end class Sort1
```

```
Unsorted array elements: [Hearts, Diamonds, Clubs, Spades]
Sorted array elements: [Clubs, Diamonds, Hearts, Spades]
```

Fig. 20.6 | `Collections` method `sort`.

Sorting in Descending Order

Figure 20.7 sorts the same list of strings used in Fig. 20.6 in descending order. The example introduces the `Comparator` interface, which is used for sorting a `Collection`'s elements in a different order. Line 18 calls `Collections`'s method `sort` to order the `List` in de-

scending order. The `static` **`Collections` method `reverseOrder`** returns a `Comparator` object that orders the collection's elements in reverse order.

```
1  // Fig. 20.7: Sort2.java
2  // Using a Comparator object with method sort.
3  import java.util.List;
4  import java.util.Arrays;
5  import java.util.Collections;
6
7  public class Sort2
8  {
9     public static void main( String[] args )
10    {
11       String[] suits = { "Hearts", "Diamonds", "Clubs", "Spades" };
12
13       // Create and display a list containing the suits array elements
14       List< String > list = Arrays.asList( suits ); // create List
15       System.out.printf( "Unsorted array elements: %s\n", list );
16
17       // sort in descending order using a comparator
18       Collections.sort( list, Collections.reverseOrder() );
19
20       // output List elements
21       System.out.printf( "Sorted list elements: %s\n", list );
22    } // end main
23 } // end class Sort2
```

```
Unsorted array elements: [Hearts, Diamonds, Clubs, Spades]
Sorted list elements: [Spades, Hearts, Diamonds, Clubs]
```

Fig. 20.7 | `Collections` method `sort` with a `Comparator` object.

Sorting with a `Comparator`

Figure 20.8 creates a custom `Comparator` class, named `TimeComparator`, that implements interface `Comparator` to compare two `Time2` objects. Class `Time2`, declared in Fig. 8.5, represents times with hours, minutes and seconds.

```
1  // Fig. 20.8: TimeComparator.java
2  // Custom Comparator class that compares two Time2 objects.
3  import java.util.Comparator;
4
5  public class TimeComparator implements Comparator< Time2 >
6  {
7     public int compare( Time2 time1, Time2 time2 )
8     {
9        int hourCompare = time1.getHour() - time2.getHour(); // compare hour
10
11       // test the hour first
12       if ( hourCompare != 0 )
13          return hourCompare;
14
```

Fig. 20.8 | Custom `Comparator` class that compares two `Time2` objects. (Part 1 of 2.)

```
15         int minuteCompare =
16            time1.getMinute() - time2.getMinute(); // compare minute
17
18         // then test the minute
19         if ( minuteCompare != 0 )
20            return minuteCompare;
21
22         int secondCompare =
23            time1.getSecond() - time2.getSecond(); // compare second
24
25         return secondCompare; // return result of comparing seconds
26      } // end method compare
27   } // end class TimeComparator
```

Fig. 20.8 | Custom `Comparator` class that compares two `Time2` objects. (Part 2 of 2.)

Class `TimeComparator` implements interface `Comparator`, a generic type that takes one type argument (in this case `Time2`). A class that implements `Comparator` must declare a `compare` method that receives two arguments and returns a negative integer if the first argument is less than the second, 0 if the arguments are equal or a positive integer if the first argument is greater than the second. Method `compare` (lines 7–26) performs comparisons between `Time2` objects. Line 9 compares the two hours of the `Time2` objects. If the hours are different (line 12), then we return this value. If this value is positive, then the first hour is greater than the second and the first time is greater than the second. If this value is negative, then the first hour is less than the second and the first time is less than the second. If this value is zero, the hours are the same and we must test the minutes (and maybe the seconds) to determine which time is greater.

Figure 20.9 sorts a list using the custom `Comparator` class `TimeComparator`. Line 11 creates an `ArrayList` of `Time2` objects. Recall that both `ArrayList` and `List` are generic types and accept a type argument that specifies the element type of the collection. Lines 13–17 create five `Time2` objects and add them to this list. Line 23 calls method `sort`, passing it an object of our `TimeComparator` class (Fig. 20.8).

```
1  // Fig. 20.9: Sort3.java
2  // Collections method sort with a custom Comparator object.
3  import java.util.List;
4  import java.util.ArrayList;
5  import java.util.Collections;
6
7  public class Sort3
8  {
9     public static void main( String[] args )
10    {
11       List< Time2 > list = new ArrayList< Time2 >(); // create List
12
13       list.add( new Time2(  6, 24, 34 ) );
14       list.add( new Time2( 18, 14, 58 ) );
15       list.add( new Time2(  6, 05, 34 ) );
16       list.add( new Time2( 12, 14, 58 ) );
```

Fig. 20.9 | `Collections` method `sort` with a custom `Comparator` object. (Part 1 of 2.)

```
17         list.add( new Time2(  6, 24, 22 ) );
18
19         // output List elements
20         System.out.printf( "Unsorted array elements:\n%s\n", list );
21
22         // sort in order using a comparator
23         Collections.sort( list, new TimeComparator() );
24
25         // output List elements
26         System.out.printf( "Sorted list elements:\n%s\n", list );
27      } // end main
28   } // end class Sort3
```

```
Unsorted array elements:
[6:24:34 AM, 6:14:58 PM, 6:05:34 AM, 12:14:58 PM, 6:24:22 AM]
Sorted list elements:
[6:05:34 AM, 6:24:22 AM, 6:24:34 AM, 12:14:58 PM, 6:14:58 PM]
```

Fig. 20.9 | `Collections` method `sort` with a custom `Comparator` object. (Part 2 of 2.)

20.7.2 Method shuffle

Method **`shuffle`** randomly orders a `List`'s elements. Chapter 7 presented a card shuffling and dealing simulation that shuffled a deck of cards with a loop. Figure 20.10 uses method `shuffle` to shuffle a deck of `Card` objects that might be used in a card-game simulator.

Class `Card` (lines 8–41) represents a card in a deck of cards. Each `Card` has a face and a suit. Lines 10–12 declare two `enum` types—`Face` and `Suit`—which represent the face and the suit of the card, respectively. Method `toString` (lines 37–40) returns a `String` containing the face and suit of the `Card` separated by the string `" of "`. When an `enum` constant is converted to a string, the constant's identifier is used as the string representation. Normally we would use all uppercase letters for `enum` constants. In this example, we chose to use capital letters for only the first letter of each `enum` constant because we want the card to be displayed with initial capital letters for the face and the suit (e.g., `"Ace of Spades"`).

```
1  // Fig. 20.10: DeckOfCards.java
2  // Card shuffling and dealing with Collections method shuffle.
3  import java.util.List;
4  import java.util.Arrays;
5  import java.util.Collections;
6
7  // class to represent a Card in a deck of cards
8  class Card
9  {
10    public static enum Face { Ace, Deuce, Three, Four, Five, Six,
11       Seven, Eight, Nine, Ten, Jack, Queen, King  };
12    public static enum Suit { Clubs, Diamonds, Hearts, Spades };
13
14    private final Face face; // face of card
15    private final Suit suit; // suit of card
```

Fig. 20.10 | Card shuffling and dealing with `Collections` method `shuffle`. (Part 1 of 3.)

```
16
17      // two-argument constructor
18      public Card( Face cardFace, Suit cardSuit )
19      {
20          face = cardFace; // initialize face of card
21          suit = cardSuit; // initialize suit of card
22      } // end two-argument Card constructor
23
24      // return face of the card
25      public Face getFace()
26      {
27         return face;
28      } // end method getFace
29
30      // return suit of Card
31      public Suit getSuit()
32      {
33         return suit;
34      } // end method getSuit
35
36      // return String representation of Card
37      public String toString()
38      {
39         return String.format( "%s of %s", face, suit );
40      } // end method toString
41   } // end class Card
42
43   // class DeckOfCards declaration
44   public class DeckOfCards
45   {
46      private List< Card > list; // declare List that will store Cards
47
48      // set up deck of Cards and shuffle
49      public DeckOfCards()
50      {
51         Card[] deck = new Card[ 52 ];
52         int count = 0; // number of cards
53
54         // populate deck with Card objects
55         for ( Card.Suit suit : Card.Suit.values() )
56         {
57            for ( Card.Face face : Card.Face.values() )
58            {
59               deck[ count ] = new Card( face, suit );
60               ++count;
61            } // end for
62         } // end for
63
64         list = Arrays.asList( deck ); // get List
65         Collections.shuffle( list );  // shuffle deck
66      } // end DeckOfCards constructor
67
```

Fig. 20.10 | Card shuffling and dealing with `Collections` method `shuffle`. (Part 2 of 3.)

```
68      // output deck
69      public void printCards()
70      {
71         // display 52 cards in two columns
72         for ( int i = 0; i < list.size(); i++ )
73            System.out.printf( "%-19s%s", list.get( i ),
74               ( ( i + 1 ) % 4 == 0 ) ? "\n" : "" );
75      } // end method printCards
76
77      public static void main( String[] args )
78      {
79         DeckOfCards cards = new DeckOfCards();
80         cards.printCards();
81      } // end main
82   } // end class DeckOfCards
```

```
Deuce of Clubs     Six of Spades      Nine of Diamonds   Ten of Hearts
Three of Diamonds  Five of Clubs      Deuce of Diamonds  Seven of Clubs
Three of Spades    Six of Diamonds    King of Clubs      Jack of Hearts
Ten of Spades      King of Diamonds   Eight of Spades    Six of Hearts
Nine of Clubs      Ten of Diamonds    Eight of Diamonds  Eight of Hearts
Ten of Clubs       Five of Hearts     Ace of Clubs       Deuce of Hearts
Queen of Diamonds  Ace of Diamonds    Four of Clubs      Nine of Hearts
Ace of Spades      Deuce of Spades    Ace of Hearts      Jack of Diamonds
Seven of Diamonds  Three of Hearts    Four of Spades     Four of Diamonds
Seven of Spades    King of Hearts     Seven of Hearts    Five of Diamonds
Eight of Clubs     Three of Clubs     Queen of Clubs     Queen of Spades
Six of Clubs       Nine of Spades     Four of Hearts     Jack of Clubs
Five of Spades     King of Spades     Jack of Spades     Queen of Hearts
```

Fig. 20.10 | Card shuffling and dealing with `Collections` method `shuffle`. (Part 3 of 3.)

Lines 55–62 populate the `deck` array with cards that have unique face and suit combinations. Both `Face` and `Suit` are `public static enum` types of class `Card`. To use these `enum` types outside of class `Card`, you must qualify each `enum`'s type name with the name of the class in which it resides (i.e., `Card`) and a dot (`.`) separator. Hence, lines 55 and 57 use `Card.Suit` and `Card.Face` to declare the control variables of the `for` statements. Recall that method `values` of an `enum` type returns an array that contains all the constants of the `enum` type. Lines 55–62 use enhanced `for` statements to construct 52 new `Card`s.

The shuffling occurs in line 65, which calls `static` method `shuffle` of class `Collections` to shuffle the elements of the array. Method `shuffle` requires a `List` argument, so we must obtain a `List` view of the array before we can shuffle it. Line 64 invokes `static` method `asList` of class `Arrays` to get a `List` view of the `deck` array.

Method `printCards` (lines 69–75) displays the deck of cards in four columns. In each iteration of the loop, lines 73–74 output a card left justified in a 19-character field followed by either a newline or an empty string based on the number of cards output so far. If the number of cards is divisible by 4, a newline is output; otherwise, the empty string is output.

20.7.3 Methods `reverse`, `fill`, `copy`, `max` and `min`

Class `Collections` provides methods for reversing, filling and copying `List`s. **`Collections` method `reverse`** reverses the order of the elements in a `List`, and **method `fill`**

overwrites elements in a `List` with a specified value. The `fill` operation is useful for reinitializing a `List`. **Method copy** takes two arguments—a destination `List` and a source `List`. Each source `List` element is copied to the destination `List`. The destination `List` must be at least as long as the source `List`; otherwise, an `IndexOutOfBoundsException` occurs. If the destination `List` is longer, the elements not overwritten are unchanged.

Each method we've seen so far operates on `Lists`. Methods **`min`** and **`max`** each operate on any `Collection`. Method `min` returns the smallest element in a `Collection`, and method `max` returns the largest element in a `Collection`. Both of these methods can be called with a `Comparator` object as a second argument to perform custom comparisons of objects, such as the `TimeComparator` in Fig. 20.9. Figure 20.11 demonstrates methods `reverse`, `fill`, `copy`, `max` and `min`.

```
1  // Fig. 20.11: Algorithms1.java
2  // Collections methods reverse, fill, copy, max and min.
3  import java.util.List;
4  import java.util.Arrays;
5  import java.util.Collections;
6
7  public class Algorithms1
8  {
9     public static void main( String[] args )
10    {
11       // create and display a List< Character >
12       Character[] letters = { 'P', 'C', 'M' };
13       List< Character > list = Arrays.asList( letters ); // get List
14       System.out.println( "list contains: " );
15       output( list );
16
17       // reverse and display the List< Character >
18       Collections.reverse( list ); // reverse order the elements
19       System.out.println( "\nAfter calling reverse, list contains: " );
20       output( list );
21
22       // create copyList from an array of 3 Characters
23       Character[] lettersCopy = new Character[ 3 ];
24       List< Character > copyList = Arrays.asList( lettersCopy );
25
26       // copy the contents of list into copyList
27       Collections.copy( copyList, list );
28       System.out.println( "\nAfter copying, copyList contains: " );
29       output( copyList );
30
31       // fill list with Rs
32       Collections.fill( list, 'R' );
33       System.out.println( "\nAfter calling fill, list contains: " );
34       output( list );
35    } // end main
36
37    // output List information
38    private static void output( List< Character > listRef )
39    {
```

Fig. 20.11 | `Collections` methods `reverse`, `fill`, `copy`, `max` and `min`. (Part 1 of 2.)

```
40         System.out.print( "The list is: " );
41
42         for ( Character element : listRef )
43            System.out.printf( "%s ", element );
44
45         System.out.printf( "\nMax: %s", Collections.max( listRef ) );
46         System.out.printf( "  Min: %s\n", Collections.min( listRef ) );
47      } // end method output
48   } // end class Algorithms1
```

```
list contains:
The list is: P C M
Max: P  Min: C

After calling reverse, list contains:
The list is: M C P
Max: P  Min: C

After copying, copyList contains:
The list is: M C P
Max: P  Min: C

After calling fill, list contains:
The list is: R R R
Max: R  Min: R
```

Fig. 20.11 | `Collections` methods `reverse`, `fill`, `copy`, `max` and `min`. (Part 2 of 2.)

Line 13 creates `List<Character>` variable `list` and initializes it with a `List` view of the `Character` array `letters`. Lines 14–15 output the current contents of the `List`. Line 18 calls `Collections` method `reverse` to reverse the order of `list`. Method `reverse` takes one `List` argument. Since `list` is a `List` view of array `letters`, the array's elements are now in reverse order. The reversed contents are output in lines 19–20. Line 27 uses `Collections` method `copy` to copy `list`'s elements into `copyList`. Changes to `copyList` do not change `letters`, because `copyList` is a separate `List` that is not a `List` view of the array `letters`. Method `copy` requires two `List` arguments—the destination `List` and the source `List`. Line 32 calls `Collections` method `fill` to place the character `'R'` in each `list` element. Because `list` is a `List` view of the array `letters`, this operation changes each element in `letters` to `'R'`. Method `fill` requires a `List` for the first argument and an `Object` for the second argument—in this case, the `Object` is the boxed version of the character `'R'`. Lines 45–46 call `Collections` methods `max` and `min` to find the largest and the smallest element of a `Collection`, respectively. Recall that interface `List` extends interface `Collection`, so a `List` *is a* `Collection`.

20.7.4 Method `binarySearch`

In Section 19.2.2, we studied the high-speed binary search algorithm. This algorithm is built into the Java collections framework as a `static` **`Collections` method `binarySearch`**, which locates an object in a `List` (e.g., a `LinkedList` or an `ArrayList`). If the object is found, its index is returned. If the object is not found, `binarySearch` returns a negative

value. Method `binarySearch` determines this negative value by first calculating the insertion point and making its sign negative. Then, `binarySearch` subtracts 1 from the insertion point to obtain the return value, which guarantees that method `binarySearch` returns positive numbers (>= 0) if and only if the object is found. If multiple elements in the list match the search key, there's no guarantee which one will be located first. Figure 20.12 uses method `binarySearch` to search for a series of strings in an `ArrayList`.

```
1  // Fig. 20.12: BinarySearchTest.java
2  // Collections method binarySearch.
3  import java.util.List;
4  import java.util.Arrays;
5  import java.util.Collections;
6  import java.util.ArrayList;
7
8  public class BinarySearchTest
9  {
10    public static void main( String[] args )
11    {
12       // create an ArrayList< String > from the contents of colors array
13       String[] colors = { "red", "white", "blue", "black", "yellow",
14          "purple", "tan", "pink" };
15       List< String > list =
16          new ArrayList< String >( Arrays.asList( colors ) );
17
18       Collections.sort( list ); // sort the ArrayList
19       System.out.printf( "Sorted ArrayList: %s\n", list );
20
21       // search list for various values
22       printSearchResults( list, colors[ 3 ] ); // first item
23       printSearchResults( list, colors[ 0 ] ); // middle item
24       printSearchResults( list, colors[ 7 ] ); // last item
25       printSearchResults( list, "aqua" ); // below lowest
26       printSearchResults( list, "gray" ); // does not exist
27       printSearchResults( list, "teal" ); // does not exist
28    } // end main
29
30    // perform search and display result
31    private static void printSearchResults(
32       List< String > list, String key )
33    {
34       int result = 0;
35
36       System.out.printf( "\nSearching for: %s\n", key );
37       result = Collections.binarySearch( list, key );
38
39       if ( result >= 0 )
40          System.out.printf( "Found at index %d\n", result );
41       else
42          System.out.printf( "Not Found (%d)\n",result );
43    } // end method printSearchResults
44 } // end class BinarySearchTest
```

Fig. 20.12 | `Collections` method `binarySearch`. (Part 1 of 2.)

```
Sorted ArrayList: [black, blue, pink, purple, red, tan, white, yellow]

Searching for: black
Found at index 0

Searching for: red
Found at index 4

Searching for: pink
Found at index 2

Searching for: aqua
Not Found (-1)

Searching for: gray
Not Found (-3)

Searching for: teal
Not Found (-7)
```

Fig. 20.12 | `Collections` method `binarySearch`. (Part 2 of 2.)

Lines 15–16 initialize `list` with an `ArrayList` containing a copy of the elements in array `colors`. `Collections` method `binarySearch` expects its `List` argument's elements to be sorted in ascending order, so line 18 uses `Collections` method `sort` to sort the list. If the `List` argument's elements are not sorted, the result of using `binarySearch` is undefined. Line 19 outputs the sorted list. Lines 22–27 call method `printSearchResults` (lines 31–43) to perform searches and output the results. Line 37 calls `Collections` method `binarySearch` to search `list` for the specified `key`. Method `binarySearch` takes a `List` as the first argument and an `Object` as the second argument. Lines 39–42 output the results of the search. An overloaded version of `binarySearch` takes a `Comparator` object as its third argument, which specifies how `binarySearch` should compare the search key to the `List`'s elements.

20.7.5 Methods `addAll`, `frequency` and `disjoint`

Class `Collections` also provides the methods `addAll`, `frequency` and `disjoint`. **`Collections` method `addAll`** takes two arguments—a `Collection` into which to insert the new element(s) and an array that provides elements to be inserted. **`Collections` method `frequency`** takes two arguments—a `Collection` to be searched and an `Object` to be searched for in the collection. Method `frequency` returns the number of times that the second argument appears in the collection. **`Collections` method `disjoint`** takes two `Collections` and returns `true` if they have no elements in common. Figure 20.13 demonstrates the use of methods `addAll`, `frequency` and `disjoint`.

```
1  // Fig. 20.13: Algorithms2.java
2  // Collections methods addAll, frequency and disjoint.
3  import java.util.ArrayList;
4  import java.util.List;
5  import java.util.Arrays;
6  import java.util.Collections;
```

Fig. 20.13 | `Collections` methods `addAll`, `frequency` and `disjoint`. (Part 1 of 2.)

```
7
8  public class Algorithms2
9  {
10    public static void main( String[] args )
11    {
12       // initialize list1 and list2
13       String[] colors = { "red", "white", "yellow", "blue" };
14       List< String > list1 = Arrays.asList( colors );
15       ArrayList< String > list2 = new ArrayList< String >();
16
17       list2.add( "black" ); // add "black" to the end of list2
18       list2.add( "red" ); // add "red" to the end of list2
19       list2.add( "green" ); // add "green" to the end of list2
20
21       System.out.print( "Before addAll, list2 contains: " );
22
23       // display elements in list2
24       for ( String s : list2 )
25          System.out.printf( "%s ", s );
26
27       Collections.addAll( list2, colors ); // add colors Strings to list2
28
29       System.out.print( "\nAfter addAll, list2 contains: " );
30
31       // display elements in list2
32       for ( String s : list2 )
33          System.out.printf( "%s ", s );
34
35       // get frequency of "red"
36       int frequency = Collections.frequency( list2, "red" );
37       System.out.printf(
38          "\nFrequency of red in list2: %d\n", frequency );
39
40       // check whether list1 and list2 have elements in common
41       boolean disjoint = Collections.disjoint( list1, list2 );
42
43       System.out.printf( "list1 and list2 %s elements in common\n",
44          ( disjoint ? "do not have" : "have" ) );
45    } // end main
46 } // end class Algorithms2
```

```
Before addAll, list2 contains: black red green
After addAll, list2 contains: black red green red white yellow blue
Frequency of red in list2: 2
list1 and list2 have elements in common
```

Fig. 20.13 | `Collections` methods `addAll`, `frequency` and `disjoint`. (Part 2 of 2.)

Line 14 initializes `list1` with elements in array `colors`, and lines 17–19 add `Strings` `"black"`, `"red"` and `"green"` to `list2`. Line 27 invokes method `addAll` to add elements in array `colors` to `list2`. Line 36 gets the frequency of `String` `"red"` in `list2` using method `frequency`. Line 41 invokes method `disjoint` to test whether `Collections` `list1` and `list2` have elements in common, which they do in this example.

20.8 Stack Class of Package java.util

We introduced the concept of a stack in Section 6.6 when we discussed the method-call stack. In Chapter 22, Custom Generic Data Structures, we'll learn how to build data structures, including linked lists, stacks, queues and trees. In a world of software reuse, rather than building data structures as we need them, we can often take advantage of existing data structures. In this section, we investigate class **Stack** in the Java utilities package (`java.util`).

Class `Stack` extends class `Vector` to implement a stack data structure. Because class `Stack` extends class `Vector`, the entire `public` interface of class `Vector` is available to clients of class `Stack`. Figure 20.14 demonstrates several `Stack` methods. For the details of class `Stack`, visit `download.oracle.com/javase/6/docs/api/java/util/Stack.html`.



Error-Prevention Tip 20.1

Because `Stack` extends `Vector`, all `public` `Vector` methods can be called on `Stack` objects, even if the methods do not represent conventional stack operations. For example, `Vector` method `add` can be used to insert an element anywhere in a stack—an operation that could "corrupt" the stack. When manipulating a `Stack`, only methods `push` and `pop` should be used to add elements to and remove elements from the `Stack`, respectively.

```
1  // Fig. 20.14: StackTest.java
2  // Stack class of package java.util.
3  import java.util.Stack;
4  import java.util.EmptyStackException;
5
6  public class StackTest
7  {
8     public static void main( String[] args )
9     {
10       Stack< Number > stack = new Stack< Number >(); // create a Stack
11
12       // use push method
13       stack.push( 12L ); // push long value 12L
14       System.out.println( "Pushed 12L" );
15       printStack( stack );
16       stack.push( 34567 ); // push int value 34567
17       System.out.println( "Pushed 34567" );
18       printStack( stack );
19       stack.push( 1.0F ); // push float value 1.0F
20       System.out.println( "Pushed 1.0F" );
21       printStack( stack );
22       stack.push( 1234.5678 ); // push double value 1234.5678
23       System.out.println( "Pushed 1234.5678 " );
24       printStack( stack );
25
26       // remove items from stack
27       try
28       {
29          Number removedObject = null;
30
```

Fig. 20.14 | `Stack` class of package `java.util`. (Part 1 of 2.)

```
31            // pop elements from stack
32            while ( true )
33            {
34               removedObject = stack.pop(); // use pop method
35               System.out.printf( "Popped %s\n", removedObject );
36               printStack( stack );
37            } // end while
38         } // end try
39         catch ( EmptyStackException emptyStackException )
40         {
41            emptyStackException.printStackTrace();
42         } // end catch
43      } // end main
44
45      // display Stack contents
46      private static void printStack( Stack< Number > stack )
47      {
48         if ( stack.isEmpty() )
49            System.out.println( "stack is empty\n" ); // the stack is empty
50         else // stack is not empty
51            System.out.printf( "stack contains: %s (top)\n", stack );
52      } // end method printStack
53   } // end class StackTest
```

```
Pushed 12L
stack contains: [12] (top)
Pushed 34567
stack contains: [12, 34567] (top)
Pushed 1.0F
stack contains: [12, 34567, 1.0] (top)
Pushed 1234.5678
stack contains: [12, 34567, 1.0, 1234.5678] (top)
Popped 1234.5678
stack contains: [12, 34567, 1.0] (top)
Popped 1.0
stack contains: [12, 34567] (top)
Popped 34567
stack contains: [12] (top)
Popped 12
stack is empty

java.util.EmptyStackException
        at java.util.Stack.peek(Unknown Source)
        at java.util.Stack.pop(Unknown Source)
        at StackTest.main(StackTest.java:34)
```

Fig. 20.14 | `Stack` class of package `java.util`. (Part 2 of 2.)

Line 10 creates an empty `Stack` of `Numbers`. Class `Number` (in package `java.lang`) is the superclass of the type-wrapper classes for the primitive numeric types (e.g., `Integer`, `Double`). By creating a `Stack` of `Numbers`, objects of any class that extends `Number` can be pushed onto the `Stack`. Lines 13, 16, 19 and 22 each call `Stack` method **`push`** to add a `Number` object to the top of the stack. Note the literals `12L` (line 13) and `1.0F` (line 19). Any integer literal that has the **suffix `L`** is a `long` value. An integer literal without a suffix

is an int value. Similarly, any floating-point literal that has the **suffix F** is a float value. A floating-point literal without a suffix is a double value. You can learn more about numeric literals in the *Java Language Specification* at java.sun.com/docs/books/jls/third_edition/html/expressions.html#15.8.1.

An infinite loop (lines 32–37) calls **Stack method pop** to remove the top element of the stack. The method returns a Number reference to the removed element. If there are no elements in the Stack, method pop throws an **EmptyStackException**, which terminates the loop. Class Stack also declares **method peek**. This method returns the top element of the stack without popping the element off the stack.

Method printStack (lines 46–52) displays the stack's contents. The current top of the stack (the last value pushed onto the stack) is the first value printed. Line 48 calls **Stack method isEmpty** (inherited by Stack from class Vector) to determine whether the stack is empty. If it's empty, the method returns true; otherwise, false.

20.9 Class PriorityQueue and Interface Queue

Recall that a queue is a collection that represents a waiting line—typically, insertions are made at the back of a queue and deletions are made from the front. In Section 22.6, we'll discuss and implement a queue data structure. In this section, we investigate Java's **Queue** interface and **PriorityQueue** class from package java.util. Interface Queue extends interface Collection and provides additional operations for inserting, removing and inspecting elements in a queue. PriorityQueue, which implements the Queue interface, orders elements by their natural ordering as specified by Comparable elements' compareTo method or by a Comparator object that is supplied to the constructor.

Class PriorityQueue provides functionality that enables insertions in sorted order into the underlying data structure and deletions from the front of the underlying data structure. When adding elements to a PriorityQueue, the elements are inserted in priority order such that the highest-priority element (i.e., the largest value) will be the first element removed from the PriorityQueue.

The common PriorityQueue operations are **offer** to insert an element at the appropriate location based on priority order, **poll** to remove the highest-priority element of the priority queue (i.e., the head of the queue), **peek** to get a reference to the highest-priority element of the priority queue (without removing that element), **clear** to remove all elements in the priority queue and **size** to get the number of elements in the priority queue. Figure 20.15 demonstrates the PriorityQueue class.

```
1  // Fig. 20.15: PriorityQueueTest.java
2  // PriorityQueue test program.
3  import java.util.PriorityQueue;
4
5  public class PriorityQueueTest
6  {
7     public static void main( String[] args )
8     {
9        // queue of capacity 11
10       PriorityQueue< Double > queue = new PriorityQueue< Double >();
```

Fig. 20.15 | PriorityQueue test program. (Part 1 of 2.)

```
11
12          // insert elements to queue
13          queue.offer( 3.2 );
14          queue.offer( 9.8 );
15          queue.offer( 5.4 );
16
17          System.out.print( "Polling from queue: " );
18
19          // display elements in queue
20          while ( queue.size() > 0 )
21          {
22             System.out.printf( "%.1f ", queue.peek() ); // view top element
23             queue.poll(); // remove top element
24          } // end while
25       } // end main
26    } // end class PriorityQueueTest
```

```
Polling from queue: 3.2 5.4 9.8
```

Fig. 20.15 | `PriorityQueue` test program. (Part 2 of 2.)

Line 10 creates a `PriorityQueue` that stores `Double`s with an initial capacity of `11` elements and orders the elements according to the object's natural ordering (the defaults for a `PriorityQueue`). `PriorityQueue` is a generic class. Line 10 instantiates a `PriorityQueue` with a type argument `Double`. Class `PriorityQueue` provides five additional constructors. One of these takes an `int` and a `Comparator` object to create a `PriorityQueue` with the initial capacity specified by the `int` and the ordering by the `Comparator`. Lines 13–15 use method `offer` to add elements to the priority queue. Method `offer` throws a `NullPointerException` if the program attempts to add a `null` object to the queue. The loop in lines 20–24 uses method `size` to determine whether the priority queue is empty (line 20). While there are more elements, line 22 uses `PriorityQueue` method `peek` to retrieve the highest-priority element in the queue for output (without actually removing it from the queue). Line 23 removes the highest-priority element in the queue with method `poll`, which returns the removed element.

20.10 Sets

A **`Set`** is an unordered `Collection` of unique elements (i.e., no duplicate elements). The collections framework contains several `Set` implementations, including **`HashSet`** and **`TreeSet`**. `HashSet` stores its elements in a hash table, and `TreeSet` stores its elements in a tree. Hash tables are presented in Section 20.11. Trees are discussed in Section 22.7.

Figure 20.16 uses a `HashSet` to remove duplicate strings from a `List`. Recall that both `List` and `Collection` are generic types, so line 16 creates a `List` that contains `String` objects, and line 20 passes a `Collection` of `String`s to method `printNonDuplicates`.

Method `printNonDuplicates` (lines 24–35) takes a `Collection` argument. Line 27 constructs a `HashSet<String>` from the `Collection<String>` argument. By definition, `Set`s do not contain duplicates, so when the `HashSet` is constructed, it removes any duplicates in the `Collection`. Lines 31–32 output elements in the `Set`.

```
1  // Fig. 20.16: SetTest.java
2  // HashSet used to remove duplicate values from array of strings.
3  import java.util.List;
4  import java.util.Arrays;
5  import java.util.HashSet;
6  import java.util.Set;
7  import java.util.Collection;
8
9  public class SetTest
10 {
11    public static void main( String[] args )
12    {
13       // create and display a List< String >
14       String[] colors = { "red", "white", "blue", "green", "gray",
15          "orange", "tan", "white", "cyan", "peach", "gray", "orange" };
16       List< String > list = Arrays.asList( colors );
17       System.out.printf( "List: %s\n", list );
18
19       // eliminate duplicates then print the unique values
20       printNonDuplicates( list );
21    } // end main
22
23    // create a Set from a Collection to eliminate duplicates
24    private static void printNonDuplicates( Collection< String > values )
25    {
26       // create a HashSet
27       Set< String > set = new HashSet< String >( values );
28
29       System.out.print( "\nNonduplicates are: " );
30
31       for ( String value : set )
32          System.out.printf( "%s ", value );
33
34       System.out.println();
35    } // end method printNonDuplicates
36 } // end class SetTest
```

```
List: [red, white, blue, green, gray, orange, tan, white, cyan, peach, gray,
orange]

Nonduplicates are: orange green white peach gray cyan red blue tan
```

Fig. 20.16 | `HashSet` used to remove duplicate values from an array of strings.

Sorted Sets

The collections framework also includes the **`SortedSet` interface** (which extends `Set`) for sets that maintain their elements in sorted order—either the elements' natural order (e.g., numbers are in ascending order) or an order specified by a `Comparator`. Class `TreeSet` implements `SortedSet`. The program in Fig. 20.17 places strings into a `TreeSet`. The strings are sorted as they're added to the `TreeSet`. This example also demonstrates range-view methods, which enable a program to view a portion of a collection.

Lines 14–15 of create a `TreeSet<String>` that contains the elements of array `colors`, then assigns the new `TreeSet<String>` to `SortedSet<String>` variable `tree`. Line 18

```
1  // Fig. 20.17: SortedSetTest.java
2  // Using SortedSets and TreeSets.
3  import java.util.Arrays;
4  import java.util.SortedSet;
5  import java.util.TreeSet;
6
7  public class SortedSetTest
8  {
9     public static void main( String[] args )
10    {
11       // create TreeSet from array colors
12       String[] colors = { "yellow", "green", "black", "tan", "grey",
13          "white", "orange", "red", "green" };
14       SortedSet< String > tree =
15          new TreeSet< String >( Arrays.asList( colors ) );
16
17       System.out.print( "sorted set: " );
18       printSet( tree ); // output contents of tree
19
20       // get headSet based on "orange"
21       System.out.print( "headSet (\"orange\"):  " );
22       printSet( tree.headSet( "orange" ) );
23
24       // get tailSet based upon "orange"
25       System.out.print( "tailSet (\"orange\"):  " );
26       printSet( tree.tailSet( "orange" ) );
27
28       // get first and last elements
29       System.out.printf( "first: %s\n", tree.first() );
30       System.out.printf( "last : %s\n", tree.last() );
31    } // end main
32
33    // output SortedSet using enhanced for statement
34    private static void printSet( SortedSet< String > set )
35    {
36       for ( String s : set )
37          System.out.printf( "%s ", s );
38
39       System.out.println();
40    } // end method printSet
41 } // end class SortedSetTest
```

```
sorted set: black green grey orange red tan white yellow
headSet ("orange"):  black green grey
tailSet ("orange"):  orange red tan white yellow
first: black
last : yellow
```

Fig. 20.17 | Using `SortedSet`s and `TreeSet`s.

outputs the initial set of strings using method `printSet` (lines 34–40), which we discuss momentarily. Line 22 calls **`TreeSet` method `headSet`** to get a subset of the `TreeSet` in which every element is less than `"orange"`. The view returned from `headSet` is then output

with `printSet`. If any changes are made to the subset, they'll also be made to the original `TreeSet`, because the subset returned by `headSet` is a view of the `TreeSet`.

Line 26 calls **`TreeSet` method `tailSet`** to get a subset in which each element is greater than or equal to `"orange"`, then outputs the result. Any changes made through the `tailSet` view are made to the original `TreeSet`. Lines 29–30 call **`SortedSet` methods `first`** and **`last`** to get the smallest and largest elements of the set, respectively.

Method `printSet` (lines 34–40) accepts a `SortedSet` as an argument and prints it. Lines 36–37 print each element of the `SortedSet` using the enhanced `for` statement.

20.11 Maps

`Map`s associate keys to values. The keys in a `Map` must be unique, but the associated values need not be. If a `Map` contains both unique keys and unique values, it's said to implement a **one-to-one mapping**. If only the keys are unique, the `Map` is said to implement a **many-to-one mapping**—many keys can map to one value.

`Map`s differ from `Set`s in that `Map`s contain keys and values, whereas `Set`s contain only values. Three of the several classes that implement interface `Map` are **`Hashtable`**, **`HashMap`** and **`TreeMap`**. `Hashtable`s and `HashMap`s store elements in hash tables, and `TreeMap`s store elements in trees. This section discusses hash tables and provides an example that uses a `HashMap` to store key/value pairs. **Interface `SortedMap`** extends `Map` and maintains its keys in sorted order—either the elements' natural order or an order specified by a `Comparator`. Class `TreeMap` implements `SortedMap`.

`Map` Implementation with Hash Tables

When a program creates objects of new or existing types, it may need to store and retrieve them efficiently. Storing and retrieving information with arrays is efficient if some aspect of your data directly matches a numerical key value and if the keys are unique and tightly packed. If you have 100 employees with nine-digit social security numbers and you want to store and retrieve employee data by using the social security number as a key, the task will require an array with over 700 million elements, because nine-digit Social Security numbers must begin with 001–733 as per the Social Security Administration's website

```
www.socialsecurity.gov/employer/stateweb.htm
```

This is impractical for virtually all applications that use social security numbers as keys. A program having an array that large could achieve high performance for both storing and retrieving employee records by simply using the social security number as the array index.

Numerous applications have this problem—namely, that either the keys are of the wrong type (e.g., not positive integers that correspond to array subscripts) or they're of the right type, but sparsely spread over a huge range. What is needed is a high-speed scheme for converting keys such as social security numbers, inventory part numbers and the like into unique array indices. Then, when an application needs to store something, the scheme could convert the application's key rapidly into an index, and the record could be stored at that slot in the array. Retrieval is accomplished the same way: Once the application has a key for which it wants to retrieve a data record, the application simply applies the conversion to the key—this produces the array index where the data is stored and retrieved.

The scheme we describe here is the basis of a technique called **hashing**. Why the name? When we convert a key into an array index, we literally scramble the bits, forming

a kind of "mishmashed," or hashed, number. The number actually has no real significance beyond its usefulness in storing and retrieving a particular data record.

A glitch in the scheme is called a **collision**—this occurs when two different keys "hash into" the same cell (or element) in the array. We cannot store two values in the same space, so we need to find an alternative home for all values beyond the first that hash to a particular array index. There are many schemes for doing this. One is to "hash again" (i.e., to apply another hashing transformation to the key to provide a next candidate cell in the array). The hashing process is designed to distribute the values throughout the table, so the assumption is that an available cell will be found with just a few hashes.

Another scheme uses one hash to locate the first candidate cell. If that cell is occupied, successive cells are searched in order until an available cell is found. Retrieval works the same way: The key is hashed once to determine the initial location and check whether it contains the desired data. If it does, the search is finished. If it does not, successive cells are searched linearly until the desired data is found.

The most popular solution to hash-table collisions is to have each cell of the table be a hash "bucket," typically a linked list of all the key/value pairs that hash to that cell. This is the solution that Java's `Hashtable` and `HashMap` classes (from package `java.util`) implement. Both `Hashtable` and `HashMap` implement the `Map` interface. The primary differences between them are that `HashMap` is unsynchronized (multiple threads should not modify a `HashMap` concurrently) and allows `null` keys and `null` values.

A hash table's **load factor** affects the performance of hashing schemes. The load factor is the ratio of the number of occupied cells in the hash table to the total number of cells in the hash table. The closer this ratio gets to 1.0, the greater the chance of collisions.



Performance Tip 20.2

The load factor in a hash table is a classic example of a memory-space/execution-time trade-off: By increasing the load factor, we get better memory utilization, but the program runs slower, due to increased hashing collisions. By decreasing the load factor, we get better program speed, because of reduced hashing collisions, but we get poorer memory utilization, because a larger portion of the hash table remains empty.

Hash tables are complex to program. Computer science students study hashing schemes in courses called "Data Structures" and "Algorithms." Classes `Hashtable` and `HashMap` enable you to use hashing without having to implement hash-table mechanisms. This concept is profoundly important in our study of object-oriented programming. As discussed in earlier chapters, classes encapsulate and hide complexity (i.e., implementation details) and offer user-friendly interfaces. Properly crafting classes to exhibit such behavior is one of the most valued skills in the field of object-oriented programming. Figure 20.18 uses a `HashMap` to count the number of occurrences of each word in a string.

```
1  // Fig. 20.18: WordTypeCount.java
2  // Program counts the number of occurrences of each word in a String.
3  import java.util.Map;
4  import java.util.HashMap;
5  import java.util.Set;
6  import java.util.TreeSet;
```

Fig. 20.18 | Program counts the number of occurrences of each word in a `String`. (Part 1 of 3.)

```
7  import java.util.Scanner;
8
9  public class WordTypeCount
10 {
11    public static void main( String[] args )
12    {
13       // create HashMap to store String keys and Integer values
14       Map< String, Integer > myMap = new HashMap< String, Integer >();
15
16       createMap( myMap ); // create map based on user input
17       displayMap( myMap ); // display map content
18    } // end main
19
20    // create map from user input
21    private static void createMap( Map< String, Integer > map )
22    {
23       Scanner scanner = new Scanner( System.in ); // create scanner
24       System.out.println( "Enter a string:" ); // prompt for user input
25       String input = scanner.nextLine();
26
27       // tokenize the input
28       String[] tokens = input.split( " " );
29
30       // processing input text
31       for ( String token : tokens )
32       {
33          String word = token.toLowerCase(); // get lowercase word
34
35          // if the map contains the word
36          if ( map.containsKey( word ) ) // is word in map
37          {
38             int count = map.get( word ); // get current count
39             map.put( word, count + 1 ); // increment count
40          } // end if
41          else
42             map.put( word, 1 ); // add new word with a count of 1 to map
43       } // end for
44    } // end method createMap
45
46    // display map content
47    private static void displayMap( Map< String, Integer > map )
48    {
49       Set< String > keys = map.keySet(); // get keys
50
51       // sort keys
52       TreeSet< String > sortedKeys = new TreeSet< String >( keys );
53
54       System.out.println( "\nMap contains:\nKey\t\tValue" );
55
56       // generate output for each key in map
57       for ( String key : sortedKeys )
58          System.out.printf( "%-10s%10s\n", key, map.get( key ) );
59
```

Fig. 20.18 | Program counts the number of occurrences of each word in a `String`. (Part 2 of 3.)

```
60         System.out.printf(
61            "\nsize: %d\nisEmpty: %b\n", map.size(), map.isEmpty() );
62      } // end method displayMap
63   } // end class WordTypeCount
```

```
Enter a string:
this is a sample sentence with several words this is another sample
sentence with several different words

Map contains:
Key             Value
a                  1
another            1
different          1
is                 2
sample             2
sentence           2
several            2
this               2
with               2
words              2

size: 10
isEmpty: false
```

Fig. 20.18 | Program counts the number of occurrences of each word in a `String`. (Part 3 of 3.)

Line 14 creates an empty `HashMap` with a default initial capacity (16 elements) and a default load factor (0.75)—these defaults are built into the implementation of `HashMap`. When the number of occupied slots in the `HashMap` becomes greater than the capacity times the load factor, the capacity is doubled automatically. `HashMap` is a generic class that takes two type arguments—the type of key (i.e., `String`) and the type of value (i.e., `Integer`). Recall that the type arguments passed to a generic class must be reference types, hence the second type argument is `Integer`, not `int`.

Line 16 calls method `createMap` (lines 21–44), which uses a `map` to store the number of occurrences of each word in the sentence. Line 25 obtains the user input, and line 28 tokenizes it. The loop in lines 31–43 converts the next token to lowercase letters (line 33), then calls **`Map` method `containsKey`** (line 36) to determine whether the word is in the map (and thus has occurred previously in the string). If the `Map` does not contain a mapping for the word, line 42 uses **`Map` method `put`** to create a new entry in the map, with the word as the key and an `Integer` object containing `1` as the value. Autoboxing occurs when the program passes integer `1` to method `put`, because the map stores the number of occurrences of the word as an `Integer`. If the word does exist in the map, line 38 uses **`Map` method `get`** to obtain the key's associated value (the count) in the map. Line 39 increments that value and uses `put` to replace the key's associated value in the map. Method `put` returns the key's prior associated value, or `null` if the key was not in the map.

Method `displayMap` (lines 47–62) displays all the entries in the map. It uses **`HashMap` method `keySet`** (line 49) to get a set of the keys. The keys have type `String` in the `map`, so method `keySet` returns a generic type `Set` with type parameter specified to be `String`. Line 52 creates a `TreeSet` of the keys, in which the keys are sorted. The loop in lines 57–58

accesses each key and its value in the map. Line 58 displays each key and its value using format specifier `%-10s` to left justify each key and format specifier `%10s` to right justify each value. The keys are displayed in ascending order. Line 61 calls **`Map` method `size`** to get the number of key/value pairs in the `Map`. Line 61 also calls **`Map` method `isEmpty`**, which returns a `boolean` indicating whether the `Map` is empty.

20.12 Properties Class

A **`Properties`** object is a persistent `Hashtable` that normally stores key/value pairs of strings—assuming that you use methods **`setProperty`** and **`getProperty`** to manipulate the table rather than inherited `Hashtable` methods `put` and `get`. By "persistent," we mean that the `Properties` object can be written to an output stream (possibly a file) and read back in through an input stream. A common use of `Properties` objects in prior versions of Java was to maintain application-configuration data or user preferences for applications. [*Note:* The **Preferences API** (package **`java.util.prefs`**) is meant to replace this particular use of class `Properties` but is beyond the scope of this book. To learn more, visit `bit.ly/JavaPreferences`.]

Class `Properties` extends class `Hashtable<Object, Object>`. Figure 20.19 demonstrates several methods of class `Properties`.

```
1  // Fig. 20.19: PropertiesTest.java
2  // Demonstrates class Properties of the java.util package.
3  import java.io.FileOutputStream;
4  import java.io.FileInputStream;
5  import java.io.IOException;
6  import java.util.Properties;
7  import java.util.Set;
8
9  public class PropertiesTest
10 {
11    public static void main( String[] args )
12    {
13       Properties table = new Properties(); // create Properties table
14
15       // set properties
16       table.setProperty( "color", "blue" );
17       table.setProperty( "width", "200" );
18
19       System.out.println( "After setting properties" );
20       listProperties( table ); // display property values
21
22       // replace property value
23       table.setProperty( "color", "red" );
24
25       System.out.println( "After replacing properties" );
26       listProperties( table ); // display property values
27
28       saveProperties( table ); // save properties
29
```

Fig. 20.19 | `Properties` class of package `java.util`. (Part 1 of 3.)

```
30          table.clear(); // empty table
31
32          System.out.println( "After clearing properties" );
33          listProperties( table ); // display property values
34
35          loadProperties( table ); // load properties
36
37          // get value of property color
38          Object value = table.getProperty( "color" );
39
40          // check if value is in table
41          if ( value != null )
42             System.out.printf( "Property color's value is %s\n", value );
43          else
44             System.out.println( "Property color is not in table" );
45       } // end main
46
47       // save properties to a file
48       private static void saveProperties( Properties props )
49       {
50          // save contents of table
51          try
52          {
53             FileOutputStream output = new FileOutputStream( "props.dat" );
54             props.store( output, "Sample Properties" ); // save properties
55             output.close();
56             System.out.println( "After saving properties" );
57             listProperties( props ); // display property values
58          } // end try
59          catch ( IOException ioException )
60          {
61             ioException.printStackTrace();
62          } // end catch
63       } // end method saveProperties
64
65       // load properties from a file
66       private static void loadProperties( Properties props )
67       {
68          // load contents of table
69          try
70          {
71             FileInputStream input = new FileInputStream( "props.dat" );
72             props.load( input ); // load properties
73             input.close();
74             System.out.println( "After loading properties" );
75             listProperties( props ); // display property values
76          } // end try
77          catch ( IOException ioException )
78          {
79             ioException.printStackTrace();
80          } // end catch
81       } // end method loadProperties
82
```

Fig. 20.19 | `Properties` class of package `java.util`. (Part 2 of 3.)

```
83      // output property values
84      private static void listProperties( Properties props )
85      {
86         Set< Object > keys = props.keySet(); // get property names
87
88         // output name/value pairs
89         for ( Object key : keys )
90            System.out.printf(
91               "%s\t%s\n", key, props.getProperty( ( String ) key ) );
92
93         System.out.println();
94      } // end method listProperties
95   } // end class PropertiesTest
```

```
After setting properties
color   blue
width   200

After replacing properties
color   red
width   200

After saving properties
color   red
width   200

After clearing properties

After loading properties
color   red
width   200

Property color's value is red
```

Fig. 20.19 | `Properties` class of package `java.util`. (Part 3 of 3.)

Line 13 uses the no-argument constructor to create an empty `Properties` `table` with no default properties. Class `Properties` also provides an overloaded constructor that receives a reference to a `Properties` object containing default property values. Lines 16 and 17 each call `Properties` method `setProperty` to store a value for the specified key. If the key does not exist in the `table`, `setProperty` returns `null`; otherwise, it returns the previous value for that key.

Line 38 calls `Properties` method `getProperty` to locate the value associated with the specified key. If the key is not found in this `Properties` object, `getProperty` returns `null`. An overloaded version of this method receives a second argument that specifies the default value to return if `getProperty` cannot locate the key.

Line 54 calls **`Properties` method `store`** to save the `Properties` object's contents to the `OutputStream` specified as the first argument (in this case, a `FileOutputStream`). The second argument, a `String`, is a description written into the file. **`Properties` method `list`**, which takes a `PrintStream` argument, is useful for displaying the list of properties.

Line 72 calls **`Properties` method `load`** to restore the contents of the `Properties` object from the `InputStream` specified as the first argument (in this case, a `FileInput-`

Stream). Line 86 calls Properties method keySet to obtain a Set of the property names. Because class Properties stores its contents as Objects, a Set of Object references is returned. Line 91 obtains the value of a property by passing a key to method getProperty.

20.13 Synchronized Collections

In Chapter 26, we discuss multithreading. Except for Vector and Hashtable, the collections in the collections framework are unsynchronized by default, so they can operate efficiently when multithreading is not required. Because they're unsynchronized, however, concurrent access to a Collection by multiple threads could cause indeterminate results or fatal errors. To prevent potential threading problems, **synchronization wrappers** are used for collections that might be accessed by multiple threads. A **wrapper** object receives method calls, adds thread synchronization (to prevent concurrent access to the collection) and delegates the calls to the wrapped collection object. The Collections API provides a set of static methods for wrapping collections as synchronized versions. Method headers for the synchronization wrappers are listed in Fig. 20.20. Details about these methods are available at download.oracle.com/javase/6/docs/api/java/util/Collections.html. All these methods take a generic type and return a synchronized view of the generic type. For example, the following code creates a synchronized List (list2) that stores String objects:

```
List< String > list1 = new ArrayList< String >();
List< String > list2 = Collections.synchronizedList( list1 );
```

public static method headers
< T > Collection< T > synchronizedCollection(Collection< T > c)
< T > List< T > synchronizedList(List< T > aList)
< T > Set< T > synchronizedSet(Set< T > s)
< T > SortedSet< T > synchronizedSortedSet(SortedSet< T > s)
< K, V > Map< K, V > synchronizedMap(Map< K, V > m)
< K, V > SortedMap< K, V > synchronizedSortedMap(SortedMap< K, V > m)

Fig. 20.20 | Synchronization wrapper methods.

20.14 Unmodifiable Collections

The Collections class provides a set of static methods that create **unmodifiable wrappers** for collections. Unmodifiable wrappers throw UnsupportedOperationExceptions if attempts are made to modify the collection. Headers for these methods are listed in Fig. 20.21. Details about these methods are available at download.oracle.com/javase/6/docs/api/java/util/Collections.html. All these methods take a generic type and return an unmodifiable view of the generic type. For example, the following code creates an unmodifiable List (list2) that stores String objects:

```
List< String > list1 = new ArrayList< String >();
List< String > list2 = Collections.unmodifiableList( list1 );
```

public static method headers

```
< T > Collection< T > unmodifiableCollection( Collection< T > c )
< T > List< T > unmodifiableList( List< T > aList )
< T > Set< T > unmodifiableSet( Set< T > s )
< T > SortedSet< T > unmodifiableSortedSet( SortedSet< T > s )
< K, V > Map< K, V > unmodifiableMap( Map< K, V > m )
< K, V > SortedMap< K, V > unmodifiableSortedMap( SortedMap< K, V > m )
```

Fig. 20.21 | Unmodifiable wrapper methods.



Software Engineering Observation 20.5

You can use an unmodifiable wrapper to create a collection that offers read-only access to others, while allowing read/write access to yourself. You do this simply by giving others a reference to the unmodifiable wrapper while retaining for yourself a reference to the original collection.

20.15 Abstract Implementations

The collections framework provides various abstract implementations of `Collection` interfaces from which you can quickly "flesh out" complete customized implementations. These abstract implementations include a thin `Collection` implementation called an **`AbstractCollection`**, a `List` implementation that allows random access to its elements called an **`AbstractList`**, a `Map` implementation called an **`AbstractMap`**, a `List` implementation that allows sequential access to its elements called an **`AbstractSequentialList`**, a `Set` implementation called an **`AbstractSet`** and a `Queue` implementation called **`AbstractQueue`**. You can learn more about these classes at `download.oracle.com/javase/6/docs/api/java/util/package-summary.html`.

To write a custom implementation, you can extend the abstract implementation that best meets your needs, and implement each of the class's `abstract` methods. Then, if your collection is to be modifiable, override any concrete methods that prevent modification.

20.16 Wrap-Up

This chapter introduced the Java collections framework. You learned the collection hierarchy and how to use the collections-framework interfaces to program with collections polymorphically. You used classes `ArrayList` and `LinkedList`, which both implement the `List` interface. We presented Java's built-in interfaces and classes for manipulating stacks and queues. You used several predefined methods for manipulating collections. Next, you learned how to use the `Set` interface and class `HashSet` to manipulate an unordered collection of unique values. We continued our presentation of sets with the `SortedSet` interface and class `TreeSet` for manipulating a sorted collection of unique values. You then learned about Java's interfaces and classes for manipulating key/value pairs—`Map`, `SortedMap`, `Hashtable`, `HashMap` and `TreeMap`. We discussed the specialized `Properties` class for manipulating key/value pairs of `String`s that can be stored to a file and retrieved from a file.

Finally, we discussed the `Collections` class's `static` methods for obtaining unmodifiable and synchronized views of collections. Chapter 21 demonstrates how to use Java's generics capabilities to implement your own generic methods and classes.

Summary

Section 20.1 Introduction

- The Java collections framework (p. 830) provides prebuilt data structures and methods to manipulate them.

Section 20.2 Collections Overview

- A collection is an object that can hold references to other objects.
- The classes and interfaces of the collections framework are in package `java.util`.

Section 20.3 Type-Wrapper Classes for Primitive Types

- Type-wrapper classes (e.g., `Integer`, `Double`, `Boolean`) enable programmers to manipulate primitive-type values as objects (p. 831). Objects of these classes can be used in collections.

Section 20.4 Autoboxing and Auto-Unboxing

- Boxing (p. 832) converts a primitive value to an object of the corresponding type-wrapper class. Unboxing (p. 832) converts a type-wrapper object to the corresponding primitive value.
- Java performs boxing conversions and unboxing conversions automatically.

Section 20.5 Interface `Collection` and Class `Collections`

- Interfaces `Set` and `List` extend `Collection` (p. 832), which contains operations for adding, clearing, comparing and retaining objects in a collection, and method `iterator` (p. 836) to obtain a collection's `Iterator` (p. 833).
- Class `Collections` (p. 833) provides `static` methods for manipulating collections.

Section 20.6 Lists

- A `List` (p. 839) is an ordered `Collection` that can contain duplicate elements.
- Interface `List` is implemented by classes `ArrayList`, `LinkedList` and `Vector`. `ArrayList` (p. 833) is a resizable-array implementation. `LinkedList` (p. 833) is a linkedlist implementation of a `List`.
- Java SE 7 supports type inferencing with the <> notation in statements that declare and create generic type variables and objects.
- `Iterator` method `hasNext` (p. 836) determines whether a `Collection` contains another element. Method `next` returns a reference to the next object in the `Collection` and advances the `Iterator`.
- Method `subList` (p. 839) returns a view into a `List`. Changes made to this view are also made to the `List`.
- Method `clear` (p. 839) removes elements from a `List`.
- Method `toArray` (p. 840) returns the contents of a collection as an array.

Section 20.7 Collections Methods

- Algorithms `sort` (p. 842), `binarySearch`, `reverse` (p. 847), `shuffle` (p. 845), `fill` (p. 847), `copy`, `addAll` (p. 838), `frequency` and `disjoint` operate on `Lists`. Algorithms `min` and `max` (p. 848) operate on `Collections`.

- Algorithm `addAll` appends all the elements in an array to a collection (p. 851), `frequency` (p. 851) calculates how many elements in the collection are equal to the specified element, and `disjoint` (p. 851) determines whether two collections have elements in common.
- Algorithms `min` and `max` find the smallest and largest items in a collection.
- The `Comparator` interface (p. 842) provides a means of sorting a `Collection`'s elements in an order other than their natural order.
- `Collections` method `reverseOrder` (p. 843) returns a `Comparator` object that can be used with `sort` to sort a collection's elements in reverse order.
- Algorithm `shuffle` (p. 845) randomly orders the elements of a `List`.
- Algorithm `binarySearch` (p. 849) locates an `Object` in a sorted `List`.

Section 20.8 `Stack` Class of Package `java.util`

- Class `Stack` (p. 853) extends `Vector`. `Stack` method `push` (p. 854) adds its argument to the top of the stack. Method `pop` (p. 855) removes the top element of the stack. Method `peek` returns a reference to the top element without removing it. `Stack` method `isEmpty` (p. 855) determines whether the stack is empty.

Section 20.9 Class `PriorityQueue` and Interface `Queue`

- Interface `Queue` (p. 832) extends interface `Collection` and provides additional operations for inserting, removing and inspecting elements in a queue.
- `PriorityQueue` (p. 855) implements interface `Queue` and orders elements by their natural ordering or by a `Comparator` object that is supplied to the constructor.
- `PriorityQueue` method `offer` (p. 855) inserts an element at the appropriate location based on priority order. Method `poll` (p. 855) removes the highest-priority element of the priority queue. Method `peek` (`peek`) gets a reference to the highest-priority element of the priority queue. Method `clear` (p. 855) removes all elements in the priority queue. Method `size` (p. 855) gets the number of elements in the priority queue.

Section 20.10 Sets

- A `Set` (p. 832) is an unordered `Collection` that contains no duplicate elements. `HashSet` (p. 856) stores its elements in a hash table. `TreeSet` (p. 856) stores its elements in a tree.
- Interface `SortedSet` (p. 857) extends `Set` and represents a set that maintains its elements in sorted order. Class `TreeSet` implements `SortedSet`.
- `TreeSet` method `headSet` (p. 858) gets a `TreeSet` view containing elements that are less than a specified element. Method `tailSet` (p. 859) gets a `TreeSet` view containing elements that are greater than or equal to a specified element. Any changes made to these views are made to the original `TreeSet`.

Section 20.11 Maps

- `Maps` (p. 859) store key/value pairs and cannot contain duplicate keys. `HashMaps` (p. 859) and `Hashtables` (p. 859) store elements in a hash table, and `TreeMaps` (p. 859) store elements in a tree.
- `HashMap` takes two type arguments—the type of key and the type of value.
- `HashMap` method `put` (p. 862) adds a key/value pair to a `HashMap`. Method `get` (p. 862) locates the value associated with the specified key. Method `isEmpty` (p. 863) determines if the map is empty.
- `HashMap` method `keySet` (p. 862) returns a set of the keys. `Map` method `size` (p. 863) returns the number of key/value pairs in the `Map`.
- Interface `SortedMap` (p. 859) extends `Map` and represents a map that maintains its keys in sorted order. Class `TreeMap` implements `SortedMap`.

Section 20.12 `Properties` Class

- A `Properties` object (p. 863) is a persistent subclass of `Hashtable`.
- The `Properties` no-argument constructor creates an empty `Properties` table. An overloaded constructor receives a `Properties` object containing default property values.
- `Properties` method `setProperty` (p. 863) specifies the value associated with its key argument. Method `getProperty` (p. 863) locates the value of the key specified as an argument. Method `store` (p. 865) saves the contents of a `Properties` object to specified `OutputStream`. Method `load` (p. 865) restores the contents of a `Properties` object from the specified `InputStream`.

Section 20.13 Synchronized Collections

- Collections from the collections framework are unsynchronized. Synchronization wrappers (p. 866) are provided for collections that can be accessed by multiple threads simultaneously.

Section 20.14 Unmodifiable Collections

- Unmodifiable collection wrappers (p. 866) throw `UnsupportedOperationExceptions` (p. 839) if attempts are made to modify the collection.

Section 20.15 Abstract Implementations

- The collections framework provides various abstract implementations of collection interfaces from which you can quickly flesh out complete customized implementations.

Self-Review Exercises

20.1 Fill in the blanks in each of the following statements:

a) A(n) _____ is used to iterate through a collection and can remove elements from the collection during the iteration.
b) An element in a `List` can be accessed by using the element's _____.
c) Assuming that `myArray` contains references to `Double` objects, _____ occurs when the statement "`myArray[ 0 ] = 1.25;`" executes.
d) Java classes _____ and _____ provide the capabilities of arraylike data structures that can resize themselves dynamically.
e) If you do not specify a capacity increment, the system will _____ the size of the `Vector` each time additional capacity is needed.
f) You can use a(n) _____ to create a collection that offers only read-only access to others while allowing read/write access to yourself.
g) Assuming that `myArray` contains references to `Double` objects, _____ occurs when the statement "`double number = myArray[ 0 ];`" executes.
h) `Collections` algorithm _____ determines if two collections have elements in common.

20.2 Determine whether each statement is *true* or *false*. If *false*, explain why.

a) Values of primitive types may be stored directly in a collection.
b) A `Set` can contain duplicate values.
c) A `Map` can contain duplicate keys.
d) A `LinkedList` can contain duplicate values.
e) `Collections` is an `interface`.
f) `Iterators` can remove elements.
g) With hashing, as the load factor increases, the chance of collisions decreases.
h) A `PriorityQueue` permits `null` elements.

Answers to Self-Review Exercises

20.1 a) `Iterator`. b) index. c) autoboxing. d) `ArrayList`, `Vector`. e) double. f) unmodifiable wrapper. g) auto-unboxing. h) `disjoint`.

20.2 a) False. Autoboxing occurs when adding a primitive type to a collection, which means the primitive type is converted to its corresponding type-wrapper class.
b) False. A `Set` cannot contain duplicate values.
c) False. A `Map` cannot contain duplicate keys.
d) True.
e) False. `Collections` is a `class`; `Collection` is an `interface`.
f) True.
g) False. As the load factor increases, fewer slots are available relative to the total number of slots, so the chance of a collision increases.
h) False. Attempting to insert a `null` element causes a `NullPointerException`.

Execises

20.3 Define each of the following terms:
a) `Collection`
b) `Collections`
c) `Comparator`
d) `List`
e) load factor
f) collision
g) space/time trade-off in hashing
h) `HashMap`

20.4 Explain briefly the operation of each of the following methods of class `Vector`:
a) `add`
b) `set`
c) `remove`
d) `removeAllElements`
e) `removeElementAt`
f) `firstElement`
g) `lastElement`
h) `contains`
i) `indexOf`
j) `size`
k) `capacity`

20.5 Explain why inserting additional elements into a `Vector` object whose current size is less than its capacity is a relatively fast operation and why inserting additional elements into a `Vector` object whose current size is at capacity is a relatively slow operation.

20.6 By extending class `Vector`, Java's designers were able to create class `Stack` quickly. What are the negative aspects of this use of inheritance, particularly for class `Stack`?

20.7 Briefly answer the following questions:
a) What is the primary difference between a `Set` and a `Map`?
b) What happens when you add a primitive type (e.g., `double`) value to a collection?
c) Can you print all the elements in a collection without using an `Iterator`? If yes, how?

20.8 Explain briefly the operation of each of the following `Iterator`-related methods:
a) `iterator`
b) `hasNext`
c) `next`

20.9 Explain briefly the operation of each of the following methods of class `HashMap`:
a) `put`

b) `get`
c) `isEmpty`
d) `containsKey`
e) `keySet`

20.10 Determine whether each of the following statements is *true* or *false*. If *false*, explain why.
a) Elements in a `Collection` must be sorted in ascending order before a `binarySearch` may be performed.
b) Method `first` gets the first element in a `TreeSet`.
c) A `List` created with `Arrays` method `asList` is resizable.

20.11 Explain the operation of each of the following methods of the `Properties` class:
a) `load`
b) `store`
c) `getProperty`
d) `list`

20.12 Rewrite lines 16–25 in Fig. 20.3 to be more concise by using the `asList` method and the `LinkedList` constructor that takes a `Collection` argument.

20.13 ***(Duplicate Elimination)*** Write a program that reads in a series of first names and eliminates duplicates by storing them in a `Set`. Allow the user to search for a first name.

20.14 ***(Counting Letters)*** Modify the program of Fig. 20.18 to count the number of occurrences of each letter rather than of each word. For example, the string `"HELLO THERE"` contains two `H`s, three `E`s, two `L`s, one `O`, one `T` and one `R`. Display the results.

20.15 ***(Color Chooser)*** Use a `HashMap` to create a reusable class for choosing one of the 13 predefined colors in class `Color`. The names of the colors should be used as keys, and the predefined `Color` objects should be used as values. Place this class in a package that can be imported into any Java program. Use your new class in an application that allows the user to select a color and draw a shape in that color.

20.16 ***(Counting Duplicate Words)*** Write a program that determines and prints the number of duplicate words in a sentence. Treat uppercase and lowercase letters the same. Ignore punctuation.

20.17 ***(Inserting Elements in a `LinkedList` in Sorted Order)*** Write a program that inserts 25 random integers from 0 to 100 in order into a `LinkedList` object. The program should sort the elements, then calculate the sum of the elements and the floating-point average of the elements.

20.18 ***(Copying and Reversing `LinkedList`s)*** Write a program that creates a `LinkedList` object of 10 characters, then creates a second `LinkedList` object containing a copy of the first list, but in reverse order.

20.19 ***(Prime Numbers and Prime Factors)*** Write a program that takes a whole number input from a user and determines whether it's prime. If the number is not prime, display its unique prime factors. Remember that a prime number's factors are only 1 and the prime number itself. Every number that is not prime has a unique prime factorization. For example, consider the number 54. The prime factors of 54 are 2, 3, 3 and 3. When the values are multiplied together, the result is 54. For the number 54, the prime factors output should be 2 and 3. Use `Set`s as part of your solution.

20.20 ***(Sorting Words with a `TreeSet`)*** Write a program that uses a `String` method `split` to tokenize a line of text input by the user and places each token in a `TreeSet`. Print the elements of the `TreeSet`. [*Note:* This should cause the elements to be printed in ascending sorted order.]

20.21 ***(Changing a `PriorityQueue`'s Sort Order)*** The output of Fig. 20.15 (`PriorityQueueTest`) shows that `PriorityQueue` orders `Double` elements in ascending order. Rewrite Fig. 20.15 so that it orders `Double` elements in descending order (i.e., `9.8` should be the highest-priority element rather than `3.2`).

Generic Classes and Methods

21



Every man of genius sees the world at a different angle from his fellows.
—Havelock Ellis

…our special individuality, as distinguished from our generic humanity.
—Oliver Wendell Holmes, Sr.

Born under one law, to another bound.
—Lord Brooke

Objectives

In this chapter you'll learn:

- To create generic methods that perform identical tasks on arguments of different types.
- To create a generic `Stack` class that can be used to store objects of any class or interface type.
- To understand how to overload generic methods with nongeneric methods or with other generic methods.
- To understand raw types and how they help achieve backward compatibility.
- To use wildcards when precise type information about a parameter is not required in the method body.

Outline

21.1 Introduction
21.2 Motivation for Generic Methods
21.3 Generic Methods: Implementation and Compile-Time Translation
21.4 Additional Compile-Time Translation Issues: Methods That Use a Type Parameter as the Return Type
21.5 Overloading Generic Methods
21.6 Generic Classes
21.7 Raw Types
21.8 Wildcards in Methods That Accept Type Parameters
21.9 Generics and Inheritance: Notes
21.10 Wrap-Up

Summary | *Self-Review Exercises* | *Answers to Self-Review Exercises* | *Exercises*

21.1 Introduction

You've used existing generic methods and classes in Chapters 7 and 20. In this chapter, you'll learn how to write your own. You'll also learn the relationships between generics and other Java features, such as overloading and inheritance.

It would be nice if we could write a single `sort` method to sort the elements in an `Integer` array, a `String` array or an array of any type that supports ordering (i.e., its elements can be compared). It would also be nice if we could write a single `Stack` class that could be used as a `Stack` of integers, a `Stack` of floating-point numbers, a `Stack` of `String`s or a `Stack` of any other type. It would be even nicer if we could detect type mismatches at *compile time*—known as **compile-time type safety**. For example, if a `Stack` stores only integers, attempting to push a `String` onto that `Stack` should issue a *compile-time* error.

This chapter discusses **generics**, which provide the means to create the general models mentioned above. **Generic methods** enable you to specify, with a single method declaration, a set of related methods. **Generic classes** (and interfaces) enable you to specify, with a single class (or interface) declaration, a set of related types, respectively. Generics also provide compile-time type safety that allows you to catch invalid types at compile time.

We might write a generic method for sorting an array of objects, then invoke the generic method with `Integer` arrays, `Double` arrays, `String` arrays and so on, to sort the array elements. The compiler could perform type checking to ensure that the array passed to the sorting method contains the same type elements. We might write a single generic `Stack` class that manipulates a stack of objects, then instantiate `Stack` objects for a stack of `Integer`s, a stack of `Double`s, a stack of `String`s and so on. The compiler could perform type checking to ensure that the `Stack` stores elements of the same type.



Software Engineering Observation 21.1

Generic methods and classes are among Java's most powerful capabilities for software reuse with compile-time type safety.

21.2 Motivation for Generic Methods

Overloaded methods are often used to perform *similar* operations on *different* types of data. To motivate generic methods, let's begin with an example (Fig. 21.1) containing overloaded `printArray` methods (lines 21–28, 31–38 and 41–48) that print the `String` representations of the elements of an `Integer` array, a `Double` array and a `Character` array, respectively. We could have used arrays of primitive types `int`, `double` and `char`. We're

using arrays of the type-wrapper classes to set up our generic method example, because *only reference types can be used with generic methods and classes*.

```
1  // Fig. 21.1: OverloadedMethods.java
2  // Printing array elements using overloaded methods.
3  public class OverloadedMethods
4  {
5     public static void main( String[] args )
6     {
7        // create arrays of Integer, Double and Character
8        Integer[] integerArray = { 1, 2, 3, 4, 5, 6 };
9        Double[] doubleArray = { 1.1, 2.2, 3.3, 4.4, 5.5, 6.6, 7.7 };
10       Character[] characterArray = { 'H', 'E', 'L', 'L', 'O' };
11
12       System.out.println( "Array integerArray contains:" );
13       printArray( integerArray ); // pass an Integer array
14       System.out.println( "\nArray doubleArray contains:" );
15       printArray( doubleArray ); // pass a Double array
16       System.out.println( "\nArray characterArray contains:" );
17       printArray( characterArray ); // pass a Character array
18    } // end main
19
20    // method printArray to print Integer array
21    public static void printArray( Integer[] inputArray )
22    {
23       // display array elements
24       for ( Integer element : inputArray )
25          System.out.printf( "%s ", element );
26
27       System.out.println();
28    } // end method printArray
29
30    // method printArray to print Double array
31    public static void printArray( Double[] inputArray )
32    {
33       // display array elements
34       for ( Double element : inputArray )
35          System.out.printf( "%s ", element );
36
37       System.out.println();
38    } // end method printArray
39
40    // method printArray to print Character array
41    public static void printArray( Character[] inputArray )
42    {
43       // display array elements
44       for ( Character element : inputArray )
45          System.out.printf( "%s ", element );
46
47       System.out.println();
48    } // end method printArray
49 } // end class OverloadedMethods
```

Fig. 21.1 | Printing array elements using overloaded methods. (Part 1 of 2.)

```
Array integerArray contains:
1 2 3 4 5 6

Array doubleArray contains:
1.1 2.2 3.3 4.4 5.5 6.6 7.7

Array characterArray contains:
H E L L O
```

Fig. 21.1 | Printing array elements using overloaded methods. (Part 2 of 2.)

The program begins by declaring and initializing three arrays—six-element `Integer` array `integerArray` (line 8), seven-element `Double` array `doubleArray` (line 9) and five-element `Character` array `characterArray` (line 10). Then lines 12–17 display the contents of each array.

When the compiler encounters a method call, it attempts to locate a method declaration with the same name and parameters that match the argument types in the call. In this example, each `printArray` call matches one of the `printArray` method declarations. For example, line 13 calls `printArray` with `integerArray` as its argument. The compiler determines the argument's type (i.e., `Integer[]`) and attempts to locate a `printArray` method that specifies an `Integer[]` parameter (lines 21–28), then sets up a call to that method. Similarly, when the compiler encounters the call at line 15, it determines the argument's type (i.e., `Double[]`), then attempts to locate a `printArray` method that specifies a `Double[]` parameter (lines 31–38), then sets up a call to that method. Finally, when the compiler encounters the call at line 17, it determines the argument's type (i.e., `Character[]`), then attempts to locate a `printArray` method that specifies a `Character[]` parameter (lines 41–48), then sets up a call to that method.

Study each `printArray` method. The array element type appears in each method's header (lines 21, 31 and 41) and `for`-statement header (lines 24, 34 and 44). If we were to replace the element types in each method with a generic name—`T` by convention—then all three methods would look like the one in Fig. 21.2. It appears that if we can replace the array element type in each of the three methods with a *single generic type*, then we should be able to declare *one* `printArray` method that can display the `String` representations of the elements of *any* array that contains objects. The method in Fig. 21.2 is similar to the generic `printArray` method declaration we discuss in Section 21.3.

```
1  public static void printArray( T[] inputArray )
2  {
3     // display array elements
4     for ( T element : inputArray )
5        System.out.printf( "%s ", element );
6
7     System.out.println();
8  } // end method printArray
```

Fig. 21.2 | `printArray` method in which actual type names are replaced by convention with the generic name `T`.

21.3 Generic Methods: Implementation and Compile-Time Translation

If the operations performed by several overloaded methods are *identical* for each argument type, the overloaded methods can be more compactly and conveniently coded using a generic method. You can write a single generic method declaration that can be called with arguments of different types. Based on the types of the arguments passed to the generic method, the compiler handles each method call appropriately.

Figure 21.3 reimplements the application of Fig. 21.1 using a generic `printArray` method (lines 22–29). The `printArray` method calls in lines 14, 16 and 18 are identical to those of Fig. 21.1 (lines 14, 16 and 18) and the outputs of the two applications are identical. This dramatically demonstrates the expressive power of generics.

```
1  // Fig. 21.3: GenericMethodTest.java
2  // Printing array elements using generic method printArray.
3
4  public class GenericMethodTest
5  {
6     public static void main( String[] args )
7     {
8        // create arrays of Integer, Double and Character
9        Integer[] intArray = { 1, 2, 3, 4, 5 };
10       Double[] doubleArray = { 1.1, 2.2, 3.3, 4.4, 5.5, 6.6, 7.7 };
11       Character[] charArray = { 'H', 'E', 'L', 'L', 'O' };
12
13       System.out.println( "Array integerArray contains:" );
14       printArray( integerArray ); // pass an Integer array
15       System.out.println( "\nArray doubleArray contains:" );
16       printArray( doubleArray ); // pass a Double array
17       System.out.println( "\nArray characterArray contains:" );
18       printArray( characterArray ); // pass a Character array
19    } // end main
20
21    // generic method printArray
22    public static < T > void printArray( T[] inputArray )
23    {
24       // display array elements
25       for ( T element : inputArray )
26          System.out.printf( "%s ", element );
27
28       System.out.println();
29    } // end method printArray
30 } // end class GenericMethodTest
```

```
Array integerArray contains:
1 2 3 4 5 6

Array doubleArray contains:
1.1 2.2 3.3 4.4 5.5 6.6 7.7

Array characterArray contains:
H E L L O
```

Fig. 21.3 | Printing array elements using generic method `printArray`.

Line 22 begins method `printArray`'s declaration. All generic method declarations have a **type-parameter section** delimited by **angle brackets** (< and >) that precedes the method's return type (`< T >` in this example). Each type-parameter section contains one or more **type parameters** (also called **formal type parameters**), separated by commas. A type parameter, also known as a **type variable**, is an identifier that specifies a generic type name. The type parameters can be used to declare the return type, parameter types and local variable types in a generic method declaration, and they act as placeholders for the types of the arguments passed to the generic method, which are known as **actual type arguments**. A generic method's body is declared like that of any other method. *Type parameters can represent only reference types*—not primitive types (like `int`, `double` and `char`). Note, too, that the type-parameter names throughout the method declaration must match those declared in the type-parameter section. For example, line 25 declares `element` as type `T`, which matches the type parameter (`T`) declared in line 22. Also, a type parameter can be declared only once in the type-parameter section but can appear more than once in the method's parameter list. For example, the type-parameter name `T` appears twice in the following method's parameter list:

```
public static < T > void printTwoArrays( T[] array1, T[] array2 )
```

Type-parameter names need not be unique among different generic methods.



Common Programming Error 21.1

When declaring a generic method, failing to place a type-parameter section before the return type of a method is a syntax error—the compiler will not understand the type-parameter names when they're encountered in the method.

Method `printArray`'s type-parameter section declares type parameter `T` as the *placeholder* for the array element type that `printArray` will output. `T` appears in the parameter list as the array element type (line 22). The `for`-statement header (line 25) also uses `T` as the element type. These are the same two locations where the overloaded `printArray` methods of Fig. 21.1 specified `Integer`, `Double` or `Character` as the array element type. The remainder of `printArray` is identical to the versions presented in Fig. 21.1.



Good Programming Practice 21.1

It's recommended that type parameters be specified as individual capital letters. Typically, a type parameter that represents an array element's type (or other collection) is named `T`.

As in Fig. 21.1, the program begins by declaring and initializing six-element `Integer` array `integerArray` (line 9), seven-element `Double` array `doubleArray` (line 10) and five-element `Character` array `characterArray` (line 11). Then the program outputs each array by calling `printArray` (lines 14, 16 and 18)—once with argument `integerArray`, once with argument `doubleArray` and once with argument `characterArray`.

When the compiler encounters line 14, it first determines argument `integerArray`'s type (i.e., `Integer[]`) and attempts to locate a method named `printArray` that specifies a single `Integer[]` parameter. There's no such method in this example. Next, the compiler determines whether there's a generic method named `printArray` that specifies a single array parameter and uses a type parameter to represent the array element type. The compiler determines that `printArray` (lines 22–29) is a match and sets up a call to the method. The same process is repeated for the calls to method `printArray` at lines 16 and 18.



Common Programming Error 21.2

If the compiler cannot match a method call to a nongeneric or a generic method declaration, a compilation error occurs.



Common Programming Error 21.3

If the compiler doesn't find a method declaration that matches a method call exactly, but does find two or more methods that can satisfy the method call, a compilation error occurs.

In addition to setting up the method calls, the compiler also determines whether the operations in the method body can be applied to elements of the type stored in the array argument. The only operation performed on the array elements in this example is to output their `String` representation. Line 26 performs an *implicit `toString` call* on every `element`. *To work with generics, every element of the array must be an object of a class or interface type.* Since all objects have a `toString` method, the compiler is satisfied that line 26 performs a *valid* operation for any object in `printArray`'s array argument. The `toString` methods of classes `Integer`, `Double` and `Character` return the `String` representation of the underlying `int`, `double` or `char` value, respectively.

Erasure at Compilation Time

When the compiler translates generic method `printArray` into Java bytecodes, it removes the type-parameter section and *replaces the type parameters with actual types*. This process is known as **erasure**. By default all generic types are replaced with type `Object`. So the compiled version of method `printArray` appears as shown in Fig. 21.4—there's only *one* copy of this code, which is used for all `printArray` calls in the example. This is quite different from other, similar mechanisms, such as C++'s templates, in which a *separate copy of the source code* is generated and compiled for *every* type passed as an argument to the method. As you'll see in Section 21.4, the translation and compilation of generics is a bit more involved than what we've discussed in this section.

By declaring `printArray` as a generic method in Fig. 21.3, we eliminated the need for the overloaded methods of Fig. 21.1, saving 19 lines of code and creating a reusable method that can output the `String` representations of the elements in any array that contains objects. However, this particular example could have simply declared the `printArray` method as shown in Fig. 21.4, using an `Object` array as the parameter. This would have yielded the same results, because any `Object` can be output as a `String`. In a generic method, the benefits become apparent when the method also uses a type parameter as the method's return type, as we demonstrate in the next section.

```
1  public static void printArray( Object[] inputArray )
2  {
3     // display array elements
4     for ( Object element : inputArray )
5        System.out.printf( "%s ", element );
6
7     System.out.println();
8  } // end method printArray
```

Fig. 21.4 | Generic method `printArray` after erasure is performed by the compiler.

21.4 Additional Compile-Time Translation Issues: Methods That Use a Type Parameter as the Return Type

Let's consider a generic method example in which type parameters are used in the return type and in the parameter list (Fig. 21.5). The application uses a generic method `maximum` to determine and return the largest of its three arguments of the same type. Unfortunately, *the relational operator > cannot be used with reference types*. However, it's possible to compare two objects of the same class if that class implements the generic **interface `Comparable<T>`** (package `java.lang`). All the type-wrapper classes for primitive types implement this interface. Like generic classes, **generic interfaces** enable you to specify, with a single interface declaration, a set of related types. `Comparable<T>` objects have a **`compareTo` method**. For example, if we have two `Integer` objects, `integer1` and `integer2`, they can be compared with the expression:

```
integer1.compareTo( integer2 )
```

It's your responsibility when you declare a class that implements `Comparable<T>` to declare method `compareTo` such that it compares the contents of two objects of that class and returns the comparison results. As specified in interface `Comparable<T>`'s documentation, `compareTo` *must* return `0` if the objects are equal, a negative integer if `object1` is less than `object2` or a positive integer if `object1` is greater than `object2`. For example, class `Integer`'s `compareTo` method compares the `int` values stored in two `Integer` objects. A benefit of implementing interface `Comparable<T>` is that `Comparable<T>` objects can be used with the sorting and searching methods of class `Collections` (package `java.util`). We discussed those methods in Chapter 20. In this example, we'll use method `compareTo` in method `maximum` to help determine the largest value.

```
1  // Fig. 21.5: MaximumTest.java
2  // Generic method maximum returns the largest of three objects.
3
4  public class MaximumTest
5  {
6     public static void main( String[] args )
7     {
8        System.out.printf( "Maximum of %d, %d and %d is %d\n\n", 3, 4, 5,
9           maximum( 3, 4, 5 ) );
10       System.out.printf( "Maximum of %.1f, %.1f and %.1f is %.1f\n\n",
11          6.6, 8.8, 7.7, maximum( 6.6, 8.8, 7.7 ) );
12       System.out.printf( "Maximum of %s, %s and %s is %s\n", "pear",
13          "apple", "orange", maximum( "pear", "apple", "orange" ) );
14    } // end main
15
16    // determines the largest of three Comparable objects
17    public static < T extends Comparable< T > > T maximum( T x, T y, T z )
18    {
19       T max = x; // assume x is initially the largest
20
21       if ( y.compareTo( max ) > 0 )
22          max = y; // y is the largest so far
```

Fig. 21.5 | Generic method `maximum` with an upper bound on its type parameter. (Part 1 of 2.)

```
23
24          if ( z.compareTo( max ) > 0 )
25             max = z; // z is the largest
26
27          return max; // returns the largest object
28       } // end method maximum
29    } // end class MaximumTest
```

```
Maximum of 3, 4 and 5 is 5

Maximum of 6.6, 8.8 and 7.7 is 8.8

Maximum of pear, apple and orange is pear
```

Fig. 21.5 | Generic method `maximum` with an upper bound on its type parameter. (Part 2 of 2.)

Generic Method `maximum`

Generic method `maximum` (lines 17–28) uses type parameter `T` as the return type of the method (line 17), as the type of method parameters `x`, `y` and `z` (line 17), and as the type of local variable `max` (line 19). The type-parameter section specifies that `T` extends `Comparable<T>`—only objects of classes that implement interface `Comparable<T>` can be used with this method. In this case, `Comparable` is known as the **upper bound** of the type parameter. By default, `Object` is the upper bound. Type-parameter declarations that bound the parameter always use keyword `extends` regardless of whether the type parameter extends a class or implements an interface. This type parameter is more restrictive than the one specified for `printArray` in Fig. 21.3, which was able to output arrays containing any type of object. The restriction of using `Comparable<T>` objects is important, because not all objects can be compared. However, `Comparable<T>` objects are guaranteed to have a `compareTo` method.

Method `maximum` uses the same algorithm that we used in Section 6.4 to determine the largest of its three arguments. The method assumes that its first argument (`x`) is the largest and assigns it to local variable `max` (line 19). Next, the `if` statement at lines 21–22 determines whether `y` is greater than `max`. The condition invokes `y`'s `compareTo` method with the expression `y.compareTo(max)`, which returns a negative integer, `0` or a positive integer, to determine `y`'s relationship to `max`. If the return value of the `compareTo` is greater than `0`, then `y` is greater and is assigned to variable `max`. Similarly, the `if` statement at lines 24–25 determines whether `z` is greater than `max`. If so, line 25 assigns `z` to `max`. Then line 27 returns `max` to the caller.

Calling Method `maximum`

In `main` (lines 6–14), line 9 calls `maximum` with the integers `3`, `4` and `5`. When the compiler encounters this call, it first looks for a `maximum` method that takes three arguments of type `int`. There's no such method, so the compiler looks for a generic method that can be used and finds generic method `maximum`. However, recall that the arguments to a generic method must be of a *reference type*. So the compiler autoboxes the three `int` values as `Integer` objects and specifies that the three `Integer` objects will be passed to `maximum`. Class `Integer` (package `java.lang`) implements the `Comparable<Integer>` interface such that method `compareTo` compares the `int` values in two `Integer` objects. Therefore, `Integer`s are valid arguments to method `maximum`. When the `Integer` representing the maximum is re-

turned, we attempt to output it with the `%d` format specifier, which outputs an `int` primitive-type value. So `maximum`'s return value is output as an `int` value.

A similar process occurs for the three `double` arguments passed to `maximum` in line 11. Each `double` is autoboxed as a `Double` object and passed to `maximum`. Again, this is allowed because class `Double` (package `java.lang`) implements the `Comparable<Double>` interface. The `Double` returned by `maximum` is output with the format specifier `%.1f`, which outputs a `double` primitive-type value. So `maximum`'s return value is auto-unboxed and output as a `double`. The call to `maximum` in line 13 receives three `String`s, which are also `Comparable<String>` objects. We intentionally placed the largest value in a different position in each method call (lines 9, 11 and 13) to show that the generic method always finds the maximum value, regardless of its position in the argument list.

Upper Bound of a Type Parameter

When the compiler translates method `maximum` into bytecodes, it uses erasure (introduced in Section 21.3) to replace the type parameters with actual types. In Fig. 21.3, all generic types were replaced with type `Object`. Actually, all type parameters are replaced with the so-called *upper bound* of the type parameter, which is specified in the type-parameter section. To indicate the upper bound, follow the type parameter's name with the keyword `extends` and the class or interface name that represents the upper bound, or a comma-separated list of the types that represent the upper bound. The list may contain zero or one class and zero or more interfaces. For example, in method `maximum`'s type-parameter section (Fig. 21.5), we specified the upper bound of the type parameter `T` as type `Comparable<T>` as follows:

```
T extends Comparable< T >
```

Thus, only `Comparable<T>` objects can be passed as arguments to `maximum`—anything that is not a `Comparable<T>` will result in compilation errors. Unless specified otherwise, `Object` is the default upper bound. Figure 21.6 simulates the erasure of method maximum's types by showing the method's source code after the type-parameter section is removed and type parameter `T` is replaced with the upper bound, `Comparable`, throughout the method declaration. The erasure of `Comparable<T>` is simply `Comparable`.

```
1  public static Comparable maximum(Comparable x, Comparable y, Comparable z)
2  {
3     Comparable max = x; // assume x is initially the largest
4
5     if ( y.compareTo( max ) > 0 )
6        max = y; // y is the largest so far
7
8     if ( z.compareTo( max ) > 0 )
9        max = z; // z is the largest
10
11    return max; // returns the largest object
12 } // end method maximum
```

Fig. 21.6 | Generic method `maximum` after erasure is performed by the compiler.

After erasure, method `maximum` specifies that it returns type `Comparable`. However, the calling method does not expect to receive a `Comparable`. It expects to receive an object of the same type that was passed to `maximum` as an argument—`Integer`, `Double` or `String`

in this example. When the compiler replaces the type-parameter information with the upper-bound type in the method declaration, it also inserts *explicit cast operations* in front of each method call to ensure that the returned value is of the type expected by the caller. Thus, the call to `maximum` in line 9 (Fig. 21.5) is preceded by an `Integer` cast, as in

```
(Integer) maximum( 3, 4, 5 )
```

the call to `maximum` in line 11 is preceded by a `Double` cast, as in

```
(Double) maximum( 6.6, 8.8, 7.7 )
```

and the call to `maximum` in line 13 is preceded by a `String` cast, as in

```
(String) maximum( "pear", "apple", "orange" )
```

In each case, the type of the cast for the return value is *inferred* from the types of the method arguments in the particular method call, because, according to the method declaration, the return type and the argument types match.

Possible `ClassCastException`*s*

In this example, you cannot use a method that accepts `Objects`, because class `Object` provides only an equality comparison. Also, without generics, you'd be responsible for implementing the cast operation. Using generics ensures that the inserted cast will never throw a `ClassCastException`, assuming that generics are used throughout your code (i.e., you do not mix old code with new generics code).

21.5 Overloading Generic Methods

A generic method may be overloaded. A class can provide two or more generic methods that specify the same method name but different method parameters. For example, generic method `printArray` of Fig. 21.3 could be overloaded with another `printArray` generic method with the additional parameters `lowSubscript` and `highSubscript` to specify the portion of the array to output (see Exercise 21.5).

A generic method can also be overloaded by nongeneric methods. When the compiler encounters a method call, it searches for the method declaration that most precisely matches the method name and the argument types specified in the call. For example, generic method `printArray` of Fig. 21.3 could be overloaded with a version that's specific to `Strings`, which outputs the `Strings` in neat, tabular format (see Exercise 21.6).

When the compiler encounters a method call, it performs a matching process to determine which method to invoke. The compiler tries to find and use a precise match in which the method name and argument types of the method call match those of a specific method declaration. If there's no such method, the compiler attempts to find a method with compatible types or a matching generic method.

21.6 Generic Classes

The concept of a data structure, such as a stack, can be understood *independently* of the element type it manipulates. Generic classes provide a means for describing the concept of a stack (or any other class) in a type-independent manner. We can then instantiate type-specific objects of the generic class. This capability provides a wonderful opportunity for software reusability.

Once you have a generic class, you can use a simple, concise notation to indicate the type(s) that should be used in place of the class's type parameter(s). At compilation time, the compiler ensures the *type safety* of your code and uses the *erasure* techniques described in Sections 21.3–21.4 to enable your client code to interact with the generic class.

One generic `Stack` class, for example, could be the basis for creating many logical `Stack` classes (e.g., "`Stack` of `Double`," "`Stack` of `Integer`," "`Stack` of `Character`," "`Stack` of `Employee`"). These classes are known as **parameterized classes** or **parameterized types** because they accept one or more type parameters. Recall that type parameters represent only *reference types*, which means the `Stack` generic class cannot be instantiated with primitive types. However, we can instantiate a `Stack` that stores objects of Java's type-wrapper classes and allow Java to use *autoboxing* to convert the primitive values into objects. Recall that autoboxing occurs when a value of a primitive type (e.g., `int`) is pushed onto a `Stack` that contains wrapper-class objects (e.g., `Integer`). *Auto-unboxing* occurs when an object of the wrapper class is popped off the `Stack` and assigned to a primitive-type variable.

Implementing a Generic `Stack` *Class*

Figure 21.7 presents a generic `Stack` class declaration. A generic class declaration looks like a nongeneric one, but the class name is followed by *a type-parameter section* (line 5). In this case, type parameter `T` represents the element type the `Stack` will manipulate. As with generic methods, the type-parameter section of a generic class can have one or more type parameters separated by commas. (You'll create a generic class with two type parameters in Exercise 21.8.) Type parameter `T` is used throughout the `Stack` class declaration to represent the element type. This example implements a `Stack` as an `ArrayList`.

```
1  // Fig. 21.7: Stack.java
2  // Stack generic class declaration.
3  import java.util.ArrayList;
4
5  public class Stack< T >
6  {
7     private ArrayList< T > elements; // ArrayList stores stack elements
8
9     // no-argument constructor creates a stack of the default size
10    public Stack()
11    {
12       this( 10 ); // default stack size
13    } // end no-argument Stack constructor
14
15    // constructor creates a stack of the specified number of elements
16    public Stack( int capacity )
17    {
18       int initCapacity = capacity > 0 ? capacity : 10; // validate
19       elements = new ArrayList< T >( initCapacity ); // create ArrayList
20    } // end one-argument Stack constructor
21
22    // push element onto stack
23    public void push( T pushValue )
24    {
```

Fig. 21.7 | `Stack` generic class declaration. (Part 1 of 2.)

```
25          elements.add( pushValue ); // place pushValue on Stack
26       } // end method push
27
28       // return the top element if not empty; else throw EmptyStackException
29       public T pop()
30       {
31          if ( elements.isEmpty() ) // if stack is empty
32             throw new EmptyStackException( "Stack is empty, cannot pop" );
33
34          // remove and return top element of Stack
35          return elements.remove( elements.size() - 1 );
36       } // end method pop
37    } // end class Stack< T >
```

Fig. 21.7 | `Stack` generic class declaration. (Part 2 of 2.)

Class `Stack` declares variable `elements` as an `ArrayList<T>` (line 7). This `ArrayList` will store the `Stack`'s elements. As you know, an `ArrayList` can grow dynamically, so objects of our `Stack` class can also grow dynamically. The `Stack` class's no-argument constructor (lines 10–13) invokes the one-argument constructor (lines 16–20) to create a `Stack` in which the underlying `ArrayList` has a capacity of 10 elements. The one-argument constructor can also be called directly to create a `Stack` with a specified initial capacity. Line 18 validates the constructor's argument. Line 19 creates the `ArrayList` of the specified capacity (or 10 if the capacity was invalid).

Method `push` (lines 23–26) uses `ArrayList` method `add` to append the pushed item to the end of the `ArrayList` `elements`. The last element in the `ArrayList` represents the *top* of the stack.

Method `pop` (lines 29–36) first determines whether an attempt is being made to pop an element from an empty `Stack`. If so, line 32 throws an `EmptyStackException` (declared in Fig. 21.8). Otherwise, line 35 returns the top element of the `Stack` by removing the last element in the underlying `ArrayList`.

Class `EmptyStackException` (Fig. 21.8) provides a no-argument constructor and a one-argument constructor. The no-argument constructor sets the default error message, and the one-argument constructor sets a custom error message.

```
1  // Fig. 21.8: EmptyStackException.java
2  // EmptyStackException class declaration.
3  public class EmptyStackException extends RuntimeException
4  {
5     // no-argument constructor
6     public EmptyStackException()
7     {
8        this( "Stack is empty" );
9     } // end no-argument EmptyStackException constructor
10
11    // one-argument constructor
12    public EmptyStackException( String message )
13    {
```

Fig. 21.8 | `EmptyStackException` class declaration. (Part 1 of 2.)

```
14         super( message );
15      } // end one-argument EmptyStackException constructor
16   } // end class EmptyStackException
```

Fig. 21.8 | `EmptyStackException` class declaration. (Part 2 of 2.)

As with generic methods, when a generic class is compiled, the compiler performs *erasure* on the class's type parameters and replaces them with their upper bounds. For class `Stack` (Fig. 21.7), no upper bound is specified, so the default upper bound, `Object`, is used. The scope of a generic class's type parameter is the entire class. However, type parameters *cannot* be used in a class's `static` variable declarations.

Testing the Generic `Stack` *Class of Fig. 21.7*

Now, let's consider the application (Fig. 21.9) that uses the `Stack` generic class (Fig. 21.7). Lines 12–13 create and initialize variables of type `Stack<Double>` (pronounced "`Stack` of `Double`") and `Stack<Integer>` (pronounced "`Stack` of `Integer`"). The types `Double` and `Integer` are known as the `Stack`'s **type arguments**. The compiler uses them to replace the type parameters so that it can perform type checking and insert cast operations as necessary. We'll discuss the cast operations in more detail shortly. Lines 12–13 instantiate `doubleStack` with a capacity of `5` and `integerStack` with a capacity of 10 (the default). Lines 16–17 and 20–21 call methods `testPushDouble` (lines 25–36), `testPopDouble` (lines 39–59), `testPushInteger` (lines 62–73) and `testPopInteger` (lines 76–96), respectively, to demonstrate the two `Stacks` in this example.

```
1  // Fig. 21.9: StackTest.java
2  // Stack generic class test program.
3
4  public class StackTest
5  {
6     public static void main( String[] args )
7     {
8        double[] doubleElements = { 1.1, 2.2, 3.3, 4.4, 5.5 };
9        int[] integerElements = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
10
11       // Create a Stack< Double > and a Stack< Integer >
12       Stack< Double > doubleStack = new Stack< Double >( 5 );
13       Stack< Integer > integerStack = new Stack< Integer >();
14
15       // push elements of doubleElements onto doubleStack
16       testPushDouble( doubleStack, doubleElements );
17       testPopDouble( doubleStack ); // pop from doubleStack
18
19       // push elements of integerElements onto integerStack
20       testPushInteger( integerStack, integerElements );
21       testPopInteger( integerStack ); // pop from integerStack
22    } // end main
23
```

Fig. 21.9 | `Stack` generic class test program. (Part 1 of 3.)

```
24      // test push method with double stack
25      private static void testPushDouble(
26         Stack< Double > stack, double[] values )
27      {
28         System.out.println( "\nPushing elements onto doubleStack" );
29
30         // push elements to Stack
31         for ( double value : values )
32         {
33            System.out.printf( "%.1f ", value );
34            stack.push( value ); // push onto doubleStack
35         } // end for
36      } // end method testPushDouble
37
38      // test pop method with double stack
39      private static void testPopDouble( Stack< Double > stack )
40      {
41         // pop elements from stack
42         try
43         {
44            System.out.println( "\nPopping elements from doubleStack" );
45            double popValue; // store element removed from stack
46
47            // remove all elements from Stack
48            while ( true )
49            {
50               popValue = stack.pop(); // pop from doubleStack
51               System.out.printf( "%.1f ", popValue );
52            } // end while
53         } // end try
54         catch( EmptyStackException emptyStackException )
55         {
56            System.err.println();
57            emptyStackException.printStackTrace();
58         } // end catch EmptyStackException
59      } // end method testPopDouble
60
61      // test push method with integer stack
62      private static void testPushInteger(
63         Stack< Integer > stack, int[] values )
64      {
65         System.out.println( "\nPushing elements onto integerStack" );
66
67         // push elements to Stack
68         for ( int value : values )
69         {
70            System.out.printf( "%d ", value );
71            stack.push( value ); // push onto integerStack
72         } // end for
73      } // end method testPushInteger
74
```

Fig. 21.9 | Stack generic class test program. (Part 2 of 3.)

```
75      // test pop method with integer stack
76      private static void testPopInteger( Stack< Integer > stack )
77      {
78         // pop elements from stack
79         try
80         {
81            System.out.println( "\nPopping elements from integerStack" );
82            int popValue; // store element removed from stack
83
84            // remove all elements from Stack
85            while ( true )
86            {
87               popValue = stack.pop(); // pop from intStack
88               System.out.printf( "%d ", popValue );
89            } // end while
90         } // end try
91         catch( EmptyStackException emptyStackException )
92         {
93            System.err.println();
94            emptyStackException.printStackTrace();
95         } // end catch EmptyStackException
96      } // end method testPopInteger
97   } // end class StackTest
```

```
Pushing elements onto doubleStack
1.1 2.2 3.3 4.4 5.5
Popping elements from doubleStack
5.5 4.4 3.3 2.2 1.1
EmptyStackException: Stack is empty, cannot pop
        at Stack.pop(Stack.java:32)
        at StackTest.testPopDouble(StackTest.java:50)
        at StackTest.main(StackTest.java:17)

Pushing elements onto integerStack
1 2 3 4 5 6 7 8 9 10
Popping elements from integerStack
10 9 8 7 6 5 4 3 2 1
EmptyStackException: Stack is empty, cannot pop
        at Stack.pop(Stack.java:32)
        at StackTest.testPopInteger(StackTest.java:87)
        at StackTest.main(StackTest.java:21)
```

Fig. 21.9 | `Stack` generic class test program. (Part 3 of 3.)

Methods `testPushDouble` *and* `testPopDouble`

Method `testPushDouble` (lines 25–36) invokes method `push` (line 34) to place the `double` values 1.1, 2.2, 3.3, 4.4 and 5.5 from array `doubleElements` onto `doubleStack`. *Autoboxing* occurs in line 34 when the program tries to push a primitive `double` value onto the `doubleStack`, which stores only references to `Double` objects.

Method `testPopDouble` (lines 39–59) invokes `Stack` method `pop` (line 50) in an infinite `while` loop (lines 48–52) to remove all the values from the stack. Note in the output that the values indeed `pop` off in last-in, first-out order (the defining characteristic of stacks). When the loop attempts to `pop` a sixth value, the `doubleStack` is empty, so the `pop`

throws an `EmptyStackException`, which causes the program to proceed to the `catch` block (lines 54–58) to handle the exception. The stack trace indicates the exception that occurred and shows that `Stack` method `pop` generated the exception at line 32 of the file `Stack.java` (Fig. 21.7). The trace also shows that method `pop` was called by `StackTest` method `testPopDouble` at line 50 of `StackTest.java` and that method `testPopDouble` was called from method `main` at line 17 of `StackTest.java`. This information enables you to determine the methods that were on the method-call stack at the time that the exception occurred. Because the program catches the exception, the exception is considered to have been handled and the program can continue executing.

Auto-unboxing occurs in line 50 when the program assigns the `Double` object popped from the stack to a `double` primitive variable. Recall from Section 21.4 that the compiler inserts casts to ensure that the proper types are returned from generic methods. After erasure, `Stack` method `pop` returns type `Object`, but the client code in `testPopDouble` expects to receive a `double` when method `pop` returns. So the compiler inserts a `Double` cast, as in

```
popValue = ( Double ) stack.pop();
```

The value assigned to `popValue` will be *unboxed* from the `Double` object returned by `pop`.

Methods `testPushInteger` *and* `testPopInteger`

Method `testPushInteger` (lines 62–73) invokes `Stack` method `push` to place values onto `integerStack` until it's full. Method `testPopInteger` (lines 76–96) invokes `Stack` method `pop` to remove values from `integerStack`. Once again, the values are popped in last-in, first-out order. During *erasure*, the compiler recognizes that the client code in method `testPopInteger` expects to receive an `int` when method `pop` returns. So the compiler inserts an `Integer` cast, as in

```
popValue = ( Integer ) stack.pop();
```

The value assigned to `popValue` will be unboxed from the `Integer` object returned by `pop`.

Creating Generic Methods to Test Class `Stack<T>`

The code in methods `testPushDouble` and `testPushInteger` is *almost identical* for pushing values onto a `Stack<Double>` or a `Stack<Integer>`, respectively, and the code in methods `testPopDouble` and `testPopInteger` is almost identical for popping values from a `Stack<Double>` or a `Stack<Integer>`, respectively. This presents another opportunity to use generic methods. Figure 21.10 declares generic method `testPush` (lines 24–35) to perform the same tasks as `testPushDouble` and `testPushInteger` in Fig. 21.9—that is, `push` values onto a `Stack<T>`. Similarly, generic method `testPop` (lines 38–58) performs the same tasks as `testPopDouble` and `testPopInteger` in Fig. 21.9—that is, `pop` values off a `Stack<T>`. The output of Fig. 21.10 precisely matches that of Fig. 21.9.

```
1  // Fig. 21.10: StackTest2.java
2  // Passing generic Stack objects to generic methods.
3  public class StackTest2
4  {
5     public static void main( String[] args )
6     {
```

Fig. 21.10 | Passing generic `Stack` objects to generic methods. (Part 1 of 3.)

```
7         Double[] doubleElements = { 1.1, 2.2, 3.3, 4.4, 5.5 };
8         Integer[] integerElements = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
9
10        // Create a Stack< Double > and a Stack< Integer >
11        Stack< Double > doubleStack = new Stack< Double >( 5 );
12        Stack< Integer > integerStack = new Stack< Integer >();
13
14        // push elements of doubleElements onto doubleStack
15        testPush( "doubleStack", doubleStack, doubleElements );
16        testPop( "doubleStack", doubleStack ); // pop from doubleStack
17
18        // push elements of integerElements onto integerStack
19        testPush( "integerStack", integerStack, integerElements );
20        testPop( "integerStack", integerStack ); // pop from integerStack
21     } // end main
22
23     // generic method testPush pushes elements onto a Stack
24     public static < T > void testPush( String name , Stack< T > stack,
25        T[] elements )
26     {
27        System.out.printf( "\nPushing elements onto %s\n", name );
28
29        // push elements onto Stack
30        for ( T element : elements )
31        {
32           System.out.printf( "%s ", element );
33           stack.push( element ); // push element onto stack
34        } // end for
35     } // end method testPush
36
37     // generic method testPop pops elements from a Stack
38     public static < T > void testPop( String name, Stack< T > stack )
39     {
40        // pop elements from stack
41        try
42        {
43           System.out.printf( "\nPopping elements from %s\n", name );
44           T popValue; // store element removed from stack
45
46           // remove all elements from Stack
47           while ( true )
48           {
49              popValue = stack.pop();
50              System.out.printf( "%s ", popValue );
51           } // end while
52        } // end try
53        catch( EmptyStackException emptyStackException )
54        {
55           System.out.println();
56           emptyStackException.printStackTrace();
57        } // end catch EmptyStackException
58     } // end method testPop
59  } // end class StackTest2
```

Fig. 21.10 | Passing generic `Stack` objects to generic methods. (Part 2 of 3.)

```
Pushing elements onto doubleStack
1.1 2.2 3.3 4.4 5.5
Popping elements from doubleStack
5.5 4.4 3.3 2.2 1.1
EmptyStackException: Stack is empty, cannot pop
        at Stack.pop(Stack.java:32)
        at StackTest2.testPop(StackTest2.java:50)
        at StackTest2.main(StackTest2.java:17)

Pushing elements onto integerStack
1 2 3 4 5 6 7 8 9 10
Popping elements from integerStack
10 9 8 7 6 5 4 3 2 1
EmptyStackException: Stack is empty, cannot pop
        at Stack.pop(Stack.java:32)
        at StackTest2.testPop(StackTest2.java:50)
        at StackTest2.main(StackTest2.java:21
```

Fig. 21.10 | Passing generic `Stack` objects to generic methods. (Part 3 of 3.)

Lines 11–12 create the `Stack<Double>` and `Stack<Integer>` objects, respectively. Lines 15–16 and 19–20 invoke generic methods `testPush` and `testPop` to test the `Stack` objects. Because type parameters can represent only reference types, to be able to pass arrays `doubleElements` and `integerElements` to generic method `testPush`, the arrays declared in lines 7–8 must be declared with the wrapper types `Double` and `Integer`. When these arrays are initialized with primitive values, the compiler *autoboxes* each primitive value.

Generic method `testPush` (lines 24–35) uses type parameter `T` (specified at line 24) to represent the data type stored in the `Stack<T>`. The generic method takes three arguments—a `String` that represents the name of the `Stack<T>` object for output purposes, a reference to an object of type `Stack<T>` and an array of type `T`—the type of elements that will be `push`ed onto `Stack<T>`. The compiler enforces *consistency* between the type of the `Stack` and the elements that will be pushed onto the `Stack` when `push` is invoked, which is the real value of the generic method call. Generic method `testPop` (lines 38–58) takes two arguments—a `String` that represents the name of the `Stack<T>` object for output purposes and a reference to an object of type `Stack<T>`.

21.7 Raw Types

The test programs for generic class `Stack` in Section 21.6 instantiate `Stacks` with type arguments `Double` and `Integer`. It's also possible to instantiate generic class `Stack` without specifying a type argument, as follows:

```
Stack objectStack = new Stack( 5 ); // no type-argument specified
```

In this case, the `objectStack` is said to have a **raw type**, which means that the compiler implicitly uses type `Object` throughout the generic class for each type argument. Thus the preceding statement creates a `Stack` that can store objects of any type. This is important for *backward compatibility* with prior versions of Java. For example, the data structures of the Java Collections Framework (see Chapter 20) all stored references to `Objects`, but are now implemented as generic types.

A raw-type `Stack` variable can be assigned a `Stack` that specifies a type argument, such as a `Stack<Double>` object, as follows:

```
Stack rawTypeStack2 = new Stack< Double >( 5 );
```

because type `Double` is a subclass of `Object`. This assignment is allowed because the elements in a `Stack<Double>` (i.e., `Double` objects) are certainly objects—class `Double` is an indirect subclass of `Object`.

Similarly, a `Stack` variable that specifies a type argument in its declaration can be assigned a raw-type `Stack` object, as in:

```
Stack< Integer > integerStack = new Stack( 10 );
```

Although this assignment is permitted, it's *unsafe*, because a `Stack` of raw type might store types other than `Integer`. In this case, the compiler issues a warning message which indicates the unsafe assignment.

Using Raw Types with Generic Class Stack

The test program of Fig. 21.11 uses the notion of raw type. Line 11 instantiates generic class `Stack` with raw type, which indicates that `rawTypeStack1` can hold objects of any type. Line 14 assigns a `Stack<Double>` to variable `rawTypeStack2`, which is declared as a `Stack` of raw type. Line 17 assigns a `Stack` of raw type to `Stack<Integer>` variable, which is legal but causes the compiler to issue a warning message (Fig. 21.12) indicating a *potentially unsafe assignment*—again, this occurs because a `Stack` of raw type might store types other than `Integer`. Also, the calls to generic methods `testPush` and `testPop` in lines 19–22 result in compiler warning messages (Fig. 21.12). These occur because `rawTypeStack1` and `rawTypeStack2` are declared as `Stacks` of raw type, but methods `testPush` and `testPop` each expect a second argument that is a `Stack` with a specific type argument. The warnings indicate that the compiler cannot guarantee the types manipulated by the stacks to be the correct types, since we did not supply a variable declared with a type argument. Methods `testPush` (lines 28–39) and `testPop` (lines 42–62) are the same as in Fig. 21.10.

Figure 21.12 shows the warning messages generated by the compiler when the file `RawTypeTest.java` (Fig. 21.11) is compiled with the `-Xlint:unchecked` option, which provides more information about potentially unsafe operations in code that uses generics. The first warning is generated for line 17, which assigned a raw-type `Stack` to a `Stack<Integer>` variable—the compiler cannot ensure that all objects in the `Stack` will be `Integer` objects. The next warning occurs at line 19. The compiler determines method `testPush`'s type argument from the `Double` array passed as the third argument, because the second method argument is a raw-type `Stack` variable. In this case, `Double` is the type argument, so the compiler expects a `Stack<Double>` as the second argument. The warning occurs because the compiler cannot ensure that a raw-type `Stack` contains only `Doubles`. The warning at line 21 occurs for the same reason, even though the actual `Stack` that `rawTypeStack2` references is a `Stack<Double>`. The compiler cannot guarantee that the variable will always refer to the same `Stack` object, so it must use the variable's declared type to perform all type checking. Lines 20 and 22 each generate warnings because method `testPop` expects as an argument a `Stack` for which a type argument has been specified. However, in each call to `testPop`, we pass a raw-type `Stack` variable. Thus, the compiler indicates a warning because it cannot check the types used in the body of the method. In general, you should avoid using raw types.

```
1  // Fig. 21.11: RawTypeTest.java
2  // Raw type test program.
3  public class RawTypeTest
4  {
5     public static void main( String[] args )
6     {
7        Double[] doubleElements = { 1.1, 2.2, 3.3, 4.4, 5.5 };
8        Integer[] integerElements = {  1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
9
10       // Stack of raw types assigned to Stack of raw types variable
11       Stack rawTypeStack1 = new Stack( 5 );
12
13       // Stack< Double > assigned to Stack of raw types variable
14       Stack rawTypeStack2 = new Stack< Double >( 5 );
15
16       // Stack of raw types assigned to Stack< Integer > variable
17       Stack< Integer > integerStack = new Stack( 10 );
18
19       testPush( "rawTypeStack1", rawTypeStack1, doubleElements );
20       testPop( "rawTypeStack1", rawTypeStack1 );
21       testPush( "rawTypeStack2", rawTypeStack2, doubleElements );
22       testPop( "rawTypeStack2", rawTypeStack2 );
23       testPush( "integerStack", integerStack, integerElements );
24       testPop( "integerStack", integerStack );
25    } // end main
26
27    // generic method pushes elements onto stack
28    public static < T > void testPush( String name, Stack< T > stack,
29       T[] elements )
30    {
31       System.out.printf( "\nPushing elements onto %s\n", name );
32
33       // push elements onto Stack
34       for ( T element : elements )
35       {
36          System.out.printf( "%s ", element );
37          stack.push( element ); // push element onto stack
38       } // end for
39    } // end method testPush
40
41    // generic method testPop pops elements from stack
42    public static < T > void testPop( String name, Stack< T > stack )
43    {
44       // pop elements from stack
45       try
46       {
47          System.out.printf( "\nPopping elements from %s\n", name );
48          T popValue; // store element removed from stack
49
50          // remove elements from Stack
51          while ( true )
52          {
53             popValue = stack.pop(); // pop from stack
```

Fig. 21.11 | Raw-type test program. (Part 1 of 2.)

```
54              System.out.printf( "%s ", popValue );
55           } // end while
56        } // end try
57        catch( EmptyStackException emptyStackException )
58        {
59           System.out.println();
60           emptyStackException.printStackTrace();
61        } // end catch EmptyStackException
62     } // end method testPop
63  } // end class RawTypeTest
```

```
Pushing elements onto rawTypeStack1
1.1 2.2 3.3 4.4 5.5
Popping elements from rawTypeStack1
5.5 4.4 3.3 2.2 1.1
EmptyStackException: Stack is empty, cannot pop
        at Stack.pop(Stack.java:32)
        at RawTypeTest.testPop(RawTypeTest.java:53)
        at RawTypeTest.main(RawTypeTest.java:20)

Pushing elements onto rawTypeStack2
1.1 2.2 3.3 4.4 5.5
Popping elements from rawTypeStack2
5.5 4.4 3.3 2.2 1.1
EmptyStackException: Stack is empty, cannot pop
        at Stack.pop(Stack.java:32)
        at RawTypeTest.testPop(RawTypeTest.java:53)
        at RawTypeTest.main(RawTypeTest.java:22)

Pushing elements onto integerStack
1 2 3 4 5 6 7 8 9 10
Popping elements from integerStack
10 9 8 7 6 5 4 3 2 1
EmptyStackException: Stack is empty, cannot pop
        at Stack.pop(Stack.java:32)
        at RawTypeTest.testPop(RawTypeTest.java:53)
        at RawTypeTest.main(RawTypeTest.java:24)
```

Fig. 21.11 | Raw-type test program. (Part 2 of 2.)

```
RawTypeTest.java:17: warning: [unchecked] unchecked conversion
found   : Stack
required: Stack<java.lang.Integer>
      Stack< Integer > integerStack = new Stack( 10 );
                                      ^

RawTypeTest.java:19: warning: [unchecked] unchecked conversion
found   : Stack
required: Stack<java.lang.Double>
      testPush( "rawTypeStack1", rawTypeStack1, doubleElements );
                                 ^
```

Fig. 21.12 | Warning messages from the compiler. (Part 1 of 2.)

```
RawTypeTest.java:19: warning: [unchecked] unchecked method invocation:
<T>testPush(java.lang.String,Stack<T>,T[]) in RawTypeTest is applied to
(java.lang.String,Stack,java.lang.Double[])
      testPush( "rawTypeStack1", rawTypeStack1, doubleElements );
               ^
RawTypeTest.java:20: warning: [unchecked] unchecked conversion
found   : Stack
required: Stack<T>
      testPop( "rawTypeStack1", rawTypeStack1 );
                                ^
RawTypeTest.java:20: warning: [unchecked] unchecked method invocation:
<T>testPop(java.lang.String,Stack<T>) in RawTypeTest is applied to
(java.lang.String,Stack)
      testPop( "rawTypeStack1", rawTypeStack1 );
              ^
RawTypeTest.java:21: warning: [unchecked] unchecked conversion
found   : Stack
required: Stack<java.lang.Double>
      testPush( "rawTypeStack2", rawTypeStack2, doubleElements );
                                 ^
RawTypeTest.java:21: warning: [unchecked] unchecked method invocation:
<T>testPush(java.lang.String,Stack<T>,T[]) in RawTypeTest is applied to
(java.lang.String,Stack,java.lang.Double[])
      testPush( "rawTypeStack2", rawTypeStack2, doubleElements );
               ^
RawTypeTest.java:22: warning: [unchecked] unchecked conversion
found   : Stack
required: Stack<T>
      testPop( "rawTypeStack2", rawTypeStack2 );
                                ^
RawTypeTest.java:22: warning: [unchecked] unchecked method invocation:
<T>testPop(java.lang.String,Stack<T>) in RawTypeTest is applied to
(java.lang.String,Stack)
      testPop( "rawTypeStack2", rawTypeStack2 );
              ^
9 warnings
```

Fig. 21.12 | Warning messages from the compiler. (Part 2 of 2.)

21.8 Wildcards in Methods That Accept Type Parameters

In this section, we introduce a powerful generics concept known as **wildcards**. For this purpose, we'll also introduce a new data structure from package `java.util`. In Chapter 20, we discussed the Java Collections Framework, which provides many generic data structures and algorithms that manipulate the elements of those data structures. Perhaps the simplest of these data structures is class `ArrayList`—a dynamically resizable, arraylike data structure. As part of this discussion, you'll learn how to create an `ArrayList`, add elements to it and traverse those elements using an enhanced `for` statement.

Let's consider an example that motivates wildcards. Suppose that you'd like to implement a generic method `sum` that totals the numbers in a collection, such as an `ArrayList`. You'd begin by inserting the numbers in the collection. Because generic classes can be used only with class or interface types, the numbers would be *autoboxed* as objects of the type-

wrapper classes. For example, any `int` value would be *autoboxed* as an `Integer` object, and any `double` value would be *autoboxed* as a `Double` object. We'd like to be able to total all the numbers in the `ArrayList` regardless of their type. For this reason, we'll declare the `ArrayList` with the type argument `Number`, which is the superclass of both `Integer` and `Double`. In addition, method `sum` will receive a parameter of type `ArrayList<Number>` and total its elements. Figure 21.13 demonstrates totaling the elements of an `ArrayList` of `Number`s.

```
1  // Fig. 21.13: TotalNumbers.java
2  // Totaling the numbers in an ArrayList<Number>.
3  import java.util.ArrayList;
4
5  public class TotalNumbers
6  {
7     public static void main( String[] args )
8     {
9        // create, initialize and output ArrayList of Numbers containing
10       // both Integers and Doubles, then display total of the elements
11       Number[] numbers = { 1, 2.4, 3, 4.1 }; // Integers and Doubles
12       ArrayList< Number > numberList = new ArrayList< Number >();
13
14       for ( Number element : numbers )
15          numberList.add( element ); // place each number in numberList
16
17       System.out.printf( "numberList contains: %s\n", numberList );
18       System.out.printf( "Total of the elements in numberList: %.1f\n",
19          sum( numberList ) );
20    } // end main
21
22    // calculate total of ArrayList elements
23    public static double sum( ArrayList< Number > list )
24    {
25       double total = 0; // initialize total
26
27       // calculate sum
28       for ( Number element : list )
29          total += element.doubleValue();
30
31       return total;
32    } // end method sum
33 } // end class TotalNumbers
```

```
numberList contains: [1, 2.4, 3, 4.1]
Total of the elements in numberList: 10.5
```

Fig. 21.13 | Totaling the numbers in an `ArrayList<Number>`.

Line 11 declares and initializes an array of `Number`s. Because the initializers are primitive values, Java *autoboxes* each primitive value as an object of its corresponding wrapper type. The `int` values `1` and `3` are *autoboxed* as `Integer` objects, and the `double` values `2.4` and `4.1` are *autoboxed* as `Double` objects. Line 12 declares and creates an `ArrayList` object that stores `Number`s and assigns it to variable `numberList`. We do not have to specify the size of the `ArrayList` because it will grow automatically as we insert objects.

Lines 14–15 traverse array `numbers` and place each element in `numberList`. Line 17 outputs the contents of the `ArrayList` as a `String`. This statement implicitly invokes the `ArrayList`'s `toString` method, which returns a `String` of the form `"[`*elements*`]"` in which *elements* is a comma-separated list of the elements' `String` representations. Lines 18–19 display the sum of the elements that is returned by the call to method `sum`.

Method `sum` (lines 23–32) receives an `ArrayList` of `Number`s and calculates the total of the `Number`s in the collection. The method uses `double` values to perform the calculations and returns the result as a `double`. Lines 28–29 use the enhanced `for` statement, which is designed to work with both arrays and the collections of the Collections Framework, to total the elements of the `ArrayList`. The `for` statement assigns each `Number` in the `ArrayList` to variable `element`, then uses **`Number` method `doubleValue`** to obtain the `Number`'s underlying primitive value as a `double` value. The result is added to `total`. When the loop terminates, the method returns the `total`.

Implementing Method `sum` With a Wildcard Type Argument in Its Parameter

Recall that the purpose of method `sum` in Fig. 21.13 was to total any type of `Number`s stored in an `ArrayList`. We created an `ArrayList` of `Number`s that contained both `Integer` and `Double` objects. The output of Fig. 21.13 demonstrates that method `sum` worked properly. Given that method `sum` can total the elements of an `ArrayList` of `Number`s, you might expect that the method would also work for `ArrayList`s that contain elements of only one numeric type, such as `ArrayList<Integer>`. So we modified class `TotalNumbers` to create an `ArrayList` of `Integer`s and pass it to method `sum`. When we compile the program, the compiler issues the following error message:

```
sum(java.util.ArrayList<java.lang.Number>) in TotalNumbersErrors
cannot be applied to (java.util.ArrayList<java.lang.Integer>)
```

Although `Number` is the superclass of `Integer`, the compiler does not consider the parameterized type `ArrayList<Number>` to be a superclass of `ArrayList<Integer>`. If it were, then every operation we could perform on `ArrayList<Number>` would also work on an `ArrayList<Integer>`. Consider the fact that you can add a `Double` object to an `ArrayList<Number>` because a `Double` *is a* `Number`, but you cannot add a `Double` object to an `ArrayList<Integer>` because a `Double` *is not an* `Integer`. Thus, the subtype relationship does not hold.

How do we create a more flexible version of the `sum` method that can total the elements of any `ArrayList` containing elements of any subclass of `Number`? This is where **wildcard type arguments** are important. Wildcards enable you to specify method parameters, return values, variables or fields, and so on, that act as supertypes or subtypes of parameterized types. In Fig. 21.14, method `sum`'s parameter is declared in line 50 with the type:

```
ArrayList< ? extends Number >
```

A wildcard type argument is denoted by a question mark (**?**), which by itself represents an "unknown type." In this case, the wildcard extends class `Number`, which means that the wildcard has an upper bound of `Number`. Thus, the unknown-type argument must be either `Number` or a subclass of `Number`. With the parameter type shown here, method `sum` can receive an `ArrayList` argument that contains any type of `Number`, such as `ArrayList<Integer>` (line 20), `ArrayList<Double>` (line 33) or `ArrayList<Number>` (line 46).

```
1  // Fig. 21.14: WildcardTest.java
2  // Wildcard test program.
3  import java.util.ArrayList;
4
5  public class WildcardTest
6  {
7     public static void main( String[] args )
8     {
9        // create, initialize and output ArrayList of Integers, then
10       // display total of the elements
11       Integer[] integers = { 1, 2, 3, 4, 5 };
12       ArrayList< Integer > integerList = new ArrayList< Integer >();
13
14       // insert elements in integerList
15       for ( Integer element : integers )
16          integerList.add( element );
17
18       System.out.printf( "integerList contains: %s\n", integerList );
19       System.out.printf( "Total of the elements in integerList: %.0f\n\n",
20          sum( integerList ) );
21
22       // create, initialize and output ArrayList of Doubles, then
23       // display total of the elements
24       Double[] doubles = { 1.1, 3.3, 5.5 };
25       ArrayList< Double > doubleList = new ArrayList< Double >();
26
27       // insert elements in doubleList
28       for ( Double element : doubles )
29          doubleList.add( element );
30
31       System.out.printf( "doubleList contains: %s\n", doubleList );
32       System.out.printf( "Total of the elements in doubleList: %.1f\n\n",
33          sum( doubleList ) );
34
35       // create, initialize and output ArrayList of Numbers containing
36       // both Integers and Doubles, then display total of the elements
37       Number[] numbers = { 1, 2.4, 3, 4.1 }; // Integers and Doubles
38       ArrayList< Number > numberList = new ArrayList< Number >();
39
40       // insert elements in numberList
41       for ( Number element : numbers )
42          numberList.add( element );
43
44       System.out.printf( "numberList contains: %s\n", numberList );
45       System.out.printf( "Total of the elements in numberList: %.1f\n",
46          sum( numberList ) );
47    } // end main
48
49    // total the elements; using a wildcard in the ArrayList parameter
50    public static double sum( ArrayList< ? extends Number > list )
51    {
52       double total = 0; // initialize total
53
```

Fig. 21.14 | Generic wildcard test program. (Part 1 of 2.)

```
54         // calculate sum
55         for ( Number element : list )
56            total += element.doubleValue();
57
58         return total;
59      } // end method sum
60   } // end class WildcardTest
```

```
integerList contains: [1, 2, 3, 4, 5]
Total of the elements in integerList: 15

doubleList contains: [1.1, 3.3, 5.5]
Total of the elements in doubleList: 9.9

numberList contains: [1, 2.4, 3, 4.1]
Total of the elements in numberList: 10.5
```

Fig. 21.14 | Generic wildcard test program. (Part 2 of 2.)

Lines 11–20 create and initialize an `ArrayList<Integer>`, output its elements and total them by calling method `sum` (line 20). Lines 24–33 perform the same operations for an `ArrayList<Double>`. Lines 37–46 perform the same operations for an `ArrayList<Number>` that contains `Integer`s and `Double`s.

In method `sum` (lines 50–59), although the `ArrayList` argument's element types are not directly known by the method, they're known to be at least of type `Number`, because the wildcard was specified with the upper bound `Number`. For this reason line 56 is allowed, because all `Number` objects have a `doubleValue` method.

Although wildcards provide flexibility when passing parameterized types to a method, they also have some disadvantages. Because the wildcard (`?`) in the method's header (line 50) does not specify a type-parameter name, you cannot use it as a type name throughout the method's body (i.e., you cannot replace `Number` with `?` in line 55). You could, however, declare method `sum` as follows:

```
public static <T extends Number> double sum( ArrayList< T > list )
```

which allows the method to receive an `ArrayList` that contains elements of any `Number` subclass. You could then use the type parameter `T` throughout the method body.

If the wildcard is specified without an upper bound, then only the methods of type `Object` can be invoked on values of the wildcard type. Also, methods that use wildcards in their parameter's type arguments cannot be used to add elements to a collection referenced by the parameter.



Common Programming Error 21.4

Using a wildcard in a method's type-parameter section or using a wildcard as an explicit type of a variable in the method body is a syntax error.

21.9 Generics and Inheritance: Notes

Generics can be used with inheritance in several ways:

- A generic class can be derived from a nongeneric class. For example, the `Object` class is a direct or indirect superclass of every generic class.

- A generic class can be derived from another generic class. For example, generic class `Stack` (in package `java.util`) is a subclass of generic class `Vector` (in package `java.util`). We discussed these classes in Chapter 20.
- A nongeneric class can be derived from a generic class. For example, nongeneric class `Properties` (in package `java.util`) is a subclass of generic class `Hashtable` (in package `java.util`). We also discussed these classes in Chapter 20.
- Finally, a generic method in a subclass can override a generic method in a superclass if both methods have the same signatures.

21.10 Wrap-Up

This chapter introduced generics. You learned how to declare generic methods and classes. We discussed how backward compatibility is achieved via raw types. You also learned how to use wildcards in a generic method or a generic class. In Chapter 22, you'll learn how to implement your own custom dynamic data structures that can grow or shrink at execution time. In particular, you'll implement these data structures using the generics capabilities you learned in this chapter. For more information on generics, please visit our Java Resource Center at `www.deitel.com/Java/` and click the topic **Java Generics** under the heading **Resource Center Contents**.

Summary

Section 21.1 Introduction

- Generic methods enable you to specify, with one method declaration, a set of related methods.
- Generic classes and interfaces enable you to specify sets of related types.

Section 21.2 Motivation for Generic Methods

- Overloaded methods are often used to perform similar operations on different types of data.
- When the compiler encounters a method call, it attempts to locate a method declaration with a name and parameters that are compatible with the argument types in the method call.

Section 21.3 Generic Methods: Implementation and Compile-Time Translation

- If the operations performed by several overloaded methods are identical for each argument type, they can be more compactly and conveniently coded using a generic method. A single generic method declaration can be called with arguments of different data types. Based on the types of the arguments passed to a generic method, the compiler handles each method call appropriately.
- All generic method declarations have a type-parameter section (p. 878) delimited by angle brackets (< and >) that precedes the method's return type (p. 878).
- A type-parameter section contains one or more type parameters separated by commas.
- A type parameter (p. 878) is an identifier that specifies a generic type name. Type parameters can be used as the return type, parameter types and local variable types in a generic method declaration, and they act as placeholders for the types of the arguments passed to the generic method, which are known as actual type arguments (p. 878). Type parameters can represent only reference types.
- Type-parameter names used throughout a method declaration must match those declared in the type-parameter section. A type-parameter name can be declared only once in the type-parameter section but can appear more than once in the method's parameter list.

- When the compiler encounters a method call, it determines the argument types and attempts to locate a method with the same name and parameters that match the argument types. If there's no such method, the compiler searches for methods with the same name and compatible parameters and for matching generic methods.
- Objects of a class that implements generic interface `Comparable` (`Comparable`) can be compared with method `compareTo` (p. 880), which returns `0` if the objects are equal, a negative integer if the first object is less than the second or a positive integer if the first object is greater than the second.
- All the type-wrapper classes for primitive types implement `Comparable`.
- `Comparable` objects can be used with the sorting and searching methods of class `Collections`.
- When a generic method is compiled, the compiler performs erasure (p. 879) to remove the type-parameter section and replace the type parameters with actual types. By default each type parameter is replaced with its upper bound (p. 881), which is `Object` unless specified otherwise.

Section 21.4 Additional Compile-Time Translation Issues: Methods That Use a Type Parameter as the Return Type

- When erasure is performed on a method that returns a type variable (p. 878), explicit casts are inserted in front of each method call to ensure that the returned value has the type expected by the caller.

Section 21.5 Overloading Generic Methods

- A generic method may be overloaded with other generic methods or with nongeneric methods.

Section 21.6 Generic Classes

- Generic classes provide a means for describing a class in a type-independent manner. We can then instantiate type-specific objects of the generic class.
- A generic class declaration looks like a nongeneric class declaration, except that the class name is followed by a type-parameter section. The type-parameter section of a generic class can have one or more type parameters separated by commas.
- When a generic class is compiled, the compiler performs erasure on the class's type parameters and replaces them with their upper bounds.
- Type parameters cannot be used in a class's `static` declarations.
- When instantiating an object of a generic class, the types specified in angle brackets after the class name are known as type arguments (p. 886). The compiler uses them to replace the type parameters so that it can perform type checking and insert cast operations as necessary.

Section 21.7 Raw Types

- It's possible to instantiate a generic class without specifying a type argument. In this case, the new object of the class is said to have a raw type (p. 891)—the compiler implicitly uses type `Object` (or the type parameter's upper bound) throughout the generic class for each type argument.

Section 21.8 Wildcards in Methods That Accept Type Parameters

- Class `Number` is the superclass of both `Integer` and `Double`.
- `Number` method `doubleValue` (p. 897) obtains the `Number`'s underlying primitive value as a `double` value.
- Wildcard type arguments enable you to specify method parameters, return values, variables, and so on, that act as supertypes of parameterized types. A wildcard-type argument is denoted by `?` (p. 897), which represents an "unknown type." A wildcard can also have an upper bound.

- Because a wildcard (?) is not a type-parameter name, you cannot use it as a type name throughout a method's body.
- If a wildcard is specified without an upper bound, then only the methods of type `Object` can be invoked on values of the wildcard type.
- Methods that use wildcards as type arguments (p. 897) cannot be used to add elements to a collection referenced by the parameter.

Section 21.9 Generics and Inheritance: Notes

- A generic class can be derived from a nongeneric class. For example, `Object` is a direct or indirect superclass of every generic class.
- A generic class can be derived from another generic class.
- A nongeneric class can be derived from a generic class.
- A generic method in a subclass can override a generic method in a superclass if both methods have the same signatures.

Self-Review Exercises

21.1 *(True or False)* State whether each of the following is *true* or *false*. If *false*, explain why.

a) A generic method cannot have the same method name as a nongeneric method.
b) All generic method declarations have a type-parameter section that immediately precedes the method name.
c) A generic method can be overloaded by another generic method with the same method name but different method parameters.
d) A type parameter can be declared only once in the type-parameter section but can appear more than once in the method's parameter list.
e) Type-parameter names among different generic methods must be unique.
f) The scope of a generic class's type parameter is the entire class except its `static` members.

21.2 *(Fill in the Blanks)* Fill in the blanks in each of the following:

a) _____ and _____ enable you to specify, with a single method declaration, a set of related methods, or with a single class declaration, a set of related types, respectively.
b) A type-parameter section is delimited by _____.
c) A generic method's _____ can be used to specify the method's argument types, to specify the method's return type and to declare variables within the method.
d) The statement "`Stack objectStack = new Stack();`" indicates that `objectStack` stores _____.
e) In a generic class declaration, the class name is followed by a(n) _____.
f) The syntax _____ specifies that the upper bound of a wildcard is type `T`.

Answers to Self-Review Exercises

21.1 a) False. Generic and nongeneric methods can have the same method name. A generic method can overload another generic method with the same method name but different method parameters. A generic method also can be overloaded by providing nongeneric methods with the same method name and number of arguments. b) False. All generic method declarations have a type-parameter section that immediately precedes the method's return type. c) True. d) True. e) False. Type-parameter names among different generic methods need not be unique. f) True.

21.2 a) Generic methods, generic classes. b) angle brackets (< and >). c) type parameters. d) a raw type. e) type-parameter section. f) `? extends T`.

Exercises

21.3 *(Explain Notation)* Explain the use of the following notation in a Java program:

```
public class Array< T > { }
```

21.4 *(Generic Method `selectionSort`)* Write a generic method `selectionSort` based on the sort program of Figs. 19.6–19.7. Write a test program that inputs, sorts and outputs an `Integer` array and a `Float` array. [*Hint:* Use `<T extends Comparable<T> >` in the type-parameter section for method `selectionSort`, so that you can use method `compareTo` to compare the objects of the type that `T` represents.]

21.5 *(Overloaded Generic Method `printArray`)* Overload generic method `printArray` of Fig. 21.3 so that it takes two additional integer arguments, `lowSubscript` and `highSubscript`. A call to this method prints only the designated portion of the array. Validate `lowSubscript` and `highSubscript`. If either is out of range, the overloaded `printArray` method should throw an `InvalidSubscriptException`; otherwise, `printArray` should return the number of elements printed. Then modify `main` to exercise both versions of `printArray` on arrays `integerArray`, `doubleArray` and `characterArray`. Test all capabilities of both versions of `printArray`.

21.6 *(Overloading a Generic Method with a Nongeneric Method))* Overload generic method `printArray` of Fig. 21.3 with a nongeneric version that specifically prints an array of `String`s in neat, tabular format, as shown in the sample output that follows:

```
Array stringArray contains:
one      two      three    four
five     six      seven    eight
```

21.7 *(Generic `isEqualTo` Method)* Write a simple generic version of method `isEqualTo` that compares its two arguments with the `equals` method and returns `true` if they're equal and `false` otherwise. Use this generic method in a program that calls `isEqualTo` with a variety of built-in types, such as `Object` or `Integer`. What result do you get when you attempt to run this program?

21.8 *(Generic Class `Pair`)* Write a generic class `Pair` which has two type parameters—`F` and `S`—each representing the type of the first and second element of the pair, respectively. Add *get* and *set* methods for the first and second elements of the pair. [*Hint:* The class header should be `public class Pair<F, S>`.]

21.9 *(Overloading Generic Methods)* How can generic methods be overloaded?

21.10 *(Overload Resolution)* The compiler performs a matching process to determine which method to call when a method is invoked. Under what circumstances does an attempt to make a match result in a compile-time error?

21.11 *(What Does this Statement Do?)* Explain why a Java program might use the statement

```
ArrayList< Employee > workerList = new ArrayList< Employee >();
```

22

Custom Generic Data Structures



Much that I bound, I could not free;
Much that I freed returned to me.
—Lee Wilson Dodd

'Will you walk a little faster?' said a whiting to a snail,
'There's a porpoise close behind us, and he's treading on my tail.'
—Lewis Carroll

There is always room at the top.
—Daniel Webster

Push on—keep moving.
—Thomas Morton

I'll turn over a new leaf.
—Miguel de Cervantes

Objectives

In this chapter you'll learn:

- To form linked data structures using references, self-referential classes, recursion and generics.
- To create and manipulate dynamic data structures, such as linked lists, queues, stacks and binary trees.
- Various important applications of linked data structures.
- How to create reusable data structures with classes, inheritance and composition.

Outline

22.1 Introduction
22.2 Self-Referential Classes
22.3 Dynamic Memory Allocation
22.4 Linked Lists
22.5 Stacks
22.6 Queues
22.7 Trees
22.8 Wrap-Up

Summary | *Self-Review Exercises* | *Answers to Self-Review Exercises* | *Exercises* | *Special Section: Building Your Own Compiler*

22.1 Introduction

This chapter shows how to build **dynamic data structures** that grow and shrink at execution time. **Linked lists** are collections of data items "linked up in a chain"; insertions and deletions can be made anywhere in a linked list. **Stacks** are important in compilers and operating systems; insertions and deletions are made only at one end of a stack—its **top**. **Queues** represent waiting lines; insertions are made at the back (also referred to as the **tail**) of a queue and deletions are made from the front (also referred to as the **head**). **Binary trees** facilitate high-speed searching and sorting of data, eliminating duplicate data items efficiently, representing file-system directories, compiling expressions into machine language and many other interesting applications.

We discuss each of these major data-structure types and implement programs that create and manipulate them. We use classes, inheritance and composition to create and package them for reusability and maintainability. In general, you'd use one of the predefined collection classes that we discussed in Chapter 20. However, the techniques we present here can be used if you ever need to build your own *custom* collections.

For simplicity, this chapter's examples manipulate primitive values. However, the data-structure implementations we develop here can store objects of most types.

If you feel ambitious, you might want to attempt the major project described in the special section entitled Building Your Own Compiler, which we've posted online at `www.deitel.com/books/jhtp9/`. You've been using a Java compiler to translate your Java programs to bytecodes so that you could execute these programs. In this project, you'll actually build your own compiler. It will read statements written in a simple, yet powerful high-level language similar to early versions of the popular language BASIC and translate these statements into Simpletron Machine Language (SML) instructions—SML is the language you learned in the Chapter 7 special section, Building Your Own Computer. Your Simpletron Simulator program will then execute the SML program produced by your compiler! Implementing this project by using an object-oriented approach will give you an opportunity to exercise most of what you've learned in this book. The special section carefully walks you through the specifications of the high-level language and describes the algorithms you'll need to convert each high-level language statement into machine-language instructions. If you enjoy being challenged, you might attempt the enhancements to both the compiler and the Simpletron Simulator suggested in the exercises.

22.2 Self-Referential Classes

A **self-referential class** contains an instance variable that refers to another object of the same class type. For example, the generic `Node` class declaration

```
class Node< T >
{
   private T data;
   private Node< T > nextNode; // reference to next linked node

   public Node( T data ) { /* constructor body */ }
   public void setData( T data ) { /* method body */ }
   public T getData() { /* method body */ }
   public void setNext( Node< T > next ) { /* method body */ }
   public Node< T > getNext() { /* method body */ }
} // end class Node< T >
```

has two `private` instance variables—`data` (of the generic type `T`) and `Node<T>` variable `nextNode`. Variable `nextNode` references a `Node<T>` object, an object of the same class being declared here—hence the term "self-referential class." Field `nextNode` is a **link**—it "links" an object of type `Node<T>` to another object of the same type. Type `Node<T>` also has five methods: a constructor that receives a value to initialize `data`, a `setData` method to set the value of `data`, a `getData` method to return the value of `data`, a `setNext` method to set the value of `nextNode` and a `getNext` method to return a reference to the next node.

Programs can link self-referential objects together to form such useful data structures as lists, queues, stacks and trees. Figure 22.1 illustrates two self-referential objects linked together to form a list. A backslash—representing a `null` reference—is placed in the link member of the second self-referential object to indicate that the link does *not* refer to another object. The backslash is illustrative; it does not correspond to the backslash character in Java. We use the `null` reference to indicate the end of a data structure.



Fig. 22.1 | Self-referential-class objects linked together.

22.3 Dynamic Memory Allocation

Creating and maintaining dynamic data structures requires **dynamic memory allocation**—allowing a program to obtain more memory space at execution time to hold new nodes and to release space no longer needed. Remember that Java programs do not explicitly release dynamically allocated memory. Rather, Java performs automatic *garbage collection* of objects that are no longer referenced in a program.

The limit for dynamic memory allocation can be as large as the amount of available physical memory in the computer or the amount of available disk space in a virtual-memory system. Often the limits are much smaller, because the computer's available memory must be shared among many applications.

The declaration and class-instance-creation expression

```
// 10 is nodeToAdd's data
Node< Integer > nodeToAdd = new Node< Integer >( 10 );
```

allocates a `Node<Integer>` object and returns a reference to it, which is assigned to `nodeToAdd`. If *insufficient memory* is available, the expression throws an `OutOfMemoryError`. The sections that follow discuss lists, stacks, queues and trees—all of which use dynamic memory allocation and self-referential classes to create dynamic data structures.

22.4 Linked Lists

A linked list is a linear collection (i.e., a sequence) of self-referential-class objects, called **nodes**, connected by reference links—hence, the term "linked" list. Typically, a program accesses a linked list via a reference to its first node. The program accesses each subsequent node via the link reference stored in the previous node. By convention, the link reference in the last node of the list is set to `null`. Data is stored in a linked list dynamically—the program creates each node as necessary. Stacks and queues are also linear data structures and, as we'll see, are constrained versions of linked lists. Trees are *nonlinear* data structures.

Lists of data can be stored in arrays, but linked lists provide several advantages. A linked list is appropriate when the number of data elements to be represented in the data structure is *unpredictable*. Linked lists are dynamic, so the length of a list can increase or decrease as necessary, whereas the size of a "conventional" Java array cannot be altered—it's fixed when the program creates the array. "Conventional" arrays can become full. Linked lists become full only when the system has *insufficient memory* to satisfy dynamic storage allocation requests. Package `java.util` contains class `LinkedList` (discussed in Chapter 20) for implementing and manipulating linked lists that grow and shrink during program execution.



Performance Tip 22.1

An array can be declared to contain more elements than the number of items expected, but this wastes memory. In these situations, linked lists provide better memory utilization, allowing the program to adapt to storage needs at runtime.



Performance Tip 22.2

Insertion into a linked list is fast—only two references have to be modified (after locating the insertion point). All existing node objects remain at their current locations in memory.

Linked lists can be maintained in sorted order simply by inserting each new element at the proper point in the list. (It does, of course, take time to *locate* the proper insertion point.) *Existing list elements do not need to be moved.*



Performance Tip 22.3

Insertion and deletion in a sorted array can be time consuming—all the elements following the inserted or deleted element must be shifted appropriately.

Singly Linked Lists

Linked list nodes normally are *not stored contiguously* in memory. Rather, they're logically contiguous. Figure 22.2 illustrates a linked list with several nodes. This diagram presents a **singly linked list**—each node contains one reference to the next node in the list. Often, linked lists are implemented as *doubly linked lists*—each node contains a reference to the next node in the list *and* a reference to the preceding one.



Performance Tip 22.4

Normally, the elements of an array are contiguous in memory. This allows immediate access to any array element, because its address can be calculated directly as its offset from the beginning of the array. Linked lists do not afford such immediate access—an element can be accessed only by traversing the list from the front (or the back in a doubly linked list).

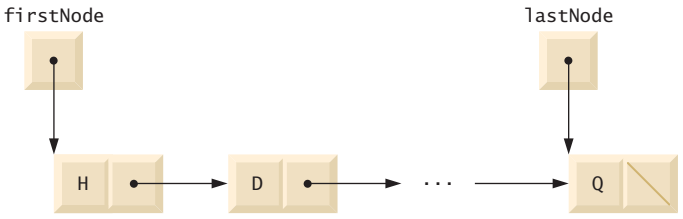


Fig. 22.2 | Linked-list graphical representation.

Implementing a Generic `List` Class

The program of Figs. 22.3–22.5 uses an object of our generic `List` class to manipulate a list of miscellaneous objects. The program consists of four classes—`ListNode` (Fig. 22.3, lines 6–37), `List` (Fig. 22.3, lines 40–147), `EmptyListException` (Fig. 22.4) and `ListTest` (Fig. 22.5). The `List`, `ListNode` and `EmptyListException` classes are placed in package `com.deitel.ch22`, so that they can be reused throughout this chapter. Encapsulated in each `List` object is a linked list of `ListNode` objects. [*Note:* Many of the classes in this chapter are declared in the package `com.deitel.ch22`. Each such class should be compiled with the `-d` command-line option to `javac`. When compiling the classes that are not in this package and when running the programs, be sure to use the option `-classpath` with `javac` and `java`, respectively.]

```
1  // Fig. 22.3: List.java
2  // ListNode and List class declarations.
3  package com.deitel.ch22;
4
5  // class to represent one node in a list
6  class ListNode< T >
7  {
8     // package access members; List can access these directly
9     T data; // data for this node
10    ListNode< T > nextNode; // reference to the next node in the list
11
12    // constructor creates a ListNode that refers to object
13    ListNode( T object )
14    {
15       this( object, null );
16    } // end ListNode one-argument constructor
17
18    // constructor creates ListNode that refers to the specified
19    // object and to the next ListNode
20    ListNode( T object, ListNode< T > node )
21    {
22       data = object;
23       nextNode = node;
24    } // end ListNode two-argument constructor
```

Fig. 22.3 | `ListNode` and `List` class declarations. (Part 1 of 4.)

```
25
26      // return reference to data in node
27      T getData()
28      {
29         return data; // return item in this node
30      } // end method getData
31
32      // return reference to next node in list
33      ListNode< T > getNext()
34      {
35         return nextNode; // get next node
36      } // end method getNext
37   } // end class ListNode< T >
38
39   // class List definition
40   public class List< T >
41   {
42      private ListNode< T > firstNode;
43      private ListNode< T > lastNode;
44      private String name; // string like "list" used in printing
45
46      // constructor creates empty List with "list" as the name
47      public List()
48      {
49         this( "list" );
50      } // end List no-argument constructor
51
52      // constructor creates an empty List with a name
53      public List( String listName )
54      {
55         name = listName;
56         firstNode = lastNode = null;
57      } // end List one-argument constructor
58
59      // insert item at front of List
60      public void insertAtFront( T insertItem )
61      {
62         if ( isEmpty() ) // firstNode and lastNode refer to same object
63            firstNode = lastNode = new ListNode< T >( insertItem );
64         else // firstNode refers to new node
65            firstNode = new ListNode< T >( insertItem, firstNode );
66      } // end method insertAtFront
67
68      // insert item at end of List
69      public void insertAtBack( T insertItem )
70      {
71         if ( isEmpty() ) // firstNode and lastNode refer to same object
72            firstNode = lastNode = new ListNode< T >( insertItem );
73         else // lastNode's nextNode refers to new node
74            lastNode = lastNode.nextNode = new ListNode< T >( insertItem );
75      } // end method insertAtBack
76
```

Fig. 22.3 | `ListNode` and `List` class declarations. (Part 2 of 4.)

```
77      // remove first node from List
78      public T removeFromFront() throws EmptyListException
79      {
80         if ( isEmpty() ) // throw exception if List is empty
81            throw new EmptyListException( name );
82
83         T removedItem = firstNode.data; // retrieve data being removed
84
85         // update references firstNode and lastNode
86         if ( firstNode == lastNode )
87            firstNode = lastNode = null;
88         else
89            firstNode = firstNode.nextNode;
90
91         return removedItem; // return removed node data
92      } // end method removeFromFront
93
94      // remove last node from List
95      public T removeFromBack() throws EmptyListException
96      {
97         if ( isEmpty() ) // throw exception if List is empty
98            throw new EmptyListException( name );
99
100        T removedItem = lastNode.data; // retrieve data being removed
101
102        // update references firstNode and lastNode
103        if ( firstNode == lastNode )
104           firstNode = lastNode = null;
105        else // locate new last node
106        {
107           ListNode< T > current = firstNode;
108
109           // loop while current node does not refer to lastNode
110           while ( current.nextNode != lastNode )
111              current = current.nextNode;
112
113           lastNode = current; // current is new lastNode
114           current.nextNode = null;
115        } // end else
116
117        return removedItem; // return removed node data
118     } // end method removeFromBack
119
120     // determine whether list is empty
121     public boolean isEmpty()
122     {
123        return firstNode == null; // return true if list is empty
124     } // end method isEmpty
125
126     // output list contents
127     public void print()
128     {
```

Fig. 22.3 | `ListNode` and `List` class declarations. (Part 3 of 4.)

```
129        if ( isEmpty() )
130        {
131           System.out.printf( "Empty %s\n", name );
132           return;
133        } // end if
134
135        System.out.printf( "The %s is: ", name );
136        ListNode< T > current = firstNode;
137
138        // while not at end of list, output current node's data
139        while ( current != null )
140        {
141           System.out.printf( "%s ", current.data );
142           current = current.nextNode;
143        } // end while
144
145        System.out.println( "\n" );
146     } // end method print
147  } // end class List< T >
```

Fig. 22.3 | `ListNode` and `List` class declarations. (Part 4 of 4.)

```
1  // Fig. 22.4: EmptyListException.java
2  // Class EmptyListException declaration.
3  package com.deitel.ch22;
4
5  public class EmptyListException extends RuntimeException
6  {
7     // no-argument constructor
8     public EmptyListException()
9     {
10       this( "List" ); // call other EmptyListException constructor
11    } // end EmptyListException no-argument constructor
12
13    // one-argument constructor
14    public EmptyListException( String name )
15    {
16       super( name + " is empty" ); // call superclass constructor
17    } // end EmptyListException one-argument constructor
18 } // end class EmptyListException
```

Fig. 22.4 | `EmptyListException` class declaration.

```
1  // Fig. 22.5: ListTest.java
2  // ListTest class to demonstrate List capabilities.
3  import com.deitel.ch22.List;
4  import com.deitel.ch22.EmptyListException;
5
6  public class ListTest
7  {
```

Fig. 22.5 | `ListTest` class to demonstrate `List` capabilities. (Part 1 of 2.)

```
8      public static void main( String[] args )
9      {
10        List< Integer > list = new List< Integer >(); // create a List
11
12        // insert integers in list
13        list.insertAtFront( -1 );
14        list.print();
15        list.insertAtFront( 0 );
16        list.print();
17        list.insertAtBack( 1 );
18        list.print();
19        list.insertAtBack( 5 );
20        list.print();
21
22        // remove objects from list; print after each removal
23        try
24        {
25           int removedItem = list.removeFromFront();
26           System.out.printf( "\n%d removed\n", removedItem );
27           list.print();
28
29           removedItem = list.removeFromFront();
30           System.out.printf( "\n%d removed\n", removedItem );
31           list.print();
32
33           removedItem = list.removeFromBack();
34           System.out.printf( "\n%d removed\n", removedItem );
35           list.print();
36
37           removedItem = list.removeFromBack();
38           System.out.printf( "\n%d removed\n", removedItem );
39           list.print();
40        } // end try
41        catch ( EmptyListException emptyListException )
42        {
43           emptyListException.printStackTrace();
44        } // end catch
45     } // end main
46  } // end class ListTest
```

```
The list is: -1
The list is: 0 -1
The list is: 0 -1 1
The list is: 0 -1 1 5

0 removed
The list is: -1 1 5

-1 removed
The list is: 1 5

5 removed
The list is: 1

1 removed
Empty list
```

Fig. 22.5 | `ListTest` class to demonstrate `List` capabilities. (Part 2 of 2.)

Generic Classes `ListNode` *and* `List`

Generic class `ListNode` (Fig. 22.3, lines 6–37) declares package-access fields `data` and `nextNode`. The `data` field is a reference of type `T`, so its type will be determined when the client code creates the corresponding `List` object. Variable `nextNode` stores a reference to the next `ListNode` object in the linked list (or `null` if the node is the last one in the list).

Lines 42–43 of class `List` (Fig. 22.3, lines 40–47) declare references to the first and last `ListNodes` in a `List` (`firstNode` and `lastNode`, respectively). The constructors (lines 47–50 and 53–57) initialize both references to `null`. The most important methods of class `List` are `insertAtFront` (lines 60–66), `insertAtBack` (lines 69–75), `removeFromFront` (lines 78–92) and `removeFromBack` (lines 95–118). Method `isEmpty` (lines 121–124) is a *predicate method* that determines whether the list is empty (i.e., the reference to the first node of the list is `null`). Predicate methods typically test a condition and do not modify the object on which they're called. If the list is empty, method `isEmpty` returns `true`; otherwise, it returns `false`. Method `print` (lines 127–146) displays the list's contents. We discuss class `List`'s methods in more detail after we discuss class `ListTest`.

Class `ListTest`

Method `main` of class `ListTest` (Fig. 22.5) creates a `List<Integer>` object (line 10), then inserts objects at the beginning of the list using method `insertAtFront`, inserts objects at the end of the list using method `insertAtBack`, deletes objects from the front of the list using method `removeFromFront` and deletes objects from the end of the list using method `removeFromBack`. After each insert and remove operation, `ListTest` calls `List` method `print` to display the current list contents. If an attempt is made to remove an item from an empty list, an `EmptyListException` (Fig. 22.4) is thrown, so the method calls to `removeFromFront` and `removeFromBack` are placed in a `try` block that's followed by an appropriate exception handler. Notice in lines 13, 15, 17 and 19 that the application passes literal primitive `int` values to methods `insertAtFront` and `insertAtBack`. Each of these methods was declared with a parameter of the generic type `T` (Fig. 22.3, lines 60 and 69). Since this example manipulates a `List<Integer>`, the type `T` represents the type-wrapper class `Integer`. In this case, the JVM *autoboxes* each literal value in an `Integer` object, and that object is actually inserted into the list.

`List` *Method* `insertAtFront`

Now we discuss each method of class `List` (Fig. 22.3) in detail and provide diagrams showing the reference manipulations performed by methods `insertAtFront`, `insertAtBack`, `removeFromFront` and `removeFromBack`. Method `insertAtFront` (lines 60–66 of Fig. 22.3) places a new node at the front of the list. The steps are:

1. Call `isEmpty` to determine whether the list is empty (line 62).
2. If the list is empty, assign to `firstNode` and `lastNode` the new `ListNode` that was initialized with `insertItem` (line 63). (Recall that assignment operators evaluate right to left.) The `ListNode` constructor at lines 13–16 calls the `ListNode` constructor at lines 20–24 to set instance variable `data` to refer to the `insertItem` passed as an argument and to set reference `nextNode` to `null`, because this is the first and last node in the list.

3. If the list is not empty, the new node is "linked" into the list by setting `firstNode` to a new `ListNode` object and initializing that object with `insertItem` and `firstNode` (line 65). When the `ListNode` constructor (lines 20–24) executes, it sets instance variable `data` to refer to the `insertItem` passed as an argument and performs the insertion by setting the `nextNode` reference of the new node to the `ListNode` passed as an argument, which previously was the first node.

In Fig. 22.6, part (a) shows a list and a new node during the `insertAtFront` operation and before the program links the new node into the list. The dotted arrows in part (b) illustrate *Step 3* of the `insertAtFront` operation that enables the node containing `12` to become the new first node in the list.

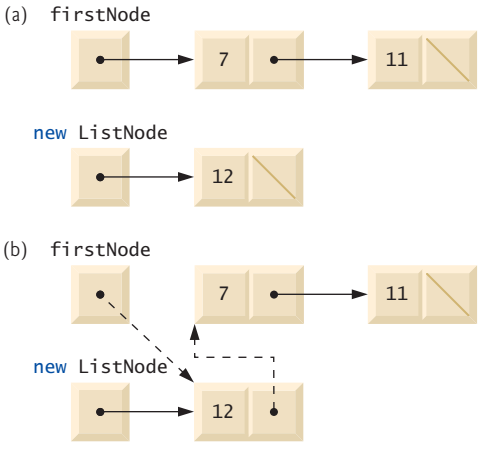


Fig. 22.6 | Graphical representation of operation `insertAtFront`.

`List` Method `insertAtBack`

Method `insertAtBack` (lines 69–75 of Fig. 22.3) places a new node at the back of the list. The steps are:

1. Call `isEmpty` to determine whether the list is empty (line 71).
2. If the list is empty, assign to `firstNode` and `lastNode` the new `ListNode` that was initialized with `insertItem` (line 72). The `ListNode` constructor at lines 13–16 calls the constructor at lines 20–24 to set instance variable `data` to refer to the `insertItem` passed as an argument and to set reference `nextNode` to `null`.
3. If the list is not empty, line 74 links the new node into the list by assigning to `lastNode` and `lastNode.nextNode` the reference to the new `ListNode` that was initialized with `insertItem`. `ListNode`'s constructor (lines 13–16) sets instance variable `data` to refer to the `insertItem` passed as an argument and sets reference `nextNode` to `null`, because this is the last node in the list.

In Fig. 22.7, part (a) shows a list and a new node during the `insertAtBack` operation and before linking the new node into the list. The dotted arrows in part (b) illustrate *Step 3* of method `insertAtBack`, which adds the new node to the end of a list that's not empty.

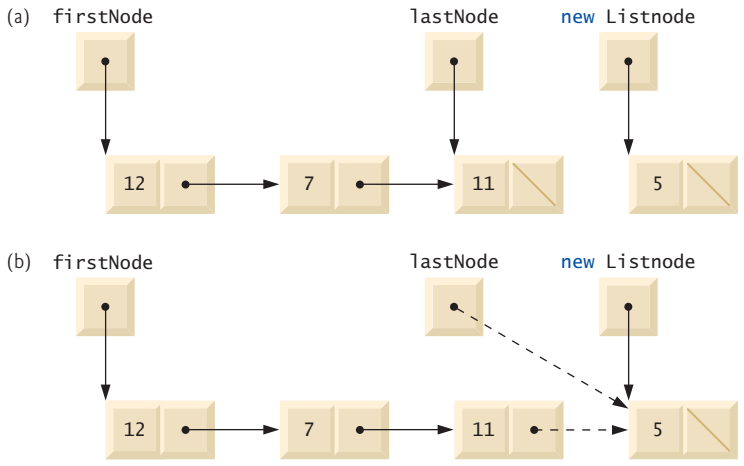


Fig. 22.7 | Graphical representation of operation `insertAtBack`.

`List` Method `removeFromFront`

Method `removeFromFront` (lines 78–92 of Fig. 22.3) removes the first node of the list and returns a reference to the removed data. If the list is empty when the program calls this method, the method throws an `EmptyListException` (lines 80–81). Otherwise, the method returns a reference to the removed data. The steps are:

1. Assign `firstNode.data` (the data being removed) to `removedItem` (line 83).
2. If `firstNode` and `lastNode` refer to the same object (line 86), the list has only one element at this time. So, the method sets `firstNode` and `lastNode` to `null` (line 87) to remove the node from the list (leaving the list empty).
3. If the list has more than one node, then the method leaves reference `lastNode` as is and assigns the value of `firstNode.nextNode` to `firstNode` (line 89). Thus, `firstNode` references the node that was previously the second node in the list.
4. Return the `removedItem` reference (line 91).

In Fig. 22.8, part (a) illustrates the list before the removal operation. The dashed lines and arrows in part (b) show the reference manipulations.

`List` Method `removeFromBack`

Method `removeFromBack` (lines 95–118 of Fig. 22.3) removes the last node of a list and returns a reference to the removed data. The method throws an `EmptyListException` (lines 97–98) if the list is empty when the program calls this method. The steps are:

1. Assign `lastNode.data` (the data being removed from the list) to `removedItem` (line 100).
2. If the `firstNode` and `lastNode` refer to the same object (line 103), the list has only one element at this time. So, line 104 sets `firstNode` and `lastNode` to `null` to remove that node from the list (leaving the list empty).

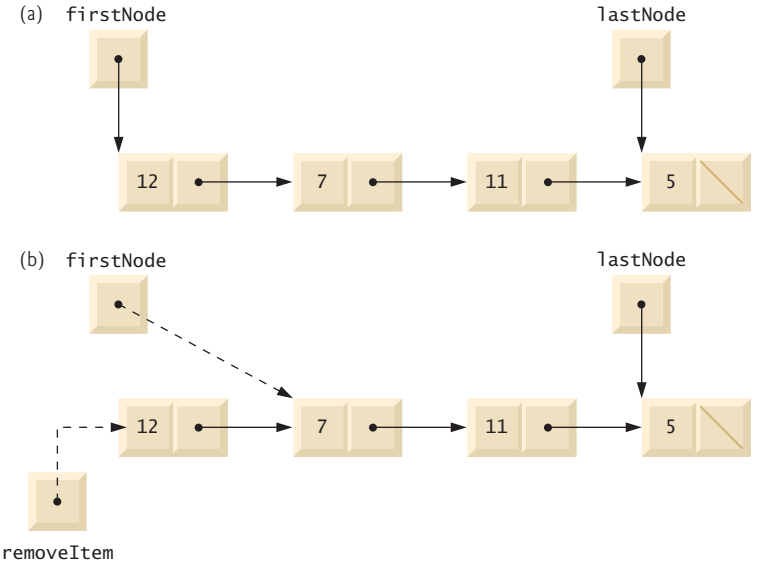


Fig. 22.8 | Graphical representation of operation `removeFromFront`.

3. If the list has more than one node, create the `ListNode` reference `current` and assign it `firstNode` (line 107).
4. Now "walk the list" with `current` until it references the node before the last node. The `while` loop (lines 110–111) assigns `current.nextNode` to `current` as long as `current.nextNode` (the next node in the list) is not `lastNode`.
5. After locating the second-to-last node, assign `current` to `lastNode` (line 113) to update which node is last in the list.
6. Set the `current.nextNode` to `null` (line 114) to remove the last node from the list and terminate the list at the current node.
7. Return the `removedItem` reference (line 117).

In Fig. 22.9, part (a) illustrates the list before the removal operation. The dashed lines and arrows in part (b) show the reference manipulations.

`List` Method `print`

Method `print` (lines 127–146) first determines whether the list is empty (lines 129–133). If so, `print` displays a message indicating that the list is empty and returns control to the calling method. Otherwise, `print` outputs the list's data. Line 136 creates `ListNode current` and initializes it with `firstNode`. While `current` is not `null`, there are more items in the list. Therefore, line 141 outputs a string representation of `current.data`. Line 142 moves to the next node in the list by assigning the value of reference `current.nextNode` to `current`. This printing algorithm is identical for linked lists, stacks and queues.

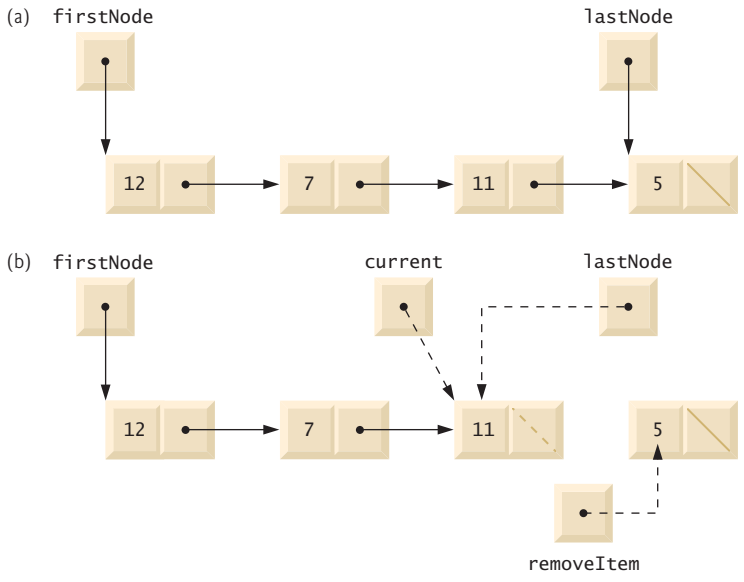


Fig. 22.9 | Graphical representation of operation `removeFromBack`.

22.5 Stacks

A stack is a constrained version of a list—*new nodes can be added to and removed from a stack only at the top*. For this reason, a stack is referred to as a **last-in, first-out** (**LIFO**) data structure. The link member in the bottom node is set to `null` to indicate the bottom of the stack. A stack is not required to be implemented as a linked list—it can also be implemented using an array.

The primary methods for manipulating a stack are **push** and **pop**, which add a new node to the top of the stack and remove a node from the top of the stack, respectively. Method `pop` also returns the data from the popped node.

Stacks have many interesting applications. For example, when a program calls a method, the called method must know how to return to its caller, so the return address of the calling method is pushed onto the program-execution stack (discussed in Section 6.6). If a series of method calls occurs, the successive return addresses are pushed onto the stack in last-in, first-out order so that each method can return to its caller. Stacks support recursive method calls in the *same* manner as they do conventional nonrecursive method calls.

The program-execution stack also contains the memory for local variables on each invocation of a method during a program's execution. When the method returns to its caller, the memory for that method's local variables is popped off the stack, and those variables are no longer known to the program. If the local variable is a reference and the object to which it referred has no other variables referring to it, the object can be garbage collected.

Compilers use stacks to evaluate arithmetic expressions and generate machine-language code to process them. The exercises in this chapter explore several applications of stacks, including using them to develop a complete working compiler. Also, package

`java.util` contains class `Stack` (see Chapter 20) for implementing and manipulating stacks that can grow and shrink during program execution.

In this section, we take advantage of the close relationship between lists and stacks to implement a stack class by reusing the `List<T>` class of Fig. 22.3. We demonstrate two different forms of reusability. First, we implement the stack class by extending class `List`. Then we implement an identically performing stack class through *composition* by including a reference to a `List` object as a `private` instance variable. The list, stack and queue data structures in this chapter are implemented to store references to objects of any type to encourage further reusability.

Stack Class That Inherits from `List<T>`

Figures 22.10 and 22.11 create and manipulate a stack class that extends the `List<T>` class of Fig. 22.3. We want the stack to have methods `push`, `pop`, `isEmpty` and `print`. Essentially, these are the `List<T>` methods `insertAtFront`, `removeFromFront`, `isEmpty` and `print`. Of course, class `List<T>` contains other methods (such as `insertAtBack` and `removeFromBack`) that we would rather not make accessible through the `public` interface to the stack class. It's important to remember that all methods in `List<T>`'s `public` interface class also are `public` methods of the subclass `StackInheritance<T>` (Fig. 22.10). Each method of `StackInheritance<T>` calls the appropriate `List<T>` method—for example, method `push` calls `insertAtFront` and method `pop` calls `removeFromFront`. `StackInheritance<T>` clients can call methods `isEmpty` and `print` because they're inherited from `List<T>`. Class `StackInheritance<T>` is declared in package `com.deitel.ch22` (line 3) for reuse. `StackInheritance<T>` does not import `List<T>`—the classes are in the same package.

```
1  // Fig. 22.10: StackInheritance.java
2  // StackInheritance extends class List.
3  package com.deitel.ch22;
4
5  public class StackInheritance< T > extends List< T >
6  {
7     // no-argument constructor
8     public StackInheritance()
9     {
10       super( "stack" );
11    } // end StackInheritance no-argument constructor
12
13    // add object to stack
14    public void push( T object )
15    {
16       insertAtFront( object );
17    } // end method push
18
19    // remove object from stack
20    public T pop() throws EmptyListException
21    {
22       return removeFromFront();
23    } // end method pop
24 } // end class StackInheritance
```

Fig. 22.10 | `StackInheritance` extends class `List`.

Class `StackInheritanceTest`'s method `main` (Fig. 22.11) creates an object of class `StackInheritance<T>` called `stack` (lines 10–11). The program pushes integers onto the stack (lines 14, 16, 18 and 20). *Autoboxing* is used here to insert `Integer` objects into the data structure. Lines 28–33 pop the objects from the stack in an infinite `while` loop. If method `pop` is invoked on an empty stack, the method throws an `EmptyListException`. In this case, the program displays the exception's stack trace, which shows the methods on the program-execution stack at the time the exception occurred. The program uses method `print` (inherited from `List`) to output the contents of the stack.

```
1  // Fig. 22.11: StackInheritanceTest.java
2  // Stack manipulation program.
3  import com.deitel.ch22.StackInheritance;
4  import com.deitel.ch22.EmptyListException;
5
6  public class StackInheritanceTest
7  {
8     public static void main( String[] args )
9     {
10       StackInheritance< Integer > stack =
11          new StackInheritance< Integer >();
12
13       // use push method
14       stack.push( -1 );
15       stack.print();
16       stack.push( 0 );
17       stack.print();
18       stack.push( 1 );
19       stack.print();
20       stack.push( 5 );
21       stack.print();
22
23       // remove items from stack
24       try
25       {
26          int removedItem;
27
28          while ( true )
29          {
30             removedItem = stack.pop(); // use pop method
31             System.out.printf( "\n%d popped\n", removedItem );
32             stack.print();
33          } // end while
34       } // end try
35       catch ( EmptyListException emptyListException )
36       {
37          emptyListException.printStackTrace();
38       } // end catch
39    } // end main
40 } // end class StackInheritanceTest
```

Fig. 22.11 | Stack manipulation program. (Part 1 of 2.)

```
The stack is: -1
The stack is: 0 -1

The stack is: 1  0 -1
The stack is: 5  1  0 -1

5 popped
The stack is: 1 0 -1

1 popped
The stack is: 0 -1

0 popped
The stack is: -1

-1 popped
Empty stack
com.deitel.ch22.EmptyListException: stack is empty
        at com.deitel.ch22.List.removeFromFront(List.java:81)
        at com.deitel.ch22.StackInheritance.pop(StackInheritance.java:22)
        at StackInheritanceTest.main(StackInheritanceTest.java:30)
```

Fig. 22.11 | Stack manipulation program. (Part 2 of 2.)

Stack Class That Contains a Reference to a `List`

You can also implement a class by reusing a list class through composition. Figure 22.12 uses a `private` `List<T>` (line 7) in class `StackComposition<T>`'s declaration. Composition enables us to hide the `List<T>` methods that should not be in our stack's `public` interface. We provide `public` interface methods that use only the required `List<T>` methods. Implementing each stack method as a call to a `List<T>` method is called **delegation**—the stack method invoked delegates the call to the appropriate `List<T>` method. In particular, `StackComposition<T>` delegates calls to `List<T>` methods `insertAtFront`, `removeFromFront`, `isEmpty` and `print`. In this example, we do not show class `StackCompositionTest`, because the only difference is that we change the type of the stack from `StackInheritance` to `StackComposition` (lines 3 and 10–11 of Fig. 22.11).

```
1  // Fig. 22.12: StackComposition.java
2  // StackComposition uses a composed List object.
3  package com.deitel.ch22;
4
5  public class StackComposition< T >
6  {
7     private List< T > stackList;
8
9     // no-argument constructor
10    public StackComposition()
11    {
12       stackList = new List< T >( "stack" );
13    } // end StackComposition no-argument constructor
14
```

Fig. 22.12 | `StackComposition` uses a composed `List` object. (Part 1 of 2.)

```
15      // add object to stack
16      public void push( T object )
17      {
18         stackList.insertAtFront( object );
19      } // end method push
20
21      // remove object from stack
22      public T pop() throws EmptyListException
23      {
24         return stackList.removeFromFront();
25      } // end method pop
26
27      // determine if stack is empty
28      public boolean isEmpty()
29      {
30         return stackList.isEmpty();
31      } // end method isEmpty
32
33      // output stack contents
34      public void print()
35      {
36         stackList.print();
37      } // end method print
38   } // end class StackComposition
```

Fig. 22.12 | `StackComposition` uses a composed `List` object. (Part 2 of 2.)

22.6 Queues

Another commonly used data structure is the queue. A queue is similar to a checkout line in a supermarket—the cashier services the person at the *beginning* of the line *first*. Other customers enter the line only at the end and wait for service. *Queue nodes are removed only from the head (or front) of the queue and are inserted only at the tail (or end).* For this reason, a queue is a **first-in, first-out** (**FIFO**) data structure. The insert and remove operations are known as **enqueue** and **dequeue**.

Queues have many uses in computer systems. Each CPU in a computer can service only one application at a time. Each application requiring processor time is placed in a queue. The application at the front of the queue is the next to receive service. Each application gradually advances to the front as the applications before it receive service.

Queues are also used to support **print spooling**. For example, a single printer might be shared by all users of a network. Many users can send print jobs to the printer, even when the printer is already busy. These print jobs are placed in a queue until the printer becomes available. A program called a **spooler** manages the queue to ensure that, as each print job completes, the next one is sent to the printer.

Information packets also *wait* in queues in computer networks. Each time a packet arrives at a network node, it must be routed to the next node along the path to the packet's final destination. The routing node routes one packet at a time, so additional packets are enqueued until the router can route them.

A file server in a computer network handles file-access requests from many clients throughout the network. Servers have a limited capacity to service requests from clients. When that capacity is exceeded, client requests *wait* in queues.

Figure 22.13 creates a `Queue<T>` class that contains a `List<T>` (Fig. 22.3) object and provides methods `enqueue`, `dequeue`, `isEmpty` and `print`. Class `List<T>` contains some methods (e.g., `insertAtFront` and `removeFromBack`) that we'd rather not make accessible through `Queue<T>`'s `public` interface. Using composition enables us to hide class `List<T>`'s other `public` methods from clients of class `Queue<T>`. Each `Queue<T>` method calls an appropriate `List<T>` method—method `enqueue` calls `List<T>` method `insertAtBack`, method `dequeue` calls `List<T>` method `removeFromFront`, method `isEmpty` calls `List<T>` method `isEmpty` and method `print` calls `List<T>` method `print`. For reuse, class `Queue<T>` is declared in package `com.deitel.ch22`.

Class `QueueTest`'s (Fig. 22.14) `main` method creates and initializes `Queue<T>` variable `queue` (line 10). Lines 13, 15, 17 and 19 enqueue four integers, taking advantage of *auto-*

```
1  // Fig. 22.13: Queue.java
2  // Queue uses class List.
3  package com.deitel.ch22;
4
5  public class Queue
6  {
7     private List< T > queueList;
8
9     // no-argument constructor
10    public Queue()
11    {
12       queueList = new List< T >( "queue" );
13    } // end Queue no-argument constructor
14
15    // add object to queue
16    public void enqueue( T object )
17    {
18       queueList.insertAtBack( object );
19    } // end method enqueue
20
21    // remove object from queue
22    public T dequeue() throws EmptyListException
23    {
24       return queueList.removeFromFront();
25    } // end method dequeue
26
27    // determine if queue is empty
28    public boolean isEmpty()
29    {
30       return queueList.isEmpty();
31    } // end method isEmpty
32
33    // output queue contents
34    public void print()
35    {
36       queueList.print();
37    } // end method print
38 } // end class Queue
```

Fig. 22.13 | `Queue` uses class `List`.

boxing to insert `Integer` objects into the queue. Lines 27–32 use an infinite loop to dequeue the objects in first-in, first-out order. When the queue is empty, method `dequeue` throws an `EmptyListException`, and the program displays the exception's stack trace.

```
1  // Fig. 22.14: QueueTest.java
2  // Class QueueTest.
3  import com.deitel.ch22.Queue;
4  import com.deitel.ch22.EmptyListException;
5
6  public class QueueTest
7  {
8     public static void main( String[] args )
9     {
10       Queue< Integer > queue = new Queue< Integer >();
11
12       // use enqueue method
13       queue.enqueue( -1 );
14       queue.print();
15       queue.enqueue( 0 );
16       queue.print();
17       queue.enqueue( 1 );
18       queue.print();
19       queue.enqueue( 5 );
20       queue.print();
21
22       // remove objects from queue
23       try
24       {
25          int removedItem;
26
27          while ( true )
28          {
29             removedItem = queue.dequeue(); // use dequeue method
30             System.out.printf( "\n%d dequeued\n", removedItem );
31             queue.print();
32          } // end while
33       } // end try
34       catch ( EmptyListException emptyListException )
35       {
36          emptyListException.printStackTrace();
37       } // end catch
38    } // end main
39 } // end class QueueTest
```

```
The queue is: -1
The queue is: -1 0
The queue is: -1 0 1
The queue is: -1 0 1 5

-1 dequeued
The queue is: 0 1 5
```

Fig. 22.14 | Queue processing program. (Part 1 of 2.)

```
0 dequeued
The queue is: 1 5

1 dequeued
The queue is: 5

5 dequeued
Empty queue
com.deitel.ch22.EmptyListException: queue is empty
        at com.deitel.ch22.List.removeFromFront(List.java:81)
        at com.deitel.ch22.Queue.dequeue(Queue.java:24)
        at QueueTest.main(QueueTest.java:29)
```

Fig. 22.14 | Queue processing program. (Part 2 of 2.)

22.7 Trees

Lists, stacks and queues are **linear data structures** (i.e., **sequences**). A tree is a nonlinear, two-dimensional data structure with special properties. Tree nodes contain two or more links. This section discusses binary trees (Fig. 22.15)—trees whose nodes each contain two links (one or both of which may be `null`). The **root node** is the first node in a tree. Each link in the root node refers to a **child**. The **left child** is the first node in the **left subtree** (also known as the root node of the left subtree), and the **right child** is the first node in the **right subtree** (also known as the root node of the right subtree). The children of a specific node are called **siblings**. A node with no children is called a **leaf node**. Computer scientists normally draw trees from the root node down—the opposite of the way most trees grow in nature.

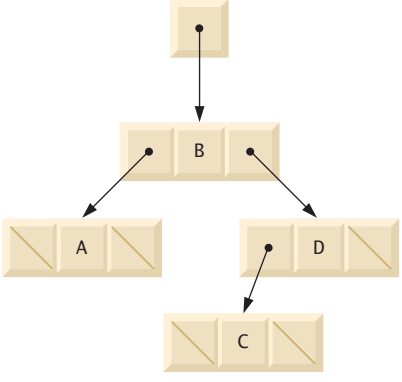


Fig. 22.15 | Binary tree graphical representation.

In our example, we create a special binary tree called a **binary search tree**. A binary search tree (with no duplicate node values) has the characteristic that the values in any left subtree are less than the value in that subtree's parent node, and the values in any right subtree are greater than the value in that subtree's parent node. Figure 22.16 illustrates a binary search tree with 12 integer values. The shape of the binary search tree that corre-

sponds to a set of data can vary, depending on the order in which the values are inserted into the tree.

Figures 22.17 and 22.18 create a generic binary search tree class and use it to manipulate a tree of integers. The application in Fig. 22.18 *traverses* the tree (i.e., walks through all its nodes) three ways—using recursive **inorder**, **preorder** and **postorder traversals**. The program generates 10 random numbers and inserts each into the tree. Class `Tree<T>` is declared in package `com.deitel.ch22` for reuse.

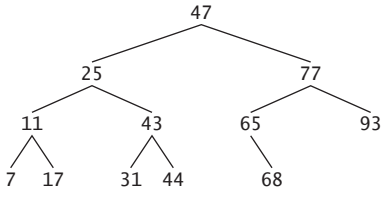


Fig. 22.16 | Binary search tree containing 12 values.

```
1  // Fig. 22.17: Tree.java
2  // TreeNode and Tree class declarations for a binary search tree.
3  package com.deitel.ch22;
4
5  // class TreeNode definition
6  class TreeNode< T extends Comparable< T > >
7  {
8     // package access members
9     TreeNode< T > leftNode; // left node
10    T data; // node value
11    TreeNode< T > rightNode; // right node
12
13    // constructor initializes data and makes this a leaf node
14    public TreeNode( T nodeData )
15    {
16       data = nodeData;
17       leftNode = rightNode = null; // node has no children
18    } // end TreeNode constructor
19
20    // locate insertion point and insert new node; ignore duplicate values
21    public void insert( T insertValue )
22    {
23       // insert in left subtree
24       if ( insertValue.compareTo( data ) < 0 )
25       {
26          // insert new TreeNode
27          if ( leftNode == null )
28             leftNode = new TreeNode< T >( insertValue );
29          else // continue traversing left subtree recursively
30             leftNode.insert( insertValue );
31       } // end if
```

Fig. 22.17 | `TreeNode` and `Tree` class declarations for a binary search tree. (Part 1 of 3.)

```
32          // insert in right subtree
33          else if ( insertValue.compareTo( data ) > 0 )
34          {
35             // insert new TreeNode
36             if ( rightNode == null )
37                rightNode = new TreeNode< T >( insertValue );
38             else // continue traversing right subtree recursively
39                rightNode.insert( insertValue );
40          } // end else if
41       } // end method insert
42    } // end class TreeNode
43
44    // class Tree definition
45    public class Tree< T extends Comparable< T > >
46    {
47       private TreeNode< T > root;
48
49       // constructor initializes an empty Tree of integers
50       public Tree()
51       {
52          root = null;
53       } // end Tree no-argument constructor
54
55       // insert a new node in the binary search tree
56       public void insertNode( T insertValue )
57       {
58          if ( root == null )
59             root = new TreeNode< T >( insertValue ); // create root node
60          else
61             root.insert( insertValue ); // call the insert method
62       } // end method insertNode
63
64       // begin preorder traversal
65       public void preorderTraversal()
66       {
67          preorderHelper( root );
68       } // end method preorderTraversal
69
70       // recursive method to perform preorder traversal
71       private void preorderHelper( TreeNode< T > node )
72       {
73          if ( node == null )
74             return;
75
76          System.out.printf( "%s ", node.data ); // output node data
77          preorderHelper( node.leftNode );       // traverse left subtree
78          preorderHelper( node.rightNode );      // traverse right subtree
79       } // end method preorderHelper
80
81       // begin inorder traversal
82       public void inorderTraversal()
83       {
```

Fig. 22.17 | `TreeNode` and `Tree` class declarations for a binary search tree. (Part 2 of 3.)

```
84          inorderHelper( root );
85       } // end method inorderTraversal
86
87       // recursive method to perform inorder traversal
88       private void inorderHelper( TreeNode< T > node )
89       {
90          if ( node == null )
91             return;
92
93          inorderHelper( node.leftNode );        // traverse left subtree
94          System.out.printf( "%s ", node.data ); // output node data
95          inorderHelper( node.rightNode );       // traverse right subtree
96       } // end method inorderHelper
97
98       // begin postorder traversal
99       public void postorderTraversal()
100      {
101         postorderHelper( root );
102      } // end method postorderTraversal
103
104      // recursive method to perform postorder traversal
105      private void postorderHelper( TreeNode< T > node )
106      {
107         if ( node == null )
108            return;
109
110         postorderHelper( node.leftNode );      // traverse left subtree
111         postorderHelper( node.rightNode );     // traverse right subtree
112         System.out.printf( "%s ", node.data ); // output node data
113      } // end method postorderHelper
114   } // end class Tree
```

Fig. 22.17 | TreeNode and Tree class declarations for a binary search tree. (Part 3 of 3.)

```
1   // Fig. 22.18: TreeTest.java
2   // Binary tree test program.
3   import java.util.Random;
4   import com.deitel.ch22.Tree;
5
6   public class TreeTest
7   {
8      public static void main( String[] args )
9      {
10        Tree< Integer > tree = new Tree< Integer >();
11        int value;
12        Random randomNumber = new Random();
13
14        System.out.println( "Inserting the following values: " );
15
16        // insert 10 random integers from 0-99 in tree
17        for ( int i = 1; i <= 10; i++ )
18        {
```

Fig. 22.18 | Binary tree test program. (Part 1 of 2.)

```
19            value = randomNumber.nextInt( 100 );
20            System.out.printf( "%d ", value );
21            tree.insertNode( value );
22         } // end for
23
24         System.out.println ( "\n\nPreorder traversal" );
25         tree.preorderTraversal(); // perform preorder traversal of tree
26
27         System.out.println ( "\n\nInorder traversal" );
28         tree.inorderTraversal(); // perform inorder traversal of tree
29
30         System.out.println ( "\n\nPostorder traversal" );
31         tree.postorderTraversal(); // perform postorder traversal of tree
32         System.out.println();
33      } // end main
34   } // end class TreeTest
```

```
Inserting the following values:
49 64 14 34 85 64 46 14 37 55

Preorder traversal
49 14 34 46 37 64 55 85

Inorder traversal
14 34 37 46 49 55 64 85

Postorder traversal
37 46 34 14 55 85 64 49
```

Fig. 22.18 | Binary tree test program. (Part 2 of 2.)

Let's walk through the binary tree program. Method `main` of class `TreeTest` (Fig. 22.18) begins by instantiating an empty `Tree<T>` object and assigning its reference to variable `tree` (line 10). Lines 17–22 randomly generate 10 integers, each of which is inserted into the binary tree by calling method `insertNode` (line 21). The program then performs preorder, inorder and postorder traversals (these will be explained shortly) of `tree` (lines 25, 28 and 31, respectively).

Overview of Class `Tree`

Class `Tree` (Fig. 22.17, lines 45–114) requires its type argument to implement interface `Comparable`, so that each value inserted in the tree can be *compared* with the existing values to find the insertion point. The class has `private` field `root` (line 47)—a `TreeNode` reference to the root node of the tree. `Tree`'s constructor (lines 50–53) initializes `root` to `null` to indicate that the tree is *empty*. The class contains method `insertNode` (lines 56–62) to insert a new node in the tree and methods `preorderTraversal` (lines 65–68), `inorderTraversal` (lines 82–85) and `postorderTraversal` (lines 99–102) to begin traversals of the tree. Each of these methods calls a recursive utility method to perform the traversal operations on the internal representation of the tree.

`Tree` *Method* `insertNode`

Class `Tree`'s method `insertNode` (lines 56–62) first determines whether the tree is empty. If so, line 59 allocates a new `TreeNode`, initializes the node with the value being inserted

in the tree and assigns the new node to reference `root`. If the tree is not empty, line 61 calls `TreeNode` method `insert` (lines 21–41). This method uses recursion to determine the location for the new node in the tree and inserts the node at that location. A node can be inserted only as a leaf node in a binary search tree.

`TreeNode` *Method* `insert`

`TreeNode` method `insert` compares the value to insert with the `data` value in the root node. If the insert value is less than the root node data (line 24), the program determines whether the left subtree is empty (line 27). If so, line 28 allocates a new `TreeNode`, initializes it with the value being inserted and assigns the new node to reference `leftNode`. Otherwise, line 30 recursively calls `insert` for the left subtree to insert the value into the left subtree. If the insert value is greater than the root node data (line 33), the program determines whether the right subtree is empty (line 36). If so, line 37 allocates a new `TreeNode`, initializes it with the value being inserted and assigns the new node to reference `rightNode`. Otherwise, line 39 recursively calls `insert` for the right subtree to insert the value in the right subtree. If the `insertValue` is already in the tree, it's simply ignored.

`Tree` *Methods* `inorderTraversal`, `preorderTraversal` *and* `postorderTraversal`

Methods `inorderTraversal`, `preorderTraversal` and `postorderTraversal` call `Tree` helper methods `inorderHelper` (lines 88–96), `preorderHelper` (lines 71–79) and `postorderHelper` (lines 105–113), respectively, to traverse the tree and print the node values. The helper methods in class `Tree` enable you to start a traversal without having to pass the `root` node to the method. Reference `root` is an *implementation detail* that a programmer should not be able to access. Methods `inorderTraversal`, `preorderTraversal` and `postorderTraversal` simply take the private `root` reference and pass it to the appropriate helper method to initiate a traversal of the tree. The base case for each helper method determines whether the reference it receives is `null` and, if so, returns immediately.

Method `inorderHelper` (lines 88–96) defines the steps for an inorder traversal:

1. Traverse the left subtree with a call to `inorderHelper` (line 93).
2. Process the value in the node (line 94).
3. Traverse the right subtree with a call to `inorderHelper` (line 95).

The inorder traversal does not process the value in a node until the values in that node's left subtree are processed. The inorder traversal of the tree in Fig. 22.19 is

```
6 13 17 27 33 42 48
```

The inorder traversal of a binary search tree prints the node values in *ascending order*. The process of creating a binary search tree actually sorts the data; thus, it's called the **binary tree sort**.

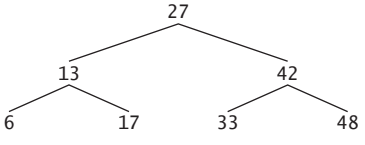


Fig. 22.19 | Binary search tree with seven values.

Method `preorderHelper` (lines 71–79) defines the steps for a preorder traversal:

1. Process the value in the node (line 76).
2. Traverse the left subtree with a call to `preorderHelper` (line 77).
3. Traverse the right subtree with a call to `preorderHelper` (line 78).

The preorder traversal processes the value in each node as the node is visited. After processing the value in a particular node, it processes the values in the left subtree, then processes the values in the right subtree. The preorder traversal of the tree in Fig. 22.19 is

```
27 13 6 17 42 33 48
```

Method `postorderHelper` (lines 105–113) defines the steps for a postorder traversal:

1. Traverse the left subtree with a call to `postorderHelper` (line 110).
2. Traverse the right subtree with a call to `postorderHelper` (line 111).
3. Process the value in the node (line 112).

The postorder traversal processes the value in each node after the values of all that node's children are processed. The `postorderTraversal` of the tree in Fig. 22.19 is

```
6 17 13 33 48 42 27
```

The binary search tree facilitates **duplicate elimination**. While building a tree, the insertion operation recognizes attempts to insert a duplicate value, because a duplicate follows the same "go left" or "go right" decisions on each comparison as the original value did. Thus, the insertion operation eventually compares the duplicate with a node containing the same value. At this point, the insertion operation can decide to *discard* the duplicate value (as we do in this example).

Searching a binary tree for a value that matches a key value is fast, especially for **tightly packed** (or **balanced**) **trees**. In a tightly packed tree, each level contains about twice as many elements as the previous level. Figure 22.19 is a tightly packed binary tree. A tightly packed binary search tree with n elements has $\log_2 n$ levels. Thus, at most $\log_2 n$ comparisons are required either to find a match or to determine that no match exists. Searching a (tightly packed) 1000-element binary search tree requires at most 10 comparisons, because $2^{10} > 1000$. Searching a (tightly packed) 1,000,000-element binary search tree requires at most 20 comparisons, because $2^{20} > 1{,}000{,}000$.

The chapter exercises present algorithms for several other binary tree operations, such as deleting an item from a binary tree, printing a binary tree in a two-dimensional tree format and performing a **level-order traversal of a binary tree**. The level-order traversal visits the nodes of the tree row by row, starting at the root node level. On each level of the tree, a level-order traversal visits the nodes from left to right. Other binary tree exercises include allowing a binary search tree to contain duplicate values, inserting string values in a binary tree and determining how many levels are contained in a binary tree.

22.8 Wrap-Up

This chapter completes our presentation of data structures. We began in Chapter 20 with an introduction to the built-in collections of the Java Collections Framework and continued in Chapter 21 by showing you how to implement generic methods and collections. In this chapter, you learned to build generic dynamic data structures that grow and shrink at

execution time. You learned that linked lists are collections of data items that are "linked up in a chain." You also saw that an application can perform insertions and deletions at the beginning and end of a linked list. You learned that the stack and queue data structures are constrained versions of lists. For stacks, you saw that insertions and deletions are made only at the top. For queues that represent waiting lines, you saw that insertions are made at the tail and deletions are made from the head. You also learned the binary tree data structure. You saw a binary search tree that facilitated high-speed searching and sorting of data and eliminating duplicate data items efficiently. Throughout the chapter, you learned how to create and package these data structures for reusability and maintainability.

Next, we introduce Java applets—Java programs that typically execute in a browser. We overview the JDK's sample applets, then show you how to write and execute your own applets. We then introduce the Java Web Start capabilities for launching an applet and installing a desktop shortcut to relaunch the applet in the future without having to revisit the applet's website.

Summary

Section 22.1 Introduction

- Dynamic data structures (p. 905) can grow and shrink at execution time.
- Linked lists (p. 905) are collections of data items "linked up in a chain"—insertions and deletions can be made anywhere in a linked list.
- Stacks (p. 905) are important in compilers and operating systems—insertions and deletions are made only at the top (p. 905) of a stack.
- In a queue, insertions are made at the tail (p. 905) and deletions are made from the head (p. 905).
- Binary trees (p. 905) facilitate high-speed searching and sorting, eliminating duplicate data items efficiently, representing file-system directories and compiling expressions into machine language.

Section 22.2 Self-Referential Classes

- A self-referential class (p. 905) contains a reference that refers to another object of the same class type. Self-referential objects can be linked together to form dynamic data structures.

Section 22.3 Dynamic Memory Allocation

- The limit for dynamic memory allocation (p. 906) can be as large as the available physical memory in the computer or the available disk space in a virtual-memory system. Often the limits are much smaller, because the computer's available memory must be shared among many users.
- If no memory is available, an `OutOfMemoryError` is thrown.

Section 22.4 Linked Lists

- A linked list is accessed via a reference to the first node of the list. Each subsequent node is accessed via the link-reference member stored in the previous node.
- By convention, the link reference in the last node of a list is set to `null` to mark the end of the list.
- A node can contain data of any type, including objects of other classes.
- A linked list is appropriate when the number of data elements to be stored is unpredictable. Linked lists are dynamic, so the length of a list can increase or decrease as necessary.
- The size of a "conventional" Java array cannot be altered—it's fixed at creation time.
- List nodes normally are not stored in contiguous memory. Rather, they're logically contiguous.

Section 22.5 Stacks

- A stack is a last-in, first-out (LIFO) data structure (p. 917). The primary methods used to manipulate a stack are `push` (p. 917) and `pop` (p. 917), which add a new node to the stack's top and remove a node from the top, respectively. Method `pop` returns the removed node's data.
- When a method call is made, the called method must know how to return to its caller, so the return address is pushed onto the program-execution stack. If a series of method calls occurs, the successive return values are pushed onto the stack in last-in, first-out order.
- The program-execution stack contains the space created for local variables on each invocation of a method. When the method returns to its caller, the space for that method's local variables is popped off the stack, and those variables are no longer available to the program.
- Stacks are used by compilers to evaluate arithmetic expressions and generate machine-language code to process the expressions.
- The technique of implementing each stack method as a call to a `List` method is called delegation—the stack method invoked delegates (p. 920) the call to the appropriate `List` method.

Section 22.6 Queues

- A queue (p. 905) is similar to a checkout line in a supermarket—the first person in line is serviced first, and other customers enter the line only at the end and wait to be serviced.
- Queue nodes are removed only from the head (p. 905) of the queue and are inserted only at the tail. For this reason, a queue is referred to as a first-in, first-out FIFOaaaa data structure.
- The insert and remove operations for a queue are known as `enqueue` (p. 921) and `dequeue` (p. 921).
- Queues have many uses in computer systems. Most computers have only a single processor, so only one application at a time can be serviced. Entries for the other applications are placed in a queue. The entry at the front of the queue is the next to receive service. Each entry gradually advances to the front of the queue as applications receive service.

Section 22.7 Trees

- A tree is a nonlinear, two-dimensional data structure. Tree nodes contain two or more links.
- A binary tree (p. 905) is a tree whose nodes all contain two links. The root node (p. 924) is the first node in a tree.
- Each link in the root node refers to a child (p. 924). The left child (p. 924) is the first node in the left subtree (p. 924), and the right child (p. 924) is the first node in the right subtree (p. 924).
- The children of a node are called siblings (p. 924). A node with no children is a leaf node (p. 924).
- In a binary search tree (p. 924) with no duplicate values, the values in any left subtree are less than the value in the subtree's parent node, and the values in any right subtree are greater than the value in the subtree's parent node. A node can be inserted only as a leaf node in a binary search tree.
- An inorder traversal (p. 925) of a binary search tree processes the node values in ascending order.
- In a preorder traversal (p. 925), the value in each node is processed as the node is visited. Then the values in the left subtree are processed, then the values in the right subtree.
- In a postorder traversal (p. 925), the value in each node is processed after the values of its children.
- The binary search tree facilitates duplicate elimination (p. 930). As the tree is created, attempts to insert a duplicate value are recognized, because a duplicate follows the same "go left" or "go right" decisions on each comparison as the original value did. Thus, the duplicate eventually is compared with a node containing the same value. The duplicate value can be discarded at this point.

- In a tightly packed tree (p. 930), each level contains about twice as many elements as the previous one. So a tightly packed binary search tree with n elements has $\log_2 n$ levels, and thus at most $\log_2 n$ comparisons would have to be made either to find a match or to determine that no match exists. Searching a (tightly packed) 1000-element binary search tree requires at most 10 comparisons, because $2^{10} > 1000$. Searching a (tightly packed) 1,000,000-element binary search tree requires at most 20 comparisons, because $2^{20} > 1{,}000{,}000$.

Self-Review Exercises

22.1 Fill in the blanks in each of the following statements:

a) A self-_____ class is used to form dynamic data structures that can grow and shrink at execution time.
b) A(n) _____ is a constrained version of a linked list in which nodes can be inserted and deleted only from the start of the list.
c) A method that does not alter a linked list, but simply looks at it to determine whether it's empty, is referred to as a(n) _____ method.
d) A queue is referred to as a(n) _____ data structure because the first nodes inserted are the first ones removed.
e) The reference to the next node in a linked list is referred to as a(n) _____.
f) Automatically reclaiming dynamically allocated memory in Java is called _____.
g) A(n) _____ is a constrained version of a linked list in which nodes can be inserted only at the end of the list and deleted only from the start of the list.
h) A(n) _____ is a nonlinear, two-dimensional data structure that contains nodes with two or more links.
i) A stack is referred to as a(n) _____ data structure because the last node inserted is the first node removed.
j) The nodes of a(n) _____ tree contain two link members.
k) The first node of a tree is the _____ node.
l) Each link in a tree node refers to a(n) _____ or _____ of that node.
m) A tree node that has no children is called a(n) _____ node.
n) The three traversal algorithms we mentioned in the text for binary search trees are _____, _____ and _____.

22.2 What are the differences between a linked list and a stack?

22.3 What are the differences between a stack and a queue?

22.4 Comment on how each of the following entities or concepts contributes to the reusability of data structures:

a) classes
b) inheritance
c) composition

22.5 Provide the inorder, preorder and postorder traversals of the binary search tree of Fig. 22.20.

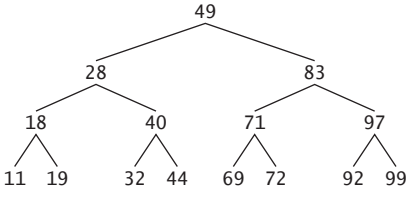


Fig. 22.20 | Binary search tree with 15 nodes.

Answers to Self-Review Exercises

22.1 a) referential. b) stack. c) predicate. d) first-in, first-out (FIFO). e) link. f) garbage collection. g) queue. h) tree i) last-in, first-out (LIFO). j) binary. k) root. l) child or subtree. m) leaf. n) inorder, preorder, postorder.

22.2 It's possible to insert a node anywhere in a linked list and remove a node from anywhere in a linked list. Nodes in a stack may be inserted only at the top of the stack and removed only from the top.

22.3 A queue is a FIFO data structure that has references to both its head and its tail, so that nodes may be inserted at the tail and deleted from the head. A stack is a LIFO data structure that has a single reference to the stack's top, where both insertion and deletion of nodes are performed.

22.4 a) Classes allow us to create as many data structure objects as we wish.
b) Inheritance enables a subclass to reuse the functionality from a superclass. Public and protected superclass methods can be accessed through a subclass to eliminate duplicate logic.
c) Composition enables a class to reuse code by storing a reference to an instance of another class in a field. Public methods of the instance can be called by methods in the class that contains the reference.

22.5 The inorder traversal is

```
11 18 19 28 32 40 44 49 69 71 72 83 92 97 99
```

The preorder traversal is

```
49 28 18 11 19 40 32 44 83 71 69 72 97 92 99
```

The postorder traversal is

```
11 19 18 32 44 40 28 69 72 71 92 99 97 83 49
```

Exercises

22.6 ***(Concatenating Lists)*** Write a program that concatenates two linked list objects of characters. Class `ListConcatenate` should include a `static` method `concatenate` that takes references to both list objects as arguments and concatenates the second list to the first list.

22.7 ***(Inserting into an Ordered List)*** Write a program that inserts 25 random integers from 0 to 100 in order into a linked-list object. For this exercise, you'll need to modify the `List<T>` class (Fig. 22.3) to maintain an ordered list. Name the new version of the class `SortedList`.

22.8 ***(Merging Ordered Lists)*** Modify the `SortedList` class from Exercise 22.7 to include a `merge` method that can merge the `SortedList` it receives as an argument with the `SortedList` that calls the method. Write an application to test method `merge`.

22.9 ***(Copying a List Backward)*** Write a `static` method `reverseCopy` that receives a `List<T>` as an argument and returns a copy of that `List<T>` with its elements reversed. Test this method in an application.

22.10 ***(Printing a Sentence in Reverse Using a Stack)*** Write a program that inputs a line of text and uses a stack to display the words of the line in reverse order.

22.11 ***(Palindrome Tester)*** Write a program that uses a stack to determine whether a string is a palindrome (i.e., the string is spelled identically backward and forward). The program should ignore spaces and punctuation.

22.12 ***(Infix-to-Postfix Converter)*** Stacks are used by compilers to help in the process of evaluating expressions and generating machine-language code. In this and the next exercise, we investigate how compilers evaluate arithmetic expressions consisting only of constants, operators and parentheses.

Humans generally write expressions like `3 + 4` and `7 / 9` in which the operator (`+` or `/` here) is written between its operands—this is called *infix notation*. Computers "prefer" *postfix notation*, in

which the operator is written to the right of its two operands. The preceding infix expressions would appear in postfix notation as `3 4 +` and `7 9 /`, respectively.

To evaluate a complex infix expression, a compiler would first convert the expression to postfix notation and evaluate the postfix version. Each of these algorithms requires only a single left-to-right pass of the expression. Each algorithm uses a stack object in support of its operation, but each uses the stack for a different purpose.

In this exercise, you'll write a Java version of the infix-to-postfix conversion algorithm. In the next exercise, you'll write a Java version of the postfix expression evaluation algorithm. In a later exercise, you'll discover that code you write in this exercise can help you implement a complete working compiler.

Write class `InfixToPostfixConverter` to convert an ordinary infix arithmetic expression (assume a valid expression is entered) with single-digit integers such as

```
(6 + 2) * 5 - 8 / 4
```

to a postfix expression. The postfix version (no parentheses are needed) of the this infix expression is

```
6 2 + 5 * 8 4 / -
```

The program should read the expression into `StringBuffer infix` and use one of the stack classes implemented in this chapter to help create the postfix expression in `StringBuffer postfix`. The algorithm for creating a postfix expression is as follows:

a) Push a left parenthesis `'('` onto the stack.
b) Append a right parenthesis `')'` to the end of `infix`.
c) While the stack is not empty, read `infix` from left to right and do the following:
    If the current character in `infix` is a digit, append it to `postfix`.
    If the current character in `infix` is a left parenthesis, push it onto the stack.
    If the current character in `infix` is an operator:
        Pop operators (if there are any) at the top of the stack while they have equal or higher precedence than the current operator, and append the popped operators to `postfix`.
        Push the current character in `infix` onto the stack.
    If the current character in `infix` is a right parenthesis:
        Pop operators from the top of the stack and append them to `postfix` until a left parenthesis is at the top of the stack.
        Pop (and discard) the left parenthesis from the stack.

The following arithmetic operations are allowed in an expression:

`+` addition
`-` subtraction
`*` multiplication
`/` division
`^` exponentiation
`%` remainder

The stack should be maintained with stack nodes that each contain an instance variable and a reference to the next stack node. Some methods you may want to provide are as follows:

a) Method `convertToPostfix`, which converts the infix expression to postfix notation.
b) Method `isOperator`, which determines whether `c` is an operator.
c) Method `precedence`, which determines whether the precedence of `operator1` (from the infix expression) is less than, equal to or greater than that of `operator2` (from the stack). The method returns `true` if `operator1` has lower precedence than `operator2`. Otherwise, `false` is returned.
d) Method `peek` (this should be added to the stack class), which returns the top value of the stack without popping the stack.

22.13 ***(Postfix Evaluator)*** Write class `PostfixEvaluator` that evaluates a postfix expression such as

```
6 2 + 5 * 8 4 / -
```

The program should read a postfix expression consisting of digits and operators into a `StringBuffer`. Using modified versions of the stack methods implemented earlier in this chapter, the program should scan the expression and evaluate it (assume it's valid). The algorithm is as follows:

a) Append a right parenthesis `')'` to the end of the postfix expression. When the right-parenthesis character is encountered, no further processing is necessary.
b) Until the right parenthesis is encountered, read the expression from left to right.
    If the current character is a digit, do the following:
        Push its integer value onto the stack (the integer value of a digit character is its value in the Unicode character set minus the value of `'0'` in Unicode).
    Otherwise, if the current character is an *operator*:
        Pop the two top elements of the stack into variables `x` and `y`.
        Calculate `y` *operator* `x`.
        Push the result of the calculation onto the stack.
c) When the right parenthesis is encountered in the expression, pop the top value of the stack. This is the result of the postfix expression.

[*Note:* In b) above (based on the sample expression at the beginning of this exercise), if the operator is `'/'`, the top of the stack is `4` and the next element in the stack is `40`, then pop `4` into `x`, pop `40` into `y`, evaluate `40 / 4` and push the result, `10`, back on the stack. This note also applies to operator `'-'`.] The arithmetic operations allowed in an expression are: `+` (addition), `-` (subtraction), `*` (multiplication), `/` (division), `^` (exponentiation) and `%` (remainder).

The stack should be maintained with one of the stack classes introduced in this chapter. You may want to provide the following methods:

a) Method `evaluatePostfixExpression`, which evaluates the postfix expression.
b) Method `calculate`, which evaluates the expression `op1 operator op2`.

22.14 ***(Postfix Evaluator Modification)*** Modify the postfix evaluator program of Exercise 22.13 so that it can process integer operands larger than 9.

22.15 ***(Supermarket Simulation)*** Write a program that simulates a checkout line at a supermarket. The line is a queue object. Customers (i.e., customer objects) arrive in random integer intervals of from 1 to 4 minutes. Also, each customer is serviced in random integer intervals of from 1 to 4 minutes. Obviously, the rates need to be balanced. If the average arrival rate is larger than the average service rate, the queue will grow infinitely. Even with "balanced" rates, randomness can still cause long lines. Run the supermarket simulation for a 12-hour day (720 minutes), using the following algorithm:

a) Choose a random integer between 1 and 4 to determine the minute at which the first customer arrives.
b) At the first customer's arrival time, do the following:
    Determine customer's service time (random integer from 1 to 4).
    Begin servicing the customer.
    Schedule arrival time of next customer (random integer 1 to 4 added to the current time).
c) For each simulated minute of the day, consider the following:
    If the next customer arrives, proceed as follows:
        Say so.
        Enqueue the customer.
        Schedule the arrival time of the next customer.
    If service was completed for the last customer, do the following:
        Say so.
        Dequeue next customer to be serviced.
        Determine customer's service completion time (random integer from 1 to 4 added to the current time).

Now run your simulation for 720 minutes and answer each of the following:

a) What is the maximum number of customers in the queue at any time?
b) What is the longest wait any one customer experiences?
c) What happens if the arrival interval is changed from 1 to 4 minutes to 1 to 3 minutes?

22.16 ***(Allowing Duplicates in a Binary Tree)*** Modify Figs. 22.17 and 22.18 to allow the binary tree to contain duplicates.

22.17 ***(Processing a Binary Search Tree of `Strings`)*** Write a program based on the program of Figs. 22.17 and 22.18 that inputs a line of text, tokenizes it into separate words, inserts the words in a binary search tree and prints the inorder, preorder and postorder traversals of the tree.

22.18 ***(Duplicate Elimination)*** In this chapter, we saw that duplicate elimination is straightforward when creating a binary search tree. Describe how you'd perform duplicate elimination when using only a one-dimensional array. Compare the performance of array-based duplicate elimination with the performance of binary-search-tree-based duplicate elimination.

22.19 ***(Depth of a Binary Tree)*** Modify Figs. 22.17 and 22.18 so the `Tree` class provides a method `getDepth` that determines how many levels are in the tree. Test the method in an application that inserts 20 random integers in a `Tree`.

22.20 ***(Recursively Print a List Backward)*** Modify the `List<T>` class of Fig. 22.3 to include method `printListBackward` that recursively outputs the items in a linked-list object in reverse order. Write a test program that creates a list of integers and prints the list in reverse order.

22.21 ***(Recursively Search a List)*** Modify the `List<T>` class of Fig. 22.3 to include method `search` that recursively searches a linked-list object for a specified value. The method should return a reference to the value if it's found; otherwise, it should return `null`. Use your method in a test program that creates a list of integers. The program should prompt the user for a value to locate in the list.

22.22 ***(Binary Tree Delete)*** In this exercise, we discuss deleting items from binary search trees. The deletion algorithm is not as straightforward as the insertion algorithm. Three cases are encountered when deleting an item—the item is contained in a leaf node (i.e., it has no children), or in a node that has one child or in a node that has two children.

If the item to be deleted is contained in a leaf node, the node is deleted and the reference in the parent node is set to null.

If the item to be deleted is contained in a node with one child, the reference in the parent node is set to reference the child node and the node containing the data item is deleted. This causes the child node to take the place of the deleted node in the tree.

The last case is the most difficult. When a node with two children is deleted, another node in the tree must take its place. However, the reference in the parent node cannot simply be assigned to reference one of the children of the node to be deleted. In most cases, the resulting binary search tree would not embody the following characteristic of binary search trees (with no duplicate values): *The values in any left subtree are less than the value in the parent node, and the values in any right subtree are greater than the value in the parent node.*

Which node is used as a *replacement node* to maintain this characteristic? It's either the node containing the largest value in the tree less than the value in the node being deleted, or the node containing the smallest value in the tree greater than the value in the node being deleted. Let's consider the node with the smaller value. In a binary search tree, the largest value less than a parent's value is located in the left subtree of the parent node and is guaranteed to be contained in the rightmost node of the subtree. This node is located by walking down the left subtree to the right until the reference to the right child of the current node is null. We're now referencing the replacement node, which is either a leaf node or a node with one child to its left. If the replacement node is a leaf node, the steps to perform the deletion are as follows:

a) Store the reference to the node to be deleted in a temporary reference variable.

b) Set the reference in the parent of the node being deleted to reference the replacement node.
c) Set the reference in the parent of the replacement node to `null`.
d) Set the reference to the right subtree in the replacement node to reference the right subtree of the node to be deleted.
e) Set the reference to the left subtree in the replacement node to reference the left subtree of the node to be deleted.

The deletion steps for a replacement node with a left child are similar to those for a replacement node with no children, but the algorithm also must move the child into the replacement node's position in the tree. If the replacement node is a node with a left child, the steps to perform the deletion are as follows:

a) Store the reference to the node to be deleted in a temporary reference variable.
b) Set the reference in the parent of the node being deleted to refer to the replacement node.
c) Set the reference in the parent of the replacement node to reference the left child of the replacement node.
d) Set the reference to the right subtree in the replacement node to reference the right subtree of the node to be deleted.
e) Set the reference to the left subtree in the replacement node to reference the left subtree of the node to be deleted.

Write method `deleteNode`, which takes as its argument the value to delete. Method `deleteNode` should locate in the tree the node containing the value to delete and use the algorithms discussed here to delete the node. If the value is not found in the tree, the method should display a message saying so. Modify the program of Figs. 22.17 and 22.18 to use this method. After deleting an item, call the methods `inorderTraversal`, `preorderTraversal` and `postorderTraversal` to confirm that the delete operation was performed correctly.

22.23 ***(Binary Tree Search)*** Modify class `Tree` of Fig. 22.17 to include method `contains`, which attempts to locate a specified value in a binary-search-tree object. The method should take as an argument a search key to locate. If the node containing the search key is found, the method should return a reference to that node's data; otherwise, it should return `null`.

22.24 ***(Level-Order Binary Tree Traversal)*** The program of Figs. 22.17 and 22.18 illustrated three recursive methods of traversing a binary tree—inorder, preorder and postorder traversals. This exercise presents the *level-order traversal* of a binary tree, in which the node values are printed level by level, starting at the root node level. The nodes on each level are printed from left to right. The level-order traversal is not a recursive algorithm. It uses a queue object to control the output of the nodes. The algorithm is as follows:

a) Insert the root node in the queue.
b) While there are nodes left in the queue, do the following:
    Get the next node in the queue.
    Print the node's value.
    If the reference to the left child of the node is not null:
        Insert the left child node in the queue.
    If the reference to the right child of the node is not null:
        Insert the right child node in the queue.

Write method `levelOrder` to perform a level-order traversal of a binary tree object. Modify the program of Figs. 22.17 and 22.18 to use this method. [*Note:* You'll also need to use the queue-processing methods of Fig. 22.13 in this program.]

22.25 ***(Printing Trees)*** Modify class `Tree` of Fig. 22.17 to include a recursive `outputTree` method to display a binary tree object. The method should output the tree row by row, with the top of the tree at the left of the screen and the bottom of the tree toward the right. Each row is output vertically. For example, the binary tree illustrated in Fig. 22.20 is output as shown in Fig. 22.21.

The rightmost leaf node appears at the top of the output in the rightmost column and the root node appears at the left of the output. Each column starts five spaces to the right of the preceding column. Method `outputTree` should receive an argument `totalSpaces` representing the number of spaces preceding the value to be output. (This variable should start at zero so that the root node is output at the left of the screen.) The method uses a modified inorder traversal to output the tree—it starts at the rightmost node in the tree and works back to the left. The algorithm is as follows:

```
            99
        97
            92
    83
            72
        71
            69
49
            44
        40
            32
    28
            19
        18
            11
```

Fig. 22.21 | Sample output of recursive method `outputTree`.

While the reference to the current node is not null, perform the following:
Recursively call `outputTree` with the right subtree of the current node and `totalSpaces + 5`.
Use a `for` statement to count from 1 to `totalSpaces` and output spaces.
Output the value in the current node.
Set the reference to the current node to refer to the left subtree of the current node.
Increment `totalSpaces` by 5.

22.26 ***(Insert/Delete Anywhere in a Linked List)*** Our linked list class allowed insertions and deletions at only the front and the back of the linked list. These capabilities were convenient for us when we used inheritance or composition to produce a stack class and a queue class with a minimal amount of code simply by reusing the list class. Linked lists are normally more general than those we provided. Modify the linked list class we developed in this chapter to handle insertions and deletions anywhere in the list.

22.27 ***(Lists and Queues without Tail References)*** Our linked-list implementation (Fig. 22.3) used both a `firstNode` and a `lastNode`. The `lastNode` was useful for the `insertAtBack` and `removeFromBack` methods of the `List` class. The `insertAtBack` method corresponds to the `enqueue` method of the `Queue` class. Rewrite the `List` class so that it does not use a `lastNode`. Thus, any operations on the tail of a list must begin searching the list from the front. Does this affect our implementation of the `Queue` class (Fig. 22.13)?

22.28 ***(Performance of Binary Tree Sorting and Searching)*** One problem with the binary tree sort is that the order in which the data is inserted affects the shape of the tree—for the same collection of data, different orderings can yield binary trees of dramatically different shapes. The performance of the binary tree sorting and searching algorithms is sensitive to the shape of the binary tree. What shape would a binary tree have if its data were inserted in increasing order? in decreasing order? What shape should the tree have to achieve maximal searching performance?

22.29 ***(Indexed Lists)*** As presented in the text, linked lists must be searched sequentially. For large lists, this can result in poor performance. A common technique for improving list-searching performance is to create and maintain an index to the list. An index is a set of references to key places in the

list. For example, an application that searches a large list of names could improve performance by creating an index with 26 entries—one for each letter of the alphabet. A search operation for a last name beginning with 'Y' would then first search the index to determine where the 'Y' entries began, then "jump into" the list at that point and search linearly until the desired name was found. This would be much faster than searching the linked list from the beginning. Use the `List` class of Fig. 22.3 as the basis of an `IndexedList` class. Write a program that demonstrates the operation of indexed lists. Be sure to include methods `insertInIndexedList`, `searchIndexedList` and `deleteFromIndexedList`.

22.30 ***(Queue Class that Inherits from a List Class)*** In Section 22.5, we created a stack class from class `List` with inheritance (Fig. 22.10) and with composition (Fig. 22.12). In Section 22.6 we created a queue class from class `List` with composition (Fig. 22.13). Create a queue class by inheriting from class `List`. What are the differences between this class and the one we created with composition?

Special Section: Building Your Own Compiler

In Exercises 7.35–7.37, we introduced Simpletron Machine Language (SML), and you implemented a Simpletron computer simulator to execute SML programs. In Exercises 22.31–22.35, we build a compiler that converts programs written in a high-level programming language to SML. This section "ties" together the entire programming process. You'll write programs in this new high-level language, compile them on the compiler you build and run them on the simulator you built in Exercise 7.36. You should make every effort to implement your compiler in an object-oriented manner. [*Note:* Due to the size of the descriptions for Exercises 22.31–22.35, we've posted them in a PDF document located at `www.deitel.com/books/jhtp9/`.]

23

Applets and Java Web Start



Observe due measure, for right timing is in all things the most important factor.
—Hesiod

Painting is only a bridge linking the painter's mind with that of the viewer.
—Eugene Delacroix

The direction in which education starts a man will determine his future in life.
—Plato

Objectives

In this chapter you'll learn:

- What applets are and how they're used in web pages.
- To observe some of Java's exciting capabilities through the JDK's demonstration applets.
- To write simple applets.
- To write a simple HyperText Markup Language (HTML) document to load an applet into an applet container and execute the applet.
- Applet life-cycle methods.
- About the sandbox security model for running downloaded code safely.
- What Java Web Start is and how to use it to download, install and run applets outside of the web browser.

Outline

23.1 Introduction
23.2 Sample Applets Provided with the JDK
23.3 Simple Java Applet: Drawing a String
23.3.1 Executing `WelcomeApplet` in the `appletviewer`
23.3.2 Executing an Applet in a Web Browser
23.4 Applet Life-Cycle Methods
23.5 Initialization with Method `init`
23.6 Sandbox Security Model
23.7 Java Web Start and the Java Network Launch Protocol (JNLP)
23.7.1 Packaging the `DrawTest` Applet for Use with Java Web Start
23.7.2 JNLP Document for the `DrawTest` Applet
23.8 Wrap-Up

Summary | *Self-Review Exercise* | *Answers to Self-Review Exercise* | *Exercises*

23.1 Introduction

[*Note:* This chapter is intentionally small and simple for readers who wish to study applets after reading only the book's first few chapters. We present more complex applets in Chapter 24, Multimedia: Applets and Applications, and Chapter 27, Networking. Also, the examples in this chapter require some knowledge of HTML to create a web page that loads an applet. With each example we supply sample HTML documents that you can modify for your own purposes.

This chapter introduces **applets**—Java programs that are typically embedded in **HTML (HyperText Markup Language) documents**—also called web pages. When a Java-enabled web browser loads a web page containing an applet, the applet downloads into the browser and executes.

Applet Containers

The application in which an applet executes is known as the **applet container**. It's the applet container's responsibility to load the applet's class(es), create an instance of the applet and manage its life cycle (which we discuss in more detail in Section 23.4). The Java Development Kit (JDK) includes one called the **`appletviewer`** for testing applets as you develop them and before you embed them in web pages. We demonstrate applets using both the `appletviewer` and web browsers, which execute Java applets via the **Java Plug-In**. Some browsers don't come with the plug-in by default. You can visit `java.com` to determine whether your browser is ready to execute Java applets. If not, you can click the **Free Java Download** button to install Java for your browser. Several popular browsers are supported. We tested our applets in Mozilla's Firefox 3.6, Microsoft's Internet Explorer 8, Google's Chrome, Opera 11 and Apple's Safari 5.

Java Web Start and the Java Network Launch Protocol (JNLP)

This chapter concludes with an introduction to Java Web Start and the Java Network Launch Protocol (JNLP). Together, these enable you to package your applets and applications so that they can be installed onto the user's desktop. As you'll learn in Chapter 24, Java Web Start also enables you to give the user control over whether an applet or application downloaded from the web can have limited access to resources on the local file system. For example, if you create a downloadable text-editor program in Java, users would probably want to store their documents on their own computers.

23.2 Sample Applets Provided with the JDK

Before we discuss our own applets, let's consider several demonstration applets provided with the JDK. Each sample applet comes with its source code.

The demonstration programs provided with the JDK are located in a directory called `demo`. For Windows, the default location of the JDK 6.0's `demo` directory is

```
C:\Program Files\Java\jdk1.6.0_##\demo
```

where `_##` represents the JDK update number. On UNIX/Linux, the default location is the directory in which you install the JDK followed by `jdk1.6.0_##/demo`—for example,

```
/usr/local/jdk1.6.0_##/demo
```

Other platforms use a similar directory (or folder) structure. You may need to update the locations specified here to reflect your chosen installation directory and disk drive, or a different version of the JDK. The demonstration programs are also available on JDK 7.

If you're using a Java development tool that does not come with the Java demos, you can download the current JDK from `www.oracle.com/technetwork/java/javase/downloads/index.html`. Mac OS X users should visit `developer.apple.com/java` for information about Java SE 6 on the Mac, or use virtualization software to run the Windows or Linux versions of Java in a virtual machine. Apple recently joined the OpenJDK project (`openjdk.java.net`). Eventually a Mac OS X version of the JDK for Java SE 7 will be available from this project's website.

Overview of the Demonstration Applets

Open a command window and use the `cd` command to change directories to the JDK's `demo` directory. The `demo` directory contains several subdirectories. You can list them by issuing the `dir` command on Windows or the `ls` command on UNIX/Linux/Max OS X. We discuss sample programs in the `applets` and `jfc` subdirectories. The `applets` directory contains demonstration applets. The `jfc` (Java Foundation Classes) directory contains applets and applications that demonstrate Java's powerful graphics and GUI capabilities.

Change to the `applets` directory and list its contents to see the directory names for the demonstration applets. Figure 23.1 provides a brief description of each. If your browser supports Java, you can test an applet by opening the HTML document for it in the applet's directory. We'll demonstrate three of these applets by using the `appletviewer` command in a command window.

Example	Description
`Animator`	Performs one of four separate animations.
`ArcTest`	Demonstrates drawing arcs. You can interact with the applet to change attributes of the arc that's displayed.
`BarChart`	Draws a simple bar chart.
`Blink`	Displays blinking text in different colors.
`CardTest`	Demonstrates several GUI components and layouts.

Fig. 23.1 | The examples from the `applets` directory. (Part 1 of 2.)

Example	Description
`Clock`	Draws a clock with rotating hands, the current date and the current time. The clock updates once per second.
`DitherTest`	Demonstrates drawing with a graphics technique known as *dithering* that allows gradual transformation from one color to another.
`DrawTest`	Allows the user to draw lines and points in different colors by dragging the mouse.
`Fractal`	Draws a fractal. Fractals typically require complex calculations to determine how they're displayed. We discuss fractals in Section 18.8.
`GraphicsTest`	Draws shapes to illustrate graphics capabilities.
`GraphLayout`	Draws a graph consisting of many nodes (represented as rectangles) connected by lines. Drag a node to see the other nodes in the graph adjust on the screen and demonstrate complex graphical interactions.
`JumpingBox`	Moves a rectangle randomly around the screen. Try to catch it by clicking it with the mouse!
`MoleculeViewer`	Presents a three-dimensional view of several chemical molecules. Drag the mouse to view the molecule from different angles.
`NervousText`	Draws text that jumps around the applet.
`SimpleGraph`	Draws a complex curve.
`SortDemo`	Compares three sorting techniques. Sorting (described in Chapter 19) arranges information in order—like alphabetizing words. When you execute this example with the `appletviewer`, three windows appear. When you execute it in a browser, the three demos appear side by side. Click in each demo to start the sort. The sorts all operate at different speeds.
`SpreadSheet`	Demonstrates a simple spreadsheet of rows and columns.
`TicTacToe`	Allows the user to play Tic-Tac-Toe against the computer.
`WireFrame`	Draws a three-dimensional shape as a wire frame. Drag the mouse to view the shape from different angles.

Fig. 23.1 | The examples from the `applets` directory. (Part 2 of 2.)

`TicTacToe` Applet

This `TicTacToe` demonstration applet allows you to play Tic-Tac-Toe against the computer. Change directories to subdirectory `TicTacToe`, where you'll find the file `example1.html` that loads the applet. In the command window, type the command

```
appletviewer example1.html
```

and press *Enter*. This executes the `appletviewer` applet container, which loads the HTML document `example1.html` specified as its command-line argument. The `appletviewer` determines from the document which applet to load and executes it. Figure 23.2 shows several screen captures of playing Tic-Tac-Toe with this applet. You can open the HTML document in your browser to execute the applet in the browser.

You are player **X**. To interact with the applet, point the mouse at the square where you want to place an **X** and click the mouse button. The applet plays a sound and places an **X**

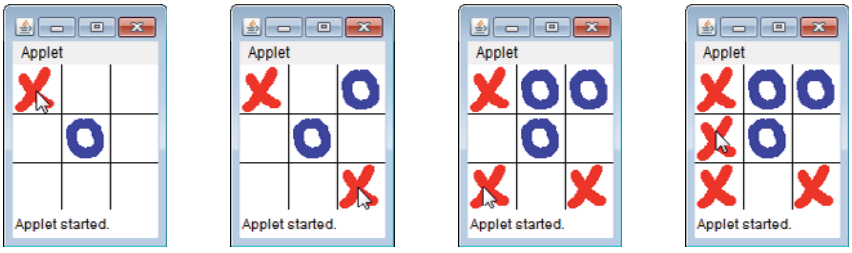


Fig. 23.2 | `TicTacToe` applet sample execution.

in the square if it's open. If the square is occupied, this is an invalid move, and the applet plays a different sound, indicating that you cannot make the specified move. After you make a valid move, the applet responds by making its own move.

To play again, click the `appletviewer`'s **Applet** menu and select the **Reload** menu item (Fig. 23.3), or click the applet again when the game is over. To terminate the `appletviewer`, click the `appletviewer`'s **Applet** menu and select the **Quit** menu item.

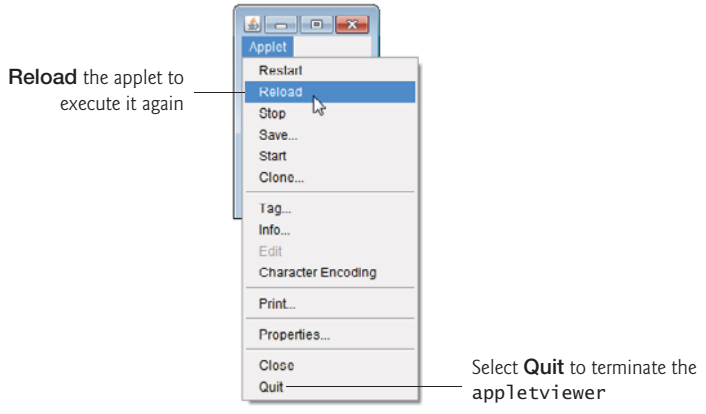


Fig. 23.3 | **Applet** menu in the `appletviewer`.

`DrawTest` Applet

The `DrawTest` applet allows you to draw lines and points in different colors. In the command window, change directories to directory `applets`, then to subdirectory `DrawTest`. You can move up the directory tree incrementally toward `demo` by issuing the command "`cd ..`" in the command window. The `DrawTest` directory contains the `example1.html` document that's used to execute the applet. In the command window, type the command

```
appletviewer example1.html
```

and press *Enter*. The `appletviewer` loads `example1.html`, determines from the document which applet to load and executes it. Figure 23.4 shows a screen capture after some lines and points have been drawn.

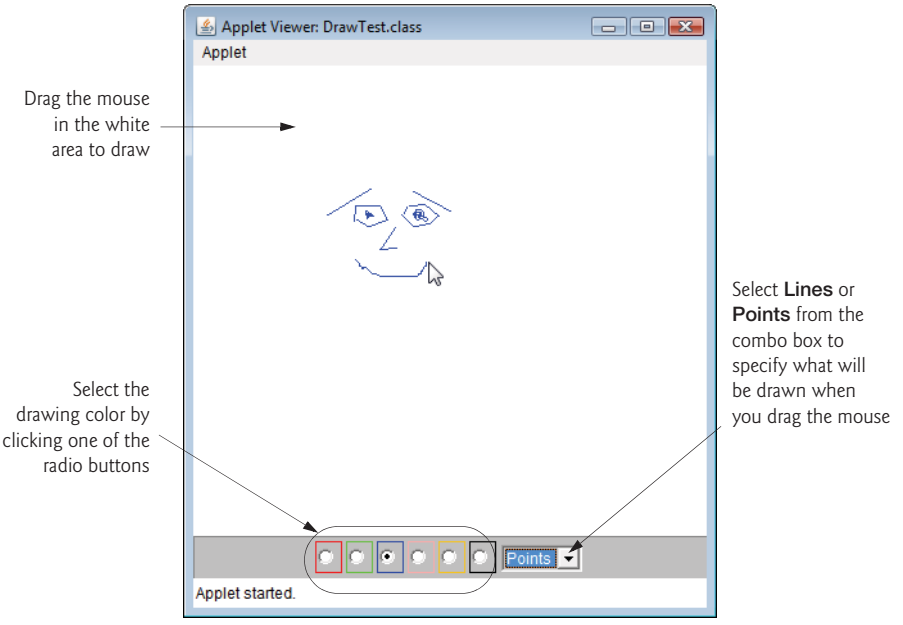


Fig. 23.4 | `DrawTest` applet sample execution.

By default the applet allows you to draw black lines by dragging the mouse across the applet. When you drag the mouse, the line's start point always remains in the same place and its endpoint follows the mouse pointer around the applet. The line is not permanent until you release the mouse button.

Select a color by clicking one of the radio buttons at the bottom of the applet. You can select from red, green, blue, pink, orange and black. Change the shape to draw from **Lines** to **Points** by selecting **Points** from the combo box. To start a new drawing, select **Reload** from the `appletviewer`'s **Applet** menu.

Java2D Applet

The `Java2D` applet demonstrates many features of the Java 2D API, which we introduced in Chapter 15. This demo can also be found at `java.sun.com/products/java-media/2D/samples/index.html`. Change directories to the `jfc` directory in the JDK's `demo` directory, then change to the `Java2D` directory. In the command window, type the command

```
appletviewer Java2Demo.html
```

and press *Enter*. The `appletviewer` loads `Java2Demo.html`, determines from the document which applet to load and executes it. Figure 23.5 shows a screen capture of one of this applet's many demonstrations of Java's two-dimensional graphics capabilities.

At the top of the applet are tabs that look like file folders in a filing cabinet. This demo provides 12 tabs, each demonstrating Java 2D API features. To change to a different part of the demo, simply click a different tab. Also, try changing the options in the upper-right corner of the applet. Some of these affect the speed with which the applet draws the graphics. For example, click the checkbox to the left of the word **Anti-Aliasing** to turn off

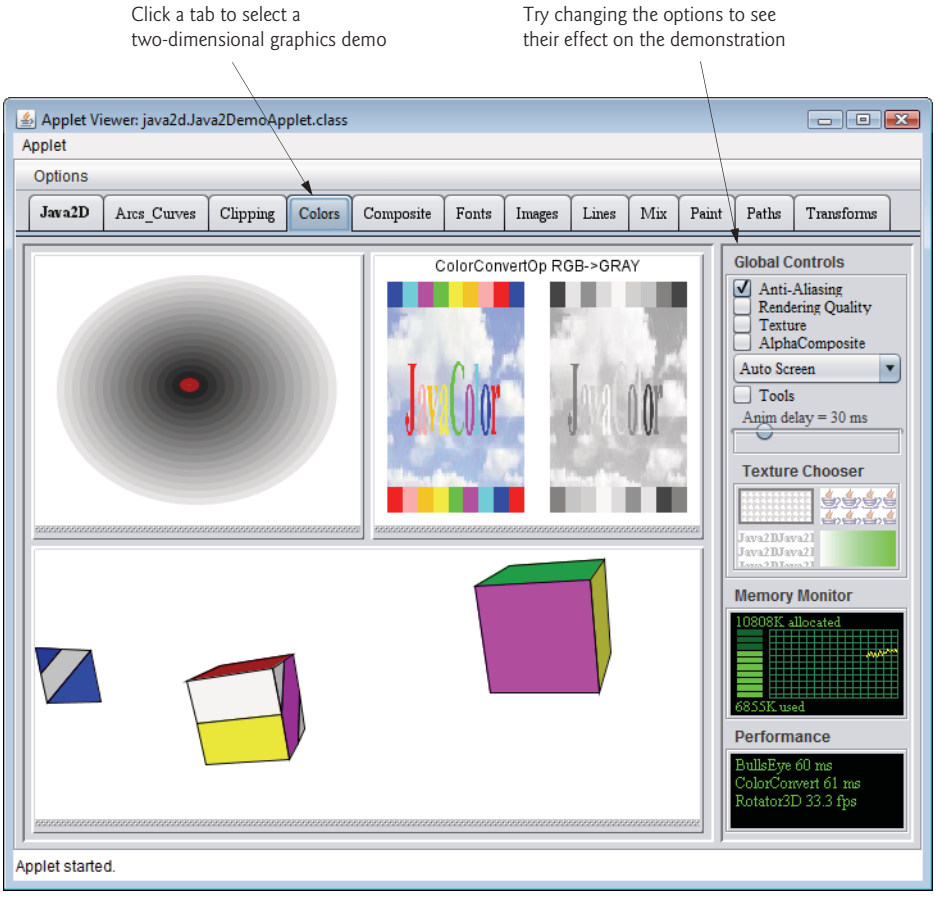


Fig. 23.5 | `Java2D` applet sample execution.

antialiasing (a graphics technique for producing smoother on-screen graphics in which the edges are blurred). Shapes that are not antialiased are less complex to draw. Accordingly, when the antialiasing feature is turned off, the animation speed increases for the animated shapes at the bottom of the demo (Fig. 23.5).

23.3 Simple Java Applet: Drawing a String

Every Java applet is a graphical user interface on which you can place GUI components using the techniques introduced in Chapter 14 or draw using the techniques demonstrated in Chapter 15. In this chapter, we'll demonstrate drawing on an applet. Examples in Chapters 24 and 27 demonstrate building an applet's graphical user interface.

Now let's build an applet of our own. We begin with a simple applet (Fig. 23.6) that draws `"Welcome to Java Programming!"` on the applet. We show this applet executing in two applet containers—the `appletviewer` and the Mozilla Firefox web browser. At the end of this section, you'll learn how to execute the applet in a web browser.

```
1  // Fig. 23.6: WelcomeApplet.java
2  // Applet that draws a String.
3  import java.awt.Graphics;   // program uses class Graphics
4  import javax.swing.JApplet; // program uses class JApplet
5
6  public class WelcomeApplet extends JApplet
7  {
8     // draw text on applet's background
9     public void paint( Graphics g )
10    {
11       // call superclass version of method paint
12       super.paint( g );
13
14       // draw a String at x-coordinate 25 and y-coordinate 25
15       g.drawString( "Welcome to Java Programming!", 25, 25 );
16    } // end method paint
17 } // end class WelcomeApplet
```

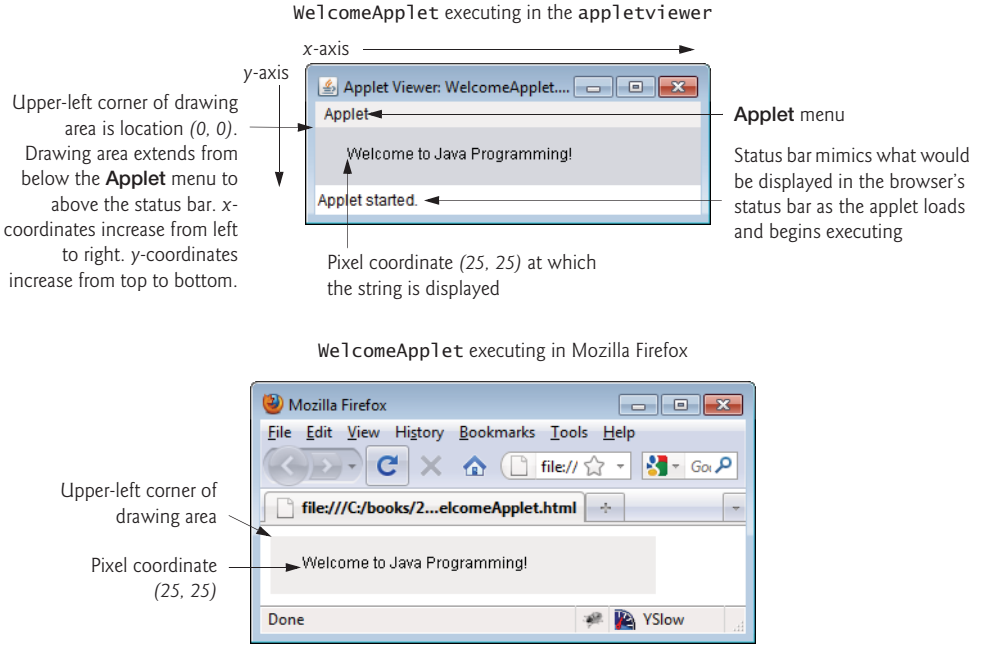


Fig. 23.6 | Applet that draws a `String`.

Creating the Applet Class

Line 3 imports class `Graphics` to enable the applet to draw graphics, such as lines, rectangles, ovals and strings of characters. Class **`JApplet`** (imported at line 4) from package `javax.swing` is used to create applets. As with applications, every Java applet contains at least one `public` class declaration. An applet container can create only objects of classes that are `public` and extend `JApplet` (or its superclass `Applet`). For this reason, class `WelcomeApplet` (lines 6–17) extends `JApplet`.

An applet container expects every Java applet to have methods named **`init`**, **`start`**, **`paint`**, **`stop`** and **`destroy`**, each of which is declared in class `JApplet`. Each new applet class you create inherits default implementations of these methods from class `JApplet`. These methods can be overridden (redefined) to perform tasks that are specific to your applet. Section 23.4 discusses each method in more detail.

When an applet container loads class `WelcomeApplet`, the container creates a `WelcomeApplet` object, then calls its methods `init`, `start` and `paint` in sequence. If you do not declare these methods in your applet, the applet container calls the inherited versions. The superclass methods `init` and `start` have empty bodies, so they do not perform any tasks. The superclass method `paint` does not draw anything on the applet.

You might wonder why it's necessary to inherit methods `init`, `start` and `paint` if their default implementations do not perform tasks. Some applets do not use all three of these methods. However, the applet container does not know that. Thus, it expects every applet to have these methods, so that it can provide a consistent start-up sequence. This is similar to applications' always starting execution with `main`. Inheriting the "default" versions of these methods guarantees that the applet container can execute each applet uniformly. Also, inheriting default implementations of these methods allows you to concentrate on defining only the methods required for a particular applet.

Overriding Method **`paint`** *for Drawing*

To enable our applet to draw, class `WelcomeApplet` overrides method `paint` (lines 9–16) by placing statements in the body of `paint` that draw a message on the screen. Method `paint` receives a parameter of type `Graphics` (called `g` by convention), which is used to draw graphics on the applet. You do not call method `paint` explicitly in an applet. Rather, the applet container calls `paint` to tell the applet when to draw, and the applet container is responsible for passing a `Graphics` object as an argument.

Line 12 calls the superclass version of method `paint` that was inherited from `JApplet`. This should be the first statement in every applet's `paint` method. Omitting it can cause subtle drawing errors in applets that combine drawing and GUI components.

Line 15 uses `Graphics` method `drawString` to draw `Welcome to Java Programming!` on the applet. The method receives as arguments the `String` to draw and the *x*-*y* coordinates at which the bottom-left corner of the `String` should appear in the drawing area. When line 15 executes, it draws the `String` on the applet at the coordinates `25` and `25`.

23.3.1 Executing `WelcomeApplet` in the `appletviewer`

After creating class `WelcomeApplet` and saving it in the file `WelcomeApplet.java`, open a command window, change to the directory in which you saved the applet class declaration and compile class `WelcomeApplet`.

Recall that applets are embedded in web pages for execution in an applet container (`appletviewer` or a browser). Before you can execute the applet, you must create an HTML document that specifies which applet to execute in the applet container. Typically, an HTML document ends with an **`.html`** or **`.htm`** file-name extension. Figure 23.7 shows a simple HTML document—`WelcomeApplet.html`—that loads the applet defined in Fig. 23.6 into an applet container.

Most HTML elements are delimited by pairs of **tags**—e.g., lines 1 and 6 delimit the HTML document's beginning and end, respectively. Each tag is enclosed in angle brackets

```
1  <html>
2     <body>
3        <applet code = "WelcomeApplet.class" width = "300" height = "45">
4        </applet>
5     </body>
6  </html>
```

Fig. 23.7 | `WelcomeApplet.html` loads `WelcomeApplet` (Fig. 23.6) into an applet container.

(< and >). Lines 2–5 specify the **body element** element of the document—this represents the elements that will be displayed in the web page. Lines 3–4 specify an **`applet` element** that tells the applet container to load a specific applet and defines the size of its display area (its width and height in pixels) in the applet container.

The applet and its corresponding HTML document are normally stored in the same directory on disk. Typically, a browser loads an HTML document from a computer (other than your own) connected to the Internet. However, HTML documents also can reside on your computer (as in Section 23.2). When an applet container encounters an `applet` element in an HTML document, it loads the applet's `.class` file (or files) from the same location that contains the HTML document.

The `applet` element has several **attributes**. The first attribute in line 3, `code = "WelcomeApplet.class"`, indicates that the file `WelcomeApplet.class` contains the compiled applet class. The second and third attributes in line 3 indicate the **`width`** (300) and the **`height`** (45) of the applet in pixels. The `</applet>` tag (line 4) terminates the `applet` element that began at line 2. The `</html>` tag (line 6) terminates the HTML document.



Common Programming Error 23.1

Forgetting the ending `</applet>` tag prevents the applet from executing in some applet containers. The `appletviewer` terminates without indicating an error. Some web browsers simply ignore the incomplete `applet` element.



Error-Prevention Tip 23.1

If you receive a `MissingResourceException` error message when loading an applet into the `appletviewer` or a browser, check the `<applet>` tag in the HTML document carefully for syntax errors, such as commas (`,`) between the attributes.

The `appletviewer` understands only the `<applet>` and `</applet>` HTML tags and ignores all other tags in the document. The `appletviewer` is an ideal place to test an applet and ensure that it executes properly. Once the applet's execution is verified, you can add its HTML tags to a web page that others can view in their web browsers.

To execute `WelcomeApplet` in the `appletviewer`, open a command window, change to the directory containing your applet and HTML document, then type

```
appletviewer WelcomeApplet.html
```



Error-Prevention Tip 23.2

Test your applets in the `appletviewer` before executing them in a web browser. Browsers often save a copy of an applet in memory until all the browser's windows are closed. If you change an applet, recompile it, then reload it in your browser, the browser may still execute the original version of the applet.



Error-Prevention Tip 23.3

Test your applets in every web browser in which they'll execute to ensure that they operate correctly.

23.3.2 Executing an Applet in a Web Browser

The sample program executions in Fig. 23.6 demonstrate `WelcomeApplet` executing in the `appletviewer` and in the Mozilla Firefox web browser. To execute an applet in Firefox, perform the following steps:

1. Select **Open File...** from the **File** menu.
2. In the dialog box that appears, locate the directory containing the HTML document for the applet you wish to execute.
3. Select the HTML document.
4. Click the **Open** button.

The steps for executing applets in other web browsers are similar. In most browsers, you can simply type *<Ctrl> O* to open a dialog that enables you to select an HTML document from your local computer.



Error-Prevention Tip 23.4

If your applet executes in the `appletviewer` but not in your web browser, Java may not be installed and configured for your browser. In this case, visit the website `java.com` and click the **Free Java Download** *button to install Java for your browser.*

23.4 Applet Life-Cycle Methods

Now that you've created an applet, let's consider the five applet methods that are called by the applet container from the time the applet is loaded into the browser to the time it's terminated by the browser. These methods correspond to various aspects of an applet's life cycle. Figure 23.8 lists these methods, which are inherited into your applet classes from class `JApplet`. The table specifies when each method gets called and explains its purpose. Other than method `paint`, these methods have empty bodies by default. If you'd like to declare any of them in your applets and have the applet container call them, you must use the method headers shown in Fig. 23.8.

Method	Description
`public void init()`	Called *once* by the applet container when an applet is loaded for execution. This method initializes an applet. Typical actions performed here are initializing fields, creating GUI components, loading sounds to play, loading images to display (see Chapter 24) and creating threads (see Chapter 26).

Fig. 23.8 | `JApplet` life-cycle methods that are called by an applet container during an applet's execution. (Part 1 of 2.)

Method	Description
`public void start()`	Called by the applet container after method `init` completes execution. In addition, if the user browses to another website and later returns to the applet's HTML page, method `start` is called again. The method performs any tasks that must be completed when the applet is loaded for the first time and that must be performed every time the applet's HTML page is revisited. Actions performed here might include starting an animation or starting other threads of execution.
`public void paint( Graphics g )`	Called by the applet container after methods `init` and `start`. Method `paint` is also called when the applet needs to be repainted. For example, if the user covers the applet with another open window on the screen and later uncovers it, the `paint` method is called. Typical actions performed here involve drawing with the `Graphics` object `g` that's passed to the `paint` method by the applet container.
`public void stop()`	This method is called by the applet container when the user leaves the applet's web page by browsing to another web page. Since it's possible that the user might return to the web page containing the applet, method `stop` performs tasks that might be required to suspend the applet's execution, so that the applet does not use computer processing time when it's not displayed on the screen. Typical actions performed here would stop the execution of animations and threads.
`public void destroy()`	This method is called by the applet container when the applet is being removed from memory. This occurs when the user exits the browsing session by closing all the browser windows and may also occur at the browser's discretion when the user has browsed to other web pages. The method performs any tasks that are required to clean up resources allocated to the applet.

Fig. 23.8 | `JApplet` life-cycle methods that are called by an applet container during an applet's execution. (Part 2 of 2.)



Common Programming Error 23.2

Declaring methods `init`, `start`, `paint`, `stop` or `destroy` with method headers that differ from those shown in Fig. 23.8 results in methods that will not be called by the applet container. The code specified in your versions of the methods will not execute. The `@Override` annotation can be applied to each method to prevent this problem.

23.5 Initialization with Method `init`

Our next applet (Fig. 23.9) computes the sum of two values entered into input dialogs by the user and displays the result by drawing a `String` inside a rectangle on the applet. The sum is stored in an instance variable of the `AdditionApplet` class, so it can be used in both the `init` method and the `paint` method. The HTML document that you can use to load this applet into an applet container (i.e., the `appletviewer` or a web browser) is shown in Fig. 23.10.

```
1  // Fig. 23.9: AdditionApplet.java
2  // Applet that adds two double values entered via input dialogs.
3  import java.awt.Graphics; // program uses class Graphics
4  import javax.swing.JApplet; // program uses class JApplet
5  import javax.swing.JOptionPane; // program uses class JOptionPane
6
7  public class AdditionApplet extends JApplet
8  {
9     private double sum; // sum of values entered by user
10
11    // initialize applet by obtaining values from user
12    public void init()
13    {
14       // obtain first number from user
15       String firstNumber = JOptionPane.showInputDialog(
16          "Enter first floating-point value" );
17
18       // obtain second number from user
19       String secondNumber = JOptionPane.showInputDialog(
20          "Enter second floating-point value" );
21
22       // convert numbers from type String to type double
23       double number1 = Double.parseDouble( firstNumber );
24       double number2 = Double.parseDouble( secondNumber );
25
26       sum = number1 + number2; // add numbers
27    } // end method init
28
29    // draw results in a rectangle on applet's background
30    public void paint( Graphics g )
31    {
32       super.paint( g ); // call superclass version of method paint
33
34       // draw rectangle starting from (15, 10) that is 270
35       // pixels wide and 20 pixels tall
36       g.drawRect( 15, 10, 270, 20 );
37
38       // draw results as a String at (25, 25)
39       g.drawString( "The sum is " + sum, 25, 25 );
40    } // end method paint
41 } // end class AdditionApplet
```

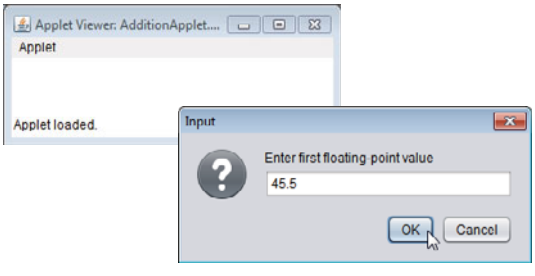


Fig. 23.9 | Applet that adds two `double` values entered via input dialogs. (Part 1 of 2.)

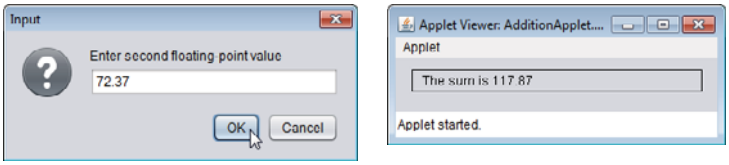


Fig. 23.9 | Applet that adds two `double` values entered via input dialogs. (Part 2 of 2.)

```
1  <html>
2  <body>
3  <applet code = "AdditionApplet.class" width = "300" height = "50">
4  </applet>
5  </body>
6  </html>
```

Fig. 23.10 | `AdditionApplet.html` loads class `AdditionApplet` of Fig. 23.9 into an applet container.

The applet requests that the user enter two floating-point numbers. In Fig. 23.9, line 9 declares instance variable `sum` of type `double`. The applet contains two methods—`init` (lines 12–27) and `paint` (lines 30–40). When an applet container loads this applet, the container creates an instance of class `AdditionApplet` and calls its `init` method—this occurs only *once* during an applet's execution. Method `init` normally initializes the applet's fields (if they need to be initialized to values other than their defaults) and performs other tasks that should occur only once when the applet begins execution. The first line of `init` always appears as shown in line 12, which indicates that `init` is a `public` method that receives no arguments and returns no information when it completes.

Lines 15–24 declare variables to store the values entered by the user, obtain the user input and convert the `String`s entered by the user to `double` values. Line 26 adds the values stored in variables `number1` and `number2`, and assigns the result to instance variable `sum`. At this point, the applet's `init` method returns program control to the applet container, which then calls the applet's `start` method. We did not declare `start` in this applet, so the one inherited from class `JApplet` is called here.

Next, the applet container calls the applet's `paint` method, which draws a rectangle (line 36) where the addition result will appear. Line 39 calls the `Graphics` object's `drawString` method to display the results. The statement concatenates the value of instance variable `sum` to the `String` `"The sum is "` and displays the concatenated `String`.



Software Engineering Observation 23.1

The only statements that should be placed in an applet's `init` method are those that should execute only once when the applet is initialized.

23.6 Sandbox Security Model

For security reasons, it's generally considered dangerous to allow applets or any other program that you execute from a web browser to access your local computer. So, you must

decide whether *you trust the source*. For example, if you choose to download a new version of the Firefox web browser from Mozilla's `firefox.com` website, you must decide whether you trust Mozilla. After all, their installer program is going to modify your system and place the files to execute Firefox on your computer. Once it's installed, Firefox will be able to access files and other local resources. Most of what you do with your web browsers—such as shopping, browsing the web and downloading software—requires you to trust the sites you visit and to trust the organizations that maintain those sites. If you're not careful, a malicious downloaded program could gain control of your computer, access personal information stored there, corrupt your data and possibly even be used to attack other computers on the Internet—as so often happens with computer viruses today.

Preventing Malicious Applets

Applets are typically downloaded from the Internet. What would happen if you downloaded a malicious applet? Consider the fact that a browser downloads and executes a Java applet automatically—the user *is not asked for approval*. In fact, an applet typically downloads *without the user's knowledge*—it's just another element of the web page the user happens to be visiting.

The designers of Java considered this issue thoroughly, since Java was intended for use in networked environments. To combat malicious code, the Java platform uses a so-called **sandbox security model** that provides a mechanism for executing downloaded code safely. Such code executes in the "sandbox" and is not allowed to "play outside the sandbox." *By default, downloaded code cannot access local system resources, and an applet can interact only with the server from which the applet was downloaded.*

Digitally Signed Applets

Unfortunately, executing in a sandbox makes it difficult for applets to perform useful tasks. It's possible, however, for an organization that wishes to create applets with access to the local system to obtain a security certificate (also called a digital certificate) from one of several certificate authorities (see `en.wikipedia.org/wiki/Certificate_Authority` for a list of authorities and more information about certificate authorities). The organization can then use tools provided with the JDK to *digitally sign* an applet that requires access to local system resources. When a user downloads a digitally signed applet, a dialog prompts the user asking whether he or she trusts the applet's source. In that dialog, the user can view the organization's security certificate and see which certificate authority issued it. If the user indicates that he/she trusts the source, only then will the applet be able to access to the local computer's resources.

In the next section, we introduce Java Web Start and the Java Network Launch Protocol (JNLP). These technologies enable applets or applications to interact with the user to request access to specific local system resources. With the user's permission, this enables Java programmers to *extend the sandbox*, but it does *not* give their programs access to all of the user's local resources—so the sandbox principles are still in effect. For example, it would be useful for a downloadable text editor program to store the user's files in a folder on the user's computer. The text editor can prompt the user to ask for permission to do this. If the user grants permission for a specific directory on disk, the program can then access *only* that local directory and its subdirectories.

For more information on digitally signed applets, visit `java.sun.com/developer/onlineTraining/Programming/JDCBook/signed.html`. For information on the Java Plat-

form security model, visit `download.oracle.com/javase/6/docs/technotes/guides/security/`.

23.7 Java Web Start and the Java Network Launch Protocol (JNLP)

Java Web Start is a framework for running downloaded applets and applications outside the browser. Typically, such programs are stored on a web server for access via the Internet, but they can also be stored on an organization's network for internal distribution, or even on CDs, DVDs or other media. As you'll learn in Chapter 24, Java Web Start enables you to ask the user if a downloaded program can have access to the resources of the user's computer.

Java Web Start Features

Some key Java Web Start features include:

- ***Desktop integration:*** Users can launch robust applets and applications by clicking a hyperlink in a web page, and can quickly and easily install the programs on their computers. *Java Web Start can be configured to ask the user if a desktop icon should be created so the user can launch the program directly from the desktop.* Downloaded programs can also have an "offline mode" for execution when the computer is not connected to the Internet.
- ***Automatic updating:*** When you execute a program via Java Web Start, the program is downloaded and cached (stored) on the user's computer. The next time the user executes that program, Java Web Start launches it from the cache. If the program has been updated since it was last launched, Java Web Start can automatically download the updates, so a user always has the most up-to-date version. This makes installing and updating software simple and seamless to the user.
- ***Draggable applets:*** With a small change to the `applet` element that invokes an applet from an HTML document, you can allow users to execute an applet in its own window by holding the *Alt* key and dragging the applet out of the web browser. The applet continues to execute even after the web browser closes.

Java Network Launch Protocol (JNLP)

A **Java Network Launch Protocol (JNLP)** document provides the information that Java Web Start needs in order to download and run a program. Also, you must package your program in one or more *Java archive (JAR) files* that contain the program's code and resources (e.g., images, media files, text files).

By default, programs launched via Java Web Start execute using the sandbox security model. If the user gives permission, such programs *can* access the local file system, the clipboard and other services via the JNLP APIs of package `javax.jnlp`. We discuss some of these features in Chapter 24. *Digitally signed programs can gain greater access to the local system if the user trusts the source.*

23.7.1 Packaging the `DrawTest` Applet for Use with Java Web Start

Let's package the JDK's `DrawTest` demonstration applet (discussed in Section 23.2) so that you can execute it via Java Web Start. To do so, you must first wrap the applet's `.class` files and the resources it uses (if any) into a Java archive (JAR) file. In a command

window, change to the `DrawTest` directory, as you did in Section 23.2. Once in that folder, execute the following command:

```
jar cvf DrawTest.jar *.class
```

which creates in the current directory a JAR file named `DrawTest.jar` containing the applet's `.class` files—`DrawControls.class`, `DrawPanel.class` and `DrawTest.class`. If the program had other resources, you'd simply add the file names or the folder names in which those resources are stored to the end of the preceding command. The letters `cvf` are command-line options to the **`jar` command**. The **`c` option** indicates that the command should create a new JAR file. The **`v` option** indicates that the command should produce verbose output so you can see the list of files and directories being included in the JAR file. The **`f` option** indicates that the next argument in the command line (`DrawTest.jar`) is the new JAR file's name. Figure 23.11 shows the preceding command's verbose output, which shows the files that were placed into the JAR.

```
added manifest
adding: DrawControls.class(in = 2611) (out= 1488)(deflated 43%)
adding: DrawPanel.class(in = 2703) (out= 1406)(deflated 47%)
adding: DrawTest.class(in = 1170) (out= 706)(deflated 39%)
```

Fig. 23.11 | Output of the `jar` command.

23.7.2 JNLP Document for the `DrawTest` Applet

Next, you must create a JNLP document that describes the contents of the JAR file and specifies which file in the JAR is the so-called **`main-class`** that begins the program's execution. For an applet, the `main-class` is the one that extends `JApplet` (i.e., `DrawTest` in this example). For an application, the `main-class` is the one that contains the `main` method. A basic JNLP document for the `DrawTest` applet is shown in Fig. 23.12. We describe this document's elements momentarily.

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <jnlp
3     codebase=PathToJNLPFile
4     href="DrawTest.jnlp">
5
6     <information>
7        <title>DrawTest Applet</title>
8        <vendor>Oracle Corporation</vendor>
9        <shortcut>
10          <desktop/>
11       </shortcut>
12       <offline-allowed/>
13    </information>
14
15    <resources>
16       <java version="1.6+"/>
17       <jar href="DrawTest.jar" main="true"/>
18    </resources>
```

Fig. 23.12 | `DrawTest.jnlp` document for launching the `DrawTest` applet. (Part 1 of 2.)

```
19
20      <applet-desc
21         name="DrawTest"
22         main-class="DrawTest"
23         width="400"
24         height="400">
25      </applet-desc>
26   </jnlp>
```

Fig. 23.12 | `DrawTest.jnlp` document for launching the `DrawTest` applet. (Part 2 of 2.)

Overview of XML

JNLP documents are written in **Extensible Markup Language (XML)**—a widely supported standard for describing data. XML is commonly used to exchange data between applications over the Internet, and many applications now use XML to specify configuration information as well—as is the case with JNLP documents for Java Web Start. XML permits you to create markup for virtually any type of information. This enables you to create entirely new markup languages for describing any type of data, such as mathematical formulas, software-configuration instructions, chemical molecular structures, music, news, recipes and financial reports. XML describes data in a way that both humans and computers can understand. JNLP is a so-called XML vocabulary that describes the information Java Web Start needs to launch a program.

XML documents contain **elements** that specify the document's structure, such as `title` (line 7), and text that represents content (i.e., data), such as `DrawTest Applet` (line 7). XML documents delimit elements with **start tags** and **end tags**. A start tag consists of the element name in **angle brackets** (e.g., `<title>` and `<vendor>` in lines 7 and 8). Start tags may also contain attributes of the form *name*=*value*—for example, the `jnlp` start tag contains the attribute `href="DrawTest.jnlp"`. An end tag consists of the element name preceded by a **forward slash** (`/`) in angle brackets (e.g., `</title>` and `</vendor>` in lines 7 and 8). An element's start and end tags enclose text that represents a piece of data (e.g., the `vendor` of the program—`Oracle Corporation`—in line 8, which is enclosed by the `<vendor>` start tag and `</vendor>` end tag) or other elements (e.g., the `title`, `vendor`, `shortcut` and `offline-allowed` elements in the `information` element of lines 6–13). Every XML document must have exactly one **root element** that contains all the other elements. In Fig. 23.12, the **`jnlp` element** (lines 2–26) is the root element.

JNLP Document: `jnlp` Element

The `jnlp` element's start tag (lines 2–4) has two attributes—`codebase` and `href`. The `codebase` attribute's value is a URL that specifies the path where the JNLP document and the JAR file are stored—this is specified in Fig. 23.12 as *PathToJNLPFile*, since this value depends on the location from which the applet is loaded. The `href` attribute specifies the JNLP file that launches the program. We saved the JNLP file and the JAR file in the `DrawTest` demonstration applet's directory within the JDK's directory structure. We used the following local file system URL as the `codebase`:

```
file:.
```

which indicates that the code is in the current directory (`.`). Typically, the `codebase` references a directory on a web server with an `http://` URL. If you'd like to serve your applet

or application from a web server so users can access it online, you'll need to configure your web server correctly, as described at `java.sun.com/javase/6/docs/technotes/guides/javaws/developersguide/setup.html`.

JNLP Document: `information` *Element*

The **`information` element** (lines 6–13) provides details about the program. The **`title` element** specifies a title for the program. The **`vendor` element** specifies who created the program. The values of these elements appear in Java Web Start's security warnings and errors that are presented to the user. The `title`'s value also appears in the title bar of the window in which the program executes.

The **`desktop` element** that's nested in the **`shortcut` element** (lines 9–11) tells Java Web Start to ask whether the user wishes to install a desktop shortcut. If the user accepts, an icon will appear on the desktop. The user can then launch the program in its own window by double-clicking the desktop icon. Note the syntax of the `<desktop/>` element—a so-called empty XML element. When nothing appears between an element's start and end tags, the element can be written using one tag that ends with `/>`.

The **`offline-allowed` element** (line 12) indicates that once the program is installed on the user's computer, it can be launched via Java Web Start—even when the computer is not connected to the Internet. This is particularly useful for any program that can be used with files stored on the user's computer.

JNLP Document: `resources` *Element*

The **`resources` element** (lines 15–18) contains two nested elements. The **`java` element** lists the minimum version of Java required to execute the program (line 16) and the **`jar` element** (line 17) specifies the location of the JAR file that contains the program and whether that JAR file contains the class that launches the program. There can be multiple `jar` elements, as you'll see in the next chapter.

JNLP Document: `applet-desc` *Element*

The **`applet-desc` element** (lines 20–25) is similar to the `applet` element in HTML. The `name` attribute specifies the applet's name. The `main-class` attribute specifies the main applet class (the one that extends `JApplet`). The `width` and `height` attributes specify the width and height in pixels, respectively, of the window in which the applet will execute. Chapter 24 discusses a similar element for applications—`application-desc`.

Launching the Applet with Java Web Start

You're now ready to launch the applet via Java Web Start. There are several ways to do this. You can use the **`javaws` command** in a command window from the folder that contains the JNLP document, as in

```
javaws DrawTest.jnlp
```

You can also use your operating system's file manager to locate the JNLP on your computer and double click its file name. Normally, the JNLP file is referenced from a web page via a hyperlink. The `DrawTestWebPage.html` document in Fig. 23.13 (which was saved in the same directory as the JNLP file) contains an anchor (`a`) element (line 4), which links to the `DrawTest.jnlp` file. Clicking this hyperlink in the web page downloads the JNLP file (in this case, it's loaded from the local file system) and executes the corresponding applet.

```
1  <html>
2     <head><title>DrawTest Launcher Page</title></head>
3     <body>
4        <a href="DrawTest.jnlp">Launch DrawTest via Java Web Start</a>
5     </body>
6  </html>
```

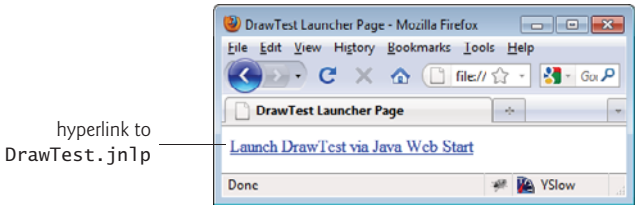


Fig. 23.13 | HTML document that launches the `DrawTest` applet when the user clicks the link.

When you run the applet via Java Web Start the first time, you'll be presented with the dialog in Fig. 23.14. This dialog enables the user to decide if a desktop icon will be installed. If the user clicks **OK**, a new icon labeled with the title specified in the JNLP document appears on the user's desktop. The applet is also cached for future use. After the user clicks **OK** or **Skip** in this dialog, the program executes (Fig. 23.15).

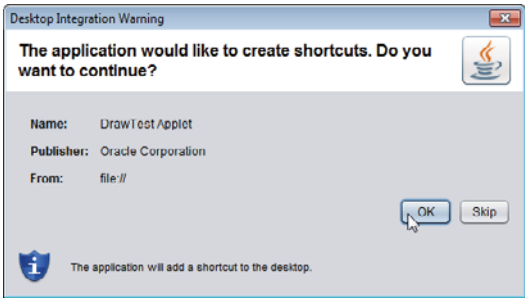


Fig. 23.14 | Dialog asking whether the user wishes to install a desktop shortcut.

Viewing the Installed Java Web Start Programs

You can view the installed Java Web Start programs in the **Java Cache Viewer** by typing the following command in a command window:

```
javaws -viewer
```

This displays the window in Fig. 23.16. The **Java Cache Viewer** enables you to manage the Java Web Start programs on your system. You can run a selected program, create a desktop shortcut for a program (if there isn't one already), delete installed programs, and more.

For more information on Java Web Start, visit `download.oracle.com/javase/6/docs/technotes/guides/javaws/`. This site provides an overview of Java Web Start and includes links to the Developer's Guide, an FAQ, the JNLP Specification and the API documentation for the `javax.jnlp` package.

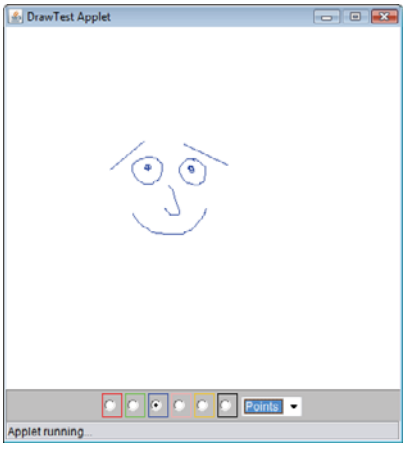


Fig. 23.15 | `DrawTest` applet running with Java Web Start.

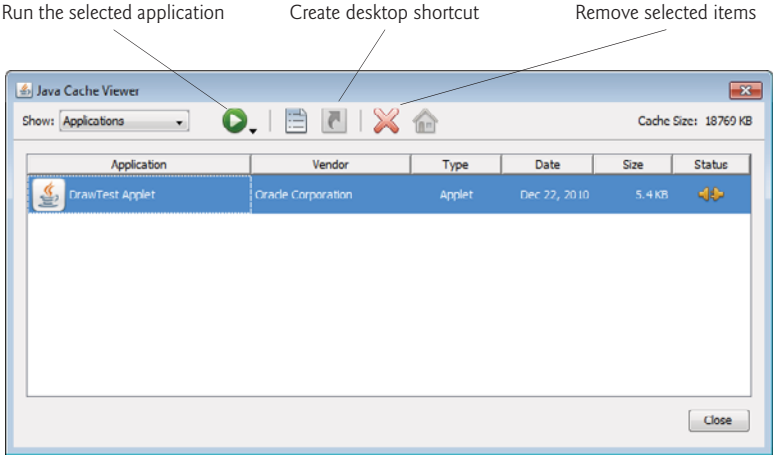


Fig. 23.16 | Viewing installed Java Web Start programs in the **Java Cache Viewer**.

23.8 Wrap-Up

In this chapter, you learned the fundamentals of Java applets and Java Web Start. You leaned HTML concepts for embedding an applet in a web page and executing it in an applet container such as the `appletviewer` or a web browser. You learned the five methods that are called automatically by the applet container during an applet's life cycle. We discussed Java's sandbox security model for executing downloaded code. Then we introduced Java Web Start and the Java Network Launch Protocol (JNLP). You packaged a program into a JAR file so that it could be executed via Java Web Start. We also discussed the basic elements of a JNLP document. Next, you'll see additional applets as we present basic multimedia capabilities. You'll also learn more features of Java Web Start and JNLP.

Summary

Section 23.1 Introduction

- Applets (p. 942) are Java programs that are typically embedded in HTML (Extensible HyperText Markup Language) documents—also called web pages. When a Java-enabled web browser loads a web page containing an applet, the applet downloads into the browser and executes.
- The application in which an applet executes is known as the applet container (p. 942). It's the applet container's responsibility to load the applet's class(es), create an instance of the applet and manage its life cycle. The JDK includes an applet container called the `appletviewer` (p. 942) for testing applets as you develop them and before you embed them in web pages.
- Web browsers execute Java applets via the Java Plug-In (p. 942).

Section 23.2 Sample Applets Provided with the JDK

- To re-execute an applet in the `appletviewer`, click the `appletviewer`'s **Applet** menu and select the **Reload** menu item.
- To terminate the `appletviewer`, select **Quit** from the `appletviewer`'s **Applet** menu.

Section 23.3 Simple Java Applet: Drawing a String

- Every Java applet is a graphical user interface on which you can place GUI components or draw.
- Class `JApplet` (p. 948) from package `javax.swing` is used to create applets.
- An applet container can create only objects of classes that are `public` and extend `JApplet` (or the `Applet` class from early versions of Java).
- An applet container expects every Java applet to have methods named `init`, `start`, `paint`, `stop` and `destroy` (p. 949), each of which is declared in class `JApplet`. Each new applet class you create inherits default implementations of these methods from class `JApplet`.
- When an applet container loads an applet, it creates an object of the applet's type, then calls the applet's `init`, `start` and `paint` methods.
- To enable an applet to draw, override its `paint` method (p. 949). The applet container calls `paint` to tell the applet when to draw.
- The first statement in method `paint` should be a call to the superclass method `paint`.
- Before you can execute an applet, you must create an HTML (Extensible HyperText Markup Language) document that specifies which applet to execute in the applet container. Typically, an HTML document ends with an "`.html`" or "`.htm`" file-name extension.
- An `applet` element tells the applet container to load a specific applet and defines the size of its display area (its width and height in pixels) in the applet container.
- Normally, an applet and its corresponding HTML document are stored in the same directory.
- When an applet container encounters an HTML document that contains an applet, the container automatically loads the applet's `.class` file(s) from the same location as the HTML document.
- The `appletviewer` understands only the `<applet>` and `</applet>` HTML tags (p. 950) and ignores all other tags in the document.

Section 23.4 Applet Life-Cycle Methods

- Method `init` (p. 949) is called once by the applet container to initialize an applet when it's loaded.
- Method `start` (p. 949) is called by the applet container after method `init` completes execution. In addition, if the user browses to another website and later returns to the applet's HTML page, method `start` is called again.
- Method `paint` is called by the applet container after methods `init` and `start`.

- Method `stop` (p. 949) is called by the applet container when the user leaves the applet's web page by browsing to another web page.
- Method `destroy` (p. 949) is called by the applet container when the applet is being removed from memory. This occurs when the user exits the browsing session by closing all the browser windows and may also occur at the browser's discretion when the user has browsed to other web pages.

Section 23.5 Initialization with Method `init`

- `Graphics` method `drawString` draws a `String` at a specified location.
- `Graphics` method `drawRect` draws a rectangle at the specified upper-left corner, width and height.

Section 23.6 Sandbox Security Model

- A browser downloads an applet *without the user's knowledge*—it's just another element of the web page the user happens to be visiting.
- To combat malicious code, the Java platform uses a sandbox security model (p. 955) that provides a mechanism for executing downloaded code safely. Downloaded code cannot access local system resources, and an applet can interact only with the server from which it was downloaded.
- You can "digitally sign" an applet that requires access to local system resources. If the user indicates that he/she trusts the applet's source, only then will the applet be able to access the local computer's resources.

Section 23.7 Java Web Start and the Java Network Launch Protocol (JNLP)

- Java Web Start (p. 956) is a framework for running downloaded programs outside the browser.
- Users can launch robust applets and applications by clicking a hyperlink in a web page, and can quickly and easily install the programs on their computers.
- Java Web Start can be configured to ask the user if a desktop icon should be created so the user can launch the program directly from the desktop. Downloaded programs can also have an "offline mode" for execution when the computer is not connected to the Internet.
- When you execute a program via Java Web Start, the program is downloaded and cached (stored) on the user's computer. The next time the user executes that program, Java Web Start launches it from the cache.
- If the program has been updated since it was last lauched, Java Web Start can automatically download the new version, so a user always has the most up-to-date version.
- A Java Network Launch Protocol (JNLP; p. 956) document provides the information that Java Web Start needs to download and run a program.
- Programs launched via Java Web Start execute using the sandbox security model. The user can permit access to the local file system, the clipboard and other services via the JNLP APIs.

Section 23.7.1 Packaging the `DrawTest` Applet for Use with Java Web Start

- The `jar` command (p. 957) is used to create JAR files. Option `c` indicates that the command should create a new JAR file (p. 957). Option `v` indicates that the command should produce verbose output (p. 957). Option `f` indicates that the next argument in the command line is the new JAR file's name (p. 957).

Section 23.7.2 JNLP Document for the `DrawTest` Applet

- A JNLP document describes the contents of the JAR file and specifies which file in the JAR is the so-called `main-class` (p. 957) that begins the program's execution.
- JNLP documents are written in Extensible Markup Language (XML)—a widely supported standard for describing data (p. 957).

- JNLP is a so-called XML vocabulary that describes the information Java Web Start needs to launch a program.
- The `jnlp` element's (p. 958) `codebase` attribute specifies the path where the JNLP document and the JAR file are stored. The `href` attribute specifies the JNLP file that launches the program.
- Typically, the `codebase` references a directory on a web server with an `http://` URL.
- The `information` element (p. 959) provides details about the program.
- The `title` element (p. 959) specifies a title for the program.
- The `vendor` element (p. 959) specifies who created the program.
- The `desktop` element nested in the `shortcut` element (p. 959) tells Java Web Start to ask users whether they wish to install a desktop shortcut.
- The `offline-allowed` (p. 959) element indicates that a program can be launched via Java Web Start even when the computer is not connected to the Internet.
- The `resources` element (p. 959) contains a `java` element (p. 959) that lists the minimum version of Java required to execute the program and a `jar` element (p. 959) that specifies the location of the JAR file.
- The `applet-desc` (p. 959) element's `name` attribute specifies the applet's name. The `main-class` attribute specifies the main applet class. The `width` and `height` (p. 950) attributes specify the width and height in pixels, respectively, of the window in which the applet will execute.
- To launch the applet via Java Web Start you can use the `javaws` command (p. 959). You can also use your operating system's file manager to locate the JNLP on your computer and double click its file name. Normally, a JNLP file is referenced from a web page via a hyperlink.
- When you run an applet via Java Web Start the first time and the JNLP document specifies that a desktop icon should be installed, you'll be presented with a dialog that enables you to decide whether to install the desktop icon. If you click **OK**, a new icon labeled with the title specified in the JNLP document appears on the desktop.
- You can view the installed Java Web Start programs in the **Java Cache Viewer** by typing the command `javaws -viewer` in a command window.
- The **Java Cache Viewer** enables you to manage the installed Java Web Start programs. You can run a selected program, create a desktop shortcut, delete installed programs, and more.

Self-Review Exercise

23.1 Fill in the blanks in each of the following:

a) Java applets begin execution with a series of three method calls: _____, _____ and _____.

b) The _____ method is invoked for an applet each time a browser's user leaves an HTML page on which the applet resides.

c) Every applet should extend class _____.

d) The _____ or a browser can be used to execute a Java applet.

e) The _____ method is called each time the user of a browser revisits the HTML page on which an applet resides.

f) To load an applet into a browser, you must first define a(n) _____ file.

g) Method _____ is called once when an applet begins execution.

h) Method _____ is invoked to draw on an applet.

i) Method _____ is invoked for an applet when the browser removes it from memory.

j) The _____ and _____ HTML tags specify that an applet should be loaded into an applet container and executed.

k) _____ is a framework for running downloaded programs outside the browser.
l) A(n) _____ document provides the information that Java Web Start needs to download and run a program.
m) The _____ enables you to manage the Java Web Start programs on your system.

Answers to Self-Review Exercise

23.1 a) `init`, `start`, `paint`. b) `stop`. c) `JApplet` (or `Applet`). d) `appletviewer`. e) `start`. f) HTML. g) `init`. h) `paint`. i) `destroy`. j) `<applet>`, `</applet>`. k) Java Web Start. l) Java Network Launch Protocol (JNLP). m) Java Cache Viewer.

Exercises

23.2 *(Arithmetic)* Write an applet that asks the user to enter two floating-point numbers, obtains the two numbers from the user and draws their sum, product (multiplication), difference and quotient (division). Use the techniques shown in Fig. 23.9.

23.3 *(Comparing Numbers)* Write an applet that asks the user to enter two floating-point numbers, obtains the numbers from the user and displays the two numbers, then displays the larger number followed by the words `"is larger"` as a string on the applet. If the numbers are equal, the applet should print the message `"These numbers are equal."` Use the techniques shown in Fig. 23.9.

23.4 *(Arithmetic and Numeric Comparisons)* Write an applet that inputs three floating-point numbers from the user and displays the sum, average, product, smallest and largest of these numbers as strings on the applet. Use the techniques shown in Fig. 23.9.

23.5 *(Diameter, Circumference and Area of a Circle)* Write an applet that asks the user to input the radius of a circle as a floating-point number and draws the circle's diameter, circumference and area. Use the value 3.14159 for π. Use the techniques shown in Fig. 23.9. [*Note:* You may also use the predefined constant `Math.PI` for the value of π. This constant is more precise than the value 3.14159. Class `Math` is defined in the `java.lang` package, so you do not need to `import` it.] Use the following formulas (r is the radius):

$$diameter = 2r$$
$$circumference = 2\pi r$$
$$area = \pi r^2$$

23.6 *(Largest and Smallest)* Write an applet that reads five integers, determines which are the largest and smallest integers in the group and prints them. Draw the results on the applet.

23.7 *(Draw a Checkerboard Pattern)* Write an applet that draws a checkerboard pattern as follows:

```
* * * * * * * *
 * * * * * * * *
* * * * * * * *
 * * * * * * * *
* * * * * * * *
 * * * * * * * *
* * * * * * * *
 * * * * * * * *
```

23.8 *(Drawing Rectangles)* Write an applet that draws rectangles of different sizes and locations.

23.9 *(Drawing Rectangles Based on User Input)* Write an applet that allows the user to input values for the arguments required by method `drawRect`, then draws a rectangle using the four input values.

23.10 *(Drawing Ovals and Rectangles)* Class `Graphics` contains method `drawOval`, which takes as arguments the same four arguments as method `drawRect`. The arguments for method `drawOval` specify the "bounding box" for the oval—the sides of the bounding box are the boundaries of the oval. Write a Java applet that draws an oval and a rectangle with the same four arguments. The oval will touch the rectangle at the center of each side.

23.11 *(Drawing Ovals and Rectangles)* Modify the solution to Exercise 23.10 to output ovals of different shapes and sizes.

23.12 *(Drawing Ovals Based on User Input)* Write an applet that allows the user to input the four arguments required by method `drawOval`, then draws an oval using the four input values.

23.13 *(`TicTacToe` Demonstration Applet with Java Web Start)* Package the `TicTacToe` demonstration applet from the JDK (discussed in Section 23.2) for use with Java Web Start, then copy the JNLP document in Fig. 23.12 and modify it so that it launches the `TicTacToe` applet.

Multimedia: Applets and Applications

24



The wheel that squeaks the loudest … gets the grease.
—John Billings (Henry Wheeler Shaw)

We'll use a signal I have tried and found far-reaching and easy to yell. Waa-hoo!
—Zane Grey

There is a natural hootchy-kootchy motion to a goldfish.
—Walt Disney

Between the motion and the act falls the shadow.
—Thomas Stearns Eliot

Objectives

In this chapter you'll learn:

- How to get, display and scale images.
- How to create animations from sequences of images.
- How to create image maps that can sense when the cursor is over them.
- How to get, play, loop and stop sounds using an `AudioClip`.
- How to play video using interface `Player`.

Outline

24.1 Introduction
24.2 Loading, Displaying and Scaling Images
24.3 Animating a Series of Images
24.4 Image Maps
24.5 Loading and Playing Audio Clips
24.6 Playing Video and Other Media with Java Media Framework
24.7 Wrap-Up
24.8 Web Resources

Summary | Self-Review Exercises | Answers to Self-Review Exercises | Exercises
Special Section: Challenging Multimedia Projects | Making a Difference

24.1 Introduction

Multimedia—using sound, images, graphics, animation and video—makes applications "come alive." Although most multimedia in Java applications is two-dimensional, you can use the **Java 3D API** to create 3D graphics applications (`www.oracle.com/technetwork/java/javase/tech/index-jsp-138252.html`).

Most new computers sold today are "multimedia ready," with DVD drives and audio and video capabilities. Economical desktop computers, laptops and smartphones are so powerful that they can store and play DVD-quality (and often, HD-quality) sound and video.

Among users who want graphics, many now want three-dimensional, high-resolution, color graphics. True three-dimensional imaging is already available. We expect high-resolution, "theater-in-the-round," three-dimensional television to eventually become common. Sporting and entertainment events will seem to take place on your living room floor! Medical students worldwide will see operations being performed thousands of miles away, as if they were occurring in the same room. People will learn how to drive with incredibly realistic driving simulators in their homes before they get behind the wheel. The possibilities are endless and exciting.

Multimedia demands extraordinary computing power. Today's ultrapowerful processors make effective multimedia economical. Users are eager to own faster processors, larger memories and wider communications channels that support demanding multimedia applications. Ironically, these enhanced capabilities may not cost more—fierce competition keeps driving prices down.

The Java APIs provide multimedia facilities that enable you to start developing powerful multimedia applications immediately. This chapter presents several examples, including:

1. the basics of manipulating images
2. creating smooth animations
3. playing audio files with the `AudioClip` interface
4. creating image maps that can sense when the cursor is over them, even without a mouse click
5. playing video files using the `Player` interface

We introduce additional JNLP features that, with the user's permission, enable an applet or application to access files on the user's local computer. The exercises suggest dozens of challenging and interesting projects. Multimedia leverages creativity in ways that we did not experience with "conventional" computer capabilities. [*Note:* Java's multimedia

capabilities go far beyond those presented in this chapter. They include the **Java Media Framework (JMF) API** (for adding audio and video media to an application), **Java Sound API** (for playing, recording and modifying audio), Java 3D API (for creating and modifying 3D graphics), **Java Advanced Imaging API** (for image-processing capabilities, such as cropping and scaling), **Java Speech API** (for inputting speech from the user and converting it to text, or outputting text to the user as speech), Java 2D API (for creating and modifying 2D graphics, covered in Chapter 15) and **Java Image I/O API** (for reading images from and outputting images to files). Section 24.8 provides web links for these APIs.]

24.2 Loading, Displaying and Scaling Images

We begin our discussion with images. We'll use several different images in this chapter. You can create your own images with software such as Adobe® Photoshop®, Corel® Paint Shop Pro®, Microsoft® Paint and G.I.M.P. (`gimp.org`).

The applet of Fig. 24.1 uses Java Web Start and the JNLP **`FileOpenService`** (package `javax.jnlp`) to allow the user to select an image, then displays that image and allows the user to scale it. After the user selects an image, the applet gets the bytes from the file, then passes them to the `ImageIcon` (package `javax.swing`) constructor to create the image that will be displayed. Class `ImageIcon`'s constructors can receive arguments of several different formats, including a `byte` array containing the bytes of an image, an **`Image`** (package `java.awt`) already loaded in memory, or a `String` or a `URL` representing the image's location. Java supports various image formats, including **Graphics Interchange Format (GIF)**, **Joint Photographic Experts Group (JPEG)** and **Portable Network Graphics (PNG)**. File names for these types typically end with **`.gif`**, **`.jpg`** (or **`.jpeg`**) and **`.png`**, respectively.

```
1  // Fig. 24.1: LoadImageAndScale.java
2  // Loading, displaying and scaling an image in an applet
3  import java.awt.BorderLayout;
4  import java.awt.Graphics;
5  import java.awt.event.ActionEvent;
6  import java.awt.event.ActionListener;
7  import javax.jnlp.FileContents;
8  import javax.jnlp.FileOpenService;
9  import javax.jnlp.ServiceManager;
10 import javax.swing.ImageIcon;
11 import javax.swing.JApplet;
12 import javax.swing.JButton;
13 import javax.swing.JFrame;
14 import javax.swing.JLabel;
15 import javax.swing.JOptionPane;
16 import javax.swing.JPanel;
17 import javax.swing.JTextField;
18
19 public class LoadImageAndScale extends JApplet
20 {
21    private ImageIcon image; // references image to display
22    private JPanel scaleJPanel; // JPanel containing the scale-selector
```

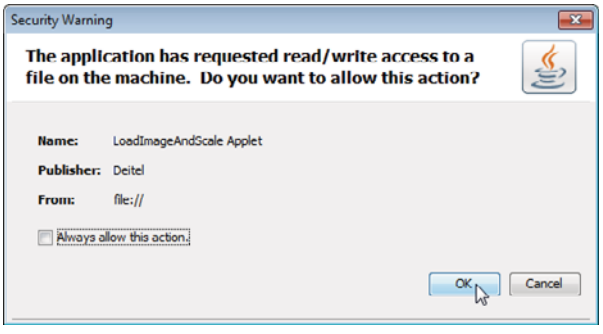
Fig. 24.1 | Loading, displaying and scaling an image in an applet. (Part 1 of 4.)

```
23      private JLabel percentJLabel; // label for JTextField
24      private JTextField scaleInputJTextField; // obtains user's input
25      private JButton scaleChangeJButton; // initiates scaling of image
26      private double scaleValue = 1.0;  //scale percentage for image
27
28      // load image when applet is loaded
29      public void init()
30      {
31         scaleJPanel = new JPanel();
32         percentJLabel = new JLabel( "scale percent:" );
33         scaleInputJTextField = new JTextField( "100" );
34         scaleChangeJButton = new JButton( "Set Scale" );
35
36         // add components and place scaleJPanel in applet's NORTH region
37         scaleJPanel.add( percentJLabel );
38         scaleJPanel.add( scaleInputJTextField );
39         scaleJPanel.add( scaleChangeJButton );
40         add( scaleJPanel, BorderLayout.NORTH );
41
42         // register event handler for scaleChangeJButton
43         scaleChangeJButton.addActionListener(
44            new ActionListener()
45            {
46               // when the JButton is pressed, set scaleValue and repaint
47               public void actionPerformed( ActionEvent e )
48               {
49                  scaleValue = Double.parseDouble(
50                     scaleInputJTextField.getText() ) / 100.0;
51                  repaint(); // causes image to be redisplayed at new scale
52               } // end method actionPerformed
53            } // end anonymous inner class
54         ); // end call to addActionListener
55
56         // use JNLP services to open an image file that the user selects
57         try
58         {
59            // get a reference to the FileOpenService
60            FileOpenService fileOpenService =
61               (FileOpenService) ServiceManager.lookup(
62                  "javax.jnlp.FileOpenService" );
63
64            // get file's contents from the FileOpenService
65            FileContents contents =
66               fileOpenService.openFileDialog( null, null );
67
68            // byte array to store image's data
69            byte[] imageData = new byte[ (int) contents.getLength() ];
70            contents.getInputStream().read( imageData ); // read image bytes
71            image = new ImageIcon( imageData ); // create the image
72
73            // if image successfully loaded, create and add DrawJPanel
74            add( new DrawJPanel(), BorderLayout.CENTER );
75         } // end try
```

Fig. 24.1 | Loading, displaying and scaling an image in an applet. (Part 2 of 4.)

```
76          catch( Exception e )
77          {
78             e.printStackTrace();
79          } // end catch
80       } // end method init
81
82       // DrawJPanel used to display loaded image
83       private class DrawJPanel extends JPanel
84       {
85          // display image
86          public void paintComponent( Graphics g )
87          {
88             super.paintComponent( g );
89
90             // the following values are used to center the image
91             double spareWidth =
92                getWidth() - scaleValue * image.getIconWidth();
93             double spareHeight =
94                getHeight() - scaleValue * image.getIconHeight();
95
96             // draw image with scaled width and height
97             g.drawImage( image.getImage(),
98                (int) ( spareWidth ) / 2, (int) ( spareHeight ) / 2,
99                (int) ( image.getIconWidth() * scaleValue ),
100               (int) ( image.getIconHeight() * scaleValue ), this );
101         } // end method paint
102      } // end class DrawJPanel
103   } // end class LoadImageAndScale
```

(a) Java Web Start security dialog that appears because this applet is requesting access to a file on the local computer



(b) **Open** dialog that appears if the user clicks **OK** in the security dialog

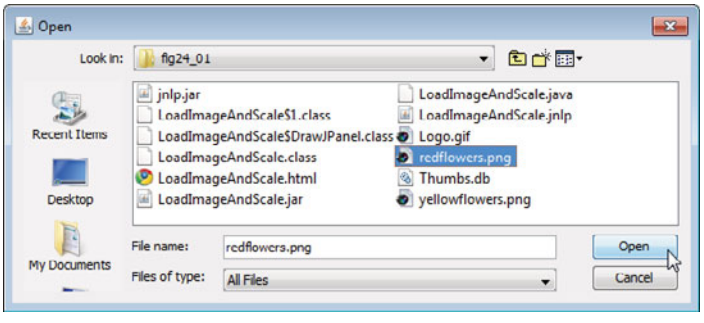


Fig. 24.1 | Loading, displaying and scaling an image in an applet. (Part 3 of 4.)

(c) Scaling the image

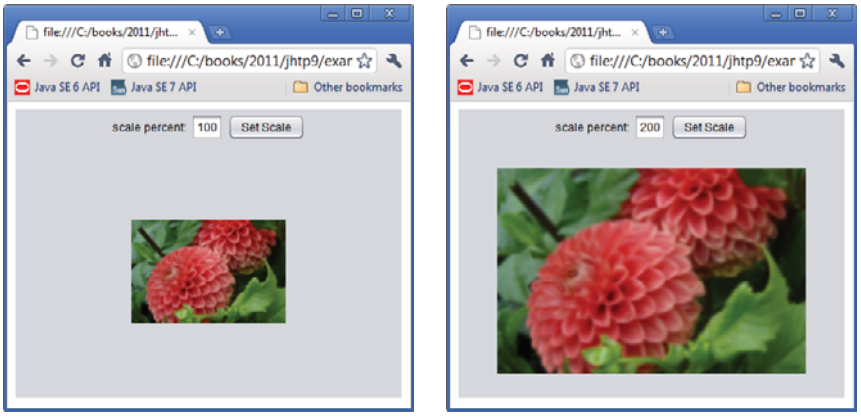


Fig. 24.1 | Loading, displaying and scaling an image in an applet. (Part 4 of 4.)

Configuring the GUI and the `JButton`'s Event Handler

The applet's `init` method (lines 29–80) configures the GUI and an event handler. It also uses JNLP services to enable the user to select an image to display from the local computer. Line 31 creates the `JPanel` that will contain the `JLabel`, `JTextField` and `JButton` created in lines 32–34. Lines 37–39 add these components to the `JPanel`'s default `FlowLayout`. Line 40 places this `JPanel` in the `NORTH` region of the `JApplet`'s default `BorderLayout`.

Lines 43–54 create the event handler for the `scaleChangeJButton`. When the user clicks this `JButton`, lines 49–50 obtain the user's input from the `scaleInputJTextField`, divide it by `100.0` to calculate the scale percentage and assign the result to `scaleValue`. This value will be used in later calculations to scale the image. For example, if the user enters `50`, the scale value will be `0.5` and the image will be displayed at half its original size. Line 51 then repaints the applet to display the image at its new scale.

Opening the Image File Using JNLP's `FileOpenService`

As we mentioned in Section 23.7, with the user's permission, Java Web Start programs can access the local file system via the JNLP APIs of package `javax.jnlp`. In this example, we'd like the user to select an image from the local computer to display in the applet. (We've provided two images in this example's directory with the source code.) You can use JNLP's `FileOpenService` to request limited access to the local file system.

Lines 7–9 import the interfaces and class we need to use the `FileOpenService`. Lines 60–62 use the JNLP `ServiceManager` class's `static` `lookup` method to obtain a reference to the `FileOpenService`. JNLP provides several services, so this method returns an `Object` that you must cast to the appropriate type. Lines 65–66 use the `FileOpenService`'s `openFileDialog` method to display a file-selection dialog. Java Web Start prompts the user (Fig. 24.1(a)) to approve the applet's request for local file-system access. If the user gives permission, the **Open** dialog (Fig. 24.1(b)) is displayed. Method `openFileDialog`'s parameters are a `String` to suggest a directory to open and a `String` array of acceptable file extensions (such as `"png"` and `"jpg"`). For simplicity, we passed `null` for each, which displays an open dialog showing the user's default directory and allows any file type to be selected.

When the user selects an image file and clicks the **Open** button in the dialog, method `openFileDialog` returns a `FileContents` object, which for security reasons does not give the program access to the file's exact location on disk. Instead, the program can get an `InputStream` and read the file's bytes. Line 69 creates a `byte` array in which the image's data will be stored. `FileContents` method **`getLength`** returns the number of bytes (as a `long`) in the file. Line 70 obtains the `InputStream`, then invokes its **`read`** method to fill the `imageData` `byte` array. Line 71 creates an `ImageIcon` using the `byte` array as the source of the image's data. Finally, line 74 adds a new `DrawJPanel` to the `CENTER` of the applet's `BorderLayout`. When the applet is displayed, its components' `paintComponent` methods are called, which causes the `DrawJPanel` to display the image. You can learn more about the JNLP APIs at `download.oracle.com/javase/6/docs/jre/api/javaws/jnlp/`.

Displaying the Image with Class `DrawJPanel`*'s* `paintComponent` *Method*

To separate the GUI from the area in which the image is displayed, we use a subclass of `JPanel` named `DrawJPanel` (lines 83–102). Its `paintComponent` method (lines 86–101) displays the image. We'd like to center the image in the `DrawJPanel`, so lines 91–94 calculate the difference between the width of the `DrawJPanel` and that of the scaled image, then the height of the `DrawJPanel` and that of the scaled image. `DrawJPanel`'s **`getWidth`** and **`getHeight`** methods (inherited indirectly from class `Component`) return the `DrawJPanel`'s width and height, respectively. The `ImageIcon`'s **`getIconWidth`** and **`getIconHeight`** methods return the image's width and height, respectively. The `scaleValue` is set to `1.0` by default (line 26), and is changed when the user clicks the **Set Scale** `JButton`.

Lines 97–100 use `Graphics`'s method **`drawImage`** to display a scaled `ImageIcon`. The first argument invokes the `ImageIcon`'s **`getImage`** method to obtain the `Image` to draw. The second and third arguments represent the image's *upper-left* corner coordinates with respect to the `DrawJPanel`'s upper-left corner. The fourth and fifth arguments specify the `Image`'s scaled width and height, respectively. Line 99 scales the image's width by invoking the `ImageIcon`'s `getIconWidth` method and multiplying its return value by `scaleValue`. Similarly, line 100 scales the image's height. The last argument is an **`ImageObserver`**—an interface implemented by class `Component`. Since class `DrawJPanel` indirectly extends `Component`, a `DrawJPanel` *is an* `ImageObserver`. This argument is important when displaying large images that require a long time to load (or download from the Internet). It's possible that a program will attempt to display the image before it has completely loaded (or downloaded). As the `Image` loads, the `ImageObserver` receives notifications and updates the image on the screen as necessary. In this example, the images are being loaded from the user's computer, so it's likely that entire image will be displayed immediately.

Compiling the Applet

Compiling and running this applet requires the `jnlp.jar` file that contains the JNLP APIs. This file can be found in your JDK installation directory under the directories

```
sample
   jnlp
      servlet
```

To compile the applet, use the following command:

```
javac -classpath PathToJnlpJarFile LoadImageAndScale.java
```

where *PathToJnlpJarFile* includes both the path and the file name `jnlp.jar`. For example, on our Windows Vista computer, the *PathToJnlpJarFile* is

```
"C:\Program Files\Java\jdk1.6.0_11\sample\jnlp\servlet\jnlp.jar"
```

Packaging the Applet for Use with Java Web Start

To package the applet for use with Java Web Start, you must create a JAR file that contains the applet's code and the `jnlp.jar` file. To do so, use the command

```
jar cvf LoadImageAndScale.jar *.class PathToJnlpJarFile
```

where *PathToJnlpJarFile* includes both the path and the file name `jnlp.jar`. This will place all the `.class` files for the applet and a copy of the `jnlp.jar` file in the new JAR file `LoadImageAndScale.jar`.

JNLP Document for `LoadImageAndScale` *Applet*

The JNLP document in Fig. 24.2 is similar to the one introduced in Fig. 23.12. The only new feature in this document is that the `resources` element (lines 10–14) contains a second `jar` element (line 13) that references the `jnlp.jar` file, which is embedded in the file `LoadImageAndScale.jar`.

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <jnlp codebase="file:." href="LoadImageAndScale.jnlp">
3
4     <information>
5        <title>LoadImageAndScale Applet</title>
6        <vendor>Deitel</vendor>
7        <offline-allowed/>
8     </information>
9
10    <resources>
11       <java version="1.6+"/>
12       <jar href="LoadImageAndScale.jar" main="true"/>
13       <jar href="jnlp.jar"/>
14    </resources>
15
16    <applet-desc
17       name="LoadImageAndScale"
18       main-class="LoadImageAndScale"
19       width="400"
20       height="300">
21    </applet-desc>
22 </jnlp>
```

Fig. 24.2 | JNLP document for the `LoadImageAndScale` applet.

Making the Applet Draggable Outside the Browser Window

The HTML document in Fig. 24.3 loads the applet into a web browser. In this example, we use an `applet` element to specify the applet's class and provide two `param` elements between the `applet` element's tags. The first (line 4) specifies that this applet should be **draggable**. That is, the user can hold the *Alt* key and use the mouse to drag the applet outside the browser window. The applet will then continue executing, even if the browser is

closed. Clicking the close box on the applet when it's executing outside the browser causes the applet to move back into the browser window if it's still open, or to terminate otherwise. The second `param` element shows an alternate way to specify the JNLP file that launches an applet. We discuss applet parameters in more detail in Section 27.2.

```
<html>
<body>
<applet code="LoadImageAndScale.class" width="400" height="300">
    <param name="draggable" value="true">
    <param name="jnlp_href" value="LoadImageAndScale.jnlp">
</applet>
</body>
</html>
```

Fig. 24.3 | HTML document to load the `LoadImageAndScale` applet and make it draggable outside the browser window.

24.3 Animating a Series of Images

Next, we animate a series of images stored in an array of `ImageIcons`. In this example, we use the JNLP `FileOpenService` to enable the user to choose a group of images that will be animated by displaying one image at a time at 50-millisecond intervals. The animation presented in Figs. 24.4–24.5 is implemented using a subclass of `JPanel` called `LogoAnimatorJPanel` (Fig. 24.4) that can be attached to an application window or a `JApplet`. Class `LogoAnimator` (Fig. 24.5) declares a `main` method (lines 8–20 of Fig. 24.5) to execute the animation as an application. Method `main` declares an instance of class `JFrame` and attaches a `LogoAnimatorJPanel` object to the `JFrame` to display the animation.

```
// Fig. 24.4: LogoAnimatorJPanel.java
// Animating a series of images.
import java.awt.Dimension;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.Graphics;
import javax.jnlp.FileContents;
import javax.jnlp.FileOpenService;
import javax.jnlp.ServiceManager;
import javax.swing.ImageIcon;
import javax.swing.JPanel;
import javax.swing.Timer;

public class LogoAnimatorJPanel extends JPanel
{
   protected ImageIcon images[]; // array of images
   private int currentImage = 0; // current image index
   private final int ANIMATION_DELAY = 50; // millisecond delay
   private int width; // image width
   private int height; // image height

```

Fig. 24.4 | Animating a series of images. (Part 1 of 3.)

```
22      private Timer animationTimer; // Timer drives animation
23
24      // constructor initializes LogoAnimatorJPanel by loading images
25      public LogoAnimatorJPanel()
26      {
27         try
28         {
29            // get reference to FileOpenService
30            FileOpenService fileOpenService =
31               (FileOpenService) ServiceManager.lookup(
32                  "javax.jnlp.FileOpenService" );
33
34            // display dialog that allows user to select multiple files
35            FileContents[] contents =
36               fileOpenService.openMultiFileDialog( null, null );
37
38            // create array to store ImageIcon references
39            images = new ImageIcon[ contents.length ];
40
41            // load the selected images
42            for ( int count = 0; count < images.length; count++ )
43            {
44               // create byte array to store an image's data
45               byte[] imageData =
46                   new byte[ (int) contents[ count ].getLength() ];
47
48               // get image's data and create image
49               contents[ count ].getInputStream().read( imageData );
50               images[ count ] = new ImageIcon( imageData );
51            } // end for
52
53            // this example assumes all images have the same width and height
54            width = images[ 0 ].getIconWidth();   // get icon width
55            height = images[ 0 ].getIconHeight(); // get icon height
56         } // end try
57         catch( Exception e )
58         {
59            e.printStackTrace();
60         } // end catch
61      } // end LogoAnimatorJPanel constructor
62
63      // display current image
64      public void paintComponent( Graphics g )
65      {
66         super.paintComponent( g ); // call superclass paintComponent
67
68         images[ currentImage ].paintIcon( this, g, 0, 0 );
69
70         // set next image to be drawn only if Timer is running
71         if ( animationTimer.isRunning() )
72            currentImage = ( currentImage + 1 ) % images.length;
73      } // end method paintComponent
```

Fig. 24.4 | Animating a series of images. (Part 2 of 3.)

```
74
75     // start animation, or restart if window is redisplayed
76     public void startAnimation()
77     {
78        if ( animationTimer == null )
79        {
80           currentImage = 0; // display first image
81
82           // create timer
83           animationTimer =
84              new Timer( ANIMATION_DELAY, new TimerHandler() );
85
86           animationTimer.start(); // start Timer
87        } // end if
88        else // animationTimer already exists, restart animation
89        {
90           if ( ! animationTimer.isRunning() )
91              animationTimer.restart();
92        } // end else
93     } // end method startAnimation
94
95     // stop animation Timer
96     public void stopAnimation()
97     {
98        animationTimer.stop();
99     } // end method stopAnimation
100
101    // return minimum size of animation
102    public Dimension getMinimumSize()
103    {
104       return getPreferredSize();
105    } // end method getMinimumSize
106
107    // return preferred size of animation
108    public Dimension getPreferredSize()
109    {
110       return new Dimension( width, height );
111    } // end method getPreferredSize
112
113    // inner class to handle action events from Timer
114    private class TimerHandler implements ActionListener
115    {
116       // respond to Timer's event
117       public void actionPerformed( ActionEvent actionEvent )
118       {
119          repaint(); // repaint animator
120       } // end method actionPerformed
121    } // end class TimerHandler
122 } // end class LogoAnimatorJPanel
```

Fig. 24.4 | Animating a series of images. (Part 3 of 3.)

```
1  // Fig. 24.5: LogoAnimator.java
2  // Displaying animated images on a JFrame.
3  import javax.swing.JFrame;
4
5  public class LogoAnimator
6  {
7     // execute animation in a JFrame
8     public static void main( String args[] )
9     {
10       LogoAnimatorJPanel animation = new LogoAnimatorJPanel();
11
12       JFrame window = new JFrame( "Animator test" ); // set up window
13       window.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
14       window.add( animation ); // add panel to frame
15
16       window.pack();  // make window just large enough for its GUI
17       window.setVisible( true );   // display window
18
19       animation.startAnimation();  // begin animation
20    } // end main
21 } // end class LogoAnimator
```



Fig. 24.5 | Displaying animated images on a `JFrame`.

Class `LogoAnimatorPanel`

Class `LogoAnimatorJPanel` (Fig. 24.4) maintains an array of `ImageIcons` (declared at line 16) that are loaded in the constructor (lines 25–61). The constructor begins by using the JNLP `FileOpenService`'s **`openMultiFileDialog`** method to display a file-selection dialog that allows the user to select multiple files at once. We named our sample images such that they all have the same base name ("`deitel`") followed by a two-digit number from 00–29. This ensures that our images are in the proper order for the animation. As in this chapter's first example, first the user is prompted to give permission, then the **Open** dialog appears if permission is granted. `FileOpenService` method `openMultiFileDialog` takes the same arguments as method `openFileDialog` but returns an array of `FileContents` objects representing the set of files selected by the user. When you run this application, navigate to a folder containing the images you wish to use and select the images. If you wish, you can use the 30 images we provide in this example's subdirectory named `images`.

Line 39 creates the array of `ImageIcons`, then lines 42–51 populate the array by creating a byte array (lines 45–46) for the current image's data, reading the bytes of the image into the array (line 49) and creating an `ImageIcon` object from the byte array. Lines 54–55 determine the width and height of the animation from the size of the first image in array `images`—we assume that all the images have the same width and height.

Method `startAnimation`

After the `LogoAnimatorJPanel` constructor loads the images, method `main` of Fig. 24.5 sets up the window in which the animation will appear (lines 12–17), and line 19 calls the `LogoAnimatorJPanel`'s `startAnimation` method (declared at lines 76–93 of Fig. 24.4). This method starts the program's animation for the first time or restarts the animation that the program stopped previously. The animation is driven by an instance of class **`Timer`** (from package `javax.swing`).

When the program is first run, method `startAnimation` is called to begin the animation. Although we provide the functionality for this method to restart the animation if it has been stopped, the example does not call the method for this purpose. We've added the functionality, however, should the reader choose to add GUI components that enable the user to start and stop the animation.

A `Timer` generates `ActionEvents` at a fixed interval in milliseconds (normally specified as an argument to the `Timer`'s constructor) and notifies all its `ActionListeners` each time an `ActionEvent` occurs. Line 78 determines whether the `Timer` reference `animationTimer` is `null`. If it is, method `startAnimation` is being called for the first time, and a `Timer` needs to be created so that the animation can begin. Line 80 sets `currentImage` to 0, which indicates that the animation should begin with the first element of array `images`. Lines 83–84 assign a new `Timer` object to `animationTimer`. The `Timer` constructor receives two arguments—the delay in milliseconds (`ANIMATION_DELAY` is 50, as specified in line 18) and the `ActionListener` that will respond to the `Timer`'s `ActionEvents`. For the second argument, an object of class `TimerHandler` is created. This class, which implements `ActionListener`, is declared in lines 114–121. Line 86 calls the `Timer` object's **`start`** method to start the `Timer`. Once started, `animationTimer` will generate an `ActionEvent` every 50 milliseconds and call the `Timer`'s event handler `actionPerformed` (lines 117–120). Line 119 calls `LogoAnimatorJPanel`'s `repaint` method to schedule a call to `LogoAnimatorJPanel`'s `paintComponent` method (lines 64–73). Remember that any subclass of `JComponent` that draws should do so in its `paintComponent` method. Recall that the first statement in any `paintComponent` method should be a call to the superclass's `paintComponent` method, to ensure that Swing components are displayed correctly.

If the animation started earlier, then our `Timer` was created and the condition in line 78 evaluates to `false`. The program continues with lines 90–91, which restarts the animation that the program stopped previously. The `if` condition at line 90 uses `Timer` method **`isRunning`** to determine whether the `Timer` is running (i.e., generating events). If it's not running, line 91 calls `Timer` method **`restart`** to indicate that the `Timer` should start generating events again. Once this occurs, method `actionPerformed` (the `Timer`'s event handler) is again called at regular intervals.

Method `paintComponent`

Line 68 calls the `ImageIcon`'s **`paintIcon`** method to display the image stored at element `currentImage` in the array. The arguments represent the `Component` on which to draw (`this`), the `Graphics` object that performs the drawing (`g`) and the coordinates of the image's upper-left corner. Lines 71–72 determine whether the `animationTimer` is running and, if so, prepare for the next image to be displayed by incrementing `currentImage` by 1. The remainder calculation ensures that the value of `currentImage` is set to 0 (to repeat the animation sequence) when it's incremented past the last element index in the array. The

`if` statement ensures that the same image will be displayed if `paintComponent` is called while the `Timer` is stopped. This can be useful if a GUI is provided that enables the user to start and stop the animation. For example, if the animation is stopped and the user covers it with another window, then uncovers it, method `paintComponent` will be called. In this case, we do not want the animation to show the next image (because the animation has been stopped). We simply want the window to display the same image until the animation is restarted.

Method `stopAnimation`

Method `stopAnimation` (lines 96–99) stops the animation by calling `Timer` method **`stop`** to indicate that the `Timer` should stop generating events. This prevents `actionPerformed` from calling `repaint` to initiate the painting of the next image in the array. Just as with restarting the animation, this example defines but does not use method `stopAnimation`. We've provided this method for demonstration purposes, or to allow the user to modify this example to stop and restart the animation.



Software Engineering Observation 24.1

When creating an animation for use in an applet, provide a mechanism for disabling the animation when the user browses a new web page different from the one on which the animation applet resides.

Methods `getPreferredSize` *and* `getMinimumSize`

By extending class `JPanel`, we're creating a new GUI component. So, we must ensure that it works like other components for layout purposes. Layout managers often use a component's **`getPreferredSize`** method (inherited from class `java.awt.Component`) to determine the component's preferred width and height. If a new component has a preferred width and height, it should override method `getPreferredSize` (lines 108–111) to return that width and height as an object of class **`Dimension`** (package `java.awt`). The `Dimension` class represents the width and height of a GUI component. In this example, the images we provide are 160 pixels wide and 80 pixels tall, so method `getPreferredSize` would return a `Dimension` object containing the numbers 160 and 80 (if you use these images).



Look-and-Feel Observation 24.1

The default size of a `JPanel` object is 10 pixels wide and 10 pixels tall.



Look-and-Feel Observation 24.2

When subclassing `JPanel` (or any other `JComponent`), override method `getPreferredSize` if the new component is to have a specific preferred width and height.

Lines 102–105 override method **`getMinimumSize`**. This method determines the minimum width and height of the component. As with method `getPreferredSize`, new components should override method `getMinimumSize` (also inherited from class `Component`). Method `getMinimumSize` simply calls `getPreferredSize` (a common programming practice) to indicate that the minimum size and preferred size are the same. Some layout managers ignore the dimensions specified by these methods. For example, a `BorderLayout`'s `NORTH` and `SOUTH` regions use only the component's preferred height.



Look-and-Feel Observation 24.3

If a new GUI component has a minimum width and height (i.e., smaller dimensions would render the component ineffective on the display), override method `getMinimumSize` *to return the minimum width and height as an instance of class* `Dimension`.



Look-and-Feel Observation 24.4

For many GUI components, method `getMinimumSize` *is implemented to return the result of a call to the component's* `getPreferredSize` *method.*

Compiling the Application

Compiling and running this application requires the `jnlp.jar` file that contains the JNLP APIs. To compile the application use the following command:

```
javac -classpath PathToJnlpJarFile *.java
```

where *PathToJnlpJarFile* includes both the path and the file name `jnlp.jar`.

Packaging the Application for Use with Java Web Start

To package the application for use with Java Web Start, you must create a JAR file that contains the applet's code and the `jnlp.jar` file. To do so, use the command

```
jar cvf LogoAnimator.jar *.class PathToJnlpJarFile
```

where *PathToJnlpJarFile* includes both the path and the file name `jnlp.jar`.

JNLP Document for `LoadImageAndScale` *Applet*

The JNLP document in Fig. 24.6 is similar to the one in Fig. 24.2. The only new feature in this document is the `application-desc` element (lines 16–19), which specifies the name of the application and its main class. To run this application, use the command

```
javaws LogoAnimator.jnlp
```

Recall that you can also run Java Web Start applications via a link in a web page, as we showed in Fig. 23.13.

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <jnlp codebase="file:." href="LogoAnimator.jnlp">
3
4     <information>
5        <title>LogoAnimator</title>
6        <vendor>Deitel</vendor>
7        <offline-allowed/>
8     </information>
9
10    <resources>
11       <java version="1.6+"/>
12       <jar href="LogoAnimator.jar" main="true"/>
13       <jar href="jnlp.jar"/>
14    </resources>
15
```

Fig. 24.6 | JNLP document for the `LoadImageAndScale` applet. (Part 1 of 2.)

```
16      <application-desc
17         name="LogoAnimator"
18         main-class="LogoAnimator">
19      </application-desc>
20   </jnlp>
```

Fig. 24.6 | JNLP document for the `LoadImageAndScale` applet. (Part 2 of 2.)

24.4 Image Maps

Image maps are commonly used to create interactive web pages. An image map is an image with **hot areas** that the user can click to accomplish a task, such as loading a different web page into a browser. When the user positions the mouse pointer over a hot area, normally a descriptive message appears in the status area of the browser or in a tool tip.

Figure 24.7 loads an image containing several of the programming-tip icons used in this book. The program allows the user to position the mouse pointer over an icon to display a descriptive message associated with it. Event handler `mouseMoved` (lines 39–43) takes the mouse coordinates and passes them to method `translateLocation` (lines 58–69). Method `translateLocation` tests the coordinates to determine the icon over which the mouse was positioned when the `mouseMoved` event occurred—the method then returns a message indicating what the icon represents. This message is displayed in the applet container's status bar using method **`showStatus`** of class `Applet`.

```
1  // Fig. 24.7: ImageMap.java
2  // Image map.
3  import java.awt.event.MouseAdapter;
4  import java.awt.event.MouseEvent;
5  import java.awt.event.MouseMotionAdapter;
6  import java.awt.Graphics;
7  import javax.swing.ImageIcon;
8  import javax.swing.JApplet;
9
10 public class ImageMap extends JApplet
11 {
12    private ImageIcon mapImage;
13
14    private static final String captions[] = { "Common Programming Error",
15       "Good Programming Practice", "Look-and-Feel Observation",
16       "Performance Tip", "Portability Tip",
17       "Software Engineering Observation", "Error-Prevention Tip" };
18
19    // sets up mouse listeners
20    public void init()
21    {
22       addMouseListener(
23
24          new MouseAdapter() // anonymous inner class
25          {
```

Fig. 24.7 | Image map. (Part 1 of 3.)

```
            // indicate when mouse pointer exits applet area
            public void mouseExited( MouseEvent event )
            {
               showStatus( "Pointer outside applet" );
            } // end method mouseExited
         } // end anonymous inner class
      ); // end call to addMouseListener

      addMouseMotionListener(

         new MouseMotionAdapter() // anonymous inner class
         {
            // determine icon over which mouse appears
            public void mouseMoved( MouseEvent event )
            {
               showStatus( translateLocation(
                  event.getX(), event.getY() ) );
            } // end method mouseMoved
         } // end anonymous inner class
      ); // end call to addMouseMotionListener

      mapImage = new ImageIcon( "icons.png" ); // get image
   } // end method init

   // display mapImage
   public void paint( Graphics g )
   {
      super.paint( g );
      mapImage.paintIcon( this, g, 0, 0 );
   } // end method paint

   // return tip caption based on mouse coordinates
   public String translateLocation( int x, int y )
   {
      // if coordinates outside image, return immediately
      if ( x >= mapImage.getIconWidth() || y >= mapImage.getIconHeight() )
         return "";

      // determine icon number (0 - 6)
      double iconWidth = ( double ) mapImage.getIconWidth() / 7.0;
      int iconNumber = ( int )( ( double ) x / iconWidth );

      return captions[ iconNumber ]; // return appropriate icon caption
   } // end method translateLocation
} // end class ImageMap
```

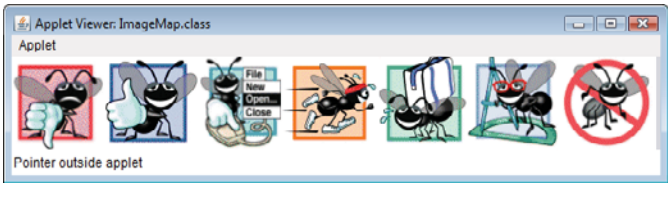


Fig. 24.7 | Image map. (Part 2 of 3.)



Fig. 24.7 | Image map. (Part 3 of 3.)

Clicking in the applet of Fig. 24.7 will not cause any action. In Chapter 27, we discuss the techniques for loading another web page into a browser via URLs and the `AppletCon-`

text interface. Using those techniques, this applet could associate each icon with a URL that the browser would display when the user clicks the icon.

24.5 Loading and Playing Audio Clips

Java programs can manipulate and play **audio clips**. Users can capture their own audio clips, and many clips are available in software products and over the Internet. Your system needs to be equipped with audio hardware (speakers and a sound card) to be able to play the audio clips.

Java provides several mechanisms for playing sounds in an applet. The two simplest are the Applet's **play** method and the **play** method of the **AudioClip** interface. Additional audio capabilities are available in the Java Media Framework and Java Sound APIs. If you'd like to play a sound once in a program, the Applet method play loads the sound and plays it once, then the sound can be garbage collected. The Applet method play has two versions:

```
public void play( URL location, String soundFileName );
public void play( URL soundURL );
```

The first version loads the audio clip stored in file soundFileName from location and plays the sound. The first argument is normally a call to the applet's getDocumentBase or **getCodeBase** method. Method getDocumentBase returns the location of the HTML file that loaded the applet. (If the applet is in a package, the method returns the location of the package or the JAR file containing the package.) Method getCodeBase indicates the location of the applet's .class file. The second version of method play takes a URL that contains the location and the file name of the audio clip. The statement

```
play( getDocumentBase(), "hi.au" );
```

loads the audio clip in file hi.au and plays it once.

The **sound engine** that plays the audio clips supports several audio file formats, including **Sun Audio file format** (**.au** extension), **Windows Wave file format** (**.wav** extension), **Macintosh AIFF file format** (**.aif** or **.aiff** extensions) and **Musical Instrument Digital Interface** (**MIDI**) **file format** (**.mid** or **.rmi** extensions). The Java Media Framework (JMF) and Java Sound APIs support additional formats.

The program of Fig. 24.8 demonstrates loading and playing an AudioClip (package java.applet). This technique is more flexible than Applet method play. An applet can use an AudioClip to store audio for repeated use throughout a program's execution. Applet method **getAudioClip** has two forms that take the same arguments as method play described previously. Method getAudioClip returns a reference to an AudioClip. An AudioClip has three methods—play, loop and stop. As mentioned earlier, method play plays the audio clip once. Method **loop** continuously loops through the audio clip in the background. Method **stop** terminates an audio clip that is currently playing. In the program, each of these methods is associated with a button on the applet.

```
1  // Fig. 24.8: LoadAudioAndPlay.java
2  // Loading and playing an AudioClip.
3  import java.applet.AudioClip;
4  import java.awt.event.ItemListener;
```

Fig. 24.8 | Loading and playing an AudioClip. (Part 1 of 3.)

```
5  import java.awt.event.ItemEvent;
6  import java.awt.event.ActionListener;
7  import java.awt.event.ActionEvent;
8  import java.awt.FlowLayout;
9  import javax.swing.JApplet;
10 import javax.swing.JButton;
11 import javax.swing.JComboBox;
12
13 public class LoadAudioAndPlay extends JApplet
14 {
15    private AudioClip sound1, sound2, currentSound;
16    private JButton playJButton, loopJButton, stopJButton;
17    private JComboBox soundJComboBox;
18
19    // load the audio when the applet begins executing
20    public void init()
21    {
22       setLayout( new FlowLayout() );
23
24       String choices[] = { "Welcome", "Hi" };
25       soundJComboBox = new JComboBox( choices ); // create JComboBox
26
27       soundJComboBox.addItemListener(
28
29          new ItemListener() // anonymous inner class
30          {
31             // stop sound and change sound to user's selection
32             public void itemStateChanged( ItemEvent e )
33             {
34                currentSound.stop();
35                currentSound = soundJComboBox.getSelectedIndex() == 0 ?
36                   sound1 : sound2;
37             } // end method itemStateChanged
38          } // end anonymous inner class
39       ); // end addItemListener method call
40
41       add( soundJComboBox ); // add JComboBox to applet
42
43       // set up button event handler and buttons
44       ButtonHandler handler = new ButtonHandler();
45
46       // create Play JButton
47       playJButton = new JButton( "Play" );
48       playJButton.addActionListener( handler );
49       add( playJButton );
50
51       // create Loop JButton
52       loopJButton = new JButton( "Loop" );
53       loopJButton.addActionListener( handler );
54       add( loopJButton );
55
56       // create Stop JButton
57       stopJButton = new JButton( "Stop" );
```

Fig. 24.8 | Loading and playing an `AudioClip`. (Part 2 of 3.)

```
58          stopJButton.addActionListener( handler );
59          add( stopJButton );
60
61          // load sounds and set currentSound
62          sound1 = getAudioClip( getDocumentBase(), "welcome.wav" );
63          sound2 = getAudioClip( getDocumentBase(), "hi.au" );
64          currentSound = sound1;
65       } // end method init
66
67       // stop the sound when the user switches web pages
68       public void stop()
69       {
70          currentSound.stop(); // stop AudioClip
71       } // end method stop
72
73       // private inner class to handle button events
74       private class ButtonHandler implements ActionListener
75       {
76          // process play, loop and stop button events
77          public void actionPerformed( ActionEvent actionEvent )
78          {
79             if ( actionEvent.getSource() == playJButton )
80                currentSound.play(); // play AudioClip once
81             else if ( actionEvent.getSource() == loopJButton )
82                currentSound.loop(); // play AudioClip continuously
83             else if ( actionEvent.getSource() == stopJButton )
84                currentSound.stop(); // stop AudioClip
85          } // end method actionPerformed
86       } // end class ButtonHandler
87    } // end class LoadAudioAndPlay
```



Fig. 24.8 | Loading and playing an `AudioClip`. (Part 3 of 3.)

Lines 62–63 in the applet's `init` method use `getAudioClip` to load two audio files—a Windows Wave file (`welcome.wav`) and a Sun Audio file (`hi.au`). The user can select which audio clip to play from the `JComboBox soundJComboBox`. The applet's `stop` method is overridden at lines 68–71. When the user switches web pages, the applet container calls the applet's `stop` method. This enables the applet to stop playing the audio clip. Otherwise, it continues to play in the background—even if the applet is not displayed in the browser. This is not necessarily a problem, but it can be annoying to the user if the audio clip is looping. The `stop` method is provided here as a convenience to the user.



Look-and-Feel Observation 24.5

When playing audio clips in an applet or application, provide a mechanism for the user to disable the audio.

24.6 Playing Video and Other Media with Java Media Framework

A simple video can concisely and effectively convey a great deal of information. Using the Java Media Framework (JMF) API, you can create Java applications that play, edit, stream and capture many popular media types. This section briefly introduces some popular media formats and demonstrates playing video using the JMF API.

JMF 2.1.1e supports media file types such as **Microsoft Audio/Video Interleave** (`.avi`), **Macromedia Flash movies** (`.swf`), **Future Splash** (`.spl`), **MPEG Layer 3 Audio** (`.mp3`), Musical Instrument Digital Interface (MIDI; `.mid` or `.rmi` extensions), **MPEG-1 videos** (`.mpeg`, `.mpg`), **QuickTime** (`.mov`), Sun Audio file format (`.au` extension), and Macintosh AIFF file format (`.aif` or `.aiff` extensions). You've already seen some of these file types.

Currently, JMF is available as an extension separate from the JDK. The most recent JMF implementation (2.1.1e) can be downloaded from:

```
www.oracle.com/technetwork/java/javase/download-142937.html
```

[*Note:* Keep track of where you install the Java Media Framework on your computer. To compile and run this application, you must include in the class path the `jmf.jar` file that is installed with the Java Media Framework. Recall that you can specify the class path with both the `javac` and `java` commands via the `-classpath` command-line option.]

The JMF website provides versions of the JMF that take advantage of the performance features of certain platforms. For example, the JMF Windows Performance Pack provides extensive media and device support for Java programs running on Microsoft Windows platforms. The JMF's website (`www.oracle.com/technetwork/java/javase/tech/index-jsp-140239.html`) provides information and resources for JMF programmers.

Creating a Simple Media Player

JMF offers several mechanisms for playing media. The simplest is using objects that implement interface **`Player`** declared in package **`javax.media`**. Package `javax.media` and its subpackages contain the classes that compose the Java Media Framework. To play a media clip, you must first create a `URL` object that refers to it. Then pass the `URL` as an argument to `static` method **`createRealizedPlayer`** of class `Manager` to obtain a `Player` for the media clip. Class **`Manager`** declares utility methods for accessing system resources to play and to manipulate media. Figure 24.9 declares a `JPanel` that demonstrates some of these methods.

```
1  // Fig. 24.9: MediaPanel.java
2  // JPanel that plays a media file from a URL.
3  import java.awt.BorderLayout;
4  import java.awt.Component;
5  import java.io.IOException;
6  import java.net.URL;
7  import javax.media.CannotRealizeException;
8  import javax.media.Manager;
9  import javax.media.NoPlayerException;
```

Fig. 24.9 | `JPanel` that plays a media file from a URL. (Part 1 of 2.)

```
10 import javax.media.Player;
11 import javax.swing.JPanel;
12
13 public class MediaPanel extends JPanel
14 {
15    public MediaPanel( URL mediaURL )
16    {
17       setLayout( new BorderLayout() ); // use a BorderLayout
18
19       // Use lightweight components for Swing compatibility
20       Manager.setHint( Manager.LIGHTWEIGHT_RENDERER, true );
21
22       try
23       {
24          // create a player to play the media specified in the URL
25          Player mediaPlayer = Manager.createRealizedPlayer( mediaURL );
26
27          // get the components for the video and the playback controls
28          Component video = mediaPlayer.getVisualComponent();
29          Component controls = mediaPlayer.getControlPanelComponent();
30
31          if ( video != null )
32             add( video, BorderLayout.CENTER ); // add video component
33
34          if ( controls != null )
35             add( controls, BorderLayout.SOUTH ); // add controls
36
37          mediaPlayer.start(); // start playing the media clip
38       } // end try
39       catch ( NoPlayerException noPlayerException )
40       {
41          System.err.println( "No media player found" );
42       } // end catch
43       catch ( CannotRealizeException cannotRealizeException )
44       {
45          System.err.println( "Could not realize media player" );
46       } // end catch
47       catch ( IOException iOException )
48       {
49          System.err.println( "Error reading from the source" );
50       } // end catch
51    } // end MediaPanel constructor
52 } // end class MediaPanel
```

Fig. 24.9 | `JPanel` that plays a media file from a URL. (Part 2 of 2.)

The constructor (lines 15–51) sets up the `JPanel` to play the media file specified by the constructor's `URL` parameter. `MediaPanel` uses a `BorderLayout` (line 17). Line 20 invokes `static` method **`setHint`** to set the flag **`Manager.LIGHTWEIGHT_RENDERER`** to `true`. This instructs the `Manager` to use a lightweight renderer that is compatible with lightweight Swing components, as opposed to the default heavyweight renderer. Inside the `try` block (lines 22–38), line 25 invokes `static` method `createRealizedPlayer` of class `Manager` to create and realize a `Player` that plays the media file. When a `Player` realizes, it identifies

the system resources it needs to play the media. Depending on the file, realizing can be a resource-consuming and time-consuming process. Method `createRealizedPlayer` throws three checked exceptions, **`NoPlayerException`**, **`CannotRealizeException`** and `IOException`. A `NoPlayerException` indicates that the system could not find a player that can play the file format. A `CannotRealizeException` indicates that the system could not properly identify the resources a media file needs. An `IOException` indicates that there was an error while reading the file. These exceptions are handled in the `catch` block in lines 39–50.

Line 28 invokes method **`getVisualComponent`** of `Player` to get a `Component` that displays the visual (generally video) aspect of the media file. Line 29 invokes method **`getControlPanelComponent`** of `Player` to get a `Component` that provides playback and media controls. These components are assigned to local variables `video` and `controls`, respectively. The `if` statements in lines 31–32 and lines 34–35 add the `video` and the `controls` if they exist. The `video Component` is added to the `CENTER` region (line 32), so it fills any available space on the `JPanel`. The `controls Component`, which is added to the `SOUTH` region, typically provides the following controls:

1. A *positioning slider* to jump to certain points in the media clip
2. A *pause button*
3. A *volume button* that provides volume control by right clicking and a mute function by left clicking
4. A *media properties button* that provides detailed media information by left clicking and frame-rate control by right clicking

Line 37 calls `Player` method **`start`** to begin playing the media file. Lines 39–50 handle the various exceptions that `createRealizedPlayer` throws.

The application in Fig. 24.10 displays a `JFileChooser` dialog for the user to choose a media file. It then creates a `MediaPanel` that plays the selected file and creates a `JFrame` to display the `MediaPanel`.

```
1  // Fig. 24.10: MediaTest.java
2  // Test application that creates a MediaPanel from a user-selected file.
3  import java.io.File;
4  import java.net.MalformedURLException;
5  import java.net.URL;
6  import javax.swing.JFileChooser;
7  import javax.swing.JFrame;
8
9  public class MediaTest
10 {
11    // launch the application
12    public static void main( String args[] )
13    {
14       // create a file chooser
15       JFileChooser fileChooser = new JFileChooser();
16
17       // show open file dialog
18       int result = fileChooser.showOpenDialog( null );
```

Fig. 24.10 | Test application that creates a MediaPanel from a user-selected file. (Part 1 of 2.)

```
19
20          if ( result == JFileChooser.APPROVE_OPTION ) // user chose a file
21          {
22             URL mediaURL = null;
23
24             try
25             {
26                // get the file as URL
27                mediaURL = fileChooser.getSelectedFile().toURI().toURL();
28             } // end try
29             catch ( MalformedURLException malformedURLException )
30             {
31                System.err.println( "Could not create URL for the file" );
32             } // end catch
33
34             if ( mediaURL != null ) // only display if there is a valid URL
35             {
36                JFrame mediaTest = new JFrame( "Media Tester" );
37                mediaTest.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
38
39                MediaPanel mediaPanel = new MediaPanel( mediaURL );
40                mediaTest.add( mediaPanel );
41
42                mediaTest.setSize( 300, 300 );
43                mediaTest.setVisible( true );
44             } // end inner if
45          } // end outer if
46       } // end main
47    } // end class MediaTest
```

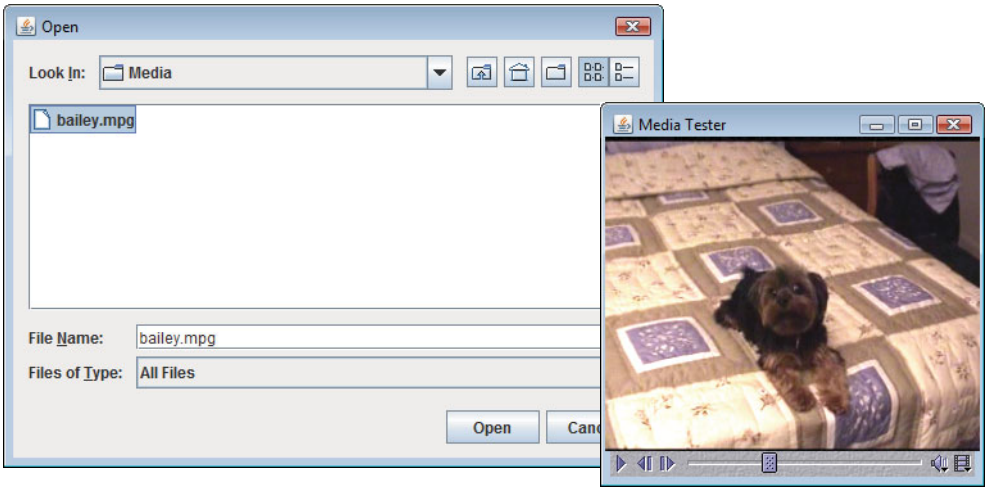


Fig. 24.10 | Test application that creates a MediaPanel from a user-selected file. (Part 2 of 2.)

Method `main` (lines 12–46) assigns a new `JFileChooser` to local variable `fileChooser` (line 15), shows an open-file dialog (line 18) and assigns the return value to `result`. Line 20 checks `result` to determine whether the user chose a file. To create a

`Player` to play the selected media file, you must convert the `File` object returned by `JFileChooser` to a `URL` object. Method **`toURI`** of class `File` returns a `URI` that points to the `File` on the system. We then invoke method **`toURL`** of class `URI` to get the file's `URL`. The `try` statement (lines 24–32) creates a `URL` for the selected file and assigns it to `mediaURL`. The `if` statement in lines 34–44 checks that `mediaURL` is not `null` and creates the GUI components to play the media.

24.7 Wrap-Up

In this chapter, you learned how to build multimedia-rich applications with sound, images, graphics and video. We introduced Java's multimedia capabilities, including the Java Media Framework API and Java Sound API. You used class `ImageIcon` to display and manipulate images stored in files, and you learned about the different image formats supported by Java. You used the JNLP `FileOpenService` to enable the user of a Java Web Start application to select files from the local file system, then used streams to load the contents of those files for use in your programs. You created an animation by displaying a series of images in a specific order. You used image maps to make an application more interactive. You learned how to load audio clips and how to play them either once or in a continuous loop. The chapter concluded with a demonstration of loading and playing video. In the next chapter, you'll continue your study of GUI concepts, building on the techniques you learned in Chapter 14.

24.8 Web Resources

`www.nasa.gov/multimedia/index.html`
The *NASA Multimedia Gallery* contains a wide variety of images, audio clips and video clips that you can download and use to test your Java multimedia programs.

`commons.wikimedia.org/wiki/Main_Page`
The *Wikimedia Commons* site provides access to millions of media files.

`www.anbg.gov.au/gardens/index.html`
The *Australian National Botanic Gardens* website provides links to the sounds of many animals. Try, for example, the *Common Birds* link under the "Animals in the Gardens" section.

`www.thefreesite.com`
This site has links to free sounds and clip art.

`www.soundcentral.com`
SoundCentral provides audio clips in WAV, AU, AIFF and MIDI formats.

`www.animationfactory.com`
The *Animation Factory* provides thousands of free GIF animations for personal use.

`www.clipart.com`
This site is a subscription-based service for images and sounds.

`java.sun.com/developer/techDocs/hi/repository/`
The *Java look-and-feel Graphics Repository* provides images designed for use in a Swing GUI, including toolbar button images.

`www.freebyte.com/graphicprograms/`
This guide contains links to several free graphics software programs. The software can be used to modify images and draw graphics.

`graphicssoft.about.com/od/pixelbasedfreewin/`
This site provides links to free graphics programs designed for use on Windows machines.

Java Multimedia API References

`www.oracle.com/technetwork/java/javase/tech/media-141984.html`
The online home of the Java Media APIs.

`www.oracle.com/technetwork/java/index-139508.html`
The *Java Sound API* home page. Java Sound provides capabilities for playing and recording audio.

`java3d.dev.java.net/`
The *Java 3D API* home page. This API can be used to produce three-dimensional images typical of today's video games.

`java.sun.com/products/java-media/speech/`
The *Java Speech API* enables programs to perform speech synthesis and speech recognition.

`freetts.sourceforge.net/docs/index.php`
FreeTTS is an implementation of the Java Speech API.

Summary

Section 24.2 Loading, Displaying and Scaling Images

- Class `ImageIcon`'s constructors can receive arguments of several different formats, including a byte array containing the bytes of an image, an `Image` (package `java.awt`) already loaded in memory, or a `String` or a URL representing the image's location.
- Java supports several image formats, including Graphics Interchange Format (GIF), Joint Photographic Experts Group (JPEG) and Portable Network Graphics (PNG). The file names for these types typically end with `.gif`, `.jpg` (or `.jpeg`) and `.png`, respectively.
- Java Web Start programs can access the local file system via the JNLP APIs (package `javax.jnlp`). You can use JNLP's `FileOpenService` (p. 969) to request limited access to the local file system.
- The JNLP `ServiceManager` class's `static lookup` (p. 972) method obtains a reference to the `FileOpenService` (p. 969). Since other services are provided by JNLP, this method returns an `Object` reference, which you must cast to the appropriate type.
- `FileOpenService` method `openFileDialog` method (p. 972) displays a file-selection dialog. Java Web Start prompts the user to approve the program's request local file-system access. If the user gives permission, the **Open** dialog is displayed. The `openFileDialog` method has two parameters—a `String` to suggest a directory to open and a `String` array of acceptable file extensions.
- Method `openFileDialog` returns a `FileContents` object, which for security reasons does not give the program access to the file's exact location on disk. Instead, the program can get an `InputStream` and read the file's bytes.
- `FileContents` method `getLength` (p. 973) returns the number of bytes in the file.
- `Component` methods `getWidth` and `getHeight` (p. 973) return the width and height of a `Component`.
- `ImageIcon` methods `getIconWidth` and `getIconHeight` (p. 973) return the width and height of an image, respectively.
- Class `Graphics` provides overloaded `drawImage` methods (p. 973), one of which displays a scaled version of an `Image` (p. 969). The first argument is the `Image` to draw. The second and third arguments represent the upper-left corner coordinates of the `Image`. The fourth and fifth arguments specify the `Image`'s scaled width and height, respectively. The last argument is a reference to an `ImageObserver` (p. 973)—an interface implemented by class `Component`.
- As an `Image` loads, its `ImageObserver` receives notifications and updates the image on the screen as necessary.

- A draggable applet (p. 974) can be dragged outside the browser window by holding the *Alt* key and dragging the applet with the mouse. The applet will then continue executing, even if the browser is closed. Clicking the close box on the applet when it's executing outside the browser causes the applet to move back into the browser window if it's still open, or to terminate otherwise.

Section 24.3 Animating a Series of Images

- The `FileOpenService`'s `openMultiFileDialog` method (p. 978) displays a file-selection dialog that allows the user to select multiple files at once. `FileOpenService` method `openMultiFileDialog` takes the same arguments as method `openFileDialog`, but returns an array of `FileContents` objects representing the set of files selected by the user.
- `Timer` objects generate `ActionEvents` at fixed millisecond intervals (`Timer`). The `Timer` constructor receives a delay in milliseconds and an `ActionListener`. `Timer` method `start` (p. 979) starts the `Timer`. Method `stop` indicates that the `Timer` should stop generating events. Method `restart` (p. 979) indicates that the `Timer` should start generating events again.
- `ImageIcon` method `paintIcon` (p. 979) displays the `ImageIcon`'s image. Its four arguments are a reference to the `Component` on which the image will be displayed, a reference to the `Graphics` object used to render the image, the *x*-coordinate of the upper-left corner of the image and the *y*-coordinate of the upper-left corner of the image.

Section 24.4 Image Maps

- An image map (image map) is an image that has hot areas (p. 982) that the user can click to accomplish a task, such as loading a different web page into a browser.

Section 24.5 Loading and Playing Audio Clips

- `Applet` method `play` (p. 985) has two forms:

```
public void play( URL location, String soundFileName );
public void play( URL soundURL );
```

  One version loads the audio clip (p. 985) stored in file `soundFileName` from `location` and plays the sound. The other takes a `URL` that contains the location and the file name of the audio clip.
- `Applet` method `getDocumentBase` indicates the location of the HTML file that loaded the applet. Method `getCodeBase` (p. 985) indicates where the `.class` file for an applet is located.
- The sound engine (p. 985) that plays audio clips supports several audio file formats, including Sun Audio file format (`.au` extension), Windows Wave file format (`.wav` extension), Macintosh AIFF file format (`.aif` or `.aiff` extensions) and Musical Instrument Digital Interface (MIDI) file format (`.mid` or `.rmi` extensions). The Java Media Framework (JMF; p. 969) supports additional formats.
- `Applet` method `getAudioClip` (p. 985) has two forms that take the same arguments as the `play` method. Method `getAudioClip` returns a reference to an `AudioClip` (p. 985). `AudioClips` have three methods—`play`, `loop` and `stop`. Method `play` plays the audio clip once (p. 985). Method `loop` (p. 985) continuously loops the audio clip. Method `stop` terminates an audio clip that is currently playing.

Section 24.6 Playing Video and Other Media with Java Media Framework

- Package `javax.media` and its subpackages contain the Java Media Framework classes.
- Class `Manager` (p. 988) declares methods for accessing system resources to play and to manipulate media.
- Method `toURI` of class `File` (p. 992) returns a `URI` that points to the `File` on the system.

Self-Review Exercises

24.1 Fill in the blanks in each of the following statements:

a) `Graphics` method _____ displays an image on an applet.
b) Java provides two mechanisms for playing sounds in an applet—the `Applet`'s `play` method and the `play` method of the _____ interface.
c) A(n) _____ is an image that has hot areas that the user can click to accomplish a task such as loading a web page.
d) Method _____ of class `ImageIcon` displays the `ImageIcon`'s image.
e) Java supports several image formats, including _____, _____ and _____.
f) The JNLP _____ class's `static lookup` method obtains a reference to the `FileOpenService`.

24.2 Determine whether each of the following statements is *true* or *false*. If *false*, explain why.

a) A sound is marked for garbage collection after it plays via the play method of class `Applet`.
b) Class `ImageIcon` provides constructors that allow an `ImageIcon` object to be initialized only with an image from the local computer.
c) Method `play` of class `AudioClip` continuously loops an audio clip.
d) The Java Image I/O API is used for adding 3D graphics to a Java application.
e) `Applet` method `getDocumentBase` returns, as an object of class `URL`, the location on the Internet of the HTML file that invoked the applet.
f) `FileOpenService` methods `openFileDialog` and `openMultiFileDialog` return a `FileContents` object and an array of `FileContents` objects, respectively.

Answers to Self-Review Exercises

24.1 a) `drawImage`. b) `AudioClip`. c) image map. d) `paintIcon`. e) Graphics Interchange Format (GIF), Joint Photographic Experts Group (JPEG), Portable Network Graphics (PNG). f) `ServiceManager`.

24.2 a) True. b) False. `ImageIcon` can load images from the Internet as well. c) False. Method `play` of class `AudioClip` plays an audio clip once. Method `loop` of class `AudioClip` continuously loops an audio clip. d) False. The Java 3D API is used for creating and modifying 3D graphics. The Java Image I/O API is used for reading from and outputting images to files. e) True. f) True.

Exercises

24.3 ***("Browser-Friendly" Animation)*** Describe how to make an animation "browser friendly."

24.4 ***(Audio Clips)*** Describe the Java methods for playing and manipulating audio clips.

24.5 ***(Image Maps)*** Explain how image maps are used. List several examples of their use.

24.6 ***(Randomly Erasing an Image)*** Suppose an image is displayed in a rectangular screen area. One way to erase the image is simply to set every pixel to the same color immediately, but the visual effect is dull. Write a Java program that displays an image, then erases it by using random-number generation to select individual pixels to erase. After most of the image is erased, erase all the remaining pixels at once. You can draw individual pixels as a line that starts and ends at the same coordinates. You might try several variants of this problem. For example, you might display lines randomly or display shapes randomly to erase regions of the screen.

24.7 ***(Text Flasher)*** Create a Java program that repeatedly flashes text on the screen. Do this by alternating the text with a plain background-color image. Allow the user to control the "blink speed" and the background color or pattern. You'll need to use methods **`getDelay`** and **`setDelay`** of class

`Timer`. These methods are used to retrieve and set the interval in milliseconds between `ActionEvents`, respectively

24.8 *(Image Flasher)* Create a Java program that repeatedly flashes an image on the screen. Do this by alternating the image with a plain background-color image.

24.9 *(Digital Clock)* Implement a program that displays a digital clock on the screen.

24.10 *(Calling Attention to an Image)* If you want to emphasize an image, you might place a row of simulated light bulbs around it. You can let the light bulbs flash in unison or fire on and off in sequence one after the other.

24.11 *(Image Zooming)* Create a program that enables you to zoom in on or out from an image.

24.12 *(`LoadImageAndScale` Modification)* Modify applet `LoadImageAndScale` (Fig. 24.1) to provide a second button that enables the user to choose a new image. The button's event handler should use the JNLP `FileOpenService` to display an **Open** dialog, so that the user can select a new image.

24.13 *(Image Viewer)* Using the JNLP techniques you learned in Sections 24.2–24.3, create an image viewer application that enables the user to select a group of images to display. The application should display a `JList` containing the names of the selected files. You can obtain the name of the file represented by a `FileContents` object by calling its `getName` method. When the user clicks the name of an image in the `JList`, the application should display the image in the window.

Special Section: Challenging Multimedia Projects

The preceding exercises are keyed to the text and designed to test your understanding of fundamental multimedia concepts. This section includes a collection of advanced multimedia projects. You should find these problems challenging, yet entertaining. The problems vary in difficulty. Some require an hour or two of program writing and implementation. Others are useful for lab assignments that might require two or three weeks of study and implementation. Some are challenging term projects. [*Note to Instructors:* Solutions are *not* provided for these exercises.]

24.14 *(Animation)* Create a general-purpose Java animation program. It should allow the user to specify the sequence of frames to be displayed, the speed at which the images are displayed, audios to be played while the animation is running and so on.

24.15 *(Limericks)* Modify the limerick-writing program you wrote in Exercise 16.6 to sing the limericks your program creates.

24.16 *(Random Interimage Transition)* This provides a nice visual effect. If you're displaying one image in a given area on the screen and you'd like to transition to another image in the same area, store the new screen image in an off-screen buffer and randomly copy pixels from it to the display area, overlaying the pixels already at those locations. When the vast majority of the pixels have been copied, copy the entire new image to the display area to be sure you're displaying the complete new image. To implement this program, you may need to use the `PixelGrabber` and `MemoryImageSource` classes (see the Java API documentation for descriptions of these classes). You might try several variants of this problem. For example, select all the pixels in a randomly chosen straight line or shape in the new image and overlay them above the corresponding positions of the old image.

24.17 *(Background Audio)* Add background audio to one of your favorite applications by using the `loop` method of class `AudioClip` to play the sound in the background while you interact with your application in the normal way.

24.18 *(Scrolling Marquee Sign)* Create a program that scrolls dotted characters from right to left (or from left to right if that is appropriate for your language) across a marquee-like display sign. As an option, display the text in a continuous loop, so that after the text disappears at one end, it reappears at the other.

24.19 *(Scrolling Image Marquee)* Create a program that scrolls an image across a marquee screen.

24.20 *(Analog Clock)* Create a program that displays an analog clock with hour, minute and second hands that move appropriately as the time changes.

24.21 *(Dynamic Audio and Graphical Kaleidoscope)* Write a kaleidoscope program that displays reflected graphics to simulate the popular children's toy. Incorporate audio effects that "mirror" your program's dynamically changing graphics.

24.22 *(Automatic Jigsaw Puzzle Generator)* Create a jigsaw puzzle generator and manipulator. The user specifies an image. Your program loads and displays the image, then breaks it into randomly selected shapes and shuffles them. The user then uses the mouse to move the pieces around to solve the puzzle. Add appropriate audio sounds as the pieces are moved around and snapped back into place. You might keep tabs on each piece and where it really belongs—then use audio effects to help the user get the pieces into the correct positions.

24.23 *(Maze Generator and Walker)* Develop a multimedia-based maze generator and traverser program based on the maze programs you wrote in Exercises 18.20–18.22. Let the user customize the maze by specifying the number of rows and columns and by indicating the level of difficulty. Have an animated mouse walk the maze. Use audio to dramatize the movement of your mouse character.

24.24 *(One-Armed Bandit)* Develop a multimedia simulation of a "one-armed bandit." Have three spinning wheels. Place symbols and images of various fruits on each wheel. Use random-number generation to simulate the spinning of each wheel and the stopping of each wheel on a symbol.

24.25 *(Horse Race)* Create a simulation of a horse race. Have multiple contenders. Use audios for a race announcer. Play the appropriate audios to indicate the correct status of each contender throughout the race. Use audios to announce the final results. You might try to simulate the kinds of horse-racing games that are often played at carnivals. The players take turns at the mouse and have to perform some skill-oriented manipulation with it to advance their horses.

24.26 *(Shuffleboard)* Develop a multimedia-based simulation of the game of shuffleboard. Use appropriate audio and visual effects.

24.27 *(Game of Pool)* Create a multimedia-based simulation of the game of pool. Each player takes turns using the mouse to position a pool cue and hit it against the ball at the appropriate angle to try to make other balls fall into the pockets. Your program should keep score.

24.28 *(Artist)* Design an art program that will give an artist a great variety of capabilities to draw, use images and use animations to create a dynamic multimedia art display.

24.29 *(Fireworks Designer)* Create a Java program that someone might use to create a fireworks display. Create a variety of fireworks demonstrations. Then orchestrate the firing of the fireworks for maximum effect.

24.30 *(Floor Planner)* Develop a program that will help someone arrange furniture in a home. Add features that enable the person to achieve the best possible arrangement.

24.31 *(Crossword)* Crossword puzzles are among the most popular pastimes. Develop a multimedia-based crossword-puzzle program. Your program should enable the player to place and erase words easily. Tie your program to a large computerized dictionary. Your program also should be able to suggest words on which letters have already been filled in. Provide other features that will make the crossword-puzzle enthusiast's job easier.

24.32 *(15 Puzzle)* Write a multimedia-based program that enables the user to play the game of 15. The game is played on a 4-by-4 board having a total of 16 slots. One slot is empty, the others are occupied by 15 tiles numbered 1 through 15. The user can move any tile next to the currently empty slot into that slot by clicking on the tile. Your program should create the board with the tiles in random order. The goal is to arrange the tiles into sequential order, row by row.

24.33 *(Reaction Time/Reaction Precision Tester)* Create a program that moves a randomly created shape around the screen. The user moves the mouse to catch and click on the shape. The shape's speed and size can be varied. Keep statistics on how long the user typically takes to catch a shape of a given size. The user will probably have more difficulty catching faster-moving, smaller shapes.

24.34 *(Calendar/Tickler File)* Using both audio and images, create a general-purpose calendar and "tickler" file. For example, the program should sing "Happy Birthday" when you use it on your birthday. Have the program display images and play audios associated with important events. Also, have it remind you in advance of these important events. It would be nice, for example, to have the program give you a week's notice so you can pick up an appropriate greeting card for that special person.

24.35 *(Rotating Images)* Create a program that lets you rotate an image through some number of degrees (out of a maximum of 360 degrees). The program should let you specify that you want to spin the image continuously. It should let you adjust the spin speed dynamically.

24.36 *(Coloring Black-and-White Photographs and Images)* Create a program that lets you paint a black-and-white photograph with color. Provide a color palette for selecting colors. Your program should let you apply different colors to different regions of the image.

24.37 *(Multimedia-Based Simpletron Simulator)* Modify the Simpletron simulator that you developed in Exercises 7.35–7.37 to include multimedia features. Add computer-like sounds to indicate that the Simpletron is executing instructions. Add a breaking-glass sound when a fatal error occurs. Use flashing lights to indicate which cells of memory or which registers are currently being manipulated. Use other multimedia techniques, as appropriate, to make your Simpletron simulator more valuable to its users as an educational tool.

Making a Difference

24.38 *(Accessibility Project: Speech Synthesis)* Computers can help people who are blind or have low vision by speaking web pages, e-mails and other documents using text-to-speech (TTS) or speech-synthesis "engines." Similarly, to help people who have difficulty interacting with a computer via the mouse and keyboard, speech-recognition engines enable computers to recognize spoken commands. With speech synthesis and speech recognition, users can "talk" with computers. In this exercise, you'll research and explore speech synthesis with the Java Speech API (`java.sun.com/products/java-media/speech/`). Download and install the open-source FreeTTS speech synthesizer (`freetts.sourceforge.net/docs/index.php`). Explore the FreeTTS documentation, then implement an application in which the user can enter text in a `JTextArea`. When the user clicks a **Speak** `JButton`, the program should use FreeTTS to speak the text aloud.

24.39 *(Accessibility Project: Speech Recognition)* In this exercise, you'll research and explore speech recognition with the Java Speech API. Download and install the open-source Sphinx-4 speech-recognition engine (`cmusphinx.sourceforge.net/sphinx4/`). Write a program that enables a user to speak to the computer. Use the speech-recognition capabilities to display what the user says in a `JTextArea`. Enable the user to save the contents of the `JTextArea` to a file on disk by speaking the command "save".

24.40 *(Project: Simbad Robotics Simulator)* Robotics holds tremendous promise for handling jobs that are dangerous for humans, such as mining coal, mining and farming the depths of the ocean and exploring deep space. Simbad (`simbad.sourceforge.net`) is an open-source Java-based 3D robotics simulator. According to the project's web page, it supports single- and multi-robot simulations; vision, range and contact sensors; and more. You can download Simbad from `simbad.sourceforge.net/index.php#download`. You'll also need to download and install Java 3D — instructions are provided on this site for Mac OS X, Windows and Linux.

Once you've downloaded Simbad and installed Java 3D, you can try the simple example provided at `simbad.sourceforge.net/example1.php`. After getting this running, read the Simbad Programming Guide at `simbad.sourceforge.net/guide.php` and try modifying the simple example to perform some different tasks. If you're interested in exploring robotics further, study the API documentation at `simbad.sourceforge.net/doc/` and create your own robot simulation program using Simbad. For example, create a simulation of a robot vacuum cleaner that travels in the direction it's facing until it encounters an obstacle, then randomly chooses another direction.

25

GUI Components: Part 2



An actor entering through the door, you've got nothing. But if he enters through the window, you've got a situation.
—Billy Wilder

...the force of events wakes slumberous talents.
—Edward Hoagland

You and I would see more interesting photography if they would stop worrying, and instead, apply horse-sense to the problem of recording the look and feel of their own era.
—Jessie Tarbox Beals

Objectives

In this chapter you'll learn:

- To create and manipulate sliders, menus, pop-up menus and windows.
- To programatically change the look-and-feel of a GUI, using Swing's pluggable look-and-feel.
- To create a multiple-document interface with `JDesktopPane` and `JInternalFrame`.
- To use additional layout managers.

Outline

25.1 Introduction
25.2 JSlider
25.3 Windows: Additional Notes
25.4 Using Menus with Frames
25.5 JPopupMenu
25.6 Pluggable Look-and-Feel
25.7 JDesktopPane and JInternalFrame
25.8 JTabbedPane
25.9 Layout Managers: BoxLayout and GridBagLayout
25.10 Wrap-Up

Summary | Self-Review Exercises | Answers to Self-Review Exercises | Exercises

25.1 Introduction

In this chapter, we continue our study of GUIs. We discuss additional components and layout managers and lay the groundwork for building more complex GUIs.

We begin our discussion with sliders that enable you to select from a range of integer values. Next, we discuss some additional details of windows. You'll learn to use menus that enable the user to effectively perform tasks in the program. The look-and-feel of a Swing GUI can be uniform across all platforms on which a Java program executes, or the GUI can be customized by using Swing's **pluggable look-and-feel** (**PLAF**). We provide an example that illustrates how to change between Swing's default metal look-and-feel (which looks and behaves the same across platforms), the Nimbus look-and-feel (introduced in Chapter 14), a look-and-feel that simulates **Motif** (a popular UNIX look-and-feel) and one that simulates Microsoft's Windows look-and-feel.

Many of today's applications use a multiple-document interface (MDI)—a main window (often called the *parent window*) containing other windows (often called *child windows*) to manage several open documents in parallel. For example, many e-mail programs allow you to have several e-mail windows open at the same time so that you can compose or read multiple e-mail messages. We demonstrate Swing's classes for creating multiple-document interfaces. The chapter finishes with a series of examples discussing additional layout managers for organizing graphical user interfaces.

Swing is a large and complex topic. There are many more GUI components and capabilities than can be presented here. Several more Swing GUI components are introduced in the remaining chapters of this book as they're needed.

25.2 JSlider

`JSliders` enable a user to select from a range of integer values. Class `JSlider` inherits from `JComponent`. Figure 25.1 shows a horizontal `JSlider` with **tick marks** and the **thumb** that allows a user to select a value. `JSliders` can be customized to display *major tick marks*, *minor tick marks* and labels for the tick marks. They also support **snap-to ticks**, which cause the *thumb*, when positioned between two tick marks, to snap to the closest one.



Fig. 25.1 | `JSlider` component with horizontal orientation.

Most Swing GUI components support user interactions through the mouse and the keyboard. For example, if a `JSlider` has the focus (i.e., it's the currently selected GUI component in the user interface), the left arrow key and right arrow key cause the thumb of the `JSlider` to decrease or increase by 1, respectively. The down arrow key and up arrow key also cause the thumb to decrease or increase by 1 tick, respectively. The *PgDn* (page down) *key* and *PgUp* (page up) *key* cause the thumb to decrease or increase by **block increments** of one-tenth of the range of values, respectively. The *Home key* moves the thumb to the minimum value of the `JSlider`, and the *End key* moves the thumb to the maximum value of the `JSlider`.

`JSlider`s have either a horizontal or a vertical orientation. For a horizontal `JSlider`, the minimum value is at the left end and the maximum is at the right end. For a vertical `JSlider`, the minimum value is at the bottom and the maximum is at the top. The minimum and maximum value positions on a `JSlider` can be reversed by invoking `JSlider` method **`setInverted`** with `boolean` argument `true`. The relative position of the thumb indicates the current value of the `JSlider`.

The program in Figs. 25.2–25.4 allows the user to size a circle drawn on a subclass of `JPanel` called `OvalPanel` (Fig. 25.2). The user specifies the circle's diameter with a horizontal `JSlider`. Class `OvalPanel` knows how to draw a circle on itself, using its own instance variable `diameter` to determine the diameter of the circle—the `diameter` is used as the width and height of the bounding box in which the circle is displayed. The `diameter` value is set when the user interacts with the `JSlider`. The event handler calls method `setDiameter` in class `OvalPanel` to set the `diameter` and calls `repaint` to draw the new circle. The `repaint` call results in a call to `OvalPanel`'s `paintComponent` method.

```
1  // Fig. 25.2: OvalPanel.java
2  // A customized JPanel class.
3  import java.awt.Graphics;
4  import java.awt.Dimension;
5  import javax.swing.JPanel;
6
7  public class OvalPanel extends JPanel
8  {
9     private int diameter = 10; // default diameter of 10
10
11    // draw an oval of the specified diameter
12    public void paintComponent( Graphics g )
13    {
14       super.paintComponent( g );
15
16       g.fillOval( 10, 10, diameter, diameter ); // draw circle
17    } // end method paintComponent
18
19    // validate and set diameter, then repaint
20    public void setDiameter( int newDiameter )
21    {
22       // if diameter invalid, default to 10
23       diameter = ( newDiameter >= 0 ? newDiameter : 10 );
```

Fig. 25.2 | `JPanel` subclass for drawing circles of a specified diameter. (Part 1 of 2.)

```
24          repaint(); // repaint panel
25       } // end method setDiameter
26
27       // used by layout manager to determine preferred size
28       public Dimension getPreferredSize()
29       {
30          return new Dimension( 200, 200 );
31       } // end method getPreferredSize
32
33       // used by layout manager to determine minimum size
34       public Dimension getMinimumSize()
35       {
36          return getPreferredSize();
37       } // end method getMinimumSize
38    } // end class OvalPanel
```

Fig. 25.2 | `JPanel` subclass for drawing circles of a specified diameter. (Part 2 of 2.)

```
1  // Fig. 25.3: SliderFrame.java
2  // Using JSliders to size an oval.
3  import java.awt.BorderLayout;
4  import java.awt.Color;
5  import javax.swing.JFrame;
6  import javax.swing.JSlider;
7  import javax.swing.SwingConstants;
8  import javax.swing.event.ChangeListener;
9  import javax.swing.event.ChangeEvent;
10
11 public class SliderFrame extends JFrame
12 {
13    private JSlider diameterJSlider; // slider to select diameter
14    private OvalPanel myPanel; // panel to draw circle
15
16    // no-argument constructor
17    public SliderFrame()
18    {
19       super( "Slider Demo" );
20
21       myPanel = new OvalPanel(); // create panel to draw circle
22       myPanel.setBackground( Color.YELLOW ); // set background to yellow
23
24       // set up JSlider to control diameter value
25       diameterJSlider =
26          new JSlider( SwingConstants.HORIZONTAL, 0, 200, 10 );
27       diameterJSlider.setMajorTickSpacing( 10 ); // create tick every 10
28       diameterJSlider.setPaintTicks( true ); // paint ticks on slider
29
30       // register JSlider event listener
31       diameterJSlider.addChangeListener(
32
33          new ChangeListener() // anonymous inner class
34          {
```

Fig. 25.3 | `JSlider` value used to determine the diameter of a circle. (Part 1 of 2.)

```
35          // handle change in slider value
36          public void stateChanged( ChangeEvent e )
37          {
38             myPanel.setDiameter( diameterJSlider.getValue() );
39          } // end method stateChanged
40       } // end anonymous inner class
41    ); // end call to addChangeListener
42
43    add( diameterJSlider, BorderLayout.SOUTH ); // add slider to frame
44    add( myPanel, BorderLayout.CENTER ); // add panel to frame
45 } // end SliderFrame constructor
46 } // end class SliderFrame
```

Fig. 25.3 | `JSlider` value used to determine the diameter of a circle. (Part 2 of 2.)

```
1  // Fig. 25.4: SliderDemo.java
2  // Testing SliderFrame.
3  import javax.swing.JFrame;
4
5  public class SliderDemo
6  {
7     public static void main( String[] args )
8     {
9        SliderFrame sliderFrame = new SliderFrame();
10       sliderFrame.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
11       sliderFrame.setSize( 220, 270 ); // set frame size
12       sliderFrame.setVisible( true ); // display frame
13    } // end main
14 } // end class SliderDemo
```

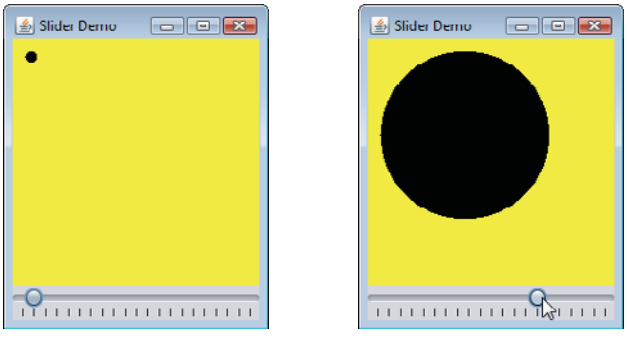


Fig. 25.4 | Test class for `SliderFrame`.

Class `OvalPanel` (Fig. 25.2) contains a `paintComponent` method (lines 12–17) that draws a filled oval (a circle in this example), a `setDiameter` method (lines 20–25) that changes the circle's `diameter` and `repaints` the `OvalPanel`, a `getPreferredSize` method (lines 28–31) that returns the preferred width and height of an `OvalPanel` and a `getMinimumSize` method (lines 34–37) that returns an `OvalPanel`'s minimum width and height. Section 24.3 introduced `getPreferredSize` and `getMinimumSize`, which are used by some layout managers to determine the size of a component.

Class SliderFrame (Fig. 25.3) creates the JSlider that controls the diameter of the circle. Class SliderFrame's constructor (lines 17–45) creates OvalPanel object myPanel (line 21) and sets its background color (line 22). Lines 25–26 create JSlider object diameterSlider to control the diameter of the circle drawn on the OvalPanel. The JSlider constructor takes four arguments. The first argument specifies the orientation of diameterSlider, which is HORIZONTAL (a constant in interface SwingConstants). The second and third arguments indicate the minimum and maximum integer values in the range of values for this JSlider. The last argument indicates that the initial value of the JSlider (i.e., where the thumb is displayed) should be 10.

Lines 27–28 customize the appearance of the JSlider. Method **setMajorTickSpacing** indicates that each major tick mark represents 10 values in the range of values supported by the JSlider. Method **setPaintTicks** with a true argument indicates that the tick marks should be displayed (they aren't displayed by default). For other methods that are used to customize a JSlider's appearance, see the JSlider on-line documentation (download.oracle.com/javase/6/docs/api/javax/swing/JSlider.html).

JSliders generate **ChangeEvents** (package javax.swing.event) in response to user interactions. An object of a class that implements interface **ChangeListener** (package javax.swing.event) and declares method **stateChanged** can respond to ChangeEvents. Lines 31–41 register a ChangeListener to handle diameterSlider's events. When method stateChanged (lines 36–39) is called in response to a user interaction, line 38 calls myPanel's setDiameter method and passes the current value of the JSlider as an argument. JSlider method **getValue** returns the current thumb position.

25.3 Windows: Additional Notes

A JFrame is a **window** with a **title bar** and a **border**. Class JFrame is a subclass of Frame (package java.awt), which is a subclass of Window (package java.awt). As such, JFrame is one of the *heavyweight* Swing GUI components. When you display a window from a Java program, the window is provided by the local platform's windowing toolkit, and therefore the window will look like every other window displayed on that platform. When a Java application executes on a Macintosh and displays a window, the window's title bar and borders will look like those of other Macintosh applications. When a Java application executes on a Microsoft Windows system and displays a window, the window's title bar and borders will look like those of other Microsoft Windows applications. And when a Java application executes on a UNIX platform and displays a window, the window's title bar and borders will look like other UNIX applications on that platform.

By default, when the user closes a JFrame window, it's hidden (i.e., removed from the screen), but you can control this with JFrame method **setDefaultCloseOperation**. Interface **WindowConstants** (package javax.swing), which class JFrame implements, declares three constants—DISPOSE_ON_CLOSE, DO_NOTHING_ON_CLOSE and HIDE_ON_CLOSE (the default)—for use with this method. Some platforms allow only a limited number of windows to be displayed on the screen. Thus, a window is a valuable resource that should be given back to the system when it's no longer needed. Class Window (an indirect superclass of JFrame) declares method **dispose** for this purpose. When a Window is no longer needed in an application, you should explicitly dispose of it. This can be done by calling the Window's dispose method or by calling method setDefaultCloseOperation with the argument WindowConstants.DISPOSE_ON_CLOSE. Terminating an application also returns

window resources to the system. Using `DO_NOTHING_ON_CLOSE` indicates that the program will determine what to do when the user attempts to close the window. For example, the program might want to ask whether to save a file's changes before closing a window.



Performance Tip 25.1

A window is an expensive system resource. Return it to the system by calling its `dispose` method when the window is no longer needed.

By default, a window is not displayed on the screen until the program invokes the window's `setVisible` method (inherited from class `java.awt.Component`) with a `true` argument. A window's size should be set with a call to method `setSize` (inherited from class `java.awt.Component`). The position of a window when it appears on the screen is specified with method **`setLocation`** (inherited from class `java.awt.Component`).



Common Programming Error 25.1

Forgetting to call method `setVisible` on a window is a runtime logic error—the window is not displayed.



Common Programming Error 25.2

Forgetting to call the `setSize` method on a window is a runtime logic error—only the title bar appears.

When the user manipulates the window, this action generates **window events**. Event listeners are registered for window events with `Window` method **`addWindowListener`**. Interface **`WindowListener`** provides seven window-event-handling methods—**`windowActivated`** (called when the user makes a window the active window), **`windowClosed`** (called after the window is closed), **`windowClosing`** (called when the user initiates closing of the window), **`windowDeactivated`** (called when the user makes another window the active window), **`windowDeiconified`** (called when the user restores a window from being minimized), **`windowIconified`** (called when the user minimizes a window) and **`windowOpened`** (called when a program first displays a window on the screen).

25.4 Using Menus with Frames

Menus are an integral part of GUIs. They allow the user to perform actions without unnecessarily cluttering a GUI with extra components. In Swing GUIs, menus can be attached only to objects of the classes that provide method **`setJMenuBar`**. Two such classes are `JFrame` and `JApplet`. The classes used to declare menus are `JMenuBar`, `JMenu`, `JMenuItem`, `JCheckBoxMenuItem` and class `JRadioButtonMenuItem`.



Look-and-Feel Observation 25.1

Menus simplify GUIs because components can be hidden within them. These components will be visible only when the user looks for them by selecting the menu.

Overview of Several Menu-Related Components

Class **`JMenuBar`** (a subclass of `JComponent`) contains the methods necessary to manage a **menu bar**, which is a container for menus. Class **`JMenu`** (a subclass of `javax.swing.JMenuItem`) contains the methods necessary for managing menus. Menus contain menu items

and are added to menu bars or to other menus as submenus. When a menu is clicked, it expands to show its list of menu items.

Class **`JMenuItem`** (a subclass of `javax.swing.AbstractButton`) contains the methods necessary to manage **menu items**. A menu item is a GUI component inside a menu that, when selected, causes an action event. A menu item can be used to initiate an action, or it can be a **submenu** that provides more menu items from which the user can select. Submenus are useful for grouping related menu items in a menu.

Class **`JCheckBoxMenuItem`** (a subclass of `javax.swing.JMenuItem`) contains the methods necessary to manage menu items that can be toggled on or off. When a `JCheckBoxMenuItem` is selected, a check appears to the left of the menu item. When the `JCheckBoxMenuItem` is selected again, the check is removed.

Class **`JRadioButtonMenuItem`** (a subclass of `javax.swing.JMenuItem`) contains the methods necessary to manage menu items that can be toggled on or off like `JCheckBoxMenuItems`. When multiple `JRadioButtonMenuItems` are maintained as part of a `ButtonGroup`, only one item in the group can be selected at a given time. When a `JRadioButtonMenuItem` is selected, a filled circle appears to the left of the menu item. When another `JRadioButtonMenuItem` is selected, the filled circle of the previously selected menu item is removed.

Using Menus in an Application

Figures 25.5–25.6 demonstrate various menu items and how to specify special characters called **mnemonics** that can provide quick access to a menu or menu item from the keyboard. Mnemonics can be used with all subclasses of `javax.swing.AbstractButton`. Class `MenuFrame` (Fig. 25.5) creates the GUI and handles the menu-item events. Most of the code in this application appears in the class's constructor (lines 34–151).

```
1  // Fig. 25.5: MenuFrame.java
2  // Demonstrating menus.
3  import java.awt.Color;
4  import java.awt.Font;
5  import java.awt.BorderLayout;
6  import java.awt.event.ActionListener;
7  import java.awt.event.ActionEvent;
8  import java.awt.event.ItemListener;
9  import java.awt.event.ItemEvent;
10 import javax.swing.JFrame;
11 import javax.swing.JRadioButtonMenuItem;
12 import javax.swing.JCheckBoxMenuItem;
13 import javax.swing.JOptionPane;
14 import javax.swing.JLabel;
15 import javax.swing.SwingConstants;
16 import javax.swing.ButtonGroup;
17 import javax.swing.JMenu;
18 import javax.swing.JMenuItem;
19 import javax.swing.JMenuBar;
20
21 public class MenuFrame extends JFrame
22 {
```

Fig. 25.5 | `JMenus` and mnemonics. (Part 1 of 5.)

```
23      private final Color[] colorValues =
24         { Color.BLACK, Color.BLUE, Color.RED, Color.GREEN };
25      private JRadioButtonMenuItem[] colorItems; // color menu items
26      private JRadioButtonMenuItem[] fonts; // font menu items
27      private JCheckBoxMenuItem[] styleItems; // font style menu items
28      private JLabel displayJLabel; // displays sample text
29      private ButtonGroup fontButtonGroup; // manages font menu items
30      private ButtonGroup colorButtonGroup; // manages color menu items
31      private int style; // used to create style for font
32
33      // no-argument constructor set up GUI
34      public MenuFrame()
35      {
36         super( "Using JMenus" );
37
38         JMenu fileMenu = new JMenu( "File" ); // create file menu
39         fileMenu.setMnemonic( 'F' ); // set mnemonic to F
40
41         // create About... menu item
42         JMenuItem aboutItem = new JMenuItem( "About..." );
43         aboutItem.setMnemonic( 'A' ); // set mnemonic to A
44         fileMenu.add( aboutItem ); // add about item to file menu
45         aboutItem.addActionListener(
46
47            new ActionListener() // anonymous inner class
48            {
49               // display message dialog when user selects About...
50               public void actionPerformed( ActionEvent event )
51               {
52                  JOptionPane.showMessageDialog( MenuFrame.this,
53                     "This is an example\nof using menus",
54                     "About", JOptionPane.PLAIN_MESSAGE );
55               } // end method actionPerformed
56            } // end anonymous inner class
57         ); // end call to addActionListener
58
59         JMenuItem exitItem = new JMenuItem( "Exit" ); // create exit item
60         exitItem.setMnemonic( 'x' ); // set mnemonic to x
61         fileMenu.add( exitItem ); // add exit item to file menu
62         exitItem.addActionListener(
63
64            new ActionListener() // anonymous inner class
65            {
66               // terminate application when user clicks exitItem
67               public void actionPerformed( ActionEvent event )
68               {
69                  System.exit( 0 ); // exit application
70               } // end method actionPerformed
71            } // end anonymous inner class
72         ); // end call to addActionListener
73
74         JMenuBar bar = new JMenuBar(); // create menu bar
75         setJMenuBar( bar ); // add menu bar to application
```

Fig. 25.5 | JMenus and mnemonics. (Part 2 of 5.)

```
76          bar.add( fileMenu ); // add file menu to menu bar
77
78          JMenu formatMenu = new JMenu( "Format" ); // create format menu
79          formatMenu.setMnemonic( 'r' ); // set mnemonic to r
80
81          // array listing string colors
82          String[] colors = { "Black", "Blue", "Red", "Green" };
83
84          JMenu colorMenu = new JMenu( "Color" ); // create color menu
85          colorMenu.setMnemonic( 'C' ); // set mnemonic to C
86
87          // create radio button menu items for colors
88          colorItems = new JRadioButtonMenuItem[ colors.length ];
89          colorButtonGroup = new ButtonGroup(); // manages colors
90          ItemHandler itemHandler = new ItemHandler(); // handler for colors
91
92          // create color radio button menu items
93          for ( int count = 0; count < colors.length; count++ )
94          {
95             colorItems[ count ] =
96                new JRadioButtonMenuItem( colors[ count ] ); // create item
97             colorMenu.add( colorItems[ count ] ); // add item to color menu
98             colorButtonGroup.add( colorItems[ count ] ); // add to group
99             colorItems[ count ].addActionListener( itemHandler );
100         } // end for
101
102         colorItems[ 0 ].setSelected( true ); // select first Color item
103
104         formatMenu.add( colorMenu ); // add color menu to format menu
105         formatMenu.addSeparator(); // add separator in menu
106
107         // array listing font names
108         String[] fontNames = { "Serif", "Monospaced", "SansSerif" };
109         JMenu fontMenu = new JMenu( "Font" ); // create font menu
110         fontMenu.setMnemonic( 'n' ); // set mnemonic to n
111
112         // create radio button menu items for font names
113         fonts = new JRadioButtonMenuItem[ fontNames.length ];
114         fontButtonGroup = new ButtonGroup(); // manages font names
115
116         // create Font radio button menu items
117         for ( int count = 0; count < fonts.length; count++ )
118         {
119            fonts[ count ] = new JRadioButtonMenuItem( fontNames[ count ] );
120            fontMenu.add( fonts[ count ] ); // add font to font menu
121            fontButtonGroup.add( fonts[ count ] ); // add to button group
122            fonts[ count ].addActionListener( itemHandler ); // add handler
123         } // end for
124
125         fonts[ 0 ].setSelected( true ); // select first Font menu item
126         fontMenu.addSeparator(); // add separator bar to font menu
127
128         String[] styleNames = { "Bold", "Italic" }; // names of styles
```

Fig. 25.5 | `JMenus` and mnemonics. (Part 3 of 5.)

```
129         styleItems = new JCheckBoxMenuItem[ styleNames.length ];
130         StyleHandler styleHandler = new StyleHandler(); // style handler
131
132         // create style checkbox menu items
133         for ( int count = 0; count < styleNames.length; count++ )
134         {
135            styleItems[ count ] =
136               new JCheckBoxMenuItem( styleNames[ count ] ); // for style
137            fontMenu.add( styleItems[ count ] ); // add to font menu
138            styleItems[ count ].addItemListener( styleHandler ); // handler
139         } // end for
140
141         formatMenu.add( fontMenu ); // add Font menu to Format menu
142         bar.add( formatMenu ); // add Format menu to menu bar
143
144         // set up label to display text
145         displayJLabel = new JLabel( "Sample Text", SwingConstants.CENTER );
146         displayJLabel.setForeground( colorValues[ 0 ] );
147         displayJLabel.setFont( new Font( "Serif", Font.PLAIN, 72 ) );
148
149         getContentPane().setBackground( Color.CYAN ); // set background
150         add( displayJLabel, BorderLayout.CENTER ); // add displayJLabel
151      } // end MenuFrame constructor
152
153      // inner class to handle action events from menu items
154      private class ItemHandler implements ActionListener
155      {
156         // process color and font selections
157         public void actionPerformed( ActionEvent event )
158         {
159            // process color selection
160            for ( int count = 0; count < colorItems.length; count++ )
161            {
162               if ( colorItems[ count ].isSelected() )
163               {
164                  displayJLabel.setForeground( colorValues[ count ] );
165                  break;
166               } // end if
167            } // end for
168
169            // process font selection
170            for ( int count = 0; count < fonts.length; count++ )
171            {
172               if ( event.getSource() == fonts[ count ] )
173               {
174                  displayJLabel.setFont(
175                     new Font( fonts[ count ].getText(), style, 72 ) );
176               } // end if
177            } // end for
178
179            repaint(); // redraw application
180         } // end method actionPerformed
181      } // end class ItemHandler
```

Fig. 25.5 | `JMenus` and mnemonics. (Part 4 of 5.)

```
182
183    // inner class to handle item events from checkbox menu items
184    private class StyleHandler implements ItemListener
185    {
186       // process font style selections
187       public void itemStateChanged( ItemEvent e )
188       {
189          String name = displayJLabel.getFont().getName(); // current Font
190          Font font; // new font based on user selections
191
192          // determine which items are checked and create Font
193          if ( styleItems[ 0 ].isSelected() &&
194               styleItems[ 1 ].isSelected() )
195             font = new Font( name, Font.BOLD + Font.ITALIC, 72 );
196          else if ( styleItems[ 0 ].isSelected() )
197             font = new Font( name, Font.BOLD, 72 );
198          else if ( styleItems[ 1 ].isSelected() )
199             font = new Font( name, Font.ITALIC, 72 );
200          else
201             font = new Font( name, Font.PLAIN, 72 );
202
203          displayJLabel.setFont( font );
204          repaint(); // redraw application
205       } // end method itemStateChanged
206    } // end class StyleHandler
207 } // end class MenuFrame
```

Fig. 25.5 | `JMenus` and mnemonics. (Part 5 of 5.)

```
1  // Fig. 25.6: MenuTest.java
2  // Testing MenuFrame.
3  import javax.swing.JFrame;
4
5  public class MenuTest
6  {
7     public static void main( String[] args )
8     {
9        MenuFrame menuFrame = new MenuFrame(); // create MenuFrame
10       menuFrame.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
11       menuFrame.setSize( 500, 200 ); // set frame size
12       menuFrame.setVisible( true ); // display frame
13    } // end main
14 } // end class MenuTest
```

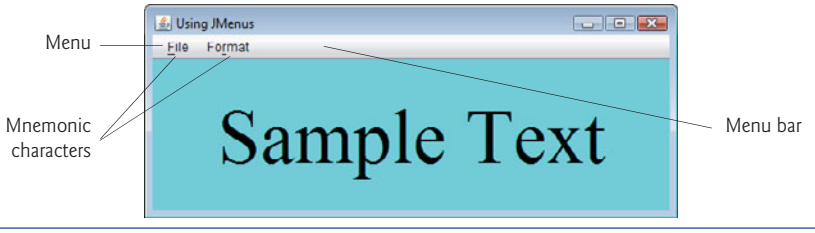


Fig. 25.6 | Test class for `MenuFrame`. (Part 1 of 2.)

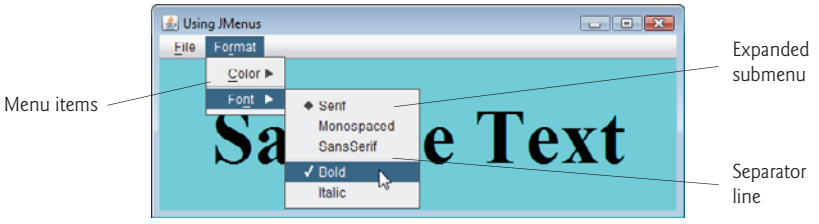


Fig. 25.6 | Test class for `MenuFrame`. (Part 2 of 2.)

Setting Up the File *Menu*

Lines 38–76 set up the **File** menu and attach it to the menu bar. The **File** menu contains an **About...** menu item that displays a message dialog when the menu item is selected and an **Exit** menu item that can be selected to terminate the application.

Line 38 creates a `JMenu` and passes to the constructor the string `"File"` as the name of the menu. Line 39 uses `JMenu` method **`setMnemonic`** (inherited from class `AbstractButton`) to indicate that `F` is the mnemonic for this menu. Pressing the *Alt* key and the letter *F* opens the menu, just as clicking the menu name with the mouse would. In the GUI, the mnemonic character in the menu's name is displayed with an underline. (See the screen captures in Fig. 25.6.)



Look-and-Feel Observation 25.2

Mnemonics provide quick access to menu commands and button commands through the keyboard.



Look-and-Feel Observation 25.3

Different mnemonics should be used for each button or menu item. Normally, the first letter in the label on the menu item or button is used as the mnemonic. If several buttons or menu items start with the same letter, choose the next most prominent letter in the name (e.g., **x** *is commonly chosen for an* **Exit** *button or menu item). Mnemonics are case insensitive.*

Lines 42–43 create `JMenuItem` `aboutItem` with the text "`About...`" and set its mnemonic to the letter `A`. This menu item is added to `fileMenu` at line 44 with `JMenu` method **`add`**. To access the **About...** menu item through the keyboard, press the *Alt* key and letter *F* to open the **File** menu, then press *A* to select the **About...** menu item. Lines 47–56 create an `ActionListener` to process `aboutItem`'s action event. Lines 52–54 display a message dialog box. In most prior uses of `showMessageDialog`, the first argument was `null`. The purpose of the first argument is to specify the **parent window** that helps determine where the dialog box will be displayed. If the parent window is specified as `null`, the dialog box appears in the center of the screen. Otherwise, it appears centered over the specified parent window. In this example, the program specifies the parent window with `MenuFrame.this`—the `this` reference of the `MenuFrame` object. When using the `this` reference in an inner class, specifying `this` by itself refers to the inner-class object. To reference the outer-class object's `this` reference, qualify `this` with the outer-class name and a dot (`.`).

Dialog boxes are typically modal. A modal dialog box does not allow any other window in the application to be accessed until the dialog box is dismissed. The dialogs dis-

played with class `JOptionPane` are modal dialogs. Class **`JDialog`** can be used to create your own modal or nonmodal dialogs.

Lines 59–72 create menu item `exitItem`, set its mnemonic to `x`, add it to `fileMenu` and register an `ActionListener` that terminates the program when the user selects `exitItem`.

Lines 74–76 create the `JMenuBar`, attach it to the window with `JFrame` method `setJMenuBar` and use `JMenuBar` method **`add`** to attach the `fileMenu` to the `JMenuBar`.



Common Programming Error 25.3

Forgetting to set the menu bar with `JFrame` method `setJMenuBar` prevents the menu bar from displaying in the `JFrame`.



Look-and-Feel Observation 25.4

Menus appear left to right in the order they're added to a `JMenuBar`.

Setting Up the Format *Menu*

Lines 78–79 create menu `formatMenu` and set its mnemonic to `r`. (`F` is not used because that is the **File** menu's mnemonic.)

Lines 84–85 create menu `colorMenu` (this will be a submenu in the **Format** menu) and set its mnemonic to `C`. Line 88 creates `JRadioButtonMenuItem` array `colorItems`, which refers to the menu items in `colorMenu`. Line 89 creates `ButtonGroup` `colorButtonGroup`, which will ensure that only one of the menu items in the **Color** submenu is selected at a time. Line 90 creates an instance of inner class `ItemHandler` (declared at lines 154–181) that responds to selections from the **Color** and **Font** submenus (discussed shortly). The `for` statement at lines 93–100 creates each `JRadioButtonMenuItem` in array `colorItems`, adds each menu item to `colorMenu` and to `colorButtonGroup` and registers the `ActionListener` for each menu item.

Line 102 invokes `AbstractButton` method **`setSelected`** to select the first element in array `colorItems`. Line 104 adds `colorMenu` as a submenu of `formatMenu`. Line 105 invokes `JMenu` method **`addSeparator`** to add a horizontal **separator** line to the menu.



Look-and-Feel Observation 25.5

A submenu is created by adding a menu as a menu item in another menu. When the mouse is positioned over a submenu (or the submenu's mnemonic is pressed), the submenu expands to show its menu items.



Look-and-Feel Observation 25.6

Separators can be added to a menu to group menu items logically.



Look-and-Feel Observation 25.7

Any lightweight GUI component (i.e., a component that is a subclass of `JComponent`) can be added to a `JMenu` or to a `JMenuBar`.

Lines 108–126 create the **Font** submenu and several `JRadioButtonMenuItems` and select the first element of `JRadioButtonMenuItem` array `fonts`. Line 129 creates a `JCheck-`

`BoxMenuItem` array to represent the menu items for specifying bold and italic styles for the fonts. Line 130 creates an instance of inner class `StyleHandler` (declared at lines 184–206) to respond to the `JCheckBoxMenuItem` events. The `for` statement at lines 133–139 creates each `JCheckBoxMenuItem`, adds it to `fontMenu` and registers its `ItemListener`. Line 141 adds `fontMenu` as a submenu of `formatMenu`. Line 142 adds the `formatMenu` to `bar` (the menu bar).

Creating the Rest of the GUI and Defining the Event Handlers

Lines 145–147 create a `JLabel` for which the **Format** menu items control the font, font color and font style. The initial foreground color is set to the first element of array `colorValues` (`Color.BLACK`) by invoking `JComponent` method **`setForeground`**. The initial font is set to `Serif` with `PLAIN` style and `72`-point size. Line 149 sets the background color of the window's content pane to cyan, and line 150 attaches the `JLabel` to the `CENTER` of the content pane's `BorderLayout`.

`ItemHandler` method `actionPerformed` (lines 157–180) uses two `for` statements to determine which font or color menu item generated the event and sets the font or color of the `JLabel displayLabel`, respectively. The `if` condition at line 162 uses `AbstractButton` method **`isSelected`** to determine the selected `JRadioButtonMenuItem`. The `if` condition at line 172 invokes the event object's `getSource` method to get a reference to the `JRadioButtonMenuItem` that generated the event. Line 175 invokes `AbstractButton` method `getText` to obtain the name of the font from the menu item.

`StyleHandler` method `itemStateChanged` (lines 187–205) is called if the user selects a `JCheckBoxMenuItem` in the `fontMenu`. Lines 193–201 determine which `JCheckBoxMenuItems` are selected and use their combined state to determine the new font style.

25.5 JPopupMenu

Many of today's computer applications provide so-called **context-sensitive pop-up menus**. In Swing, such menus are created with class **`JPopupMenu`** (a subclass of `JComponent`). These menus provide options that are specific to the component for which the **pop-up trigger event** was generated. On most systems, the pop-up trigger event occurs when the user presses and releases the right mouse button.



Look-and-Feel Observation 25.8

The pop-up trigger event is platform specific. On most platforms that use a mouse with multiple buttons, the pop-up trigger event occurs when the user clicks the right mouse button on a component that supports a pop-up menu.

The application in Figs. 25.7–25.8 creates a `JPopupMenu` that allows the user to select one of three colors and change the background color of the window. When the user clicks the right mouse button on the `PopupFrame` window's background, a `JPopupMenu` containing colors appears. If the user clicks a `JRadioButtonMenuItem` for a color, `ItemHandler` method `actionPerformed` changes the background color of the window's content pane.

Line 25 of the `PopupFrame` constructor (Fig. 25.7, lines 21–69) creates an instance of class `ItemHandler` (declared in lines 72–87) that will process the item events from the menu items in the pop-up menu. Line 29 creates the `JPopupMenu`. The `for` statement (lines 33–39) creates a `JRadioButtonMenuItem` object (line 35), adds it to `popupMenu` (line 36), adds it to `ButtonGroup colorGroup` (line 37) to maintain one selected `JRadioButton-`

`MenuItem` at a time and registers its `ActionListener` (line 38). Line 41 sets the initial background to white by invoking method `setBackground`.

```
1  // Fig. 25.7: PopupFrame.java
2  // Demonstrating JPopupMenus.
3  import java.awt.Color;
4  import java.awt.event.MouseAdapter;
5  import java.awt.event.MouseEvent;
6  import java.awt.event.ActionListener;
7  import java.awt.event.ActionEvent;
8  import javax.swing.JFrame;
9  import javax.swing.JRadioButtonMenuItem;
10 import javax.swing.JPopupMenu;
11 import javax.swing.ButtonGroup;
12
13 public class PopupFrame extends JFrame
14 {
15    private JRadioButtonMenuItem[] items; // holds items for colors
16    private final Color[] colorValues =
17       { Color.BLUE, Color.YELLOW, Color.RED }; // colors to be used
18    private JPopupMenu popupMenu; // allows user to select color
19
20    // no-argument constructor sets up GUI
21    public PopupFrame()
22    {
23       super( "Using JPopupMenus" );
24
25       ItemHandler handler = new ItemHandler(); // handler for menu items
26       String[] colors = { "Blue", "Yellow", "Red" }; // array of colors
27
28       ButtonGroup colorGroup = new ButtonGroup(); // manages color items
29       popupMenu = new JPopupMenu(); // create pop-up menu
30       items = new JRadioButtonMenuItem[ colors.length ]; // color items
31
32       // construct menu item, add to pop-up menu, enable event handling
33       for ( int count = 0; count < items.length; count++ )
34       {
35          items[ count ] = new JRadioButtonMenuItem( colors[ count ] );
36          popupMenu.add( items[ count ] ); // add item to pop-up menu
37          colorGroup.add( items[ count ] ); // add item to button group
38          items[ count ].addActionListener( handler ); // add handler
39       } // end for
40
41       setBackground( Color.WHITE ); // set background to white
42
43       // declare a MouseListener for the window to display pop-up menu
44       addMouseListener(
45
46          new MouseAdapter() // anonymous inner class
47          {
48             // handle mouse press event
49             public void mousePressed( MouseEvent event )
50             {
```

Fig. 25.7 | `JPopupMenu` for selecting colors. (Part 1 of 2.)

```
51                checkForTriggerEvent( event ); // check for trigger
52             } // end method mousePressed
53
54             // handle mouse release event
55             public void mouseReleased( MouseEvent event )
56             {
57                checkForTriggerEvent( event ); // check for trigger
58             } // end method mouseReleased
59
60             // determine whether event should trigger pop-up menu
61             private void checkForTriggerEvent( MouseEvent event )
62             {
63                if ( event.isPopupTrigger() )
64                   popupMenu.show(
65                      event.getComponent(), event.getX(), event.getY() );
66             } // end method checkForTriggerEvent
67          } // end anonymous inner class
68       ); // end call to addMouseListener
69    } // end PopupFrame constructor
70
71    // private inner class to handle menu item events
72    private class ItemHandler implements ActionListener
73    {
74       // process menu item selections
75       public void actionPerformed( ActionEvent event )
76       {
77          // determine which menu item was selected
78          for ( int i = 0; i < items.length; i++ )
79          {
80             if ( event.getSource() == items[ i ] )
81             {
82                getContentPane().setBackground( colorValues[ i ] );
83                return;
84             } // end if
85          } // end for
86       } // end method actionPerformed
87    } // end private inner class ItemHandler
88 } // end class PopupFrame
```

Fig. 25.7 | `JPopupMenu` for selecting colors. (Part 2 of 2.)

```
1  // Fig. 25.8: PopupTest.java
2  // Testing PopupFrame.
3  import javax.swing.JFrame;
4
5  public class PopupTest
6  {
7     public static void main( String[] args )
8     {
9        PopupFrame popupFrame = new PopupFrame(); // create PopupFrame
10       popupFrame.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
```

Fig. 25.8 | Test class for `PopupFrame`. (Part 1 of 2.)

```
11          popupFrame.setSize( 300, 200 ); // set frame size
12          popupFrame.setVisible( true ); // display frame
13       } // end main
14    } // end class PopupTest
```

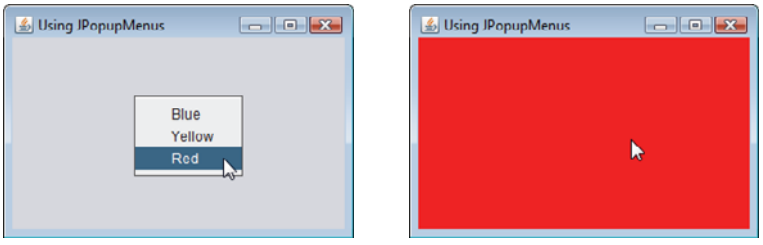


Fig. 25.8 | Test class for `PopupFrame`. (Part 2 of 2.)

Lines 44–68 register a `MouseListener` to handle the mouse events of the application window. Methods `mousePressed` (lines 49–52) and `mouseReleased` (lines 55–58) check for the pop-up trigger event. Each method calls private utility method `checkForTriggerEvent` (lines 61–66) to determine whether the pop-up trigger event occurred. If it did, `MouseEvent` method **`isPopupTrigger`** returns `true`, and `JPopupMenu` method **`show`** displays the `JPopupMenu`. The first argument to method `show` specifies the **origin component**, whose position helps determine where the `JPopupMenu` will appear on the screen. The last two arguments are the *x*-*y* coordinates (measured from the origin component's upper-left corner) at which the `JPopupMenu` is to appear.



Look-and-Feel Observation 25.9

Displaying a `JPopupMenu` for the pop-up trigger event of multiple GUI components requires registering mouse-event handlers for each of those GUI components.

When the user selects a menu item from the pop-up menu, class `ItemHandler`'s method `actionPerformed` (lines 75–86) determines which `JRadioButtonMenuItem` the user selected and sets the background color of the window's content pane.

25.6 Pluggable Look-and-Feel

A program that uses Java's AWT GUI components (package `java.awt`) takes on the look-and-feel of the platform on which the program executes. A Java application running on a Mac OS X looks like other Mac OS X applications. One running on Microsoft Windows looks like other Windows applications. One running on a Linux platform looks like other applications on that Linux platform. This is sometimes desirable, because it allows users of the application on each platform to use GUI components with which they're already familiar. However, it also introduces interesting portability issues.



Portability Tip 25.1

GUI components look different on different platforms and may require different amounts of space to display. This could change their layout and alignments.



Portability Tip 25.2

GUI components on different platforms have different default functionality (e.g., some platforms allow a button with the focus to be "pressed" with the space bar, and some don't).

Swing's lightweight GUI components eliminate many of these issues by providing uniform functionality across platforms and by defining a uniform cross-platform look-and-feel. Recent versions of Java SE 6 and the upcoming Java SE 7 include the *Nimbus* look-and-feel that we discussed in Section 14.2. Earlier versions of Java used the **metal look-and-feel**, which is still the default. Swing also provides the flexibility to customize the look-and-feel to appear as a Microsoft Windows-style look-and-feel (only on Window systems), a Motif-style (UNIX) look-and-feel (across all platforms) or a Macintosh look-and-feel (only on Mac systems).

Figures 25.9–25.10 demonstrate a way to change the look-and-feel of a Swing GUI. It creates several GUI components, so you can see the change in their look-and-feel at the same time. The output windows show the Metal, Nimbus, CDE/Motif, Windows and Windows Classic look-and-feels that are available on Windows systems. The installed look-and-feels will vary by platform.

```
1  // Fig. 25.9: LookAndFeelFrame.java
2  // Changing the look-and-feel.
3  import java.awt.GridLayout;
4  import java.awt.BorderLayout;
5  import java.awt.event.ItemListener;
6  import java.awt.event.ItemEvent;
7  import javax.swing.JFrame;
8  import javax.swing.UIManager;
9  import javax.swing.JRadioButton;
10 import javax.swing.ButtonGroup;
11 import javax.swing.JButton;
12 import javax.swing.JLabel;
13 import javax.swing.JComboBox;
14 import javax.swing.JPanel;
15 import javax.swing.SwingConstants;
16 import javax.swing.SwingUtilities;
17
18 public class LookAndFeelFrame extends JFrame
19 {
20    private UIManager.LookAndFeelInfo[] looks; // look and feels
21    private String[] lookNames; // names of look and feels
22    private JRadioButton[] radio; // radio buttons to select look-and-feel
23    private ButtonGroup group; // group for radio buttons
24    private JButton button; // displays look of button
25    private JLabel label; // displays look of label
26    private JComboBox comboBox; // displays look of combo box
27
28    // set up GUI
29    public LookAndFeelFrame()
30    {
```

Fig. 25.9 | Look-and-feel of a Swing-based GUI. (Part 1 of 3.)

```
31          super( "Look and Feel Demo" );
32
33          // get installed look-and-feel information
34          looks = UIManager.getInstalledLookAndFeels();
35          lookNames = new String[ looks.length ];
36
37          // get names of installed look-and-feels
38          for ( int i = 0; i < looks.length; i++ )
39             lookNames[ i ] = looks[ i ].getName();
40
41          JPanel northPanel = new JPanel(); // create north panel
42          northPanel.setLayout( new GridLayout( 3, 1, 0, 5 ) );
43
44          label = new JLabel( "This is a " + lookNames[0] + " look-and-feel",
45             SwingConstants.CENTER ); // create label
46          northPanel.add( label ); // add label to panel
47
48          button = new JButton( "JButton" ); // create button
49          northPanel.add( button ); // add button to panel
50
51          comboBox = new JComboBox( lookNames ); // create combobox
52          northPanel.add( comboBox ); // add combobox to panel
53
54          // create array for radio buttons
55          radio = new JRadioButton[ looks.length ];
56
57          JPanel southPanel = new JPanel(); // create south panel
58
59          // use a GridLayout with 3 buttons in each row
60          int rows = (int) Math.ceil( radio.length / 3.0 );
61          southPanel.setLayout( new GridLayout( rows, 3 ) );
62
63          group = new ButtonGroup(); // button group for looks-and-feels
64          ItemHandler handler = new ItemHandler(); // look-and-feel handler
65
66          for ( int count = 0; count < radio.length; count++ )
67          {
68             radio[ count ] = new JRadioButton( lookNames[ count ] );
69             radio[ count ].addItemListener( handler ); // add handler
70             group.add( radio[ count ] ); // add radio button to group
71             southPanel.add( radio[ count ] ); // add radio button to panel
72          } // end for
73
74          add( northPanel, BorderLayout.NORTH ); // add north panel
75          add( southPanel, BorderLayout.SOUTH ); // add south panel
76
77          radio[ 0 ].setSelected( true ); // set default selection
78       } // end LookAndFeelFrame constructor
79
80       // use UIManager to change look-and-feel of GUI
81       private void changeTheLookAndFeel( int value )
82       {
```

Fig. 25.9 | Look-and-feel of a Swing-based GUI. (Part 2 of 3.)

```
83          try // change look-and-feel
84          {
85             // set look-and-feel for this application
86             UIManager.setLookAndFeel( looks[ value ].getClassName() );
87
88             // update components in this application
89             SwingUtilities.updateComponentTreeUI( this );
90          } // end try
91          catch ( Exception exception )
92          {
93             exception.printStackTrace();
94          } // end catch
95       } // end method changeTheLookAndFeel
96
97       // private inner class to handle radio button events
98       private class ItemHandler implements ItemListener
99       {
100         // process user's look-and-feel selection
101         public void itemStateChanged( ItemEvent event )
102         {
103            for ( int count = 0; count < radio.length; count++ )
104            {
105               if ( radio[ count ].isSelected() )
106               {
107                  label.setText( String.format(
108                     "This is a %s look-and-feel", lookNames[ count ] ) );
109                  comboBox.setSelectedIndex( count ); // set combobox index
110                  changeTheLookAndFeel( count ); // change look-and-feel
111               } // end if
112            } // end for
113         } // end method itemStateChanged
114      } // end private inner class ItemHandler
115   } // end class LookAndFeelFrame
```

Fig. 25.9 | Look-and-feel of a Swing-based GUI. (Part 3 of 3.)

```
1  // Fig. 25.10: LookAndFeelDemo.java
2  // Changing the look-and-feel.
3  import javax.swing.JFrame;
4
5  public class LookAndFeelDemo
6  {
7     public static void main( String[] args )
8     {
9        LookAndFeelFrame lookAndFeelFrame = new LookAndFeelFrame();
10       lookAndFeelFrame.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
11       lookAndFeelFrame.setSize( 400, 220 ); // set frame size
12       lookAndFeelFrame.setVisible( true ); // display frame
13    } // end main
14 } // end class LookAndFeelDemo
```

Fig. 25.10 | Test class for `LookAndFeelFrame`. (Part 1 of 2.)

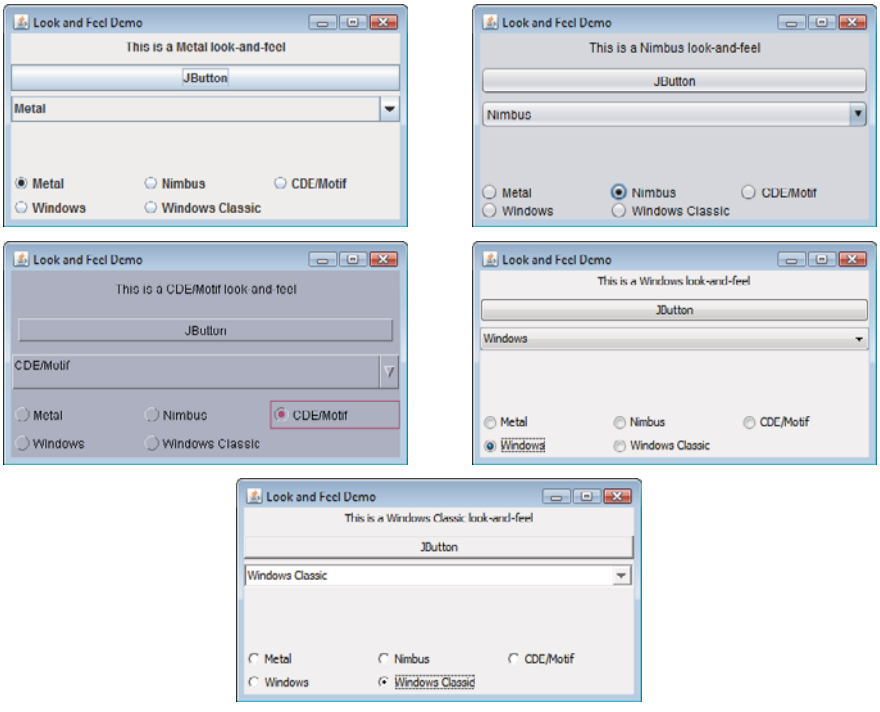


Fig. 25.10 | Test class for `LookAndFeelFrame`. (Part 2 of 2.)

We've covered the GUI components and event-handling concepts in this example previously, so we focus here on the mechanism for changing the look-and-feel. Class **`UIManager`** (package `javax.swing`) contains nested class **`LookAndFeelInfo`** (a `public static` class) that maintains information about a look-and-feel. Line 20 declares an array of type `UIManager.LookAndFeelInfo` (note the syntax used to identify the `static` inner class `LookAndFeelInfo`). Line 34 uses `UIManager` `static` method **`getInstalledLookAndFeels`** to get the array of `UIManager.LookAndFeelInfo` objects that describe each look-and-feel available on your system.



Performance Tip 25.2

Each look-and-feel is represented by a Java class. `UIManager` method `getInstalledLookAndFeels` does not load each class. Rather, it provides the names of the available look-and-feel classes so that a choice can be made (presumably once at program start-up). This reduces the overhead of having to load all the look-and-feel classes even if the program will not use some of them.

Our utility method `changeTheLookAndFeel` (lines 81–95) is called by the event handler for the `JRadioButtons` at the bottom of the user interface. The event handler (declared in `private` inner class `ItemHandler` at lines 98–114) passes an integer representing the element in array `looks` that should be used to change the look-and-feel. Line 86 invokes `static` method **`setLookAndFeel`** of `UIManager` to change the look-and-feel. Method **`getClassName`** of class `UIManager.LookAndFeelInfo` determines the name of the look-and-

feel class that corresponds to the `UIManager.LookAndFeelInfo` object. If the look-and-feel class is not already loaded, it will be loaded as part of the call to `setLookAndFeel`. Line 89 invokes `static` method **`updateComponentTreeUI`** of class **`SwingUtilities`** (package `javax.swing`) to change the look-and-feel of every GUI component attached to its argument (`this` instance of our application class `LookAndFeelFrame`) to the new look-and-feel.

25.7 JDesktopPane and JInternalFrame

Many of today's applications use a **multiple-document interface** (**MDI**)—a main window (called the **parent window**) containing other windows (called **child windows**), to manage several open documents that are being processed in parallel. For example, many e-mail programs allow you to have several windows open at the same time, so you can compose or read multiple e-mail messages simultaneously. Similarly, many word processors allow the user to open multiple documents in separate windows within a main window, making it possible to switch between them without having to close one to open another. The application in Figs. 25.11–25.12 demonstrates Swing's **`JDesktopPane`** and **`JInternalFrame`** classes for implementing multiple-document interfaces.

```
1  // Fig. 25.11: DesktopFrame.java
2  // Demonstrating JDesktopPane.
3  import java.awt.BorderLayout;
4  import java.awt.Dimension;
5  import java.awt.Graphics;
6  import java.awt.event.ActionListener;
7  import java.awt.event.ActionEvent;
8  import java.util.Random;
9  import javax.swing.JFrame;
10 import javax.swing.JDesktopPane;
11 import javax.swing.JMenuBar;
12 import javax.swing.JMenu;
13 import javax.swing.JMenuItem;
14 import javax.swing.JInternalFrame;
15 import javax.swing.JPanel;
16 import javax.swing.ImageIcon;
17
18 public class DesktopFrame extends JFrame
19 {
20    private JDesktopPane theDesktop;
21
22    // set up GUI
23    public DesktopFrame()
24    {
25       super( "Using a JDesktopPane" );
26
27       JMenuBar bar = new JMenuBar(); // create menu bar
28       JMenu addMenu = new JMenu( "Add" ); // create Add menu
29       JMenuItem newFrame = new JMenuItem( "Internal Frame" );
30
31       addMenu.add( newFrame ); // add new frame item to Add menu
32       bar.add( addMenu ); // add Add menu to menu bar
```

Fig. 25.11 | Multiple-document interface. (Part 1 of 3.)

```
33         setJMenuBar( bar ); // set menu bar for this application
34
35         theDesktop = new JDesktopPane(); // create desktop pane
36         add( theDesktop ); // add desktop pane to frame
37
38         // set up listener for newFrame menu item
39         newFrame.addActionListener(
40
41            new ActionListener() // anonymous inner class
42            {
43               // display new internal window
44               public void actionPerformed( ActionEvent event )
45               {
46                  // create internal frame
47                  JInternalFrame frame = new JInternalFrame(
48                     "Internal Frame", true, true, true, true );
49
50                  MyJPanel panel = new MyJPanel(); // create new panel
51                  frame.add( panel, BorderLayout.CENTER ); // add panel
52                  frame.pack(); // set internal frame to size of contents
53
54                  theDesktop.add( frame ); // attach internal frame
55                  frame.setVisible( true ); // show internal frame
56               } // end method actionPerformed
57            } // end anonymous inner class
58         ); // end call to addActionListener
59      } // end DesktopFrame constructor
60   } // end class DesktopFrame
61
62   // class to display an ImageIcon on a panel
63   class MyJPanel extends JPanel
64   {
65      private static Random generator = new Random();
66      private ImageIcon picture; // image to be displayed
67      private final static String[] images = { "yellowflowers.png",
68         "purpleflowers.png", "redflowers.png", "redflowers2.png",
69         "lavenderflowers.png" };
70
71      // load image
72      public MyJPanel()
73      {
74         int randomNumber = generator.nextInt( images.length );
75         picture = new ImageIcon( images[ randomNumber ] ); // set icon
76      } // end MyJPanel constructor
77
78      // display imageIcon on panel
79      public void paintComponent( Graphics g )
80      {
81         super.paintComponent( g );
82         picture.paintIcon( this, g, 0, 0 ); // display icon
83      } // end method paintComponent
84
```

Fig. 25.11 | Multiple-document interface. (Part 2 of 3.)

```
85      // return image dimensions
86      public Dimension getPreferredSize()
87      {
88         return new Dimension( picture.getIconWidth(),
89            picture.getIconHeight() );
90      } // end method getPreferredSize
91   } // end class MyJPanel
```

Fig. 25.11 | Multiple-document interface. (Part 3 of 3.)

Lines 27–33 create a `JMenuBar`, a `JMenu` and a `JMenuItem`, add the `JMenuItem` to the `JMenu`, add the `JMenu` to the `JMenuBar` and set the `JMenuBar` for the application window. When the user selects the `JMenuItem` `newFrame`, the application creates and displays a new `JInternalFrame` object containing an image.

Line 35 assigns `JDesktopPane` (package `javax.swing`) variable `theDesktop` a new `JDesktopPane` object that will be used to manage the `JInternalFrame` child windows. Line 36 adds the `JDesktopPane` to the `JFrame`. By default, the `JDesktopPane` is added to the center of the content pane's `BorderLayout`, so the `JDesktopPane` expands to fill the entire application window.

Lines 39–58 register an `ActionListener` to handle the event when the user selects the `newFrame` menu item. When the event occurs, method `actionPerformed` (lines 44–56) creates a `JInternalFrame` object in lines 47–48. The `JInternalFrame` constructor used here takes five arguments—a `String` for the title bar of the internal window, a `boolean` indicating whether the internal frame can be resized by the user, a `boolean` indicating whether the internal frame can be closed by the user, a `boolean` indicating whether the internal frame can be maximized by the user and a `boolean` indicating whether the internal frame can be minimized by the user. For each of the `boolean` arguments, a `true` value indicates that the operation should be allowed (as is the case here).

As with `JFrames` and `JApplets`, a `JInternalFrame` has a content pane to which GUI components can be attached. Line 50 (Fig. 25.11) creates an instance of our class `MyJPanel` (declared at lines 63–91) that is added to the `JInternalFrame` at line 51.

```
1  // Fig. 25.12: DesktopTest.java
2  // Demonstrating JDesktopPane.
3  import javax.swing.JFrame;
4
5  public class DesktopTest
6  {
7     public static void main( String[] args )
8     {
9        DesktopFrame desktopFrame = new DesktopFrame();
10       desktopFrame.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
11       desktopFrame.setSize( 600, 480 ); // set frame size
12       desktopFrame.setVisible( true ); // display frame
13    } // end main
14 } // end class DesktopTest
```

Fig. 25.12 | Test class for `DeskTopFrame`. (Part 1 of 2.)

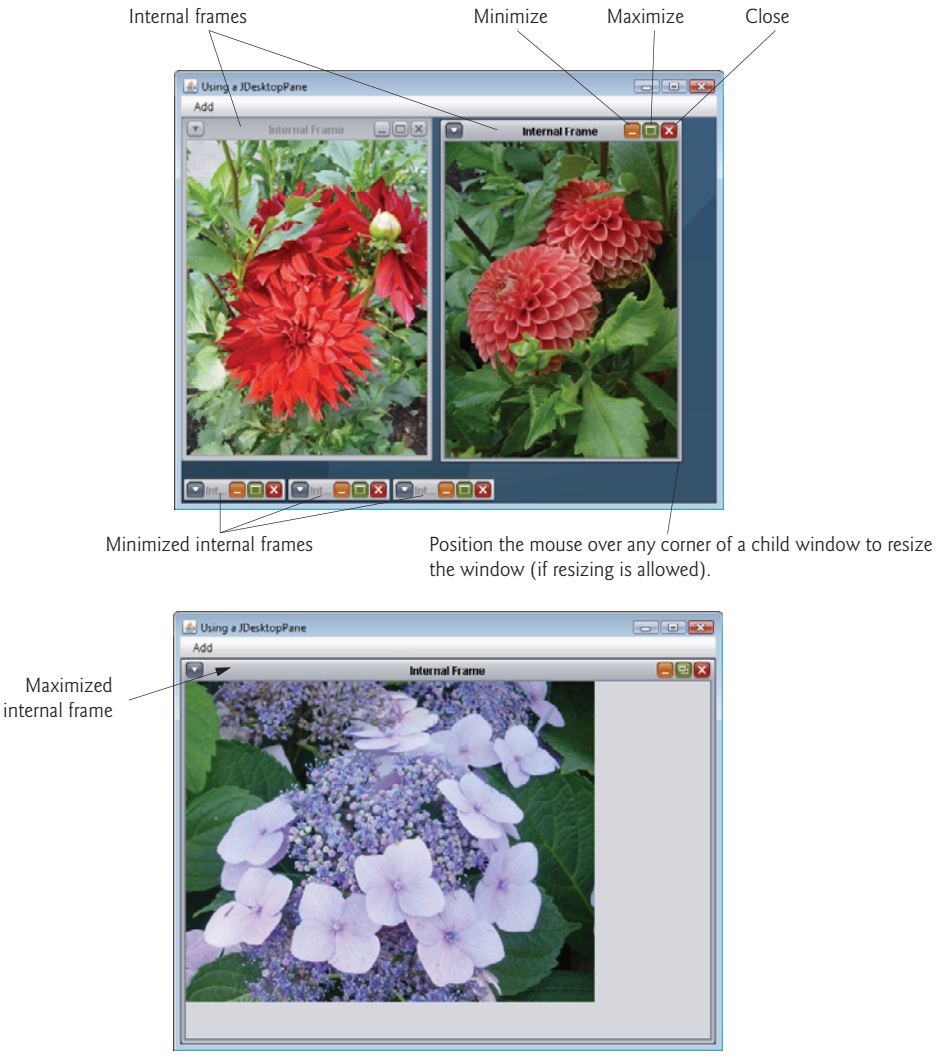


Fig. 25.12 | Test class for `DeskTopFrame`. (Part 2 of 2.)

Line 52 uses `JInternalFrame` method **`pack`** to set the size of the child window. Method `pack` uses the preferred sizes of the components to determine the window's size. Class `MyJPanel` declares method `getPreferredSize` (lines 86–90) to specify the panel's preferred size for use by the `pack` method. Line 54 adds the `JInternalFrame` to the `JDesktopPane`, and line 55 displays the `JInternalFrame`.

Classes `JInternalFrame` and `JDesktopPane` provide many methods for managing child windows. See the `JInternalFrame` and `JDesktopPane` online API documentation for complete lists of these methods:

```
download.oracle.com/javase/6/docs/api/javax/swing/JInternalFrame.html
download.oracle.com/javase/6/docs/api/javax/swing/JDesktopPane.html
```

25.8 JTabbedPane

A **JTabbedPane** arranges GUI components into layers, of which only one is visible at a time. Users access each layer via a tab—similar to folders in a file cabinet. When the user clicks a tab, the appropriate layer is displayed. The tabs appear at the top by default but also can be positioned at the left, right or bottom of the `JTabbedPane`. Any component can be placed on a tab. If the component is a container, such as a panel, it can use any layout manager to lay out several components on the tab. Class `JTabbedPane` is a subclass of `JComponent`. The application in Figs. 25.13–25.14 creates one tabbed pane with three tabs. Each tab displays one of the `JPanel`s—`panel1`, `panel2` or `panel3`.

```
1  // Fig. 25.13: JTabbedPaneFrame.java
2  // Demonstrating JTabbedPane.
3  import java.awt.BorderLayout;
4  import java.awt.Color;
5  import javax.swing.JFrame;
6  import javax.swing.JTabbedPane;
7  import javax.swing.JLabel;
8  import javax.swing.JPanel;
9  import javax.swing.JButton;
10 import javax.swing.SwingConstants;
11
12 public class JTabbedPaneFrame extends JFrame
13 {
14    // set up GUI
15    public JTabbedPaneFrame()
16    {
17       super( "JTabbedPane Demo " );
18
19       JTabbedPane tabbedPane = new JTabbedPane(); // create JTabbedPane
20
21       // set up pane11 and add it to JTabbedPane
22       JLabel label1 = new JLabel( "panel one", SwingConstants.CENTER );
23       JPanel panel1 = new JPanel(); // create first panel
24       panel1.add( label1 ); // add label to panel
25       tabbedPane.addTab( "Tab One", null, panel1, "First Panel" );
26
27       // set up panel2 and add it to JTabbedPane
28       JLabel label2 = new JLabel( "panel two", SwingConstants.CENTER );
29       JPanel panel2 = new JPanel(); // create second panel
30       panel2.setBackground( Color.YELLOW ); // set background to yellow
31       panel2.add( label2 ); // add label to panel
32       tabbedPane.addTab( "Tab Two", null, panel2, "Second Panel" );
33
34       // set up panel3 and add it to JTabbedPane
35       JLabel label3 = new JLabel( "panel three" );
36       JPanel panel3 = new JPanel(); // create third panel
37       panel3.setLayout( new BorderLayout() ); // use borderlayout
38       panel3.add( new JButton( "North" ), BorderLayout.NORTH );
39       panel3.add( new JButton( "West" ), BorderLayout.WEST );
40       panel3.add( new JButton( "East" ), BorderLayout.EAST );
```

Fig. 25.13 | `JTabbedPane` used to organize GUI components. (Part 1 of 2.)

```
41          panel3.add( new JButton( "South" ), BorderLayout.SOUTH );
42          panel3.add( label3, BorderLayout.CENTER );
43          tabbedPane.addTab( "Tab Three", null, panel3, "Third Panel" );
44
45          add( tabbedPane ); // add JTabbedPane to frame
46       } // end JTabbedPaneFrame constructor
47    } // end class JTabbedPaneFrame
```

Fig. 25.13 | `JTabbedPane` used to organize GUI components. (Part 2 of 2.)

```
1  // Fig. 25.14: JTabbedPaneDemo.java
2  // Demonstrating JTabbedPane.
3  import javax.swing.JFrame;
4
5  public class JTabbedPaneDemo
6  {
7     public static void main( String[] args )
8     {
9        JTabbedPaneFrame tabbedPaneFrame = new JTabbedPaneFrame();
10       tabbedPaneFrame.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
11       tabbedPaneFrame.setSize( 250, 200 ); // set frame size
12       tabbedPaneFrame.setVisible( true ); // display frame
13    } // end main
14 } // end class JTabbedPaneDemo
```

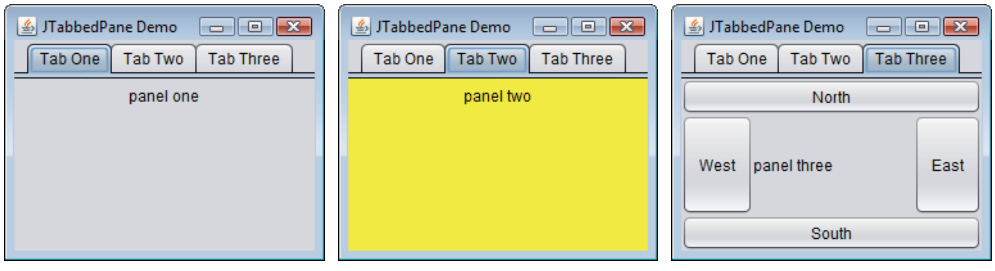


Fig. 25.14 | Test class for `JTabbedPaneFrame`.

The constructor (lines 15–46) builds the GUI. Line 19 creates an empty `JTabbedPane` with default settings—that is, tabs across the top. If the tabs do not fit on one line, they'll wrap to form additional lines of tabs. Next the constructor creates the `JPanel`s `panel1`, `panel2` and `panel3` and their GUI components. As we set up each panel, we add it to `tabbedPane`, using `JTabbedPane` method **`addTab`** with four arguments. The first argument is a `String` that specifies the title of the tab. The second argument is an `Icon` reference that specifies an icon to display on the tab. If the `Icon` is a `null` reference, no image is displayed. The third argument is a `Component` reference that represents the GUI component to display when the user clicks the tab. The last argument is a `String` that specifies the tool tip for the tab. For example, line 25 adds `JPanel` `panel1` to `tabbedPane` with title `"Tab One"` and the tool tip `"First Panel"`. `JPanel`s `panel2` and `panel3` are added to `tabbedPane` at lines 32 and 43. To view a tab, click it with the mouse or use the arrow keys to cycle through the tabs.

25.9 Layout Managers: BoxLayout and GridBagLayout

In Chapter 14, we introduced three layout managers—`FlowLayout`, `BorderLayout` and `GridLayout`. This section presents two additional layout managers (summarized in Fig. 25.15). We discuss them in the examples that follow. We also discuss the extremely flexible `GroupLayout` in Appendix I.

Layout manager	Description
`BoxLayout`	A layout manager that allows GUI components to be arranged left-to-right or top-to-bottom in a container. Class `Box` declares a container with `BoxLayout` as its default layout manager and provides `static` methods to create a `Box` with a horizontal or vertical `BoxLayout`.
`GridBagLayout`	A layout manager similar to `GridLayout`, but the components can vary in size and can be added in any order.

Fig. 25.15 | Additional layout managers.

`BoxLayout` *Layout Manager*

The `BoxLayout` layout manager (in package `javax.swing`) arranges GUI components horizontally along a container's *x*-axis or vertically along its *y*-axis. The application in Figs. 25.16–25.17 demonstrates `BoxLayout` and the container class `Box` that uses `BoxLayout` as its default layout manager.

```
1  // Fig. 25.16: BoxLayoutFrame.java
2  // Demonstrating BoxLayout.
3  import java.awt.Dimension;
4  import javax.swing.JFrame;
5  import javax.swing.Box;
6  import javax.swing.JButton;
7  import javax.swing.BoxLayout;
8  import javax.swing.JPanel;
9  import javax.swing.JTabbedPane;
10
11 public class BoxLayoutFrame extends JFrame
12 {
13    // set up GUI
14    public BoxLayoutFrame()
15    {
16       super( "Demonstrating BoxLayout" );
17
18       // create Box containers with BoxLayout
19       Box horizontal1 = Box.createHorizontalBox();
20       Box vertical1 = Box.createVerticalBox();
21       Box horizontal2 = Box.createHorizontalBox();
22       Box vertical2 = Box.createVerticalBox();
23
24       final int SIZE = 3; // number of buttons on each Box
25
```

Fig. 25.16 | `BoxLayout` layout manager. (Part 1 of 2.)

```
26          // add buttons to Box horizontal1
27          for ( int count = 0; count < SIZE; count++ )
28             horizontal1.add( new JButton( "Button " + count ) );
29
30          // create strut and add buttons to Box vertical1
31          for ( int count = 0; count < SIZE; count++ )
32          {
33             vertical1.add( Box.createVerticalStrut( 25 ) );
34             vertical1.add( new JButton( "Button " + count ) );
35          } // end for
36
37          // create horizontal glue and add buttons to Box horizontal2
38          for ( int count = 0; count < SIZE; count++ )
39          {
40             horizontal2.add( Box.createHorizontalGlue() );
41             horizontal2.add( new JButton( "Button " + count ) );
42          } // end for
43
44          // create rigid area and add buttons to Box vertical2
45          for ( int count = 0; count < SIZE; count++ )
46          {
47             vertical2.add( Box.createRigidArea( new Dimension( 12, 8 ) ) );
48             vertical2.add( new JButton( "Button " + count ) );
49          } // end for
50
51          // create vertical glue and add buttons to panel
52          JPanel panel = new JPanel();
53          panel.setLayout( new BoxLayout( panel, BoxLayout.Y_AXIS ) );
54
55          for ( int count = 0; count < SIZE; count++ )
56          {
57             panel.add( Box.createGlue() );
58             panel.add( new JButton( "Button " + count ) );
59          } // end for
60
61          // create a JTabbedPane
62          JTabbedPane tabs = new JTabbedPane(
63             JTabbedPane.TOP, JTabbedPane.SCROLL_TAB_LAYOUT );
64
65          // place each container on tabbed pane
66          tabs.addTab( "Horizontal Box", horizontal1 );
67          tabs.addTab( "Vertical Box with Struts", vertical1 );
68          tabs.addTab( "Horizontal Box with Glue", horizontal2 );
69          tabs.addTab( "Vertical Box with Rigid Areas", vertical2 );
70          tabs.addTab( "Vertical Box with Glue", panel );
71
72          add( tabs ); // place tabbed pane on frame
73       } // end BoxLayoutFrame constructor
74    } // end class BoxLayoutFrame
```

Fig. 25.16 | BoxLayout layout manager. (Part 2 of 2.)

Lines 19–22 create Box containers. References horizontal1 and horizontal2 are initialized with static Box method createHorizontalBox, which returns a Box container

```
1  // Fig. 25.17: BoxLayoutDemo.java
2  // Demonstrating BoxLayout.
3  import javax.swing.JFrame;
4
5  public class BoxLayoutDemo
6  {
7     public static void main( String[] args )
8     {
9        BoxLayoutFrame boxLayoutFrame = new BoxLayoutFrame();
10       boxLayoutFrame.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
11       boxLayoutFrame.setSize( 400, 220 ); // set frame size
12       boxLayoutFrame.setVisible( true ); // display frame
13    } // end main
14 } // end class BoxLayoutDemo
```



Fig. 25.17 | Test class for `BoxLayoutFrame`.

with a horizontal `BoxLayout` in which GUI components are arranged left-to-right. Variables `vertical1` and `vertical2` are initialized with `static` `Box` method **`createVerticalBox`**, which returns references to `Box` containers with a vertical `BoxLayout` in which GUI components are arranged top-to-bottom.

The loop at lines 27–28 adds three `JButtons` to `horizontal1`. The `for` statement at lines 31–35 adds three `JButtons` to `vertical1`. Before adding each button, line 33 adds a **vertical strut** to the container with `static` `Box` method **`createVerticalStrut`**. A vertical strut is an invisible GUI component that has a fixed pixel height and is used to guarantee a fixed amount of space between GUI components. The `int` argument to method `createVerticalStrut` determines the height of the strut in pixels. When the container is resized,

the distance between GUI components separated by struts does not change. Class `Box` also declares method **`createHorizontalStrut`** for horizontal `BoxLayouts`.

The `for` statement at lines 38–42 adds three `JButtons` to `horizontal2`. Before adding each button, line 40 adds **horizontal glue** to the container with `static Box` method **`createHorizontalGlue`**. Horizontal glue is an invisible GUI component that can be used between fixed-size GUI components to occupy additional space. Normally, extra space appears to the right of the last horizontal GUI component or below the last vertical one in a `BoxLayout`. Glue allows the extra space to be placed between GUI components. When the container is resized, components separated by glue components remain the same size, but the glue stretches or contracts to occupy the space between them. Class `Box` also declares method **`createVerticalGlue`** for vertical `BoxLayouts`.

The `for` statement at lines 45–49 adds three `JButtons` to `vertical2`. Before each button is added, line 47 adds a **rigid area** to the container with `static Box` method **`createRigidArea`**. A rigid area is an invisible GUI component that always has a fixed pixel width and height. The argument to method `createRigidArea` is a `Dimension` object that specifies the area's width and height.

Lines 52–53 create a `JPanel` object and set its layout to a `BoxLayout` in the conventional manner, using `Container` method `setLayout`. The `BoxLayout` constructor receives a reference to the container for which it controls the layout and a constant indicating whether the layout is horizontal (**`BoxLayout.X_AXIS`**) or vertical (**`BoxLayout.Y_AXIS`**).

The `for` statement at lines 55–59 adds three `JButtons` to `panel`. Before adding each button, line 57 adds a glue component to the container with `static Box` method **`createGlue`**. This component expands or contracts based on the size of the `Box`.

Lines 62–63 create a `JTabbedPane` to display the five containers in this program. The argument **`JTabbedPane.TOP`** sent to the constructor indicates that the tabs should appear at the top of the `JTabbedPane`. The argument **`JTabbedPane.SCROLL_TAB_LAYOUT`** specifies that the tabs should wrap to a new line if there are too many to fit on one line.

The `Box` containers and the `JPanel` are attached to the `JTabbedPane` at lines 66–70. Try executing the application. When the window appears, resize the window to see how the glue components, strut components and rigid area affect the layout on each tab.

`GridBagLayout` *Layout Manager*

One of the most powerful predefined layout managers is **`GridBagLayout`** (in package `java.awt`). This layout is similar to `GridLayout` in that it arranges components in a grid, but it's more flexible. The components can vary in size (i.e., they can occupy multiple rows and columns) and can be added in any order.

The first step in using `GridBagLayout` is determining the appearance of the GUI. For this step you need only a piece of paper. Draw the GUI, then draw a grid over it, dividing the components into rows and columns. The initial row and column numbers should be 0, so that the `GridBagLayout` layout manager can use the row and column numbers to properly place the components in the grid. Figure 25.18 demonstrates drawing the lines for the rows and columns over a GUI.

A **`GridBagConstraints`** object describes how a component is placed in a `GridBagLayout`. Several `GridBagConstraints` fields are summarized in Fig. 25.19.

`GridBagConstraints` field **`anchor`** specifies the relative position of the component in an area that it does not fill. The variable `anchor` is assigned one of the following `GridBag-`

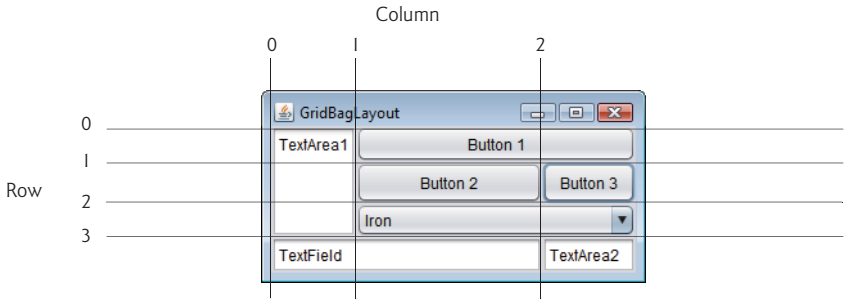


Fig. 25.18 | Designing a GUI that will use `GridBagLayout`.

Field	Description
`anchor`	Specifies the relative position (`NORTH`, `NORTHEAST`, `EAST`, `SOUTHEAST`, `SOUTH`, `SOUTHWEST`, `WEST`, `NORTHWEST`, `CENTER`) of the component in an area that it does not fill.
`fill`	Resizes the component in the specified direction (`NONE`, `HORIZONTAL`, `VERTICAL`, `BOTH`) when the display area is larger than the component.
`gridx`	The column in which the component will be placed.
`gridy`	The row in which the component will be placed.
`gridwidth`	The number of columns the component occupies.
`gridheight`	The number of rows the component occupies.
`weightx`	The amount of extra space to allocate horizontally. The grid slot can become wider when extra space is available.
`weighty`	The amount of extra space to allocate vertically. The grid slot can become taller when extra space is available.

Fig. 25.19 | `GridBagConstraints` fields.

`Constraints` constants: **`NORTH`**, **`NORTHEAST`**, **`EAST`**, **`SOUTHEAST`**, **`SOUTH`**, **`SOUTHWEST`**, **`WEST`**, **`NORTHWEST`** or **`CENTER`**. The default value is `CENTER`.

`GridBagConstraints` field `fill` defines how the component grows if the area in which it can be displayed is larger than the component. The variable `fill` is assigned one of the following `GridBagConstraints` constants: **`NONE`**, **`VERTICAL`**, **`HORIZONTAL`** or **`BOTH`**. The default value is `NONE`, which indicates that the component will not grow in either direction. `VERTICAL` indicates that it will grow vertically. `HORIZONTAL` indicates that it will grow horizontally. `BOTH` indicates that it will grow in both directions.

Variables **`gridx`** and **`gridy`** specify where the upper-left corner of the component is placed in the grid. Variable `gridx` corresponds to the column, and variable `gridy` corresponds to the row. In Fig. 25.18, the `JComboBox` (displaying "`Iron`") has a `gridx` value of 1 and a `gridy` value of 2.

Variable **gridwidth** specifies the number of columns a component occupies. The JComboBox occupies two columns. Variable **gridheight** specifies the number of rows a component occupies. The JTextArea on the left side of Fig. 25.18 occupies three rows.

Variable **weightx** specifies how to distribute extra horizontal space to grid slots in a GridBagLayout when the container is resized. A zero value indicates that the grid slot does not grow horizontally on its own. However, if the component spans a column containing a component with nonzero weightx value, the component with zero weightx value will grow horizontally in the same proportion as the other component(s) in that column. This is because each component must be maintained in the same row and column in which it was originally placed.

Variable **weighty** specifies how to distribute extra vertical space to grid slots in a GridBagLayout when the container is resized. A zero value indicates that the grid slot does not grow vertically on its own. However, if the component spans a row containing a component with nonzero weighty value, the component with zero weighty value grows vertically in the same proportion as the other component(s) in the same row.

In Fig. 25.18, the effects of weighty and weightx cannot easily be seen until the container is resized and additional space becomes available. Components with larger weight values occupy more of the additional space than those with smaller weight values.

Components should be given nonzero positive weight values—otherwise they'll "huddle" together in the middle of the container. Figure 25.20 shows the GUI of Fig. 25.18 with all weights set to zero.

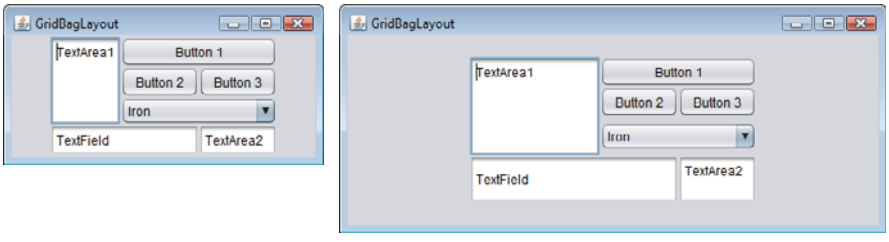


Fig. 25.20 | GridBagLayout with the weights set to zero.

The application in Figs. 25.21–25.22 uses the GridBagLayout layout manager to arrange the components of the GUI in Fig. 25.18. The application does nothing except demonstrate how to use GridBagLayout.

```
1  // Fig. 25.21: GridBagFrame.java
2  // Demonstrating GridBagLayout.
3  import java.awt.GridBagLayout;
4  import java.awt.GridBagConstraints;
5  import java.awt.Component;
6  import javax.swing.JFrame;
7  import javax.swing.JTextArea;
8  import javax.swing.JTextField;
```

Fig. 25.21 | GridBagLayout layout manager. (Part 1 of 3.)

```
import javax.swing.JButton;
import javax.swing.JComboBox;

public class GridBagFrame extends JFrame
{
   private GridBagLayout layout; // layout of this frame
   private GridBagConstraints constraints; // constraints of this layout

   // set up GUI
   public GridBagFrame()
   {
      super( "GridBagLayout" );
      layout = new GridBagLayout();
      setLayout( layout ); // set frame layout
      constraints = new GridBagConstraints(); // instantiate constraints

      // create GUI components
      JTextArea textArea1 = new JTextArea( "TextArea1", 5, 10 );
      JTextArea textArea2 = new JTextArea( "TextArea2", 2, 2 );

      String[] names = { "Iron", "Steel", "Brass" };
      JComboBox comboBox = new JComboBox( names );

      JTextField textField = new JTextField( "TextField" );
      JButton button1 = new JButton( "Button 1" );
      JButton button2 = new JButton( "Button 2" );
      JButton button3 = new JButton( "Button 3" );

      // weightx and weighty for textArea1 are both 0: the default
      // anchor for all components is CENTER: the default
      constraints.fill = GridBagConstraints.BOTH;
      addComponent( textArea1, 0, 0, 1, 3 );

      // weightx and weighty for button1 are both 0: the default
      constraints.fill = GridBagConstraints.HORIZONTAL;
      addComponent( button1, 0, 1, 2, 1 );

      // weightx and weighty for comboBox are both 0: the default
      // fill is HORIZONTAL
      addComponent( comboBox, 2, 1, 2, 1 );

      // button2
      constraints.weightx = 1000;  // can grow wider
      constraints.weighty = 1;     // can grow taller
      constraints.fill = GridBagConstraints.BOTH;
      addComponent( button2, 1, 1, 1, 1 );

      // fill is BOTH for button3
      constraints.weightx = 0;
      constraints.weighty = 0;
      addComponent( button3, 1, 2, 1, 1 );

```

Fig. 25.21 | `GridBagLayout` layout manager. (Part 2 of 3.)

```
61          // weightx and weighty for textField are both 0, fill is BOTH
62          addComponent( textField, 3, 0, 2, 1 );
63
64          // weightx and weighty for textArea2 are both 0, fill is BOTH
65          addComponent( textArea2, 3, 2, 1, 1 );
66       } // end GridBagFrame constructor
67
68       // method to set constraints on
69       private void addComponent( Component component,
70          int row, int column, int width, int height )
71       {
72          constraints.gridx = column; // set gridx
73          constraints.gridy = row; // set gridy
74          constraints.gridwidth = width; // set gridwidth
75          constraints.gridheight = height; // set gridheight
76          layout.setConstraints( component, constraints ); // set constraints
77          add( component ); // add component
78       } // end method addComponent
79    } // end class GridBagFrame
```

Fig. 25.21 | `GridBagLayout` layout manager. (Part 3 of 3.)

```
1  // Fig. 25.22: GridBagDemo.java
2  // Demonstrating GridBagLayout.
3  import javax.swing.JFrame;
4
5  public class GridBagDemo
6  {
7     public static void main( String[] args )
8     {
9        GridBagFrame gridBagFrame = new GridBagFrame();
10       gridBagFrame.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
11       gridBagFrame.setSize( 300, 150 ); // set frame size
12       gridBagFrame.setVisible( true ); // display frame
13    } // end main
14 } // end class GridBagDemo
```

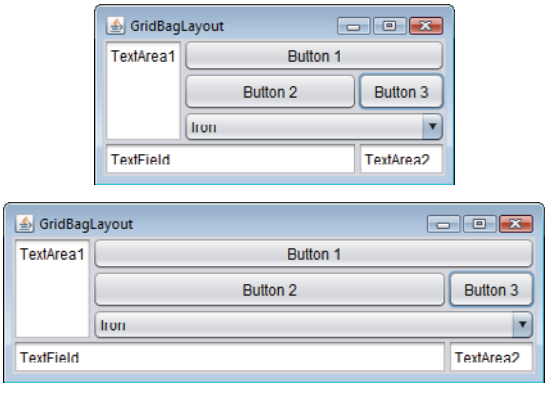


Fig. 25.22 | Test class for `GridBagFrame`. (Part 1 of 2.)

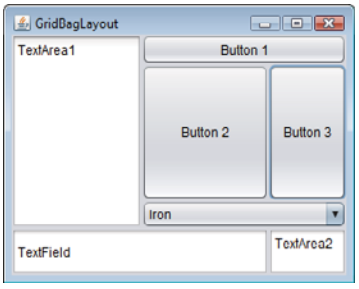


Fig. 25.22 | Test class for `GridBagFrame`. (Part 2 of 2.)

The GUI contains three `JButtons`, two `JTextAreas`, a `JComboBox` and a `JTextField`. The layout manager is `GridBagLayout`. Lines 21–22 create the `GridBagLayout` object and set the layout manager for the `JFrame` to `layout`. Line 23 creates the `GridBagConstraints` object used to determine the location and size of each component in the grid. Lines 26–35 create each GUI component that will be added to the content pane.

Lines 39–40 configure `JTextArea textArea1` and add it to the content pane. The values for `weightx` and `weighty` values are not specified in `constraints`, so each has the value zero by default. Thus, the `JTextArea` will not resize itself even if space is available. However, it spans multiple rows, so the vertical size is subject to the `weighty` values of `JButtons button2` and `button3`. When either button is resized vertically based on its `weighty` value, the `JTextArea` is also resized.

Line 39 sets variable `fill` in `constraints` to `GridBagConstraints.BOTH`, causing the `JTextArea` to always fill its entire allocated area in the grid. An `anchor` value is not specified in `constraints`, so the default `CENTER` is used. We do not use variable `anchor` in this application, so all the components will use the default. Line 40 calls our utility method `addComponent` (declared at lines 69–78). The `JTextArea` object, the row, the column, the number of columns to span and the number of rows to span are passed as arguments.

`JButton button1` is the next component added (lines 43–44). By default, the `weightx` and `weighty` values are still zero. The `fill` variable is set to `HORIZONTAL`—the component will always fill its area in the horizontal direction. The vertical direction is not filled. The `weighty` value is zero, so the button will become taller only if another component in the same row has a nonzero `weighty` value. `JButton button1` is located at row 0, column 1. One row and two columns are occupied.

`JComboBox comboBox` is the next component added (line 48). By default, the `weightx` and `weighty` values are zero, and the `fill` variable is set to `HORIZONTAL`. The `JComboBox` button will grow only in the horizontal direction. The `weightx`, `weighty` and `fill` variables retain the values set in `constraints` until they're changed. The `JComboBox` button is placed at row 2, column 1. One row and two columns are occupied.

`JButton button2` is the next component added (lines 51–54). It's given a `weightx` value of `1000` and a `weighty` value of `1`. The area occupied by the button is capable of growing in the vertical and horizontal directions. The `fill` variable is set to `BOTH`, which specifies that the button will always fill the entire area. When the window is resized, `button2` will grow. The button is placed at row 1, column 1. One row and one column are occupied.

`JButton button3` is added next (lines 57–59). Both the `weightx` value and `weighty` value are set to zero, and the value of `fill` is `BOTH`. `JButton button3` will grow if the window is resized—it's affected by the weight values of `button2`. The `weightx` value for `button2` is much larger than that for `button3`. When resizing occurs, `button2` will occupy a larger percentage of the new space. The button is placed at row 1, column 2. One row and one column are occupied.

Both the `JTextField textField` (line 62) and `JTextArea textArea2` (line 65) have a `weightx` value of 0 and a `weighty` value of 0. The value of `fill` is `BOTH`. The `JTextField` is placed at row 3, column 0, and the `JTextArea` at row 3, column 2. The `JTextField` occupies one row and two columns, the `JTextArea` one row and one column.

Method `addComponent`'s parameters are a `Component` reference `component` and integers `row`, `column`, `width` and `height`. Lines 72–73 set the `GridBagConstraints` variables `gridx` and `gridy`. The `gridx` variable is assigned the column in which the `Component` will be placed, and the `gridy` value is assigned the row in which the `Component` will be placed. Lines 74–75 set the `GridBagConstraints` variables `gridwidth` and `gridheight`. The `gridwidth` variable specifies the number of columns the `Component` will span in the grid, and the `gridheight` variable specifies the number of rows the `Component` will span in the grid. Line 76 sets the `GridBagConstraints` for a component in the `GridBagLayout`. Method **`setConstraints`** of class `GridBagLayout` takes a `Component` argument and a `GridBagConstraints` argument. Line 77 adds the component to the `JFrame`.

When you execute this application, try resizing the window to see how the constraints for each GUI component affect its position and size in the window.

`GridBagConstraints` Constants `RELATIVE` and `REMAINDER`

Instead of `gridx` and `gridy`, a variation of `GridBagLayout` uses `GridBagConstraints` constants **`RELATIVE`** and **`REMAINDER`**. `RELATIVE` specifies that the next-to-last component in a particular row should be placed to the right of the previous component in the row. `REMAINDER` specifies that a component is the last component in a row. Any component that is not the second-to-last or last component on a row must specify values for `GridbagConstraints` variables `gridwidth` and `gridheight`. The application in Figs. 25.23–25.24 arranges components in `GridBagLayout`, using these constants.

```
1  // Fig. 25.23: GridBagFrame2.java
2  // Demonstrating GridBagLayout constants.
3  import java.awt.GridBagLayout;
4  import java.awt.GridBagConstraints;
5  import java.awt.Component;
6  import javax.swing.JFrame;
7  import javax.swing.JComboBox;
8  import javax.swing.JTextField;
9  import javax.swing.JList;
10 import javax.swing.JButton;
11
12 public class GridBagFrame2 extends JFrame
13 {
14    private GridBagLayout layout; // layout of this frame
15    private GridBagConstraints constraints; // constraints of this layout
```

Fig. 25.23 | `GridBagConstraints` constants `RELATIVE` and `REMAINDER`. (Part 1 of 3.)

```
16
17      // set up GUI
18      public GridBagFrame2()
19      {
20         super( "GridBagLayout" );
21         layout = new GridBagLayout();
22         setLayout( layout ); // set frame layout
23         constraints = new GridBagConstraints(); // instantiate constraints
24
25         // create GUI components
26         String[] metals = { "Copper", "Aluminum", "Silver" };
27         JComboBox comboBox = new JComboBox( metals );
28
29         JTextField textField = new JTextField( "TextField" );
30
31         String[] fonts = { "Serif", "Monospaced" };
32         JList list = new JList( fonts );
33
34         String[] names = { "zero", "one", "two", "three", "four" };
35         JButton[] buttons = new JButton[ names.length ];
36
37         for ( int count = 0; count < buttons.length; count++ )
38            buttons[ count ] = new JButton( names[ count ] );
39
40         // define GUI component constraints for textField
41         constraints.weightx = 1;
42         constraints.weighty = 1;
43         constraints.fill = GridBagConstraints.BOTH;
44         constraints.gridwidth = GridBagConstraints.REMAINDER;
45         addComponent( textField );
46
47         // buttons[0] -- weightx and weighty are 1: fill is BOTH
48         constraints.gridwidth = 1;
49         addComponent( buttons[ 0 ] );
50
51         // buttons[1] -- weightx and weighty are 1: fill is BOTH
52         constraints.gridwidth = GridBagConstraints.RELATIVE;
53         addComponent( buttons[ 1 ] );
54
55         // buttons[2] -- weightx and weighty are 1: fill is BOTH
56         constraints.gridwidth = GridBagConstraints.REMAINDER;
57         addComponent( buttons[ 2 ] );
58
59         // comboBox -- weightx is 1: fill is BOTH
60         constraints.weighty = 0;
61         constraints.gridwidth = GridBagConstraints.REMAINDER;
62         addComponent( comboBox );
63
64         // buttons[3] -- weightx is 1: fill is BOTH
65         constraints.weighty = 1;
66         constraints.gridwidth = GridBagConstraints.REMAINDER;
67         addComponent( buttons[ 3 ] );
68
```

Fig. 25.23 | `GridBagConstraints` constants `RELATIVE` and `REMAINDER`. (Part 2 of 3.)

```
69          // buttons[4] -- weightx and weighty are 1: fill is BOTH
70          constraints.gridwidth = GridBagConstraints.RELATIVE;
71          addComponent( buttons[ 4 ] );
72
73          // list -- weightx and weighty are 1: fill is BOTH
74          constraints.gridwidth = GridBagConstraints.REMAINDER;
75          addComponent( list );
76       } // end GridBagFrame2 constructor
77
78       // add a component to the container
79       private void addComponent( Component component )
80       {
81          layout.setConstraints( component, constraints );
82          add( component ); // add component
83       } // end method addComponent
84    } // end class GridBagFrame2
```

Fig. 25.23 | `GridBagConstraints` constants `RELATIVE` and `REMAINDER`. (Part 3 of 3.)

```
1  // Fig. 25.24: GridBagDemo2.java
2  // Demonstrating GridBagLayout constants.
3  import javax.swing.JFrame;
4
5  public class GridBagDemo2
6  {
7     public static void main( String[] args )
8     {
9        GridBagFrame2 gridBagFrame = new GridBagFrame2();
10       gridBagFrame.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
11       gridBagFrame.setSize( 300, 200 ); // set frame size
12       gridBagFrame.setVisible( true ); // display frame
13    } // end main
14 } // end class GridBagDemo2
```

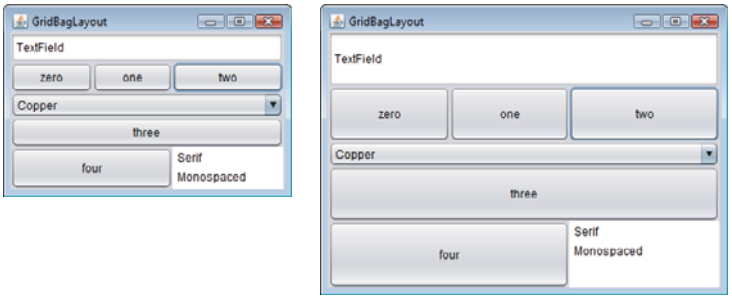


Fig. 25.24 | Test class for `GridBagDemo2`.

Lines 21–22 create a `GridBagLayout` and use it to set the `JFrame`'s layout manager. The components that are placed in `GridBagLayout` are created in lines 27–38—they are a `JComboBox`, a `JTextField`, a `JList` and five `JButtons`.

The `JTextField` is added first (lines 41–45). The `weightx` and `weighty` values are set to 1. The `fill` variable is set to `BOTH`. Line 44 specifies that the `JTextField` is the last com-

ponent on the line. The `JTextField` is added to the content pane with a call to our utility method `addComponent` (declared at lines 79–83). Method `addComponent` takes a `Component` argument and uses `GridBagLayout` method `setConstraints` to set the constraints for the `Component`. Method `add` attaches the component to the content pane.

`JButton buttons[ 0 ]` (lines 48–49) has `weightx` and `weighty` values of 1. The `fill` variable is `BOTH`. Because `buttons[ 0 ]` is not one of the last two components on the row, it's given a `gridwidth` of 1 and so will occupy one column. The `JButton` is added to the content pane with a call to utility method `addComponent`.

`JButton buttons[ 1 ]` (lines 52–53) has `weightx` and `weighty` values of 1. The `fill` variable is `BOTH`. Line 52 specifies that the `JButton` is to be placed relative to the previous component. The `Button` is added to the `JFrame` with a call to `addComponent`.

`JButton buttons[ 2 ]` (lines 56–57) has `weightx` and `weighty` values of 1. The `fill` variable is `BOTH`. This `JButton` is the last component on the line, so `REMAINDER` is used. The `JButton` is added to the content pane with a call to `addComponent`.

The `JComboBox` (lines 60–62) has a `weightx` of 1 and a `weighty` of 0. The `JComboBox` will not grow vertically. The `JComboBox` is the only component on the line, so `REMAINDER` is used. The `JComboBox` is added to the content pane with a call to `addComponent`.

`JButton buttons[ 3 ]` (lines 65–67) has `weightx` and `weighty` values of 1. The `fill` variable is `BOTH`. This `JButton` is the only component on the line, so `REMAINDER` is used. The `JButton` is added to the content pane with a call to `addComponent`.

`JButton buttons[ 4 ]` (lines 70–71) has `weightx` and `weighty` values of 1. The `fill` variable is `BOTH`. This `JButton` is the next-to-last component on the line, so `RELATIVE` is used. The `JButton` is added to the content pane with a call to `addComponent`.

The `JList` (lines 74–75) has `weightx` and `weighty` values of 1. The `fill` variable is `BOTH`. The `JList` is added to the content pane with a call to `addComponent`.

25.10 Wrap-Up

This chapter completes our introduction to GUIs. In this chapter, we discussed additional GUI topics, such as menus, sliders, pop-up menus, multiple-document interfaces, tabbed panes and Java's pluggable look-and-feel. All these components can be added to existing applications to make them easier to use and understand. We also presented additional layout managers for organizing and sizing GUI components. In the next chapter, you'll learn about multithreading, which allows you to specify that an application should perform multiple tasks at once.

Summary

Section 25.2 `JSlider`

- `JSliders` enable you to select from a range of integer values. They can display major and minor tick marks, and labels for the tick marks (p. 1001). They also support snap-to ticks (p. 1001)—positioning the thumb (p. 1001) between two tick marks causes snaps it to the closest tick mark.
- `JSliders` (p. 1001) have either horizontal or vertical orientation. For a horizontal `JSlider`, the minimum value is at the extreme left and the maximum value at the extreme right. For a vertical `JSlider`, the minimum value is at the extreme bottom and the maximum value at the extreme top. The position of the thumb indicates the current value of the `JSlider`. Method `getValue` (p. 1005) of class `JSlider` returns the current thumb position.

- `JSlider` method `setMajorTickSpacing` () sets the spacing for tick marks on a `JSlider`. Method `setPaintTicks` (p. 1005) with a `true` argument indicates that the tick marks should be displayed.
- `JSliders` generate `ChangeEvents` when the user interacts with a `JSlider`. A `ChangeListener` (p. 1005) declares method `stateChanged` (p. 1005) that can respond to `ChangeEvents`.

Section 25.3 Windows: Additional Notes

- A window's (p. 1005) events can be handled by a `WindowListener` (p. 1006), which provides seven window-event-handling methods—`windowActivated`, `windowClosed`, `windowClosing`, `windowDeactivated`, `windowDeiconified`, `windowIconified` and `windowOpened`.

Section 25.4 Using Menus with Frames

- Menus neatly organize commands in a GUI. In Swing GUIs, menus can be attached only to objects of classes with method `setJMenuBar` (p. 1006).
- A `JMenuBar` (p. 1006) is a container for menus. A `JMenuItem` appears in a menu. A `JMenu` (p. 1006) contains menu items (p. 1007) and can be added to a `JMenuBar` or to other `JMenus` as submenus.
- When a menu is clicked, it expands to show its list of menu items.
- When a `JCheckBoxMenuItem` (p. 1007) is selected, a check appears to the left of the menu item. When the `JCheckBoxMenuItem` is selected again, the check is removed.
- In a `ButtonGroup`, only one `JRadioButtonMenuItem` (p. 1007) can be selected at a time.
- `AbstractButton` method `setMnemonic` (p. 1012) specifies the mnemonic (p. 1007) for a button. Mnemonic characters are normally displayed with an underline.
- A modal dialog box (p. 1012) does not allow access to any other window in the application until the dialog is dismissed. The dialogs displayed with class `JOptionPane` are modal dialogs. Class `JDialog` (p. 1013) can be used to create your own modal or nonmodal dialogs.

Section 25.5 `JPopupMenu`

- Context-sensitive pop-up menus (p. 1014) are created with class `JPopupMenu` (p. 1014). The pop-up trigger event (p. 1014) occurs normally when the user presses and releases the right mouse button. `MouseEvent` method `isPopupTrigger` (p. 1017) returns `true` if the pop-up trigger event occurred.
- `JPopupMenu` method `show` (p. 1017) displays a `JPopupMenu`. The first argument specifies the origin component (p. 1017), which helps determine where the `JPopupMenu` will appear. The last two arguments are the coordinates from the origin component's upper-left corner, at which the `JPopupMenu` appears.

Section 25.6 Pluggable Look-and-Feel

- Class `UIManager.LookAndFeelInfo` (p. 1021) maintains information about a look-and-feel.
- `UIManager` (p. 1021) `static` method `getInstalledLookAndFeels` (p. 1021) returns an array of `UIManager.LookAndFeelInfo` objects that describe the available look-and-feels.
- `UIManager` `static` method `setLookAndFeel` (p. 1021) changes the look-and-feel. `SwingUtilities` (p. 1022) `static` method `updateComponentTreeUI` (p. 1022) changes the look-and-feel of every component attached to its `Component` argument to the new look-and-feel.

Section 25.7 `JDesktopPane` and `JInternalFrame`

- Many of today's applications use a multiple-document interface (MDI; p. 1022) to manage several open documents that are being processed in parallel. Swing's `JDesktopPane` (p. 1022) and `JInternalFrame` (p. 1022) classes provide support for creating multiple-document interfaces.

Section 25.8 `JTabbedPane`

- A `JTabbedPane` (p. 1026) arranges GUI components into layers, of which only one is visible at a time. Users access each layer by clicking its tab.

Section 25.9 Layout Managers: `BoxLayout` and `GridBagLayout`

- `BoxLayout` arranges GUI components left-to-right or top-to-bottom in a container.
- Class `Box` represents a container with `BoxLayout` as its default layout manager and provides `static` methods to create a `Box` with a horizontal or vertical `BoxLayout`.
- `GridBagLayout` (p. 1031) is similar to `GridLayout`, but each component size can vary.
- A `GridBagConstraints` object (p. 1031) specifies how a component is placed in a `GridBagLayout`.

Self-Review Exercises

25.1 Fill in the blanks in each of the following statements:

a) The _____ class is used to create a menu object.
b) The _____ method of class `JMenu` places a separator bar in a menu.
c) `JSlider` events are handled by the _____ method of interface _____.
d) The `GridBagConstraints` instance variable _____ is set to `CENTER` by default.

25.2 State whether each of the following is *true* or *false*. If *false*, explain why.

a) When the programmer creates a `JFrame`, a minimum of one menu must be created and added to the `JFrame`.
b) The variable `fill` belongs to the `GridBagLayout` class.
c) Drawing on a GUI component is performed with respect to the (0, 0) upper-left corner coordinate of the component.
d) The default layout for a `Box` is `BoxLayout`.

25.3 Find the error(s) in each of the following and explain how to correct the error(s).

a)
```
JMenubar b;
```
b)
```
mySlider = JSlider( 1000, 222, 100, 450 );
```
c)
```
gbc.fill = GridBagConstraints.NORTHWEST;  // set fill
```
d)
```
// override to paint on a customized Swing component
public void paintcomponent( Graphics g )
{
   g.drawString( "HELLO", 50, 50 );
} // end method paintComponent
```
e)
```
// create a JFrame and display it
JFrame f = new JFrame( "A Window" );
f.setVisible( true );
```

Answers to Self-Review Exercises

25.1 a) `JMenu`. b) `addSeparator`. c) `stateChanged`, `ChangeListener`. d) `anchor`.

25.2 a) False. A `JFrame` does not require any menus.
b) False. The variable `fill` belongs to the `GridBagConstraints` class.
c) True.
d) True.

25.3 a) `JMenubar` should be `JMenuBar`.
b) The first argument to the constructor should be `SwingConstants.HORIZONTAL` or `SwingConstants.VERTICAL`, and the keyword `new` must be used after the `=` operator. Also, the minimum value should be less than the maximum and the initial value should be in range.

c) The constant should be either `BOTH`, `HORIZONTAL`, `VERTICAL` or `NONE`.
d) `paintcomponent` should be `paintComponent`, and the method should call `super.paintComponent( g )` as its first statement.
e) The `JFrame`'s `setSize` method must also be called to establish the size of the window.

Exercises

25.4 *(Fill-in-the-Blanks)* Fill in the blanks in each of the following statements:
a) A `JMenuItem` that is a `JMenu` is called a(n) _____.
b) Method _____ attaches a `JMenuBar` to a `JFrame`.
c) Container class _____ has a default `BoxLayout`.
d) A(n) _____ manages a set of child windows declared with class `JInternalFrame`.

25.5 *(True or False)* State whether each of the following is *true* or *false*. If *false*, explain why.
a) Menus require a `JMenuBar` object so they can be attached to a `JFrame`.
b) `BoxLayout` is the default layout manager for a `JFrame`.
c) `JApplets` can contain menus.

25.6 *(Find the Code Errors)* Find the error(s) in each of the following. Explain how to correct the error(s).

```
a) x.add( new JMenuItem( "Submenu Color" ) ); // create submenu
b) container.setLayout( new GridbagLayout() );
```

25.7 *(Display a Circle and its Attributes)* Write a program that displays a circle of random size and calculates and displays the area, radius, diameter and circumference. Use the following equations: *diameter* = 2 × *radius*, *area* = π × *radius*^{2}, *circumference* = 2 × π × *radius*. Use the constant `Math.PI` for pi (π). All drawing should be done on a subclass of `JPanel`, and the results of the calculations should be displayed in a read-only `JTextArea`.

25.8 *(Using a `JSlider`)* Enhance the program in Exercise 25.7 by allowing the user to alter the radius with a `JSlider`. The program should work for all radii in the range from 100 to 200. As the radius changes, the diameter, area and circumference should be updated and displayed. The initial radius should be 150. Use the equations from Exercise 25.7. All drawing should be done on a subclass of `JPanel`, and the results of the calculations should be displayed in a read-only `JTextArea`.

25.9 *(Varying `weightx` and `weighty`)* Explore the effects of varying the `weightx` and `weighty` values of the program in Fig. 25.21. What happens when a slot has a nonzero weight but is not allowed to fill the whole area (i.e., the `fill` value is not `BOTH`)?

25.10 *(Synchronizing a `JSlider` and a `JTextField`)* Write a program that uses the `paintComponent` method to draw the current value of a `JSlider` on a subclass of `JPanel`. In addition, provide a `JTextField` where a specific value can be entered. The `JTextField` should display the current value of the `JSlider` at all times. Changing the value in the `JTextField` should also update the `JSlider`. A `JLabel` should be used to identify the `JTextField`. The `JSlider` methods `setValue` and `getValue` should be used. [*Note:* The `setValue` method is a `public` method that does not return a value and takes one integer argument, the `JSlider` value, which determines the position of the thumb.]

25.11 *(Creating a Color Chooser)* Declare a subclass of `JPanel` called `MyColorChooser` that provides three `JSlider` objects and three `JTextField` objects. Each `JSlider` represents the values from 0 to 255 for the red, green and blue parts of a color. Use these values as the arguments to the `Color` constructor to create a new `Color` object. Display the current value of each `JSlider` in the corresponding `JTextField`. When the user changes the value of the `JSlider`, the `JTextField` should be changed accordingly. Use your new GUI component as part of an application that displays the current `Color` value by drawing a filled rectangle.

25.12 ***(Creating a Color Chooser: Modification)*** Modify the `MyColorChooser` class of Exercise 25.11 to allow the user to enter an integer value into a `JTextField` to set the red, green or blue value. When the user presses *Enter* in the `JTextField`, the corresponding `JSlider` should be set to the appropriate value.

25.13 ***(Creating a Color Chooser: Modification)*** Modify the application in Exercise 25.12 to draw the current color as a rectangle on an instance of a subclass of `JPanel` which provides its own `paintComponent` method to draw the rectangle and provides *set* methods to set the red, green and blue values for the current color. When any *set* method is invoked, the drawing panel should automatically `repaint` itself.

25.14 ***(Drawing Application)*** Modify the application in Exercise 25.13 to allow the user to drag the mouse across the drawing panel (a subclass of `JPanel`) to draw a shape in the current color. Enable the user to choose what shape to draw.

25.15 ***(Drawing Application Modification)*** Modify the application in Exercise 25.14 to provide the user with the ability to terminate the application by clicking the close box on the window that is displayed and by selecting `Exit` from a `File` menu. Use the techniques shown in Fig. 25.5.

25.16 ***(Complete Drawing Application)*** Using the techniques developed in this chapter and Chapter 14, create a complete drawing application. The program should use the GUI components from Chapter 14 and Chapter 25 to enable the user to select the shape, color and fill characteristics. Each shape should be stored in an array of `MyShape` objects, where `MyShape` is the superclass in your hierarchy of shape classes. Use a `JDesktopPane` and `JInternalFrames` to allow the user to create multiple separate drawings in separate child windows. Create the user interface as a separate child window containing all the GUI components that allow the user to determine the characteristics of the shape to be drawn. The user can then click in any `JInternalFrame` to draw the shape.

26

Multithreading



The most general definition of beauty…Multeity in Unity.
—Samuel Taylor Coleridge

Do not block the way of inquiry.
—Charles Sanders Peirce

A person with one watch knows what time it is; a person with two watches is never sure.
—Proverb

Learn to labor and to wait.
—Henry Wadsworth Longfellow

The world is moving so fast these days that the man who says it can't be done is generally interrupted by someone doing it.
—Elbert Hubbard

Objectives

In this chapter you'll learn:

- What threads are and why they're useful.
- How threads enable you to manage concurrent activities.
- The life cycle of a thread.
- To create and execute `Runnable`s.
- Thread synchronization.
- What producer/consumer relationships are and how they're implemented with multithreading.
- To enable multiple threads to update Swing GUI components in a thread-safe manner.

Outline

26.1 Introduction
26.2 Thread States: Life Cycle of a Thread
26.3 Creating and Executing Threads with `Executor` Framework
26.4 Thread Synchronization
- 26.4.1 Unsynchronized Data Sharing
- 26.4.2 Synchronized Data Sharing—Making Operations Atomic

26.5 Producer/Consumer Relationship without Synchronization
26.6 Producer/Consumer Relationship: `ArrayBlockingQueue`
26.7 Producer/Consumer Relationship with Synchronization
26.8 Producer/Consumer Relationship: Bounded Buffers
26.9 Producer/Consumer Relationship: The `Lock` and `Condition` Interfaces
26.10 Concurrent Collections Overview
26.11 Multithreading with GUI
- 26.11.1 Performing Computations in a Worker Thread
- 26.11.2 Processing Intermediate Results with `SwingWorker`

26.12 Interfaces `Callable` and `Future`
26.13 Java SE 7: Fork/Join Framework
26.14 Wrap-Up

Summary | *Self-Review Exercises* | *Answers to Self-Review Exercises* | *Exercises*

26.1 Introduction

It would be nice if we could focus our attention on performing only one action at a time and performing it well, but that's usually difficult to do. The human body performs a great variety of operations *in parallel*—or, as we'll say throughout this chapter, **concurrently**. Respiration, blood circulation, digestion, thinking and walking, for example, can occur concurrently, as can all the senses—sight, touch, smell, taste and hearing.

Computers, too, can perform operations concurrently. It's common for personal computers to compile a program, send a file to a printer and receive electronic mail messages over a network concurrently. Only computers that have multiple processors can truly execute multiple instructions concurrently. Operating systems on single-processor computers create the illusion of concurrent execution by rapidly switching between activities, but on such computers only a single instruction can execute at once. Today's multicore computers have multiple processors that enable computers to perform tasks truly concurrently. Multicore smartphones are starting to appear.

Historically, concurrency has been implemented with operating system primitives available only to experienced systems programmers. The Ada programming language—developed by the United States Department of Defense—made concurrency primitives widely available to defense contractors building military command-and-control systems. However, Ada has not been widely used in academia and industry.

Java Concurrency

Java makes concurrency available to you through the language and APIs. Java programs can have multiple **threads of execution**, where each thread has its own method-call stack and program counter, allowing it to execute concurrently with other threads while sharing with them application-wide resources such as memory. This capability is called **multithreading**.



Performance Tip 26.1

A problem with single-threaded applications that can lead to poor responsiveness is that lengthy activities must complete before *others can begin. In a multithreaded application, threads can be distributed across multiple processors (if available) so that multiple tasks execute* truly *concurrently and the application can operate more efficiently. Multithreading can also increase performance on single-processor systems that* simulate *concurrency—when one thread cannot proceed (because, for example, it's waiting for the result of an I/O operation), another can use the processor.*

Concurrent Programming Uses

We'll discuss many applications of **concurrent programming**. For example, when downloading a large file (e.g., an image, an audio clip or a video clip) over the Internet, the user may not want to wait until the entire clip downloads before starting the playback. To solve this problem, multiple threads can be used—one to download the clip, and another to play it. These activities proceed concurrently. To avoid choppy playback, the threads are **synchronized** (that is, their actions are coordinated) so that the player thread doesn't begin until there's a sufficient amount of the clip in memory to keep the player thread busy. The Java Virtual Machine (JVM) creates threads to run programs and threads to perform housekeeping tasks such as garbage collection.

Concurrent Programming Is Difficult

Writing multithreaded programs can be tricky. Although the human mind can perform functions concurrently, people find it difficult to jump between parallel trains of thought. To see why multithreaded programs can be difficult to write and understand, try the following experiment: Open three books to page 1, and try reading the books concurrently. Read a few words from the first book, then a few from the second, then a few from the third, then loop back and read the next few words from the first book, and so on. After this experiment, you'll appreciate many of the challenges of multithreading—switching between the books, reading briefly, remembering your place in each book, moving the book you're reading closer so that you can see it and pushing the books you're not reading aside—and, amid all this chaos, trying to comprehend the content of the books!

Use the Prebuilt Classes of the Concurrency APIs Whenever Possible

Programming concurrent applications is difficult and error prone. If you must use synchronization in a program, you should follow some simple guidelines. *Use existing classes from the Concurrency APIs (such as the* `ArrayBlockingQueue` *class we discuss in Section 26.6) that manage synchronization for you.* These classes are written by experts, have been thoroughly tested and debugged, operate efficiently and help you avoid common traps and pitfalls.

If you need even more complex capabilities, use interfaces `Lock` and `Condition` that we introduce in Section 26.9. These interfaces should be used only by advanced programmers who are familiar with concurrent programming's common traps and pitfalls. We explain these topics in this chapter for several reasons:

- They provide a solid basis for understanding how concurrent applications synchronize access to shared memory.
- The concepts are important to understand, even if an application does not use these tools explicitly.

- By showing you the complexity involved in using these low-level features, we hope to impress upon you the importance of *using prebuilt concurrency capabilities whenever possible*.

Section 26.10 provides an overview of Java's pre-built concurrent collections.

26.2 Thread States: Life Cycle of a Thread

At any time, a thread is said to be in one of several **thread states**—illustrated in the UML state diagram in Fig. 26.1. Several of the terms in the diagram are defined in later sections. We include this discussion to help you understand what's going on "under the hood" in a Java multithreaded environment. Java hides most of this detail from you, greatly simplifying the task of developing multithreaded applications.

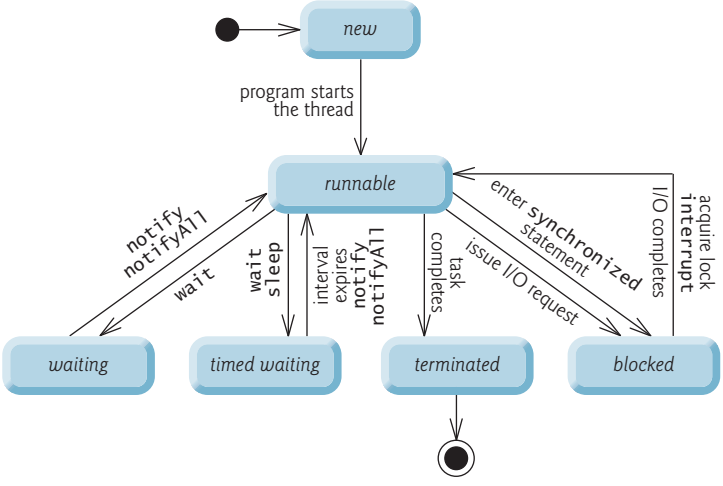


Fig. 26.1 | Thread life-cycle UML state diagram.

New *and* Runnable *States*

A new thread begins its life cycle in the ***new*** state. It remains in this state until the program starts the thread, which places it in the ***runnable*** state. A thread in the *runnable* state is considered to be executing its task.

Waiting *State*

Sometimes a *runnable* thread transitions to the ***waiting*** state while it waits for another thread to perform a task. A *waiting* thread transitions back to the *runnable* state only when another thread notifies it to continue executing.

Timed Waiting *State*

A *runnable* thread can enter the ***timed waiting*** state for a specified interval of time. It transitions back to the *runnable* state when that time interval expires or when the event it's waiting for occurs. *Timed waiting* and *waiting* threads cannot use a processor, even if one

is available. A *runnable* thread can transition to the *timed waiting* state if it provides an optional wait interval when it's waiting for another thread to perform a task. Such a thread returns to the *runnable* state when it's notified by another thread or when the timed interval expires—whichever comes first. Another way to place a thread in the *timed waiting* state is to put a *runnable* thread to sleep. A **sleeping thread** remains in the *timed waiting* state for a designated period of time (called a **sleep interval**), after which it returns to the *runnable* state. Threads sleep when they momentarily do not have work to perform. For example, a word processor may contain a thread that periodically backs up (i.e., writes a copy of) the current document to disk for recovery purposes. If the thread did not sleep between successive backups, it would require a loop in which it continually tested whether it should write a copy of the document to disk. This loop would consume processor time without performing productive work, thus reducing system performance. In this case, it's more efficient for the thread to specify a sleep interval (equal to the period between successive backups) and enter the *timed waiting* state. This thread is returned to the *runnable* state when its sleep interval expires, at which point it writes a copy of the document to disk and reenters the *timed waiting* state.

Blocked *State*

A *runnable* thread transitions to the ***blocked*** state when it attempts to perform a task that cannot be completed immediately and it must temporarily wait until that task completes. For example, when a thread issues an input/output request, the operating system blocks the thread from executing until that I/O request completes—at that point, the *blocked* thread transitions to the *runnable* state, so it can resume execution. A *blocked* thread cannot use a processor, even if one is available.

Terminated *State*

A *runnable* thread enters the ***terminated*** state (sometimes called the ***dead*** state) when it successfully completes its task or otherwise terminates (perhaps due to an error). In the UML state diagram of Fig. 26.1, the *terminated* state is followed by the UML final state (the bull's-eye symbol) to indicate the end of the state transitions.

Operating-System View of the Runnable *State*

At the operating system level, Java's *runnable* state typically encompasses *two separate* states (Fig. 26.2). The operating system hides these states from the Java Virtual Machine (JVM), which sees only the *runnable* state. When a thread first transitions to the *runnable* state from the *new* state, it's in the ***ready*** state. A *ready* thread enters the ***running*** state (i.e., begins executing) when the operating system assigns it to a processor—also known as **dispatching the thread**. In most operating systems, each thread is given a small amount of processor time—called a **quantum** or **timeslice**—with which to perform its task. Deciding how large the quantum should be is a key topic in operating systems courses. When its quantum expires, the thread returns to the *ready* state, and the operating system assigns another thread to the processor. Transitions between the *ready* and *running* states are handled solely by the operating system. The JVM does not "see" the transitions—it simply views the thread as being *runnable* and leaves it up to the operating system to transition the thread between *ready* and *running*. The process that an operating system uses to determine which thread to dispatch is called **thread scheduling** and is dependent on thread priorities.

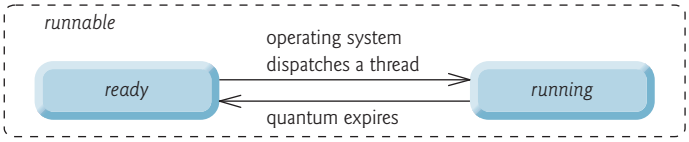


Fig. 26.2 | Operating system's internal view of Java's *runnable* state.

Thread Priorities and Thread Scheduling

Every Java thread has a **thread priority** that helps determine the order in which threads are scheduled. Each new thread inherits the priority of the thread that created it. Informally, higher-priority threads are more important to a program and should be allocated processor time before lower-priority threads. *Nevertheless, thread priorities cannot guarantee the order in which threads execute.*

It's recommended that you do not explicitly create and use `Thread` *objects to implement concurrency, but rather use the* `Executor` *interface (which is described in Section 26.3).* The Thread class does contain some useful static methods, which you *will* use later in the chapter.

Most operating systems support timeslicing, which enables threads of equal priority to share a processor. Without timeslicing, each thread in a set of equal-priority threads runs to completion (unless it leaves the *runnable* state and enters the *waiting* or *timed waiting* state, or gets interrupted by a higher-priority thread) before other threads of equal priority get a chance to execute. With timeslicing, even if a thread has *not* finished executing when its quantum expires, the processor is taken away from the thread and given to the next thread of equal priority, if one is available.

An *operating system's* **thread scheduler** determines which thread runs next. One simple thread-scheduler implementation keeps the highest-priority thread *running* at all times and, if there's more than one highest-priority thread, ensures that all such threads execute for a quantum each in **round-robin** fashion. This process continues until all threads run to completion.

When a higher-priority thread enters the *ready* state, the operating system generally preempts the currently *running* thread (an operation known as **preemptive scheduling**). Depending on the operating system, higher-priority threads could postpone—possibly indefinitely—the execution of lower-priority threads. Such **indefinite postponement** is sometimes referred to more colorfully as **starvation**. Operating systems employ a technique called *aging* to prevent starvation—as a thread waits in the *ready* state, the operating system gradually increases the thread's priority, thus ensuring that the thread will eventually run.

Java provides higher-level concurrency utilities to hide much of this complexity and make multithreaded programming less error prone. Thread priorities are used behind the scenes to interact with the operating system, but most programmers who use Java multithreading will not be concerned with setting and adjusting thread priorities.



Portability Tip 26.1

Thread scheduling is platform dependent—the behavior of a multithreaded program could vary across different Java implementations.

26.3 Creating and Executing Threads with Executor Framework

This section demonstrates how to perform concurrent tasks in an application by using `Executors` and `Runnable` objectss.

Creating Concurrent Tasks with the `Runnable` Interface

You implement the **`Runnable`** interface (of package `java.lang`) to specify a task that can execute concurrently with other tasks. The `Runnable` interface declares the single method **`run`**, which contains the code that defines the task that a `Runnable` object should perform.

Executing `Runnable` Objects with an `Executor`

To allow a `Runnable` to perform its task, you must execute it. An **`Executor`** object executes `Runnable`s. An `Executor` does this by creating and managing a group of threads called a **thread pool**. When an `Executor` begins executing a `Runnable`, the `Executor` calls the `Runnable` object's `run` method, which executes in the new thread.

The `Executor` interface declares a single method named **`execute`** which accepts a `Runnable` as an argument. The `Executor` assigns every `Runnable` passed to its `execute` method to one of the available threads in the thread pool. If there are no available threads, the `Executor` creates a new thread or waits for a thread to become available and assigns that thread the `Runnable` that was passed to method `execute`.

Using an `Executor` has many advantages over creating threads yourself. `Executors` can *reuse existing threads* to eliminate the overhead of creating a new thread for each task and can improve performance by *optimizing the number of threads* to ensure that the processor stays busy, without creating so many threads that the application runs out of resources.



Software Engineering Observation 26.1

Though it's possible to create threads explicitly, it's recommended that you use the `Executor` interface to manage the execution of `Runnable` objects.

Using Class `Executors` to Obtain an `ExecutorService`

The **`ExecutorService` interface** (of package `java.util.concurrent`) *extends* `Executor` and declares various methods for managing the life cycle of an `Executor`. An object that implements the `ExecutorService` interface can be created using `static` methods declared in class **`Executors`** (of package `java.util.concurrent`). We use interface `ExecutorService` and a method of class `Executors` in our example, which executes three tasks.

Implementing the `Runnable` Interface

Class `PrintTask` (Fig. 26.3) implements `Runnable` (line 5), *so that multiple `PrintTask`s can execute concurrently*. Variable `sleepTime` (line 7) stores a random integer value from 0 to 5 seconds created in the `PrintTask` constructor (line 17). Each thread running a `PrintTask` sleeps for the amount of time specified by `sleepTime`, then outputs its task's name and a message indicating that it's done sleeping.

A `PrintTask` executes when a thread calls the `PrintTask`'s `run` method. Lines 25–26 display a message indicating the name of the currently executing task and that the task is going to sleep for `sleepTime` milliseconds. Line 27 invokes `static` method **`sleep`** of class `Thread` to place the thread in the *timed waiting* state for the specified amount of time. At this point, the thread loses the processor, and the system allows another thread to execute.

```
1  // Fig. 26.3: PrintTask.java
2  // PrintTask class sleeps for a random time from 0 to 5 seconds
3  import java.util.Random;
4
5  public class PrintTask implements Runnable
6  {
7     private final int sleepTime; // random sleep time for thread
8     private final String taskName; // name of task
9     private final static Random generator = new Random();
10
11    // constructor
12    public PrintTask( String name )
13    {
14       taskName = name; // set task name
15
16       // pick random sleep time between 0 and 5 seconds
17       sleepTime = generator.nextInt( 5000 ); // milliseconds
18    } // end PrintTask constructor
19
20    // method run contains the code that a thread will execute
21    public void run()
22    {
23       try // put thread to sleep for sleepTime amount of time
24       {
25          System.out.printf( "%s going to sleep for %d milliseconds.\n",
26             taskName, sleepTime );
27          Thread.sleep( sleepTime ); // put thread to sleep
28       } // end try
29       catch ( InterruptedException exception )
30       {
31          System.out.printf( "%s %s\n", taskName,
32             "terminated prematurely due to interruption" );
33       } // end catch
34
35       // print task name
36       System.out.printf( "%s done sleeping\n", taskName );
37    } // end method run
38 } // end class PrintTask
```

Fig. 26.3 | `PrintTask` class sleeps for a random time from 0 to 5 seconds.

When the thread awakens, it reenters the *runnable* state. When the `PrintTask` is assigned to a processor again, line 36 outputs a message indicating that the task is done sleeping, then method `run` terminates. The `catch` at lines 29–33 is required because method `sleep` might throw a *checked* exception of type **`InterruptedException`** if a sleeping thread's **`interrupt`** method is called.

Using the `ExecutorService` *to Manage Threads that Execute* `PrintTask`s

Figure 26.4 uses an `ExecutorService` object to manage threads that execute `PrintTasks` (as defined in Fig. 26.3). Lines 11–13 create and name three `PrintTasks` to execute. Line 18 uses `Executors` method **`newCachedThreadPool`** to obtain an `ExecutorService` that's capable of creating new threads as they're needed by the application. These threads are used by `ExecutorService` (`threadExecutor`) to execute the `Runnable`s.

```
1  // Fig. 26.4: TaskExecutor.java
2  // Using an ExecutorService to execute Runnables.
3  import java.util.concurrent.Executors;
4  import java.util.concurrent.ExecutorService;
5
6  public class TaskExecutor
7  {
8     public static void main( String[] args )
9     {
10       // create and name each runnable
11       PrintTask task1 = new PrintTask( "task1" );
12       PrintTask task2 = new PrintTask( "task2" );
13       PrintTask task3 = new PrintTask( "task3" );
14
15       System.out.println( "Starting Executor" );
16
17       // create ExecutorService to manage threads
18       ExecutorService threadExecutor = Executors.newCachedThreadPool();
19
20       // start threads and place in runnable state
21       threadExecutor.execute( task1 ); // start task1
22       threadExecutor.execute( task2 ); // start task2
23       threadExecutor.execute( task3 ); // start task3
24
25       // shut down worker threads when their tasks complete
26       threadExecutor.shutdown();
27
28       System.out.println( "Tasks started, main ends.\n" );
29    } // end main
30 } // end class TaskExecutor
```

```
Starting Executor
Tasks started, main ends

task1 going to sleep for 4806 milliseconds
task2 going to sleep for 2513 milliseconds
task3 going to sleep for 1132 milliseconds
task3 done sleeping
task2 done sleeping
task1 done sleeping
```

```
Starting Executor
task1 going to sleep for 3161 milliseconds.
task3 going to sleep for 532 milliseconds.
task2 going to sleep for 3440 milliseconds.
Tasks started, main ends.

task3 done sleeping
task1 done sleeping
task2 done sleeping
```

Fig. 26.4 | Using an `ExecutorService` to execute `Runnable`s.

Lines 21–23 each invoke the `ExecutorService`'s `execute` method, which executes the `Runnable` passed to it as an argument (in this case a `PrintTask`) some time in the future. The specified task may execute in one of the threads in the `ExecutorService`'s thread pool, in a new thread created to execute it, or in the thread that called the `execute` method—the `ExecutorService` manages these details. Method `execute` returns immediately from each invocation—the program does *not* wait for each `PrintTask` to finish. Line 26 calls `ExecutorService` method **`shutdown`**, which notifies the `ExecutorService` to *stop accepting new tasks, but continues executing tasks that have already been submitted*. Once all of the previously submitted `Runnables` have completed, the `threadExecutor` terminates. Line 28 outputs a message indicating that the tasks were started and the `main` thread is finishing its execution.

The code in `main` executes in the **main thread**, a thread created by the JVM. The code in the `run` method of `PrintTask` (lines 21–37 of Fig. 26.3) executes whenever the `Executor` starts each `PrintTask`—again, this is sometime after they're passed to the `ExecutorService`'s `execute` method (Fig. 26.4, lines 21–23). When `main` terminates, the program itself continues running because there are still tasks that must finish executing. The program will not terminate until these tasks complete.

The sample outputs show each task's name and sleep time as the thread goes to sleep. The thread with the shortest sleep time *normally* awakens first, indicates that it's done sleeping and terminates. In Section 26.8, we discuss multithreading issues that could prevent the thread with the shortest sleep time from awakening first. In the first output, the `main` thread terminates *before* any of the `PrintTasks` output their names and sleep times. This shows that the `main` thread runs to completion before any of the `PrintTasks` gets a chance to run. In the second output, all of the `PrintTasks` output their names and sleep times *before* the `main` thread terminates. This shows that the `PrintTasks` started executing before the main thread terminated. Also, notice in the second example output, `task3` goes to sleep before `task2` last, even though we passed `task2` to the `ExecutorService`'s `execute` method before `task3`. This illustrates the fact that *we cannot predict the order in which the tasks will start executing, even if we know the order in which they were created and started.*

26.4 Thread Synchronization

When multiple threads share an object and it's modified by one or more of them, indeterminate results may occur (as we'll see in the examples) unless access to the shared object is managed properly. If one thread is in the process of updating a shared object and another thread also tries to update it, it's unclear which thread's update takes effect. When this happens, the program's behavior cannot be trusted—sometimes the program will produce the correct results, and sometimes it won't. In either case, there'll be no indication that the shared object was manipulated incorrectly.

The problem can be solved by giving only one thread at a time *exclusive access* to code that manipulates the shared object. During that time, other threads desiring to manipulate the object are kept waiting. When the thread with exclusive access to the object finishes manipulating it, one of the threads that was waiting is allowed to proceed. This process, called **thread synchronization**, coordinates access to shared data by multiple concurrent threads. By synchronizing threads in this manner, you can ensure that each thread accessing a shared object excludes all other threads from doing so simultaneously—this is called **mutual exclusion**.

Monitors

A common way to perform synchronization is to use Java's built-in **monitors**. Every object has a monitor and a **monitor lock** (or **intrinsic lock**). The monitor ensures that its object's monitor lock is held by a maximum of only one thread at any time. Monitors and monitor locks can thus be used to enforce mutual exclusion. If an operation requires the executing thread to hold a lock while the operation is performed, a thread must acquire the lock before proceeding with the operation. Other threads attempting to perform an operation that requires the same lock will be *blocked* until the first thread releases the lock, at which point the *blocked* threads may attempt to acquire the lock and proceed with the operation.

To specify that a thread must hold a monitor lock to execute a block of code, the code should be placed in a **`synchronized` statement**. Such code is said to be **guarded** by the monitor lock; a thread must **acquire the lock** to execute the guarded statements. The monitor allows only one thread at a time to execute statements within `synchronized` statements that lock on the same object, as only one thread at a time can hold the monitor lock. The `synchronized` statements are declared using the **`synchronized` keyword**:

```
synchronized ( object )
{
   statements
} // end synchronized statement
```

where *object* is the object whose monitor lock will be acquired; *object* is normally `this` if it's the object in which the `synchronized` statement appears. If several `synchronized` statements are trying to execute on an object at the same time, only one of them may be active on the object—all the other threads attempting to enter a `synchronized` statement on the same object are placed in the *blocked* state.

When a `synchronized` statement finishes executing, the object's monitor lock is released and one of the *blocked* threads attempting to enter a `synchronized` statement can be allowed to acquire the lock to proceed. Java also allows **`synchronized` methods**. Before executing, a non-`static` `synchronized` method must acquire the lock on the object that's used to call the method. Similary, a `static` `synchronized` method must acquire the lock on the class that's used to call the method.

26.4.1 Unsynchronized Data Sharing

First, we illustrate the dangers of sharing an object across threads without proper synchronization. In this example, two `Runnables` maintain references to a single integer array. Each `Runnable` writes three values to the array, then terminates. This may seem harmless, but we'll see that it can result in errors if the array is manipulated without synchronization.

Class `SimpleArray`

A `SimpleArray` object (Fig. 26.5) will be *shared* across multiple threads. `SimpleArray` will enable those threads to place `int` values into `array` (declared at line 8). Line 9 initializes variable `writeIndex`, which will be used to determine the array element that should be written to next. The constructor (lines 13–16) creates an integer array of the desired size.

Method `add` (lines 19–40) allows new values to be inserted at the end of the array. Line 21 stores the current `writeIndex` value. Line 26 puts the thread that invokes `add` to sleep for a random interval from 0 to 499 milliseconds. This is done to make the problems associated with *unsynchronized access to shared data* more obvious. After the thread is done

```
1  // Fig. 26.5: SimpleArray.java
2  // Class that manages an integer array to be shared by multiple threads.
3  import java.util.Arrays;
4  import java.util.Random;
5
6  public class SimpleArray // CAUTION: NOT THREAD SAFE!
7  {
8     private final int[] array; // the shared integer array
9     private int writeIndex = 0; // index of next element to be written
10    private final static Random generator = new Random();
11
12    // construct a SimpleArray of a given size
13    public SimpleArray( int size )
14    {
15       array = new int[ size ];
16    } // end constructor
17
18    // add a value to the shared array
19    public void add( int value )
20    {
21       int position = writeIndex; // store the write index
22
23       try
24       {
25          // put thread to sleep for 0-499 milliseconds
26          Thread.sleep( generator.nextInt( 500 ) );
27       } // end try
28       catch ( InterruptedException ex )
29       {
30          ex.printStackTrace();
31       } // end catch
32
33       // put value in the appropriate element
34       array[ position ] = value;
35       System.out.printf( "%s wrote %2d to element %d.\n",
36          Thread.currentThread().getName(), value, position );
37
38       ++writeIndex; // increment index of element to be written next
39       System.out.printf( "Next write index: %d\n", writeIndex );
40    } // end method add
41
42    // used for outputting the contents of the shared integer array
43    public String toString()
44    {
45       return "\nContents of SimpleArray:\n" + Arrays.toString( array );
46    } // end method toString
47 } // end class SimpleArray
```

Fig. 26.5 | Class that manages an integer array to be shared by multiple threads.

sleeping, line 34 inserts the value passed to `add` into the array at the element specified by `position`. Lines 35–36 output a message indicating the executing thread's name, the value that was inserted in the array and where it was inserted. The expression `Thread.currentThread.getName()` (line 36) first obtains a reference to the currently executing `Thread`,

then uses that Thread's getName method to obtain its name. Line 38 increments writeIndex so that the next call to add will insert a value in the array's next element. Lines 43–46 override method toString to create a String representation of the array's contents.

Class ArrayWriter

Class ArrayWriter (Fig. 26.6) implements the interface Runnable to define a task for inserting values in a SimpleArray object. The constructor (lines 10–14) takes two arguments—an integer value, which is the first value this task will insert in the SimpleArray object, and a reference to the SimpleArray object. Line 20 invokes method add on the SimpleArray object. The task completes after three consecutive integers beginning with startValue are added to the SimpleArray object.

```
1  // Fig. 26.6: ArrayWriter.java
2  // Adds integers to an array shared with other Runnables
3  import java.lang.Runnable;
4
5  public class ArrayWriter implements Runnable
6  {
7     private final SimpleArray sharedSimpleArray;
8     private final int startValue;
9
10    public ArrayWriter( int value, SimpleArray array )
11    {
12       startValue = value;
13       sharedSimpleArray = array;
14    } // end constructor
15
16    public void run()
17    {
18       for ( int i = startValue; i < startValue + 3; i++ )
19       {
20          sharedSimpleArray.add( i ); // add an element to the shared array
21       } // end for
22    } // end method run
23 } // end class ArrayWriter
```

Fig. 26.6 | Adds integers to an array shared with other Runnables.

Class SharedArrayTest

Class SharedArrayTest (Fig. 26.7) executes two ArrayWriter tasks that add values to a single SimpleArray object. Line 12 constructs a six-element SimpleArray object. Lines 15–16 create two new ArrayWriter tasks, one that places the values 1–3 in the SimpleArray object, and one that places the values 11–13. Lines 19–21 create an ExecutorService and execute the two ArrayWriters. Line 23 invokes the ExecutorService's shutDown method to *prevent additional tasks from starting* and to enable the application to terminate when the currently executing tasks complete execution.

Recall that ExecutorService method shutdown returns immediately. Thus any code that appears *after* the call to ExecutorService method shutdown in line 23 *will continue executing as long as the* main *thread is still assigned to a processor*. We'd like to output the SimpleArray object to show you the results *after* the threads complete their tasks. So, we

need the program to wait for the threads to complete before `main` outputs the `SimpleArray` object's contents. Interface `ExecutorService` provides the **`awaitTermination`** method for this purpose. This method returns control to its caller either when all tasks executing in the `ExecutorService` complete or when the specified timeout elapses. If all tasks are completed before `awaitTermination` times out, this method returns `true`; otherwise it returns `false`. The two arguments to `awaitTermination` represent a timeout value and a unit of measure specified with a constant from class `TimeUnit` (in this case, `TimeUnit.MINUTES`).

```
1  // Fig 26.7: SharedArrayTest.java
2  // Executes two Runnables to add elements to a shared SimpleArray.
3  import java.util.concurrent.Executors;
4  import java.util.concurrent.ExecutorService;
5  import java.util.concurrent.TimeUnit;
6
7  public class SharedArrayTest
8  {
9     public static void main( String[] arg )
10    {
11       // construct the shared object
12       SimpleArray sharedSimpleArray = new SimpleArray( 6 );
13
14       // create two tasks to write to the shared SimpleArray
15       ArrayWriter writer1 = new ArrayWriter( 1, sharedSimpleArray );
16       ArrayWriter writer2 = new ArrayWriter( 11, sharedSimpleArray );
17
18       // execute the tasks with an ExecutorService
19       ExecutorService executor = Executors.newCachedThreadPool();
20       executor.execute( writer1 );
21       executor.execute( writer2 );
22
23       executor.shutdown();
24
25       try
26       {
27          // wait 1 minute for both writers to finish executing
28          boolean tasksEnded = executor.awaitTermination(
29             1, TimeUnit.MINUTES );
30
31          if ( tasksEnded )
32             System.out.println( sharedSimpleArray ); // print contents
33          else
34             System.out.println(
35                "Timed out while waiting for tasks to finish." );
36       } // end try
37       catch ( InterruptedException ex )
38       {
39          System.out.println(
40             "Interrupted while waiting for tasks to finish." );
41       } // end catch
42    } // end main
43 } // end class SharedArrayTest
```

Fig. 26.7 | Executes two `Runnable`s to insert values in a shared array. (Part 1 of 2.)

```
pool-1-thread-1 wrote  1 to element 0.
Next write index: 1
pool-1-thread-1 wrote  2 to element 1.
Next write index: 2
pool-1-thread-1 wrote  3 to element 2.
Next write index: 3
pool-1-thread-2 wrote 11 to element 0.
Next write index: 4
pool-1-thread-2 wrote 12 to element 4.
Next write index: 5
pool-1-thread-2 wrote 13 to element 5.
Next write index: 6

Contents of SimpleArray:
[11, 2, 3, 0, 12, 13]
```

First `pool-1-thread-1` wrote the value `1` to element `0`. Later `pool-1-thread-2` wrote the value `11` to element `0`, thus *overwriting* the previously stored value.

Fig. 26.7 | Executes two `Runnable`s to insert values in a shared array. (Part 2 of 2.)

In this example, if *both* tasks complete before `awaitTermination` times out, line 32 displays the `SimpleArray` object's contents. Otherwise, lines 34–35 print a message indicating that the tasks did not finish executing before `awaitTermination` timed out.

The output in Fig. 26.7 demonstrates the problems (highlighted in the output) that can be *caused by failure to synchronize access to shared data*. The value `1` was written to element `0`, then *overwritten* later by the value `11`. Also, when `writeIndex` was incremented to 3, *nothing was written to that element*, as indicated by the 0 in that element of the printed array.

Recall that we added calls to `Thread` method `sleep` between operations on the shared data to emphasize the *unpredictability of thread scheduling* and increase the likelihood of producing erroneous output. Even if these operations were allowed to proceed at their normal pace, you could still see errors in the program's output. However, modern processors can handle the simple operations of the `SimpleArray` method `add` so quickly that you might not see the errors caused by the two threads executing this method concurrently, even if you tested the program dozens of times. *One of the challenges of multithreaded programming is spotting the errors—they may occur so infrequently that a broken program does not produce incorrect results during testing, creating the illusion that the program is correct.*

26.4.2 Synchronized Data Sharing—Making Operations Atomic

The output errors of Fig. 26.7 can be attributed to the fact that the shared object, `SimpleArray`, is not **thread safe**—`SimpleArray` is susceptible to errors if it's *accessed concurrently by multiple threads*. The problem lies in method `add`, which stores the value of `writeIndex`, places a new value in that element, then increments `writeIndex`. Such a method would present no problem in a single-threaded program. However, if one thread obtains the value of `writeIndex`, there's no guarantee that another thread cannot come along and increment `writeIndex` *before* the first thread has had a chance to place a value in the array. If this happens, the first thread will be writing to the array based on a **stale value** of `writeIndex`—a value that's no longer valid. Another possibility is that one thread might obtain the value of `writeIndex` *after* another thread adds an element to the array but *before* `writeIndex` is incremented. In this case, too, the first thread would write to the array based on an invalid value for `writeIndex`.

`SimpleArray` is *not thread safe because it allows any number of threads to read and modify shared data concurrently*, which can cause errors. To make `SimpleArray` thread safe, we must ensure that no two threads can access it at the same time. We also must ensure that while one thread is in the process of storing `writeIndex`, adding a value to the array, and incrementing `writeIndex`, no other thread may read or change the value of `writeIndex` or modify the contents of the array at any point during these three operations. In other words, we want these three operations—storing `writeIndex`, writing to the array, incrementing `writeIndex`—to be an **atomic operation**, which cannot be divided into smaller suboperations. We can simulate atomicity by ensuring that only one thread carries out the three operations at a time. Any other threads that need to perform the operation must *wait* until the first thread has finished the `add` operation in its entirety.

Atomicity can be achieved using the `synchronized` keyword. By placing our three suboperations in a `synchronized` statement or `synchronized` method, we allow only one thread at a time to acquire the lock and perform the operations. When that thread has completed all of the operations in the `synchronized` block and releases the lock, another thread may acquire the lock and begin executing the operations. This ensures that a thread executing the operations will see the actual values of the shared data and that *these values will not change unexpectedly in the middle of the operations as a result of another thread's modifying them.*



Software Engineering Observation 26.2

Place all accesses to mutable data that may be shared by multiple threads inside `synchronized` statements or `synchronized` methods that synchronize on the same lock. When performing multiple operations on shared data, hold the lock for the entirety of the operation to ensure that the operation is effectively atomic.

Class `SimpleArray` with Synchronization

Figure 26.8 displays class `SimpleArray` with the proper synchronization. Notice that it's identical to the `SimpleArray` class of Fig. 26.5, except that `add` is now a `synchronized` method (line 20). So, only one thread at a time can execute this method. We reuse classes `ArrayWriter` (Fig. 26.6) and `SharedArrayTest` (Fig. 26.7) from the previous example.

```
1  // Fig. 26.8: SimpleArray.java
2  // Class that manages an integer array to be shared by multiple
3  // threads with synchronization.
4  import java.util.Arrays;
5  import java.util.Random;
6
7  public class SimpleArray
8  {
9     private final int[] array; // the shared integer array
10    private int writeIndex = 0; // index of next element to be written
11    private final static Random generator = new Random();
12
```

Fig. 26.8 | Class that manages an integer array to be shared by multiple threads with synchronization. (Part 1 of 2.)

```
13      // construct a SimpleArray of a given size
14      public SimpleArray( int size )
15      {
16         array = new int[ size ];
17      } // end constructor
18
19      // add a value to the shared array
20      public synchronized void add( int value )
21      {
22         int position = writeIndex; // store the write index
23
24         try
25         {
26            // put thread to sleep for 0-499 milliseconds
27            Thread.sleep( generator.nextInt( 500 ) );
28         } // end try
29         catch ( InterruptedException ex )
30         {
31            ex.printStackTrace();
32         } // end catch
33
34         // put value in the appropriate element
35         array[ position ] = value;
36         System.out.printf( "%s wrote %2d to element %d.\n",
37            Thread.currentThread().getName(), value, position );
38
39         ++writeIndex; // increment index of element to be written next
40         System.out.printf( "Next write index: %d\n", writeIndex );
41      } // end method add
42
43      // used for outputting the contents of the shared integer array
44      public String toString()
45      {
46         return "\nContents of SimpleArray:\n" + Arrays.toString( array );
47      } // end method toString
48   } // end class SimpleArray
```

```
pool-1-thread-1 wrote  1 to element 0.
Next write index: 1
pool-1-thread-2 wrote 11 to element 1.
Next write index: 2
pool-1-thread-2 wrote 12 to element 2.
Next write index: 3
pool-1-thread-2 wrote 13 to element 3.
Next write index: 4
pool-1-thread-1 wrote  2 to element 4.
Next write index: 5
pool-1-thread-1 wrote  3 to element 5.
Next write index: 6

Contents of SimpleArray:
1 11 12 13 2 3
```

Fig. 26.8 | Class that manages an integer array to be shared by multiple threads with synchronization. (Part 2 of 2.)

Line 20 declares method as `synchronized`, making all of the operations in this method behave as a single, atomic operation. Line 22 performs the first suboperation—storing the value of `writeIndex`. Line 35 defines the second suboperation, writing an element to the element at the index `position`. Line 39 increments `writeIndex`. When the method finishes executing at line 41, the executing thread implicitly releases the `SimpleArray` lock, making it possible for another thread to begin executing the `add` method.

In the `synchronized add` method, we print messages to the console indicating the progress of threads as they execute this method, in addition to performing the actual operations required to insert a value in the array. We do this so that the messages will be printed in the correct order, allowing us to see whether the method is properly synchronized by comparing these outputs with those of the previous, unsynchronized example. We continue to output messages from `synchronized` blocks in later examples for demonstration purposes only; typically, however, I/O *should not* be performed in `synchronized` blocks, because it's important to minimize the amount of time that an object is "locked." Also, line 27 in this example calls `Thread` method `sleep` to emphasize the *unpredictability of thread scheduling. You should never call* `sleep` *while holding a lock in a real application.*



Performance Tip 26.2

Keep the duration of `synchronized` *statements as short as possible while maintaining the needed synchronization. This minimizes the wait time for blocked threads. Avoid performing I/O, lengthy calculations and operations that do not require synchronization while holding a lock.*

Another note on thread safety: We've said that it's necessary to synchronize access to all data that may be shared across multiple threads. Actually, this synchronization is necessary only for **mutable data**, or data that may *change* in its lifetime. If the shared data will not change in a multithreaded program, then it's not possible for a thread to see old or incorrect values as a result of another thread's manipulating that data.

When you share immutable data across threads, declare the corresponding data fields `final` to indicate that the values of the variables will *not* change after they're initialized. This prevents accidental modification of the shared data later in a program, which could compromise thread safety. *Labeling object references as* `final` *indicates that the reference will not change, but it does not guarantee that the object itself is immutable—this depends entirely on the object's properties.* However, it's still good practice to mark references that will not change as `final`, as doing so forces the object's constructor to be atomic—the object will be fully constructed with all its fields initialized before the program accesses it.



Good Programming Practice 26.1

Always declare data fields that you do not expect to change as `final`*. Primitive variables that are declared as* `final` *can safely be shared across threads. An object reference that's declared as* `final` *ensures that the object it refers to will be fully constructed and initialized before it's used by the program, and prevents the reference from pointing to another object.*

26.5 Producer/Consumer Relationship without Synchronization

In a **producer/consumer relationship**, the **producer** portion of an application generates data and *stores it in a shared object*, and the **consumer** portion of the application *reads data*

from the shared object. The producer/consumer relationship separates the task of identifying work to be done from the tasks involved in actually carrying out the work. One example of a common producer/consumer relationship is **print spooling**. Although a printer might not be available when you want to print from an application (i.e., the producer), you can still "complete" the print task, as the data is temporarily placed on disk until the printer becomes available. Similarly, when the printer (i.e., a consumer) is available, it doesn't have to wait until a current user wants to print. The spooled print jobs can be printed as soon as the printer becomes available. Another example of the producer/consumer relationship is an application that copies data onto DVDs by placing data in a fixed-size buffer, which is emptied as the DVD drive "burns" the data onto the DVD.

In a multithreaded producer/consumer relationship, a **producer thread** generates data and places it in a shared object called a **buffer**. A **consumer thread** reads data from the buffer. This relationship requires *synchronization* to ensure that values are produced and consumed properly. All operations on mutable data that's shared by multiple threads (e.g., the data in the buffer) must be guarded with a lock to prevent corruption, as discussed in Section 26.4. Operations on the buffer data shared by a producer and consumer thread are also **state dependent**—the operations should proceed only if the buffer is in the correct state. If the buffer is in a *not-full state*, the producer may produce; if the buffer is in a *not-empty state*, the consumer may consume. All operations that access the buffer must use synchronization to ensure that data is written to the buffer or read from the buffer only if the buffer is in the proper state. If the producer attempting to put the next data into the buffer determines that it's full, the producer thread must *wait* until there's space to write a new value. If a consumer thread finds the buffer empty or finds that the previous data has already been read, the consumer must also *wait* for new data to become available.

Consider how logic errors can arise if we do not synchronize access among multiple threads manipulating shared data. Our next example (Fig. 26.9–Fig. 26.13) implements a producer/consumer relationship without the proper synchronization. A producer thread writes the numbers 1 through 10 into a shared buffer—a single memory location shared between two threads (a single `int` variable called `buffer` in line 6 of Fig. 26.12 in this example). The consumer thread reads this data from the shared buffer and displays the data. The program's output shows the values that the producer writes (produces) into the shared buffer and the values that the consumer reads (consumes) from the shared buffer.

Each value the producer thread writes to the shared buffer must be consumed *exactly once* by the consumer thread. However, the threads in this example are not synchronized. Therefore, *data can be lost or garbled if the producer places new data into the shared buffer before the consumer reads the previous data*. Also, data can be incorrectly *duplicated* if the consumer consumes data again before the producer produces the next value. To show these possibilities, the consumer thread in the following example keeps a total of all the values it reads. The producer thread produces values from 1 through 10. If the consumer reads each value produced once and only once, the total will be 55. However, if you execute this program several times, you'll see that the total is not always 55 (as shown in the outputs in Fig. 26.13). To emphasize the point, the producer and consumer threads in the example each sleep for random intervals of up to three seconds between performing their tasks. Thus, we do not know when the producer thread will attempt to write a new value, or when the consumer thread will attempt to read a value.

Implementing the Producer/Consumer Relationship

The program consists of interface `Buffer` (Fig. 26.9) and classes `Producer` (Fig. 26.10), `Consumer` (Fig. 26.11), `UnsynchronizedBuffer` (Fig. 26.12) and `SharedBufferTest` (Fig. 26.13). Interface `Buffer` (Fig. 26.9) declares methods `set` (line 6) and `get` (line 9) that a `Buffer` (such as `UnsynchronizedBuffer`) must implement to enable the `Producer` thread to place a value in the `Buffer` and the `Consumer` thread to retrieve a value from the `Buffer`, respectively. In subsequent examples, methods `set` and `get` will call methods that throw `InterruptedExceptions`. We declare each method with a `throws` clause here so that we don't have to modify this interface for the later examples.

```
1  // Fig. 26.9: Buffer.java
2  // Buffer interface specifies methods called by Producer and Consumer.
3  public interface Buffer
4  {
5     // place int value into Buffer
6     public void set( int value ) throws InterruptedException;
7
8     // return int value from Buffer
9     public int get() throws InterruptedException;
10 } // end interface Buffer
```

Fig. 26.9 | Buffer interface specifies methods called by `Producer` and `Consumer`.

Class `Producer` (Fig. 26.10) implements the `Runnable` interface, allowing it to be executed as a task in a separate thread. The constructor (lines 11–14) initializes the `Buffer` reference `sharedLocation` with an object created in `main` (line 14 of Fig. 26.13) and passed to the constructor. As we'll see, this is an `UnsynchronizedBuffer` object that implements interface `Buffer` *without synchronizing access to the shared object*. The `Producer` thread in this program executes the tasks specified in the method `run` (lines 17–39). Each iteration of the loop (lines 21–35) invokes `Thread` method `sleep` (line 25) to place the `Producer` thread into the *timed waiting* state for a random time interval between 0 and 3 seconds. When the thread awakens, line 26 passes the value of control variable `count` to the `Buffer` object's `set` method to set the shared buffer's value. Lines 27–28 keep a total of all the values produced so far and output that value. When the loop completes, lines 37–38 display a message indicating that the `Producer` has finished producing data and is terminating. Next, method `run` terminates, which indicates that the `Producer` completed its task. Any method called from a `Runnable`'s `run` method (e.g., `Buffer` method `set`) executes as part of that task's thread of execution. This fact becomes important in Sections 26.6–26.8 when we add synchronization to the producer/consumer relationship.

```
1  // Fig. 26.10: Producer.java
2  // Producer with a run method that inserts the values 1 to 10 in buffer.
3  import java.util.Random;
4
5  public class Producer implements Runnable
6  {
```

Fig. 26.10 | `Producer` with a `run` method that inserts the values 1 to 10 in buffer. (Part 1 of 2.)

```
7     private final static Random generator = new Random();
8     private final Buffer sharedLocation; // reference to shared object
9
10    // constructor
11    public Producer( Buffer shared )
12    {
13        sharedLocation = shared;
14    } // end Producer constructor
15
16    // store values from 1 to 10 in sharedLocation
17    public void run()
18    {
19       int sum = 0;
20
21       for ( int count = 1; count <= 10; count++ )
22       {
23          try // sleep 0 to 3 seconds, then place value in Buffer
24          {
25             Thread.sleep( generator.nextInt( 3000 ) ); // random sleep
26             sharedLocation.set( count ); // set value in buffer
27             sum += count; // increment sum of values
28             System.out.printf( "\t%2d\n", sum );
29          } // end try
30          // if lines 25 or 26 get interrupted, print stack trace
31          catch ( InterruptedException exception )
32          {
33             exception.printStackTrace();
34          } // end catch
35       } // end for
36
37       System.out.println(
38          "Producer done producing\nTerminating Producer" );
39    } // end method run
40 } // end class Producer
```

Fig. 26.10 | `Producer` with a `run` method that inserts the values 1 to 10 in buffer. (Part 2 of 2.)

Class `Consumer` (Fig. 26.11) also implements interface `Runnable`, allowing the `Consumer` to execute concurrently with the `Producer`. Lines 11–14 initialize `Buffer` reference `sharedLocation` with an object that implements the `Buffer` interface (created in `main`, Fig. 26.13) and passed to the constructor as the parameter `shared`. As we'll see, this is the same `UnsynchronizedBuffer` object that's used to initialize the `Producer` object—thus, the two threads share the same object. The `Consumer` thread in this program performs the tasks specified in method `run` (lines 17–39). Lines 21–35 iterate 10 times. Each iteration invokes `Thread` method `sleep` (line 26) to put the `Consumer` thread into the *timed waiting* state for up to 3 seconds. Next, line 27 uses the `Buffer`'s `get` method to retrieve the value in the shared buffer, then adds the value to variable `sum`. Line 28 displays the total of all the values consumed so far. When the loop completes, lines 37–38 display a line indicating the sum of the consumed values. Then method `run` terminates, which indicates that the `Consumer` completed its task. Once both threads enter the *terminated* state, the program ends.

```
1  // Fig. 26.11: Consumer.java
2  // Consumer with a run method that loops, reading 10 values from buffer.
3  import java.util.Random;
4
5  public class Consumer implements Runnable
6  {
7     private final static Random generator = new Random();
8     private final Buffer sharedLocation; // reference to shared object
9
10    // constructor
11    public Consumer( Buffer shared )
12    {
13       sharedLocation = shared;
14    } // end Consumer constructor
15
16    // read sharedLocation's value 10 times and sum the values
17    public void run()
18    {
19       int sum = 0;
20
21       for ( int count = 1; count <= 10; count++ )
22       {
23          // sleep 0 to 3 seconds, read value from buffer and add to sum
24          try
25          {
26             Thread.sleep( generator.nextInt( 3000 ) );
27             sum += sharedLocation.get();
28             System.out.printf( "\t\t\t%2d\n", sum );
29          } // end try
30          // if lines 26 or 27 get interrupted, print stack trace
31          catch ( InterruptedException exception )
32          {
33             exception.printStackTrace();
34          } // end catch
35       } // end for
36
37       System.out.printf( "\n%s %d\n%s\n",
38          "Consumer read values totaling", sum, "Terminating Consumer" );
39    } // end method run
40 } // end class Consumer
```

Fig. 26.11 | `Consumer` with a `run` method that loops, reading 10 values from buffer.

[*Note:* We call method `sleep` in method `run` of the `Producer` and `Consumer` classes to emphasize the fact that, *in multithreaded applications, it's unpredictable when each thread will perform its task and for how long it will perform the task when it has a processor*. Normally, these thread scheduling issues are beyond the control of the Java developer. In this program, our thread's tasks are quite simple—the `Producer` writes the values 1 to 10 to the buffer, and the `Consumer` reads 10 values from the buffer and adds each value to variable `sum`. Without the `sleep` method call, and if the `Producer` executes first, given today's phenomenally fast processors, the `Producer` would likely complete its task before the `Consumer` got a chance to execute. If the `Consumer` executed first, it would likely consume garbage data ten times, then terminate before the `Producer` could produce the first real value.]

Class `UnsynchronizedBuffer` (Fig. 26.12) implements interface `Buffer` (line 4). An object of this class is shared between the `Producer` and the `Consumer`. Line 6 declares instance variable `buffer` and initializes it with the value `-1`. This value is used to demonstrate the case in which the `Consumer` attempts to consume a value *before* the `Producer` ever places a value in `buffer`. Methods `set` (lines 9–13) and `get` (lines 16–20) do *not* synchronize access to the field `buffer`. Method `set` simply assigns its argument to `buffer` (line 12), and method `get` simply returns the value of `buffer` (line 19).

```
1  // Fig. 26.12: UnsynchronizedBuffer.java
2  // UnsynchronizedBuffer maintains the shared integer that is accessed by
3  // a producer thread and a consumer thread via methods set and get.
4  public class UnsynchronizedBuffer implements Buffer
5  {
6     private int buffer = -1; // shared by producer and consumer threads
7
8     // place value into buffer
9     public void set( int value ) throws InterruptedException
10    {
11       System.out.printf( "Producer writes\t%2d", value );
12       buffer = value;
13    } // end method set
14
15    // return value from buffer
16    public int get() throws InterruptedException
17    {
18       System.out.printf( "Consumer reads\t%2d", buffer );
19       return buffer;
20    } // end method get
21 } // end class UnsynchronizedBuffer
```

Fig. 26.12 | `UnsynchronizedBuffer` maintains the shared integer that is accessed by a producer thread and a consumer thread via methods `set` and `get`.

In class `SharedBufferTest` (Fig. 26.13), line 11 creates an `ExecutorService` to execute the `Producer` and `Consumer` `Runnable`s. Line 14 creates an `UnsynchronizedBuffer` object and assigns it to `Buffer` variable `sharedLocation`. This object stores the data that the `Producer` and `Consumer` threads will share. Lines 23–24 create and execute the `Producer` and `Consumer`. The `Producer` and `Consumer` constructors are each passed the same `Buffer` object (`sharedLocation`), so each object is initialized with a reference to the same `Buffer`. These lines also implicitly launch the threads and call each `Runnable`'s `run` method. Finally, line 26 calls method `shutdown` so that the application can terminate when the threads executing the `Producer` and `Consumer` complete their tasks. When `main` terminates (line 27), the main thread of execution enters the *terminated* state.

```
1  // Fig. 26.13: SharedBufferTest.java
2  // Application with two threads manipulating an unsynchronized buffer.
3  import java.util.concurrent.ExecutorService;
4  import java.util.concurrent.Executors;
```

Fig. 26.13 | Application with two threads manipulating an unsynchronized buffer. (Part 1 of 3.)

```
5
6  public class SharedBufferTest
7  {
8     public static void main( String[] args )
9     {
10       // create new thread pool with two threads
11       ExecutorService application = Executors.newCachedThreadPool();
12
13       // create UnsynchronizedBuffer to store ints
14       Buffer sharedLocation = new UnsynchronizedBuffer();
15
16       System.out.println(
17          "Action\t\tValue\tSum of Produced\tSum of Consumed" );
18       System.out.println(
19          "------\t\t-----\t---------------\t---------------\n" );
20
21       // execute the Producer and Consumer, giving each of them access
22       // to sharedLocation
23       application.execute( new Producer( sharedLocation ) );
24       application.execute( new Consumer( sharedLocation ) );
25
26       application.shutdown(); // terminate application when tasks complete
27    } // end main
28 } // end class SharedBufferTest
```

```
Action          Value   Sum of Produced Sum of Consumed
------          -----   --------------- ---------------

Producer writes  1        1
Producer writes  2        3                 -- 1 is lost
Producer writes  3        6                 -- 2 is lost
Consumer reads   3                        3
Producer writes  4       10
Consumer reads   4                        7
Producer writes  5       15
Producer writes  6       21                 -- 5 is lost
Producer writes  7       28                 -- 6 is lost
Consumer reads   7                       14
Consumer reads   7                       21 -- 7 read again
Producer writes  8       36
Consumer reads   8                       29
Consumer reads   8                       37 -- 8 read again
Producer writes  9       45
Producer writes 10       55                 -- 9 is lost

Producer done producing
Terminating Producer
Consumer reads  10                       47
Consumer reads  10                       57 -- 10 read again
Consumer reads  10                       67 -- 10 read again
Consumer reads  10                       77 -- 10 read again

Consumer read values totaling 77
Terminating Consumer
```

Fig. 26.13 | Application with two threads manipulating an unsynchronized buffer. (Part 2 of 3.)

```
Action          Value   Sum of Produced Sum of Consumed
------          -----   --------------- ---------------

Consumer reads  -1                      -1 ---- reads -1 bad data
Producer writes  1       1
Consumer reads   1                      0
Consumer reads   1                      1 ---- 1 read again
Consumer reads   1                      2 ---- 1 read again
Consumer reads   1                      3 ---- 1 read again
Consumer reads   1                      4 ---- 1 read again
Producer writes  2       3
Consumer reads   2                      6
Producer writes  3       6
Consumer reads   3                      9
Producer writes  4      10
Consumer reads   4                     13
Producer writes  5      15
Producer writes  6      21                ---- 5 is lost
Consumer reads   6                     19

Consumer read values totaling 19
Terminating Consumer
Producer writes  7      28                ---- 7 never read
Producer writes  8      36                ---- 8 never read
Producer writes  9      45                ---- 9 never read
Producer writes 10      55                ---- 10 never read

Producer done producing
Terminating Producer
```

Fig. 26.13 | Application with two threads manipulating an unsynchronized buffer. (Part 3 of 3.)

Recall from the overview of this example that we would like the `Producer` to execute first and every value produced by the `Producer` to be consumed exactly once by the `Consumer`. However, when we study the first output of Fig. 26.13, we see that the `Producer` writes the values `1`, `2` and `3` before the `Consumer` reads its first value (`3`). Therefore, the values `1` and `2` are lost. Later, the values `5`, `6` and `9` are lost, while `7` and `8` are read twice and `10` is read four times. So the first output produces an incorrect total of 77, instead of the correct total of 55. In the second output, the `Consumer` reads the value `-1` before the `Producer` ever writes a value. The `Consumer` reads the value `1` five times before the `Producer` writes the value `2`. Meanwhile, the values `5`, `7`, `8`, `9` and `10` are all lost—the last four because the `Consumer` terminates before the `Producer`. An incorrect consumer total of 19 is displayed. (Lines in the output where the `Producer` or `Consumer` has acted out of order are highlighted.)



Error-Prevention Tip 26.1

Access to a shared object by concurrent threads must be controlled carefully or a program may produce incorrect results.

To solve the problems of lost and duplicated data, Section 26.6 presents an example in which we use an `ArrayBlockingQueue` (from package `java.util.concurrent`) to synchronize access to the shared object, guaranteeing that each and every value will be processed once and only once.

26.6 Producer/Consumer Relationship: ArrayBlockingQueue

One way to synchronize producer and consumer threads is to use classes from Java's concurrency package that *encapsulate the synchronization for you*. Java includes the class **ArrayBlockingQueue** (from package `java.util.concurrent`)—a fully implemented, *thread-safe buffer class* that implements interface **BlockingQueue**. This interface extends the `Queue` interface discussed in Chapter 20 and declares methods **put** and **take**, the blocking equivalents of `Queue` methods `offer` and `poll`, respectively. Method `put` places an element at the end of the `BlockingQueue`, waiting if the queue is full. Method `take` removes an element from the head of the `BlockingQueue`, waiting if the queue is empty. These methods make class `ArrayBlockingQueue` a good choice for implementing a shared buffer. Because method `put` blocks until there's room in the buffer to write data, and method `take` blocks until there's new data to read, the producer must produce a value first, the consumer correctly consumes only after the producer writes a value and the producer correctly produces the next value (after the first) only after the consumer reads the previous (or first) value. `ArrayBlockingQueue` stores the shared data in an array. The array's size is specified as an argument to the `ArrayBlockingQueue` constructor. Once created, an `ArrayBlockingQueue` is fixed in size and will not expand to accommodate extra elements.

Figures 26.14–26.15 demonstrate a `Producer` and a `Consumer` accessing an `ArrayBlockingQueue`. Class `BlockingBuffer` (Fig. 26.14) uses an `ArrayBlockingQueue` object that stores an `Integer` (line 7). Line 11 creates the `ArrayBlockingQueue` and passes `1` to the constructor so that the object holds a single value, as we did with the `UnsynchronizedBuffer` of Fig. 26.12. Lines 7 and 11 use generics, which we discussed in Chapters 20–21. We discuss multiple-element buffers in Section 26.8. Because our `BlockingBuffer` class uses the thread-safe `ArrayBlockingQueue` class to manage access to the shared buffer, `BlockingBuffer` is itself *thread safe*, even though we have not implemented the synchronization ourselves.

```
1  // Fig. 26.14: BlockingBuffer.java
2  // Creating a synchronized buffer using an ArrayBlockingQueue.
3  import java.util.concurrent.ArrayBlockingQueue;
4
5  public class BlockingBuffer implements Buffer
6  {
7     private final ArrayBlockingQueue<Integer> buffer; // shared buffer
8
9     public BlockingBuffer()
10    {
11       buffer = new ArrayBlockingQueue<Integer>( 1 );
12    } // end BlockingBuffer constructor
13
14    // place value into buffer
15    public void set( int value ) throws InterruptedException
16    {
17       buffer.put( value ); // place value in buffer
```

Fig. 26.14 | Creating a synchronized buffer using an `ArrayBlockingQueue`. (Part 1 of 2.)

```
18          System.out.printf( "%s%2d\t%s%d\n", "Producer writes ", value,
19             "Buffer cells occupied: ", buffer.size() );
20       } // end method set
21
22       // return value from buffer
23       public int get() throws InterruptedException
24       {
25          int readValue = buffer.take(); // remove value from buffer
26          System.out.printf( "%s %2d\t%s%d\n", "Consumer reads ",
27             readValue, "Buffer cells occupied: ", buffer.size() );
28
29          return readValue;
30       } // end method get
31    } // end class BlockingBuffer
```

Fig. 26.14 | Creating a synchronized buffer using an `ArrayBlockingQueue`. (Part 2 of 2.)

`BlockingBuffer` implements interface `Buffer` (Fig. 26.9) and uses classes `Producer` (Fig. 26.10 modified to remove line 28) and `Consumer` (Fig. 26.11 modified to remove line 28) from the example in Section 26.5. This approach demonstrates that *the threads accessing the shared object are unaware that their buffer accesses are now synchronized*. The synchronization is handled entirely in the `set` and `get` methods of `BlockingBuffer` by calling the synchronized `ArrayBlockingQueue` methods `put` and `take`, respectively. Thus, the `Producer` and `Consumer` `Runnable`s are properly synchronized simply by calling the shared object's `set` and `get` methods.

Line 17 in method `set` (Fig. 26.14, lines 15–20) calls the `ArrayBlockingQueue` object's `put` method. This method call blocks if necessary until there's room in the `buffer` to place the `value`. Method `get` (lines 23–30) calls the `ArrayBlockingQueue` object's `take` method (line 25). This method call blocks if necessary until there's an element in the `buffer` to remove. Lines 18–19 and 26–27 use the `ArrayBlockingQueue` object's **`size`** method to display the total number of elements currently in the `ArrayBlockingQueue`.

Class `BlockingBufferTest` (Fig. 26.15) contains the `main` method that launches the application. Line 12 creates an `ExecutorService`, and line 15 creates a `BlockingBuffer` object and assigns its reference to the `Buffer` variable `sharedLocation`. Lines 17–18 execute the `Producer` and `Consumer` `Runnable`s. Line 19 calls method `shutdown` to end the application when the threads finish executing the `Producer` and `Consumer` tasks.

```
1  // Fig. 26.15: BlockingBufferTest.java
2  // Two threads manipulating a blocking buffer that properly
3  // implements the producer/consumer relationship.
4  import java.util.concurrent.ExecutorService;
5  import java.util.concurrent.Executors;
6
7  public class BlockingBufferTest
8  {
9     public static void main( String[] args )
10    {
```

Fig. 26.15 | Two threads manipulating a blocking buffer that properly implements the producer/consumer relationship. (Part 1 of 2.)

```
11         // create new thread pool with two threads
12         ExecutorService application = Executors.newCachedThreadPool();
13
14         // create BlockingBuffer to store ints
15         Buffer sharedLocation = new BlockingBuffer();
16
17         application.execute( new Producer( sharedLocation ) );
18         application.execute( new Consumer( sharedLocation ) );
19
20         application.shutdown();
21      } // end main
22   } // end class BlockingBufferTest
```

```
Producer writes  1      Buffer cells occupied: 1
Consumer reads   1      Buffer cells occupied: 0
Producer writes  2      Buffer cells occupied: 1
Consumer reads   2      Buffer cells occupied: 0
Producer writes  3      Buffer cells occupied: 1
Consumer reads   3      Buffer cells occupied: 0
Producer writes  4      Buffer cells occupied: 1
Consumer reads   4      Buffer cells occupied: 0
Producer writes  5      Buffer cells occupied: 1
Consumer reads   5      Buffer cells occupied: 0
Producer writes  6      Buffer cells occupied: 1
Consumer reads   6      Buffer cells occupied: 0
Producer writes  7      Buffer cells occupied: 1
Consumer reads   7      Buffer cells occupied: 0
Producer writes  8      Buffer cells occupied: 1
Consumer reads   8      Buffer cells occupied: 0
Producer writes  9      Buffer cells occupied: 1
Consumer reads   9      Buffer cells occupied: 0
Producer writes 10      Buffer cells occupied: 1

Producer done producing
Terminating Producer
Consumer reads  10      Buffer cells occupied: 0

Consumer read values totaling 55
Terminating Consumer
```

Fig. 26.15 | Two threads manipulating a blocking buffer that properly implements the producer/consumer relationship. (Part 2 of 2.)

While methods `put` and `take` of `ArrayBlockingQueue` are properly synchronized, `BlockingBuffer` methods `set` and `get` (Fig. 26.14) are not declared to be synchronized. Thus, the statements performed in method `set`—the `put` operation (line 17) and the output (lines 18–19)—are *not atomic*; nor are the statements in method `get`—the `take` operation (line 25) and the output (lines 26–27). So there's no guarantee that each output will occur immediately after the corresponding `put` or `take` operation, and the outputs may appear out of order. Even if they do, the `ArrayBlockingQueue` object is properly synchronizing access to the data, as evidenced by the fact that the sum of values read by the consumer is always correct.

26.7 Producer/Consumer Relationship with Synchronization

The previous example showed how multiple threads can share a single-element buffer in a thread-safe manner by using the `ArrayBlockingQueue` class that encapsulates the synchronization necessary to protect the shared data. For educational purposes, we now explain how you can implement a shared buffer yourself using the `synchronized` keyword and methods of class `Object`. Using an `ArrayBlockingQueue` will result in more-maintainable and better-performing code.

The first step in synchronizing access to the buffer is to implement methods `get` and `set` as `synchronized` methods. This requires that a thread obtain the *monitor lock* on the `Buffer` object before attempting to access the buffer data, but it does not automatically ensure that threads proceed with an operation only if the buffer is in the proper state. We need a way to allow our threads to wait, depending on whether certain conditions are true. In the case of placing a new item in the buffer, the condition that allows the operation to proceed is that the *buffer is not full*. In the case of fetching an item from the buffer, the condition that allows the operation to proceed is that the *buffer is not empty*. If the condition in question is true, the operation may proceed; if it's false, the thread must wait until it becomes true. When a thread is waiting on a condition, it's removed from contention for the processor and placed into the *waiting* state and the lock it holds is released.

Methods `wait`, `notify` *and* `notifyAll`

`Object` methods `wait`, `notify` and `notifyAll`, which are inherited by all other classes, can be used with conditions to make threads *wait* when they cannot perform their tasks. If a thread obtains the *monitor lock* on an object, then determines that it cannot continue with its task on that object until some condition is satisfied, the thread can call `Object` method **`wait`** on the `synchronized` object; this releases the monitor lock on the object, and the thread waits in the *waiting* state while the other threads try to enter the object's `synchronized` statement(s) or method(s). When a thread executing a `synchronized` statement (or method) completes or satisfies the condition on which another thread may be waiting, it can call `Object` method **`notify`** on the `synchronized` object to allow a waiting thread to transition to the *runnable* state again. At this point, the thread that was transitioned from the *waiting* state to the *runnable* state can attempt to reacquire the monitor lock on the object. Even if the thread is able to reacquire the monitor lock, it still might not be able to perform its task at this time—in which case the thread will reenter the *waiting* state and implicitly release the monitor lock. If a thread calls **`notifyAll`** on the `synchronized` object, then *all* the threads waiting for the monitor lock become eligible to reacquire the lock (that is, they all transition to the *runnable* state).

Remember that only one thread at a time can obtain the monitor lock on the object—other threads that attempt to acquire the same monitor lock will be *blocked* until the monitor lock becomes available again (i.e., until no other thread is executing in a `synchronized` statement on that object).



Common Programming Error 26.1

It's an error if a thread issues a `wait`*, a* `notify` *or a* `notifyAll` *on an object without having acquired a lock for it. This causes an* **`IllegalMonitorStateException`**.



Error-Prevention Tip 26.2

It's a good practice to use `notifyAll` *to notify* waiting *threads to become* runnable. *Doing so avoids the possibility that your program would forget about waiting threads, which would otherwise starve.*

The application in Fig. 26.16 and Fig. 26.17 demonstrates a `Producer` and a `Consumer` accessing a shared buffer with synchronization. In this case, the `Producer` always produces a value *first*, the `Consumer` correctly consumes only *after* the `Producer` produces a value and the `Producer` correctly produces the next value only after the `Consumer` consumes the previous (or first) value. We reuse interface `Buffer` and classes `Producer` and `Consumer` from the example in Section 26.5, except that line 28 is removed from class `Producer` and class `Consumer`. The synchronization is handled in the `set` and `get` methods of class `SynchronizedBuffer` (Fig. 26.16), which implements interface `Buffer` (line 4). Thus, the `Producer`'s and `Consumer`'s `run` methods simply call the shared object's `synchronized` `set` and `get` methods.

```
1  // Fig. 26.16: SynchronizedBuffer.java
2  // Synchronizing access to shared data using Object
3  // methods wait and notifyAll.
4  public class SynchronizedBuffer implements Buffer
5  {
6     private int buffer = -1; // shared by producer and consumer threads
7     private boolean occupied = false; // whether the buffer is occupied
8
9     // place value into buffer
10    public synchronized void set( int value ) throws InterruptedException
11    {
12       // while there are no empty locations, place thread in waiting state
13       while ( occupied )
14       {
15          // output thread information and buffer information, then wait
16          System.out.println( "Producer tries to write." );
17          displayState( "Buffer full. Producer waits." );
18          wait();
19       } // end while
20
21       buffer = value; // set new buffer value
22
23       // indicate producer cannot store another value
24       // until consumer retrieves current buffer value
25       occupied = true;
26
27       displayState( "Producer writes " + buffer );
28
29       notifyAll(); // tell waiting thread(s) to enter runnable state
30    } // end method set; releases lock on SynchronizedBuffer
31
```

Fig. 26.16 | Synchronizing access to shared data using `Object` methods `wait` and `notifyAll`. (Part 1 of 2.)

```
32      // return value from buffer
33      public synchronized int get() throws InterruptedException
34      {
35         // while no data to read, place thread in waiting state
36         while ( !occupied )
37         {
38            // output thread information and buffer information, then wait
39            System.out.println( "Consumer tries to read." );
40            displayState( "Buffer empty. Consumer waits." );
41            wait();
42         } // end while
43
44         // indicate that producer can store another value
45         // because consumer just retrieved buffer value
46         occupied = false;
47
48         displayState( "Consumer reads " + buffer );
49
50         notifyAll(); // tell waiting thread(s) to enter runnable state
51
52         return buffer;
53      } // end method get; releases lock on SynchronizedBuffer
54
55      // display current operation and buffer state
56      public void displayState( String operation )
57      {
58         System.out.printf( "%-40s%d\t\t%b\n\n", operation, buffer,
59            occupied );
60      } // end method displayState
61   } // end class SynchronizedBuffer
```

Fig. 26.16 | Synchronizing access to shared data using `Object` methods `wait` and `notifyAll`. (Part 2 of 2.)

Fields and Methods of Class `SynchronizedBuffer`

Class `SynchronizedBuffer` contains fields `buffer` (line 6) and `occupied` (line 7). Methods `set` (lines 10–30) and `get` (lines 33–53) are declared as `synchronized`—only one thread can call either of these methods at a time on a particular `SynchronizedBuffer` object. Field `occupied` is used to determine whether it's the `Producer`'s or the `Consumer`'s turn to perform a task. This field is used in conditional expressions in both the `set` and `get` methods. If `occupied` is `false`, then `buffer` is empty, so the `Consumer` cannot read the value of `buffer`, but the `Producer` can place a value into `buffer`. If `occupied` is `true`, the `Consumer` can read a value from `buffer`, but the `Producer` cannot place a value into `buffer`.

Method `set` *and the* `Producer` *Thread*

When the `Producer` thread's `run` method invokes `synchronized` method `set`, the thread implicitly attempts to acquire the `SynchronizedBuffer` object's monitor lock. If the monitor lock is available, the `Producer` thread implicitly acquires the lock. Then the loop at lines 13–19 first determines whether `occupied` is `true`. If so, `buffer` is full, so line 16 outputs a message indicating that the `Producer` thread is trying to write a value, and line 17 invokes method `displayState` (lines 56–60) to output another message indicating that `buffer` is

full and that the Producer thread is waiting until there's space. Line 18 invokes method wait (inherited from Object by SynchronizedBuffer) to place the thread that called method set (i.e., the Producer thread) in the *waiting* state for the SynchronizedBuffer object. The call to wait causes the calling thread to *implicitly* release the lock on the SynchronizedBuffer object. This is important because the thread cannot currently perform its task and because other threads (in this case, the Consumer) should be allowed to access the object to allow the condition (occupied) to change. Now another thread can attempt to acquire the SynchronizedBuffer object's lock and invoke the object's set or get method.

The Producer thread remains in the *waiting* state until another thread notifies the Producer that it may proceed—at which point the Producer returns to the *runnable* state and attempts to implicitly reacquire the lock on the SynchronizedBuffer object. If the lock is available, the Producer thread reacquires it, and method set continues executing with the next statement after the wait call. Because wait is called in a loop, the loop-continuation condition is tested again to determine whether the thread can proceed. If not, then wait is invoked again—otherwise, method set continues with the next statement after the loop.

Line 21 in method set assigns the value to the buffer. Line 25 sets occupied to true to indicate that the buffer now contains a value (i.e., a consumer can read the value, but a Producer cannot yet put another value there). Line 27 invokes method displayState to output a message indicating that the Producer is writing a new value into the buffer. Line 29 invokes method notifyAll (inherited from Object). If any threads are waiting on the SynchronizedBuffer object's monitor lock, those threads enter the *runnable* state and can now attempt to reacquire the lock. Method notifyAll returns immediately, and method set then returns to the caller (i.e., the Producer's run method). When method set returns, it implicitly releases the monitor lock on the SynchronizedBuffer object.

Method `get` *and the* `Consumer` *Thread*

Methods get and set are implemented similarly. When the Consumer thread's run method invokes synchronized method get, the thread attempts to acquire the *monitor lock* on the SynchronizedBuffer object. If the lock is available, the Consumer thread acquires it. Then the while loop at lines 36–42 determines whether occupied is false. If so, the buffer is empty, so line 39 outputs a message indicating that the Consumer thread is trying to read a value, and line 40 invokes method displayState to output a message indicating that the buffer is empty and that the Consumer thread is waiting. Line 41 invokes method wait to place the thread that called method get (i.e., the Consumer) in the *waiting* state for the SynchronizedBuffer object. Again, the call to wait causes the calling thread to implicitly release the lock on the SynchronizedBuffer object, so another thread can attempt to acquire the SynchronizedBuffer object's lock and invoke the object's set or get method. If the lock on the SynchronizedBuffer is not available (e.g., if the Producer has not yet returned from method set), the Consumer is blocked until the lock becomes available.

The Consumer thread remains in the *waiting* state until it's notified by another thread that it may proceed—at which point the Consumer thread returns to the *runnable* state and attempts to implicitly reacquire the lock on the SynchronizedBuffer object. If the lock is available, the Consumer reacquires it, and method get continues executing with the next statement after wait. Because wait is called in a loop, the loop-continuation condition is tested again to determine whether the thread can proceed with its execution. If not, wait is invoked again—otherwise, method get continues with the next statement after the loop.

Line 46 sets `occupied` to `false` to indicate that `buffer` is now empty (i.e., a `Consumer` cannot read the value, but a `Producer` can place another value in `buffer`), line 48 calls method `displayState` to indicate that the consumer is reading and line 50 invokes method `notifyAll`. If any threads are in the *waiting* state for the lock on this `SynchronizedBuffer` object, they enter the *runnable* state and can now attempt to reacquire the lock. Method `notifyAll` returns immediately, then method `get` returns the value of `buffer` to its caller. When method `get` returns, the lock on the `SynchronizedBuffer` object is implicitly released.



Error-Prevention Tip 26.3

Always invoke method `wait` *in a loop that tests the condition the task is waiting on. It's possible that a thread will reenter the* runnable *state (via a timed wait or another thread calling* `notifyAll`*) before the condition is satisfied. Testing the condition again ensures that the thread will not erroneously execute if it was notified early.*

Testing Class `SynchronizedBuffer`

Class `SharedBufferTest2` (Fig. 26.17) is similar to class `SharedBufferTest` (Fig. 26.13). `SharedBufferTest2` contains method `main` (lines 8–24), which launches the application. Line 11 creates an `ExecutorService` to run the `Producer` and `Consumer` tasks. Line 14 creates a `SynchronizedBuffer` object and assigns its reference to `Buffer` variable `sharedLocation`. This object stores the data that will be shared between the `Producer` and `Consumer`. Lines 16–17 display the column heads for the output. Lines 20–21 execute a `Producer` and a `Consumer`. Finally, line 23 calls method `shutdown` to end the application when the `Producer` and `Consumer` complete their tasks. When method `main` ends (line 24), the main thread of execution terminates.

```
1  // Fig. 26.17: SharedBufferTest2.java
2  // Two threads correctly manipulating a synchronized buffer.
3  import java.util.concurrent.ExecutorService;
4  import java.util.concurrent.Executors;
5
6  public class SharedBufferTest2
7  {
8     public static void main( String[] args )
9     {
10       // create a newCachedThreadPool
11       ExecutorService application = Executors.newCachedThreadPool();
12
13       // create SynchronizedBuffer to store ints
14       Buffer sharedLocation = new SynchronizedBuffer();
15
16       System.out.printf( "%-40s%s\t\t%s\n%-40s%s\n\n", "Operation",
17          "Buffer", "Occupied", "---------", "------\t\t--------" );
18
19       // execute the Producer and Consumer tasks
20       application.execute( new Producer( sharedLocation ) );
21       application.execute( new Consumer( sharedLocation ) );
22
```

Fig. 26.17 | Two threads correctly manipulating a synchronized buffer. (Part 1 of 3.)

```
23         application.shutdown();
24      } // end main
25   } // end class SharedBufferTest2
```

```
Operation                               Buffer          Occupied
---------                               ------          --------

Consumer tries to read.
Buffer empty. Consumer waits.           -1              false

Producer writes 1                       1               true

Consumer reads 1                        1               false

Consumer tries to read.
Buffer empty. Consumer waits.           1               false

Producer writes 2                       2               true

Consumer reads 2                        2               false

Producer writes 3                       3               true

Consumer reads 3                        3               false

Producer writes 4                       4               true

Producer tries to write.
Buffer full. Producer waits.            4               true

Consumer reads 4                        4               false

Producer writes 5                       5               true

Consumer reads 5                        5               false

Producer writes 6                       6               true

Producer tries to write.
Buffer full. Producer waits.            6               true

Consumer reads 6                        6               false

Producer writes 7                       7               true

Producer tries to write.
Buffer full. Producer waits.            7               true

Consumer reads 7                        7               false

Producer writes 8                       8               true

Consumer reads 8                        8               false

Consumer tries to read.
Buffer empty. Consumer waits.           8               false
```

Fig. 26.17 | Two threads correctly manipulating a synchronized buffer. (Part 2 of 3.)

```
Producer writes 9                       9               true

Consumer reads 9                        9               false

Consumer tries to read.
Buffer empty. Consumer waits.           9               false

Producer writes 10                      10              true

Consumer reads 10                       10              false


Producer done producing
Terminating Producer

Consumer read values totaling 55
Terminating Consumer
```

Fig. 26.17 | Two threads correctly manipulating a synchronized buffer. (Part 3 of 3.)

Study the outputs in Fig. 26.17. Observe that *every integer produced is consumed exactly once—no values are lost, and no values are consumed more than once*. The synchronization ensures that the `Producer` produces a value only when the buffer is empty and the `Consumer` consumes only when the buffer is full. The `Producer` always goes first, the `Consumer` waits if the `Producer` has not produced since the `Consumer` last consumed, and the `Producer` waits if the `Consumer` has not yet consumed the value that the `Producer` most recently produced. Execute this program several times to confirm that every integer produced is consumed exactly once. In the sample output, note the highlighted lines indicating when the `Producer` and `Consumer` must wait to perform their respective tasks.

26.8 Producer/Consumer Relationship: Bounded Buffers

The program in Section 26.7 uses thread synchronization to guarantee that two threads manipulate data in a shared buffer correctly. However, the application may not perform optimally. If the two threads operate at different speeds, one them will spend more (or most) of its time waiting. For example, in the program in Section 26.7 we shared a single integer variable between the two threads. If the `Producer` thread produces values faster than the `Consumer` can consume them, then the `Producer` thread *waits* for the `Consumer`, because there are no other locations in the buffer in which to place the next value. Similarly, if the `Consumer` consumes values faster than the `Producer` produces them, the `Consumer` *waits* until the `Producer` places the next value in the shared buffer. Even when we have threads that operate at the same relative speeds, those threads may occasionally become "out of sync" over a period of time, causing one of them to wait for the other. *We cannot make assumptions about the relative speeds of concurrent threads*—interactions that occur with the operating system, the network, the user and other components can cause the threads to operate at different and ever-changing speeds. When this happens, threads wait. When threads wait excessively, programs become less efficient, interactive programs become less responsive and applications suffer longer delays.

Bounded Buffers

To minimize the amount of waiting time for threads that share resources and operate at the same average speeds, we can implement a **bounded buffer** that provides a fixed number of buffer cells into which the `Producer` can place values, and from which the `Consumer` can retrieve those values. (In fact, we've already done this with the `ArrayBlockingQueue` class in Section 26.6.) If the `Producer` temporarily produces values faster than the `Consumer` can consume them, the `Producer` can write additional values into the extra buffer cells, if any are available. This capability enables the `Producer` to perform its task even though the `Consumer` is not ready to retrieve the current value being produced. Similarly, if the `Consumer` consumes faster than the `Producer` produces new values, the `Consumer` can read additional values (if there are any) from the buffer. This enables the `Consumer` to keep busy even though the `Producer` is not ready to produce additional values.

Even a *bounded buffer* is inappropriate if the `Producer` and the `Consumer` operate consistently at different speeds. If the `Consumer` always executes faster than the `Producer`, then a buffer containing one location is enough. Additional locations would simply waste memory. If the `Producer` always executes faster, only a buffer with an "infinite" number of locations would be able to absorb the extra production. However, if the `Producer` and `Consumer` execute at about the same average speed, a bounded buffer helps to smooth the effects of any occasional speeding up or slowing down in either thread's execution.

The key to using a *bounded buffer* with a `Producer` and `Consumer` that operate at about the same speed is to provide the buffer with enough locations to handle the anticipated "extra" production. If, over a period of time, we determine that the `Producer` often produces as many as three more values than the `Consumer` can consume, we can provide a buffer of at least three cells to handle the extra production. Making the buffer too small would cause threads to wait longer; making the buffer too large would waste memory.



Performance Tip 26.3

Even when using a bounded buffer*, it's possible that a producer thread could fill the buffer, which would force the producer to wait until a consumer consumed a value to free an element in the buffer. Similarly, if the buffer is empty at any given time, a consumer thread must wait until the producer produces another value. The key to using a bounded buffer is to optimize the buffer size to minimize the amount of thread wait time, while not wasting space.*

Bounded Buffers Using `ArrayBlockingQueue`

The simplest way to implement a bounded buffer is to use an `ArrayBlockingQueue` for the buffer so that *all of the synchronization details are handled for you*. This can be done by modifying the example from Section 26.6 to pass the desired size for the bounded buffer into the `ArrayBlockingQueue` constructor. Rather than repeat our previous `ArrayBlockingQueue` example with a different size, we instead present an example that illustrates how you can build a bounded buffer yourself. Again, using an `ArrayBlockingQueue` will result in more-maintainable and better-performing code. In Exercise 26.11, we ask you to reimplement this section's example, using the Java Concurrency API techniques presented in Section 26.9.

Implementing Your Own Bounded Buffer as a Circular Buffer

The program in Fig. 26.18 and Fig. 26.19 demonstrates a `Producer` and a `Consumer` accessing a *bounded buffer with synchronization*. Again, we reuse interface `Buffer` and classes

`Producer` and `Consumer` from the example in Section 26.5, except that line 28 is removed from class `Producer` and class `Consumer`. We implement the bounded buffer in class `CircularBuffer` (Fig. 26.18) as a **circular buffer** that uses a shared array of three elements. A circular buffer writes into and reads from the array elements in order, beginning at the first cell and moving toward the last. When a `Producer` or `Consumer` reaches the last element, it returns to the first and begins writing or reading, respectively, from there. In this version of the producer/consumer relationship, the `Consumer` consumes a value only when the array is not empty and the `Producer` produces a value only when the array is not full. The statements that created and started the thread objects in the `main` method of class `SharedBufferTest2` (Fig. 26.17) now appear in class `CircularBufferTest` (Fig. 26.19).

```
1  // Fig. 26.18: CircularBuffer.java
2  // Synchronizing access to a shared three-element bounded buffer.
3  public class CircularBuffer implements Buffer
4  {
5     private final int[] buffer = { -1, -1, -1 }; // shared buffer
6
7     private int occupiedCells = 0; // count number of buffers used
8     private int writeIndex = 0; // index of next element to write to
9     private int readIndex = 0; // index of next element to read
10
11    // place value into buffer
12    public synchronized void set( int value ) throws InterruptedException
13    {
14       // wait until buffer has space available, then write value;
15       // while no empty locations, place thread in blocked state
16       while ( occupiedCells == buffer.length )
17       {
18          System.out.printf( "Buffer is full. Producer waits.\n" );
19          wait(); // wait until a buffer cell is free
20       } // end while
21
22       buffer[ writeIndex ] = value; // set new buffer value
23
24       // update circular write index
25       writeIndex = ( writeIndex + 1 ) % buffer.length;
26
27       ++occupiedCells; // one more buffer cell is full
28       displayState( "Producer writes " + value );
29       notifyAll(); // notify threads waiting to read from buffer
30    } // end method set
31
32    // return value from buffer
33    public synchronized int get() throws InterruptedException
34    {
35       // wait until buffer has data, then read value;
36       // while no data to read, place thread in waiting state
37       while ( occupiedCells == 0 )
38       {
39          System.out.printf( "Buffer is empty. Consumer waits.\n" );
```

Fig. 26.18 | Synchronizing access to a shared three-element bounded buffer. (Part 1 of 2.)

```
40            wait(); // wait until a buffer cell is filled
41         } // end while
42
43         int readValue = buffer[ readIndex ]; // read value from buffer
44
45         // update circular read index
46         readIndex = ( readIndex + 1 ) % buffer.length;
47
48         --occupiedCells; // one fewer buffer cells are occupied
49         displayState( "Consumer reads " + readValue );
50         notifyAll(); // notify threads waiting to write to buffer
51
52         return readValue;
53      } // end method get
54
55      // display current operation and buffer state
56      public void displayState( String operation )
57      {
58         // output operation and number of occupied buffer cells
59         System.out.printf( "%s%s%d)\n%s", operation,
60            " (buffer cells occupied: ", occupiedCells, "buffer cells:  " );
61
62         for ( int value : buffer )
63            System.out.printf( " %2d  ", value ); // output values in buffer
64
65         System.out.print( "\n                " );
66
67         for ( int i = 0; i < buffer.length; i++ )
68            System.out.print( "---- " );
69
70         System.out.print( "\n                " );
71
72         for ( int i = 0; i < buffer.length; i++ )
73         {
74            if ( i == writeIndex && i == readIndex )
75               System.out.print( " WR" ); // both write and read index
76            else if ( i == writeIndex )
77               System.out.print( " W   " ); // just write index
78            else if ( i == readIndex )
79               System.out.print( "  R  " ); // just read index
80            else
81               System.out.print( "     " ); // neither index
82         } // end for
83
84         System.out.println( "\n" );
85      } // end method displayState
86   } // end class CircularBuffer
```

Fig. 26.18 | Synchronizing access to a shared three-element bounded buffer. (Part 2 of 2.)

Line 5 initializes array `buffer` as a three-element `int` array that represents the circular buffer. Variable `occupiedCells` (line 7) counts the number of elements in `buffer` that contain data to be read. When `occupiedBuffers` is 0, there's no data in the circular buffer and the `Consumer` must wait—when `occupiedCells` is 3 (the size of the circular buffer),

the circular buffer is full and the `Producer` must wait. Variable `writeIndex` (line 8) indicates the next location in which a value can be placed by a `Producer`. Variable `readIndex` (line 9) indicates the position from which the next value can be read by a `Consumer`.

`CircularBuffer` Method `set`

`CircularBuffer` method `set` (lines 12–30) performs the same tasks as in Fig. 26.16, with a few modifications. The loop at lines 16–20 determines whether the `Producer` must wait (i.e., all buffer cells are full). If so, line 18 indicates that the `Producer` is waiting to perform its task. Then line 19 invokes method `wait`, causing the `Producer` thread to release the `CircularBuffer`'s lock and wait until there's space for a new value to be written into the buffer. When execution continues at line 22 after the `while` loop, the value written by the `Producer` is placed in the circular buffer at location `writeIndex`. Then line 25 updates `writeIndex` for the next call to `CircularBuffer` method `set`. This line is the key to the buffer's *circularity*. When `writeIndex` is incremented past the end of the buffer, the line sets it to `0`. Line 27 increments `occupiedCells`, because there's now one more value in the buffer that the `Consumer` can read. Next, line 28 invokes method `displayState` (lines 56–85) to update the output with the value produced, the number of occupied buffer cells, the contents of the buffer cells and the current `writeIndex` and `readIndex`. Line 29 invokes method `notifyAll` to transition *waiting* threads to the *runnable* state, so that a waiting `Consumer` thread (if there is one) can now try again to read a value from the buffer.

`CircularBuffer` Method `get`

`CircularBuffer` method `get` (lines 33–53) also performs the same tasks as it did in Fig. 26.16, with a few minor modifications. The loop at lines 37–41 determines whether the `Consumer` must wait (i.e., all buffer cells are empty). If the `Consumer` must wait, line 39 updates the output to indicate that the `Consumer` is waiting to perform its task. Then line 40 invokes method `wait`, causing the current thread to release the lock on the `CircularBuffer` and wait until data is available to read. When execution eventually continues at line 43 after a `notifyAll` call from the `Producer`, `readValue` is assigned the value at location `readIndex` in the circular buffer. Then line 46 updates `readIndex` for the next call to `CircularBuffer` method `get`. This line and line 25 implement the *circularity* of the buffer. Line 48 decrements `occupiedCells`, because there's now one more position in the buffer in which the `Producer` thread can place a value. Line 49 invokes method `displayState` to update the output with the consumed value, the number of occupied buffer cells, the contents of the buffer cells and the current `writeIndex` and `readIndex`. Line 50 invokes method `notifyAll` to allow any `Producer` threads waiting to write into the `CircularBuffer` object to attempt to write again. Then line 52 returns the consumed value to the caller.

`CircularBuffer` Method `displayState`

Method `displayState` (lines 56–85) outputs the application's state. Lines 62–63 output the values of the buffer cells. Line 63 uses method `printf` with a `"%2d"` format specifier to print the contents of each buffer with a leading space if it's a single digit. Lines 70–82 output the current `writeIndex` and `readIndex` with the letters `W` and `R`, respectively.

Testing Class `CircularBuffer`

Class `CircularBufferTest` (Fig. 26.19) contains the `main` method that launches the application. Line 11 creates the `ExecutorService`, and line 14 creates a `CircularBuffer` ob-

ject and assigns its reference to `CircularBuffer` variable `sharedLocation`. Line 17 invokes the `CircularBuffer`'s `displayState` method to show the initial state of the buffer. Lines 20–21 execute the `Producer` and `Consumer` tasks. Line 23 calls method `shutdown` to end the application when the threads complete the `Producer` and `Consumer` tasks.

Each time the `Producer` writes a value or the `Consumer` reads a value, the program outputs a message indicating the action performed (a read or a write), the contents of `buffer`, and the location of `writeIndex` and `readIndex`. In the output of Fig. 26.19, the `Producer` first writes the value 1. The buffer then contains the value 1 in the first cell and the value –1 (the default value that we use for output purposes) in the other two cells. The write index is updated to the second cell, while the read index stays at the first cell. Next, the `Consumer` reads 1. The buffer contains the same values, but the read index has been updated to the second cell. The `Consumer` then tries to read again, but the buffer is empty and the `Consumer` is forced to wait. Only once in this execution of the program was it necessary for either thread to wait.

```
1  // Fig. 26.19: CircularBufferTest.java
2  // Producer and Consumer threads manipulating a circular buffer.
3  import java.util.concurrent.ExecutorService;
4  import java.util.concurrent.Executors;
5
6  public class CircularBufferTest
7  {
8     public static void main( String[] args )
9     {
10       // create new thread pool with two threads
11       ExecutorService application = Executors.newCachedThreadPool();
12
13       // create CircularBuffer to store ints
14       CircularBuffer sharedLocation = new CircularBuffer();
15
16       // display the initial state of the CircularBuffer
17       sharedLocation.displayState( "Initial State" );
18
19       // execute the Producer and Consumer tasks
20       application.execute( new Producer( sharedLocation ) );
21       application.execute( new Consumer( sharedLocation ) );
22
23       application.shutdown();
24    } // end main
25 } // end class CircularBufferTest
```

```
Initial State (buffer cells occupied: 0)
buffer cells:   -1   -1   -1
               ---- ---- ----
                WR

Producer writes 1 (buffer cells occupied: 1)
buffer cells:    1   -1   -1
               ---- ---- ----
                 R   W
```

Fig. 26.19 | `Producer` and `Consumer` threads manipulating a circular buffer. (Part 1 of 3.)

```
Consumer reads 1 (buffer cells occupied: 0)
buffer cells:    1   -1   -1
               ---- ---- ----
                     WR

Buffer is empty. Consumer waits.
Producer writes 2 (buffer cells occupied: 1)
buffer cells:    1    2   -1
               ---- ---- ----
                      R   W

Consumer reads 2 (buffer cells occupied: 0)
buffer cells:    1    2   -1
               ---- ---- ----
                          WR

Producer writes 3 (buffer cells occupied: 1)
buffer cells:    1    2    3
               ---- ---- ----
                W          R

Consumer reads 3 (buffer cells occupied: 0)
buffer cells:    1    2    3
               ---- ---- ----
                WR

Producer writes 4 (buffer cells occupied: 1)
buffer cells:    4    2    3
               ---- ---- ----
                 R   W

Producer writes 5 (buffer cells occupied: 2)
buffer cells:    4    5    3
               ---- ---- ----
                 R        W

Consumer reads 4 (buffer cells occupied: 1)
buffer cells:    4    5    3
               ---- ---- ----
                      R   W

Producer writes 6 (buffer cells occupied: 2)
buffer cells:    4    5    6
               ---- ---- ----
                W     R

Producer writes 7 (buffer cells occupied: 3)
buffer cells:    7    5    6
               ---- ---- ----
                     WR

Consumer reads 5 (buffer cells occupied: 2)
buffer cells:    7    5    6
                ---- ---- ----
                     W     R

Producer writes 8 (buffer cells occupied: 3)
buffer cells:    7    8    6
               ---- ---- ----
                          WR
```

Fig. 26.19 | `Producer` and `Consumer` threads manipulating a circular buffer. (Part 2 of 3.)

```
Consumer reads 6 (buffer cells occupied: 2)
buffer cells:    7    8    6
               ---- ---- ----
                 R        W

Consumer reads 7 (buffer cells occupied: 1)
buffer cells:    7    8    6
               ---- ---- ----
                      R   W

Producer writes 9 (buffer cells occupied: 2)
buffer cells:    7    8    9
               ---- ---- ----
                W     R

Consumer reads 8 (buffer cells occupied: 1)
buffer cells:    7    8    9
               ---- ---- ----
                W          R

Consumer reads 9 (buffer cells occupied: 0)
buffer cells:    7    8    9
               ---- ---- ----
                WR

Producer writes 10 (buffer cells occupied: 1)
buffer cells:   10    8    9
               ---- ---- ----
                 R   W


Producer done producing
Terminating Producer
Consumer reads 10 (buffer cells occupied: 0)
buffer cells:   10    8    9
               ---- ---- ----
                     WR


Consumer read values totaling: 55
Terminating Consumer
```

Fig. 26.19 | `Producer` and `Consumer` threads manipulating a circular buffer. (Part 3 of 3.)

26.9 Producer/Consumer Relationship: The Lock and Condition Interfaces

Though the `synchronized` keyword provides for most basic thread-synchronization needs, Java provides other tools to assist in developing concurrent programs. In this section, we discuss the `Lock` and `Condition` interfaces. These interfaces give you more precise control over thread synchronization, but are more complicated to use.

Interface `Lock` *and Class* `ReentrantLock`

Any object can contain a reference to an object that implements the **`Lock`** interface (of package `java.util.concurrent.locks`). A thread calls the `Lock`'s **`lock`** method (analogous to entering a `synchronized` block) to acquire the lock. Once a `Lock` has been obtained by one thread, the `Lock` object will not allow another thread to obtain the `Lock` until the first thread releases the `Lock` (by calling the `Lock`'s **`unlock`** method—analogous to ex-

iting a synchronized block). If several threads are trying to call method lock on the same Lock object at the same time, only one of these threads can obtain the lock—all the others are placed in the *waiting* state for that lock. When a thread calls method unlock, the lock on the object is released and a waiting thread attempting to lock the object proceeds.

Class **ReentrantLock** (of package java.util.concurrent.locks) is a basic implementation of the Lock interface. The constructor for a ReentrantLock takes a boolean argument that specifies whether the lock has a **fairness policy**. If the argument is true, the ReentrantLock's fairness policy is "the longest-waiting thread will acquire the lock when it's available." Such a fairness policy guarantees that *indefinite postponement* (also called *starvation*) cannot occur. If the fairness policy argument is set to false, there's no guarantee as to which waiting thread will acquire the lock when it's available.



Software Engineering Observation 26.3

Using a ReentrantLock with a fairness policy avoids indefinite postponement.



Performance Tip 26.4

Using a ReentrantLock with a fairness policy can decrease program performance.

Condition Objects and Interface `Condition`

If a thread that owns a Lock determines that it cannot continue with its task until some condition is satisfied, the thread can wait on a **condition object**. Using Lock objects allows you to explicitly declare the condition objects on which a thread may need to wait. For example, in the producer/consumer relationship, producers can wait on *one* object and consumers can wait on *another*. This is not possible when using the synchronized keywords and an object's built-in monitor lock. Condition objects are associated with a specific Lock and are created by calling a Lock's **newCondition** method, which returns an object that implements the **Condition** interface (of package java.util.concurrent.locks). To wait on a condition object, the thread can call the Condition's **await** method (analogous to Object method wait). This immediately releases the associated Lock and places the thread in the *waiting* state for that Condition. Other threads can then try to obtain the Lock. When a *runnable* thread completes a task and determines that the *waiting* thread can now continue, the *runnable* thread can call Condition method **signal** (analogous to Object method notify) to allow a thread in that Condition's *waiting* state to return to the *runnable* state. At this point, the thread that transitioned from the *waiting* state to the *runnable* state can attempt to reacquire the Lock. Even if it's able to reacquire the Lock, the thread still might not be able to perform its task at this time—in which case the thread can call the Condition's await method to release the Lock and reenter the *waiting* state. If multiple threads are in a Condition's *waiting* state when signal is called, the default implementation of Condition signals the longest-waiting thread to transition to the *runnable* state. If a thread calls Condition method **signalAll** (analogous to Object method notifyAll), then all the threads waiting for that condition transition to the *runnable* state and become eligible to reacquire the Lock. Only one of those threads can obtain the Lock on the object—the others will wait until the Lock becomes available again. If the Lock has a *fairness policy*, the longest-waiting thread acquires the Lock. When a thread is finished with a shared object, it must call method unlock to release the Lock.



Common Programming Error 26.2

Deadlock occurs when a waiting thread (let's call this thread1) cannot proceed because it's waiting (either directly or indirectly) for another thread (let's call this thread2) to proceed, while simultaneously thread2 cannot proceed because it's waiting (either directly or indirectly) for thread1 to proceed. The two threads are waiting for each other, so the actions that would enable each thread to continue execution can never occur.



Error-Prevention Tip 26.4

When multiple threads manipulate a shared object using locks, ensure that if one thread calls method `await` *to enter the* waiting *state for a condition object, a separate thread eventually will call* `Condition` *method* `signal` *to transition the thread waiting on the condition object back to the* runnable *state. If multiple threads may be waiting on the condition object, a separate thread can call* `Condition` *method* `signalAll` *as a safeguard to ensure that all the waiting threads have another opportunity to perform their tasks. If this is not done, starvation might occur.*



Common Programming Error 26.3

An `IllegalMonitorStateException` *occurs if a thread issues an* `await`*, a* `signal`*, or a* `signalAll` *on a* `Condition` *object that was created from a* `ReentrantLock` *without having acquired the lock for that* `Condition` *object.*

`Lock` and `Condition` vs. the `synchronized` Keyword

In some applications, using `Lock` and `Condition` objects may be preferable to using the `synchronized` keyword. `Lock`s allow you to *interrupt* waiting threads or to specify a *timeout* for waiting to acquire a lock, which is not possible using the `synchronized` keyword. Also, a `Lock` is *not* constrained to be acquired and released in the *same* block of code, which is the case with the `synchronized` keyword. `Condition` objects allow you to specify multiple conditions on which threads may wait. Thus, it's possible to indicate to waiting threads that a specific condition object is now true by calling `signal` or `signallAll` on that `Condition` object. With `synchronized`, there's no way to explicitly state the condition on which threads are waiting, and thus there's no way to notify threads waiting on one condition that they may proceed without also signaling threads waiting on any other conditions. There are other possible advantages to using `Lock` and `Condition` objects, but generally it's best to use the `synchronized` keyword unless your application requires advanced synchronization capabilities.



Error-Prevention Tip 26.5

Using interfaces `Lock` *and* `Condition` *is error prone—*`unlock` *is not guaranteed to be called, whereas the monitor in a* `synchronized` *statement will always be released when the statement completes execution.*

Using `Lock`s and `Condition`s to Implement Synchronization

To illustrate how to use the `Lock` and `Condition` interfaces, we now implement the producer/consumer relationship using `Lock` and `Condition` objects to coordinate access to a shared single-element buffer (Fig. 26.20 and Fig. 26.21). In this case, each produced value is correctly consumed exactly once. Again, we reuse interface `Buffer` and classes `Producer` and `Consumer` from the example in Section 26.5, except that line 28 is removed from class `Producer` and class `Consumer`.

Class SynchronizedBuffer (Fig. 26.20) contains five fields. Line 11 creates a new object of type ReentrantLock and assigns its reference to Lock variable accessLock. The ReentrantLock is created without the *fairness policy* because at any time only a single Producer or Consumer will be waiting to acquire the Lock in this example. Lines 14–15 create two Conditions using Lock method newCondition. Condition canWrite contains a queue for a Producer thread waiting while the buffer is full (i.e., there's data in the buffer that the Consumer has not read yet). If the buffer is full, the Producer calls method await on this Condition. When the Consumer reads data from a full buffer, it calls method signal on this Condition. Condition canRead contains a queue for a Consumer thread waiting while the buffer is empty (i.e., there's no data in the buffer for the Consumer to read). If the buffer is empty, the Consumer calls method await on this Condition. When the Producer writes to the empty buffer, it calls method signal on this Condition. The int variable buffer (line 17) holds the shared data. The boolean variable occupied (line 18) keeps track of whether the buffer currently holds data (that the Consumer should read).

```
1  // Fig. 26.20: SynchronizedBuffer.java
2  // Synchronizing access to a shared integer using the Lock and Condition
3  // interfaces
4  import java.util.concurrent.locks.Lock;
5  import java.util.concurrent.locks.ReentrantLock;
6  import java.util.concurrent.locks.Condition;
7
8  public class SynchronizedBuffer implements Buffer
9  {
10    // Lock to control synchronization with this buffer
11    private final Lock accessLock = new ReentrantLock();
12
13    // conditions to control reading and writing
14    private final Condition canWrite = accessLock.newCondition();
15    private final Condition canRead = accessLock.newCondition();
16
17    private int buffer = -1; // shared by producer and consumer threads
18    private boolean occupied = false; // whether buffer is occupied
19
20    // place int value into buffer
21    public void set( int value ) throws InterruptedException
22    {
23       accessLock.lock(); // lock this object
24
25       // output thread information and buffer information, then wait
26       try
27       {
28          // while buffer is not empty, place thread in waiting state
29          while ( occupied )
30          {
31             System.out.println( "Producer tries to write." );
32             displayState( "Buffer full. Producer waits." );
33             canWrite.await(); // wait until buffer is empty
34          } // end while
```

Fig. 26.20 | Synchronizing access to a shared integer using the Lock and Condition interfaces. (Part 1 of 3.)

```
35
36             buffer = value; // set new buffer value
37
38             // indicate producer cannot store another value
39             // until consumer retrieves current buffer value
40             occupied = true;
41
42             displayState( "Producer writes " + buffer );
43
44             // signal any threads waiting to read from buffer
45             canRead.signalAll();
46          } // end try
47          finally
48          {
49             accessLock.unlock(); // unlock this object
50          } // end finally
51       } // end method set
52
53       // return value from buffer
54       public int get() throws InterruptedException
55       {
56          int readValue = 0; // initialize value read from buffer
57          accessLock.lock(); // lock this object
58
59          // output thread information and buffer information, then wait
60          try
61          {
62             // if there is no data to read, place thread in waiting state
63             while ( !occupied )
64             {
65                System.out.println( "Consumer tries to read." );
66                displayState( "Buffer empty. Consumer waits." );
67                canRead.await(); // wait until buffer is full
68             } // end while
69
70             // indicate that producer can store another value
71             // because consumer just retrieved buffer value
72             occupied = false;
73
74             readValue = buffer; // retrieve value from buffer
75             displayState( "Consumer reads " + readValue );
76
77             // signal any threads waiting for buffer to be empty
78             canWrite.signalAll();
79          } // end try
80          finally
81          {
82             accessLock.unlock(); // unlock this object
83          } // end finally
84
85          return readValue;
86       } // end method get
```

Fig. 26.20 | Synchronizing access to a shared integer using the `Lock` and `Condition` interfaces. (Part 2 of 3.)

```
87
88     // display current operation and buffer state
89     public void displayState( String operation )
90     {
91        System.out.printf( "%-40s%d\t\t%b\n\n", operation, buffer,
92           occupied );
93     } // end method displayState
94  } // end class SynchronizedBuffer
```

Fig. 26.20 | Synchronizing access to a shared integer using the `Lock` and `Condition` interfaces. (Part 3 of 3.)

Line 23 in method `set` calls method `lock` on the `SynchronizedBuffer`'s `accessLock`. If the lock is available (i.e., no other thread has acquired it), this thread now owns the lock and the thread continues. If the lock is unavailable (i.e., it's held by another thread), method `lock` waits until the lock is released. After the lock is acquired, lines 26–46 execute. Line 29 tests `occupied` to determine whether `buffer` is full. If it is, lines 31–32 display a message indicating that the thread will wait. Line 33 calls `Condition` method `await` on the `canWrite` condition object, which temporarily releases the `SynchronizedBuffer`'s `Lock` and waits for a signal from the `Consumer` that `buffer` is available for writing. When `buffer` is available, the method proceeds, writing to `buffer` (line 36), setting `occupied` to `true` (line 40) and displaying a message indicating that the producer wrote a value (line 42). Line 45 calls `Condition` method `signal` on condition object `canRead` to notify the waiting `Consumer` (if there is one) that the buffer has new data to be read. Line 49 calls method `unlock` from a `finally` block to release the lock and allow the `Consumer` to proceed.



Error-Prevention Tip 26.6

Place calls to `Lock` method `unlock` in a `finally` block. If an exception is thrown, `unlock` must still be called or deadlock could occur.

Line 57 of method `get` (lines 54–86) calls method `lock` to acquire the `Lock`. This method waits until the `Lock` is available. Once the `Lock` is acquired, line 63 tests whether `occupied` is `false`, indicating that the buffer is empty. If so, line 67 calls method `await` on condition object `canRead`. Recall that method `signal` is called on variable `canRead` in the `set` method (line 45). When the `Condition` object is signaled, the `get` method continues. Line 72–74 set `occupied` to `false`, store the value of `buffer` in `readValue` and output the `readValue`. Then line 78 signals the condition object `canWrite`. This awakens the `Producer` if it's indeed waiting for the buffer to be emptied. Line 82 calls method `unlock` from a `finally` block to release the lock, and line 85 returns `readValue` to the caller.



Common Programming Error 26.4

Forgetting to `signal` a waiting thread is a logic error. The thread will remain in the waiting *state, which will prevent it from proceeding. Such waiting can lead to indefinite postponement or deadlock.*

Class `SharedBufferTest2` (Fig. 26.21) is identical to that of Fig. 26.17. Study the outputs in Fig. 26.21. *Observe that every integer produced is consumed exactly once—no values are lost, and no values are consumed more than once.* The `Lock` and `Condition` objects ensure that the `Producer` and `Consumer` cannot perform their tasks unless it's their turn.

The `Producer` must go first, the `Consumer` must wait if the `Producer` has not produced since the `Consumer` last consumed and the `Producer` must wait if the `Consumer` has not yet consumed the value that the `Producer` most recently produced. Execute this program several times to confirm that every integer produced is consumed exactly once. In the sample output, note the highlighted lines indicating when the `Producer` and `Consumer` must wait to perform their respective tasks.

```
1  // Fig. 26.21: SharedBufferTest2.java
2  // Two threads manipulating a synchronized buffer.
3  import java.util.concurrent.ExecutorService;
4  import java.util.concurrent.Executors;
5
6  public class SharedBufferTest2
7  {
8     public static void main( String[] args )
9     {
10       // create new thread pool with two threads
11       ExecutorService application = Executors.newCachedThreadPool();
12
13       // create SynchronizedBuffer to store ints
14       Buffer sharedLocation = new SynchronizedBuffer();
15
16       System.out.printf( "%-40s%s\t\t%s\n%-40s%s\n\n", "Operation",
17          "Buffer", "Occupied", "---------", "------\t\t--------" );
18
19       // execute the Producer and Consumer tasks
20       application.execute( new Producer( sharedLocation ) );
21       application.execute( new Consumer( sharedLocation ) );
22
23       application.shutdown();
24    } // end main
25 } // end class SharedBufferTest2
```

```
Operation                               Buffer          Occupied
---------                               ------          --------

Producer writes 1                       1               true

Producer tries to write.
Buffer full. Producer waits.            1               true

Consumer reads 1                        1               false

Producer writes 2                       2               true

Producer tries to write.
Buffer full. Producer waits.            2               true

Consumer reads 2                        2               false

Producer writes 3                       3               true

Consumer reads 3                        3               false
```

Fig. 26.21 | Two threads manipulating a synchronized buffer. (Part 1 of 2.)

```
Producer writes 4                       4               true

Consumer reads 4                        4               false

Consumer tries to read.
Buffer empty. Consumer waits.           4               false

Producer writes 5                       5               true

Consumer reads 5                        5               false

Consumer tries to read.
Buffer empty. Consumer waits.           5               false

Producer writes 6                       6               true

Consumer reads 6                        6               false

Producer writes 7                       7               true

Consumer reads 7                        7               false

Producer writes 8                       8               true

Consumer reads 8                        8               false

Producer writes 9                       9               true

Consumer reads 9                        9               false

Producer writes 10                      10              true


Producer done producing
Terminating Producer
Consumer reads 10                       10              false


Consumer read values totaling 55
Terminating Consumer
```

Fig. 26.21 | Two threads manipulating a synchronized buffer. (Part 2 of 2.)

26.10 Concurrent Collections Overview

In Chapter 20, we introduced various collections from the Java Collections API. We also mentioned that you can obtain synchronized versions of those collections to allow only one thread at a time to access a collection that might be shared among several threads. The collections from the `java.util.concurrent` package are specifically designed and optimized for use in programs that share collections among multiple threads.

Figure 26.22 lists the many concurrent collections in package `java.util.concurrent`. For more information on these collections, visit

```
download.oracle.com/javase/6/docs/api/java/util/concurrent/
   package-summary.html
```

For information on the additional concurrent collections that are new in Java SE 7, visit

```
download.java.net/jdk7/docs/api/java/util/concurrent/
   package-summary.html
```

Collection	Description
`ArrayBlockingQueue`	A fixed-size queue that supports the producer/consumer relationship—possibly with many producers and consumers.
`ConcurrentHashMap`	A hash-based map that allows an arbitrary number of reader threads and a limited number of writer threads.
`ConcurrentLinkedQueue`	A concurrent linked-list implementation of a queue that can grow dynamically.
`ConcurrentSkipListMap`	A concurrent map that is sorted by its keys.
`ConcurrentSkipListSet`	A sorted concurrent set.
`CopyOnWriteArrayList`	A thread-safe `ArrayList`. Each operation that modifies the collection first creates a new copy of the contents. Used when the collection is traversed much more frequently than the collection's contents are modified.
`CopyOnWriteArraySet`	A set that's implemented using `CopyOnWriteArrayList`.
`DelayQueue`	A variable-size queue containing `Delayed` objects. An object can be removed only after its delay has expired.
`LinkedBlockingDeque`	A double-ended blocking queue implemented as a linked list that can optionally be fixed in size.
`LinkedBlockingQueue`	A blocking queue implemented as a linked list that can optionally be fixed in size.
`PriorityBlockingQueue`	A variable-length priority-based blocking queue (like a `PriorityQueue`).
`SynchronousQueue`	A blocking queue implementation that does not have an internal capacity. Each insert operation by one thread must wait for a remove operation from another thread and vice versa.
Concurrent Collections Added in Java SE 7	
`ConcurrentLinkedDeque`	A concurrent linked-list implementation of a double-ended queue.
`LinkedTransferQueue`	A linked-list implementation of interface `TransferQueue`. Each producer has the option of waiting for a consumer to take an element being inserted (via method `transfer`) or simply placing the element into the queue (via method put). Also provides overloaded method `tryTransfer` to immediately transfer an element to a waiting consumer or to do so within a specified timeout period. If the transfer cannot be completed, the element is not placed in the queue. Typically used in applications that pass messages between threads.

Fig. 26.22 | Concurrent collections summary (package `java.util.concurrent`).

26.11 Multithreading with GUI

Swing applications present a unique set of challenges for multithreaded programming. All Swing applications have a single thread, called the **event dispatch thread**, to handle interactions with the application's GUI components. Typical interactions include *updating GUI components* or *processing user actions* such as mouse clicks. All tasks that require interaction with an application's GUI are placed in an *event queue* and are executed sequentially by the event dispatch thread.

Swing GUI components are not thread safe—they cannot be manipulated by multiple threads without the risk of incorrect results. Unlike the other examples presented in this chapter, thread safety in GUI applications is achieved not by synchronizing thread actions, but by *ensuring that Swing components are accessed from only a single thread*—the event dispatch thread. This technique is called **thread confinement**. Allowing just one thread to access non-thread-safe objects eliminates the possibility of corruption due to multiple threads accessing these objects concurrently.

Usually it's sufficient to perform simple calculations on the event dispatch thread in sequence with GUI component manipulations. If an application must perform a lengthy computation in response to a user interface interaction, the event dispatch thread cannot attend to other tasks in the event queue while the thread is tied up in that computation. This causes the GUI components to become unresponsive. It's preferable to handle a long-running computation in a separate thread, freeing the event dispatch thread to continue managing other GUI interactions. Of course, to update the GUI based on the computation's results, you must update the GUI from the event dispatch thread, rather than from the worker thread that performed the computation.

Class `SwingWorker`

Class `SwingWorker` (in package `javax.swing`) perform long-running computations in a worker thread and to update Swing components from the event dispatch thread based on the computations' results. `SwingWorker` implements the `Runnable` interface, meaning that *a `SwingWorker` object can be scheduled to execute in a separate thread*. The `SwingWorker` class provides several methods to simplify performing computations in a worker thread and making the results available for display in a GUI. Some common `SwingWorker` methods are described in Fig. 26.23.

Method	Description
`doInBackground`	Defines a long computation and is called in a worker thread.
`done`	Executes on the event dispatch thread when `doInBackground` returns.
`execute`	Schedules the `SwingWorker` object to be executed in a worker thread.
`get`	Waits for the computation to complete, then returns the result of the computation (i.e., the return value of `doInBackground`).
`publish`	Sends intermediate results from the `doInBackground` method to the `process` method for processing on the event dispatch thread.

Fig. 26.23 | Commonly used `SwingWorker` methods. (Part 1 of 2.)

Method	Description
`process`	Receives intermediate results from the `publish` method and processes these results on the event dispatch thread.
`setProgress`	Sets the progress property to notify any property change listeners on the event dispatch thread of progress bar updates.

Fig. 26.23 | Commonly used `SwingWorker` methods. (Part 2 of 2.)

26.11.1 Performing Computations in a Worker Thread

In the next example, the user enters a number *n* and the program gets the *n*th Fibonacci number, which we calculate using the recursive algorithm discussed in Section 18.4. Since the algorithm is time consuming for large values, we use a `SwingWorker` object to perform the calculation in a worker thread. The GUI also provides a separate set of components that get the next Fibonacci number in the sequence with each click of a button, beginning with `fibonacci(1)`. This set of components performs its short computation directly in the event dispatch thread. This program is capable of producing up to the 92nd Fibonacci number—subsequent values are outside the range that can be represented by a `long`. Recall that you can use class `BigInteger` to represent arbitrarily large integer values.

Class `BackgroundCalculator` (Fig. 26.24) performs the recursive Fibonacci calculation in a *worker thread*. This class extends `SwingWorker` (line 8), overriding the methods `doInBackground` and `done`. Method `doInBackground` (lines 21–24) computes the *n*th Fibonacci number in a worker thread and returns the result. Method `done` (lines 27–43) displays the result in a `JLabel`.

```
1  // Fig. 26.24: BackgroundCalculator.java
2  // SwingWorker subclass for calculating Fibonacci numbers
3  // in a background thread.
4  import javax.swing.SwingWorker;
5  import javax.swing.JLabel;
6  import java.util.concurrent.ExecutionException;
7
8  public class BackgroundCalculator extends SwingWorker< Long, Object >
9  {
10    private final int n; // Fibonacci number to calculate
11    private final JLabel resultJLabel; // JLabel to display the result
12
13    // constructor
14    public BackgroundCalculator( int number, JLabel label )
15    {
16       n = number;
17       resultJLabel = label;
18    } // end BackgroundCalculator constructor
```

Fig. 26.24 | `SwingWorker` subclass for calculating Fibonacci numbers in a background thread. (Part 1 of 2.)

```
19
20     // long-running code to be run in a worker thread
21     public Long doInBackground()
22     {
23        return nthFib = fibonacci( n );
24     } // end method doInBackground
25
26     // code to run on the event dispatch thread when doInBackground returns
27     protected void done()
28     {
29        try
30        {
31           // get the result of doInBackground and display it
32           resultJLabel.setText( get().toString() );
33        } // end try
34        catch ( InterruptedException ex )
35        {
36           resultJLabel.setText( "Interrupted while waiting for results." );
37        } // end catch
38        catch ( ExecutionException ex )
39        {
40           resultJLabel.setText(
41              "Error encountered while performing calculation." );
42        } // end catch
43     } // end method done
44
45     // recursive method fibonacci; calculates nth Fibonacci number
46     public long fibonacci( long number )
47     {
48        if ( number == 0 || number == 1 )
49           return number;
50        else
51           return fibonacci( number - 1 ) + fibonacci( number - 2 );
52     } // end method fibonacci
53  } // end class BackgroundCalculator
```

Fig. 26.24 | `SwingWorker` subclass for calculating Fibonacci numbers in a background thread. (Part 2 of 2.)

`SwingWorker` is a *generic class*. In line 8, the first type parameter is `Long` and the second is `Object`. The first type parameter indicates the type returned by the `doInBackground` method; the second indicates the type that's passed between the `publish` and `process` methods to handle intermediate results. Since we do not use `publish` and `process` in this example, we simply use `Object` as the second type parameter. We discuss `publish` and `process` in Section 26.11.2.

A `BackgroundCalculator` object can be instantiated from a class that controls a GUI. A `BackgroundCalculator` maintains instance variables for an integer that represents the Fibonacci number to be calculated and a `JLabel` that displays the results of the calculation (lines 10–11). The `BackgroundCalculator` constructor (lines 14–18) initializes these instance variables with the arguments that are passed to the constructor.



Software Engineering Observation 26.4

Any GUI components that will be manipulated by `SwingWorker` methods, such as components that will be updated from methods `process` or `done`, should be passed to the `SwingWorker` subclass's constructor and stored in the subclass object. This gives these methods access to the GUI components they'll manipulate.

When method `execute` is called on a `BackgroundCalculator` object, the object is scheduled for execution in a worker thread. Method `doInBackground` is called from the worker thread and invokes the `fibonacci` method (lines 46–52), passing instance variable `n` as an argument (line 23). Method `fibonacci` uses recursion to compute the Fibonacci of `n`. When `fibonacci` returns, method `doInBackground` returns the result.

After `doInBackground` returns, method `done` is called from the event dispatch thread. This method attempts to set the result `JLabel` to the return value of `doInBackground` by calling method `get` to retrieve this return value (line 32). Method `get` waits for the result to be ready if necessary, but since we call it from method `done`, the computation will be complete before `get` is called. Lines 34–37 catch `InterruptedException` if the current thread is interrupted while waiting for `get` to return. This exception will not occur in this example since the calculation will have already completed by the time `get` is called. Lines 38–42 catch `ExecutionException`, which is thrown if an exception occurs during the computation.

Class `FibonacciNumbers`

Class `FibonacciNumbers` (Fig. 26.25) displays a window containing two sets of GUI components—one set to compute a Fibonacci number in a worker thread and another to get the next Fibonacci number in response to the user's clicking a `JButton`. The constructor (lines 38–109) places these components in separate titled `JPanels`. Lines 46–47 and 78–79 add two `JLabels`, a `JTextField` and a `JButton` to the `workerJPanel` to allow the user to enter an integer whose Fibonacci number will be calculated by the `BackgroundWorker`. Lines 84–85 and 103 add two `JLabels` and a `JButton` to the event dispatch thread panel to allow the user to get the next Fibonacci number in the sequence. Instance variables `n1` and `n2` contain the previous two Fibonacci numbers in the sequence and are initialized to `0` and `1`, respectively (lines 29–30). Instance variable `count` stores the most recently computed sequence number and is initialized to `1` (line 31). The two `JLabels` display `count` and `n2` initially, so that the user will see the text `Fibonacci of 1: 1` in the `eventThreadJPanel` when the GUI starts.

```
1  // Fig. 26.25: FibonacciNumbers.java
2  // Using SwingWorker to perform a long calculation with
3  // results displayed in a GUI.
4  import java.awt.GridLayout;
5  import java.awt.event.ActionEvent;
6  import java.awt.event.ActionListener;
7  import javax.swing.JButton;
8  import javax.swing.JFrame;
9  import javax.swing.JPanel;
```

Fig. 26.25 | Using `SwingWorker` to perform a long calculation with results displayed in a GUI. (Part 1 of 4.)

```
10  import javax.swing.JLabel;
11  import javax.swing.JTextField;
12  import javax.swing.border.TitledBorder;
13  import javax.swing.border.LineBorder;
14  import java.awt.Color;
15  import java.util.concurrent.ExecutionException;
16
17  public class FibonacciNumbers extends JFrame
18  {
19     // components for calculating the Fibonacci of a user-entered number
20     private final JPanel workerJPanel =
21        new JPanel( new GridLayout( 2, 2, 5, 5 ) );
22     private final JTextField numberJTextField = new JTextField();
23     private final JButton goJButton = new JButton( "Go" );
24     private final JLabel fibonacciJLabel = new JLabel();
25
26     // components and variables for getting the next Fibonacci number
27     private final JPanel eventThreadJPanel =
28        new JPanel( new GridLayout( 2, 2, 5, 5 ) );
29     private long n1 = 0; // initialize with first Fibonacci number
30     private long n2 = 1; // initialize with second Fibonacci number
31     private int count = 1; // current Fibonacci number to display
32     private final JLabel nJLabel = new JLabel( "Fibonacci of 1: " );
33     private final JLabel nFibonacciJLabel =
34        new JLabel( String.valueOf( n2 ) );
35     private final JButton nextNumberJButton = new JButton( "Next Number" );
36
37     // constructor
38     public FibonacciNumbers()
39     {
40        super( "Fibonacci Numbers" );
41        setLayout( new GridLayout( 2, 1, 10, 10 ) );
42
43        // add GUI components to the SwingWorker panel
44        workerJPanel.setBorder( new TitledBorder(
45           new LineBorder( Color.BLACK ), "With SwingWorker" ) );
46        workerJPanel.add( new JLabel( "Get Fibonacci of:" ) );
47        workerJPanel.add( numberJTextField );
48        goJButton.addActionListener(
49           new ActionListener()
50           {
51              public void actionPerformed( ActionEvent event )
52              {
53                 int n;
54
55                 try
56                 {
57                    // retrieve user's input as an integer
58                    n = Integer.parseInt( numberJTextField.getText() );
59                 } // end try
60                 catch( NumberFormatException ex )
61                 {
```

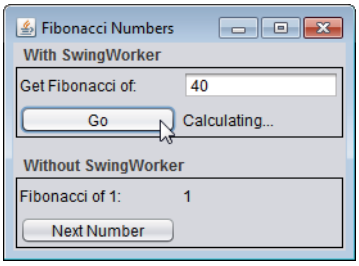
Fig. 26.25 | Using `SwingWorker` to perform a long calculation with results displayed in a GUI. (Part 2 of 4.)

```
62                   // display an error message if the user did not
63                   // enter an integer
64                   fibonacciJLabel.setText( "Enter an integer." );
65                   return;
66                } // end catch
67
68                // indicate that the calculation has begun
69                fibonacciJLabel.setText( "Calculating..." );
70
71                // create a task to perform calculation in background
72                BackgroundCalculator task =
73                   new BackgroundCalculator( n, fibonacciJLabel );
74                task.execute(); // execute the task
75             } // end method actionPerformed
76          } // end anonymous inner class
77       ); // end call to addActionListener
78       workerJPanel.add( goJButton );
79       workerJPanel.add( fibonacciJLabel );
80
81       // add GUI components to the event-dispatching thread panel
82       eventThreadJPanel.setBorder( new TitledBorder(
83          new LineBorder( Color.BLACK ), "Without SwingWorker" ) );
84       eventThreadJPanel.add( nJLabel );
85       eventThreadJPanel.add( nFibonacciJLabel );
86       nextNumberJButton.addActionListener(
87          new ActionListener()
88          {
89             public void actionPerformed( ActionEvent event )
90             {
91                // calculate the Fibonacci number after n2
92                long temp = n1 + n2;
93                n1 = n2;
94                n2 = temp;
95                ++count;
96
97                // display the next Fibonacci number
98                nJLabel.setText( "Fibonacci of " + count + ": " );
99                nFibonacciJLabel.setText( String.valueOf( n2 ) );
100            } // end method actionPerformed
101         } // end anonymous inner class
102      ); // end call to addActionListener
103      eventThreadJPanel.add( nextNumberJButton );
104
105      add( workerJPanel );
106      add( eventThreadJPanel );
107      setSize( 275, 200 );
108      setVisible( true );
109   } // end constructor
110
111   // main method begins program execution
112   public static void main( String[] args )
113   {
```

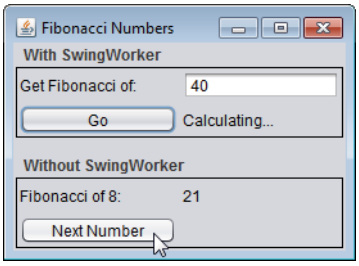
Fig. 26.25 | Using `SwingWorker` to perform a long calculation with results displayed in a GUI. (Part 3 of 4.)

```
114         FibonacciNumbers application = new FibonacciNumbers();
115         application.setDefaultCloseOperation( EXIT_ON_CLOSE );
116      } // end main
117   } // end class FibonacciNumbers
```

a) Begin calculating Fibonacci of 40 in the background



b) Calculating other Fibonacci values while Fibonacci of 40 continues calculating



c) Fibonacci of 40 calculation finishes

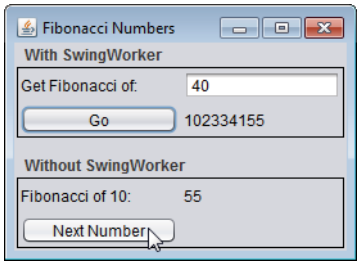


Fig. 26.25 | Using `SwingWorker` to perform a long calculation with results displayed in a GUI. (Part 4 of 4.)

Lines 48–77 register the event handler for the `goJButton`. If the user clicks this `JButton`, line 58 gets the value entered in the `numberJTextField` and attempts to parse it as an integer. Lines 72–73 create a new `BackgroundCalculator` object, passing in the user-entered value and the `fibonacciJLabel` that's used to display the calculation's results. Line 74 calls method `execute` on the `BackgroundCalculator`, scheduling it for execution in a separate worker thread. Method `execute` does not wait for the `BackgroundCalculator` to finish executing. It returns immediately, allowing the GUI to continue processing other events while the computation is performed.

If the user clicks the `nextNumberJButton` in the `eventThreadJPanel`, the event handler registered in lines 86–102 executes.Lines 92–95 add the previous two Fibonacci numbers stored in `n1` and `n2` to determine the next number in the sequence, update `n1` and `n2` to their new values and increment `count`. Then lines 98–99 update the GUI to display the next number. The code for these calculations is in method `actionPerformed`, so they're performed on the *event dispatch thread*. Handling such short computations in the event dispatch thread does not cause the GUI to become unresponsive, as with the recursive algorithm for calculating the Fibonacci of a large number. Because the longer Fibonacci computation is performed in a separate worker thread using the `SwingWorker`, it's possible to get the next Fibonacci number while the recursive computation is still in progress.

26.11.2 Processing Intermediate Results with SwingWorker

We've presented an example that uses the `SwingWorker` class to execute a long process in a *background thread* and update the GUI when the process is finished. We now present an example of updating the GUI with intermediate results before the long process completes. Figure 26.26 presents class `PrimeCalculator`, which extends `SwingWorker` to compute the first *n* prime numbers in a *worker thread*. In addition to the `doInBackground` and `done` methods used in the previous example, this class uses `SwingWorker` methods `publish`, `process` and `setProgress`. In this example, method `publish` sends prime numbers to method `process` as they're found, method `process` displays these primes in a GUI component and method `setProgress` updates the progress property. We later show how to use this property to update a `JProgressBar`.

```
1  // Fig. 26.26: PrimeCalculator.java
2  // Calculates the first n primes, displaying them as they are found.
3  import javax.swing.JTextArea;
4  import javax.swing.JLabel;
5  import javax.swing.JButton;
6  import javax.swing.SwingWorker;
7  import java.util.Arrays;
8  import java.util.Random;
9  import java.util.List;
10 import java.util.concurrent.CancellationException;
11 import java.util.concurrent.ExecutionException;
12
13 public class PrimeCalculator extends SwingWorker< Integer, Integer >
14 {
15    private final Random generator = new Random();
16    private final JTextArea intermediateJTextArea; // displays found primes
17    private final JButton getPrimesJButton;
18    private final JButton cancelJButton;
19    private final JLabel statusJLabel; // displays status of calculation
20    private final boolean[] primes; // boolean array for finding primes
21
22    // constructor
23    public PrimeCalculator( int max, JTextArea intermediate, JLabel status,
24       JButton getPrimes, JButton cancel )
25    {
26       intermediateJTextArea = intermediate;
27       statusJLabel = status;
28       getPrimesJButton = getPrimes;
29       cancelJButton = cancel;
30       primes = new boolean[ max ];
31
32       // initialize all prime array values to true
33       Arrays.fill( primes, true );
34    } // end constructor
35
36    // finds all primes up to max using the Sieve of Eratosthenes
37    public Integer doInBackground()
38    {
```

Fig. 26.26 | Calculates the first *n* primes, displaying them as they are found. (Part 1 of 3.)

```
39          int count = 0; // the number of primes found
40
41          // starting at the third value, cycle through the array and put
42          // false as the value of any greater number that is a multiple
43          for ( int i = 2; i < primes.length; i++ )
44          {
45             if ( isCancelled() ) // if calculation has been canceled
46                return count;
47             else
48             {
49                setProgress( 100 * ( i + 1 ) / primes.length );
50
51                try
52                {
53                   Thread.sleep( generator.nextInt( 5 ) );
54                } // end try
55                catch ( InterruptedException ex )
56                {
57                   statusJLabel.setText( "Worker thread interrupted" );
58                   return count;
59                } // end catch
60
61                if ( primes[ i ] ) // i is prime
62                {
63                   publish( i ); // make i available for display in prime list
64                   ++count;
65
66                   for ( int j = i + i; j < primes.length; j += i )
67                      primes[ j ] = false; // i is not prime
68                } // end if
69             } // end else
70          } // end for
71
72          return count;
73       } // end method doInBackground
74
75       // displays published values in primes list
76       protected void process( List< Integer > publishedVals )
77       {
78          for ( int i = 0; i < publishedVals.size(); i++ )
79             intermediateJTextArea.append( publishedVals.get( i ) + "\n" );
80       } // end method process
81
82       // code to execute when doInBackground completes
83       protected void done()
84       {
85          getPrimesJButton.setEnabled( true ); // enable Get Primes button
86          cancelJButton.setEnabled( false ); // disable Cancel button
87
88          int numPrimes;
89
90          try
91          {
```

Fig. 26.26 | Calculates the first *n* primes, displaying them as they are found. (Part 2 of 3.)

```
92            numPrimes = get(); // retrieve doInBackground return value
93         } // end try
94         catch ( InterruptedException ex )
95         {
96            statusJLabel.setText( "Interrupted while waiting for results." );
97            return;
98         } // end catch
99         catch ( ExecutionException ex )
100        {
101           statusJLabel.setText( "Error performing computation." );
102           return;
103        } // end catch
104        catch ( CancellationException ex )
105        {
106           statusJLabel.setText( "Cancelled." );
107           return;
108        } // end catch
109
110        statusJLabel.setText( "Found " + numPrimes + " primes." );
111     } // end method done
112  } // end class PrimeCalculator
```

Fig. 26.26 | Calculates the first *n* primes, displaying them as they are found. (Part 3 of 3.)

Class `PrimeCalculator` extends `SwingWorker` (line 13), with the first type parameter indicating the return type of method `doInBackground` and the second indicating the type of intermediate results passed between methods `publish` and `process`. In this case, both type parameters are `Integer`s. The constructor (lines 23–34) takes as arguments an integer that indicates the upper limit of the prime numbers to locate, a `JTextArea` used to display primes in the GUI, one `JButton` for initiating a calculation and one for canceling it, and a `JLabel` used to display the status of the calculation.

Sieve of Eratosthenes

Line 33 initializes the elements of the `boolean` array `primes` to `true` with `Arrays` method `fill`. `PrimeCalculator` uses this array and the **Sieve of Eratosthenes** algorithm (described in Exercise 7.27) to find all primes less than `max`. The Sieve of Eratosthenes takes a list of integers and, beginning with the first prime number, filters out all multiples of that prime. It then moves to the next prime, which will be the next number that's not yet filtered out, and eliminates all of its multiples. It continues until the end of the list is reached and all nonprimes have been filtered out. Algorithmically, we begin with element `2` of the `boolean` array and set the cells corresponding to all values that are multiples of 2 to `false` to indicate that they're divisible by 2 and thus not prime. We then move to the next array element, check whether it's `true`, and if so set all of its multiples to `false` to indicate that they're divisible by the current index. When the whole array has been traversed in this way, all indices that contain `true` are prime, as they have no divisors.

Method `doInBackground`

In method `doInBackground` (lines 37–73), the control variable `i` for the loop (lines 43–70) controls the current index for implementing the Sieve of Eratosthenes. Line 45 calls the inherited `SwingWorker` method **`isCancelled`** to determine whether the user has

clicked the **Cancel** button. If `isCancelled` returns `true`, method `doInBackground` returns the number of primes found so far (line 46) without finishing the computation.

If the calculation isn't canceled, line 49 calls `setProgress` to update the percentage of the array that's been traversed so far. Line 53 puts the currently executing thread to sleep for up to 4 milliseconds. We discuss the reason for this shortly. Line 61 tests whether the element of array `primes` at the current index is `true` (and thus prime). If so, line 63 passes the index to method `publish` so that it can be displayed as an intermediate result in the GUI and line 64 increments the number of primes found. Lines 66–67 set all multiples of the current index to `false` to indicate that they're not prime. When the entire array has been traversed, line 72 returns the number of primes found.

Method `process`

Lines 76–80 declare method `process`, which executes in the event dispatch thread and receives its argument `publishedVals` from method `publish`. The passing of values between `publish` in the worker thread and `process` in the event dispatch thread is asynchronous; `process` might not be invoked for every call to `publish`. All `Integers` published since the last call to `process` are received as a `List` by method `process`. Lines 78–79 iterate through this list and display the published values in a `JTextArea`. Because the computation in method `doInBackground` progresses quickly, publishing values often, updates to the `JTextArea` can pile up on the event dispatch thread, causing the GUI to become sluggish. In fact, when searching for a large number of primes, the *event dispatch thread* may receive so many requests in quick succession to update the `JTextArea` that it *runs out of memory in its event queue*. This is why we put the worker thread to sleep for a few milliseconds between calls to `publish`. The calculation is slowed just enough to allow the event dispatch thread to keep up with requests to update the `JTextArea` with new primes, enabling the GUI to update smoothly and remain responsive.

Method `done`

Lines 83–111 define method `done`. When the calculation is finished or canceled, method `done` enables the **Get Primes** button and disables the **Cancel** button (lines 85–86). Line 92 gets the return value—the number of primes found—from method `doInBackground`. Lines 94–108 catch the exceptions thrown by method `get` and display an appropriate message in the `statusJLabel`. If no exceptions occur, line 110 sets the `statusJLabel` to indicate the number of primes found.

Class `FindPrimes`

Class `FindPrimes` (Fig. 26.27) displays a `JTextField` that allows the user to enter a number, a `JButton` to begin finding all primes less than that number and a `JTextArea` to display the primes. A `JButton` allows the user to cancel the calculation, and a `JProgressBar` indicates the calculation's progress. The `FindPrimes` constructor (lines 32–125) sets up the application's GUI.

Lines 42–94 register the event handler for the `getPrimesJButton`. When the user clicks this `JButton`, lines 47–49 reset the `JProgressBar` and clear the `displayPrimesJTextArea` and the `statusJLabel`. Lines 53–63 parse the value in the `JTextField` and display an error message if the value is not an integer. Lines 66–68 construct a new `PrimeCalculator` object, passing as arguments the integer the user entered, the `displayPrimesJTextArea` for displaying the primes, the `statusJLabel` and the two `JButtons`.

```
1  // Fig 26.27: FindPrimes.java
2  // Using a SwingWorker to display prime numbers and update a JProgressBar
3  // while the prime numbers are being calculated.
4  import javax.swing.JFrame;
5  import javax.swing.JTextField;
6  import javax.swing.JTextArea;
7  import javax.swing.JButton;
8  import javax.swing.JProgressBar;
9  import javax.swing.JLabel;
10 import javax.swing.JPanel;
11 import javax.swing.JScrollPane;
12 import javax.swing.ScrollPaneConstants;
13 import java.awt.BorderLayout;
14 import java.awt.GridLayout;
15 import java.awt.event.ActionListener;
16 import java.awt.event.ActionEvent;
17 import java.util.concurrent.ExecutionException;
18 import java.beans.PropertyChangeListener;
19 import java.beans.PropertyChangeEvent;
20
21 public class FindPrimes extends JFrame
22 {
23    private final JTextField highestPrimeJTextField = new JTextField();
24    private final JButton getPrimesJButton = new JButton( "Get Primes" );
25    private final JTextArea displayPrimesJTextArea = new JTextArea();
26    private final JButton cancelJButton = new JButton( "Cancel" );
27    private final JProgressBar progressJProgressBar = new JProgressBar();
28    private final JLabel statusJLabel = new JLabel();
29    private PrimeCalculator calculator;
30
31    // constructor
32    public FindPrimes()
33    {
34       super( "Finding Primes with SwingWorker" );
35       setLayout( new BorderLayout() );
36
37       // initialize panel to get a number from the user
38       JPanel northJPanel = new JPanel();
39       northJPanel.add( new JLabel( "Find primes less than: " ) );
40       highestPrimeJTextField.setColumns( 5 );
41       northJPanel.add( highestPrimeJTextField );
42       getPrimesJButton.addActionListener(
43          new ActionListener()
44          {
45             public void actionPerformed( ActionEvent e )
46             {
47                progressJProgressBar.setValue( 0 ); // reset JProgressBar
48                displayPrimesJTextArea.setText( "" ); // clear JTextArea
49                statusJLabel.setText( "" ); // clear JLabel
50
51                int number; // search for primes up through this value
```

Fig. 26.27 | Using a `SwingWorker` to display prime numbers and update a `JProgressBar` while the prime numbers are being calculated. (Part 1 of 3.)

```
52
53                try
54                {
55                   // get user input
56                   number = Integer.parseInt(
57                      highestPrimeJTextField.getText() );
58                } // end try
59                catch ( NumberFormatException ex )
60                {
61                   statusJLabel.setText( "Enter an integer." );
62                   return;
63                } // end catch
64
65                // construct a new PrimeCalculator object
66                calculator = new PrimeCalculator( number,
67                   displayPrimesJTextArea, statusJLabel, getPrimesJButton,
68                   cancelJButton );
69
70                // listen for progress bar property changes
71                calculator.addPropertyChangeListener(
72                   new PropertyChangeListener()
73                   {
74                      public void propertyChange( PropertyChangeEvent e )
75                      {
76                         // if the changed property is progress,
77                         // update the progress bar
78                         if ( e.getPropertyName().equals( "progress" ) )
79                         {
80                            int newValue = ( Integer ) e.getNewValue();
81                            progressJProgressBar.setValue( newValue );
82                         } // end if
83                      } // end method propertyChange
84                   } // end anonymous inner class
85                ); // end call to addPropertyChangeListener
86
87                // disable Get Primes button and enable Cancel button
88                getPrimesJButton.setEnabled( false );
89                cancelJButton.setEnabled( true );
90
91                calculator.execute(); // execute the PrimeCalculator object
92             } // end method ActionPerformed
93          } // end anonymous inner class
94       ); // end call to addActionListener
95       northJPanel.add( getPrimesJButton );
96
97       // add a scrollable JList to display results of calculation
98       displayPrimesJTextArea.setEditable( false );
99       add( new JScrollPane( displayPrimesJTextArea,
100         ScrollPaneConstants.VERTICAL_SCROLLBAR_ALWAYS,
101         ScrollPaneConstants.HORIZONTAL_SCROLLBAR_NEVER ) );
102
```

Fig. 26.27 | Using a `SwingWorker` to display prime numbers and update a `JProgressBar` while the prime numbers are being calculated. (Part 2 of 3.)

```
103         // initialize a panel to display cancelJButton,
104         // progressJProgressBar, and statusJLabel
105         JPanel southJPanel = new JPanel( new GridLayout( 1, 3, 10, 10 ) );
106         cancelJButton.setEnabled( false );
107         cancelJButton.addActionListener(
108            new ActionListener()
109            {
110               public void actionPerformed( ActionEvent e )
111               {
112                  calculator.cancel( true ); // cancel the calculation
113               } // end method ActionPerformed
114            } // end anonymous inner class
115         ); // end call to addActionListener
116         southJPanel.add( cancelJButton );
117         progressJProgressBar.setStringPainted( true );
118         southJPanel.add( progressJProgressBar );
119         southJPanel.add( statusJLabel );
120
121         add( northJPanel, BorderLayout.NORTH );
122         add( southJPanel, BorderLayout.SOUTH );
123         setSize( 350, 300 );
124         setVisible( true );
125      } // end constructor
126
127      // main method begins program execution
128      public static void main( String[] args )
129      {
130         FindPrimes application = new FindPrimes();
131         application.setDefaultCloseOperation( EXIT_ON_CLOSE );
132      } // end main
133   } // end class FindPrimes
```

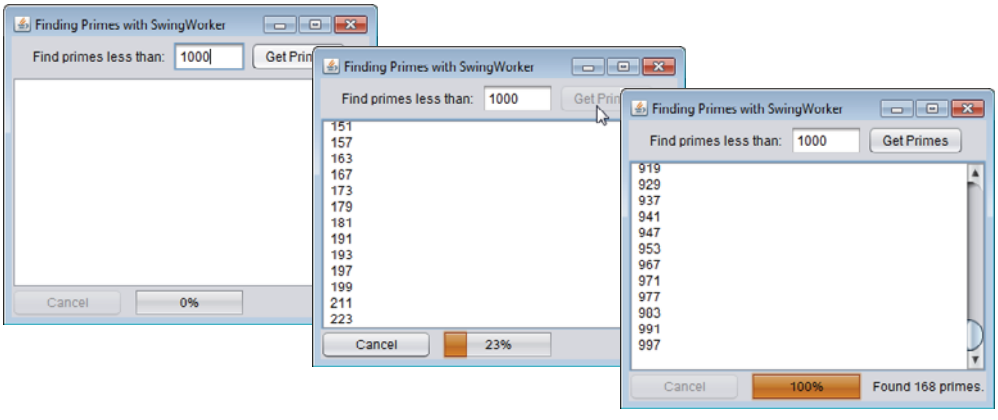


Fig. 26.27 | Using a `SwingWorker` to display prime numbers and update a `JProgressBar` while the prime numbers are being calculated. (Part 3 of 3.)

Lines 71–85 register a `PropertyChangeListener` for the `PrimeCalculator` object. **`PropertyChangeListener`** is an interface from package `java.beans` that defines a single method, `propertyChange`. Every time method `setProgress` is invoked on a `PrimeCalcu-`

lator, the PrimeCalculator generates a PropertyChangeEvent to indicate that the progress property has changed. Method propertyChange listens for these events. Line 78 tests whether a given PropertyChangeEvent indicates a change to the progress property. If so, line 80 gets the new value of the property and line 81 updates the JProgressBar with the new progress property value.

The **Get Primes** JButton is disabled (line 88) so only one calculation that updates the GUI can execute at a time, and the **Cancel** JButton is enabled (line 89) to allow the user to stop the computation before it completes. Line 91 executes the PrimesCalculator to begin finding primes. If the user clicks the cancelJButton, the event handler registered at lines 107–115 calls PrimeCalculator's method **cancel** (line 112), which is inherited from class SwingWorker, and the calculation returns early. The argument true to method cancel indicates that the thread performing the task should be interrupted in an attempt to cancel the task.

26.12 Interfaces Callable and Future

Interface Runnable provides only the most basic functionality for multithreaded programming. In fact, this interface has several limitations. Suppose a Runnable encounters a problem and tries to throw a *checked* exception. The run method is not declared to throw any exceptions, so the problem must be handled within the Runnable—the exception *cannot* be passed to the calling thread. Now suppose a Runnable is performing a long calculation and the application wants to retrieve the result of that calculation. The run method cannot return a value, so the application must use shared data to pass the value back to the calling thread. This also involves the overhead of synchronizing access to the data. The developers of the concurrency APIs recognized these limitations and created a new interface to fix them. The **Callable** interface (of package java.util.concurrent) declares a single method named **call**. This interface is designed to be similar to the Runnable interface—allowing an action to be performed concurrently in a separate thread—but the call method allows the thread to return a value or to throw a *checked* exception.

An application that creates a Callable likely wants to run it concurrently with other Runnables and Callables. The ExecutorService interface provides method **submit**, which will execute a Callable passed in as its argument. The submit method returns an object of type **Future** (of package java.util.concurrent), which is an interface that represents the executing Callable. The Future interface declares method **get** to return the result of the Callable and provides other methods to manage a Callable's execution.

26.13 Java SE 7: Fork/Join Framework

Java SE 7's concurrency APIs include the new fork/join framework, which helps programmers parallelize algorithms. The framework is beyond the scope of this book. Experts tell us that most Java programmers will benefit by this framework being used "behind the scenes" in the Java API and other third party libraries.

The fork/join framework is particularly well suited to divide-and-conquer-style algorithms, such as the merge sort that we implemented in Section 19.3.3. Recall that the recursive algorithm sorts an array by *splitting* it into two equal-sized subarrays, *sorting* each subarray, then *merging* them into one larger array. Each subarray is sorted by performing the same algorithm on the subarray. For algorithms like merge sort, the fork/join frame-

work can be used to create parallel tasks so that they can be distributed across multiple processors and be truly performed in parallel—the details of assigning the parallel tasks to different processors are handled for you by the framework.

To learn more about the fork/join framework and Java multithreading in general, please visit the sites listed in our Java Multithreading Resource Center at

```
www.deitel.com/JavaMultithreading
```

26.14 Wrap-Up

In this chapter, you learned that concurrency has historically been implemented with operating-system primitives available only to experienced systems programmers, but that Java makes concurrency available to you through the language and APIs. You also learned that the JVM itself creates threads to run a program, and that it also can create threads to perform housekeeping tasks such as garbage collection.

We discussed the life cycle of a thread and the states that a thread may occupy during its lifetime. Next, we presented the interface `Runnable`, which is used to specify a task that can execute concurrently with other tasks. This interface's `run` method is invoked by the thread executing the task. We showed how to execute a `Runnable` object by associating it with an object of class `Thread`. Then we showed how to use the `Executor` interface to manage the execution of `Runnable` objects via thread pools, which can reuse existing threads to eliminate the overhead of creating a new thread for each task and can improve performance by optimizing the number of threads to ensure that the processor stays busy.

You learned that when multiple threads share an object and one or more of them modify that object, indeterminate results may occur unless access to the shared object is managed properly. We showed you how to solve this problem via thread synchronization, which coordinates access to shared data by multiple concurrent threads. You learned several techniques for performing synchronization—first with the built-in class `ArrayBlockingQueue` (which handles *all* the synchronization details for you), then with Java's built-in monitors and the `synchronized` keyword, and finally with interfaces `Lock` and `Condition`.

We discussed the fact that Swing GUIs are not thread safe, so all interactions with and modifications to the GUI must be performed in the event dispatch thread. We also discussed the problems associated with performing long-running calculations in the event dispatch thread. Then we showed how you can use the `SwingWorker` class to perform long-running calculations in worker threads. You learned how to display the results of a `SwingWorker` in a GUI when the calculation completed and how to display intermediate results while the calculation was still in process.

Finally, we discussed the `Callable` and `Future` interfaces, which enable you to execute tasks that return results and to obtain those results, respectively. We use the multithreading techniques introduced in this chapter again in Chapter 27, Networking, to help build multithreaded servers that can interact with multiple clients concurrently.

Summary

Section 26.1 Introduction

- Historically, concurrency (p. 1046) has been implemented with operating-system primitives available only to experienced systems programmers.

- The Ada programming language made concurrency primitives widely available.
- Java makes concurrency available to you through the language and APIs.
- The JVM creates threads to run a program and for housekeeping tasks such as garbage collection.

Section 26.2 Thread States: Life Cycle of a Thread

- A new thread begins its life cycle in the *new* state (p. 1048). When the program starts the thread, it's placed in the *runnable* state. A thread in the *runnable* state is considered to be executing its task.
- A *runnable* thread transitions to the *waiting* state (p. 1048) to wait for another thread to perform a task. A *waiting* thread transitions to *runnable* when another thread notifies it to continue executing.
- A *runnable* thread can enter the *timed waiting* state (p. 1048) for a specified interval of time, transitioning back to *runnable* when that time interval expires or when the event it's waiting for occurs.
- A *runnable* thread can transition to the *timed waiting* state if it provides an optional wait interval when it's waiting for another thread to perform a task. Such a thread will return to the *runnable* state when it's notified by another thread or when the timed interval expires.
- A sleeping thread (p. 1049) remains in the *timed waiting* state for a designated period of time, after which it returns to the *runnable* state.
- A *runnable* thread transitions to the *blocked* state (p. 1049) when it attempts to perform a task that cannot be completed immediately and the thread must temporarily wait until that task completes. At that point, the *blocked* thread transitions to the *runnable* state, so it can resume execution.
- A *runnable* thread enters the *terminated* state (p. 1049) when it successfully completes its task or otherwise terminates (perhaps due to an error).
- At the operating-system level, the *runnable* state (p. 1048) encompasses two separate states. When a thread first transitions to the *runnable* state from the *new* state, it's in the *ready* state (p. 1049). A *ready* thread enters the *running* state (p. 1049) when the operating system dispatches it.
- Most operating systems allot a quantum (p. 1049) or timeslice in which a thread performs its task. When this expires, the thread returns to the *ready* state and another thread is assigned to the processor.
- Thread scheduling determines which thread to dispatch based on thread priorities.
- The job of an operating system's thread scheduler (p. 1050) is to determine which thread runs next.
- When a higher-priority thread enters the *ready* state, the operating system generally preempts the currently *running* thread (an operation known as preemptive scheduling; p. 1050).
- Depending on the operating system, higher-priority threads could postpone—possibly indefinitely (p. 1050)—the execution of lower-priority threads.

Section 26.3 Creating and Executing Threads with `Executor` Framework

- A `Runnable` (p. 1051) object represents a task that can execute concurrently with other tasks.
- Interface `Runnable` declares method `run` (p. 1051) in which you place the code that defines the task to perform. The thread executing a `Runnable` calls method `run` to perform the task.
- A program will not terminate until its last thread completes execution.
- You cannot predict the order in which threads will be scheduled, even if you know the order in which they were created and started.
- It's recommended that you use the `Executor` interface (p. 1051) to manage the execution of `Runnable` objects. An `Executor` object typically creates and manages a group of threads—called a thread pool (p. 1051).
- `Executors` (p. 1051) can reuse existing threads and can improve performance by optimizing the number of threads to ensure that the processor stays busy.

- `Executor` method `execute` (p. 1051) receives a `Runnable` and assigns it to an available thread in a thread pool. If there are none, the `Executor` creates a new thread or waits for one to become available.
- Interface `ExecutorService` (of package `java.util.concurrent`; p. 1051) extends interface `Executor` and declares other methods for managing the life cycle of an `Executor`.
- An object that implements the `ExecutorService` interface can be created using `static` methods declared in class `Executors` (of package `java.util.concurrent`).
- `Executors` method `newCachedThreadPool` (p. 1052) returns an `ExecutorService` that creates new threads as they're needed by the application.
- `ExecutorService` method `execute` executes its `Runnable` sometime in the future. The method returns immediately from each invocation—the program does not wait for each task to finish.
- `ExecutorService` method `shutdown` (p. 1054) notifies the `ExecutorService` to stop accepting new tasks, but continues executing existing tasks and terminates when those tasks complete execution.

Section 26.4 Thread Synchronization

- Thread synchronization (p. 1054) coordinates access to shared data by multiple concurrent threads.
- By synchronizing threads, you can ensure that each thread accessing a shared object excludes all other threads from doing so simultaneously—this is called mutual exclusion (p. 1054).
- A common way to perform synchronization is to use Java's built-in monitors. Every object has a monitor and a monitor lock (p. 1055). The monitor ensures that its object's monitor lock is held by a maximum of only one thread at any time, and thus can be used to enforce mutual exclusion.
- If an operation requires the executing thread to hold a lock while the operation is performed, a thread must acquire the lock (p. 1055) before it can proceed with the operation. Any other threads attempting to perform an operation that requires the same lock will be *blocked* until the first thread releases the lock, at which point the *blocked* threads may attempt to acquire the lock.
- To specify that a thread must hold a monitor lock to execute a block of code, the code should be placed in a `synchronized` statement (p. 1055). Such code is said to be guarded by the monitor lock (p. 1055).
- The `synchronized` statements are declared using the `synchronized` keyword:

```
synchronized ( object )
{
   statements
} // end synchronized statement
```

  where *object* is the object whose monitor lock will be acquired; *object* is normally `this` if it's the object in which the `synchronized` statement appears.
- Java also allows `synchronized` methods (p. 1055). Before executing, a non-`static` `synchronized` method must acquire the lock on the object that's used to call the method. Similary, a `static` `synchronized` method must acquire the lock on the class that's used to call the method.
- `ExecutorService` method `awaitTermination` (p. 1058) forces a program to wait for threads to terminate. It returns control to its caller either when all tasks executing in the `ExecutorService` complete or when the specified timeout elapses. If all tasks complete before the timeout elapses, the method returns `true`; otherwise, it returns `false`.
- You can simulate atomicity (p. 1060) by ensuring that only one thread performs a set of operations at a time. Atomicity can be achieved with `synchronized` statements or `synchronized` methods.
- When you share immutable data across threads, you should declare the corresponding data fields `final` to indicate that variables' values will not change after they're initialized.

Section 26.5 Producer/Consumer Relationship without Synchronization

- In a multithreaded producer/consumer relationship (p. 1062), a producer thread generates data and places it in a shared object called a buffer. A consumer thread reads data from the buffer.
- Operations on a buffer data shared by a producer and a consumer should proceed only if the buffer is in the correct state. If the buffer is not full, the producer may produce; if the buffer is not empty, the consumer may consume. If the buffer is full when the producer attempts to write into it, the producer must wait until there's space. If the buffer is empty or the previous value was already read, the consumer must wait for new data to become available.

Section 26.6 Producer/Consumer Relationship: `ArrayBlockingQueue`

- `ArrayBlockingQueue` (p. 1070) is a fully implemented buffer class from package `java.util.concurrent` that implements the `BlockingQueue` interface.
- An `ArrayBlockingQueue` can implement a shared buffer in a producer/consumer relationship. Method `put` (p. 1070) places an element at the end of the `BlockingQueue`, waiting if the queue is full. Method `take` (p. 1070) removes an element from the head of the `BlockingQueue`, waiting if the queue is empty.
- `ArrayBlockingQueue` stores shared data in an array that's sized with an argument passed to the constructor. Once created, an `ArrayBlockingQueue` is fixed in size.

Section 26.7 Producer/Consumer Relationship with Synchronization

- You can implement a shared buffer yourself using the `synchronized` keyword and `Object` methods `wait` (p. 1073), `notify` and `notifyAll`.
- A thread can call `Object` method `wait` to release an object's monitor lock, and wait in the *waiting* state while the other threads try to enter the object's `synchronized` statement(s) or method(s).
- When a thread executing a `synchronized` statement (or method) completes or satisfies the condition on which another thread may be waiting, it can call `Object` method `notify` (p. 1073) to allow a waiting thread to transition to the *runnable* state. At this point, the thread that was transitioned can attempt to reacquire the monitor lock on the object.
- If a thread calls `notifyAll` (p. 1073), then all the threads waiting for the monitor lock become eligible to reacquire the lock (that is, they all transition to the *runnable* state).

Section 26.8 Producer/Consumer Relationship: Bounded Buffers

- You cannot make assumptions about the relative speeds of concurrent threads.
- A bounded buffer (p. 1080) can be used to minimize the amount of waiting time for threads that share resources and operate at the same average speeds. If the producer temporarily produces values faster than the consumer can consume them, the producer can write additional values into the extra buffer space (if any are available). If the consumer consumes faster than the producer produces new values, the consumer can read additional values (if there are any) from the buffer.
- The key to using a bounded buffer with a producer and consumer that operate at about the same speed is to provide the buffer with enough locations to handle the anticipated "extra" production.
- The simplest way to implement a bounded buffer is to use an `ArrayBlockingQueue` for the buffer so that all of the synchronization details are handled for you.

Section 26.9 Producer/Consumer Relationship: The `Lock` and `Condition` Interfaces

- The `Lock` and `Condition` interfaces (p. 1087) give programmers more precise control over thread synchronization, but are more complicated to use.
- Any object can contain a reference to an object that implements the `Lock` interface (of package `java.util.concurrent.locks`). A thread calls the `Lock`'s `lock` method (p. 1086) to acquire the

lock. Once a `Lock` has been obtained by one thread, the `Lock` object will not allow another thread to obtain the `Lock` until the first thread releases the `Lock` (by calling the `Lock`'s `unlock` method; p. 1086).

- If several threads are trying to call method `lock` on the same `Lock` object at the same time, only one thread can obtain the lock—the others are placed in the *waiting* state. When a thread calls `unlock`, the object's lock is released and a waiting thread attempting to lock the object proceeds.
- Class `ReentrantLock` (p. 1087) is a basic implementation of the `Lock` interface.
- The `ReentrantLock` constructor takes a `boolean` that specifies whether the lock has a fairness policy (p. 1087). If `true`, the `ReentrantLock`'s fairness policy is "the longest-waiting thread will acquire the lock when it's available"—this prevents indefinite postponement. If the argument is set to `false`, there's no guarantee as to which waiting thread will acquire the lock when it's available.
- If a thread that owns a `Lock` determines that it cannot continue with its task until some condition is satisfied, the thread can wait on a condition object (p. 1087). Using `Lock` objects allows you to explicitly declare the condition objects on which a thread may need to wait.
- `Condition` (p. 1087) objects are associated with a specific `Lock` and are created by calling `Lock` method `newCondition`, which returns a `Condition` object. To wait on a `Condition`, the thread can call the `Condition`'s `await` method. This immediately releases the associated `Lock` and places the thread in the *waiting* state for that `Condition`. Other threads can then try to obtain the `Lock`.
- When a *runnable* thread completes a task and determines that a *waiting* thread can now continue, the *runnable* thread can call `Condition` method `signal` to allow a thread in that `Condition`'s *waiting* state to return to the *runnable* state. At this point, the thread that transitioned from the *waiting* state to the *runnable* state can attempt to reacquire the `Lock`.
- If multiple threads are in a `Condition`'s *waiting* state when `signal` is called, the default implementation of `Condition` signals the longest-waiting thread to transition to the *runnable* state.
- If a thread calls `Condition` method `signalAll`, then all the threads waiting for that condition transition to the *runnable* state and become eligible to reacquire the `Lock`.
- When a thread is finished with a shared object, it must call method `unlock` to release the `Lock`.
- `Lock`s allow you to interrupt waiting threads or to specify a timeout for waiting to acquire a lock—not possible with `synchronized`. Also, a `Lock` object is not constrained to be acquired and released in the same block of code, which is the case with the `synchronized` keyword.
- `Condition` objects allow you to specify multiple conditions on which threads may wait. Thus, it's possible to indicate to waiting threads that a specific condition object is now true by calling that `Condition` object's `signal` or `signallAll` methods (p. 1087). With `synchronized`, there's no way to explicitly state the condition on which threads are waiting.

Section 26.11 Multithreading with GUI

- The event dispatch thread (p. 1095) handles interactions with the application's GUI components. All tasks that interact with the GUI are placed in an event queue and executed sequentially by this thread.
- Swing GUI components are not thread safe. Thread safety is achieved by ensuring that Swing components are accessed from only the event dispatch thread.
- Performing a lengthy computation in response to a user interface interaction ties up the event dispatch thread, preventing it from attending to other tasks and causing the GUI components to become unresponsive. Long-running computations should be handled in separate threads.
- You can extend generic class `SwingWorker` (p. 1095; package `javax.swing`), which implements `Runnable`, to perform long-running computations in a worker thread and to update Swing components from the event dispatch thread based on the computations' results. You override its

`doInBackground` and `done` methods. Method `doInBackground` performs the computation and returns the result. Method `done` displays the results in the GUI.

- Class `SwingWorker`'s first type parameter indicates the type returned by the `doInBackground` method; the second indicates the type that's passed between the `publish` and `process` methods to handle intermediate results.
- Method `doInBackground` is called from a worker thread. After `doInBackground` returns, method `done` is called from the event dispatch thread to display the results.
- An `ExecutionException` is thrown if an exception occurs during the computation.
- `SwingWorker` method `publish` repeatedly sends intermediate results to method `process`, which displays the results in a GUI component. Method `setProgress` updates the progress property.
- Method `process` executes in the event dispatch thread and receives data from method `publish`. The passing of values between `publish` in the worker thread and `process` in the event dispatch thread is asynchronous; `process` is not necessarily invoked for every call to `publish`.
- `PropertyChangeListener` (p. 1108) is an interface from package `java.beans` that defines a single method, `propertyChange`. Every time method `setProgress` is invoked, a `PropertyChangeEvent` is generated to indicate that the progress property has changed.

Section 26.12 Interfaces `Callable` and `Future`

- The `Callable` (p. 1109) interface (of package `java.util.concurrent`) declares a single method named `call` that allows the thread to return a value or to throw a checked exception.
- `ExecutorService` method `submit` (p. 1109) executes a `Callable` passed in as its argument.
- Method `submit` returns an object of type `Future` (of package `java.util.concurrent`) that represents the executing `Callable`. Interface `Future` (p. 1109) declares method `get` to return the result of the `Callable` and provides other methods to manage a `Callable`'s execution.

Section 26.13 Java SE 7: Fork/Join Framework

- Java SE 7's concurrency APIs include the new fork/join framework, which helps programmers parallelize algorithms. The fork/join framework particularly well suited to divide-and-conquer-style algorithms, like the merge sort.

Self-Review Exercises

26.1 Fill in the blanks in each of the following statements:

a) A thread enters the *terminated* state when _____.

b) To pause for a designated number of milliseconds and resume execution, a thread should call method _____ of class _____.

c) Method _____ of class `Condition` moves a single thread in an object's *waiting* state to the *runnable* state.

d) Method _____ of class `Condition` moves every thread in an object's *waiting* state to the *runnable* state.

e) A(n) _____ thread enters the _____ state when it completes its task or otherwise terminates.

f) A *runnable* thread can enter the _____ state for a specified interval of time.

g) At the operating-system level, the *runnable* state actually encompasses two separate states, _____ and _____.

h) `Runnable`s are executed using a class that implements the _____ interface.

i) `ExecutorService` method _____ ends each thread in an `ExecutorService` as soon as it finishes executing its current `Runnable`, if any.

j) A thread can call method _____ on a `Condition` object to release the associated `Lock` and place that thread in the _____ state.
k) In a(n) _____ relationship, the _____ generates data and stores it in a shared object, and the _____ reads data from the shared object.
l) Class _____ implements the `BlockingQueue` interface using an array.
m) Keyword _____ indicates that only one thread at a time should execute on an object.

26.2 State whether each of the following is *true* or *false*. If *false*, explain why.
a) A thread is not *runnable* if it has terminated.
b) Some operating systems use timeslicing with threads. Therefore, they can enable threads to preempt threads of the same priority.
c) When the thread's quantum expires, the thread returns to the *running* state as the operating system assigns it to a processor.
d) On a single-processor system without timeslicing, each thread in a set of equal-priority threads (with no other threads present) runs to completion before other threads of equal priority get a chance to execute.

Answers to Self-Review Exercises

26.1 a) its `run` method ends. b) `sleep`, `Thread`. c) `signal`. d) `signalAll`. e) *runnable*, *terminated*. f) *timed waiting*. g) *ready*, *running*. h) `Executor`. i) `shutdown`. j) `await`, *waiting*. k) producer/consumer, producer, consumer. l) `ArrayBlockingQueue`. m) `synchronized`.

26.2 a) True. b) False. Timeslicing allows a thread to execute until its timeslice (or quantum) expires. Then other threads of equal priority can execute. c) False. When a thread's quantum expires, the thread returns to the *ready* state and the operating system assigns to the processor another thread. d) True.

Exercises

26.3 ***(True or False)*** State whether each of the following is *true* or *false*. If *false*, explain why.
a) Method `sleep` does not consume processor time while a thread sleeps.
b) Declaring a method `synchronized` guarantees that deadlock cannot occur.
c) Once a `ReentrantLock` has been obtained by a thread, the `ReentrantLock` object will not allow another thread to obtain the lock until the first thread releases it.
d) Swing components are thread safe.

26.4 ***(Multithreading Terms)*** Define each of the following terms.
a) thread
b) multithreading
c) *runnable* state
d) *timed waiting* state
e) preemptive scheduling
f) `Runnable` interface
g) `notifyAll` method
h) producer/consumer relationship
i) quantum

26.5 ***(Multithreading Terms)*** Discuss each of the following terms in the context of Java's threading mechanisms:
a) `synchronized`
b) producer
c) consumer

d) `wait`
e) `notify`
f) `Lock`
g) `Condition`

26.6 *(Blocked State)* List the reasons for entering the *blocked* state. For each of these, describe how the program will normally leave the *blocked* state and enter the *runnable* state.

26.7 *(Deadlock and Indefinite Postponement)* Two problems that can occur in systems that allow threads to wait are deadlock, in which one or more threads will wait forever for an event that cannot occur, and indefinite postponement, in which one or more threads will be delayed for some unpredictably long time. Give an example of how each of these problems can occur in multithreaded Java programs.

26.8 *(Bouncing Ball)* Write a program that bounces a blue ball inside a `JPanel`. The ball should begin moving with a `mousePressed` event. When the ball hits the edge of the `JPanel`, it should bounce off the edge and continue in the opposite direction. The ball should be updated using a `Runnable`.

26.9 *(Bouncing Balls)* Modify the program in Exercise 26.8 to add a new ball each time the user clicks the mouse. Provide for a minimum of 20 balls. Randomly choose the color for each new ball.

26.10 *(Bouncing Balls with Shadows)* Modify the program in Exercise 26.9 to add shadows. As a ball moves, draw a solid black oval at the bottom of the `JPanel`. You may consider adding a 3-D effect by increasing or decreasing the size of each ball when it hits the edge of the `JPanel`.

26.11 *(Circular Buffer with `Lock`s and `Condition`s)* Reimplement the example in Section 26.8 using the `Lock` and `Condition` concepts presented in Section 26.9.

27

Networking



If the presence of electricity can be made visible in any part of a circuit, I see no reason why intelligence may not be transmitted instantaneously by electricity.
—Samuel F. B. Morse

Protocol is everything.
—Francois Giuliani

What networks of railroads, highways and canals were in another age, the networks of telecommunications, information and computerization … are today.
—Bruno Kreisky

The port is near, the bells I hear, the people all exulting.
—Walt Whitman

Objectives

In this chapter you'll learn:

- Java networking with URLs, sockets and datagrams.
- To implement Java networking applications by using sockets and datagrams.
- To implement Java clients and servers that communicate with one another.
- To implement network-based collaborative applications.
- To construct a simple multithreaded server.

Outline

27.1 Introduction
27.2 Manipulating URLs
27.3 Reading a File on a Web Server
27.4 Establishing a Simple Server Using Stream Sockets
27.5 Establishing a Simple Client Using Stream Sockets
27.6 Client/Server Interaction with Stream Socket Connections
27.7 Datagrams: Connectionless Client/Server Interaction
27.8 Client/Server Tic-Tac-Toe Using a Multithreaded Server
27.9 [Web Bonus] Case Study: `DeitelMessenger`
27.10 Wrap-Up

Summary | *Self-Review Exercises* | *Answers to Self-Review Exercises* | *Exercises*

27.1 Introduction

Java provides a number of built-in networking capabilities that make it easy to develop Internet-based and web-based applications. Java can enable programs to search the world for information and to collaborate with programs running on other computers internationally, nationally or just within an organization (subject to security constraints).

Java's fundamental networking capabilities are declared by the classes and interfaces of package **`java.net`**, through which Java offers **stream-based communications** that enable applications to view networking as streams of data. The classes and interfaces of package `java.net` also offer **packet-based communications** for transmitting individual **packets** of information—commonly used to transmit data images, audio and video over the Internet. In this chapter, we show how to communicate with packets and streams of data.

We focus on both sides of the **client/server relationship**. The **client** *requests* that some action be performed, and the **server** performs the action and *responds* to the client. A common implementation of the *request-response model* is between web browsers and web servers. When a user selects a website to browse through a browser (the client application), a request is sent to the appropriate web server (the server application). The server normally responds to the client by sending an appropriate web page to be rendered by the browser.

We introduce Java's **socket-based communications**, which enable applications to view networking as if it were *file I/O*—a program can read from a **socket** or write to a socket as simply as reading from a file or writing to a file. The socket is simply a software construct that represents one endpoint of a connection. We show how to create and manipulate *stream sockets* and *datagram sockets*.

With **stream sockets**, a process establishes a **connection** to another process. While the connection is in place, data flows between the processes in continuous **streams**. Stream sockets are said to provide a **connection-oriented service**. The protocol used for transmission is the popular **TCP** (**Transmission Control Protocol**).

With **datagram sockets**, individual **packets** of information are transmitted. The protocol used—**UDP**, the **User Datagram Protocol**—is a **connectionless service** and does *not* guarantee that packets arrive in any particular *order*. With UDP, packets can even be *lost* or *duplicated*. Significant extra programming is required on your part to deal with these problems (if you choose to do so). UDP is most appropriate for network applications that

do not require the error checking and reliability of TCP. Stream sockets and the TCP protocol will be more desirable for the vast majority of Java networking applications.



Performance Tip 27.1

Connectionless services generally offer greater performance but less reliability than connection-oriented services.



Portability Tip 27.1

TCP, UDP and related protocols enable heterogeneous computer systems (i.e., those with different processors and different operating systems) to intercommunicate.

On the web at `www.deitel.com/books/jhtp9/`, we present a case study that implements a client/server chat application similar to popular instant-messaging services. The application introduces **multicasting**, in which a server can publish information and *many* clients can *subscribe* to it. When the server publishes information, *all* subscribers receive it.

27.2 Manipulating URLs

The Internet offers many protocols. The **HyperText Transfer Protocol** (**HTTP**), which forms the basis of the web, uses **URIs** (**Uniform Resource Identifiers**) to identify data on the Internet. URIs that specify the locations of websites and web pages are called **URLs** (**Uniform Resource Locators**). Common URLs refer to files or directories and can reference objects that perform complex tasks, such as database lookups and Internet searches. If you know the URL of a publicly available web page, you can access it through HTTP.

Java makes it easy to manipulate URLs. When you use a URL that refers to the exact location of a resource (e.g., a web page) as an argument to the **`showDocument`** method of interface **`AppletContext`**, the browser in which the applet is executing will access and display that resource. The applet in Figs. 27.1–27.2 demonstrates simple networking capabilities. It enables the user to select a web page from a `JList` and causes the browser to display the corresponding page. In this example, the networking is performed by the browser.

Processing Applet Parameters

This applet takes advantage of **applet parameters** specified in the HTML document that invokes the applet. When browsing the web, you'll often come across applets that are in the public domain—you can use them free of charge on your own web pages (normally in exchange for crediting the applet's creator). Many applets can be customized via parameters supplied from the HTML file that invokes the applet. For example, Fig. 27.1 contains the HTML that invokes the applet `SiteSelector` in Fig. 27.2.

```
1  <html>
2  <head>
3     <title>Site Selector</title>
4  </head>
5  <body>
6     <applet code = "SiteSelector.class" width = "300" height = "75">
7        <param name = "title0" value = "Java Home Page">
8        <param name = "location0"
```

Fig. 27.1 | HTML document to load `SiteSelector` applet. (Part 1 of 2.)

```
9           value = "http://www.oracle.com/technetwork/java/">
10        <param name = "title1" value = "Deitel">
11        <param name = "location1" value = "http://www.deitel.com/">
12        <param name = "title2" value = "JGuru">
13        <param name = "location2" value = "http://www.jGuru.com/">
14        <param name = "title3" value = "JavaWorld">
15        <param name = "location3" value = "http://www.javaworld.com/">
16     </applet>
17  </body>
18  </html>
```

Fig. 27.1 | HTML document to load `SiteSelector` applet. (Part 2 of 2.)

The HTML document contains eight parameters specified with the **`param` element**—these lines must appear between the starting and ending `applet` tags. The applet can read these values and use them to customize itself. Any number of `param` elements can appear between the starting and ending `applet` tags. Each parameter has a unique **`name`** and a **`value`**. `Applet` method **`getParameter`** returns the `value` associated with a specific parameter name as a `String`. The argument passed to `getParameter` is a `String` containing the name of the parameter in the `param` element. In this example, parameters represent the title and location of each website the user can select. Parameters specified for this applet are named `title#`, where the value of `#` starts at `0` and increments by 1 for each new title. Each title should have a corresponding location parameter of the form `location#`, where the value of `#` starts at `0` and increments by 1 for each new location. The statement

```
String title = getParameter( "title0" );
```

gets the value associated with parameter `"title0"` and assigns it to reference `title`. If there's no `param` tag containing the specified parameter, `getParameter` returns `null`.

Storing the Website Names and URLs

The applet (Fig. 27.2) obtains from the HTML document (Fig. 27.1) the choices that will be displayed in the applet's `JList`. Class `SiteSelector` uses a `HashMap` (package `java.util`) to store the website names and URLs. In this example, the *key* is the `String` in the `JList` that represents the website name, and the *value* is a `URL` object that stores the location of the website to display in the browser.

```
1  // Fig. 27.2: SiteSelector.java
2  // Loading a document from a URL into a browser.
3  import java.net.MalformedURLException;
4  import java.net.URL;
5  import java.util.HashMap;
6  import java.util.ArrayList;
7  import java.awt.BorderLayout;
8  import java.applet.AppletContext;
9  import javax.swing.JApplet;
10 import javax.swing.JLabel;
11 import javax.swing.JList;
12 import javax.swing.JScrollPane;
```

Fig. 27.2 | Loading a document from a URL into a browser. (Part 1 of 3.)

```
13  import javax.swing.event.ListSelectionEvent;
14  import javax.swing.event.ListSelectionListener;
15
16  public class SiteSelector extends JApplet
17  {
18     private HashMap< String, URL > sites; // site names and URLs
19     private ArrayList< String > siteNames; // site names
20     private JList siteChooser; // list of sites to choose from
21
22     // read parameters and set up GUI
23     public void init()
24     {
25        sites = new HashMap< String, URL >(); // create HashMap
26        siteNames = new ArrayList< String >(); // create ArrayList
27
28        // obtain parameters from HTML document
29        getSitesFromHTMLParameters();
30
31        // create GUI components and lay out interface
32        add( new JLabel( "Choose a site to browse" ), BorderLayout.NORTH );
33
34        siteChooser = new JList( siteNames.toArray() ); // populate JList
35        siteChooser.addListSelectionListener(
36           new ListSelectionListener() // anonymous inner class
37           {
38              // go to site user selected
39              public void valueChanged( ListSelectionEvent event )
40              {
41                 // get selected site name
42                 Object object = siteChooser.getSelectedValue();
43
44                 // use site name to locate corresponding URL
45                 URL newDocument = sites.get( object );
46
47                 // get applet container
48                 AppletContext browser = getAppletContext();
49
50                 // tell applet container to change pages
51                 browser.showDocument( newDocument );
52              } // end method valueChanged
53           } // end anonymous inner class
54        ); // end call to addListSelectionListener
55
56        add( new JScrollPane( siteChooser ), BorderLayout.CENTER );
57     } // end method init
58
59     // obtain parameters from HTML document
60     private void getSitesFromHTMLParameters()
61     {
62        String title; // site title
63        String location; // location of site
64        URL url; // URL of location
65        int counter = 0; // count number of sites
```

Fig. 27.2 | Loading a document from a URL into a browser. (Part 2 of 3.)

```
66
67          title = getParameter( "title" + counter ); // get first site title
68
69          // loop until no more parameters in HTML document
70          while ( title != null )
71          {
72             // obtain site location
73             location = getParameter( "location" + counter );
74
75             try // place title/URL in HashMap and title in ArrayList
76             {
77                url = new URL( location ); // convert location to URL
78                sites.put( title, url ); // put title/URL in HashMap
79                siteNames.add( title ); // put title in ArrayList
80             } // end try
81             catch ( MalformedURLException urlException )
82             {
83                urlException.printStackTrace();
84             } // end catch
85
86             ++counter;
87             title = getParameter( "title" + counter ); // get next site title
88          } // end while
89       } // end method getSitesFromHTMLParameters
90    } // end class SiteSelector
```

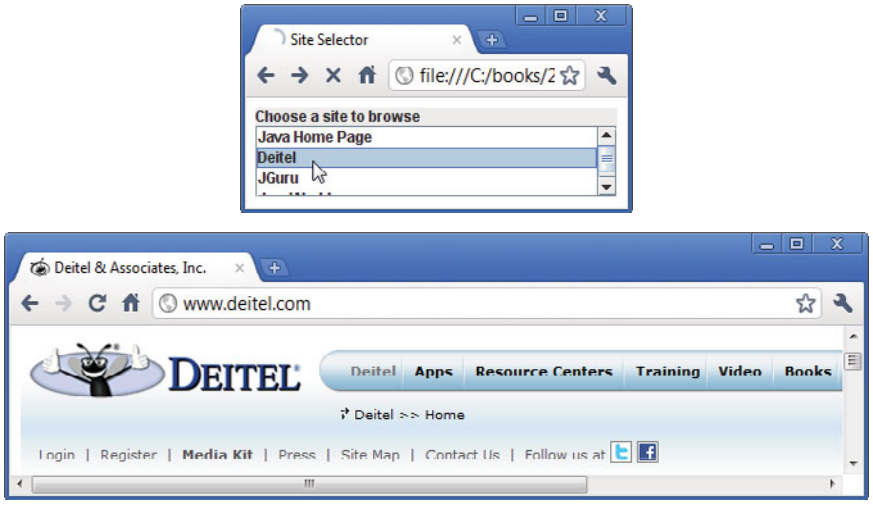


Fig. 27.2 | Loading a document from a URL into a browser. (Part 3 of 3.)

Class `SiteSelector` also contains an `ArrayList` (package `java.util`) in which the site names are placed so that they can be used to initialize the `JList` (one version of the `JList` constructor receives an array of `Objects` which is returned by `ArrayList`'s `toArray` method). An `ArrayList` is a dynamically resizable array of references. Class `ArrayList` provides method `add` to add a new element to the end of the `ArrayList`. (`ArrayList` and `HashMap` were discussed in Chapter 20.)

Lines 25–26 in the applet's `init` method (lines 23–57) create a `HashMap` object and an `ArrayList` object. Line 29 calls our utility method `getSitesFromHTMLParameters` (declared at lines 60–89) to obtain the HTML parameters from the HTML document that invoked the applet.

Method `getSitesFromHTMLParameters` uses `Applet` method `getParameter` (line 67) to obtain a website title. If the `title` is not `null`, lines 73–87 execute. Line 73 uses `Applet` method `getParameter` to obtain the website location. Line 77 uses the `location` as the value of a new `URL` object. The `URL` constructor determines whether its argument represents a valid URL. If not, the `URL` constructor throws a **`MalformedURLException`**. The `URL` constructor must be called in a `try` block. If the `URL` constructor generates a `MalformedURLException`, the call to `printStackTrace` (line 83) causes the program to output a stack trace to the Java console. On Windows machines, the Java console can be viewed by right clicking the Java icon in the notification area of the taskbar. On a Mac, go to **Applications > Utilities** and launch the **Java Preferences** app. Then on the **Advanced** tab under **Java console**, select **Show console**. On other platforms, this is typically accessible through a desktop icon. Then the program attempts to obtain the next website title. The program does not add the site for the invalid URL to the `HashMap`, so the title will not be displayed in the `JList`.

For a proper `URL`, line 78 places the `title` and `URL` into the `HashMap`, and line 79 adds the `title` to the `ArrayList`. Line 87 gets the next title from the HTML document. When the call to `getParameter` at line 87 returns `null`, the loop terminates.

Building the Applet's GUI

When method `getSitesFromHTMLParameters` returns to `init`, lines 32–56 construct the applet's GUI. Line 32 adds the `JLabel` "`Choose a site to browse`" to the `NORTH` of the `JApplet`'s `BorderLayout`. Line 34 creates `JList` `siteChooser` to allow the user to select a web page to view. Lines 35–54 register a `ListSelectionListener` to handle the `JList`'s events. Line 56 adds `siteChooser` to the `CENTER` of the `JFrame`'s `BorderLayout`.

Processing a User Selection

When the user selects a website in `siteChooser`, the program calls method `valueChanged` (lines 39–52). Line 42 obtains the selected site name from the `JList`. Line 45 passes the selected site name (the *key*) to `HashMap` method `get`, which locates and returns a reference to the corresponding `URL` object (the *value*) that's assigned to reference `newDocument`.

Line 48 uses `Applet` method **`getAppletContext`** to get a reference to an `AppletContext` object that represents the applet container. Line 51 uses this reference to invoke method `showDocument`, which receives a `URL` object as an argument and passes it to the `AppletContext` (i.e., the browser). The browser displays in the current browser window the resource associated with that `URL`. In this example, all the resources are HTML documents.

Specifying the Target Frame for Method **`showDocument`**

A second version of `AppletContext` method `showDocument` enables an applet to specify the **target frame** in which to display the web resource. This takes as arguments a `URL` object specifying the resource to display and a `String` representing the target frame. There are some special target frames that can be used as the second argument. The target frame **`_blank`** results in a new web browser window to display the content from the specified URL. The target frame **`_self`** specifies that the content from the specified URL should be displayed in the same frame as the applet (the applet's HTML page is replaced in this case).

The target frame `_top` specifies that the browser should remove the current frames in the browser window, then display the content from the specified URL in the current window.



Error-Prevention Tip 27.1

The applet in Fig. 27.2 must be run from a web browser to show the results of displaying another web page. The `appletviewer` *is capable only of executing applets—it ignores all other HTML tags. If the websites in the program contained Java applets, only those applets would appear in the* `appletviewer` *when the user selected a website. Each applet would execute in a separate* `appletviewer` *window.*

27.3 Reading a File on a Web Server

The application in Fig. 27.3 uses Swing GUI component **`JEditorPane`** (from package `javax.swing`) to display the contents of a file on a web server. The user enters a URL in the `JTextField` at the top of the window, and the application displays the corresponding document (if it exists) in the `JEditorPane`. Class `JEditorPane` is able to render both plain text and basic HTML-formatted text, as illustrated in the two screen captures (Fig. 27.4), so this application acts as a simple web browser. The application also demonstrates how to process **`HyperlinkEvents`** when the user clicks a hyperlink in the HTML document. The techniques shown in this example can also be used in applets. However, an applet is allowed to read files only on the server from which it was downloaded. [*Note:* This program might not work if your web browser must access the web through a proxy server. If you create a JNLP document for this program and use Java Web Start to launch it, Java Web Start will use the proxy server settings from your default web browser. See Chapters 23–24 for more information on Java Web Start.]

```
1  // Fig. 27.3: ReadServerFile.java
2  // Reading a file by opening a connection through a URL.
3  import java.awt.BorderLayout;
4  import java.awt.event.ActionEvent;
5  import java.awt.event.ActionListener;
6  import java.io.IOException;
7  import javax.swing.JEditorPane;
8  import javax.swing.JFrame;
9  import javax.swing.JOptionPane;
10 import javax.swing.JScrollPane;
11 import javax.swing.JTextField;
12 import javax.swing.event.HyperlinkEvent;
13 import javax.swing.event.HyperlinkListener;
14
15 public class ReadServerFile extends JFrame
16 {
17    private JTextField enterField; // JTextField to enter site name
18    private JEditorPane contentsArea; // to display website
19
20    // set up GUI
21    public ReadServerFile()
22    {
```

Fig. 27.3 | Reading a file by opening a connection through a URL. (Part 1 of 2.)

```
23          super( "Simple Web Browser" );
24
25          // create enterField and register its listener
26          enterField = new JTextField( "Enter file URL here" );
27          enterField.addActionListener(
28             new ActionListener()
29             {
30                // get document specified by user
31                public void actionPerformed( ActionEvent event )
32                {
33                   getThePage( event.getActionCommand() );
34                } // end method actionPerformed
35             } // end inner class
36          ); // end call to addActionListener
37
38          add( enterField, BorderLayout.NORTH );
39
40          contentsArea = new JEditorPane(); // create contentsArea
41          contentsArea.setEditable( false );
42          contentsArea.addHyperlinkListener(
43             new HyperlinkListener()
44             {
45                // if user clicked hyperlink, go to specified page
46                public void hyperlinkUpdate( HyperlinkEvent event )
47                {
48                   if ( event.getEventType() ==
49                        HyperlinkEvent.EventType.ACTIVATED )
50                      getThePage( event.getURL().toString() );
51                } // end method hyperlinkUpdate
52             } // end inner class
53          ); // end call to addHyperlinkListener
54
55          add( new JScrollPane( contentsArea ), BorderLayout.CENTER );
56          setSize( 400, 300 ); // set size of window
57          setVisible( true ); // show window
58       } // end ReadServerFile constructor
59
60       // load document
61       private void getThePage( String location )
62       {
63          try // load document and display location
64          {
65             contentsArea.setPage( location ); // set the page
66             enterField.setText( location ); // set the text
67          } // end try
68          catch ( IOException ioException )
69          {
70             JOptionPane.showMessageDialog( this,
71                "Error retrieving specified URL", "Bad URL",
72                JOptionPane.ERROR_MESSAGE );
73          } // end catch
74       } // end method getThePage
75    } // end class ReadServerFile
```

Fig. 27.3 | Reading a file by opening a connection through a URL. (Part 2 of 2.)

```
1  // Fig. 27.4: ReadServerFileTest.java
2  // Create and start a ReadServerFile.
3  import javax.swing.JFrame;
4
5  public class ReadServerFileTest
6  {
7     public static void main( String[] args )
8     {
9        ReadServerFile application = new ReadServerFile();
10       application.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
11    } // end main
12 } // end class ReadServerFileTest
```

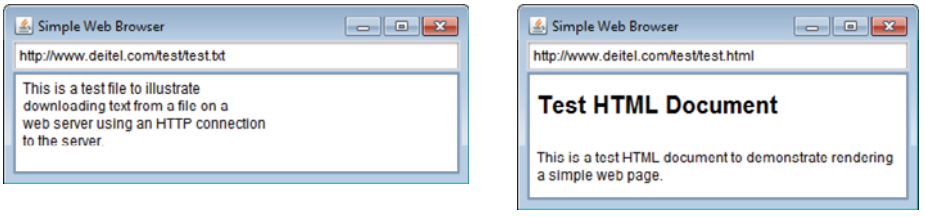


Fig. 27.4 | Test class for `ReadServerFile`.

The application class `ReadServerFile` contains `JTextField enterField`, in which the user enters the URL of the file to read and `JEditorPane contentsArea` to display the file's contents. When the user presses the *Enter* key in `enterField`, the application calls method `actionPerformed` (lines 31–34). Line 33 uses `ActionEvent` method `getActionCommand` to get the `String` the user input in the `JTextField` and passes the `String` to utility method `getThePage` (lines 61–74).

Line 65 invokes `JEditorPane` method **`setPage`** to download the document specified by `location` and display it in the `JEditorPane`. If there's an error downloading the document, method `setPage` throws an `IOException`. Also, if an invalid URL is specified, a `MalformedURLException` (a subclass of `IOException`) occurs. If the document loads successfully, line 66 displays the current location in `enterField`.

Typically, an HTML document contains **hyperlinks** that, when clicked, provide quick access to another document on the web. If a `JEditorPane` contains an HTML document and the user clicks a hyperlink, the `JEditorPane` generates a **`HyperlinkEvent`** (package `javax.swing.event`) and notifies all registered **`HyperlinkListeners`** (package `javax.swing.event`) of that event. Lines 42–53 register a `HyperlinkListener` to handle `HyperlinkEvents`. When a `HyperlinkEvent` occurs, the program calls method **`hyperlinkUpdate`** (lines 46–51). Lines 48–49 use `HyperlinkEvent` method **`getEventType`** to determine the type of the `HyperlinkEvent`. Class `HyperlinkEvent` contains a `public` nested class called **`EventType`** that declares three `static EventType` objects, which represent the hyperlink event types. **`ACTIVATED`** indicates that the user clicked a hyperlink to change web pages, **`ENTERED`** indicates that the user moved the mouse over a hyperlink and **`EXITED`** indicates that the user moved the mouse away from a hyperlink. If a hyperlink was `ACTIVATED`, line 50 uses `HyperlinkEvent` method **`getURL`** to obtain the `URL` represented by the hyperlink. Method `toString` converts the returned `URL` to a `String` that can be passed to utility method `getThePage`.



Look-and-Feel Observation 27.1

A `JEditorPane` generates `HyperlinkEvents` only if it's uneditable.

27.4 Establishing a Simple Server Using Stream Sockets

The two examples discussed so far use *high-level* Java networking capabilities to communicate between applications. In the examples, it was not your responsibility to establish the connection between a client and a server. The first program relied on the web browser to communicate with a web server. The second program relied on a `JEditorPane` to perform the connection. This section begins our discussion of creating your own applications that can communicate with one another.

Step 1: Create a `ServerSocket`

Establishing a simple server in Java requires five steps. *Step 1* is to create a **`ServerSocket`** object. A call to the `ServerSocket` constructor, such as

```
ServerSocket server = new ServerSocket( portNumber, queueLength );
```

registers an available TCP **port number** and specifies the maximum number of clients that can wait to connect to the server (i.e., the **queue length**). The port number is used by clients to locate the server application on the server computer. This is often called the **handshake point**. If the queue is full, the server refuses client connections. The constructor establishes the port where the server waits for connections from clients—a process known as **binding the server to the port**. Each client will ask to connect to the server on this **port**. Only one application at a time can be bound to a specific port on the server.



Software Engineering Observation 27.1

Port numbers can be between 0 and 65,535. Most operating systems reserve port numbers below 1024 for system services (e.g., e-mail and World Wide Web servers). Generally, these ports should not be specified as connection ports in user programs. In fact, some operating systems require special access privileges to bind to port numbers below 1024.

Step 2: Wait for a Connection

Programs manage each client connection with a **`Socket`** object. In *Step 2*, the server listens indefinitely (or **blocks**) for an attempt by a client to connect. To listen for a client connection, the program calls `ServerSocket` method **`accept`**, as in

```
Socket connection = server.accept();
```

which returns a `Socket` when a connection with a client is established. The `Socket` allows the server to interact with the client. The interactions with the client actually occur at a different server port from the *handshake point*. This allows the port specified in *Step 1* to be used again in a multithreaded server to accept another client connection. We demonstrate this concept in Section 27.8.

Step 3: Get the `Socket`*'s I/O Streams*

Step 3 is to get the `OutputStream` and `InputStream` objects that enable the server to communicate with the client by sending and receiving bytes. The server sends information to

the client via an `OutputStream` and receives information from the client via an `InputStream`. The server invokes method **`getOutputStream`** on the `Socket` to get a reference to the `Socket`'s `OutputStream` and invokes method **`getInputStream`** on the `Socket` to get a reference to the `Socket`'s `InputStream`.

The stream objects can be used to send or receive individual bytes or sequences of bytes with the `OutputStream`'s method `write` and the `InputStream`'s method `read`, respectively. Often it's useful to send or receive values of primitive types (e.g., `int` and `double`) or `Serializable` objects (e.g., `String`s or other serializable types) rather than sending bytes. In this case, we can use the techniques discussed in Chapter 17 to wrap other stream types (e.g., `ObjectOutputStream` and `ObjectInputStream`) around the `OutputStream` and `InputStream` associated with the `Socket`. For example,

```
ObjectInputStream input =
   new ObjectInputStream( connection.getInputStream() );
ObjectOutputStream output =
   new ObjectOutputStream( connection.getOutputStream() );
```

The beauty of establishing these relationships is that whatever the server writes to the `ObjectOutputStream` is sent via the `OutputStream` and is available at the client's `InputStream`, and whatever the client writes to its `OutputStream` (with a corresponding `ObjectOutputStream`) is available via the server's `InputStream`. The transmission of the data over the network is seamless and is handled completely by Java.

Step 4: Perform the Processing

Step 4 is the *processing* phase, in which the server and the client communicate via the `OutputStream` and `InputStream` objects.

Step 5: Close the Connection

In *Step 5*, when the transmission is complete, the server closes the connection by invoking the **`close`** method on the streams and on the `Socket`.



Software Engineering Observation 27.2

With sockets, network I/O appears to Java programs to be similar to sequential file I/O. Sockets hide much of the complexity of network programming.



Software Engineering Observation 27.3

A multithreaded server can take the `Socket` returned by each call to `accept` and create a new thread that manages network I/O across that `Socket`. Alternatively, a multithreaded server can maintain a pool of threads (a set of already existing threads) ready to manage network I/O across the new `Sockets` as they're created. These techniques enable multithreaded servers to manage many simultaneous client connections.



Performance Tip 27.2

In high-performance systems in which memory is abundant, a multithreaded server can create a pool of threads that can be assigned quickly to handle network I/O for new `Sockets` as they're created. Thus, when the server receives a connection, it need not incur thread-creation overhead. *When the connection is closed, the thread is returned to the pool for reuse.*

27.5 Establishing a Simple Client Using Stream Sockets

Establishing a simple client in Java requires four steps.

Step 1: Create a `Socket` *to Connect to the sServer*

In *Step 1*, we create a `Socket` to connect to the server. The `Socket` constructor establishes the connection. For example, the statement

```
Socket connection = new Socket( serverAddress, port );
```

uses the `Socket` constructor with two arguments—the server's address (*serverAddress*) and the *port* number. If the connection attempt is successful, this statement returns a `Socket`. A connection attempt that fails throws an instance of a subclass of `IOException`, so many programs simply catch `IOException`. An **`UnknownHostException`** occurs specifically when the system is unable to resolve the server name specified in the call to the `Socket` constructor to a corresponding IP address.

Step 2: Get the `Socket`*'s I/O Streams*

In *Step 2*, the client uses `Socket` methods `getInputStream` and `getOutputStream` to obtain references to the `Socket`'s `InputStream` and `OutputStream`. As we mentioned in the preceding section, we can use the techniques of Chapter 17 to wrap other stream types around the `InputStream` and `OutputStream` associated with the `Socket`. If the server is sending information in the form of actual types, the client should receive the information in the same format. Thus, if the server sends values with an `ObjectOutputStream`, the client should read those values with an `ObjectInputStream`.

Step 3: Perform the Processing

Step 3 is the processing phase in which the client and the server communicate via the `InputStream` and `OutputStream` objects.

Step 4: Close the Connection

In *Step 4*, the client closes the connection when the transmission is complete by invoking the `close` method on the streams and on the `Socket`. The client must determine when the server is finished sending information so that it can call `close` to close the `Socket` connection. For example, the `InputStream` method `read` returns the value –1 when it detects end-of-stream (also called EOF—end-of-file). If an `ObjectInputStream` reads information from the server, an `EOFException` occurs when the client attempts to read a value from a stream on which end-of-stream is detected.

27.6 Client/Server Interaction with Stream Socket Connections

Figures 27.5 and 27.7 use stream sockets, `ObjectInputStream` and `ObjectOutputStream` to demonstrate a simple **client/server chat application**. The server waits for a client connection attempt. When a client connects to the server, the server application sends the client a `String` object (recall that `String`s are `Serializable` objects) indicating that the connection was successful. Then the client displays the message. The client and server applications each provide text fields that allow the user to type a message and send it to the other application. When the client or the server sends the `String` `"TERMINATE"`, the con-

nection terminates. Then the server waits for the next client to connect. The declaration of class `Server` appears in Fig. 27.5. The declaration of class `Client` appears in Fig. 27.7. The screen captures showing the execution between the client and the server are shown in Fig. 27.8.

Server Class

`Server`'s constructor (Fig. 27.5, lines 30–55) creates the server's GUI, which contains a `JTextField` and a `JTextArea`. `Server` displays its output in the `JTextArea`. When the `main` method (lines 6–11 of Fig. 27.6) executes, it creates a `Server` object, specifies the window's default close operation and calls method `runServer` (Fig. 27.5, lines 57–86).

```
1  // Fig. 27.5: Server.java
2  // Server portion of a client/server stream-socket connection.
3  import java.io.EOFException;
4  import java.io.IOException;
5  import java.io.ObjectInputStream;
6  import java.io.ObjectOutputStream;
7  import java.net.ServerSocket;
8  import java.net.Socket;
9  import java.awt.BorderLayout;
10 import java.awt.event.ActionEvent;
11 import java.awt.event.ActionListener;
12 import javax.swing.JFrame;
13 import javax.swing.JScrollPane;
14 import javax.swing.JTextArea;
15 import javax.swing.JTextField;
16 import javax.swing.SwingUtilities;
17
18 public class Server extends JFrame
19 {
20    private JTextField enterField; // inputs message from user
21    private JTextArea displayArea; // display information to user
22    private ObjectOutputStream output; // output stream to client
23    private ObjectInputStream input; // input stream from client
24    private ServerSocket server; // server socket
25    private Socket connection; // connection to client
26    private int counter = 1; // counter of number of connections
27
28    // set up GUI
29    public Server()
30    {
31       super( "Server" );
32
33       enterField = new JTextField(); // create enterField
34       enterField.setEditable( false );
35       enterField.addActionListener(
36          new ActionListener()
37          {
38             // send message to client
39             public void actionPerformed( ActionEvent event )
40             {
```

Fig. 27.5 | Server portion of a client/server stream-socket connection. (Part 1 of 4.)

```
41                sendData( event.getActionCommand() );
42                enterField.setText( "" );
43             } // end method actionPerformed
44          } // end anonymous inner class
45       ); // end call to addActionListener
46
47       add( enterField, BorderLayout.NORTH );
48
49       displayArea = new JTextArea(); // create displayArea
50       add( new JScrollPane( displayArea ), BorderLayout.CENTER );
51
52       setSize( 300, 150 ); // set size of window
53       setVisible( true ); // show window
54    } // end Server constructor
55
56    // set up and run server
57    public void runServer()
58    {
59       try // set up server to receive connections; process connections
60       {
61          server = new ServerSocket( 12345, 100 ); // create ServerSocket
62
63          while ( true )
64          {
65             try
66             {
67                waitForConnection(); // wait for a connection
68                getStreams(); // get input & output streams
69                processConnection(); // process connection
70             } // end try
71             catch ( EOFException eofException )
72             {
73                displayMessage( "\nServer terminated connection" );
74             } // end catch
75             finally
76             {
77                closeConnection(); // close connection
78                ++counter;
79             } // end finally
80          } // end while
81       } // end try
82       catch ( IOException ioException )
83       {
84          ioException.printStackTrace();
85       } // end catch
86    } // end method runServer
87
88    // wait for connection to arrive, then display connection info
89    private void waitForConnection() throws IOException
90    {
91       displayMessage( "Waiting for connection\n" );
92       connection = server.accept(); // allow server to accept connection
```

Fig. 27.5 | Server portion of a client/server stream-socket connection. (Part 2 of 4.)

```
93          displayMessage( "Connection " + counter + " received from: " +
94             connection.getInetAddress().getHostName() );
95       } // end method waitForConnection
96
97       // get streams to send and receive data
98       private void getStreams() throws IOException
99       {
100         // set up output stream for objects
101         output = new ObjectOutputStream( connection.getOutputStream() );
102         output.flush(); // flush output buffer to send header information
103
104         // set up input stream for objects
105         input = new ObjectInputStream( connection.getInputStream() );
106
107         displayMessage( "\nGot I/O streams\n" );
108      } // end method getStreams
109
110      // process connection with client
111      private void processConnection() throws IOException
112      {
113         String message = "Connection successful";
114         sendData( message ); // send connection successful message
115
116         // enable enterField so server user can send messages
117         setTextFieldEditable( true );
118
119         do // process messages sent from client
120         {
121            try // read message and display it
122            {
123               message = ( String ) input.readObject(); // read new message
124               displayMessage( "\n" + message ); // display message
125            } // end try
126            catch ( ClassNotFoundException classNotFoundException )
127            {
128               displayMessage( "\nUnknown object type received" );
129            } // end catch
130
131         } while ( !message.equals( "CLIENT>>> TERMINATE" ) );
132      } // end method processConnection
133
134      // close streams and socket
135      private void closeConnection()
136      {
137         displayMessage( "\nTerminating connection\n" );
138         setTextFieldEditable( false ); // disable enterField
139
140         try
141         {
142            output.close(); // close output stream
143            input.close(); // close input stream
144            connection.close(); // close socket
145         } // end try
```

Fig. 27.5 | Server portion of a client/server stream-socket connection. (Part 3 of 4.)

```
146         catch ( IOException ioException )
147         {
148            ioException.printStackTrace();
149         } // end catch
150      } // end method closeConnection
151
152      // send message to client
153      private void sendData( String message )
154      {
155         try // send object to client
156         {
157            output.writeObject( "SERVER>>> " + message );
158            output.flush(); // flush output to client
159            displayMessage( "\nSERVER>>> " + message );
160         } // end try
161         catch ( IOException ioException )
162         {
163            displayArea.append( "\nError writing object" );
164         } // end catch
165      } // end method sendData
166
167      // manipulates displayArea in the event-dispatch thread
168      private void displayMessage( final String messageToDisplay )
169      {
170         SwingUtilities.invokeLater(
171            new Runnable()
172            {
173               public void run() // updates displayArea
174               {
175                  displayArea.append( messageToDisplay ); // append message
176               } // end method run
177            } // end anonymous inner class
178         ); // end call to SwingUtilities.invokeLater
179      } // end method displayMessage
180
181      // manipulates enterField in the event-dispatch thread
182      private void setTextFieldEditable( final boolean editable )
183      {
184         SwingUtilities.invokeLater(
185            new Runnable()
186            {
187               public void run() // sets enterField's editability
188               {
189                  enterField.setEditable( editable );
190               } // end method run
191            }  // end inner class
192         ); // end call to SwingUtilities.invokeLater
193      } // end method setTextFieldEditable
194   } // end class Server
```

Fig. 27.5 | Server portion of a client/server stream-socket connection. (Part 4 of 4.)

```
1  // Fig. 27.6: ServerTest.java
2  // Test the Server application.
3  import javax.swing.JFrame;
4
5  public class ServerTest
6  {
7     public static void main( String[] args )
8     {
9        Server application = new Server(); // create server
10       application.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
11       application.runServer(); // run server application
12    } // end main
13 } // end class ServerTest
```

Fig. 27.6 | Test class for `Server`.

Method `runServer`

Method `runServer` (Fig. 27.5, lines 57–86) sets up the server to receive a connection and processes one connection at a time. Line 61 creates a `ServerSocket` called `server` to wait for connections. The `ServerSocket` listens for a connection from a client at port `12345`. The second argument to the constructor is the number of connections that can wait in a queue to connect to the server (`100` in this example). If the queue is full when a client attempts to connect, the server refuses the connection.



Common Programming Error 27.1

Specifying a port that's already in use or specifying an invalid port number when creating a `ServerSocket` *results in a* **`BindException`**.

Line 67 calls method `waitForConnection` (declared at lines 89–95) to wait for a client connection. After the connection is established, line 68 calls method `getStreams` (declared at lines 98–108) to obtain references to the connection's streams. Line 69 calls method `processConnection` (declared at lines 111–132) to send the initial connection message to the client and to process all messages received from the client. The `finally` block (lines 75–79) terminates the client connection by calling method `closeConnection` (lines 135–150), even if an exception occurs. These methods call `displayMessage` (lines 168–179), which uses the event-dispatch thread to display messages in the application's `JTextArea`. **`SwingUtilities`** method **`invokeLater`** receives a `Runnable` object as its argument and places it into the event-dispatch thread for execution. This ensures that we don't modify a GUI component from a thread other than the event-dispatch thread, which is important since *Swing GUI components are not thread safe*. We use a similar technique in method `setTextFieldEditable` (lines 182–193), to set the editability of `enterField`. For more information on interface `Runnable`, see Chapter 26.

Method `waitForConnection`

Method `waitForConnection` (lines 89–95) uses `ServerSocket` method `accept` (line 92) to wait for a connection from a client. When a connection occurs, the resulting `Socket` is assigned to `connection`. Method `accept` blocks until a connection is received (i.e., the thread in which `accept` is called stops executing until a client connects). Lines 93–94 output the host name of the computer that made the connection. `Socket` method **`getInet-`**

Address returns an **InetAddress** (package java.net) containing information about the client computer. InetAddress method **getHostName** returns the host name of the client computer. For example, a special IP address (**127.0.0.1**) and host name (**localhost**) are useful for testing networking applications on your local computer (this is also known as the **loopback address**). If getHostName is called on an InetAddress containing 127.0.0.1, the corresponding host name returned by the method will be localhost.

Method getStreams

Method getStreams (lines 98–108) obtains the Socket's streams and uses them to initialize an ObjectOutputStream (line 101) and an ObjectInputStream (line 105), respectively. Note the call to ObjectOutputStream method **flush** at line 102. This statement causes the ObjectOutputStream on the server to send a **stream header** to the corresponding client's ObjectInputStream. The stream header contains such information as the version of object serialization being used to send objects. This information is required by the ObjectInputStream so that it can prepare to receive those objects correctly.



Software Engineering Observation 27.4

When using ObjectOutputStream and ObjectInputStream to send and receive data over a network connection, always create the ObjectOutputStream first and flush the stream so that the client's ObjectInputStream can prepare to receive the data. This is required for networking applications that communicate using ObjectOutputStream and ObjectInputStream.



Performance Tip 27.3

A computer's I/O components are typically much slower than its memory. Output buffers are used to increase the efficiency of an application by sending larger amounts of data fewer times, reducing the number of times an application accesses the computer's I/O components.

Method processConnection

Line 114 of method processConnection (lines 111–132) calls method sendData to send "SERVER>>> Connection successful" as a String to the client. The loop at lines 119–131 executes until the server receives the message "CLIENT>>> TERMINATE". Line 123 uses ObjectInputStream method readObject to read a String from the client. Line 124 invokes method displayMessage to append the message to the JTextArea.

Method closeConnection

When the transmission is complete, method processConnection returns, and the program calls method closeConnection (lines 135–150) to close the streams associated with the Socket and close the Socket. Then the server waits for the next connection attempt from a client by continuing with line 67 at the beginning of the while loop.

Server receives a connection, processes it, closes it and waits for the next connection. A more likely scenario would be a Server that receives a connection, sets it up to be processed as a separate thread of execution, then immediately waits for new connections. The separate threads that process existing connections can continue to execute while the Server concentrates on new connection requests. This makes the server more efficient, because multiple client requests can be processed concurrently. We demonstrate a *multithreaded server* in Section 27.8.

Processing User Interactions

When the user of the server application enters a `String` in the text field and presses the *Enter* key, the program calls method `actionPerformed` (lines 39–43), which reads the `String` from the text field and calls utility method `sendData` (lines 153–165) to send the `String` to the client. Method `sendData` writes the object, flushes the output buffer and appends the same `String` to the text area in the server window. It's not necessary to invoke `displayMessage` to modify the text area here, because method `sendData` is called from an event handler—thus, `sendData` executes as part of the *event-dispatch thread*.

`Client` Class

Like class `Server`, class `Client`'s constructor (Fig. 27.7, lines 29–56) creates the GUI of the application (a `JTextField` and a `JTextArea`). `Client` displays its output in the text area. When method `main` (lines 7–19 of Fig. 27.8) executes, it creates an instance of class `Client`, specifies the window's default close operation and calls method `runClient` (Fig. 27.7, lines 59–79). In this example, you can execute the client from any computer on the Internet and specify the IP address or host name of the server computer as a command-line argument to the program. For example, the command

```
java Client 192.168.1.15
```

attempts to connect to the `Server` on the computer with IP address `192.168.1.15`.

```
1  // Fig. 27.7: Client.java
2  // Client portion of a stream-socket connection between client and server.
3  import java.io.EOFException;
4  import java.io.IOException;
5  import java.io.ObjectInputStream;
6  import java.io.ObjectOutputStream;
7  import java.net.InetAddress;
8  import java.net.Socket;
9  import java.awt.BorderLayout;
10 import java.awt.event.ActionEvent;
11 import java.awt.event.ActionListener;
12 import javax.swing.JFrame;
13 import javax.swing.JScrollPane;
14 import javax.swing.JTextArea;
15 import javax.swing.JTextField;
16 import javax.swing.SwingUtilities;
17
18 public class Client extends JFrame
19 {
20    private JTextField enterField; // enters information from user
21    private JTextArea displayArea; // display information to user
22    private ObjectOutputStream output; // output stream to server
23    private ObjectInputStream input; // input stream from server
24    private String message = ""; // message from server
25    private String chatServer; // host server for this application
26    private Socket client; // socket to communicate with server
27
```

Fig. 27.7 | Client portion of a stream-socket connection between client and server. (Part 1 of 5.)

```
28    // initialize chatServer and set up GUI
29    public Client( String host )
30    {
31       super( "Client" );
32
33       chatServer = host; // set server to which this client connects
34
35       enterField = new JTextField(); // create enterField
36       enterField.setEditable( false );
37       enterField.addActionListener(
38          new ActionListener()
39          {
40             // send message to server
41             public void actionPerformed( ActionEvent event )
42             {
43                sendData( event.getActionCommand() );
44                enterField.setText( "" );
45             } // end method actionPerformed
46          } // end anonymous inner class
47       ); // end call to addActionListener
48
49       add( enterField, BorderLayout.NORTH );
50
51       displayArea = new JTextArea(); // create displayArea
52       add( new JScrollPane( displayArea ), BorderLayout.CENTER );
53
54       setSize( 300, 150 ); // set size of window
55       setVisible( true ); // show window
56    } // end Client constructor
57
58    // connect to server and process messages from server
59    public void runClient()
60    {
61       try // connect to server, get streams, process connection
62       {
63          connectToServer(); // create a Socket to make connection
64          getStreams(); // get the input and output streams
65          processConnection(); // process connection
66       } // end try
67       catch ( EOFException eofException )
68       {
69          displayMessage( "\nClient terminated connection" );
70       } // end catch
71       catch ( IOException ioException )
72       {
73          ioException.printStackTrace();
74       } // end catch
75       finally
76       {
77          closeConnection(); // close connection
78       } // end finally
79    } // end method runClient
```

Fig. 27.7 | Client portion of a stream-socket connection between client and server. (Part 2 of 5.)

```
80
81     // connect to server
82     private void connectToServer() throws IOException
83     {
84        displayMessage( "Attempting connection\n" );
85
86        // create Socket to make connection to server
87        client = new Socket( InetAddress.getByName( chatServer ), 12345 );
88
89        // display connection information
90        displayMessage( "Connected to: " +
91           client.getInetAddress().getHostName() );
92     } // end method connectToServer
93
94     // get streams to send and receive data
95     private void getStreams() throws IOException
96     {
97        // set up output stream for objects
98        output = new ObjectOutputStream( client.getOutputStream() );
99        output.flush(); // flush output buffer to send header information
100
101       // set up input stream for objects
102       input = new ObjectInputStream( client.getInputStream() );
103
104       displayMessage( "\nGot I/O streams\n" );
105    } // end method getStreams
106
107    // process connection with server
108    private void processConnection() throws IOException
109    {
110       // enable enterField so client user can send messages
111       setTextFieldEditable( true );
112
113       do // process messages sent from server
114       {
115          try // read message and display it
116          {
117             message = ( String ) input.readObject(); // read new message
118             displayMessage( "\n" + message ); // display message
119          } // end try
120          catch ( ClassNotFoundException classNotFoundException )
121          {
122             displayMessage( "\nUnknown object type received" );
123          } // end catch
124
125       } while ( !message.equals( "SERVER>>> TERMINATE" ) );
126    } // end method processConnection
127
128    // close streams and socket
129    private void closeConnection()
130    {
131       displayMessage( "\nClosing connection" );
132       setTextFieldEditable( false ); // disable enterField
```

Fig. 27.7 | Client portion of a stream-socket connection between client and server. (Part 3 of 5.)

```
133
134        try
135        {
136           output.close(); // close output stream
137           input.close(); // close input stream  1
138           client.close(); // close socket
139        } // end try
140        catch ( IOException ioException )
141        {
142           ioException.printStackTrace();
143        } // end catch
144     } // end method closeConnection
145
146     // send message to server
147     private void sendData( String message )
148     {
149        try // send object to server
150        {
151           output.writeObject( "CLIENT>>> " + message );
152           output.flush(); // flush data to output
153           displayMessage( "\nCLIENT>>> " + message );
154        } // end try
155        catch ( IOException ioException )
156        {
157           displayArea.append( "\nError writing object" );
158        } // end catch
159     } // end method sendData
160
161     // manipulates displayArea in the event-dispatch thread
162     private void displayMessage( final String messageToDisplay )
163     {
164        SwingUtilities.invokeLater(
165           new Runnable()
166           {
167              public void run() // updates displayArea
168              {
169                 displayArea.append( messageToDisplay );
170              } // end method run
171           }  // end anonymous inner class
172        ); // end call to SwingUtilities.invokeLater
173     } // end method displayMessage
174
175     // manipulates enterField in the event-dispatch thread
176     private void setTextFieldEditable( final boolean editable )
177     {
178        SwingUtilities.invokeLater(
179           new Runnable()
180           {
181              public void run() // sets enterField's editability
182              {
183                 enterField.setEditable( editable );
184              } // end method run
185           } // end anonymous inner class
```

Fig. 27.7 | Client portion of a stream-socket connection between client and server. (Part 4 of 5.)

```
186       ); // end call to SwingUtilities.invokeLater
187    } // end method setTextFieldEditable
188 } // end class Client
```

Fig. 27.7 | Client portion of a stream-socket connection between client and server. (Part 5 of 5.)

```
1  // Fig. 27.8: ClientTest.java
2  // Class that tests the Client.
3  import javax.swing.JFrame;
4
5  public class ClientTest
6  {
7     public static void main( String[] args )
8     {
9        Client application; // declare client application
10
11       // if no command line args
12       if ( args.length == 0 )
13          application = new Client( "127.0.0.1" ); // connect to localhost
14       else
15          application = new Client( args[ 0 ] ); // use args to connect
16
17       application.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
18       application.runClient(); // run client application
19    } // end main
20 } // end class ClientTest
```

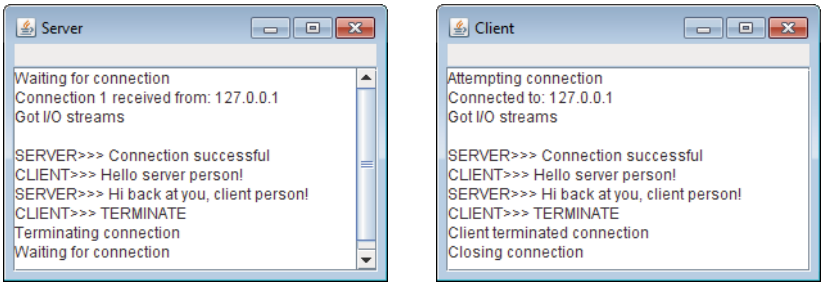


Fig. 27.8 | Class that tests the `Client`.

Method `runClient`

`Client` method `runClient` (Fig. 27.7, lines 59–79) sets up the connection to the server, processes messages received from the server and closes the connection when communication is complete. Line 63 calls method `connectToServer` (declared at lines 82–92) to perform the connection. After connecting, line 64 calls method `getStreams` (declared at lines 95–105) to obtain references to the `Socket`'s stream objects. Then line 65 calls method `processConnection` (declared at lines 108–126) to receive and display messages sent from the server. The `finally` block (lines 75–78) calls `closeConnection` (lines 129–144) to close the streams and the `Socket` even if an exception occurred. Method `displayMessage` (lines 162–173) is called from these methods to use the event-dispatch thread to display messages in the application's text area.

Method `connectToServer`

Method `connectToServer` (lines 82–92) creates a `Socket` called `client` (line 87) to establish a connection. The arguments to the `Socket` constructor are the IP address of the server computer and the port number (12345) where the server application is awaiting client connections. In the first argument, `InetAddress` `static` method **`getByName`** returns an `InetAddress` object containing the IP address specified as a command-line argument to the application (or `127.0.0.1` if none was specified). Method `getByName` can receive a `String` containing either the actual IP address or the host name of the server. The first argument also could have been written other ways. For the `localhost` address `127.0.0.1`, the first argument could be specified with either of the following expressions:

```
InetAddress.getByName( "localhost" )
InetAddress.getLocalHost()
```

Other versions of the `Socket` constructor receive the IP address or host name as a `String`. The first argument could have been specified as the IP address `"127.0.0.1"` or the host name `"localhost"`. We chose to demonstrate the client/server relationship by connecting between applications on the same computer (`localhost`). Normally, this first argument would be the IP address of another computer. The `InetAddress` object for another computer can be obtained by specifying the computer's IP address or host name as the argument to `InetAddress` method `getByName`. The `Socket` constructor's second argument is the server port number. This *must* match the port number at which the server is waiting for connections (called the *handshake point*). Once the connection is made, lines 90–91 display a message in the text area indicating the name of the server computer to which the client has connected.

The `Client` uses an `ObjectOutputStream` to send data to the server and an `ObjectInputStream` to receive data from the server. Method `getStreams` (lines 95–105) creates the `ObjectOutputStream` and `ObjectInputStream` objects that use the streams associated with the `client` socket.

Methods `processConnection` *and* `closeConnection`

Method `processConnection` (lines 108–126) contains a loop that executes until the client receives the message `"SERVER>>> TERMINATE"`. Line 117 reads a `String` object from the server. Line 118 invokes `displayMessage` to append the message to the text area. When the transmission is complete, method `closeConnection` (lines 129–144) closes the streams and the `Socket`.

Processing User Interactions

When the client application user enters a `String` in the text field and presses *Enter*, the program calls method `actionPerformed` (lines 41–45) to read the `String`, then invokes utility method `sendData` (147–159) to send the `String` to the server. Method `sendData` writes the object, flushes the output buffer and appends the same `String` to the client window's `JTextArea`. Once again, it's not necessary to invoke utility method `displayMessage` to modify the text area here, because method `sendData` is called from an event handler.

27.7 Datagrams: Connectionless Client/Server Interaction

We've been discussing connection-oriented, streams-based transmission. Now we consider **connectionless transmission with datagrams**.

Connection-oriented transmission is like the telephone system in which you dial and are given a connection to the telephone of the person with whom you wish to communicate. The connection is maintained for your phone call, *even when you're not talking*.

Connectionless transmission with datagrams is more like the way mail is carried via the postal service. If a large message will not fit in one envelope, you break it into separate pieces that you place in sequentially numbered envelopes. All of the letters are then mailed at once. The letters could arrive *in order*, *out of order* or *not at all* (the last case is rare). The person at the receiving end *reassembles* the pieces into sequential order before attempting to make sense of the message.

If your message is small enough to fit in one envelope, you need not worry about the "out-of-sequence" problem, but it's still possible that your message might not arrive. One advantage of datagrams over postal mail is that duplicates of datagrams can arrive at the receiving computer.

Figures 27.9–27.12 use datagrams to send packets of information via the User Datagram Protocol (UDP) between a client application and a server application. In the `Client` application (Fig. 27.11), the user types a message into a text field and presses *Enter*. The program converts the message into a `byte` array and places it in a datagram packet that's sent to the server. The `Server` (Figs. 27.9–27.10) receives the packet and displays the information in it, then **echoes** the packet back to the client. Upon receiving the packet, the client displays the information it contains.

Server Class

Class `Server` (Fig. 27.9) declares two **`DatagramPackets`** that the server uses to send and receive information and one **`DatagramSocket`** that sends and receives the packets. The constructor (lines 19–37), which is called from `main` (Fig. 27.10, lines 7–12), creates the GUI in which the packets of information will be displayed. Line 30 creates the `DatagramSocket` in a `try` block. Line 30 in Fig. 27.9 uses the `DatagramSocket` constructor that takes an integer port-number argument (`5000` in this example) to bind the server to a port where it can receive packets from clients. `Clients` sending packets to this `Server` specify the same port number in the packets they send. A **`SocketException`** is thrown if the `DatagramSocket` constructor fails to bind the `DatagramSocket` to the specified port.



Common Programming Error 27.2

Specifying a port that's already in use or specifying an invalid port number when creating a `DatagramSocket` results in a `SocketException`.

```
1  // Fig. 27.9: Server.java
2  // Server side of connectionless client/server computing with datagrams.
3  import java.io.IOException;
4  import java.net.DatagramPacket;
5  import java.net.DatagramSocket;
6  import java.net.SocketException;
7  import java.awt.BorderLayout;
8  import javax.swing.JFrame;
9  import javax.swing.JScrollPane;
10 import javax.swing.JTextArea;
```

Fig. 27.9 | Server side of connectionless client/server computing with datagrams. (Part 1 of 3.)

```
11  import javax.swing.SwingUtilities;
12
13  public class Server extends JFrame
14  {
15     private JTextArea displayArea; // displays packets received
16     private DatagramSocket socket; // socket to connect to client
17
18     // set up GUI and DatagramSocket
19     public Server()
20     {
21        super( "Server" );
22
23        displayArea = new JTextArea(); // create displayArea
24        add( new JScrollPane( displayArea ), BorderLayout.CENTER );
25        setSize( 400, 300 ); // set size of window
26        setVisible( true ); // show window
27
28        try // create DatagramSocket for sending and receiving packets
29        {
30           socket = new DatagramSocket( 5000 );
31        } // end try
32        catch ( SocketException socketException )
33        {
34           socketException.printStackTrace();
35           System.exit( 1 );
36        } // end catch
37     } // end Server constructor
38
39     // wait for packets to arrive, display data and echo packet to client
40     public void waitForPackets()
41     {
42        while ( true )
43        {
44           try // receive packet, display contents, return copy to client
45           {
46              byte[] data = new byte[ 100 ]; // set up packet
47              DatagramPacket receivePacket =
48                 new DatagramPacket( data, data.length );
49
50              socket.receive( receivePacket ); // wait to receive packet
51
52              // display information from received packet
53              displayMessage( "\nPacket received:" +
54                 "\nFrom host: " + receivePacket.getAddress() +
55                 "\nHost port: " + receivePacket.getPort() +
56                 "\nLength: " + receivePacket.getLength() +
57                 "\nContaining:\n\t" + new String( receivePacket.getData(),
58                    0, receivePacket.getLength() ) );
59
60              sendPacketToClient( receivePacket ); // send packet to client
61           } // end try
62           catch ( IOException ioException )
63           {
```

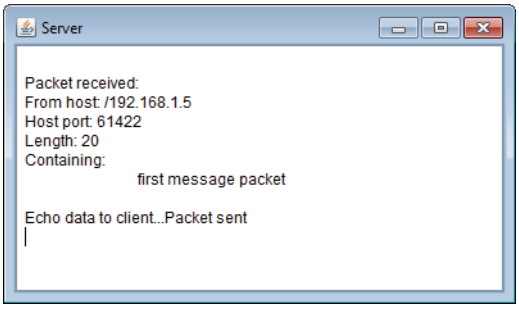
Fig. 27.9 | Server side of connectionless client/server computing with datagrams. (Part 2 of 3.)

```
64              displayMessage( ioException + "\n" );
65              ioException.printStackTrace();
66           } // end catch
67        } // end while
68     } // end method waitForPackets
69
70     // echo packet to client
71     private void sendPacketToClient( DatagramPacket receivePacket )
72        throws IOException
73     {
74        displayMessage( "\n\nEcho data to client..." );
75
76        // create packet to send
77        DatagramPacket sendPacket = new DatagramPacket(
78           receivePacket.getData(), receivePacket.getLength(),
79           receivePacket.getAddress(), receivePacket.getPort() );
80
81        socket.send( sendPacket ); // send packet to client
82        displayMessage( "Packet sent\n" );
83     } // end method sendPacketToClient
84
85     // manipulates displayArea in the event-dispatch thread
86     private void displayMessage( final String messageToDisplay )
87     {
88        SwingUtilities.invokeLater(
89           new Runnable()
90           {
91              public void run() // updates displayArea
92              {
93                 displayArea.append( messageToDisplay ); // display message
94              } // end method run
95           } // end anonymous inner class
96        ); // end call to SwingUtilities.invokeLater
97     } // end method displayMessage
98  } // end class Server
```

Fig. 27.9 | Server side of connectionless client/server computing with datagrams. (Part 3 of 3.)

```
1  // Fig. 27.10: ServerTest.java
2  // Class that tests the Server.
3  import javax.swing.JFrame;
4
5  public class ServerTest
6  {
7     public static void main( String[] args )
8     {
9        Server application = new Server(); // create server
10       application.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
11       application.waitForPackets(); // run server application
12    } // end main
13 } // end class ServerTest
```

Fig. 27.10 | Class that tests the `Server`. (Part 1 of 2.)



`Server` window after packet of data is received from `Client`

Fig. 27.10 | Class that tests the `Server`. (Part 2 of 2.)

Method `waitForPackets`

`Server` method `waitForPackets` (Fig. 27.9, lines 40–68) uses an infinite loop to wait for packets to arrive at the `Server`. Lines 47–48 create a `DatagramPacket` in which a received packet of information can be stored. The `DatagramPacket` constructor for this purpose receives two arguments—a `byte` array in which the data will be stored and the length of the array. Line 50 uses `DatagramSocket` method **`receive`** to wait for a packet to arrive at the `Server`. Method `receive` blocks until a packet arrives, then stores the packet in its `DatagramPacket` argument. The method throws an `IOException` if an error occurs while receiving a packet.

Method `displayMessage`

When a packet arrives, lines 53–58 call method `displayMessage` (declared at lines 86–97) to append the packet's contents to the text area. `DatagramPacket` method **`getAddress`** (line 54) returns an `InetAddress` object containing the IP address of the computer from which the packet was sent. Method **`getPort`** (line 55) returns an integer specifying the port number through which the client computer sent the packet. Method **`getLength`** (line 56) returns an integer representing the number of bytes of data received. Method **`getData`** (line 57) returns a `byte` array containing the data. Lines 57–58 initialize a `String` object using a three-argument constructor that takes a `byte` array, the offset and the length. This `String` is then appended to the text to display.

Method `sendPacketToClient`

After displaying a packet, line 60 calls method `sendPacketToClient` (declared at lines 71–83) to create a new packet and send it to the client. Lines 77–79 create a `DatagramPacket` and pass four arguments to its constructor. The first argument specifies the `byte` array to send. The second argument specifies the number of bytes to send. The third argument specifies the client computer's IP address, to which the packet will be sent. The fourth argument specifies the port where the client is waiting to receive packets. Line 81 sends the packet over the network. Method **`send`** of `DatagramSocket` throws an `IOException` if an error occurs while sending a packet.

`Client` *Class*

The `Client` (Figs. 27.11–27.12) works similarly to class `Server`, except that the `Client` sends packets only when the user types a message in a text field and presses the *Enter* key.

When this occurs, the program calls method `actionPerformed` (Fig. 27.11, lines 32–57), which converts the `String` the user entered into a `byte` array (line 41). Lines 44–45 create a `DatagramPacket` and initialize it with the `byte` array, the length of the `String` that was entered by the user, the IP address to which the packet is to be sent (`InetAddress.getLocalHost()` in this example) and the port number at which the `Server` is waiting for packets (5000 in this example). Line 47 sends the packet. The client in this example must know that the server is receiving packets at port 5000—otherwise, the server will *not* receive the packets.

The `DatagramSocket` constructor call (Fig. 27.11, line 71) in this application does not specify any arguments. This no-argument constructor allows the computer to select the next available port number for the `DatagramSocket`. The client does not need a specific port number, because the server receives the client's port number as part of each `DatagramPacket` sent by the client. Thus, the server can send packets back to the same computer and port number from which it receives a packet of information.

```
1  // Fig. 27.11: Client.java
2  // Client side of connectionless client/server computing with datagrams.
3  import java.io.IOException;
4  import java.net.DatagramPacket;
5  import java.net.DatagramSocket;
6  import java.net.InetAddress;
7  import java.net.SocketException;
8  import java.awt.BorderLayout;
9  import java.awt.event.ActionEvent;
10 import java.awt.event.ActionListener;
11 import javax.swing.JFrame;
12 import javax.swing.JScrollPane;
13 import javax.swing.JTextArea;
14 import javax.swing.JTextField;
15 import javax.swing.SwingUtilities;
16
17 public class Client extends JFrame
18 {
19    private JTextField enterField; // for entering messages
20    private JTextArea displayArea; // for displaying messages
21    private DatagramSocket socket; // socket to connect to server
22
23    // set up GUI and DatagramSocket
24    public Client()
25    {
26       super( "Client" );
27
28       enterField = new JTextField( "Type message here" );
29       enterField.addActionListener(
30          new ActionListener()
31          {
32             public void actionPerformed( ActionEvent event )
33             {
34                try // create and send packet
35                {
```

Fig. 27.11 | Client side of connectionless client/server computing with datagrams. (Part 1 of 3.)

```
36                   // get message from textfield
37                   String message = event.getActionCommand();
38                   displayArea.append( "\nSending packet containing: " +
39                      message + "\n" );
40
41                   byte[] data = message.getBytes(); // convert to bytes
42
43                   // create sendPacket
44                   DatagramPacket sendPacket = new DatagramPacket( data,
45                      data.length, InetAddress.getLocalHost(), 5000 );
46
47                   socket.send( sendPacket ); // send packet
48                   displayArea.append( "Packet sent\n" );
49                   displayArea.setCaretPosition(
50                      displayArea.getText().length() );
51                } // end try
52                catch ( IOException ioException )
53                {
54                   displayMessage( ioException + "\n" );
55                   ioException.printStackTrace();
56                } // end catch
57             } // end actionPerformed
58          } // end inner class
59       ); // end call to addActionListener
60
61       add( enterField, BorderLayout.NORTH );
62
63       displayArea = new JTextArea();
64       add( new JScrollPane( displayArea ), BorderLayout.CENTER );
65
66       setSize( 400, 300 ); // set window size
67       setVisible( true ); // show window
68
69       try // create DatagramSocket for sending and receiving packets
70       {
71          socket = new DatagramSocket();
72       } // end try
73       catch ( SocketException socketException )
74       {
75          socketException.printStackTrace();
76          System.exit( 1 );
77       } // end catch
78    } // end Client constructor
79
80    // wait for packets to arrive from Server, display packet contents
81    public void waitForPackets()
82    {
83       while ( true )
84       {
85          try // receive packet and display contents
86          {
87             byte[] data = new byte[ 100 ]; // set up packet
```

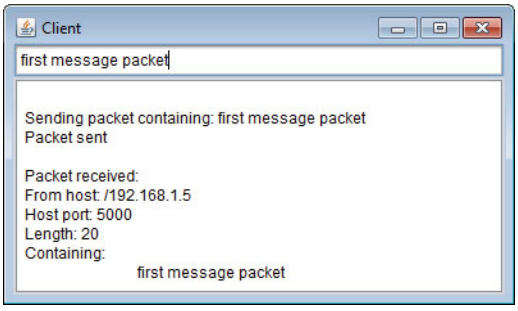
Fig. 27.11 | Client side of connectionless client/server computing with datagrams. (Part 2 of 3.)

```
88              DatagramPacket receivePacket = new DatagramPacket(
89                 data, data.length );
90
91              socket.receive( receivePacket ); // wait for packet
92
93              // display packet contents
94              displayMessage( "\nPacket received:" +
95                 "\nFrom host: " + receivePacket.getAddress() +
96                 "\nHost port: " + receivePacket.getPort() +
97                 "\nLength: " + receivePacket.getLength() +
98                 "\nContaining:\n\t" + new String( receivePacket.getData(),
99                    0, receivePacket.getLength() ) );
100          } // end try
101          catch ( IOException exception )
102          {
103             displayMessage( exception + "\n" );
104             exception.printStackTrace();
105          } // end catch
106       } // end while
107    } // end method waitForPackets
108
109    // manipulates displayArea in the event-dispatch thread
110    private void displayMessage( final String messageToDisplay )
111    {
112       SwingUtilities.invokeLater(
113          new Runnable()
114          {
115             public void run() // updates displayArea
116             {
117                displayArea.append( messageToDisplay );
118             } // end method run
119          }  // end inner class
120       ); // end call to SwingUtilities.invokeLater
121    } // end method displayMessage
122 }  // end class Client
```

Fig. 27.11 | Client side of connectionless client/server computing with datagrams. (Part 3 of 3.)

```
1  // Fig. 27.12: ClientTest.java
2  // Tests the Client class.
3  import javax.swing.JFrame;
4
5  public class ClientTest
6  {
7     public static void main( String[] args )
8     {
9        Client application = new Client(); // create client
10       application.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
11       application.waitForPackets(); // run client application
12    } // end main
13 }  // end class ClientTest
```

Fig. 27.12 | Class that tests the `Client`. (Part 1 of 2.)



`Client` window after sending packet to `Server` and receiving packet back from `Server`

Fig. 27.12 | Class that tests the `Client`. (Part 2 of 2.)

Method `waitForPackets`

`Client` method `waitForPackets` (lines 81–107) uses an infinite loop to wait for packets from the server. Line 91 blocks until a packet arrives. This does not prevent the user from sending a packet, because the *GUI events are handled in the event-dispatch thread*. It only prevents the `while` loop from continuing until a packet arrives at the `Client`. When a packet arrives, line 91 stores it in `receivePacket`, and lines 94–99 call method `displayMessage` (declared at lines 110–121) to display the packet's contents in the text area.

27.8 Client/Server Tic-Tac-Toe Using a Multithreaded Server

This section presents the popular game Tic-Tac-Toe implemented by using client/server techniques with stream sockets. The program consists of a `TicTacToeServer` application (Figs. 27.13–27.14) that allows two `TicTacToeClient` applications (Figs. 27.15–27.16) to connect to the server and play Tic-Tac-Toe. Sample outputs are shown in Fig. 27.17.

`TicTacToeServer` *Class*

As the `TicTacToeServer` receives each client connection, it creates an instance of inner-class `Player` (Fig. 27.13, lines 182–304) to process the client in a *separate thread*. These threads enable the clients to play the game independently. The first client to connect to the server is player X and the second is player O. Player X makes the first move. The server maintains the information about the board so it can determine if a player's move is valid.

```
1  // Fig. 27.13: TicTacToeServer.java
2  // Server side of client/server Tic-Tac-Toe program.
3  import java.awt.BorderLayout;
4  import java.net.ServerSocket;
5  import java.net.Socket;
6  import java.io.IOException;
7  import java.util.Formatter;
8  import java.util.Scanner;
9  import java.util.concurrent.ExecutorService;
10 import java.util.concurrent.Executors;
```

Fig. 27.13 | Server side of client/server Tic-Tac-Toe program. (Part 1 of 7.)

```
11 import java.util.concurrent.locks.Lock;
12 import java.util.concurrent.locks.ReentrantLock;
13 import java.util.concurrent.locks.Condition;
14 import javax.swing.JFrame;
15 import javax.swing.JTextArea;
16 import javax.swing.SwingUtilities;
17
18 public class TicTacToeServer extends JFrame
19 {
20    private String[] board = new String[ 9 ]; // tic-tac-toe board
21    private JTextArea outputArea; // for outputting moves
22    private Player[] players; // array of Players
23    private ServerSocket server; // server socket to connect with clients
24    private int currentPlayer; // keeps track of player with current move
25    private final static int PLAYER_X = 0; // constant for first player
26    private final static int PLAYER_O = 1; // constant for second player
27    private final static String[] MARKS = { "X", "O" }; // array of marks
28    private ExecutorService runGame; // will run players
29    private Lock gameLock; // to lock game for synchronization
30    private Condition otherPlayerConnected; // to wait for other player
31    private Condition otherPlayerTurn; // to wait for other player's turn
32
33    // set up tic-tac-toe server and GUI that displays messages
34    public TicTacToeServer()
35    {
36       super( "Tic-Tac-Toe Server" ); // set title of window
37
38       // create ExecutorService with a thread for each player
39       runGame = Executors.newFixedThreadPool( 2 );
40       gameLock = new ReentrantLock(); // create lock for game
41
42       // condition variable for both players being connected
43       otherPlayerConnected = gameLock.newCondition();
44
45       // condition variable for the other player's turn
46       otherPlayerTurn = gameLock.newCondition();
47
48       for ( int i = 0; i < 9; i++ )
49          board[ i ] = new String( "" ); // create tic-tac-toe board
50       players = new Player[ 2 ]; // create array of players
51       currentPlayer = PLAYER_X; // set current player to first player
52
53       try
54       {
55          server = new ServerSocket( 12345, 2 ); // set up ServerSocket
56       } // end try
57       catch ( IOException ioException )
58       {
59          ioException.printStackTrace();
60          System.exit( 1 );
61       } // end catch
62
63       outputArea = new JTextArea(); // create JTextArea for output
```

Fig. 27.13 | Server side of client/server Tic-Tac-Toe program. (Part 2 of 7.)

```
64          add( outputArea, BorderLayout.CENTER );
65          outputArea.setText( "Server awaiting connections\n" );
66
67          setSize( 300, 300 ); // set size of window
68          setVisible( true ); // show window
69       } // end TicTacToeServer constructor
70
71       // wait for two connections so game can be played
72       public void execute()
73       {
74          // wait for each client to connect
75          for ( int i = 0; i < players.length; i++ )
76          {
77             try // wait for connection, create Player, start runnable
78             {
79                players[ i ] = new Player( server.accept(), i );
80                runGame.execute( players[ i ] ); // execute player runnable
81             } // end try
82             catch ( IOException ioException )
83             {
84                ioException.printStackTrace();
85                System.exit( 1 );
86             } // end catch
87          } // end for
88
89          gameLock.lock(); // lock game to signal player X's thread
90
91          try
92          {
93             players[ PLAYER_X ].setSuspended( false ); // resume player X
94             otherPlayerConnected.signal(); // wake up player X's thread
95          } // end try
96          finally
97          {
98             gameLock.unlock(); // unlock game after signalling player X
99          } // end finally
100      } // end method execute
101
102      // display message in outputArea
103      private void displayMessage( final String messageToDisplay )
104      {
105         // display message from event-dispatch thread of execution
106         SwingUtilities.invokeLater(
107            new Runnable()
108            {
109               public void run() // updates outputArea
110               {
111                  outputArea.append( messageToDisplay ); // add message
112               } // end  method run
113            } // end inner class
114         ); // end call to SwingUtilities.invokeLater
115      } // end method displayMessage
```

Fig. 27.13 | Server side of client/server Tic-Tac-Toe program. (Part 3 of 7.)

```
116
117    // determine if move is valid
118    public boolean validateAndMove( int location, int player )
119    {
120       // while not current player, must wait for turn
121       while ( player != currentPlayer )
122       {
123          gameLock.lock(); // lock game to wait for other player to go
124
125          try
126          {
127             otherPlayerTurn.await(); // wait for player's turn
128          } // end try
129          catch ( InterruptedException exception )
130          {
131             exception.printStackTrace();
132          } // end catch
133          finally
134          {
135             gameLock.unlock(); // unlock game after waiting
136          } // end finally
137       } // end while
138
139       // if location not occupied, make move
140       if ( !isOccupied( location ) )
141       {
142          board[ location ] = MARKS[ currentPlayer ]; // set move on board
143          currentPlayer = ( currentPlayer + 1 ) % 2; // change player
144
145          // let new current player know that move occurred
146          players[ currentPlayer ].otherPlayerMoved( location );
147
148          gameLock.lock(); // lock game to signal other player to go
149
150          try
151          {
152             otherPlayerTurn.signal(); // signal other player to continue
153          } // end try
154          finally
155          {
156             gameLock.unlock(); // unlock game after signaling
157          } // end finally
158
159          return true; // notify player that move was valid
160       } // end if
161       else // move was not valid
162          return false; // notify player that move was invalid
163    } // end method validateAndMove
164
165    // determine whether location is occupied
166    public boolean isOccupied( int location )
167    {
```

Fig. 27.13 | Server side of client/server Tic-Tac-Toe program. (Part 4 of 7.)

```
168        if ( board[ location ].equals( MARKS[ PLAYER_X ] ) ||
169           board [ location ].equals( MARKS[ PLAYER_O ] ) )
170           return true; // location is occupied
171        else
172           return false; // location is not occupied
173     } // end method isOccupied
174
175     // place code in this method to determine whether game over
176     public boolean isGameOver()
177     {
178        return false; // this is left as an exercise
179     } // end method isGameOver
180
181     // private inner class Player manages each Player as a runnable
182     private class Player implements Runnable
183     {
184        private Socket connection; // connection to client
185        private Scanner input; // input from client
186        private Formatter output; // output to client
187        private int playerNumber; // tracks which player this is
188        private String mark; // mark for this player
189        private boolean suspended = true; // whether thread is suspended
190
191        // set up Player thread
192        public Player( Socket socket, int number )
193        {
194           playerNumber = number; // store this player's number
195           mark = MARKS[ playerNumber ]; // specify player's mark
196           connection = socket; // store socket for client
197
198           try // obtain streams from Socket
199           {
200              input = new Scanner( connection.getInputStream() );
201              output = new Formatter( connection.getOutputStream() );
202           } // end try
203           catch ( IOException ioException )
204           {
205              ioException.printStackTrace();
206              System.exit( 1 );
207           } // end catch
208        } // end Player constructor
209
210        // send message that other player moved
211        public void otherPlayerMoved( int location )
212        {
213           output.format( "Opponent moved\n" );
214           output.format( "%d\n", location ); // send location of move
215           output.flush(); // flush output
216        } // end method otherPlayerMoved
217
```

Fig. 27.13 | Server side of client/server Tic-Tac-Toe program. (Part 5 of 7.)

```
218         // control thread's execution
219         public void run()
220         {
221            // send client its mark (X or O), process messages from client
222            try
223            {
224               displayMessage( "Player " + mark + " connected\n" );
225               output.format( "%s\n", mark ); // send player's mark
226               output.flush(); // flush output
227
228               // if player X, wait for another player to arrive
229               if ( playerNumber == PLAYER_X )
230               {
231                  output.format( "%s\n%s", "Player X connected",
232                     "Waiting for another player\n" );
233                  output.flush(); // flush output
234
235                  gameLock.lock(); // lock game to  wait for second player
236
237                  try
238                  {
239                     while( suspended )
240                     {
241                        otherPlayerConnected.await(); // wait for player O
242                     } // end while
243                  } // end try
244                  catch ( InterruptedException exception )
245                  {
246                     exception.printStackTrace();
247                  } // end catch
248                  finally
249                  {
250                     gameLock.unlock(); // unlock game after second player
251                  } // end finally
252
253                  // send message that other player connected
254                  output.format( "Other player connected. Your move.\n" );
255                  output.flush(); // flush output
256               } // end if
257               else
258               {
259                  output.format( "Player O connected, please wait\n" );
260                  output.flush(); // flush output
261               } // end else
262
263               // while game not over
264               while ( !isGameOver() )
265               {
266                  int location = 0; // initialize move location
267
268                  if ( input.hasNext() )
269                     location = input.nextInt(); // get move location
270
```

Fig. 27.13 | Server side of client/server Tic-Tac-Toe program. (Part 6 of 7.)

```
271                // check for valid move
272                if ( validateAndMove( location, playerNumber ) )
273                {
274                   displayMessage( "\nlocation: " + location );
275                   output.format( "Valid move.\n" ); // notify client
276                   output.flush(); // flush output
277                } // end if
278                else // move was invalid
279                {
280                   output.format( "Invalid move, try again\n" );
281                   output.flush(); // flush output
282                } // end else
283             } // end while
284          } // end try
285          finally
286          {
287             try
288             {
289                connection.close(); // close connection to client
290             } // end try
291             catch ( IOException ioException )
292             {
293                ioException.printStackTrace();
294                System.exit( 1 );
295             } // end catch
296          } // end finally
297       } // end method run
298
299       // set whether or not thread is suspended
300       public void setSuspended( boolean status )
301       {
302          suspended = status; // set value of suspended
303       } // end method setSuspended
304    } // end class Player
305 } // end class TicTacToeServer
```

Fig. 27.13 | Server side of client/server Tic-Tac-Toe program. (Part 7 of 7.)

```
1  // Fig. 27.14: TicTacToeServerTest.java
2  // Class that tests Tic-Tac-Toe server.
3  import javax.swing.JFrame;
4
5  public class TicTacToeServerTest
6  {
7     public static void main( String[] args )
8     {
9        TicTacToeServer application = new TicTacToeServer();
10       application.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
11       application.execute();
12    } // end main
13 } // end class TicTacToeServerTest
```

Fig. 27.14 | Class that tests Tic-Tac-Toe server. (Part 1 of 2.)

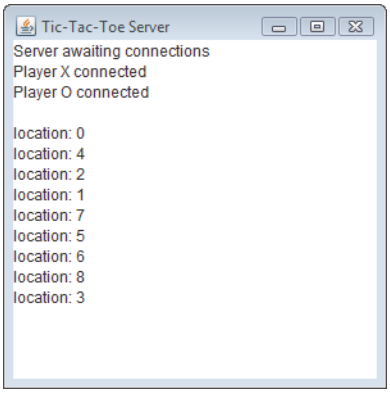


Fig. 27.14 | Class that tests Tic-Tac-Toe server. (Part 2 of 2.)

We begin with a discussion of the server side of the Tic-Tac-Toe game. When the `TicTacToeServer` application executes, the `main` method (lines 7–12 of Fig. 27.14) creates a `TicTacToeServer` object called `application`. The constructor (Fig. 27.13, lines 34–69) attempts to set up a `ServerSocket`. If successful, the program displays the server window, then `main` invokes the `TicTacToeServer` method `execute` (lines 72–100). Method `execute` loops twice, blocking at line 79 each time while waiting for a client connection. When a client connects, line 79 creates a new `Player` object to manage the connection as a separate thread, and line 80 executes the `Player` in the `runGame` thread pool.

When the `TicTacToeServer` creates a `Player`, the `Player` constructor (lines 192–208) receives the `Socket` object representing the connection to the client and gets the associated input and output streams. Line 201 creates a `Formatter` (see Chapter 17) by wrapping it around the output stream of the socket. The `Player`'s `run` method (lines 219–297) controls the information that's sent to and received from the client. First, it passes to the client the character that the client will place on the board when a move is made (line 225). Line 226 calls `Formatter` method **`flush`** to force this output to the client. Line 241 suspends player X's thread as it starts executing, because player X can move only after player O connects.

When player O connects, the game can be played, and the `run` method begins executing its `while` statement (lines 264–283). Each iteration of this loop reads an integer (line 269) representing the location where the client wants to place a mark (blocking to wait for input, if necessary), and line 272 invokes the `TicTacToeServer` method `validateAndMove` (declared at lines 118–163) to check the move. If the move is valid, line 275 sends a message to the client to this effect. If not, line 280 sends a message indicating that the move was invalid. The program maintains board locations as numbers from 0 to 8 (0 through 2 for the first row, 3 through 5 for the second row and 6 through 8 for the third row).

Method `validateAndMove` (lines 118–163 in class `TicTacToeServer`) allows only one player at a time to move, thereby preventing them from modifying the state information of the game simultaneously. If the `Player` attempting to validate a move is *not* the current player (i.e., the one allowed to make a move), it's placed in a *wait* state until its turn to move. If the position for the move being validated is already occupied on the board,

`validMove` returns `false`. Otherwise, the server places a mark for the player in its local representation of the board (line 142), notifies the other `Player` object (line 146) that a move has been made (so that the client can be sent a message), invokes method `signal` (line 152) so that the waiting `Player` (if there is one) can validate a move and returns `true` (line 159) to indicate that the move is valid.

`TicTacToeClient` Class

Each `TicTacToeClient` application (Figs. 27.15–27.16; sample outputs in Fig. 27.17) maintains its own GUI version of the Tic-Tac-Toe board on which it displays the state of the game. The clients can place a mark only in an empty square. Inner class `Square` (Fig. 27.15, lines 205–261) implements each of the nine squares on the board. When a `TicTacToeClient` begins execution, it creates a `JTextArea` in which messages from the server and a representation of the board using nine `Square` objects are displayed. The `startClient` method (lines 80–100) opens a connection to the server and gets the associated input and output streams from the `Socket` object. Lines 85–86 make a connection to the server. Class `TicTacToeClient` implements interface `Runnable` so that a separate thread can read messages from the server. This approach enables the user to interact with the board (in the event-dispatch thread) while waiting for messages from the server. After establishing the connection to the server, line 99 executes the client with the `worker ExecutorService`. The `run` method (lines 103–126) controls the separate thread of execution. The method first reads the mark character (X or O) from the server (line 105), then loops continuously (lines 121–125) and reads messages from the server (line 124). Each message is passed to the `processMessage` method (lines 129–156) for processing.

```
1  // Fig. 27.15: TicTacToeClient.java
2  // Client side of client/server Tic-Tac-Toe program.
3  import java.awt.BorderLayout;
4  import java.awt.Dimension;
5  import java.awt.Graphics;
6  import java.awt.GridLayout;
7  import java.awt.event.MouseAdapter;
8  import java.awt.event.MouseEvent;
9  import java.net.Socket;
10 import java.net.InetAddress;
11 import java.io.IOException;
12 import javax.swing.JFrame;
13 import javax.swing.JPanel;
14 import javax.swing.JScrollPane;
15 import javax.swing.JTextArea;
16 import javax.swing.JTextField;
17 import javax.swing.SwingUtilities;
18 import java.util.Formatter;
19 import java.util.Scanner;
20 import java.util.concurrent.Executors;
21 import java.util.concurrent.ExecutorService;
22
23 public class TicTacToeClient extends JFrame implements Runnable
24 {
```

Fig. 27.15 | Client side of client/server Tic-Tac-Toe program. (Part 1 of 6.)

```
25      private JTextField idField; // textfield to display player's mark
26      private JTextArea displayArea; // JTextArea to display output
27      private JPanel boardPanel; // panel for tic-tac-toe board
28      private JPanel panel2; // panel to hold board
29      private Square[][] board; // tic-tac-toe board
30      private Square currentSquare; // current square
31      private Socket connection; // connection to server
32      private Scanner input; // input from server
33      private Formatter output; // output to server
34      private String ticTacToeHost; // host name for server
35      private String myMark; // this client's mark
36      private boolean myTurn; // determines which client's turn it is
37      private final String X_MARK = "X"; // mark for first client
38      private final String O_MARK = "O"; // mark for second client
39
40      // set up user-interface and board
41      public TicTacToeClient( String host )
42      {
43         ticTacToeHost = host; // set name of server
44         displayArea = new JTextArea( 4, 30 ); // set up JTextArea
45         displayArea.setEditable( false );
46         add( new JScrollPane( displayArea ), BorderLayout.SOUTH );
47
48         boardPanel = new JPanel(); // set up panel for squares in board
49         boardPanel.setLayout( new GridLayout( 3, 3, 0, 0 ) );
50
51         board = new Square[ 3 ][ 3 ]; // create board
52
53         // loop over the rows in the board
54         for ( int row = 0; row < board.length; row++ )
55         {
56            // loop over the columns in the board
57            for ( int column = 0; column < board[ row ].length; column++ )
58            {
59               // create square
60               board[ row ][ column ] = new Square( ' ', row * 3 + column );
61               boardPanel.add( board[ row ][ column ] ); // add square
62            } // end inner for
63         } // end outer for
64
65         idField = new JTextField(); // set up textfield
66         idField.setEditable( false );
67         add( idField, BorderLayout.NORTH );
68
69         panel2 = new JPanel(); // set up panel to contain boardPanel
70         panel2.add( boardPanel, BorderLayout.CENTER ); // add board panel
71         add( panel2, BorderLayout.CENTER ); // add container panel
72
73         setSize( 300, 225 ); // set size of window
74         setVisible( true ); // show window
75
76         startClient();
77      } // end TicTacToeClient constructor
```

Fig. 27.15 | Client side of client/server Tic-Tac-Toe program. (Part 2 of 6.)

```
78
79      // start the client thread
80      public void startClient()
81      {
82         try // connect to server and get streams
83         {
84            // make connection to server
85            connection = new Socket(
86               InetAddress.getByName( ticTacToeHost ), 12345 );
87
88            // get streams for input and output
89            input = new Scanner( connection.getInputStream() );
90            output = new Formatter( connection.getOutputStream() );
91         } // end try
92         catch ( IOException ioException )
93         {
94            ioException.printStackTrace();
95         } // end catch
96
97         // create and start worker thread for this client
98         ExecutorService worker = Executors.newFixedThreadPool( 1 );
99         worker.execute( this ); // execute client
100     } // end method startClient
101
102     // control thread that allows continuous update of displayArea
103     public void run()
104     {
105        myMark = input.nextLine(); // get player's mark (X or O)
106
107        SwingUtilities.invokeLater(
108           new Runnable()
109           {
110              public void run()
111              {
112                 // display player's mark
113                 idField.setText( "You are player \"" + myMark + "\"" );
114              } // end method run
115           } // end anonymous inner class
116        ); // end call to SwingUtilities.invokeLater
117
118        myTurn = ( myMark.equals( X_MARK ) ); // determine if client's turn
119
120        // receive messages sent to client and output them
121        while ( true )
122        {
123           if ( input.hasNextLine() )
124              processMessage( input.nextLine() );
125        } // end while
126     } // end method run
127
128     // process messages received by client
129     private void processMessage( String message )
130     {
```

Fig. 27.15 | Client side of client/server Tic-Tac-Toe program. (Part 3 of 6.)

```
131         // valid move occurred
132         if ( message.equals( "Valid move." ) )
133         {
134            displayMessage( "Valid move, please wait.\n" );
135            setMark( currentSquare, myMark ); // set mark in square
136         } // end if
137         else if ( message.equals( "Invalid move, try again" ) )
138         {
139            displayMessage( message + "\n" ); // display invalid move
140            myTurn = true; // still this client's turn
141         } // end else if
142         else if ( message.equals( "Opponent moved" ) )
143         {
144            int location = input.nextInt(); // get move location
145            input.nextLine(); // skip newline after int location
146            int row = location / 3; // calculate row
147            int column = location % 3; // calculate column
148
149            setMark(  board[ row ][ column ],
150               ( myMark.equals( X_MARK ) ? O_MARK : X_MARK ) ); // mark move
151            displayMessage( "Opponent moved. Your turn.\n" );
152            myTurn = true; // now this client's turn
153         } // end else if
154         else
155            displayMessage( message + "\n" ); // display the message
156      } // end method processMessage
157
158      // manipulate displayArea in event-dispatch thread
159      private void displayMessage( final String messageToDisplay )
160      {
161         SwingUtilities.invokeLater(
162            new Runnable()
163            {
164               public void run()
165               {
166                  displayArea.append( messageToDisplay ); // updates output
167               } // end method run
168            }  // end inner class
169         ); // end call to SwingUtilities.invokeLater
170      } // end method displayMessage
171
172      // utility method to set mark on board in event-dispatch thread
173      private void setMark( final Square squareToMark, final String mark )
174      {
175         SwingUtilities.invokeLater(
176            new Runnable()
177            {
178               public void run()
179               {
180                  squareToMark.setMark( mark ); // set mark in square
181               } // end method run
```

Fig. 27.15 | Client side of client/server Tic-Tac-Toe program. (Part 4 of 6.)

```
182            } // end anonymous inner class
183         ); // end call to SwingUtilities.invokeLater
184      } // end method setMark
185
186      // send message to server indicating clicked square
187      public void sendClickedSquare( int location )
188      {
189         // if it is my turn
190         if ( myTurn )
191         {
192            output.format( "%d\n", location ); // send location to server
193            output.flush();
194            myTurn = false; // not my turn any more
195         } // end if
196      } // end method sendClickedSquare
197
198      // set current Square
199      public void setCurrentSquare( Square square )
200      {
201         currentSquare = square; // set current square to argument
202      } // end method setCurrentSquare
203
204      // private inner class for the squares on the board
205      private class Square extends JPanel
206      {
207         private String mark; // mark to be drawn in this square
208         private int location; // location of square
209
210         public Square( String squareMark, int squareLocation )
211         {
212            mark = squareMark; // set mark for this square
213            location = squareLocation; // set location of this square
214
215            addMouseListener(
216               new MouseAdapter()
217               {
218                  public void mouseReleased( MouseEvent e )
219                  {
220                     setCurrentSquare( Square.this ); // set current square
221
222                     // send location of this square
223                     sendClickedSquare( getSquareLocation() );
224                  } // end method mouseReleased
225               } // end anonymous inner class
226            ); // end call to addMouseListener
227         } // end Square constructor
228
229         // return preferred size of Square
230         public Dimension getPreferredSize()
231         {
232            return new Dimension( 30, 30 ); // return preferred size
233         } // end method getPreferredSize
```

Fig. 27.15 | Client side of client/server Tic-Tac-Toe program. (Part 5 of 6.)

```
234
235        // return minimum size of Square
236        public Dimension getMinimumSize()
237        {
238           return getPreferredSize(); // return preferred size
239        } // end method getMinimumSize
240
241        // set mark for Square
242        public void setMark( String newMark )
243        {
244           mark = newMark; // set mark of square
245           repaint(); // repaint square
246        } // end method setMark
247
248        // return Square location
249        public int getSquareLocation()
250        {
251           return location; // return location of square
252        } // end method getSquareLocation
253
254        // draw Square
255        public void paintComponent( Graphics g )
256        {
257           super.paintComponent( g );
258
259           g.drawRect( 0, 0, 29, 29 ); // draw square
260           g.drawString( mark, 11, 20 ); // draw mark
261        } // end method paintComponent
262     } // end inner-class Square
263  } // end class TicTacToeClient
```

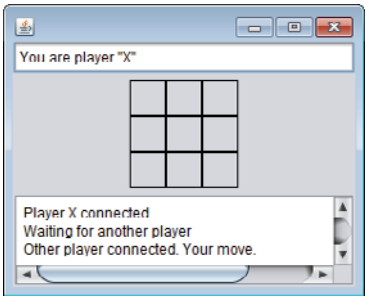
Fig. 27.15 | Client side of client/server Tic-Tac-Toe program. (Part 6 of 6.)

```
1  // Fig. 27.16: TicTacToeClientTest.java
2  // Test class for Tic-Tac-Toe client.
3  import javax.swing.JFrame;
4
5  public class TicTacToeClientTest
6  {
7     public static void main( String[] args )
8     {
9        TicTacToeClient application; // declare client application
10
11       // if no command line args
12       if ( args.length == 0 )
13          application = new TicTacToeClient( "127.0.0.1" ); // localhost
14       else
15          application = new TicTacToeClient( args[ 0 ] ); // use args
16
17       application.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
18    } // end main
19 } // end class TicTacToeClientTest
```

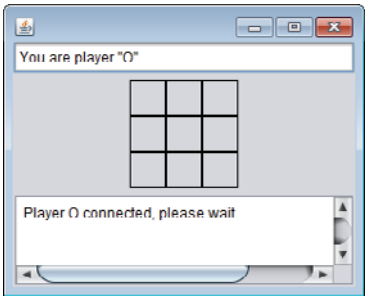
Fig. 27.16 | Test class for Tic-Tac-Toe client.

If the message received is `"Valid move."`, lines 134–135 display the message `"Valid move, please wait."` and call method `setMark` (lines 173–184) to set the client's mark in the current square (the one in which the user clicked), using `SwingUtilities` method `invokeLater` to ensure that the GUI updates occur in the event-dispatch thread. If the message received is `"Invalid move, try again."`, line 139 displays the message so that the user can click a different square. If the message received is `"Opponent moved."`, line 144 reads an integer from the server indicating where the opponent moved, and lines 149–150 place a mark in that square of the board (again using `SwingUtilities` method `invokeLater` to ensure that the GUI updates occur in the event-dispatch thread). If any other message is received, line 155 simply displays the message.

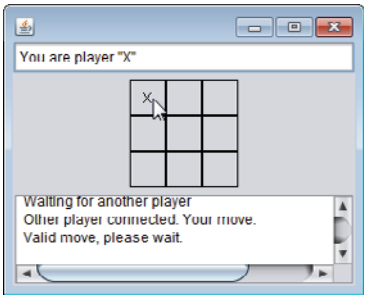
a) Player X connected to server.



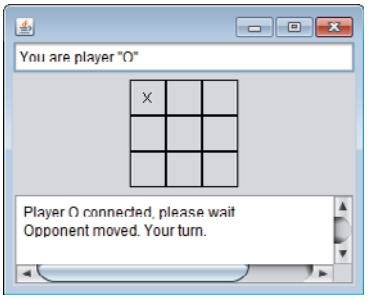
b) Player O connected to server.



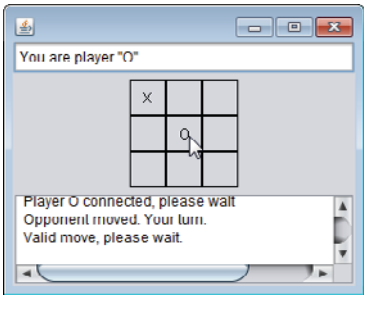
c) Player X moved.



d) Player O sees Player X's move.



e) Player O moved.



f) Player X sees Player O's move.

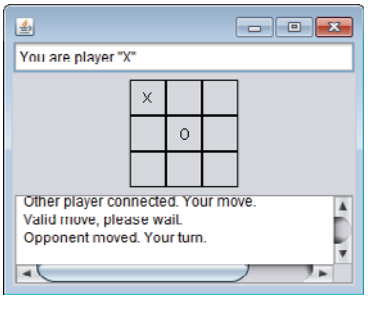
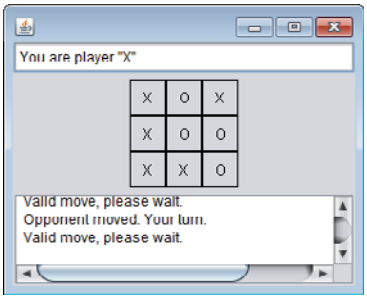


Fig. 27.17 | Sample outputs from the client/server Tic-Tac-Toe program. (Part 1 of 2.)

g) Player X moved.



h) Player O sees Player X's last move.

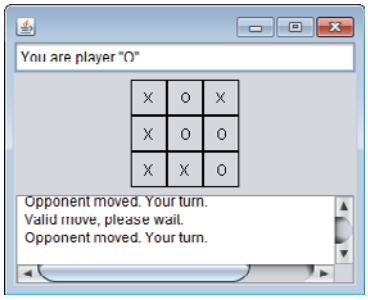


Fig. 27.17 | Sample outputs from the client/server Tic-Tac-Toe program. (Part 2 of 2.)

27.9 [Web Bonus] Case Study: DeitelMessenger

This case study is available at `www.deitel.com/books/jhtp9/`. Chat rooms provide a central location where users can chat with each other via short text messages. Each participant can see all the messages that the other users post, and each user can post messages. This case study integrates many of the Java networking, multithreading and Swing GUI features you've learned thus far to build an online chat system. We also introduce **multicasting**, which enables an application to send `DatagramPackets` to *groups* of clients.

The `DeitelMessenger` case study is a significant application that uses many intermediate Java features, such as networking with `Sockets`, `DatagramPackets` and `MulticastSockets`, multithreading and Swing GUI. The case study also demonstrates good software engineering practices by separating interface from implementation and enabling developers to support different network protocols and provide different user interfaces. After reading this case study, you'll be able to build more significant networking applications.

27.10 Wrap-Up

In this chapter, you learned the basics of network programming in Java. We began with a simple applet and application in which Java performed the networking for you. You then learned two different methods of sending data over a network—streams-based networking using TCP/IP and datagrams-based networking using UDP. We showed how to build simple client/server chat programs using both streams-based and datagram-based networking. You then saw a client/server Tic-Tac-Toe game that enables two clients to play by interacting with a multithreaded server that maintains the game's state and logic. In the next chapter, you'll learn basic database concepts, how to interact with data in a database using SQL and how to use JDBC to allow Java applications to manipulate database data.

Summary

Section 27.1 Introduction

- Java provides stream sockets and datagram sockets (p. 1119). With stream sockets (p. 1119), a process establishes a connection (p. 1119) to another process. While the connection is in place, data flows between the processes in streams. Stream sockets are said to provide a connection-ori-

ented service (p. 1119). The protocol used for transmission is the popular TCP (Transmission Control Protocol; p. 1119).

- With datagram sockets (datagram socket), individual packets of information are transmitted. UDP (User Datagram Protocol; p. 1119) is a connectionless service that does not guarantee that packets will not be lost, duplicated or arrive out of sequence.

Section 27.2 Manipulating URLs

- The HTTP protocol (p. 1120) uses URIs (p. 1120) to locate data on the Internet. Common URIs represent files or directories and can represent complex tasks such as database lookups and Internet searches. A URI that represents a website or web page is called a URL (p. 1120).
- `Applet` method `getAppletContext` (p. 1124) returns an `AppletContext` (p. 1120) that represents the browser in which the applet is executing. `AppletContext` method `showDocument` (p. 1120) receives a `URL` and passes it to the `AppletContext`, which displays the corresponding web resource. A second version of `showDocument` enables an applet to specify the target frame (p. 1124) in which to display a web resource.

Section 27.3 Reading a File on a Web Server

- `JEditorPane` (p. 1125) method `setPage` (p. 1127) downloads the document specified by its argument and displays it.
- Typically, an HTML document contains hyperlinks that link to other documents on the web. If an HTML document is displayed in an uneditable `JEditorPane` and the user clicks a hyperlink (p. 1127), a `HyperlinkEvent` (p. 1125) occurs and the `HyperlinkListeners` are notified.
- `HyperlinkEvent` method `getEventType` (p. 1127) determines the event type. `HyperlinkEvent` contains nested class `EventType` (p. 1127), which declares event types `ACTIVATED`, `ENTERED` and `EXITED`. `HyperlinkEvent` method `getURL` (p. 1127) obtains the `URL` represented by the hyperlink.

Section 27.4 Establishing a Simple Server Using Stream Sockets

- Stream-based connections (p. 1119) are managed with `Socket` objects (p. 1128).
- A `ServerSocket` object (p. 1128) establishes the port (p. 1128) where a server (p. 1119) waits for connections from clients (p. 1119). `ServerSocket` method `accept` (p. 1128) waits indefinitely for a connection from a client and returns a `Socket` object when a connection is established.
- `Socket` methods `getOutputStream` and `getInputStream` (p. 1129) get references to a `Socket`'s `OutputStream` and `InputStream`, respectively. Method `close` (p. 1129) terminates a connection.

Section 27.5 Establishing a Simple Client Using Stream Sockets

- A server name and port number (p. 1128) are specified when creating a `Socket` object to enable it to connect a client to the server. A failed connection attempt throws an `IOException`.
- `InetAddress` method `getByName` (p. 1142) returns an `InetAddress` object (p. 1136) containing the IP address of the specified computer. `InetAddress` method `getLocalHost` (p. 1142) returns an `InetAddress` object containing the IP address of the local computer executing the program.

Section 27.7 Datagrams: Connectionless Client/Server Interaction

- Connection-oriented transmission is like the telephone system—you dial and are given a connection to the telephone of the person with whom you wish to communicate. The connection is maintained for the duration of your phone call, even when you aren't talking.
- Connectionless transmission (p. 1142) with datagrams is similar to mail carried via the postal service. A large message that will not fit in one envelope can be broken into separate message pieces that are placed in separate, sequentially numbered envelopes. All the letters are then mailed at once. They could arrive in order, out of order or not at all.

- `DatagramPacket` objects store packets of data that are to be sent or that are received by an application. `DatagramSockets` send and receive `DatagramPackets`.
- The `DatagramSocket` constructor that takes no arguments binds the `DatagramSocket` to a port chosen by the computer executing the program. The one that takes an integer port-number argument binds the `DatagramSocket` to the specified port. If a `DatagramSocket` constructor fails to bind the `DatagramSocket` to a port, a `SocketException` occurs (p. 1143). `DatagramSocket` method `receive` (p. 1146) blocks (waits) until a packet arrives, then stores the packet in its argument.
- `DatagramPacket` method `getAddress` (p. 1146) returns an `InetAddress` object containing information about the computer from or to which the packet was sent. Method `getPort` (p. 1146) returns an integer specifying the port number (p. 1128) through which the `DatagramPacket` was sent or received. Method `getLength` (`getLength`) returns the number of bytes of data in a `DatagramPacket`. Method `getData` (p. 1146) returns a byte array containing the data.
- The `DatagramPacket` constructor for a packet to be sent takes four arguments—the byte array to be sent, the number of bytes to be sent, the client address to which the packet will be sent and the port number where the client is waiting to receive packets.
- `DatagramSocket` method `send` (p. 1146) sends a `DatagramPacket` out over the network.
- If an error occurs when receiving or sending a `DatagramPacket`, an `IOException` occurs.

Self-Review Exercises

27.1 Fill in the blanks in each of the following statements:

a) Exception _____ occurs when an input/output error occurs when closing a socket.
b) Exception _____ occurs when a hostname indicated by a client cannot be resolved to an address.
c) If a `DatagramSocket` constructor fails to set up a `DatagramSocket` properly, an exception of type _____ occurs.
d) Many of Java's networking classes are contained in package _____.
e) Class _____ binds the application to a port for datagram transmission.
f) An object of class _____ contains an IP address.
g) The two types of sockets we discussed in this chapter are _____ and _____.
h) The acronym URL stands for _____.
i) The acronym URI stands for _____.
j) The key protocol that forms the basis of the World Wide Web is _____.
k) `AppletContext` method _____ receives a `URL` object as an argument and displays in a browser the World Wide Web resource associated with that `URL`.
l) Method `getLocalHost` returns a(n) _____ object containing the local IP address of the computer on which the program is executing.
m) The `URL` constructor determines whether its `String` argument is a valid URL. If so, the `URL` object is initialized with that location. If not, a(n) _____ exception occurs.

27.2 State whether each of the following is *true or false.* If *false*, explain why.

a) UDP is a connection-oriented protocol.
b) With stream sockets a process establishes a connection to another process.
c) A server waits at a port for connections from a client.
d) Datagram packet transmission over a network is reliable—packets are guaranteed to arrive in sequence.

Answers to Self-Review Exercises

27.1 a) `IOException`. b) `UnknownHostException`. c) `SocketException`. d) `java.net`. e) `DatagramSocket`. f) `InetAddress`. g) stream sockets, datagram sockets. h) Uniform Resource Locator.

i) Uniform Resource Identifier. j) HTTP. k) `showDocument`. l) `InetAddress`. m) `MalformedURLException`.

27.2 a) False; UDP is a connectionless protocol and TCP is a connection-oriented protocol. b) True. c) True. d) False; packets can be lost, arrive out of order or be duplicated.

Exercises

27.3 Distinguish between connection-oriented and connectionless network services.

27.4 How does a client determine the hostname of the client computer?

27.5 Under what circumstances would a `SocketException` be thrown?

27.6 How can a client get a line of text from a server?

27.7 Describe how a client connects to a server.

27.8 Describe how a server sends data to a client.

27.9 Describe how to prepare a server to receive a stream-based connection from a single client.

27.10 How does a server listen for streams-based socket connections at a port?

27.11 What determines how many connect requests from clients can wait in a queue to connect to a server?

27.12 As described in the text, what reasons might cause a server to refuse a connection request from a client?

27.13 Use a socket connection to allow a client to specify a file name of a text file and have the server send the contents of the file or indicate that the file does not exist.

27.14 Modify Exercise 27.13 to allow the client to modify the contents of the file and send the file back to the server for storage. The user can edit the file in a `JTextArea`, then click a *save changes* button to send the file back to the server.

27.15 Modify the program in Fig. 27.2 to allow users to add their own sites to the list and remove sites from the list.

27.16 ***(Multithreaded Server)*** Multithreaded servers are quite popular today, especially because of the increasing use of multicore servers. Modify the simple server application presented in Section 27.6 to be a multithreaded server. Then use several client applications and have each of them connect to the server simultaneously. Use an `ArrayList` to store the client threads. `ArrayList` provides several methods to use in this exercise. Method `size` determines the number of elements in an `ArrayList`. Method `get` returns the element in the location specified by its argument. Method `add` places its argument at the end of the `ArrayList`. Method `remove` deletes its argument from the `ArrayList`.

27.17 ***(Checkers Game)*** In the text, we presented a Tic-Tac-Toe program controlled by a multithreaded server. Develop a checkers program modeled after the Tic-Tac-Toe program. The two users should alternate making moves. Your program should mediate the players' moves, determining whose turn it is and allowing only valid moves. The players themselves will determine when the game is over.

27.18 ***(Chess Game)*** Develop a chess-playing program modeled after Exercise 27.17.

27.19 ***(Blackjack Game)*** Develop a blackjack card game program in which the server application deals cards to each of the clients. The server should deal additional cards (per the rules of the game) to each player as requested.

27.20 ***(Poker Game)*** Develop a poker game in which the server application deals cards to each client. The server should deal additional cards (per the rules of the game) to each player as requested.

27.21 ***(Modifications to the Multithreaded Tic-Tac-Toe Program)*** The programs in Figs. 27.13 and 27.15 implemented a multithreaded, client/server version of the game of Tic-Tac-Toe. Our goal in developing this game was to demonstrate a multithreaded server that could process multiple connections from clients at the same time. The server in the example is really a mediator between

the two client applets—it makes sure that each move is valid and that each client moves in the proper order. The server does not determine who won or lost or whether there was a draw. Also, there's no capability to allow a new game to be played or to terminate an existing game.

The following is a list of suggested modifications to Figs. 27.13 and 27.15:

a) Modify the `TicTacToeServer` class to test for a win, loss or draw after each move. Send a message to each client that indicates the result of the game when the game is over.
b) Modify the `TicTacToeClient` class to display a button that when clicked allows the client to play another game. The button should be enabled only when a game completes. Both class `TicTacToeClient` and class `TicTacToeServer` must be modified to reset the board and all state information. Also, the other `TicTacToeClient` should be notified that a new game is about to begin so that its board and state can be reset.
c) Modify the `TicTacToeClient` class to provide a button that allows a client to terminate the program at any time. When the user clicks the button, the server and the other client should be notified. The server should then wait for a connection from another client so that a new game can begin.
d) Modify the `TicTacToeClient` class and the `TicTacToeServer` class so that the winner of a game can choose game piece X or O for the next game. Remember: X always goes first.
e) If you'd like to be ambitious, allow a client to play against the server while the server waits for a connection from another client.

27.22 ***(3-D Multithreaded Tic-Tac-Toe)*** Modify the multithreaded, client/server Tic-Tac-Toe program to implement a three-dimensional 4-by-4-by-4 version of the game. Implement the server application to mediate between the two clients. Display the three-dimensional board as four boards containing four rows and four columns each. If you're ambitious, try the following modifications:

a) Draw the board in a three-dimensional manner.
b) Allow the server to test for a win, loss or draw. Beware! There are many possible ways to win on a 4-by-4-by-4 board!

27.23 ***(Networked Morse Code)*** Perhaps the most famous of all coding schemes is the Morse code, developed by Samuel Morse in 1832 for use with the telegraph system. The Morse code assigns a series of dots and dashes to each letter of the alphabet, each digit, and a few special characters (e.g., period, comma, colon and semicolon). In sound-oriented systems, the dot represents a short sound and the dash a long sound. Other representations of dots and dashes are used with light-oriented systems and signal-flag systems. Separation between words is indicated by a space or, simply, the absence of a dot or dash. In a sound-oriented system, a space is indicated by a short time during which no sound is transmitted. The international version of the Morse code appears in Fig. 27.18.

Character	Code	Character	Code	Character	Code	Character	Code
A	.-	J	.---	S	...	1	.----
B	-...	K	-.-	T	-	2	..---
C	-.-.	L	.-..	U	..-	3	...--
D	-..	M	--	V	...-	4	....-
E	.	N	-.	W	.--	5	.....
F	..-.	O	---	X	-..-	6	-....
G	--.	P	.--.	Y	-.--	7	--...
H	....	Q	--.-	Z	--..	8	---..
I	..	R	.-.			9	----.
						0	-----

Fig. 27.18 | Letters and digits in international Morse code.

Write a client/server application in which two clients can send Morse-code messages to each other through a multithreaded server application. The client application should allow the user to type English-language phrases in a `JTextArea`. When the user sends the message, the client application encodes the text into Morse code and sends the coded message through the server to the other client. Use one blank between each Morse-coded letter and three blanks between each Morse-coded word. When messages are received, they should be decoded and displayed as normal characters and as Morse code. The client should have one `JTextField` for typing and one `JTextArea` for displaying the other client's messages.

Accessing Databases with JDBC

28



It is a capital mistake to theorize before one has data.
—Arthur Conan Doyle

Now go, write it before them in a table, and note it in a book, that it may be for the time to come for ever and ever.
—The Holy Bible, Isaiah 30:8

Get your facts first, and then you can distort them as much as you please.
—Mark Twain

I like two kinds of men: domestic and foreign.
—Mae West

Objectives

In this chapter you'll learn:

- Relational database concepts.
- To use Structured Query Language (SQL) to retrieve data from and manipulate data in a database.
- To use the JDBC™ API to access databases.
- To use the `RowSet` interface from package `javax.sql` to manipulate databases.
- To use JDBC 4's automatic JDBC driver discovery.
- To create precompiled SQL statements with parameters via `PreparedStatement`s.
- How transaction processing makes database applications more robust.

Outline

28.1 Introduction
28.2 Relational Databases
28.3 Relational Database Overview: The `books` Database
28.4 SQL
28.4.1 Basic `SELECT` Query
28.4.2 `WHERE` Clause
28.4.3 `ORDER BY` Clause
28.4.4 Merging Data from Multiple Tables: `INNER JOIN`
28.4.5 `INSERT` Statement
28.4.6 `UPDATE` Statement
28.4.7 `DELETE` Statement
28.5 Instructions for Installing MySQL and MySQL Connector/J
28.6 Instructions for Setting Up a MySQL User Account
28.7 Creating Database `books` in MySQL
28.8 Manipulating Databases with JDBC
28.8.1 Connecting to and Querying a Database
28.8.2 Querying the `books` Database
28.9 `RowSet` Interface
28.10 Java DB/Apache Derby
28.11 `PreparedStatement`s
28.12 Stored Procedures
28.13 Transaction Processing
28.14 Wrap-Up
28.15 Web Resources

Summary | *Self-Review Exercise* | *Answers to Self-Review Exercise* | *Exercises*

28.1 Introduction[1]

A **database** is an organized collection of data. There are many different strategies for organizing data to facilitate easy access and manipulation. A **database management system** (**DBMS**) provides mechanisms for storing, organizing, retrieving and modifying data for many users. Database management systems allow for the access and storage of data without concern for the internal representation of data.

Today's most popular database systems are *relational databases* (Section 28.2). A language called **SQL**—pronounced "sequel," or as its individual letters—is the international standard language used almost universally with relational databases to perform **queries** (i.e., to request information that satisfies given criteria) and to manipulate data. [*Note:* As you learn about SQL, you'll see some authors writing "a SQL statement" (which assumes the pronunciation "sequel") and others writing "an SQL statement" (which assumes that the individual letters are pronounced). In this book we pronounce SQL as "sequel."]

Some popular **relational database management systems** (**RDBMSs**) are Microsoft SQL Server, Oracle, Sybase, IBM DB2, Informix, PostgreSQL and MySQL. The JDK now comes with a pure-Java RDBMS called Java DB—Oracles's version of Apache Derby. In this chapter, we present examples using MySQL and Java DB.

Java programs communicate with databases and manipulate their data using the **Java Database Connectivity (JDBC™) API**. A **JDBC driver** enables Java applications to connect to a database in a particular DBMS and allows you to manipulate that database using the JDBC API.



Software Engineering Observation 28.1

Using the JDBC API enables developers to change the underlying DBMS (for example, from Java DB to MySQL) without modifying the Java code that accesses the database.

1. Before using this chapter, please review the Before You Begin section of the book.

Most popular database management systems now provide JDBC drivers. There are also many third-party JDBC drivers available. In this chapter, we introduce JDBC and use it to manipulate MySQL and Java DB databases. The techniques demonstrated here can also be used to manipulate other databases that have JDBC drivers. Check your DBMS's documentation to determine whether your DBMS comes with a JDBC driver. If not, third-party vendors provide JDBC drivers for many DBMSs.



Software Engineering Observation 28.2

Most major database vendors provide their own JDBC database drivers, and many third-party vendors provide JDBC drivers as well.

For more information on JDBC, visit

```
www.oracle.com/technetwork/java/javase/tech/index-jsp-136101.html
```

which contains JDBC information including the JDBC specification, FAQs, a learning resource center and software downloads.

28.2 Relational Databases

A **relational database** is a logical representation of data that allows the data to be accessed without consideration of its physical structure. A relational database stores data in **tables**. Figure 28.1 illustrates a sample table that might be used in a personnel system. The table name is `Employee`, and its primary purpose is to store the attributes of employees. Tables are composed of **rows**, and rows are composed of **columns** in which values are stored. This table consists of six rows. The `Number` column of each row is the table's **primary key**—a column (or group of columns) with a *unique* value that cannot be duplicated in other rows. This guarantees that each row can be identified by its primary key. Good examples of primary-key columns are a social security number, an employee ID number and a part number in an inventory system, as values in each of these columns are guaranteed to be unique. The rows in Fig. 28.1 are displayed in order by primary key. In this case, the rows are listed in increasing order, but we could also use decreasing order.

Rows in tables are not guaranteed to be stored in any particular order. As we'll demonstrate in an upcoming example, programs can specify ordering criteria when requesting data from a database.

Number	Name	Department	Salary	Location
23603	Jones	413	1100	New Jersey
24568	Kerwin	413	2000	New Jersey
34589	Larson	642	1800	Los Angeles
35761	Myers	611	1400	Orlando
47132	Neumann	413	9000	New Jersey
78321	Stephens	611	8500	Orlando

Row

Primary key

Column

Fig. 28.1 | `Employee` table sample data.

Each column represents a different data attribute. Rows are normally unique (by primary key) within a table, but particular column values may be duplicated between rows. For example, three different rows in the `Employee` table's `Department` column contain number 413.

Different users of a database are often interested in different data and different relationships among the data. Most users require only subsets of the rows and columns. Queries specify which subsets of the data to select from a table. You use SQL to define queries. For example, you might select data from the `Employee` table to create a result that shows where each department is located, presenting the data sorted in increasing order by department number. This result is shown in Fig. 28.2. SQL is discussed in Section 28.4.

```
Department      Location

413             New Jersey
611             Orlando
642             Los Angeles
```

Fig. 28.2 | Result of selecting distinct `Department` and `Location` data from table `Employee`.

28.3 Relational Database Overview: The books Database

We now overview relational databases in the context of a sample `books` database we created for this chapter. Before we discuss SQL, we discuss the *tables* of the `books` database. We use this database to introduce various database concepts, including how to use SQL to obtain information from the database and to manipulate the data. We provide a script to create the database. You can find the script in the examples directory for this chapter. Section 28.7 explains how to use this script. The database consists of three tables: `Authors`, `AuthorISBN` and `Titles`.

Authors Table

The `Authors` table (described in Fig. 28.3) consists of three columns that maintain each author's unique ID number, first name and last name. Figure 28.4 contains sample data from the `Authors` table of the `books` database.

Column	Description
`AuthorID`	Author's ID number in the database. In the `books` database, this integer column is defined as **autoincremented**—for each row inserted in this table, the `AuthorID` value is increased by 1 automatically to ensure that each row has a unique `AuthorID`. This column represents the table's primary key.
`FirstName`	Author's first name (a string).
`LastName`	Author's last name (a string).

Fig. 28.3 | `Authors` table from the `books` database.

AuthorID	FirstName	LastName
1	Paul	Deitel
2	Harvey	Deitel
3	Abbey	Deitel
4	Michael	Morgano
5	Eric	Kern

Fig. 28.4 | Sample data from the `Authors` table.

AuthorISBN Table

The `AuthorISBN` table (described in Fig. 28.5) consists of two columns that maintain each ISBN and the corresponding author's ID number. This table associates authors with their books. Both columns are foreign keys that represent the relationship between the tables `Authors` and `Titles`—one row in table `Authors` may be associated with many rows in table `Titles`, and vice versa. The combined columns of the `AuthorISBN` table represent the table's *primary key*—thus, each row in this table must be a *unique* combination of an `AuthorID` and an `ISBN`. Figure 28.6 contains sample data from the `AuthorISBN` table of the `books` database. [*Note:* To save space, we have split the contents of this table into two columns, each containing the `AuthorID` and `ISBN` columns.] The `AuthorID` column is a **foreign key**—a column in this table that matches the primary-key column in another table (i.e., `AuthorID` in the `Authors` table). Foreign keys are specified when creating a table. The foreign key helps maintain the **Rule of Referential Integrity**—every foreign-key value must appear as another table's primary-key value. This enables the DBMS to determine whether the `AuthorID` value for a particular book is *valid*. Foreign keys also allow related data in multiple tables to be selected from those tables for analytic purposes—this is known as **joining** the data.

Column	Description
`AuthorID`	The author's ID number, a foreign key to the `Authors` table.
`ISBN`	The ISBN for a book, a foreign key to the `Titles` table.

Fig. 28.5 | `AuthorISBN` table from the `books` database.

AuthorID	ISBN	AuthorID	ISBN
1	0132152134	2	0132575663
2	0132152134	1	0132662361
1	0132151421	2	0132662361
2	0132151421	1	0132404168
1	0132575663	2	0132404168

Fig. 28.6 | Sample data from the `AuthorISBN` table of `books`. (Part 1 of 2.)

AuthorID	ISBN	AuthorID	ISBN
1	013705842X	1	0132121360
2	013705842X	2	0132121360
3	013705842X	3	0132121360
4	013705842X	4	0132121360
5	013705842X		

Fig. 28.6 | Sample data from the `AuthorISBN` table of `books`. (Part 2 of 2.)

`Titles` Table

The `Titles` table described in Fig. 28.7 consists of four columns that stand for the ISBN, the title, the edition number and the copyright year. The table is in Fig. 28.8.

Column	Description
`ISBN`	ISBN of the book (a string). The table's primary key. ISBN is an abbreviation for "International Standard Book Number"—a numbering scheme that publishers use to give every book a unique identification number.
`Title`	Title of the book (a string).
`EditionNumber`	Edition number of the book (an integer).
`Copyright`	Copyright year of the book (a string).

Fig. 28.7 | `Titles` table from the `books` database.

ISBN	Title	EditionNumber	Copyright
0132152134	Visual Basic 2010 How to Program	5	2011
0132151421	Visual C# 2010 How to Program	4	2011
0132575663	Java How to Program	9	2012
0132662361	C++ How to Program	8	2012
0132404168	C How to Program	6	2010
013705842X	iPhone for Programmers: An App-Driven Approach	1	2010
0132121360	Android for Programmers: An App-Driven Approach	1	2012

Fig. 28.8 | Sample data from the `Titles` table of the `books` database .

Entity-Relationship (ER) Diagram

There's a one-to-many relationship between a primary key and a corresponding foreign key (e.g., one author can write many books). A foreign key can appear many times in its own table, but only once (as the primary key) in another table. Figure 28.9 is an **entity-**

relationship (ER) diagram for the `books` database. This diagram shows the *database tables* and the *relationships* among them. The first compartment in each box contains the table's name and the remaining compartments contain the table's columns. The names in italic are primary keys. *A table's primary key uniquely identifies each row in the table.* Every row must have a primary-key value, and that value must be unique in the table. This is known as the **Rule of Entity Integrity**. Again, for the `AuthorISBN` table, the primary key is the combination of both columns.



Fig. 28.9 | Table relationships in the `books` database.



Common Programming Error 28.1

Not providing a value for every column in a primary key breaks the Rule of Entity Integrity and causes the DBMS to report an error.



Common Programming Error 28.2

Providing the same primary-key value in multiple rows causes the DBMS to report an error.

The lines connecting the tables (Fig. 28.9) represent the relationships between the tables. Consider the line between the `AuthorISBN` and `Authors` tables. On the `Authors` end of the line is a `1`, and on the `AuthorISBN` end is an infinity symbol (∞), indicating a **one-to-many relationship** in which every author in the `Authors` table can have an arbitrary number of books in the `AuthorISBN` table. The relationship line links the `AuthorID` column in `Authors` (i.e., its primary key) to the `AuthorID` column in `AuthorISBN` (i.e., its foreign key). The `AuthorID` column in the `AuthorISBN` table is a foreign key.



Common Programming Error 28.3

Providing a foreign-key value that does not appear as a primary-key value in another table breaks the Rule of Referential Integrity and causes the DBMS to report an error.

The line between `Titles` and `AuthorISBN` illustrates another *one-to-many relationship*; a title can be written by any number of authors. In fact, the sole purpose of the `AuthorISBN` table is to provide a *many-to-many relationship* between `Authors` and `Titles`—an author can write many books and a book can have many authors.

28.4 SQL

We now overview SQL in the context of our `books` database. You'll be able to use the SQL discussed here in the examples later in the chapter and in examples in Chapters 30–31.

The next several subsections discuss the SQL keywords listed in Fig. 28.10 in the context of SQL queries and statements. Other SQL keywords are beyond this text's scope. To

learn other keywords, refer to the SQL reference guide supplied by the vendor of the RDBMS you're using.

SQL keyword	Description
`SELECT`	Retrieves data from one or more tables.
`FROM`	Tables involved in the query. Required in every `SELECT`.
`WHERE`	Criteria for selection that determine the rows to be retrieved, deleted or updated. Optional in a SQL query or a SQL statement.
`GROUP BY`	Criteria for grouping rows. Optional in a `SELECT` query.
`ORDER BY`	Criteria for ordering rows. Optional in a `SELECT` query.
`INNER JOIN`	Merge rows from multiple tables.
`INSERT`	Insert rows into a specified table.
`UPDATE`	Update rows in a specified table.
`DELETE`	Delete rows from a specified table.

Fig. 28.10 | SQL query keywords.

28.4.1 Basic SELECT Query

Let us consider several SQL queries that extract information from database `books`. A SQL query "selects" rows and columns from one or more tables in a database. Such selections are performed by queries with the **`SELECT`** keyword. The basic form of a `SELECT` query is

```
SELECT * FROM tableName
```

in which the **asterisk** (`*`) *wildcard character* indicates that all columns from the *tableName* table should be retrieved. For example, to retrieve all the data in the `Authors` table, use

```
SELECT * FROM Authors
```

Most programs do not require all the data in a table. To retrieve only specific columns, replace the `*` with a comma-separated list of column names. For example, to retrieve only the columns `AuthorID` and `LastName` for all rows in the `Authors` table, use the query

```
SELECT AuthorID, LastName FROM Authors
```

This query returns the data listed in Fig. 28.11.

AuthorID	LastName
1	Deitel
2	Deitel
3	Deitel
4	Morgano
5	Kern

Fig. 28.11 | Sample `AuthorID` and `LastName` data from the `Authors` table.



Software Engineering Observation 28.3

In general, you process results by knowing in advance the order of the columns in the result—for example, selecting `AuthorID` *and* `LastName` *from table* `Authors` *ensures that the columns will appear in the result with* `AuthorID` *as the first column and* `LastName` *as the second column. Programs typically process result columns by specifying the column number in the result (starting from number 1 for the first column). Selecting columns by name avoids returning unneeded columns and protects against changes in the actual order of the columns in the table(s) by returning the columns in the exact order specified.*



Common Programming Error 28.4

If you assume that the columns are always returned in the same order from a query that uses the asterisk (*), the program may process the results incorrectly. If the column order in the table(s) changes or if additional columns are added at a later time, the order of the columns in the result will change accordingly.

28.4.2 WHERE Clause

In most cases, it's necessary to locate rows in a database that satisfy certain **selection criteria**. Only rows that satisfy the selection criteria (formally called **predicates**) are selected. SQL uses the optional **`WHERE` clause** in a query to specify the selection criteria for the query. The basic form of a query with selection criteria is

```
SELECT columnName1, columnName2, ... FROM tableName WHERE criteria
```

For example, to select the `Title`, `EditionNumber` and `Copyright` columns from table `Titles` for which the `Copyright` date is greater than `2010`, use the query

```
SELECT Title, EditionNumber, Copyright
   FROM Titles
   WHERE Copyright > '2010'
```

Strings in SQL are delimited by single (') rather than double (") quotes.Figure 28.12 shows the result of the preceding query.

Title	EditionNumber	Copyright
Visual Basic 2010 How to Program	5	2011
Visual C# 2010 How to Program	4	2011
Java How to Program	9	2012
C++ How to Program	8	2012
Android for Programmers: An App-Driven Approach	1	2012

Fig. 28.12 | Sampling of titles with copyrights after 2005 from table `Titles`.

Pattern Matching: Zero or More Characters

The `WHERE` clause criteria can contain the operators <, >, <=, >=, =, <> and `LIKE`. Operator **`LIKE`** is used for **pattern matching** with wildcard characters **percent** (**`%`**) and **underscore** (_). Pattern matching allows SQL to search for strings that match a given pattern.

A pattern that contains a percent character (%) searches for strings that have zero or more characters at the percent character's position in the pattern. For example, the next query locates the rows of all the authors whose last name starts with the letter D:

```
SELECT AuthorID, FirstName, LastName
   FROM Authors
   WHERE LastName LIKE 'D%'
```

This query selects the two rows shown in Fig. 28.13—three of the five authors have a last name starting with the letter D (followed by zero or more characters). The % symbol in the WHERE clause's LIKE pattern indicates that any number of characters can appear after the letter D in the LastName. The pattern string is surrounded by single-quote characters.

AuthorID	FirstName	LastName
1	Paul	Deitel
2	Harvey	Deitel
3	Abbey	Deitel

Fig. 28.13 | Authors whose last name starts with D from the Authors table.



Portability Tip 28.1

See the documentation for your database system to determine whether SQL is case sensitive on your system and to determine the syntax for SQL keywords.



Portability Tip 28.2

Read your database system's documentation carefully to determine whether it supports the LIKE operator as discussed here.

Pattern Matching: Any Character

An underscore (_) in the pattern string indicates a single wildcard character at that position in the pattern. For example, the following query locates the rows of all the authors whose last names start with any character (specified by _), followed by the letter o, followed by any number of additional characters (specified by %):

```
SELECT AuthorID, FirstName, LastName
   FROM Authors
   WHERE LastName LIKE '_o%'
```

The preceding query produces the row shown in Fig. 28.14, because only one author in our database has a last name that contains the letter o as its second letter.

AuthorID	FirstName	LastName
4	Michael	Morgano

Fig. 28.14 | The only author from the Authors table whose last name contains o as the second letter.

28.4.3 ORDER BY Clause

The rows in the result of a query can be sorted into ascending or descending order by using the optional **ORDER BY clause**. The basic form of a query with an ORDER BY clause is

```
SELECT columnName1, columnName2, ... FROM tableName ORDER BY column ASC
SELECT columnName1, columnName2, ... FROM tableName ORDER BY column DESC
```

where ASC specifies ascending order (lowest to highest), DESC specifies descending order (highest to lowest) and *column* specifies the column on which the sort is based. For example, to obtain the list of authors in ascending order by last name (Fig. 28.15), use the query

```
SELECT AuthorID, FirstName, LastName
   FROM Authors
   ORDER BY LastName ASC
```

AuthorID	FirstName	LastName
1	Paul	Deitel
2	Harvey	Deitel
3	Abbey	Deitel
5	Eric	Kern
4	Michael	Morgano

Fig. 28.15 | Sample data from table Authors in ascending order by LastName.

Sorting in Descending Order

The default sorting order is ascending, so ASC is optional. To obtain the same list of authors in descending order by last name (Fig. 28.16), use the query

```
SELECT AuthorID, FirstName, LastName
   FROM Authors
   ORDER BY LastName DESC
```

AuthorID	FirstName	LastName
4	Michael	Morgano
5	Eric	Kern
1	Paul	Deitel
2	Harvey	Deitel
3	Abbey	Deitel

Fig. 28.16 | Sample data from table Authors in descending order by LastName.

Sorting By Multiple Columns

Multiple columns can be used for sorting with an ORDER BY clause of the form

```
ORDER BY column1 sortingOrder, column2 sortingOrder, ...
```

where *sortingOrder* is either `ASC` or `DESC`. The *sortingOrder* does not have to be identical for each column. The query

```
SELECT AuthorID, FirstName, LastName
   FROM Authors
   ORDER BY LastName, FirstName
```

sorts all the rows in ascending order by last name, then by first name. If any rows have the same last-name value, they're returned sorted by first name (Fig. 28.17).

AuthorID	FirstName	LastName
3	Abbey	Deitel
2	Harvey	Deitel
1	Paul	Deitel
5	Eric	Kern
4	Michael	Morgano

Fig. 28.17 | Sample data from `Authors` in ascending order by `LastName` and `FirstName`.

Combining the `WHERE` *and* `ORDER BY` *Clauses*

The `WHERE` and `ORDER BY` clauses can be combined in one query, as in

```
SELECT ISBN, Title, EditionNumber, Copyright
   FROM Titles
   WHERE Title LIKE '%How to Program'
   ORDER BY Title ASC
```

which returns the `ISBN`, `Title`, `EditionNumber` and `Copyright` of each book in the `Titles` table that has a `Title` ending with `"How to Program"` and sorts them in ascending order by `Title`. The query results are shown in Fig. 28.18.

ISBN	Title	Edition-Number	Copy-right
0132404168	C How to Program	6	2010
0132662361	C++ How to Program	8	2012
0132575663	Java How to Program	9	2012
0132152134	Visual Basic 2005 How to Program	5	2011
0132151421	Visual C# 2005 How to Program	4	2011

Fig. 28.18 | Sampling of books from table `Titles` whose titles end with `How to Program` in ascending order by `Title`.

28.4.4 Merging Data from Multiple Tables: `INNER JOIN`

Database designers often split related data into separate tables to ensure that a database does not store data redundantly. For example, in the `books` database, we use an `AuthorISBN` table to store the relationship data between authors and their corresponding titles. If we did

not separate this information into individual tables, we'd need to include author information with each entry in the `Titles` table. This would result in the database's storing *duplicate* author information for authors who wrote multiple books. Often, it's necessary to merge data from multiple tables into a single result. Referred to as joining the tables, this is specified by an **`INNER JOIN`** operator, which merges rows from two tables by matching values in columns that are common to the tables. The basic form of an `INNER JOIN` is:

```
SELECT columnName1, columnName2, ...
FROM table1
INNER JOIN table2
   ON table1.columnName = table2.columnName
```

The **`ON` clause** of the `INNER JOIN` specifies the columns from each table that are compared to determine which rows are merged. For example, the following query produces a list of authors accompanied by the ISBNs for books written by each author:

```
SELECT FirstName, LastName, ISBN
FROM Authors
INNER JOIN AuthorISBN
   ON Authors.AuthorID = AuthorISBN.AuthorID
ORDER BY LastName, FirstName
```

The query merges the `FirstName` and `LastName` columns from table `Authors` with the `ISBN` column from table `AuthorISBN`, sorting the result in ascending order by `LastName` and `FirstName`. Note the use of the syntax *tableName*`.`*columnName* in the `ON` clause. This syntax, called a **qualified name**, specifies the columns from each table that should be compared to join the tables. The "*tableName*`.`" syntax is required if the columns have the same name in both tables. The same syntax can be used in any SQL statement to distinguish columns in different tables that have the same name. In some systems, table names qualified with the database name can be used to perform cross-database queries. As always, the query can contain an `ORDER BY` clause. Figure 28.19 shows the results of the preceding query, ordered by `LastName` and `FirstName`. [*Note:* To save space, we split the result of the query into two columns, each containing the `FirstName`, `LastName` and `ISBN` columns.]

FirstName	LastName	ISBN	FirstName	LastName	ISBN
Abbey	Deitel	013705842X	Paul	Deitel	0132151421
Abbey	Deitel	0132121360	Paul	Deitel	0132575663
Harvey	Deitel	0132152134	Paul	Deitel	0132662361
Harvey	Deitel	0132151421	Paul	Deitel	0132404168
Harvey	Deitel	0132575663	Paul	Deitel	013705842X
Harvey	Deitel	0132662361	Paul	Deitel	0132121360
Harvey	Deitel	0132404168	Eric	Kern	013705842X
Harvey	Deitel	013705842X	Michael	Morgano	013705842X
Harvey	Deitel	0132121360	Michael	Morgano	0132121360
Paul	Deitel	0132152134			

Fig. 28.19 | Sampling of authors and ISBNs for the books they have written in ascending order by `LastName` and `FirstName`.



Software Engineering Observation 28.4

If a SQL statement includes columns with the same name from multiple tables, the statement must precede those column names with their table names and a dot (e.g., `Authors.AuthorID`).



Common Programming Error 28.5

Failure to qualify names for columns that have the same name in two or more tables is an error.

28.4.5 INSERT Statement

The **`INSERT`** statement inserts a row into a table. The basic form of this statement is

```
INSERT INTO tableName ( columnName1, columnName2, ..., columnNameN )
   VALUES ( value1, value2, ..., valueN )
```

where *tableName* is the table in which to insert the row. The *tableName* is followed by a comma-separated list of column names in parentheses (this list is not required if the `INSERT` operation specifies a value for every column of the table in the correct order). The list of column names is followed by the SQL keyword **`VALUES`** and a comma-separated list of values in parentheses. The values specified here must match the columns specified after the table name in both order and type (e.g., if *columnName1* is supposed to be the `FirstName` column, then *value1* should be a string in single quotes representing the first name). Always explicitly list the columns when inserting rows. If the table's column order changes or a new column is added, using only `VALUES` may cause an error. The `INSERT` statement

```
INSERT INTO Authors ( FirstName, LastName )
   VALUES ( 'Sue', 'Red' )
```

inserts a row into the `Authors` table. The statement indicates that values are provided for the `FirstName` and `LastName` columns. The corresponding values are `'Sue'` and `'Smith'`. We do not specify an `AuthorID` in this example because `AuthorID` is an autoincremented column in the `Authors` table. For every row added to this table, the DBMS assigns a unique `AuthorID` value that is the next value in the autoincremented sequence (i.e., 1, 2, 3 and so on). In this case, Sue Red would be assigned `AuthorID` number 6. Figure 28.20 shows the `Authors` table after the `INSERT` operation. [*Note:* Not every database management system supports autoincremented columns. Check the documentation for your DBMS for alternatives to autoincremented columns.]

AuthorID	FirstName	LastName
1	Paul	Deitel
2	Harvey	Deitel
3	Abbey	Deitel
4	Michael	Morgano
5	Eric	Kern
6	Sue	Red

Fig. 28.20 | Sample data from table `Authors` after an `INSERT` operation.



Common Programming Error 28.6

It's normally an error to specify a value for an autoincrement column.



Common Programming Error 28.7

SQL delimits strings with single quotes ('). A string containing a single quote (e.g., O'Malley) must have two single quotes in the position where the single quote appears (e.g., 'O''Malley'). The first acts as an escape character for the second. Not escaping single-quote characters in a string that's part of a SQL statement is a SQL syntax error.

28.4.6 UPDATE Statement

An **UPDATE** statement modifies data in a table. Its basic form is

```
UPDATE tableName
   SET columnName1 = value1, columnName2 = value2, ..., columnNameN = valueN
   WHERE criteria
```

where *tableName* is the table to update. The *tableName* is followed by keyword **SET** and a comma-separated list of column name/value pairs in the format *columnName* = *value*. The optional `WHERE` clause provides criteria that determine which rows to update. Though not required, the `WHERE` clause is typically used, unless a change is to be made to every row. The `UPDATE` statement

```
UPDATE Authors
   SET LastName = 'Black'
   WHERE LastName = 'Red' AND FirstName = 'Sue'
```

updates a row in the `Authors` table. The statement indicates that `LastName` will be assigned the value `Black` for the row in which `LastName` is equal to `Red` and `FirstName` is equal to `Sue`. [*Note:* If there are multiple rows with the first name "Sue" and the last name "Red," this statement will modify all such rows to have the last name "Black."] If we know the `AuthorID` in advance of the `UPDATE` operation (possibly because we searched for it previously), the `WHERE` clause can be simplified as follows:

```
WHERE AuthorID = 6
```

Figure 28.21 shows the `Authors` table after the `UPDATE` operation has taken place.

AuthorID	FirstName	LastName
1	Paul	Deitel
2	Harvey	Deitel
3	Abbey	Deitel
4	Michael	Morgano
5	Eric	Kern
6	Sue	Black

Fig. 28.21 | Sample data from table `Authors` after an `UPDATE` operation.

28.4.7 DELETE Statement

A SQL **DELETE** statement removes rows from a table. Its basic form is

```
DELETE FROM tableName WHERE criteria
```

where *tableName* is the table from which to delete. The optional `WHERE` clause specifies the criteria used to determine which rows to delete. If this clause is omitted, all the table's rows are deleted. The `DELETE` statement

```
DELETE FROM Authors
   WHERE LastName = 'Black' AND FirstName = 'Sue'
```

deletes the row for Sue Black in the `Authors` table. If we know the `AuthorID` in advance of the `DELETE` operation, the `WHERE` clause can be simplified as follows:

```
WHERE AuthorID = 5
```

Figure 28.22 shows the `Authors` table after the `DELETE` operation has taken place.

AuthorID	FirstName	LastName
1	Paul	Deitel
2	Harvey	Deitel
3	Abbey	Deitel
4	Michael	Morgano
5	Eric	Kern

Fig. 28.22 | Sample data from table `Authors` after a `DELETE` operation.

28.5 Instructions for Installing MySQL and MySQL Connector/J

MySQL Community Edition is an open-source database management system that executes on many platforms, including Windows, Linux, and Mac OS X. Complete information about MySQL is available from `www.mysql.com`. The examples in Sections 28.8–28.9 manipulate MySQL databases using MySQL 5.5.8—the latest release at the time of this writing.

Installing MySQL

To install MySQL Community Edition on Windows, Linux or Mac OS X, see the installation overview for your platform at:

- Windows: `dev.mysql.com/doc/refman/5.5/en/windows-installation.html`
- Linux: `dev.mysql.com/doc/refman/5.5/en/linux-installation-rpm.html`
- Mac OS X: `dev.mysql.com/doc/refman/5.5/en/macosx-installation.html`

Carefully follow the instructions for downloading and installing the software on your platform. The downloads are available from:

```
dev.mysql.com/downloads/mysql/
```

For the following steps, we assume that you're installing MySQL on Windows. When you execute the installer, the **MySQL Server 5.5 Setup Wizard** window will appear. Perform the following steps:

1. Click the **Next** button.
2. Read the license agreement, then check the **I accept the terms in the License Agreement** checkbox and click the **Next** button. [*Note:* If you do not accept the license terms, you will not be able to install MySQL.]
3. Click the **Typical** button in the **Choose Setup Type** screen then click **Install**.
4. When the installation completes, click **Next >** twice.
5. In the **Completed the MySQL Server 5.5 Setup Wizard** screen, ensure that the **Launch the MySQL Instance Configuration Wizard** checkbox is checked, then click **Finish** to begin configuring the server.

The **MySQL Instance Configuration Wizard** window appears. To configure the server:

1. Click **Next >**, then select **Standard Configuration** and click **Next >** again.
2. You have the option of installing MySQL as a Windows service, which enables the MySQL server to begin executing automatically each time your system starts. For our examples, this is unnecessary, so you can uncheck **Install as a Windows Service** if you wish. Check **Include Bin Directory in Windows PATH**. This will enable you to use the MySQL commands in the Windows Command Prompt. Click **Next >**, then click **Execute** to perform the server configuration.
3. Click **Finish** to close the wizard.

You've now completed the MySQL installation.

Installing MySQL Connector/J

To use MySQL with JDBC, you also need to install **MySQL Connector/J** (the J stands for Java)—a JDBC driver that allows programs to use JDBC to interact with MySQL. MySQL Connector/J can be downloaded from

```
dev.mysql.com/downloads/connector/j/
```

The documentation for Connector/J is located at

```
dev.mysql.com/doc/refman/5.5/en/connector-j.html
```

At the time of this writing, the current generally available release of MySQL Connector/J is 5.1.14. To install MySQL Connector/J, carefully follow the installation instructions at:

```
dev.mysql.com/doc/refman/5.5/en/connector-j-installing.html
```

We *do not* recommend modifying your system's `CLASSPATH` environment variable, which is discussed in the installation instructions. Instead, we'll show you how use MySQL Connector/J by specifying it as a command-line option when you execute your applications.

28.6 Instructions for Setting Up a MySQL User Account

For the MySQL examples to execute correctly, you need to set up a user account that allows users to create, delete and modify a database. After MySQL is installed, follow the

steps below to set up a user account (these steps assume MySQL is installed in its default installation directory):

1. Open a Command Prompt and start the database server by executing the command `mysqld.exe`. This command has no output—it simply starts the MySQL server. Do not close this window—doing so terminates the server.
1. Next, you'll start the MySQL monitor so you can set up a user account, open another Command Prompt and execute the command

```
mysql -h localhost -u root
```

   The `-h` option indicates the host (i.e., computer) on which the MySQL server is running—in this case your local computer (`localhost`). The `-u` option indicates the user account that will be used to log in to the server—`root` is the default user account that is created during installation to allow you to configure the server. Once you've logged in, you'll see a `mysql>` prompt at which you can type commands to interact with the MySQL server.

1. At the `mysql>` prompt, type

```
USE mysql;
```

   and press *Enter* to select the built-in database named `mysql`, which stores server information, such as user accounts and their privileges for interacting with the server. Each command must end with a semicolon. To confirm the command, MySQL issues the message "`Database changed`."

1. Next, you'll add the `deitel` user account to the `mysql` built-in database. The `mysql` database contains a table called `user` with columns that represent the user's name, password and various privileges. To create the `deitel` user account with the password `deitel`, execute the following commands from the `mysql>` prompt:

```
create user 'deitel'@'localhost' identified by 'deitel';
grant select, insert, update, delete, create, drop, references,
   execute on *.* to 'deitel'@'localhost';
```

   This creates the `deitel` user with the privileges needed to create the databases used in this chapter and manipulate them.

1. Type the command

```
exit;
```

   to terminate the MySQL monitor.

28.7 Creating Database books in MySQL

For each MySQL database we discuss, we provide a SQL script in a `.sql` file that sets up the database and its tables. You can execute these scripts in the MySQL monitor. In this chapter's examples directory, you'll find the script `books.sql` to create the `books` database. For the following steps, we assume that the MySQL server (`mysqld.exe`) is still running. To execute the `books.sql` script:

1. Open a Command Prompt and use the `cd` command to change directories to the location that contains the `books.sql` script.

2. Start the MySQL monitor by typing

```
mysql -h localhost -u deitel -p
```

The `-p` option prompts you for the password for the `deitel` user account. When prompted, enter the password `deitel`.

3. Execute the script by typing

```
source books.sql;
```

This creates a new directory named `books` in the server's `data` directory—located by default on Windows at `C:\ProgramData\MySQL\MySQL Server 5.5\data`. This new directory contains the `books` database.

4. Type the command

```
exit;
```

to terminate the MySQL monitor. You're now ready to proceed to the first JDBC example.

28.8 Manipulating Databases with JDBC

This section presents two examples. The first introduces how to connect to a database and query it. The second demonstrates how to display the result of the query in a `JTable`.

28.8.1 Connecting to and Querying a Database

The example of Fig. 28.23 performs a simple query on the `books` database that retrieves the entire `Authors` table and displays the data. The program illustrates connecting to the database, querying the database and processing the result. The discussion that follows presents the key JDBC aspects of the program. [*Note:* Sections 28.5–28.7 demonstrate how to start the MySQL server, configure a user account and create the `books` database. These steps *must* be performed before executing the program of Fig. 28.23.]

```
1  // Fig. 28.23: DisplayAuthors.java
2  // Displaying the contents of the Authors table.
3  import java.sql.Connection;
4  import java.sql.Statement;
5  import java.sql.DriverManager;
6  import java.sql.ResultSet;
7  import java.sql.ResultSetMetaData;
8  import java.sql.SQLException;
9
10 public class DisplayAuthors
11 {
12    // database URL
13    static final String DATABASE_URL = "jdbc:mysql://localhost/books";
14
15    // launch the application
16    public static void main( String args[] )
17    {
```

Fig. 28.23 | Displaying the contents of the `Authors` table. (Part 1 of 3.)

```
18         Connection connection = null; // manages connection
19         Statement statement = null; // query statement
20         ResultSet resultSet = null; // manages results
21
22         // connect to database books and query database
23         try
24         {
25            // establish connection to database
26            connection = DriverManager.getConnection(
27               DATABASE_URL, "deitel", "deitel" );
28
29            // create Statement for querying database
30            statement = connection.createStatement();
31
32            // query database
33            resultSet = statement.executeQuery(
34               "SELECT AuthorID, FirstName, LastName FROM Authors" );
35
36            // process query results
37            ResultSetMetaData metaData = resultSet.getMetaData();
38            int numberOfColumns = metaData.getColumnCount();
39            System.out.println( "Authors Table of Books Database:\n" );
40
41            for ( int i = 1; i <= numberOfColumns; i++ )
42               System.out.printf( "%-8s\t", metaData.getColumnName( i ) );
43            System.out.println();
44
45            while ( resultSet.next() )
46            {
47               for ( int i = 1; i <= numberOfColumns; i++ )
48                  System.out.printf( "%-8s\t", resultSet.getObject( i ) );
49               System.out.println();
50            } // end while
51         }  // end try
52         catch ( SQLException sqlException )
53         {
54            sqlException.printStackTrace();
55         } // end catch
56         finally // ensure resultSet, statement and connection are closed
57         {
58            try
59            {
60               resultSet.close();
61               statement.close();
62               connection.close();
63            } // end try
64            catch ( Exception exception )
65            {
66               exception.printStackTrace();
67            } // end catch
68         } // end finally
69      } // end main
70   } // end class DisplayAuthors
```

Fig. 28.23 | Displaying the contents of the `Authors` table. (Part 2 of 3.)

```
Authors Table of Books Database:

AuthorID        FirstName       LastName
1               Harvey          Deitel
2               Paul            Deitel
3               Andrew          Goldberg
4               David           Choffnes
```

Fig. 28.23 | Displaying the contents of the `Authors` table. (Part 3 of 3.)

Lines 3–8 import the JDBC interfaces and classes from package `java.sql` used in this program. Line 13 declares a string constant for the database URL. This identifies the name of the database to connect to, as well as information about the protocol used by the JDBC driver (discussed shortly). Method `main` (lines 16–69) connects to the `books` database, queries the database, displays the result of the query and closes the database connection.

In past versions of Java, programs were required to load an appropriate database driver before connecting to a database. JDBC 4.0 and higher support **automatic driver discovery**—you're no longer required to load the database driver in advance. To ensure that the program can locate the database driver class, you must include the class's location in the program's classpath when you execute the program. For MySQL, you include the file `mysql-connector-java-5.1.14-bin.jar` (in the `C:\mysql-connector-java-5.1.14` directory) in your program's classpath, as in:

```
java -classpath .;c:\mysql-connector-java-5.1.14\mysql-connector-
java-5.1.14-bin.jar DisplayAuthors
```

If the period (`.`) at the beginning of the classpath information is missing, the JVM will not look for classes in the current directory and thus will not find the `DisplayAuthors` class file. You may also copy the `mysql-connector-java-5.1.14-bin.jar` file to your JDK's `\jre\lib\ext` folder. After doing so, you can run the application simply using the command

```
java DisplayAuthors
```

Connecting to the Database

Lines 26–27 of Fig. 28.23 create a **`Connection`** object (package `java.sql`) referenced by `connection`. An object that implements interface `Connection` manages the connection between the Java program and the database. `Connection` objects enable programs to create SQL statements that manipulate databases. The program initializes `connection` with the result of a call to `static` method **`getConnection`** of class **`DriverManager`** (package `java.sql`), which attempts to connect to the database specified by its URL. Method `getConnection` takes three arguments—a `String` that specifies the database URL, a `String` that specifies the username and a `String` that specifies the password. The username and password are set in Section 28.6. If you used a different username and password, you need to replace the username (second argument) and password (third argument) passed to method `getConnection` in line 27. The URL locates the database (possibly on a network or in the local file system of the computer). The URL `jdbc:mysql://localhost/books` specifies the protocol for communication (`jdbc`), the **subprotocol** for communication (`mysql`) and the location of the database (`//localhost/books`, where `localhost` is the host running the MySQL server and `books` is the database name). The subprotocol `mysql`

indicates that the program uses a MySQL-specific subprotocol to connect to the MySQL database. If the `DriverManager` cannot connect to the database, method `getConnection` throws a **`SQLException`** (package `java.sql`). Figure 28.24 lists the JDBC driver names and database URL formats of several popular RDBMSs.

RDBMS	Database URL format
MySQL	`jdbc:mysql://`*hostname*`:`*portNumber*`/`*databaseName*
ORACLE	`jdbc:oracle:thin:@`*hostname*`:`*portNumber*`:`*databaseName*
DB2	`jdbc:db2:`*hostname*`:`*portNumber*`/`*databaseName*
PostgreSQL	`jdbc:postgresql://`*hostname*`:`*portNumber*`/`*databaseName*
Java DB/Apache Derby	`jdbc:derby:`*dataBaseName* (embedded) `jdbc:derby://`*hostname*`:`*portNumber*`/`*databaseName* (network)
Microsoft SQL Server	`jdbc:sqlserver://`*hostname*`:`*portNumber*`;databaseName=`*dataBaseName*
Sybase	`jdbc:sybase:Tds:`*hostname*`:`*portNumber*`/`*databaseName*

Fig. 28.24 | Popular JDBC database URL formats.



Software Engineering Observation 28.5

Most database management systems require the user to log in before accessing the database contents. `DriverManager` method `getConnection` is overloaded with versions that enable the program to supply the user name and password to gain access.

Creating a `Statement` for Executing Queries

Line 30 invokes `Connection` method **`createStatement`** to obtain an object that implements interface `Statement` (package `java.sql`). The program uses the **`Statement`** object to submit SQL statements to the database.

Executing a Query

Lines 33–34 use the `Statement` object's **`executeQuery`** method to submit a query that selects all the author information from table `Authors`. This method returns an object that implements interface **`ResultSet`** and contains the query results. The `ResultSet` methods enable the program to manipulate the query result.

Processing a Query's `ResultSet`

Lines 37–50 process the `ResultSet`. Line 37 obtains the metadata for the `ResultSet` as a **`ResultSetMetaData`** (package `java.sql`) object. The **metadata** describes the `ResultSet`'s contents. Programs can use metadata programmatically to obtain information about the `ResultSet`'s column names and types. Line 38 uses `ResultSetMetaData` method **`getColumnCount`** to retrieve the number of columns in the `ResultSet`. Lines 41–42 display the column names.



Software Engineering Observation 28.6

Metadata enables programs to process `ResultSet` contents dynamically when detailed information about the `ResultSet` is not known in advance.

Lines 45–50 display the data in each `ResultSet` row. First, the program positions the `ResultSet` cursor (which points to the row being processed) to the first row in the `ResultSet` with method **`next`** (line 45). Method `next` returns `boolean` value `true` if it's able to position to the next row; otherwise, the method returns `false`.



Common Programming Error 28.8

Initially, a `ResultSet` cursor is positioned before the first row. A `SQLException` occurs if you attempt to access a `ResultSet`'s contents before positioning the `ResultSet` cursor to the first row with method `next`.

If there are rows in the `ResultSet`, lines 47–48 extract and display the contents of each column in the current row. When a `ResultSet` is processed, each column can be extracted as a specific Java type. In fact, `ResultSetMetaData` method **`getColumnType`** returns a constant integer from class **`Types`** (package `java.sql`) indicating the type of a specified column. Programs can use these values in a `switch` statement to invoke `ResultSet` methods that return the column values as appropriate Java types. If the type of a column is `Types.INTEGER`, `ResultSet` method **`getInt`** returns the column value as an `int`. `ResultSet` *get* methods typically receive as an argument either a column number (as an `int`) or a column name (as a `String`) indicating which column's value to obtain. Visit

```
java.sun.com/javase/6/docs/technotes/guides/jdbc/getstart/
   GettingStartedTOC.fm.html
```

for detailed mappings of SQL data types to Java types and to determine the appropriate `ResultSet` method to call for each SQL data type.



Performance Tip 28.1

If a query specifies the exact columns to select from the database, the `ResultSet` contains the columns in the specified order. In this case, using the column number to obtain the column's value is more efficient than using the column name. The column number provides direct access to the specified column. Using the column name requires a search of the column names to locate the appropriate column.



Error-Prevention Tip 28.1

Using column names to obtain values from a `ResultSet` produces code that is less error prone than obtaining values by column number—you don't need to remember the column order. Also, if the column order changes, your code does not have to change.

For simplicity, this example treats each value as an `Object`. We retrieve each column value with `ResultSet` method **`getObject`** (line 48) then print the `Object`'s `String` representation. Unlike array indices, `ResultSet` *column numbers start at 1*. The `finally` block (lines 56–68) closes the `ResultSet`, the `Statement` and the database `Connection`. [*Note:* Lines 60–62 will throw `NullPointerExceptions` if the `ResultSet`, `Statement` or `Connection` objects were not created properly. For code used in industry, you should check the variables that refer to these objects to see if they're `null` before you call `close`.]



Common Programming Error 28.9

Specifying column 0 when obtaining values from a `ResultSet` causes a `SQLException`.



Common Programming Error 28.10

A `SQLException` occurs if you attempt to manipulate a `ResultSet` after closing the `Statement` that created it. The `ResultSet` is discarded when the `Statement` is closed.



Software Engineering Observation 28.7

Each `Statement` object can open only one `ResultSet` object at a time. When a `Statement` returns a new `ResultSet`, the `Statement` closes the prior `ResultSet`. To use multiple `ResultSet`s in parallel, separate `Statement` objects must return the `ResultSet`s.

Java SE 7: Automatically Closing `Connection`s, `Statement`s and `ResultSet`s

As of Java SE 7, the interfaces `Connection`, `Statement` and `ResultSet` each extend the `AutoCloseable` interface, so you can use objects that implement these interfaces with the new `try`-with-resources statement, which was introduced in Section 11.13. In the folder for the example of Fig. 28.23, the subfolder `JavaSE7Version` contains a version of the example that uses the `try`-with-resources statement to allocate the `Connection`, `Statement` and `ResultSet` objects. These objects are automatically closed at the end of the `try` block or if an exception occurs while executing the code in the `try` block.

28.8.2 Querying the books Database

The next example (Fig. 28.25 and Fig. 28.28) allows the user to enter any query into the program. The example displays the result of a query in a **`JTable`**, using a **`TableModel`** object to provide the `ResultSet` data to the `JTable`. A `JTable` is a swing GUI component that can be bound to a database to display the results of a query. Class `ResultSetTableModel` (Fig. 28.25) performs the connection to the database via a `TableModel` and maintains the `ResultSet`. Class `DisplayQueryResults` (Fig. 28.28) creates the GUI and specifies an instance of class `ResultSetTableModel` to provide data for the `JTable`.

`ResultSetTableModel` Class

Class `ResultSetTableModel` (Fig. 28.25) extends class **`AbstractTableModel`** (package `javax.swing.table`), which implements interface `TableModel`. `ResultSetTableModel` overrides `TableModel` methods **`getColumnClass`**, **`getColumnCount`**, **`getColumnName`**, **`getRowCount`** and **`getValueAt`**. The default implementations of `TableModel` methods `isCellEditable` and `setValueAt` (provided by `AbstractTableModel`) are not overridden, because this example does not support editing the `JTable` cells. The default implementations of `TableModel` methods **`addTableModelListener`** and **`removeTableModelListener`** (provided by `AbstractTableModel`) are not overridden, because the implementations of these methods in `AbstractTableModel` properly add and remove event listeners.

```
1  // Fig. 28.25: ResultSetTableModel.java
2  // A TableModel that supplies ResultSet data to a JTable.
3  import java.sql.Connection;
4  import java.sql.Statement;
5  import java.sql.DriverManager;
6  import java.sql.ResultSet;
7  import java.sql.ResultSetMetaData;
```

Fig. 28.25 | A `TableModel` that supplies `ResultSet` data to a `JTable`. (Part 1 of 5.)

```
8  import java.sql.SQLException;
9  import javax.swing.table.AbstractTableModel;
10
11 // ResultSet rows and columns are counted from 1 and JTable
12 // rows and columns are counted from 0. When processing
13 // ResultSet rows or columns for use in a JTable, it is
14 // necessary to add 1 to the row or column number to manipulate
15 // the appropriate ResultSet column (i.e., JTable column 0 is
16 // ResultSet column 1 and JTable row 0 is ResultSet row 1).
17 public class ResultSetTableModel extends AbstractTableModel
18 {
19    private Connection connection;
20    private Statement statement;
21    private ResultSet resultSet;
22    private ResultSetMetaData metaData;
23    private int numberOfRows;
24
25    // keep track of database connection status
26    private boolean connectedToDatabase = false;
27
28    // constructor initializes resultSet and obtains its meta data object;
29    // determines number of rows
30    public ResultSetTableModel( String url, String username,
31       String password, String query ) throws SQLException
32    {
33       // connect to database
34       connection = DriverManager.getConnection( url, username, password );
35
36       // create Statement to query database
37       statement = connection.createStatement(
38          ResultSet.TYPE_SCROLL_INSENSITIVE,
39          ResultSet.CONCUR_READ_ONLY );
40
41       // update database connection status
42       connectedToDatabase = true;
43
44       // set query and execute it
45       setQuery( query );
46    } // end constructor ResultSetTableModel
47
48    // get class that represents column type
49    public Class getColumnClass( int column ) throws IllegalStateException
50    {
51       // ensure database connection is available
52       if ( !connectedToDatabase )
53          throw new IllegalStateException( "Not Connected to Database" );
54
55       // determine Java class of column
56       try
57       {
58          String className = metaData.getColumnClassName( column + 1 );
59
```

Fig. 28.25 | A `TableModel` that supplies `ResultSet` data to a `JTable`. (Part 2 of 5.)

```
60            // return Class object that represents className
61            return Class.forName( className );
62         } // end try
63         catch ( Exception exception )
64         {
65            exception.printStackTrace();
66         } // end catch
67
68         return Object.class; // if problems occur above, assume type Object
69      } // end method getColumnClass
70
71      // get number of columns in ResultSet
72      public int getColumnCount() throws IllegalStateException
73      {
74         // ensure database connection is available
75         if ( !connectedToDatabase )
76            throw new IllegalStateException( "Not Connected to Database" );
77
78         // determine number of columns
79         try
80         {
81            return metaData.getColumnCount();
82         } // end try
83         catch ( SQLException sqlException )
84         {
85            sqlException.printStackTrace();
86         } // end catch
87
88         return 0; // if problems occur above, return 0 for number of columns
89      } // end method getColumnCount
90
91      // get name of a particular column in ResultSet
92      public String getColumnName( int column ) throws IllegalStateException
93      {
94         // ensure database connection is available
95         if ( !connectedToDatabase )
96            throw new IllegalStateException( "Not Connected to Database" );
97
98         // determine column name
99         try
100        {
101           return metaData.getColumnName( column + 1 );
102        } // end try
103        catch ( SQLException sqlException )
104        {
105           sqlException.printStackTrace();
106        } // end catch
107
108        return ""; // if problems, return empty string for column name
109     } // end method getColumnName
110
```

Fig. 28.25 | A `TableModel` that supplies `ResultSet` data to a `JTable`. (Part 3 of 5.)

```
111    // return number of rows in ResultSet
112    public int getRowCount() throws IllegalStateException
113    {
114       // ensure database connection is available
115       if ( !connectedToDatabase )
116          throw new IllegalStateException( "Not Connected to Database" );
117
118       return numberOfRows;
119    } // end method getRowCount
120
121    // obtain value in particular row and column
122    public Object getValueAt( int row, int column )
123       throws IllegalStateException
124    {
125       // ensure database connection is available
126       if ( !connectedToDatabase )
127          throw new IllegalStateException( "Not Connected to Database" );
128
129       // obtain a value at specified ResultSet row and column
130       try
131       {
132          resultSet.absolute( row + 1 );
133          return resultSet.getObject( column + 1 );
134       } // end try
135       catch ( SQLException sqlException )
136       {
137          sqlException.printStackTrace();
138       } // end catch
139
140       return ""; // if problems, return empty string object
141    } // end method getValueAt
142
143    // set new database query string
144    public void setQuery( String query )
145       throws SQLException, IllegalStateException
146    {
147       // ensure database connection is available
148       if ( !connectedToDatabase )
149          throw new IllegalStateException( "Not Connected to Database" );
150
151       // specify query and execute it
152       resultSet = statement.executeQuery( query );
153
154       // obtain meta data for ResultSet
155       metaData = resultSet.getMetaData();
156
157       // determine number of rows in ResultSet
158       resultSet.last(); // move to last row
159       numberOfRows = resultSet.getRow(); // get row number
160
161       // notify JTable that model has changed
162       fireTableStructureChanged();
163    } // end method setQuery
```

Fig. 28.25 | A `TableModel` that supplies `ResultSet` data to a `JTable`. (Part 4 of 5.)

```
164
165    // close Statement and Connection
166    public void disconnectFromDatabase()
167    {
168       if ( connectedToDatabase )
169       {
170          // close Statement and Connection
171          try
172          {
173             resultSet.close();
174             statement.close();
175             connection.close();
176          } // end try
177          catch ( SQLException sqlException )
178          {
179             sqlException.printStackTrace();
180          } // end catch
181          finally  // update database connection status
182          {
183             connectedToDatabase = false;
184          } // end finally
185       } // end if
186    } // end method disconnectFromDatabase
187 } // end class ResultSetTableModel
```

Fig. 28.25 | A `TableModel` that supplies `ResultSet` data to a `JTable`. (Part 5 of 5.)

`ResultSetTableModel` Constructor

The `ResultSetTableModel` constructor (lines 30–46) accepts four `String` arguments—the URL of the database, the username, the password and the default query to perform. The constructor throws any exceptions that occur in its body back to the application that created the `ResultSetTableModel` object, so that the application can determine how to handle the exception (e.g., report an error and terminate the application). Line 34 establishes a connection to the database. Lines 37–39 invoke `Connection` method `createStatement` to create a `Statement` object. This example uses a version of method `createStatement` that takes two arguments—the result set type and the result set concurrency. The **result set type** (Fig. 28.26) specifies whether the `ResultSet`'s cursor is able to scroll in both directions or forward only and whether the `ResultSet` is sensitive to changes made to the underlying data.

`ResultSet` constant	Description
`TYPE_FORWARD_ONLY`	Specifies that a `ResultSet`'s cursor can move only in the forward direction (i.e., from the first to the last row in the `ResultSet`).
`TYPE_SCROLL_INSENSITIVE`	Specifies that a `ResultSet`'s cursor can scroll in either direction and that the changes made to the underlying data during `ResultSet` processing are not reflected in the `ResultSet` unless the program queries the database again.

Fig. 28.26 | `ResultSet` constants for specifying `ResultSet` type. (Part 1 of 2.)

ResultSet constant	Description
TYPE_SCROLL_SENSITIVE	Specifies that a ResultSet's cursor can scroll in either direction and that the changes made to the underlying data during ResultSet processing are reflected immediately in the ResultSet.

Fig. 28.26 | ResultSet constants for specifying ResultSet type. (Part 2 of 2.)



Portability Tip 28.3

Some JDBC drivers do not support scrollable ResultSets. In such cases, the driver typically returns a ResultSet in which the cursor can move only forward. For more information, see your database driver documentation.



Common Programming Error 28.11

Attempting to move the cursor backward through a ResultSet when the database driver does not support backward scrolling causes a SQLFeatureNotSupportedException.

ResultSets that are sensitive to changes reflect those changes immediately after they're made with methods of interface ResultSet. If a ResultSet is insensitive to changes, the query that produced the ResultSet must be executed again to reflect any changes made. The **result set concurrency** (Fig. 28.27) specifies whether the ResultSet can be updated with ResultSet's update methods.

ResultSet static concurrency constant	Description
CONCUR_READ_ONLY	Specifies that a ResultSet cannot be updated (i.e., changes to the ResultSet contents cannot be reflected in the database with ResultSet's update methods).
CONCUR_UPDATABLE	Specifies that a ResultSet can be updated (i.e., changes to its contents can be reflected in the database with ResultSet's update methods).

Fig. 28.27 | ResultSet constants for specifying result properties.



Portability Tip 28.4

Some JDBC drivers do not support updatable ResultSets. In such cases, the driver typically returns a read-only ResultSet. For more information, see your database driver documentation.



Common Programming Error 28.12

Attempting to update a ResultSet when the database driver does not support updatable ResultSets causes SQLFeatureNotSupportedExceptions.

This example uses a ResultSet that is scrollable, insensitive to changes and read only. Line 45 invokes our method setQuery (lines 144–163) to perform the default query.

`ResultSetTableModel` *Method* `getColumnClass`

Method `getColumnClass` (lines 49–69) returns a `Class` object that represents the superclass of all objects in a particular column. The `JTable` uses this information to configure the default cell renderer and cell editor for that column in the `JTable`. Line 58 uses `ResultSetMetaData` method **`getColumnClassName`** to obtain the fully qualified class name for the specified column. Line 61 loads the class and returns the corresponding `Class` object. If an exception occurs, the `catch` in lines 63–66 prints a stack trace and line 68 returns `Object.class`—the `Class` instance that represents class `Object`—as the default type. [*Note:* Line 58 uses the argument `column + 1`. Like arrays, `JTable` row and column numbers are counted from 0. However, `ResultSet` row and column numbers are counted from 1. Thus, when processing `ResultSet` rows or columns for use in a `JTable`, it's necessary to add 1 to the row or column number to manipulate the appropriate `ResultSet` row or column.]

`ResultSetTableModel` *Method* `getColumnCount`

Method `getColumnCount` (lines 72–89) returns the number of columns in the model's underlying `ResultSet`. Line 81 uses `ResultSetMetaData` method **`getColumnCount`** to obtain the number of columns in the `ResultSet`. If an exception occurs, the `catch` in lines 83–86 prints a stack trace and line 88 returns 0 as the default number of columns.

`ResultSetTableModel` *Method* `getColumnName`

Method `getColumnName` (lines 92–109) returns the name of the column in the model's underlying `ResultSet`. Line 101 uses `ResultSetMetaData` method **`getColumnName`** to obtain the column name from the `ResultSet`. If an exception occurs, the `catch` in lines 103–106 prints a stack trace and line 108 returns the empty string as the default column name.

`ResultSetTableModel` *Method* `getRowCount`

Method `getRowCount` (lines 112–119) returns the number of rows in the model's underlying `ResultSet`. When method `setQuery` (lines 144–163) performs a query, it stores the number of rows in variable `numberOfRows`.

`ResultSetTableModel` *Method* `getValueAt`

Method `getValueAt` (lines 122–141) returns the `Object` in a particular row and column of the model's underlying `ResultSet`. Line 132 uses `ResultSet` method **`absolute`** to position the `ResultSet` cursor at a specific row. Line 133 uses `ResultSet` method `getObject` to obtain the `Object` in a specific column of the current row. If an exception occurs, the `catch` in lines 135–138 prints a stack trace and line 140 returns an empty string as the default value.

`ResultSetTableModel` *Method* `setQuery`

Method `setQuery` (lines 144–163) executes the query it receives as an argument to obtain a new `ResultSet` (line 152). Line 155 gets the `ResultSetMetaData` for the new `ResultSet`. Line 158 uses `ResultSet` method **`last`** to position the `ResultSet` cursor at the last row in the `ResultSet`. [*Note:* This can be slow if the table contains many rows.] Line 159 uses `ResultSet` method **`getRow`** to obtain the row number for the current row in the `ResultSet`. Line 162 invokes method **`fireTableStructureChanged`** (inherited from class `AbstractTableModel`) to notify any `JTable` using this `ResultSetTableModel` object as its model that the structure of the model has changed. This causes the `JTable` to repopulate its rows and columns with the new `ResultSet` data. Method `setQuery` throws any exceptions that occur in its body back to the application that invoked `setQuery`.

`ResultSetTableModel` *Method* `disconnectFromDatabase`

Method `disconnectFromDatabase` (lines 166–186) implements an appropriate termination method for class `ResultSetTableModel`. A class designer should provide a `public` method that clients of the class must invoke explicitly to free resources that an object has used. In this case, method `disconnectFromDatabase` closes the `ResultSet`, `Statement` and `Connection` (lines 173–175), which are considered limited resources. Clients of the `ResultSetTableModel` class should always invoke this method when the instance of this class is no longer needed. Before releasing resources, line 168 verifies whether the connection is already terminated. If not, the method proceeds. The other methods in class `ResultSetTableModel` each throw an `IllegalStateException` if `connectedToDatabase` is `false`. Method `disconnectFromDatabase` sets `connectedToDatabase` to `false` (line 183) to ensure that clients do not use an instance of `ResultSetTableModel` after that instance has already been terminated. `IllegalStateException` is an exception from the Java libraries that is appropriate for indicating this error condition.

`DisplayQueryResults` *Class*

Class `DisplayQueryResults` (Fig. 28.28) implements the application's GUI and interacts with the `ResultSetTableModel` via a `JTable` object. This application also demonstrates the `JTable` sorting and filtering capabilities.

```
1  // Fig. 28.28: DisplayQueryResults.java
2  // Display the contents of the Authors table in the books database.
3  import java.awt.BorderLayout;
4  import java.awt.event.ActionListener;
5  import java.awt.event.ActionEvent;
6  import java.awt.event.WindowAdapter;
7  import java.awt.event.WindowEvent;
8  import java.sql.SQLException;
9  import java.util.regex.PatternSyntaxException;
10 import javax.swing.JFrame;
11 import javax.swing.JTextArea;
12 import javax.swing.JScrollPane;
13 import javax.swing.ScrollPaneConstants;
14 import javax.swing.JTable;
15 import javax.swing.JOptionPane;
16 import javax.swing.JButton;
17 import javax.swing.Box;
18 import javax.swing.JLabel;
19 import javax.swing.JTextField;
20 import javax.swing.RowFilter;
21 import javax.swing.table.TableRowSorter;
22 import javax.swing.table.TableModel;
23
24 public class DisplayQueryResults extends JFrame
25 {
26    // database URL, username and password
27    static final String DATABASE_URL = "jdbc:mysql://localhost/books";
28    static final String USERNAME = "deitel";
29    static final String PASSWORD = "deitel";
```

Fig. 28.28 | Displays contents of the database `books`. (Part 1 of 5.)

```
30
31      // default query retrieves all data from Authors table
32      static final String DEFAULT_QUERY = "SELECT * FROM Authors";
33
34      private ResultSetTableModel tableModel;
35      private JTextArea queryArea;
36
37      // create ResultSetTableModel and GUI
38      public DisplayQueryResults()
39      {
40         super( "Displaying Query Results" );
41
42         // create ResultSetTableModel and display database table
43         try
44         {
45            // create TableModel for results of query SELECT * FROM Authors
46            tableModel = new ResultSetTableModel( DATABASE_URL,
47               USERNAME, PASSWORD, DEFAULT_QUERY );
48
49            // set up JTextArea in which user types queries
50            queryArea = new JTextArea( DEFAULT_QUERY, 3, 100 );
51            queryArea.setWrapStyleWord( true );
52            queryArea.setLineWrap( true );
53
54            JScrollPane scrollPane = new JScrollPane( queryArea,
55               ScrollPaneConstants.VERTICAL_SCROLLBAR_AS_NEEDED,
56               ScrollPaneConstants.HORIZONTAL_SCROLLBAR_NEVER );
57
58            // set up JButton for submitting queries
59            JButton submitButton = new JButton( "Submit Query" );
60
61            // create Box to manage placement of queryArea and
62            // submitButton in GUI
63            Box boxNorth = Box.createHorizontalBox();
64            boxNorth.add( scrollPane );
65            boxNorth.add( submitButton );
66
67            // create JTable based on the tableModel
68            JTable resultTable = new JTable( tableModel );
69
70            JLabel filterLabel = new JLabel( "Filter:" );
71            final JTextField filterText = new JTextField();
72            JButton filterButton = new JButton( "Apply Filter" );
73            Box boxSouth = Box.createHorizontalBox();
74
75            boxSouth.add( filterLabel );
76            boxSouth.add( filterText );
77            boxSouth.add( filterButton );
78
79            // place GUI components on content pane
80            add( boxNorth, BorderLayout.NORTH );
81            add( new JScrollPane( resultTable ), BorderLayout.CENTER );
82            add( boxSouth, BorderLayout.SOUTH );
```

Fig. 28.28 | Displays contents of the database books. (Part 2 of 5.)

```
83
84          // create event listener for submitButton
85          submitButton.addActionListener(
86
87             new ActionListener()
88             {
89                // pass query to table model
90                public void actionPerformed( ActionEvent event )
91                {
92                   // perform a new query
93                   try
94                   {
95                      tableModel.setQuery( queryArea.getText() );
96                   } // end try
97                   catch ( SQLException sqlException )
98                   {
99                      JOptionPane.showMessageDialog( null,
100                        sqlException.getMessage(), "Database error",
101                        JOptionPane.ERROR_MESSAGE );
102
103                     // try to recover from invalid user query
104                     // by executing default query
105                     try
106                     {
107                        tableModel.setQuery( DEFAULT_QUERY );
108                        queryArea.setText( DEFAULT_QUERY );
109                     } // end try
110                     catch ( SQLException sqlException2 )
111                     {
112                        JOptionPane.showMessageDialog( null,
113                           sqlException2.getMessage(), "Database error",
114                           JOptionPane.ERROR_MESSAGE );
115
116                        // ensure database connection is closed
117                        tableModel.disconnectFromDatabase();
118
119                        System.exit( 1 ); // terminate application
120                     } // end inner catch
121                  } // end outer catch
122               } // end actionPerformed
123            }  // end ActionListener inner class
124         ); // end call to addActionListener
125
126         final TableRowSorter< TableModel > sorter =
127            new TableRowSorter< TableModel >( tableModel );
128         resultTable.setRowSorter( sorter );
129         setSize( 500, 250 ); // set window size
130         setVisible( true ); // display window
131
132         // create listener for filterButton
133         filterButton.addActionListener(
134            new ActionListener()
135            {
```

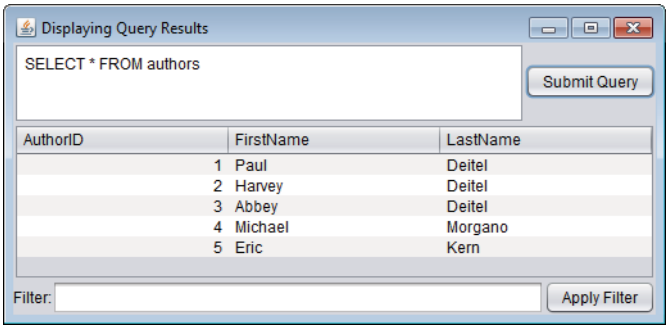
Fig. 28.28 | Displays contents of the database books. (Part 3 of 5.)

```
136                // pass filter text to listener
137                public void actionPerformed( ActionEvent e )
138                {
139                   String text = filterText.getText();
140
141                   if ( text.length() == 0 )
142                      sorter.setRowFilter( null );
143                   else
144                   {
145                      try
146                      {
147                         sorter.setRowFilter(
148                            RowFilter.regexFilter( text ) );
149                      } // end try
150                      catch ( PatternSyntaxException pse )
151                      {
152                         JOptionPane.showMessageDialog( null,
153                            "Bad regex pattern", "Bad regex pattern",
154                            JOptionPane.ERROR_MESSAGE );
155                      } // end catch
156                   } // end else
157                } // end method actionPerfomed
158             } // end annonymous inner class
159          ); // end call to addActionLister
160       } // end try
161       catch ( SQLException sqlException )
162       {
163          JOptionPane.showMessageDialog( null, sqlException.getMessage(),
164             "Database error", JOptionPane.ERROR_MESSAGE );
165
166          // ensure database connection is closed
167          tableModel.disconnectFromDatabase();
168
169          System.exit( 1 ); // terminate application
170       } // end catch
171
172       // dispose of window when user quits application (this overrides
173       // the default of HIDE_ON_CLOSE)
174       setDefaultCloseOperation( DISPOSE_ON_CLOSE );
175
176       // ensure database connection is closed when user quits application
177       addWindowListener(
178
179          new WindowAdapter()
180          {
181             // disconnect from database and exit when window has closed
182             public void windowClosed( WindowEvent event )
183             {
184                tableModel.disconnectFromDatabase();
185                System.exit( 0 );
186             } // end method windowClosed
187          } // end WindowAdapter inner class
```

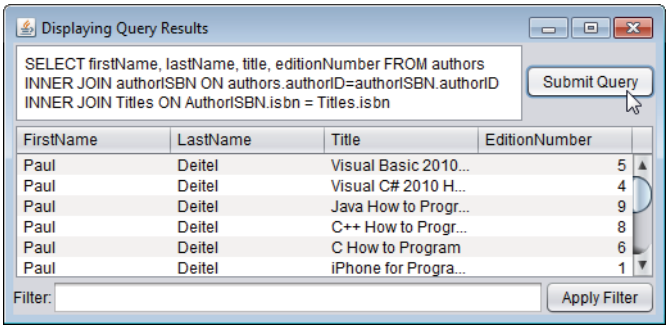
Fig. 28.28 | Displays contents of the database books. (Part 4 of 5.)

```
188         ); // end call to addWindowListener
189      } // end DisplayQueryResults constructor
190
191      // execute application
192      public static void main( String args[] )
193      {
194         new DisplayQueryResults();
195      } // end main
196   } // end class DisplayQueryResults
```

a) Displaying all authors from the `Authors` table



b) Displaying the the authors' first and last names joined with the titles and edition numbers of the books they've authored



c) Filtering the results of the previous query to show only the books with Java in the title

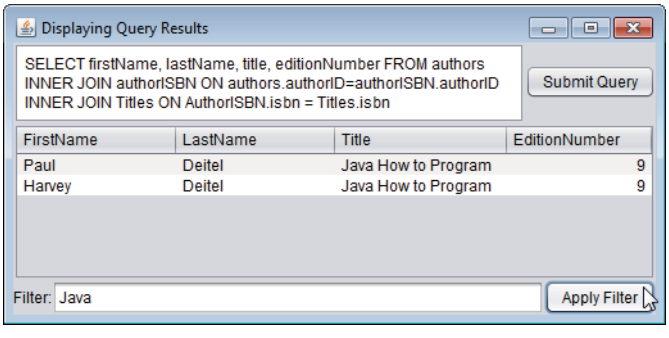


Fig. 28.28 | Displays contents of the database `books`. (Part 5 of 5.)

Lines 27–29 and 32 declare the URL, username, password and default query that are passed to the `ResultSetTableModel` constructor to make the initial connection to the

database and perform the default query. The `DisplayQueryResults` constructor (lines 38–189) creates a `ResultSetTableModel` object and the GUI for the application. Line 68 creates the `JTable` object and passes a `ResultSetTableModel` object to the `JTable` constructor, which then registers the `JTable` as a listener for `TableModelEvents` generated by the `ResultSetTableModel`.

The local variables `filterText` (line 71) and `sorter` (lines 126–127) are declared `final`. These are both used from an event handler that is implemented as an anonymous inner class (lines 134–158). Any local variable that will be used in an anonymous inner class *must* be declared final; otherwise, a compilation error occurs.

Lines 85–124 register an event handler for the `submitButton` that the user clicks to submit a query to the database. When the user clicks the button, method `actionPerformed` (lines 90–122) invokes method `setQuery` from the class `ResultSetTableModel` to execute the new query (line 95). If the user's query fails (e.g., because of a syntax error in the user's input), lines 107–108 execute the default query. If the default query also fails, there could be a more serious error, so line 117 ensures that the database connection is closed and line 119 exits the program. The screen captures in Fig. 28.28 show the results of two queries. The first screen capture shows the default query that retrieves all the data from table `Authors` of database `books`. The second screen capture shows a query that selects each author's first name and last name from the `Authors` table and combines that information with the title and edition number from the `Titles` table. Try entering your own queries in the text area and clicking the **Submit Query** button to execute the query.

Lines 177–188 register a **`WindowListener`** for the **`windowClosed`** event, which occurs when the user closes the window. Since `WindowListeners` can handle several window events, we extend class **`WindowAdapter`** and override only the `windowClosed` event handler.

Sorting Rows in a `JTable`

`JTables` allow users to sort rows by the data in a specific column. Lines 126–127 use the **`TableRowSorter`** class (from package **`javax.swing.table`**) to create an object that uses our `ResultSetTableModel` to sort rows in the `JTable` that displays query results. When the user clicks the title of a particular `JTable` column, the `TableRowSorter` interacts with the underlying `TableModel` to reorder the rows based on the data in that column. Line 128 uses `JTable` method **`setRowSorter`** to specify the `TableRowSorter` for `resultTable`.

Filtering Rows in a `JTable`

`JTables` can now show subsets of the data from the underlying `TableModel`. This is known as filtering the data. Lines 133–159 register an event handler for the `filterButton` that the user clicks to filter the data. In method `actionPerformed` (lines 137–157), line 139 obtains the filter text. If the user did not specify filter text, line 142 uses `JTable` method **`setRowFilter`** to remove any prior filter by setting the filter to `null`. Otherwise, lines 147–148 use `setRowFilter` to specify a **`RowFilter`** (from package `javax.swing`) based on the user's input. Class `RowFilter` provides several methods for creating filters. The `static` method **`regexFilter`** receives a `String` containing a regular expression pattern as its argument and an optional set of indices that specify which columns to filter. If no indices are specified, then all the columns are searched. In this example, the regular expression pattern is the text the user typed. Once the filter is set, the data displayed in the `JTable` is updated based on the filtered `TableModel`.

28.9 RowSet Interface

In the preceding examples, you learned how to query a database by explicitly establishing a `Connection` to the database, preparing a `Statement` for querying the database and executing the query. In this section, we demonstrate the **`RowSet` interface**, which configures the database connection and prepares query statements automatically. The interface `RowSet` provides several *set* methods that allow you to specify the properties needed to establish a connection (such as the database URL, user name and password of the database) and create a `Statement` (such as a query). `RowSet` also provides several *get* methods that return these properties.

Connected and Disconnected `RowSet`*s*

There are two types of `RowSet` objects—connected and disconnected. A **connected `RowSet`** object connects to the database once and remains connected while the object is in use. A **disconnected `RowSet`** object connects to the database, executes a query to retrieve the data from the database and then closes the connection. A program may change the data in a disconnected `RowSet` while it's disconnected. Modified data can be updated in the database after a disconnected `RowSet` reestablishes the connection with the database.

Package **`javax.sql.rowset`** contains two subinterfaces of `RowSet`—`JdbcRowSet` and `CachedRowSet`. **`JdbcRowSet`**, a connected `RowSet`, acts as a wrapper around a `ResultSet` object and allows you to scroll through and update the rows in the `ResultSet`. Recall that by default, a `ResultSet` object is nonscrollable and read only—you must explicitly set the result set type constant to `TYPE_SCROLL_INSENSITIVE` and set the result set concurrency constant to `CONCUR_UPDATABLE` to make a `ResultSet` object scrollable and updatable. A `JdbcRowSet` object is scrollable and updatable by default. **`CachedRowSet`**, a disconnected `RowSet`, caches the data of a `ResultSet` in memory and disconnects from the database. Like `JdbcRowSet`, a `CachedRowSet` object is scrollable and updatable by default. A `CachedRowSet` object is also *serializable*, so it can be passed between Java applications through a network, such as the Internet. However, `CachedRowSet` has a limitation—the amount of data that can be stored in memory is limited. Package `javax.sql.rowset` contains three other subinterfaces of `RowSet`.



Portability Tip 28.5

A `RowSet` can provide scrolling capability for drivers that do not support scrollable `ResultSet`s.

Using a `RowSet`

Figure 28.29 reimplements the example of Fig. 28.23 using a `RowSet`. Rather than establish the connection and create a `Statement` explicitly, Fig. 28.29 uses a `JdbcRowSet` object to create a `Connection` and a `Statement` automatically.

```
1  // Fig. 28.29: JdbcRowSetTest.java
2  // Displaying the contents of the Authors table using JdbcRowSet.
3  import java.sql.ResultSetMetaData;
4  import java.sql.SQLException;
```

Fig. 28.29 | Displaying the `Authors` table using `JdbcRowSet`. (Part 1 of 3.)

```
5  import javax.sql.rowset.JdbcRowSet;
6  import com.sun.rowset.JdbcRowSetImpl; // Sun's JdbcRowSet implementation
7
8  public class JdbcRowSetTest
9  {
10    // JDBC driver name and database URL
11    static final String DATABASE_URL = "jdbc:mysql://localhost/books";
12    static final String USERNAME = "deitel";
13    static final String PASSWORD = "deitel";
14
15    // constructor connects to database, queries database, processes
16    // results and displays results in window
17    public JdbcRowSetTest()
18    {
19       // connect to database books and query database
20       try
21       {
22          // specify properties of JdbcRowSet
23          JdbcRowSet rowSet = new JdbcRowSetImpl();
24          rowSet.setUrl( DATABASE_URL ); // set database URL
25          rowSet.setUsername( USERNAME ); // set username
26          rowSet.setPassword( PASSWORD ); // set password
27          rowSet.setCommand( "SELECT * FROM Authors" ); // set query
28          rowSet.execute(); // execute query
29
30          // process query results
31          ResultSetMetaData metaData = rowSet.getMetaData();
32          int numberOfColumns = metaData.getColumnCount();
33          System.out.println( "Authors Table of Books Database:\n" );
34
35          // display rowset header
36          for ( int i = 1; i <= numberOfColumns; i++ )
37             System.out.printf( "%-8s\t", metaData.getColumnName( i ) );
38          System.out.println();
39
40          // display each row
41          while ( rowSet.next() )
42          {
43             for ( int i = 1; i <= numberOfColumns; i++ )
44                System.out.printf( "%-8s\t", rowSet.getObject( i ) );
45             System.out.println();
46          } // end while
47
48          // close the underlying ResultSet, Statement and Connection
49          rowSet.close();
50       } // end try
51       catch ( SQLException sqlException )
52       {
53          sqlException.printStackTrace();
54          System.exit( 1 );
55       } // end catch
56    } // end DisplayAuthors constructor
57
```

Fig. 28.29 | Displaying the `Authors` table using `JdbcRowSet`. (Part 2 of 3.)

```
58      // launch the application
59      public static void main( String args[] )
60      {
61         JdbcRowSetTest application = new JdbcRowSetTest();
62      } // end main
63   } // end class JdbcRowSetTest
```

```
Authors Table of Books Database:

AuthorID        FirstName       LastName
1               Paul            Deitel
2               Harvey          Deitel
3               Abbey           Deitel
4               Michael         Morgano
5               Eric            Kern
```

Fig. 28.29 | Displaying the `Authors` table using `JdbcRowSet`. (Part 3 of 3.)

The package **`com.sun.rowset`** provides Oracle's reference implementations of the interfaces in package `javax.sql.rowset`. Line 23 uses Sun's reference implementation of the `JdbcRowSet` interface—**`JdbcRowSetImpl`**—to create a `JdbcRowSet` object. We used class `JdbcRowSetImpl` here to demonstrate the capability of the `JdbcRowSet` interface. Other databases may provide their own `RowSet` implementations.

Lines 24–26 set the `RowSet` properties that the `DriverManager` uses to establish a database connection. Line 24 invokes `JdbcRowSet` method **`setUrl`** to specify the database URL. Line 25 invokes `JdbcRowSet` method **`setUsername`** to specify the username. Line 26 invokes `JdbcRowSet` method **`setPassword`** to specify the password. Line 27 invokes `JdbcRowSet` method **`setCommand`** to specify the SQL query that will be used to populate the `RowSet`. Line 28 invokes `JdbcRowSet` method **`execute`** to execute the SQL query. Method `execute` performs four actions—it establishes a `Connection` to the database, prepares the query `Statement`, executes the query and stores the `ResultSet` returned by query. The `Connection`, `Statement` and `ResultSet` are encapsulated in the `JdbcRowSet` object.

The remaining code is almost identical to Fig. 28.23, except that line 31 obtains a `ResultSetMetaData` object from the `JdbcRowSet`, line 41 uses the `JdbcRowSet`'s `next` method to get the next row of the result and line 44 uses the `JdbcRowSet`'s `getObject` method to obtain a column's value. Line 49 invokes `JdbcRowSet` method **`close`**, which closes the `RowSet`'s encapsulated `ResultSet`, `Statement` and `Connection`. In a `CachedRowSet`, invoking `close` also releases the resources held by that `RowSet`. The output of this application is the same as that of Fig. 28.23.

28.10 Java DB/Apache Derby

In this section and Section 28.11, we use Oracle's pure Java database **Java DB**. Please refer to the Before You Begin section after the Preface for information on installing Java DB. Section 28.11 uses the embedded version of Java DB. There's also a network version that executes similarly to the MySQL DBMS introduced earlier in the chapter.

Before you can execute the application in Section 28.11, you must set up the `AddressBook` database in Java DB. For the purpose of the following steps, we assume

you're running Microsoft Windows with Java installed in its default location. Mac OS X and Linux will need to perform similar steps.

1. Java DB comes with several batch files to configure and run it. Before executing these batch files from a command prompt, you must set the environment variable `JAVA_HOME` to refer to the JDK's installation directory—for example, `C:\Program Files\Java\jdk1.6.0_23`. Be sure to use the exact installation directory of the JDK on your computer.
2. Open the batch file `setEmbeddedCP.bat` (typically located in `C:\Program Files\Sun\JavaDB\bin`) in a text editor such as Notepad. Locate the line

```
@rem set DERBY_INSTALL=
```

   and change it to

```
@set DERBY_INSTALL=C:\Program Files\Sun\JavaDB
```

   Save your changes and close this file. [*Note:* You might need to run Notepad as an Administrator to edit this file. To do so, open the Start menu and type `Notepad` in the **Search programs and files** field. Then, right click **Notepad** at the top of the menu and select **Run as administrator**.]
3. Open a Command Prompt as an administrator (as you did for Notepad in the previous step) and change directories to

```
C:\Program Files\Sun\JavaDB\bin
```

   Then, type `setEmbeddedCP.bat` and press *Enter* to set the environment variables required by Java DB.
4. An embedded Java DB database must reside in the same location as the application that manipulates the database. For this reason, change to the directory that contains the code for Figs. 28.30–28.32. This directory contains a SQL script `address.sql` that builds the `AddressBook` database.
5. Execute the command

```
"C:\Program Files\Sun\JavaDB\bin\ij"
```

   to start the command-line tool for interacting with Java DB. The double quotes are necessary because the path contains a space. This will display the `ij>` prompt.
6. At the `ij>` prompt type

```
connect 'jdbc:derby:AddressBook;create=true;user=deitel;
   password=deitel';
```

   and press *Enter* to create the `AddressBook` database in the current directory and to create the user `deitel` with the password `deitel` for accessing the database.
7. To create the database table and insert sample data in it, we've provided the file `address.sql` in this example's directory. To execute this SQL script, type

```
run 'address.sql';
```

8. To terminate the Java DB command-line tool, type

```
exit;
```

You're now ready to execute the `AddressBook` application in Section 28.11. MySQL or any other database that supports JDBC `PreparedStatements` could also be used.

28.11 PreparedStatements

A **`PreparedStatement`** enables you to create compiled SQL statements that execute more efficiently than `Statements`. `PreparedStatements` can also specify parameters, making them more flexible than `Statements`—you can execute the same query repeatedly with different parameter values. For example, in the `books` database, you might want to locate all book titles for an author with a specific last and first name, and you might want to execute that query for several authors. With a `PreparedStatement`, that query is defined as follows:

```
PreparedStatement authorBooks = connection.prepareStatement(
   "SELECT LastName, FirstName, Title " +
   "FROM Authors INNER JOIN AuthorISBN " +
      "ON Authors.AuthorID=AuthorISBN.AuthorID " +
   "INNER JOIN Titles " +
      "ON AuthorISBN.ISBN=Titles.ISBN " +
   "WHERE LastName = ? AND FirstName = ?" );
```

The two question marks (?) in the the preceding SQL statement's last line are placeholders for values that will be passed as part of the query to the database. Before executing a `PreparedStatement`, the program must specify the parameter values by using the `PreparedStatement` interface's *set* methods.

For the preceding query, both parameters are strings that can be set with `PreparedStatement` method **`setString`** as follows:

```
authorBooks.setString( 1, "Deitel" );
authorBooks.setString( 2, "Paul" );
```

Method `setString`'s first argument represents the parameter number being set, and the second argument is that parameter's value. Parameter numbers are *counted from 1*, starting with the first question mark (?). When the program executes the preceding `PreparedStatement` with the parameter values set above, the SQL passed to the database is

```
SELECT LastName, FirstName, Title
FROM Authors INNER JOIN AuthorISBN
   ON Authors.AuthorID=AuthorISBN.AuthorID
INNER JOIN Titles
   ON AuthorISBN.ISBN=Titles.ISBN
WHERE LastName = 'Deitel' AND FirstName = 'Paul'
```

Method `setString` automatically escapes `String` parameter values as necessary. For example, if the last name is O'Brien, the statement

```
authorBooks.setString( 1, "O'Brien" );
```

escapes the `'` character in O'Brien by replacing it with two single-quote characters, so that the `'` appears correctly in the database.



Performance Tip 28.2

`PreparedStatements` are more efficient than `Statements` when executing SQL statements multiple times and with different parameter values.



Error-Prevention Tip 28.2

Use `PreparedStatements` *with parameters for queries that receive* `String` *values as arguments to ensure that the* `Strings` *are quoted properly in the SQL statement.*

Interface `PreparedStatement` provides *set* methods for each supported SQL type. It's important to use the *set* method that is appropriate for the parameter's SQL type in the database—`SQLExceptions` occur when a program attempts to convert a parameter value to an incorrect type.

Address Book Application that Uses `PreparedStatements`

We now present an address book application that enables you to browse existing entries, add new entries and search for entries with a specific last name. Our `AddressBook` Java DB database contains an `Addresses` table with the columns `addressID`, `FirstName`, `LastName`, `Email` and `PhoneNumber`. The column `addressID` is a so-called *identity column*. This is the SQL standard way to represent an *autoincremented column*. The SQL script we provide for this database uses the SQL `IDENTITY` keyword to mark the `addressID` column as an identity column. For more information on using the `IDENTITY` keyword and creating databases, see the Java DB Developer's Guide at `download.oracle.com/javadb/10.6.1.0/devguide/devguide-single.html`.

Class `Person`

Our address book application consists of three classes—`Person` (Fig. 28.30), `PersonQueries` (Fig. 28.31) and `AddressBookDisplay` (Fig. 28.32). Class `Person` is a simple class that represents one person in the address book. The class contains fields for the address ID, first name, last name, email address and phone number, as well as *set* and *get* methods for manipulating these fields.

```
1  // Fig. 28.30: Person.java
2  // Person class that represents an entry in an address book.
3  public class Person
4  {
5     private int addressID;
6     private String firstName;
7     private String lastName;
8     private String email;
9     private String phoneNumber;
10
11    // no-argument constructor
12    public Person()
13    {
14    } // end no-argument Person constructor
15
16    // constructor
17    public Person( int id, String first, String last,
18       String emailAddress, String phone )
19    {
20       setAddressID( id );
21       setFirstName( first );
```

Fig. 28.30 | `Person` class that represents an entry in an `AddressBook`. (Part 1 of 3.)

```
22          setLastName( last );
23          setEmail( emailAddress );
24          setPhoneNumber( phone );
25       } // end five-argument Person constructor
26
27       // sets the addressID
28       public void setAddressID( int id )
29       {
30          addressID = id;
31       } // end method setAddressID
32
33       // returns the addressID
34       public int getAddressID()
35       {
36          return addressID;
37       } // end method getAddressID
38
39       // sets the firstName
40       public void setFirstName( String first )
41       {
42          firstName = first;
43       } // end method setFirstName
44
45       // returns the first name
46       public String getFirstName()
47       {
48          return firstName;
49       } // end method getFirstName
50
51       // sets the lastName
52       public void setLastName( String last )
53       {
54          lastName = last;
55       } // end method setLastName
56
57       // returns the last name
58       public String getLastName()
59       {
60          return lastName;
61       } // end method getLastName
62
63       // sets the email address
64       public void setEmail( String emailAddress )
65       {
66          email = emailAddress;
67       } // end method setEmail
68
69       // returns the email address
70       public String getEmail()
71       {
72          return email;
73       } // end method getEmail
74
```

Fig. 28.30 | `Person` class that represents an entry in an `AddressBook`. (Part 2 of 3.)

```
75      // sets the phone number
76      public void setPhoneNumber( String phone )
77      {
78         phoneNumber = phone;
79      } // end method setPhoneNumber
80
81      // returns the phone number
82      public String getPhoneNumber()
83      {
84         return phoneNumber;
85      } // end method getPhoneNumber
86   } // end class Person
```

Fig. 28.30 | `Person` class that represents an entry in an `AddressBook`. (Part 3 of 3.)

Class `PersonQueries`

Class `PersonQueries` (Fig. 28.31) manages the address book application's database connection and creates the `PreparedStatements` that the application uses to interact with the database. Lines 18–20 declare three `PreparedStatement` variables. The constructor (lines 23–49) connects to the database at lines 27–28.

```
1   // Fig. 28.31: PersonQueries.java
2   // PreparedStatements used by the Address Book application.
3   import java.sql.Connection;
4   import java.sql.DriverManager;
5   import java.sql.PreparedStatement;
6   import java.sql.ResultSet;
7   import java.sql.SQLException;
8   import java.util.List;
9   import java.util.ArrayList;
10
11  public class PersonQueries
12  {
13     private static final String URL = "jdbc:derby:AddressBook";
14     private static final String USERNAME = "deitel";
15     private static final String PASSWORD = "deitel";
16
17     private Connection connection = null; // manages connection
18     private PreparedStatement selectAllPeople = null;
19     private PreparedStatement selectPeopleByLastName = null;
20     private PreparedStatement insertNewPerson = null;
21
22     // constructor
23     public PersonQueries()
24     {
25        try
26        {
27           connection =
28              DriverManager.getConnection( URL, USERNAME, PASSWORD );
29
```

Fig. 28.31 | `PreparedStatements` used by the Address Book application. (Part 1 of 4.)

```
30            // create query that selects all entries in the AddressBook
31            selectAllPeople =
32               connection.prepareStatement( "SELECT * FROM Addresses" );
33
34            // create query that selects entries with a specific last name
35            selectPeopleByLastName = connection.prepareStatement(
36               "SELECT * FROM Addresses WHERE LastName = ?" );
37
38            // create insert that adds a new entry into the database
39            insertNewPerson = connection.prepareStatement(
40               "INSERT INTO Addresses " +
41               "( FirstName, LastName, Email, PhoneNumber ) " +
42               "VALUES ( ?, ?, ?, ? )" );
43         } // end try
44         catch ( SQLException sqlException )
45         {
46            sqlException.printStackTrace();
47            System.exit( 1 );
48         } // end catch
49      } // end PersonQueries constructor
50
51      // select all of the addresses in the database
52      public List< Person > getAllPeople()
53      {
54         List< Person > results = null;
55         ResultSet resultSet = null;
56
57         try
58         {
59            // executeQuery returns ResultSet containing matching entries
60            resultSet = selectAllPeople.executeQuery();
61            results = new ArrayList< Person >();
62
63            while ( resultSet.next() )
64            {
65               results.add( new Person(
66                  resultSet.getInt( "addressID" ),
67                  resultSet.getString( "FirstName" ),
68                  resultSet.getString( "LastName" ),
69                  resultSet.getString( "Email" ),
70                  resultSet.getString( "PhoneNumber" ) ) );
71            } // end while
72         } // end try
73         catch ( SQLException sqlException )
74         {
75            sqlException.printStackTrace();
76         } // end catch
77         finally
78         {
79            try
80            {
81               resultSet.close();
82            } // end try
```

Fig. 28.31 | `PreparedStatements` used by the Address Book application. (Part 2 of 4.)

```
83            catch ( SQLException sqlException )
84            {
85               sqlException.printStackTrace();
86               close();
87            } // end catch
88         } // end finally
89
90         return results;
91      } // end method getAllPeople
92
93      // select person by last name
94      public List< Person > getPeopleByLastName( String name )
95      {
96         List< Person > results = null;
97         ResultSet resultSet = null;
98
99         try
100        {
101           selectPeopleByLastName.setString( 1, name ); // specify last name
102
103           // executeQuery returns ResultSet containing matching entries
104           resultSet = selectPeopleByLastName.executeQuery();
105
106           results = new ArrayList< Person >();
107
108           while ( resultSet.next() )
109           {
110              results.add( new Person( resultSet.getInt( "addressID" ),
111                 resultSet.getString( "FirstName" ),
112                 resultSet.getString( "LastName" ),
113                 resultSet.getString( "Email" ),
114                 resultSet.getString( "PhoneNumber" ) ) );
115           } // end while
116        } // end try
117        catch ( SQLException sqlException )
118        {
119           sqlException.printStackTrace();
120        } // end catch
121        finally
122        {
123           try
124           {
125              resultSet.close();
126           } // end try
127           catch ( SQLException sqlException )
128           {
129              sqlException.printStackTrace();
130              close();
131           } // end catch
132        } // end finally
133
134        return results;
135     } // end method getPeopleByName
```

Fig. 28.31 | `PreparedStatements` used by the Address Book application. (Part 3 of 4.)

```
136
137    // add an entry
138    public int addPerson(
139       String fname, String lname, String email, String num )
140    {
141       int result = 0;
142
143       // set parameters, then execute insertNewPerson
144       try
145       {
146          insertNewPerson.setString( 1, fname );
147          insertNewPerson.setString( 2, lname );
148          insertNewPerson.setString( 3, email );
149          insertNewPerson.setString( 4, num );
150
151          // insert the new entry; returns # of rows updated
152          result = insertNewPerson.executeUpdate();
153       } // end try
154       catch ( SQLException sqlException )
155       {
156          sqlException.printStackTrace();
157          close();
158       } // end catch
159
160       return result;
161    } // end method addPerson
162
163    // close the database connection
164    public void close()
165    {
166       try
167       {
168          connection.close();
169       } // end try
170       catch ( SQLException sqlException )
171       {
172          sqlException.printStackTrace();
173       } // end catch
174    } // end method close
175 } // end class PersonQueries
```

Fig. 28.31 | `PreparedStatements` used by the Address Book application. (Part 4 of 4.)

Creating `PreparedStatements`

Lines 31–32 invoke `Connection` method **`prepareStatement`** to create the `PreparedStatement` named `selectAllPeople` that selects all the rows in the `Addresses` table. Lines 35–36 create the `PreparedStatement` named `selectPeopleByLastName` with a parameter. This statement selects all the rows in the `Addresses` table that match a particular last name. Notice the `?` character that's used to specify the last-name parameter. Lines 39–42 create the `PreparedStatement` named `insertNewPerson` with four parameters that represent the first name, last name, email address and phone number for a new entry. Again, notice the `?` characters used to represent these parameters.

PersonQueries Method getAllPeople

Method `getAllPeople` (lines 52–91) executes `PreparedStatement` `selectAllPeople` (line 60) by calling method **`executeQuery`**, which returns a `ResultSet` containing the rows that match the query (in this case, all the rows in the `Addresses` table). Lines 61–71 place the query results in an `ArrayList` of `Person` objects, which is returned to the caller at line 90. Method `getPeopleByLastName` (lines 94–135) uses `PreparedStatement` method `setString` to set the parameter to `selectPeopleByLastName` (line 101). Then, line 104 executes the query and lines 106–115 place the query results in an `ArrayList` of `Person` objects. Line 134 returns the `ArrayList` to the caller.

PersonQueries Methods addPerson *and* Close

Method `addPerson` (lines 138–161) uses `PreparedStatement` method `setString` (lines 146–149) to set the parameters for the `insertNewPerson` `PreparedStatement`. Line 152 uses `PreparedStatement` method **`executeUpdate`** to insert the new record. This method returns an integer indicating the number of rows that were updated (or inserted) in the database. Method `close` (lines 164–174) simply closes the database connection.

Class AddressBookDisplay

The `AddressBookDisplay` (Fig. 28.32) application uses a `PersonQueries` object to interact with the database. Line 59 creates the `PersonQueries` object. When the user presses the **Browse All Entries** `JButton`, the `browseButtonActionPerformed` handler (lines 309–335) is called. Line 313 calls the method `getAllPeople` on the `PersonQueries` object to obtain all the entries in the database. The user can then scroll through the entries using the **Previous** and **Next** `JButtons`. When the user presses the **Find** `JButton`, the `queryButtonActionPerformed` handler (lines 265–287) is called. Lines 267–268 call method `getPeopleByLastName` on the `PersonQueries` object to obtain the entries in the database that match the specified last name. If there are several such entries, the user can then scroll through them using the **Previous** and **Next** `JButtons`.

```
1  // Fig. 28.32: AddressBookDisplay.java
2  // A simple address book
3  import java.awt.event.ActionEvent;
4  import java.awt.event.ActionListener;
5  import java.awt.event.WindowAdapter;
6  import java.awt.event.WindowEvent;
7  import java.awt.FlowLayout;
8  import java.awt.GridLayout;
9  import java.util.List;
10 import javax.swing.JButton;
11 import javax.swing.Box;
12 import javax.swing.JFrame;
13 import javax.swing.JLabel;
14 import javax.swing.JPanel;
15 import javax.swing.JTextField;
16 import javax.swing.WindowConstants;
17 import javax.swing.BoxLayout;
18 import javax.swing.BorderFactory;
19 import javax.swing.JOptionPane;
```

Fig. 28.32 | A simple address book. (Part 1 of 9.)

```
20
21  public class AddressBookDisplay extends JFrame
22  {
23     private Person currentEntry;
24     private PersonQueries personQueries;
25     private List< Person > results;
26     private int numberOfEntries = 0;
27     private int currentEntryIndex;
28
29     private JButton browseButton;
30     private JLabel emailLabel;
31     private JTextField emailTextField;
32     private JLabel firstNameLabel;
33     private JTextField firstNameTextField;
34     private JLabel idLabel;
35     private JTextField idTextField;
36     private JTextField indexTextField;
37     private JLabel lastNameLabel;
38     private JTextField lastNameTextField;
39     private JTextField maxTextField;
40     private JButton nextButton;
41     private JLabel ofLabel;
42     private JLabel phoneLabel;
43     private JTextField phoneTextField;
44     private JButton previousButton;
45     private JButton queryButton;
46     private JLabel queryLabel;
47     private JPanel queryPanel;
48     private JPanel navigatePanel;
49     private JPanel displayPanel;
50     private JTextField queryTextField;
51     private JButton insertButton;
52
53     // no-argument constructor
54     public AddressBookDisplay()
55     {
56        super( "Address Book" );
57
58        // establish database connection and set up PreparedStatements
59        personQueries = new PersonQueries();
60
61        // create GUI
62        navigatePanel = new JPanel();
63        previousButton = new JButton();
64        indexTextField = new JTextField( 2 );
65        ofLabel = new JLabel();
66        maxTextField = new JTextField( 2 );
67        nextButton = new JButton();
68        displayPanel = new JPanel();
69        idLabel = new JLabel();
70        idTextField = new JTextField( 10 );
71        firstNameLabel = new JLabel();
72        firstNameTextField = new JTextField( 10 );
```

Fig. 28.32 | A simple address book. (Part 2 of 9.)

```
73          lastNameLabel = new JLabel();
74          lastNameTextField = new JTextField( 10 );
75          emailLabel = new JLabel();
76          emailTextField = new JTextField( 10 );
77          phoneLabel = new JLabel();
78          phoneTextField = new JTextField( 10 );
79          queryPanel = new JPanel();
80          queryLabel = new JLabel();
81          queryTextField = new JTextField( 10 );
82          queryButton = new JButton();
83          browseButton = new JButton();
84          insertButton = new JButton();
85
86          setLayout( new FlowLayout( FlowLayout.CENTER, 10, 10 ) );
87          setSize( 400, 300 );
88          setResizable( false );
89
90          navigatePanel.setLayout(
91             new BoxLayout( navigatePanel, BoxLayout.X_AXIS ) );
92
93          previousButton.setText( "Previous" );
94          previousButton.setEnabled( false );
95          previousButton.addActionListener(
96             new ActionListener()
97             {
98                public void actionPerformed( ActionEvent evt )
99                {
100                  previousButtonActionPerformed( evt );
101               } // end method actionPerformed
102            } // end anonymous inner class
103         ); // end call to addActionListener
104
105         navigatePanel.add( previousButton );
106         navigatePanel.add( Box.createHorizontalStrut( 10 ) );
107
108         indexTextField.setHorizontalAlignment(
109            JTextField.CENTER );
110         indexTextField.addActionListener(
111            new ActionListener()
112            {
113               public void actionPerformed( ActionEvent evt )
114               {
115                  indexTextFieldActionPerformed( evt );
116               } // end method actionPerformed
117            } // end anonymous inner class
118         ); // end call to addActionListener
119
120         navigatePanel.add( indexTextField );
121         navigatePanel.add( Box.createHorizontalStrut( 10 ) );
122
123         ofLabel.setText( "of" );
124         navigatePanel.add( ofLabel );
125         navigatePanel.add( Box.createHorizontalStrut( 10 ) );
```

Fig. 28.32 | A simple address book. (Part 3 of 9.)

```
126
127        maxTextField.setHorizontalAlignment(
128           JTextField.CENTER );
129        maxTextField.setEditable( false );
130        navigatePanel.add( maxTextField );
131        navigatePanel.add( Box.createHorizontalStrut( 10 ) );
132
133        nextButton.setText( "Next" );
134        nextButton.setEnabled( false );
135        nextButton.addActionListener(
136           new ActionListener()
137           {
138              public void actionPerformed( ActionEvent evt )
139              {
140                 nextButtonActionPerformed( evt );
141              } // end method actionPerformed
142           } // end anonymous inner class
143        ); // end call to addActionListener
144
145        navigatePanel.add( nextButton );
146        add( navigatePanel );
147
148        displayPanel.setLayout( new GridLayout( 5, 2, 4, 4 ) );
149
150        idLabel.setText( "Address ID:" );
151        displayPanel.add( idLabel );
152
153        idTextField.setEditable( false );
154        displayPanel.add( idTextField );
155
156        firstNameLabel.setText( "First Name:" );
157        displayPanel.add( firstNameLabel );
158        displayPanel.add( firstNameTextField );
159
160        lastNameLabel.setText( "Last Name:" );
161        displayPanel.add( lastNameLabel );
162        displayPanel.add( lastNameTextField );
163
164        emailLabel.setText( "Email:" );
165        displayPanel.add( emailLabel );
166        displayPanel.add( emailTextField );
167
168        phoneLabel.setText( "Phone Number:" );
169        displayPanel.add( phoneLabel );
170        displayPanel.add( phoneTextField );
171        add( displayPanel );
172
173        queryPanel.setLayout(
174           new BoxLayout( queryPanel, BoxLayout.X_AXIS) );
175
176        queryPanel.setBorder( BorderFactory.createTitledBorder(
177           "Find an entry by last name" ) );
178        queryLabel.setText( "Last Name:" );
```

Fig. 28.32 | A simple address book. (Part 4 of 9.)

```
179         queryPanel.add( Box.createHorizontalStrut( 5 ) );
180         queryPanel.add( queryLabel );
181         queryPanel.add( Box.createHorizontalStrut( 10 ) );
182         queryPanel.add( queryTextField );
183         queryPanel.add( Box.createHorizontalStrut( 10 ) );
184
185         queryButton.setText( "Find" );
186         queryButton.addActionListener(
187            new ActionListener()
188            {
189               public void actionPerformed( ActionEvent evt )
190               {
191                  queryButtonActionPerformed( evt );
192               } // end method actionPerformed
193            } // end anonymous inner class
194         ); // end call to addActionListener
195
196         queryPanel.add( queryButton );
197         queryPanel.add( Box.createHorizontalStrut( 5 ) );
198         add( queryPanel );
199
200         browseButton.setText( "Browse All Entries" );
201         browseButton.addActionListener(
202            new ActionListener()
203            {
204               public void actionPerformed( ActionEvent evt )
205               {
206                  browseButtonActionPerformed( evt );
207               } // end method actionPerformed
208            } // end anonymous inner class
209         ); // end call to addActionListener
210
211         add( browseButton );
212
213         insertButton.setText( "Insert New Entry" );
214         insertButton.addActionListener(
215            new ActionListener()
216            {
217               public void actionPerformed( ActionEvent evt )
218               {
219                  insertButtonActionPerformed( evt );
220               } // end method actionPerformed
221            } // end anonymous inner class
222         ); // end call to addActionListener
223
224         add( insertButton );
225
226         addWindowListener(
227            new WindowAdapter()
228            {
229               public void windowClosing( WindowEvent evt )
230               {
231                  personQueries.close(); // close database connection
```

Fig. 28.32 | A simple address book. (Part 5 of 9.)

```
232                System.exit( 0 );
233             } // end method windowClosing
234          } // end anonymous inner class
235       ); // end call to addWindowListener
236
237       setVisible( true );
238    } // end no-argument constructor
239
240    // handles call when previousButton is clicked
241    private void previousButtonActionPerformed( ActionEvent evt )
242    {
243       currentEntryIndex--;
244
245       if ( currentEntryIndex < 0 )
246          currentEntryIndex = numberOfEntries - 1;
247
248       indexTextField.setText( "" + ( currentEntryIndex + 1 ) );
249       indexTextFieldActionPerformed( evt );
250    } // end method previousButtonActionPerformed
251
252    // handles call when nextButton is clicked
253    private void nextButtonActionPerformed( ActionEvent evt )
254    {
255       currentEntryIndex++;
256
257       if ( currentEntryIndex >= numberOfEntries )
258          currentEntryIndex = 0;
259
260       indexTextField.setText( "" + ( currentEntryIndex + 1 ) );
261       indexTextFieldActionPerformed( evt );
262    } // end method nextButtonActionPerformed
263
264    // handles call when queryButton is clicked
265    private void queryButtonActionPerformed( ActionEvent evt )
266    {
267       results =
268          personQueries.getPeopleByLastName( queryTextField.getText() );
269       numberOfEntries = results.size();
270
271       if ( numberOfEntries != 0 )
272       {
273          currentEntryIndex = 0;
274          currentEntry = results.get( currentEntryIndex );
275          idTextField.setText( "" + currentEntry.getAddressID() );
276          firstNameTextField.setText( currentEntry.getFirstName() );
277          lastNameTextField.setText( currentEntry.getLastName() );
278          emailTextField.setText( currentEntry.getEmail() );
279          phoneTextField.setText( currentEntry.getPhoneNumber() );
280          maxTextField.setText( "" + numberOfEntries );
281          indexTextField.setText( "" + ( currentEntryIndex + 1 ) );
282          nextButton.setEnabled( true );
283          previousButton.setEnabled( true );
284       } // end if
```

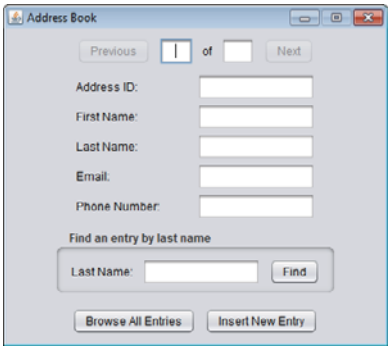
Fig. 28.32 | A simple address book. (Part 6 of 9.)

```
285        else
286           browseButtonActionPerformed( evt );
287     } // end method queryButtonActionPerformed
288
289     // handles call when a new value is entered in indexTextField
290     private void indexTextFieldActionPerformed( ActionEvent evt )
291     {
292        currentEntryIndex =
293           ( Integer.parseInt( indexTextField.getText() ) - 1 );
294
295        if ( numberOfEntries != 0 && currentEntryIndex < numberOfEntries )
296        {
297           currentEntry = results.get( currentEntryIndex );
298           idTextField.setText("" + currentEntry.getAddressID() );
299           firstNameTextField.setText( currentEntry.getFirstName() );
300           lastNameTextField.setText( currentEntry.getLastName() );
301           emailTextField.setText( currentEntry.getEmail() );
302           phoneTextField.setText( currentEntry.getPhoneNumber() );
303           maxTextField.setText( "" + numberOfEntries );
304           indexTextField.setText( "" + ( currentEntryIndex + 1 ) );
305        } // end if
306      } // end method indexTextFieldActionPerformed
307
308     // handles call when browseButton is clicked
309     private void browseButtonActionPerformed( ActionEvent evt )
310     {
311        try
312        {
313           results = personQueries.getAllPeople();
314           numberOfEntries = results.size();
315
316           if ( numberOfEntries != 0 )
317           {
318              currentEntryIndex = 0;
319              currentEntry = results.get( currentEntryIndex );
320              idTextField.setText( "" + currentEntry.getAddressID() );
321              firstNameTextField.setText( currentEntry.getFirstName() );
322              lastNameTextField.setText( currentEntry.getLastName() );
323              emailTextField.setText( currentEntry.getEmail() );
324              phoneTextField.setText( currentEntry.getPhoneNumber() );
325              maxTextField.setText( "" + numberOfEntries );
326              indexTextField.setText( "" + ( currentEntryIndex + 1 ) );
327              nextButton.setEnabled( true );
328              previousButton.setEnabled( true );
329           } // end if
330        } // end try
331        catch ( Exception e )
332        {
333           e.printStackTrace();
334        } // end catch
335     } // end method browseButtonActionPerformed
336
```

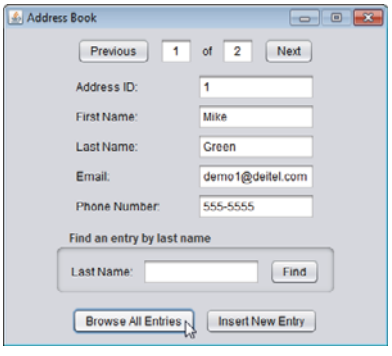
Fig. 28.32 | A simple address book. (Part 7 of 9.)

```
337    // handles call when insertButton is clicked
338    private void insertButtonActionPerformed( ActionEvent evt )
339    {
340       int result = personQueries.addPerson( firstNameTextField.getText(),
341          lastNameTextField.getText(), emailTextField.getText(),
342          phoneTextField.getText() );
343
344       if ( result == 1 )
345          JOptionPane.showMessageDialog( this, "Person added!",
346             "Person added", JOptionPane.PLAIN_MESSAGE );
347       else
348          JOptionPane.showMessageDialog( this, "Person not added!",
349             "Error", JOptionPane.PLAIN_MESSAGE );
350
351       browseButtonActionPerformed( evt );
352    } // end method insertButtonActionPerformed
353
354    // main method
355    public static void main( String args[] )
356    {
357       new AddressBookDisplay();
358    } // end method main
359 } // end class AddressBookDisplay
```

a) Initial **Address Book** screen.



b) Results of clicking **Browse All Entries**.



c) Browsing to the next entry.



d) Finding entries with the last name `Green`.

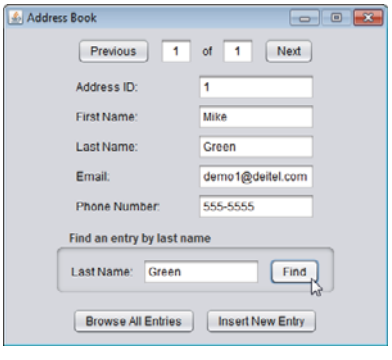


Fig. 28.32 | A simple address book. (Part 8 of 9.)

e) After adding a new entry and browsing to it.

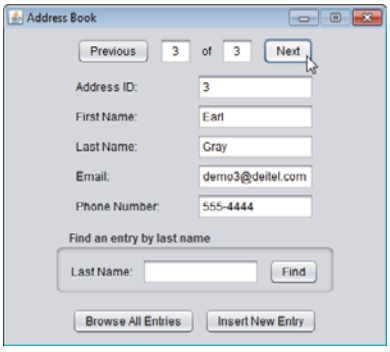


Fig. 28.32 | A simple address book. (Part 9 of 9.)

To add a new entry into the `AddressBook` database, the user can enter the first name, last name, email and phone number (the `AddressID` will *autoincrement*) in the `JTextFields` and press the **Insert New Entry** `JButton`. The `insertButtonActionPerformed` handler (lines 338–352) is called. Lines 340–342 call the method `addPerson` on the `PersonQueries` object to add a new entry to the database. Line 351 calls `browseButtonActionPerformed` to obtain the updated set of people in the address book and update the GUI accordingly.

The user can then view different entries by pressing the **Previous** `JButton` or **Next** `JButton`, which results in calls to methods `previousButtonActionPerformed` (lines 241–250) or `nextButtonActionPerformed` (lines 253–262), respectively. Alternatively, the user can enter a number in the `indexTextField` and press *Enter* to view a particular entry. This results in a call to method `indexTextFieldActionPerformed` (lines 290–306) to display the specified record.

28.12 Stored Procedures

Many database management systems can store individual or sets of SQL statements in a database, so that programs accessing that database can invoke them. Such named collections of SQL statements are called **stored procedures**. JDBC enables programs to invoke stored procedures using objects that implement the interface **`CallableStatement`**. `CallableStatements` can receive arguments specified with the methods inherited from interface `PreparedStatement`. In addition, `CallableStatements` can specify **output parameters** in which a stored procedure can place return values. Interface `CallableStatement` includes methods to specify which parameters in a stored procedure are output parameters. The interface also includes methods to obtain the values of output parameters returned from a stored procedure.



Portability Tip 28.6

Although the syntax for creating stored procedures differs across database management systems, the interface `CallableStatement` provides a uniform interface for specifying input and output parameters for stored procedures and for invoking stored procedures.



Portability Tip 28.7

According to the Java API documentation for interface `CallableStatement`*, for maximum portability between database systems, programs should process the update counts (which indicate how many rows were updated) or* `ResultSet`*s returned from a* `CallableStatement` *before obtaining the values of any output parameters.*

28.13 Transaction Processing

Many database applications require guarantees that a series of database insertions, updates and deletions executes properly before the application continues processing the next database operation. For example, when you transfer money electronically between bank accounts, several factors determine if the transaction is successful. You begin by specifying the source account and the amount you wish to transfer from that account to a destination account. Next, you specify the destination account. The bank checks the source account to determine whether its funds are sufficient to complete the transfer. If so, the bank withdraws the specified amount and, if all goes well, deposits it into the destination account to complete the transfer. What happens if the transfer fails after the bank withdraws the money from the source account? In a proper banking system, the bank redeposits the money in the source account. How would you feel if the money was subtracted from your source account and the bank *did not* deposit the money in the destination account?

Transaction processing enables a program that interacts with a database to *treat a database operation (or set of operations) as a single operation*. Such an operation also is known as an **atomic operation** or a **transaction**. At the end of a transaction, a decision can be made either to **commit the transaction** or **roll back the transaction**. Committing the transaction finalizes the database operation(s); all insertions, updates and deletions performed as part of the transaction cannot be reversed without performing a new database operation. Rolling back the transaction leaves the database in its state prior to the database operation. This is useful when a portion of a transaction fails to complete properly. In our bank-account-transfer discussion, the transaction would be rolled back if the deposit could not be made into the destination account.

Java provides transaction processing via methods of interface `Connection`. Method **`setAutoCommit`** specifies whether each SQL statement commits after it completes (a `true` argument) or whether several SQL statements should be grouped as a transaction (a `false` argument). If the argument to `setAutoCommit` is `false`, the program must follow the last SQL statement in the transaction with a call to `Connection` method **`commit`** (to commit the changes to the database) or `Connection` method **`rollback`** (to return the database to its state prior to the transaction). Interface `Connection` also provides method **`getAutoCommit`** to determine the autocommit state for the `Connection`.

28.14 Wrap-Up

In this chapter, you learned basic database concepts, how to query and manipulate data in a database using SQL and how to use JDBC to allow Java applications to interact with MySQL and Java DB databases. You learned about the SQL commands `SELECT`, `INSERT`, `UPDATE` and `DELETE`, as well as clauses such as `WHERE`, `ORDER BY` and `INNER JOIN`. You learned the steps for obtaining a `Connection` to the database, creating a `Statement` to interact with the database's data, executing the statement and processing the results. Then you used a

RowSet to simplify the process of connecting to a database and creating statements. You used PreparedStatements to create precompiled SQL statements. You also learned how to create and configure databases in both MySQL and Java DB by using predefined SQL scripts. We also provided overviews of CallableStatements and transaction processing. In the next chapter, you'll learn about web application development with JavaServer Faces.

28.15 Web Resources

`www.oracle.com/technetwork/java/javadb/overview/index.html`
Oracle Java DB home page.

`db.apache.org/derby/papers/DerbyTut/index.html`
Apache Derby tutorial. Includes Linux installation instructions.

`download.oracle.com/javase/tutorial/jdbc/index.html`
The Java Tutorial's JDBC track.

`www.sql.org`
This SQL portal provides links to many resources, including SQL syntax, tips, tutorials, books, magazines, discussion groups, companies with SQL services, SQL consultants and free software.

`download.oracle.com/javase/6/docs/technotes/guides/jdbc/index.html`
Oracle JDBC API documentation.

`www.mysql.com`
This site is the MySQL database home page. You can download the latest versions of MySQL and MySQL Connector/J and access their online documentation.

`dev.mysql.com/doc/refman/5.5/en/index.html`
MySQL reference manual.

`download.oracle.com/javase/6/docs/technotes/guides/jdbc/getstart/rowsetImpl.html`
Overviews the RowSet interface and its subinterfaces. This site also discusses the reference implementations of these interfaces from Sun and their usage.

Summary

Section 28.1 Introduction

- A database (p. 1172) is an integrated collection of data. A database management system (DBMS; p. 1172) provides mechanisms for storing, organizing, retrieving and modifying data.
- Today's most popular database management systems are relational database (p. 1173) systems.
- SQL (p. 1172) is the international standard language used to query (p. 1172) and manipulate relational data.
- Programs connect to, and interact with, relational databases via an interface—software that facilitates communications between a database management system and a program.
- A JDBC driver (p. 1172) enables Java applications to connect to a database in a particular DBMS and allows you to retrieve and manipulate database data.

Section 28.2 Relational Databases

- A relational database (p. 1173) stores data in tables (p. 1173). Tables are composed of rows (p. 1173), and rows are composed of columns in which values are stored.
- A table's primary key (p. 1173) provides a unique value that cannot be duplicated among rows.
- Each column (p. 1173) of a table represents a different attribute.

- The primary key can be composed of more than one column.
- Every column in a primary key must have a value, and the value of the primary key must be unique. This is known as the Rule of Entity Integrity (p. 1177).
- A one-to-many relationship (p. 1177) between tables indicates that a row in one table can have many related rows in a separate table.
- A foreign key (p. 1175) is a column in a table that must match the primary-key column in another table. This is known as the Rule of Referential Integrity (p. 1175).
- Foreign keys enable information from multiple tables to be joined together. There's a one-to-many relationship between a primary key and its corresponding foreign key.

Section 28.4.1 Basic SELECT Query

- The basic form of a query (p. 1172) is

```
* FROM tableName
```

  where the asterisk (`*`; p. 1178) indicates that all columns from *tableName* should be selected, and *tableName* specifies the table in the database from which rows will be retrieved.
- To retrieve specific columns, replace the `*` with a comma-separated list of column names.

Section 28.4.2 WHERE Clause

- The optional `WHERE` clause (p. 1179) in a query specifies the selection criteria for the query. The basic form of a query with selection criteria (p. 1178) is

```
SELECT columnName1, columnName2, ... FROM tableName WHERE criteria
```

- The `WHERE` clause can contain operators `<`, `>`, `<=`, `>=`, `=`, `<>` and `LIKE`. `LIKE` (p. 1179) is used for string pattern matching (p. 1179) with wildcard characters percent (`%`) and underscore (`_`).
- A percent character (`%`; p. 1179) in a pattern indicates that a string matching the pattern can have zero or more characters at the percent character's location in the pattern.
- An underscore (`_` ; p. 1179) in the pattern string indicates a single character at that position in the pattern.

Section 28.4.3 ORDER BY Clause

- A query's result can be sorted with the `ORDER BY` clause (p. 1181). The simplest form of an `ORDER BY` clause is

```
SELECT columnName1, columnName2, ... FROM tableName ORDER BY column ASC
SELECT columnName1, columnName2, ... FROM tableName ORDER BY column DESC
```

  where `ASC` specifies ascending order, `DESC` specifies descending order and *column* specifies the column on which the sort is based. The default sorting order is ascending, so `ASC` is optional.
- Multiple columns can be used for ordering purposes with an `ORDER BY` clause of the form

```
ORDER BY column1 sortingOrder, column2 sortingOrder, ...
```

- The `WHERE` and `ORDER BY` clauses can be combined in one query. If used, `ORDER BY` must be the last clause in the query.

Section 28.4.4 Merging Data from Multiple Tables: INNER JOIN

- An `INNER JOIN` (p. 1183) merges rows from two tables by matching values in columns that are common to the tables. The basic form for the `INNER JOIN` operator is:

```
SELECT columnName1, columnName2, ...
FROM table1
INNER JOIN table2
   ON table1.columnName = table2.columnName
```

The ON clause (p. 1183) specifies the columns from each table that are compared to determine which rows are joined. If a SQL statement uses columns with the same name from multiple tables, the column names must be fully qualified (p. 1183) by prefixing them with their table names and a dot (.).

Section 28.4.5 INSERT Statement

- An INSERT statement (p. 1184) inserts a new row into a table. The basic form of this statement is

```
INSERT INTO tableName ( columnName1, columnName2, ..., columnNameN )
   VALUES ( value1, value2, ..., valueN )
```

  where *tableName* is the table in which to insert the row. The *tableName* is followed by a comma-separated list of column names in parentheses. The list of column names is followed by the SQL keyword VALUES (p. 1184) and a comma-separated list of values in parentheses.
- SQL uses single quotes (') to delimit strings. To specify a string containing a single quote in SQL, escape the single quote with another single quote (i.e., '').

Section 28.4.6 UPDATE Statement

- An UPDATE statement (p. 1185) modifies data in a table. The basic form of an UPDATE statement is

```
UPDATE tableName
   SET columnName1 = value1, columnName2 = value2, ..., columnNameN = valueN
   WHERE criteria
```

  where *tableName* is the table to update. Keyword SET (p. 1185) is followed by a comma-separated list of *columnName* = *value* pairs. The optional WHERE clause determines which rows to update.

Section 28.4.7 DELETE Statement

- A DELETE statement (p. 1186) removes rows from a table. The simplest form for a DELETE statement is

```
DELETE FROM tableName WHERE criteria
```

  where *tableName* is the table from which to delete a row (or rows). The optional WHERE *criteria* determines which rows to delete. If this clause is omitted, all the table's rows are deleted.

Section 28.8.1 Connecting to and Querying a Database

- Package java.sql contains classes and interfaces for accessing relational databases in Java.
- A Connection object (p. 1191) manages the connection between a Java program and a database. Connection objects enable programs to create SQL statements that access data.
- DriverManager (p. 1191) method getConnection (p. 1191) attempts to connect to a database at a URL that specifies the protocol for communication, the subprotocol (p. 1191) for communication and the database name.
- Connection method createStatement (p. 1192) creates a Statement object (p. 1192), which can be used to submit SQL statements to the database.
- Statement method executeQuery (p. 1192) executes a query and returns a ResultSet object (p. 1192). ResultSet methods enable a program to manipulate query results.
- A ResultSetMetaData object (p. 1192) describes a ResultSet's contents. Programs can use metadata programmatically to obtain information about the ResultSet column names and types.
- ResultSetMetaData method getColumnCount (p. 1192) retrieves the number of ResultSet columns.
- ResultSet method next (p. 1193) positions the ResultSet cursor to the next row and returns true if the row exists; otherwise, it returns false. This method must be called to begin processing a ResultSet because the cursor is intially positioned before the first row.

- It's possible to extract each `ResultSet` column as a specific Java type. `ResultSetMetaData` method `getColumnType` (p. 1193) returns a `Types` (p. 1193) constant (package `java.sql`) indicating the column's type.
- `ResultSet` *get* methods typically receive as an argument either a column number (as an `int`) or a column name (as a `String`) indicating which column's value to obtain.
- `ResultSet` row and column numbers start at 1.
- Each `Statement` object can open only one `ResultSet` at a time. When a `Statement` returns a new `ResultSet`, the `Statement` closes the prior `ResultSet`.
- `Connection` method `createStatement` has an overloaded version that receives the result type and concurrency. The result type specifies whether the `ResultSet`'s cursor is able to scroll in both directions or forward only and whether the `ResultSet` is sensitive to changes. The result concurrency (p. 1199) specifies whether the `ResultSet` can be updated.
- Some JDBC drivers (p. 1172) do not support scrollable or updatable `ResultSet`s.

Section 28.8.2 Querying the `books` *Database*

- `TableModel` (p. 1194) method `getColumnClass` (p. 1194) returns a `Class` object that represents the superclass of all objects in a particular column. A `JTable` (p. 1194) uses this information to set up the default cell renderer and cell editor for that column in a `JTable`.
- `ResultSetMetaData` method `getColumnClassName` (p. 1200) obtains a column's fully qualified class name (p. 1183).
- `TableModel` method `getColumnCount` (p. 1194) returns the number of columns in the `ResultSet`.
- `TableModel` method `getColumnName` (p. 1194) returns the column name in the `ResultSet`.
- `ResultSetMetaData` method `getColumnName` (p. 1200) obtains a column's name from the `ResultSet`.
- `TableModel` method `getRowCount` (p. 1194) returns the number of rows in the model's `ResultSet`.
- `TableModel` method `getValueAt` (p. 1194) returns the `Object` at a particular row and column of the model's underlying `ResultSet`.
- `ResultSet` method `absolute` (p. 1200) positions the `ResultSet` cursor at a specific row.
- `AbstractTableModel` (p. 1194) method `fireTableStructureChanged` (p. 1200) notifies any `JTable` using a particular `TableModel` object as its model that the data in the model has changed.

Section 28.9 `RowSet` *Interface*

- Interface `RowSet` (p. 1207) configures a database connection and executes a query automatically.
- A connected `RowSet` (p. 1207) remains connected to the database while the object is in use. A disconnected `RowSet` (p. 1207) connects, executes a query, then closes the connection.
- `JdbcRowSet` (p. 1207), a connected `RowSet`, wraps a `ResultSet` object and allows you to scroll and update its rows. Unlike a `ResultSet` object, a `JdbcRowSet` object is scrollable and updatable by default.
- `CachedRowSet` (p. 1207), a disconnected `RowSet`, caches a `ResultSet`'s data in memory. A `CachedRowSet` is scrollable and updatable. A `CachedRowSet` is also serializable.

Section 28.10 Java DB/Apache Derby

- Java DB (p. 1209) has both an embedded version and a network version.

Section 28.11 `PreparedStatement`*s*

- `PreparedStatement`s (p. 1211) are compiled, so they execute more efficiently than `Statement`s.

- `PreparedStatements` can have parameters, so the same query can execute with different arguments.
- A parameter is specified with a question mark (`?`) in the SQL statement. Before executing a `PreparedStatement`, you must use `PreparedStatement`'s *set* methods to specify the arguments.
- `PreparedStatement` method `setString`'s (p. 1211) first argument represents the parameter number being set and the second argument is that parameter's value.
- Parameter numbers are counted from 1, starting with the first question mark (`?`).
- Method `setString` automatically escapes `String` parameter values as necessary.
- Interface `PreparedStatement` provides set methods for each supported SQL type.
- An identity column is the SQL standard way to represent an autoincremented (p. 1174) column. The SQL `IDENTITY` keyword (p. 1212) marks a column as an identity column.

Section 28.12 Stored Procedures

- JDBC enables programs to invoke stored procedures (p. 1226) using `CallableStatement` (p. 1226) objects.
- `CallableStatement` can specify input parameters. `CallableStatement` can specify output parameters (p. 1226) in which a stored procedure can place return values.

Section 28.13 Transaction Processing

- Transaction processing (p. 1227) enables a program that interacts with a database to treat a database operation (or set of operations) as a single operation—known as an atomic operation (p. 1227) or a transaction (p. 1227).
- At the end of a transaction, a decision can be made to either commit or roll back the transaction.
- Committing a transaction (p. 1227) finalizes the database operation(s)—inserts, updates and deletes cannot be reversed without performing a new database operation.
- Rolling back a transaction (p. 1227) leaves the database in its state prior to the database operation.
- Java provides transaction processing via methods of interface `Connection`.
- Method `setAutoCommit` (p. 1227) specifies whether each SQL statement commits after it completes (a `true` argument) or whether several SQL statements should be grouped as a transaction.
- When autocommit is disabled, the program must follow the last SQL statement in the transaction with a call to `Connection` method `commit` (to commit the changes to the database; p. 1227) or `Connection` method `rollback` (to return the database to its state prior to the transaction; p. 1227).
- Method `getAutoCommit` (p. 1227) determines the autocommit state for the `Connection`.

Self-Review Exercise

28.1 Fill in the blanks in each of the following statements:

a) The international standard database language is _____.
b) A table in a database consists of _____ and _____.
c) Statement objects return SQL query results as _____ objects.
d) The _____ uniquely identifies each row in a table.
e) SQL keyword _____ is followed by the selection criteria that specify the rows to select in a query.
f) SQL keywords _____ specify the order in which rows are sorted in a query.
g) Merging rows from multiple database tables is called _____ the tables.
h) A(n) _____ is an organized collection of data.
i) A(n) _____ is a set of columns whose values match the primary-key values of another table.

j) _____ method _____ is used to obtain a `Connection` to a database.
k) Interface _____ helps manage the connection between a Java program and a database.
l) A(n) _____ object is used to submit a query to a database.
m) Unlike a `ResultSet` object, _____ and _____ objects are scrollable and updatable by default.
n) _____, a disconnected `RowSet`, caches the data of a `ResultSet` in memory.

Answers to Self-Review Exercise

28.1 a) SQL. b) rows, columns. c) `ResultSet`. d) primary key. e) `WHERE`. f) `ORDER BY`. g) joining. h) database. i) foreign key. j) `DriverManager`, `getConnection`. k) `Connection`. l) `Statement`. m) `JdbcRowSet`, `CachedRowSet` n) `CachedRowSet`.

Exercises

28.2 *(Query Application for the `books` Database)* Using the techniques shown in this chapter, define a complete query application for the `books` database. Provide the following predefined queries:

a) Select all authors from the `Authors` table.
b) Select a specific author and list all books for that author. Include each book's title, year and ISBN. Order the information alphabetically by the author's last then first name.
c) Select a specific title and list all authors for that title. Order the authors alphabetically by last name then by first name.
d) Provide any other queries you feel are appropriate.

Display a `JComboBox` with appropriate names for each predefined query. Also allow users to supply their own queries.

28.3 *(Data Manipulation Application for the `books` Database)* Define a data-manipulation application for the `books` database. The user should be able to edit existing data and add new data to the database (obeying referential and entity integrity constraints). Allow the user to edit the database in the following ways:

a) Add a new author.
b) Edit the existing information for an author.
c) Add a new title for an author. (Remember that the book must have an entry in the `AuthorISBN` table.).
d) Add a new entry in the `AuthorISBN` table to link authors with titles.

28.4 *(Employee Database)* In Section 10.7, we introduced an employee-payroll hierarchy to calculate each employee's payroll. In this exercise, we provide a database of employees that corresponds to the employee-payroll hierarchy. (A SQL script to create the `employees` MySQL database is provided with the examples for this chapter.) Write an application that allows the user to:

a) Add employees to the `employee` table.
b) Add payroll information to the appropriate table for each new employee. For example, for a salaried employee add the payroll information to the `salariedEmployees` table.

Figure 28.33 is the entity-relationship diagram for the `employees` database.

28.5 *(Employee Database Query Application)* Modify Exercise 28.4 to provide a `JComboBox` and a `JTextArea` to allow the user to perform a query that is either selected from the `JComboBox` or defined in the `JTextArea`. Sample predefined queries are:

a) Select all employees working in Department SALES.
b) Select hourly employees working over 30 hours.
c) Select all commission employees in descending order of the commission rate.

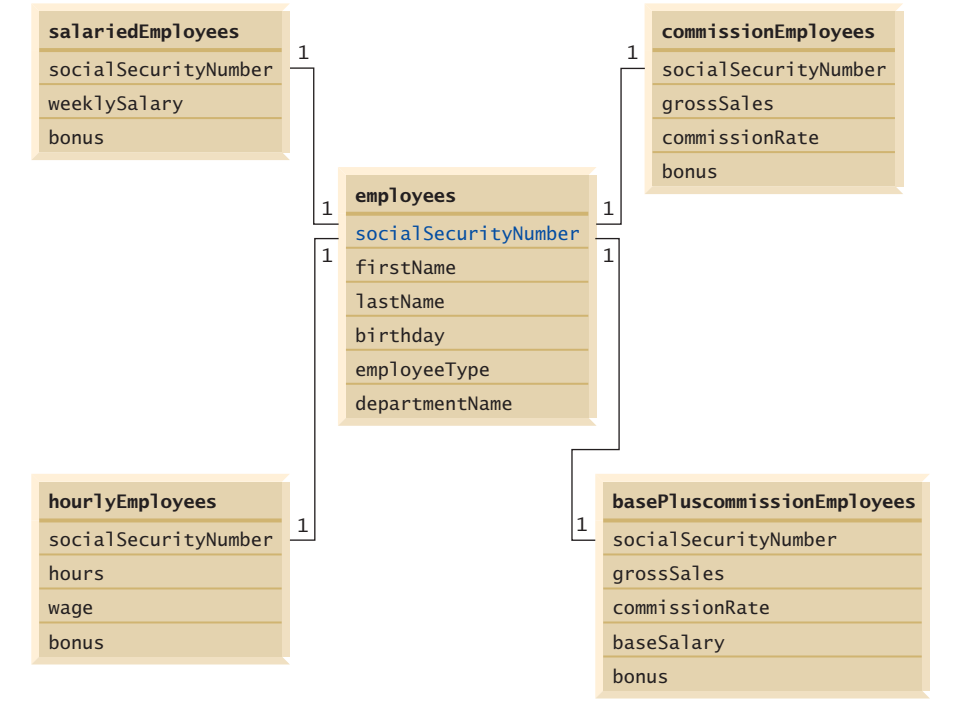


Fig. 28.33 | Table relationships in the `employees` database.

28.6 ***(Employee Database Data Manipulation Application)*** Modify Exercise 28.5 to perform the following tasks:

a) Increase base salary by 10% for all base-plus-commission employees.
b) If the employee's birthday is in the current month, add a $100 bonus.
c) For all commission employees with gross sales over $10,000, add a $100 bonus.

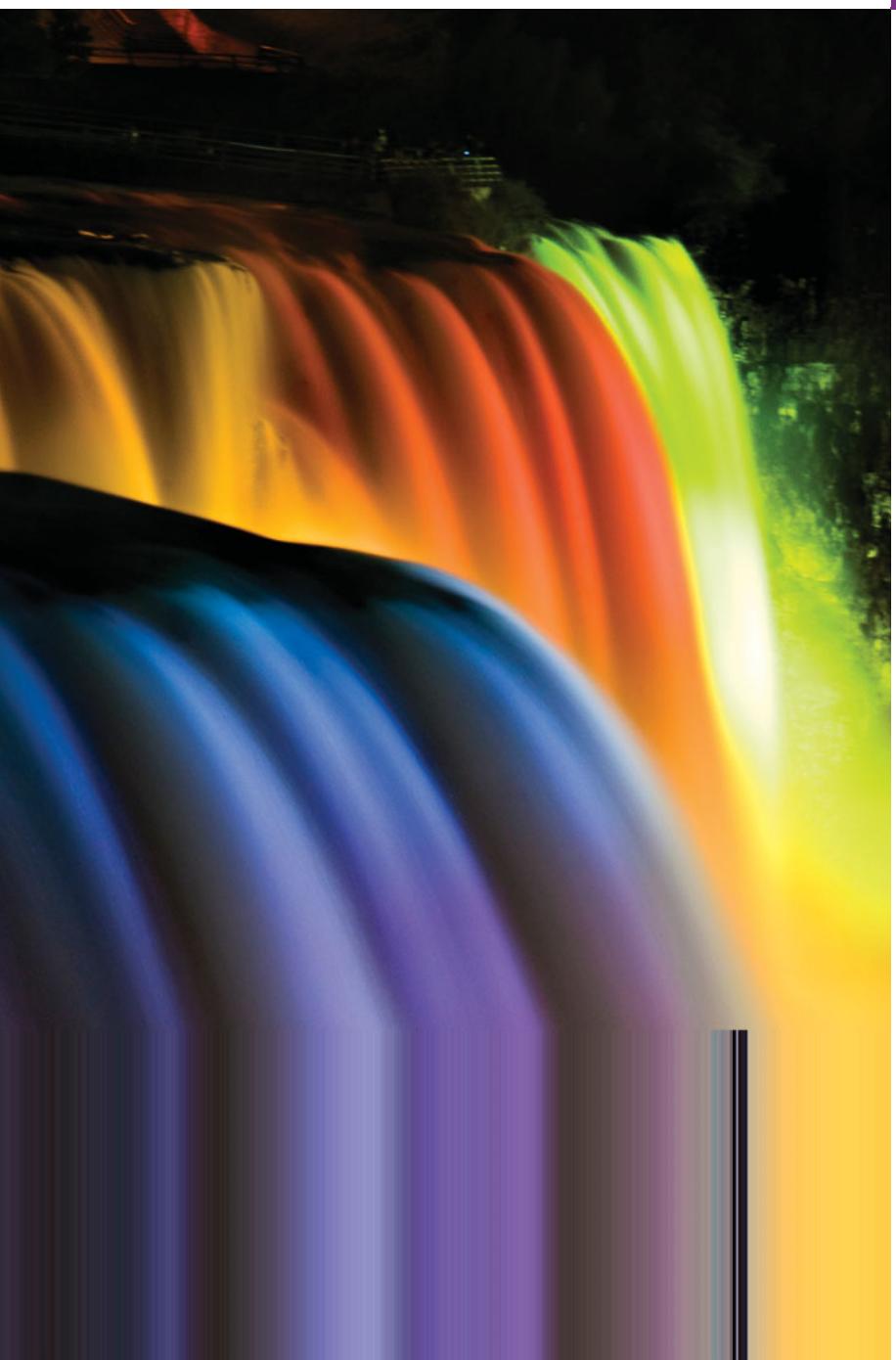
28.7 ***(Address Book Modification: Update an Existing Entry)*** Modify the program in Figs. 28.30–28.32 to provide a `JButton` that allows the user to call a method named `updatePerson` in `PersonQueries` class to update the current entry in the `AddressBook` database.

28.8 ***(Address Book Modification: Delete an Existing Entry)*** Modify the program of Exercise 28.7 to provide a `JButton` that allows the user to call a method named `deletePerson` in `PersonQueries` class to delete the current entry in the `AddressBook` database.

28.9 ***(Project: ATM Case Study with a Database)*** Modify the ATM Case Study (Chapters 12–13) to use an actual database to store the account information. We provide a SQL script to create a MySQL `BankDatabase`, which has a single table consisting of four columns—`AccountNumber` (an `int`), `PIN` (an `int`), `AvailableBalance` (a `double`) and `TotalBalance` (a `double`).

JavaServer™ Faces Web Apps: Part 1

29



If any man will draw up his case, and put his name at the foot of the first page, I will give him an immediate reply. Where he compels me to turn over the sheet, he must wait my leisure.
—Lord Sandwich

Rule One:
Our client is always right.
Rule Two: If you think our client is wrong, see Rule One.
—Anonymous

A fair question should be followed by a deed in silence.
—Dante Alighieri

You will come here and get books that will open your eyes, and your ears, and your curiosity, and turn you inside out or outside in.
—Ralph Waldo Emerson

Objectives

In this chapter you'll learn:

- To create JavaServer Faces web apps.
- To create web apps consisting of multiple pages.
- To validate user input on a web page.
- To maintain user-specific state information throughout a web app with session tracking.

Outline

29.1 Introduction
29.2 HyperText Transfer Protocol (HTTP) Transactions
29.3 Multitier Application Architecture
29.4 Your First JSF Web App
- 29.4.1 The Default `index.xhtml` Document: Introducing Facelets
- 29.4.2 Examining the `WebTimeBean` Class
- 29.4.3 Building the `WebTime` JSF Web App in NetBeans

29.5 Model-View-Controller Architecture of JSF Apps
29.6 Common JSF Components
29.7 Validation Using JSF Standard Validators
29.8 Session Tracking
- 29.8.1 Cookies
- 29.8.2 Session Tracking with `@SessionScoped` Beans

29.9 Wrap-Up

Summary | *Self-Review Exercises* | *Answers to Self-Review Exercises* | *Exercises*

29.1 Introduction

In this chapter, we introduce web app development in Java with JavaServer Faces (JSF). Web-based apps create content for web browser clients. This content includes eXtensible HyperText Markup Language (XHTML), JavaScript client-side scripting, Cascading Style Sheets (CSS), images and binary data. XHTML is an XML (eXtensible Markup Language) vocabulary that is based on HTML (HyperText Markup Language). We discuss only the features of these technologies that are required to understand the examples in this chapter. If you'd like more information on XHTML, XML, JavaScript and CSS, please visit our Resource Centers on each of these topics at

```
www.deitel.com/ResourceCenters.html
```

where you'll find links to introductions, tutorials and other valuable resources.

This chapter begins with an overview of how interactions between a web browser and web server work. We then present several web apps implemented with JSF. We continue this discussion in Chapter 30 with more advanced web applications.

Java multitier applications are typically implemented using Java Enterprise Edition (Java EE). The technologies we use to develop web apps here and in Chapter 30 are part of Java EE 6 (`www.oracle.com/technetwork/java/javaee/overview/index.html`). After you study this chapter and the next, you can learn more about JavaServer Faces 2.0 in Oracle's extensive Java EE 6 tutorial at `download.oracle.com/javaee/6/tutorial/doc/`.

We focus on the JavaServer Faces 2.0[1] subset of Java EE. JavaServer Faces is a **web-application framework** that enables you to build multitier web apps by extending the framework with your application-specific capabilities. The framework handles the details of receiving client requests and returning responses for you so that you can focus on your application's functionality.

Required Software for This Chapter

To work with and implement the examples in this chapter and Chapters 30–31, you must install the **NetBeans 6.9.1** IDE and the **GlassFish 3.0.1** open-source application server. Both are available in a bundle from `netbeans.org/downloads/index.html`. You're prob-

1. The JavaServer Faces Specification: `http://bit.ly/JSF20Spec`.

ably using a computer with the Windows, Linux or Max OS X operating system—installers are provided for each of these platforms. Download and execute the installer for the **Java** or **All** version—both include the required **Java Web and EE** and **Glassfish Server Open Source Edition** options. We assume you use the default installation options for your platform. Once you've installed NetBeans, run it. Then, use the **Help** menu's **Check for Updates** option to make sure you have the most up-to-date components.

29.2 HyperText Transfer Protocol (HTTP) Transactions

To learn how JSF web apps work, it's important to understand the basics of what occurs behind the scenes when a user requests a web page in a web browser. If you're already familiar with this and with multitier application architecture, you can skip to Section 29.4.

XHTML Documents

In its simplest form, a web page is nothing more than an XHTML document (also called an XHTML page) that describes content to display in a web browser. HTML documents normally contain *hyperlinks* that link to different pages or to other parts of the same page. When the user clicks a hyperlink, the requested web page loads into the user's web browser. Similarly, the user can type the address of a page into the browser's address field.

URLs

Computers that run **web-server** software make resources available, such as web pages, images, PDF documents and even objects that perform complex tasks such as database lookups and web searches. The HyperText Transfer Protocol (HTTP) is used by web browsers to communicate with web servers, so they can exchange information in a uniform and reliable manner. URLs (Uniform Resource Locators) identify the locations on the Internet of resources, such as those mentioned above. If you know the URL of a publicly available web resource, you can access it through HTTP.

Parts of a URL

When you enter a URL into a web browser, the browser uses the information in the URL to locate the web server that contains the resource and to request that resource from the server. Let's examine the components of the URL

```
http://www.deitel.com/books/downloads.html
```

The `http://` indicates that the resource is to be obtained using the HTTP protocol. The next portion, `www.deitel.com`, is the server's fully qualified **hostname**—the name of the *server* on which the *resource* resides. The computer that houses and maintains resources is usually is referred to as the **host**. The hostname `www.deitel.com` is translated into an **IP (Internet Protocol) address**—a unique numerical value that identifies the server, much as a telephone number uniquely defines a particular phone line. This translation is performed by a **domain-name system (DNS) server**—a computer that maintains a database of hostnames and their corresponding IP addresses—and the process is called a **DNS lookup**. To test web apps, you'll often use your computer as the host. This host is referred to using the reserved domain name `localhost`, which translates to the IP address `127.0.0.1`. The fully qualified hostname can be followed by a colon (`:`) and a port number. Web servers typically await requests on port 80 by default; however, many development web servers use a different port number, such as 8080—as you'll see in Section 29.4.3.

The remainder of the URL (i.e., `/books/downloads.html`) specifies both the name of the requested resource (the HTML document `downloads.html`) and its path, or location (`/books`), on the web server. The path could specify the location of an actual directory on the web server's file system. For security reasons, however, the path normally specifies the location of a **virtual directory**. The server translates the virtual directory into a real location on the server (or on another computer on the server's network), thus hiding the resource's true location. Some resources are created dynamically using other information, such as data from a database.

Making a Request and Receiving a Response

When given a URL, a web browser performs an HTTP transaction to retrieve and display the web page at that address. Figure 29.1 illustrates the transaction, showing the interaction between the web browser (the client) and the web server (the server).

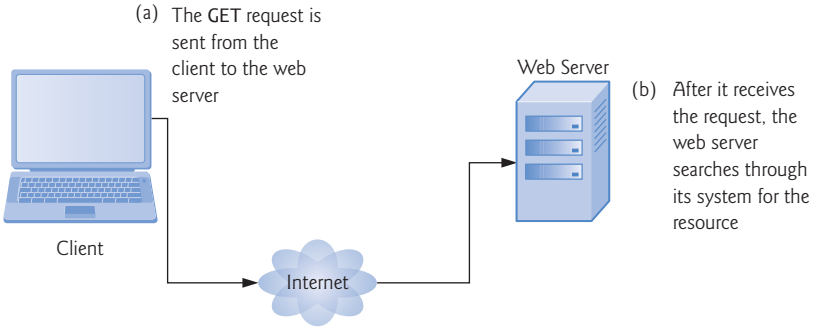


Fig. 29.1 | Client interacting with the web server. *Step 1:* The `GET` request.

In Fig. 29.1, the web browser sends an HTTP request to the server. Underneath the hood, the request (in its simplest form) is

```
GET /books/downloads.html HTTP/1.1
```

The word **`GET`** is an **HTTP method** indicating that the client wishes to obtain a resource from the server. The remainder of the request provides the path name of the resource (e.g., an HTML document) and the protocol's name and version number (`HTTP/1.1`). As part of the client request, the browser also sends other required and optional information, such as the `Host` (which identifies the server computer) or the `User-Agent` (which identifies the web browser type and version number).

Any server that understands HTTP (version 1.1) can translate this request and respond appropriately. Figure 29.2 depicts the server responding to a request.

The server first responds by sending a line of text that indicates the HTTP version, followed by a numeric code and a phrase describing the status of the transaction. For example,

```
HTTP/1.1 200 OK
```

indicates success, whereas

```
HTTP/1.1 404 Not found
```

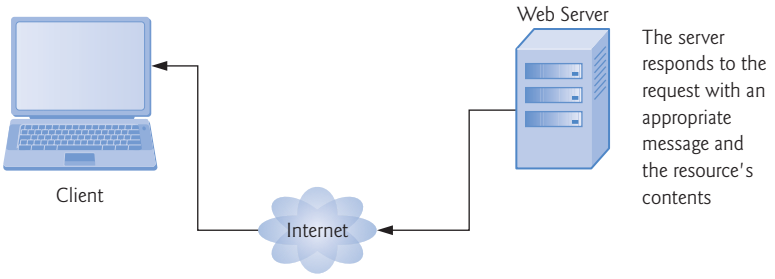


Fig. 29.2 | Client interacting with the web server. *Step 2:* The HTTP response.

informs the client that the web server could not locate the requested resource. On a successful request, the server appends the requested resource to the HTTP response. A complete list of numeric codes indicating the status of an HTTP transaction can be found at `www.w3.org/Protocols/rfc2616/rfc2616-sec10.html`.

HTTP Headers

The server then sends one or more **HTTP headers**, which provide additional information about the data that will be sent. If the server is sending an HTML text document, one HTTP header would read:

```
Content-type: text/html
```

The information provided in this header specifies the **Multipurpose Internet Mail Extensions (MIME)** type of the content that the server is transmitting to the browser. MIME is an Internet standard that specifies *data formats* so that programs can interpret data correctly. For example, the MIME type `text/plain` indicates that the sent information is text that can be displayed directly, without any interpretation of the content as HTML markup. Similarly, the MIME type `image/jpeg` indicates that the content is a JPEG image. When the browser receives this MIME type, it attempts to display the image. For a list of available MIME types, visit `www.w3schools.com/media/media_mimeref.asp`.

The header or set of headers is followed by a blank line, which indicates to the client browser that the server is finished sending HTTP headers. The server then sends the contents of the requested resource (such as, `downloads.html`). In the case of an HTML document, the web browser parses the HTML markup it receives and **renders** (or displays) the results.

HTTP `GET` and `POST` Requests

The two most common **HTTP request types** (also known as **request methods**) are `GET` and `POST`. A `GET` request typically asks for a resource on a server. Common uses of `GET` requests are to retrieve an HTML document or an image or to fetch search results from a search engine based on a user-submitted search term. A **`POST`** request typically sends data to a server. Common uses of `POST` requests are to send form data or documents to a server.

When a web page contains an HTML form in which the user can enter data, an HTTP request typically posts that data to a **server-side form handler** for processing. For example, when a user performs a search or participates in a web-based survey, the web server receives the information specified in the form as part of the request.

GET requests and POST requests can both send form data to a web server, yet each request type sends the information differently. A GET request sends information to the server in the URL, as in www.google.com/search?q=deitel. Here, search is the name of Google's server-side form handler, q is the name of a variable in Google's search form and deitel is the search term. A ? separates the **query string** from the rest of the URL in a request. A *name/value* pair is passed to the server with the *name* and the *value* separated by an equals sign (=). If more than one *name/value* pair is submitted, each is separated from the next by an ampersand (&). The server uses data passed in a query string to retrieve an appropriate resource. The server then sends a **response** to the client. A GET request may be initiated by submitting an HTML form whose method attribute is set to "get", by typing the URL (possibly containing a query string) directly into the browser's address bar or through a hyperlink when the user clicks the link.

A POST request sends form data as part of the HTTP message, not as part of the URL. The specification for GET requests does not limit the query string's number of characters, but some web browsers do—for example, Internet Explorer restricts the length to 2083 characters), so it's often necessary to send large pieces of information using POST. Sometimes POST is preferred because it hides the submitted data from the user by embedding it in an HTTP message.



Software Engineering Observation 29.1

The data sent in a POST request is not *part of the URL, and the user can't see the data by default. However, tools are available that expose this data, so you should not assume that the data is secure just because a POST request is used.*

Client-Side Caching

Browsers often **cache** (save on disk) web pages for quick reloading. If there are no changes between the version stored in the cache and the current version on the web, the browser uses the cached copy to speed up your browsing experience. An HTTP response can indicate the length of time for which the content remains "fresh." If this amount of time has not been reached, the browser can avoid another request to the server. Otherwise, the browser requests the document from the server. Thus, the browser minimizes the amount of data that must be downloaded for you to view a web page. Browsers typically do not cache the server's response to a POST request, because the next POST might not return the same result. For example, in a survey, many users could visit the same web page and answer a question. The survey results could then be displayed for the user. Each new answer changes the survey results.

When you use a web-based search engine, the browser normally supplies the information you specify in an HTML form to the search engine with a GET request. The search engine performs the search, then returns the results to you as a web page. Such pages are sometimes cached by the browser in case you perform the same search again.

29.3 Multitier Application Architecture

Web apps are **multitier applications** (sometimes referred to as ***n*-tier applications**). Multitier applications divide functionality into separate **tiers** (i.e., logical groupings of functionality). Although tiers can be located on the same computer, the tiers of web apps often reside on separate computers. Figure 29.3 presents the basic structure of a three-tier web app.

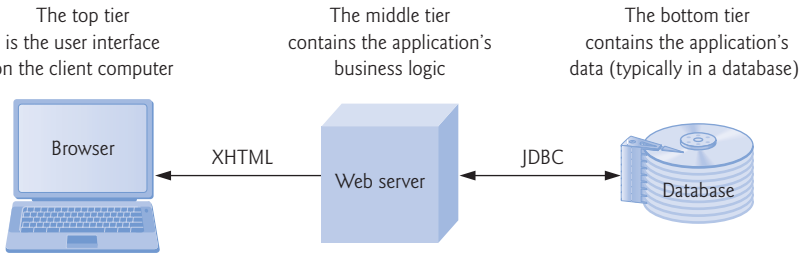


Fig. 29.3 | Three-tier architecture.

The **information tier** (also called the **data tier** or the **bottom tier**) maintains data pertaining to the application. This tier typically stores data in a *relational database management system* (*RDBMS*). We discussed RDBMSs in Chapter 28. For example, a retail store might have a database for storing product information, such as descriptions, prices and quantities in stock. The same database also might contain customer information, such as user names, billing addresses and credit card numbers. This tier can contain multiple databases, which together comprise the data needed for our application.

The **middle tier** implements **business logic**, **controller logic** and **presentation logic** to control interactions between the application's clients and the application's data. The middle tier acts as an intermediary between data in the information tier and the application's clients. The middle-tier controller logic processes client requests (such as requests to view a product catalog) and retrieves data from the database. The middle-tier presentation logic then processes data from the information tier and presents the content to the client. Web apps typically present data to clients as HTML documents.

Business logic in the middle tier enforces **business rules** and ensures that data is reliable before the server application updates the database or presents the data to users. Business rules dictate how clients can and cannot access application data, and how applications process data. For example, a business rule in the middle tier of a retail store's web app might ensure that all product quantities remain positive. A client request to set a negative quantity in the bottom tier's product-information database would be rejected by the middle tier's business logic.

The **client tier**, or **top tier**, is the application's user interface, which gathers input and displays output. Users interact directly with the application through the user interface (typically viewed in a web browser), keyboard and mouse. In response to user actions (e.g., clicking a hyperlink), the client tier interacts with the middle tier to make requests and to retrieve data from the information tier. The client tier then displays the data retrieved from the middle tier to the user. The client tier never directly interacts with the information tier.

29.4 Your First JSF Web App

Let's begin with a simple example. Figure 29.4 shows the output of our `WebTime` app. When you invoke this app from a web browser, the browser requests the app's default JSF page. The web server receives this request and passes it to the **JSF web-application framework** for processing. This framework is available in any Java EE 6-compliant application server (such as the GlassFish application server used in this chapter) or any JavaServer

Faces 2.0-compliant container (such as Apache Tomcat). The framework includes the **Faces servlet**—a software component running on the server that processes each requested JSF page so that the server can eventually return a response to the client. In this example, the Faces servlet processes the JSF document in Fig. 29.5 and forms a response containing the text `"Current time on the web server:"` followed by the web server's local time. We demonstrate this chapter's examples on the GlassFish server that you installed with NetBeans locally on your computer.



Fig. 29.4 | Sample output of the `WebTime` app.

Executing the `WebTime` *App*

To run this example on your own computer, perform the following steps:

1. Open the NetBeans IDE.
2. Select **File > Open Project...** to display the **Open Project** dialog.
3. Navigate to the `ch29` folder in the book's examples and select `WebTime`.
4. Click the **Open Project** button.
5. Right click the project's name in the **Projects** tab (in the upper-left corner of the IDE, below the toolbar) and select **Run** from the pop-up menu.

This launches the GlassFish application server (if it isn't already running), installs the web app onto the server, then opens your computer's default web browser which requests the `WebTime` app's default JSF page. The browser should display a web page similar to that in Fig. 29.4.

29.4.1 The Default `index.xhtml` Document: Introducing Facelets

This app contains a single web page and consists of two related files—a JSF document named `index.xhtml` (Fig. 29.5) and a supporting Java source-code file (Fig. 29.6), which we discuss in Section 29.4.2. First we discuss the markup in `index.xhtml` and the supporting source code, then we provide step-by-step instructions for creating this web app in Section 29.4.3. Most of the markup in Fig. 29.5 was generated by NetBeans. We've reformatted the generated code to match our coding conventions used throughout the book.

```
1  <?xml version='1.0' encoding='UTF-8' ?>
2
3  <!-- index.xhtml -->
4  <!-- JSF page that displays the current time on the web server -->
```

Fig. 29.5 | JSF page that displays the current time on the web server. (Part 1 of 2.)

```
5  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
6     "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
7  <html xmlns="http://www.w3.org/1999/xhtml"
8     xmlns:h="http://java.sun.com/jsf/html">
9     <h:head>
10       <title>WebTime: A Simple Example</title>
11       <meta http-equiv="refresh" content="60" />
12    </h:head>
13    <h:body>
14       <h1>Current time on the web server: #{webTimeBean.time}</h1>
15    </h:body>
16 </html>
```

Fig. 29.5 | JSF page that displays the current time on the web server. (Part 2 of 2.)

Facelets: XHTML and JSF Markup

You present your web app's content in JSF using **Facelets**—a combination of XHTML markup and JSF markup. **XHTML**—the **Extensible HyperText Markup Language**—specifies the content of a web page that is displayed in a web browser. XHTML separates the **presentation** of a document (that is, the document's appearance when rendered by a browser) from the **structure** of the document's data. A document's presentation might specify where the browser should place an element in a web page or what fonts and colors should be used to display an element. The XHTML 1.0 Strict Recommendation allows only a document's structure to appear in a valid XHTML document, and not its presentation. Presentation is specified with Cascading Style Sheets (CSS). JSF uses the XHTML 1.0 Transitional Recommendation by default. Transitional markup may include some non-CSS formatting, but this is not recommended.

XML Declaration, Comments and the `DOCTYPE` Declaration

With the exception of lines 3–4, 10–11 and 14, the code shown in Fig. 29.5 was generated by NetBeans. Line 1 is an XML declaration, indicating that the JSF document is expressed in XML 1.0 syntax. Lines 3–4 are comments that we added to the document to indicate its file name and purpose. Lines 5–6 are a `DOCTYPE` declaration indicating the version of XHTML used in the markup. This can be used by a web browser to validate the syntax of the document.

Specifying the XML Namespaces Used in the Document

Line 7 begins the document's root `html` element, which spans lines 7–16. Each element typically consists of a starting and ending tag. The starting `<html>` tag (lines 7–8) may contain one or more `xmlns` attributes. Each **`xmlns` attribute** has a **name** and a **value** separated by an equal sign (=), and specifies an XML namespace of elements that are used in the document. Just as Java packages can be used to differentiate class names, XML namespaces can be used to differentiate sets of elements. When there's a naming conflict, fully qualified tag names can be used to resolve the conflict.

Line 7 specifies a required `xmlns` attribute and its value (`http://www.w3.org/1999/xhtml`) for the `html` element. This indicates that the `html` element and any other unqualified element names are part of the default XML namespace that's used in this document.

The `xmlns:h` attribute (line 8) specifies a prefix and a URL for JSF's **HTML Tag Library**, allowing the document to use JSF's elements from that library. A tag library defines

a set of elements that can be inserted into the XHTML markup. The elements in the HTML Tag Library generate XHTML elements. Based on line 7, each element we use from the HTML Tag Library must be preceded by the `h:` prefix. This tag library is one of several suppoorted by JSF that can be used to create Facelets pages. We'll discuss others as we use them. For a complete list of JSF tag libraries and their elements and attributes, visit

```
javaserverfaces.java.net/nonav/docs/2.0/pdldocs/facelets/
```

The `h:head` *and* `h:body` *Elements*

The **`h:head`** element (lines 9–12) defines the XHTML page's `head` element. In this example the `head` contains an HTML `title` element and a `meta` element. The document's `title` (line 10) typically appears in the browser window's title bar, or a browser tab if you have multiple web pages open in the browser at once. The `title` is also used when search engines index your web pages. The **`meta`** element (line 11) tells the browser to refresh the page every 60 seconds. This forces the browser to re-request the page once per minute.

The **`h:body`** element (lines 13–15) represent's the page's content. In this example, it contains a XHTML `h1` header element (line 14) that represents the text to display when this document is rendered in the web browser. The `h1` element contains some literal text (`Current time on the web server:`) that's simply placed into the response to the client and a **JSF Expression Language (EL)** expression that obtains a value dynamically and inserts it into the response. The expression

```
#{webTimeBean.time}
```

indicates that the web app has an object named `webTimeBean` which contains a property named `time`. The property's value replaces the expression in the response that's sent to the client. We'll discuss this EL expression in more detail shortly.

29.4.2 Examining the `WebTimeBean` Class

JSF documents typically interact with one or more Java objects to perform the app's tasks. As you saw, this example obtains the time on the server and sends it as part of the response.

JavaBeans

JavaBeans objects are instances of classes that follow certain conventions for class design. Each JavaBean class typically contains data and methods. A JavaBean exposes its data to a JSF document as **properties**. Depending on their use, these properties can be read/write, read-only or write-only. To define a read/write property, a JavaBean class provides *set* and *get* methods for that property. For example, to create a `String` property `firstName`, the class would provide methods with the following first lines:

```
public String getFirstName()
public void setFirstName( String name )
```

The fact that both method names contain "`FirstName`" with an uppercase "`F`" indicates that the class exposes a `firstName` property with a lowercase "`F`." This naming convention is part of the JavaBeans Specification (available at `bit.ly/JavaBeansSpecification`). A read-only property would have only a *get* method and a write-only property only a *set* method. The JavaBeans used in JSF are also **POJOs (plain old Java objects)**, meaning that—unlike prior versions of JSF—you do *not* need to extend a special class to create the beans used in JSF applications. Instead various annotations are used to "inject" function-

ality into your beans so they can be used easily in JSF applications. The JSF framework is responsible for creating and managing objects of your JavaBean classes for you—you'll see how to enable this momentarily.

Class `WebTimeBean`

Figure 29.6 presents the `WebTimeBean` class that allows the JSF document to obtain the web server's time. You can name your bean classes like any other class. We chose to end the class name with "`Bean`" to indicate that the class represents a JavaBean. The class contains just a `getTime` method (lines 13–17), which defines the read-only `time` property of the class. Recall that we access this property at line 14 of Fig. 29.5. Lines 15–16 create a `Date` object, then format and return the time as a `String`.

```
1  // WebTimeBean.java
2  // Bean that enables the JSF page to retrieve the time from the server
3  package webtime;
4
5  import java.text.DateFormat;
6  import java.util.Date;
7  import javax.faces.bean.ManagedBean;
8
9  @ManagedBean( name="webTimeBean" )
10 public class WebTimeBean
11 {
12    // return the time on the server at which the request was received
13    public String getTime()
14    {
15       return DateFormat.getTimeInstance( DateFormat.LONG ).format(
16          new Date() );
17    } // end method getTime
18 } // end class WebTimeBean
```

Fig. 29.6 | Bean that enables the JSF page to retrieve the time from the server.

The `@ManagedBean` *Annotation*

Line 9 uses the **`@ManagedBean` annotation** (from the package **`javax.faces.bean`**) to indicate that the JSF framework should create and manage the `WebTimeBean` object(s) used in the application. The parentheses following the annotation contain the optional **`name` attribute**—in this case, indicating that the bean object created by the JSF framework should be called `webTimeBean`. If you specify the annotation without the parentheses and the `name` attribute, the JSF framework will use the class name with a lowercase first letter (that is, `webTimeBean`) as the default bean name.

Processing the EL Expression

When the Faces servlet encounters an EL expression that accesses a bean property, it automatically invokes the property's *set* or *get* method based on the context in which the property is used. In line 14 of Fig. 29.5, accessing the property `webTimeBean.time` results in a call to the bean's `getTime` method, which returns the web server's time. If this bean object does not yet exist, the JSF framework instantiates it, then calls the `getTime` method on the bean object. The framework can also discard beans that are no longer being used. [*Note:* We discuss only the EL expressions that we use in this chapter. For more EL details,

see Chapter 6 of the Java EE 6 tutorial at `download.oracle.com/javaee/6/tutorial/doc/` and Chapter 5 of the JSF 2.0 specification at `bit.ly/JSF20Spec`.]

29.4.3 Building the `WebTime` JSF Web App in NetBeans

We'll now build the `WebTime` app from scratch using NetBeans.

Creating the JSF Web Application Project

Begin by opening the NetBeans IDE and performing the following steps:

1. Select **File > New Project...** to display the **New Project** dialog. Select **Java Web** in the **Categories** pane, **Web Application** in the **Projects** pane and click **Next >**.
2. In the dialog's **Name and Location** step, specify `WebTime` as the **Project Name**. In the **Project Location** field, specify where you'd like to store the project (or keep the default location). These settings will create a `WebTime` directory to store the project's files in the parent directory you specified. Keep the other default settings and click **Next >**.
3. In the dialog's **Server and Settings** step, specify **GlassFish Server 3** as the **Server** and **Java EE 6 Web** as the **Java EE Version** (these may be the default). Keep the default **Context Path** and click **Next >**.
4. In the dialog's **Frameworks** step, select **JavaServer Faces**, then click **Finish** to create the web application project.

Examining the NetBeans Projects Window

Figure 29.7 displays the **Projects** window, which appears in the upper-left corner of the IDE. This window displays the contents of the project. The app's XHTML documents are placed in the **Web Pages** node. NetBeans supplies the default web page `index.xhtml` that will be displayed when a user requests this web app from a browser. When you add Java source code to the project, it will be placed in the **Source Packages** node.

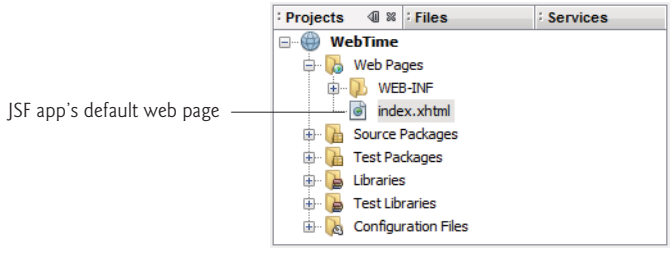


Fig. 29.7 | **Projects** window for the `WebTime` project.

Examining the Default `index.xhtml` Page

Figure 29.8 displays `index.xhtml`—the default page that will be displayed when a user requests this web app. We reformatted the code to match our coding conventions. When this file is first created, it contains elements for setting up the page, including linking to the page's style sheet and declaring the JSF libraries that will be used. By default, NetBeans

does not show line numbers in the source-code editor. To view the line numbers, select **View > Show Line Numbers**.



Fig. 29.8 | Default `index.xhtml` page generated by NetBeans for the web app.

Editing the `h:head` *Element's Contents*

Modify line 7 of Fig. 29.8 by changing the `title` element's content from `"Facelet Title"` to `"Web Time: A Simple Example"`. After the closing `</title>` tag, press *Enter*, then insert the `meta` element

```
<meta http-equiv="refresh" content="60" />
```

which will cause the browser to refresh this page once per minute. As you type, notice that NetBeans provides a code-completion window to help you write your code. For example, after typing "`<meta`" and a space, the IDE displays the code-completion window in Fig. 29.9, which shows the list of valid attributes for the starting tag of a `meta` element. You can then double click an item in the list to insert it into your code. Code-completion support is provided for XHTML elements, JSF elements and Java code.

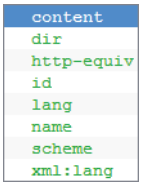


Fig. 29.9 | NetBeans code-completion window.

Editing the `h:body` *Element's Contents*

In the `h:body` element, replace `"Hello from Facelets"` with the `h1` header element

```
<h1>Current time on the web server: </h1>
```

Don't insert the expression `#{webTimeBean.time}` yet. After we define the `WebTimeBean` class, we'll come back to this file and insert this expression to demonstrate that the IDE provides code-completion support for the Java classes you define in your project.

Defining the Page's Logic: Class `WebTimeBean`

We'll now create the `WebTimeBean` class—the `@ManagedBean` class that will allow the JSF page to obtain the web server's time. To create the class, perform the following steps:

1. In the NetBeans **Projects** tab, right click the `WebTime` project's **Source Packages** node and select **New > Other...** to display the **New File** dialog.
2. In the **Categories** list, select **JavaServer Faces**, then in the **File Types** list select **JSF Managed Bean**. Click **Next >**.
3. In the **Name and Location** step, specify `WebTimeBean` as the **Class Name** and `webtime` as the `Package`, then click **Finish**.

NetBeans creates the `WebTimeBean.java` file and places it within the `webtime` package in the project's **Source Packages** node. Figure 29.10 shows this file's default source code displayed in the IDE. At line 16, notice that NetBeans added the **`@RequestScoped` annotation** to the class—this indicates that an object of this class exists only for the duration of the request that's being processed. (We'll discuss `@RequestScoped` and other bean scopes in more detail in Section 29.8.) We did not include this annotation in Fig. 29.6, because all JSF beans are request scoped by default. Replace the code in Fig. 29.10 with the code in Fig. 29.6.

index.xhtml | WebTimeBean.java

```
1  /*
2   * To change this template, choose Tools | Templates
3   * and open the template in the editor.
4   */
5
6  package webtime;
7
8  import javax.faces.bean.ManagedBean;
9  import javax.faces.bean.RequestScoped;
10
11 /**
12  *
13  * @author Paul Deitel
14  */
15 @ManagedBean
16 @RequestScoped
17 public class WebTimeBean {
18
19     /** Creates a new instance of WebTimeBean */
20     public WebTimeBean() {
21     }
22
23 }
```

Fig. 29.10 | Default source code for the `WebTimeBean` class.

Adding the EL Expression to the `index.xhtml` *Page*

Now that you've created the `WebTimeBean`, let's go back to the `index.xhtml` file and add the EL expression that will obtain the time. In the `index.xhtml` file, modify the line

```
<h1>Current time on the web server: </h1>
```

by inserting the expression `#{webTimeBean.time}` before the `h1` element's closing tag. After you type the characters `#` and `{`, the IDE automatically inserts the closing `}`, inserts the

cursor between the braces and displays the code-completion window. This shows various items that could be placed in the braces of the EL expression, including the `webTimeBean` object (of type `WebTimeBean`). To insert `webTimeBean` in the code, you can type the object's name or double click it in the code-completion window. As you type, the list of items in the code-completion window is filtered by what you've typed so far.

When you type the dot (`.`) after `webTimeBean`, the code-completion window reappears, showing you the `WebTimeBean` methods and properties that can be used in this context (Fig. 29.11). In this list, you can double click the `time` property, or you can simply type its name.

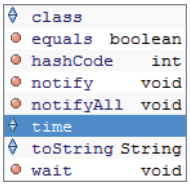


Fig. 29.11 | NetBeans code-completion window for the `webTimeBean` object.

Running the Application

You've now completed the `WebTime` app. To test it, right click the project's name in the **Projects** tab and select **Run** from the pop-up menu. The IDE will compile the code and deploy (that is, install) the `WebTime` app on the GlassFish application server running on your local machine. Then, the IDE will launch your default web browser and request the WebTime app's default web page (`index.xhtml`). Because GlassFish is installed on your local computer, the URL displayed in the browser's address bar will be

```
http://localhost:8080/WebTime/
```

where `8080` is the port number on which the GlassFish server runs by default. Depending on your web browser, the `http://` may not be displayed (Fig. 29.5).

Debugging the Application

If there's a problem with your web app's logic, you can press *<Ctrl> F5* to build the application and run it in debug mode—the NetBeans built-in debugger can help you troubleshoot applications. If you press *F6*, the program executes without debugging enabled.

Testing the Application from Other Web Browsers

After deploying your project, you can test it from another web browser on your computer by entering the app's URL into the other browser's address field. Since your application resides on the local file system, GlassFish must be running. If you've already executed the application using one of the techniques above and have not closed NetBeans, GlassFish will still be running. Otherwise, you can start the server from the IDE by opening the **Services** tab (located in the same panel as the **Projects**), expanding the **Servers** node, right clicking **GlassFish Server 3** and selecting **Start**. Then you can type the URL in the browser to execute the application.

29.5 Model-View-Controller Architecture of JSF Apps

JSF applications adhere to the **Model-View-Controller (MVC) architecture**, which separates an application's data (contained in the **model**) from the graphical presentation (the **view**) and the processing logic (the **controller**). Figure 29.12 shows the relationships between components in MVC.

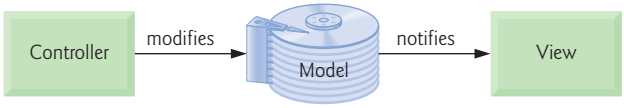


Fig. 29.12 | Model-View-Controller architecture.

In JSF, the controller is the JSF framework and is responsible for coordinating interactions between the view and the model. The model contains the application's data (typically in a database), and the view presents the data stored in the model (typically as web pages). When a user interacts with a JSF web app's view, the framework interacts with the model to store and/or retrieve data. When the model changes, the view is updated with the changed data.

29.6 Common JSF Components

As mentioned in Section 29.4, JSF supports several tag libraries. In this section, we introduce several of the JSF HTML Tag Library's elements and one element from the **JSF Core Tag Library**. Figure 29.13 summarizes elements discussed in this section.

JSF component	Description
`h:form`	Inserts an XHTML `form` element into a page.
`h:commandButton`	Displays a button that triggers an event when clicked. Typically, such a button is used to submit a form's user input to the server for processing.
`h:graphicImage`	Displays an image (e.g., GIF and JPG).
`h:inputText`	Displays a text box in which the user can enter input.
`h:outputLink`	Displays a hyperlink.
`h:panelGrid`	Displays an XHTML `table` element.
`h:selectOneMenu`	Displays a drop-down list of choices from which the user can make a selection.
`h:selectOneRadio`	Displays a set of radio buttons.
`f:selectItem`	Specifies an item in an `h:selectOneMenu` or `h:selectOneRadio` (and other similar components).

Fig. 29.13 | Commonly used JSF components.

All of these elements are mapped by JSF framework to a combination of XHTML elements and JavaScript code that enables the browser to render the page. JavaScript is a scripting language that's interpreted in all of today's popular web browsers. It can be used to perform tasks that manipulate web-page elements in a web browser and provide interactivity with the user. You can learn more about JavaScript in our JavaScript Resource Center at `www.deitel.com/JavaScript/`.

Figure 29.14 displays a form for gathering user input. [*Note:* To create this application from scratch, review the steps in Section 29.4.3 and name the application `WebComponents`.] The **`h:form`** element (lines 14–55) contains the components with which a user interacts to provide data, such as registration or login information, to a JSF app. This example uses the components summarized in Fig. 29.13. This example does not perform a task when the user clicks the **Register** button. Later, we demonstrate how to add functionality to many of these components.

```
1  <?xml version='1.0' encoding='UTF-8' ?>
2
3  <!-- index.xhtml -->
4  <!-- Registration form that demonstrates various JSF components -->
5  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
6     "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
7  <html xmlns="http://www.w3.org/1999/xhtml"
8        xmlns:h="http://java.sun.com/jsf/html"
9        xmlns:f="http://java.sun.com/jsf/core">
10    <h:head>
11       <title>Sample Registration Form</title>
12    </h:head>
13    <h:body>
14       <h:form>
15          <h1>Registration Form</h1>
16          <p>Please fill in all fields and click Register</p>
17          <h:panelGrid columns="4" style="height: 96px; width:456px;">
18             <h:graphicImage name="fname.png" library="images"/>
19             <h:inputText id="firstNameInputText"/>
20             <h:graphicImage name="lname.png" library="images"/>
21             <h:inputText id="lastNameInputText"/>
22             <h:graphicImage name="email.png" library="images"/>
23             <h:inputText id="emailInputText"/>
24             <h:graphicImage name="phone.png" library="images"/>
25             <h:inputText id="phoneInputText"/>
26          </h:panelGrid>
27          <p><h:graphicImage name="publications.png" library="images"/>
28             <br/>Which book would you like information about?</p>
29          <h:selectOneMenu id="booksSelectOneMenu">
30             <f:selectItem itemValue="CHTP"
31                itemLabel="C How to Program" />
32             <f:selectItem itemValue="CPPHTP"
33                itemLabel="C++ How to Program" />
34             <f:selectItem itemValue="IW3HTP"
35                itemLabel="Internet &amp;amp; World Wide Web How to Program" />
```

Fig. 29.14 | Registration form that demonstrates various JSF components. (Part 1 of 2.)

```
36              <f:selectItem itemValue="JHTP"
37                 itemLabel="Java How to Program" />
38              <f:selectItem itemValue="VBHTP"
39                 itemLabel="Visual Basic How to Program" />
40              <f:selectItem itemValue="VCSHTP"
41                 itemLabel="Visual C# How to Program" />
42           </h:selectOneMenu>
43           <p><h:outputLink value="http://www.deitel.com">
44              Click here to learn more about our books
45           </h:outputLink></p>
46           <h:graphicImage name="os.png" library="images"/>
47           <h:selectOneRadio id="osSelectOneRadio">
48              <f:selectItem itemValue="WinVista" itemLabel="Windows Vista"/>
49              <f:selectItem itemValue="Win7" itemLabel="Windows 7"/>
50              <f:selectItem itemValue="OSX" itemLabel="Mac OS X"/>
51              <f:selectItem itemValue="Linux" itemLabel="Linux"/>
52              <f:selectItem itemValue="Other" itemLabel="Other"/>
53           </h:selectOneRadio>
54           <h:commandButton value="Register"/>
55        </h:form>
56     </h:body>
57  </html>
```

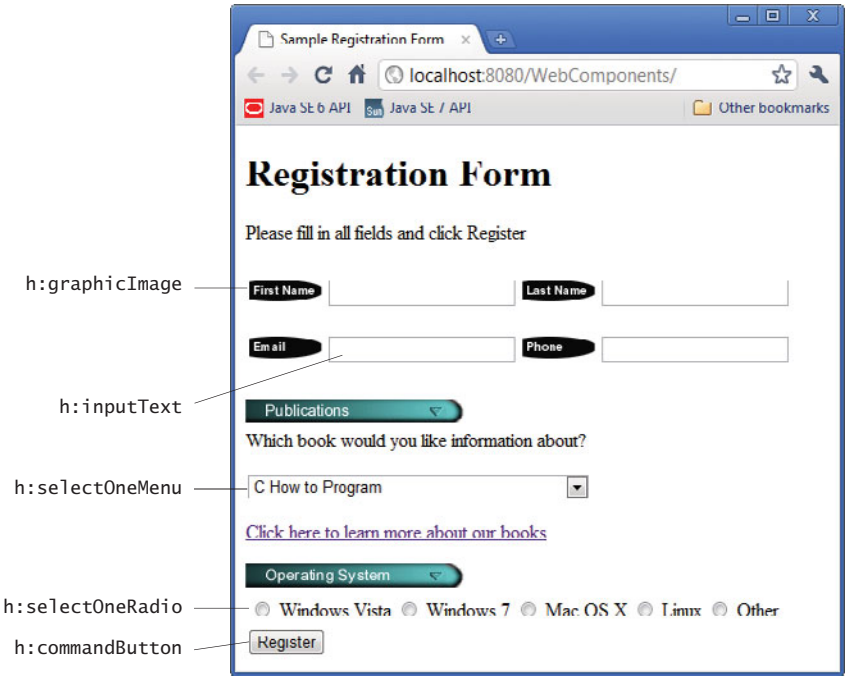


Fig. 29.14 | Registration form that demonstrates various JSF components. (Part 2 of 2.)

h:panelGrid Element

Lines 17–26 define an **h:panelGrid** element for organizing elements in the page. This element inserts an XHTML `table` in the page. The `h:` prefix indicates that `panelGrid` is

from the JSF HTML Tag Library. The **`columns` attribute** specifies the number of columns in the `table`. The elements between the `h:panelGrid`'s start tag (line 17) and end tag (line 26) are automatically placed into the `table`'s columns from left to right in the order they appear in the JSF page. When the number of elements in a row exceeds the number of columns, the `h:panelGrid` creates a new row. We use the `h:panelGrid` to control the positions of the `h:graphicImage` and `h:inputText` elements in the user information section of the page. In this case, there are eight elements in the `h:panelGrid`, so the first four (lines 18–21) are placed in the table's first row and the last four are placed in the second row. The `h:panelGrid`'s **`style` attribute** specifies the CSS formatting for the `table`. We use the CSS attributes `width` and `height` to specify the width and height of the table in pixels (`px`).The `h:panelGrid` contains pairs of `h:graphicImage` and `h:inputText` elements.

`h:graphicImage` *Element and Resource Libraries*

Each **`h:graphicImage`** displays an image in the page. For example, line 18 inserts the image `fname.png`—as specified by the **`name` attribute**. As of JSF 2.0, you add resources that are used throughout your app—such as images, CSS files, JavaScript files—to your web apps by placing them in the app's **`resources` folder** within your project's **Web Pages** node. Each subfolder of `resources` represents a **resource library**. Typically, images are placed in an `images` library and CSS files in a `css` library. In line 18, we specify that the image is located in the `images` library with the **`library` attribute**. JSF knows that the value of this attribute represents a folder within the `resources` folder.

You can create any library you like in the `resources` folder. To create this folder:

1. Expand your app's node in the NetBeans **Projects** tab.
2. Right click the **Web Pages** node and select **New > Folder...** to display the **New Folder** dialog. [*Note:* If the **Folder...** option is not available in the popup menu, select **Other...**, then in the **Categories** pane select **Other** and in the **File Types** pane select **Folder** and click **Next >**.
3. Specify `resources` as the *Folder Name* and press *Finish*.

Next, right click the `resources` folder you just created and create an `images` subfolder. You can then drag the images from your file system onto the `images` folder to add them as resources. The images in this example are located in the `images` directory with the chapter's examples.

The `h:graphicImage` in line 18 is a so-called **empty element**—an element that does not have content between its start and end tags. In such an element, data is typically specified as attributes in the start tag, such as the `name` and `library` attributes in line 18. You can close an empty element either by placing a slash immediately preceding the start tag's right angle bracket, as shown in line 18, or by explicitly writing an end tag.

`h:inputText` *Element*

Line 19 defines an **`h:inputText` element** in which the user can enter text or the app can display text. For any element that might be accessed by other elements of the page or that might be used in server-side code, you should specify an **`id` attribute**. We specified these attributes in this example, even though the app does not provide any functionality. We'll use the `id` attribute starting with the next example.

`h:selectOneMenu` *Element*

Lines 29–42 define an **`h:selectOneMenu` element**, which is typically rendered in a web page as a drop-down list. When a user clicks the drop-down list, it expands and displays a list from which the user can make a selection. Each item to display appears between the start and end tags of this element as an **`f:selectItem` element** (lines 30–41). This element is part of the JSF Core Tag Library. The XML namespace for this tag library is specified in the `html` element's start tag at line 9. Each `f:selectItem` has `itemValue` and `itemLabel` attributes. The **`itemLabel`** is the string that the user will see in the browser, and the **`itemValue`** is the value that's returned when you programmatically retrieve the user's selection from the drop-down list (as you'll see in a later example).

`h:outputLink` *Element*

The **`h:outputLink` element** (lines 43–45) inserts a hyperlink in a web page. Its **`value` attribute** specifies the resource (`http://www.deitel.com` in this case) that's requested when a user clicks the hyperlink. By default, `h:outputLink` elements cause pages to open in the same browser window, but you can set the element's `target` attribute to change this behavior.

`h:selectOneMenu` *Element*

Lines 47–53 define an **`h:selectOneRadio` element**, which provides a series of radio buttons from which the user can select only one. Like an `h:selectOneMenu`, an `h:selectOneRadio` displays items that are specified with `f:selectItem` elements.

`h:commandButton` *Element*

Lines 54 defines an **`h:commandButton` element** that triggers an action when clicked—in this example, we don't specify the action to trigger, so the default action occurs (re-requesting the same page from the server) when the user clicks this button. An `h:commandButton` typically maps to an XHTML `input` element with its `type` attribute set to `"submit"`. Such elements are often used to submit a form's user input values to the server for processing.

29.7 Validation Using JSF Standard Validators

Validating input is an important step in collecting information from users. **Validation** helps prevent processing errors due to incomplete, incorrect or improperly formatted user input. For example, you may perform validation to ensure that all required fields contain data or that a zip-code field has the correct number of digits. The JSF Core Tag Library provides several standard validator components and allows you to create your own custom validators. Multiple validators can be specified for each input element. The validators are:

- **`f:validateLength`**—determines whether a field contains an acceptable number of characters.
- **`f:validateDoubleRange`** and **`f:validateLongRange`**—determine whether numeric input falls within acceptable ranges of `double` or `long` values, respectively.
- **`f:validateRequired`**—determines whether a field contains a value.
- **`f:validateRegex`**—determines whether a field contains a string that matches a specified regular expression pattern.
- **`f:validateBean`**—allows you to invoke a bean method that performs custom validation.

Validating Form Data in a Web Application

[*Note:* To create this application from scratch, review the steps in Section 29.4.3 and name the application `Validation`.] The example in this section prompts the user to enter a name, e-mail address and phone number in a form. When the user enters any data and presses the **Submit** button to submit the form's contents to the web server, validation ensures that the user entered a value in each field, that the entered name does not exceed 30 characters, and that the e-mail address and phone-number values are in an acceptable format. In this example, (555) 123-4567, 555-123-4567 and 123-4567 are all considered valid phone numbers. Once valid data is submitted, the JSF framework stores the submitted values in a bean object of class `ValidationBean` (Fig. 29.15), then sends a response back to the web browser. We simply display the validated data in the page to demonstrate that the server received the data. A real business application would typically store the submitted data in a database or in a file on the server.

Class `ValidationBean`

Class `ValidationBean` (Fig. 29.15) provides the read/write properties name, email and phone, and the read-only property `result`. Each read/write property has an instance variable (lines 11–13) and corresponding *set*/*get* methods (lines 16–25, 28–37 and 40–49) for manipulating the instance variables. The read-only property response has only a `getResult` method (lines 52–60), which returns a paragraph (`p`) element containing the validated data. (You can create the `ValidationBean` managed bean class by using the steps presented in Fig. 29.4.3.)

```
1  // ValidationBean.java
2  // Validating user input.
3  package validation;
4
5  import java.io.Serializable;
6  import javax.faces.bean.ManagedBean;
7
8  @ManagedBean( name="validationBean" )
9  public class ValidationBean implements Serializable
10 {
11    private String name;
12    private String email;
13    private String phone;
14
15    // return the name String
16    public String getName()
17    {
18       return name;
19    } // end method getName
20
21    // set the name String
22    public void setName( String name )
23    {
24       this.name = name;
25    } // end method setName
```

Fig. 29.15 | `ValidationBean` stores the validated data, which is then used as part of the response to the client. (Part 1 of 2.)

```
26
27      // return the email String
28      public String getEmail()
29      {
30         return email;
31      } // end method getEmail
32
33      // set the email String
34      public void setEmail( String email )
35      {
36         this.email = email;
37      } // end method setEmail
38
39      // return the phone String
40      public String getPhone()
41      {
42         return phone;
43      } // end method getPhone
44
45      // set the phone String
46      public void setPhone( String phone )
47      {
48         this.phone = phone;
49      } // end method setPhone
50
51      // returns result for rendering on the client
52      public String getResult()
53      {
54         if ( name != null && email != null && phone != null )
55            return "<p style=\"background-color:yellow;width:200px;" +
56               "padding:5px\">Name: " + getName() + "<br/>E-Mail: " +
57               getEmail() + "<br/>Phone: " + getPhone() + "</p>";
58         else
59            return ""; // request has not yet been made
60      } // end method getResult
61   } // end class ValidationBean
```

Fig. 29.15 | `ValidationBean` stores the validated data, which is then used as part of the response to the client. (Part 2 of 2.)

index.xhtml

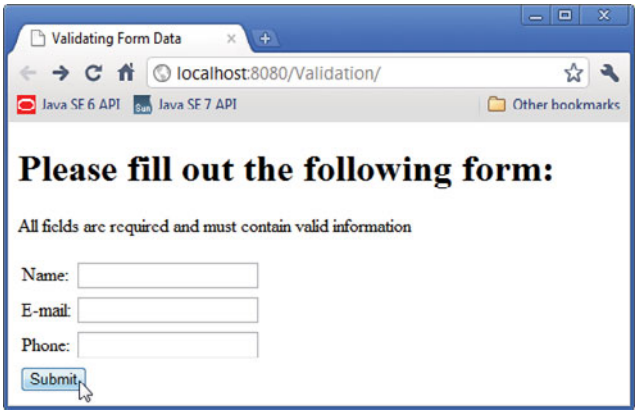
Figure 29.16 shows this app's `index.xhtml` file. The initial request to this web app displays the page shown in Fig. 29.16(a). When this app is initially requested, the beginning of the **JSF application lifecycle** uses this `index.xhtml` document to build the app's facelets view and sends it as the response to the client browser, which displays the form for user input. During this initial request, the EL expressions (lines 22, 30, 39 and 49) are evaluated to obtain the values that should be displayed in various parts of the page. Nothing is displayed initially as a result of these four EL expressions being evaluated, because no default values are specified for the bean's properties. The page's `h:form` element contains an `h:panelGrid` (lines 18–45) with three columns and an `h:commandButton` (line 46), which by default submits the contents of the form's fields to the server.

```
<?xml version='1.0' encoding='UTF-8' ?>

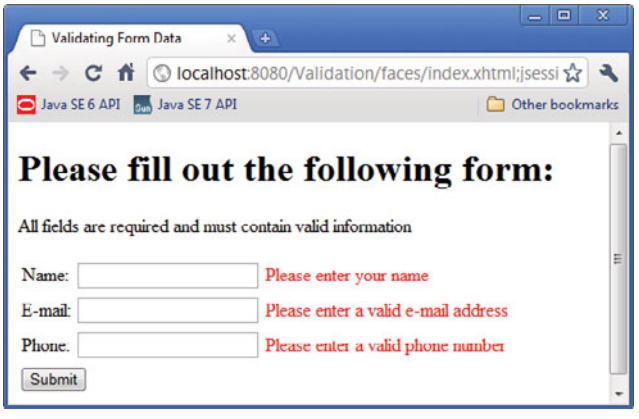
<!-- index.xhtml -->
<!-- Validating user input -->
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
   "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml"
   xmlns:h="http://java.sun.com/jsf/html"
   xmlns:f="http://java.sun.com/jsf/core">
   <h:head>
      <title>Validating Form Data</title>
      <h:outputStylesheet name="style.css" library="css"/>
   </h:head>
   <h:body>
      <h:form>
         <h1>Please fill out the following form:</h1>
         <p>All fields are required and must contain valid information</p>
         <h:panelGrid columns="3">
            <h:outputText value="Name:"/>
            <h:inputText id="nameInputText" required="true"
               requiredMessage="Please enter your name"
               value="#{validationBean.name}"
               validatorMessage="Name must be fewer than 30 characters">
               <f:validateLength maximum="30" />
            </h:inputText>
            <h:message for="nameInputText" styleClass="error"/>
            <h:outputText value="E-mail:"/>
            <h:inputText id="emailInputText" required="true"
               requiredMessage="Please enter a valid e-mail address"
               value="#{validationBean.email}"
               validatorMessage="Invalid e-mail address format">
               <f:validateRegex pattern=
                  "\w+([-+.']\w+)*@\w+([-.]\w+)*\.\w+([-.]\w+)*" />
            </h:inputText>
            <h:message for="emailInputText" styleClass="error"/>
            <h:outputText value="Phone:"/>
            <h:inputText id="phoneInputText" required="true"
               requiredMessage="Please enter a valid phone number"
               value="#{validationBean.phone}"
               validatorMessage="Invalid phone number format">
               <f:validateRegex pattern=
                  "((\(\d{3}\) ?)|(\d{3}-))?\d{3}-\d{4}" />
            </h:inputText>
            <h:message for="phoneInputText" styleClass="error"/>
         </h:panelGrid>
         <h:commandButton value="Submit"/>
         <h:outputText escape="false" value="#{validationBean.response}"/>
      </h:form>
   </h:body>
</html>
```

Fig. 29.16 | Form to demonstrate validating user input. (Part 1 of 3.)

a) Submitting the form before entering any information



b) Error messages displayed after submitting the empty form



c) Error messages displayed after submitting invalid information

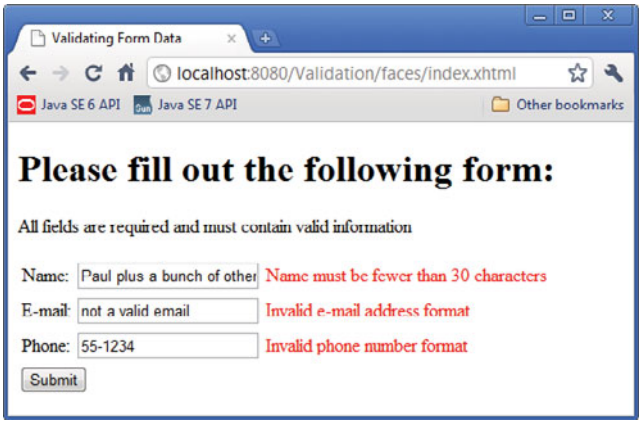


Fig. 29.16 | Form to demonstrate validating user input. (Part 2 of 3.)

d) Successfully submitted form

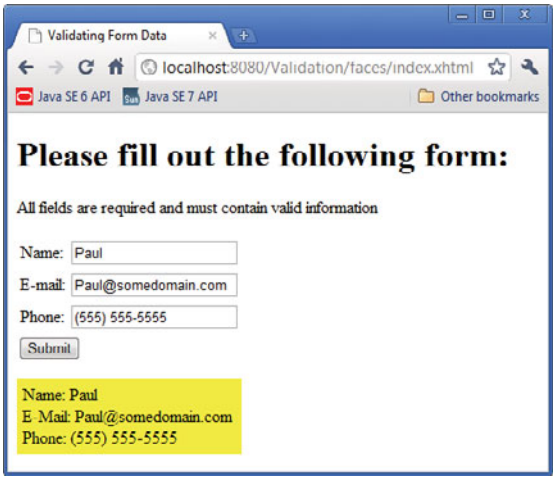


Fig. 29.16 | Form to demonstrate validating user input. (Part 3 of 3.)

First Row of the `h:panelGrid`

In this application, we demonstrate several new elements and attributes. The first new element is the **`h:outputText` element** (line 19; from the JSF HTML Tag Library), which inserts text in the page. In this case, we insert a literal string (`"Name:"`) that is specified with the element's `value` attribute.

The `h:inputText` element (lines 20–25) displays a text box in which the user can enter a name. We've specified several attributes for this element:

- `id`—This enables other elements or server-side code to reference this element.
- **`required`**—Ensuring that the user has made a selection or entered some text in a required input element is a basic type of validation. When set to `"true"`, this attribute specifies that the element *must* contain a value.
- **`requiredMessage`**—This specifies the message that should be displayed if the user submits the form without first providing a value for this required element.
- `value`—This specifies the value to display in the field or to be saved into a bean on the server. In this case, the EL expression indicates the bean property that's associated with this field.
- **`validatorMessage`**—This specifies the message to display if a validator is associated with this `h:inputText` and the data the user enters is invalid.

The messages specified by the `requiredMessage` and `validatorMessage` attributes are displayed in an associated **`h:message` element** (line 26) when validation fails. The element's **`for` attribute** specifies the `id` of the specific element for which messages will be displayed (`nameInputText`), and the **`styleClass` attribute** specifies the name of a CSS style class that will format the message. For this example, we defined a CSS style sheet, which was inserted into the document's `head` element at line 12 using the **`h:outputStylesheet`**

element. We placed the style sheet in the `css` library within the `resources` folder. The style sheet contains the following CSS rule:

```
.error
{
   color:red;
}
```

which creates a style class named `error` (the dot indicates that it's a style class) and specifies that any text to which this is applied, such as the error messages, should be red. We use this CSS style for all the `h:message` elements in this example.

Validating the `nameInputText` *Element's Contents*

If the user submits the form without a value in the `nameInputText`, the `requiredMessage` `"Please enter your name"` is displayed in the corresponding `h:message` element. If the user specifies a value for the `nameInputText`, the JSF framework executes the `f:validateLength` validator that's nested in the `h:inputText` element. Here, we check that the name contains no more than 30 characters—as specified by the validator's **`maximum` attribute**. This might be useful to ensure that a value will fit within a particular database field.

Users can type as much text in the `nameInputText` as they wish. If the name is too long, the `validatorMessage` is displayed in the `h:message` element after the user submits the form. It's also possible to limit the length of user input in an `h:inputText` without using validation by setting its **`maxlength` attribute**, in which case the element's cursor will not advance beyond the maximum allowable number of characters. This would prevent the user from submitting data that exceeds the length limit.

Second and Third Rows of the `h:panelGrid`

The next two rows of the `h:panelGrid` have elements similar to those in the first row. In addition to being required elements, the `h:inputText` elements at lines 28–34 and 37–43 are each validated by `h:validateRegex` validators as described next.

Validating the e-Mail Address

The `h:validateRegex` element at lines 32–33 uses the regular expression

```
\w+([-+.']\w+)*@\w+([-.]\w+)*\.\w+([-.]\w+)*
```

which indicates that an e-mail address is valid if the part before the `@` symbol contains one or more word characters (that is, alphanumeric characters or underscores), followed by zero or more strings comprised of a hyphen, plus sign, period or apostrophe and additional word characters. After the `@` symbol, a valid e-mail address must contain one or more groups of word characters potentially separated by hyphens or periods, followed by a required period and another group of one or more word characters potentially separated by hyphens or periods. For example, `bob's-personal.email@white.email.com`, `bob-white@my-email.com` and `bob.white@email.com` are all valid e-mail addresses. If the address the user enters has an invalid format, the `validatorMessage` (line 31) will be displayed in the corresponding `h:message` element (line 35).

Validating the Phone Number

The `h:validateRegex` element at lines 41–42 uses the regular expression

```
((\(\d{3}\) ?)|(\d{3}-))?\d{3}-\d{4}
```

which indicates that a phone number can contain a three-digit area code either in parentheses and followed by an optional space or without parentheses and followed by a required hyphen. After an optional area code, a phone number must contain three digits, a hyphen and another four digits. For example, `(555) 123-4567`, `555-123-4567` and `123-4567` are all valid phone numbers. If the phone number the user enters has an invalid format, the `validatorMessage` (line 40) will be displayed in the corresponding `h:message` element (line 44).

Submitting the Form—More Details of the JSF Lifecycle

As we mentioned earlier in this section, when the app receives the initial request, it returns the page shown in Fig. 29.16(a). When a request does not contain any request values, such as those the user enters in a form, the JSF framework simply creates the view and returns it as the response.

The user submits the form to the server by pressing the **Submit** `h:commandButton` (defined at line 46). Since we did not specify an `action` attribute for this `h:commandButton`, the `action` is configured by default to perform a **postback**—the browser re-requests the page `index.xhtml` and sends the values of the form's fields to the server for processing. Next, the JSF framework performs the validations of all the form elements. If any of the elements is invalid, the framework renders the appropriate error message as part of the response.

If the values of all the elements are valid, the framework uses the values of the elements to set the properties of the `validateBean`—as specified in the EL expressions in lines 22, 30 and 39. Each property's *set* method is invoked, passing the value of the corresponding element as an argument. The framework then formulates the response to the client. In the response, the form elements are populated with the values of the `validateBean`'s properties (by calling their *get* methods), and the `h:outputText` element at line 47 is populated with the value of the read-only `result` property. The value of this property is determined by the `getResult` method (lines 52–60 of Fig. 29.15), which uses the submitted form data in the string that it returns.

When you execute this app, try submitting the form with no data (Fig. 29.16(b)), with invalid data (Fig. 29.16(c)) and with valid data (Fig. 29.16(d)).

29.8 Session Tracking

Originally, critics accused the Internet and e-business of failing to provide the customized service typically experienced in "brick-and-mortar" stores. To address this problem, businesses established mechanisms by which they could *personalize* users' browsing experiences, tailoring content to individual users. They tracked each customer's movement through the Internet and combined the collected data with information the consumer provided, including billing information, personal preferences, interests and hobbies.

Personalization

Personalization enables businesses to communicate effectively with their customers and also helps users locate desired products and services. Companies that provide content of particular interest to users can establish relationships with customers and build on those relationships over time. Furthermore, by targeting consumers with personal offers, recommendations, advertisements, promotions and services, businesses create customer loyalty.

Websites can use sophisticated technology to allow visitors to customize home pages to suit their individual needs and preferences. Similarly, online shopping sites often store personal information for customers, tailoring notifications and special offers to their interests. Such services encourage customers to visit sites more frequently and make purchases more regularly.

Privacy

A trade-off exists between personalized business service and protection of privacy. Some consumers embrace tailored content, but others fear the possible adverse consequences if the info they provide to businesses is released or collected by tracking technologies. Consumers and privacy advocates ask: What if the business to which we give personal data sells or gives that information to another organization without our knowledge? What if we do not want our actions on the Internet—a supposedly anonymous medium—to be tracked and recorded by unknown parties? What if unauthorized parties gain access to sensitive private data, such as credit-card numbers or medical history? These are questions that must be addressed by programmers, consumers, businesses and lawmakers alike.

Recognizing Clients

To provide personalized services, businesses must be able to recognize clients when they request information from a site. As we have discussed, the request/response system on which the web operates is facilitated by HTTP. Unfortunately, HTTP is a *stateless protocol*—it *does not* provide information that would enable web servers to maintain state information regarding particular clients. This means that web servers cannot determine whether a request comes from a particular client or whether the same or different clients generate a series of requests.

To circumvent this problem, sites can provide mechanisms by which they identify individual clients. A session represents a unique client on a website. If the client leaves a site and then returns later, the client will still be recognized as the same user. When the user closes the browser, the session typically ends. To help the server distinguish among clients, each client must identify itself to the server. Tracking individual clients is known as **session tracking**. One popular session-tracking technique uses cookies (discussed in Section 29.8.1); another uses beans that are marked with the `@SessionScoped` **annotation** (used in Section 29.8.2). Additional session-tracking techniques are beyond this book's scope.

29.8.1 Cookies

Cookies provide you with a tool for personalizing web pages. A cookie is a piece of data stored by web browsers in a small text file on the user's computer. A cookie maintains information about the client during and between browser sessions. The first time a user visits the website, the user's computer might receive a cookie from the server; this cookie is then reactivated each time the user revisits that site. The collected information is intended to be an anonymous record containing data that is used to personalize the user's future visits to the site. For example, cookies in a shopping application might store unique identifiers for users. When a user adds items to an online shopping cart or performs another task resulting in a request to the web server, the server receives a cookie containing the user's unique identifier. The server then uses the unique identifier to locate the shopping cart and perform any necessary processing.

In addition to identifying users, cookies also can indicate users' shopping preferences. When a Web Form receives a request from a client, the Web Form can examine the cookie(s) it sent to the client during previous communications, identify the user's preferences and immediately display products of interest to the client.

Every HTTP-based interaction between a client and a server includes a header containing information either about the request (when the communication is from the client to the server) or about the response (when the communication is from the server to the client). When a Web Form receives a request, the header includes information such as the request type and any cookies that have been sent previously from the server to be stored on the client machine. When the server formulates its response, the header information contains any cookies the server wants to store on the client computer and other information, such as the MIME type of the response.

The **expiration date** of a cookie determines how long the cookie remains on the client's computer. If you do not set an expiration date for a cookie, the web browser maintains the cookie for the duration of the browsing session. Otherwise, the web browser maintains the cookie until the expiration date occurs. Cookies are deleted by the web browser when they **expire**.



Portability Tip 29.1

Users may disable cookies in their web browsers to help ensure their privacy. Such users will experience difficulty using web applications that depend on cookies to maintain state information.

29.8.2 Session Tracking with `@SessionScoped` Beans

The previous web applications used `@RequestScoped` beans by default—the beans existed only for the duration of each request. In the next application, we use a **`@SessionScoped`** bean to maintain selections throughout the user's session. Such a bean is created when a session begins and exists throughout the entire session. A `@SessionScoped` bean can be accessed by all of the app's pages during the session, and the app server maintains a separate `@SessionScoped` bean for each user. By default a session expires after 30 minutes of inactivity or when the user closes the browser that was used to begin the session. When the session expires, the server discards the bean associated with that session.



Software Engineering Observation 29.2

`@SessionScoped` beans should implement the `Serializable` interface. Websites with heavy traffic often use groups of servers (sometimes hundreds or thousands of them) to respond to requests. Such groups are known as server farms. Server farms often balance the number of requests being handled on each server by moving some sessions to other servers. Making a bean `Serializable` enables the session to be moved properly among servers.

Test-Driving the App

This example consists of a `SelectionsBean` class that is `@SessionScoped` and two pages (`index.xhtml` and `recommendations.xhtml`) that store data in and retrieve data from a `SelectionsBean` object. To understand how these pieces fit together, let's walk through a sample execution of the app. When you first execute the app, the `index.xhtml` page is displayed. The user selects a topic from a group of radio buttons and submits the form (Fig. 29.17).

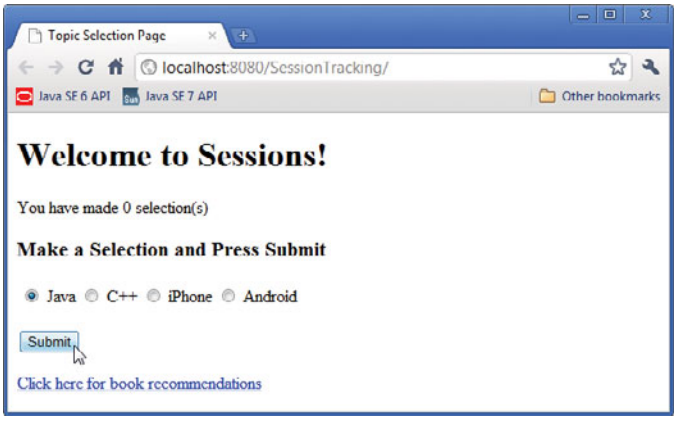


Fig. 29.17 | `index.xhtml` after the user has made a selection and is about to submit the form for the first time.

When the form is submitted, the JSF framework creates a `SelectionsBean` object that is specific to this user, stores the selected topic in the bean and returns the `index.xhtml` page. The page now shows how many selections have been made (1) and allows the user to make another selection (Fig. 29.18).

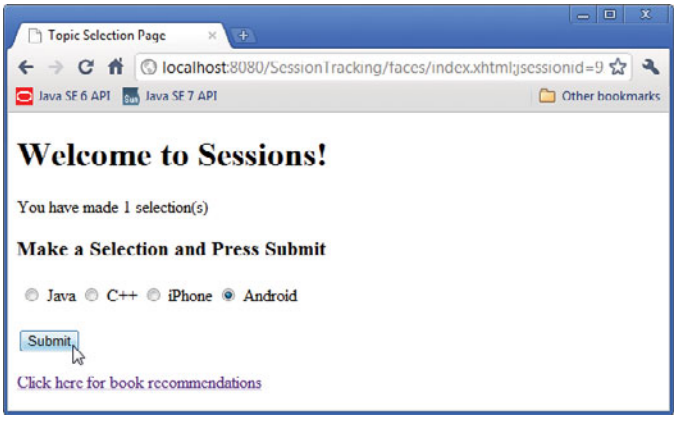


Fig. 29.18 | `index.xhtml` after the user has submitted the form the first time, made another selection and is about to submit the form again.

The user makes a second topic selection and submits the form again. The app stores the selection in this user's existing `SelectionsBean` object and returns the `index.xhtml` page (Fig. 29.19), which shows how many selections have been made so far (2).

At any time, the user can click the link at the bottom of the `index.xhtml` page to open `recommendations.xhtml`, which obtains the information from this user's `SelectionsBean` object and creates a recommended books list (Fig. 29.20) for the user's selected topics.

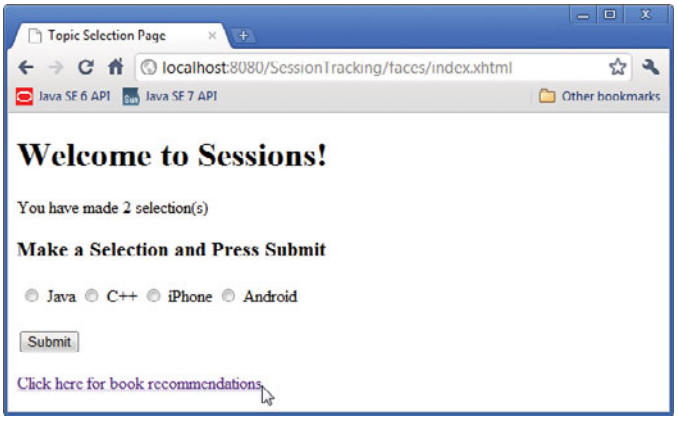


Fig. 29.19 | `index.xhtml` after the user has submitted the form the second time and is about to click the link to the `recommendations.xhtml` page.

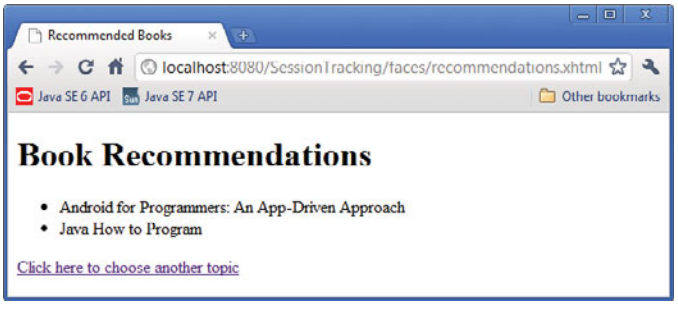


Fig. 29.20 | `recommendations.hxtml` showing book recommendations for the topic selections made by the user in Figs. 29.18 and 29.19.

@SessionScoped Class `SelectionsBean`

Class `SelectionsBean` (Fig. 29.21) uses the `@SessionScoped` annotation (line 13) to indicate that the server should maintain separate instances of this class for each user session. The class maintains a `static HashMap` (created at lines 17–18) of topics and their corresponding book titles. We made this object `static`, because its values can be shared among all `SelectionsBean` objects. The `static` initializer block (lines 23–28) specifies the `HashMap`'s key/value pairs. Class `SelectionsBean` maintains each user's selections in a `Set<String>` (line 32), which allows only unique keys, so selecting the same topic multiple times does not increase the number of selections.

```
1  // SelectionsBean.java
2  // Manages a user's topic selections
3  package sessiontracking;
```

Fig. 29.21 | `@SessionScoped SelectionsBean` class. (Part 1 of 3.)

```
4
5  import java.io.Serializable;
6  import java.util.HashMap;
7  import java.util.Set;
8  import java.util.TreeSet;
9  import javax.faces.bean.ManagedBean;
10 import javax.faces.bean.SessionScoped;
11
12 @ManagedBean( name="selectionsBean" )
13 @SessionScoped
14 public class SelectionsBean implements Serializable
15 {
16    // map of topics to book titles
17    private static final HashMap< String, String > booksMap =
18       new HashMap< String, String >();
19
20    // initialize booksMap
21    static
22    {
23       booksMap.put( "java", "Java How to Program" );
24       booksMap.put( "cpp", "C++ How to Program" );
25       booksMap.put( "iphone",
26          "iPhone for Programmers: An App-Driven Approach" );
27       booksMap.put( "android",
28          "Android for Programmers: An App-Driven Approach" );
29    } // end static initializer block
30
31    // stores individual user's selections
32    private Set< String > selections = new TreeSet< String >();
33    private String selection; // stores the current selection
34
35    // return number of selections
36    public int getNumberOfSelections()
37    {
38       return selections.size();
39    } // end method getNumberOfSelections
40
41    // returns the current selection
42    public String getSelection()
43    {
44       return selection;
45    } // end method getSelection
46
47    // store user's selection
48    public void setSelection( String topic )
49    {
50       selection = booksMap.get( topic );
51       selections.add( selection );
52    } // end method setSelection
53
54    // return the Set of selections
55    public String[] getSelections()
56    {
```

Fig. 29.21 | `@SessionScoped SelectionsBean` class. (Part 2 of 3.)

```
57          return selections.toArray( new String[ selections.size() ] );
58       } // end method getSelections
59    } // end class SelectionsBean
```

Fig. 29.21 | `@SessionScoped SelectionsBean` class. (Part 3 of 3.)

Methods of Class `SelectionsBean`

Method `getNumberOfSelections` (lines 36–39) returns the number of topics the user has selected and represents the read-only property `numberOfSelections`. We use this property in the `index.xhtml` document to display the number of selections the user has made so far.

Methods `getSelection` (lines 42–45) and `setSelection` (lines 48–52) represent the read/write `selection` property. When a user makes a selection in `index.xhtml` and submits the form, method setSelection looks up the corresponding book title in the `booksMap` (line 50), then stores that title in `selections` (line 51).

Method `getSelections` (lines 55–58) represents the read-only property `selections`, which returns an array of `String`s containing the book titles for the topics selected by the user so far. When the `recommendations.xhtml` page is requested, it uses the `selections` property to get the list of book titles and display them in the page.

`index.xhtml`

The `index.xhtml` document (Fig. 29.22) contains an `h:selectOneRadio` element (lines 19–26) with the options **Java**, **C++**, **iPhone** and **Android**. The user selects a topic by clicking a radio button, then pressing **Submit** to send the selection. As the user makes each selection and submits the form, the `selectionsBean` object's `selection` property is updated and this document is returned. The EL expression at line 15 inserts the number of selections that have been made so far into the page. When the user clicks the `h:outputLink` (lines 29–31) the `recommendations.xhtml` page is requested. The `value` attribute specifies only `recommendations.xhtml`, so the browser assumes that this page is on the same server and at the same location as `index.xhtml`.

```
1  <?xml version='1.0' encoding='UTF-8' ?>
2
3  <!-- index.xhtml -->
4  <!-- Allow the user to select a topic -->
5  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
6     "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
7  <html xmlns="http://www.w3.org/1999/xhtml"
8     xmlns:h="http://java.sun.com/jsf/html"
9     xmlns:f="http://java.sun.com/jsf/core">
10    <h:head>
11       <title>Topic Selection Page</title>
12    </h:head>
13    <h:body>
14       <h1>Welcome to Sessions!</h1>
15       <p>You have made #{selectionsBean.numberOfSelections} selection(s)
16       </p>
17       <h3>Make a Selection and Press Submit</h3>
```

Fig. 29.22 | `index.xhtml` allows the user to select a topic. (Part 1 of 2.)

```
18         <h:form>
19            <h:selectOneRadio id="topicSelectOneRadio" required="true"
20               requiredMessage="Please choose a topic, then press Submit"
21               value="#{selectionsBean.selection}">
22               <f:selectItem itemValue="java" itemLabel="Java"/>
23               <f:selectItem itemValue="cpp" itemLabel="C++"/>
24               <f:selectItem itemValue="iphone" itemLabel="iPhone"/>
25               <f:selectItem itemValue="android" itemLabel="Android"/>
26            </h:selectOneRadio>
27            <p><h:commandButton value="Submit"/></p>
28         </h:form>
29         <p><h:outputLink value="recommendations.xhtml">
30            Click here for book recommendations
31         </h:outputLink></p>
32      </h:body>
33   </html>
```

Fig. 29.22 | `index.xhtml` allows the user to select a topic. (Part 2 of 2.)

recommendations.xhtml

When the user clicks the `h:outputLink` in the `index.xhtml` page, the browser requests the `recommendations.xhtml` (Fig. 29.23), which displays book recommendations in an XHTML unordered (bulleted) list (lines 15–19). The `h:outputLink` (lines 20–22) allows the user to return to `index.xhtml` to select additional topics.

```
1   <?xml version='1.0' encoding='UTF-8' ?>
2
3   <!-- recommendations.xhtml -->
4   <!-- Display recommended books based on the user's selected topics -->
5   <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
6      "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
7   <html xmlns="http://www.w3.org/1999/xhtml"
8      xmlns:h="http://java.sun.com/jsf/html"
9      xmlns:ui="http://java.sun.com/jsf/facelets">
10     <h:head>
11        <title>Recommended Books</title>
12     </h:head>
13     <h:body>
14        <h1>Book Recommendations</h1>
15        <ul>
16           <ui:repeat value="#{selectionsBean.selections}" var="book">
17              <li>#{book}</li>
18           </ui:repeat>
19        </ul>
20        <p><h:outputLink value="index.xhtml">
21              Click here to choose another topic
22           </h:outputLink></p>
23     </h:body>
24  </html>
```

Fig. 29.23 | `recommendations.xhtml` displays book recommendations based on the user's selections.

Iterating Through the List of Books

Line 9 enables us to use elements from the **JSF Facelets Tag Library**. This library includes the **`ui:repeat` element** (lines 16–18), which can be thought of as an enhanced `for` loop that iterates through collections JSF Expression Language. The element inserts its nested element(s) once for each element in a collection. The collection is specified by the **`value` attribute**'s EL expression, which *must* return an array, a `List`, a `java.sql.ResultSet` or an `Object`. If the EL expression does not return an array, a `List` or a `ResultSet`, the `ui:repeat` element inserts its nested element(s) only once for the returned `Object`. In this example, the `ui:repeat` element renders the items returned by the `selectionsBean`'s `selections` property.

The `ui:repeat` element's **`var` attribute** creates a variable named `book` to which each item in the collection is assigned in sequence. You can use this variable in EL expressions in the nested elements. For example, the expression `#{book}` in line 17 inserts between the `<li>` and `</li>` tags the `String` representation of one item in the collection. You can also use the variable to invoke methods on, or access properties of, the referenced object.

29.9 Wrap-Up

In this chapter, we introduced web application development using JavaServer Faces in NetBeans. We began by discussing the simple HTTP transactions that take place when you request and receive a web page through a web browser. We then discussed the three tiers (i.e., the client or top tier, the business logic or middle tier and the information or bottom tier) that comprise most web applications.

You learned how to use NetBeans and the GlassFish Application Server to create, compile and execute web applications. We demonstrated several common JSF components. We also showed how to use validators to ensure that user input satisfies the requirements of an application.

We discussed the benefits of maintaining user information across multiple pages of a website. We then demonstrated how you can include such functionality in a web application using `@SessionScoped` beans.

In Chapter 30, we continue our discussion of Java web application development with more advanced concepts. You'll learn how to access a database from a JSF web application and how to use AJAX to help web-based applications provide the interactivity and responsiveness that users typically expect of *desktop* applications.

Summary

Section 29.1 Introduction

- Web-based apps create content for web browser clients. This content includes eXtensible HyperText Markup Language (XHTML), JavaScript client-side scripting, Cascading Style Sheets (CSS), images and binary data.
- XHTML is an XML (eXtensible Markup Language) vocabulary that is based on HTML (HyperText Markup Language).
- Java multitier applications are typically implemented using the features of Java Enterprise Edition (Java EE).

- The JavaServer Faces subset of Java EE is a web-application framework (p. 1236) for building multitier web apps by extending the framework with your application-specific capabilities. The framework handles the details of receiving client requests and returning responses for you.

Section 29.2 HyperText Transfer Protocol (HTTP) Transactions

- In its simplest form, a web page is nothing more than an XHTML document that describes to a web browser how to display and format the document's information.
- XHTML documents normally contain hyperlinks that link to different pages or to other parts of the same page. When the user clicks a hyperlink, the requested web page loads into the browser.
- Computers that run web-server software (p. 1237) make resources available, such as web pages, images, PDF documents and even objects that perform complex tasks.
- The HTTP protocol allows clients and servers to interact and exchange information.
- HTTP uses URLs (Uniform Resource Locators) to locate resources on the Internet.
- A URL contains information that directs a browser to the resource that the user wishes to access.
- The computer that houses and maintains resources is usually referred to as the host (p. 1237).
- Host names are translated into IP addresses by domain-name system (DNS) servers (p. 1237).
- The path in a URL typically specifies a virtual directory on the server. The server translates the this into a real location, thus hiding a resource's true location.
- When given a URL, a web browser performs an HTTP transaction to retrieve and display the web page at that address.
- HTTP headers (p. 1239) provide additional information about the data that will be sent.
- Multipurpose Internet Mail Extensions (MIME; p. 1239) is an Internet standard that specifies data formats so that programs can interpret data correctly.
- The two most common HTTP request types are `GET` and `POST` (p. 1239). A `GET` request typically asks for a specific resource on a server. A `POST` request typically posts (or sends) data to a server.
- `GET` requests and `POST` requests can both be used to send form data to a web server, yet each request type sends the information differently. A `GET` request sends information to the server in the URL's query string (p. 1240). A `POST` request sends form data as part of the HTTP message.
- Browsers often cache (p. 1240) web pages for quick reloading. If there are no changes between the cached version and the current version on the web, this speeds up your browsing experience.
- An HTTP response can indicate the length of time for which the content remains "fresh." If this amount of time has not been reached, the browser can avoid another request to the server.
- Browsers typically do not cache the server's response to a `POST` request, because the next `POST` might not return the same result.

Section 29.3 Multitier Application Architecture

- Web-based applications are multitier (*n*-tier) applications (p. 1240) that divide functionality into separate tiers (i.e., logical groupings of functionality). Although tiers can be located on the same computer, the tiers of web-based applications often reside on separate computers.
- The information tier (p. 1241) maintains data pertaining to the application.
- The middle tier (p. 1241) implements business logic, controller logic and presentation logic to control interactions between the application's clients and the application's data. Business logic in the middle tier enforces business rules and ensures that data is reliable before the server application updates the database or presents the data to users. Business rules dictate how clients can and cannot access application data, and how applications process data.

- The client tier (p. 1241) is the application's user interface, which gathers input and displays output. Users interact directly with the application through the user interface. In response to user actions (e.g., clicking a hyperlink), the client tier interacts with the middle tier to make requests and to retrieve data from the information tier.

Section 29.4 Your First JSF Web App

- The JSF web-application framework's Faces servlet (p. 1242) processes each requested JSF page so that the server can eventually return a response to the client.

Section 29.4.1 The Default `index.xhtml` Document: Introducing Facelets

- You present your web app's content in JSF using Facelets (p. 1243)—a combination of XHTML markup and JSF markup.
- XHTML (p. 1243) specifies the content of a web page that is displayed in a web browser. XHTML separates the presentation of a document from the structure of the document's data.
- Presentation is specified with Cascading Style Sheets (CSS).
- JSF uses the XHTML 1.0 Transitional Recommendation by default. Transitional markup may include some non-CSS formatting, but this is not recommended.
- The starting `<html>` tag may contain one or more `xmlns` attributes (p. 1243). Each has a name and a value separated by an equal sign (=), and specifies an XML namespace of elements that are used in the document.
- The attribute `xmlns:h="http://java.sun.com/jsf/html"` specifies a prefix and a URL for JSF's HTML Tag Library (p. 1243), allowing the document to use JSF's elements from that library.
- A tag library defines a set of elements that can be inserted into the XHTML markup.
- The elements in the HTML Tag Library generate XHTML elements.
- The `h:head` element (p. 1244) defines the XHTML page's `head` element.
- The document's `title` typically appears in the browser window's title bar, or a browser tab if you have multiple web pages open in the browser at once.
- The `h:body` (p. 1244) element represents the page's content.
- A JSF Expression Language (EL; p. 1244) expression can interact with a JavaBean to obtain data.

Section 29.4.2 Examining the `WebTimeBean` Class

- JSF documents typically interact with one or more Java objects to perform the app's tasks.
- JavaBeans objects (p. 1244) are instances of classes that follow certain conventions for class design. A JavaBean exposes its data as properties (p. 1244). Properties can be read/write, read-only or write-only. To define a read/write property, a JavaBean class provides *set* and *get* methods for that property. A read-only property would have only a *get* method and a write-only property only a *set* method.
- The JavaBeans used in JSF are also POJOs (plain old Java objects; p. 1244)
- The JSF framework creates and manages objects of your JavaBean classes for you.
- The `@ManagedBean` annotation (from the package `javax.faces.bean`; p. 1245) indicates that the JSF framework should create and manage instances of the class. The parentheses following the annotation contain the optional `name` attribute. If you specify the annotation without the parentheses and the `name` attribute, the JSF framework will use the class name with a lowercase first letter as the default bean name.
- When the Faces servlet encounters an EL expression that accesses a bean property, it automatically invokes the property's *set* or *get* method based on the context in which the property is used.

Section 29.5 Model-View-Controller Architecture of JSF Apps

- JSF applications adhere to the Model-View-Controller (MVC; p. 1250) architecture, which separates an application's data (contained in the model) from the graphical presentation (the view) and the processing logic (the controller).
- In JSF, the controller is the JSF framework and is responsible for coordinating interactions between the view and the model. The model contains the application's data (typically in a database), and the view presents the data stored in the model (typically as web pages).

Section 29.6 Common JSF Components

- Elements from the JSF HTML Tag Library are mapped by the JSF framework to a combination of XHTML elements and JavaScript code that enables the browser to render the page.
- The `h:form` element (p. 1251) contains the components with which a user interacts to provide data, such as registration or login information, to a JSF app.
- An `h:panelGrid` element (p. 1252) organizes elements in an XHTML table. The `columns` attribute specifies the number of columns in the table. The `style` attribute specifies the CSS formatting for the table.
- A `h:graphicImage` (p. 1253) displays an image (specified by the `name` attribute) in the page.
- As of JSF 2.0, you add resources (p. 1253) that are used throughout your app—such as images, CSS files, JavaScript files—to your web apps by placing them in the app's `resources` folder within your project's **Web Pages** node. Each subfolder of resources represents a resource library (p. 1253).
- An empty element (p. 1253) does not have content between its start and end tags. In such an element, data can be specified as attributes in the start tag. You can close an empty element either by placing a slash immediately preceding the start tag's right angle bracket or by explicitly writing an end tag.
- An `h:selectOneMenu` element (p. 1254) is typically rendered in a web page as a drop-down list. Each item to display appears between the start and end tags of this element as an `f:selectItem` element (from the JSF Core Tag Library; p. 1254). An `f:selectItem`'s `itemLabel` is the string that the user will see in the browser, and its `itemValue` is the value that's returned when you programmatically retrieve the user's selection from the drop-down list.
- An `h:outputLink` element (p. 1254) inserts a hyperlink in a web page. Its `value` attribute specifies the resource that's requested when a user clicks the hyperlink.
- An `h:selectOneRadio` element (p. 1254) provides a series of radio buttons from which the user can select only one.
- An `h:commandButton` element (p. 1254) triggers an action when clicked. An `h:commandButton` typically maps to an XHTML `input` element with its `type` attribute set to `"submit"`. Such elements are often used to submit a form's user input values to the server for processing.

Section 29.7 Validation Using JSF Standard Validators

- Form validation (p. 1254) helps prevent processing errors due to incomplete or improperly formatted user input.
- An `f:validateLength` validator (p. 1254) determines whether a field contains an acceptable number of characters.
- `f:validateDoubleRange` and `f:validateLongRange` validators (p. 1254) determine whether numeric input falls within acceptable ranges.
- An `f:validateRequired` validator (p. 1254) determines whether a field contains a value.
- An `f:validateRegex` validator (p. 1254) determines whether a field contains a string that matches a specified regular expression pattern.

- An `f:validateBean` validator (p. 1254) invokes a bean method that performs custom validation.
- An `h:outputText` element (p. 1259) inserts text in a page.
- An input element's `required` attribute (when set to `"true"`; p. 1259) ensures that the user has made a selection or entered some text in a required input element is a basic type of validation.
- An input element's `requiredMessage` attribute (p. 1259) specifies the message that should be displayed if the user submits the form without first providing a value for the required element.
- An input element's `validatorMessage` attribute (p. 1259) specifies the message to display if a validator is associated with the element and the data the user enters is invalid.
- The messages specified by the `requiredMessage` and `validatorMessage` attributes are displayed in an associated `h:message` element (p. 1259) when validation fails.
- To limit the length of user input in an `h:inputText`, set its `maxlength` attribute (p. 1260)—the element's cursor will not advance beyond the maximum allowable number of characters.
- In a postback (p. 1261), the browser re-requests the page and sends the values of the form's fields to the server for processing.

Section 29.8 Session Tracking

- Personalization (p. 1261) makes it possible for e-businesses to communicate effectively with their customers and also improves the user's ability to locate desired products and services.
- A trade-off exists between personalized e-business service and protection of privacy. Some consumers embrace the idea of tailored content, but others fear the possible adverse consequences if the information they provide to e-businesses is released or collected by tracking technologies.
- HTTP is a stateless protocol—it does not provide information that would enable web servers to maintain state information regarding particular clients.
- To help the server distinguish among clients, each client must identify itself to the server. Tracking individual clients, known as session tracking, can be achieved in a number of ways. One popular technique uses cookies; another uses the `@SessionScoped` annotation.

Section 29.8.1 Cookies

- A cookie (p. 1262) is a piece of data stored in a small text file on the user's computer. A cookie maintains information about the client during and between browser sessions.
- The expiration date (p. 1263) of a cookie determines how long the cookie remains on the client's computer. If you do not set an expiration date for a cookie, the web browser maintains the cookie for the duration of the browsing session.

Section 29.8.2 Session Tracking with `@SessionScoped` Beans

- A `@SessionScoped` bean (p. 1263) can maintain a user's selections throughout the user's session. Such a bean is created when a session begins and exists throughout the entire session.
- A `@SessionScoped` bean can be accessed by all of the app's pages, and the app server maintains a separate `@SessionScoped` bean for each user.
- By default a session expires after 30 minutes of inactivity or when the user closes the browser that was used to begin the session. When the session expires, the server discards the bean that was associated with that session.
- The `ui:repeat` element (from the JSF Facelets Tag Library; p. 1269) inserts its nested element(s) once for each element in a collection. The collection is specified by the `value` attribute's EL expression, which must return an array, a `List`, a `java.sql.ResultSet` or an `Object`.
- The `ui:repeat` element's `var` attribute creates a variable named book to which each item in the collection is assigned in sequence.

Self-Review Exercises

29.1 State whether each of the following is *true* or *false*. If *false*, explain why.

a) A URL contains information that directs a browser to the resource that the user wishes to access.
b) Host names are translated into IP addresses by web servers.
c) The path in a URL typically specifies a resource's exact location on the server.
d) `GET` requests and `POST` requests can both be used to send form data to a web server.
e) Browsers typically cache the server's response to a `POST` request.
f) A tag library defines a set of elements that can be inserted into the XHTML markup.
g) You must create and manage the JavaBean objects that are used in your JSF web applications.
h) When the Faces servlet encounters an EL expression that accesses a bean property, it automatically invokes the property's *set* or *get* method based on the context in which the property is used.
i) An `h:panelGrid` element organizes elements in an XHTML table.
j) An `h:selectOneMenu` element is typically rendered in a web page as a set of radio buttons.
k) The messages specified by an element's `requiredMessage` and `validatorMessage` attributes are displayed in an associated `h:message` element when validation fails.
l) The HTTP protocol provides information that enables web servers to maintain state information regarding particular clients.
m) The `ui:repeat` element inserts its nested element(s) once for each element in a collection. The collection can be any `IEnumerable` type.

29.2 Fill in the blanks in each of the following statements:

a) Java multitier applications are typically implemented using the features of _____.
b) Computers that run _____ software make resources available, such as web pages, images, PDF documents and even objects that perform complex tasks.
c) The JSF web-application framework's _____ processes each requested JSF page.
d) A(n) _____ exposes its data as read/write, read-only or write-only properties.
e) The _____ annotation indicates that the JSF framework should create and manage instances of the class.
f) A(n) _____ element contains the components with which a user interacts to provide data, such as registration or login information, to a JSF app.
g) A(n) _____ element triggers an action when clicked.
h) A(n) _____ validator determines whether a field contains an acceptable number of characters.
i) A(n) _____ validator determines whether a field contains a string that matches a specified regular expression pattern.
j) In a(n) _____, the browser re-requests the page and sends the values of the form's fields to the server for processing.
k) A(n) _____bean is created when a session begins and exists throughout the entire session.

Answers to Self-Review Exercises

29.1 a) True. b) False. Host names are translated into IP addresses by DNS servers. c) False. The server translates a virtual directory into a real location, thus hiding a resource's true location. d) True. e) False. Browsers typically do not cache the server's response to a `POST` request, because the next `POST` might not return the same result. f) True. g) False. The JSF framework creates and manages objects of your JavaBean classes for you. h) True. i) True. j) False. An `h:selectOneRadio` ele-

ment is rendered as a set of radio buttons. An `h:selectOneMenu` is rendered as a drop-down list. k) True. l) False. HTTP is a stateless protocol that does not provide information that enables web servers to maintain state information regarding particular clients—a separate tracking technology must be used. m) False. A `ui:repeat` element can iterate over only arrays, `Lists` and `ResultSets`. For any other object, the elements in a `ui:repeat` element will be inserted once.

29.2 a) Java Enterprise Edition (Java EE). b) web-server. c) Faces servlet. d) JavaBean. e) `@ManagedBean`. f) `h:form`. g) `h:commandButton`. h) `f:validateLength`. i) `f:validateRegex`. j) postback. k) `@SessionScoped`.

Exercises

29.3 *(Registration Form Modification)* Modify the `WebComponents` application to add functionality to the **Register** button. When the user clicks **Register**, validate all input fields to make sure the user has filled out the form completely and entered a valid email address and phone number. Then, display a message indicating successful registration and show the user's registration information at the bottom of the page. (This is similar to the example in Section 29.7.) You'll need to create an appropriate bean class to store the user's registration information.

29.4 *(Shopping Cart Application)* Using the techniques you learned in Section 29.8.2, create a simple shopping cart application. Display a list of books as an `h:selectOneRadio` element. When the user submits the form, store the user's selection in a `@SessionScoped` managed bean. Allow the user to return to the list of books and make additional selections. Provide a link to view the shopping cart. On the shopping cart page, display the list of selections the user made, the price of each book and the total of all books in the cart.

29.5 *(`Guestbook` Application)* In Section 29.8.2, you used an `@SessionScoped` managed bean to maintain an individual user's selections. JSF also provides the **`@ApplicationScoped` annotation** for managed beans that should be shared among all users of a JSF app. For this exercise, create an `@ApplicationScoped GuestbookBean` that maintains a `List` of `GuestbookEntry` objects. The application should provide a form, similar to the one in Fig. 29.16 that enables a user to enter a name, an email address and a message. When the user submits the form, the `GuestbookBean` should create a `GuestbookEntry` object containing the submitted values and insert it at the beginning of the `List` of `GuestbookEntry` objects—this places the most recent entry first. The `GuestbookBean` should also provide a read-only property that returns the `List` of `GuestbookEntry` objects. The page should use a `ui:repeat` element to display all of the items in the List so the user can see all the guestbook entries so far. [*Note:* In the next chapter, you'll implement a similar exercise that stores this information in a database.]

30

JavaServer™ Faces Web Apps: Part 2



Whatever is in any way beautiful hath its source of beauty in itself, and is complete in itself; praise forms no part of it.
—Marcus Aurelius Antoninus

There is something in a face, An air, and a peculiar grace, Which boldest painters cannot trace.
—William Somerville

Cato said the best way to keep good acts in memory was to refresh them with new.
—Francis Bacon

I never forget a face, but in your case I'll make an exception.
—Groucho Marx

Objectives

In this chapter you'll learn:

- To access databases from JSF applications.
- The basic principles and advantages of Ajax technology.
- To use Ajax in a JSF web app.

Outline

30.1 Introduction
30.2 Accessing Databases in Web Apps
30.2.1 Setting Up the Database
30.2.2 `@ManagedBean` Class `AddressBean`
30.2.3 `index.xhtml` Facelets Page
30.2.4 `addentry.xhtml` Facelets Page
30.3 Ajax
30.4 Adding Ajax Functionality to the Validation App
30.5 Wrap-Up

Summary | *Self-Review Exercise* | *Answers to Self-Review Exercise* | *Exercises*

30.1 Introduction

This chapter continues our discussion of JSF web application development with two additional examples. In the first, we present a simple address book app that retrieves data from and inserts data into a Java DB database. The app allows users to view the existing contacts in the address book and to add new contacts. In the second example, we add so-called *Ajax* capabilities to the `Validation` example from Section 29.7. As you'll learn, Ajax improves application performance and responsiveness. This chapter's examples, like those in Chapter 29, were developed in NetBeans.

30.2 Accessing Databases in Web Apps

Many web apps access databases to store and retrieve persistent data. In this section, we build an address book web app that uses a Java DB database display contacts from the address book on a web page and to store contacts in the address book. Figure 30.1 shows sample interactions with the `AddressBook` app.

a) Table of addresses displayed when the `AddressBook` app is first requested

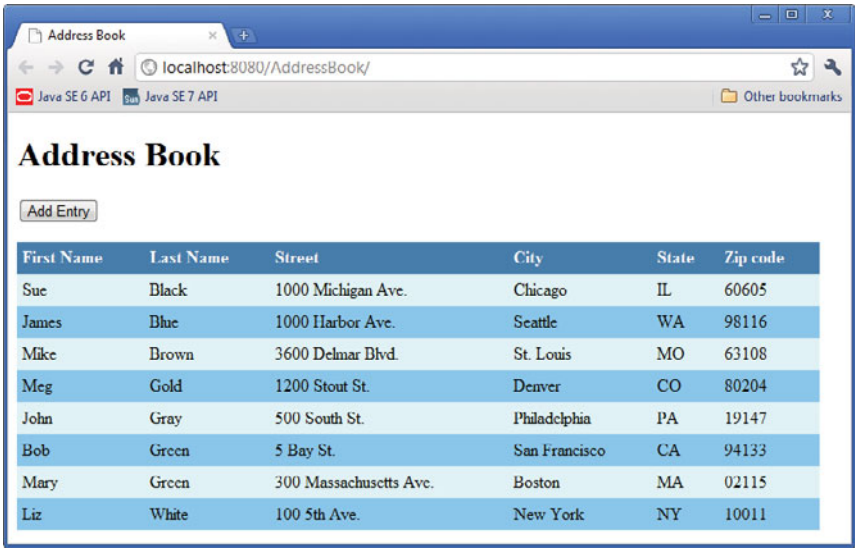


Fig. 30.1 | Sample outputs from the `AddressBook` app. (Part 1 of 2.)

b) Form for adding an entry



c) Table of addresses updated with the new entry added in Part (b)



Fig. 30.1 | Sample outputs from the `AddressBook` app. (Part 2 of 2.)

If the app's database already contains addresses, the initial request to the app displays those addresses as shown in Fig. 30.1(a). We populated the database with the sample addresses shown. When the user clicks **Add Entry**, the `addentry.xhtml` page is displayed (Fig. 30.1(b)). When the user clicks **Save Address**, the form's fields are validated. If the validations are successful, the address is added to the database and the app returns to the `index.xhtml` page to show the updated list of addresses (Fig. 30.1(c)). This example also introduces the `h:dataTable` element for displaying data in tabular format.

The next several sections explain how to build the `AddressBook` application. First, we set up the database (Section 30.2.1). Next, we present class `AddressBean` (Section 30.2.2), which enables the app's Facelets pages to interact with the database. Finally, we present the `index.xthml` (Section 30.2.3) and `addentry.xhtml` (Section 30.2.4) Facelets pages.

30.2.1 Setting Up the Database

You'll now create a *data source* that enables the app to interact with the database. As part of this process, you'll create the `addressbook` database and populate it with sample data.

Open NetBeans and Ensure that Java DB and GlassFish Are Running

Before you can create the data source in NetBeans, the IDE must be open and the Java DB and GlassFish servers must be running. Perform the following steps:

1. Open the NetBeans IDE.
2. On the **Services** tab, expand the **Databases** node then right click **Java DB**. If **Java DB** is not already running the **Start Server** option will be enabled. In this case, **Select Start** server to launch the Java DB server.
3. On the **Services** tab, expand the **Servers** node then right click **GlassFish Server 3**. If **GlassFish Server 3** is not already running the **Start** option will be enabled. In this case, **Start** server to launch GlassFish.

You may need to wait a few moments for the servers to begin executing.

Creating a Connection Pool

In web apps that receive many requests, it's inefficient to create separate database connections for each request. Instead, you should set up a **connection pool** to allow the server to manage a limited number of database connections and share them among requests. To create a connection pool for this app, perform the following steps:

1. On the **Services** tab, expand the **Servers** node, right click **GlassFish Server 3** and select **View Admin Console**. This opens your default web browser and displays a web page for configuring the GlassFish server.
2. In the left column of the page under **Common Tasks**, expand the **Resources** node, then expand its **JDBC** node to show the **JDBC Resources** and **Connection Pools** nodes (Fig. 30.2).

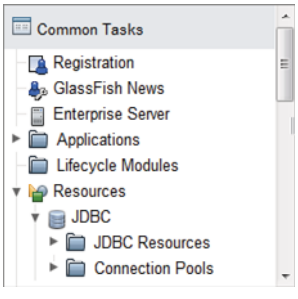


Fig. 30.2 | **Common Tasks** window in the GlassFish server configuration web page.

3. Click the **Connection Pools** node to display the list of existing connection pools, then click the **New...** button above the list to create a new connection pool.
4. In the **New JDBC Connection Pool (Step 1 of 2)** page (Fig. 30.3), specify `AddressBookPool` for the **Name**, select `javax.sql.DataSource` for the **Resource Type** and select **JavaDB** for the **Database Vendor**, then click **Next**.
5. In the **New JDBC Connection Pool (Step 2 of 2)** page (Fig. 30.4), scroll to the **Additional Properties** table and specify the following values (leave the other entries in the table unchanged):
   - **ConnectionAttributes:** `;create=true` (specifies that the database should be created when the connection pool is created)
   - **DatabaseName:** `addressbook` (specifies the name of the database to create)
   - **Password:** `APP` (specifies the password for the database—the **User** name is already specified as `APP` in the **Additional Properties** table; you can specify any **User** name and **Password** you like)

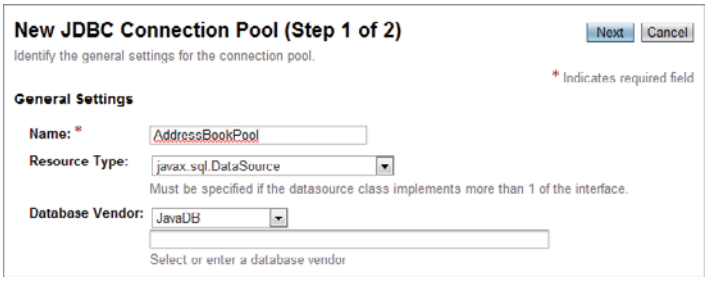


Fig. 30.3 | **New JDBC Connection Pool (Step 1 of 2)** page.

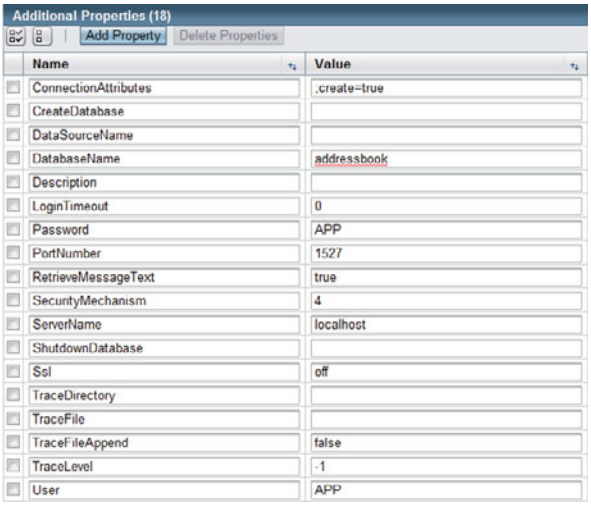


Fig. 30.4 | **New JDBC Connection Pool (Step 2 of 2)** page.

6. Click **Finish** to create the connection pool and return to the connection pools list.
7. Click `AddressBookPool` in the connection pools list to display the **Edit JDBC Connection Pool** page, then click **Ping** in that page to test the database connection and ensure that you set it up correctly.

Creating a Data Source Name

To connect to the database from the web app, you must configure a **data source name** that will be used to locate the database. The data source name must be associated with the connection pool that manages the connections to the database. Perform the following steps:

1. In the left column of the GlassFish configuration web page, click the **JDBC Resources** node to display the list of data source names, then click the **New...** button to display the **New JDBC Resource** page (Fig. 30.5).
2. Specify `jdbc/addressbook` as the **JNDI Name** and select `AddressBookPool` as the **Pool Name**. Then click **OK**. JNDI (Java Naming and Directory Interface) is a technology for locating application components (such as databases) in a distributed application (such as a multitier web application). You can now close the GlassFish configuration web page.

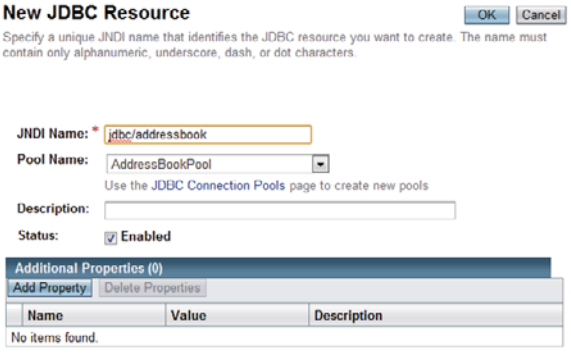


Fig. 30.5 | **New JDBC Resource** page.

Populate the `addressbook` *Database with Sample Data*

You'll now populate the database with sample data using the `AddressBook.sql` SQL script that's provided with this chapter's examples. To do so, you must create a connection to the new `addressbook` database from NetBeans. Perform the following steps:

1. On the NetBeans **Services** tab, right click the **Databases** node and select **New Connection...**.
2. In the **New Database Connection** dialog, specify `localhost` as the **Host**, `1527` as the **Port**, `addressbook` as the **Database**, `APP` as the **User Name** and `APP` as the **Password**, then select the **Remember password** checkbox and click **OK**.

The preceding steps create a new entry in the **Databases** node showing the database's URL (`jdbc:derby://localhost:1527/addressbook`). The database server that provides access to this database resides on the local machine and accepts connections on port 1527.

NetBeans must be connected to the database to execute SQL statements. If NetBeans is already connected to the database, the icon is displayed next to the database's URL; otherwise, the icon is displayed. In this case, right click the icon and select **Connect...**.

To populate the database with sample data, perform the following steps:

1. Click the + next to `jdbc:derby://localhost:1527/addressbook` node to expand it, then expand the database's **APP** node.
2. Right click the **Tables** node and select **Execute Command...** to open a **SQL Command** editor tab in NetBeans. In a text editor, open the file `AddressBook.sql` from this chapter's examples folder, then copy the SQL statements and paste them into the **SQL Command** editor in NetBeans. Next, right click in the **SQL Command** editor and select **Run File**. This will create the `Addresses` table with the sample data in Fig. 30.1(a). [*Note:* The SQL script attempts to remove the database's `Addresses` table if it already exists. If it doesn't exist, you'll receive an error message, but the table will still be created properly.] Expand the **Tables** node to see the new table. You can view the table's data by right clicking **ADDRESSES** and selecting **View Data...**. Notice that we named the columns with all capital letters. We'll be using these names in Section 30.2.3.

30.2.2 @ManagedBean Class AddressBean

[*Note:* To build this app from scratch, use the techniques you learned in Chapter 29 to create a JSF web application named `AddressBook` and add a second Facelets page named `addentry.xhtml` to the app.] Class `AddressBean` (Fig. 30.6) enables the `AddressBook` app to interact with the `addressbook` database. The class provides properties that represent the first name, last name, street, city, state and zip code for an entry in the database. These are used by the `addentry.xhtml` page when adding a new entry to the database. In addition, this class declares a `DataSource` (lines 26–27) for interacting with the database method `getAddresses` (lines 102–130) for obtaining the list of addresses from the database and method `save` (lines 133–169) for saving a new address into the database. These methods use various JDBC techniques you learned in Chapter 28.

```
1  // AddressBean.java
2  // Bean for interacting with the AddressBook database
3  package addressbook;
4
5  import java.sql.Connection;
6  import java.sql.PreparedStatement;
7  import java.sql.ResultSet;
8  import java.sql.SQLException;
9  import javax.annotation.Resource;
10 import javax.faces.bean.ManagedBean;
11 import javax.sql.DataSource;
12 import javax.sql.rowset.CachedRowSet;
13
14 @ManagedBean( name="addressBean" )
15 public class AddressBean
16 {
```

Fig. 30.6 | `AddressBean` interacts with a database to store and retrieve addresses. (Part 1 of 4.)

```
17    // instance variables that represent one address
18    private String firstName;
19    private String lastName;
20    private String street;
21    private String city;
22    private String state;
23    private String zipcode;
24
25    // allow the server to inject the DataSource
26    @Resource( name="jdbc/addressbook" )
27    DataSource dataSource;
28
29    // get the first name
30    public String getFirstName()
31    {
32       return firstName;
33    } // end method getFirstName
34
35    // set the first name
36    public void setFirstName( String firstName )
37    {
38       this.firstName = firstName;
39    } // end method setFirstName
40
41    // get the last name
42    public String getLastName()
43    {
44       return lastName;
45    } // end method getLastName
46
47    // set the last name
48    public void setLastName( String lastName )
49    {
50       this.lastName = lastName;
51    } // end method setLastName
52
53    // get the street
54    public String getStreet()
55    {
56       return street;
57    } // end method getStreet
58
59    // set the street
60    public void setStreet( String street )
61    {
62       this.street = street;
63    } // end method setStreet
64
65    // get the city
66    public String getCity()
67    {
68       return city;
69    } // end method getCity
```

Fig. 30.6 | `AddressBean` interacts with a database to store and retrieve addresses. (Part 2 of 4.)

```
70
71      // set the city
72      public void setCity( String city )
73      {
74         this.city = city;
75      } // end method setCity
76
77      // get the state
78      public String getState()
79      {
80         return state;
81      } // end method getState
82
83      // set the state
84      public void setState( String state )
85      {
86         this.state = state;
87      } // end method setState
88
89      // get the zipcode
90      public String getZipcode()
91      {
92         return zipcode;
93      } // end method getZipcode
94
95      // set the zipcode
96      public void setZipcode( String zipcode )
97      {
98         this.zipcode = zipcode;
99      } // end method setZipcode
100
101     // return a ResultSet of entries
102     public ResultSet getAddresses() throws SQLException
103     {
104        // check whether dataSource was injected by the server
105        if ( dataSource == null )
106           throw new SQLException( "Unable to obtain DataSource" );
107
108        // obtain a connection from the connection pool
109        Connection connection = dataSource.getConnection();
110
111        // check whether connection was successful
112        if ( connection == null )
113           throw new SQLException( "Unable to connect to DataSource" );
114
115        try
116        {
117           // create a PreparedStatement to insert a new address book entry
118           PreparedStatement getAddresses = connection.prepareStatement(
119              "SELECT FIRSTNAME, LASTNAME, STREET, CITY, STATE, ZIP " +
120              "FROM ADDRESSES ORDER BY LASTNAME, FIRSTNAME" );
121
122           CachedRowSet rowSet = new com.sun.rowset.CachedRowSetImpl();
```

Fig. 30.6 | `AddressBean` interacts with a database to store and retrieve addresses. (Part 3 of 4.)

```
123            rowSet.populate( getAddresses.executeQuery() );
124            return rowSet;
125         } // end try
126         finally
127         {
128            connection.close(); // return this connection to pool
129         } // end finally
130      } // end method getAddresses
131
132      // save a new address book entry
133      public String save() throws SQLException
134      {
135         // check whether dataSource was injected by the server
136         if ( dataSource == null )
137            throw new SQLException( "Unable to obtain DataSource" );
138
139         // obtain a connection from the connection pool
140         Connection connection = dataSource.getConnection();
141
142         // check whether connection was successful
143         if ( connection == null )
144            throw new SQLException( "Unable to connect to DataSource" );
145
146         try
147         {
148            // create a PreparedStatement to insert a new address book entry
149            PreparedStatement addEntry =
150               connection.prepareStatement( "INSERT INTO ADDRESSES " +
151                  "(FIRSTNAME,LASTNAME,STREET,CITY,STATE,ZIP)" +
152                  "VALUES ( ?, ?, ?, ?, ?, ? )" );
153
154            // specify the PreparedStatement's arguments
155            addEntry.setString( 1, getFirstName() );
156            addEntry.setString( 2, getLastName() );
157            addEntry.setString( 3, getStreet() );
158            addEntry.setString( 4, getCity() );
159            addEntry.setString( 5, getState() );
160            addEntry.setString( 6, getZipcode() );
161
162            addEntry.executeUpdate(); // insert the entry
163            return "index"; // go back to index.xhtml page
164         } // end try
165         finally
166         {
167            connection.close(); // return this connection to pool
168         } // end finally
169      } // end method save
170   } // end class AddressBean
```

Fig. 30.6 | `AddressBean` interacts with a database to store and retrieve addresses. (Part 4 of 4.)

Injecting the `DataSource` *into Class* `AddressBean`

A **`DataSource`** (package `javax.sql`) enables a web application to obtain a `Connection` to a database. Lines 26–27 use annotation **`@Resource`** to inject a `DataSource` object into the

`AddressBean`. The annotation's `name` attribute specifies `java/addressbook`—the JNDI name from the *Creating a Data Source Name* step of Section 30.2.1. The `@Resource` annotation enables the server (GlassFish in our case) to hide all the complex details of locating the connection pool that we set up for interacting with the `addressbook` database. The server creates a `DataSource` for you that's configured to use that connection pool and assigns the `DataSource` object to the annotated variable declared at line 27. You can now trivially obtain a `Connection` for interacting with the database.

`AddressBean` Method `getAddresses`

Method `getAddresses` (lines 102–130) is called when the `index.xhtml` page is requested. The method returns a list of addresses for display in the page (Section 30.2.3). First, we check whether variable `dataSource` is `null` (lines 105–106), which would indicate that the server was unable to create the `DataSource` object. If the `DataSource` was created successfully, we use it to obtain a `Connection` to the database (line 109). Next, we check whether variable `connection` is `null` (lines 112–113), which would indicate that we were unable to connect. If the connection was successful, lines 118–124 get the set of addresses from the database and return them.

The `PreparedStatement` at lines 118–120 obtains all the addresses. Because database connections are a limited resources, you should use and close them quickly in your web apps. For this reason, we create a `CachedRowSet` and populate it with the `ResultSet` returned by the `PreparedStatement`'s `executeQuery` method (lines 122–123). We then return the `CachedRowSet` (a disconnected `RowSet`) for use in the `index.xhtml` page (line 124) and close the `connection` object (line 128) in the `finally` block.

`AddressBean` Method `save`

Method `save` (lines 133–169) stores a new address in the database (Section 30.2.4). This occurs when the user submits the `addentry.xhtml` form—assuming the form's fields validate successfully. As in `getAddresses`, we ensure that the `DataSource` is not `null`, then obtain the `Connection` object and ensure that its not `null`. Lines 149–152 create a `PreparedStatement` for inserting a new record in the database. Lines 155–160 specify the values for each of the parameters in the `PreparedStatement`. Line 162 then executes the `PreparedStatement` to insert the new record. Line 163 returns the string `"index"`, which as you'll see in Section 30.2.4 causes the app to display the `index.xhtml` page again.

30.2.3 `index.xhtml` Facelets Page

`index.xhtml` (Fig. 30.7) is the default web page for the `AddressBook` app. When this page is requested, it obtains the list of addresses from the `AddressBean` and displays them in tabular format using an **`h:dataTable` element**. The user can click the **Add Entry** button (line 17) to view the `addentry.xhtml` page. Recall that the default action for an `h:commandButton` is to submit a form. In this case, we specify the button's **`action` attribute** with the value `"addentry"`. The JSF framework assumes this is a page in the app, appends `.xhtml` extension to the `action` attribute's value and returns the `addentry.xhtml` page to the client browser.

The `h:dataTable` Element

The `h:dataTable` element (lines 19–46) inserts tabular data into a page. We discuss only the attributes and nested elements that we use here. For more details on this element, its attributes and other JSF tag library elements, visit `bit.ly/JSF2TagLibraryReference`.

```
1  <?xml version='1.0' encoding='UTF-8' ?>
2
3  <!-- index.html -->
4  <!-- Displays an h:dataTable of the addresses in the address book -->
5  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
6     "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
7  <html xmlns="http://www.w3.org/1999/xhtml"
8     xmlns:h="http://java.sun.com/jsf/html"
9     xmlns:f="http://java.sun.com/jsf/core">
10    <h:head>
11       <title>Address Book</title>
12       <h:outputStylesheet name="style.css" library="css"/>
13    </h:head>
14    <h:body>
15       <h1>Address Book</h1>
16       <h:form>
17          <p><h:commandButton value="Add Entry" action="addentry"/></p>
18       </h:form>
19       <h:dataTable value="#{addressBean.addresses}" var="address"
20          rowClasses="oddRows,evenRows" headerClass="header"
21          styleClass="table" cellpadding="5" cellspacing="0">
22          <h:column>
23             <f:facet name="header">First Name</f:facet>
24             #{address.FIRSTNAME}
25          </h:column>
26          <h:column>
27             <f:facet name="header">Last Name</f:facet>
28             #{address.LASTNAME}
29          </h:column>
30          <h:column>
31             <f:facet name="header">Street</f:facet>
32             #{address.STREET}
33          </h:column>
34          <h:column>
35             <f:facet name="header">City</f:facet>
36             #{address.CITY}
37          </h:column>
38          <h:column>
39             <f:facet name="header">State</f:facet>
40             #{address.STATE}
41          </h:column>
42          <h:column>
43             <f:facet name="header">Zip code</f:facet>
44             #{address.ZIP}
45          </h:column>
46       </h:dataTable>
47    </h:body>
48 </html>
```

Fig. 30.7 | Displays an `h:dataTable` of the addresses in the address book.

The `h:dataTable` element's **`value` attribute** (line 19) specifies the collection of data you wish to display. In this case, we use `AddressBean`'s `addresses` property, which calls

the `getAddresses` method (Fig. 30.6). The collection returned by this method is a `CachedRowSet`, which is a type of `ResultSet`.

The `h:dataTable` iterates over its `value` collection and, one at a time, assigns each element to the variable specified by the **`var` attribute**. This variable is used in the `h:dataTable`'s nested elements to access each element of the collection—each element in this case represents one row (i.e., address) in the `CachedRowSet`.

The **`rowClasses` attribute** (line 20) is a space-separated list of CSS style class names that are used to style the rows in the tabular output. These style classes are defined in the app's `styles.css` file in the `css` library (which is inserted into the document at line 12). You can open this file to view the various style class definitions. We specified two style classes— all the odd numbered rows will have the first style (`oddRows`) and all the even numbered rows the second style (`evenRows`). You can specify as many styles as you like—they'll be applied in the order you list them one row at a time until all the styles have been applied, then the `h:DataTable` will automatically cycle through the styles again for the next set of rows. The **`columnClasses` attribute** works similarly for columns in the table.

The **`headerClass` attribute** (line 20) specifies the column header CSS style. Headers are defined with `f:facet` elements nested in `h:column` elements (discussed momentarily). The **`footerClass` attribute** works similarly for column footers in the table.

The **`styleClass` attribute** (line 21) specifies the CSS styles for the entire table. The **`cellpadding`** and **`cellspacing`** attributes (line 21) specify the number of pixels around each table cell's contents and the number of pixels between table cells, respectively.

The `h:column` Elements

Lines 22–45 define the table's columns with six nested **`h:column` elements**. We focus here on the one at lines 22–25. When the `CachedRowSet` is populated in the `AddressBean` class, it automatically uses the database's column names as property names for each row object in the `CachedRowSet`. Line 28 inserts into the column the `FIRSTNAME` property of the `CachedRowSet`'s current row. To display a column header above the column, you define an **`f:facet` element** (line 23) and set its `name` attribute to `"header"`. Similarly, to display a column footer, use an `f:facet` with its `name` attribute set to `"footer"`. The header is formatted with the CSS style specified in the `h:dataTable`'s `headerClass` attribute (line 20). The remaining `h:column` elements perform similar tasks for the current row's `LASTNAME`, `STREET`, `CITY`, `STATE` and `ZIP` properties.

30.2.4 `addentry.xhtml` Facelets Page

When the user clicks **Add Entry** in the `index.xhtml` page, `addentry.xhtml` (Fig. 30.8) is displayed. Each `h:inputText` in this page has its `required` attribute set to `"true"` and includes a `maxlength` attribute that restricts the user's input to the maximum length of the corresponding database field. When the user clicks **Save** (lines 48–49), the input element's values are validated and (if successful) assigned to the properties of the `addressBean` managed object. In addition, the button specifies as its `action` the EL expression

```
#{addressBean.save}
```

which invokes the `addressBean` object's `save` method to store the new address in the database. When you call a method with the `action` attribute, if the method returns a value (in

this case, it returns the string "index"), that value is used to request the corresponding page from the app. If the method does not return a value, the current page is re-requested.

```
<?xml version='1.0' encoding='UTF-8' ?>

<!-- addentry.html -->
<!-- Form for adding an entry to an address book -->
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
   "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml"
   xmlns:h="http://java.sun.com/jsf/html">
   <h:head>
      <title>Address Book: Add Entry</title>
      <h:outputStylesheet name="style.css" library="css"/>
   </h:head>
   <h:body>
      <h1>Address Book: Add Entry</h1>
      <h:form>
         <h:panelGrid columns="3">
            <h:outputText value="First name:"/>
            <h:inputText id="firstNameInputText" required="true"
               requiredMessage="Please enter first name"
               value="#{addressBean.firstName}" maxlength="30"/>
            <h:message for="firstNameInputText" styleClass="error"/>
            <h:outputText value="Last name:"/>
            <h:inputText id="lastNameInputText" required="true"
               requiredMessage="Please enter last name"
               value="#{addressBean.lastName}" maxlength="30"/>
            <h:message for="lastNameInputText" styleClass="error"/>
            <h:outputText value="Street:"/>
            <h:inputText id="streetInputText" required="true"
               requiredMessage="Please enter the street address"
               value="#{addressBean.street}" maxlength="150"/>
            <h:message for="streetInputText" styleClass="error"/>
            <h:outputText value="City:"/>
            <h:inputText id="cityInputText" required="true"
               requiredMessage="Please enter the city"
               value="#{addressBean.city}" maxlength="30"/>
            <h:message for="cityInputText" styleClass="error"/>
            <h:outputText value="State:"/>
            <h:inputText id="stateInputText" required="true"
               requiredMessage="Please enter state"
               value="#{addressBean.state}" maxlength="2"/>
            <h:message for="stateInputText" styleClass="error"/>
            <h:outputText value="Zipcode:"/>
            <h:inputText id="zipcodeInputText" required="true"
               requiredMessage="Please enter zipcode"
               value="#{addressBean.zipcode}" maxlength="5"/>
            <h:message for="zipcodeInputText" styleClass="error"/>
         </h:panelGrid>
         <h:commandButton value="Save Address"
            action="#{addressBean.save}"/>
      </h:form>
```

Fig. 30.8 | Form for adding an entry to an address book. (Part 1 of 2.)

```
51          <h:outputLink value="index.xhtml">Return to Addresses</h:outputLink>
52       </h:body>
53    </html>
```

Fig. 30.8 | Form for adding an entry to an address book. (Part 2 of 2.)

30.3 Ajax

The term **Ajax**—short for **Asynchronous JavaScript and XML**—was coined by Jesse James Garrett of Adaptive Path, Inc., in 2005 to describe a range of technologies for developing highly responsive, dynamic web applications. Ajax applications include Google Maps, Yahoo's FlickR and many more. Ajax separates the *user interaction* portion of an application from its *server interaction*, enabling both to proceed *in parallel*. This enables Ajax web-based applications to perform at speeds approaching those of desktop applications, reducing or even eliminating the performance advantage that desktop applications have traditionally had over web-based applications. This has huge ramifications for the desktop applications industry—the applications platform of choice is shifting from the desktop to the web. Many people believe that the web—especially in the context of abundant open-source software, inexpensive computers and exploding Internet bandwidth—will create the next major growth phase for Internet companies.

Ajax makes **asynchronous** calls to the server to exchange small amounts of data with each call. *Where normally the entire page would be submitted and reloaded with every user interaction on a web page, Ajax allows only the necessary portions of the page to reload, saving time and resources.*

Ajax applications typically make use of client-side scripting technologies such as JavaScript to interact with page elements. They use the browser's **`XMLHttpRequest` object** to perform the asynchronous exchanges with the web server that make Ajax applications so responsive. This object can be used by most scripting languages to pass XML data from the client to the server and to process XML data sent from the server back to the client.

Using Ajax technologies in web applications can dramatically improve performance, but programming Ajax directly is complex and error prone. It requires page designers to know both scripting and markup languages. As you'll soon see, JSF 2.0 makes adding Ajax capabilities to your web apps fairly simple.

Traditional Web Applications

Figure 30.9 presents the typical interactions between the client and the server in a traditional web application, such as one that uses a user registration form. The user first fills in the form's fields, then *submits* the form (Fig. 30.9, *Step 1*). The browser generates a request to the server, which receives the request and processes it (*Step 2*). The server generates and sends a response containing the exact page that the browser will render (*Step 3*), which causes the browser to load the new page (*Step 4*) and temporarily makes the browser window blank. The client *waits* for the server to respond and *reloads the entire page* with the data from the response (*Step 4*). While such a **synchronous request** is being processed on the server, *the user cannot interact with the client web page*. If the user interacts with and submits another form, the process begins again (*Steps 5–8*).

This model was originally designed for a web of *hypertext documents*—what some people call the "brochure web." As the web evolved into a full-scale applications platform,

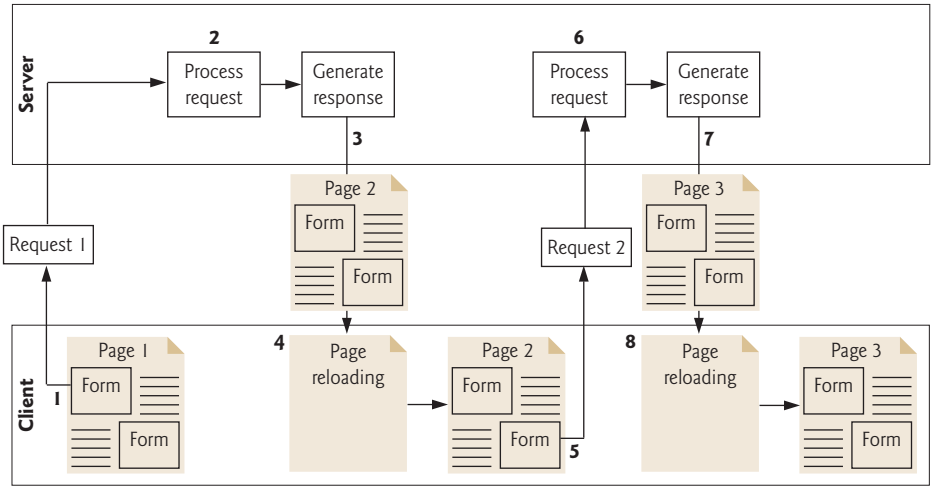


Fig. 30.9 | Classic web application reloading the page for every user interaction.

the model shown in Fig. 30.9 yielded "choppy" application performance. Every full-page refresh required users to reestablish their understanding of the full-page contents. Users began to demand a model that would yield the responsiveness of desktop applications.

Ajax Web Applications

Ajax applications add a layer between the client and the server to manage communication between the two (Fig. 30.10). When the user interacts with the page, the client creates an

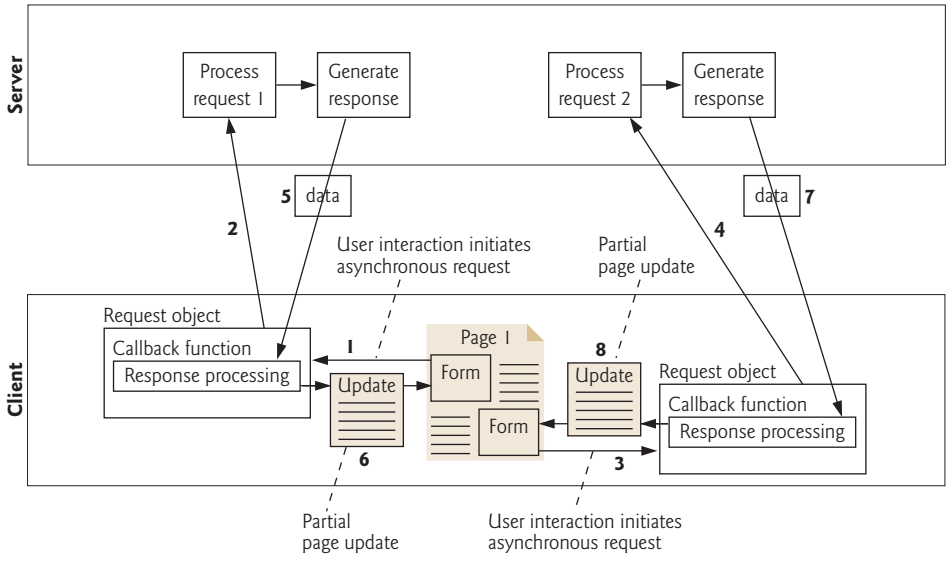


Fig. 30.10 | Ajax-enabled web application interacting with the server asynchronously.

`XMLHttpRequest` object to manage a request (*Step 1*). This object sends the request to the server (*Step 2*) and awaits the response. The requests are asynchronous, so the user can continue interacting with the application on the client side while the server processes the earlier request concurrently. Other user interactions could result in additional requests to the server (*Steps 3* and *4*). Once the server responds to the original request (*Step 5*), the `XMLHttpRequest` object that issued the request calls a client-side function to process the data returned by the server. This function—known as a **callback function**—uses **partial page updates** (*Step 6*) to display the data in the existing web page *without reloading the entire page*. At the same time, the server may be responding to the second request (*Step 7*) and the client side may be starting to do another partial page update (*Step 8*). The callback function updates only a designated part of the page. Such partial page updates help make web applications more responsive, making them feel more like desktop applications. The web application does not load a new page while the user interacts with it.

30.4 Adding Ajax Functionality to the Validation App

The example in this section adds Ajax capabilities to the `Validation` app that we presented in Section 29.7. Figure 30.11 shows the sample outputs from the `ValidationAjax` version of the app that we'll build momentarily. Part (a) shows the initial form that's displayed when this app first executes. Parts (b) and (c) show validation errors that are displayed when the user submits an empty form and invalid data, respectively. Part (d) shows the page after the form is submitted successfully.

As you can see, the app has the same functionality as the version in Section 29.7; however, you'll notice a couple of changes in how the app works. First, the URL displayed in the web browser always reads `localhost:8080/ValidationAjax/`, whereas the URL in the Section 29.7 changes after the form is submitted the first time. Also, in the non-Ajax version of the app, the page refreshes each time you press the **Submit** button. In the Ajax version, only the parts of the page that need updating actually change.

a) Submitting the form before entering any information

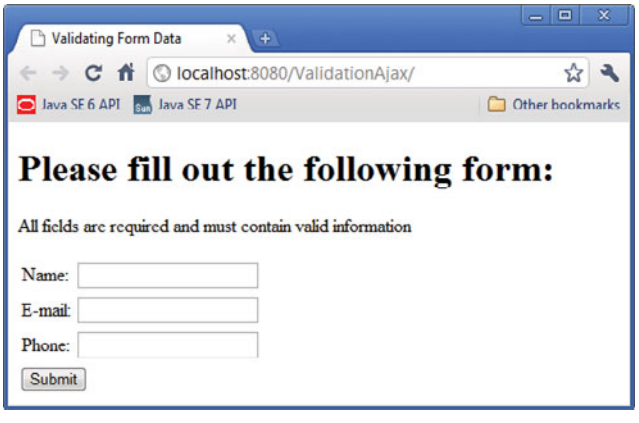
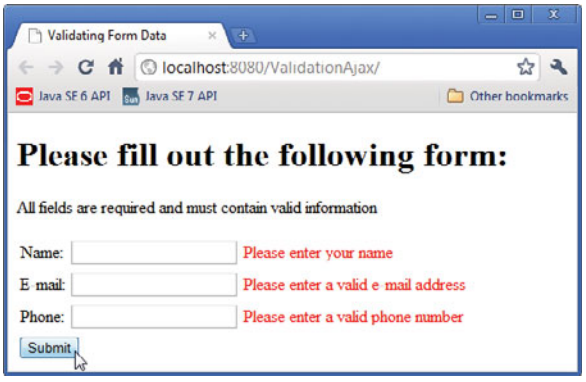
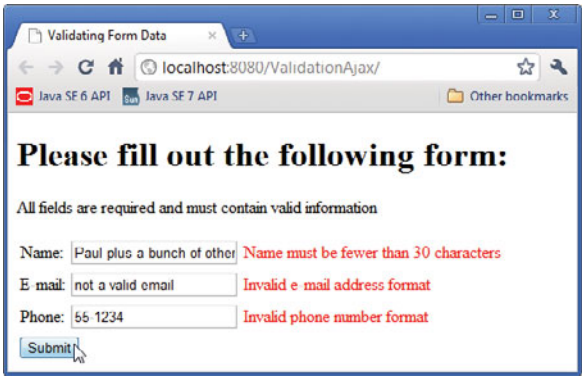


Fig. 30.11 | JSP that demonstrates validation of user input. (Part 1 of 2.)

b) Error messages displayed after submitting the empty form



c) Error messages displayed after submitting invalid information



d) Successfully submitted form

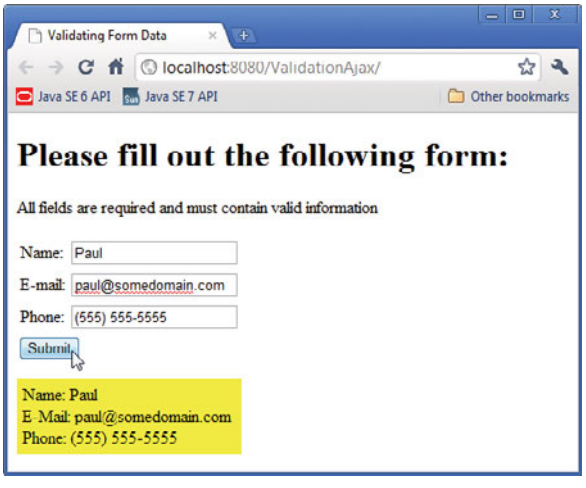


Fig. 30.11 | JSP that demonstrates validation of user input. (Part 2 of 2.)

index.xhtml

The changes required to add Ajax functionality to this app are minimal. All of the changes are in the `index.xhtml` file (Fig. 30.12) and are highlighted. The `ValidationBean` class is identical to the version in Section 29.7, so we don't show it here.

```
1  <?xml version='1.0' encoding='UTF-8' ?>
2
3  <!-- index.xhtml -->
4  <!-- Validating user input -->
5  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
6     "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
7  <html xmlns="http://www.w3.org/1999/xhtml"
8     xmlns:h="http://java.sun.com/jsf/html"
9     xmlns:f="http://java.sun.com/jsf/core">
10    <h:head>
11       <title>Validating Form Data</title>
12       <h:outputStylesheet name="style.css" library="css"/>
13    </h:head>
14    <h:body>
15       <h:form>
16          <h1>Please fill out the following form:</h1>
17          <p>All fields are required and must contain valid information</p>
18          <h:panelGrid columns="3">
19             <h:outputText value="Name:"/>
20             <h:inputText id="nameInputText" required="true"
21                requiredMessage="Please enter your name"
22                value="#{validationBean.name}"
23                validatorMessage="Name must be fewer than 30 characters">
24                <f:validateLength maximum="30" />
25             </h:inputText>
26             <h:message id="nameMessage" for="nameInputText"
27                styleClass="error"/>
28             <h:outputText value="E-mail:"/>
29             <h:inputText id="emailInputText" required="true"
30                requiredMessage="Please enter a valid e-mail address"
31                value="#{validationBean.email}"
32                validatorMessage="Invalid e-mail address format">
33                <f:validateRegex pattern=
34                   "\w+([-+.']\w+)*@\w+([-.]\w+)*\.\w+([-.]\w+)*" />
35             </h:inputText>
36             <h:message id="emailMessage" for="emailInputText"
37                styleClass="error"/>
38             <h:outputText value="Phone:"/>
39             <h:inputText id="phoneInputText" required="true"
40                requiredMessage="Please enter a valid phone number"
41                value="#{validationBean.phone}"
42                validatorMessage="Invalid phone number format">
43                <f:validateRegex pattern=
44                   "((\(\d{3}\) ?)|(\d{3}-))?\d{3}-\d{4}" />
45             </h:inputText>
46             <h:message id="phoneMessage" for="phoneInputText"
47                styleClass="error"/>
48          </h:panelGrid>
```

Fig. 30.12 | Ajax enabling the Validation app. (Part 1 of 2.)

```
49            <h:commandButton value="Submit">
50               <f:ajax execute="nameInputText emailInputText phoneInputText"
51                  render=
52                  "nameMessage emailMessage phoneMessage resultOutputText"/>
53            </h:commandButton>
54            <h:outputText id="resultOutputText" escape="false"
55               value="#{validationBean.response}"/>
56         </h:form>
57      </h:body>
58   </html>
```

Fig. 30.12 | Ajax enabling the Validation app. (Part 2 of 2.)

Adding `id` Attributes to Elements

The Facelets elements that will be submitted as part of an Ajax request and the Facelets elements that will participate in the partial page updates must have `id` attributes. The `h:inputText` elements in the original `Validation` example already had `id` attributes. These elements will be submitted to the server as part of an Ajax request. We'd like the `h:Message` elements that show validation errors and the `h:outputText` element that displays the result to be updated with partial page updates. For this reason, we've added `id` attributes to these elements.

`f:ajax` Element

The other key change to this page is at lines 49–53 where the `h:commandButton` now contains an **`f:ajax` element**, which intercepts the form submission when the user clicks the button and makes an Ajax request instead. The `f:ajax` element's **`execute` attribute** specifies a space-separated list of element `id`s—the values of these elements are submitted as part of the Ajax request. The `f:ajax` element's **`render` attribute** specifies a space-separated list of element `id`s for the elements that should be updated via partial page updates.

30.5 Wrap-Up

In this chapter, we built an `AddressBook` application that allowed a user to add and view contacts. You learned how to insert user input into a Java DB database and how to display the contents of a database on a web page using an `h:dataTable` JSF element. We also demonstrated how to add Ajax capabilities to JSF web apps by enhancing the `Validation` app from Section 29.7. In Chapter 31, you'll use NetBeans to create web services and consume them from desktop and web applications.

Summary

Section 30.2.1 Setting Up the Database

- A data source enables a web app to interact with a database.
- In web apps that receive many requests, it's inefficient to create separate database connections for each request. Instead, you should set up a connection pool (p. 1279) to allow the server to manage a limited number of database connections and share them among requests.

- To connect to the database from a web app, you configure a data source name (p. 1281) that will be used to locate the database. The data source name is associated with a connection pool that manages the connections to the database.
- JNDI (Java Naming and Directory Interface) is a technology for locating application components (such as databases) in a distributed application (such as a multitier web application).

Section 30.2.2 `@ManagedBean` Class `AddressBean`

- A `DataSource` (p. 1285; package `javax.sql`) enables a web application to obtain a `Connection` to a database.
- The annotation `@Resource` (p. 1285) can be used to inject a `DataSource` object into a managed bean. The annotation's `name` attribute specifies the JNDI name of a data source.
- The `@Resource` annotation enables the server to hide all the complex details of locating a connection pool. The server creates a `DataSource` for you that's configured to use a connection pool and assigns the `DataSource` object to the annotated variable. You can then trivially obtain a `Connection` for interacting with the database.
- Database connections are limited resources, so you should use and close them quickly in your web apps. You can use a `CachedRowSet` to store the results of a query for use later.

Section 30.2.3 `index.xhtml` Facelets Page

- You can use an `h:dataTable` element (p. 1286) to display a collection of objects, such as the rows in a `CachedRowSet`, in tabular format.
- If you specify an `h:commandButton`'s `action` attribute (p. 1286) with a value that is the name of a web page (without the file name extension), the JSF framework assumes this is a page in the app, appends `.xhtml` extension to the `action` attribute's value and returns the page to the client browser.
- The `h:dataTable` element's `value` attribute (p. 1287) specifies the collection of data you wish to display. The `h:dataTable` iterates over its `value` collection and, one at a time, assigns each element to the variable specified by the `var` attribute (p. 1288). This variable is used in the `h:dataTable`'s nested elements to access each element of the collection.
- The `h:dataTable` `rowClasses` attribute (p. 1288) is a space-separated list of CSS style class names that are used to style the rows in the tabular output. You can specify as many styles as you like—they'll be applied in the order you list them one row at a time until all the styles have been applied, then the `h:DataTable` will automatically cycle through the styles again for the next set of rows. The `columnClasses` attribute works similarly for columns in the table.
- The `headerClass` attribute (p. 1288) specifies the column header CSS style. The `footerClass` attribute (p. 1288) works similarly for column footers in the table.
- The `styleClass` attribute (p. 1288) specifies the CSS styles for the entire table. The `cellpadding` and `cellspacing` attributes (p. 1288) specify the number of pixels around each table cell's contents and the number of pixels between table cells, respectively.
- An `h:column` element (p. 1288) defines a column in an `h:dataTable`.
- To display a column header above a column, define an `f:facet` element (p. 1288) and set its `name` attribute to `"header"`. Similarly, to display a column footer, use an `f:facet` with its `name` attribute set to `"footer"`.

Section 30.2.4 `addentry.xhtml` Facelets Page

- You can call a managed bean's methods in EL expressions.
- When you call a managed bean method with the `action` attribute, if the method returns a value, that value is used to request the corresponding page from the app. If the method does not return a value, the current page is re-requested.

Section 30.3 Ajax

- The term Ajax—short for Asynchronous JavaScript and XML—was coined by Jesse James Garrett of Adaptive Path, Inc., in February 2005 to describe a range of technologies for developing highly responsive, dynamic web applications.
- Ajax separates the user interaction portion of an application from its server interaction, enabling both to proceed asynchronously in parallel. This enables Ajax web-based applications to perform at speeds approaching those of desktop applications.
- Ajax makes asynchronous calls to the server to exchange small amounts of data with each call. Where normally the entire page would be submitted and reloaded with every user interaction on a web page, Ajax reloads only the necessary portions of the page, saving time and resources.
- Ajax applications typically make use of client-side scripting technologies such as JavaScript to interact with page elements. They use the browser's `XMLHttpRequest` object to perform the asynchronous exchanges with the web server that make Ajax applications so responsive.
- In a traditional web application, the user fills in a form's fields, then submits the form. The browser generates a request to the server, which receives the request and processes it. The server generates and sends a response containing the exact page that the browser will render. The browser loads the new page, temporarily making the browser window blank. The client waits for the server to respond and reloads the entire page with the data from the response. While such a synchronous request is being processed on the server, the user cannot interact with the web page. This model yields "choppy" application performance.
- In an Ajax application, when the user interacts with the page, the client creates an `XMLHttpRequest` object to manage a request. This object sends the request to the server and awaits the response. The requests are asynchronous, so the user can interact with the application on the client side while the server processes the earlier request concurrently. Other user interactions could result in additional requests to the server. Once the server responds to the original request, the `XMLHttpRequest` object that issued the request calls a client-side function to process the data returned by the server. This callback function uses partial page updates to display the data in the existing web page without reloading the entire page. At the same time, the server may be responding to the second request and the client side may be starting to do another partial page update.
- Partial page updates help make web applications more responsive, making them feel more like desktop applications.

Section 30.4 Adding Ajax Functionality to the Validation App

- The Facelets elements that will be submitted as part of an Ajax request and the Facelets elements that will participate in the partial page updates must have `id` attributes.
- When you nest an `f:ajax` element (p. 1295) in an `h:commandButton` element, the `f:ajax` element intercepts the form submission and makes an Ajax request instead.
- The `f:ajax` element's `execute` attribute (p. 1295) specifies a space-separated list of element `id`s—the values of these elements are submitted as part of the Ajax request.
- The `f:ajax` element's `render` attribute (p. 1295) specifies a space-separated list of element `id`s for the elements that should be updated via partial page updates.

Self-Review Exercise

30.1 Fill in the blanks in each of the following statements.

a) Ajax is an acronym for _____.

b) A(n) _____ allows the server to manage a limited number of database connections and share them among requests.

c) is a technology for locating application components (such as databases) in a distributed application.
d) A(n) _____ enables a web application to obtain a `Connection` to a database.
e) The annotation _____ can be used to inject a `DataSource` object into a managed bean.
f) A(n) _____ element displays a collection of objects in tabular format.
g) An `h:commandButton`'s _____ attribute can specify the name of another page in the web app that should be returned to the client.
h) To specify headers or footers for the columns in `h:dataTables`, use _____ elements nested with their `name` attributes set to _____ and _____, respectively.
i) _____ separates the user interaction portion of an application from its server interaction, enabling both to proceed asynchronously in parallel.
j) _____ help make web applications more responsive, making them feel more like desktop applications.
k) The `f:ajax` element's _____ attribute specifies a space-separated list of element `id`s—the values of these elements are submitted as part of the Ajax request.
l) The `f:ajax` element's _____ attribute specifies a space-separated list of element `id`s for the elements that should be updated via partial page updates.

Answers to Self-Review Exercise

30.1 a) Asynchronous JavaScript and XML. b) connection pool. c) JNDI (Java Naming and Directory Interface). d) `DataSource`. e) `@Resource`. f) `h:dataTable`. g) `action`. h) `f:facet`, `"header"`, `"footer"`. i) Ajax. j) partial page updates. k) `execute`. l) `render`.

Exercises

30.2 ***(`Guestbook` Application)*** Create a JSF web app that allows users to sign and view a guestbook. Use the `Guestbook` database to store guestbook entries. [*Note:* A SQL script to create the `Guestbook` database is provided in the examples directory for this chapter.] The `Guestbook` database has a single table, `Messages`, which has four columns: `Date`, `Name`, `Email` and `Message`. The database already contains a few sample entries. Using the `AddressBook` app in Section 30.2 as your guide, create two Facelets pages and a managed bean. The `index.xhtml` page should show the Guestbook entries in tabular format and should provide a button to add an entry to the Guestbook. When the user clicks this button, display an `addentry.xhtml` page. Provide `h:inputText` elements for the user's name and email address, an `h:inputTextarea` for the message and a **Sign Guestbook** button to submit the form. When the form is submitted, you should store in the `Guestbook` database a new entry containing the user's input and the date of the entry.

30.3 ***(`AddressBook` Application Modification: Ajax)*** Combine the two Facelets pages of the `AddressBook` application (Section 30.2) into a single page. Use Ajax capabilities to submit the new address book entry and to perform a partial page update that rerenders `h:dataTable` with the updated list of addresses.

30.4 ***(`AddressBook` Application Modification)*** Modify your solution to Exercise 30.3 to add a search capability that allows the user to search by last name. When the user presses the **Search** button, use Ajax to submit the search key and perform a partial page update that displays only the matching addresses in the `h:dataTable`.

31

Web Services



A client is to me a mere unit, a factor in a problem.
—Sir Arthur Conan Doyle

They also serve who only stand and wait.
—John Milton

...if the simplest things of nature have a message that you understand, rejoice, for your soul is alive.
—Eleonora Duse

Protocol is everything.
—Francoise Giuliani

Objectives

In this chapter you will learn:

- What a web service is.
- How to publish and consume web services in NetBeans.
- How XML, JSON, XML-Based Simple Object Access Protocol (SOAP) and Representational State Transfer (REST) Architecture enable Java web services.
- How to create client desktop and web applications that consume web services.
- How to use session tracking in web services to maintain client state information.
- How to connect to databases from web services.
- How to pass objects of user-defined types to and return them from a web service.

Outline

31.1 Introduction
31.2 Web Service Basics
31.3 Simple Object Access Protocol (SOAP)
31.4 Representational State Transfer (REST)
31.5 JavaScript Object Notation (JSON)
31.6 Publishing and Consuming SOAP-Based Web Services
- 31.6.1 Creating a Web Application Project and Adding a Web Service Class in NetBeans
- 31.6.2 Defining the `WelcomeSOAP` Web Service in NetBeans
- 31.6.3 Publishing the `WelcomeSOAP` Web Service from NetBeans
- 31.6.4 Testing the `WelcomeSOAP` Web Service with GlassFish Application Server's `Tester` Web Page
- 31.6.5 Describing a Web Service with the Web Service Description Language (WSDL)
- 31.6.6 Creating a Client to Consume the `WelcomeSOAP` Web Service
- 31.6.7 Consuming the `WelcomeSOAP` Web Service

31.7 Publishing and Consuming REST-Based XML Web Services
- 31.7.1 Creating a REST-Based XML Web Service
- 31.7.2 Consuming a REST-Based XML Web Service

31.8 Publishing and Consuming REST-Based JSON Web Services
- 31.8.1 Creating a REST-Based JSON Web Service
- 31.8.2 Consuming a REST-Based JSON Web Service

31.9 Session Tracking in a SOAP Web Service
- 31.9.1 Creating a `Blackjack` Web Service
- 31.9.2 Consuming the `Blackjack` Web Service

31.10 Consuming a Database-Driven SOAP Web Service
- 31.10.1 Creating the `Reservation` Database
- 31.10.2 Creating a Web Application to Interact with the `Reservation` Service

31.11 Equation Generator: Returning User-Defined Types
- 31.11.1 Creating the `EquationGeneratorXML` Web Service
- 31.11.2 Consuming the `EquationGeneratorXML` Web Service
- 31.11.3 Creating the `EquationGeneratorJSON` Web Service
- 31.11.4 Consuming the `EquationGeneratorJSON` Web Service

31.12 Wrap-Up

Summary | *Self-Review Exercises* | *Answers to Self-Review Exercises* | *Exercises* | *Making a Difference*

31.1 Introduction

This chapter introduces web services, which promote software portability and reusability in applications that operate over the Internet. A **web service** is a software component stored on one computer that can be accessed by an application (or other software component) on another computer over a network. Web services communicate using such technologies as XML, JSON and HTTP. In this chapter, we use two Java APIs that facilitate web services. The first, **JAX-WS**, is based on the **Simple Object Access Protocol (SOAP)**—an XML-based protocol that allows web services and clients to communicate, even if the client and the web service are written in different languages. The second, **JAX-RS**, uses **Representational State Transfer (REST)**—a network architecture that uses the web's traditional request/response mechanisms such as `GET` and `POST` requests. For more information on SOAP-based and REST-based web services, visit our Web Services Resource Centers:

```
www.deitel.com/WebServices/
www.deitel.com/RESTWebServices/
```

These Resource Centers include information about designing and implementing web services in many languages and about web services offered by companies such as Google, Amazon and eBay. You'll also find many additional tools for publishing and consuming web services. For more information about REST-based Java web services, check out the Jersey project:

```
jersey.java.net/
```

The XML used in this chapter is created and manipulated for you by the APIs, so you need not know the details of XML to use it here. To learn more about XML, read the following tutorials:

```
www.deitel.com/articles/xml_tutorials/20060401/XMLBasics/
www.deitel.com/articles/xml_tutorials/20060401/XMLStructuringData/
```

and visit our XML Resource Center:

```
www.deitel.com/XML/
```

Business-to-Business Transactions

Rather than relying on proprietary applications, businesses can conduct transactions via standardized, widely available web services. This has important implications for **business-to-business** (**B2B**) **transactions**. Web services are platform and language independent, enabling companies to collaborate without worrying about the compatibility of their hardware, software and communications technologies. Companies such as Amazon, Google, eBay, PayPal and many others are benefiting by making their server-side applications available to partners via web services.

By purchasing some web services and using other free ones that are relevant to their businesses, companies can spend less time developing applications and can create new ones that are more innovative. E-businesses for example, can provide their customers with enhanced shopping experiences. Consider an online music store. The store's website links to information about various CDs, enabling users to purchase them, to learn about the artists, to find more titles by those artists, to find other artists' music they may enjoy, and more. The store's website may also link to the site of a company that sells concert tickets and provides a web service that displays upcoming concert dates for various artists, allowing users to buy tickets. By consuming the concert-ticket web service on its site, the online music store can provide an additional service to its customers, increase its site traffic and perhaps earn a commission on concert-ticket sales. The company that sells concert tickets also benefits from the business relationship by selling more tickets and possibly by receiving revenue from the online music store for the use of the web service.

Any Java programmer with a knowledge of web services can write applications that "consume" web services. The resulting applications would invoke web services running on servers that could be thousands of miles away.

NetBeans

NetBeans is one of many tools that enable you to *publish* and/or *consume* web services. We demonstrate how to use NetBeans to implement web services using the JAX-WS and JAX-RS APIs and how to invoke them from client applications. For each example, we provide the web service's code, then present a client application that uses the web service. Our first examples build simple web services and client applications in NetBeans. Then we demon-

strate web services that use more sophisticated features, such as manipulating databases with JDBC and manipulating class objects. For information on downloading and installing the NetBeans and the GlassFish server, see Section 29.1.

31.2 Web Service Basics

The machine on which a web service resides is referred to as a **web service host**. The client application sends a request over a network to the web service host, which processes the request and returns a response over the network to the application. This kind of distributed computing benefits systems in various ways. For example, an application without direct access to data on another system might be able to retrieve the data via a web service. Similarly, an application lacking the processing power to perform specific computations could use a web service to take advantage of another system's superior resources.

In Java, a web service is implemented as a class that resides on a server—it's not part of the client application. Making a web service available to receive client requests is known as **publishing a web service**; using a web service from a client application is known as **consuming a web service**.

31.3 Simple Object Access Protocol (SOAP)

The Simple Object Access Protocol (SOAP) is a platform-independent protocol that uses XML to interact with web services, typically over HTTP. You can view the SOAP specification at `www.w3.org/TR/soap/`. Each request and response is packaged in a **SOAP message**—XML markup containing the information that a web service requires to process the message. SOAP messages are written in XML so that they're computer readable, human readable and platform independent. Most **firewalls**—security barriers that restrict communication among networks—allow HTTP traffic to pass through, so that clients can browse the web by sending requests to and receiving responses from web servers. Thus, SOAP-based services can send and receive SOAP messages over HTTP connections with few limitations.

SOAP supports an extensive set of types, including the primitive types (e.g., `int`), as well as `DateTime`, `XmlNode` and others. SOAP can also transmit arrays of these types. When a program invokes a method of a SOAP web service, the request and all relevant information are packaged in a SOAP message enclosed in a **SOAP envelope** and sent to the server on which the web service resides. When the web service receives this SOAP message, it parses the XML representing the message, then processes the message's contents. The message specifies the *method* that the client wishes to execute and the *arguments* the client passed to that method. Next, the web service calls the method with the specified arguments (if any) and sends the response back to the client in another SOAP message. The client parses the response to retrieve the method's result. In Section 31.6, you'll build and consume a basic SOAP web service.

31.4 Representational State Transfer (REST)

Representational State Transfer (REST) refers to an architectural style for implementing web services. Such web services are often called **RESTful web services**. Though REST itself is not a standard, RESTful web services are implemented using web standards. Each method in a

RESTful web service is identified by a unique URL. Thus, when the server receives a request, it immediately knows what operation to perform. Such web services can be used in a program or directly from a web browser. The results of a particular operation may be cached locally by the browser when the service is invoked with a `GET` request. This can make subsequent requests for the same operation faster by loading the result directly from the browser's cache. Amazon's web services (`aws.amazon.com`) are RESTful, as are many others.

RESTful web services are alternatives to those implemented with SOAP. Unlike SOAP-based web services, the request and response of REST services are not wrapped in envelopes. REST is also not limited to returning data in XML format. It can use a variety of formats, such as XML, JSON, HTML, plain text and media files. In Sections 31.7–31.8, you'll build and consume basic RESTful web services.

31.5 JavaScript Object Notation (JSON)

JavaScript Object Notation (JSON) is an alternative to XML for representing data. JSON is a text-based data-interchange format used to represent objects in JavaScript as collections of name/value pairs represented as `Strings`. It's commonly used in Ajax applications. JSON is a simple format that makes objects easy to read, create and parse and, because it's much less verbose than XML, allows programs to transmit data efficiently across the Internet. Each JSON object is represented as a list of property names and values contained in curly braces, in the following format:

```
{ propertyName1 : value1, propertyName2 : value2 }
```

Arrays are represented in JSON with square brackets in the following format:

```
[ value1, value2, value3 ]
```

Each value in an array can be a string, a number, a JSON object, `true`, `false` or `null`. To appreciate the simplicity of JSON data, examine this representation of an array of address-book entries:

```
[ { first: 'Cheryl', last: 'Black' },
  { first: 'James', last: 'Blue' },
  { first: 'Mike', last: 'Brown' },
  { first: 'Meg', last: 'Gold' } ]
```

Many programming languages now support the JSON data format. An extensive list of JSON libraries sorted by language can be found at `www.json.org`.

31.6 Publishing and Consuming SOAP-Based Web Services

This section presents our first example of publishing (enabling for client access) and consuming (using) a web service. We begin with a SOAP-based web service.

31.6.1 Creating a Web Application Project and Adding a Web Service Class in NetBeans

When you create a web service in NetBeans, you focus on its logic and let the IDE and server handle its infrastructure. First you create a **Web Application** project. NetBeans uses this project type for web services that are invoked by other applications.

Creating a Web Application Project in NetBeans

To create a web application, perform the following steps:

1. Select **File > New Project...** to open the **New Project** dialog.
2. Select **Java Web** from the dialog's **Categories** list, then select **Web Application** from the **Projects** list. Click **Next >**.
3. Specify the name of your project (`WelcomeSOAP`) in the **Project Name** field and specify where you'd like to store the project in the **Project Location** field. You can click the **Browse** button to select the location. Click **Next >**.
4. Select **GlassFish Server 3** from the **Server** drop-down list and **Java EE 6 Web** from the **Java EE Version** drop-down list.
5. Click **Finish** to create the project.

This creates a web application that will run in a web browser, similar to the projects used in Chapters 29 and 30.

Adding a Web Service Class to a Web Application Project

Perform the following steps to add a web service class to the project:

1. In the **Projects** tab in NetBeans, right click the **WelcomeSOAP** project's node and select **New > Web Service...** to open the **New Web Service** dialog.
2. Specify `WelcomeSOAP` in the **Web Service Name** field.
3. Specify `com.deitel.welcomesoap` in the **Package** field.
4. Click **Finish** to create the web service class.

The IDE generates a sample web service class with the name from *Step 2* in the package from *Step 3*. You can find this class in your project's **Web Services** node. In this class, you'll define the methods that your web service makes available to client applications. When you eventually build your application, the IDE will generate other supporting files for your web service.

31.6.2 Defining the `WelcomeSOAP` Web Service in NetBeans

Figure 31.1 contains the completed `WelcomeSOAPService` code (reformatted to match the coding conventions we use in this book). First we discuss this code, then show how to use the NetBeans web service design view to add the `welcome` method to the class.

```
1  // Fig. 31.1: WelcomeSOAP.java
2  // Web service that returns a welcome message via SOAP.
3  package com.deitel.welcomesoap;
4
5  import javax.jws.WebService; // program uses the annotation @WebService
6  import javax.jws.WebMethod; // program uses the annotation @WebMethod
7  import javax.jws.WebParam; // program uses the annotation @WebParam
8
9  @WebService() // annotates the class as a web service
10 public class WelcomeSOAP
11 {
```

Fig. 31.1 | Web service that returns a welcome message via SOAP. (Part 1 of 2.)

```
12      // WebMethod that returns welcome message
13      @WebMethod( operationName = "welcome" )
14      public String welcome( @WebParam( name = "name" ) String name )
15      {
16         return "Welcome to JAX-WS web services with SOAP, " + name + "!";
17      } // end method welcome
18   } // end class WelcomeSOAP
```

Fig. 31.1 | Web service that returns a welcome message via SOAP. (Part 2 of 2.)

Annotation `import` *Declarations*

Lines 5–7 import the annotations used in this example. By default, each new web service class created with the JAX-WS APIs is a POJO (plain old Java object), so you do *not* need to extend a class or implement an interface to create a web service.

`@WebService` *Annotation*

Line 9 contains a **`@WebService` annotation** (imported at line 5) which indicates that class `WelcomeSOAP` implements a web service. The annotation is followed by parentheses that may contain optional annotation attributes. The optional **`name` attribute** specifies the name of the service endpoint interface class that will be generated for the client. A **service endpoint interface (SEI)** class (sometimes called a **proxy class**) is used to interact with the web service—a client application consumes the web service by invoking methods on the service endpoint interface object. The optional **`serviceName` attribute** specifies the service name, which is also the name of the class that the client uses to obtain a service endpoint interface object. If the `serviceName` attribute is not specified, the web service's name is assumed to be the Java class name followed by the word `Service`. NetBeans places the `@WebService` annotation at the beginning of each new web service class you create. You can then add the `name` and `serviceName` properties in the parentheses following the annotation.

When you deploy a web application containing a class that uses the `@WebService` annotation, the server (GlassFish in our case) recognizes that the class implements a web service and creates all the **server-side artifacts** that support the web service—that is, the framework that allows the web service to wait for client requests and respond to those requests once it's deployed on an application server. Some popular open-source application servers that support Java web services include GlassFish (`glassfish.dev.java.net`), Apache Tomcat (`tomcat.apache.org`) and JBoss Application Server (`www.jboss.com/products/platforms/application`).

`WelcomeSOAP` *Service's* `welcome` *Method*

The `WelcomeSOAP` service has only one method, `welcome` (lines 13–17), which takes the user's name as a `String` and returns a `String` containing a welcome message. This method is tagged with the **`@WebMethod` annotation** to indicate that it can be called remotely. Any methods that are not tagged with `@WebMethod` are *not* accessible to clients that consume the web service. Such methods are typically utility methods within the web service class. The `@WebMethod` annotation uses the **`operationName`** attribute to specify the method name that is exposed to the web service's client. If the `operationName` is not specified, it's set to the actual Java method's name.



Common Programming Error 31.1

Failing to expose a method as a web method by declaring it with the `@WebMethod` *annotation prevents clients of the web service from accessing the method. There's one exception—if none of the class's methods are declared with the* `@WebMethod` *annotation, then all the* `public` *methods of the class will be exposed as web methods.*



Common Programming Error 31.2

Methods with the `@WebMethod` *annotation cannot be* `static`*. An object of the web service class must exist for a client to access the service's web methods.*

The `name` parameter to `welcome` is annotated with the **`@WebParam` annotation** (line 14). The optional `@WebParam` attribute **`name`** indicates the parameter name that is exposed to the web service's clients. If you don't specify the name, the actual parameter name is used.

Completing the Web Service's Code

NetBeans provides a web service design view in which you can define the method(s) and parameter(s) for your web services. To define the `WelcomeSOAP` class's `welcome` method, perform the following steps:

1. In the project's **Web Services** node, double click `WelcomeSOAP` to open the file `WelcomeSOAPService.java` in the code editor.
2. Click the **Design** button at the top of the code editor to show the web service design view (Fig. 31.2).

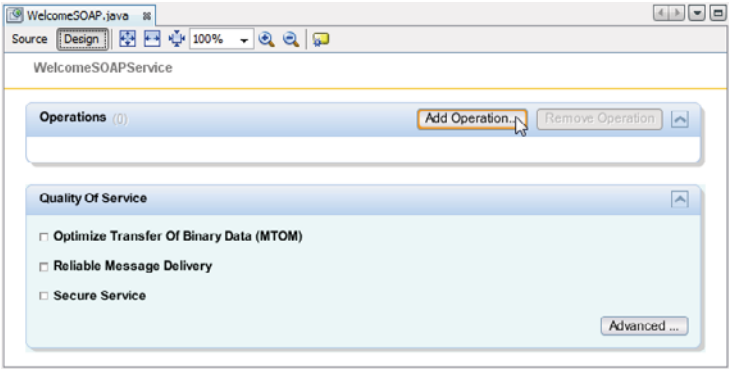


Fig. 31.2 | Web service design view.

3. Click the **Add Operation...** button to display the **Add Operation...** dialog (Fig. 31.3).
4. Specify the method name `welcome` in the **Name** field. The default **Return Type** (`String`) is correct for this example.
5. Add the method's `name` parameter by clicking the **Add** button to the right of the **Parameters** tab then entering `name` in the **Name** field. The parameter's default **Type** (`String`) is correct for this example.

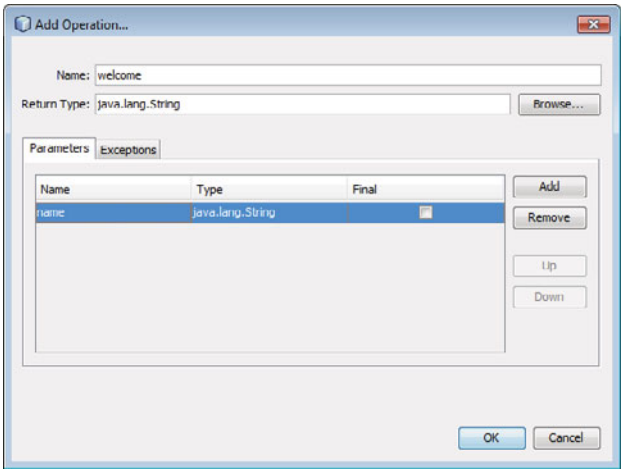


Fig. 31.3 | Adding an operation to a web service.

6. Click **OK** to create the `welcome` method. The design view should now appear as shown in Fig. 31.3.
7. At the top of the design view, click the **Source** button to display the class's source code and add the code line 18 of Fig. 31.1 to the body of method `welcome`.

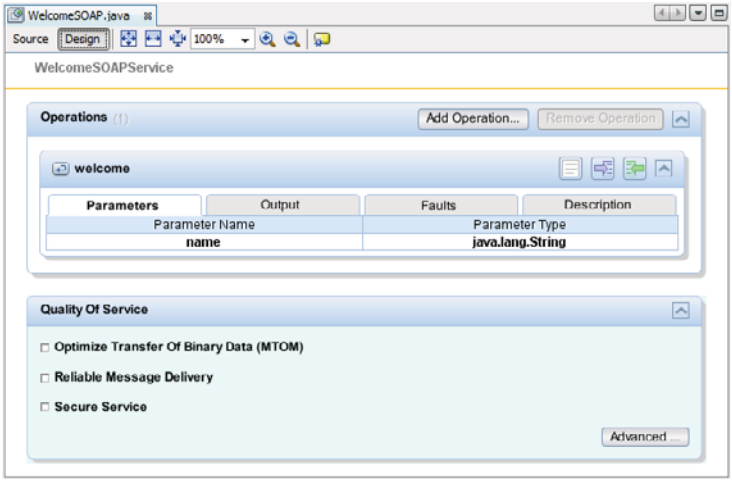


Fig. 31.4 | Web service design view after new operation is added.

31.6.3 Publishing the `WelcomeSOAP` Web Service from NetBeans

Now that you've created the `WelcomeSOAP` web service class, you'll use NetBeans to build and *publish* (that is, deploy) the web service so that clients can consume its services. NetBeans handles all the details of building and deploying a web service for you. This includes

creating the framework required to support the web service. Right click the project name `WelcomeSOAP` in the **Projects** tab and select **Deploy** to build and deploy the web application to the GlassFish server.

31.6.4 Testing the `WelcomeSOAP` Web Service with GlassFish Application Server's `Tester` Web Page

Next, you'll test the `WelcomeSOAP` web service. We previously selected the GlassFish application server to execute this web application. This server can dynamically create a web page that allows you to test a web service's methods from a web browser. To use this capability:

1. Expand the project's **Web Services** in the NetBeans **Projects** tab.
2. Right click the web service class name (`WelcomeSOAP`) and select **Test Web Service**.

The GlassFish application server builds the `Tester` web page and loads it into your web browser. Figure 31.5 shows the `Tester` web page for the `WelcomeSOAP` web service. The web service's name is automatically the class name followed by `Service`.

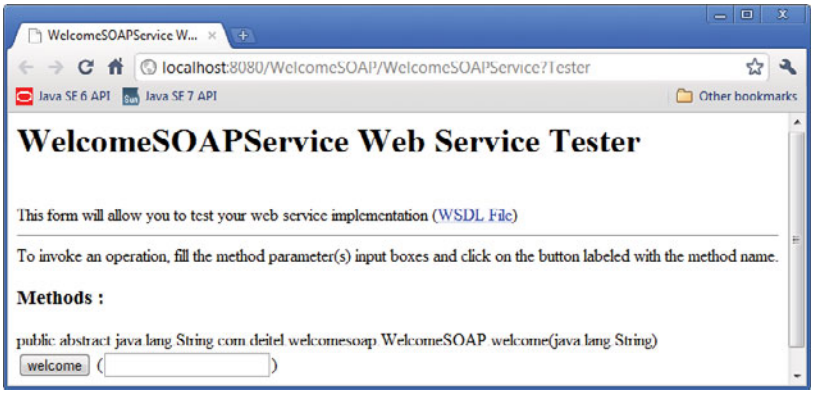


Fig. 31.5 | `Tester` web page created by GlassFish for the `WelcomeSOAP` web service.

Once you've deployed the web service, you can also type the URL

```
http://localhost:8080/WelcomeSOAP/WelcomeSOAPService?Tester
```

in your web browser to view the `Tester` web page. `WelcomeSOAPService` is the name (specified in line 11 of Fig. 31.1) that clients use to access the web service.

To test `WelcomeSOAP`'s `welcome` web method, type your name in the text field to the right of the **welcome** button and click the button to invoke the method. Figure 31.6 shows the results of invoking `WelcomeSOAP`'s `welcome` method with the value `Paul`.

Application Server Note

You can access the web service only when the application server is running. If NetBeans launches GlassFish for you, it will automatically shut it down when you close NetBeans. To keep it up and running, you can launch it independently of NetBeans before you deploy or run web applications. The GlassFish Quick Start Guide at `glassfish.java.net/downloads/quickstart/index.html` shows how to manually start and stop the server.

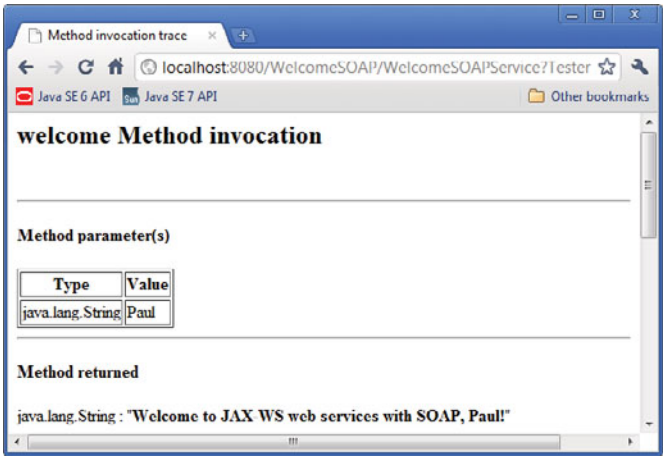


Fig. 31.6 | Testing `WelcomeSOAP`'s `welcome` method.

Testing the `WelcomeSOAP` Web Service from Another Computer

If your computer is connected to a network and allows HTTP requests, then you can test the web service from another computer on the network by typing the following URL (where *host* is the hostname or IP address of the computer on which the web service is deployed) into a browser on another computer:

```
http://host:8080/WelcomeSOAP/WelcomeSOAPService?Tester
```

31.6.5 Describing a Web Service with the Web Service Description Language (WSDL)

To consume a web service, a client must determine its functionality and how to use it. For this purpose, web services normally contain a **service description**. This is an XML document that conforms to the **Web Service Description Language** (**WSDL**)—an XML vocabulary that defines the methods a web service makes available and how clients interact with them. The WSDL document also specifies lower-level information that clients might need, such as the required formats for requests and responses.

WSDL documents help applications determine how to interact with the web services described in the documents. You do not need to understand WSDL to take advantage of it—the GlassFish application server generates a web service's WSDL dynamically for you, and client tools can parse the WSDL to help create the client-side service endpoint interface class that a client uses to access the web service. Since GlassFish (and most other servers) generate the WSDL dynamically, clients always receive a deployed web service's most up-to-date description. To access the `WelcomeSOAP` web service, the client code will need the following WSDL URL:

```
http://localhost:8080/WelcomeSOAP/WelcomeSOAPService?WSDL
```

Accessing the `WelcomeSOAP` Web Service's WSDL from Another Computer

Eventually, you'll want clients on other computers to use your web service. Such clients need the web service's WSDL, which they would access with the following URL:

```
http://host:8080/WelcomeSOAP/WelcomeSOAPService?WSDL
```

where *host* is the hostname or IP address of the server that hosts the web service. As we discussed in Section 31.6.4, this works only if your computer allows HTTP connections from other computers—as is the case for publicly accessible web and application servers.

31.6.6 Creating a Client to Consume the `WelcomeSOAP` Web Service

Now you'll consume the web service from a client application. A web service client can be any type of application or even another web service. You enable a client application to consume a web service by **adding a web service reference** to the application.

Service Endpoint Interface (SEI)

An application that consumes a web service consists of an object of a service endpoint interface (SEI) class (sometimes called a *proxy class*) that's used to interact with the web service and a client application that consumes the web service by invoking methods on the service endpoint interface object. The client code invokes methods on the service endpoint interface object, which handles the details of passing method arguments to and receiving return values from the web service on the client's behalf. This communication can occur over a local network, over the Internet or even with a web service on the same computer. The web service performs the corresponding task and returns the results to the service endpoint interface object, which then returns the results to the client code. Figure 31.7 depicts the interactions among the client code, the SEI object and the web service. As you'll soon see, NetBeans creates these service endpoint interface classes for you.

Requests to and responses from web services created with JAX-WS (one of many different web service frameworks) are typically transmitted via SOAP. Any client capable of generating and processing SOAP messages can interact with a web service, regardless of the language in which the web service is written.

We now use NetBeans to create a client Java desktop GUI application. Then you'll add a web service reference to the project so the client can access the web service. When you add the reference, the IDE creates and compiles the **client-side artifacts**—the framework of Java code that supports the client-side service endpoint interface class. The client then calls methods on an object of the service endpoint interface class, which uses the rest of the artifacts to interact with the web service.

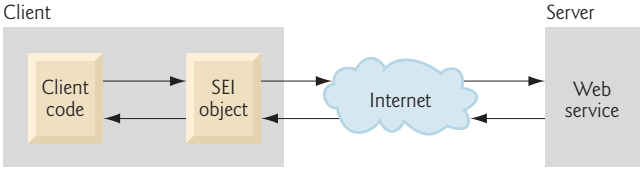


Fig. 31.7 | Interaction between a web service client and a web service.

Creating a Desktop Application Project in NetBeans

Before performing the steps in this section, ensure that the `WelcomeSOAP` web service has been deployed and that the GlassFish application server is running (see Section 31.6.3). Perform the following steps to create a client Java desktop application in NetBeans:

1. Select **File > New Project...** to open the **New Project** dialog.
2. Select **Java** from the **Categories** list and **Java Application** from the **Projects** list, then click **Next >**.
3. Specify the name `WelcomeSOAPClient` in the **Project Name** field and uncheck the **Create Main Class** checkbox. Later, you'll add a subclass of `JFrame` that contains a `main` method.
4. Click **Finish** to create the project.

Step 2: Adding a Web Service Reference to an Application

Next, you'll add a web service reference to your application so that it can interact with the `WelcomeSOAP` web service. To add a web service reference, perform the following steps.

1. Right click the project name (`WelcomeSOAPClient`) in the NetBeans **Projects** tab and select **New > Web Service Client...** from the pop-up menu to display the **New Web Service Client** dialog.
2. In the **WSDL URL** field, specify the URL `http://localhost:8080/WelcomeSOAP/WelcomeSOAPService?WSDL` (Fig. 31.8). This URL tells the IDE where to find the web service's WSDL description. [*Note:* If the GlassFish application server is located on a different computer, replace `localhost` with the hostname or IP address of that computer.] The IDE uses this WSDL description to generate the client-side artifacts that compose and support the service endpoint interface.

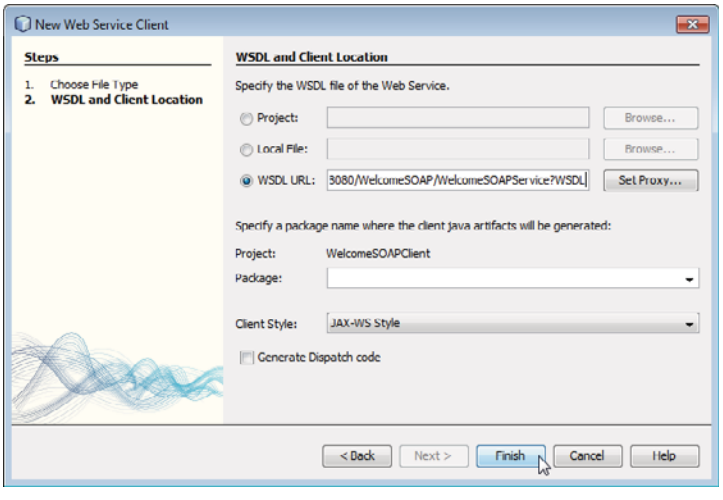


Fig. 31.8 | **New Web Service Client** dialog.

3. For the other options, leave the default settings, then click **Finish** to create the web service reference and dismiss the **New Web Service Client** dialog.

In the NetBeans **Projects** tab, the `WelcomeSOAPClient` project now contains a **Web Service References** folder with the `WelcomeSOAP` web service's service endpoint interface (Fig. 31.9). The service endpoint interface's name is listed as `WelcomeSOAPService`.

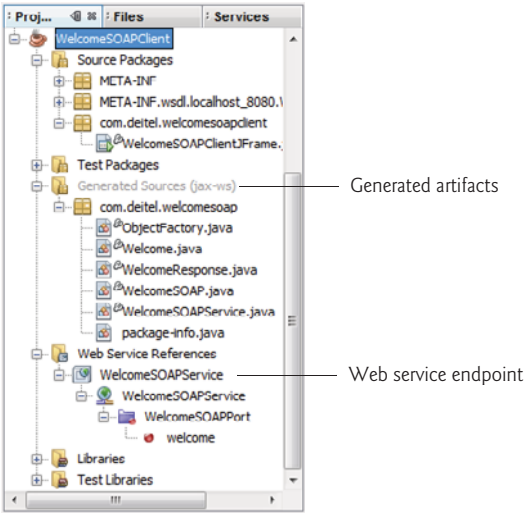


Fig. 31.9 | NetBeans **Project** tab after adding a web service reference to the project.

When you specify the web service you want to consume, NetBeans accesses and copies its WSDL information to a file in your project (named `WelcomeSOAPService.wsdl` in this example). You can view this file by double clicking the `WelcomeSOAPService` node in the project's **Web Service References** folder. If the web service changes, the client-side artifacts and the client's copy of the WSDL file can be regenerated by right clicking the `WelcomeSOAPService` node shown in Fig. 31.9 and selecting **Refresh...**. Figure 31.9 also shows the IDE-generated client-side artifacts, which appear in the **Generated Sources (jax-ws)** folder.

31.6.7 Consuming the `WelcomeSOAP` Web Service

For this example, we use a GUI application[1] to interact with the `WelcomeSOAP` web service. To build the client application's GUI, add a subclass of `JFrame` to the project by performing the following steps:

1. Right click the project name (`WelcomeSOAPClient`) in the NetBeans **Project** tab and select **New > JFrame Form...** to display the **New JFrame Form** dialog.
2. Specify `WelcomeSOAPClientJFrame` in the **Class Name** field.
3. Specify `com.deitel.welcomesoapclient` in the **Package** field.
4. Click **Finish** to close the **New JFrame Form** dialog.

Next, use the NetBeans GUI design tools to build the GUI shown in the sample screen captures at the end of Fig. 31.10. The GUI consists of a **Label**, a **Text Field** and a **Button**.

The application in Fig. 31.10 uses the `WelcomeSOAP` web service to display a welcome message to the user. To save space, we do not show the NetBeans autogenerated `initComponents` method, which contains the code that creates the GUI components, positions

1. We assume you're already familiar with using the NetBeans GUI designer. If not, see Appendix I.

them and registers their event handlers. To view the complete source code, open the `WelcomeSOAPClientJFrame.java` file in this example's folder under `src\java\com\deitel\welcomesoapclient`. NetBeans places the GUI component instance-variable declarations at the end of the class (lines 114–116). Java allows instance variables to be declared anywhere in a class's body as long as they're placed outside the class's methods. We continue to declare our own instance variables at the top of the class.

```
1  // Fig. 31.10: WelcomeSOAPClientJFrame.java
2  // Client desktop application for the WelcomeSOAP web service.
3  package com.deitel.welcomesoapclient;
4
5  import com.deitel.welcomesoap.WelcomeSOAP;
6  import com.deitel.welcomesoap.WelcomeSOAPService;
7  import javax.swing.JOptionPane;
8
9  public class WelcomeSOAPClientJFrame extends javax.swing.JFrame
10 {
11    // references the service endpoint interface object (i.e., the proxy)
12    private WelcomeSOAP welcomeSOAPProxy;
13
14    // no-argument constructor
15    public WelcomeSOAPClientJFrame()
16    {
17       initComponents();
18
19       try
20       {
21          // create the objects for accessing the WelcomeSOAP web service
22          WelcomeSOAPService service = new WelcomeSOAPService();
23          welcomeSOAPProxy = service.getWelcomeSOAPPort();
24       } // end try
25       catch ( Exception exception )
26       {
27          exception.printStackTrace();
28          System.exit( 1 );
29       } // end catch
30    } // end WelcomeSOAPClientJFrame constructor
31
32    // The initComponents method is autogenerated by NetBeans and is called
33    // from the constructor to initialize the GUI. This method is not shown
34    // here to save space. Open WelcomeSOAPClientJFrame.java in this
35    // example's folder to view the complete generated code.
36
87    // call the web service with the supplied name and display the message
88    private void submitJButtonActionPerformed(
89       java.awt.event.ActionEvent evt )
90    {
91       String name = nameJTextField.getText(); // get name from JTextField
92
93       // retrieve the welcome string from the web service
94       String message = welcomeSOAPProxy.welcome( name );
```

Fig. 31.10 | Client desktop application for the `WelcomeSOAP` web service. (Part 1 of 2.)

```
95          JOptionPane.showMessageDialog( this, message,
96             "Welcome", JOptionPane.INFORMATION_MESSAGE );
97       } // end method submitJButtonActionPerformed
98
99       // main method begins execution
100      public static void main( String args[] )
101      {
102         java.awt.EventQueue.invokeLater(
103            new Runnable()
104            {
105               public void run()
106               {
107                  new WelcomeSOAPClientJFrame().setVisible( true );
108               } // end method run
109            } // end anonymous inner class
110         ); // end call to java.awt.EventQueue.invokeLater
111      } // end main
112
113      // Variables declaration - do not modify
114      private javax.swing.JLabel nameJLabel;
115      private javax.swing.JTextField nameJTextField;
116      private javax.swing.JButton submitJButton;
117      // End of variables declaration
118   } // end class WelcomeSOAPClientJFrame
```

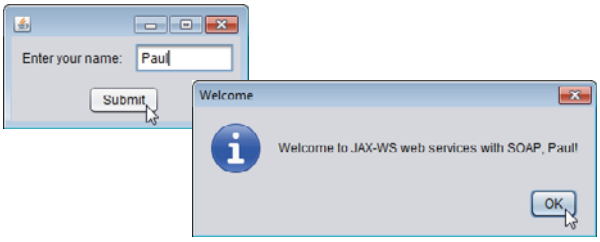


Fig. 31.10 | Client desktop application for the `WelcomeSOAP` web service. (Part 2 of 2.)

Lines 5–6 import the classes `WelcomeSOAP` and `WelcomeSOAPService` that enable the client application to interact with the web service. Notice that we do not have `import` declarations for most of the GUI components used in this example. When you create a GUI in NetBeans, it uses fully qualified class names (such as `javax.swing.JFrame` in line 9), so `import` declarations are unnecessary.

Line 12 declares a variable of type `WelcomeSOAP` that will refer to the service endpoint interface object. Line 22 in the constructor creates an object of type `WelcomeSOAPService`. Line 23 uses this object's `getWelcomeSOAPPort` method to obtain the `WelcomeSOAP` service endpoint interface object that the application uses to invoke the web service's methods.

The event handler for the **Submit** button (lines 88–97) first retrieves the name the user entered from `nameJTextField`. It then calls the `welcome` method on the service endpoint interface object (line 94) to retrieve the welcome message from the web service. This object communicates with the web service on the client's behalf. Once the message has been retrieved, lines 95–96 display it in a message box by calling `JOptionPane`'s `showMessageDialog` method.

31.7 Publishing and Consuming REST-Based XML Web Services

The previous section used a service endpoint interface (proxy) object to pass data to and from a Java web service using the SOAP protocol. Now, we access a Java web service using the REST architecture. We recreate the `WelcomeSOAP` example to return data in plain XML format. You can create a **Web Application** project as you did in Section 31.6 to begin. Name the project `WelcomeRESTXML`.

31.7.1 Creating a REST-Based XML Web Service

NetBeans provides various templates for creating RESTful web services, including ones that can interact with databases on the client's behalf. In this chapter, we focus on simple RESTful web services. To create a RESTful web service:

1. Right-click the **WelcomeRESTXML** node in the **Projects** tab, and select **New > Other...** to display the **New File** dialog.
2. Select **Web Services** under **Categories**, then select **RESTful Web Services from Patterns** and click **Next >**.
3. Under **Select Pattern**, ensure **Simple Root Resource** is selected, and click **Next >**.
4. Set the **Resource Package** to `com.deitel.welcomerestxml`, the **Path** to `welcome` and the **Class Name** to `WelcomeRESTXMLResource`. Leave the **MIME Type** and **Representation Class** set to `application/xml` and `java.lang.String`, respectively. The correct configuration is shown in Fig. 31.11.
5. Click **Finish** to create the web service.

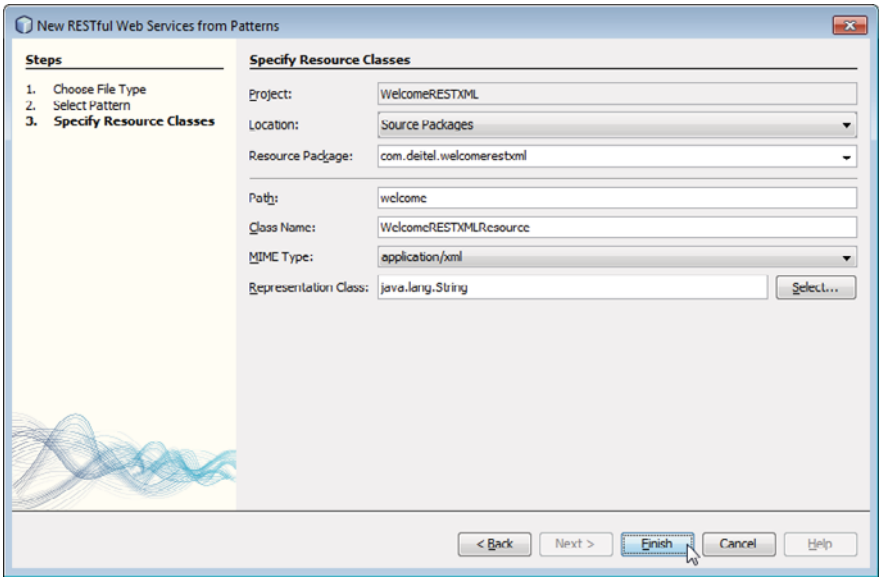


Fig. 31.11 | Creating the `WelcomeRESTXML` RESTful web service.

NetBeans generates the class and sets up the proper annotations. The class is placed in the project's **RESTful Web Services** folder. The code for the completed service is shown in Fig. 31.12. You'll notice that the completed code does not include some of the code generated by NetBeans. We removed the pieces that were unnecessary for this simple web service. The autogenerated `putXml` method is not necessary, because this example does not modify state on the server. The `UriInfo` instance variable is not needed, because we do not use HTTP query parameters. We also removed the autogenerated constructor, because we have no code to place in it.

```
1  // Fig. 31.12: WelcomeRESTXMLResource.java
2  // REST web service that returns a welcome message as XML.
3  package com.deitel.welcomerestxml;
4
5  import java.io.StringWriter;
6  import javax.ws.rs.GET; // annotation to indicate method uses HTTP GET
7  import javax.ws.rs.Path; // annotation to specify path of resource
8  import javax.ws.rs.PathParam; // annotation to get parameters from URI
9  import javax.ws.rs.Produces; // annotation to specify type of data
10 import javax.xml.bind.JAXB; // utility class for common JAXB operations
11
12 @Path( "welcome" ) // URI used to access the resource
13 public class WelcomeRESTXMLResource
14 {
15    // retrieve welcome message
16    @GET // handles HTTP GET requests
17    @Path( "{name}" ) // URI component containing parameter
18    @Produces( "application/xml" ) // response formatted as XML
19    public String getXml( @PathParam( "name" ) String name )
20    {
21       String message = "Welcome to JAX-RS web services with REST and " +
22          "XML, " + name + "!"; // our welcome message
23       StringWriter writer = new StringWriter();
24       JAXB.marshal( message, writer ); // marshal String as XML
25       return writer.toString(); // return XML as String
26    } // end method getXml
27 } // end class WelcomeRESTXMLResource
```

Fig. 31.12 | REST web service that returns a welcome message as XML.

Lines 6–9 contain the `imports` for the JAX-RS annotations that help define the RESTful web service. The **`@Path` annotation** on the `WelcomeRESTXMLResource` class (line 12) indicates the URI for accessing the web service. This URI is appended to the web application project's URL to invoke the service. Methods of the class can also use the `@Path` annotation (line 17). Parts of the path specified in curly braces indicate parameters—they're placeholders for values that are passed to the web service as part of the path. The base path for the service is the project's `resources` directory. For example, to get a welcome message for someone named John, the complete URL is

```
http://localhost:8080/WelcomeRESTXML/resources/welcome/John
```

Arguments in a URL can be used as arguments to a web service method. To do so, you bind the parameters specified in the `@Path` specification to parameters of the web service

method with the **`@PathParam` annotation**, as shown in line 19. When the request is received, the server passes the argument(s) in the URL to the appropriate parameter(s) in the web service method.

The **`@GET` annotation** denotes that this method is accessed via an HTTP `GET` request. The `putXml` method the IDE created for us had an `@PUT` annotation, which indicates that the method is accessed using the HTTP `PUT` method. Similar annotations exist for HTTP `POST`, `DELETE` and `HEAD` requests.

The **`@Produces` annotation** denotes the content type returned to the client. It's possible to have multiple methods with the same HTTP method and path but different `@Produces` annotations, and JAX-RS will call the method matching the content type requested by the client. Standard Java method overloading rules apply, so such methods must have different names. The **`@Consumes` annotation** for the autogenerated `putXml` method (which we deleted) restricts the content type that the web service will accept from a `PUT` operation.

Line 10 imports the **`JAXB` class** from package `javax.xml.bind`. **JAXB (Java Architecture for XML Binding)** is a set of classes for converting POJOs to and from XML. There are many related classes in the same package that implement the serializations we perform, but the `JAXB` class contains easy-to-use wrappers for common operations. After creating the welcome message (lines 21–22), we create a `StringWriter` (line 23) to which JAXB will output the XML. Line 24 calls the `JAXB` class's `static` method **`marshal`** to convert the `String` containing our message to XML format. Line 25 calls `StringWriter`'s `toString` method to retrieve the XML text to return to the client.

Testing RESTful Web Services

Section 31.6.4 demonstrated testing a SOAP service using GlassFish's `Tester` page. GlassFish does not provide a testing facility for RESTful services, but NetBeans automatically generates a test page that can be accessed by right clicking the **WelcomeRESTXML** node in the **Projects** tab and selecting **Test RESTful Web Services**. This will compile and deploy the web service, if you have not yet done so, then open the test page. Your browser will probably require you to acknowledge a potential security issue before allowing the test page to perform its tasks. The test page is loaded from your computer's local file system, *not* the GlassFish server. Browsers consider the local file system and GlassFish as two different servers, even though they're both on the local computer. For security reasons, browsers do not allow so-called cross-site scripting in which a web page tries to interact with a server other than the one that served the page.

On the test page (Fig. 31.13), expand the **welcome** element in the left column and select **{name}**. The form on the right side of the page allows you to choose the MIME type of the data (`application/xml` by default) and lets you enter the `name` parameter's value. Click the **Test** button to invoke the web service and display the returned XML.



Error-Prevention Tip 31.1

At the time of this writing, the test page did not work in Google's Chrome web browser. If this is your default web browser, copy the test page's URL from Chrome's address field and paste it into another web browser's address field. Fig. 31.13 shows the test page in Mozilla Firefox.

The test page shows several tabs containing the results and various other information. The **Raw View** tab shows the actual XML response. The **Headers** tab shows the HTTP

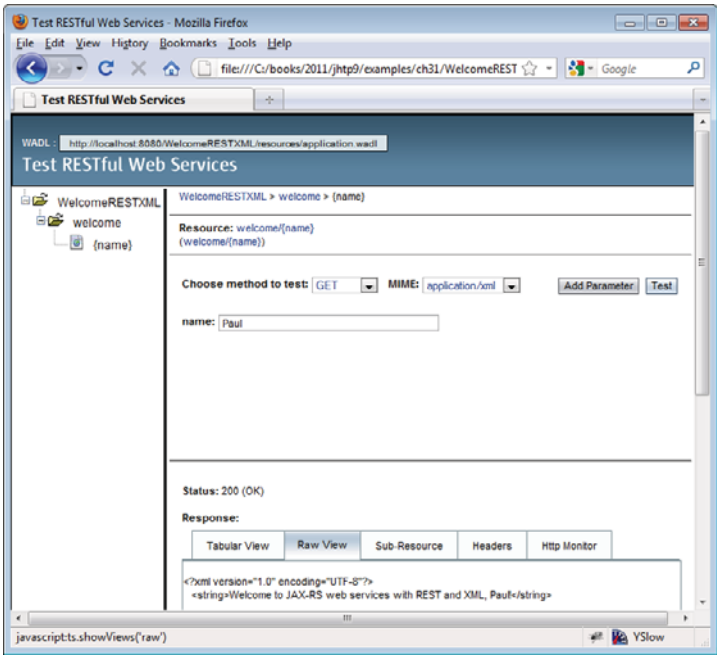


Fig. 31.13 | Test page for the `WelcomeRESTXML` web service.

headers returned by the server. The **Http Monitor** tab shows a log of the HTTP transactions that took place to complete the request and response. The Sub-Resource tab shows the actual URL that was used to invoke the web service

```
http://localhost:8080/WelcomeRESTXML/resources/welcome/Paul
```

You can enter this URL in any browser on your computer to invoke the web service with the value Paul.

The test page provides its functionality by reading a WADL file from the server—you can see the URL of the WADL file in the upper-left corner of the test page. **WADL (Web Application Description Language)** has similar design goals to WSDL, but describes RESTful services instead of SOAP services.

31.7.2 Consuming a REST-Based XML Web Service

As we did with SOAP, we create a Java application that retrieves the welcome message from the web service and displays it to the user. First, create a Java application with the name `WelcomeRESTXMLClient`. RESTful web services do *not* require web service references, so you can begin building the GUI immediately by creating a `JFrame` form called `WelcomeRESTXMLClientJFrame` and placing it in the `com.deitel.welcomerestxmlclient` package. The GUI is identical to the one in Fig. 31.10, including the names of the GUI elements. To create the GUI quickly, you can simply copy and paste the GUI from the **Design** view of the `WelcomeSOAPClientJFrame` class and paste it into the **Design** view of the `WelcomeRESTXMLClientJFrame` class. Figure 31.14 contains the completed code.

```
1  // Fig. 31.14: WelcomeRESTXMLClientJFrame.java
2  // Client that consumes the WelcomeRESTXML service.
3  package com.deitel.welcomerestxmlclient;
4
5  import javax.swing.JOptionPane;
6  import javax.xml.bind.JAXB; // utility class for common JAXB operations
7
8  public class WelcomeRESTXMLClientJFrame extends javax.swing.JFrame
9  {
10    // no-argument constructor
11    public WelcomeRESTXMLClientJFrame()
12    {
13       initComponents();
14    } // end constructor
15
16    // The initComponents method is autogenerated by NetBeans and is called
17    // from the constructor to initialize the GUI. This method is not shown
18    // here to save space. Open WelcomeRESTXMLClientJFrame.java in this
19    // example's folder to view the complete generated code.
20
21    // call the web service with the supplied name and display the message
22    private void submitJButtonActionPerformed(
23       java.awt.event.ActionEvent evt)
24    {
25       String name = nameJTextField.getText(); // get name from JTextField
26
27       // the URL for the REST service
28       String url =
29          "http://localhost:8080/WelcomeRESTXML/resources/welcome/" + name;
30
31       // read from URL and convert from XML to Java String
32       String message = JAXB.unmarshal( url, String.class );
33
34       // display the message to the user
35       JOptionPane.showMessageDialog( this, message,
36             "Welcome", JOptionPane.INFORMATION_MESSAGE );
37    } // end method submitJButtonActionPerformed
38
39    // main method begins execution
40    public static void main( String args[] )
41    {
42       java.awt.EventQueue.invokeLater(
43          new Runnable()
44          {
45             public void run()
46             {
47                new WelcomeRESTXMLClientJFrame().setVisible( true );
48             } // end method run
49          } // end anonymous inner class
50       ); // end call to java.awt.EventQueue.invokeLater
51    } // end main
52
```

Fig. 31.14 | Client that consumes the `WelcomeRESTXML` service. (Part 1 of 2.)

```
103     // Variables declaration - do not modify
104     private javax.swing.JLabel nameJLabel;
105     private javax.swing.JTextField nameJTextField;
106     private javax.swing.JButton submitJButton;
107     // End of variables declaration
108  } // end class WelcomeRESTXMLClientJFrame
```



Fig. 31.14 | Client that consumes the `WelcomeRESTXML` service. (Part 2 of 2.)

You can access a RESTful web service with classes from Java API. As in the RESTful XML web service, we use the JAXB library. The JAXB class (imported on line 6) has a `static` **`unmarshal`** method that takes as arguments a file name or URL as a `String`, and a `Class<T>` object indicating the Java class to which the XML will be converted (line 82). In this example, the XML contains a `String` object, so we use the Java compiler shortcut `String.class` to create the `Class<String>` object we need as the second argument. The `String` returned from the call to the `unmarshal` method is then displayed to the user via `JOptionPane`'s `showMessageDialog` method (lines 85–86), as it was with the SOAP service. The URL used in this example to extract data from the web service matches the URL used by the test page.

31.8 Publishing and Consuming REST-Based JSON Web Services

While XML was designed primarily as a document interchange format, JSON is designed as a *data* exchange format. Data structures in most programming languages do not map directly to XML constructs—for example, the distinction between elements and attributes is not present in programming-language data structures. JSON is a subset of the JavaScript programming language, and its components—objects, arrays, strings, numbers—can be easily mapped to constructs in Java and other programming languages.

The standard Java libraries do not currently provide capabilities for working with JSON, but there are many open-source JSON libraries for Java and other languages; you can find a list of them at `json.org`. We chose the Gson library from `code.google.com/p/google-gson/`, which provides a simple way to convert POJOs to and from JSON.

31.8.1 Creating a REST-Based JSON Web Service

To begin, create a WelcomeRESTJSON web application, then create the web service by following the steps in Section 31.7.1. In *Step 4*, change the **Resource Package** to `com.deitel.welcomerestjson`, the **Class Name** to `WelcomeRESTJSONResource` and the **MIME Type** to `application/json`. Additionally, you must download the Gson library's

JAR file, then add it to the project as a library. To do so, right click your project's **Libraries** folder, select **Add JAR/Folder...** locate the downloaded Gson JAR file and click **Open**. The complete code for the service is shown in Fig. 31.15.

```
1  // Fig. 31.15: WelcomeRESTJSONResource.java
2  // REST web service that returns a welcome message as JSON.
3  package com.deitel.welcomerestjson;
4
5  import com.google.gson.Gson; // converts POJO to JSON and back again
6  import javax.ws.rs.GET; // annotation to indicate method uses HTTP GET
7  import javax.ws.rs.Path; // annotation to specify path of resource
8  import javax.ws.rs.PathParam; // annotation to get parameters from URI
9  import javax.ws.rs.Produces; // annotation to specify type of data
10
11 @Path( "welcome" ) // path used to access the resource
12 public class WelcomeRESTJSONResource
13 {
14    // retrieve welcome message
15    @GET // handles HTTP GET requests
16    @Path( "{name}" ) // takes name as a path parameter
17    @Produces( "application/json" ) // response formatted as JSON
18    public String getJson( @PathParam( "name" ) String name )
19    {
20       // add welcome message to field of TextMessage object
21       TextMessage message = new TextMessage(); // create wrapper object
22       message.setMessage( String.format( "%s, %s!",
23          "Welcome to JAX-RS web services with REST and JSON", name ) );
24
25       return new Gson().toJson( message ); // return JSON-wrapped message
26    } // end method getJson
27 } // end class WelcomeRESTJSONResource
28
29 // private class that contains the message we wish to send
30 class TextMessage
31 {
32    private String message; // message we're sending
33
34    // returns the message
35    public String getMessage()
36    {
37       return message;
38    } // end method getMessage
39
40    // sets the message
41    public void setMessage( String value )
42    {
43       message = value;
44    } // end method setMessage
45 } // end class TextMessage
```

Fig. 31.15 | REST web service that returns a welcome message as JSON.

All the annotations and the basic structure of the `WelcomeRESTJSONResource` class are the same as REST XML example. The argument to the `@Produces` attribute (line 17) is

"application/json". The TextMessage class (lines 30–45) addresses a difference between JSON and XML. JSON does not permit strings or numbers to stand on their own—they must be encapsulated in a composite data type. So, we created class TextMessage to encapsulate the String representing the message.

When a client invokes this web service, line 21 creates the TextMessage object, then lines 22–23 set its contained message. Next, line 25 creates a **Gson** object (from package com.google.gson.Gson) and calls its **toJson** method to convert the TextMessage into its JSON String representation. We return this String, which is then sent back to the client in the web service's response. There are multiple overloads of the toJson method, such as one that sends its output to a Writer instead of returning a String.

RESTful services returning JSON can be tested in the same way as those returning XML. Follow the procedure outlined in Section 31.7.1, but be sure to change the MIME type to application/json in the test web page; otherwise, the web service will return an error stating that it cannot produce the desired response.

31.8.2 Consuming a REST-Based JSON Web Service

We now create a Java application that retrieves the welcome message from the web service and displays it to the user. First, create a Java application with the name WelcomeRESTJSONClient. Then, create a JFrame form called WelcomeRESTXMLClientJFrame and place it in the com.deitel.welcomerestjsonclient package. The GUI is identical to the one in Fig. 31.10. To create the GUI quickly, copy it from the **Design** view of the WelcomeSOAPClientJFrame class and paste it into the **Design** view of the WelcomeRESTJSONClientJFrame class. Figure 31.16 contains the completed code.

```
1  // Fig. 31.16: WelcomeRESTJSONClientJFrame.java
2  // Client that consumes the WelcomeRESTJSON service.
3  package com.deitel.welcomerestjsonclient;
4
5  import com.google.gson.Gson; // converts POJO to JSON and back again
6  import java.io.InputStreamReader;
7  import java.net.URL;
8  import javax.swing.JOptionPane;
9
10 public class WelcomeRESTJSONClientJFrame extends javax.swing.JFrame
11 {
12    // no-argument constructor
13    public WelcomeRESTJSONClientJFrame()
14    {
15       initComponents();
16    } // end constructor
17
18    // The initComponents method is autogenerated by NetBeans and is called
19    // from the constructor to initialize the GUI. This method is not shown
20    // here to save space. Open WelcomeRESTJSONClientJFrame.java in this
21    // example's folder to view the complete generated code.
22
```

Fig. 31.16 | Client that consumes the WelcomeRESTJSON service. (Part 1 of 3.)

```
23     // call the web service with the supplied name and display the message
24     private void submitJButtonActionPerformed(
25        java.awt.event.ActionEvent evt )
26     {
27        String name = nameJTextField.getText(); // get name from JTextField
28
29        // retrieve the welcome string from the web service
30        try
31        {
32           // the URL of the web service
33           String url = "http://localhost:8080/WelcomeRESTJSON/" +
34              "resources/welcome/" + name;
35
36           // open URL, using a Reader to convert bytes to chars
37           InputStreamReader reader =
38              new InputStreamReader( new URL( url ).openStream() );
39
40           // parse the JSON back into a TextMessage
41           TextMessage message =
42              new Gson().fromJson( reader, TextMessage.class );
43
44           // display message to the user
45           JOptionPane.showMessageDialog( this, message.getMessage(),
46              "Welcome", JOptionPane.INFORMATION_MESSAGE );
47        } // end try
48        catch ( Exception exception )
49        {
50           exception.printStackTrace(); // show exception details
51        } // end catch
52     } // end method submitJButtonActionPerformed
53
54     // main method begin execution
55     public static void main( String args[] )
56     {
57        java.awt.EventQueue.invokeLater(
58           new Runnable()
59           {
60              public void run()
61              {
62                 new WelcomeRESTJSONClientJFrame().setVisible( true );
63              } // end method run
64           } // end anonymous inner class
65        ); // end call to java.awt.EventQueue.invokeLater
66     } // end main
67
68     // Variables declaration - do not modify
69     private javax.swing.JLabel nameJLabel;
70     private javax.swing.JTextField nameJTextField;
71     private javax.swing.JButton submitJButton;
72     // End of variables declaration
73  } // end class WelcomeRESTJSONClientJFrame
74
```

Fig. 31.16 | Client that consumes the `WelcomeRESTJSON` service. (Part 2 of 3.)

```
75  // private class that contains the message we are receiving
76  class TextMessage
77  {
78     private String message; // message we're receiving
79
80     // returns the message
81     public String getMessage()
82     {
83        return message;
84     } // end method getMessage
85
86     // sets the message
87     public void setMessage( String value )
88     {
89        message = value;
90     } // end method setMessage
91  } // end class TextMessage
```

Fig. 31.16 | Client that consumes the `WelcomeRESTJSON` service. (Part 3 of 3.)

Lines 83–84 create the URL `String` that is used to invoke the web service. Lines 87–88 create a `URL` object using this `String`, then call the `URL`'s **`openStream`** method to invoke the web service and obtain an `InputStream` from which the client can read the response. The `InputStream` is wrapped in an `InputStreamReader` so it can be passed as the first argument to the `Gson` class's **`fromJson`** method. This method is overloaded. The version we use takes as arguments a `Reader` from which to read a JSON `String` and a `Class<T>` object indicating the Java class to which the JSON `String` will be converted (line 92). In this example, the JSON `String` contains a `TextMessage` object, so we use the Java compiler shortcut `TextMessage.class` to create the `Class<TextMessage>` object we need as the second argument. Lines 95–96 display the message in the `TextMessage` object.

The `TextMessage` classes in the web service and client are unrelated. Technically, the client can be written in any programming language, so the manner in which a response is processed can vary greatly. Since our client is written in Java, we duplicated the `TextMessage` class in the client so we could easily convert the JSON object back to Java.

31.9 Session Tracking in a SOAP Web Service

Section 29.8 described the advantages of using session tracking to maintain client-state information so you can personalize the users' browsing experiences. Now we'll incorporate *session tracking* into a web service. Suppose a client application needs to call several methods from the same web service, possibly several times each. In such a case, it can be beneficial for the web service to maintain state information for the client, thus eliminating the need for client information to be passed between the client and the web service multiple times. For example, a web service that provides local restaurant reviews could store the client user's street address during the initial request, then use it to return personalized, localized results in subsequent requests. Storing session information also enables a web service to distinguish between clients.

31.9.1 Creating a Blackjack Web Service

Our next example is a web service that assists you in developing a blackjack card game. The `Blackjack` web service (Fig. 31.17) provides web methods to shuffle a deck of cards, deal a card from the deck and evaluate a hand of cards. After presenting the web service, we use it to serve as the dealer for a game of blackjack (Fig. 31.18). The `Blackjack` web service uses an `HttpSession` object to maintain a unique deck of cards for each client application. Several clients can use the service at the same time, but web method calls made by a specific client use only the deck of cards stored in that client's session. Our example uses the following blackjack rules:

> *Two cards each are dealt to the dealer and the player. The player's cards are dealt face up. Only the first of the dealer's cards is dealt face up. Each card has a value. A card numbered 2 through 10 is worth its face value. Jacks, queens and kings each count as 10. Aces can count as 1 or 11—whichever value is more beneficial to the player (as we'll soon see). If the sum of the player's two initial cards is 21 (i.e., the player was dealt a card valued at 10 and an ace, which counts as 11 in this situation), the player has "blackjack" and immediately wins the game—if the dealer does not also have blackjack (which would result in a "push"—i.e., a tie). Otherwise, the player can begin taking additional cards one at a time. These cards are dealt face up, and the player decides when to stop taking cards. If the player "busts" (i.e., the sum of the player's cards exceeds 21), the game is over, and the player loses. When the player is satisfied with the current set of cards, the player "stands" (i.e., stops taking cards), and the dealer's hidden card is revealed. If the dealer's total is 16 or less, the dealer must take another card; otherwise, the dealer must stand. The dealer must continue taking cards until the sum of the dealer's cards is greater than or equal to 17. If the dealer exceeds 21, the player wins. Otherwise, the hand with the higher point total wins. If the dealer and the player have the same point total, the game is a "push," and no one wins. The value of an ace for a dealer depends on the dealer's other card(s) and the casino's house rules. A dealer typically must hit for totals of 16 or less and must stand for totals of 17 or more. However, for a "soft 17"—a hand with a total of 17 with one ace counted as 11—some casinos require the dealer to hit and some require the dealer to stand (we require the dealer to stand). Such a hand is known as a "soft 17" because taking another card cannot bust the hand.*

The web service (Fig. 31.17) stores each card as a `String` consisting of a number, `1`–`13`, representing the card's face (ace through king, respectively), followed by a space and a digit, `0`–`3`, representing the card's suit (hearts, diamonds, clubs or spades, respectively). For example, the jack of clubs is represented as `"11 2"` and the two of hearts as `"2 0"`. To create and deploy this web service, follow the steps that we presented in Sections 31.6.2–31.6.3 for the `WelcomeSOAP` service.

```
1  // Fig. 31.17: Blackjack.java
2  // Blackjack web service that deals cards and evaluates hands
3  package com.deitel.blackjack;
4
5  import com.sun.xml.ws.developer.servlet.HttpSessionScope;
6  import java.util.ArrayList;
7  import java.util.Random;
```

Fig. 31.17 | `Blackjack` web service that deals cards and evaluates hands. (Part 1 of 3.)

```
8  import javax.jws.WebMethod;
9  import javax.jws.WebParam;
10 import javax.jws.WebService;
11
12 @HttpSessionScope // enable web service to maintain session state
13 @WebService()
14 public class Blackjack
15 {
16    private ArrayList< String > deck; // deck of cards for one user session
17    private static final Random randomObject = new Random();
18
19    // deal one card
20    @WebMethod( operationName = "dealCard" )
21    public String dealCard()
22    {
23       String card = "";
24       card = deck.get( 0 ); // get top card of deck
25       deck.remove( 0 ); // remove top card of deck
26       return card;
27    } // end WebMethod dealCard
28
29    // shuffle the deck
30    @WebMethod( operationName = "shuffle" )
31    public void shuffle()
32    {
33       // create new deck when shuffle is called
34       deck = new ArrayList< String >();
35
36       // populate deck of cards
37       for ( int face = 1; face <= 13; face++ ) // loop through faces
38          for ( int suit = 0; suit <= 3; suit++ ) // loop through suits
39             deck.add( face + " " + suit ); // add each card to deck
40
41       String tempCard; // holds card temporarily during swapping
42       int index; // index of randomly selected card
43
44       for ( int i = 0; i < deck.size() ; i++ ) // shuffle
45       {
46          index = randomObject.nextInt( deck.size() - 1 );
47
48          // swap card at position i with randomly selected card
49          tempCard = deck.get( i );
50          deck.set( i, deck.get( index ) );
51          deck.set( index, tempCard );
52       } // end for
53    } // end WebMethod shuffle
54
55    // determine a hand's value
56    @WebMethod( operationName = "getHandValue" )
57    public int getHandValue( @WebParam( name = "hand" ) String hand )
58    {
59       // split hand into cards
60       String[] cards = hand.split( "\t" );
```

Fig. 31.17 | `Blackjack` web service that deals cards and evaluates hands. (Part 2 of 3.)

```
61         int total = 0; // total value of cards in hand
62         int face; // face of current card
63         int aceCount = 0; // number of aces in hand
64
65         for ( int i = 0; i < cards.length; i++ )
66         {
67            // parse string and get first int in String
68            face = Integer.parseInt(
69               cards[ i ].substring( 0, cards[ i ].indexOf( " " ) ) );
70
71            switch ( face )
72            {
73               case 1: // if ace, increment aceCount
74                  ++aceCount;
75                  break;
76               case 11: // jack
77               case 12: // queen
78               case 13: // king
79                  total += 10;
80                  break;
81               default: // otherwise, add face
82                  total += face;
83                  break;
84            } // end switch
85         } // end for
86
87         // calculate optimal use of aces
88         if ( aceCount > 0 )
89         {
90            // if possible, count one ace as 11
91            if ( total + 11 + aceCount - 1 <= 21 )
92               total += 11 + aceCount - 1;
93            else // otherwise, count all aces as 1
94               total += aceCount;
95          } // end if
96
97         return total;
98      } // end WebMethod getHandValue
99   } // end class Blackjack
```

Fig. 31.17 | `Blackjack` web service that deals cards and evaluates hands. (Part 3 of 3.)

Session Tracking in Web Services: `@HttpSessionScope` *Annotation*

In JAX-WS 2.2, it's easy to enable session tracking in a web service. You simply precede your web service class with the **`@HttpSessionScope` annotation**. This annotation is located in package `com.sun.xml.ws.developer.servlet`. To use this package you must add the JAX-WS 2.2 library to your project. To do so, right click the **Libraries** node in your `Blackjack` web application project and select **Add Library...**. Then, in the dialog that appears, locate and select **JAX-WS 2.2**, then click **Add Library**. Once a web service is annotated with `@HttpSessionScope`, the server automatically maintains a separate instance of the class for each client session. Thus, the `deck` instance variable (line 16) will be maintained separately for each client.

Client Interactions with the `Blackjack` Web Service

A client first calls the `Blackjack` web service's `shuffle` web method (lines 30–53) to create a new deck of cards (line 34), populate it (lines 37–39) and shuffle it (lines 41–52). Lines 37–39 generate `String`s in the form "*face suit*" to represent each possible card in the deck.

Lines 20–27 define the `dealCard` web method. Method `shuffle` *must* be called before method `dealCard` is called the first time for a client—otherwise, `deck` could be `null`. The method gets the top card from the deck (line 24), removes it from the deck (line 25) and returns the card's value as a `String` (line 26). Without using session tracking, the deck of cards would need to be passed back and forth with each method call. Session tracking makes the `dealCard` method easy to call (it requires no arguments) and eliminates the overhead of sending the deck over the network multiple times.

Method `getHandValue` (lines 56–98) determines the total value of the cards in a hand by trying to attain the highest score possible without going over 21. Recall that an ace can be counted as either 1 or 11, and all face cards count as 10. This method does not use the `session` object, because the deck of cards is not used in this method.

As you'll soon see, the client application maintains a hand of cards as a `String` in which each card is separated by a tab character. Line 60 splits the hand of cards (represented by `hand`) into individual cards by calling `String` method `split` and passing to it a `String` containing the delimiter characters (in this case, just a tab). Method `split` uses the delimiter characters to separate tokens in the `String`. Lines 65–85 count the value of each card. Lines 68–69 retrieve the first integer—the face—and use that value in the `switch` statement (lines 71–84). If the card is an ace, the method increments variable `aceCount`. We discuss how this variable is used shortly. If the card is an 11, 12 or 13 (jack, queen or king), the method adds 10 to the total value of the hand (line 79). If the card is anything else, the method increases the total by that value (line 82).

Because an ace can have either of two values, additional logic is required to process aces. Lines 88–95 process the aces after all the other cards. If a hand contains several aces, only one ace can be counted as 11. The condition in line 91 determines whether counting one ace as 11 and the rest as 1 will result in a total that does not exceed 21. If this is possible, line 92 adjusts the total accordingly. Otherwise, line 94 adjusts the total, counting each ace as 1.

Method `getHandValue` maximizes the value of the current cards without exceeding 21. Imagine, for example, that the dealer has a 7 and receives an ace. The new total could be either 8 or 18. However, `getHandValue` always maximizes the value of the cards without going over 21, so the new total is 18.

31.9.2 Consuming the `Blackjack` Web Service

The blackjack application in Fig. 31.18 keeps track of the player's and dealer's cards, and the web service tracks the cards that have been dealt. The constructor (lines 34–83) sets up the GUI (line 36), changes the window's background color (line 40) and creates the `Blackjack` web service's service endpoint interface object (lines 46–47). In the GUI, each player has 11 `JLabel`s—the maximum number of cards that can be dealt without automatically exceeding 21 (i.e., four aces, four twos and three threes). These `JLabel`s are placed in an `ArrayList` of `JLabel`s (lines 59–82), so we can index the `ArrayList` during the game to determine the `JLabel` that will display a particular card image.

```
1  // Fig. 31.18: BlackjackGameJFrame.java
2  // Blackjack game that uses the Blackjack Web Service.
3  package com.deitel.blackjackclient;
4
5  import com.deitel.blackjack.Blackjack;
6  import com.deitel.blackjack.BlackjackService;
7  import java.awt.Color;
8  import java.util.ArrayList;
9  import javax.swing.ImageIcon;
10 import javax.swing.JLabel;
11 import javax.swing.JOptionPane;
12 import javax.xml.ws.BindingProvider;
13
14 public class BlackjackGameJFrame extends javax.swing.JFrame
15 {
16    private String playerCards;
17    private String dealerCards;
18    private ArrayList<JLabel> cardboxes; // list of card image JLabels
19    private int currentPlayerCard; // player's current card number
20    private int currentDealerCard; // blackjackProxy's current card number
21    private BlackjackService blackjackService; // used to obtain proxy
22    private Blackjack blackjackProxy; // used to access the web service
23
24    // enumeration of game states
25    private enum GameStatus
26    {
27       PUSH, // game ends in a tie
28       LOSE, // player loses
29       WIN, // player wins
30       BLACKJACK // player has blackjack
31    } // end enum GameStatus
32
33    // no-argument constructor
34    public BlackjackGameJFrame()
35    {
36       initComponents();
37
38       // due to a bug in NetBeans, we must change the JFrame's background
39       // color here rather than in the designer
40       getContentPane().setBackground( new Color( 0, 180, 0 ) );
41
42       // initialize the blackjack proxy
43       try
44       {
45          // create the objects for accessing the Blackjack web service
46          blackjackService = new BlackjackService();
47          blackjackProxy = blackjackService.getBlackjackPort();
48
49          // enable session tracking
50          ( (BindingProvider) blackjackProxy ).getRequestContext().put(
51             BindingProvider.SESSION_MAINTAIN_PROPERTY, true );
52       } // end try
```

Fig. 31.18 | Blackjack game that uses the `Blackjack` web service. (Part 1 of 10.)

```
53          catch ( Exception e )
54          {
55             e.printStackTrace();
56          } // end catch
57
58          // add JLabels to cardBoxes ArrayList for programmatic manipulation
59          cardboxes = new ArrayList<JLabel>();
60
61          cardboxes.add( dealerCard1JLabel );
62          cardboxes.add( dealerCard2JLabel );
63          cardboxes.add( dealerCard3JLabel );
64          cardboxes.add( dealerCard4JLabel );
65          cardboxes.add( dealerCard5JLabel );
66          cardboxes.add( dealerCard6JLabel );
67          cardboxes.add( dealerCard7JLabel );
68          cardboxes.add( dealerCard8JLabel );
69          cardboxes.add( dealerCard9JLabel );
70          cardboxes.add( dealerCard10JLabel );
71          cardboxes.add( dealerCard11JLabel );
72          cardboxes.add( playerCard1JLabel );
73          cardboxes.add( playerCard2JLabel );
74          cardboxes.add( playerCard3JLabel );
75          cardboxes.add( playerCard4JLabel );
76          cardboxes.add( playerCard5JLabel );
77          cardboxes.add( playerCard6JLabel );
78          cardboxes.add( playerCard7JLabel );
79          cardboxes.add( playerCard8JLabel );
80          cardboxes.add( playerCard9JLabel );
81          cardboxes.add( playerCard10JLabel );
82          cardboxes.add( playerCard11JLabel );
83       } // end constructor
84
85       // play the dealer's hand
86       private void dealerPlay()
87       {
88          try
89          {
90             // while the value of the dealers's hand is below 17
91             // the dealer must continue to take cards
92             String[] cards = dealerCards.split( "\t" );
93
94             // display dealer's cards
95             for ( int i = 0; i < cards.length; i++ )
96             {
97                displayCard( i, cards[i] );
98             }
99
100            while ( blackjackProxy.getHandValue( dealerCards ) < 17 )
101            {
102               String newCard = blackjackProxy.dealCard(); // deal new card
103               dealerCards += "\t" + newCard; // deal new card
104               displayCard( currentDealerCard, newCard );
```

Fig. 31.18 | Blackjack game that uses the `Blackjack` web service. (Part 2 of 10.)

```
105             ++currentDealerCard;
106             JOptionPane.showMessageDialog( this, "Dealer takes a card",
107                "Dealer's turn", JOptionPane.PLAIN_MESSAGE );
108          } // end while
109
110          int dealersTotal = blackjackProxy.getHandValue( dealerCards );
111          int playersTotal = blackjackProxy.getHandValue( playerCards );
112
113          // if dealer busted, player wins
114          if ( dealersTotal > 21 )
115          {
116             gameOver( GameStatus.WIN );
117             return;
118          } // end if
119
120          // if dealer and player are below 21
121          // higher score wins, equal scores is a push
122          if ( dealersTotal > playersTotal )
123          {
124             gameOver( GameStatus.LOSE );
125          }
126          else if ( dealersTotal < playersTotal )
127          {
128             gameOver( GameStatus.WIN );
129          }
130          else
131          {
132             gameOver( GameStatus.PUSH );
133          }
134       } // end try
135       catch ( Exception e )
136       {
137          e.printStackTrace();
138       } // end catch
139    } // end method dealerPlay
140
141    // displays the card represented by cardValue in specified JLabel
142    private void displayCard( int card, String cardValue )
143    {
144       try
145       {
146          // retrieve correct JLabel from cardBoxes
147          JLabel displayLabel = cardboxes.get( card );
148
149          // if string representing card is empty, display back of card
150          if ( cardValue.equals( "" ) )
151          {
152             displayLabel.setIcon( new ImageIcon( getClass().getResource(
153                "/com/deitel/java/blackjackclient/" +
154                "blackjack_images/cardback.png" ) ) );
155             return;
156          } // end if
```

Fig. 31.18 | Blackjack game that uses the `Blackjack` web service. (Part 3 of 10.)

```
157
158          // retrieve the face value of the card
159          String face = cardValue.substring( 0, cardValue.indexOf( " " ) );
160
161          // retrieve the suit of the card
162          String suit =
163             cardValue.substring( cardValue.indexOf( " " ) + 1 );
164
165          char suitLetter; // suit letter used to form image file
166
167          switch ( Integer.parseInt( suit ) )
168          {
169             case 0: // hearts
170                suitLetter = 'h';
171                break;
172             case 1: // diamonds
173                suitLetter = 'd';
174                break;
175             case 2: // clubs
176                suitLetter = 'c';
177                break;
178             default: // spades
179                suitLetter = 's';
180                break;
181          } // end switch
182
183          // set image for displayLabel
184          displayLabel.setIcon( new ImageIcon( getClass().getResource(
185             "/com/deitel/java/blackjackclient/blackjack_images/" +
186             face + suitLetter + ".png" ) ) );
187       } // end try
188       catch ( Exception e )
189       {
190          e.printStackTrace();
191       } // end catch
192    } // end method displayCard
193
194    // displays all player cards and shows appropriate message
195    private void gameOver( GameStatus winner )
196    {
197       String[] cards = dealerCards.split( "\t" );
198
199       // display blackjackProxy's cards
200       for ( int i = 0; i < cards.length; i++ )
201       {
202          displayCard( i, cards[i] );
203       }
204
205       // display appropriate status image
206       if ( winner == GameStatus.WIN )
207       {
208          statusJLabel.setText( "You win!" );
209       }
```

Fig. 31.18 | Blackjack game that uses the `Blackjack` web service. (Part 4 of 10.)

```
210         else if ( winner == GameStatus.LOSE )
211         {
212            statusJLabel.setText( "You lose." );
213         }
214         else if ( winner == GameStatus.PUSH )
215         {
216            statusJLabel.setText( "It's a push." );
217         }
218         else // blackjack
219         {
220            statusJLabel.setText( "Blackjack!" );
221         }
222
223         // display final scores
224         int dealersTotal = blackjackProxy.getHandValue( dealerCards );
225         int playersTotal = blackjackProxy.getHandValue( playerCards );
226         dealerTotalJLabel.setText( "Dealer: " + dealersTotal );
227         playerTotalJLabel.setText( "Player: " + playersTotal );
228
229         // reset for new game
230         standJButton.setEnabled( false );
231         hitJButton.setEnabled( false );
232         dealJButton.setEnabled( true );
233      } // end method gameOver
234
235      // The initComponents method is autogenerated by NetBeans and is called
236      // from the constructor to initialize the GUI. This method is not shown
237      // here to save space. Open BlackjackGameJFrame.java in this
238      // example's folder to view the complete generated code
239
542      // handles dealJButton click
543      private void dealJButtonActionPerformed(
544         java.awt.event.ActionEvent evt )
545      {
546         String card; // stores a card temporarily until it's added to a hand
547
548         // clear card images
549         for ( int i = 0; i < cardboxes.size(); i++ )
550         {
551            cardboxes.get( i ).setIcon( null );
552         }
553
554         statusJLabel.setText( "" );
555         dealerTotalJLabel.setText( "" );
556         playerTotalJLabel.setText( "" );
557
558         // create a new, shuffled deck on remote machine
559         blackjackProxy.shuffle();
560
561         // deal two cards to player
562         playerCards = blackjackProxy.dealCard(); // add first card to hand
563         displayCard( 11, playerCards ); // display first card
564         card = blackjackProxy.dealCard(); // deal second card
```

Fig. 31.18 | Blackjack game that uses the `Blackjack` web service. (Part 5 of 10.)

```
565         displayCard( 12, card ); // display second card
566         playerCards += "\t" + card; // add second card to hand
567
568         // deal two cards to blackjackProxy, but only show first
569         dealerCards = blackjackProxy.dealCard(); // add first card to hand
570         displayCard( 0, dealerCards ); // display first card
571         card = blackjackProxy.dealCard(); // deal second card
572         displayCard( 1, "" ); // display back of card
573         dealerCards += "\t" + card; // add second card to hand
574
575         standJButton.setEnabled( true );
576         hitJButton.setEnabled( true );
577         dealJButton.setEnabled( false );
578
579         // determine the value of the two hands
580         int dealersTotal = blackjackProxy.getHandValue( dealerCards );
581         int playersTotal = blackjackProxy.getHandValue( playerCards );
582
583         // if hands both equal 21, it is a push
584         if ( playersTotal == dealersTotal && playersTotal == 21 )
585         {
586            gameOver( GameStatus.PUSH );
587         }
588         else if ( dealersTotal == 21 ) // blackjackProxy has blackjack
589         {
590            gameOver( GameStatus.LOSE );
591         }
592         else if ( playersTotal == 21 ) // blackjack
593         {
594            gameOver( GameStatus.BLACKJACK );
595         }
596
597         // next card for blackjackProxy has index 2
598         currentDealerCard = 2;
599
600         // next card for player has index 13
601         currentPlayerCard = 13;
602      } // end method dealJButtonActionPerformed
603
604      // handles standJButton click
605      private void hitJButtonActionPerformed(
606         java.awt.event.ActionEvent evt )
607      {
608         // get player another card
609         String card = blackjackProxy.dealCard(); // deal new card
610         playerCards += "\t" + card; // add card to hand
611
612         // update GUI to display new card
613         displayCard( currentPlayerCard, card );
614         ++currentPlayerCard;
615
616         // determine new value of player's hand
617         int total = blackjackProxy.getHandValue( playerCards );
```

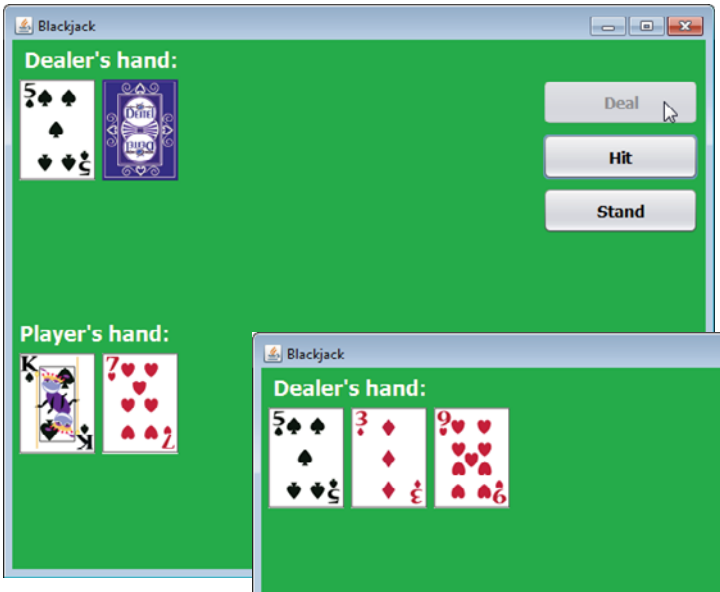
Fig. 31.18 | Blackjack game that uses the `Blackjack` web service. (Part 6 of 10.)

```
618
619        if ( total > 21 ) // player busts
620        {
621           gameOver( GameStatus.LOSE );
622        }
623        else if ( total == 21 ) // player cannot take any more cards
624        {
625           hitJButton.setEnabled( false );
626           dealerPlay();
627        } // end if
628     } // end method hitJButtonActionPerformed
629
630     // handles standJButton click
631     private void standJButtonActionPerformed(
632        java.awt.event.ActionEvent evt )
633     {
634        standJButton.setEnabled( false );
635        hitJButton.setEnabled( false );
636        dealJButton.setEnabled( true );
637        dealerPlay();
638     } // end method standJButtonActionPerformed
639
640     // begins application execution
641     public static void main( String args[] )
642     {
643        java.awt.EventQueue.invokeLater(
644           new Runnable()
645           {
646              public void run()
647              {
648                 new BlackjackGameJFrame().setVisible( true );
649              }
650           }
651        ); // end call to java.awt.EventQueue.invokeLater
652     } // end main
653
654     // Variables declaration - do not modify
655     private javax.swing.JButton dealJButton;
656     private javax.swing.JLabel dealerCard10JLabel;
657     private javax.swing.JLabel dealerCard11JLabel;
658     private javax.swing.JLabel dealerCard1JLabel;
659     private javax.swing.JLabel dealerCard2JLabel;
660     private javax.swing.JLabel dealerCard3JLabel;
661     private javax.swing.JLabel dealerCard4JLabel;
662     private javax.swing.JLabel dealerCard5JLabel;
663     private javax.swing.JLabel dealerCard6JLabel;
664     private javax.swing.JLabel dealerCard7JLabel;
665     private javax.swing.JLabel dealerCard8JLabel;
666     private javax.swing.JLabel dealerCard9JLabel;
667     private javax.swing.JLabel dealerJLabel;
668     private javax.swing.JLabel dealerTotalJLabel;
669     private javax.swing.JButton hitJButton;
670     private javax.swing.JLabel playerCard10JLabel;
```

Fig. 31.18 | Blackjack game that uses the `Blackjack` web service. (Part 7 of 10.)

```
671     private javax.swing.JLabel playerCard11JLabel;
672     private javax.swing.JLabel playerCard1JLabel;
673     private javax.swing.JLabel playerCard2JLabel;
674     private javax.swing.JLabel playerCard3JLabel;
675     private javax.swing.JLabel playerCard4JLabel;
676     private javax.swing.JLabel playerCard5JLabel;
677     private javax.swing.JLabel playerCard6JLabel;
678     private javax.swing.JLabel playerCard7JLabel;
679     private javax.swing.JLabel playerCard8JLabel;
680     private javax.swing.JLabel playerCard9JLabel;
681     private javax.swing.JLabel playerJLabel;
682     private javax.swing.JLabel playerTotalJLabel;
683     private javax.swing.JButton standJButton;
684     private javax.swing.JLabel statusJLabel;
685     // End of variables declaration
686 } // end class BlackjackGameJFrame
```

a) Dealer and player hands after the user clicks the **Deal** `JButton`



b) Dealer and player hands after the user clicks **Stand**. In this case, the result is a push



Fig. 31.18 | Blackjack game that uses the `Blackjack` web service. (Part 8 of 10.)

c) Dealer and player hands after the user clicks **Hit** and draws 21. In this case, the player wins



d) Dealer and player hands after the player is dealt blackjack

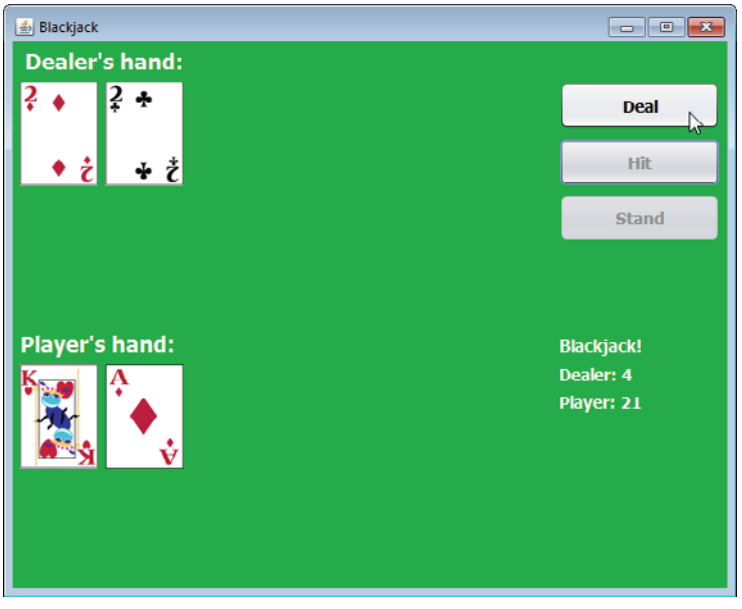


Fig. 31.18 | Blackjack game that uses the `Blackjack` web service. (Part 9 of 10.)

e) Dealer and player hands after the dealer is dealt blackjack

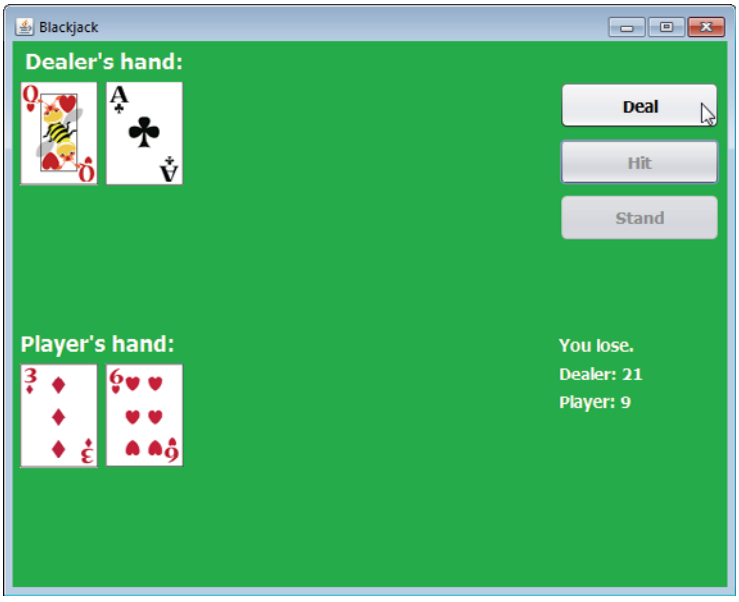


Fig. 31.18 | Blackjack game that uses the `Blackjack` web service. (Part 10 of 10.)

Configuring the Client for Session Tracking

When interacting with a JAX-WS web service that performs session tracking, the client application must indicate whether it wants to allow the web service to maintain session information. Lines 50–51 in the constructor perform this task. We first cast the service endpoint interface object to interface type `BindingProvider`. A **`BindingProvider`** enables the client to manipulate the request information that will be sent to the server. This information is stored in an object that implements interface **`RequestContext`**. The `BindingProvider` and `RequestContext` are part of the framework that is created by the IDE when you add a web service client to the application. Next, we invoke the `BindingProvider`'s **`getRequestContext` method** to obtain the `RequestContext` object. Then we call the `RequestContext`'s **`put` method** to set the property

```
BindingProvider.SESSION_MAINTAIN_PROPERTY
```

to `true`. This enables the client side of the session-tracking mechanism, so that the web service knows which client is invoking the service's web methods.

Method `gameOver`

Method `gameOver` (lines 195–233) displays all the dealer's cards, shows the appropriate message in `statusJLabel` and displays the final point totals of both the dealer and the player. Method `gameOver` receives as an argument a member of the `GameStatus` enumeration (defined in lines 25–31). The enumeration represents whether the player tied, lost or won the game; its four members are `PUSH`, `LOSE`, `WIN` and `BLACKJACK`.

Method `dealJButtonActionPerformed`

When the player clicks the **Deal** `JButton`, method `dealJButtonActionPerformed` (lines 543–602) clears all of the `JLabel`s that display cards or game status information. Next, the deck is shuffled (line 559), and the player and dealer receive two cards each (lines 562–573). Lines 580–581 then total each hand. If the player and the dealer both obtain scores of 21, the program calls method `gameOver`, passing `GameStatus.PUSH` (line 586). If only the dealer has 21, the program passes `GameStatus.LOSE` to method `gameOver` (line 590). If only the player has 21 after the first two cards are dealt, the program passes `GameStatus.BLACKJACK` to method `gameOver` (line 594).

Method `hitJButtonActionPerformed`

If `dealJButtonActionPerformed` does not call `gameOver`, the player can take more cards by clicking the **Hit** `JButton`, which calls `hitJButtonActionPerformed` in lines 605–628. Each time a player clicks **Hit**, the program deals the player one more card (line 609) and displays it in the GUI (line 613). If the player exceeds 21, the game is over and the player loses (line 621). If the player has exactly 21, the player is not allowed to take any more cards (line 625), and method `dealerPlay` is called (line 626).

Method `dealerPlay`

Method `dealerPlay` (lines 86–139) displays the dealer's cards, then deals cards to the dealer until the dealer's hand has a value of 17 or more (lines 100–108). If the dealer exceeds 21, the player wins (line 116); otherwise, the values of the hands are compared, and `gameOver` is called with the appropriate argument (lines 122–133).

Method `standJButtonActionPerformed`

Clicking the **Stand** `JButton` indicates that a player does not want to be dealt another card. Method `standJButtonActionPerformed` (lines 631–638) disables the **Hit** and **Stand** buttons, enables the **Deal** button, then calls method `dealerPlay`.

Method `displayCard`

Method `displayCard` (lines 142–192) updates the GUI to display a newly dealt card. The method takes as arguments an integer index for the `JLabel` in the `ArrayList` that must have its image set and a `String` representing the card. An empty `String` indicates that we wish to display the card face down. If method `displayCard` receives a `String` that's not empty, the program extracts the face and suit from the `String` and uses this information to display the correct image. The `switch` statement (lines 167–181) converts the number representing the suit to an integer and assigns the appropriate character to variable `suitLetter` (`h` for hearts, `d` for diamonds, `c` for clubs and `s` for spades). The character in `suitLetter` is used to complete the image's file name (lines 184–186). *You must add the folder* `blackjack_images` *to your project so that lines 152–154 and 184–186 can access the images properly.* To do so, copy the folder `blackjack_images` from this chapter's examples folder and paste it into the project's `src\com\deitel\java\blackjackclient` folder.

31.10 Consuming a Database-Driven SOAP Web Service

Our prior examples accessed web services from desktop applications created in NetBeans. However, we can just as easily use them in web applications created with NetBeans. In fact, because web-based businesses are becoming increasingly popular, it's common for

web applications to consume web services. In this section, we present an airline reservation web service that receives information regarding the type of seat a customer wishes to reserve and makes a reservation if such a seat is available. Later in the section, we present a web application that allows a customer to specify a reservation request, then uses the airline reservation web service to attempt to execute the request.

31.10.1 Creating the `Reservation` Database

Our web service uses a `reservation` database containing a single table named `Seats` to locate a seat matching a client's request. Review the steps presented in Section 30.2.1 for configuring a data source and the `addressbook` database. Then perform those steps for the reservation database used in this example. Create a data source named `jdbc/reservation`. This chapter's examples directory contains the `Seats.sql` SQL script to create the `seats` table and populate it with sample data. The sample data is shown in Fig. 31.19.

number	location	class	taken
1	Aisle	Economy	0
2	Aisle	Economy	0
3	Aisle	First	0
4	Middle	Economy	0
5	Middle	Economy	0
6	Middle	First	0
7	Window	Economy	0
8	Window	Economy	0
9	Window	First	0
10	Window	First	0

Fig. 31.19 | Data from the `seats` table.

Creating the Reservation Web Service

You can now create a web service that uses the `Reservation` database (Fig. 31.20). The airline reservation web service has a single web method—`reserve` (lines 23–78)—which searches the `Seats` table to locate a seat matching a user's request. The method takes two arguments—a `String` representing the desired seat type (i.e., `"Window"`, `"Middle"` or `"Aisle"`) and a `String` representing the desired class type (i.e., `"Economy"` or `"First"`). If it finds an appropriate seat, method `reserve` updates the database to make the reservation and returns `true`; otherwise, no reservation is made, and the method returns `false`. The statements at lines 34–39 and lines 45–48 that query and update the database use objects of JDBC types `ResultSet` and `PreparedStatement`.



Software Engineering Observation 31.1

Using `PreparedStatements` to create SQL statements is highly recommended to secure against so-called SQL injection attacks in which executable code is inserted into SQL code. The site `www.owasp.org/index.php/Preventing_SQL_Injection_in_Java` provides a summary of SQL injection attacks and ways to mitigate against them.

```
1  // Fig. 31.20: Reservation.java
2  // Airline reservation web service.
3  package com.deitel.reservation;
4
5  import java.sql.Connection;
6  import java.sql.PreparedStatement;
7  import java.sql.ResultSet;
8  import java.sql.SQLException;
9  import javax.annotation.Resource;
10 import javax.jws.WebMethod;
11 import javax.jws.WebParam;
12 import javax.jws.WebService;
13 import javax.sql.DataSource;
14
15 @WebService()
16 public class Reservation
17 {
18    // allow the server to inject the DataSource
19    @Resource( name="jdbc/reservation" )
20    DataSource dataSource;
21
22    // a WebMethod that can reserve a seat
23    @WebMethod( operationName = "reserve" )
24    public boolean reserve( @WebParam( name = "seatType" ) String seatType,
25       @WebParam( name = "classType" ) String classType )
26    {
27       Connection connection = null;
28       PreparedStatement lookupSeat = null;
29       PreparedStatement reserveSeat = null;
30
31       try
32       {
33          connection = DriverManager.getConnection(
34             DATABASE_URL, USERNAME, PASSWORD );
35          lookupSeat = connection.prepareStatement(
36             "SELECT \"number\" FROM \"seats\" WHERE (\"taken\" = 0) " +
37             "AND (\"location\" = ?) AND (\"class\" = ?)" );
38          lookupSeat.setString( 1, seatType );
39          lookupSeat.setString( 2, classType );
40          ResultSet resultSet = lookupSeat.executeQuery();
41
42          // if requested seat is available, reserve it
43          if ( resultSet.next() )
44          {
45             int seat = resultSet.getInt( 1 );
46             reserveSeat = connection.prepareStatement(
47                "UPDATE \"seats\" SET \"taken\"=1 WHERE \"number\"=?" );
48             reserveSeat.setInt( 1, seat );
49             reserveSeat.executeUpdate();
50             return true;
51          } // end if
52
```

Fig. 31.20 | Airline reservation web service. (Part 1 of 2.)

```
53             return false;
54          } // end try
55          catch ( SQLException e )
56          {
57             e.printStackTrace();
58             return false;
59          } // end catch
60          catch ( Exception e )
61          {
62             e.printStackTrace();
63             return false;
64          } // end catch
65          finally
66          {
67             try
68             {
69                lookupSeat.close();
70                reserveSeat.close();
71                connection.close();
72             } // end try
73             catch ( Exception e )
74             {
75                e.printStackTrace();
76                return false;
77             } // end catch
78          } // end finally
79       } // end WebMethod reserve
80    } // end class Reservation
```

Fig. 31.20 | Airline reservation web service. (Part 2 of 2.)

Our database contains four columns—the seat number (i.e., 1–10), the seat type (i.e., `Window`, `Middle` or `Aisle`), the class type (i.e., `Economy` or `First`) and a column containing either `1` (true) or `0` (false) to indicate whether the seat is taken. Lines 34–39 retrieve the seat numbers of any available seats matching the requested seat and class type. This statement fills the `resultSet` with the results of the query

```
SELECT number
FROM seats
WHERE (taken = 0) AND (type = type) AND (class = class)
```

The parameters *type* and *class* in the query are replaced with values of method `reserve`'s `seatType` and `classType` parameters.

If `resultSet` is not empty (i.e., at least one seat is available that matches the selected criteria), the condition in line 42 is `true` and the web service reserves the first matching seat number. Recall that `ResultSet` method `next` returns `true` if a nonempty row exists, and positions the cursor on that row. We obtain the seat number (line 44) by accessing `resultSet`'s first column (i.e., `resultSet.getInt(1)`—the first column in the row). Then lines 45–48 configure a `PreparedStatement` and execute the SQL:

```
UPDATE seats
SET taken = 1
WHERE (number = number)
```

which marks the seat as taken in the database. The parameter *number* is replaced with the value of `seat`. Method `reserve` returns `true` (line 49) to indicate that the reservation was successful. If there are no matching seats, or if an exception occurred, method `reserve` returns `false` (lines 52, 57, 62 and 75) to indicate that no seats matched the user's request.

31.10.2 Creating a Web Application to Interact with the `Reservation` Service

This section presents a `ReservationClient` JSF web application that consumes the `Reservation` web service. The application allows users to select `"Aisle"`, `"Middle"` or `"Window"` seats in `"Economy"` or `"First"` class, then submit their requests to the web service. If the database request is not successful, the application instructs the user to modify the request and try again. The application presented here was built using the techniques presented in Chapters 29–30. We assume that you've already read those chapters and thus know how to build a Facelets page and a corresponding JavaBean.

`index.xhtml`

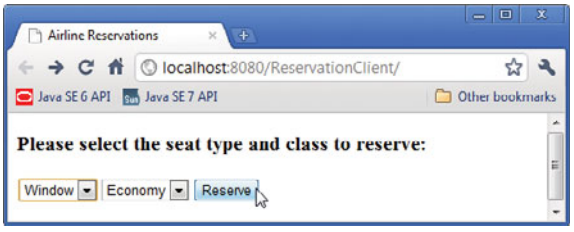
`index.xhtml` (Fig. 31.21) defines two `h:selectOneMenus` and an `h:commandButton`. The `h:selectOneMenu` at lines 16–20) displays all the seat types from which users can select. The one at lines 21–24) provides choices for the class type. The values of these are stored in the `seatType` and `classType` properties of the `reservationBean` (Fig. 31.22). Users click the **Reserve** button (lines 25–26) to submit requests after making selections from the `h:selectOneMenus`. Clicking the button calls the `reservationBean`'s `reserveSeat` method. The page displays the result of each attempt to reserve a seat in line 28.

```
1  <?xml version='1.0' encoding='UTF-8' ?>
2
3  <!-- Fig. 31.21: index.xhtml -->
4  <!-- Facelets page that allows a user to select a seat -->
5  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
6     "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
7  <html xmlns="http://www.w3.org/1999/xhtml"
8     xmlns:h="http://java.sun.com/jsf/html"
9     xmlns:f="http://java.sun.com/jsf/core">
10    <h:head>
11       <title>Airline Reservations</title>
12    </h:head>
13    <h:body>
14       <h:form>
15          <h3>Please select the seat type and class to reserve:</h3>
16          <h:selectOneMenu value="#{reservationBean.seatType}">
17             <f:selectItem itemValue="Aisle" itemLabel="Aisle" />
18             <f:selectItem itemValue="Middle" itemLabel="Middle" />
19             <f:selectItem itemValue="Window" itemLabel="Window" />
20          </h:selectOneMenu>
21          <h:selectOneMenu value="#{reservationBean.classType}">
22             <f:selectItem itemValue="Economy" itemLabel="Economy" />
23             <f:selectItem itemValue="First" itemLabel="First" />
24          </h:selectOneMenu>
```

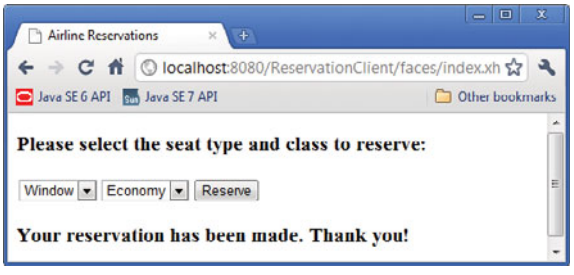
Fig. 31.21 | Facelets page that allows a user to select a seat. (Part 1 of 2.)

```
25            <h:commandButton value="Reserve"
26               action="#{reservationBean.reserveSeat}"/>
27         </h:form>
28         <h3>#{reservationBean.result}</h3>
29      </h:body>
30   </html>
```

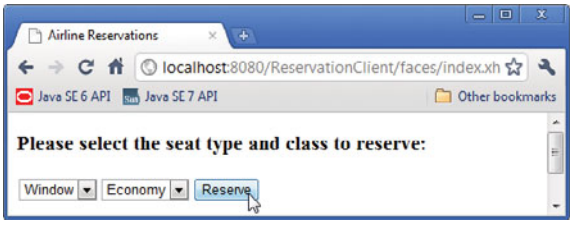
a) Selecting a seat



b) Seat reserved successfully



c) Attempting to reserve another window seat in economy when there are no such seats available



d) No seats match the requested seat type and class

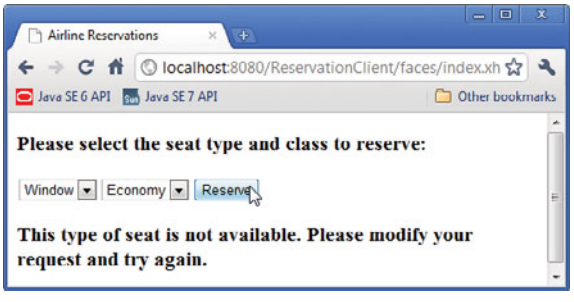


Fig. 31.21 | Facelets page that allows a user to select a seat. (Part 2 of 2.)

ReservationBean.java

Class `ReservationBean` (Fig. 31.22) defines the `seatType`, `classType` and `result` properties and the `reserveSeat` method that are used in the `index.xhtml` page. When the user clicks the **Reserve** button in `index.xhtml`, method `reserveSeat` (lines 57–74) executes. Lines 61–62 use the service endpoint interface object (created in lines 22–23) to invoke

the web service's `reserve` method, passing the selected seat type and class type as arguments. If `reserve` returns `true`, line 65 sets `result` to a message thanking the user for making a reservation; otherwise, lines 67–68 set `result` to a message notifying the user that the requested seat type is not available and instructing the user to try again.

```
1  // Fig. 31.22: ReservationBean.java
2  // Bean for seat reservation client.
3  package reservationclient;
4
5  import com.deitel.reservation.Reservation;
6  import com.deitel.reservation.ReservationService;
7  import javax.faces.bean.ManagedBean;
8
9  @ManagedBean( name = "reservationBean" )
10 public class ReservationBean
11 {
12    // references the service endpoint interface object (i.e., the proxy)
13    private Reservation reservationServiceProxy; // reference to proxy
14    private String seatType; // type of seat to reserve
15    private String classType; // class of seat to reserve
16    private String result; // result of reservation attempt
17
18    // no-argument constructor
19    public ReservationBean()
20    {
21       // get service endpoint interface
22       ReservationService reservationService = new ReservationService();
23       reservationServiceProxy = reservationService.getReservationPort();
24    } // end constructor
25
26    // return classType
27    public String getClassType()
28    {
29       return classType;
30    } // end method getClassType
31
32    // set classType
33    public void setClassType( String classType )
34    {
35       this.classType = classType;
36    } // end method setClassType
37
38    // return seatType
39    public String getSeatType()
40    {
41       return seatType;
42    } // end method getSeatType
43
44    // set seatType
45    public void setSeatType( String seatType )
46    {
```

Fig. 31.22 | Page bean for seat reservation client. (Part 1 of 2.)

```
47          this.seatType = seatType;
48       } // end method setSeatType
49
50       // return result
51       public String getResult()
52       {
53          return result;
54       } // end method getResult
55
56       // invoke the web service when the user clicks Reserve button
57       public void reserveSeat()
58       {
59          try
60          {
61             boolean reserved = reservationServiceProxy.reserve(
62                getSeatType(), getClassType() );
63
64             if ( reserved )
65                result = "Your reservation has been made. Thank you!";
66             else
67                result = "This type of seat is not available. " +
68                   "Please modify your request and try again.";
69          } // end try
70          catch ( Exception e )
71          {
72             e.printStackTrace();
73          } // end catch
74       } // end method reserveSeat
75    } // end class ReservationBean
```

Fig. 31.22 | Page bean for seat reservation client. (Part 2 of 2.)

31.11 Equation Generator: Returning User-Defined Types

Most of the web services we've demonstrated received and returned primitive-type instances. It's also possible to process instances of class types in a web service. These types can be passed to or returned from web service methods.

This section presents a RESTful `EquationGenerator` web service that generates random arithmetic equations of type `Equation`. The client is a math-tutoring application that accepts information about the mathematical question that the user wishes to attempt (addition, subtraction or multiplication) and the skill level of the user (1 specifies equations using numbers from 1 through 9, 2 specifies equations involving numbers from 10 through 99, and 3 specifies equations containing numbers from 100 through 999). The web service then generates an equation consisting of random numbers in the proper range. The client application receives the `Equation` and displays the sample question to the user.

Defining Class `Equation`

We define class `Equation` in Fig. 31.23. All the programs in this section have a copy of this class in their corresponding package. Except for the package name, the class is identical in each project, so we show it only once. Like the `TextMessage` class used earlier, the server-

side and client-side copies of class `Equation` are unrelated to each other. The only requirement for *serialization* and *deserialization* to work with the JAXB and Gson classes is that class `Equation` must have the same `public` properties on both the server and the client. Such properties can be public instance variables or private instance variables that have corresponding *set* and *get* methods.

```
1  // Fig. 31.23: Equation.java
2  // Equation class that contains information about an equation.
3  package com.deitel.equationgeneratorxml;
4
5  public class Equation
6  {
7     private int leftOperand;
8     private int rightOperand;
9     private int result;
10    private String operationType;
11
12    // required no-argument constructor
13    public Equation()
14    {
15       this( 0, 0, "add" );
16    } // end no-argument constructor
17
18    // constructor that receives the operands and operation type
19    public Equation( int leftValue, int rightValue, String type )
20    {
21       leftOperand = leftValue;
22       rightOperand = rightValue;
23
24       // determine result
25       if ( type.equals( "add" ) ) // addition
26       {
27          result = leftOperand + rightOperand;
28          operationType = "+";
29       } // end if
30       else if ( type.equals( "subtract" ) ) // subtraction
31       {
32          result = leftOperand - rightOperand;
33          operationType = "-";
34       } // end if
35       else // multiplication
36       {
37          result = leftOperand * rightOperand;
38          operationType = "*";
39       } // end else
40    } // end three argument constructor
41
42    // gets the leftOperand
43    public int getLeftOperand()
44    {
45       return leftOperand;
46    } // end method getLeftOperand
```

Fig. 31.23 | `Equation` class that contains information about an equation. (Part 1 of 3.)

```
47
48      // required setter
49      public void setLeftOperand( int value )
50      {
51         leftOperand = value;
52      } // end method setLeftOperand
53
54      // gets the rightOperand
55      public int getRightOperand()
56      {
57         return rightOperand;
58      } // end method getRightOperand
59
60      // required setter
61      public void setRightOperand( int value )
62      {
63         rightOperand = value;
64      } // end method setRightOperand
65
66      // gets the resultValue
67      public int getResult()
68      {
69         return result;
70      } // end method getResult
71
72      // required setter
73      public void setResult( int value )
74      {
75         result = value;
76      } // end method setResult
77
78      // gets the operationType
79      public String getOperationType()
80      {
81         return operationType;
82      } // end method getOperationType
83
84      // required setter
85      public void setOperationType( String value )
86      {
87         operationType = value;
88      } // end method setOperationType
89
90      // returns the left hand side of the equation as a String
91      public String getLeftHandSide()
92      {
93         return leftOperand + " " + operationType + " " + rightOperand;
94      } // end method getLeftHandSide
95
96      // returns the right hand side of the equation as a String
97      public String getRightHandSide()
98      {
```

Fig. 31.23 | `Equation` class that contains information about an equation. (Part 2 of 3.)

```
99          return "" + result;
100      } // end method getRightHandSide
101
102      // returns a String representation of an Equation
103      public String toString()
104      {
105         return getLeftHandSide() + " = " + getRightHandSide();
106      } // end method toString
107   } // end class Equation
```

Fig. 31.23 | `Equation` class that contains information about an equation. (Part 3 of 3.)

Lines 19–40 define a constructor that takes two `int`s representing the left and right operands, and a `String` representing the arithmetic operation. The constructor stores this information, then calculates the result. The parameterless constructor (lines 13–16) calls the three-argument constructor (lines 19–40) and passes default values.

Class `Equation` defines *get* and *set* methods for instance variables `leftOperand` (lines 43–52), `rightOperand` (lines 55–64), `result` (line 67–76) and `operationType` (lines 79–88). It also provides *get* methods for the left-hand and right-hand sides of the equation and a `toString` method that returns the entire equation as a `String`. An instance variable can be serialized only if it has both a *get* and a *set* method. Because the different sides of the equation and the result of `toString` can be generated from the other instance variables, there's no need to send them across the wire. The client in this case study does not use the `getRightHandSide` method, but we included it in case future clients choose to use it.

31.11.1 Creating the `EquationGeneratorXML` Web Service

Figure 31.24 presents the `EquationGeneratorXML` web service's class for creating randomly generated `Equations`. Method `getXml` (lines 19–38) takes two parameters—a `String` representing the mathematical operation (`"add"`, `"subtract"` or `"multiply"`) and an `int` representing the difficulty level. JAX-RS automatically converts the arguments to the correct type and will return a "not found" error to the client if the argument cannot be converted from a `String` to the destination type. Supported types for conversion include integer types, floating-point types, `boolean` and the corresponding *type-wrapper classes*.

```
1  // Fig. 31.24: EquationGeneratorXMLResource.java
2  // RESTful equation generator that returns XML.
3  package com.deitel.equationgeneratorxml;
4
5  import java.io.StringWriter;
6  import java.util.Random;
7  import javax.ws.rs.PathParam;
8  import javax.ws.rs.Path;
9  import javax.ws.rs.GET;
10 import javax.ws.rs.Produces;
11 import javax.xml.bind.JAXB; // utility class for common JAXB operations
12
```

Fig. 31.24 | RESTful equation generator that returns XML. (Part 1 of 2.)

```
13  @Path( "equation" )
14  public class EquationGeneratorXMLResource
15  {
16     private static Random randomObject = new Random();
17
18     // retrieve an equation formatted as XML
19     @GET
20     @Path( "{operation}/{level}" )
21     @Produces( "application/xml" )
22     public String getXml( @PathParam( "operation" ) String operation,
23        @PathParam( "level" ) int level )
24     {
25        // compute minimum and maximum values for the numbers
26        int minimum = ( int ) Math.pow( 10, level - 1 );
27        int maximum = ( int ) Math.pow( 10, level );
28
29        // create the numbers on the left-hand side of the equation
30        int first = randomObject.nextInt( maximum - minimum ) + minimum;
31        int second = randomObject.nextInt( maximum - minimum ) + minimum;
32
33        // create Equation object and marshal it into XML
34        Equation equation = new Equation( first, second, operation );
35        StringWriter writer = new StringWriter(); // XML output here
36        JAXB.marshal( equation, writer ); // write Equation to StringWriter
37        return writer.toString(); // return XML string
38     } // end method getXml
39  } // end class EquationGeneratorXMLResource
```

Fig. 31.24 | RESTful equation generator that returns XML. (Part 2 of 2.)

The `getXml` method first determines the minimum (inclusive) and maximum (exclusive) values for the numbers in the equation it will return (lines 26–27). It then uses a static member of the `Random` class (line 16) to generate two random numbers in that range (lines 30–31). Line 34 creates an `Equation` object, passing these two numbers and the requested operation to the constructor. The `getXml` method then uses JAXB to convert the `Equation` object to XML (line 36), which is output to the `StringWriter` created on line 35. Finally, it retrieves the data that was written to the `StringWriter` and returns it to the client. [*Note:* We'll reimplement this web service with JSON in Section 31.11.3.]

31.11.2 Consuming the EquationGeneratorXML Web Service

The `EquationGeneratorXMLClient` application (Fig. 31.25) retrieves an XML-formatted `Equation` object from the `EquationGeneratorXML` web service. The application then displays the `Equation`'s left-hand side and waits for user to submit an answer.

```
1  // Fig. 31.25: EquationGeneratorXMLClientJFrame.java
2  // Math-tutoring program using REST and XML to generate equations.
3  package com.deitel.equationgeneratorxmlclient;
4
5  import javax.swing.JOptionPane;
```

Fig. 31.25 | Math-tutoring program using REST and XML to generate equations. (Part 1 of 4.)

```
6  import javax.xml.bind.JAXB; // utility class for common JAXB operations
7
8  public class EquationGeneratorXMLClientJFrame extends javax.swing.JFrame
9  {
10    private String operation = "add"; // operation user is tested on
11    private int difficulty = 1; // 1, 2, or 3 digits in each number
12    private int answer; // correct answer to the question
13
14    // no-argument constructor
15    public EquationGeneratorXMLClientJFrame()
16    {
17       initComponents();
18    } // end no-argument constructor
19
20    // The initComponents method is autogenerated by NetBeans and is called
21    // from the constructor to initialize the GUI. This method is not shown
22    // here to save space. Open EquationGeneratorXMLClientJFrame.java in
23    // this example's folder to view the complete generated code.
24
143   // determine if the user answered correctly
144   private void checkAnswerJButtonActionPerformed(
145      java.awt.event.ActionEvent evt)
146   {
147      if ( answerJTextField.getText().equals( "" ) )
148      {
149         JOptionPane.showMessageDialog(
150            this, "Please enter your answer." );
151      } // end if
152
153      int userAnswer = Integer.parseInt( answerJTextField.getText() );
154
155      if ( userAnswer == answer )
156      {
157         equationJLabel.setText( "" ); // clear label
158         answerJTextField.setText( "" ); // clear text field
159         checkAnswerJButton.setEnabled( false );
160         JOptionPane.showMessageDialog( this, "Correct! Good Job!",
161            "Correct", JOptionPane.PLAIN_MESSAGE );
162      } // end if
163      else
164      {
165         JOptionPane.showMessageDialog( this, "Incorrect. Try again.",
166            "Incorrect", JOptionPane.PLAIN_MESSAGE );
167      } // end else
168   } // end method checkAnswerJButtonActionPerformed
169
170   // retrieve equation from web service and display left side to user
171   private void generateJButtonActionPerformed(
172      java.awt.event.ActionEvent evt)
173   {
174      try
175      {
```

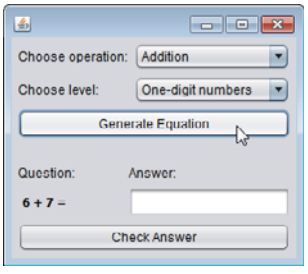
Fig. 31.25 | Math-tutoring program using REST and XML to generate equations. (Part 2 of 4.)

```
176          String url = String.format( "http://localhost:8080/" +
177             "EquationGeneratorXML/resources/equation/%s/%d",
178             operation, difficulty );
179
180          // convert XML back to an Equation object
181          Equation equation = JAXB.unmarshal( url, Equation.class );
182
183          answer = equation.getResult();
184          equationJLabel.setText( equation.getLeftHandSide() + " =" );
185          checkAnswerJButton.setEnabled( true );
186       } // end try
187       catch ( Exception exception )
188       {
189          exception.printStackTrace();
190       } // end catch
191    } // end method generateJButtonActionPerformed
192
193    // obtains the mathematical operation selected by the user
194    private void operationJComboBoxItemStateChanged(
195       java.awt.event.ItemEvent evt)
196    {
197       String item = ( String ) operationJComboBox.getSelectedItem();
198
199       if ( item.equals( "Addition" ) )
200          operation = "add"; // user selected addition
201       else if ( item.equals( "Subtraction" ) )
202          operation = "subtract"; // user selected subtraction
203       else
204          operation = "multiply"; // user selected multiplication
205    } // end method operationJComboBoxItemStateChanged
206
207    // obtains the difficulty level selected by the user
208    private void levelJComboBoxItemStateChanged(
209       java.awt.event.ItemEvent evt)
210    {
211       // indices start at 0, so add 1 to get the difficulty level
212       difficulty = levelJComboBox.getSelectedIndex() + 1;
213    } // end method levelJComboBoxItemStateChanged
214
215    // main method begins execution
216    public static void main(String args[])
217    {
218       java.awt.EventQueue.invokeLater(
219          new Runnable()
220          {
221             public void run()
222             {
223                new EquationGeneratorXMLClientJFrame().setVisible( true );
224             } // end method run
225          } // end anonymous inner class
226       ); // end call to java.awt.EventQueue.invokeLater
227    } // end main
228
```

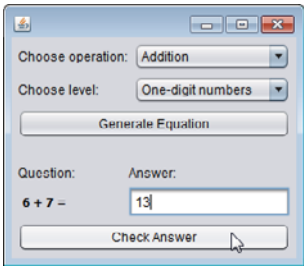
Fig. 31.25 | Math-tutoring program using REST and XML to generate equations. (Part 3 of 4.)

```
229     // Variables declaration - do not modify
230     private javax.swing.JLabel answerJLabel;
231     private javax.swing.JTextField answerJTextField;
232     private javax.swing.JButton checkAnswerJButton;
233     private javax.swing.JLabel equationJLabel;
234     private javax.swing.JButton generateJButton;
235     private javax.swing.JComboBox levelJComboBox;
236     private javax.swing.JLabel levelJLabel;
237     private javax.swing.JComboBox operationJComboBox;
238     private javax.swing.JLabel operationJLabel;
239     private javax.swing.JLabel questionJLabel;
240     // End of variables declaration
241  } // end class EquationGeneratorXMLClientJFrame
```

a) Generating a simple equation.



b) Sumbitting the answer.



c) Dialog indicating correct answer.

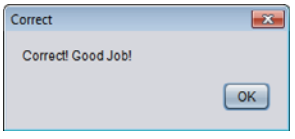


Fig. 31.25 | Math-tutoring program using REST and XML to generate equations. (Part 4 of 4.)

The default setting for the difficulty level is `1`, but the user can change this by choosing a level from the **Choose level** `JComboBox`. Changing the selected value invokes the `levelJComboBoxItemStateChanged` event handler (lines 208–213), which sets the `difficulty` instance variable to the level selected by the user. Although the default setting for the question type is **Addition**, the user also can change this by choosing from the **Choose operation** `JComboBox`. This invokes the `operationJComboBoxItemStateChanged` event handler in lines 194–205, which assigns to instance variable `operation` the `String` corresponding to the user's selection.

The event handler for `generateJButton` (lines 171–191) constructs the URL to invoke the web service, then passes this URL to the `unmarshal` method, along with an instance of `Class<Equation>`, so that JAXB can convert the XML into an `Equation` object (line 181). Once the XML has been converted back into an `Equation`, lines 183–184 retrieve the correct answer and display the left-hand side of the equation. The **Check Answer** button is then enabled (line 185), and the user must solve the problem and enter the answer.

When the user enters a value and clicks **Check Answer**, the `checkAnswerJButtonActionPerformed` event handler (lines 144–168) retrieves the user's answer from the dialog box (line 153) and compares it to the correct answer that was stored earlier (line 155). If they match, lines 157–161 reset the GUI elements so the user can generate another equation and tell the user that the answer was correct. If they do not match, a message box asking the user to try again is displayed (lines 165–166).

31.11.3 Creating the EquationGeneratorJSON Web Service

As you saw in Section 31.8, RESTful web services can return data formatted as JSON as well. Figure 31.26 is a reimplementation of the `EquationGeneratorXML` service that returns an `Equation` in JSON format.

```
1  // Fig. 31.26: EquationGeneratorJSONResource.java
2  // RESTful equation generator that returns JSON.
3  package com.deitel.equationgeneratorjson;
4
5  import com.google.gson.Gson; // converts POJO to JSON and back again
6  import java.util.Random;
7  import javax.ws.rs.GET;
8  import javax.ws.rs.Path;
9  import javax.ws.rs.PathParam;
10 import javax.ws.rs.Produces;
11
12 @Path( "equation" )
13 public class EquationGeneratorJSONResource
14 {
15    static Random randomObject = new Random(); // random number generator
16
17    // retrieve an equation formatted as JSON
18    @GET
19    @Path( "{operation}/{level}" )
20    @Produces( "application/json" )
21    public String getJson( @PathParam( "operation" ) String operation,
22       @PathParam( "level" ) int level )
23    {
24       // compute minimum and maximum values for the numbers
25       int minimum = ( int ) Math.pow( 10, level - 1 );
26       int maximum = ( int ) Math.pow( 10, level );
27
28       // create the numbers on the left-hand side of the equation
29       int first = randomObject.nextInt( maximum - minimum ) + minimum;
30       int second = randomObject.nextInt( maximum - minimum ) + minimum;
31
32       // create Equation object and return result
33       Equation equation = new Equation( first, second, operation );
34       return new Gson().toJson( equation ); // convert to JSON and return
35    } // end method getJson
36 } // end class EquationGeneratorJSONResource
```

Fig. 31.26 | RESTful equation generator that returns JSON.

The logic implemented here is the same as the XML version except for the last line (line 34), which uses Gson to convert the `Equation` object into JSON instead of using JAXB to convert it into XML. The `@Produces` annotation (line 20) has also changed to reflect the JSON data format.

31.11.4 Consuming the EquationGeneratorJSON Web Service

The program in Fig. 31.27 consumes the `EquationGeneratorJSON` service and performs the same function as `EquationGeneratorXMLClient`—the only difference is in how the

`Equation` object is retrieved from the web service. Lines 181–183 construct the URL that is used to invoke the `EquationGeneratorJSON` service. As in the `WelcomeRESTJSONClient` example, we use the `URL` class and an `InputStreamReader` to invoke the web service and read the response (lines 186–187). The retrieved JSON is *deserialized* using Gson (line 191) and converted back into an `Equation` object. As before, we use the `getResult` method (line 194) of the deserialized object to obtain the answer and the `getLeftHandSide` method (line 195) to display the left side of the equation.

```
1   // Fig. 31.27: EquationGeneratorJSONClientJFrame.java
2   // Math-tutoring program using REST and JSON to generate equations.
3   package com.deitel.equationgeneratorjsonclient;
4
5   import com.google.gson.Gson; // converts POJO to JSON and back again
6   import java.io.InputStreamReader;
7   import java.net.URL;
8   import javax.swing.JOptionPane;
9
10  public class EquationGeneratorJSONClientJFrame extends javax.swing.JFrame
11  {
12     private String operation = "add"; // operation user is tested on
13     private int difficulty = 1; // 1, 2, or 3 digits in each number
14     private int answer; // correct answer to the question
15
16     // no-argument constructor
17     public EquationGeneratorJSONClientJFrame()
18     {
19        initComponents();
20     } // end no-argument constructor
21
22     // The initComponents method is autogenerated by NetBeans and is called
23     // from the constructor to initialize the GUI. This method is not shown
24     // here to save space. Open EquationGeneratorJSONClientJFrame.java in
25     // this example's folder to view the complete generated code.
26
147    // determine if the user answered correctly
148    private void checkAnswerJButtonActionPerformed(
149       java.awt.event.ActionEvent evt)
150    {
151       if ( answerJTextField.getText().equals( "" ) )
152       {
153          JOptionPane.showMessageDialog(
154             this, "Please enter your answer." );
155       } // end if
156
157       int userAnswer = Integer.parseInt( answerJTextField.getText() );
158
159       if ( userAnswer == answer )
160       {
161          equationJLabel.setText( "" ); // clear label
162          answerJTextField.setText( "" ); // clear text field
163          checkAnswerJButton.setEnabled( false );
```

Fig. 31.27 | Math-tutoring program using REST and JSON to generate equations. (Part 1 of 3.)

```
164            JOptionPane.showMessageDialog( this, "Correct! Good Job!",
165               "Correct", JOptionPane.PLAIN_MESSAGE );
166         } // end if
167         else
168         {
169            JOptionPane.showMessageDialog( this, "Incorrect. Try again.",
170               "Incorrect", JOptionPane.PLAIN_MESSAGE );
171         } // end else
172      } // end method checkAnswerJButtonActionPerformed
173
174      // retrieve equation from web service and display left side to user
175      private void generateJButtonActionPerformed(
176         java.awt.event.ActionEvent evt)
177      {
178         try
179         {
180            // URL of the EquationGeneratorJSON service, with parameters
181            String url = String.format( "http://localhost:8080/" +
182               "EquationGeneratorJSON/resources/equation/%s/%d",
183               operation, difficulty );
184
185            // open URL and create a Reader to read the data
186            InputStreamReader reader =
187               new InputStreamReader( new URL( url ).openStream() );
188
189            // convert the JSON back into an Equation object
190            Equation equation =
191               new Gson().fromJson( reader, Equation.class );
192
193            // update the internal state and GUI to reflect the equation
194            answer = equation.getResult();
195            equationJLabel.setText( equation.getLeftHandSide() + " =" );
196            checkAnswerJButton.setEnabled( true );
197         } // end try
198         catch ( Exception exception )
199         {
200            exception.printStackTrace();
201         } // end catch
202      } // end method generateJButtonActionPerformed
203
204      // obtains the mathematical operation selected by the user
205      private void operationJComboBoxItemStateChanged(
206         java.awt.event.ItemEvent evt)
207      {
208         String item = ( String ) operationJComboBox.getSelectedItem();
209
210         if ( item.equals( "Addition" ) )
211            operation = "add"; // user selected addition
212         else if ( item.equals( "Subtraction" ) )
213            operation = "subtract"; // user selected subtraction
214         else
215            operation = "multiply"; // user selected multiplication
216      } // end method operationJComboBoxItemStateChanged
```

Fig. 31.27 | Math-tutoring program using REST and JSON to generate equations. (Part 2 of 3.)

```
217
218    // obtains the difficulty level selected by the user
219    private void levelJComboBoxItemStateChanged(
220       java.awt.event.ItemEvent evt)
221    {
222       // indices start at 0, so add 1 to get the difficulty level
223       difficulty = levelJComboBox.getSelectedIndex() + 1;
224    } // end method levelJComboBoxItemStateChanged
225
226    // main method begins execution
227    public static void main( String args[] )
228    {
229       java.awt.EventQueue.invokeLater(
230          new Runnable()
231          {
232             public void run()
233             {
234                new EquationGeneratorJSONClientJFrame().setVisible( true );
235             } // end method run
236          } // end anonymous inner class
237       ); // end call to java.awt.EventQueue.invokeLater
238    } // end main
239
240    // Variables declaration - do not modify
241    private javax.swing.JLabel answerJLabel;
242    private javax.swing.JTextField answerJTextField;
243    private javax.swing.JButton checkAnswerJButton;
244    private javax.swing.JLabel equationJLabel;
245    private javax.swing.JButton generateJButton;
246    private javax.swing.JComboBox levelJComboBox;
247    private javax.swing.JLabel levelJLabel;
248    private javax.swing.JComboBox operationJComboBox;
249    private javax.swing.JLabel operationJLabel;
250    private javax.swing.JLabel questionJLabel;
251    // End of variables declaration
252 } // end class EquationGeneratorJSONClientJFrame
```

Fig. 31.27 | Math-tutoring program using REST and JSON to generate equations. (Part 3 of 3.)

31.12 Wrap-Up

This chapter introduced web services—a set of technologies for building distributed systems in which system components communicate with one another over networks. In particular, we presented JAX-WS SOAP-based web services and JAX-RS REST-based web services. You learned that a web service is a class that allows client software to call the web service's methods remotely via common data formats and protocols, such as XML, JSON, HTTP, SOAP and REST. We also benefits of distributed computing with web services.

We explained how NetBeans and the JAX-WS and JAX-RS APIs facilitate publishing and consuming web services. You learned how to define web services and methods using both SOAP protocol and REST architecture, and how to return data in both XML and JSON formats. You consumed SOAP-based web services using proxy classes to call the web service's methods. You also consumed REST-based web services by using class `URL` to

invoke the services and open `InputStreams` from which the clients could read the services' responses. You learned how to define web services and web methods, as well as how to consume them both from Java desktop applications and from web applications. After explaining the mechanics of web services through our `Welcome` examples, we demonstrated more sophisticated web services that use session tracking, database access and user-defined types. We also explained XML and JSON serialization and showed how to retrieve objects of user-defined types from web services.

Summary

Section 31.1 Introduction

- A web service (p. 1300) is a software component stored on one computer that can be accessed by an application (or other software component) on another computer over a network.
- Web services communicate using such technologies as XML, JSON and HTTP.
- JAX-WS (p. 1300) is based on the Simple Object Access Protocol (SOAP; p. 1300)—an XML-based protocol that allows web services and clients to communicate.
- JAX-RS (p. 1300) uses Representational State Transfer (REST; p. 1300)—a network architecture that uses the web's traditional request/response mechanisms such as `GET` and `POST` requests.
- Web services enable businesses to conduct transactions via standardized, widely available web services rather than relying on proprietary applications.
- Web services are platform and language independent, so companies can collaborate via web services without hardware, software and communications compatibility issues.
- NetBeans is one of the many tools that enable you to publish and/or consume web services.

Section 31.2 Web Service Basics

- The machine on which a web service resides is referred to as a web service host.
- A client application that accesses the web service sends a method call over a network to the web service host, which processes the call and returns a response over the network to the application.
- In Java, a web service is implemented as a class. The class that represents the web service resides on a server—it's not part of the client application.
- Making a web service available to receive client requests is known as publishing a web service (p. 1302); using a web service from a client application is known as consuming a web service (p. 1302).

Section 31.3 Simple Object Access Protocol (SOAP)

- SOAP is a platform-independent protocol that uses XML to make remote procedure calls, typically over HTTP. Each request and response is packaged in a SOAP message (p. 1302)—an XML message containing the information that a web service requires to process the message.
- SOAP messages are written in XML so that they're computer readable, human readable and platform independent.
- SOAP supports an extensive set of types—the primitive types, as well as `DateTime`, `XmlNode` and others. SOAP can also transmit arrays of these types.
- When a program invokes a method of a SOAP web service, the request and all relevant information are packaged in a SOAP message, enclosed in a SOAP envelope (p. 1302) and sent to the server on which the web service resides.

- When a web service receives a SOAP message, it parses the XML representing the message, then processes the message's contents. The message specifies the method that the client wishes to execute and the arguments the client passed to that method.
- After a web service parses a SOAP message, it calls the appropriate method with the specified arguments (if any) and sends the response back to the client in another SOAP message. The client parses the response to retrieve the method's result.

Section 31.4 Representational State Transfer (REST)

- Representational State Transfer (REST) refers to an architectural style for implementing web services. Such web services are often called RESTful web services (p. 1302). Though REST itself is not a standard, RESTful web services are implemented using web standards.
- Each operation in a RESTful web service is identified by a unique URL.
- REST can return data in many formats, including XML and JSON.

Section 31.5 JavaScript Object Notation (JSON)

- JavaScript Object Notation (JSON; p. 1303) is an alternative to XML for representing data.
- JSON is a text-based data-interchange format used to represent objects in JavaScript as collections of name/value pairs represented as `String`s.
- JSON is a simple format that makes objects easy to read, create and parse and allows programs to transmit data efficiently across the Internet, because it's much less verbose than XML.
- Each value in a JSON array can be a string, a number, a JSON object, `true`, `false` or `null`.

Section 31.6.1 Creating a Web Application Project and Adding a Web Service Class in NetBeans

- When you create a web service in NetBeans, you focus on the web service's logic and let the IDE handle the web service's infrastructure.
- To create a web service in NetBeans, you first create a **Web Application** project (p. 1303).

Section 31.6.2 Defining the `WelcomeSOAP` Web Service in NetBeans

- By default, each new web service class created with the JAX-WS APIs is a POJO (plain old Java object)—you do not need to extend a class or implement an interface to create a web service.
- When you deploy a web application containing a JAX-WS web service, the server creates the server-side artifacts that support the web service.
- The `@WebService` annotation (p. 1305) indicates that a class represents a web service. The optional `name` attribute (p. 1305) specifies the service endpoint interface (SEI; p. 1305) class's name. The optional `serviceName` attribute (p. 1305) specifies the name of the class that the client uses to obtain an SEI object.
- Methods that are tagged with the `@WebMethod` annotation (p. 1305) can be called remotely.
- The `@WebMethod` annotation's optional `operationName` attribute (p. 1305) specifies the method name that is exposed to the web service's clients.
- Web method parameters are annotated with the `@WebParam` annotation (p. 1306). The optional `name` attribute (p. 1306) indicates the parameter name that is exposed to the web service's clients.

Section 31.6.3 Publishing the `WelcomeSOAP` Web Service from NetBeans

- NetBeans handles all the details of building and deploying a web service for you. This includes creating the framework required to support the web service.

Section 31.6.4 Testing the `WelcomeSOAP` Web Service with GlassFish Application Server's `Tester` Web Page

- GlassFish can dynamically create a web page for testing a web service's methods from a web browser. To open the test page, expand the project's **Web Services** node in the NetBeans **Projects** tab, then right click the web service class name and select **Test Web Service**.
- A client can access a web service only when the application server is running. If NetBeans launches the application server for you, the server will shut down when you close NetBeans. To keep the application server up and running, you can launch it independently of NetBeans.

Section 31.6.5 Describing a Web Service with the Web Service Description Language (WSDL)

- To consume a web service, a client must know where to find it and must be provided with the web service's description.
- JAX-WS uses the Web Service Description Language (WSDL; p. 1309)—a standard XML vocabulary for describing web services in a platform-independent manner.
- The server generates a web service's WSDL dynamically for you, and client tools can parse the WSDL to help create the client-side proxy class that a client uses to access the web service.

Section 31.6.6 Creating a Client to Consume the `WelcomeSOAP` Web Service

- A web service reference (p. 1310) defines the service endpoint interface class so that a client can access the a service.
- An application that consumes a SOAP-based web service invokes methods on a service endpoint interface (SEI) object that interact with the web service on the client's behalf.
- The service endpoint interface object handles the details of passing method arguments to and receiving return values from the web service. This communication can occur over a local network, over the Internet or even with a web service on the same computer.
- NetBeans creates these service endpoint interface classes for you.
- When you add the web service reference, the IDE creates and compiles the client-side artifacts—the framework of Java code that supports the client-side service endpoint interface class. The service endpoint interface class uses the rest of the artifacts to interact with the web service.
- A web service reference is added by giving NetBeans the URL of the web service's WSDL file.

Section 31.6.7 Consuming the `WelcomeSOAP` Web Service

- To consume a JAX-WS web service, you must obtain an SEI object. You then invoke the web service's methods through the SEI object.

Section 31.7.1 Creating a REST-Based XML Web Service

- The **RESTful Web Services** plug-in for NetBeans provides various templates for creating RESTful web services, including ones that can interact with databases on the client's behalf.
- The `@Path` annotation (p. 1316) on a JAX-RS web service class indicates the URI for accessing the web service. This is appended to the web application project's URL to invoke the service. Methods of the class can also use the `@Path` annotation.
- Parts of the path specified in curly braces indicate parameters—they're placeholders for arguments that are passed to the web service as part of the path. The base path for the service is the project's `resources` directory.
- Arguments in a URL can be used as arguments to a web service method. To do so, you bind the parameters specified in the `@Path` specification to parameters of a web service method with the

`@PathParam` annotation (p. 1317). When the request is received, the server passes the argument(s) in the URL to the appropriate parameter(s) in the web service method.

- The `@GET` annotation (p. 1317) denotes that a method is accessed via an HTTP `GET` request. Similar annotations exist for HTTP `PUT`, `POST`, `DELETE` and `HEAD` requests.
- The `@Produces` annotation (p. 1317) denotes the content type returned to the client. It's possible to have multiple methods with the same HTTP method and path but different `@Produces` annotations, and JAX-RS will call the method matching the content type requested by the client.
- The `@Consumes` annotation (p. 1317) restricts the content type that a web service accepts from a `PUT` request.
- JAXB (Java Architecture for XML Binding; p. 1317) is a set of classes for converting POJOs to and from XML. Class `JAXB` (package `javax.xml.bind`) contains `static` methods for common operations.
- Class `JAXB`'s `static` method `marshal` (p. 1317) converts a Java object to XML format.
- GlassFish does not provide test pages for RESTful services, but NetBeans generates a test page that can be accessed by right clicking the project's node in the **Projects** tab and selecting **Test RESTful Web Services**.
- On the test page, select a method element in the left column. The right side of the page displays a form that allows you to choose the MIME type of the data and lets you enter the method's arguments. Click the **Test** button to invoke the web service and display the returned data.
- WADL (Web Application Description Language; p. 1318) has similar design goals to WSDL, but describes RESTful services instead of SOAP services.

Section 31.7.2 Consuming a REST-Based XML Web Service

- Clients of RESTful web services do not require web service references.
- The `JAXB` class has a `static` `unmarshal` method that takes as arguments a file name or URL as a `String`, and a `Class<T>` object indicating the Java class to which the XML will be converted.

Section 31.8 Publishing and Consuming REST-Based JSON Web Services

- JSON components—objects, arrays, strings, numbers—can be easily mapped to constructs in Java and other programming languages.
- There are many open-source JSON libraries for Java and other languages. The Gson library from `code.google.com/p/google-gson/` provides a simple way to convert POJOs to and from JSON.

Section 31.8.1 Creating a REST-Based JSON Web Service

- To add a JAR file as a library in NetBeans, right click your project's **Libraries** folder, select **Add JAR/Folder...**, locate the JAR file and click **Open**.
- For a web service method that returns JSON text, the argument to the `@Produces` attribute must be `"application/json"`.
- In JSON, all data must be encapsulated in a composite data type.
- Create a `Gson` object (from package `com.google.gson`) and call its `toJson` method to convert an object into its JSON `String` representation.

Section 31.8.2 Consuming a REST-Based JSON Web Service

- To read JSON data from a URL, create a `URL` object and call its `openStream` method (p. 1324). This invokes the web service and returns an `InputStream` from which the client can read the response. Wrap the `InputStream` in an `InputStreamReader` so it can be passed as the first argument to the `Gson` class's `fromJson` method (p. 1324).

Section 31.9 Session Tracking in a SOAP Web Service

- It can be beneficial for a web service to maintain client state information, thus eliminating the need to pass client information between the client and the web service multiple times. Storing session information also enables a web service to distinguish between clients.

Section 31.9.1 Creating a `Blackjack` Web Service

- In JAX-WS 2.2, to enable session tracking in a web service, you simply precede your web service class with the `@HttpSessionScope` annotation (p. 1327) from package `com.sun.xml.ws.developer.servlet`. To use this package you must add the JAX-WS 2.2 library to your project.
- Once a web service is annotated with `@HttpSessionScope`, the server automatically maintains a separate instance of the class for each client session.

Section 31.9.2 Consuming the `Blackjack` Web Service

- In the JAX-WS framework, the client must indicate whether it wants to allow the web service to maintain session information. To do this, first cast the proxy object to interface type `BindingProvider`. A `BindingProvider` enables the client to manipulate the request information that will be sent to the server. This information is stored in an object that implements interface `RequestContext`. The `BindingProvider` and `RequestContext` are part of the framework that is created by the IDE when you add a web service client to the application.
- Next, invoke the `BindingProvider`'s `getRequestContext` method to obtain the `RequestContext` object. Then call the `RequestContext`'s `put` method to set the property `BindingProvider.SESSION_MAINTAIN_PROPERTY` to `true`, which enables session tracking from the client side so that the web service knows which client is invoking the service's web methods.

Section 31.11 Equation Generator: Returning User-Defined Types

- It's also possible to process instances of class types in a web service. These types can be passed to or returned from web service methods.
- An instance variable can be serialized only if it's `public` or has both a *get* and a *set* method.
- Properties that can be generated from the values of other properties should not be serialized to prevent redundancy.
- JAX-RS automatically converts arguments from an `@Path` annotation to the correct type, and it will return a "not found" error to the client if the argument cannot be converted from the `String` passed as part of the URL to the destination type. Supported types for conversion include integer types, floating-point types, `boolean` and the corresponding type-wrapper classes.

Self-Review Exercises

31.1 State whether each of the following is *true* or *false*. If *false*, explain why.

a) All methods of a web service class can be invoked by clients of that web service.
b) When consuming a web service in a client application created in NetBeans, you must create the proxy class that enables the client to communicate with the web service.
c) A proxy class communicating with a web service normally uses SOAP to send and receive messages.
d) Session tracking is automatically enabled in a client of a web service.
e) Web methods cannot be declared `static`.
f) A user-defined type used in a web service must define both *get* and *set* methods for any property that will be serialized.
g) Operations in a REST web service are defined by their own unique URLs.
h) A SOAP-based web service can return data in JSON format.

31.2 Fill in the blanks for each of the following statements:

a) A key difference between SOAP and REST is that SOAP messages have data wrapped in a(n) _____.

b) A web service in Java is a(n) _____—it does not need to implement any interfaces or extend any classes.

c) Web service requests are typically transported over the Internet via the _____ protocol.

d) To set the exposed name of a web method, use the _____ element of the `@WebMethod` annotation.

e) _____ transforms an object into a format that can be sent between a web service and a client.

f) To return data in JSON format from a method of a REST-based web service, the `@Produces` annotation is set to _____.

g) To return data in XML format from a method of a REST-based web service, the `@Produces` annotation is set to _____.

Answers to Self-Review Exercises

31.1 a) False. Only methods declared with the `@WebMethod` annotation can be invoked by a web service's clients. b) False. The proxy class is created by NetBeans when you add a web service client to the application. c) True. d) False. In the JAX-WS framework, the client must indicate whether it wants to allow the web service to maintain session information. First, you must cast the proxy object to interface type `BindingProvider`, then use the `BindingProvider`'s `getRequestContext` method to obtain the `RequestContext` object. Finally, you must use the `RequestContext`'s `put` method to set the property `BindingProvider.SESSION_MAINTAIN_PROPERTY` to `true`. e) True. f) True. g) True. h) False. A SOAP web service implicitly returns data in XML format.

31.2 a) SOAP message or SOAP envelope. b) POJO (plain old Java object) c) HTTP. d) `operationName`. e) serialization. f) `"application/json"`. g) `"application/xml"`.

Exercises

31.3 ***(Phone Book Web Service)*** Create a RESTful web service that stores phone book entries in the database `PhoneBookDB` and a web client application that consumes this service. The web service should output XML. Use the steps in Section 30.2.1 to create the `PhoneBook` database and a data source name for accessing it. The database contains one table—`PhoneBook`—with three columns—`LastName`, `FirstName` and `PhoneNumber`. The `LastName` and `FirstName` columns store up to 30 characters. The `PhoneNumber` column supports phone numbers of the form `(800) 555-1212` that contain 14 characters. Use the `PhoneBookDB.sql` script provided in the examples folder to create the `PhoneBook` table.

Give the client user the capability to enter a new contact (web method `addEntry`) and to find contacts by last name (web method `getEntries`). Pass only `String`s as arguments to the web service. The `getEntries` web method should return an array of `String`s that contains the matching phone book entries. Each `String` in the array should consist of the last name, first name and phone number for one phone book entry. These values should be separated by commas.

The `SELECT` query that will find a `PhoneBook` entry by last name should be:

```
SELECT LastName, FirstName, PhoneNumber
FROM PhoneBook
WHERE (LastName = LastName)
```

The `INSERT` statement that inserts a new entry into the `PhoneBook` database should be:

```
INSERT INTO PhoneBook (LastName, FirstName, PhoneNumber)
VALUES (LastName, FirstName, PhoneNumber)
```

31.4 *(Phone Book Web Service Modification)* Modify Exercise 31.3 so that it uses a class named `PhoneBookEntry` to represent a row in the database. The web service should return objects of type `PhoneBookEntry` in XML format for the `getEntries` method, and the client application should use the `JAXB` method `unmarshal` to retrieve the `PhoneBookEntry` objects.

31.5 *(Phone-Book Web Service with JSON)* Modify Exercise 31.4 so that the `PhoneBookEntry` class is passed to and from the web service as a JSON object. Use serialization to convert the JSON object into an object of type `PhoneBookEntry`.

31.6 *(`Blackjack` Web Service Modification)* Modify the `Blackjack` web service example in Section 31.9 to include class `Card`. Modify web method `dealCard` so that it returns an object of type `Card` and modify web method `getHandValue` so that it receives an array of `Card` objects from the client. Also modify the client application to keep track of what cards have been dealt by using `ArrayLists` of `Card` objects. The proxy class created by NetBeans will treat a web method's array parameter as a `List`, so you can pass these `ArrayLists` of `Card` objects directly to the `getHandValue` method. Your `Card` class should include *set* and *get* methods for the face and suit of the card.

31.7 *(Project: Airline Reservation Web-Service Modification)* Modify the airline reservation web service in Section 31.10 so that it contains two separate methods—one that allows users to view all available seats, and another that allows users to reserve a particular seat that is currently available. Use an object of type `Ticket` to pass information to and from the web service. The web service must be able to handle cases in which two users view available seats, one reserves a seat and the second user tries to reserve the same seat, not knowing that it's now taken. The names of the methods that execute should be `reserve` and `getAllAvailableSeats`.

31.8 *(Project: Morse Code Web Service)* In Exercise 16.22, you learned about Morse Code and wrote applications that could translate English phrases into Morse Code and vice versa. Create a SOAP-based web service that provides two methods—one that translates an English phrase into Morse Code and one that translates Morse Code into English. Next, build a Morse Code translator GUI application that invokes the web service to perform these translations.

Making a Difference

31.9 *(Project: Spam Scanner Web Service)* In Exercise 16.27, you created a spam scanner application that scanned an e-mail and gave it a point rating based on the occurrence of certain words and phrases that commonly appear in spam e-mails and how many times the words and phrases occurred in the e-mail. Create a SOAP-based Spam scanner web service. Next, modify the GUI application you created in Exercise 16.27 to use the web service to scan an e-mail. Then display the point rating returned by the web service.

31.10 *(Project: SMS Web Service)* In Exercise 16.28, you created an SMS message-translator application. Create a SOAP-based web service with three methods:

a) one that receives an SMS abbreviation and returns the corresponding English word or phrase,
b) one that receives an entire SMS message and returns the corresponding English text, and
c) one that translates English text into an SMS message.

Use the web service from a GUI application that displays the web service's responses.

31.11 *(Project: Gender-Neutrality Web Service)* In Exercise 1.12, you researched eliminating sexism in all forms of communication. You then described the algorithm you'd use to read through a paragraph of text and replace gender-specific words with gender-neutral equivalents. Create a SOAP-based web service that receives a paragraph of text, then replaces gender-specific words with gender-neutral ones. Use the web service from a GUI application that displays the resulting gender-neutral text.



A

Operator Precedence Chart

Operators are shown in decreasing order of precedence from top to bottom (Fig. A.1).

Operator	Description	Associativity
`++` `--`	unary postfix increment unary postfix decrement	right to left

`++`
`--`
`+`
`-`
`!`
`~`
`( type )`
	unary prefix increment
	unary prefix decrement
	unary plus
	unary minus
	unary logical negation
	unary bitwise complement
	unary cast
		right to left

`*`
`/`
`%`
	multiplication
	division
	remainder
		left to right

`+` `-`	addition or string concatenation subtraction	left to right

`<<`
`>>`
`>>>`
	left shift
	signed right shift
	unsigned right shift
		left to right

`<`
`<=`
`>`
`>=`
`instanceof`
	less than
	less than or equal to
	greater than
	greater than or equal to
	type comparison
		left to right

`==` `!=`	is equal to is not equal to	left to right
`&`	bitwise AND boolean logical AND	left to right
`^`	bitwise exclusive OR boolean logical exclusive OR	left to right

Fig. A.1 | Operator precedence chart. (Part 1 of 2.)

Operator	Description	Associativity
|	bitwise inclusive OR boolean logical inclusive OR	left to right
&&	conditional AND	left to right
||	conditional OR	left to right
?:	conditional	right to left
=	assignment	right to left
+=	addition assignment	
-=	subtraction assignment	
*=	multiplication assignment	
/=	division assignment	
%=	remainder assignment	
&=	bitwise AND assignment	
^=	bitwise exclusive OR assignment	
|=	bitwise inclusive OR assignment	
<<=	bitwise left-shift assignment	
>>=	bitwise signed-right-shift assignment	
>>>=	bitwise unsigned-right-shift assignment	

Fig. A.1 | Operator precedence chart. (Part 2 of 2.)



B

ASCII Character Set

	0	1	2	3	4	5	6	7	8	9
0	nul	soh	stx	etx	eot	enq	ack	bel	bs	ht
1	nl	vt	ff	cr	so	si	dle	dc1	dc2	dc3
2	dc4	nak	syn	etb	can	em	sub	esc	fs	gs
3	rs	us	sp	!	"	#	$	%	&	‘
4	(	)	*	+	,	-	.	/	0	1
5	2	3	4	5	6	7	8	9	:	;
6	<	=	>	?	@	A	B	C	D	E
7	F	G	H	I	J	K	L	M	N	O
8	P	Q	R	S	T	U	V	W	X	Y
9	Z	[	\	]	^	_	’	a	b	c
10	d	e	f	g	h	i	j	k	l	m
11	n	o	p	q	r	s	t	u	v	w
12	x	y	z	{	|	}	~	del		

Fig. B.1 | ASCII character set.

The digits at the left of the table are the left digits of the decimal equivalents (0–127) of the character codes, and the digits at the top of the table are the right digits of the character codes. For example, the character code for “F” is 70, and the character code for “&” is 38.

Most users of this book are interested in the ASCII character set used to represent English characters on many computers. The ASCII character set is a subset of the Unicode character set used by Java to represent characters from most of the world’s languages. For more information on the Unicode character set, see the web bonus Appendix N.



C

Keywords and Reserved Words

Java Keywords				
`abstract`	`assert`	`boolean`	`break`	`byte`
`case`	`catch`	`char`	`class`	`continue`
`default`	`do`	`double`	`else`	`enum`
`extends`	`final`	`finally`	`float`	`for`
`if`	`implements`	`import`	`instanceof`	`int`
`interface`	`long`	`native`	`new`	`package`
`private`	`protected`	`public`	`return`	`short`
`static`	`strictfp`	`super`	`switch`	`synchronized`
`this`	`throw`	`throws`	`transient`	`try`
`void`	`volatile`	`while`		
Keywords that are not currently used				
`const`	`goto`			

Fig. C.1 | Java keywords.

Java also contains the reserved words `true` and `false`, which are `boolean` literals, and `null`, which is the literal that represents a reference to nothing. Like keywords, these reserved words cannot be used as identifiers.



D

Primitive Types

Type	Size in bits	Values	Standard
`boolean`		`true` or `false`	
[*Note:* A `boolean`'s representation is specific to the Java Virtual Machine on each platform.]			
`char`	16	`'\u0000'` to `'\uFFFF'` (0 to 65535)	(ISO Unicode character set)
`byte`	8	–128 to +127 (-2^7 to $2^7 - 1$)	
`short`	16	–32,768 to +32,767 (-2^{15} to $2^{15} - 1$)	
`int`	32	–2,147,483,648 to +2,147,483,647 (-2^{31} to $2^{31} - 1$)	
`long`	64	–9,223,372,036,854,775,808 to +9,223,372,036,854,775,807 (-2^{63} to $2^{63} - 1$)	
`float`	32	*Negative range:* –3.4028234663852886E+38 to –1.40129846432481707e–45 *Positive range:* 1.40129846432481707e–45 to 3.4028234663852886E+38	(IEEE 754 floating point)
`double`	64	*Negative range:* –1.7976931348623157E+308 to –4.94065645841246544e–324 *Positive range:* 4.94065645841246544e–324 to 1.7976931348623157E+308	(IEEE 754 floating point)

Fig. D.1 | Java primitive types.

For more information on IEEE 754 visit `grouper.ieee.org/groups/754/`. For more information on Unicode, see Appendix N.



E

Using the Java API Documentation

E.1 Introduction

The Java class library contains thousands of predefined classes and interfaces that programmers can use to write their own applications. These classes are grouped into packages based on their functionality. For example, the classes and interfaces used for file processing are grouped into the `java.io` package, and the classes and interfaces for networking applications are grouped into the `java.net` package. The **Java API documentation** lists the `public` and `protected` members of each class and the `public` members of each interface in the Java class library. The documentation overviews all the classes and interfaces, summarizes their members (i.e., the fields, constructors and methods of classes, and the fields and methods of interfaces) and provides detailed descriptions of each member. Most Java programmers rely on this documentation when writing programs. Normally, programmers would search the API to find the following:

1. The package that contains a particular class or interface.
2. Relationships between a particular class or interface and other classes and interfaces.
3. Class or interface constants—normally declared as `public static final` fields.
4. Constructors to determine how an object of the class can be initialized.
5. The methods of a class to determine whether they're `static` or non-`static`, the number and types of the arguments you need to pass, the return types and any exceptions that might be thrown from the method.

In addition, programmers often rely on the documentation to discover classes and interfaces that they have not used before. For this reason, we demonstrate the documentation with classes you know and classes you may not have studied yet. We show how to use the documentation to locate the information you need to use a class or interface effectively.

E.2 Navigating the Java API

The Java API documentation can be downloaded to your local hard disk or viewed online. To download the Java API documentation, go to `www.oracle.com/technetwork/java/javase/downloads/index.html/` scroll down to the **Additional Resources** section and click the **Download Zip** button to the right of **Java SE 6 Documentation**. You'll be asked to accept a license agreement. To do this, click **Accept**, then click **Continue**. Click the link to

the ZIP file to begin downloading it. After downloading the file, you can use a program such as WinZip (`www.winzip.com`) to extract the files. If you're using Windows, extract the contents to your JDK's installation directory. To view the API documentation on your local hard disk in Microsoft Windows, open `C:\Program Files\Java\`*YourJDKVersion*`\docs\api\index.html` page in your browser. To view the API documentation online, go to `download.oracle.com/javase/6/docs/api/` (Fig. E.1).

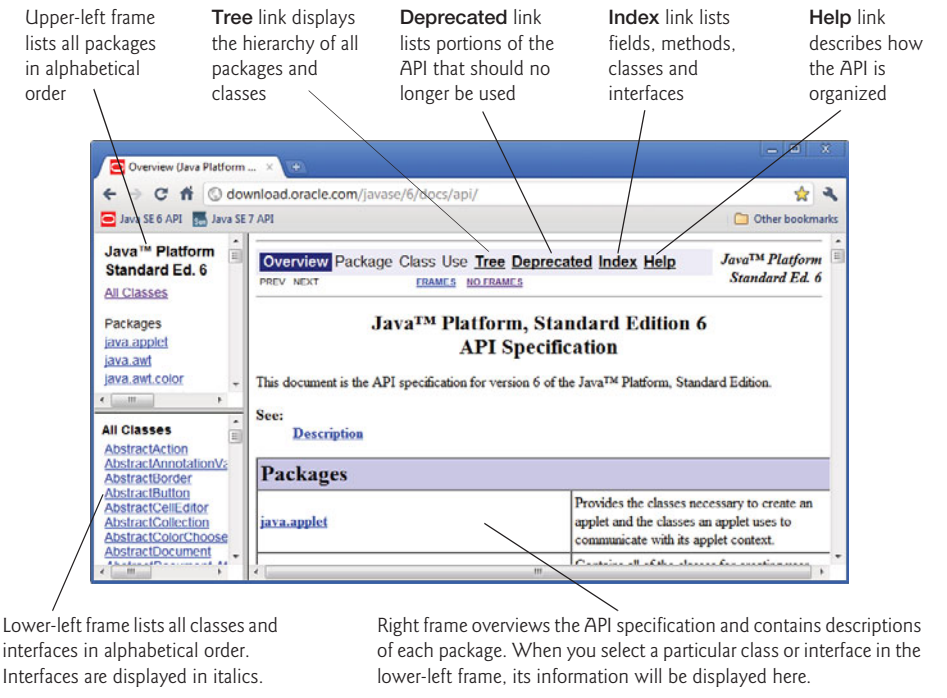


Fig. E.1 | Java API overview. (Courtesy of Oracle Corporation)

Frames in the API Documentation's `index.html` *Page*

The API documentation is divided into three frames (see Fig. E.1). The upper-left frame lists all of the Java API's packages in alphabetical order. The lower-left frame initially lists the Java API's classes and interfaces in alphabetical order. Interface names are displayed in italic. When you click a specific package in the upper-left frame, the lower-left frame lists the classes and interfaces of the selected package. The right frame initially provides a brief description of each package of the Java API specification—read this overview to become familiar wth the general capabilities of the Java APIs. If you select a class or interface in the lower-left frame, the right frame displays information about that class or interface.

Important Links in the `index.html` *Page*

At the top of the right frame (Fig. E.1), there are four links—**Tree**, **Deprecated**, **Index** and **Help**. The **Tree** link displays the hierarchy of all packages, classes and interfaces in a tree structure. The **Deprecated** link displays interfaces, classes, exceptions, fields, constructors and methods that should no longer be used. The **Index** link displays classes, interfaces,

fields, constructors and methods in alphabetical order. The **Help** link describes how the API documentation is organized. You should probably begin by reading the **Help** page.

Viewing the Index *Page*

If you do not know the name of the class you're looking for, but you do know the name of a method or field, you can use the documentation's index to locate the class. The **Index** link is located near the upper-right corner of the right frame. The index page (Fig. E.2) displays fields, constructors, methods, interfaces and classes in alphabetical order. For example, if you're looking for `Scanner` method `hasNextInt`, but do not know the class name, you can click the **H** link to go to the alphabetical listing of all items in the Java API that begin with `"h"`. Scroll to method `hasNextInt` (Fig. E.3). Once there, each method named `hasNextInt` is listed with the package name and class to which the method belongs. From there, you can click the class name to view the class's complete details, or you can click the method name to view the method's details.

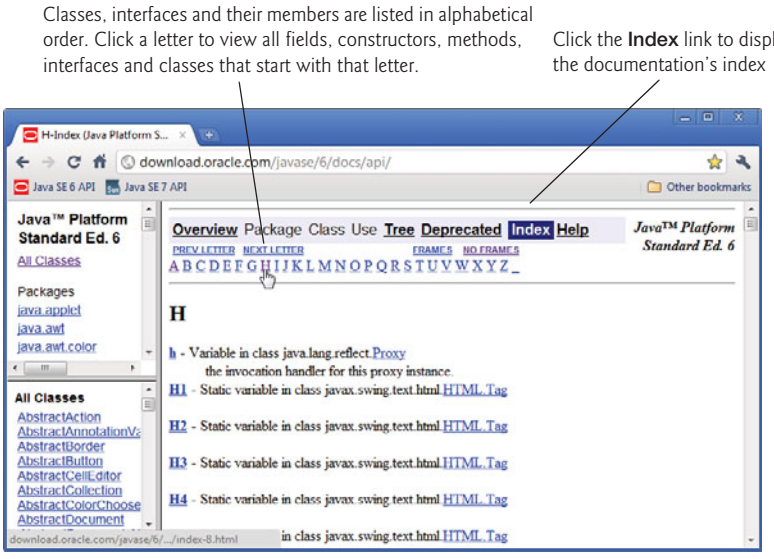


Fig. E.2 | Viewing the **Index** page. (Courtesy of Oracle Corporation.)

Viewing a Specific Package

When you click the package name in the upper-left frame, all classes and interfaces from that package are displayed in the lower-left frame and are divided into five subsections—**Interfaces**, **Classes**, **Enums**, **Exceptions** and **Errors**—each listed alphabetically. For example, the contents of package `javax.swing` are displayed in the lower-left frame (Fig. E.4) when you click `javax.swing` in the upper-left frame. You can click the package name in the lower-left frame to get an overview of the package. If you think that a package contains several classes that could be useful in your application, the package overview can be especially helpful.

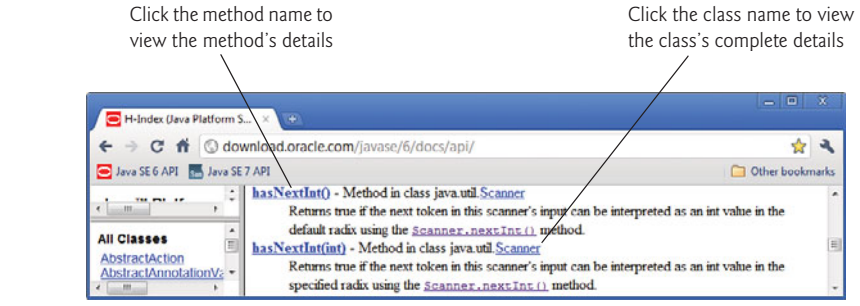


Fig. E.3 | Scroll to method `hasNextInt`. (Courtesy of Oracle Corporation)

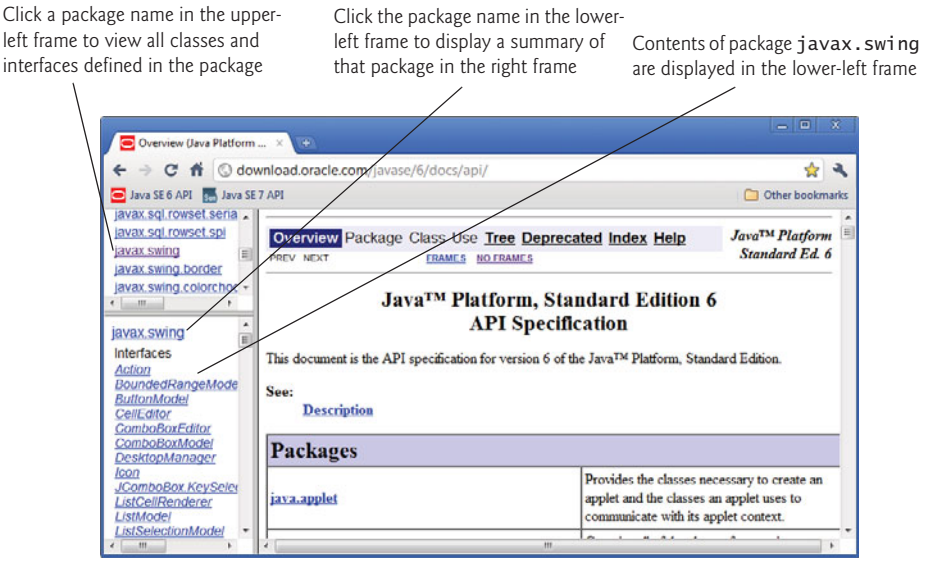


Fig. E.4 | Clicking a package name in the upper-left frame to view all classes and interfaces declared in this package. (Courtesy of Oracle Corporation)

Viewing the Details of a Class

When you click a class name or interface name in the lower-left frame, the right frame displays the details of that class or interface. First you'll see the class's package name followed by a hierarchy that shows the class's relationship to other classes. You'll also see a list of the interfaces implemented by the class and the class's known subclasses. Figure E.5 shows the beginning of the documentation page for class `JButton` from the `javax.swing` package. The page first shows the package name in which the class appears. This is followed by the class hierarchy that leads to class `JButton`, the interfaces class `JButton` implements and the subclasses of class `JButton`. The bottom of the right frame shows the beginning of class `JButton`'s description. When you look at the documentation for an interface, the right frame does not display a hierarchy for that interface. Instead, the right frame lists the interface's superinterfaces, known subinterfaces and known implementing classes.

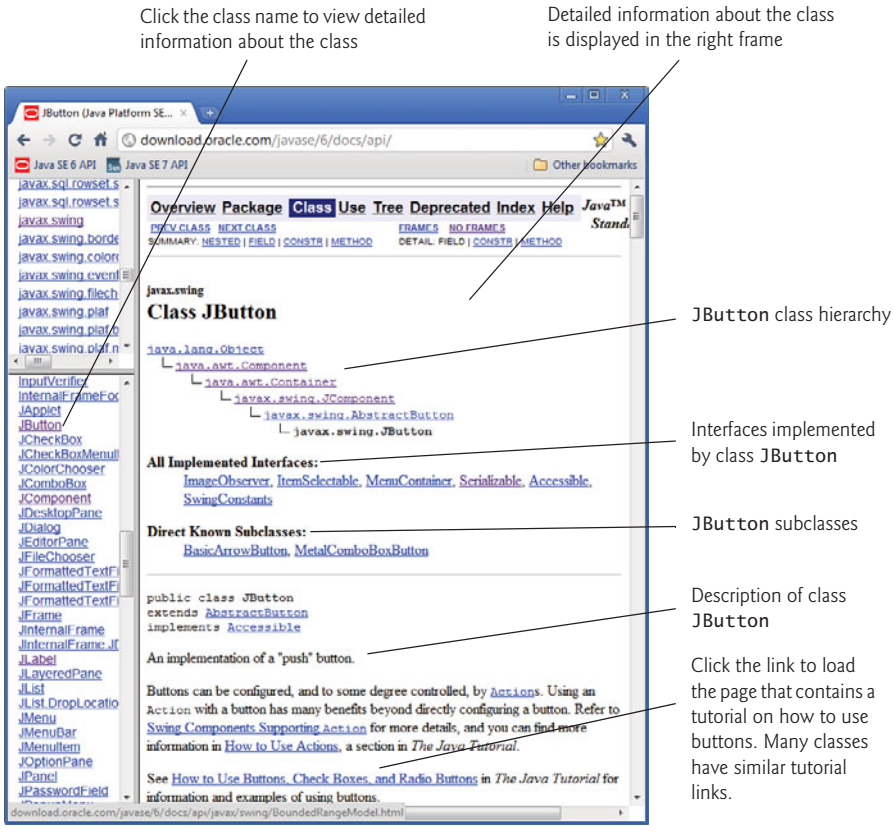


Fig. E.5 | Clicking a class name to view detailed information about the class. (Courtesy of Oracle Corporation)

Summary Sections in a Class's Documentation Page

Other parts of each API page are listed below. Each part is presented only if the class contains or inherits the items specified. Class members shown in the summary sections are `public` unless they're explicitly marked as `protected`. A class's `private` members are not shown in the documentation, because they cannot be used directly in your programs.

1. The **Nested Class Summary** section summarizes the class's `public` and `protected` nested classes—i.e., classes that are defined inside the class. Unless explicitly specified, these classes are `public` and non-`static`.
2. The **Field Summary** section summarizes the class's `public` and `protected` fields. Unless explicitly specified, these fields are `public` and non-`static`. Figure E.6 shows the **Field Summary** section of class `Color`.
3. The **Constructor Summary** section summarizes the class's constructors. Constructors are not inherited, so this section appears in the documentation for a class only if the class declares one or more constructors. Figure E.7 shows the **Constructor Summary** section of class `JButton`.

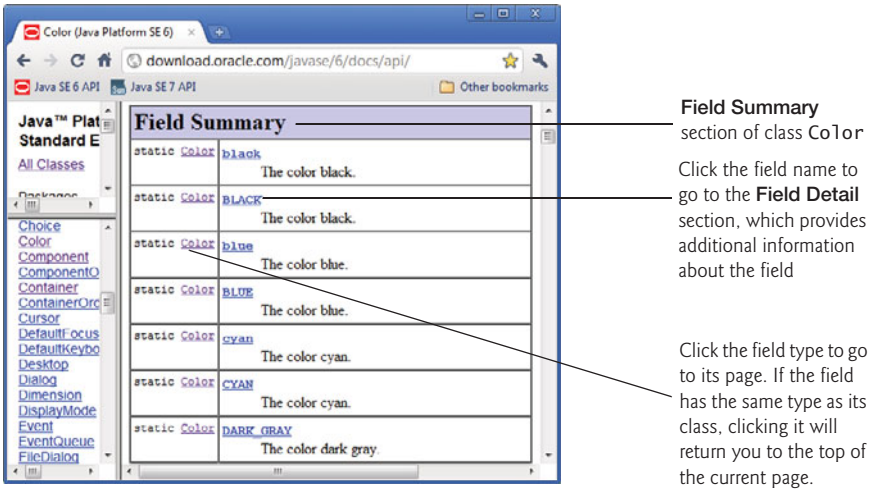


Fig. E.6 | **Field Summary** section of class `Color`. (Courtesy of Oracle Corporation)

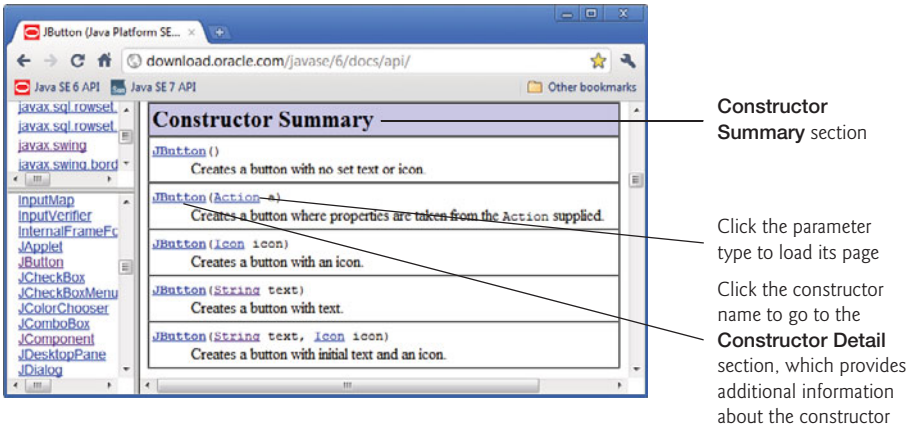


Fig. E.7 | **Constructor Summary** section of class `JButton`. (Courtesy of Oracle Corporation)

4. The **Method Summary** section summarizes the class's `public` and `protected` methods. Unless explicitly specified, these methods are `public` and non-`static`. Figure E.8 shows the **Method Summary** section of class `BufferedInputStream`.

The summary sections typically provide only a one-sentence description of a class member. Additional details are presented in the detail sections discussed next.

Detail Sections in a Class's Documentation Page

After the summary sections are detail sections that normally provide more discussion of particular class members. There isn't a detail section for nested classes. When you click the

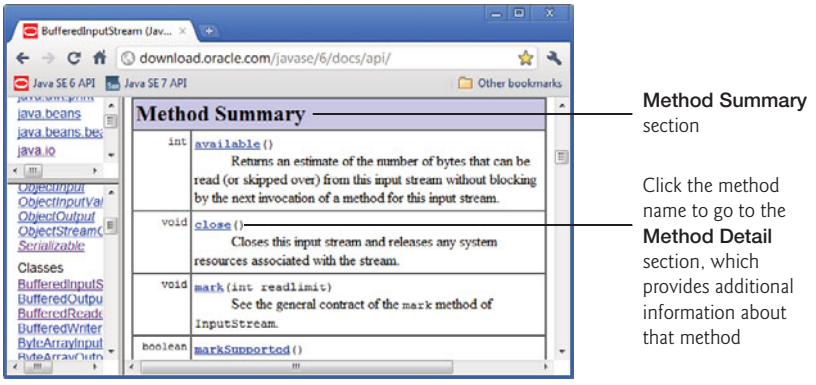


Fig. E.8 | **Method Summary** section of class `BufferedInputStream`. (Courtesy of Oracle Corporation)

link in the **Nested Class Summary** for a particular nested class, a documentation page describing that nested class is displayed. The detail sections are described below.

1. The **Field Detail** section provides the declaration of each field. It also discusses each field, including the field's modifiers and meaning. Figure E.9 shows the **Field Detail** section of class `Color`.
2. The **Constructor Detail** section provides the first line of each constructor's declaration and discusses the constructors. The discussion includes the modifiers of each constructor, a description of each constructor, each constructor's parameters and any exceptions thrown by each constructor. Figure E.10 shows the **Constructor Detail** section of class `JButton`.
3. The **Method Detail** section provides the first line of each method. The discussion of each method includes its modifiers, a more complete method description, the method's parameters, the method's return type and any exceptions thrown by the method. Figure E.11 shows class `BufferedInputStream`'s **Method Detail** section.

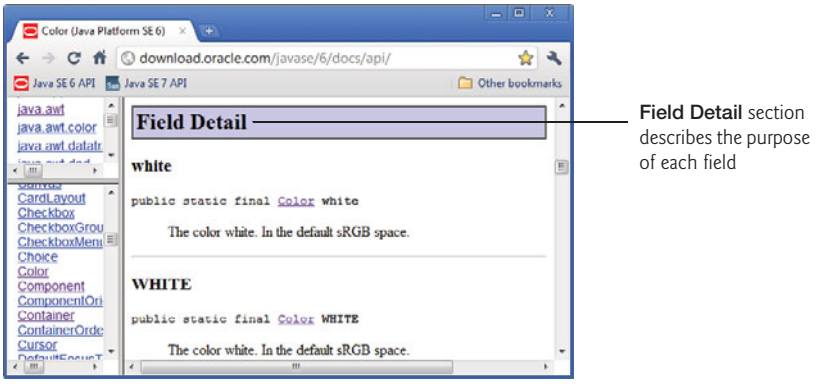


Fig. E.9 | **Field Detail** section of class `Color`. (Courtesy of Oracle Corporation)

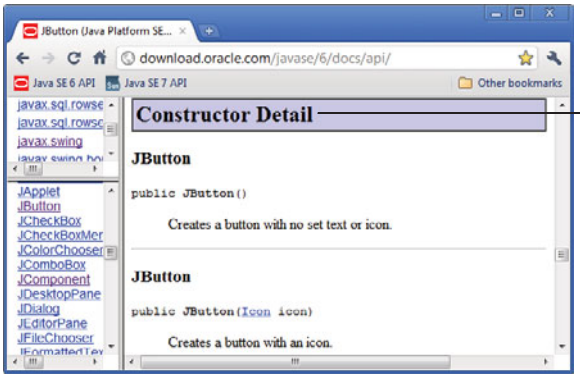


Fig. E.10 | **Constructor Detail** section of class `JButton`. (Courtesy of Oracle Corporation)

The method details show you other methods that might be of interest (labeled as **See Also**). If the method overrides a method of the superclass, the name of the superclass method and the name of the superclass are provided so you can link to the method or superclass for more information.

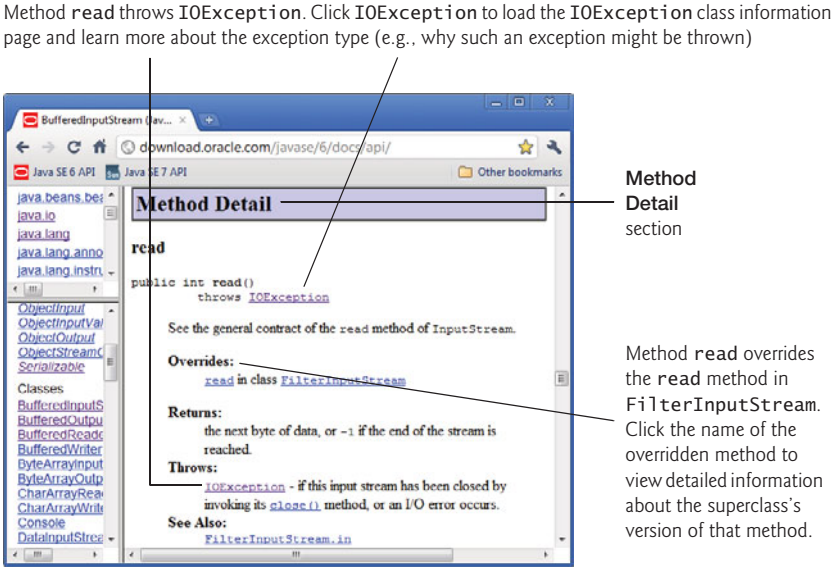


Fig. E.11 | **Method Detail** section of class `BufferedInputStream`. (Courtesy of Oracle Corporation)

As you look through the documentation, you'll notice that there are often links to other fields, methods, nested-classes and top-level classes. These links enable you to jump from the class you're looking at to another relevant portion of the documentation.

F

Using the Debugger



And so shall I catch the fly.
—William Shakespeare

We are built to make mistakes, coded for error.
—Lewis Thomas

What we anticipate seldom occurs; what we least expect generally happens.
—Benjamin Disraeli

Objectives

In this appendix you'll learn:

- To set breakpoints to debug applications.
- To use the `run` command to run an application through the debugger.
- To use the `stop` command to set a breakpoint.
- To use the `cont` command to continue execution.
- To use the `print` command to evaluate expressions.
- To use the `set` command to change variable values during program execution.
- To use the `step`, `step up` and `next` commands to control execution.
- To use the `watch` command to see how a field is modified during program execution.
- To use the `clear` command to list breakpoints or remove a breakpoint.

Outline

F.1 Introduction
F.2 Breakpoints and the `run`, `stop`, `cont` and `print` Commands
F.3 The `print` and `set` Commands
F.4 Controlling Execution Using the `step`, `step up` and `next` Commands
F.5 The `watch` Command
F.6 The `clear` Command
F.7 Wrap-Up

Self-Review Exercises | *Answers to Self-Review Exercises*

F.1 Introduction

In Chapter 2, you learned that there are two types of errors—syntax errors and logic errors—and you learned how to eliminate syntax errors from your code. Logic errors do not prevent the application from compiling successfully, but they do cause an application to produce erroneous results when it runs. The JDK includes software called a **debugger** that allows you to monitor the execution of your applications so you can locate and remove logic errors. The debugger will be one of your most important application development tools. Many IDEs provide their own debuggers similar to the one included in the JDK or provide a graphical user interface to the JDK's debugger.

This appendix demonstrates key features of the JDK's debugger using command-line applications that receive no input from the user. The same debugger features discussed here can be used to debug applications that take user input, but debugging such applications requires a slightly more complex setup. To focus on the debugger features, we've opted to demonstrate the debugger with simple command-line applications involving no user input. For more information on the Java debugger visit `download.oracle.com/javase/6/docs/technotes/tools/windows/jdb.html`.

F.2 Breakpoints and the `run`, `stop`, `cont` and `print` Commands

We begin our study of the debugger by investigating **breakpoints**, which are markers that can be set at any executable line of code. When application execution reaches a breakpoint, execution pauses, allowing you to examine the values of variables to help determine whether logic errors exist. For example, you can examine the value of a variable that stores the result of a calculation to determine whether the calculation was performed correctly. Setting a breakpoint at a line of code that is not executable (such as a comment) causes the debugger to display an error message.

To illustrate the features of the debugger, we use application `AccountTest` (Fig. F.1), which creates and manipulates an object of class `Account` (Fig. 3.13). Execution of `AccountTest` begins in `main` (lines 7–24). Line 9 creates an `Account` object with an initial balance of $50.00. Recall that `Account`'s constructor accepts one argument, which specifies the `Account`'s initial `balance`. Lines 12–13 output the initial account balance using `Account` method `getBalance`. Line 15 declares and initializes a local variable `depositAmount`. Lines 17–19 then print `depositAmount` and add it to the `Account`'s `balance` using

its `credit` method. Finally, lines 22–23 display the new `balance`. [*Note:* The Appendix F examples directory contains a copy of `Account.java` identical to the one in Fig. 3.13.]

```
1  // Fig. F.1: AccountTest.java
2  // Create and manipulate an Account object.
3
4  public class AccountTest
5  {
6     // main method begins execution
7     public static void main( String[] args )
8     {
9        Account account = new Account( 50.00 ); // create Account object
10
11       // display initial balance of Account object
12       System.out.printf( "initial account balance: $%.2f\n",
13          account.getBalance() );
14
15       double depositAmount = 25.0; // deposit amount
16
17       System.out.printf( "\nadding %.2f to account balance\n\n",
18          depositAmount );
19       account.credit( depositAmount ); // add to account balance
20
21       // display new balance
22       System.out.printf( "new account balance: $%.2f\n",
23          account.getBalance() );
24    } // end main
25
26 } // end class AccountTest
```

```
initial account balance: $50.00

adding 25.00 to account balance

new account balance: $75.00
```

Fig. F.1 | `AccountTest` class creates and manipulates an `Account` object.

In the following steps, you'll use breakpoints and various debugger commands to examine the value of the variable `depositAmount` declared in `AccountTest` (Fig. F.1).

1. ***Opening the* Command Prompt *window and changing directories.*** Open the **Command Prompt** window by selecting **Start > Programs > Accessories > Command Prompt**. Change to the directory containing the Appendix F examples by typing `cd C:\examples\debugger` [*Note:* If your examples are in a different directory, use that directory here.]

2. ***Compiling the application for debugging.*** The Java debugger works only with `.class` files that were compiled with the **`-g`** compiler option, which generates information that is used by the debugger to help you debug your applications. Compile the application with the `-g` command-line option by typing `javac -g AccountTest.java Account.java`. Recall from Chapter 3 that this command compiles both `AccountTest.java` and `Account.java`. The command `java -g *.java` compiles all of the working directory's `.java` files for debugging.

3. ***Starting the debugger.*** In the **Command Prompt**, type **`jdb`** (Fig. F.2). This command will start the Java debugger and enable you to use its features. [*Note:* We modified the colors of our **Command Prompt** window for readability.]

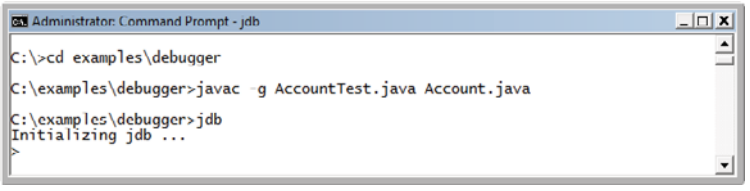


Fig. F.2 | Starting the Java debugger.

4. ***Running an application in the debugger.*** Run the `AccountTest` application through the debugger by typing **`run`** `AccountTest` (Fig. F.3). If you do not set any breakpoints before running your application in the debugger, the application will run just as it would using the `java` command.

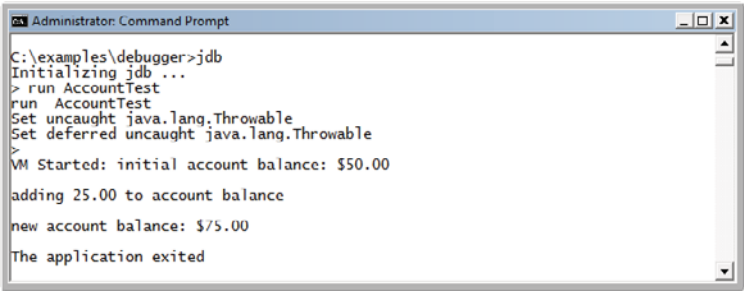


Fig. F.3 | Running the `AccountTest` application through the debugger.

5. ***Restarting the debugger.*** To make proper use of the debugger, you must set at least one breakpoint before running the application. Restart the debugger by typing `jdb`.

6. ***Inserting breakpoints in Java.*** You set a breakpoint at a specific line of code in your application. The line numbers used in these steps are from the source code in Fig. F.1. Set a breakpoint at line 12 in the source code by typing `stop at AccountTest:12` (Fig. F.4). The **`stop` command** inserts a breakpoint at the line number specified after the command. You can set as many breakpoints as necessary. Set another breakpoint at line 19 by typing `stop at AccountTest:19` (Fig. F.4). When the application runs, it suspends execution at any line that contains a breakpoint. The application is said to be in **break mode** when the debugger pauses the application's execution. Breakpoints can be set even after the debugging process has begun. The debugger command `stop in`, followed by a class name, a period and a method name (e.g., `stop in Account.credit`) instructs the debugger to set a breakpoint at the first executable statement in the specified method. The debugger pauses execution when program control enters the method.

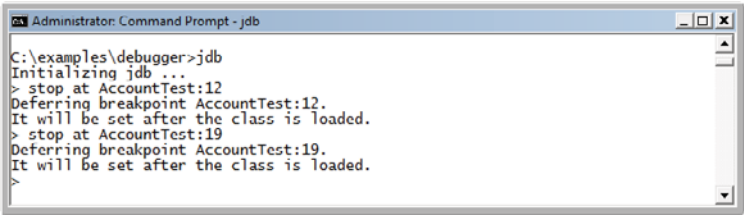


Fig. F.4 | Setting breakpoints at lines 12 and 19.

7. ***Running the application and beginning the debugging process.*** Type `run AccountTest` to execute the application and begin the debugging process (Fig. F.5). The debugger prints text indicating that breakpoints were set at lines 12 and 19. It calls each breakpoint a "deferred breakpoint" because each was set before the application began running in the debugger. The application pauses when execution reaches the breakpoint on line 12. At this point, the debugger notifies you that a breakpoint has been reached and it displays the source code at that line (12). That line of code is the next statement that will execute.

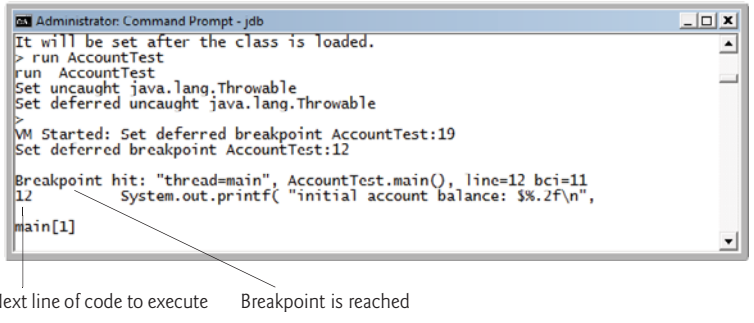


Fig. F.5 | Restarting the `AccountTest` application.

8. ***Using the `cont` command to resume execution.*** Type `cont`. The **`cont` command** causes the application to continue running until the next breakpoint is reached (line 19), at which point the debugger notifies you (Fig. F.6). `AccountTest`'s normal output appears between messages from the debugger.

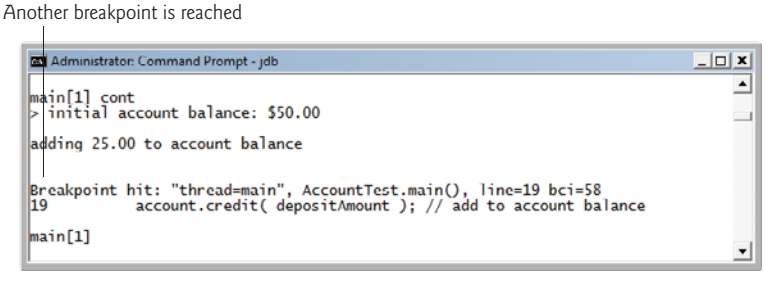


Fig. F.6 | Execution reaches the second breakpoint.

9. *Examining a variable's value.* Type `print depositAmount` to display the current value stored in the `depositAmount` variable (Fig. F.7). The **`print` command** allows you to peek inside the computer at the value of one of your variables. This command will help you find and eliminate logic errors in your code. The value displayed is `25.0`—the value assigned to `depositAmount` in line 15 of Fig. F.1.



Fig. F.7 | Examining the value of variable `depositAmount`.

10. *Continuing application execution.* Type `cont` to continue the application's execution. There are no more breakpoints, so the application is no longer in break mode. The application continues executing and eventually terminates (Fig. F.8). The debugger will stop when the application ends.

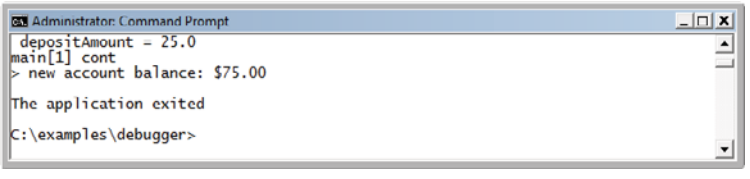


Fig. F.8 | Continuing application execution and exiting the debugger.

F.3 The `print` and `set` Commands

In the preceding section, you learned how to use the debugger's `print` command to examine the value of a variable during program execution. In this section, you'll learn how to use the `print` command to examine the value of more complex expressions. You'll also learn the **`set` command**, which allows the programmer to assign new values to variables.

For this section, we assume that you've followed *Step 1* and *Step 2* in Section F.2 to open the **Command Prompt** window, change to the directory containing the Appendix F examples (e.g., `C:\examples\debugger`) and compile the `AccountTest` application (and class `Account`) for debugging.

1. *Starting debugging.* In the **Command Prompt**, type `jdb` to start the Java debugger.
2. *Inserting a breakpoint.* Set a breakpoint at line 19 in the source code by typing `stop at AccountTest:19`.
3. *Running the application and reaching a breakpoint.* Type `run AccountTest` to begin the debugging process (Fig. F.9). This will cause `AccountTest`'s `main` to execute until the breakpoint at line 19 is reached. This suspends application execution and switches the application into break mode. At this point, the statements in lines 9–13 created an `Account` object and printed the initial balance of the `Ac-`

`count` obtained by calling its `getBalance` method. The statement in line 15 (Fig. F.1) declared and initialized local variable `depositAmount` to `25.0`. The statement in line 19 is the next statement that will execute.

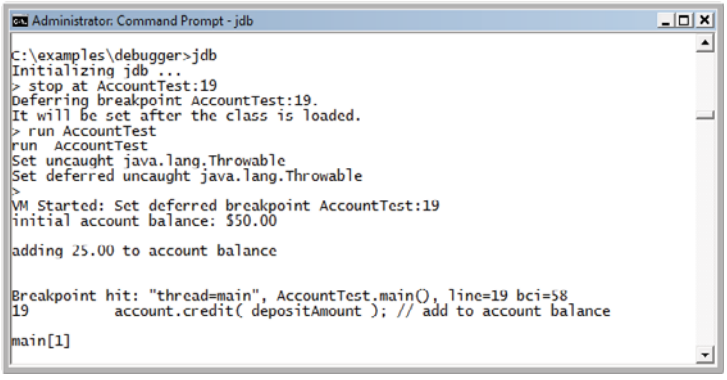


Fig. F.9 | Application execution suspended when debugger reaches the breakpoint at line 19.

4. ***Evaluating arithmetic and boolean expressions.*** Recall from Section F.2 that once the application has entered break mode, you can explore the values of the application's variables using the debugger's `print` command. You can also use the `print` command to evaluate arithmetic and `boolean` expressions. In the **Command Prompt** window, type `print depositAmount - 2.0`. The `print` command returns the value `23.0` (Fig. F.10). However, this command does not actually change the value of `depositAmount`. In the **Command Prompt** window, type `print depositAmount == 23.0`. Expressions containing the `==` symbol are treated as `boolean` expressions. The value returned is `false` (Fig. F.10) because `depositAmount` does not currently contain the value `23.0`—`depositAmount` is still `25.0`.

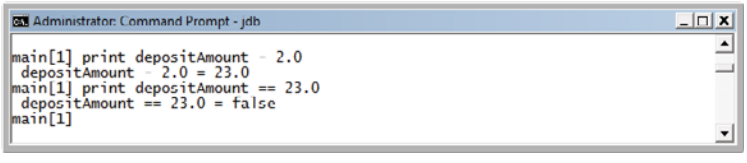


Fig. F.10 | Examining the values of an arithmetic and `boolean` expression.

5. ***Modifying values.*** The debugger allows you to change the values of variables during the application's execution. This can be valuable for experimenting with different values and for locating logic errors in applications. You can use the debugger's `set` command to change the value of a variable. Type `set depositAmount = 75.0`. The debugger changes the value of `depositAmount` and displays its new value (Fig. F.11).



Fig. F.11 | Modifying values.

6. *Viewing the application result.* Type `cont` to continue application execution. Line 19 of `AccountTest` (Fig. F.1) executes, passing `depositAmount` to `Account` method `credit`. Method `main` then displays the new balance. The result is `$125.00` (Fig. F.12). This shows that the preceding step changed the value of `depositAmount` from its initial value (`25.0`) to `75.0`.

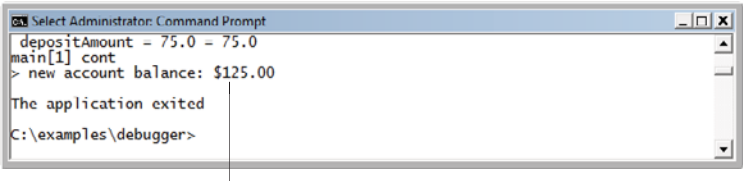


Fig. F.12 | Output displayed after the debugging process.

F.4 Controlling Execution Using the `step`, `step up` and `next` Commands

Sometimes you'll need to execute an application line by line to find and fix errors. Walking through a portion of your application this way can help you verify that a method's code executes correctly. In this section, you'll learn how to use the debugger for this task. The commands you learn in this section allow you to execute a method line by line, execute all the statements of a method at once or execute only the remaining statements of a method (if you've already executed some statements within the method).

Once again, we assume you're working in the directory containing the Appendix F examples and have compiled for debugging with the `-g` compiler option.

1. *Starting the debugger.* Start the debugger by typing `jdb`.
2. *Setting a breakpoint.* Type `stop at AccountTest:19` to set a breakpoint at line 19.
3. *Running the application.* Run the application by typing `run AccountTest`. After the application displays its two output messages, the debugger indicates that the breakpoint has been reached and displays the code at line 19 (Fig. F.13). The debugger and application then pause and wait for the next command to be entered.
4. *Using the* **`step`** *command.* The **`step` command** executes the next statement in the application. If the next statement to execute is a method call, control transfers to the called method. The `step` command enables you to enter a method and study

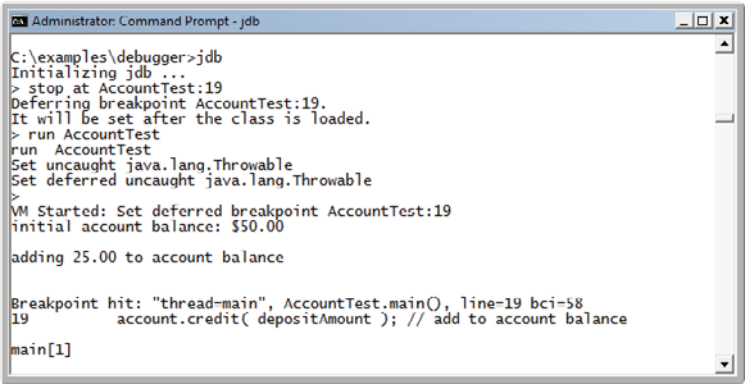


Fig. F.13 | Reaching the breakpoint in the `AccountTest` application.

the individual statements of that method. For instance, you can use the `print` and `set` commands to view and modify the variables within the method. You'll now use the `step` command to enter the `credit` method of class `Account` (Fig. 3.13) by typing `step` (Fig. F.14). The debugger indicates that the step has been completed and displays the next executable statement—in this case, line 21 of class `Account` (Fig. 3.13).

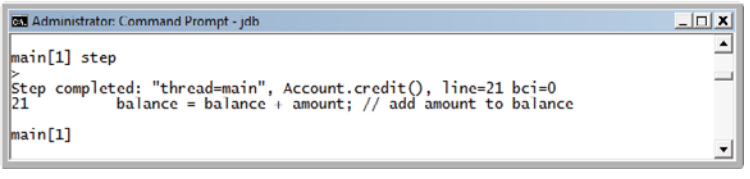


Fig. F.14 | Stepping into the `credit` method.

5. *Using the* **`step up`** *command.* After you've stepped into the `credit` method, type **`step up`**. This command executes the remaining statements in the method and returns control to the place where the method was called. The `credit` method contains only one statement to add the method's parameter `amount` to instance variable `balance`. The `step up` command executes this statement, then pauses before line 22 in `AccountTest`. Thus, the next action to occur will be to print the new account balance (Fig. F.15). In lengthy methods, you may want to look at a few key lines of code, then continue debugging the caller's code. The `step up` command is useful for situations in which you do not want to continue stepping through the entire method line by line.
6. *Using the* **`cont`** *command to continue execution.* Enter the `cont` command (Fig. F.16) to continue execution. The statement at lines 22–23 executes, displaying the new balance, then the application and the debugger terminate.
7. *Restarting the debugger.* Restart the debugger by typing `jdb`.

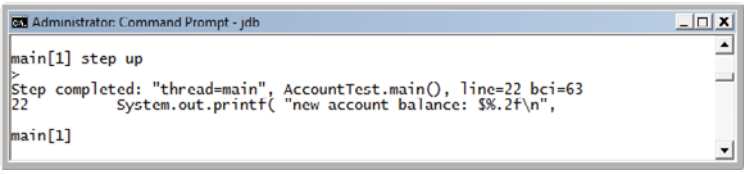


Fig. F.15 | Stepping out of a method.



Fig. F.16 | Continuing execution of the `AccountTest` application.

8. ***Setting a breakpoint.*** Breakpoints persist only until the end of the debugging session in which they're set—once the debugger exits, all breakpoints are removed. (In Section F.6, you'll learn how to manually clear a breakpoint before the end of the debugging session.) Thus, the breakpoint set for line 19 in *Step 2* no longer exists upon restarting the debugger in *Step 7*. To reset the breakpoint at line 19, once again type `stop at AccountTest:19`.
9. ***Running the application.*** Type `run AccountTest` to run the application. As in *Step 3*, `AccountTest` runs until the breakpoint at line 19 is reached, then the debugger pauses and waits for the next command (Fig. F.17).

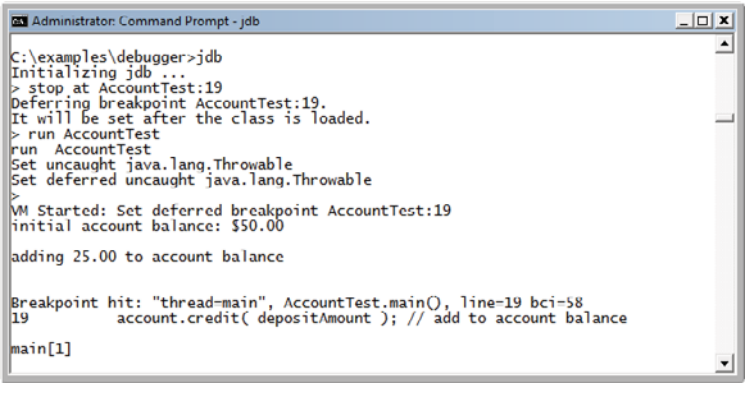


Fig. F.17 | Reaching the breakpoint in the `AccountTest` application.

10. ***Using the `next` command.*** Type **`next`**. This command behaves like the `step` command, except when the next statement to execute contains a method call. In that case, the called method executes in its entirety and the application advances to the

next executable line after the method call (Fig. F.18). Recall from *Step 4* that the `step` command would enter the called method. In this example, the `next` command causes `Account` method `credit` to execute, then the debugger pauses at line 22 in `AccountTest`.

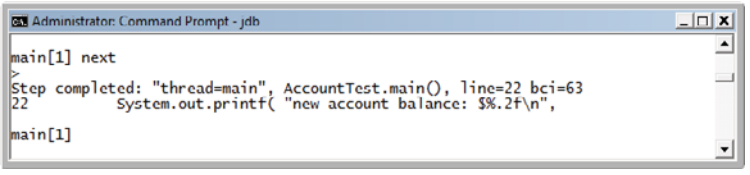


Fig. F.18 | Stepping over a method call.

11. *Using the* **`exit`** *command.* Use the **`exit` command** to end the debugging session (Fig. F.19). This command causes the `AccountTest` application to immediately terminate rather than execute the remaining statements in `main`. When debugging some types of applications (e.g., GUI applications), the application continues to execute even after the debugging session ends.



Fig. F.19 | Exiting the debugger.

F.5 The `watch` Command

In this section, we present the **`watch` command**, which tells the debugger to watch a field. When that field is about to change, the debugger will notify you. In this section, you'll learn how to use the `watch` command to see how the `Account` object's field `balance` is modified during the execution of the `AccountTest` application.

As in the preceding two sections, we assume that you've followed *Step 1* and *Step 2* in Section F.2 to open the **Command Prompt**, change to the correct examples directory and compile classes `AccountTest` and `Account` for debugging (i.e., with the `-g` compiler option).

1. *Starting the debugger.* Start the debugger by typing `jdb`.
2. *Watching a class's field.* Set a watch on `Account`'s `balance` field by typing `watch Account.balance` (Fig. F.20). You can set a watch on any field during execution of the debugger. Whenever the value in a field is about to change, the debugger enters break mode and notifies you that the value will change. Watches can be placed only on fields, not on local variables.
3. *Running the application.* Run the application with the command `run AccountTest`. The debugger will now notify you that field `balance`'s value will change

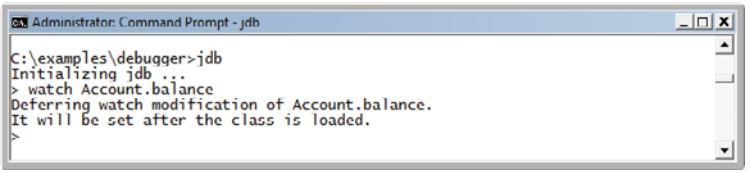


Fig. F.20 | Setting a watch on `Account`'s `balance` field.

(Fig. F.21). When the application begins, an instance of `Account` is created with an initial balance of $50.00 and a reference to the `Account` object is assigned to the local variable `account` (line 9, Fig. F.1). Recall from Fig. 3.13 that when the constructor for this object runs, if parameter `initialBalance` is greater than `0.0`, instance variable `balance` is assigned the value of parameter `initialBalance`. The debugger notifies you that the value of `balance` will be set to `50.0`.

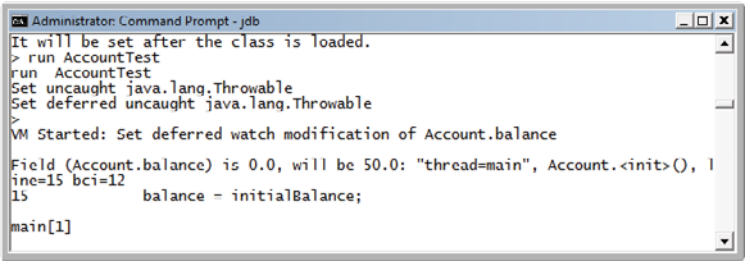


Fig. F.21 | `AccountTest` application stops when `account` is created and its `balance` field will be modified.

4. ***Adding money to the account.*** Type `cont` to continue executing the application. The application executes normally before reaching the code on line 19 of Fig. F.1 that calls `Account` method `credit` to raise the `Account` object's `balance` by a specified `amount`. The debugger notifies you that instance variable `balance` will change (Fig. F.22). Although line 19 of class `AccountTest` calls method `credit`, line 21 in `Account`'s method `credit` actually changes the value of `balance`.

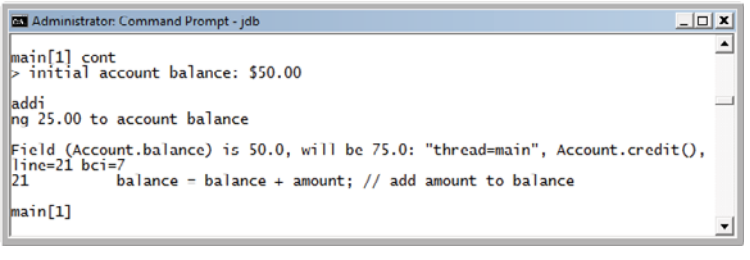


Fig. F.22 | Changing the value of `balance` by calling `Account` method `credit`.

5. ***Continuing execution.*** Type `cont`—the application will finish executing because the application does not attempt any additional changes to `balance` (Fig. F.23).



Fig. F.23 | Continuing execution of `AccountTest`.

6. ***Restarting the debugger and resetting the watch on the variable.*** Type `jdb` to restart the debugger. Once again, set a watch on the `Account` instance variable `balance` by typing the `watch Account.balance`, then type `run AccountTest` to run the application (Fig. F.24).

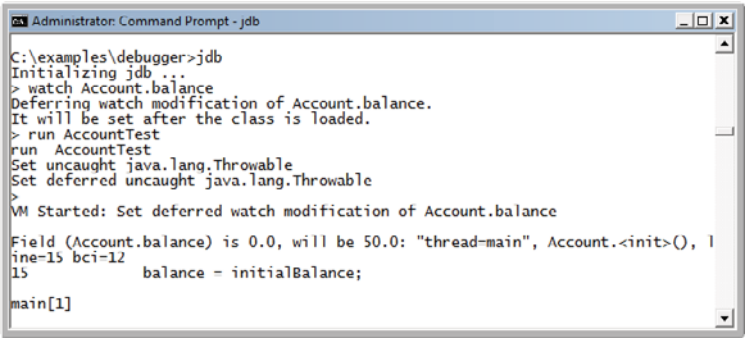


Fig. F.24 | Restarting the debugger and resetting the watch on the variable `balance`.

7. ***Removing the watch on the field.*** Suppose you want to watch a field for only part of a program's execution. You can remove the debugger's watch on variable `balance` by typing **`unwatch`** `Account.balance` (Fig. F.25). Type `cont`—the application will finish executing without reentering break mode.

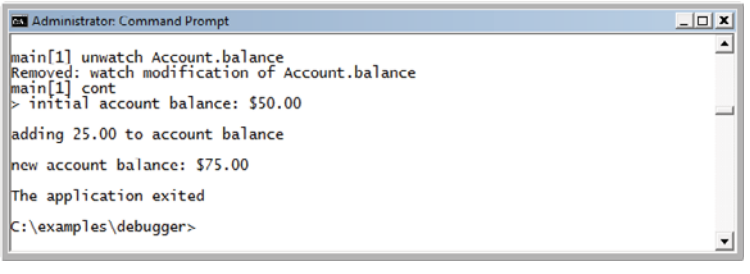


Fig. F.25 | Removing the watch on variable `balance`.

8. *Closing the* **Command Prompt** *window.* Close the **Command Prompt** window by clicking its close button.

F.6 The clear Command

In the preceding section, you learned to use the `unwatch` command to remove a watch on a field. The debugger also provides the `clear` command to remove a breakpoint from an application. You'll often need to debug applications containing repetitive actions, such as a loop. You may want to examine the values of variables during several, but possibly not all, of the loop's iterations. If you set a breakpoint in the body of a loop, the debugger will pause before each execution of the line containing a breakpoint. After determining that the loop is working properly, you may want to remove the breakpoint and allow the remaining iterations to proceed normally. In this section, we use the compound interest application in Fig. 5.6 to demonstrate how the debugger behaves when you set a breakpoint in the body of a `for` statement and how to remove a breakpoint in the middle of a debugging session.

1. *Opening the* **Command Prompt** *window, changing directories and compiling the application for debugging.* Open the **Command Prompt** window, then change to the directory containing the Appendix F examples. For your convenience, we've provided a copy of the `Interest.java` file in this directory. Compile the application for debugging by typing `javac -g Interest.java`.
2. *Starting the debugger and setting breakpoints.* Start the debugger by typing `jdb`. Set breakpoints at lines 13 and 22 of class `Interest` by typing `stop at Interest:13`, then `stop at Interest:22` (Fig. F.26).

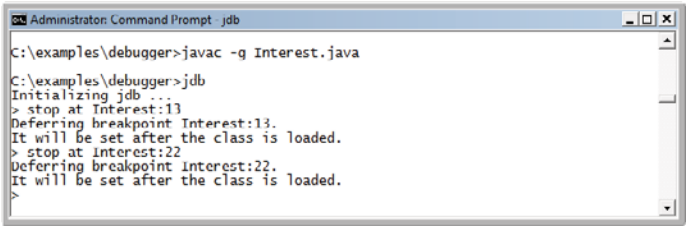


Fig. F.26 | Setting breakpoints in the `Interest` application.

3. *Running the application.* Run the application by typing `run Interest`. The application executes until reaching the breakpoint at line 13 (Fig. F.27).
4. *Continuing execution.* Type `cont` to continue—the application executes line 13, printing the column headings `"Year"` and `"Amount on deposit"`. Line 13 appears before the `for` statement at lines 16–23 in `Interest` (Fig. 5.6) and thus executes only once. Execution continues past line 13 until the breakpoint at line 22 is reached during the first iteration of the `for` statement (Fig. F.28).
5. *Examining variable values.* Type `print year` to examine the current value of variable `year` (i.e., the `for`'s control variable). Print the value of variable `amount` too (Fig. F.29).

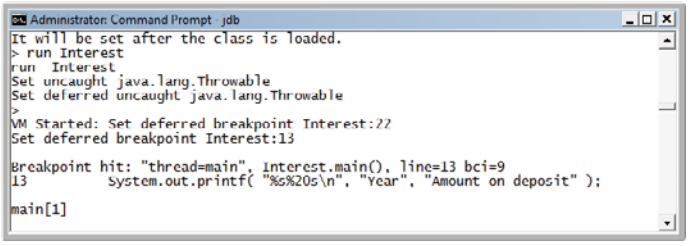


Fig. F.27 | Reaching the breakpoint at line 13 in the `Interest` application.

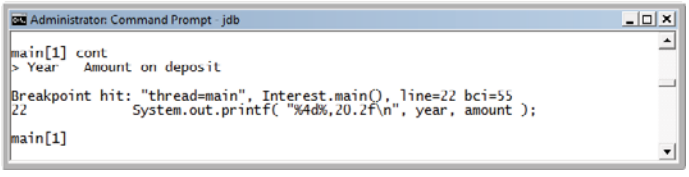


Fig. F.28 | Reaching the breakpoint at line 22 in the `Interest` application.

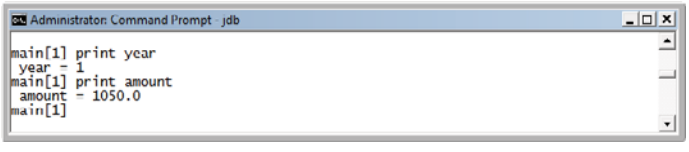


Fig. F.29 | Printing `year` and `amount` during the first iteration of `Interest`'s `for`.

6. ***Continuing execution.*** Type `cont` to continue execution. Line 22 executes and prints the current values of `year` and `amount`. After the `for` enters its second iteration, the debugger notifies you that the breakpoint at line 22 has been reached a second time. The debugger pauses each time a line where a breakpoint has been set is about to execute—when the breakpoint appears in a loop, the debugger pauses during each iteration. Print the values of variables `year` and `amount` again to see how the values have changed since the first iteration of the `for` (Fig. F.30).

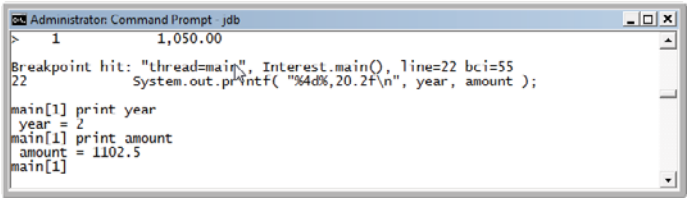


Fig. F.30 | Printing `year` and `amount` during the second iteration of `Interest`'s `for`.

7. ***Removing a breakpoint.*** You can display a list of all of the breakpoints in the application by typing **`clear`** (Fig. F.31). Suppose you're satisfied that the `Interest`

application's `for` statement is working properly, so you want to remove the breakpoint at line 22 and allow the remaining iterations of the loop to proceed normally. You can remove the breakpoint at line 22 by typing `clear Interest:22`. Now type `clear` to list the remaining breakpoints in the application. The debugger should indicate that only the breakpoint at line 13 remains (Fig. F.31). This breakpoint has already been reached and thus will no longer affect execution.

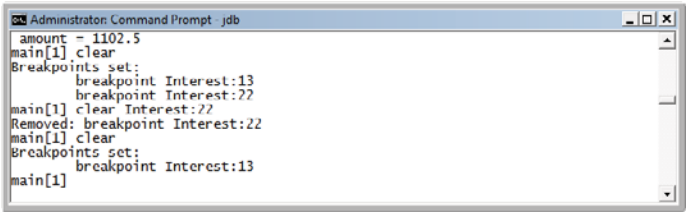


Fig. F.31 | Removing the breakpoint at line 22.

8. ***Continuing execution after removing a breakpoint.*** Type `cont` to continue execution. Recall that execution last paused before the `printf` statement in line 22. If the breakpoint at line 22 was removed successfully, continuing the application will produce the correct output for the current and remaining iterations of the `for` statement without the application halting (Fig. F.32).

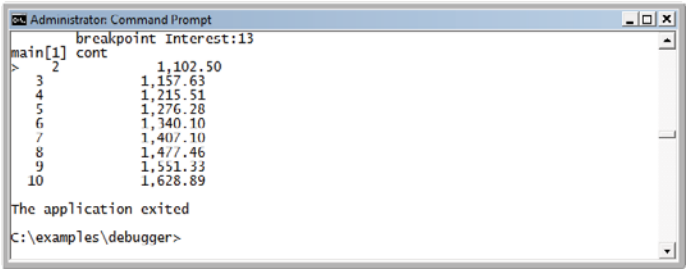


Fig. F.32 | Application executes without a breakpoint set at line 22.

F.7 Wrap-Up

In this appendix, you learned how to insert and remove breakpoints in the debugger. Breakpoints allow you to pause application execution so you can examine variable values with the debugger's `print` command. This capability will help you locate and fix logic errors in your applications. You saw how to use the `print` command to examine the value of an expression and how to use the `set` command to change the value of a variable. You also learned debugger commands (including the `step`, `step up` and `next` commands) that can be used to determine whether a method is executing correctly. You learned how to use the `watch` command to keep track of a field throughout the life of an application. Finally, you learned how to use the `clear` command to list all the breakpoints set for an application or remove individual breakpoints to continue execution without breakpoints.

Self-Review Exercises

F.1 Fill in the blanks in each of the following statements:

a) A breakpoint cannot be set at a(n) _____.

b) You can examine the value of an expression by using the debugger's _____ command.

c) You can modify the value of a variable by using the debugger's _____ command.

d) During debugging, the _____ command executes the remaining statements in the current method and returns program control to the place where the method was called.

e) The debugger's _____ command behaves like the `step` command when the next statement to execute does not contain a method call.

f) The `watch` debugger command allows you to view all changes to a(n) _____.

F.2 State whether each of the following is *true* or *false*. If *false*, explain why.

a) When application execution suspends at a breakpoint, the next statement to be executed is the statement after the breakpoint.

b) Watches can be removed using the debugger's `clear` command.

c) The `-g` compiler option must be used when compiling classes for debugging.

d) When a breakpoint appears in a loop, the debugger pauses only the first time that the breakpoint is encountered.

Answers to Self-Review Exercises

F.1 a) comment. b) `print`. c) `set`. d) `step up`. e) `next`. f) field.

F.2 a) False. When application execution suspends at a breakpoint, the next statement to be executed is the statement at the breakpoint. b) False. Watches can be removed using the debugger's `unwatch` command. c) True. d) False. When a breakpoint appears in a loop, the debugger pauses during each iteration.

G

Formatted Output



All the news that's fit to print.
—Adolph S. Ochs

What mad pursuit? What struggle to escape?
—John Keats

Remove not the landmark on the boundary of the fields.
—Amenehope

Objectives

In this appendix you'll learn:

- To understand input and output streams.
- To use `printf` formatting.
- To print with field widths and precisions.
- To use formatting flags in the `printf` format string.
- To print with an argument index.
- To output literals and escape sequences.
- To format output with class `Formatter`.

Outline

G.1 Introduction
G.2 Streams
G.3 Formatting Output with `printf`
G.4 Printing Integers
G.5 Printing Floating-Point Numbers
G.6 Printing Strings and Characters
G.7 Printing Dates and Times
G.8 Other Conversion Characters
G.9 Printing with Field Widths and Precisions
G.10 Using Flags in the `printf` Format String
G.11 Printing with Argument Indices
G.12 Printing Literals and Escape Sequences
G.13 Formatting Output with Class `Formatter`
G.14 Wrap-Up

Summary | *Self-Review Exercises* | *Answers to Self-Review Exercises* | *Exercises*

G.1 Introduction

In this appendix, we discuss the formatting features of method `printf` and class `Formatter` (package `java.util`). Class **`Formatter`** formats and outputs data to a specified destination, such as a string or a file output stream. Many features of `printf` were discussed earlier in the text. This appendix summarizes those features and introduces others, such as displaying date and time data in various formats, reordering output based on the index of the argument and displaying numbers and strings with various flags.

G.2 Streams

Input and output are usually performed with streams, which are sequences of bytes. In input operations, the bytes flow from a device (e.g., a keyboard, a disk drive, a network connection) to main memory. In output operations, bytes flow from main memory to a device (e.g., a display screen, a printer, a disk drive, a network connection).

When program execution begins, three streams are created. The standard input stream typically reads bytes from the keyboard, and the standard output stream typically outputs characters to a command window. A third stream, the **standard error stream** (`System.err`), typically outputs characters to a command window and is used to output error messages so they can be viewed immediately. Operating systems typically allow these streams to be redirected to other devices. Streams are discussed in detail in Chapter 17, Files, Streams and Object Serialization, and Chapter 27, Networking.

G.3 Formatting Output with `printf`

Precise output formatting is accomplished with `printf`. Java borrowed (and enhanced) this feature from the C programming language. Method `printf` can perform the following formatting capabilities, each of which is discussed in this appendix:

1. Rounding floating-point values to an indicated number of decimal places.
2. Aligning a column of numbers with decimal points appearing one above the other.
3. Right justification and left justification of outputs.

4. Inserting literal characters at precise locations in a line of output.
5. Representing floating-point numbers in exponential format.
6. Representing integers in octal and hexadecimal format.
7. Displaying all types of data with fixed-size field widths and precisions.
8. Displaying dates and times in various formats.

Every call to `printf` supplies as the first argument a **format string** that describes the output format. The format string may consist of **fixed text** and **format specifiers**. Fixed text is output by `printf` just as it would be output by `System.out` methods `print` or `println`. Each format specifier is a placeholder for a value and specifies the type of data to output. Format specifiers also may include optional formatting information.

In the simplest form, each format specifier begins with a percent sign (%) and is followed by a **conversion character** that represents the data type of the value to output. For example, the format specifier `%s` is a placeholder for a `string`, and the format specifier `%d` is a placeholder for an `int` value. The optional formatting information, such as an argument index, flags, field width and precision, is specified between the percent sign and the conversion character. We demonstrate each of these capabilities.

G.4 Printing Integers

Figure G.1 describes the **integer conversion characters**. (See Appendix H for an overview of the binary, octal, decimal and hexadecimal number systems.) Figure G.2 uses each to print an integer. In lines 9–10, the plus sign is not displayed by default, but the minus sign is. Later in this appendix (Fig. G.14) we'll see how to force plus signs to print.

Conversion character	Description
d	Display a decimal (base 10) integer.
o	Display an octal (base 8) integer.
x or **X**	Display a hexadecimal (base 16) integer. `X` uses uppercase letters.

Fig. G.1 | Integer conversion characters.

```
1  // Fig. G.2: IntegerConversionTest.java
2  // Using the integer conversion characters.
3
4  public class IntegerConversionTest
5  {
6     public static void main( String[] args )
7     {
8        System.out.printf( "%d\n", 26 );
9        System.out.printf( "%d\n", +26 );
10       System.out.printf( "%d\n", -26 );
11       System.out.printf( "%o\n", 26 );
12       System.out.printf( "%x\n", 26 );
```

Fig. G.2 | Using the integer conversion characters. (Part 1 of 2.)

```
13          System.out.printf( "%X\n", 26 );
14       } // end main
15    } // end class IntegerConversionTest
```

```
26
26
-26
32
1a
1A
```

Fig. G.2 | Using the integer conversion characters. (Part 2 of 2.)

The `printf` method has the form

```
printf( format-string, argument-list );
```

where *format-string* describes the output format, and the optional *argument-list* contains the values that correspond to each format specifier in *format-string*. There can be many format specifiers in one format string.

Each format string in lines 8–10 specifies that `printf` should output a decimal integer (`%d`) followed by a newline character. At the format specifier's position, `printf` substitutes the value of the first argument after the format string. If the format string contains multiple format specifiers, at each subsequent format specifier's position `printf` substitutes the value of the next argument in the argument list. The `%o` format specifier in line 11 outputs the integer in octal format. The `%x` format specifier in line 12 outputs the integer in hexadecimal format. The `%X` format specifier in line 13 outputs the integer in hexadecimal format with capital letters.

G.5 Printing Floating-Point Numbers

Figure G.3 describes the floating-point conversions. The **conversion characters `e`** and **`E`** display floating-point values in **computerized scientific notation** (also called **exponential notation**). Exponential notation is the computer equivalent of the scientific notation used in mathematics. For example, the value `150.4582` is represented in scientific notation in mathematics as

$$1.504582 \times 10^2$$

and is represented in exponential notation as

```
1.504582e+02
```

in Java. This notation indicates that `1.504582` is multiplied by `10` raised to the second power (`e+02`). The `e` stands for "exponent."

Values printed with the conversion characters `e`, `E` and `f` are output with six digits of precision to the right of the decimal point by default (e.g., `1.045921`)—other precisions must be specified explicitly. For values printed with the conversion character `g`, the precision represents the total number of digits displayed, excluding the exponent. The default is six digits (e.g., `12345678.9` is displayed as `1.23457e+07`). **Conversion character `f`** always prints at least one digit to the left of the decimal point. Conversion characters `e` and `E` print

Conversion character	Description
e or E	Display a floating-point value in exponential notation. Conversion character E displays the output in uppercase letters.
f	Display a floating-point value in decimal format.
g or G	Display a floating-point value in either the floating-point format f or the exponential format e based on the magnitude of the value. If the magnitude is less than 10^{-3}, or greater than or equal to 10^{7}, the floating-point value is printed with e (or E). Otherwise, the value is printed in format f. When conversion character G is used, the output is displayed in uppercase letters.
a or A	Display a floating-point number in hexadecimal format. Conversion character A displays the output in uppercase letters.

Fig. G.3 | Floating-point conversion characters.

lowercase e and uppercase E preceding the exponent and always print exactly one digit to the left of the decimal point. Rounding occurs if the value being formatted has more significant digits than the precision.

Conversion character g (or **G**) prints in either e (E) or f format, depending on the floating-point value. For example, the values 0.0000875, 87500000.0, 8.75, 87.50 and 875.0 are printed as 8.750000e-05, 8.750000e+07, 8.750000, 87.500000 and 875.000000 with the conversion character g. The value 0.0000875 uses e notation because the magnitude is less than 10-3. The value 87500000.0 uses e notation because the magnitude is greater than 10^{7}. Figure G.4 demonstrates the floating-point conversion characters.

```
1  // Fig. G.4: FloatingNumberTest.java
2  // Using floating-point conversion characters.
3
4  public class FloatingNumberTest
5  {
6     public static void main( String[] args )
7     {
8        System.out.printf( "%e\n", 12345678.9 );
9        System.out.printf( "%e\n", +12345678.9 );
10       System.out.printf( "%e\n", -12345678.9 );
11       System.out.printf( "%E\n", 12345678.9 );
12       System.out.printf( "%f\n", 12345678.9 );
13       System.out.printf( "%g\n", 12345678.9 );
14       System.out.printf( "%G\n", 12345678.9 );
15    } // end main
16 } // end class FloatingNumberTest
```

```
1.234568e+07
1.234568e+07
-1.234568e+07
```

Fig. G.4 | Using floating-point conversion characters. (Part 1 of 2.)

```
1.234568E+07
12345678.900000
1.23457e+07
1.23457E+07
```

Fig. G.4 | Using floating-point conversion characters. (Part 2 of 2.)

G.6 Printing Strings and Characters

The `c` and `s` conversion characters print individual characters and strings, respectively. **Conversion characters `c` and `C`** require a `char` argument. **Conversion characters `s` and `S`** can take a `String` or any `Object` as an argument. When conversion characters `C` and `S` are used, the output is displayed in uppercase letters. Figure G.5 displays characters, strings and objects with conversion characters `c` and `s`. Autoboxing occurs at line 9 when an `int` constant is assigned to an `Integer` object. Line 15 outputs an `Integer` argument with the conversion character `s`, which implicitly invokes the `toString` method to get the integer value. You can also output an `Integer` object using the `%d` format specifier. In this case, the `int` value in the `Integer` object will be unboxed and output.



Common Programming Error G.1

Using `%c` to print a `String` causes an `IllegalFormatConversionException`—a `String` cannot be converted to a character.

```
1  // Fig. G.5: CharStringConversion.java
2  // Using character and string conversion characters.
3  public class CharStringConversion
4  {
5     public static void main( String[] args )
6     {
7        char character = 'A';  // initialize char
8        String string = "This is also a string";  // String object
9        Integer integer = 1234;  // initialize integer (autoboxing)
10
11       System.out.printf( "%c\n", character );
12       System.out.printf( "%s\n", "This is a string" );
13       System.out.printf( "%s\n", string );
14       System.out.printf( "%S\n", string );
15       System.out.printf( "%s\n", integer ); // implicit call to toString
16    } // end main
17 } // end class CharStringConversion
```

```
A
This is a string
This is also a string
THIS IS ALSO A STRING
1234
```

Fig. G.5 | Using character and string conversion characters.

G.7 Printing Dates and Times

The **conversion character** `t` (or `T`) is used to print dates and times in various formats. It's always followed by a **conversion suffix character** that specifies the date and/or time format. When conversion character `T` is used, the output is displayed in uppercase letters. Figure G.6 lists the common conversion suffix characters for formatting **date and time compositions** that display both the date and the time. Figure G.7 lists the common conversion suffix characters for formatting dates. Figure G.8 lists the common conversion suffix characters for formatting times. For the complete list of conversion suffix characters, visit `java.sun.com/javase/6/docs/api/java/util/Formatter.html`.

Conversion suffix character	Description
`c`	Display date and time formatted as `day month date hour:minute:second time-zone year` with three characters for `day` and `month`, two digits for `date`, `hour`, `minute` and `second` and four digits for `year`—for example, `Wed Mar 03 16:30:25 GMT-05:00 2004`. The 24-hour clock is used. `GMT-05:00` is the time zone.
`F`	Display date formatted as `year-month-date` with four digits for the `year` and two digits each for the `month` and `date` (e.g., `2004-05-04`).
`D`	Display date formatted as `month/day/year` with two digits each for the `month`, `day` and `year` (e.g., `03/03/04`).
`r`	Display time in 12-hour format as `hour:minute:second AM|PM` with two digits each for the `hour`, `minute` and `second` (e.g., `04:30:25 PM`).
`R`	Display time formatted as `hour:minute` with two digits each for the `hour` and `minute` (e.g., `16:30`). The 24-hour clock is used.
`T`	Display time as `hour:minute:second` with two digits for the `hour`, `minute` and `second` (e.g., `16:30:25`). The 24-hour clock is used.

Fig. G.6 | Date and time composition conversion suffix characters.

Conversion suffix character	Description
`A`	Display full name of the day of the week (e.g., `Wednesday`).
`a`	Display the three-character name of the day of the week (e.g., `Wed`).
`B`	Display full name of the month (e.g., `March`).
`b`	Display the three-character short name of the month (e.g., `Mar`).
`d`	Display the day of the month with two digits, padding with leading zeros as necessary (e.g., `03`).
`m`	Display the month with two digits, padding with leading zeros as necessary (e.g., `07`).

Fig. G.7 | Date formatting conversion suffix characters. (Part 1 of 2.)

Conversion suffix character	Description
`e`	Display the day of month without leading zeros (e.g., 3).
`Y`	Display the year with four digits (e.g., 2004).
`y`	Display the last two digits of the year with leading zeros (e.g., 04).
`j`	Display the day of the year with three digits, padding with leading zeros as necessary (e.g., 016).

Fig. G.7 | Date formatting conversion suffix characters. (Part 2 of 2.)

Conversion suffix character	Description
`H`	Display hour in 24-hour clock with a leading zero as necessary (e.g., 16).
`I`	Display hour in 12-hour clock with a leading zero as necessary (e.g., 04).
`k`	Display hour in 24-hour clock without leading zeros (e.g., 16).
`l`	Display hour in 12-hour clock without leading zeros (e.g., 4).
`M`	Display minute with a leading zero as necessary (e.g., 06).
`S`	Display second with a leading zero as necessary (e.g., 05).
`Z`	Display the abbreviation for the time zone (e.g., EST, stands for Eastern Standard Time, which is 5 hours behind Greenwich Mean Time).
`p`	Display morning or afternoon marker in lowercase (e.g., pm).
`P`	Display morning or afternoon marker in uppercase (e.g., PM).

Fig. G.8 | Time formatting conversion suffix characters.

Figure G.9 uses the conversion characters `t` and `T` with the conversion suffix characters to display dates and times in various formats. Conversion character `t` requires the corresponding argument to be a date or time of type `long`, `Long`, **`Calendar`** (package `java.util`) or **`Date`** (package `java.util`)—objects of each of these classes can represent dates and times. Class `Calendar` is preferred for this purpose because some constructors

```
1  // Fig. G.9: DateTimeTest.java
2  // Formatting dates and times with conversion characters t and T.
3  import java.util.Calendar;
4
5  public class DateTimeTest
6  {
7     public static void main( String[] args )
8     {
9        // get current date and time
10       Calendar dateTime = Calendar.getInstance();
```

Fig. G.9 | Formatting dates and times with conversion characters `t` and `T`. (Part 1 of 2.)

```
11
12         // printing with conversion characters for date/time compositions
13         System.out.printf( "%tc\n", dateTime );
14         System.out.printf( "%tF\n", dateTime );
15         System.out.printf( "%tD\n", dateTime );
16         System.out.printf( "%tr\n", dateTime );
17         System.out.printf( "%tT\n", dateTime );
18
19         // printing with conversion characters for date
20         System.out.printf( "%1$tA, %1$tB %1$td, %1$tY\n", dateTime );
21         System.out.printf( "%1$TA, %1$TB %1$Td, %1$TY\n", dateTime );
22         System.out.printf( "%1$ta, %1$tb %1$te, %1$ty\n", dateTime );
23
24         // printing with conversion characters for time
25         System.out.printf( "%1$tH:%1$tM:%1$tS\n", dateTime );
26         System.out.printf( "%1$tZ %1$tI:%1$tM:%1$tS %tP", dateTime );
27      } // end main
28   } // end class DateTimeTest
```

```
Wed Feb 25 15:00:22 EST 2009
2009-02-25
02/25/09
03:00:22 PM
15:00:22
Wednesday, February 25, 2009
WEDNESDAY, FEBRUARY 25, 2009
Wed, Feb 25, 09
15:00:22
EST 03:00:22 PM
```

Fig. G.9 | Formatting dates and times with conversion characters `t` and `T`. (Part 2 of 2.)

and methods in class `Date` are replaced by those in class `Calendar`. Line 10 invokes `static` method **`getInstance`** of `Calendar` to obtain a calendar with the current date and time. Lines 13–17, 20–22 and 25–26 use this `Calendar` object in `printf` statements as the value to be formatted with conversion character `t`. Lines 20–22 and 25–26 use the optional **argument index** (`"1$"`) to indicate that all format specifiers in the format string use the first argument after the format string in the argument list. You'll learn more about argument indices in Section G.11. Using the argument index eliminates the need to repeatedly list the same argument.

G.8 Other Conversion Characters

The remaining conversion characters are **`b`**, **`B`**, **`h`**, **`H`**, **`%`** and **`n`**. These are described in Fig. G.10. Lines 9–10 of Fig. G.11 use `%b` to print the value of `boolean` (or `Boolean`) values `false` and `true`. Line 11 associates a `String` to `%b`, which returns `true` because it's not `null`. Line 12 associates a `null` object to `%B`, which displays `FALSE` because `test` is `null`. Lines 13–14 use `%h` to print the string representations of the hash-code values for strings `"hello"` and `"Hello"`. These values could be used to store or locate the strings in a `Hashtable` or `HashMap` (both discussed in Chapter 20, Generic Collections). The hash-code values for these two strings differ, because one string starts with a lowercase letter and

the other with an uppercase letter. Line 15 uses `%H` to print `null` in uppercase letters. The last two `printf` statements (lines 16–17) use `%%` to print the `%` character in a string and `%n` to print a platform-specific line separator.

Conversion character	Description
`b` or `B`	Print `"true"` or `"false"` for the value of a `boolean` or `Boolean`. These conversion characters can also format the value of any reference. If the reference is non-`null`, `"true"` is output; otherwise, `"false"`. When conversion character `B` is used, the output is displayed in uppercase letters.
`h` or `H`	Print the string representation of an object's hash-code value in hexadecimal format. If the corresponding argument is `null`, `"null"` is printed. When conversion character `H` is used, the output is displayed in uppercase letters.
`%`	Print the percent character.
`n`	Print the platform-specific line separator (e.g., `\r\n` on Windows or `\n` on UNIX/LINUX).

Fig. G.10 | Other conversion characters.

```
1  // Fig. G.11: OtherConversion.java
2  // Using the b, B, h, H, % and n conversion characters.
3
4  public class OtherConversion
5  {
6     public static void main( String[] args )
7     {
8        Object test = null;
9        System.out.printf( "%b\n", false );
10       System.out.printf( "%b\n", true );
11       System.out.printf( "%b\n", "Test" );
12       System.out.printf( "%B\n", test );
13       System.out.printf( "Hashcode of \"hello\" is %h\n", "hello" );
14       System.out.printf( "Hashcode of \"Hello\" is %h\n", "Hello" );
15       System.out.printf( "Hashcode of null is %H\n", test );
16       System.out.printf( "Printing a %% in a format string\n" );
17       System.out.printf( "Printing a new line %nnext line starts here" );
18    } // end main
19 } // end class OtherConversion
```

```
false
true
true
FALSE
Hashcode of "hello" is 5e918d2
```

Fig. G.11 | Using the `b`, `B`, `h`, `H`, `%` and `n` conversion characters. (Part 1 of 2.)

```
Hashcode of "Hello" is 42628b2
Hashcode of null is NULL
Printing a % in a format string
Printing a new line
next line starts here
```

Fig. G.11 | Using the `b`, `B`, `h`, `H`, `%` and `n` conversion characters. (Part 2 of 2.)



Common Programming Error G.2

Trying to print a literal percent character using `%` rather than `%%` in the format string might cause a difficult-to-detect logic error. When `%` appears in a format string, it must be followed by a conversion character in the string. The single percent could accidentally be followed by a legitimate conversion character, thus causing a logic error.

G.9 Printing with Field Widths and Precisions

The size of a field in which data is printed is specified by a **field width**. If the field width is larger than the data being printed, the data is right justified in that field by default. We discuss left justification in Section G.10. You insert an integer representing the field width between the `%` and the conversion character (e.g., `%4d`) in the format specifier. Figure G.12 prints two groups of five numbers each, right justifying those numbers that contain fewer digits than the field width. The field width is increased to print values wider than the field and that the minus sign for a negative value uses one character position in the field. Also, if no field width is specified, the data prints in exactly as many positions as it needs. Field widths can be used with all format specifiers except the line separator (`%n`).

```
1  // Fig. G.12: FieldWidthTest.java
2  // Right justifying integers in fields.
3
4  public class FieldWidthTest
5  {
6     public static void main( String[] args )
7     {
8        System.out.printf( "%4d\n", 1 );
9        System.out.printf( "%4d\n", 12 );
10       System.out.printf( "%4d\n", 123 );
11       System.out.printf( "%4d\n", 1234 );
12       System.out.printf( "%4d\n\n", 12345 ); // data too large
13
14       System.out.printf( "%4d\n", -1 );
15       System.out.printf( "%4d\n", -12 );
16       System.out.printf( "%4d\n", -123 );
17       System.out.printf( "%4d\n", -1234 ); // data too large
18       System.out.printf( "%4d\n", -12345 ); // data too large
19    } // end main
20 } // end class RightJustifyTest
```

Fig. G.12 | Right justifying integers in fields. (Part 1 of 2.)

```
   1
  12
 123
1234
12345

  -1
 -12
-123
-1234
-12345
```

Fig. G.12 | Right justifying integers in fields. (Part 2 of 2.)



Common Programming Error G.3

Not providing a sufficiently large field width to handle a value to be printed can offset other data being printed and produce confusing outputs. Know your data!

Method `printf` also provides the ability to specify the precision with which data is printed. Precision has different meanings for different types. When used with floating-point conversion characters `e` and `f`, the precision is the number of digits that appear after the decimal point. When used with conversion characters `g`, `a` or `A`, the precision is the maximum number of significant digits to be printed. When used with conversion character `s`, the precision is the maximum number of characters to be written from the string. To use precision, place between the percent sign and the conversion specifier a decimal point (`.`) followed by an integer representing the precision. Figure G.13 demonstrates the use of precision in format strings. When a floating-point value is printed with a precision smaller than the original number of decimal places in the value, the value is rounded. Also, the format specifier `%.3g` indicates that the total number of digits used to display the floating-point value is 3. Because the value has three digits to the left of the decimal point, the value is rounded to the ones position.

The field width and the precision can be combined by placing the field width, followed by a decimal point, followed by a precision between the percent sign and the conversion character, as in the statement

```
printf( "%9.3f", 123.456789 );
```

which displays `123.457` with three digits to the right of the decimal point right justified in a nine-digit field—this number will be preceded in its field by two blanks.

```
1  // Fig. G.13: PrecisionTest.java
2  // Using precision for floating-point numbers and strings.
3  public class PrecisionTest
4  {
5     public static void main( String[] args )
6     {
7        double f = 123.94536;
8        String s = "Happy Birthday";
```

Fig. G.13 | Using precision for floating-point numbers and strings. (Part 1 of 2.)

```
9
10         System.out.printf( "Using precision for floating-point numbers\n" );
11         System.out.printf( "\t%.3f\n\t%.3e\n\t%.3g\n\n", f, f, f );
12
13         System.out.printf( "Using precision for strings\n" );
14         System.out.printf( "\t%.11s\n", s );
15      } // end main
16   } // end class PrecisionTest
```

```
Using precision for floating-point numbers
        123.945
        1.239e+02
        124

Using precision for strings
        Happy Birth
```

Fig. G.13 | Using precision for floating-point numbers and strings. (Part 2 of 2.)

G.10 Using Flags in the printf Format String

Various flags may be used with method `printf` to supplement its output formatting capabilities. Seven flags are available for use in format strings (Fig. G.14).

Flag	Description
- (minus sign)	Left justify the output within the specified field.
+ (plus sign)	Display a plus sign preceding positive values and a minus sign preceding negative values.
space	Print a space before a positive value not printed with the + flag.
#	Prefix `0` to the output value when used with the octal conversion character `o`. Prefix `0x` to the output value when used with the hexadecimal conversion character `x`.
0 (zero)	Pad a field with leading zeros.
, (comma)	Use the locale-specific thousands separator (i.e., `','` for U.S. locale) to display decimal and floating-point numbers.
(	Enclose negative numbers in parentheses.

Fig. G.14 | Format string flags.

To use a flag in a format string, place it immediately to the right of the percent sign. Several flags may be used in the same format specifier. Figure G.15 demonstrates right justification and left justification of a string, an integer, a character and a floating-point number. Line 9 serves as a counting mechanism for the screen output.

Figure G.16 prints a positive number and a negative number, each with and without the **+ flag**. The minus sign is displayed in both cases, the plus sign only when the + flag is used.

```
1  // Fig. G.15: MinusFlagTest.java
2  // Right justifying and left justifying values.
3
4  public class MinusFlagTest
5  {
6     public static void main( String[] args )
7     {
8        System.out.println( "Columns:" );
9        System.out.println( "01234567890123456789012345678901234567890123456789\n" );
10       System.out.printf( "%10s%10d%10c%10f\n\n", "hello", 7, 'a', 1.23 );
11       System.out.printf(
12          "%-10s%-10d%-10c%-10f\n", "hello", 7, 'a', 1.23 );
13    } // end main
14 } // end class MinusFlagTest
```

```
Columns:
01234567890123456789012345678901234567890123456789

     hello         7         a  1.230000

hello     7         a         1.230000
```

Fig. G.15 | Right justifying and left justifying values.

```
1  // Fig. G.16: PlusFlagTest.java
2  // Printing numbers with and without the + flag.
3
4  public class PlusFlagTest
5  {
6     public static void main( String[] args )
7     {
8        System.out.printf( "%d\t%d\n", 786, -786 );
9        System.out.printf( "%+d\t%+d\n", 786, -786 );
10    } // end main
11 } // end class PlusFlagTest
```

```
786     -786
+786    -786
```

Fig. G.16 | Printing numbers with and without the + flag.

Figure G.17 prefixes a space to the positive number with the **space flag**. This is useful for aligning positive and negative numbers with the same number of digits. The value -547 is not preceded by a space in the output because of its minus sign. Figure G.18 uses the **# flag** to prefix 0 to the octal value and 0x to the hexadecimal value.

```
1  // Fig. G.17: SpaceFlagTest.java
2  // Printing a space before non-negative values.
3
```

Fig. G.17 | Printing a space before nonnegative values. (Part 1 of 2.)

```
4  public class SpaceFlagTest
5  {
6     public static void main( String[] args )
7     {
8        System.out.printf( "% d\n% d\n", 547, -547 );
9     } // end main
10 } // end class SpaceFlagTest
```

```
 547
-547
```

Fig. G.17 | Printing a space before nonnegative values. (Part 2 of 2.)

```
1  // Fig. G.18: PoundFlagTest.java
2  // Using the # flag with conversion characters o and x.
3
4  public class PoundFlagTest
5  {
6     public static void main( String[] args )
7     {
8        int c = 31;       // initialize c
9
10       System.out.printf( "%#o\n", c );
11       System.out.printf( "%#x\n", c );
12    } // end main
13 } // end class PoundFlagTest
```

```
037
0x1f
```

Fig. G.18 | Using the # flag with conversion characters o and x.

Figure G.19 combines the + flag the **0 flag** and the space flag to print 452 in a field of width 9 with a + sign and leading zeros, next prints 452 in a field of width 9 using only the 0 flag, then prints 452 in a field of width 9 using only the space flag.

```
1  // Fig. G.19: ZeroFlagTest.java
2  // Printing with the 0 (zero) flag fills in leading zeros.
3
4  public class ZeroFlagTest
5  {
6     public static void main( String[] args )
7     {
8        System.out.printf( "%+09d\n", 452 );
9        System.out.printf( "%09d\n", 452 );
10       System.out.printf( "% 9d\n", 452 );
11    } // end main
12 } // end class ZeroFlagTest
```

Fig. G.19 | Printing with the 0 (zero) flag fills in leading zeros. (Part 1 of 2.)

```
+00000452
000000452
      452
```

Fig. G.19 | Printing with the 0 (zero) flag fills in leading zeros. (Part 2 of 2.)

Figure G.20 uses the comma (,) flag to display a decimal and a floating-point number with the thousands separator. Figure G.21 encloses negative numbers in parentheses using the (flag. The value 50 is not enclosed in parentheses in the output because it's a positive number.

```
1  // Fig. G.20: CommaFlagTest.java
2  // Using the comma (,) flag to display numbers with thousands separator.
3
4  public class CommaFlagTest
5  {
6     public static void main( String[] args )
7     {
8        System.out.printf( "%,d\n", 58625 );
9        System.out.printf( "%,.2f", 58625.21 );
10       System.out.printf( "%,.2f", 12345678.9 );
11    } // end main
12 } // end class CommaFlagTest
```

```
58,625
58,625.21
12,345,678.90
```

Fig. G.20 | Using the comma (,) flag to display numbers with the thousands separator.

```
1  // Fig. G.21: ParenthesesFlagTest.java
2  // Using the ( flag to place parentheses around negative numbers.
3
4  public class ParenthesesFlagTest
5  {
6     public static void main( String[] args )
7     {
8        System.out.printf( "%(d\n", 50 );
9        System.out.printf( "%(d\n", -50 );
10       System.out.printf( "%(.1e\n", -50.0 );
11    } // end main
12 } // end class ParenthesesFlagTest
```

```
50
(50)
(5.0e+01)
```

Fig. G.21 | Using the (flag to place parentheses around negative numbers.

G.11 Printing with Argument Indices

An **argument index** is an optional integer followed by a `$` sign that indicates the argument's position in the argument list. For example, lines 20–22 and 25–26 in Fig. G.9 use argument index `"1$"` to indicate that all format specifiers use the first argument in the argument list. Argument indices enable programmers to reorder the output so that the arguments in the argument list are not necessarily in the order of their corresponding format specifiers. Argument indices also help avoid duplicating arguments. Figure G.22 prints arguments in the argument list in reverse order using the argument index.

```
1  // Fig. G.22: ArgumentIndexTest
2  // Reordering output with argument indices.
3
4  public class ArgumentIndexTest
5  {
6     public static void main( String[] args )
7     {
8        System.out.printf(
9           "Parameter list without reordering: %s %s %s %s\n",
10          "first", "second", "third", "fourth" );
11       System.out.printf(
12          "Parameter list after reordering: %4$s %3$s %2$s %1$s\n",
13          "first", "second", "third", "fourth" );
14    } // end main
15 } // end class ArgumentIndexTest
```

```
Parameter list without reordering: first second third fourth
Parameter list after reordering: fourth third second first
```

Fig. G.22 | Reordering output with argument indices.

G.12 Printing Literals and Escape Sequences

Most literal characters to be printed in a `printf` statement can simply be included in the format string. However, there are several "problem" characters, such as the quotation mark (") that delimits the format string itself. Various control characters, such as newline and tab, must be represented by escape sequences. An escape sequence is represented by a backslash (\), followed by an escape character. Figure G.23 lists the escape sequences and the actions they cause.

Escape sequence	Description
\' (single quote)	Output the single quote (') character.
\" (double quote)	Output the double quote (") character.
\\ (backslash)	Output the backslash (\) character.
\b (backspace)	Move the cursor back one position on the current line.

Fig. G.23 | Escape sequences. (Part 1 of 2.)

Escape sequence	Description
\f (new page or form feed)	Move the cursor to the start of the next logical page.
\n (newline)	Move the cursor to the beginning of the next line.
\r (carriage return)	Move the cursor to the beginning of the current line.
\t (horizontal tab)	Move the cursor to the next horizontal tab position.

Fig. G.23 | Escape sequences. (Part 2 of 2.)



Common Programming Error G.4

Attempting to print as literal data in a `printf` *statement a double quote or backslash character without preceding that character with a backslash to form a proper escape sequence might result in a syntax error.*

G.13 Formatting Output with Class `Formatter`

So far, we've discussed displaying formatted output to the standard output stream. What should we do if we want to send formatted outputs to other output streams or devices, such as a `JTextArea` or a file? The solution relies on class `Formatter` (in package `java.util`), which provides the same formatting capabilities as `printf`. `Formatter` is a utility class that enables programmers to output formatted data to a specified destination, such as a file on disk. By default, a `Formatter` creates a string in memory. Figure G.24 demonstrates how to use a `Formatter` to build a formatted string, which is then displayed in a message dialog.

Line 11 creates a `Formatter` object using the default constructor, so this object will build a string in memory. Other constructors are provided to allow you to specify the destination to which the formatted data should be output. For details, see `java.sun.com/javase/6/docs/api/java/util/Formatter.html`.

```
1  // Fig. Fig. G.24: FormatterTest.java
2  // Formatting output with class Formatter.
3  import java.util.Formatter;
4  import javax.swing.JOptionPane;
5
6  public class FormatterTest
7  {
8     public static void main( String[] args )
9     {
10       // create Formatter and format output
11       Formatter formatter = new Formatter();
12       formatter.format( "%d = %#o = %#X", 10, 10, 10 );
13
14       // display output in JOptionPane
15       JOptionPane.showMessageDialog( null, formatter.toString() );
16    } // end main
17 } // end class FormatterTest
```

Fig. G.24 | Formatting output with class `Formatter`. (Part 1 of 2.)

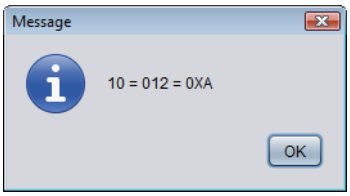


Fig. G.24 | Formatting output with class `Formatter`. (Part 2 of 2.)

Line 12 invokes method **`format`** to format the output. Like `printf`, method `format` takes a format string and an argument list. The difference is that `printf` sends the formatted output directly to the standard output stream, while `format` sends the formatted output to the destination specified by its constructor (a string in memory in this program). Line 15 invokes the `Formatter`'s `toString` method to get the formatted data as a string, which is then displayed in a message dialog.

Class `String` also provides a `static` convenience method named `format` that enables you to create a string in memory without the need to first create a `Formatter` object. Lines 11–12 and line 15 in Fig. G.24 could have been replaced by

```
String s = String.format( "%d = %#o = %#x", 10, 10, 10 );
JOptionPane.showMessageDialog( null, s );
```

G.14 Wrap-Up

This appendix summarized how to display formatted output with various format characters and flags. We displayed decimal numbers using format characters `d`, `o`, `x` and `X`; floating-point numbers using format characters `e`, `E`, `f`, `g` and `G`; and dates and times in various format using format characters `t` and `T` and their conversion suffix characters. You learned how to display output with field widths and precisions. We introduced the flags `+`, `-`, space, `#`, `0`, comma and `(` that are used together with the format characters to produce output. We also demonstrated how to format output with class `Formatter`.

Summary

Section G.2 Streams

- Input and output are usually performed with streams, which are sequences of bytes.
- Normally, the standard input stream is connected to the keyboard, and the standard output stream is connected to the computer screen.

Section G.3 Formatting Output with `printf`

- The `printf` format string describes the formats in which the output values appear. The format specifier consists of argument index, flags, field widths, precisions and conversion characters.

Section G.4 Printing Integers

- Integers are printed with the conversion characters `d` for decimal integers, `o` for integers in octal form and `x` (or `X`) for integers in hexadecimal form. `X` displays uppercase letters.

Section G.5 Printing Floating-Point Numbers

- Floating-point values are printed with the conversion characters `e` (or `E`) for exponential notation, `f` for regular floating-point notation, and `g` (or `G`) for either `e` (or `E`) notation or `f` notation. For the `g` conversion specifier, the `e` conversion character is used if the value is less than 10^{-3} or greater than or equal to 10^{7}; otherwise, the `f` conversion character is used.

Section G.6 Printing Strings and Characters

- The conversion character `c` prints a character.
- The conversion character `s` (or `S`) prints a string of characters. Conversion character `S` displays the output in uppercase letters.

Section G.7 Printing Dates and Times

- The conversion character `t` (or `T`) followed by a conversion suffix character prints the date and time in various forms. Conversion character `T` displays the output in uppercase letters.
- Conversion character `t` (or `T`) requires the argument to be of type `long`, `Long`, `Calendar` or `Date`.

Section G.8 Other Conversion Characters

- Conversion character `b` (or `B`) outputs the string representation of a `boolean` or `Boolean`. These conversion characters also output `"true"` for non-`null` references and `"false"` for `null` references. Conversion character `B` outputs uppercase letters.
- Conversion character `h` (or `H`) returns `null` for a `null` reference and a `String` representation of the hash-code value (in base 16) of an object . Hash codes are used to store and retrieve objects in `Hashtables` and `HashMaps`. Conversion character `H` outputs uppercase letters.
- The conversion character `n` prints the platform-specific line separator.
- The conversion character `%` is used to display a literal `%`.

Section G.9 Printing with Field Widths and Precisions

- If the field width is larger than the object being printed, the object is right justified in the field.
- Field widths can be used with all conversion characters except the line-separator conversion.
- Precision used with floating-point conversion characters `e` and `f` indicates the number of digits that appear after the decimal point. Precision used with floating-point conversion character `g` indicates the number of significant digits to appear.
- Precision used with conversion character `s` indicates the number of characters to be printed.
- The field width and the precision can be combined by placing the field width, followed by a decimal point, followed by the precision between the percent sign and the conversion character.

Section G.10 Using Flags in the `printf` *Format String*

- The `-` flag left justifies its argument in a field.
- The `+` flag prints a plus sign for positive values and a minus sign for negative values.
- The space flag prints a space preceding a positive value. The space flag and the `+` flag cannot be used together in an integer conversion character.
- The `#` flag prefixes `0` to octal values and `0x` to hexadecimal values.
- The `0` flag prints leading zeros for a value that does not occupy its entire field.
- The comma `(,)` flag uses the locale-specific thousands separator to display integer and floating-point numbers.
- The `(` flag encloses a negative number in parentheses.

Section G.11 Printing with Argument Indices

- An argument index is an optional decimal integer followed by a `$` sign that indicates the position of the argument in the argument list.
- Argument indices enable programmers to reorder the output so that the arguments in the argument list are not necessarily in the order of their corresponding format specifiers. Argument indices also help avoid duplicating arguments.

Section G.13 Formatting Output with Class `Formatter`

- Class `Formatter` (in package `java.util`) provides the same formatting capabilities as `printf`. `Formatter` is a utility class that enables programmers to print formatted output to various destinations, including GUI components, files and other output streams.
- `Formatter` method `format` outputs formatted data to the destination specified by the `Formatter` constructor.
- `String` `static` method `format` formats data and returns the formatted data as a `String`.

Self-Review Exercises

G.1 Fill in the blanks in each of the following:

a) All input and output is dealt with in the form of _____.
b) The _____ stream is normally connected to the keyboard.
c) The _____ stream is normally connected to the computer screen.
d) `System.out`'s _____ method can be used to format text that is displayed on the standard output.
e) The conversion character _____ may be used to output a decimal integer.
f) The conversion characters _____ and _____ are used to display integers in octal and hexadecimal form, respectively.
g) The conversion character _____ is used to display a floating-point value in exponential notation.
h) The conversion characters `e` and `f` are displayed with _____ digits of precision to the right of the decimal point if no precision is specified.
i) The conversion characters _____ and _____ are used to print strings and characters, respectively.
j) The conversion character _____ and conversion suffix character _____ are used to print time for the 24-hour clock as `hour:minute:second`.
k) The _____ flag causes output to be left justified in a field.
l) The _____ flag causes values to be displayed with either a plus sign or a minus sign.
m) The argument index _____ corresponds to the second argument in the argument list.
n) Class _____ has the same capability as `printf`, but allows programmers to print formatted output to various destinations besides the standard output stream.

G.2 Find the error in each of the following and explain how it can be corrected.

a) The following statement should print the character `'c'`.
```
System.out.printf( "%c\n", "c" );
```
b) The following statement should print `9.375%`.
```
System.out.printf( "%.3f%", 9.375 );
```
c) The following statement should print the third argument in the argument list.
```
System.out.printf( "%2$s\n", "Mon", "Tue", "Wed", "Thu", "Fri" );
```
d)
```
System.out.printf( ""A string in quotes"" );
```
e)
```
System.out.printf( %d %d, 12, 20 );
```
f)
```
System.out.printf( "%s\n", 'Richard' );
```

G.3 Write a statement for each of the following:

a) Print `1234` right justified in a 10-digit field.
b) Print `123.456789` in exponential notation with a sign (+ or -) and 3 digits of precision.
c) Print `100` in octal form preceded by `0`.
d) Given a `Calendar` object `calendar`, print a date formatted as month/day/year (each with two digits).
e) Given a `Calendar` object `calendar`, print a time for the 24-hour clock as `hour:minute:second` (each with two digits) using argument index and conversion suffix characters for formatting time.
f) Print `3.333333` with a sign (+ or -) in a field of 20 characters with a precision of 3.

Answers to Self-Review Exercises

G.1 a) Streams. b) Standard input. c) Standard output. d) `printf`. e) `d`. f) `o`, `x` or `X`. g) `e` or `E`. h) 6. i) `s` or `S`, `c` or `C`. j) `t`, `T`. k) - (minus). l) + (plus). m) `2$`. n) `Formatter`.

G.2 a) Error: Conversion character `c` expects an argument of primitive type `char`.
Correction: To print the character `'c'`, change `"c"` to `'c'`.
b) Error: Trying to print the literal character `%` without using the format specifier `%%`.
Correction: Use `%%` to print a literal `%` character.
c) Error: Argument index does not start with 0; e.g., the first argument is `1$`.
Correction: To print the third argument use `3$`.
d) Error: Trying to print the literal character " without using the \" escape sequence.
Correction: Replace each quote in the inner set of quotes with \".
e) Error: The format string is not enclosed in double quotes.
Correction: Enclose `%d %d` in double quotes.
f) Error: The string to be printed is enclosed in single quotes.
Correction: Use double quotes instead of single quotes to represent a string.

G.3

```
a) System.out.printf( "%10d\n", 1234 );
b) System.out.printf( "%+.3e\n", 123.456789 );
c) System.out.printf( "%#o\n", 100 );
d) System.out.printf( "%tD\n", calendar );
e) System.out.printf( "%1$tH:%1$tM:%1$tS\n", calendar );
f) System.out.printf( "%+20.3f\n", 3.333333 );
```

Exercises

G.4 Write statement(s) for each of the following:

a) Print integer `40000` right justified in a 15-digit field.
b) Print `200` with and without a sign.
c) Print `100` in hexadecimal form preceded by `0x`.
d) Print `1.234` with three digits of precision in a 9-digit field with preceding zeros.

G.5 Show what is printed by each of the following statements. If a statement is incorrect, indicate why.

```
a) System.out.printf( "%-10d\n", 10000 );
b) System.out.printf( "%c\n", "This is a string" );
c) System.out.printf( "%8.3f\n", 1024.987654 );
d) System.out.printf( "%#o\n%#X\n", 17, 17 );
e) System.out.printf( "% d\n%+d\n", 1000000, 1000000 );
f) System.out.printf( "%10.2e\n", 444.93738 );
g) System.out.printf( "%d\n", 10.987 );
```

G.6 Find the error(s) in each of the following program segments. Show the corrected statement.

a) `System.out.printf( "%s\n", 'Happy Birthday' );`

b) `System.out.printf( "%c\n", 'Hello' );`

c) `System.out.printf( "%c\n", "This is a string" );`

d) The following statement should print `"Bon Voyage"` with the double quotes:
```
System.out.printf( ""%s"", "Bon Voyage" );
```

e) The following statement should print "Today is Friday":
```
System.out.printf( "Today is %s\n", "Monday", "Friday" );
```

f) `System.out.printf( 'Enter your name: ' );`

g) `System.out.printf( %f, 123.456 );`

h) The following statement should print the current time in the format `"hh:mm:ss"`:
```
Calendar dateTime = Calendar.getInstance();
System.out.printf( "%1$tk:1$%tl:%1$tS\n", dateTime );
```

G.7 ***(Printing Dates and Times)*** Write a program that prints dates and times in the following forms:

```
GMT-05:00 04/30/04 09:55:09 AM
GMT-05:00 April 30 2004 09:55:09
2004-04-30 day-of-the-month:30
2004-04-30 day-of-the-year:121
Fri Apr 30 09:55:09 GMT-05:00 2004
```

[*Note:* Depending on your location, you may get a time zone other than GMT-05:00.]

G.8 Write a program to test the results of printing the integer value `12345` and the floating-point value `1.2345` in fields of various sizes.

G.9 ***(Rounding Numbers)*** Write a program that prints the value `100.453627` rounded to the nearest digit, tenth, hundredth, thousandth and ten-thousandth.

G.10 Write a program that inputs a word from the keyboard and determines its length. Print the word using twice the length as the field width.

G.11 ***(Converting Fahrenheit Temperature to Celsius)*** Write a program that converts integer Fahrenheit temperatures from `0` to `212` degrees to floating-point Celsius temperatures with three digits of precision. Use the formula

```
celsius = 5.0 / 9.0 * ( fahrenheit - 32 );
```

to perform the calculation. The output should be printed in two right-justified columns of 10 characters each, and the Celsius temperatures should be preceded by a sign for both positive and negative values.

G.12 Write a program to test all the escape sequences in Fig. G.23. For those that move the cursor, print a character before and after the escape sequence so that it's clear where the cursor has moved.

G.13 Write a program that uses the conversion character `g` to output the value `9876.12345`. Print the value with precisions ranging from 1 to 9.

H

Number Systems



Objectives

In this appendix you'll learn:

- To understand basic number systems concepts, such as base, positional value and symbol value.
- To understand how to work with numbers represented in the binary, octal and hexadecimal number systems.
- To abbreviate binary numbers as octal numbers or hexadecimal numbers.
- To convert octal numbers and hexadecimal numbers to binary numbers.
- To convert back and forth between decimal numbers and their binary, octal and hexadecimal equivalents.
- To understand binary arithmetic and how negative binary numbers are represented using two's complement notation.

Outline

H.1 Introduction
H.2 Abbreviating Binary Numbers as Octal and Hexadecimal Numbers
H.3 Converting Octal and Hexadecimal Numbers to Binary Numbers
H.4 Converting from Binary, Octal or Hexadecimal to Decimal
H.5 Converting from Decimal to Binary, Octal or Hexadecimal
H.6 Negative Binary Numbers: Two's Complement Notation

Summary | *Self-Review Exercises* | *Answers to Self-Review Exercises* | *Exercises*

H.1 Introduction

In this appendix, we introduce the key number systems that Java programmers use, especially when they're working on software projects that require close interaction with machine-level hardware. Projects like this include operating systems, computer networking software, compilers, database systems and applications requiring high performance.

When we write an integer such as 227 or –63 in a Java program, the number is assumed to be in the decimal (base 10) number system. The digits in the decimal number system are 0, 1, 2, 3, 4, 5, 6, 7, 8 and 9. The lowest digit is 0 and the highest digit is 9—one less than the base of 10. Internally, computers use the binary (base 2) number system. The binary number system has only two digits, namely 0 and 1. Its lowest digit is 0 and its highest digit is 1—one less than the base of 2.

As we'll see, binary numbers tend to be much longer than their decimal equivalents. Programmers who work in assembly languages and in high-level languages like Java that enable programmers to reach down to the machine level find it cumbersome to work with binary numbers. So two other number systems—the octal number system (base 8) and the hexadecimal number system (base 16)—are popular primarily because they make it convenient to abbreviate binary numbers.

In the octal number system, the digits range from 0 to 7. Because both the binary number system and the octal number system have fewer digits than the decimal number system, their digits are the same as the corresponding digits in decimal.

The hexadecimal number system poses a problem because it requires 16 digits—a lowest digit of 0 and a highest digit with a value equivalent to decimal 15 (one less than the base of 16). By convention, we use the letters A through F to represent the hexadecimal digits corresponding to decimal values 10 through 15. Thus in hexadecimal we can have numbers like 876 consisting solely of decimal-like digits, numbers like 8A55F consisting of digits and letters and numbers like FFE consisting solely of letters. Occasionally, a hexadecimal number spells a common word such as FACE or FEED—this can appear strange to programmers accustomed to working with numbers. The digits of the binary, octal, decimal and hexadecimal number systems are summarized in Fig. H.1 and Fig. H.2.

Each of these number systems uses positional notation—each position in which a digit is written has a different positional value. For example, in the decimal number 937 (the 9, the 3 and the 7 are referred to as symbol values), we say that the 7 is written in the ones position, the 3 is written in the tens position and the 9 is written in the hundreds position. Each of these positions is a power of the base (base 10) and that these powers begin at 0 and increase by 1 as we move left in the number (Fig. H.3).

Binary digit	Octal digit	Decimal digit	Hexadecimal digit
0	0	0	0
1	1	1	1
	2	2	2
	3	3	3
	4	4	4
	5	5	5
	6	6	6
	7	7	7
		8	8
		9	9
			A (decimal value of 10)
			B (decimal value of 11)
			C (decimal value of 12)
			D (decimal value of 13)
			E (decimal value of 14)
			F (decimal value of 15)

Fig. H.1 | Digits of the binary, octal, decimal and hexadecimal number systems.

Attribute	Binary	Octal	Decimal	Hexadecimal
Base	2	8	10	16
Lowest digit	0	0	0	0
Highest digit	1	7	9	F

Fig. H.2 | Comparing the binary, octal, decimal and hexadecimal number systems.

Positional values in the decimal number system			
Decimal digit	9	3	7
Position name	Hundreds	Tens	Ones
Positional value	100	10	1
Positional value as a power of the base (10)	10^2	10^1	10^0

Fig. H.3 | Positional values in the decimal number system.

For longer decimal numbers, the next positions to the left would be the thousands position (10 to the 3rd power), the ten-thousands position (10 to the 4th power), the hundred-thousands position (10 to the 5th power), the millions position (10 to the 6th power), the ten-millions position (10 to the 7th power) and so on.

In the binary number 101, the rightmost 1 is written in the ones position, the 0 is written in the twos position and the leftmost 1 is written in the fours position. Each position is a power of the base (base 2) and that these powers begin at 0 and increase by 1 as we move left in the number (Fig. H.4). So, $101 = 2^2 + 2^0 = 4 + 1 = 5$.

Positional values in the binary number system			
Binary digit	1	0	1
Position name	Fours	Twos	Ones
Positional value	4	2	1
Positional value as a power of the base (2)	2^2	2^1	2^0

Fig. H.4 | Positional values in the binary number system.

For longer binary numbers, the next positions to the left would be the eights position (2 to the 3rd power), the sixteens position (2 to the 4th power), the thirty-twos position (2 to the 5th power), the sixty-fours position (2 to the 6th power) and so on.

In the octal number 425, we say that the 5 is written in the ones position, the 2 is written in the eights position and the 4 is written in the sixty-fours position. Each of these positions is a power of the base (base 8) and that these powers begin at 0 and increase by 1 as we move left in the number (Fig. H.5).

Positional values in the octal number system			
Decimal digit	4	2	5
Position name	Sixty-fours	Eights	Ones
Positional value	64	8	1
Positional value as a power of the base (8)	8^2	8^1	8^0

Fig. H.5 | Positional values in the octal number system.

For longer octal numbers, the next positions to the left would be the five-hundred-and-twelves position (8 to the 3rd power), the four-thousand-and-ninety-sixes position (8 to the 4th power), the thirty-two-thousand-seven-hundred-and-sixty-eights position (8 to the 5th power) and so on.

In the hexadecimal number 3DA, we say that the A is written in the ones position, the D is written in the sixteens position and the 3 is written in the two-hundred-and-fifty-sixes position. Each of these positions is a power of the base (base 16) and that these powers begin at 0 and increase by 1 as we move left in the number (Fig. H.6).

For longer hexadecimal numbers, the next positions to the left would be the four-thousand-and-ninety-sixes position (16 to the 3rd power), the sixty-five-thousand-five-hundred-and-thirty-sixes position (16 to the 4th power) and so on.

Positional values in the hexadecimal number system			
Decimal digit	3	D	A
Position name	Two-hundred-and-fifty-sixes	Sixteens	Ones
Positional value	256	16	1
Positional value as a power of the base (16)	16^2	16^1	16^0

Fig. H.6 | Positional values in the hexadecimal number system.

H.2 Abbreviating Binary Numbers as Octal and Hexadecimal Numbers

The main use for octal and hexadecimal numbers in computing is for abbreviating lengthy binary representations. Figure H.7 highlights the fact that lengthy binary numbers can be expressed concisely in number systems with higher bases than the binary number system.

Decimal number	Binary representation	Octal representation	Hexadecimal representation
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Fig. H.7 | Decimal, binary, octal and hexadecimal equivalents.

A particularly important relationship that both the octal number system and the hexadecimal number system have to the binary system is that the bases of octal and hexadec-

imal (8 and 16 respectively) are powers of the base of the binary number system (base 2). Consider the following 12-digit binary number and its octal and hexadecimal equivalents. See if you can determine how this relationship makes it convenient to abbreviate binary numbers in octal or hexadecimal. The answer follows the numbers.

```
Binary number   Octal equivalent   Hexadecimal equivalent
100011010001    4321               8D1
```

To see how the binary number converts easily to octal, simply break the 12-digit binary number into groups of three consecutive bits each and write those groups over the corresponding digits of the octal number as follows:

```
100     011     010     001
4       3       2       1
```

The octal digit you've written under each group of three bits corresponds precisely to the octal equivalent of that 3-digit binary number, as shown in Fig. H.7.

The same kind of relationship can be observed in converting from binary to hexadecimal. Break the 12-digit binary number into groups of four consecutive bits each and write those groups over the corresponding digits of the hexadecimal number as follows:

```
1000    1101    0001
8       D       1
```

Notice that the hexadecimal digit you wrote under each group of four bits corresponds precisely to the hexadecimal equivalent of that 4-digit binary number as shown in Fig. H.7.

H.3 Converting Octal and Hexadecimal Numbers to Binary Numbers

In the previous section, we saw how to convert binary numbers to their octal and hexadecimal equivalents by forming groups of binary digits and simply rewriting them as their equivalent octal digit values or hexadecimal digit values. This process may be used in reverse to produce the binary equivalent of a given octal or hexadecimal number.

For example, the octal number 653 is converted to binary simply by writing the 6 as its 3-digit binary equivalent 110, the 5 as its 3-digit binary equivalent 101 and the 3 as its 3-digit binary equivalent 011 to form the 9-digit binary number 110101011.

The hexadecimal number FAD5 is converted to binary simply by writing the F as its 4-digit binary equivalent 1111, the A as its 4-digit binary equivalent 1010, the D as its 4-digit binary equivalent 1101 and the 5 as its 4-digit binary equivalent 0101 to form the 16-digit 1111101011010101.

H.4 Converting from Binary, Octal or Hexadecimal to Decimal

We're accustomed to working in decimal, and therefore it's often convenient to convert a binary, octal, or hexadecimal number to decimal to get a sense of what the number is "really" worth. Our diagrams in Section H.1 express the positional values in decimal. To convert a number to decimal from another base, multiply the decimal equivalent of each digit

by its positional value and sum these products. For example, the binary number 110101 is converted to decimal 53, as shown in Fig. H.8.

Converting a binary number to decimal						
Postional values:	32	16	8	4	2	1
Symbol values:	1	1	0	1	0	1
Products:	1*32=32	1*16=16	0*8=0	1*4=4	0*2=0	1*1=1
Sum:	= 32 + 16 + 0 + 4 + 0s + 1 = 53					

Fig. H.8 | Converting a binary number to decimal.

To convert octal 7614 to decimal 3980, we use the same technique, this time using appropriate octal positional values, as shown in Fig. H.9.

Converting an octal number to decimal				
Positional values:	512	64	8	1
Symbol values:	7	6	1	4
Products	7*512=3584	6*64=384	1*8=8	4*1=4
Sum:	= 3584 + 384 + 8 + 4 = 3980			

Fig. H.9 | Converting an octal number to decimal.

To convert hexadecimal AD3B to decimal 44347, we use the same technique, this time using appropriate hexadecimal positional values, as shown in Fig. H.10.

Converting a hexadecimal number to decimal				
Postional values:	4096	256	16	1
Symbol values:	A	D	3	B
Products	A*4096=40960	D*256=3328	3*16=48	B*1=11
Sum:	= 40960 + 3328 + 48 + 11 = 44347			

Fig. H.10 | Converting a hexadecimal number to decimal.

H.5 Converting from Decimal to Binary, Octal or Hexadecimal

The conversions in Section H.4 follow naturally from the positional notation conventions. Converting from decimal to binary, octal, or hexadecimal also follows these conventions.

Suppose we wish to convert decimal 57 to binary. We begin by writing the positional values of the columns right to left until we reach a column whose positional value is greater than the decimal number. We don't need that column, so we discard it. Thus, we first write:

```
Positional values: 64 32 16 8 4 2 1
```

Then we discard the column with positional value 64, leaving:

```
Positional values: 32 16 8 4 2 1
```

Next we work from the leftmost column to the right. We divide 32 into 57 and observe that there's one 32 in 57 with a remainder of 25, so we write 1 in the 32 column. We divide 16 into 25 and observe that there's one 16 in 25 with a remainder of 9 and write 1 in the 16 column. We divide 8 into 9 and observe that there's one 8 in 9 with a remainder of 1. The next two columns each produce quotients of 0 when their positional values are divided into 1, so we write 0s in the 4 and 2 columns. Finally, 1 into 1 is 1, so we write 1 in the 1 column. This yields:

```
Positional values: 32 16 8 4 2 1
Symbol values:     1  1  1 0 0 1
```

and thus decimal 57 is equivalent to binary 111001.

To convert decimal 103 to octal, we begin by writing the positional values of the columns until we reach a column whose positional value is greater than the decimal number. We do not need that column, so we discard it. Thus, we first write:

```
Positional values: 512 64 8 1
```

Then we discard the column with positional value 512, yielding:

```
Positional values: 64 8 1
```

Next we work from the leftmost column to the right. We divide 64 into 103 and observe that there's one 64 in 103 with a remainder of 39, so we write 1 in the 64 column. We divide 8 into 39 and observe that there are four 8s in 39 with a remainder of 7 and write 4 in the 8 column. Finally, we divide 1 into 7 and observe that there are seven 1s in 7 with no remainder, so we write 7 in the 1 column. This yields:

```
Positional values: 64 8 1
Symbol values:     1  4 7
```

and thus decimal 103 is equivalent to octal 147.

To convert decimal 375 to hexadecimal, we begin by writing the positional values of the columns until we reach a column whose positional value is greater than the decimal number. We do not need that column, so we discard it. Thus, we first write:

```
Positional values: 4096 256 16 1
```

Then we discard the column with positional value 4096, yielding:

```
Positional values: 256 16 1
```

Next we work from the leftmost column to the right. We divide 256 into 375 and observe that there's one 256 in 375 with a remainder of 119, so we write 1 in the 256 column. We divide 16 into 119 and observe that there are seven 16s in 119 with a

remainder of 7 and write 7 in the 16 column. Finally, we divide 1 into 7 and observe that there are seven 1s in 7 with no remainder, so we write 7 in the 1 column. This yields:

```
Positional values:  256     16      1
Symbol values:      1       7       7
```

and thus decimal 375 is equivalent to hexadecimal 177.

H.6 Negative Binary Numbers: Two's Complement Notation

The discussion so far in this appendix has focused on positive numbers. In this section, we explain how computers represent negative numbers using *two's complement notation*. First we explain how the two's complement of a binary number is formed, then we show why it represents the negative value of the given binary number.

Consider a machine with 32-bit integers. Suppose

```
int value = 13;
```

The 32-bit representation of `value` is

```
00000000 00000000 00000000 00001101
```

To form the negative of `value` we first form its *one's complement* by applying Java's bitwise complement operator (~):

```
onesComplementOfValue = ~value;
```

Internally, `~value` is now `value` with each of its bits reversed—ones become zeros and zeros become ones, as follows:

```
value:
00000000 00000000 00000000 00001101

~value (i.e., value's ones complement):
11111111 11111111 11111111 11110010
```

To form the two's complement of `value`, we simply add 1 to `value`'s one's complement. Thus

```
Two's complement of value:
11111111 11111111 11111111 11110011
```

Now if this is in fact equal to –13, we should be able to add it to binary 13 and obtain a result of 0. Let's try this:

```
 00000000 00000000 00000000 00001101
+11111111 11111111 11111111 11110011
-----------------------------------
 00000000 00000000 00000000 00000000
```

The carry bit coming out of the leftmost column is discarded and we indeed get 0 as a result. If we add the one's complement of a number to the number, the result would be all 1s. The key to getting a result of all zeros is that the two's complement is one more than the one's complement. The addition of 1 causes each column to add to 0 with a carry of 1. The carry keeps moving leftward until it's discarded from the leftmost bit, and thus the resulting number is all zeros.

Computers actually perform a subtraction, such as

```
x = a - value;
```

by adding the two's complement of `value` to `a`, as follows:

```
x = a + (~value + 1);
```

Suppose `a` is 27 and `value` is 13 as before. If the two's complement of `value` is actually the negative of `value`, then adding the two's complement of value to `a` should produce the result 14. Let's try this:

```
a (i.e., 27)          00000000 00000000 00000000 00011011
+(~value + 1)        +11111111 11111111 11111111 11110011
                     -----------------------------------
                      00000000 00000000 00000000 00001110
```

which is indeed equal to 14.

Summary

- An integer such as 19 or 227 or –63 in a Java program is assumed to be in the decimal (base 10) number system. The digits in the decimal number system are 0, 1, 2, 3, 4, 5, 6, 7, 8 and 9. The lowest digit is 0 and the highest digit is 9—one less than the base of 10.
- Computers use the binary (base 2) number system. The binary number system has only two digits, namely 0 and 1. Its lowest digit is 0 and its highest digit is 1—one less than the base of 2.
- The octal number system (base 8) and the hexadecimal number system (base 16) are popular primarily because they make it convenient to abbreviate binary numbers.
- The digits of the octal number system range from 0 to 7.
- The hexadecimal number system poses a problem because it requires 16 digits—a lowest digit of 0 and a highest digit with a value equivalent to decimal 15 (one less than the base of 16). By convention, we use the letters A through F to represent the hexadecimal digits corresponding to decimal values 10 through 15.
- Each number system uses positional notation—each position in which a digit is written has a different positional value.
- A particularly important relationship of both the octal number system and the hexadecimal number system to the binary system is that the bases of octal and hexadecimal (8 and 16 respectively) are powers of the base of the binary number system (base 2).
- To convert an octal to a binary number, replace each octal digit with its three-digit binary equivalent.
- To convert a hexadecimal number to a binary number, simply replace each hexadecimal digit with its four-digit binary equivalent.
- Because we're accustomed to working in decimal, it's convenient to convert a binary, octal or hexadecimal number to decimal to get a sense of the number's "real" worth.
- To convert a number to decimal from another base, multiply the decimal equivalent of each digit by its positional value and sum the products.
- Computers represent negative numbers using two's complement notation.
- To form the negative of a value in binary, first form its one's complement by applying Java's bitwise complement operator (~). This reverses the bits of the value. To form the two's complement of a value, simply add one to the value's one's complement.

Self-Review Exercises

H.1 The bases of the decimal, binary, octal and hexadecimal number systems are _____, _____, _____ and _____ respectively.

H.2 In general, the decimal, octal and hexadecimal representations of a given binary number contain (more/fewer) digits than the binary number contains.

H.3 (*True/False*) A popular reason for using the decimal number system is that it forms a convenient notation for abbreviating binary numbers simply by substituting one decimal digit per group of four binary bits.

H.4 The (octal / hexadecimal / decimal) representation of a large binary value is the most concise (of the given alternatives).

H.5 (*True/False*) The highest digit in any base is one more than the base.

H.6 (*True/False*) The lowest digit in any base is one less than the base.

H.7 The positional value of the rightmost digit of any number in either binary, octal, decimal or hexadecimal is always _____.

H.8 The positional value of the digit to the left of the rightmost digit of any number in binary, octal, decimal or hexadecimal is always equal to _____.

H.9 Fill in the missing values in this chart of positional values for the rightmost four positions in each of the indicated number systems:

decimal	1000	100	10	1
hexadecimal	...	256	...	...
binary	...	...	...	...
octal	512	...	8	...

H.10 Convert binary 110101011000 to octal and to hexadecimal.

H.11 Convert hexadecimal FACE to binary.

H.12 Convert octal 7316 to binary.

H.13 Convert hexadecimal 4FEC to octal. (*Hint:* First convert 4FEC to binary, then convert that binary number to octal.)

H.14 Convert binary 1101110 to decimal.

H.15 Convert octal 317 to decimal.

H.16 Convert hexadecimal EFD4 to decimal.

H.17 Convert decimal 177 to binary, to octal and to hexadecimal.

H.18 Show the binary representation of decimal 417. Then show the one's complement of 417 and the two's complement of 417.

H.19 What is the result when a number and its two's complement are added to each other?

Answers to Self-Review Exercises

H.1 10, 2, 8, 16.

H.2 Fewer.

H.3 False. Hexadecimal does this.

H.4 Hexadecimal.

H.5 False. The highest digit in any base is one less than the base.

H.6 False. The lowest digit in any base is zero.

H.7 1 (the base raised to the zero power).

H.8 The base of the number system.

H.9 See below:

decimal	1000	100	10	1
hexadecimal	4096	256	16	1
binary	8	4	2	1
octal	512	64	8	1

H.10 Octal 6530; hexadecimal D58.

H.11 Binary 1111 1010 1100 1110.

H.12 Binary 111 011 001 110.

H.13 Binary 0 100 111 111 101 100; Octal 47754.

H.14 Decimal 2+4+8+32+64=110.

H.15 Decimal 7+1*8+3*64=7+8+192=207.

H.16 Decimal 4+13*16+15*256+14*4096=61396.

H.17 Decimal 177
to binary:

```
256 128 64 32 16 8 4 2 1
128 64 32 16 8 4 2 1
(1*128)+(0*64)+(1*32)+(1*16)+(0*8)+(0*4)+(0*2)+(1*1)
10110001
```

to octal:

```
512 64 8 1
64 8 1
(2*64)+(6*8)+(1*1)
261
```

to hexadecimal:

```
256 16 1
16 1
(11*16)+(1*1)
(B*16)+(1*1)
B1
```

H.18 Binary:

```
512 256 128 64 32 16 8 4 2 1
256 128 64 32 16 8 4 2 1
(1*256)+(1*128)+(0*64)+(1*32)+(0*16)+(0*8)+(0*4)+(0*2)+(1*1)
110100001
```

One's complement: 001011110
Two's complement: 001011111
Check: Original binary number + its two's complement

```
110100001
001011111
---------
000000000
```

H.19 Zero.

Exercises

H.20 Some people argue that many of our calculations would be easier in the base 12 number system because 12 is divisible by so many more numbers than 10 (for base 10). What is the lowest digit in base 12? What would be the highest symbol for the digit in base 12? What are the positional values of the rightmost four positions of any number in the base 12 number system?

H.21 Complete the following chart of positional values for the rightmost four positions in each of the indicated number systems:

decimal	1000	100	10	1
base 6	...	...	6	...
base 13	...	169	...	...
base 3	27	...	...	...

H.22 Convert binary 100101111010 to octal and to hexadecimal.

H.23 Convert hexadecimal 3A7D to binary.

H.24 Convert hexadecimal 765F to octal. (*Hint:* First convert 765F to binary, then convert that binary number to octal.)

H.25 Convert binary 1011110 to decimal.

H.26 Convert octal 426 to decimal.

H.27 Convert hexadecimal FFFF to decimal.

H.28 Convert decimal 299 to binary, to octal and to hexadecimal.

H.29 Show the binary representation of decimal 779. Then show the one's complement of 779 and the two's complement of 779.

H.30 Show the two's complement of integer value –1 on a machine with 32-bit integers.



I

GroupLayout

I.1 Introduction

Java SE 6 includes a powerful layout manager called **`GroupLayout`**, which is the default layout manager in the NetBeans IDE (`www.netbeans.org`). In this appendix, we overview `GroupLayout`, then demonstrate how to use the NetBeans IDE's **Matisse GUI designer** to create a GUI using `GroupLayout` to position the components. NetBeans generates the `GroupLayout` code for you automatically. Though you can write `GroupLayout` code by hand, in most cases you'll use a GUI design tool like the one provided by NetBeans to take advantage of `GroupLayout`'s power. For more details on `GroupLayout`, see the list of web resources at the end of this appendix.

I.2 GroupLayout Basics

Chapters 14 and 25 presented several layout managers that provide basic GUI layout capabilities. We also discussed how to combine layout managers and multiple containers to create more complex layouts. Most layout managers do not give you precise control over the positioning of components. In Chapter 25, we discussed the `GridBagLayout`, which provides more precise control over the position and size of your GUI components. It allows you to specify the horizontal and vertical position of each component, the number of rows and columns each component occupies in the grid, and how components grow and shrink as the size of the container changes. This is all specified at once with a `GridBagConstraints` object. Class `GroupLayout` is the next step in layout management. `GroupLayout` is more flexible, because you can specify the horizontal and vertical layouts of your components independently.

Sequential and Parallel Arrangements

Components are arranged either sequentially or in parallel. The three `JButtons` in Fig. I.1 are arranged with **sequential horizontal orientation**—they appear left to right in sequence. Vertically, the components are arranged in parallel, so, in a sense, they "occupy the same vertical space." Components can also be arranged sequentially in the vertical direction and in parallel in the horizontal direction, as you'll see in Section I.3. To prevent overlapping components, components with parallel vertical orientation are normally arranged with sequential horizontal orientation (and vice versa).

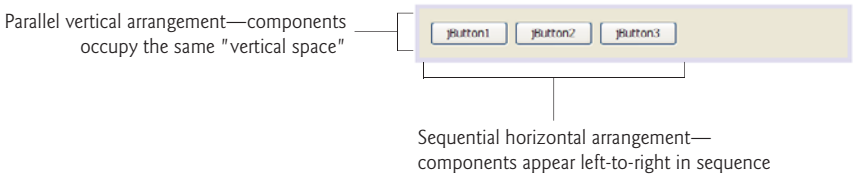


Fig. I.1 | JButtons arranged sequentially for their horizontal orientation and in parallel for their vertical orientation.

Groups and Alignment

To create more complex user interfaces, `GroupLayout` allows you to create **groups** that contain sequential or parallel elements. Within a group you can have GUI components, other groups and gaps. Placing a group within another group is similar to building a GUI using nested containers, such as a `JPanel` that contains other `JPanel`s, which in turn contain GUI components.

When you create a group, you can specify the **alignment** of the group's elements. Class `GroupLayout` contains four constants for this purpose—`LEADING`, `TRAILING`, `CENTER` and `BASELINE`. The constant `BASELINE` applies only to vertical orientations. In horizontal orientation, the constants `LEADING`, `TRAILING` and `CENTER` represent left justified, right justified and centered, respectively. In vertical orientation, `LEADING`, `TRAILING` and `CENTER` align the components at their tops, bottoms or vertical centers, respectively. Aligning components with `BASELINE` indicates they should be aligned using the baseline of the font for the components' text. For more information about font baselines, see Section 15.4.

Spacing

`GroupLayout` by default uses the recommended GUI design guidelines of the underlying platform for spacing between components. The **`addGap`** method of `GroupLayout` nested classes **`GroupLayout.Group`**, **`GroupLayout.SequentialGroup`** and **`GroupLayout.ParallelGroup`** allows you to control the spacing between components.

Sizing Components

By default, `GroupLayout` uses each component's `getMinimumSize`, `getMaximumSize` and `getPreferredSize` methods to help determine the component's size. You can override the default settings.

I.3 Building a ColorChooser

We now present a `ColorChooser` application to demonstrate the `GroupLayout` layout manager. The application consists of three `JSlider` objects, each representing the values from `0` to `255` for specifying the red, green and blue values of a color. The selected values for each `JSlider` will be used to display a filled rectangle of the specified color. We build the application using NetBeans. For an more detailed introduction to developing GUI applications in the NetBeans IDE, see `www.netbeans.org/kb/trails/matisse.html`.

Creating a New Project

Begin by opening a new NetBeans project. Select **File > New Project...**. In the **New Project** dialog, choose **Java** from the **Categories** list and **Java Application** from the **Projects** list then click **Next >**. Specify `ColorChooser` as the project name and uncheck the **Create Main Class** checkbox. You can also specify the location of your project in the **Project Location** field. Click **Finish** to create the project.

Adding a New Subclass of `JFrame` to the Project

In the IDE's **Projects** tab just below the **File** menu and toolbar (Fig. I.2), expand the **Source Packages** node. Right-click the **<default package>** node that appears and select **New > JFrame Form**. In the **New JFrame Form** dialog, specify `ColorChooser` as the class name and click **Finish**. This subclass of `JFrame` will display the application's GUI components. The NetBeans window should now appear similar to Fig. I.3 with the `ColorChooser` class shown in **Design** view. The **Source** and **Design** buttons at the top of the `ColorChooser.java` window allow you to switch between editing the source code and designing the GUI.

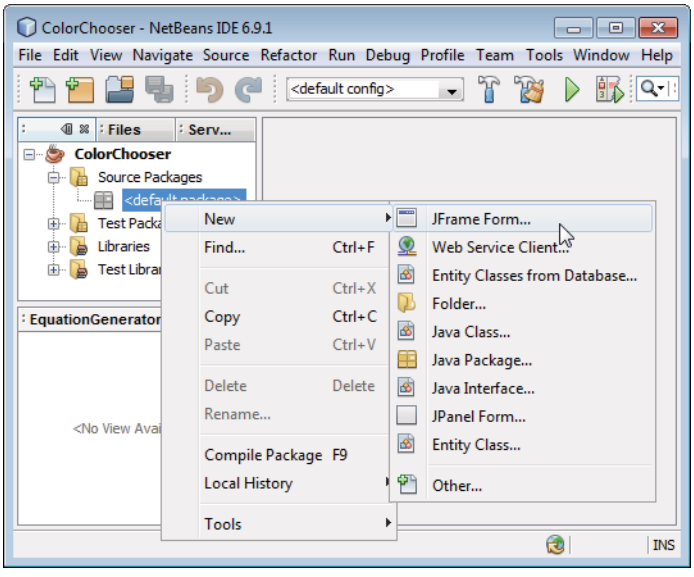


Fig. I.2 | Adding a new **JFrame Form** to the `ColorChooser` project.

Design view shows only the `ColorChooser`'s client area (i.e., the area that will appear inside the window's borders). To build a GUI visually, you can drag GUI components from the **Palette** window onto the client area. You can configure the properties of each component by selecting it, then modifying the property values that appear in the **Properties** window (Fig. I.3). When you select a component, the **Properties** window displays three buttons—**Properties**, **Bindings**, **Events**, **Code** (see Fig. I.4)—that enable you to configure various aspects of the component.

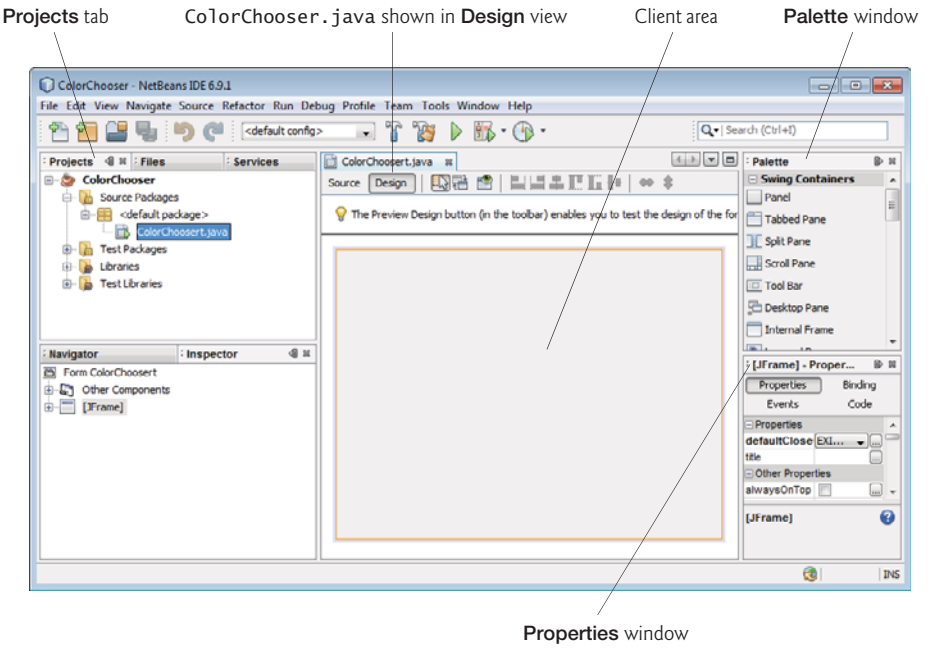


Fig. I.3 | Class `ColorChooser` shown in the NetBeans **Design** view.

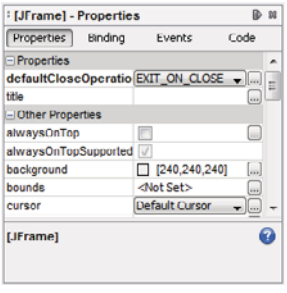


Fig. I.4 | **Properties** window with buttons that enable you to configure various aspects of the component.

Build the GUI

Drag three **Sliders** (objects of class `JSlider`) from the **Palette** onto the `JFrame` (you may need to scroll through the **Palette**). As you drag components near the edges of the client area or near other components, NetBeans displays **guide lines** (Fig. I.5) that show you the recommended distances and alignments between the component you're dragging, the edges of the client area and other components. As you follow the steps to build the GUI, use the guide lines to arrange the components into three rows and three columns as in Fig. I.6. Next, rename the `JSliders` to `redJSlider`, `greenJSlider` and `blueJSlider`. To do so, select the first `JSlider`, then click the **Code** button in the **Properties** window and

change the **Variable Name** property to `redSlider`. Repeat this process to rename the other two `JSliders`. Then, click the **Properties** button in the **Properties** window, select each `JSlider` and change its **maximum** property to 255 so that it will produce values in the range 0–255, and change its **value** property to 0 so the `JSlider`'s thumb will initially be at the left of the `JSlider`.

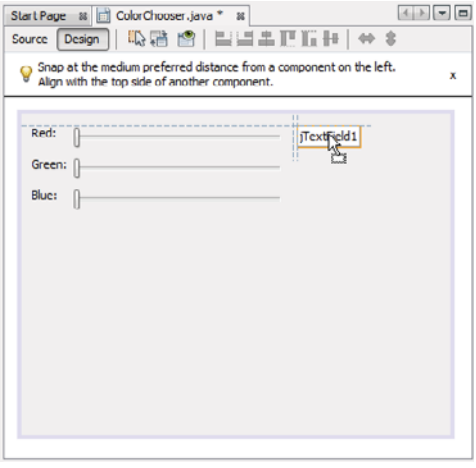


Fig. I.5 | Positioning the first `JTextField`.

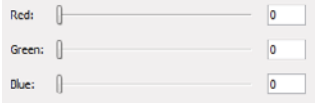


Fig. I.6 | Layout of the `JLabels`, `JSLiders` and `JTextFields`.

Drag three **Labels** (objects of class `JLabel`) from the **Palette** to the `JFrame` to label each `JSlider` with the color it represents. Name the `JLabels` `redJLabel`, `greenJLabel` and `blueJLabel`, respectively. Each `JLabel` should be placed to the left of the corresponding `JSlider` (Fig. I.6). Change each `JLabel`'s **text** property either by double clicking the `JLabel` and typing the new text, or by selecting the `JLabel` and changing the **text** property in the **Properties** window.

Add a **Text Field** (an object of class `JTextField`) next to each of the `JSliders` to display the value of the slider. Name the `JTextFields` `redJTextField`, `greenJTextField` and `blueJTextField`, respectively. Change each `JTextField`'s **text** property to `0` using the same techniques as you did for the `JLabels`. Change each `JTextField`'s **columns** property to `4`. To align each **Label**, **Slider** and **Text Field** nicely, you can select them by dragging the mouse across all three and use the alignment buttons at the top of the Design window.

Next, add a **Panel** named `colorJPanel` to the right of this group of components. Use the guide lines as shown in Fig. I.7 to place the `JPanel`. Change this `JPanel`'s **background** color to black (the initially selected RGB color). Finally, drag the bottom-right border of the client area toward the top-left of the **Design** area until you see the snap-to lines that

show the recommended client area dimensions (which are based on the components in the client area) as shown in Fig. I.8.

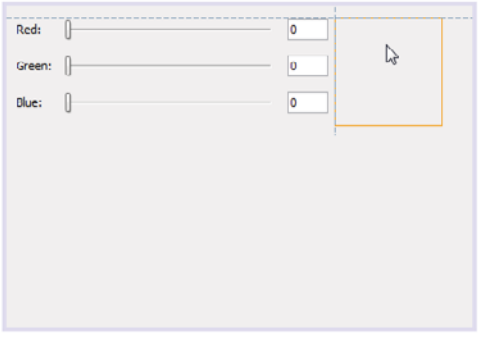


Fig. I.7 | Positioning the `JPanel`.

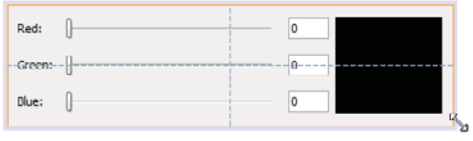


Fig. I.8 | Setting the height of the client area.

Editing the Source Code and Adding Event Handlers

The IDE automatically generated the GUI code, including methods for initializing components and aligning them using the `GroupLayout` layout manager. We must add the desired functionality to the components' event handlers. To add an event handler for a component, right click it and position the mouse over the **Events** option in the pop-up menu. You can then select the category of event you wish to handle and the specific event within that category. For example, to add the `JSlider` event handlers for this example, right click each `JSlider` and select **Events > Change > stateChanged**. When you do this, NetBeans adds a `ChangeListener` to the `JSlider` and switches from **Design** view to **Source** view where you can place code in the event handler. Use the **Design** button to return to **Design** view and repeat the preceding steps to add the event handlers for the other two `JSliders`. To complete the event handlers, first add the method in Fig. I.9 following the class's constructor. In each `JSlider` event handler set the corresponding `JTextField` to the new value of the `JSlider`, then call method `changeColor`. Figure I.10 shows the completed `ColorChooser` class as it's generated in NetBeans. We did not restyle the code to match our coding conventions that you've seen throughout the book. You can now run the program to see it in action. Drag each slider and watch the `colorJPanel`'s background color change.

Method `initComponents` (lines 39–162) was entirely generated by NetBeans based on your interactions with the GUI designer. This method contains the code that creates and formats the GUI. Lines 41–93 construct and initialize the GUI components. Lines

95–161 specify the layout of those components using `GroupLayout`. Lines 108–136 specify the horizontal group and lines 137–159 specify the vertical group. Notice how complex the code is. More and more software development is done with tools that generate complex code like this, saving you the time and effort of doing it yourself.

We manually added the `changeColor` method in lines 25–30. When the user moves the thumb on one of the `JSliders`, the `JSlider`'s event handler sets the text in its corresponding `JTextField` to the `JSlider`'s new value (lines 166, 172 and 178), then calls method `changeColor` (lines 167, 173 and 179) to update the `colorJPanel`'s background color. Method `changeColor` gets the current value of each `JSlider` (lines 28–29) and uses these values as the arguments to the `Color` constructor to create a new `Color`.

```
1  // changes the colorJPanel's background color based on the current
2  // values of the JSliders
3  public void changeColor()
4  {
5     colorJPanel.setBackground( new java.awt.Color(
6        redJSlider.getValue(), greenJSlider.getValue(),
7        blueJSlider.getValue() ) );
8  } // end method changeColor
```

Fig. I.9 | Method that changes the `colorJPanel`'s background color based on the values of the three `JSliders`.

```
1  /*
2   * To change this template, choose Tools | Templates
3   * and open the template in the editor.
4   */
5
6  /*
7   * ColorChooser.java
8   *
9   * Created on Feb 8, 2011, 9:20:27 AM
10  */
11
12 /**
13  *
14  * @author Paul Deitel
15  */
16 public class ColorChooser extends javax.swing.JFrame {
17
18     /** Creates new form ColorChooser */
19     public ColorChooser() {
20         initComponents();
21     }
22
23     // changes the colorJPanel's background color based on the current
24     // values of the JSliders
25     public void changeColor()
26     {
```

Fig. I.10 | `ColorChooser` class that uses `GroupLayout` for its GUI layout. (Part 1 of 6.)

```
27          colorJPanel.setBackground( new java.awt.Color(
28             redJSlider.getValue(), greenJSlider.getValue(),
29             blueJSlider.getValue() ) );
30       } // end method changeColor
31
32       /** This method is called from within the constructor to
33        * initialize the form.
34        * WARNING: Do NOT modify this code. The content of this method is
35        * always regenerated by the Form Editor.
36        */
37       @SuppressWarnings("unchecked")
38      // <editor-fold defaultstate="collapsed" desc="Generated Code">
39      private void initComponents() {
40
41         redJSlider = new javax.swing.JSlider();
42         greenJSlider = new javax.swing.JSlider();
43         blueJSlider = new javax.swing.JSlider();
44         redJLabel = new javax.swing.JLabel();
45         greenJLabel = new javax.swing.JLabel();
46         blueJLabel = new javax.swing.JLabel();
47         redJTextField = new javax.swing.JTextField();
48         greenJTextField = new javax.swing.JTextField();
49         blueJTextField = new javax.swing.JTextField();
50         colorJPanel = new javax.swing.JPanel();
51
52         setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
53
54         redJSlider.setMaximum(255);
55         redJSlider.setValue(0);
56         redJSlider.addChangeListener(new javax.swing.event.ChangeListener()
{
57            public void stateChanged(javax.swing.event.ChangeEvent evt) {
58               redJSliderStateChanged(evt);
59            }
60         });
61
62         greenJSlider.setMaximum(255);
63         greenJSlider.setValue(0);
64         greenJSlider.addChangeListener(new
javax.swing.event.ChangeListener() {
65            public void stateChanged(javax.swing.event.ChangeEvent evt) {
66               greenJSliderStateChanged(evt);
67            }
68         });
69
70         blueJSlider.setMaximum(255);
71         blueJSlider.setValue(0);
72        blueJSlider.addChangeListener(new javax.swing.event.ChangeListener()
{
73            public void stateChanged(javax.swing.event.ChangeEvent evt) {
74               blueJSliderStateChanged(evt);
75            }
76         });
```

Fig. I.10 | `ColorChooser` class that uses `GroupLayout` for its GUI layout. (Part 2 of 6.)

```
77
78          redJLabel.setText("Red:");
79
80          greenJLabel.setText("Green:");
81
82          blueJLabel.setText("Blue:");
83
84          redJTextField.setColumns(4);
85          redJTextField.setText("0");
86
87          greenJTextField.setColumns(4);
88          greenJTextField.setText("0");
89
90          blueJTextField.setColumns(4);
91          blueJTextField.setText("0");
92
93          colorJPanel.setBackground(new java.awt.Color(0, 0, 0));
94
95          javax.swing.GroupLayout colorJPanelLayout = new
javax.swing.GroupLayout(colorJPanel);
96          colorJPanel.setLayout(colorJPanelLayout);
97          colorJPanelLayout.setHorizontalGroup(
98
colorJPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADIN
G)
99             .addGap(0, 100, Short.MAX_VALUE)
100         );
101         colorJPanelLayout.setVerticalGroup(
102
colorJPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADIN
G)
103            .addGap(0, 91, Short.MAX_VALUE)
104         );
105
106         javax.swing.GroupLayout layout = new
javax.swing.GroupLayout(getContentPane());
107         getContentPane().setLayout(layout);
108         layout.setHorizontalGroup(
109
layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
110            .addGroup(layout.createSequentialGroup()
111
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING
)
112                  .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
layout.createSequentialGroup()
113                     .addContainerGap()
114                     .addComponent(redJLabel)
115                     .addGap(20, 20, 20)
116                     .addComponent(redJSlider,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)
```

Fig. I.10 | `ColorChooser` class that uses `GroupLayout` for its GUI layout. (Part 3 of 6.)

```
117
.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
118                     .addComponent(redJTextField,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE))
119                 .addGroup(layout.createSequentialGroup()
120                     .addContainerGap()
121                     .addComponent(greenJLabel)
122
.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
123                     .addComponent(greenJSlider,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)
124                     .addGap(10, 10, 10)
125                     .addComponent(greenJTextField,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE))
126                 .addGroup(layout.createSequentialGroup()
127                     .addContainerGap()
128                     .addComponent(blueJLabel)
129                     .addGap(19, 19, 19)
130                     .addComponent(blueJSlider,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)
131
.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
132                     .addComponent(blueJTextField,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)))
133
.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
134                 .addComponent(colorJPanel,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)
135                 .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE))
136         );
137         layout.setVerticalGroup(
138
layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
139             .addGroup(layout.createSequentialGroup()
140                 .addContainerGap()
141
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILIN
G, false)
142                     .addComponent(colorJPanel,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE)
143                     .addGroup(javax.swing.GroupLayout.Alignment.LEADING,
layout.createSequentialGroup()
144
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.CENTER)
```

Fig. I.10 | `ColorChooser` class that uses `GroupLayout` for its GUI layout. (Part 4 of 6.)

```
145                         .addComponent(redJSlider,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)
146                         .addComponent(redJTextField,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)
147                         .addComponent(redJLabel))
148
.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
149
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.CENTER)
150                         .addComponent(greenJSlider,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)
151                         .addComponent(greenJLabel)
152                         .addComponent(greenJTextField,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE))
153
.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
154
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.CENTER)
155                         .addComponent(blueJLabel)
156                         .addComponent(blueJSlider,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)
157                         .addComponent(blueJTextField,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE))))
158                 .addContainerGap())
159         );
160
161         pack();
162      }// </editor-fold>
163
164      private void redJSliderStateChanged(javax.swing.event.ChangeEvent evt)
165      {
166         redJTextField.setText( String.valueOf( redJSlider.getValue() ) );
167         changeColor();
168      }
169
170      private void greenJSliderStateChanged(javax.swing.event.ChangeEvent
evt)
171      {
172         greenJTextField.setText( String.valueOf( greenJSlider.getValue() )
);
173         changeColor();
174      }
175
176      private void blueJSliderStateChanged(javax.swing.event.ChangeEvent
evt)
177      {
178         blueJTextField.setText( String.valueOf( blueJSlider.getValue() ) );
```

Fig. I.10 | `ColorChooser` class that uses `GroupLayout` for its GUI layout. (Part 5 of 6.)

```
179             changeColor();
180         }
181
182         /**
183         * @param args the command line arguments
184         */
185         public static void main(String args[]) {
186             java.awt.EventQueue.invokeLater(new Runnable() {
187                 public void run() {
188                     new ColorChooser().setVisible(true);
189                 }
190             });
191         }
192
193         // Variables declaration - do not modify
194         private javax.swing.JLabel blueJLabel;
195         private javax.swing.JSlider blueJSlider;
196         private javax.swing.JTextField blueJTextField;
197         private javax.swing.JPanel colorJPanel;
198         private javax.swing.JLabel greenJLabel;
199         private javax.swing.JSlider greenJSlider;
200         private javax.swing.JTextField greenJTextField;
201         private javax.swing.JLabel redJLabel;
202         private javax.swing.JSlider redJSlider;
203         private javax.swing.JTextField redJTextField;
204         // End of variables declaration
205
206 }
```

Fig. I.10 | `ColorChooser` class that uses `GroupLayout` for its GUI layout. (Part 6 of 6.)

I.4 GroupLayout Web Resources

`download.oracle.com/javase/6/docs/api/javax/swing/GroupLayout.html`
API documentation for class `GroupLayout`.

`wiki.java.net/bin/view/Javadesktop/GroupLayoutExample`
Provides an Address Book demo of a GUI built manually with `GroupLayout` with source code.

`www.developer.com/java/ent/article.php/3589961`
Tutorial: "Building Java GUIs with Matisse: A Gentle Introduction," by Dick Wall.



J

Java Desktop Integration Components

J.1 Introduction

The **Java Desktop Integration Components (JDIC)** are part of an open-source project aimed at allowing better integration between Java applications and the platforms on which they execute. Some JDIC features include:

- interacting with the underlying platform to launch native applications (such as web browsers and e-mail clients)
- displaying a splash screen when an application begins execution to indicate to the user that the application is loading
- creating icons in the system tray (also called the taskbar status area or notification area) to provide access to Java applications running in the background
- registering file-type associations, so that files of specified types will automatically open in corresponding Java applications
- creating installer packages, and more.

The JDIC homepage (`jdic.dev.java.net/`) includes an introduction to JDIC, downloads, documentation, FAQs, demos, articles, blogs, announcements, incubator projects, a developer's page, forums, mailing lists, and more. Java SE 6 includes some of the features mentioned above. We discuss several of these features here.

J.2 Splash Screens

Java application users often perceive a performance problem, because nothing appears on the screen when you first launch an application. One way to show a user that your program is loading is to display a **splash screen**—a borderless window that appears temporarily while an application loads. Java SE 6 provides the new command-line option **`-splash`** for the `java` command to accomplish this task. This option enables you to specify a PNG, GIF or JPG image that should display when your application begins loading. To demonstrate this new option, we created a program (Fig. J.1) that sleeps for 5 seconds (so you can view the splash screen) then displays a message at the command line. The directory for this example includes a PNG format image to use as the splash screen. To display the splash screen when this application loads, use the command

```
java -splash:DeitelBug.png SplashDemo
```

```
1  // Fig. J.1: SplashDemo.java
2  // Splash screen demonstration.
3  public class SplashDemo
4  {
5     public static void main( String[] args )
6     {
7        try
8        {
9           Thread.sleep( 5000 );
10       } // end try
11       catch ( InterruptedException e )
12       {
13          e.printStackTrace();
14       } // end catch
15
16       System.out.println(
17          "This was the splash screen demo." );
18    } // end method main
19 } // end class SplashDemo
```

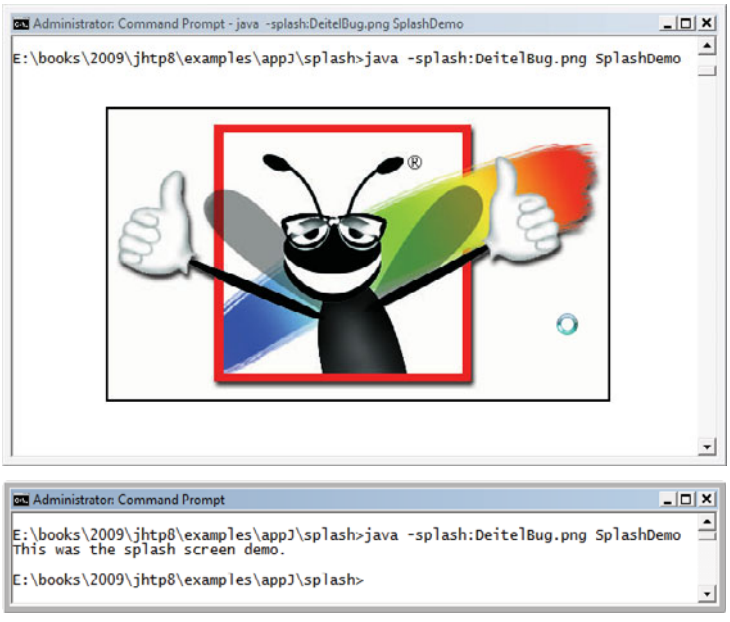


Fig. J.1 | Spash screen displayed with the `-splash` option to the `java` command.

Once you've initiated the splash screen display, you can interact with it programmatically via the **`SplashScreen`** class of the `java.awt` package. You might do this to add some dynamic content to the splash screen. For more information on working with splash screens, see the following sites:

```
java.sun.com/developer/technicalArticles/J2SE/Desktop/javase6/
   splashscreen/
download.oracle.com/javase/6/docs/api/java/awt/SplashScreen.html
```

J.3 Desktop Class

The **Desktop** class enables you to specify a file or URI that you'd like to open using the underlying platform's appropriate application. For example, if Firefox is your computer's default browser, you can use the Desktop class's browse method to open a website in Firefox. In addition, you can open an e-mail composition window in your system's default e-mail client, open a file in its associated application and print a file using the associated application's print command. Figure J.2 demonstrates the first three of these capabilities.

The event handler at lines 86–116 obtains the index number of the task the user selects in the tasksJComboBox (line 89) and the String that represents the file or URI to process (line 90). Line 92 uses Desktop static method **isDesktopSupported** to determine whether class Desktop's features are supported on the platform on which this application runs. If they are, line 96 uses Desktop static method **getDesktop** to obtain a Desktop object. If the user selected the option to open the default browser, line 101 creates a new URI object using the String input as the site to display in the browser, then passes the URI object to Desktop method **browse** which invokes the system's default browser and passes the URI to the browser for display. If the user selects the option to open a file in its associated program, line 104 creates a new File object using the String input as the file to open, then passes the File object to Desktop method **open** which passes the file to the appropriate application to open the file. Finally, if the user selects the option to compose an e-mail, line 107 creates a new URI object using the String input as the e-mail address to which the e-mail will be sent, then passes the URI object to Desktop method **mail** which invokes the system's default e-mail client and passes the URI to the e-mail client as the e-mail recipient. You can learn more about class Desktop at

```
download.oracle.com/javase/6/docs/api/java/awt/Desktop.html
```

```
1  // Fig. J.2: DesktopDemo.java
2  // Use Desktop to launch default browser, open a file in its associated
3  // application and an email in the default email client.
4  import java.awt.Desktop;
5  import java.io.File;
6  import java.io.IOException;
7  import java.net.URI;
8
9  public class DesktopDemo extends javax.swing.JFrame
10 {
11    // constructor
12    public DesktopDemo()
13    {
14       initComponents();
15    } // end DesktopDemo constructor
16
17    // To save space, lines 20-84 of the NetBeans autogenerated GUI code
18    // are not shown here. The complete code for this example is located in
19    // the file DesktopDemo.java in this example's directory.
20
```

Fig. J.2 | Use Desktop to launch the default browser, open a file in its associated application and compose an e-mail in the default e-mail client. (Part 1 of 3.)

```
21      // determine selected task and perform the task
22      private void doTaskJButtonActionPerformed(
23         java.awt.event.ActionEvent evt)
24      {
25         int index = tasksJComboBox.getSelectedIndex();
26         String input = inputJTextField.getText();
27
28         if ( Desktop.isDesktopSupported() )
29         {
30            try
31            {
32               Desktop desktop = Desktop.getDesktop();
33
34               switch ( index )
35               {
36                  case 0: // open browser
37                     desktop.browse( new URI( input ) );
38                     break;
39                  case 1: // open file
40                     desktop.open( new File( input ) );
41                     break;
42                  case 2: // open email composition window
43                     desktop.mail( new URI( input ) );
44                     break;
45               } // end switch
46            } // end try
47            catch ( Exception e )
48            {
49               e.printStackTrace();
50            } // end catch
51         } // end if
52      } // end method doTaskJButtonActionPerformed
53
54      public static void main(String[] args)
55      {
56         java.awt.EventQueue.invokeLater(
57            new Runnable()
58            {
59               public void run()
60               {
61                  new DesktopDemo().setVisible(true);
62               }
63            }
64         );
65      } // end method main
66
67      // Variables declaration - do not modify
68      private javax.swing.JButton doTaskJButton;
69      private javax.swing.JLabel inputJLabel;
70      private javax.swing.JTextField inputJTextField;
71      private javax.swing.JLabel instructionLabel;
```

Fig. J.2 | Use `Desktop` to launch the default browser, open a file in its associated application and compose an e-mail in the default e-mail client. (Part 2 of 3.)

```
72      private javax.swing.JComboBox tasksJComboBox;
73      // End of variables declaration
74  }
```

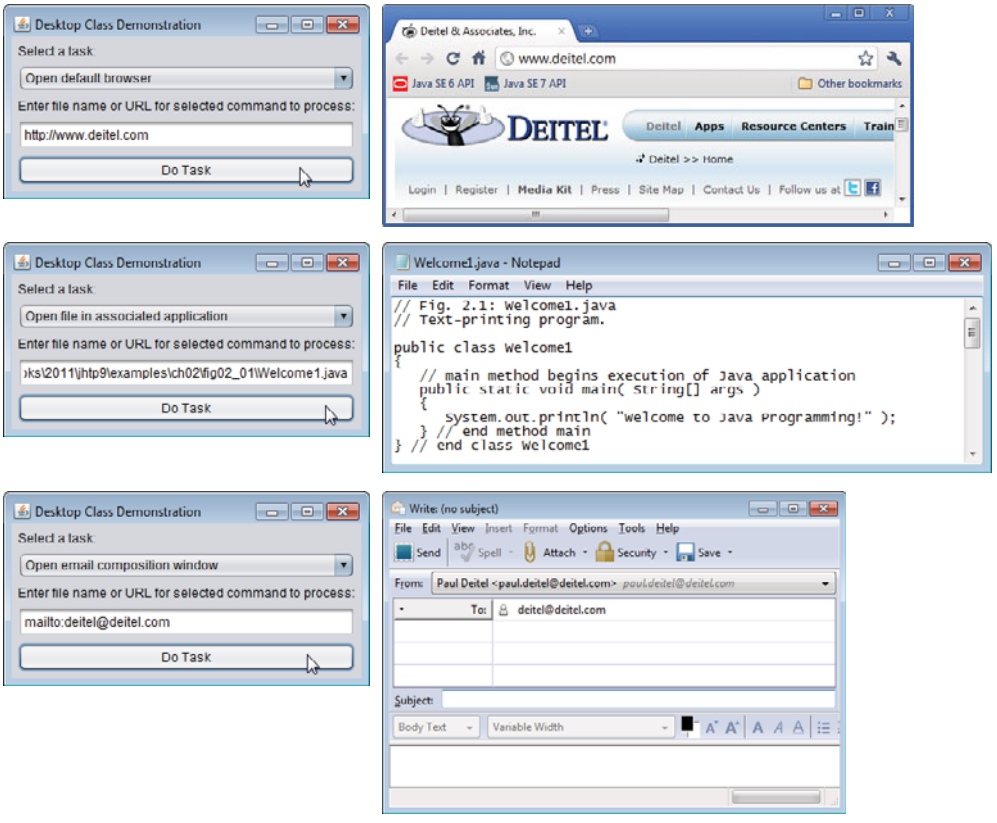


Fig. J.2 | Use `Desktop` to launch the default browser, open a file in its associated application and compose an e-mail in the default e-mail client. (Part 3 of 3.)

J.4 Tray Icons

Tray icons generally appear in your system's system tray, taskbar status area or notification area. They typically provide quick access to applications that are executing in the background on your system. When you position the mouse over one of these icons, a tooltip appears indicating what application the icon represents. If you click the icon, a popup menu appears with options for that application.

Classes `SystemTray` and `TrayIcon` (both from package `java.awt`) enable you to create and manage your own tray icons in a platform independent manner. Class **`SystemTray`** provides access to the underlying platform's system tray—the class consists of three methods:

- `static` method **`getDefaultSystemTray`** returns the system tray
- method **`addTrayIcon`** adds a new `TrayIcon` to the system tray
- method **`removeTrayIcon`** removes an icon from the system tray

Class **TrayIcon** consists of several methods allowing users to specify an icon, a tooltip and a pop-up menu for the icon. In addition, tray icons support `ActionListeners`, `MouseListeners` and `MouseMotionListeners`. You can learn more about classes `SystemTray` and `TrayIcon` at

```
download.oracle.com/javase/6/docs/api/java/awt/SystemTray.html
download.oracle.com/javase/6/docs/api/java/awt/TrayIcon.html
```



K

Mashups

K.1 Introduction

Building web application mashups is one of the signature features of Web 2.0. The term mashup originated in the music world—a music mashup is a remix of two or more songs to create a new song. You can listen to some music mashups at `www.ccmixter.org/`. A web application mashup combines complementary functionality, usually accessed via web services (Chapter 31) and RSS feeds (`www.deitel.com/rss` and `www.rssbus.com`) from multiple websites. You can create innovative and powerful Web 2.0 mashup applications much faster than if you have to write your applications from scratch. For example, `www.housingmaps.com` combines Craigslist apartment listings with Google Maps to display on a map all of the apartments for rent in a neighborhood.

K.2 Popular Mashups

Figure K.1 shows some popular mashups.

URL	APIs	Description
`www.mappr.com/`	Google Maps, FlickR	Find photos of sites across the US.
`www.housingmaps.com/`	Google Maps, Craigslist	Find available apartments and homes by neighborhood. Includes prices, pictures, the address and rental-agent contact information.
`www.estately.com/`	Google Maps	Find the approximate value of your home based on recent home sales in your area.
`www.liveplasma.com/`	Amazon eCommerce	Enter the name of a musician, band, actor, director or movie. The app displays related albums, movies, etc. Click on the images to go to the page on Amazon where you can purchase the movie or album.

Fig. K.1 | Popular mashups. (Part 1 of 2.)

URL	APIs	Description
www.secretprices.com/default.aspx	Shopping.com, Amazon A9 OpenSearch, Amazon eCommerce	Shopping comparison site that also offers promotions and coupons.
www.checkinmania.com/	FourSquare, Gowalla, Google Maps	Track the people checking in at nearby locations using Foursquare and Gowalla.
appexchange.salesforce.com/listingDetail?listingId=a0330000003z9bdAAA	Facebook, Salesforce.com	Integrates Facebook profile information for your friends into your Salesforce customer relationship management (CRM) database.

Fig. K.1 | Popular mashups. (Part 2 of 2.)

Now that you've read most of *Java How to Program, 9/e*, you're probably familiar with API categories including graphics, GUI, collections, multimedia, databases and many more. Nearly all of these provide enhanced *computing functionality*. Many web services APIs provide *business functionality*—eBay provides auction capabilities, Amazon provides book sales (and sales of other types of products, such as CDs, DVDs, electronic devices, and more), Google provides search capabilities, PayPal provides payment services, etc. These web services are typically free for non-commercial use; some impose (generally reasonable) fees for commercial use. This creates exciting possibilities for people building Internet-based applications and businesses.

K.3 APIs Commonly Used in Mashups

We've emphasized the importance of software reuse. Mashups are yet another form of software reuse that saves you time, money and effort—you can rapidly prototype starter versions of your applications, integrate business functionality, integrate search functionality and more. Figure 1.19 shows some APIs commonly used in mashups.

K.4 Deitel Mashups Resource Center

Our Mashups Resource Center, which is located at

```
www.deitel.com/mashups/
```

focuses on the enormous amount of free mashup content available online. You'll find tutorials, articles, documentation, the latest books, articles, blogs, directories, tools, forums, etc., that will help you quickly develop mashup applications.

- Check out the newest and most popular mashups, including scores of Google Maps-based mashups showing you the locations of theaters, real estate for sale or rent, properties that have sold in your area, and even the locations of the public restrooms in San Francisco!
- Search ProgrammableWeb for mashups by category.

- Check out the Flickr APIs for adding photos to your applications, updating photos, replacing photos, example requests, and asynchronous uploading.
- Check out the article: "Building Mashups for Non-Programmers."
- Check out the Smashforce tool that enables Salesforce.com users to mashup applications such as Google Maps with their Multiforce and Sforce enterprise applications.
- Check out IBM's Enterprise Mashup Tool.
- Check out the search and mapping APIs from Microsoft, Yahoo! and Google that you can use in your mashup applications.
- Use Technorati APIs to find all of the blogs that link to a specific website, search blogs for mentions of keywords, see which blogs are linked to a given blog and find blogs associated with a specific website.
- Use the Backpack API to help organize tasks and events, plan your schedule, collaborate with others, monitor your competitors online and more.

K.5 Deitel RSS Resource Center

RSS feeds are also popular information sources for mashups. To learn more about RSS feeds, visit our RSS Resource Center at `www.deitel.com/RSS/`.

K.6 Mashup Performance and Reliability Issues

There are several challenges when creating mashup applications. Your applications become susceptible to traffic and reliability problems on the Internet—circumstances generally beyond your control. Companies might suddenly change APIs that your applications use. Your application is dependent on hardware and software capabilities of other companies. Also, companies could impose fee structures on previously free web services or could increase existing fees.



L Unicode®

L.1 Introduction

The use of inconsistent **character encodings** (i.e., numeric values associated with characters) when developing global software products causes serious problems because computers process information using numbers. For example, the character "a" is converted to a numeric value so that a computer can manipulate that piece of data. Many countries and corporations have developed encoding systems that are incompatible with the encoding systems of other countries and corporations. For example, the Microsoft Windows operating system assigns the value `0xC0` to the character "A with a grave accent," while the Apple Macintosh operating system assigns the same value to an upside-down question mark. This results in the misrepresentation and possible corruption of data.

In the absence of a universal character encoding standard, global software developers had to localize their products extensively before distribution. **Localization** includes the language translation and cultural adaptation of content. The process of localization usually includes significant modifications to the source code (e.g., the conversion of numeric values and the underlying assumptions made by programmers), which results in increased costs and delays in releasing the software. For example, an English-speaking programmer might design a global software product assuming that a single character can be represented by one byte. However, when those products are localized in Asian markets, the programmer's assumptions are no longer valid because there are many more Asian characters, and therefore most, if not all, of the code needs to be rewritten. Localization is necessary with each release of a version. By the time a software product is localized for a particular market, a newer version, which needs to be localized as well, can be ready for distribution. As a result, it's cumbersome and costly to produce and distribute global software products in a market where there's no universal character encoding standard.

In response to this situation, the **Unicode Standard**, an encoding standard that facilitates the production and distribution of software, was created. The Unicode Standard outlines a specification to produce consistent encoding of the world's characters and symbols. Software products which handle text encoded in the Unicode Standard need to be localized, but the localization process is simpler and more efficient because the numeric values need not be converted and the assumptions made by programmers about the character encoding are universal. The Unicode Standard is maintained by a non-profit organization called the **Unicode Consortium**, whose members include Apple, IBM, Microsoft, Oracle, Sun Microsystems, Sybase and many others.

When the Consortium envisioned and developed the Unicode Standard, it wanted an encoding system that was **universal**, **efficient**, **uniform** and **unambiguous**. A universal encoding system encompasses all commonly used characters. An efficient encoding system allows text files to be parsed quickly. A uniform encoding system assigns fixed values to all characters. An unambiguous encoding system represents a given character in a consistent manner. These four terms are referred to as the Unicode Standard design basis.

L.2 Unicode Transformation Formats

Although Unicode incorporates the limited ASCII character set (i.e., a collection of characters), it encompasses a more comprehensive character set. In ASCII each character is represented by a byte containing 0s and 1s. One byte is capable of storing the binary numbers from 0 to 255. Each character is assigned a number between 0 and 255, thus ASCII-based systems can support only 256 characters, a tiny fraction of the world's characters. Unicode extends the ASCII character set by encoding the vast majority of the world's characters. The Unicode Standard encodes characters in a uniform numerical space from 0 to 10FFFF hexadecimal. An implementation will express these numbers in one of several transformation formats, choosing the one that best fits the particular application at hand.

Three such formats are in use, called **UTF-8**, **UTF-16** and **UTF-32**. UTF-8, a variable-width encoding form, requires one to four bytes to express each Unicode character. UTF-8 data consists of 8-bit bytes (sequences of one, two, three or four bytes depending on the character being encoded) and is well suited for ASCII-based systems when there's a predominance of one-byte characters (ASCII represents characters as one-byte). Currently, UTF-8 is widely implemented in UNIX systems and in databases.

The variable-width UTF-16 encoding form expresses Unicode characters in units of 16-bits (i.e., as two adjacent bytes, or a short integer in many machines). Most characters of Unicode are expressed in a single 16-bit unit. However, characters with values above FFFF hexadecimal are expressed with an ordered pair of 16-bit units called **surrogates**. Surrogates are 16-bit integers in the range D800 through DFFF, which are used solely for the purpose of "escaping" into higher numbered characters. Approximately one million characters can be expressed in this manner. Although a surrogate pair requires 32 bits to represent characters, it's space-efficient to use these 16-bit units. Surrogates are rare characters in current implementations. Many string-handling implementations are written in terms of UTF-16. [*Note:* Details and sample-code for UTF-16 handling are available on the Unicode Consortium website at `www.unicode.org`.]

Implementations that require significant use of rare characters or entire scripts encoded above FFFF hexadecimal, should use UTF-32, a 32-bit fixed-width encoding form that usually requires twice as much memory as UTF-16 encoded characters. The major advantage of the fixed-width UTF-32 encoding form is that it expresses all characters uniformly, so it's easy to handle in arrays.

There are few guidelines that state when to use a particular encoding form. The best encoding form to use depends on the computer system and business protocol, not on the data itself. Typically, the UTF-8 encoding form should be used where computer systems and business protocols require data to be handled in 8-bit units, particularly in legacy systems being upgraded, because it often simplifies changes to existing programs. For this reason, UTF-8 has become the encoding form of choice on the Internet. Likewise, UTF-16 is the encoding form of choice on Microsoft Windows applications. UTF-32 is likely

to become more widely used in the future as more characters are encoded with values above FFFF hexadecimal. UTF-32 requires less sophisticated handling than UTF-16 in the presence of surrogate pairs. Figure L.1 shows the different ways in which the three encoding forms handle character encoding.

Character	UTF-8	UTF-16	UTF-32
LATIN CAPITAL LETTER A	0x41	0x0041	0x00000041
GREEK CAPITAL LETTER ALPHA	0xCD 0x91	0x0391	0x00000391
CJK UNIFIED IDEOGRAPH-4E95	0xE4 0xBA 0x95	0x4E95	0x00004E95
OLD ITALIC LETTER A	0xF0 0x80 0x83 0x80	0xDC00 0xDF00	0x00010300

Fig. L.1 | Correlation between the three encoding forms.

L.3 Characters and Glyphs

The Unicode Standard consists of characters—written components (i.e., alphabets, numbers, punctuation marks, accent marks, etc.) that can be represented by numeric values. An example of such a character is U+0041 LATIN CAPITAL LETTER A. In the first character representation, **U+*yyyy*** is a **code value**, in which U+ refers to Unicode code values, as opposed to other hexadecimal values. The *yyyy* represents a four-digit hexadecimal number of an encoded character. Code values are bit combinations that represent encoded characters. Characters are represented using **glyphs**—various shapes, fonts and sizes for displaying characters. There are no code values for glyphs in the Unicode Standard. Examples of glyphs are shown in Fig. L.2.

The Unicode Standard encompasses the alphabets, ideographs, syllabaries, punctuation marks, **diacritics**, mathematical operators and other features that comprise the written languages and scripts of the world. A diacritic is a special mark added to a character to distinguish it from another letter or to indicate an accent (e.g., in Spanish, the tilde "~" above the character "n"). Currently, Unicode provides code values for 96,382 character representations, with more than 878,000 code values reserved for future expansion.



Fig. L.2 | Various glyphs of the character A.

L.4 Advantages/Disadvantages of Unicode

The Unicode Standard has several significant advantages that promote its use. One is the impact it has on the performance of the international economy. Unicode standardizes the characters for the world's writing systems to a uniform model that promotes transferring and sharing data. Programs developed using such a schema maintain their accuracy be-

cause each character has a single definition (i.e., *a* is always U+0061, *%* is always U+0025). This enables corporations to manage the high demands of international markets by processing different writing systems at the same time. All characters can be managed in an identical manner, thus avoiding any confusion caused by different character-code architectures. Moreover, managing data in a consistent manner eliminates data corruption, because data can be sorted, searched and manipulated using a consistent process.

Another advantage of the Unicode Standard is portability (i.e., software that can execute on disparate computers or with disparate operating systems). Most operating systems, databases, programming languages (including Java and Microsoft's .NET languages) and web browsers currently support, or are planning to support, Unicode.

A disadvantage of the Unicode Standard is the amount of memory required by UTF-16 and UTF-32. ASCII character sets are 8-bits in length, so they require less storage than the default 16-bit Unicode character set. The **double-byte character set** (**DBCS**) encodes Asian characters with one or two bytes per character. The **multibyte character set** (**MBCS**) encodes characters with a variable number of bytes per character. In such instances, the UTF-16 or UTF-32 encoding forms may be used with little hindrance on memory and performance.

Another disadvantage of Unicode is that although it includes more characters than any other character set in common use, it does not yet encode all of the world's written characters. Also, UTF-8 and UTF-16 are variable-width encoding forms, so characters occupy different amounts of memory.

L.5 Using Unicode

Numerous programming languages (e.g., C, Java, JavaScript, Perl, Visual Basic) provide some level of support for the Unicode Standard. The application shown in Fig. L.3–Fig. L.4 prints the text "Welcome to Unicode!" in eight different languages: English, Russian, French, German, Japanese, Portuguese, Spanish and Traditional Chinese.

```
1  // Fig. L.3: UnicodeJFrame.java
2  // Demonstrating how to use Unicode in Java programs.
3  import java.awt.GridLayout;
4  import javax.swing.JFrame;
5  import javax.swing.JLabel;
6
7  public class UnicodeJFrame extends JFrame
8  {
9     // constructor creates JLabels to display Unicode
10    public UnicodeJFrame()
11    {
12       super( "Demonstrating Unicode" );
13
14       setLayout( new GridLayout( 8, 1 ) ); // set frame layout
15
16       // create JLabels using Unicode
17       JLabel englishJLabel = new JLabel( "\u0057\u0065\u006C\u0063" +
18          "\u006F\u006D\u0065\u0020\u0074\u006F\u0020Unicode\u0021" );
```

Fig. L.3 | Java application that uses Unicode encoding (Part 1 of 2.).

```
19         englishJLabel.setToolTipText( "This is English" );
20         add( englishJLabel );
21
22         JLabel chineseJLabel = new JLabel( "\u6B22\u8FCE\u4F7F\u7528" +
23            "\u0020\u0020Unicode\u0021" );
24         chineseJLabel.setToolTipText( "This is Traditional Chinese" );
25         add( chineseJLabel );
26
27         JLabel cyrillicJLabel = new JLabel( "\u0414\u043E\u0431\u0440" +
28            "\u043E\u0020\u043F\u043E\u0436\u0430\u043B\u043E\u0432" +
29            "\u0430\u0442\u044A\u0020\u0432\u0020Unicode\u0021" );
30         cyrillicJLabel.setToolTipText( "This is Russian" );
31         add( cyrillicJLabel );
32
33         JLabel frenchJLabel = new JLabel( "\u0042\u0069\u0065\u006E\u0076" +
34            "\u0065\u006E\u0075\u0065\u0020\u0061\u0075\u0020Unicode\u0021");
35         frenchJLabel.setToolTipText( "This is French" );
36         add( frenchJLabel );
37
38         JLabel germanJLabel = new JLabel( "\u0057\u0069\u006C\u006B\u006F" +
39            "\u006D\u006D\u0065\u006E\u0020\u007A\u0075\u0020Unicode\u0021");
40         germanJLabel.setToolTipText( "This is German" );
41         add( germanJLabel );
42
43         JLabel japaneseJLabel = new JLabel( "Unicode\u3078\u3087\u3045" +
44            "\u3053\u305D\u0021" );
45         japaneseJLabel.setToolTipText( "This is Japanese" );
46         add( japaneseJLabel );
47
48         JLabel portugueseJLabel = new JLabel( "\u0053\u00E9\u006A\u0061" +
49            "\u0020\u0042\u0065\u006D\u0076\u0069\u006E\u0064\u006F\u0020" +
50            "Unicode\u0021" );
51         portugueseJLabel.setToolTipText( "This is Portuguese" );
52         add( portugueseJLabel );
53
54         JLabel spanishJLabel = new JLabel( "\u0042\u0069\u0065\u006E" +
55            "\u0076\u0065\u006E\u0069\u0064\u0061\u0020\u0061\u0020" +
56            "Unicode\u0021" );
57         spanishJLabel.setToolTipText( "This is Spanish" );
58         add( spanishJLabel );
59      } // end UnicodeJFrame constructor
60   } // end class UnicodeJFrame
```

Fig. L.3 | Java application that uses Unicode encoding (Part 2 of 2.).

```
1  // Fig. L.4: Unicode.java
2  // Displaying Unicode.
3  import javax.swing.JFrame;
4
5  public class Unicode
6  {
```

Fig. L.4 | Displaying Unicode. (Part 1 of 2.)

```
7      public static void main( String[] args )
8      {
9         UnicodeJFrame unicodeJFrame = new UnicodeJFrame();
10        unicodeJFrame.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
11        unicodeJFrame.setSize( 350, 250 );
12        unicodeJFrame.setVisible( true );
13     } // end method main
14  } // end class Unicode
```

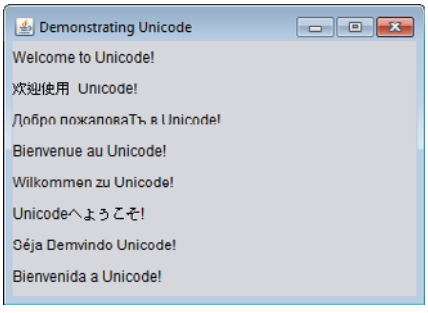


Fig. L.4 | Displaying Unicode. (Part 2 of 2.)

Class `UnicodeJFrame` (Fig. L.3) uses escape sequences to represent characters. An escape sequence is in the form `\u`*yyyy*, where *yyyy* represents the four-digit hexadecimal code value. Lines 17–18 contain the series of escape sequences necessary to display "Welcome to Unicode!" in English. The first escape sequence (`\u0057`) equates to the character "W," the second escape sequence (`\u0065`) equates to the character "e," and so on. The `\u0020` escape sequence (line 18) is the encoding for the space character. The `\u0074` and `\u006F` escape sequences equate to the word "to." "Unicode" is not encoded because it's a registered trademark and has no equivalent translation in most languages. Line 18 also contains the `\u0021` escape sequence for the exclamation point (`!`).

Lines 22–56 contain the escape sequences for the other seven languages. The Unicode Consortium's website contains a link to code charts that lists the 16-bit Unicode code values. The English, French, German, Portuguese and Spanish characters are located in the **Basic Latin** block, the Japanese characters are located in the **Hiragana** block, the Russian characters are located in the **Cyrillic** block and the Traditional Chinese characters are located in the **CJK Unified Ideographs** block. The next section discusses these blocks.

L.6 Character Ranges

The Unicode Standard assigns code values, which range from `0000` (**Basic Latin**) to `E007F` (**Tags**), to the written characters of the world. Currently, there are code values for 96,382 characters. To simplify the search for a character and its associated code value, the Unicode Standard generally groups code values by **script** and function (i.e., Latin characters are grouped in a block, mathematical operators are grouped in another block, etc.). As a rule, a script is a single writing system that is used for multiple languages (e.g., the Latin script is used for English, French, Spanish, etc.). The **Code Charts** page on the Unicode Consortium website lists all the defined blocks and their respective code values. Figure L.5 lists some blocks (scripts) from the website and their range of code values.

Script	Range of code values
Arabic	U+0600–U+06FF
Basic Latin	U+0000–U+007F
Bengali (India)	U+0980–U+09FF
Cherokee (Native America)	U+13A0–U+13FF
CJK Unified Ideographs (East Asia)	U+4E00–U+9FFF
Cyrillic (Russia and Eastern Europe)	U+0400–U+04FF
Ethiopic	U+1200–U+137F
Greek	U+0370–U+03FF
Hangul Jamo (Korea)	U+1100–U+11FF
Hebrew	U+0590–U+05FF
Hiragana (Japan)	U+3040–U+309F
Khmer (Cambodia)	U+1780–U+17FF
Lao (Laos)	U+0E80–U+0EFF
Mongolian	U+1800–U+18AF
Myanmar	U+1000–U+109F
Ogham (Ireland)	U+1680–U+169F
Runic (Germany and Scandinavia)	U+16A0–U+16FF
Sinhala (Sri Lanka)	U+0D80–U+0DFF
Telugu (India)	U+0C00–U+0C7F
Thai	U+0E00–U+0E7F

Fig. L.5 | Some character ranges.



Appendices on the Web

The following appendices are available at Java How to Program, 9/e's Companion Website (`www.pearsonhighered.com/deitel`) as PDF documents:

- Appendix M, Creating Documentation with `javadoc`
- Appendix N, Bit Manipulation
- Appendix O, Labeled `break` and `continue` Statements
- Appendix P, UML 2: Additional Diagram Types
- Appendix Q, Design Patterns

These files can be viewed in Adobe® Reader® (`get.adobe.com/reader`).

This page intentionally left blank



Index

[*Note:* Page references for defining occurrences of terms appear in **bold maroon**. Page references for online appendices are shown in red or **bold red** (for defining occurrences). The online appendic page references also include the appendix letter and an underscore, as in M_1. You can find the online appendices on the book's Companion Website at `www.pearsonhighered.com/deitel/`.]

Symbols

^, bitwise exclusive OR **N_1**
^, boolean logical exclusive OR 174, **176**
  truth table 176
^=, bitwise exclusive OR assignment operator N_11
_ SQL wildcard character **1179**, 1180
, (comma) formatting flag **161**
--, predecrement/ postdecrement **130**
-, subtraction 53, 54
!, logical NOT 174, **176**
  truth table 177
!=, not equals 57
? (wildcard type argument) **897**
?:, ternary conditional operator **108**, 133
. dot separator **75**
, flag **1407**
(flag **1407**
{, left brace 40
}, right brace 40
@ symbol **M_1**
@ApplicationScoped annotation **1275**
@HttpSessionScope annotation **1327**
@Override annotation **368**
@Resource annotation **1285**
* SQL wildcard character **1178**
* wildcard in a file name 75
*, multiplication 53, 54
*=, multiplication assignment operator 131
/ forward slash in end tags **958**
/, division 53, 54
/* */ traditional comment **39**
/** */ Java documentation comment 39, **M_1**
//, end-of-line comment **39**
/=, division assignment operator 131
\, backslash escape sequence 46
\', single-quote-character escape sequence 1411
\", double-quote escape sequence 46, 1411
\\, backslash-character escape sequence 1411
\b, escape sequence 1411
\f, form-feed escape sequence 1412
\n, newline escape sequence 45, 46, 1412
\r, carriage-return escape sequence 46, 1412
\t, horizontal tab escape sequence 46, 1412
&, bitwise AND **N_1**
&, boolean logical AND 174, **175**
&&, conditional AND **174**, 175
  truth table 174
&=, bitwise AND assignment operator N_11
character **M_7**
flag **1407**, 1408
% conversion character **1403**
% SQL wildcard character **1179**
%, remainder 53, 54
%% format specifier 1405
%=, remainder assignment operator 131
%A format specifier 1399
%a format specifier 1399
%B format specifier **1403**
%b format specifier **177**, **1403**, 1404
%C format specifier **1400**
%c format specifier 68, **1400**
%d format specifier **51**, 1397, **1397**, 1398
%E format specifier **1398**, 1399
%e format specifier **1398**, 1399
%f format specifier 68, **90**, **1398**, **1399**
%G format specifier **1399**
%g format specifier **1399**
%H format specifier **1403**
%h format specifier 1404
%n format specifier 1404
%o format specifier **1397**, 1398
%S format specifier **1400**
%s format specifier **47**, 1397, **1400**
%T format specifier **1401**
%t format specifier **1401**
%X format specifier **1397**
%x format specifier **1397**
- flag **1407**
+ flag **1407**
- (minus sign) formatting flag **160**
+, addition 53, 54
++, preincrement/ postincrement **130**
+=, addition assignment operator **130**
+=, string concatenation assignment operator 688
<, less than 57
<<, left shift **N_1**, N_8
<<=, left shift assignment operator N_11
<=, less than or equal 57
<> diamond notation for generic type inference (Java SE 7) **836**
<>, angle brackets for XML elements **958**
=, assignment operator 51
-=, subtraction assignment operator 131
== to determine whether two references refer to the same object 387
==, is equal to 57
>, greater than 57
>=, greater than or equal to 57
>>, signed right shift **N_1**, N_8
>>=, signed right shift assignment operator N_11
>>>, unsigned right shift **N_1**, N_8
>>>=, unsigned right shift assignment operator N_11
|, bitwise inclusive OR **N_1**
|, boolean logical inclusive OR 174, **175**
|=, bitwise inclusive OR assignment operator N_11
||, conditional OR **174**
  truth table 175
~ (bitwise complement) **N_1**
~, bitwise complement N_2

Numerics

0 flag **250**
0 format flag 314
0x (hexadecimal prefix) 1408
127.0.0.1 (localhost IP address) **1136**, 1237
15 Puzzle exercise 997

A

abbreviating assignment expressions 130
abs method of Math 201
absolute method of ResultSet **1200**
absolute path **722**, 723, 726
absolute value 201
abstract class 396, **400**, 401, 402, 420, Q_7
abstract data type (ADT) 313
Abstract Factory design pattern Q_3, Q_4, Q_15
abstract implementation 867
abstract keyword **401**
abstract method **401**, 403, 404, 521, 568, 1368
abstract superclass **401**, 521
Abstract Window Toolkit (AWT) **555**
  package **209**
Abstract Window Toolkit Event package **209**
AbstractButton class **571**, 573, 1007, 1012
  addActionListener method 574
  addItemListener method **576**
  isSelected method **1014**
  setMnemonic method **1012**

AbstractButton class (cont.)
setRolloverIcon method 573
setSelected method 1013
AbstractCollection class 867
AbstractList class 867
AbstractMap class 867
AbstractQueue class 867
AbstractSequentialList class 867
AbstractSet class 867
AbstractTableModel class 1194, 1200
fireTableStructureChanged method 1200
accelerometer 5, 8
accept method of class ServerSocket 1128, 1135
access modifier 72, 80, 511
private 80, 316, 363
protected 316, 363
public 72, 316, 363
access modifier in the UML
- (private) 83
+ (public) 76
access shared data 1069
accessibility 556
accessibility heuristic 300
Accessibility Project
Speech Recognition 998
Speech Synthesis 998
accessor method 327
Account class (ATM case study) 477, 480, 483, 485, 486, 493, 500, 501, 502, 504, 505, 532
Accounts Payable System Modification exercise 436, 437
accounts receivable file 763
accumulator register 304, 307
ACM/IEEE curriculum recommendations and the Computer Science Advanced Placement Examination xxiii
acquire the lock 1055
action 107, 112
action expression in the UML 105, 490
action key 601
action of an object 490
action state in the UML 105, 181, 490
action state symbol 105
action to execute 103
ActionEvent class 565, 566, 570, 617, 979
getActionCommand method 566, 574
ActionListener interface 565, 570
actionPerformed method 565, 569, 611, 617
actionPerformed method of interface ActionListener 565, 569, 611, 617
ACTIVATED constant of nested class EventType 1127
activation in a UML sequence diagram 504
activation record 207, 777
activity diagram 105, 108, 157, 181
do...while statement 163
for statement 157
if statement 107
if...else statement 108
in the UML 112, 477, 490, 491, 508
sequence statement 105
switch statement 170
while statement 113
activity in the UML 105, 477, 489, 492
actor in use case in the UML 475
actual type arguments 878
acyclic gradient 660
Ada Lovelace 16
Ada programming language 16, 1046
adapter class 594
Adapter Classes used to implement event handlers 598
Adapter design pattern Q_3, Q_5, Q_8
add a web service reference to an application in NetBeans 1311
add an event handler in Netbeans 1436
add method
ArrayList<T> 286, 1123
BigInteger 773
ButtonGroup 580
JFrame 390, 559
JFrame class 137
JMenu 1012
JMenuBar 1013
LinkedList<T> 841
List<T> 836, 838
addActionListener method of class AbstractButton 574
of class JTextField 565
addAll method
Collections 841, 851
List 838
addFirst method of LinkedList 841
addGap method of class GroupLayout.Group 1432
addGap method of class GroupLayout.ParallelGroup 1432
addGap method of class GroupLayout.SequentialGroup 1432
adding a web service reference to an application 1310
Adding Object Serialization to the MyShape Drawing Application (exercise) 764
addItemListener method of class AbstractButton 576
addition 9, 53, 54
addition compound assignment operator, += 130
addKeyListener method of class Component 601
addLast method of LinkedList 840
addListSelectionListener method of class JList 586
addMouseListener method of class Component 593
addMouseMotionListener method of class Component 593
addPoint method of class Polygon 655, 657
AddressBook Application Modification exercise 1298
addSeparator method of class JMenu 1013
addTab method of class JTabbedPane 1027
addTableModelListener method of TableModel 1194
addTrayIcon method of class SystemTray 1447
addWindowListener method of class Window 1006
"administrative" section of the computer 9
advertisement 1261
aggregation in the UML 482
Agile Alliance 29
Agile Manifesto 29
agile software development xxvi, 29, 29
.aif file extension 985, 988
.aiff file extension 985, 988
Airline Reservation System 297
Airline Reservation Web-Service Modification 1364
Ajax 1277
id attributes for elements 1295
Ajax (Asynchronous JavaScript and XML) 27, 1290, 1291
Ajax-enabled
web applications xxv
Ajax request 1295
Ajax web application 1291
algebraic notation 54
algorithm 103, 113, 121, 774, Q_12
binary search 804
bubble sort 826
bucket sort 826
in Java Collections Framework 841
insertion sort 814
algorithm (cont.)
linear search 800
merge sort 817
quicksort 827
recursive binary search 827
recursive linear search 827
selection sort 810
aligning components in GroupLayout 1432
aligning decimal points in output 1396
allClasses-frame.html generated by javadoc M_10
alpha software 30
alphabetizing 677
ALU (arithmetic and logic unit) 9
Amazon 3
Amazon S3 30
AMBER Alert 4
Analog Clock exercise 997
analysis stage of the software life cycle 475
Analytical Engine 16
anchor (a) element 959
anchor field of class GridBagConstraints 1031
AND (in SQL) 1185, 1186
and method of class BitSet N_11
Android 15, 16
Android Market 15, 16
app 27
Market 15
operating system 3, 14, 15, 15
smartphone 15
Android for Programmers: An App-Driven Approach 16
angle bracket (<>) for XML elements 958
angle brackets (< and >) 878
animated shape 947
animating a series of images 975
animation 952, 968, 980, 996
www.animationfactory.com 992
Animation exercise 996
Animator applet 943
annotation
@Override 368
Annotations
@GET 1317
@PathParam 1317
@Produces 1317
@WebMethod 1305
@WebParam 1306
@WebService 1305
Path 1316
annotations
@Resource 1285
anonymous inner class 565, 583, 599
anti-aliasing 946
Apache Derby xxv

Apache Software Foundation 14
Apache Tomcat 1305
API (application programming interface) **48**, **198**
API documentation (`download.oracle.com/javase/6/docs/api/`) 210
API links
  Deprecated **1371**
  Help **1371**
  Index **1371**
  Tree **1371**
APIs commonly used in mashups 1450
`append` method of class `StringBuilder` **691**
Apple 3
Apple Computer, Inc. 1452
Apple TV 5
applet **942**, 947, 954, 1120
  draggable **974**
applet `.class` file 950
`Applet` class
  `getAppletContext` method **1124**
  `getAudioClip` method **985**
  `getCodeBase` method **985**
  `getParameter` method **1121**
  `play` method **985**
  `showStatus` method **982**
applet container **942**, 951
Applet Package **209**
applet parameter **1120**
Applet that draws a string 948
`applet` XHTML element **950**
`AppletContext` interface **1120**
  `showDocument` method **1120**, 1124
`applet-desc` element of a JNLP document **959**
`applets` directory
  JDK sample applets 943
applets in the public domain 1120
`appletviewer` applet container **942**, 944
  Applet menu 945
  Quit menu item 945
  Reload menu item **945**
application **38**, 40, 73
  command-line arguments **202**
application mashup 1449
application programming interface (API) **18**, **198**
application server 1236
Application servers
  Apache Tomcat 1305
  GlassFish 1305
  JBoss Application Server 1305
`application-desc` element of a JNLP document 959
  `height` attribute 959
  `main-class` attribute 959
  `name` attribute 959
  `width` attribute 959
arc **651**, 943
arc angle **651**
arc width and arc height for rounded rectangles 650
`Arc2D` class 632
  `CHORD` constant **661**
  `OPEN` constant **661**
  `PIE` constant **661**
`Arc2D.Double` class **657**, 670
architectural patterns Q_3, Q_17, Q_18
archive files **344**
`ArcTest` applet 943
area of a circle 236
`args` parameter 279
argument index 1397, **1403**, **1411**
argument list 1398
argument promotion **207**
argument to a method **41**, 76
arithmetic and logic unit (ALU) **9**
arithmetic calculation 53
arithmetic compound assignment operators **130**
arithmetic mean 55
arithmetic operators 53
arithmetic overflow 447
`ArithmeticException` class **441**, 446
array **241**, 720, 1123, Q_20
  bounds checking **251**
  ignoring element zero 253
  `length` instance variable **242**
  pass an array element to a method 260
  pass an array to a method 260
array-access expression **242**
array-creation expression **243**
array initializer **245**
  for multidimensional array 269
  nested **269**
array of one-dimensional arrays 269
`ArrayBlockingQueue` class 1069, **1070**, 1080, 1094
  `size` method **1071**
`arraycopy` method of class `System` **281**, 283
`ArrayIndexOutOfBounds-Exception` class 251, **253**, 253, 657
`ArrayList<T>` generic class **284**, **833**, 849, 895, 1123, 1328
  `add` method **286**, 1123
  `clear` method 284
  `contains` method 284, **286**
  `get` method **286**
`ArrayList<t>` generic class (cont.)
  `indexOf` method 284
  `isEmpty` method 327
  `remove` method 284, **286**
  `size` method **286**
  `toString` method 897
  `trimToSize` method 284
`Arrays` class **281**
  `asList` method **839**, 840
  `binarySearch` method **281**
  `equals` method **281**
  `fill` method **281**, 1104
  `sort` method **281**, **805**
  `toString` method 706, 801
arrow 105
arrow key 601
arrowhead in a UML sequence diagram 504
artifact in the UML P_1
Artist exercise 997
ascending order 281
  `ASC` in SQL 1181, 1182
ascent **645**
ASCII (American Standard Code for Information Interchange) character set **7**, 171, 309, 1453
ASCII character set Appendix 1367
`asList` method of `Arrays` **839**, 840
assembler **10**
assembly language **10**
`assert` statement **461**, 1368
assertion **461**
`AssertionError` class **461**
assign a value to a variable 51
Assigning superclass and subclass references to superclass and subclass variables 399
assignment operator, `=` **51**, 59
assignment operators **130**
assignment statement 51
associate
  left to right 133
  right to left 125, 133
association (in the UML) **480**, 481, 482, 513, 514
  name 481
associativity of operators **54**, 60, 133
  left to right 60
  right to left 54
asterisk (*) SQL wildcard character **1178**
asynchronous call **503**
asynchronous event **447**
Asynchronous JavaScript and XML (Ajax) **1290**
asynchronous request **1290**
ATM (automated teller machine) case study 470, 475
`ATM` class (ATM case study) 480, 481, 485, 487, 489, 493, 500, 501, 502, 503, 504, 512
ATM system 475, 476, 478, 479, 489, 493, 511
`ATMCaseStudy` class (ATM case study) 546
atomic operation **1060**, **1227**
attribute 511, 513, 514
  compartment in a class diagram 487
  declaration in the UML 487, 489
  in the UML 13, 75, 480, 484, 485, **487**, 489, 492, 519, 520
  name in the UML 487
  of a class 11
  of an object 13
  of an XHTML element **950**
  type in the UML **487**
`.au` file extension **985**, 988
audio clip **985**, 987, 992
`AudioClip` interface **985**
  `loop` method **985**
  `play` method **985**
  `stop` method **985**
Austrailian Botanical Gardens (`www.anbg.gov.au/anbg/index.html`) 992
`@author` `javadoc` tag M_5
Author: note M_5
`-author` option M_9
`authorISBN` table of `books` database 1174, 1175
`authors` table of `books` database 1174
auto commit state 1227
auto-unboxing **832**
autobox an `int` 881
autoboxing 698, **832**, 881
`AutoCloseable` interface **463**, 1194
  `close` method 463
autoincremented **1174**, 1184
automated teller machine (ATM) 470, 475
  user interface 471
automatic driver discovery (JDBC 4) xxv, **1191**
automatic garbage collection 450
Automatic Jigsaw Puzzle Generator exercise 997
automatic scrolling 586
automatic updating 956
average 55, 113, 116
`.avi` file extension **988**
`await` method of interface `Condition` **1087**, 1091
`awaitTermination` method of interface `ExecutorService` **1058**
AWT (Abstract Window Toolkit) **555**
  components 556
`AWTEvent` class 567

B

B conversion character **1403**
b conversion character **1403**
B2B (business-to-business) transactions **1301**
Babbage, Charles 16
Background Audio exercise 996
background color 639, 641
backing array **839**
Backpack API 1451
backslash (\) **45**, 1411, 1412
backtracking **790**
BalanceInquiry class (ATM case study) 480, 482, 485, 486, 487, 490, 493, 501, 502, 503, 504, 512, 516, 517, 518
Balking design pattern Q_3, Q_14
BankDatabase class (ATM case study) 480, 483, 485, 493, 495, 500, 501, 502, 503, 504, 505, 512, 514
bar chart 193, 248, 249, 943
bar of asterisks 248, 249
BarChart applet 943
base N_1
base case **767**, 773, 778
base class **360**
base of a number 697
BASELINE alignment constant in GroupLayout 1432
baseline of the font 643
BASIC (Beginner's All-Purpose Symbolic Instruction Code) 16, 905
Basic Latin block 1457
BasicStroke class **632**, 660, 661
  CAP_ROUND constant **662**
  JOIN_ROUND constant **662**
batch file **731**
behavior 493
  of a class 11
  of a system 489, 490, 492, 502
behavioral design patterns Q_2, **Q_6**, Q_10, Q_19
beta software 31
bidirectional iterator **839**
bidirectional navigability in the UML **512**
Big O notation **802**, 809, 813, 817, 823
BigDecimal class 88, 162, **770**
  documentation (download.oracle.com/javase/6/docs/api/java/math/BigDecimal.html) 162
BigInteger class **770**, 1096
  add method **773**
  compareTo method **771**
  multiply method **771**
  ONE constant **771**, 773
BigInteger class (cont.)
  subtract method **771**, 773
  ZERO constant **773**
binary 238
  base 2 number system 1419
binary digit (bit) **7**
binary file **721**
binary integer 149
binary operator **51**, 53, 176
binary search algorithm **804**, 809, 849
binary search tree **924**, 929
binary tree **905**, 929
  delete 937
  search 938
  sort **929**
binarySearch method
  of Arrays **281**, 283
  of Collections 841, **849**, 851
BindException class **1135**
binding the server to the port **1128**, 1143
BindingProvider interface **1338**
  getRequestContext method **1338**
bit (binary digit) **7**
bit manipulation N_1
BitSet class **N_1**, N_11
  and method **N_11**
  clear method **N_11**
  equals method **N_12**
  get method **N_11**
  or method **N_11**
  set method **N_11**
  size method **N_12**
  toString method **N_12**
  xor method **N_11**
bitwise AND (&) **N_1**
Bitwise AND, bitwise inclusive OR, bitwise exclusive OR and bitwise complement operators N_4
bitwise assignment operators **N_11**
  ^= (bitwise exclusive OR) N_11
  &= (bitwise AND) N_11
  <<= (left shift) N_11
  >>= (signed right shift) N_11
  >>>= (unsigned right shift) N_11
  |= (bitwise inclusive OR) N_11
bitwise complement (~) operator **N_1**, N_1, N_8
bitwise exclusive OR (^) operator **N_1**, N_8
bitwise inclusive OR (|) operator **N_1**
bitwise operators 174, N_1, N_2
  ^ (bitwise exclusive OR) **N_1**
  & (bitwise AND) **N_1**
bitwise operators (cont.)
  << (left shift) **N_1**
  >> (signed right shift) **N_1**
  >>> (unsigned right shift) **N_1**
  | (bitwise inclusive OR) **N_1**
  ~ (complement) **N_1**
bitwise shift operations N_9
BlackBerry OS 14
Blackjack 1325
  game 1168
  Web Service Modification 1364
blank line 39, 121
_blank target frame **1124**
Blink applet 943
blink speed 995
block **111**, 123, **1128**, 1146
block increment of a JSlider **1002**
block until connection received 1135
blocked state **1049**, 1055
BlockingQueue interface **1070**
  put method **1070**, 1071
  take method **1070**, 1071
body
  of a class declaration **40**
  of a loop 112
  of a method **41**
  of an if statement 56
body mass index (BMI) 34
  calculator 34
Body Mass Index Calculator exercise 69
body XHTML element **950**
Bohm, C. 104
BOLD constant of class Font **643**
book-title capitalization **554**, 571
books database 1174
  table relationships 1177, 1234
Boolean
  attribute in the UML **485**
  class **831**
boolean
  expression **108**, 1384
  promotions 208
boolean logical AND, & 174, **175**
boolean logical exclusive OR, ^ 174, **176**
  truth table 176
boolean logical inclusive OR, | **175**
boolean primitive type **108**, 1368, 1369, 1384
border of a JFrame **1005**
BorderLayout class **390**, **592**, 603, 605, 608, 617
  CENTER constant 390, **592**, 608, 611
  EAST constant 390, **592**, 608
BorderLayout class (cont.)
  NORTH constant 390, **592**, 608
  SOUTH constant 390, **592**, 608
  WEST constant 390, **592**, 608
BOTH constant of class GridBagConstraints **1032**
bottom tier **1241**
bounded buffer **1080**
bounding box for an oval 966
bounding rectangle **185**, **649**, 651, 1002
bounds checking **251**
Box class **617**, 1028, 1029
  createGlue method **1031**
  createHorizontalBox method **617**, 1029
  createHorizontalGlue method **1031**
  createHorizontal-Strut method **1031**
  createRigidArea method **1031**
  createVerticalBox method **1030**
  createVerticalGlue method **1031**
  createVerticalStrut method **1030**
  X_AXIS constant **1031**
  Y_AXIS constant **1031**
boxing conversion **832**, 881
BoxLayout class **617**, 1028
BoxLayout layout manager 1028
braces ({ and }) 111, 123, 155, 164, 245
  not required 168
braille screen reader 556
branch Q_9
break 1368
break mode **1381**
break statement **168**, 172, 195
breakpoint **1379**
  inserting 1381, 1383
  listing 1392
  removing 1393
bricks-and-mortar store 1261
Bridge design pattern Q_3, Q_5, Q_9
brightness **641**
Brin, Sergey 26
brittle software **380**
browse method of class Desktop **1445**
browser 93
browsing 1120
brute force 300, 301
  Knight's Tour 301
bubble sort 826
  improving performance 826
bucket sort 826
buffer **752**, **1063**

buffered I/O **752**
BufferedImage class **661**
  createGraphics method **661**
  TYPE_INT_RGB constant **661**
BufferedInputStream class **752**
BufferedOutputStream class **752**, Q_16
  flush method **752**
BufferedReader class **753**
BufferedWriter class **753**
Builder design pattern Q_4
building block appearance 181
building blocks 103
Building Your Own Compiler 905
Building Your Own Computer 304
building-block approach to creating programs 12
bulk operation **832**
business functionality (mashups) 1450
business logic **1241**
business publications 31
business rule **1241**
business-to-business (B2B) transactions **1301**
button **551**, **571**
button label **571**
ButtonGroup class **577**, 1007, 1014
  add method **580**
byte **7**
byte-based stream **721**
Byte class **831**
byte keyword 1369
byte primitive type 164, 1368, N_1
  promotions 208
ByteArrayInputStream class 753
ByteArrayOutputStream class 753
bytecode **20**, 43, 905
bytecode verifier **21**

C

c option of the jar command **957**
C# programming language **17**
C++ programming language 17
cache **1240**
CachedRowSet interface **1207**
  close method 1209
calculations 9, 60, 105
Calendar class **1402**
  getInstance method **1403**
Calendar/Tickler File exercise 998
call-by-reference **262**
call-by-value **262**
call method of interface Callable **1109**
Callable interface **1109**
  call method **1109**
CallableStatement interface **1226**
callback function **1292**
Calling Attention to an Image exercise 996
calling method (caller) **73**, 81, 199
camera 15
Cancel button 94
cancel method of class SwingWorker **1109**
CANCEL_OPTION constant of JFileChooser **757**
CannotRealizePlayerException exception **990**
canRead method of File 723
canWrite method of File 723
CAP_ROUND constant of class BasicStroke **662**
capacity method
  of class StringBuilder **688**
capacity of a StringBuilder 687
carbon footprint calculator 34
card games 254
card shuffling
  Fisher-Yates 257
Card Shuffling and Dealing 303, 304
  with Collections method shuffle 845
CardTest applet 943
caretaker object Q_7
Car-Pool Savings Calculator exercise 70
carriage return 46
carry bit 1426
Cascading Style Sheets (CSS) 1236
case keyword **168**, 1368
case sensitive **40**
  Java commands 23
case studies xxiv
CashDispenser class (ATM case study) 480, 481, 482, 485, 486, 493, 505, 530
casino 210, 215
cast
  downcast **398**
  operator 68, **124**, 208
catch
  a superclass exception 449
  an exception 442
catch
  block **444**, 446, 447, 450, 454, 456
  clause **444**, 1368
  keyword **444**
Catch block **253**
catch handler
  multi-catch **462**
catch-or-declare requirement **448**
Catching Exceptions Using Class Exception exercise 468
Catching Exceptions Using Outer Scopes exercise 468
Catching Exceptions with Superclasses exercise 468
cd to change directories 42
ceil method of Math 201
cellpadding attribute of h:dataTable **1288**
cellspacing attribute of h:dataTable **1288**
Celsius 627, 1417
  equivalent of a Fahrenheit temperature 237
CENTER constant
  BorderLayout **592**, 608, 611
  FlowLayout **608**
  GridBagConstraints **1032**
  GroupLayout 1432
center mouse button click 596
centered 605
central processing unit (CPU) **9**
certificate authority 955
Chain-of-Responsibility design pattern Q_3, Q_6, Q_10
chained exception **457**
change directories 42, 943
ChangeEvent class **1005**
ChangeListener interface **1005**
  stateChanged method **1005**
changing look-and-feel of a Swing-based GUI 1021
chapter dependency chart xxvi
char
  array 675
  keyword 1368, 1369
  primitive type **49**, 164
  promotions 208
character **7**
  constant **171**
  literal **673**
  set 7, 68
character-based stream **721**
Character class 673, 695, **831**
  charValue method **698**
  digit method **696**
  forDigit method **696**
  isDefined method **695**
  isDigit method **695**
  isJavaIdentifierPart method **696**
  isJavaIdentifierStart method **696**
  isLetter method **696**
  isLetterOrDigit method **696**
  isLowerCase method **696**
  isUpperCase method **696**
  static conversion methods 697
Character class (cont.)
  toLowerCase method **696**
  toUpperCase method **696**
character encoding **1452**
character set 1453
character string **41**
CharArrayReader class **753**
CharArrayWriter class **753**
charAt method
  of class String 675
  of class StringBuilder **690**
CharSequence interface **707**
charValue method of class Character **698**
check protection exercise 715
checkbox **571**, 577
checkbox label **576**
checked exception **448**
checkerboard pattern 68
  exercise 965
Checkers game 1168
Checking with assert that a value is within range 461
Chess game 1168
chess-playing program 1168
child node **924**
child window 1001, **1022**, 1024, 1025
CHORD constant of class Arc2D **661**
Chrome 93
Circles Using Class Ellipse2D.Double exercise 668
circular buffer **1081**
circumference 68, 668, 965
Cisco 3
CJK Unified Ideographs block 1457
class **11**, 487, 493, 497, 511
  class keyword 72
  constructor **74**, 85, 513
  data hiding **81**
  declaration **40**, 948
  declare a method **72**
  default constructor **85**
  field **79**
  file **43**
  get method 320
  instance variable **12**, **79**, 201
  instantiating an object **72**
  name **40**, 342, 513
  set method 320
class 8
class-average problem 113, 114, 120, 121
class cannot extend a final class 419
Class class 388, **417**, 560, 1200
  getName method 388, **417**
  getResource method **560**

class diagram
for the ATM system model 483, 507
in the UML **477**, **480**, 482, 486, 493, 511, 514, 518, 519, 520
`.class` file **19**, **43**
separate one for every class 318
`.class` file extension 985
class hierarchy **360**, 402
class instance creation expression **74**, 86
`class` keyword **40**, 72, 1368
class library **361**, 386
class loader **20**, **344**, 560
class method **200**
class name
fully qualified 79
class variable **201**, **334**
classwide information **334**
`ClassCastException` class 883
Classes
`AbstractButton` **571**, 573, 1007, 1012
`AbstractCollection` **867**
`AbstractList` **867**
`AbstractMap` **867**
`AbstractQueue` **867**
`AbstractSequentialList` **867**
`AbstractSet` **867**
`AbstractTableModel` **1194**, 1200
`ActionEvent` **565**, 566, 570, 617, 979
`Arc2D` 632
`Arc2D.Double` **657**
`ArithmeticException` **441**
`ArrayBlockingQueue` 1069, **1070**, 1080, 1094
`ArrayIndexOutOfBoundsException` 251, 253
`ArrayList<T>` **284**, 284, **286**, **833**, 834, 849, 895, 1123
`Arrays` **281**
`AssertionError` **461**
`AWTEvent` 567
`BasicStroke` **632**, 660, 661
`BigDecimal` 88, 162, **770**
`BigInteger` **770**, 1096
`BindException` **1135**
`BitSet` N_1
`Boolean` **831**
`BorderLayout` **592**, 603, 605, 608, 617
`Box` **617**, 1028, 1029
`BoxLayout` **617**, 1028
`BufferedImage` **661**
`BufferedInputStream` **752**

Classes (cont.)
`BufferedOutputStream` **752**
`BufferedReader` **753**
`BufferedWriter` **753**
`ButtonGroup` **577**, 1007, 1014
`Byte` **831**
`ByteArrayInputStream` 753
`ByteArrayOutputStream` 753
`Calendar` **1402**
`ChangeEvent` **1005**
`Character` 673, 691, 695, **831**
`CharArrayReader` **753**
`CharArrayWriter` **753**
`Class` 388, **417**, 560, 1200
`ClassCastException` 883
`Collections` **833**, 880
`Color` **224**, **632**
`Component` **556**, 589, 634, 635, 973, 980, 1006, 1037
`ComponentAdapter` 594
`ComponentListener` 605
`ConcurrentHashMap` 1094
`ConcurrentLinkedDeque` 1094
`ConcurrentSkipListMap` 1094
`ConcurrentSkipListSet` 1094
`Container` **556**, 586, 605, 613
`ContainerAdapter` 594
`CopyOnWriteArrayList` 1094
`CopyOnWriteArraySet` 1094
`DatagramPacket` **1143**, 1165
`DatagramSocket` **1143**
`DataInputStream` **752**
`DataOutputStream` **752**
`Date` **1402**
`DelayQueue` 1094
`Desktop` **1445**
`Dimension` **980**
`Double` **831**, 896
`DriverManager` **1191**
`Ellipse2D` 632
`Ellipse2D.Double` **657**
`Ellipse2D.Float` **657**
`EmptyStackException` **855**
`EnumSet` **333**
`Error` **447**
`EventListenerList` **569**
`Exception` **447**
`ExecutionException` 1098
`Executors` **1051**

Classes (cont.)
`File` **722**
`FileInputStream` **721**
`FileOutputStream` **721**
`FileReader` **721**, 753
`FileWriter` **721**
`FilterInputStream` **751**
`FilterOutputStream` **751**
`Float` **831**
`FlowLayout` 559, 605
`FocusAdapter` 594
`Font` **576**, **632**, 643
`FontMetrics` **632**, **645**
`Formatter` **722**, **1396**
`Frame` 1005
`GeneralPath` 632, **662**
`GradientPaint` **632**, 660
`Graphics` **599**, **632**, 657, 973
`Graphics2D` **632**, **657**, 661
`GridBagConstraints` **1031**, 1037
`GridBagLayout` 1028, **1031**, 1033, 1037
`GridLayout` 605, **611**
`GroupLayout` 605, **1431**
`GroupLayout.Group` **1432**
`GroupLayout.ParallelGroup` **1432**
`GroupLayout.SequentialGroup` **1432**
`Gson` 1322
`HashMap` **859**, 1121
`HashSet` **856**
`Hashtable` **859**
`HyperlinkEvent` **1125**, 1127
`IllegalMonitorStateException` **1073**, 1088
`Image` **969**
`ImageIcon` **560**, 969, 978, 979
`IndexOutOfRangeException` 253
`InetAddress` **1136**, 1142, 1146, 1147
`InetAddress` Q_16
`InputEvent` **589**, 596, 601
`InputMismatchException` **441**
`InputStream` **751**, 1128, 1129, 1130
`InputStreamReader` **753**
`Integer` 554, **831**, 896
`InterruptedException` **1052**
`ItemEvent` **576**, 580
`JApplet` **948**, 1006
`JAXB` **1317**
`JButton` 555, **571**, 574, 611

Classes (cont.)
`JCheckBox` 555, **574**
`JCheckBoxMenuItem` 1006, **1007**, 1013
`JColorChooser` **639**
`JComboBox` 555, **580**, 1032
`JComponent` **556**, 557, 559, 569, 580, 584, 597, 613, 632, 634, 980
`JdbcRowSetImpl` **1209**
`JDesktopPane` **1022**, 1044
`JDialog` **1013**
`JEditorPane` **1125**
`JFileChooser` **754**
`JFrame` 1005
`JInternalFrame` **1022**, 1024
`JLabel` 555, **557**
`JList` 555, 584
`JMenu` **1006**, 1013, 1024
`JMenuBar` **1006**, 1013, 1024
`JMenuItem` **1007**, 1024
`JOptionPane` **93**, **552**, 1041
`JPanel` 555, 597, 598, 605, 613, 975, 1002
`JPasswordField` **561**, 566
`JPopupMenu` **1014**
`JProgressBar` 1105
`JRadioButton` **574**, 577, 580
`JRadioButtonMenuItem` 1006, **1007**, 1014
`JScrollPane` **586**, 588, 617, 618
`JSlider` **1001**, 1002, 1005, 1432
`JTabbedPane` **1026**, 1031
`JTable` **1194**
`JTextArea` 603, **615**, 617, 1033, 1036
`JTextComponent` **561**, 564, 615, 617
`JTextField` 555, **561**, 565, 569, 615
`JToggleButton` **574**
`KeyAdapter` 594
`KeyEvent` **570**, 601
`Line2D` 632, **661**
`Line2D.Double` **657**
`LinearGradientPaint` 660
`LineNumberReader` **753**
`LinkedBlockingDeque` 1094
`LinkedBlockingQueue` 1094
`LinkedList` **833**
`LinkedTransferQueue` 1094
`ListSelectionEvent` **584**

Classes (cont.)
ListSelectionModel **586**
Long **831**
MalformedURL-Exception **1124**
Manager **988**
Matcher 673, **707**
Math 200, 201
MemoryImageSource 996
MouseAdapter 594
MouseEvent **570**, **589**, 1017
MouseMotionAdapter 594, 598
MouseWheelEvent **590**
Number 896
Object 333
ObjectInputStream **722**, 1129, 1130, 1136
ObjectOutputStream **722**
OutputStream **751**, 1128, 1129, 1130
OutputStreamWriter **753**
Pattern 673, **707**
PipedInputStream 751
PipedOutputStream 751
PipedReader **753**
PipedWriter **753**
PixelGrabber 996
Point **599**
Polygon **632**, **654**
PrintStream **751**
PrintWriter 753
PriorityBlocking-Queue 1094
PriorityQueue **855**
Properties **863**
RadialGradientPaint 660
Random 209, **210**, 296
Reader **753**
Rectangle2D 632
Rectangle2D.Double **657**
ReentrantLock **1087**, 1089
RoundRectangle2D 632
RoundRectangle2D.Double **657**, 661
RowFilter **1206**
RuntimeException **448**
Scanner **49**, **77**
ServerSocket **1128**, 1135, 1157
ServiceManager **972**
Short **831**
Socket **1128**, 1142, 1157, 1158, Q_15
SocketException **1143**
SplashScreen **1444**
SQLException 1192
SQLFeatureNot-SupportedExceptio n 1199

Classes (cont.)
Stack **853**
StackTraceElement **457**
String **94**, 673
StringBuffer **688**
StringBuilder 673, **687**
StringIndexOutOf-BoundsException **684**, 690
StringReader **753**
StringWriter **753**, 1317
SwingUtilities **1022**, **1135**
SwingWorker **1095**
SynchronousQueue 1094
SystemColor **660**
SystemTray **1447**
TableModelEvent 1206
TableRowSorter **1206**
TexturePaint **632**, 660, 661
Throwable **447**, 456
Timer **979**, 980
TrayIcon **1448**
TreeMap **859**
TreeSet **856**
Types **1193**
UIManager **1021**
UnknownHostException **1130**
UnsupportedOperation Exception **839**
URL 985
Vector **833**
Window 1005
WindowAdapter 594, **1206**
Writer **753**
classified listings 26
ClassName.this 1012
CLASSPATH
environment variable 44, **345**
classpath **344**, 1191
-classpath command-line argument 728
to java **345**
to javac **345**
clear debugger command **1392**
clear method
of ArrayList<T> 284
of BitSet N_11
of List<T> **839**
of PriorityQueue **855**
clearRect method of class Graphics 648
click a button 561
click a tab 947
click count 594
click the mouse 574, 944
click the scroll arrows 583
clicking the close box 1044

client
object Q_16
of a class 493, 502
of an object **83**
client code 398
client connection 1128
client-server chat **1130**
client-server relationship **1119**
client-side artifacts **1310**
client tier **1241**, Q_18
clip art (www.clipart.com) 992
clock 944, 997
Clock applet 944
clone method of Object 387
clone object 742
Cloneable interface
documentation (java.sun.com/j2se/5.0/docs/api/java/lang/Cloneable.html) Q_19
cloning objects
deep copy **387**
shallow copy **387**
close a window 557, 561
close method
of CachedRowSet 1209
of Connection 1193
of Formatter **732**
of interface Connection 1193
of interface ResultSet 1193
of interface Statement 1193
of JdbcRowSet **1209**
of ObjectOutputStream **748**
of ResultSet 1193
of Socket **1129**
of Statement 1193
close method of interface AutoCloseable 463
closed polygons 654
closed tour 301, 669
closePath method of class GeneralPath **664**
cloud computing xxvi, **30**
COBOL (COmmon Business Oriented Language) 16
code **13**
code attribute of <applet> tag 950
code reuse 360
code value **1454**, 1457
codebase attribute of the jnlp element 958
code completion window (NetBeans) 1247
coin tossing 211, 238
collaboration diagram in the UML **477**
collaboration in the UML **499**, 500, 501, 503
collection **284**, **830**
collection hierarchy 832
collection implementation 866

Collection interface 831, **832**, 836, 841
contains method **836**
iterator method **836**
collections
synchronized collection 833
unmodifiable collection 833
Collections class **833**, 880
addAll method 841, **851**
binarySearch method 841, **849**, 851
copy method 841, **848**
disjoint method 841, **851**
fill method 841, **847**
frequency method 841, **851**
max method 841, **848**
min method 841, **848**
reverse method 841, **847**
reverseOrder method **843**
shuffle method 841, **845**, 847
sort method **842**
wrapper methods **833**
collections framework **830**
Collections methods reverse, fill, copy, max and min 848
collision in a hashtable **860**
color 632
color chooser dialog 641
Color class **224**, **632**
getBlue method **636**, 638
getColor method **636**
getGreen method **636**, 638
getRed method **636**, 638
setColor method **636**
Color constant 635, 638
color manipulation 634
color swatches **641**
Color.BLACK 224
Color.BLUE 224
Color.CYAN 224
Color.DARK_GRAY 224
Color.GRAY 224
Color.GREEN 224
Color.LIGHT_GRAY 224
Color.MAGENTA 224
Color.ORANGE 224
Color.PINK 224
Color.RED 224
Color.WHITE 224
Color.YELLOW 224
Coloring Black-and-White Photographs and Images exercise 998
colors 224
column 268, **1173**, 1174
column number in a result set 1179
columnClasses attribute of h:dataTable **1288**

columns attribute of h:panelGrid 1253
columns of a two-dimensional array 268
com.google.gson.Gson package 1322
com.sun.rowset package 1209
combo box 551, 580
comma (,) 159
comma (,) formatting flag 161
comma in an argument list 46
comma-separated list 159
  of arguments 46, 50
  of parameters 204
command-and-control software system 1046
command button 571
Command design pattern Q_3, Q_6, Q_11
command line 41
command-line argument 202, 279
Command Prompt 19, 41
command window 41, 943, 944, 949
comment
  end-of-line (single-line), // 39, 42
  Javadoc 39
  single line 42
commercial data processing 761
commission 146, 295
CommissionEmployee class derived from Employee 410
commit a transaction 1227
commit method of interface Connection 1227
Common Programming Errors overview xxviii
Commonly used JSF components 1250
communication diagram in the UML 477, 502, 503
Comparable interface 928
Comparable<T> interface 430, 679, 842, 880, 928
  compareTo method 842, 880
Comparator interface 842
  compare method 844
Comparator object 842, 848, 857, 859
  in sort 842
compare method of interface Comparator 844
compareTo method
  of class String 677, 679
  of Comparable 842
compareTo method of class BigInteger 771
compareTo method of Comparable<T> 880
comparing String objects 677
comparison operator 430
compartment in a UML class diagram 75
compilation error 39
compile 42, 43
compile a program 19
compile method of class Pattern 707
compile-time error 39
compile-time type safety 830
compiled applet class 950
compiler 10
compiler error 39
compiler options
  -d 342
compile-time type safety 874
compiling an application with multiple classes 75
Complex 356
complex curve 662
complex number 356
Complex Numbers (exercise) 356
complexity theory 774
component 11, 209, 588
Component class 556, 589, 634, 635, 641, 973, 980, 1006, 1037, Q_8, Q_10
  addKeyListener method 601
  addMouseListener method 593
  addMouseMotionListener method 593
  getHeight method 973
  getMaximumSize method 1432
  getMinimumSize method 980, 1004, 1432
  getPreferredSize method 980, 1004, 1432
  getWidth method 973
  repaint method 599
  setBackground method 641
  setBounds method 604
  setFont method 576
  setLocation method 604, 1006
  setSize method 604, 1006
  setVisible method 611, 1006
component diagram in the UML P_1
component in the UML P_1
component of an array 242
ComponentAdapter class 594
ComponentListener interface 594, 605
Composite design pattern Q_3, Q_5, Q_9, Q_10
composite structure diagram in the UML P_1
composition 328, 361, 363, 481, 482, 507
  in the UML 481
compound assignment operators 130, 132
compound interest 159, 193, 194
computation 5
computer-assisted instruction (CAI) 238, 239
computer-assisted instruction (CAI): Monitoring Student Performance 239
computer-assisted instruction (CAI): Reducing Student Fatigue 239
computer-assisted instruction (CAI): Varying the Types of Problems 239
computer-assisted instruction (CAI):Difficulty Levels 239
computer dump 307
computer program 6
computer simulator 306
Computerization of Health Records exercise 101
computerized scientific notation 1398
computers in education 238
concat method of class String 684
concatenate strings 337
concatenation 204
Concentric Circles Using Class Ellipse2D.Double exercise 668
Concentric Circles Using Method drawArc exercise 668
concrete class 401
concrete subclass 407
CONCUR_READ_ONLY constant 1199
CONCUR_UPDATABLE constant 1199
concurrency 1046
Concurrency API 1047
concurrency design patterns Q_3, Q_14
concurrency problem Q_14
concurrent access to a Collection by multiple threads 866
concurrent collections (Java SE 7) 1094
concurrent operations 1046
concurrent programming 1047
concurrent threads 1069
ConcurrentHashMap class 1094
ConcurrentLinkedDeque class 1094
ConcurrentSkipListMap class 1094
ConcurrentSkipListSet class 1094
condition 56, 163
Condition interface 1087, 1089
  await method 1087, 1091
  signal method 1087
  signalAll method 1087
condition object 1087
conditional AND, && 174, 175
  truth table 174
conditional expression 108
conditional operator, ?: 108, 133
conditional OR, || 174
  truth table 175
confusing the equality operator == with the assignment operator = 59
connect to a database 1189
connect to server 1128, 1130
connected lines 654
connected RowSet 1207
connection 1119, 1130, 1142, 1143, 1157, 1158
connection between client and server terminates 1130
connection between Java program and database 1191
Connection interface 1191, 1193, 1198, 1227
  close method 1193
  commit method 1227
  createStatement method 1192, 1198
  getAutoCommit method 1227
  prepareStatement method 1217
  rollBack method 1227
  setAutoCommit method 1227
connection-oriented service 1119
connection-oriented, streams-based transmission 1142
connection pool 1279
connection port 1128
connectionless service 1119, 1143
connectionless transmission 1142
consistent state 320
constant 339
  in an interface 431
  Math.PI 68
constant integral expression 164, 171
constant run time 803
constant variable 171, 247, 339
  must be initialized 247
constructor 74, 85, 513
  call another constructor of the same class using this 322
  multiple parameters 88
  no argument 322
  overloaded 320
  parameter list 86
Constructor Detail section in API 1376
Constructor failure exercise 468
Constructor Summary section in API 1374
constructors cannot specify a return type 86
consume an event 565

consume memory 777
consumer **1062**
consumer electronic device 18
consumer thread **1063**
consuming a web service **1302**, 1303, **1358**
`cont` debugger command **1382**
`Container` class **556**, 586, 605, 613, Q_10
  `setLayout` method **559**, 605, 611, 613, 1031
  `validate` method **613**
container for menus 1006
`ContainerAdapter` class 594
`ContainerListener` interface 594
`contains` method of `Collection` **836**
`contains` method of class `ArrayList<T>` 284, **286**
`containsKey` method of `Map` **862**
content pane 586, 1014
  `setBackground` method **586**
context-sensitive popup menu **1014**
`continue` statement **172**, 195, 1368, Q_2
continuous beta 31
control statement **104**, 105, 106, 107, 776
  nesting **106**, 182
  stacking **106**, 179
control variable **113**, **152**, 153, 154
controller (in MVC architecture) **1250**, 1250, Q_17
controller logic **1241**
controlling expression of a `switch` **168**
controls 550
converge on a base case 767
conversion characters **1397**
  % 1404
  A 1399
  a 1399
  B **1403**
  b **1403**, 1404
  C **1400**
  c **1400**
  d **1397**
  E **1398**, 1399
  e **1398**, 1399
  f **1398**, **1399**
  G **1399**
  g **1399**
  H **1403**
  h 1404
  n 1404
  o **1397**
  S **1400**
  s **1400**
  T **1401**
  t **1401**
  X **1397**
  x **1397**
conversion suffix characters **1401**
  A **1401**
  a **1401**
  B **1401**
  b **1401**
  c **1401**
  D **1401**
  d **1401**
  e **1402**
  F **1401**
  H **1402**
  I **1402**
  j **1402**
  k **1402**
  l **1402**
  M **1402**
  m **1401**
  P **1402**
  p **1402**
  R **1401**
  r **1401**
  S **1402**
  T **1401**
  Y **1402**
  y **1402**
  Z **1402**
convert
  a binary number to decimal 1424
  a hexadecimal number to decimal 1424
  an integral value to a floating-point value 208
  an octal number to decimal 1424
  between number systems 697
cookie **1262**, 1263
  deletion 1263
  expiration **1263**
  expiration date **1263**
  header 1263
Cooking with Healthier Ingredients 717
coordinate system **134**, **632**, 634
coordinates 949
coordinates (0, 0) 135, 632
`copy` method of `Collections` 841, **848**
copying objects
  deep copy **387**
  shallow copy **387**
`CopyOnWriteArrayList` class 1094
`CopyOnWriteArraySet` class 1094
core package 43
Core Tag Library (JSF) **1250**, 1254
correct in a mathematical sense 179
`cos` method of `Math` 201
cosine 201
counter **113**, 119, 126
counter-controlled repetition **113**, 114, 123, 126, 127, 152, 154, 306, 776
coupling Q_11, **11**
`-cp` command line argument to `java` **345**
CPU (central processing unit) **9**
CraigsList (`www.craigslist.org`) 26, 27, 1449
craps (casino game) 210, 215, 238, 297
create a desktop application in NetBeans 1310
create a package 340
create a reusable class 340
create a `Socket` 1130
create a web application in NetBeans 1304
create an object of a class 74
`createGlue` method of class `Box` **1031**
`createGraphics` method of class `BufferedImage` **661**
`createHorizontalBox` method of class `Box` **617**, 1029
`createHorizontalGlue` method of class `Box` **1031**
`createHorizontalStrut` method of class `Box` **1031**
`createRealizedPlayer` method of class `Manager` **988**
`createRigidArea` method of class `Box` **1031**
`createStatement` method of `Connection` **1192**, 1198
`createVerticalBox` method of class `Box` **1030**
`createVerticalGlue` method of class `Box` **1031**
`createVerticalStrut` method of class `Box` **1030**
creating a Java DB database in NetBeans 1286
creating and initializing an array 244
creational design patterns Q_2, Q_3, Q_7, Q_15, Q_19
credit limit on a charge account 146
cross-site scripting 1317
Crossword exercise 997
crossword puzzle generator 717
CSS
  `height` attribute **1253**
  `width` attribute **1253**
CSS (Cascading Style Sheets) 1236
CSS rule 1260
<Ctrl>-d 167
Ctrl key 586, 604
ctrl key 167
<Ctrl>-z 167
`currentThread` method of class `Thread` 1056
`currentTimeMillis` method of class `System` 797
cursor 41, 44
curve 662, 944
custom drawing area 598
customized subclass of class `JPanel` 598
cyclic gradient **660**

D

`-d` compiler option 342
`-d` option M_9
dangling-`else` problem **110**, 148
dashed lines 657
data 6
data entry 94
data hiding **81**
data hierarchy **6**, 7
data integrity 327
data source name **1281**
data structure **241**, Q_20
data tier **1241**
database **1172**, 1177
  table **1173**
database-driven multitier web address book xxv
database management system (DBMS) **1172**
datagram packet **1119**, 1142, 1143
datagram socket **1119**, 1143
`DatagramPacket` class **1143**, 1165
  `getAddress` method **1146**
  `getData` method **1146**
  `getLength` method **1146**
  `getPort` method **1146**
`DatagramSocket` class **1143**
  `receive` method **1146**
  `send` method **1146**
`DataInput` interface 752
`DataInputStream` class **752**
`DataOutput` interface 752
  `writeBoolean` method 752
  `writeByte` method 752
  `writeBytes` method 752
  `writeChar` method 752
  `writeChars` method 752
  `writeDouble` method 752
  `writeFloat` method 752
  `writeInt` method 752
  `writeLong` method 752
  `writeShort` method 752
  `writeUTF` method 752
`DataOutputStream` class **752**
`DataSource` interface **1285**
date 209
`Date` and `Time` Class (exercise) 356
date and time compositions **1401**
`Date` class **1402**
  exercise 356
date formatting 1397

DBCS (double byte character set) **1455**
DB2 1172
De Morgan's Laws 194
dead state 1049
deadlock **1088**, 1091, 1117, Q_14
dealing 254
debugger 1379
  break mode **1381**
  breakpoint **1379**
  `clear` command **1392**
  `cont` command **1382**
  defined **1379**
  `exit` command **1388**
  `-g` compiler option **1380**
  inserting breakpoints 1381
  `jdb` command **1381**
  logic error 1379
  `next` command **1387**
  `print` command **1383**, 1384
  `run` command **1381**, 1383
  `set` command **1383**, 1384
  `step` command **1385**
  `step up` command **1386**
  `stop` command **1381**, 1383
  suspending program execution 1383
  `unwatch` command 1388, **1390**
  `watch` command **1388**
decimal (base 10) number system 1419
decimal digit **7**
decimal integer 1397
decimal integer formatting 51
decision **56**, 107
  symbol in the UML **107**, 492
declaration
  class **40**
  `import` **48**, 50
  method **41**
declare a method of a class **72**
Decorator design pattern Q_3, Q_5, Q_15, Q_16
decrement of a control variable **152**
decrement operator, `--` **130**
decrypt 150
dedicated drawing area **597**
deep copy **387**
deeply nested statement 182
`default` case in a `switch` **168**, 171, 214
default constructor **85**, 326, 367
default exception handler 456
default initial value **82**
`default` keyword 1368
default layout of the content pane 617
default package **79**, 340
default upper bound (`Object`) of a type parameter 886
default value **82**, 134
define a custom drawing area 598
definite repetition **113**
degree 651
Deitel Resource Centers 31
`DelayQueue` class 1094
delegate a method call **920**
delegation event model **568**
`delete` method of class `StringBuilder` **693**
`DELETE` SQL statement 1178, **1186**
`deleteCharAt` method of class `StringBuilder` **693**
deleting an item from a binary tree 930
delimiter for tokens **699**
delimiter string 699
Dell 3
`demo` directory 946
Department of Defense (DOD) 16
dependency chart (chapters) xxvi
dependent condition 175
deploy a web app 1249
deploying a web service 1307
deployment diagram in the UML **P_1**
`Deposit` class (ATM case study) 480, 482, 485, 493, 501, 502, 509, 512, 516, 517
`DepositSlot` class (ATM case study) 480, 481, 482, 485, 493, 502, 513
`@deprecated` `javadoc` tag **M_8**
Deprecated link in API **1371**
`deprecated-list.html` generated by `javadoc` M_11
`Deprecated` note **M_8**
dequeue operation of queue **921**
derived class **360**
descending order 281
descending sort (`DESC`) 1181
descent **645**
descriptive words and phrases 485, 486
deserialized object **742**
design pattern **29**, Q_1, Q_2, Q_4, Q_5, Q_7, Q_8, Q_9, Q_10, Q_12, Q_14, Q_15, Q_19
design patterns xxvi
Design Patterns, Elements of Reusable Object-Oriented Software Q_2
design process **13**, 470, 476, 494, 499
design specification **476**
Design view in Netbeans 1433
`Desktop` class **1445**
  `browse` method **1445**
  `getDesktop` method **1445**
`Desktop` class (cont.)
  `isDesktopSupported` method **1445**
  `mail` method **1445**
  `open` method **1445**
`desktop` element of a JNLP document **959**
desktop integration 956
`destroy` method of `JApplet` **949**, 952
Determining points `C` and `D` for level 1 of "Lo fractal" 782
development tool 943
diacritic **1454**
dialog **93**, **552**
dialog box **93**, 552, 1012
`Dialog` font 643
`DialogInput` font 643
diameter 68, 668, 965
diamond in the UML **105**, 195
dice game 215
Dice Rolling 296
digit 49, 697, 700, 1419
`digit` method of class `Character` **696**
digital certificate 955
Digital Clock exercise 996
digital divide 5
digits reversed 237
`Dimension` class **980**
`dir` command on Windows 943
direct superclass 360, 362
`DIRECTORIES_ONLY` constant of `JFileChooser` **757**
directory 722, 723
  name 722
  separator **344**
  tree 945
disconnected `RowSet` **1207**, 1286
`disjoint` method of `Collections` 841, **851**
disk 6, 22, 720
disk drive 943
disk I/O completion 447
disk space 906
dismiss a dialog 553
dispatch
  a thread **1049**
  an event **570**
display a line of text 41
display area 950
display monitor 135, 632
display output 60
displaying text in a dialog box 93
`dispose` method of class `Window` **1005**
`DISPOSE_ON_CLOSE` constant of interface `WindowConstants` 1005
distance between values (random numbers) 214
dithering 944
`DitherTest` applet 944
divide-and-conquer approach **198**, 199, 767
divide by zero 22, 120, 441
division 9, 53, 54
division compound assignment operator, `/=` 131
DNS (domain name system) server **1237**
DNS lookup **1237**
`DO_NOTHING_ON_CLOSE` constant of interface `WindowConstants` 1005
`do...while` repetition statement 106, **162**, 163, 183, 1368
document 1001, 1022
document a program **39**
documentation comments **M_1**
dollar signs ($) 40
domain name system (DNS) server **1237**
Dorsey, Jack 28
dot (`.`) separator **75**, 94, 161, 200, 335, 657
dotted line in the UML **106**
double-byte character set (DBCS) **1455**
`(double)` cast **124**
`Double` class **831**, 896
  `parseDouble` method 953
double equals, `==` 59
double-precision floating-point number **88**
`double` primitive type **49**, **88**, 121, 1368, 1369
  promotions 208
double quotes, `"` 41, 45, 46
double selection 183
double-selection statement **106**, 127
`doubleValue` method of `Number` **897**
downcast 416
downcasting **398**
drag the scroll box 583
draggable applet 956, **974**
dragging the mouse to highlight 617
draw arc 943
draw complex curve 944
draw graphics 948
draw lines and points 944
`draw` method of class `Graphics2D` **660**
draw rectangle 954
draw shapes 632
`draw3DRect` method of class `Graphics` 648, **651**
`drawArc` method of class `Graphics` **288**, **651**, 668
`drawImage` method of class `Graphics` **973**
drawing color 636
drawing on the screen 634
`drawLine` method of class `Graphics` **137**, 648
`drawOval` method of class `Graphics` **184**, **185**, 648, **651**, 966

drawPolygon method of class Graphics 654, 656
drawPolyline method of class Graphics 654, 656
drawRect method of class Graphics 184, 648, 661, 668, 965, 966
drawRoundRect method of class Graphics 649
drawString method of class Graphics 638, 949, 954
DrawTest applet 944, 945
driver class 74
DriverManager class 1191
  getConnection method 1191
drop-down list 555, 580
dual-core processor 9
dummy value 118
Duplicate Elimination 296
duplicate elimination 930
duplicate of datagram 1143
duplicate values 930
Dynamic Audio and Graphical Kaleidoscope exercise 997
dynamic binding 416
dynamic content 18
dynamic data structure 905
dynamic memory allocation 906
dynamic resizing 241
dynamically resizable array 1123

E

EAST constant
  of class BorderLayout 592, 608
  of class GridBagConstraints 1032
eBay 3, 29
echo character of class JPasswordField 562
echoes a packet back to the client 1143
Eclipse
  demonstration video (www.deitel.com/books/jhtp9) 38
Eclipse (www.eclipse.org) 19
Eclipse Foundation 14
Ecofont 629
edit a program 19
editor 19
efficiency of
  binary search 809
  bubble sort 826
  insertion sort 817
  linear search 804
  merge sort 822
  selection sort 813
efficient (Unicode design principle) 1453
Eight Queens exercise 301, 795
  Brute Force Approaches 301
EL expression 1248
element (XML) 958
element of an array 242
element of chance 210
elided UML diagram 480
eligible for garbage collection 337
eliminate resource leaks 451
Ellipse2D class 632
Ellipse2D.Double class 657, 668
Ellipse2D.Float class 657
ellipsis (...) in a method parameter list 278
else keyword 108, 1368
emacs 19
email 1128
embedded system 6, 14
Employee abstract superclass 405
Employee class hierarchy test program 413
Employee class that implements Payable 425
employee identification number 8
empty set 356
empty statement (a semicolon, ;) 59, 111, 163
empty string 566, 674
empty XML element 959, 1253
EmptyStackException class 855
encapsulation 13
encrypt 150
end cap 660
End key 601
"end of data entry" 118
end-of-file (EOF) 1130
  indicator 167
  key combinations 731
  marker 720
end-of-line (single-line) comment, // 39, 42
end-of-stream 1130
end tag 958
endsWith method of class String 680
Enforcing Privacy with Cryptography exercise 150
English-like abbreviations 10
enhanced for statement 258
Enhancing Class Date (exercise) 355
Enhancing Class Time2 (exercise) 355
enqueue operation of queue 921, 922
ensureCapacity method of class StringBuilder 688
Enter (or *Return*) key 42, 569, 945, 946
ENTERED constant of nested class EventType 1127
entity-relationship diagram 1176
entry point 179
enum 218
  constant 331
enum (cont.)
  constructor 331
  declaration 331
  EnumSet class 333
  keyword 218, 1368
  values method 332
enumeration 218
enumeration constant 218
EnumSet class 333
  range method 333
environment variable
  CLASSPATH 44
  PATH 43
EOF (end-of-file) 1130
EOFException class 750
equal likelihood 212
equality operator == to compare String objects 677
equality operators 56
equals method
  of class Arrays 281
  of class BitSet N_12
  of class Object 387
  of class String 677, 679
equalsIgnoreCase method of class String 677, 679
erasure 879, 882
e-reader 3
e-reader device 15
Error class 447
Error-Prevention Tips
  overview xxviii
escape character 45, 1185
escape sequence 45, 49, 726, 1411, 1412, 1417
  \, backslash 46
  \", double-quote 46
  \t, horizontal tab 46
  newline, \n 45, 46, 49
Euclid's Algorithm 237
Euler 298
evaluating expressions 934
event 431, 489, 561, 635
event classes 567
event-dispatch thread (EDT) 634, 1095, 1135
event driven 561
event-driven process 634
event handler 431, 561
event handling 561, 564, 569
  event source 566
event ID 570
event listener 431, 567, 594
  adapter class 594
  interface 564, 565, 568, 570, 589, 594
event object 567
event registration 565
event source 566, 567
EventListenerList class 569
EventObject class
  getSource method 566
EventType nested class
  ACTIVATED constant 1127
  ENTERED constant 1127
  EXITED constant 1127
EventType nested class of HyperlinkEvent 1127
examination-results problem 128
exception 253, 439
  handler 253
  handling 251
  parameter 253
Exception class 447
exception handler 444
Exception Handling
  multi-catch 462
  try-with-resources statement 463
exception parameter 444
Exceptions 253
  IndexOutOfRange-Exception 253
execute 21, 42
execute an applet in a web browser 947, 951
execute attribute of f
  ajax 1295
execute method
  of JdbcRowSet 1209
execute method of the Executor interface 1051, 1054
executeQuery method
  of PreparedStatement 1218
  of Statement 1192
executeUpdate method of interface PreparedStatement 1218
executing an application 23
execution-time error 22
ExecutionException class 1098
Executor interface 1051
  execute method 1051, 1054
Executors class 1051
  newCachedThreadPool method 1052
ExecutorService interface 1051, 1109
  awaitTerminationmethod 1058
  shutdown method 1054
  submit method 1109
exists method of File 723
exit debugger command 1388
exit method of class System 450, 731
exit point 179
  of a control statement 106
EXIT_ON_CLOSE constant of class JFrame 137
EXITED constant of nested class EventType 1127
exiting a for statement 172
exp method of Math 201
expanded submenu 1012
expiration date of a cookie 1263
explicit conversion 124

exponential format 1397
exponential method 201
exponential notation **1398**
exponentiation 309
exponentiation operator 161
expression **51**
extend a class **360**
`extends` keyword **136**, **364**, 375, 1368
extensibility 398
eXtensible HyperText Markup Language (XHTML) 1236, 1237, **1243**
extensible language **74**
eXtensible Markup Language (XML) **958**, 1309
extension mechanism
  extending Java with additional class libraries **344**
external event **588**

F

`f` option of the `jar` command **957**
`f:ajax` element **1295**
`f:execute` element
  `execute` attribute **1295**
`f:facet` JSF element **1288**
`f:render` element
  `execute` attribute **1295**
`f:selectItem` element **1254**
`f:validateBean` element **1254**
`f:validateDoubleRange` element **1254**
`f:validateLength` element **1254**
`f:validateLongRange` element **1254**
`f:validateRegex` element **1254**
`f:validateRequired` element **1254**
Facade design pattern Q_3, Q_5, Q_16
facade object Q_16
FaceBook 26
Facebook 3, 14, 17, 28
Facelets (JSF) **1243**
Facelets Tag Library (JSF) **1269**
Faces servlet **1242**
factorial 149, 193, 768
Factorial calculations with a recursive method 769, 770
`factorial` method 768, 769
factory Q_15
factory method Q_8
Factory Method design pattern Q_3, Q_4, Q_8
Fahrenheit 627, 1417
  equivalent of a Celsius temperature 237
fairness policy of a lock **1087**
`false` keyword **56**, **109**, 1368
fatal error **111**, 309
fatal logic error **111**
fatal runtime error **22**
fault tolerant 50, 439
fault-tolerant program **253**
feature-complete 31
fetch 307
`fibonacci` method 773
Fibonacci series 303, 771, 773
  defined recursively 771
  generated with a recursive method 772
field **8**, **79**
  default initial value **82**
Field Detail section in API **1376**
field of a class 8, 220
Field Summary section in API **1374**
field width **160**, 1397, 1405
file **8**, **720**
`File` class **722**
  `canRead` method 723
  `canWrite` method 723
  `exists` method 723
  `File` methods 723
  `getAbsolutePath` method 723
  `getName` method 723
  `getParent` method 723
  `getPath` method 723
  `isAbsolute` method 723
  `isDirectory` method 723
  `lastModified` method 723
  `length` method 723
  `list` method 723
  `toURI` method **992**
  used to obtain file and directory information 724
file extensions
  `.aif` **985**, 988
  `.aiff` **985**, 988
  `.au` **985**, 988
  `.avi` **988**
  `.class` 985
  `.gif` **969**
  `.jpeg` **969**
  `.jpg` **969**
  `.mid` **985**, 988
  `.mov` **988**
  `.mp3` **988**
  `.mpeg` **988**
  `.mpg` **988**
  `.png` **969**
  `.rmi` **985**, 988
  `.spl` **988**
  `.swf` **988**
  `.wav` **985**
file folder 946
file matching
  exercise 761
  program 761
  with Multiple Transactions exercise 763
  with Object Serialization exercise 763
`File` methods 723
file processing 721
file-system APIs (Java SE 7) 720
`File.pathSeparator` 726
`FileContents` interface 978
  `getLength` method **973**
`FileInputStream` class **721**, 742, 745, 749, 751, 865
`FileNotFoundException` class **731**
`FileOpenService` interface **969**, 972
  `openFileDialog` method **972**
  `openMultiFileDialog` method **978**
`FileOutputStream` class **721**, 742, 745, 865, Q_15
`FileReader` class **721**, 753
`FILES_AND_DIRECTORIES` constant of `JFileChooser` **757**
`FILES_ONLY` constant of `JFileChooser` **757**
`FileWriter` class **721**, 753
filing cabinet 946
`fill` method
  of class `Arrays` **281**, 283
  of class `Collections` 841, **847**
  of class `Graphics2D` **660**, 661, 664, 670
`fill` method of class `Arrays` 1104
fill pattern 661
fill texture 661
fill with color 632
`fill3DRect` method of class `Graphics` 648, **651**
`fillArc` method of class `Graphics` 286, **288**, **651**
filled-in shape 224, 661
filled rectangle 636
filled three-dimensional rectangle 648
`fillOval` method of class `Graphics` **224**, **600**, 648, **651**
`fillPolygon` method of class `Graphics` 654, **657**
`fillRect` method of class `Graphics` **224**, **636**, 648, 661
`fillRoundRect` method of class `Graphics` **649**
filter a stream **751**
`FilterInputStream` class **751**
`FilterOutputStream` class **751**
`final`
  class **419**
  classes and methods 419
  keyword 171, **201**, 247, 339, 418, 1062, 1368
  local variable 583
  method **418**
  variable 247
final state in the UML **106**, 179, 490
final value 153
`finalize` method **334**, 387
`finally`
  block **444**, 450, 1091
  clause **450**, 1368
  keyword **444**
`find` method of class `Matcher` **707**
Find the Minimum Value in an Array exercise 796
Firefox web browser 93
`fireTableStructureChanged` method of `AbstractTableModel` **1200**
firewall **1302**
Fireworks Designer exercise 997
first-in, first-out (FIFO) data structure **921**
`first` method of `SortedSet` **859**
first refinement 126
first refinement in top-down, stepwise refinement 119
Fisher-Yates shuffling algorithm 257
five-pointed star 662
fixed text 51
  in a format string **47**, **1397**
flag value **118**
flags 1397, 1407
flash drive 720
Flickr 26, 1449
  APIs 1451
`float`
  literal suffix `F` **855**
  primitive type **49**, **88**, 1368, 1369
  primitive type promotions 208
`Float` class **831**
floating-point constant 159
floating-point conversion specifiers 1406
floating-point literal **88**
  `double` by default 88
floating-point number **88**, 118, 121, 123, 855, 953, 1399
  division 124
  double precision **88**
  `double` primitive type **88**
  `float` primitive type **88**
  single precision **88**
`floor` method of `Math` 201
Floor Planner exercise 997
flow of control 112, 123
flow of control in the `if...else` statement 108
`FlowLayout` class **559**, 605, 606
  `CENTER` constant **608**
  `LEFT` constant **608**
  `RIGHT` constant **608**
  `setAlignment` method **608**

flush method
of class BufferedOutputStream 752
of class Formatter 1157
of class ObjectOutputStream 1136
Flyweight design pattern Q_5
focus 562
focus for a GUI application 1002, 1018
FocusAdapter class 594
FocusListener interface 594
font
manipulation 634
name 643
size 643
style 643
Font class 576, 632, 643
BOLD constant 643
getFamily method 642, 645
getName method 642, 643
getSize method 642, 643
getStyle method 642, 645
isBold method 642, 645
isItalic method 642, 645
isPlain method 642, 645
ITALIC constant 643
PLAIN constant 643
font information 632
font manipulation 634
font metrics 645
ascent 647
descent 647
height 647
leading 647
font style 574
FontMetrics class 632, 645
getAscent method 646
getDescent method 646
getFontMetrics method 645
getHeight method 646
getLeading method 646
footerClass attribute of h:dataTable 1288
for attribute of h:message 1259
for repetition statement 106, 154, 155, 157, 159, 160, 183, 1368
activity diagram 157
enhanced 258
example 157
header 155
nested 249
forDigit method of class Character 696
foreign key 1175, 1177
fork/join framework 1109
form 1251
formal type parameter 878
format method
of class Formatter 731, 1413
of class String 94, 314, 1413
format specifiers 47, 1397
%.2f for floating-point numbers with precision 125
%% 1404
%B 1403
%b 1403
%b for boolean values 177
%c 68, 1400
%d 51, 1397, 1398
%E 1399
%e 1399
%f 68, 90, 1399
%G 1399
%g 1399
%H 1404
%h 1403
%n 1404
%n (line separator) 732
%o 1398
%S 1400
%s 47, 1397, 1400
%X 1398
%x 1398
format string 47, 1397, 1406
formatted output 1403
, (comma) formatting flag 161
%f format specifier 90
- (minus sign) formatting flag 160
0 flag 250, 314
aligning decimal points in output 1396
boolean values 177
comma (,) formatting flag 161
conversion character 1397
date and time compositions 1401
date and time conversion suffix characters 1401
dates 1397
exponential format 1397
field width 160, 1397
floating-point numbers 90
grouping separator 161
inserting literal characters 1397
integers in hexadecimal format 1397
integers in octal format 1397
left justification 1396
left justify 160
minus sign (-) formatting flag 160
precision 90, 1397
right justification 160, 1396
rounding 1396
times 1397
Formatter class 722, 728, 1396, 1412
close method 732
documentation (java.sun.com/javase/6/docs/api/java/util/Formatter.html) 1401, 1412
flush method 1157
format method 731, 1413
toString method 1413
FormatterClosedException class 732
formatting
display formatted data 46
Formatting date and time with conversion character t 1402
Formatting output with class Formatter 1412
formulating algorithms 113
Fortran (FORmula TRANslator) 16
forward slash character (/) in end tags 958
Foursquare 3, 17, 26, 29
fractal 779
"Lo fractal" at level 0 781
"Lo fractal" at level 2 783
"Lo fractal" at level 2, dashed lines from level 1 provided 783
depth 780
exercises 796
Koch Curve 780
Koch Snowflake 781
level 780
"Lo fractal" at level 1, with C and D points determined for level 2 782
order 780
self-similar property 780
strictly self-similar fractal 780
Fractal applet 944
Fractal user interface 783
fragile software 380
frame (in the UML) 505
Frame class 1005
free graphics programs (www.freebyte.com/graphicprograms) 992
FreeTTS (freetts.sourceforge.net/docs/index.php) 993
frequency method of Collections 841, 851
FROM SQL clause 1178
fromJson method of class Gson 1324
full tour 669
fully qualified class name 79, 342
function 199
function key 601
Future interface 1109
get method 1109
Future Splash (.spl) files 988

G

-g command line option to javac 1380
G.I.M.P. 969
Game of Craps 297
Game of Pool exercise 997
game playing 210
game programming 5
games
Call of Duty 2: games Modern Warfare 5
Farmville 5
Kinect for Xbox 360 5
Mafia Wars 5
social gaming 5
video game console 5
Xbox 360 5
gaming console 15
Gamma, Erich Q_2
"Gang of Four" Q_2, Q_5, Q_6, Q_14
garbage collection 1047
garbage collector 334, 447, 450, 985
GCD (greatest common divisor) 794
Gender Neutrality Web Service 1364
general class average problem 118
general path 662
generalities 397
generalization in the UML 516
GeneralPath class 632, 662, 668
closePath method 664
lineTo method 663
moveTo method 663
Generating Mazes Randomly exercise 797
generic class 284, 883
generic classes 874
generic collections xxv
generic interface 880
generic method 874, 877, 883
generics xxv, 831, 874
? (wildcard type argument) 897
actual type arguments 878
angle brackets (< and >) 878
default upper bound (Object) of a type parameter 886
erasure 879
formal type parameter 878
method 877
parameterized class 884
parameterized type 884
scope of a type parameter 886
type parameter 878
type parameter section 878
type variable 878
upper bound of a type parameter 881, 882
upper bound of a wildcard 897

generics (cont.)
wildcard type argument **897**
wildcard without an upper bound 899
wildcards **895**, 897
gesture 15
get a value **83**
@GET annotation **1317**
GET HTTP request **1238**
get method
of class ArrayList<T> **286**
of class BitSet N_11
of interface Future **1109**
of interface List<T> **836**
of interface Map **862**
get method 83, 320, 327
get request 1240
get started
java.sun.com/new2java/ 19
getAbsolutePath method of class File 723
getActionCommand method of class ActionEvent **566**, 574
getAddress method of class DatagramPacket **1146**
getAscent method of class FontMetrics 646
getAudioClip method of class Applet **985**
getAutoCommit method of interface Connection **1227**
getBlue method of class Color **636**, 638
getByName method of class InetAddress **1142**
getChars method
of class String **675**
of class StringBuilder **690**
getClass method of class Object **560**
getClass method of Object 388, **417**
getClassName method of class StackTraceElement **457**
getClassName method of class UIManager.LookAndFeelInfo **1021**
getClickCount method of class MouseEvent **597**
getCodeBase method of class Applet **985**
getColor method of class Color **636**
getColor method of class Graphics 636
getColumnClass method of TableModel **1194**, 1200
getColumnClassName method of ResultSetMetaData **1200**
getColumnCount method of ResultSetMetaData **1192**, **1200**
getColumnCount method of TableModel **1194**, 1200
getColumnName method of ResultSetMetaData **1200**
getColumnName method of TableModel **1194**, 1200
getColumnType method of ResultSetMetaData **1193**
getConnection method of DriverManager **1191**
getContentPane method of class JFrame **586**
getControlPanel-Component method of interface Player **990**
getData method of class DatagramPacket **1146**
getDefaultSystemTray method of class SystemTray **1447**
getDelay method of class Timer **995**
getDescent method of class FontMetrics 646
getDesktop method of class Desktop **1445**
getEventType method of class HyperlinkEvent **1127**
getFamily method of class Font 642, **645**
getFileName method of class StackTraceElement **457**
getFont method of class Graphics **643**, 643
getFontMetrics method of class FontMetrics **645**
getFontMetrics method of class Graphics 646
getGreen method of class Color **636**, 638
getHeight method of class Component **973**
getHeight method of class FontMetrics 646
getHeight method of class JPanel **137**
getHostName method of class InetAddress **1136**
getIcon method of class JLabel **560**
getIconHeight method of class ImageIcon **973**
getIconWidth method of class ImageIcon **973**
getImage method of class ImageIcon **973**
getInetAddress method of class Socket **1135**
getInputStream method of class Socket **1129**, 1130
getInstalledLookAndFeels method of class UIManager **1021**
getInstance method of Calendar **1403**
getInt method of ResultSet **1193**
getKeyChar method of class KeyEvent **604**
getKeyCode method of class KeyEvent **603**
getKeyModifiersText method of class KeyEvent **604**
getKeyText method of class KeyEvent **604**
getLeading method of class FontMetrics 646
getLength method of class DatagramPacket **1146**
getLength method of interface FileContents **973**
getLineNumber method of class StackTraceElement **457**
getLocalHost method of class InetAddress **1142**, 1147
getMaximumSize method of class Component 1432
getMessage method of class Throwable **456**
getMethodName method of class StackTraceElement **457**
getMinimumSize method of class Component **980**, 1004, 1432
getModifiers method of class InputEvent **604**
getName method of class Class 388, **417**
getName method of class File 723
getName method of class Font 642, **643**
getObject method of interface ResultSet **1193**, 1200
getOutputStream method of class Socket **1129**
getParameter method of class Applet **1121**
getParent method of class File 723
getPassword method of class JPasswordField **566**
getPath method of class File 723
getPoint method of class MouseEvent **599**
getPort method of class DatagramPacket **1146**
getPreferredSize method of class Component **980**, 1004, 1432
getProperty method of class Properties **863**
getRed method of class Color **636**, 638
getRequestContext method of interface Binding-Provider **1338**
getResource method of class Class **560**
getRow method of interface ResultSet **1200**
getRowCount method of interface TableModel **1194**, 1200
getSelectedFile method of class JFileChooser **757**
getSelectedIndex method of class JComboBox **583**
getSelectedIndex method of class JList **586**
getSelectedText method of class JTextComponent **617**
getSelectedValues method of class JList **589**
getSize method of class Font 642, **643**
getSource method of class EventObject 566
getStackTrace method of class Throwable **456**
getStateChange method of class ItemEvent **584**
getStyle method of class Font 642, **645**
getText method of class JLabel **560**
getText method of class JTextComponent 1014
getting started with Java 1193
getURL method of class HyperlinkEvent **1127**
getValue method of class JSlider **1005**
getValueAt method of interface TableModel **1194**, 1200
getVisualComponent method of interface Player **990**
getWidth method of class Component **973**
getWidth method of class JPanel **137**
getX method of class MouseEvent **593**
getY method of class MouseEvent **593**
GIF (Graphics Interchange Format) 560, **969**
.gif file extension **969**
gigabyte 9
glass pane 586
GlassFish application server **1236**, 1241, 1242, 1305
Tester web page 1308
Global Positioning System (GPS) 4
Global Warming Facts Quiz exercise 195
glyph **1454**
golden mean **771**
golden ratio **771**

Good Programming Practices
overview xxviii
Google 3, 26, 27
Goggles 27
Maps 27, 1449
Storage 30
TV 5
Gosling, James 18
goto elimination 104
goto statement **104**
GPS (Global Positioning System) 4
GPS device 8
gradient **660**
GradientPaint class **632**, 660, 669
Grand, Mark Q_14
graph 193
graph information 249
graphical user interface (GUI) **94**, 209, 431, **550**
component **94**
design tool **604**
graphics 597, 943, 944, 946, 968
Graphics class **135**, 224, 286, 431, 432, **599**, **632**, 634, 657, 949, 952, 953, 973
clearRect method 648
draw3DRect method 648, **651**
drawArc method **651**, 668
drawImage method **973**
drawLine method **137**, 648
drawOval method 648, **651**, 966
drawPolygon method 654, **656**
drawPolyline method 654, **656**
drawRect method 648, 661, 668, 965, 966
drawRoundRect method **649**
drawString method **638**, 949, 954
fill3DRect method 648, **651**
fillArc method **651**
fillOval method **224**, **600**, 648, **651**
fillPolygon method 654, **657**
fillRect method **224**, **636**, 648, 661
fillRoundRect method **649**
getColor method 636
getFont method **643**, 643
getFontMetrics method 646
setColor method **225**, 661
setFont method **643**
graphics context **634**
graphics demo 947
graphics in a platform-independent manner 634
Graphics Interchange Format (GIF) 560, **969**
Graphics2D class **632**, **657**, 661, 664, 668
draw method **660**
fill method **660**, 661, 664, 670
rotate method **664**
setPaint method **660**
setStroke method **660**
translate method **664**
graphicssoft.about.com/od/pixelbasedfreewin 992
GraphicsTest applet 944
GraphLayout applet 944
greatest common divisor (GCD) 237, 794
exercise 794
greedy quantifier **704**
grid 611
grid for GridBagLayout layout manager 1031
Grid Using Class Line2D.Double exercise 668
Grid Using Class Rectangle2D.Double exercise 668
Grid Using Method drawLine exercise 668
Grid Using Method drawRect exercise 668
GridBagConstraints class **1031**, 1037
anchor field **1031**
BOTH constant **1032**
CENTER constant **1032**
EAST constant **1032**
gridheight field **1033**
gridwidth field **1033**
gridx field **1032**
gridy field **1032**
HORIZONTAL constant **1032**
instance variables 1031
NONE constant **1032**
NORTH constant **1032**
NORTHEAST constant **1032**
NORTHWEST constant **1032**
RELATIVE constant **1037**
REMAINDER constant **1037**
SOUTH constant **1032**
SOUTHEAST constant **1032**
SOUTHWEST constant **1032**
VERTICAL constant **1032**
weightx field **1033**
weighty field **1033**
WEST constant **1032**
GridBagConstraints constants RELATIVE and REMAINDER 1037
GridBagLayout class 1028, **1031**, 1033, 1037
setConstraints method **1037**
GridBagLayout layout manager 1033
gridheight field of class GridBagConstraints **1033**
GridLayout class 605, **611**
GridLayout containing six buttons 612
gridwidth field of class GridBagConstraints **1033**
gridx field of class GridBagConstraints **1032**
gridy field of class GridBagConstraints **1032**
gross pay 146
GROUP BY 1178
group method of class Matcher 708
grouping separator (formatted output) **161**
GroupLayout class 605, 1028, **1431**
BASELINE alignment constant 1432
CENTER alignment constant 1432
default layout manager in Netbeans **1431**
groups **1432**
LEADING aligning components 1432
LEADING alignment constant 1432
parallel layout of GUI components 1431
recommended GUI design guidelines 1432
sequential horizontal orientation **1431**
sequential layout of GUI components 1431
spacing between components 1432
TRAILING alignment constant 1432
GroupLayout.Group class **1432**
addGap method **1432**
GroupLayout.ParallelGroup class **1432**
addGap method **1432**
GroupLayout.SequentialGroup class **1432**
addGap method **1432**
Groupon 3, 26, 28
groups in GroupLayout **1432**
Gson class 1322
code.google.com/p/google-gson/ 1320
fromJson method **1324**
toJson method 1322
guard condition in the UML 107, 492
Guarded Suspension design pattern Q_3, Q_14
guarding code with a lock **1055**
"guess the number" game 238, 627
Guestbook Application exercise 1275, 1298
GUI (Graphical User Interface) 431
component **550**, 943
design tool **604**
guide lines (Netbeans) **1434**, 1435
guillemets (« and ») in the UML **88**

H

H conversion character **1403**
h conversion character **1403**
h:body JSF element **1244**
h:column JSF element **1288**
h:commandButton element 1250, **1254**
h:dataTable element
cellpadding attribute **1288**
cellspacing attribute **1288**
columnClasses attribute **1288**
footerClass attribute **1288**
headerClass attribute **1288**
rowClasses attribute **1288**
styleClass attribute **1288**
value attribute **1287**
var attribute **1288**
h:dataTable JSF element 1278, **1286**, 1286
h:form element 1250, **1251**
h:graphicImage element 1250, **1253**
h:head JSF element **1244**
h:inputText element 1250, **1253**, 1259
h:message element **1259**
h:outputLink element 1250, **1254**, 1254, 1267, 1268
h:outputStyleSheet element **1259**
h:outputText element **1259**
h:panelGrid element 1250, **1252**
h:selectItem element 1250
h:selectOneMenu element 1250, **1254**
h:selectOneRadio element 1250, **1254**, 1267
half word 309
handle an exception 442
handshake point **1128**, 1142
hard drive 6, 9
hardcopy printer 22

hardware **6**, 10
has-a relationship **328**, **361**, 482
hash bucket 860
hash table 856, 860
`hashCode` method of `Object` 388
hashing **859**
`HashMap` class **859**, 1121
  `keySet` method **862**
`HashSet` class **856**
`Hashtable` class **859**, 860, Q_20
hash-table collisions 860
`hasNext` method
  of class `Scanner` **167**, 731
  of interface `Iterator` **836**, 839
`hasPrevious` method of `ListIterator` **839**
head of a queue **905**, 921
`headerClass` attribute of `h:dataTable` **1288**
`headSet` method of class `TreeSet` **858**
heavyweight components **556**
height **645**
`height` attribute (CSS) **1253**
`height` attribute of the `applet-desc` element **950**, 959
height of a rectangle in pixels 637
Helm, Richard Q_2
Help link in API **1371**
`helpdoc.html` generated by `javadoc` **M_11**
helper method **170**, 929
heuristic 300
Hewlett Packard 3
hexadecimal (base 16) number system 238, 309, 1419
hexadecimal integer 1397
"hidden" fields 220
hide a dialog 553
hide implementation details 199, 316
`HIDE_ON_CLOSE` constant of interface `WindowConstants` 1005
hierarchical boss-method/worker-method relationship 199
high-level language **10**
Hiragana block 1457
hollow diamonds (representing aggregation) in the UML **482**
Home key 601
Hopper, Grace 16
`HORIZONTAL` constant of class `GridBagConstraints` **1032**
horizontal coordinate **135**, **632**
horizontal gap space **611**
horizontal glue **1031**
horizontal `JSlider` component 1001
horizontal scrollbar policy 618
horizontal tab 46
`HORIZONTAL_SCROLLBAR_ALWAYS` constant of class `JScrollPane` **618**
`HORIZONTAL_SCROLLBAR_AS_NEEDED` constant of class `JScrollPane` **618**
`HORIZONTAL_SCROLLBAR_NEVER` constant of class `JScrollPane` **618**
Horse Race exercise 997
host **1237**
host name 1142
hostname **1237**
hot area **982**
hot spots in bytecode 21
`HourlyEmployee` class derived from `Employee` 409
HousingMaps.com (`www.housingmaps.com`) 27
`href` attribute of the `jnlp` element 958
`.htm` file name extension **949**
HTML (Hypertext Markup Language) 1236
HTML (HyperText Markup Language) document **942**, 949, 950
`html` element 1243
`.html` file name extension **949**
HTML Tag Library (JSF) **1243**, 1250
HTTP (HyperText Transfer Protocol) 1120, 1237, 1262
  being used with firewalls 1302
  header **1239**
  method **1238**
  request type **1239**
  transaction 1238
HTTP status codes (`www.w3.org/Protocols/rfc2616/rfc2616-sec10.html`) 1239
hue **641**
`HugeInteger` Class 357
  exercise 357
Hughes, Chris 28
Human Genome Project 3
hyperlink 1125, **1127**
`HyperlinkEvent` class **1125**, **1127**
  `EventType` nested class **1127**
  `getEventType` method **1127**
  `getURL` method **1127**
`HyperlinkListener` interface **1127**
  `hyperlinkUpdate` method **1127**
`hyperlinkUpdate` method of interface `HyperlinkListener` **1127**
HyperText Transfer Protocol (HTTP) **1120**, 1237, 1239, 1262
hypotenuse of a right triangle 235

I

I/O performance enhancement 752
IBM 3
IBM Corporation 16, 1452
IBM's Enterprise Mashup Tool 1451
icon 554
`Icon` interface **560**
`id` attribute of a JSF element **1253**
`id` attributes for elements in Ajax requests and responses 1295
IDE (integrated development environment) **19**
identifier **40**, 49
identity column 1212
`IDENTITY` keyword (SQL) **1212**
IDEs
  NetBeans 1301
IEEE 754 (`grouper.ieee.org/groups/754/`) 1369
IEEE 754 floating point 1369
`if` single-selection statement **56**, 106, 107, 164, 183, 184, 1368
  activity diagram 107
`if...else` double-selection statement 106, **107**, 108, 123, 164, 183
  activity diagram 108
ignoring array element zero 253
`IllegalArgumentException` class **314**
`IllegalMonitorStateException` class **1073**, 1088
`IllegalStateException` class **736**
image 951, 968, 992
`Image` class **969**
Image Flasher exercise 996
image map 968, **982**
Image Zooming exercise 996
`ImageIcon` class **388**, **560**, 969, 978, 979
  `getIconHeight` method **973**
  `getIconWidth` method **973**
  `getImage` method **973**
  `paintIcon` method **979**
`ImageObserver` interface **973**
imaginary part 356
immutable **675**
immutable object **337**
immutable `String` object **675**
implement an interface **396**, **420**, 427
implementation-dependent code 316
implementation of a function 406
implementation phase 521
implementation process 494, 511
`implements` 1368
`implements` keyword **420**, 424
implements multiple interfaces **590**
implicit conversion **124**
`import` declaration **48**, 50, 79, 341, 1368
improve performance of bubble sort 826
increment 159
  a control variable 153
  expression 172
  of a control variable **152**
  of a `for` statement 157
  operator, ++ 131
increment and decrement operators 131
indefinite postponement **1050**, 1091, 1117
indefinite repetition **118**
indentation 108, 110
independent software vendor (ISV) 386
index 251
index (subscript) **242**
Index link in API **1371**
index of a `JComboBox` **582**
index zero **242**
`Index_CD.html` generated by `javadoc` M_10
`Index_CD-all.html` generated by `javadoc` **M_11**
indexed lists 939
`indexOf` method of class `ArrayList<T>` 284
`indexOf` method of class `String` **681**
`IndexOutOfBoundsException` class 848
`IndexOutOfRangeException` class 253
indirect recursion **767**
indirect recursive call **767**
indirect superclass 360, 362
`InetAddress` class **1136**, 1142, 1146, 1147, Q_16
  `getByName` method **1142**
  `getHostName` method **1136**
  `getLocalHost` method **1142**, 1147
infinite loop **112**, 124, 156, 157, 1146, 1150
infinite recursion 385, **769**, 776, 777
infinite series 194
infinity symbol 1177
infix notation 934
infix-to-postfix conversion algorithm 935

information element of a JNLP document **959**
information hiding **13**, 81
information tier **1241**, Q_18
inherit **136**
inheritance **13**, 136, **360**, 516, 519, 520, 521
  examples 361
  extends keyword **364**, 375
  hierarchy **361**, 402
  hierarchy for university CommunityMembers 362
  multiple 360
  single **360**
init method
  of JApplet **949**, 951, 952, 954
initComponents
  autogenerated method in Netbeans 1436
initial state 179
initial state in the UML **105**, **489**, 490
initial value of an attribute 487
initial value of control variable **152**
initialization at the beginning of each repetition 128
initialization phase 119
initialize a variable in a declaration **49**
initialize an applet 951
initialize applet's instance variables 954
initializer block (static) 1265
initializer list **245**
initializing two-dimensional arrays in declarations 270
initiate an action 1007
inlining method calls **324**
inner class **564**, 576, 599, 1013
  anonymous **583**
  object of 577
  relationship between an inner class and its top-level class 577
INNER JOIN SQL clause 1178, **1183**
innermost set of brackets 252
inorder traversal **925**
input data from the keyboard 60
input device **8**
input dialog **94**, **552**
input/output 722
input/output operation 105, 304
input/output package 209
input unit **8**
InputEvent class **589**, 596, 601
  getModifiers method **604**
  isAltDown method 597, **604**
  isControlDown method **604**
InputEvent class (cont.)
  isMetaDown method 597, **604**
  isShiftDown method **604**
InputMismatchException class **441**, 444
InputStream class 743, **751**, 865, 1128, 1129, 1130
  read method **973**
InputStreamReader class **753**
insert method of class StringBuilder **693**
INSERT SQL statement 1178, **1184**
inserting literal characters in the output 1397
insertion point 283, 850, 907
insertion sort **814**
  algorithm **814**, 817
instance **12**
instance (non-static) method 335
instance of a class 80
instance variable **12**, **79**, 80, 89, 201
instanceof operator **416**, 1368
instantiating an object of a class **72**
instruction execution cycle 307
instructor resources for *Java How to Program, 9/e* xxx
int primitive type **49**, 121, 131, 164, 1368, 1369
  promotions 208
integer **47**
  array 245
  division **118**
  quotient 53
  value 49
Integer class 280, 554, **831**, 896
  parseInt method 280, 554
  toBinaryString method N_4
integer conversion characters **1397**
integer division **53**
integerPower method 235
integers
  suffix L 854
integral expression 171
integrated development environment (IDE) **19**
Intel 3
intelligent consumer electronic device 18
interaction between a web service client and a web service 1310
interaction diagram in the UML **502**
interaction overview diagram in the UML **P_2**
interactions among objects 499, 503
Interactive Drawing Application exercise 627
interest rate 159
interface 396, 421, 429, **1192**
  declaration **420**
  implementing more than one at a time **590**
  tagging interface **743**
interface keyword **420**, 1368
Interfaces **419**
  ActionListener **565**, 570
  AppletContext **1120**
  AudioClip **985**
  AutoCloseable **463**, 1194
  BlockingQueue **1070**
  CachedRowSet **1207**
  Callable **1109**
  CallableStatement **1226**
  ChangeListener **1005**
  CharSequence **707**
  Collection 831, **832**, 841
  Comparable 430, 679, **842**, **880**, 928
  Comparator **842**
  ComponentListener 594
  Condition **1087**, 1089
  Connection **1191**, 1193, 1198
  ContainerListener 594
  DataInput 752
  DataOutput 752
  DataSource **1285**
  Executor **1051**
  ExecutorService **1051**, 1109
  FileOpenService **969**, 972
  FocusListener 594
  Future **1109**
  HyperlinkListener **1127**
  Icon **560**
  ImageObserver **973**
  ItemListener **576**, 1014
  Iterator **833**
  JdbcRowSet **1207**
  KeyListener **570**, 594, 601
  LayoutManager **604**, 608
  LayoutManager2 **608**
  List 831, **839**
  ListIterator **833**
  ListSelectionListener **586**, 1124
  Lock **1086**
  Map 831, 859
  MouseInputListener **589**, 593
  MouseListener **570**, **589**, 594, 1017
Interfaces (cont.)
  MouseMotionListener **570**, **589**, 594
  MouseWheelListener **590**
  ObjectInput **742**
  ObjectOutput **742**
  Player **988**
  PreparedStatement 1226
  PropertyChangeListener **1108**
  Queue 831, **832**, **855**, 1070
  RequestContext 1338
  ResultSet **1192**
  ResultSetMetaData **1192**
  RowSet **1207**
  Runnable **1051**, 1158, 430
  Serializable 430, **743**
  Set 831, **832**, **856**
  SortedMap **859**
  SortedSet **857**
  Statement 1193
  SwingConstants **560**, 1005, 431
  TableModel **1194**
  WindowConstants **1005**
  WindowListener 594, **1006**, **1206**
internal frame
  closable 1024
  maximizable 1024
  minimizable 1024
  resizable 1024
Internet 3, 1120
Internet domain name in reverse order 342
Internet Explorer 93
Internet telephony 26
Internet TV 5
interpreter 11
Interpreter design pattern Q_6
interrupt method of class Thread **1052**
InterruptedException class **1052**
intersection of two sets 356
intrinsic lock **1055**
invoke a method **84**, 199
invokeLater method of class SwingUtilities **1135**
IOException class **748**
iOS 14
IP address 1146, **1237**
  of the server 1142
iPhone 27, 29
is-a relationship **361**, 397
isAbsolute method of File 723
isActionKey method of class KeyEvent **604**
isAltDown method of class InputEvent **597**, **604**
isBold method of class Font 642, **645**

isCancelled method of class SwingWorker 1104
isControlDown method of class InputEvent 604
isDefined method of class Character 695
isDesktopSupported method of class Desktop 1445
isDigit method of class Character 695
isDirectory method of File 723
isEmpty method
ArrayList 327
Map 863
Stack 855
isItalic method of class Font 642, 645
isJavaIdentifierPart method of class Character 696
isJavaIdentifierStart method of class Character 696
isLetter method of class Character 696
isLetterOrDigit method of class Character 696
isLowerCase method of class Character 696
isMetaDown method of class InputEvent 597, 604
isPlain method of class Font 642, 645
isPopupTrigger method of class MouseEvent 1017
isRunning method of class Timer 979
isSelected method
AbstractButton 1014
JCheckBox 577
isShiftDown method of class InputEvent 604
isUpperCase method of class Character 696
ITALIC constant of class Font 643
ItemEvent class 576, 580
getStateChange method 584
itemLabel attribute of f:selectItem 1254
ItemListener interface 576, 1014
itemStateChanged method 576, 577, 1014
itemStateChanged method of interface ItemListener 576, 577, 1014
itemValue attribute of f:selectItem 1254
iteration 116, 776
of a loop 152, 172
iteration (looping)
of a for loop 252
iterative (non-recursive) 768
iterative factorial solution 776
iterative model 475
iterator 830
Iterator design pattern Q_3, Q_6, Q_20
Iterator interface 833
hasNext method 836
next method 836
remove method 836
iterator method of Collection 836

J

Jacopini, G. 104
JApplet class 948, 949, 951, 1006
destroy method 949
init method 949, 954
paint method 949, 954
start method 949, 954
stop method 949
jar command 957
c option 957
f option 957
v option 957
jar element of a JNLP document 959
JAR file 974, 981
Java 2D API 632, 657, 946, 969
Java 2D shapes 657
Java 2D Shapes package 209
Java 3D 998
Java 3D API 968, 969, 993
Java Abstract Window Toolkit (AWT) package 209
Java Abstract Window Toolkit Event package 209
Java Advanced Imaging API 969
Java API 198, 430
overview 208
Java API documentation
download 52
Java API Interfaces 430
Java applet 948
Java Applet Package 209
Java Application Programming Interface (Java API) 18, 48, 198, 208
Java Architecture for XML Binding (JAXB) 1317
Java archive (JAR) file 956
Java class library 18, 48, 198
java command 20, 23, 38
-splash option 1443
Java compiler 19
Java Concurrency Package 209
Java Database Connectivity (JDBC) 1172
Java DB xxv, 1172, 1209, 1277
Java DB Developer's Guide 1212
Java debugger 1379
Java development environment 19, 20, 21, 943
Java Development Kit (JDK) 42
Java EE 6
www.oracle.com/technetwork/java/javaee/overview/index.html 1236
Java EE 6 tutorial 1236
java element of a JNLP document 959
Java-enabled web browser 942
Java Enterprise Edition (Java EE) 2, 1236
.java extension 19
.java file name extension 72
Java fonts
Dialog 643
DialogInput 643
Monospaced 643
SansSerif 643
Serif 643
Java HotSpot compiler 21
Java How to Program, 9/e
instructor resources xxx
Java IDE Resource Kit xxix, xxxiv
Java Image I/O API 969
Java Input/Output Package 209
java interpreter 43
Java Keywords 1368
Java Language Package 209
Java look and feel Graphics Repository 992
Java look-and-feel repository (java.sun.com/developer/techDocs/hi/repository) 992
Java Media Framework (JMF) API 969, 988
download 988
Java Media Framework package 209
Java Micro Edition (Java ME) 3
Java Naming and Directory Interface (JNDI) 1281
Java Network Launch Protocol (JNLP) 942, 955, 956
Java Networking Package 209
Java Plug-In 942
Java programming language 15
Java Resource Centers at www.deitel.com/ResourceCenters.html 42
Java SE 6
API documentation 210
package overview 208
Java SE 7 171
Automatically Closing Connections, Statements and ResultSets 1194
ConcurrentLinkedDeque 1094
file-system APIs 720
fork/join framework 1109
LinkedTransferQueue 1094
multi-catch 462
Java SE 7 (cont.)
new concurrent collections 1094
Strings in switch statements 171
try-with-resources statement 463
type inference with the <> notation 836
Java SE Development Kit (JDK) 18, 39
Java Sound API 969, 993
Java Speech API 969, 993
Java Speech API (java.sun.com/products/java-media/speech) 993
Java Standard Edition (Java SE) 2
6 2
7 2
Java Swing Event Package 210
Java Swing GUI Components Package 209
Java Utilities Package 209
Java Virtual Machine (JVM) 18, 20, 38, 41
Java Web Start 942, 955, 956
automatic updating 956
desktop integration 956
javaws command 959
overview (download.oracle.com/javase/6/docs/technotes/guides/javaws/) 960
Java website (java.sun.com) 208
JAVA_HOME environment variable 1210
java.applet package 209
java.awt class 1005
java.awt package 209, 555, 634, 635, 654, 657, 948, 969, 980, 1017
java.awt.color package 657
java.awt.event package 209, 210, 567, 569, 594, 604
java.awt.font package 657
java.awt.geom package 209, 657
java.awt.image package 657
java.awt.image.renderable package 657
java.awt.print package 657
java.beans package 1108
java.com 942
java.io package 209, 721
java.lang package 50, 200, 209, 364, 387, 673, 1051, Q_19
imported in every Java program 50
java.math package 88, 770

java.net package 209, 1119
java.sql package 209, 1191, 1192
java.util package 48, 209, 210, 284, 831, 853, 895, 907, 1402
  Calendar class 1402
  Date class 1402
java.util.concurrent package 209, 1051, 1070, 1093, 1109
java.util.concurrent.locks package 1086, 1087
java.util.prefs package 863
java.util.regex package 673
Java™ Language Specification (java.sun.com/docs/books/jls/) 54
Java2D API 657
Java2D applet 946
Java2D directory 946
JavaBean 1244
JavaBean property 1244
JavaBeans Specification 1244
javac compiler 19, 43
Javadoc comment 39
javadoc options
  -author M_9
  -d M_9
  -link M_9
javadoc tag M_1
javadoc tags
  {@link} M_8
  @author M_5
  @deprecated M_8
  @param M_6
  @return M_7
  @see M_5
  @since M_8
  @throws M_7
  @version M_8
javadoc utility program 39, M_1
JavaScript 1236
JavaScript Object Notation (JSON) 1303
JavaServer Faces (JSF) xxv
  XML declaration 1243
  xmlns attributes 1243
javax.faces.bean package (JSF) 1245
javax.jnlp package 956, 969, 972
javax.media package 209, 988
javax.sql package 1285
javax.sql.rowset package 1207
javax.swing package 94, 209, 210, 550, 552, 560, 569, 571, 617, 639, 948, 969, 1005, 1021, 1024
javax.swing.event package 210, 567, 586, 594, 1005
javax.swing.table package 1194, 1206
JAXB (Java Architecture for XML Binding) 1317
JAXB class 1317
  marshal method 1317
  unmarshal method 1320
JAX-RS 1300
JAX-WS 1300
JAX-WS package 210
JBoss Application Server (www.jboss.com/products/platforms/application) 1305
JButton class 555, 571, 574, 611
JCheckBox buttons and item events 575
JCheckBox class 555, 574
  isSelected method 577
JCheckBoxMenuItem class 1006, 1007, 1013
JColorChooser class 639, 641
  showDialog method 641
JColorChooser Dialog exercise 670
JComboBox class 555, 580, 1032
  getSelectedIndex method 583
  setMaximumRowCount method 583
JComboBox that displays a list of image names 581
JComponent class 556, 557, 559, 569, 580, 584, 597, 613, 632, 634, 980, Q_10
  paintComponent method 136, 597, 632, 979, 1002, 1004
  repaint method 635
  setForeground method 1014
  setOpaque method 597, 600
  setToolTipText method 559
jdb command 1381
JDBC
  API 1172, 1189, 1226
  driver 1172, 1173
JDBC 4 xxv
JDBC documentation 1173
JDBC information (www.oracle.com/technetwork/java/javase/tech/index-jsp-136101.html) 1173
JDBC Package 209
jdbc:mysql://localhost/books 1191
JdbcRowSet interface 1207
  close method 1209
  execute method 1209
  setCommand method 1209
  setPassword method 1209
  setUrl method 1209
JdbcRowSet interface (cont.)
  setUsername method 1209
JdbcRowSetImpl class 1209
JDesktopPane class 1022, 1044
JDesktopPane documentation (download.oracle.com/javase/6/docs/api/javax/swing/JDesktopPane.html) 1025
JDialog class 1013
JDIC (Java Desktop Integration Components)
  addTrayIcon method of class SystemTray 1447
  browse method of class Desktop 1445
  Desktop class 1445
  getDefaultSystemTray method of class SystemTray 1447
  getDesktop method of class Desktop 1445
  isDesktopSupported method of class Desktop 1445
  java command -splash option 1443
  mail method of class Desktop 1445
  open method of class Desktop 1445
  removeTrayIcon method of class SystemTray 1447
  -splash command-line option to the java command 1443
  splash screen 1443
  SplashScreen class 1444
  SystemTray class 1447
  Tray icons 1447
  TrayIcon class 1448
JDK 18, 42
  demo directory 943, 946
JEditorPane class 1125
  setPage method 1127
Jesse James Garrett 1290
JFileChooser class 754
  CANCEL_OPTION constant 757
  FILES_AND_DIRECTORIES constant 757
  FILES_ONLY constant 757
  getSelectedFile method 757
  setFileSelectionMode method 757
  showOpenDialog method 757
JFileChooser dialog 754
JFrame class 137, 225, 1005
  add method 137, 559
JFrame class (cont.)
  EXIT_ON_CLOSE 561
  getContentPane method 586
  setDefaultCloseOperation method 137, 561, 1005
  setJMenuBar method 1006, 1013
  setSize method 137, 561
  setVisible method 137, 561
JFrame class EXIT_ON_CLOSE constant 137
JFrame.EXIT_ON_CLOSE 561
JInternalFrame class 1022, 1024
  documentation 1025
JLabel class 388, 389, 555, 557
  documentation (download.oracle.com/javase/6/docs/api/javax/swing/JLabel.html) 557
  getIcon method 560
  getText method 560
  setHorizontalAlignment method 560
  setHorizontalTextPosition method 560
  setIcon method 560
  setText method 560
  setVerticalAlignment method 560
  setVerticalTextPosition method 560
JList class 555, 584
  addListSelectionListener method 586
  getSelectedIndex method 586
  getSelectedValues method 589
  setFixedCellHeight method 588
  setFixedCellWidth method 588
  setListData method 589
  setSelectionMode method 586
  setVisibleRowCount method 586
JMenu class 1006, 1013, 1024
  add method 1012
  addSeparator method 1013
JMenuBar class 1006, 1013, 1024
  add method 1013
JMenuItem class 1007, 1024
JMenus and mnemonics 1007
JMF (Java Media Framework) API 969, 985, 988
JNDI (Java Naming and Directory Interface) 1281

JNLP 972, 974, 981
  FileOpenService **969**, 972
  main-class **957**
  ServiceManager class **972**
JNLP (Java Network Launch Protocol) **956**
JNLP document 957
  applet-desc element **959**
  application-desc element 959
  desktop element **959**
  information element **959**
  jar element **959**
  java element **959**
  jnlp element **958**
  offline-allowed element **959**
  resources element **959**
  shortcut element **959**
  title element **959**
  vendor element **959**
JNLP documentation (download.oracle.com/javase/6/docs/jre/api/javaws/jnlp/) 973
jnlp element of a JNLP document **958**
  codebase attribute 958
  href attribute 958
jnlp.jar 973, 981
Johnson, Ralph Q_2
JOIN_ROUND constant of class BasicStroke **662**
joining database tables **1175**, 1183
Joint Photographic Experts Group (JPEG) 560, **969**
JOptionPane class **93**, 94, **552**, 553, 1041
  constants for message dialogs 555
  documentation 554
  PLAIN_MESSAGE constant **554**
  showInputDialog method **94**, **553**
  showMessageDialog method **94**, **554**
JOptionPane constants for message dialogs
  JOptionPane.ERROR_MESSAGE 555
  JOptionPane.INFORMATION_MESSAGE 555
  JOptionPane.PLAIN_MESSAGE 555
  JOptionPane.QUESTION_MESSAGE 555
  JOptionPane.WARNING_MESSAGE 555
JPanel class **135**, 136, 555, 597, 598, 605, 613, 975, 1002, Q_10
JPanel class (cont.)
  getHeight method **137**
  getWidth method **137**
JPasswordField class **561**, 566
  getPassword method **566**
JPEG (Joint Photographic Experts Group) 560, **969**
.jpeg file extension **969**
.jpg file extension **969**
JPopupMenu class **1014**
  show method **1017**
JProgressBar class 1105
JRadioButton class **574**, 577, 580
JRadioButtonMenuItem class 1006, **1007**, 1014
JScrollPane class **586**, 588, 617, 618
  HORIZONTAL_SCROLLBAR_ALWAYS constant **618**
  HORIZONTAL_SCROLLBAR_AS_NEEDED constant **618**
  HORIZONTAL_SCROLLBAR_NEVER constant **618**
  setHorizontalScrollBarPolicy method **618**
  setVerticalScrollBarPolicy method **618**
  VERTICAL_SCROLLBAR_ALWAYS constant **618**
  VERTICAL_SCROLLBAR_AS_NEEDED constant **618**
  VERTICAL_SCROLLBAR_NEVER constant **618**
JScrollPane scrollbar policies 617
JSF
  elements **1254**
  application lifecycle **1256**, 1261
  Core Tag Library **1250**, 1254
  deploy an app 1249
  f:selectItem element **1254**
  Facelets **1243**
  h:commandButton element 1250, **1254**
  h:form element 1250, **1251**
  h:graphicImage element 1250, **1253**
  h:inputText element 1250, **1253**, 1259
  h:message element **1259**
  h:outputLink element 1250, **1254**, 1254, 1267, 1268
  h:outputStyleSheet element **1259**
JSF (cont.)
  h:outputText element **1259**
  h:panelGrid element 1250, **1252**
  h:selectItem element 1250
  h:selectOneMenu element 1250, **1254**
  h:selectOneRadio element 1250, **1254**, 1267
  HTML Tag Library **1243**, 1250
  javax.faces.bean package **1245**
  @ManagedBean annotation **1245**, 1248
  @RequestScoped annotation **1248**
  resource library 1253
  resources folder **1253**
  resources library **1253**
  session expire 1263
  ui:repeat element **1269**
JSF Expression Language **1244**
JSF Facelets Tag Library **1269**
JSF web-application framework **1241**
JSlider class **1001**, 1002, 1005, 1432
  block increment **1002**
  documentation (download.oracle.com/javase/6/docs/api/javax/swing/JSlider.html) 1005
  getValue method **1005**
  major tick marks 1001
  minor tick marks 1001
  setInverted method **1002**
  setMajorTickSpacing method **1005**
  setPaintTicks method **1005**
  snap-to ticks **1001**
  thumb **1001**
  tick marks **1001**
JSON (JavaScript Object Notation) **1303**
JSON (www.json.org) 1303
JTabbedPane class **1026**, 1031
  addTab method **1027**
  SCROLL_TAB_LAYOUT constant **1031**
  TOP constant **1031**
JTable class **1194**
  RowFilter **1206**
  setRowFilter method **1206**
  setRowSorter method **1206**
  sorting and filtering xxvi
  TableRowSorter **1206**
JTextArea class 603, **615**, 617, 1033, 1036
JTextArea class (cont.)
  setLineWrap method **617**
JTextComponent class **561**, 564, 615, 617
  getSelectedText method **617**
  getText method 1014
  setDisabledTextColor method **603**
  setEditable method **564**
  setText method 617
JTextField class 555, **561**, 565, 569, 615
  addActionListener method **565**
JTextFields and JPasswordFields 562
JToggleButton class **574**
JumpingBox applet 944
just-in-time compilation **21**
just-in-time (JIT) compiler **21**

K

Kelvin temperature scale 627
kernel **14**
key constant 604, **604**
key event **570**, **601**
Key event handling 601
key value 930
key/value pair 860
KeyAdapter class 594
keyboard 6, 47, 550
KeyEvent class **570**, 601
  getKeyChar method **604**
  getKeyCode method **603**
  getKeyModifiersText method **604**
  getKeyText method **604**
  isActionKey method **604**
KeyListener interface **570**, 594, 601
  keyPressed method **601**, 603
  keyReleased method **601**
  keyTyped method **601**
Keypad class (ATM case study) 477, 480, 481, 482, 493, 500, 501, 502, 504, 513, 516, 547
keyPressed method of interface KeyListener **601**, 603
keyReleased method of interface KeyListener **601**
keySet method of class HashMap **862** of class Properties 866
keyTyped method of interface KeyListener **601**
keyword **40**, 106
Keywords
  abstract **401**
  boolean **108**, 1384

Keywords (cont.)
break 168
case 168
catch 444
char 49
class 40, 72
continue 172
default 168
do 106, 162
double 49, 88
else 106
enum 218
extends 136, 364, 375
false 109, 1368
final 171, 201, 247, 1062
finally 444
float 49, 88
for 106, 154
if 106
implements 420
import 48
instanceof 416
int 49
interface 420
new 49, 74, 243, 244
null 84, 243, 1368
private 80, 316, 327
public 40, 72, 73, 80, 203, 316
reserved but not used by Java 1368
return 80, 81, 199, 206
static 94, 161, 200
super 363, 385
switch 106
synchronized 1055
table of Keywords and reserved words 1368
this 317, 335
throw 453
true 109, 1368
try 444
void 41, 73
while 106, 162
Knight's Tour 298, 669
Brute Force Approach 300
Closed Tour Test 301
exercise 669
Koch Curve fractal 780
Koch Snowflake fractal 781
Koenig, Andrew 439

L

label 388, 388, 557, O_2
label in a switch 168
labeled block O_2
labeled break statement O_1
exiting a nested for statement O_1
labeled continue statement O_2
terminating a single iteration of a labeled-for statement O_3
labeled statement O_1, O_2
labels for tick marks 1001
Lady Ada Lovelace 16
LAMP 30, 30
language package 209
last-in, first-out (LIFO) 206
last-in, first-out (LIFO) order 888
last-in, first-out (LIFO) data structure 917
last method of ResultSet 1200
last method of SortedSet 859
lastIndexOf method of class String 681
lastModified method of class File 723
late binding 416
Layers architecture pattern Q_3, Q_18
layout 390
layout manager 559, 592, 604, 613, 1431
BorderLayout 592
FlowLayout 559
GridLayout 611
layoutContainer method of interface LayoutManager 608
LayoutManager interface 604, 608
layoutContainer method 608
LayoutManager2 interface 608
lazy quantifier 705
Lea, Doug Q_14
leading 645
LEADING alignment constant in GroupLayout 1432
leaf Q_9
leaf node 924
in a binary search tree 929
left brace, { 40, 41, 49
left child 924
LEFT constant of class FlowLayout 608
left justification 1396
left justified 108, 160, 560, 605
left-mouse-button click 596
left shift (<<) N_1, N_2, N_8
left subtree 924, 929, 937
left-to-right evaluation 55
Left, center and right mouse-button clicks 594
length field of an array 243
length instance variable of an array 242
length method of class String 675
length method of class StringBuilder 688
length method of File 723
letter 7
level of indentation 108
level-order binary tree traversal 930, 938
levels of nesting O_2
lexicographical comparison 679, 680
library attribute of h:graphicImage 1253
library of resources (JSF) 1253
life cycle of a thread 1048, 1050
lifecycle of a JSF application 1256, 1261
lifeline of an object in a UML sequence diagram 504
LIFO (last-in, first-out) 206, 888
lightweight GUI component 556, 1013
LIGHTWEIGHT_RENDERER constant of class Manager 989
LIKE operator (SQL) 1179
LIKE SQL clause 1180, 1182
Limericks 712
Limericks exercise 996
line 632, 647, 656
line join 660
line wrapping 617
Line2D class 632, 661
Line2D.Double class 657, 668
linear collection 907
linear data structure 924
linear run time 803
linear search algorithm 800, 801, 809
LinearGradientPaint class 660
LineNumberReader class 753
lineTo method of class GeneralPath 663
link 907, 924
{@link} javadoc tag M_8
-link option M_9
link to another node 906
linked list 905, 907, Q_20
LinkedBlockingDeque class 1094
LinkedBlockingQueue class 1094
LinkedList class 833, 849, 872
add method 841
addFirst method 841
addLast method 840
LinkedTransferQueue class 1094
Linux 14, 19, 41, 731, 943
Linux operating system 14, 14
list 582
List interface 831, 832, 839, 842, 847
add method 836, 838
addAll method 838
clear method 839
get method 836
listIterator method 839
size method 836, 839
subList method 839
toArray method 840
list method of File 723, 725
list method of Properties 865
listen for events 565
ListIterator interface 833
hasPrevious method 839
previous method 839
set method 839
listIterator method of interface List 839
ListSelectionEvent class 584
ListSelectionListener interface 586, 1124
valueChanged method 586
ListSelectionModel class 586
MULTIPLE_INTERVAL_SELECTION constant 586, 588
SINGLE_INTERVAL_SELECTION constant 586, 586, 588
SINGLE_SELECTION constant 586
literals
floating point 88
"Lo fractal" at level 0 781
"Lo fractal" at level 1, with C and D points determined for level 2 782
"Lo fractal" at level 2 783
"Lo fractal" at level 2, dashed lines from level 1 provided 783
load another web page into a browser 982, 984
load factor 860
load method of Properties 865
load/store operations 305
loading 20
Loading and displaying an image in an applet 969
Loading and playing an AudioClip 985
local variable 79, 116, 220, 319
localhost 1237
localhost (127.0.0.1) address 1136
localization 556, 1452
location *(0, 0)* 948
location of a variable in the computer's memory 52
lock an object 1076
Lock interface 1086
lock method 1086, 1091
newCondition method 1087, 1089
unlock method 1086, 1091
lock method of interface Lock 1086, 1091
log method of Math 201
logarithm 201
logarithmic run time 809
logic error 19, 50, 111, 154, 1379

logical complement operator, ! **176**
logical decision 5
logical input operations **752**
logical negation, ! **176**
logical negation, or logical NOT (!) operator truth table 177
logical operators **174**, 176
logical output operations **752**
logical unit **8**
Logo language 298
`long`
  literal suffix `L` **854**
`Long` class **831**
`long` keyword 1368, 1369
`long` promotions 208
look-and-feel **555**, 556, 604, 1017
  Nimbus **551**
Look-and-Feel Observations overview xxviii
Look-and-feel of a Swing-based GUI 1018
look-and-feel of an application **555**
`LookAndFeelInfo` nested class of class `UIManager` **1021**
`lookingAt` method of class `Matcher` **707**
`lookup` method of class `ServiceManager` **972**
loop 113, 116, 119, 996
  body 162
  continuation condition **106**
  counter 152
  infinite **112**, 124
  nested within a loop 127
  statement **106**, **112**
loop-continuation condition **152**, 153, 154, 155, 156, 157, 158, 162, 163, 172
`loop` method of interface `AudioClip` **985**
loopback address **1136**
looping **116**
Lord Byron 16
Lovelace, Ada 16
lowercase letter 8, 40
lowered rectangle **651**
`ls` command on UNIX 943

M

m-by-*n* array **268**
Mac OS X 14, 19, 41, 731
machine dependent 10
machine language **10**
machine language programming 304
Macintosh 634
Macintosh AIFF file format **985**, 988
Macintosh look-and-feel 1018
Macromedia Flash movies (`.swf`) **988**, 988
magnetic tape 720
`mail` method of class `Desktop` **1445**
`main` method **41**, 42, 49, 74
main thread **1054**
`main-class` attribute of the `applet-desc` element 959
`main-class` specified in an JNLP document **957**
maintainability 905
major tick marks of class `JSlider` 1001
make your point (game of craps) 215
making decisions 60
`MalformedURLException` class **1124**
`@ManagedBean` annoation (JSF) **1245**, 1248
`Manager` class **988**
  `createRealizedPlayer` method **988**
  `LIGHTWEIGHT_RENDERER` constant **989**
  `setHint` method **989**
Mandelbrot, Benoit 780
"manufacturing" section of the computer 9
many-to-many relationship 1177
many-to-one mapping **859**
many-to-one relationship in the UML **483**
`Map` interface 831, **859**
  `containsKey` method **862**
  `get` method **862**
  `isEmpty` method **863**
  `put` method **862**
  `size` method **863**
mapping APIs 1451
mappings of SQL types to Java types 1193
`marshal` method of class `JAXB` **1317**
Mashups
  Backpack API 1451
  business functionality 1450
  commonly used APIs 1450
  Craigslist 1449
  Flickr 1449
  Flickr APIs 1451
  Google Maps 1449
  IBM's Enterprise Mashup Tool 1451
  mapping APIs 1451
  Mashups Resource Center 1450
  Microsoft 1451
  of applications 1449
  PayPal 1450
  performance and reliability issues 1451
  popular mashups 1450
  ProgrammableWeb 1450
  RSS feed 1449
  RSS Resource Center 1451
  Salesforce.com 1451
  Smashforce 1451
Mashups (cont.)
  software reuse 1450
  Technorati APIs 1451
  Web 2.0 1449
  web services 1449
mashups 27
Mashups Resource Center 1450
mask N_2
master file 761
`Matcher` class 673, **707**
  `find` method **707**
  `group` method 708
  `lookingAt` method **707**
  `matches` method **707**
  `replaceAll` method **707**
  `replaceFirst` method **707**
`matcher` method of class `Pattern` **707**
`matches` method of class `Matcher` **707**
`matches` method of class `Pattern` **707**
`matches` method of class `String` **700**
matching `catch` block 444
`Math` class **161**, 200, 201
  `abs` method 201
  `ceil` method 201
  `cos` method 201
  `E` constant **201**
  `exp` method 201
  `floor` method 201
  `log` method 201
  `max` method 201
  `min` method 201
  `PI` constant **201**, 232
  `pow` method **161**, 161, 200, 201, 232
  `random` method 210
  `sqrt` method 200, 201, 207
  `tan` method 201
`Math.PI` constant 68, 668, 965
mathematical computations 16
Matisse GUI designer (Netbeans) **1431**
Matsumoto, Yukihiro "Matz" 17
`max` method of `Collections` 841, **848**
`max` method of `Math` 201
maximize a window 557, 1025
maximized internal frame 1025
`maximum` attribute of an `h` validateLength validator **1260**
`maxLength` attribute of an `h` nputText element **1260**
Maze Generator and Walker exercise 997
Maze Traversal Using Recursive Backtracking exercise 796
Mazes of Any Size exercise 797
MBCS (multi-byte character set) **1455**
MDI (Multiple Document Interface) 1001, **1022**
mean 55
Mediator design pattern Q_6
medical imaging 4
Memento design pattern Q_3, Q_6, Q_7
memento object Q_7
memory 6, **9**
memory buffer 752
memory leak 334, 450
memory location 52
memory-space/execution-time trade-off 860
memory unit **9**
memory utilization 860
`MemoryImageSource` class 996
menu **551**, 615, **1006**, 1007
menu bar **551**, **1006**, 1013
menu item **1007**, 1012
merge in the UML 492
merge records from tables 1182
merge sort **817**
merge sort algorithm **817**, 822
merge symbol in the UML **112**
merge two arrays 817
message 85, 949
message dialog **93**, **552**, 554
  types 554
message in the UML **500**, 502, 503, 504
message passing in the UML 504
Meta key 596, 597
`meta` XHTML element **1244**
metadata **1192**
metal look-and-feel 1001, **1018**
method **11**, **41**, 511
  declaration 41
  local variable **79**
  parameter **76**, 78
  parameter list **76**
  return type 81
  signature **223**
  `static` 161
method call **12**, 199, 204
method-call stack **207**
Method calls made within the call `fibonacci( 3 )` 775
Method calls on the program execution stack 775
method declaration 204
Method Detail section in API **1376**
method header **73**
method overloading **222**
method parameter list 278
Method Summary section in API **1375**
methods called automatically during applet's execution 951
methods implicitly `final` 419
Metric Conversion Program 716
microblogging 26, 28

Microsoft 3, 1451, 1452
Image Cup 35
SYNC 4
Microsoft Audio/Video Interleave (.avi) file **988**
Microsoft Audio/Video Interleave files 988
Microsoft SQL Server 1172
Microsoft Windows 167, 634, 1005, 1017
Microsoft Windows-style look-and-feel 1018
.mid file extension **985**, 988
middle mouse button 597
middle tier **1241**, Q_18
MIDI (Musical Instrument Digital Interface) file format (.mid or .rmi extensions) **985**, 988
mileage obtained by automobiles 146
MIME (Multipurpose Internet Mail Extensions) **1239**, 1263
min method of Collections 841, **848**
min method of Math 201
minimize a window 557, 1006, 1025
minimize internal frame 1025
minor tick marks of class JSlider 1001
minus sign (-) formatting flag **160**
minus sign (-) indicating private visibility in the UML 511
mnemonic **556**, **1007**, 1011, 1013
mobile application 2
mobile check-in 26
modal dialog **554**, 641
modal dialog box 1012
model (in MVC architecture) 1250, Q_17
model of a software system 480, 488, 518
Model-View-Controller (MVC) **1250**, 1250, Q_3, Q_11, Q_17
modifier key 604
Modifying the Internal Data Representation of a Class (exercise) 355
modularizing a program with methods 199
module **198**
modules in Java 198
MoleculeViewer applet 944
monetary calculations 162
monitor **1055**
monitor lock **1055**
Monospaced Java font 643
Moore's Law **6**
Morse Code 715, 1169
Web Service 1364
Moskovitz, Dustin 28
Motif-style (UNIX) look-and-feel 1001, 1018
motion information 8
Motorola 3
mouse 6, 550, 944
mouse button 944
mouse-button click 596
mouse click 594
mouse event **570**, **589**, 1017
handling 589
mouse wheel 590
MouseAdapter class 594
mousePressed method 1117
mouseClicked method of interface MouseListener 589, 594
mouseDragged method of interface MouseMotionListener 590, 598
mouseEntered method of interface MouseListener 590
MouseEvent class **570**, **589**, 1017
getClickCount method **597**
getPoint method **599**
getX method **593**
getY method **593**
isAltDown method **597**
isMetaDown method **597**
isPopupTrigger method **1017**
mouseExited method of interface MouseListener 590
MouseInputListener interface **589**, 593
MouseListener interface **570**, **589**, 594, 1017
mouseClicked method 589, 594
mouseEntered method 590
mouseExited method 590
mousePressed method 589, 1017
mouseReleased method 589, 1017
MouseMotionAdapter class 594, 598
MouseMotionListener interface **570**, **589**, 594
mouseDragged method 590, 598
mouseMoved method 590, 598
mouseMoved method of interface MouseMotionListener 590, 598
mousePressed method of class MouseAdapter 1117
mousePressed method of interface MouseListener 589, 1017
mouseReleased method of interface MouseListener 589, 1017
MouseWheelEvent class **590**
MouseWheelListener interface **590**
mouseWheelMoved method **590**
mouseWheelMoved method of interface MouseWheelListener **590**
.mov file extension **988**
moveTo method of class GeneralPath **663**
Mozilla Foundation 14
.mp3 file extension **988**
MP3 player 15
.mpeg file extension **988**
MPEG Layer 3 Audio (.mp3) files **988**
MPEG-1 videos 988
MPEG-1 videos (.mpeg, .mpg) **988**
.mpg file extension **988**
multi-button mouse 596
multi-core processor **9**
multibyte character set (MBCS) **1455**
multicast **1120**, 1165
multi-catch **462**
multidimensional array 268, 269
multimedia **968**
Multimedia-Based Simpletron Simulator exercise 998
multiple class declarations in one source-code file 318
multiple document interface (MDI) 1001, **1022**
multiple inheritance 360
multiple-selection list **584**, 586
multiple-selection statement **106**
MULTIPLE_INTERVAL_SELECTION constant of interface ListSelectionModel **586**, 588
multiplication compound assignment operator, *= 131
multiplication, * **53**, 54
multiplicative operators: *, / and % **125**
multiplicity **480**, 481
multiply method of class BigInteger **771**
Multipurpose Internet Mail Extensions (MIME) **1239**, 1263
multithreaded server 1168, 1170
multithreaded Tic-Tac-Toe Program 1168
multithreaded user interfaces xxv
multithreading 833, **1046**, Q_14
multitier application **1240**
multitouch screen **15**
Musical Instrument Digital Interface (MIDI) file format (.mid or .rmi extensions) **985**, 988
mutable data **1062**
mutator method **327**
mutual exclusion **1054**
mutually exclusive options **577**
MVC (Model-View-Controller) **1250**, Q_17
MyShape hierarchy 432
MyShape hierarchy with MyBoundedShape 433
MySQL 30, 1172, 1186, 1188
Community Edition **1186**
Connector/J **xxxiv**, **1187**
mysqld.exe 1188

N

n conversion character **1403**
%n format specifier (line separator) **732**
n-tier application **1240**
name attribute of @WebParam annotation **1306**
name attribute of @WebService annotation **1305**
name attribute of h:graphicImage **1253**
name attribute of the @ManagedBean annotation **1245**
name attribute of the appletdesc element 959
name collision **342**
name conflict **342**
name of a param 1121
name of a variable **52**
name of an array **242**
name of an attribute **1243**
named constant **247**
NASA multimedia (www.nasa.gov/multimedia/highlights/index.html) 992
NASA multimedia (www.nasa.gov/multimedia/index.html) 992
NASA Multimedia Gallery 992
native keyword 1368
natural comparison method **842**
natural logarithm 201
navigability arrow in the UML **511**
negative arc angles 652
negative degree **651**
NervousText applet 944
nested array initializers 269
nested building block 183
nested class **564**, 1021
relationship between an inner class and its top-level class 577

Nested Class Summary section in API **1374**
nested control statement Q_2
nested control statements **126**, 182, 184, 214
  Examination-results problem 128
nested `for` statement 249, 270, 271, 275, Q_1
nested `if...else` selection statement **109**, 110
nested message in the UML **504**
nested parentheses **54**
nesting rule **182**
NetBeans **1236**
  add a web service reference to an application 1311
  code-completion window 1247
  creating a Java DB database 1286
  Show Line Numbers 1247
Netbeans
  demonstration video (`www.deitel.com/books/jhtp9`) 38
NetBeans (`www.netbeans.org`) 19
NetBeans IDE **1236**, 1301, 1432
  add an event handler 1436
  create a desktop application 1310
  create a new project 1433
  create a web application 1304
  Design view 1433
  `GroupLayout` **1431**
  guide lines **1434**, 1435
  New JFrame Form dialog 1312
  New Web Service Client dialog 1311
  New Web Service dialog 1304
  Palette window 1433, 1434
  Properties window 1433
  snap-to alignment grid 1435
  Web Application project **1303**
Netbeans IDE 1431, 1432
  Design view 1433
  `initComponents` autogenerated method 1436
  Source view 1433
NetBeans Matisse GUI design tool xxvi
Netbeans Matisse GUI designer **1431**
network message arrival 447
Networked Morse Code 1169
networking package 209
New JFrame Form dialog 1312
`new` keyword **49**, **74**, 243, 244, 1368
`new Scanner(System.in)` expression 49
new state 1048
new to Java (`www.oracle.com/technetwork/topics/newtojava/overview/index.html`) 19
New Web Service Client dialog 1311
New Web Service dialog 1304
`newCachedThreadPool` method of class `Executors` **1052**
`newCondition` method of interface `Lock` **1087**, 1089
newline character **45**
newline escape sequence, `\n` 45, 46, 49, 309, 673
`next` method
  of `Iterator` **836**
  of `ResultSet` **1193**
  of `Scanner` **77**
`nextDouble` method of class `Scanner` **91**
`nextInt` method of class `Random` **210**, 214
`nextLine` method of class `Scanner` **76**
Nimbus look and feel **551**
  `swing.properties` xxxvi, 552
Nimbus look-and-feel 1018
Nirvanix 30
no-argument constructor **322**, 324
node Q_9
node in a list **907**
non-`static` class member 335
`NONE` constant of class `GridBagConstraints` **1032**
nonfatal logic error **111**
nonfatal runtime error **22**
nonlinear data structures 907
`NoPlayerException` exception **990**
`NORTH` constant of class `BorderLayout` **592**, 608
`NORTH` constant of class `GridBagConstraints` **1032**
`NORTHEAST` constant of class `GridBagConstraints` **1032**
`NORTHWEST` constant of class `GridBagConstraints` **1032**
`NoSuchElementException` class **731**, 736
note in the UML **106**
Notepad 19
`notify` method of class `Object` **1073**
`notify` method of `Object` 388
`notifyAll` method of class `Object` **1073**, 1076, 1077
`notifyAll` method of `Object` 388
noun phrase in requirements document 478
`null` 1368
`null` keyword 82, **84**, 94, 243, 554, 906
`null` reserved word 134
`Number` class 896
  `doubleValue` method **897**
number systems 697
numeric Classes 831

O

O(1) **803**
O(log *n*) **809**
O(*n* log *n*) time **823**
O(*n*) time **803**
O(n^2) time **803**
object 2, 11
object (or instance) 13, 502, 949
`Object` class 333, 360, **364**, 750
  `clone` method 387
  `equals` method 387
  `finalize` method 387
  `getClass` method 388, **417**, **560**
  `hashCode` method 388
  `notify` method 388, **1073**
  `notifyAll` method 388, **1073**, 1076, 1077
  `toString` method 367, 388
  `wait` method 388, **1073**
object diagram in the UML P_1
object of a derived class 398
object of a derived class is instantiated 385
object-oriented analysis and design (OOAD) **13**
object-oriented design (OOD) 470, 476, 478, 488, 511
object-oriented language **13**
object-oriented programming (OOP) **2**, 6, **13**, 17, 360
object serialization **742**, 1136
`ObjectInput` interface **742**
  `readObject` method **743**
`ObjectInputStream` class **722**, 742, 743, 749, 1129, 1130, 1136
`ObjectOutput` interface **742**
  `writeObject` method **743**
`ObjectOutputStream` class **722**, 742, 743, 865, Q_15
  `close` method **748**
  `flush` method **1136**
`Observable` class Q_12
Observer design pattern Q_3, Q_6, Q_12
`Observer` interface Q_12
observer object Q_12
octal integer 1397
octal number system (base 8) 238, 1419
Odersky, Martin 17
off-by-one error **154**
`offer` method of `PriorityQueue` **855**
`offline-allowed` element of a JNLP document **959**
OK button 94
`ON` clause **1183**
One-Armed Bandit exercise 997
`ONE` constant of class `BigInteger` **771**, 773
One Laptop Per Child (OLPC) 5
one statement per line 59
one-to-many relationship **1177**
one-to-one mapping **859**
one-, two- or three-button mouse 596
one's complement 1426, N_8
ones position 1419
one-to-many relationship in the UML **483**
one-to-one relationship in the UML **483**
OOAD (object-oriented analysis and design) 13
OOD (object-oriented design) 470, 476, 478, 488
OOP (object-oriented programming) **13**, 360
opaque Swing GUI components **597**
open a file **721**
`OPEN` constant of class `Arc2D` **661**
Open Handset Alliance 15
`open` method of class `Desktop` **1445**
open source **14**, 15
open source software xxvi
`openFileDialog` method of interface `FileOpenService` **972**
`openMultiFileDialog` method of interface `FileOpenService` **978**
`openStream` method of class `URL` **1324**
operand **51**, 124, 305
operating system **13**, 15
operation code 304
operation compartment in a class diagram 493
operation in the UML **76**, 480, 493, 497, 513, 514, 519, 520
operation parameter in the UML 78, 494, 497, 498, 499
`operationName` attribute of the `@WebMethod` annotation **1305**
operator **51**

operator precedence 54, 773
  operator precedence chart 125
  Operator Precedence Chart Appendix 1365
  rules **54**
Operators
  ^, boolean logical exclusive OR 174, **176**
  --, predecrement/postdecrement **130**
  --, prefix decrement/postfix decrement 131
  !, logical NOT 174, **176**
  ?:, ternary conditional operator 108, 133
  *=, multiplication assignment operator 131
  /=, division assignment operator 131
  &, boolean logical AND 174, **175**
  &&, conditional AND **174**, 175
  %=, remainder assignment operator 131
  ++, prefix increment/postfix increment 131
  ++, preincrement/postincrement 130
  +=, addition assignment operator **130**
  = **51**, 59
  -=, subtraction assignment operator 131
  |, boolean logical inclusive OR 174, **175**
  ||, conditional OR 174, **174**
  arithmetic **53**
  binary **51**, 53
  boolean logical AND, & 174, **175**
  boolean logical exclusive OR, ^ 174, **176**
  boolean logical inclusive OR, | **175**
  cast **124**
  compound assignment 130, 132
  conditional AND, && **174**, 175
  conditional operator, ?: **108**, 133
  conditional OR, || 174, **174**, 175
  decrement operator, -- 130, 131
  increment and decrement 131
  increment, ++ 131
  logical complement, ! **176**
  logical negation, ! **176**
  logical operators **174**, 176, 177
  multiplication, * **53**
  multiplicative: *, / and % **125**
Operators (cont.)
  postfix decrement **131**
  postfix increment **131**
  prefix decrement **131**
  prefix increment **131**
  remainder 149
  remainder, % **53**, 54
  subtraction, - 54
optical disk 720
optimizing compiler 161
optional package **344**
or method of class BitSet N_11
Oracle Corporation 1172, 1452
order 104
ORDER BY SQL clause 1178, **1181**, 1182
order in which actions should execute **103**
Order of catch Blocks exercise 468
order of exception handlers 468
ordering of records 1178
orientation information 8
origin component **1017**
originator object Q_7
out-of-bounds array index 447
outer set of brackets 252
OutOfMemoryError 906
output 41
output cursor 41, 44
output device **9**
output parameter for a CallableStatement **1226**
output unit **9**
OutputStream class 743, **751**, 1128, 1129, 1130
OutputStreamWriter class **753**
oval 647, 651, 948
oval bounded by a rectangle 651
oval filled with gradually changing colors 660
overflow 447
overflow error 309
overload a method **222**
overloaded constructors **320**
overloaded method 874
overloading generic methods 883
override a superclass method **363**, 368

P

PaaS (Platform as a Service) **30**
pack method of class Window **1025**
package **48**, 198, 208, 340, 1
package access **345**
package-access members of a class 346
package-access methods 346
package declaration **340**
package diagram in the UML P_1
package directory names 342
package directory structure 340
package keyword 1368
package-list generated by javadoc M_11
package name 79
package overview 208
Packages
  com.google.gson.Gson 1322
  default package **79**
  java.applet 209
  java.awt 209, **555**, 635, 657, 948, 969, 980, 1005, 1017
  java.awt.color 657
  java.awt.event 209, 210, **567**, 569, 594, 604
  java.awt.font 657
  java.awt.geom 209, 657
  java.awt.image 657
  java.awt.image.renderable 657
  java.awt.print 657
  java.beans 1108
  java.io 209, **721**
  java.lang **50**, 200, 209, 364, 387, 673, 1051
  java.math 88, 770
  java.net 209, **1119**
  java.sql 209, 1191, 1192
  java.util **48**, 209, 210, 284, 895
  java.util.concurrent 209, 1051, 1070, 1093, 1109
  java.util.concurrent.locks 1086, 1087
  java.util.prefs **863**
  java.util.regex 673
  javax.jnlp 956, 969, 972
  javax.media 209, **988**
  javax.sql 1285
  javax.sql.rowset **1207**
  javax.swing 209, 210, **550**, 552, 560, 571, 617, 639, 969, 1005, 1021, 1024
  javax.swing.event 210, **567**, 569, 586, 594, 1005
  javax.swing.table 1194, **1206**
packet **1119**, 1143
packet-based communications **1119**
Page Down key 601
page layout software 673
Page Up key 601
Page, Larry 26
paint method of JApplet **949**, 952, 954
Paint object **660**
paintComponent method of class JComponent **597**, 632, 979, 1002, 1004
paintComponent method of JComponent **136**
paintIcon method of class ImageIcon **979**
palindrome 149, 795
Palindromes exercise 795
panel 613
parallel layout of GUI components 1431
parallel operations 1046
param element **1121**
@param javadoc tag M_6
parameter **76**, 78
parameter in the UML 78, 494, 497, 498, 499
parameter list **76**, 86
parameterized class **884**
parameterized type **884**
Parameters: note M_6
parent directory 723
parent node 924, 937
parent window 94, 554, 1001, **1022**
parent window for a dialog box **1012**
parent window specified as null 1012
parentheses **41**, 54
  nested **54**
  redundant **56**
  unnecessary 56
parseDouble method of Double 953
parseInt method of class Integer 554
parseInt method of Integer 95, 186, 280
partial page update **1292**
partition step in quicksort 827, 828
Pascal programming language 16
pass an array element to a method 260
pass an array to a method 260
pass-by-reference **262**
pass-by-value 260, **262**
passing options to a program 279
password 562
@Path annotation **1316**
PATH environment variable xxxv, 43
path information **722**
path to a resource 1238
@PathParam annotation **1317**
pathSeparator static field of File 726
pattern 657
Pattern class 673, **707**
  compile method **707**
  matcher method **707**
  matches method **707**
pattern matching **1179**
pattern of 1s and 0s 7

Payable interface declaration 422
Payable interface hierarchy UML class diagram 422
Payable interface test program processing Invoices and Employees polymorphically 429
PayPal 1450
Payroll System Modification exercise 436
peek method of class PriorityQueue 855
peek method of class Stack 855
percent (%) SQL wildcard character 1179
perfect number (exercise) 237
perform a calculation 60
perform a task 73
perform an action 41
performance of binary tree sorting and searching 939
Performance Tips overview xxviii
performing operations concurrently 1046
persistent 9
persistent data 720
persistent Hashtable 863
personalization 1261
phase 119
Phishing Scanner 764
Phone-Book Web Service with JSON 1364
photo sharing 26
PHP 17, 30
physical input operation 752
physical output operation 752
PI 668, 965
"pick off" each digit 69
pie chart 670
Pie Chart exercise 670
PIE constant of class Arc2D 661
pie-shaped arc 661
Pig Latin 713
pipe 751
PipedInputStream class 751
PipedOutputStream class 751
PipedReader class 753
PipedWriter class 753
pixel ("picture element") 135, 632
pixel coordinates 948
PixelGrabber class 996
PLAF (pluggable look-and-feel) 1001
PLAIN constant of class Font 643, 643
PLAIN_MESSAGE 554
Platform as a Service (PaaS) 30, 30
platform dependency 1050
play method of class Applet 985
play method of interface AudioClip 985
Player interface 988
  getControlPanel-Component method 990
  getVisualComponent method 990
  start method 990
playing an AudioClip 985
playing audio 985
pluggable look-and-feel (PLAF) 1001
pluggable look-and-feel package 556
plus sign (+) indicating public visibility in the UML 511
PNG (Portable Network Graphics) 560, 969
.png file extension 969
point 643, 944
Point class 599
POJO (Plain Old Java Object) 1244, 1305
poker 303
Poker game 1168
poll method of PriorityQueue 855
polygon 654, 656
Polygon class 632, 654
  addPoint method 655, 657
polyline 654
polylines 654
polymorphic processing of collections 833
polymorphic processing of related exceptions 449
polymorphically process Invoices and Employees 428
polymorphism 171, 391, 395, 516, 517, 527
polynomial 55, 56
pool of threads 1129
pop method of Stack 855
pop off a stack 206
pop stack operation 917
popular mashups 1450
popup trigger event 1014, 1017
port 1128
port number 1128, 1130, 1142, 1143, 1147
portability 634, 1455
Portability Tips overview xxviii
portable 20
portable GUI 209
Portable Network Graphics (PNG) 560, 969
position number 242
positional notation 1419
positional value 1420
positional values in the decimal number system 1420
positive and negative arc angles 652
positive degrees 651
POST request 1239
postback 1261
postcondition 460
postdecrement 131
postfix decrement operator 131
postfix expression evaluation algorithm 935
postfix increment operator 131, 156
postfix notation 934
PostgreSQL 1172
postincrement 131, 133
postorder traversal 925, 928
pow method of class Math 161, 161, 200, 201, 232
power (exponent) 201, 237
power of 2 larger than 100 112
prebuilt data structures 830
precedence 54, 60, 133, 773
  arithmetic operators 54
  chart 54, 125
Precedence Chart Appendix 1365
precision 1397, 1398
  format of a floating-point number 125
precision of a floating-point value 88
precision of a formatted floating-point number 90
precondition 460
predecrement 131
predefined character class 700
predicate 1179
predicate method 327, 913
preemptive scheduling 1050
Preferences API 863
prefix decrement operator 131
prefix increment operator 131
preincrement 131, 133
Preincrementing and postincrementing 132
preorder traversal 925
PreparedStatement interface 1211, 1212, 1214, 1217, 1226
  executeQuery method 1218
  executeUpdate method 1218
  setString method 1211, 1218
prepareStatement method of interface Connection 1217
presentation logic 1241
presentation of a document 1243
previous method of ListIterator 839
primary key 1173, 1177
primary memory 9
prime 237, 872
prime number 302
primitive type 49, 84, 134, 207
  boolean 1384
  byte 164
  char 49, 164
  double 49, 88, 121
  float 49, 88
  int 49, 50, 121, 131, 164
  names are keywords 49
primitive type (cont.)
  passed by value 262
  promotions 208
  short 164
principal in an interest calculation 159
principle of least privilege 339
print a line of text 41
print an array 795
Print an Array Backward exercise 796
Print an Array exercise 795
print an array recursively 795
print debugger command 1383
print method of System.out 44
print on multiple lines 44
print spooling 921, 1063
printArray generic method 878
printf method of System.out 46, 1396
printing a binary tree in a two-dimensional tree format 930
printing trees 938
println method of System.out 41, 44
printStackTrace method of class Throwable 456
PrintStream class 751, 865
PrintWriter class 732, 753
priority of a thread 1050
PriorityBlockingQueue class 1094
PriorityQueue class 855
  clear method 855
  offer method 855
  peek method 855
  poll method 855
  size method 855
privacy protection 1262
private
  access modifier 80, 316, 317, 363
  data 327
  field 326
  keyword 327, 511, 1368
private static class member 335
probability 210
procedure 199
procedure for solving a problem 103
processing phase 119
processing unit 6
producer 1062
producer thread 1063
producer/consumer relationship 1062, 1081
@Produces annotation 1317
product of odd integer 193
program 6
program construction principles 187
program control 104
program development tool 107, 121

program execution stack **207**, 917
program in the general 395, 436
program in the specific 395
ProgrammableWeb 1450
programmer **6**
Projects window 1246
promotion **124**, 1261
  of arguments **207**
  rules 124, **207**
promotions for primitive types 208
prompt **50**
`Properties` class **863**
  `getProperty` method **863**
  `keySet` method 866
  `list` method **865**
  `load` method **865**
  `setProperty` method **863**
  `store` method **865**
property (JSF) 1244
property of a JavaBean **1244**
`propertyChange` method of interface `PropertyChangeListener` 1109
`PropertyChangeListener` interface **1108**
  `propertyChange` method 1109
proprietary class 386
`protected` access modifier 316, 363, 1368
protocol for communication (`jdbc`) 1191
Prototype design pattern Q_3, Q_4, Q_19
proxy class for a web service **1305**, 1310
Proxy design pattern Q_3, Q_5, **Q_6**
proxy object **Q_6**
pseudocode **104**, 108, 114, 125, 128
  algorithm 120
  first refinement **119**, 126
  second refinement **119**, 127
pseudorandom number **210**, 214
`public`
  `abstract` method 420
  access modifier 72, 73, 80, 203, 316, 363
  class 40
  `final static` data 420
  interface **312**
  keyword **40**, 80, 511, 513, 514, 1368
  member of a subclass 363
  method 137, 313, 316
  method encapsulated in an object 316
  service **312**
  `static` class members 335
  `static` method 335
publishing a web service **1302**, 1303, 1307, **1358**
`push` method of class `Stack` **854**
push onto a stack **206**
push stack operation **917**
`put` method
  of interface `BlockingQueue` **1070**, 1071
  of interface `Map` **862**
  of interface `RequestContext` **1338**
Pythagorean Triples 194
Python 17

Q

quad-core processor 9
quadratic run time 803
qualified name **1183**
quantifiers used in regular expressions **704**
quantum **1049**
query **1172**, 1174
query a database 1189
query application for the `books` database 1233
query method **327**
query string **1240**
`QUESTION_MESSAGE` 554
queue 832, 855, **905**, 921
`Queue` interface 831, **832**, **855**, 1070
queue length **1128**
queue to the server 1135
quicksort algorithm 827
QuickTime (`.mov`) files **988**

R

`RadialGradientPaint` class 660
radians 201
radio button **571**, **577**
radio button group **577**
radius 668, 965
radius of a circle 236
radix (base) of a number **697**
raised rectangle **651**
Random Characters exercise 668
`Random` class 209, **210**, 296
  `download.oracle.com/javase/6/docs/api/java/util/Random.html` 210
  `nextInt` method **210**, 214
  `setSeed` method 215
Random Colors exercise 670
Random Interimage Transition exercise 996
random limericks 712
Random Lines Using Class `Line2D.Double` exercise 668
`random` method of class `Math` 210
random numbers 214
  difference between values 214
  element of chance **210**
  generation 254
  generation to create sentences 712
  processing 209
  pseudorandom number **210**
  scaling **211**
  scaling factor **211**, 214
  seed **211**
  seed value **214**
  shift a range **211**
  shifting value **211**, 214
Random Triangles exercise 668
Randomly Erasing an Image exercise 995
randomly generated triangles 668
randomly sized shapes 670
`range` method of class `EnumSet` **333**
range-view methods **839**, 857
ratio of successive Fibonacci numbers 771
`Rational` class 357
Rational Numbers (exercise) 357
Rational Software Corporation 476
Rational Unified Process™ 476
raw type **891**
RDBMS (relational database management system) 1241
Reaction Time/Reaction Precision Tester exercise 998
`read` method of class `InputStream` **973**
read-only file 748
read-only text 557
Read/Write Lock design pattern Q_3, Q_14
readability 39, 127
`Reader` class **753**
reading a file on a web server 1125
`readObject` method of `ObjectInput` **743**
`readObject` method of `ObjectInputStream` 750
ready state 1049
real number 49, 121
real part 356
realization in the UML **421**
receive a connection 1135
receive data from a server 1142
`receive` method of class `DatagramSocket` **1146**
"receiving" section of the computer 8
reclaim memory 338
recognizing clients 1262
recommended GUI design guidelines used by `GroupLayout` 1432
record **8**, 726
rectangle 355, 632, 636, 648, 944, 948
`Rectangle` Class (exercise) 355
`Rectangle2D` class 632
`Rectangle2D.Double` class **657**
recursion
  overhead 777
  quicksort 827
  recursion step **767**, 773
  recursive backtracking **790**
  recursive binary search algorithm 827
  recursive call **767**, 772, 773
  recursive evaluation 768
  Recursive evaluation of 5! 768
  recursive `factorial` method 769
  recursive linear search algorithm 827
  recursive method **766**
  Recursive `power` Method exercise 794
  recursive step 827
  recursively generating Fibonacci numbers 773
  Recursively Print a List Backwards 937
  Recursively Search a List 937
Recursion Exercises
  binary search 827
  Eight Queens 795
  Find the Minimum Value in an Array 796
  Fractals 796
  Generating Mazes Randomly 797
  Greatest Common Divisor 794
  linear search 827
  Maze Traversal Using Recursive Backtracking 796
  Mazes of Any Size 797
  Palindromes 795
  Print an Array 795
  Print an Array Backward 796
  quicksort 827
  Recursive `power` Method 794
  Time to Calculate Fibonacci Numbers 797
  Visualizing Recursion 794
recursive backtracking 797
redirect a standard stream **721**
redirect a stream 721
redundant parentheses **56**
`ReentrantLock` class **1087**, 1089
refactoring xxvi, **29**
  tool 29
refer to an object **84**
reference **84**

reference type **84**, 346
refinement process 119
reflection 417
`regexFilter` method of class `RowFilter` **1206**
`regionMatches` method of class `String` **677**
register a port **1128**
register an `ActionListener` 1013
registered listener 569
registering the event handler **564**
Registration Form Modification exercise 1275
regular expression **700**
  `^` 703
  `?` 704
  `.` 707
  `{n,}` 704
  `{n,m}` 704
  `{n}` 704
  `*` 703
  `\D` 700
  `\d` 700
  `\S` 700
  `\s` 700
  `\W` 700
  `\w` 700
  `+` 703
  `|` 704
Regular Expressions Resource Center (`www.deitel.com/regularexpressions/`) 708
Reigstration Form Modification 1275
reinventing the wheel 12, 48, 281
relational database 1172, **1173**
relational database management system (RDBMS) **1172**, 1241
relational database table 1173
relational operators **56**
relationship between an inner class and its top-level class 577
`RELATIVE` constant of class `GridBagConstraints` **1037**
relative path **722**
release a lock 1076
release a resource 450, 451
release candidate 31
reload an entire web page 1290
Reload from `appletviewer`'s **Applet** menu 945, 946
reluctant quantifier **705**
remainder 53
remainder compound assignment operator, `%=` 131
`REMAINDER` constant of class `GridBagConstraints` **1037**
remainder operator, `%` **53**, 54, 149
remove duplicate `String` 856
`remove` method of class `ArrayList<T>` 284, **286**
`remove` method of interface `Iterator` **836**
`removeTableModel-Listener` method of interface `TableModel` **1194**
`removeTrayIcon` method of class `SystemTray` **1447**
`render` attribute of `f` ajax **1295**
rendering XHTML in a web browser **1239**
Reordering output with argument index 1411
`repaint` method of class `Component` **599**
`repaint` method of class `JComponent` **635**
repainted 952
repetition 106, 183
  counter controlled 114, 123, 126, 127
  definite **113**
  sentinel controlled 118, 119, 120, 121, 123
repetition statement **105**, **106**, **112**, 119, 776
  `do...while` 106, 162, 163, 183, 163, 183
  `for` 106, 157, 183
  `while` 106, **112**, 113, 116, 123, 152, 183, 184
repetition terminates 112
`replaceAll` method of class `Matcher` **707** of class `String` **705**
`replaceFirst` method of class `Matcher` **707** of class `String` **705**
Representational State Transfer (REST) **1300**, 1302
representing integers in hexadecimal format 1397
representing integers in octal format 1397
request method **1239**
`RequestContext` interface **1338**
  `put` method **1338**
`@RequestScoped` annoation (JSF) **1248**
  default 1248
`required` attribute of a JSF element **1259**
requirements **13**, 474
requirements document **470**, 474, 476
requirements gathering **474**
reservations system 297
reserved word **40**, 106, 1368
  `false` 108
  `null` 82, **84**, 134
  `true` 107
resizable array 1123
  implementation of a `List` 833
resolution 135, 632
resource leak 334, **450**
resource library (JSF) 1253
resource-release code 450
`resources` element of a JNLP document **959**
`resources` folder of a JSF app **1253**
responses to a survey 251, 253
REST (Representational State Transfer) **1300**
`restart` method of class `Timer` **979**, 994
RESTful web services **1302**
result 1179
result set concurrency **1199**
result set type **1198**
`ResultSet` interface **1192**, 1198, 1200
  `absolute` method **1200**
  `close` method 1193
  column name 1193
  column number 1193
  `CONCUR_READ_ONLY` constant **1199**
  `CONCUR_UPDATABLE` constant **1199**
  concurrency constant 1199
  `getInt` method **1193**
  `getObject` method **1193**, 1200
  `getRow` method **1200**
  `last` method **1200**
  `next` method **1193**
  `TYPE_FORWARD_ONLY` constant **1198**
  `TYPE_SCROLL_INSENSITIVE` constant **1198**
  `TYPE_SCROLL_SENSITIVE` constant **1199**
`ResultSetMetaData` interface **1192**, 1200
  `getColumnClassName` method **1200**
  `getColumnCount` method **1192**, **1200**
  `getColumnName` method **1200**
  `getColumnType` method **1193**
`ResultSetTableModel` enables a `JTable` to display the contents of a `ResultSet` 1194
resumption model of exception handling **445**
rethrow an exception **453**, 468
Rethrowing Exceptions exercise 468
`@return` `javadoc` tag **M_7**
Return key 945, 946
`return` keyword **81**, 199, 206, 1368
return message in the UML 504
`return` statement 767
return type 81
  in the UML 494, 499
  of a method **73**, 81
Returns: note **M_7**
reusability 883, 905
reusable software components 11, 208, 361
reuse 12, 48
`reverse` method of class `StringBuilder` **690**
`reverse` method of `Collections` 841, **847**
`reverseOrder` method of `Collections` **843**
RGB value 635, **636**, 641
RGB values **224**
right aligned 605
right brace, `}` **40**, 41, 49, 116, 123
right child **924**
`RIGHT` constant of class `FlowLayout` **608**
right justification 1396, 1405
right justify output **160**
right justifying integers 1405
right subtree 929, 937
rigid area of class `Box` **1031**
rise-and-shine algorithm 103
Ritchie, Dennis 16
`.rmi` file extension **985**, 988
robot 4
robotics simulator 998
robust 50
robust application 439
role in the UML 481
role name in the UML **481**
roll back a transaction **1227**
`rollback` method of interface `Connection` **1227**
rolling two dice 218
rollover `Icon` **571**
root directory **722**
root element (XML) **958**
root `html` element 1243
root node **924**
`rotate` method of class `Graphics2D` **664**
Rotating Images exercise 998
round a floating-point number for display purposes 125
round-robin scheduling **1050**
rounded rectangle 649, 661
rounded rectangle (for representing a state in a UML state diagram) **489**
rounding 1396
rounding a number 53, 118, 161, 201, 235
`RoundRectangle2D` class 632
`RoundRectangle2D.Double` class **657**, 661
row **1173**, 1177, 1178, 1179, 1180, 1184
`rowClasses` attribute of `h:dataTable` **1288**
`RowFilter` class **1206**

rows of a two-dimensional array 268
rows to be retrieved 1178
RowSet interface **1207**
RSS feed 1449
RSS Resource Center 1451
Ruby on Rails 17
Ruby programming language 17
Rule of Entity Integrity **1177**
Rule of Referential Integrity **1175**
rule of thumb (heuristic) 173
rules for forming structured programs 179
rules of operator precedence **54**, 773
run an applet in a web browser 951
run debugger command **1381**
run method of interface Runnable **1051**, 1157
Runnable interface 430, **1051**, 1158
  run method **1051**, 1157
runnable state 1048
running an application 23
running state 1049
running total 119
runtime error **22**
runtime logic error 50
RuntimeException class **448**

S

SaaS (Software as a Service) xxvi, 30
Safari 93
SalariedEmployee class that implements interface Payable method getPaymentAmount 427
SalariedEmployee concrete class extends abstract class Employee 407
Sales Commissions 295
Salesforce 26
Salesforce.com 1451
sandbox security model **955**
SansSerif Java font 643
saturation **641**
Saverin, Eduardo 28
savings account 159
SavingsAccount Class (exercise) 355
Scala 17
scalar **260**
scaling (random numbers) 211
scaling an image 973
scaling factor (random numbers) 211, 214
Scanner class 48, **49**
  hasNext method **167**
  next method **77**
  nextDouble method **91**
  nextLine method **76**
scanning images 8
scheduling threads 1050
scientific notation 1398
scope 156
scope of a declaration **220**
scope of a type parameter 886
scope of a variable **156**
screen 6, 9
Screen class (ATM case study) 480, 481, 493, 500, 501, 502, 503, 504, 506, 513
screen cursor 46
screen-manager program 397
Screen Saver exercise 668
Screen Saver for Random Number of Lines exercise 669
Screen Saver Using the Java2D API exercise 669
Screen Saver Using Timer exercise 668
Screen Saver with Shapes exercise 669
script (Unicode) **1457**
scroll 582, 586
scroll arrow **583**
scroll box **583**
SCROLL_TAB_LAYOUT constant of class JTabbedPane **1031**
scrollbar 586, 617
  of a JComboBox **583**
scrollbar policies **617**
Scrolling Image Marquee exercise 997
Scrolling Marquee Sign exercise 996
SDK (Software Development Kit) **30**
search algorithms
  binary search **804**
  linear search **800**
  recursive binary search 827
  recursive linear search 827
search engine 1240
search key **799**
searching 905
searching data **799**
second-degree polynomial 55, 56
Second Life 26
second refinement 127
second refinement in top-down, stepwise refinement 119
secondary storage 6
secondary storage devices **720**
secondary storage unit **9**
sector 652
security 21
security certificate 955
SecurityException class **731**
See Also: note M_5
@see javadoc tag M_5
seed value (random numbers) 211, **214**
SEI (service endpoint interface) **1305**, 1310
SELECT SQL keyword **1178**, 1179, 1180, 1181, 1182
selected text in a JTextArea 617
selecting an item from a menu 561
selecting data from a table 1174
Selecting Shapes exercise 670
selection 106, 182, 183
selection criteria **1179**
selection mode **586**
selection sort algorithm **810**, 813
selection statement **105**, **106**
  if 106, 107, 164, 183, 184
  if...else 106, **107**, 108, 123, 164, 183
  switch 106, **164**, 170, 183
self-documenting 50
self-documenting code 50
self-referential class **905**, 907
self-similar property **780**
_self target frame **1124**
Selvadurai, Naveen 29
semicolon (;) **42**, 49, 59
send a message to an object 12
send data to a server 1142
send message 85
send method of class DatagramSocket **1146**
sentence-style capitalization **553**
sentinel-controlled repetition **118**, 119, 120, 121, 123, 194, 305
sentinel value **118**, 119, 123
separator character **726**
separator line in a menu 1012, **1013**
sequence 106, 181, 183, **833**, **924**
sequence diagram in the UML **477**, **502**
sequence of messages in the UML **503**
sequence structure **105**
sequence-structure activity diagram 105
SequenceInputStream class 753
sequential-access file 720, 726, 1129
sequential execution **104**
sequential horizontal orientation in GroupLayout **1431**
sequential layout of GUI components 1431
Sequentially searching an array for an item 801
Serializable interface 430, **743**
serialized object **742**
serialized-form.html generated by javadoc M_11
Serif Java font 643
server **1119**
server farm 1263
server port number 1142
server response **1240**
server-side artifacts **1305**
server-side form handler **1239**
server waits for connections from clients 1128
server's Internet address 1130
server-side form handler **1239**
ServerSocket class **1128**, 1135, 1157
  accept method **1128**, 1135
service description for a web service **1309**
service endpoint interface (SEI) **1305**, 1310
service of a class 316
ServiceManager class **972**
  lookup method **972**
serviceName attribute of @WebService annotation **1305**
session 1262
session expire (JSF) 1263
session tracking **1262**
  in web services 1324
@SessionScoped annotation **1262**, **1263**, **1265**
set a value **83**
set debugger command **1383**
Set interface 831, **832**, **856**, 857, 859
set method
  of class BitSet N_11
  of interface ListIterator **839**
set method 83, 320
set of constants
  as an interface 420
Set of Integers (exercise) 356
Set of recursive calls for fibonacci(3) 773
SET SQL clause **1185**
set-theoretic intersection 356
set-theoretic union 356
set up event handling 564
setAlignment method of class FlowLayout **608**
setAutoCommit method of interface Connection **1227**
setBackground method of class Component 287, **586**, 641
setBounds method of class Component 604
setCharAt method of class StringBuilder **690**
setColor method of class Graphics **636**, 661
setColor method of Graphics **225**
setCommand method of JdbcRowSet interface **1209**
setConstraints method of class GridBagLayout **1037**

setDefaultCloseOperation method of class JFrame 137, 561, 1005
setDelay method of class Timer 995
setDisabledTextColor method of class JTextComponent 603
setEditable method of class JTextComponent 564
setErr method of class System 721
setFileSelectionMode method of class JFileChooser 757
setFixedCellHeight method of class JList 588
setFixedCellWidth method of class JList 588
setFont method of class Component 576
setFont method of class Graphics 643
setForeground method of class JComponent 1014
setHint method of class Manager 989
setHorizontalAlignment method of class JLabel 560
setHorizontalScrollBarPolicy method of class JScrollPane 618
setHorizontalTextPosition method of class JLabel 560
setIcon method of class JLabel 560
setIn method of class System 721
setInverted method of class JSlider 1002
setJMenuBar method of class JFrame 1006, 1013
setLayout method of class Container 559, 605, 611, 613, 1031
setLineWrap method of class JTextArea 617
setListData method of class JList 589
setLocation method of class Component 604, 1006
setLookAndFeel method of class UIManager 1021
setMajorTickSpacing method of class JSlider 1005
setMaximumRowCount method of class JComboBox 583
setMnemonic method of class AbstractButton 1012
setOpaque method of class JComponent 597, 600
setOut method of System 721
setPage method of class JEditorPane 1127
setPaint method of class Graphics2D 660
setPaintTicks method of class JSlider 1005
setPassword method of JdbcRowSet interface 1209
setProperty method of Properties 863
setRolloverIcon method of class AbstractButton 573
setRowFilter method of class JTable 1206
setRowSorter method of class JTable 1206
setSeed method of class Random 215
setSelected method of class AbstractButton 1013
setSelectionMode method of class JList 586
setSize method of class Component 604, 1006
setSize method of class JFrame 137, 561
setString method of interface PreparedStatement 1211, 1218
setStroke method of class Graphics2D 660
setText method of class JLabel 390, 560
setText method of class JTextComponent 617
Setting the PATH environment variable xxxv
setToolTipText method of class JComponent 559
setUrl method of JdbcRowSet interface 1209
setUsername method of JdbcRowSet interface 1209
setVerticalAlignment method of class JLabel 560
setVerticalScrollBarPolicy method of class JScrollPane 618
setVerticalTextPosition method of class JLabel 560
setVisible method of class Component 561, 611, 1006
setVisible method of class JFrame 137
setVisibleRowCount method of class JList 586
shadow a field 220
shallow copy 387, 388
shape 657
Shape class hierarchy 362, 393
Shape Hierarchy exercise 436
Shape object 660
shapes 944
shared buffer 1063
shell 41
shell prompt in UNIX 19
shell script 731
Shift 604
shift (random numbers) 211
shifting value 211
shifting value (random numbers) 214
"shipping" section of the computer 9
Shopping Cart Application 1275
shopping list 112
short-circuit evaluation 175
Short class 831
short primitive type 164, 1368, 1369
  promotions 208
shortcut element of a JNLP document 959
shortcut key 1007
Show Line Numbers 1247
show method of class JPopupMenu 1017
showDialog method of class JColorChooser 641
showDocument method of interface AppletContext 1120, 1124
showInputDialog method of class JOptionPane 94, 553
showMessageDialog method of class JOptionPane 94, 554
showOpenDialog method of class JFileChooser 757
showStatus method of class Applet 982
shuffle 254
  algorithm 845
shuffle method of class Collections 841, 845, 847
Shuffleboard exercise 997
shuffling
  Fisher-Yates 257
shutdown method of class ExecutorService 1054
sibling nodes 924
side effect 175
Sieve of Eratosthenes 302, 1104
Sieve of Eratosthenes, using a BitSet N_12
signal method of interface Condition 1087, 1091
signal value 118
signalAll method of interface Condition 1087
signature 224
signature of a method 223
signed right shift (>>) N_1, N_2, N_8, N_10
Simbad Robotics Simulator Project 998
simple condition 173
simple name 342
Simple Object Access Protocol (SOAP) 1300, 1303
SimpleGraph applet 944
simplest activity diagram 179, 181
Simpletron Machine Language (SML) 304, 905
Simpletron simulator 306, 309, 905, 998
simulate a middle-mouse-button click on a one- or two-button mouse 597
simulate a right-mouse-button click on a one-button mouse 597
simulation 210
  coin tossing 238
Simulation: Tortoise and the Hare 302, 669
simulator 304
sin method of class Math 201
@since javadoc tag M_8
Since: note M_8
sine 201
single entry point 179
single-entry/single-exit control statements 106, 179
single exit point 179
single inheritance 360
single-line (end-of-line) comment 42
single-precision floating-point number 88
single-quote character 673, 1180
single-selection list 584
single-selection statement 106, 107, 183
single static import 338
Single-Threaded Execution design pattern Q_3, Q_14
single-type-import declaration 344
SINGLE_INTERVAL_SELECTION constant of interface ListSelectionModel 586, 586, 588
SINGLE_SELECTION constant of interface ListSelectionModel 586
single-selection statement
  if 107
Singleton design pattern Q_3, Q_4
singly linked list 907
size method
  of class ArrayBlockingQueue 1071
  of class ArrayList<T> 286
  of class BitSet N_12
  of class PriorityQueue 855
  of interface List 836, 839
  of interface Map 863
size of a variable 52
size of the applet's display area 950
Skype 26
sleep interval 1049

sleep method of class Thread **1051**, 1064, 1065, 1066
sleeping thread **1049**
small circles in the UML **105**
small diamond symbol (for representing a decision in a UML activity diagram) 492
smallest integer in a group 965
smallest of several integers 193
smartphone 2, 3, 15
Smashforce 1451
SML 905
SMS Language 718
SMS Web Service 1364
snap-to ticks for JSlider **1001**
SOA (services oriented architecture) xxvi
SOAP (Simple Object Access Protocol) **1300**, 1302, 1303, 1310
  envelope **1302**
  message **1302**
social commerce 26, 28
social networking 26
socket **1119**
socket-based communication **1119**
Socket class **1128**, 1142, 1157, 1158, Q_15
  close method **1129**
  getInetAddress method **1135**
  getInputStream method **1129**, 1130
  getOutputStream method **1129**
SocketException class **1143**
SocketImpl class Q_15
software 2, **6**
Software as a Service (SaaS) **30**
Software Development Kit (SDK) **30**
software engineering 327
Software Engineering Observations overview xxviii
software life cycle **474**
software model 306
software reuse 12, **199**, 340, 360, 874, 1450
software simulation 304
solid circle (for representing an initial state in a UML diagram) in the UML **489**, 490
solid circle enclosed in an open circle (for representing the end of a UML activity diagram) 490
solid circle in the UML **105**
solid circle surrounded by a hollow circle in the UML **105**
solid diamonds (representing composition) in the UML **481**
Solves Towers of Hanoi problem with a recursive method 779
sort 281
sort algorithms
  bubble sort 826
  bucket sort 826
  insertion sort **814**
  merge sort **817**
  quicksort 827
  selection sort **810**
sort key **799**
sort method
  of class Arrays **281**, **805**
  of class Collections **842**
SortDemo applet 944
sorted array 907
sorted order 857, 859
SortedMap interface **859**
SortedSet interface **857**, 859
  first method **859**
  last method **859**
sorting 905
  descending order 842
  with a Comparator 843
sorting data **799**, 809
sorting techniques 944
sound 951, 968
sound card 985
sound engine **985**
sounds 992
source code **19**, 386
Source view in Netbeans 1433
SourceForge 14
SOUTH constant of class BorderLayout **592**, 608
SOUTH constant of class GridBagConstraints **1032**
SOUTHEAST constant of class GridBagConstraints **1032**
SOUTHWEST constant of class GridBagConstraints **1032**
space character 39
space flag 1408
spacing between components in GroupLayout 1432
Spam Scanner 717
SPAM Scanner Web Service 1364
speaker 985
speaking to a computer 8
special character 49, **673**
Special Section: Advanced String-Manipulation Exercises 714
Special Section: Building Your Own Compiler 905
Special Section: Building Your Own Computer 304
Special Section: Challenging String-Manipulation Projects 716
special symbol 7
specialization **360**
specialization in the UML **517**
specifics 397
speech recognition 998
speech synthesis 998
Spelling Checker project 716
sphere 232
spiral 669, 771
.spl file extension **988**
-splash command-line option to the java command **1443**
splash screen **1443**
SplashScreen class **1444**
split method of class String **699**, **705**
split the array in merge sort 817
spooler **921**
spooling 921
SpreadSheet applet 944
SQL 1172, 1174, 1177, 1178, 1184
  DELETE statement 1178, **1186**
  FROM clause 1178
  GROUP BY 1178
  IDENTITY keyword **1212**
  INNER JOIN clause 1178, **1183**
  INSERT statement 1178, **1184**
  LIKE clause 1180
  ON clause **1183**
  ORDER BY clause 1178, **1181**, 1182
  SELECT query **1178**, 1179, 1180, 1181, 1182
  SET clause **1185**
  UPDATE statement 1178
  VALUES clause **1184**
  WHERE clause **1179**
.sql 1188
SQL (Structured Query Language) 1211
SQL keyword 1177
SQL script 1188
SQL statement 1227
SQLException class **1192**, 1193, 1212
SQLFeatureNotSupported-Exception class 1199
sqrt method of class Math 200, 201, 207
square brackets, [] **242**
square root 201
Squares eliminated by placing a queen in the upper-left corner of a chessboard 795
stack **206**, 883, **905**, 917
  method call stack **207**
  program execution stack **207**
  stack overflow **207**
Stack class 855, 917
  isEmpty method **855**
  of package java.util **853**
  peek method **855**
  pop method **855**
  push method **854**
stack frame **207**
Stack generic class 884
  Stack< Double > 891
  Stack< Integer > 891
Stack generic class declaration 884
stack operation
  pop **917**
  push **917**
stack trace **441**
stack unwinding **454**
stacked building blocks 183
stacking control statements 184
stacking rule **181**
StackTraceElement class **457**
  getClassName method **457**
  getFileName method **457**
  getLineNumber method **457**
  getMethodName method **457**
stale value **1059**
standard error stream **444**, **453**, **1396**
standard error stream (System.err) 721, 751
standard input stream (System.in) **49**, 721
standard output stream **453**
standard output stream (System.out) **41**, 721, 751
standard reusable component **361**
standard time format 314
"warehouse" section of the computer 9
start method of class JApplet **949**, 952, 954
start method of class Timer **979**
start method of interface Player **990**
start tag **958**
starting angle **651**
startsWith method of class String **680**
starvation **1050**
state **477**
state button **574**
state dependent **1063**
State design pattern Q_3, Q_6, Q_7, Q_12
  context object Q_7
  State class Q_7
  state object Q_7
  State subclass Q_7
state diagram for the ATM object 489
state diagram in the UML **489**
state in the UML **477**, 490
state machine diagram in the UML **477**, **489**
state object Q_12
state of an object 484, 489

stateChanged method of interface ChangeListener **1005**
stateless protocol 1262
statement **42**, 73
Statement interface **1192**, 1193, 1211
  close method 1193
  executeQuery method **1192**
Statements 119
  break **168**, 172, 195
  continue **172**, 195, Q_2
  control statement **104**, 105, 106, 107
  control-statement nesting **106**
  control-statement stacking **106**
  do...while 106, **162**, 163, 183
  double selection **106**, 127
  empty **59**, 111
  empty statement 111
  enhanced for **258**
  for 106, **154**, 155, 157, 159, 160, 183
  if **56**, 106, 107, 164, 183, 184
  if...else 106, **107**, 108, 123, 164, 183
  labeled break **Q_1**, Q_1
  labeled continue **Q_2**
  looping **106**
  multiple selection **106**
  nested **126**
  nested if...else **109**, 110
  repetition **105**, **106**, **112**
  return 199, 206
  selection **105**, **106**
  single selection **106**
  switch 106, **164**, 170, 183
  switch multiple-selection statement 214
  try **253**
  while 106, 112, 113, 116, 123, 152, 183, 184
statements
  throw **314**
  try-with-resources **463**
static
  class member 334, 335
  class variable 335
  field (class variable) **334**
  import **338**
  import on demand **338**
  keyword 200, 1368
  method 74, **94**, 161
static binding **419**
static initializer block 1265
status bar 948
step debugger command **1385**
step up debugger command **1386**
Stone, Isaac "Biz" 28
stop debugger command **1381**
stop method of JApplet **949**, 952
stop method of class Timer **980**
stop method of interface AudioClip **985**
store method of Properties **865**
stored procedure **1226**
straight-line form **54**
Strategy design pattern Q_3, Q_6, Q_12
strategy object Q_12
stream **453**, 1396
stream header **1136**
stream of bytes **720**
stream socket **1119**, 1130, 1150
stream-based communications **1119**
streams **1119**
streams-based transmission 1142
strictfp keyword 1368
strictly self-similar fractal **780**
string **41**
  literal **41**
  of characters 41
String class 673
  charAt method **675**, **690**
  compareTo method 677, 679
  concat method **684**
  endsWith method **680**
  equals method 677, 679
  equalsIgnoreCase method 677, 679
  format method **94**, **314**, 1413
  getChars method **675**
  immutable **337**
  indexOf method **681**
  lastIndexOf method **681**
  length method **675**
  matches method **700**
  regionMatches method **677**
  replaceAll method **705**
  replaceFirst method **705**
  split method **699**, **705**
  startsWith method **680**
  substring method 683, 684
  toCharArray method **686**, 796
  toLowerCase **839**
  toLowerCase method **686**
  toUpperCase **839**
  toUpperCase method **685**
  trim method **686**
  valueOf method **686**
String class searching methods 681
string concatenation **204**, 337
string literal **674**
StringBuffer class **688**
StringBuilder class 673, **687**
  append method **691**
  capacity method **688**
  charAt method **690**
  constructors 688
  delete method **693**
  deleteCharAt method **693**
  ensureCapacity method **688**
  getChars method **690**
  insert method **693**
  length method **688**
  reverse method **690**
  setCharAt method **690**
StringIndexOutOfBounds-Exception class **684**, 690
StringReader class **753**
Strings in switch statements 171
StringWriter class **753**, 1317
Stroke object **660**, 661
strongly typed languages **134**
Stroustrup, Bjarne 17, 439
structural design patterns Q_2, **Q_5**, Q_8, Q_15
structure **1243**
structure of a system 488, 489
structured programming 6, **104**, 152, 173, 179
  summary 179
Structured Query Language (SQL) **1172**, 1174, 1177
Student Poll exercise 763
style attribute of h:panelGrid **1253**
styleClass attribute of a JSF element **1259**
styleClass attribute of h:dataTable **1288**
subclass **136**, **360**, 516, 517, Q_9
subdirectory 944
subject object Q_12
sublist **839**
subList method of List **839**
submenu **1007**
submit method of class ExecutorService **1109**
subprotocol for communication **1191**
subscript (index) **242**
substring method of class String 683, 684
subsystem Q_16
subtract method of class BigInteger **771**, 773
subtraction 9, 53
  operator, - 54
subtraction compound assignment operator, -= 131
suffix F for float literals **855**
suffix L for long literals **854**
sum the elements of an array 248
summarizing responses to a survey 251
Sun Audio file format (.au extension) **985**, 988
Sun Microsystems 1452, Q_2
super keyword 363, 385, 1368
  call superclass constructor 377
super.paintComponent(g) 136
superclass **136**, **360**, 516, 517, Q_10
  constructor 367
  constructor call syntax **377**
  default constructor 367
  direct **360**, 362
  indirect **360**, 362
  method overridden in a subclass 385
Supermarket Simulation 936
surrogates **1453**
suspend an applet's execution 952
swapping values 810, 814
sweep **288**, **651**
sweep counterclockwise 651
.swf file extension **988**
Swing Event Package 210
Swing GUI APIs 551
Swing GUI components **550**
Swing GUI components package 209
swing.properties file xxxvi, 552
SwingConstants interface 431, **560**, 1005
SwingSet3 demo (download.java.net/javadesktop/swingset3/SwingSet3.jnlp) 551
SwingUtilities class **1022**, **1135**
  invokeLater method **1135**
  updateComponentTreeUI method **1022**
SwingWorker class **1095**
  cancel method **1109**
  doInBackground method 1095, 1098
  done method 1095, 1098
  execute method 1095
  get method 1095
  isCancelled method **1104**
  process method 1096, 1105
  publish method 1095, 1105
  setProgress method 1096, 1105
switch logic 171
switch multiple-selection statement 106, **164**, 170, 183, 214, 1368
  activity diagram with break statements 170

switch (cont.)
case label 168
comparing Strings 171
controlling expression **168**
default case **168**, 171, 214
Sybase 1172
Sybase, Inc. 1452
synchronization **1054**, 1074
synchronization wrapper **866**
synchronize **1047**
synchronize access to a collection 833
synchronized
keyword 866, **1055**, 1368
method **1055**
statement **1055**
synchronized collection 833
synchronous call **502**
synchronous error **447**
synchronous request **1290**
SynchronousQueue class 1094
syntax error **39**, 43
system **476**
system behavior **476**
System class
arraycopy **281**, 283
currentTimeMillis method 797
exit method **450**, **731**
setErr method **721**
setIn method **721**
setOut **721**
system requirements **474**
system service 1128
system structure **476**
System.err (standard error stream) **444**, 721, 751, 1396
System.in (standard input stream) 721
System.out
print method **44**
printf method **46**
println method **41**, 44
System.out (standard output stream) **41**, 721, 751
SystemColor class **660**
SystemTray class **1447**
addTrayIcon method **1447**
getDefaultSystemTray method **1447**
removeTrayIcon method **1447**

T

tab 1411
tab character, \t 46
Tab key 41
tab stops 41, 46
table 268, 1173
table element 268
table of values 268
TableModel interface **1194**
addTableModelListener **1194**
TableModel interface (cont.)
getColumnClass method **1194**, 1200
getColumnCount method **1194**, 1200
getColumnName method **1194**, 1200
getRowCount method **1194**
getValueAt method **1194**
removeTableModelListener **1194**
TableModelEvent class 1206
TableRowSorter class **1206**
tablet 3
tablet computer 15
tabular format 245
tag (in an XHTML document) **949**
tag library (JSF) 1243
tagging interface 421, **743**
tail of a queue **905**, 921
tailSet method of class TreeSet **859**
take method of class BlockingQueue **1070**, 1071
tan method of class Math 201
tangent 201
target frame **1124**
_blank **1124**
_self **1124**
_top **1125**
Target-Heart-Rate Calculator exercise 101
Tax Plan Alternatives exercise 196
TCP (Transmission Control Protocol) **1119**
technical publications 31
Technorati APIs 1451
Telephone-Number Word Generator exercise 763
telephone system 1143
Template Method design pattern Q_3, Q_6, Q_13
temporary 124
terabyte 9
Terminal application (Max OS X) 19
terminal window **41**
terminate a loop 119
terminate an application 1012
terminate successfully 731
terminated state **1049**
termination housekeeping **334**, 387
termination model of exception handling **445**
termination phase 119
termination test 776
ternary operator **108**
test a web service 1308
testing a web service from another computer 1309
Testing the insertion sort class 816
Testing the merge sort class 821
Testing the selection sort class 812
Text analysis 714
text editor 42, 673
text field **94**
text file **721**
Text Flasher exercise 995
text that jumps 944
text-to-speech (TTS) 998
TexturePaint class **632**, 660, 661
The "FairTax" 196
The Free Site (www.thefreesite.com) 992
The Java™ Language Specification (java.sun.com/docs/books/jls/) 54
thick lines 657
this
keyword **317**, 317, 335, 1368
reference **317**
to call another constructor of the same class 322
thread **445**, 634, 951, Q_14
life cycle 1048, 1050
of execution **1046**
scheduling **1049**, 1066
state **1048**
synchronization 866, **1054**
Thread class
currentThread method 1056
interrupt method **1052**
sleep method **1051**
thread confinement **1095**
thread-life-cycle statechart diagram 1048, 1050
thread pool **1051**
thread priority **1050**
thread safe **1059**, 1095
thread scheduler **1050**
thread states
blocked **1049**, 1055
dead 1049
new **1048**
ready **1049**
runnable **1048**
running **1049**
terminated **1049**
timed waiting **1048**
waiting **1048**
three-button mouse 596
3-D Multithreaded Tic-Tac-Toe 1169
three-dimensional shape 944
three-dimensional view 944
three-dimensional rectangle 648
three-dimensional, high-resolution, color graphics 968
ThreeDimensionalShape class 393
throw an exception **253**, **440**, 444
throw an exception **314**, 323
throw keyword **453**, 1368
throw point **441**
throw statement **453**
Throwable class **447**, 456
getMessage method **456**
getStackTrace method **456**
hierarchy 448
printStackTrace method **456**
throws an exception **314**
throws clause **446**
@throws javadoc tag M_7
throws keyword 1368
thumb of class JSlider **1001**, 1005
thumb position of class JSlider 1005
Tic-Tac-Toe 1150
tick marks on a JSlider **1001**
TicTacToe 357
applet 944, 945
exercise 357
tier in a multitier application **1240**
tightly packed binary tree **930**
time formatting 1397
Time to Calculate Fibonacci Numbers exercise 797
timed waiting state 1048
Timer class 668, **979**, 980
getDelay method **995**
isRunning method **979**
restart method **979**
setDelay method **995**
start method **979**
stop method **980**
timeslice **1049**
timeslicing 1050
timing diagram in the UML P_2
title bar **551**, 557, **1005**
title bar of a window 554
title bar of internal window 1024
title element of a JNLP document **959**
title of a JSF document **1244**
titles table of books database 1174, 1176
toArray method of List **840**, 841
toBinaryString method of class Integer N_4
toCharArray method of class String **686**
toCharArray method of String 796
toggle buttons **571**
toJson method of class Gson 1322
token of a String **699**
tokenization 699
toLowerCase method of class Character **696**
toLowerCase method of class String **686**, **839**
tool tips **556**, 559, 561
top **118**, 855

TOP constant of class JTabbedPane **1031**
top-down, stepwise refinement **118**, 119, 121, 126, 12
top-level class **564**
top of a stack **905**
_top target frame **1125**
top tier **1241**
Tortoise and the Hare 302, 669
Tortoise and the Hare exercise 669
toString method
of class ArrayList 842, 897
of class Arrays 706, 801
of class BitSet N_12
of class Formatter 1413
of class Object 367, 388
total **114**, 119
Total Sales 297
toUpperCase method of class Character **696**
toUpperCase method of class String **685**, **839**
toURI method of class File **992**
toURL method of class URI **992**
Towers of Hanoi **777**
Towers of Hanoi for the case with four disks 778
track mouse events 590
tracking customers 1261
traditional comment **39**
traditional web application 1290
TRAILING alignment constant in GroupLayout 1432
trailing white-space characters 686
Transaction class (ATM case study) 516, 517, 518, 519, 521, 547
transaction file 762
transaction processing **1227**
transaction record 762
transfer of control **104**, 305, 307, 308
transient keyword **745**, 1368
transition arrow 108, 113
in the UML **105**
transition arrow in the UML 112
transition between states in the UML **489**, 492
transition in the UML **105**
translate method of class Graphics2D **664**
translation **10**
translator program **10**
transparency of a JComponent **597**
traverse a tree 929
traverse an array **270**
Tray icons **1447**
TrayIcon class **1448**
tree 856, 924, 945
Tree link in API **1371**
TreeMap class **859**
TreeSet class **856**, 857, 859
headSet method **858**
tailSet method **859**
trigger an event 555
trigonometric cosine 201
trigonometric sine 201
trigonometric tangent 201
trim method of class String **686**
trimToSize method of class ArrayList<T> 284
true **56**, 1368
true reserved word 107, **109**
truncate 53
truncate fractional part of a calculation **118**
truncated **729**
truth table **174**
truth tables
for operator ^ 176
for operator ! 177
for operator && 174
for operator || 175
try block **253**, 444, 454
terminates 445
try keyword **444**, 1368
try statement **253**, **446**
try-with-resources statement **463**
Turtle Graphics 298, 669
Turtle Graphics exercise 669
24-hour clock format 312
Twitter 3, 17, 26, 28
tweet 28
two-dimensional graphics demo 946
two-dimensional array representation of a maze 796
268, 269
two-dimensional array with three rows and four columns 268
two-dimensional data structure 924
two-dimensional graphics 657
two-dimensional shapes 632
two largest values 147
Two-Phase Termination design pattern Q_3, Q_14
two's complement 1426
TwoDimensionalShape class 393
twos position 1421
type **49**
type argument **886**
type casting **124**
type-import-on-demand declaration **344**
type inference with the <> notation (Java SE 7) 836
type of a variable **52**
type parameter **878**, 884, 891
scope 886
section **878**, 884
type variable **878**
type-wrapper class 694, **831**, 880
implements Comparable 880
TYPE_FORWARD_ONLY constant **1198**
TYPE_INT_RGB constant of class BufferedImage **661**
TYPE_SCROLL_INSENSITIVE constant **1198**
TYPE_SCROLL_SENSITIVE constant **1199**
Types class **1193**
typesetting system 673
typing in a text field 561
Typing Tutor: Tuning a Crucial Skill in the Computer Age 629

U

U+yyyy (Unicode notational convention) **1454**
UDP (User Datagram Protocol) **1119**, 1143
ui:repeat element **1269**
UIManager class **1021**
getInstalledLookAndFeels method **1021**
LookAndFeelInfo nested class **1021**
setLookAndFeel method **1021**
UIManager.LookAndFeelInfo class
getClassName method **1021**
UML (Unified Modeling Language) 13, 470, 476, 480, 487, 488, 516
activity diagram **105**, 108, 112, 157, 163
aggregation **482**
arrow 105
association **480**
class diagram **75**
compartment in a class diagram 75
diagram 476
diamond 107
dotted line **106**
elided diagram **480**
final state **106**
frame 505
guard condition **107**
guillemets (« and ») **88**
hollow diamond representing aggregation **482**
many-to-one relationship **483**
merge symbol **112**
multiplicity **480**
note **106**
one-to-many relationship **483**
one-to-one relationship **483**
UML (cont.)
Resource Center (www.deitel.com/UML/) 477
role name **481**
solid circle **105**
solid circle surrounded by a hollow circle 105
solid diamond representing composition **481**
Specification (www.omg.org/technology/documents/formal/uml.htm) 482
UML (www.uml.org) 106
UML Activity Diagram
small diamond symbol (for representing a decision) in the UML 492
solid circle (for representing an initial state) in the UML 490
solid circle enclosed in an open circle (for representing the end of an activity) in the UML 490
UML Class Diagram **480**
attribute compartment 487
operation compartment 493
UML Sequence Diagram
activation **504**
arrowhead 504
lifeline **504**
UML State Diagram
rounded rectangle (for representing a state) in the UML **489**
solid circle (for representing an initial state) in the UML **489**
UML Use Case Diagram
actor **475**
use case 476
unambiguous (Unicode design basis) **1453**
unary operator **125**, 176
cast **124**
unboxing 884, 889
unboxing conversion **832**
uncaught exception **445**
unchecked exceptions **448**
uncovering a component 635
underlying data structure 855
underscore (_) SQL wildcard character 1179, 1180
uneditable JTextArea 615
uneditable text or icons 555
Unicode character set **7**, 68, 134, 171, **673**, 679, 695, 1369
Unicode Consortium **1452**
Unicode Standard **1452**
Unicode Standard design basis 1453

Unicode value of the character typed 604
Unified Modeling Language (UML) 13, 470, 476, 480, 487, 488, 516
uniform (Unicode design principle) **1453**
Uniform Resource Identifier (URI) **722**, **1120**
Uniform Resource Locator (URL) **722**, 1120, 1237
union of two sets 356
universal (Unicode design principle) **1453**
universal-time format 312, 313, 314
UNIX 19, 41, 167, 731, 943
`UnknownHostException` class **1130**
`unlock` method of interface `Lock` **1086**, 1091
`unmarshal` method of class `JAXB` **1320**
unmodifiable collection 833
unmodifiable wrapper **866**
unnecessary parentheses 56
unsigned right shift (>>>) **N_1**, N_2, N_8, N_11
unspecified number of arguments 278
`UnsupportedOperationException` class **839**
`unwatch` debugger command **1390**
unwinding the method-call stack 454
`UPDATE` SQL statement 1178, **1185**
`updateComponentTreeUI` method of class `SwingUtilities` **1022**
upper bound 880
  of a wildcard 897
upper bound of a type parameter **881**, 882
upper-left corner *(0, 0)* coordinates of an applet 948
upper-left corner of a GUI component 134, 632
upper-left *x*-coordinate 636
upper-left *y*-coordinate 636
uppercase letter 40, 49
URI (Uniform Resource Identifier) **722**, **1120**
`URI` class
  `toURL` method **992**
`URL` Q_16
URL (Uniform Resource Locator) **722**, **1120**, 1121, 1237
`URL` class 985
  `openStream` method **1324**
`URLStreamHandler` class Q_16
Use binary search to locate an item in an array 807
use case diagram in the UML **475**, **476**
use case in the UML **475**
use case modeling **475**
User Datagram Protocol (UDP) **1119**, 1143
user interface 1241, Q_18
UTF-16 **1453**
UTF-32 **1453**
UTF-8 **1453**
Utilities Package 209
Utility class that displays bit representation of an integer N_7
utility method **170**

V

`v` option of the `jar` command **957**
va **732**
vacated bits N_11
valid identifier 49
`validate` method of class `Container` **613**
validation **1254**
`validatorMessage` attribute of a JSF element **1259**
Validators (JSF)
  `f:validateBean` **1254**
  `f:validateDoubleRange` **1254**
  `f:validateLength` **1254**
  `f:validateLongRange` **1254**
  `f:validateRegex` **1254**
  `f:validateRequired` **1254**
validity checking 327
`value` attribute of `h:dataTable` **1287**
`value` attribute of `h:inputText` 1259
`value` attribute of `h:outputLink` **1254**
`value` attribute of `ui:repeat` **1269**
value of a param 1121
value of a variable **52**
value of an attribute **1243**
value to the nearest integer 235
`valueChanged` method of interface `ListSelectionListener` **586**
`valueOf` method of class `String` **686**
`values` method of an `enum` **332**
`VALUES` SQL clause **1184**
van Rossum, Guido 17
`var` attribute of `h:dataTable` **1288**
`var` attribute of `ui:repeat` **1269**
variable **47**, **49**, 49
  name **49**, **52**
  reference type **84**
  size **52**
  type **52**
  value **52**
variable declaration statement **49**
variable is not modifiable 339
variable-length argument list **278**
variable scope 156
`Vector` class 290, **833**
`vendor` element of a JNLP document **959**
verb phrase in requirements document 493
`@version javadoc` tag **M_8**
Version note **M_8**
`VERTICAL` constant of class `GridBagConstraints` **1032**
vertical coordinate **135**, **632**
vertical gap space **611**
vertical scrolling 617
vertical strut **1030**
`VERTICAL_SCROLLBAR_ALWAYS` constant of class `JScrollPane` **618**
`VERTICAL_SCROLLBAR_AS_NEEDED` constant of class `JScrollPane` **618**
`VERTICAL_SCROLLBAR_NEVER` constant of class `JScrollPane` **618**
`vi` 19
video 968, 992
video game 211
video sharing 26
View 551
view **839**
view (in MVC) 1250, Q_17
view a shape from different angles 944
virtual directory **1238**
virtual key code **603**
virtual machine (VM) **20**
virtual world 26
visibility in the UML **511**
visibility marker in the UML **511**
Visitor design pattern Q_6
Visual Basic programming language 17
Visual C# programming language **17**
Visual C++ programming language 17
visual feedback 574
Visualizing Recursion exercise 794
Vlissides, John Q_2
`void` keyword **41**, 73, 1368
VoIP (Voice over IP 29
volatile information 9
`volatile` keyword 1368
volume of a sphere 232, 234

W

WADL (Web Application Description Language) **1318**
wait for a new connection 1135
`wait` method of class `Object` 388, **1073**
waiting line 832, 855
waiting state 1048
waiting thread 1076
`watch` debugger command **1388**
waterfall model **475**
`.wav` file extension **985**
web 1120
Web 2.0 **26**, 1449
web app development 1236
web application
  Ajax 1291
  traditional 1290
Web Application Description Language (WADL) **1318**
web application framework **1236**
Web Application project **1303**
web browser 93, 942, 1124
  execute an applet 947, 951
Web Form 1263
web page 93
web server 1128, **1237**
web service **1300**, 1449
  implemented as a class 1302, 1358
  publishing a web service 1307
Web Service Description Language (WSDL) **1309**
web service host **1302**
web service reference 1311
web services 27
  adding a web service reference to an application **1310**
  client-side artifacts **1310**
  consuming a web service **1302**, **1358**
  deploying a web service 1307
  `@GET` annotation **1317**
  GlassFish application server's `Tester` web page 1308
  JAX-RS **1300**
  JAX-WS **1300**
  `name` attribute of `@WebService` annotation **1305**
  `@Path` annotation **1316**
  `@PathParam` annotation **1317**
  POJO (Plain Old Java Object) **1244**, 1305
  processing user-defined types 1346
  `@Produces` annotation **1317**
  proxy class **1305**, 1310

web services (cont.)
publishing a web service **1302**, **1358**
RequestContext interface **1338**
REST **1300**
server-side artifacts **1305**
serviceName attribute of @WebService annotation **1305**
session tracking **1327**
SOAP 1310
test a web service 1308
testing a web service from another computer 1309
web service host **1302**
web service reference 1311
@WebMethod annotation **1305**
@WebParam annotation **1306**
@WebService annotation **1305**
@WebMethod annotation **1305**
operationName attribute **1305**
@WebParam annotation **1306**
name attribute **1306**
@WebService annotation **1305**
name attribute **1305**
serviceName attribute **1305**
weightx field of class GridBagConstraints **1033**
weighty field of class GridBagConstraints **1033**
WEST constant of class BorderLayout **592**, 608
WEST constant of class GridBagConstraints **1032**
WHERE SQL clause 1178, **1179**, 1180, 1182, 1185, 1186
while repetition statement 106, **112**, 113, 116, 123, 152, 183, 184, 1368
activity diagram in the UML 113
white space **39**, 41, 60
white-space character 686, 699, 700
whole/part relationship 481
widgets 550
width 647
width attribute (CSS) **1253**
width attribute of the applet-desc element 959
width of a rectangle in pixels 637
width of an applet in pixels **950**
Wikipedia 17, 26
wildcard 897
generic type parameter **895**
type argument **897**
upper bound 897
Williams, Evan 28
window 93, 134, 135, 137, **1005**
Window class 1005
addWindowListener method **1006**
dispose method **1005**
pack method **1025**
window event **1006**
window event-handling methods 594
window events **1006**
window gadgets 550
windowActivated method of interface WindowListener **1006**
WindowAdapter class 594, **1206**
windowClosed method of interface WindowListener **1006**, **1206**
windowClosing method of interface WindowListener **1006**
WindowConstants interface **1005**
DISPOSE_ON_CLOSE constant 1005
DO_NOTHING_ON_CLOSE constant 1005
HIDE_ON_CLOSE constant 1005
windowDeactivated method of interface WindowListener **1006**
windowDeiconified method of interface WindowListener **1006**
windowIconified method of interface WindowListener **1006**
windowing system **556**
WindowListener interface 594, **1006**, **1206**
windowActivated method **1006**
windowClosed method **1006**, **1206**
windowClosing method **1006**
windowDeactivated method **1006**
windowDeiconified method **1006**
windowIconified method **1006**
windowOpened method **1006**
windowOpened method of interface WindowListener **1006**
Windows 14, 19, 167, 731, 943
Windows look-and-feel 1001
Windows operating system **14**
Windows Performance Package 988
Windows Wave file format (.wav extension) **985**
WireFrame applet 944
Wirth, Niklaus 16
Withdrawal class (ATM case study) 480, 481, 482, 485, 491, 492, 493, 501, 502, 504, 505, 513, 514, 516, 517, 518, 521
word character **700**
word processor 673, 681
workflow **105**
workflow of an object in the UML **490**
World Community Grid 4
World Population Growth Calculator exercise 70
World Population Growth exercise 150
World Wide Web (WWW) browser 93
worst-case run time for an algorithm 802
wrap stream types 1129, 1130
wrapper methods of the Collections class **833**
wrapper object (collections) **866**
wrapping stream objects **742**, 748
wrapping text in a JTextArea 617
writeable 723
writeBoolean method of interface DataOutput 752
writeByte method of interface DataOutput 752
writeBytes method of interface DataOutput 752
writeChar method of interface DataOutput 752
writeChars method of interface DataOutput 752
writeDouble method of interface DataOutput 752
writeFloat method of interface DataOutput 752
writeInt method of interface DataOutput 752
writeLong method of interface DataOutput 752
writeObject method of class ObjectOutputStream 748
of interface ObjectOutput **743**
Writer class **753**, 753
writeShort method of interface DataOutput 752
writeUTF method of interface DataOutput 752
Writing the Word Equivalent of a Check Amount 715
WSDL (Web Service Description Language) **1309**
www 29

X

x-coordinate **135**, **632**, 656, 948
X_AXIS constant of class Box **1031**
x-axis **135**, 632
XHTML (eXtensible HyperText Markup Language) 1236, 1237, 1243, **1243**
applet element **950**
body element **950**
document 949
page 1237
tag **949**
XHTML 1.0
Strict Recommendation 1243
Transitional Recommendation 1243
XML (eXtensible Markup Language) **958**, 1236, 1309
declaration 1243
element **958**
empty element **1253**
end tag **958**
root element **958**
start tag **958**
vocabulary 958
XMLHttpRequest object **1290**
xmlns attributes **1243**
xor method of class BitSet N_11

Y

y-coordinate **135**, **632**, 656
Y_AXIS constant of class Box **1031**
Yahoo! 3
y-axis **135**, **632**
YouTube 26, 29
Yukihiro 17

Z

zero-based counting 155
ZERO constant of class BigInteger **773**
0 (zero) flag **1407**, 1409
zero-based counting **245**
zeroth element **242**
zooming 996
Zuckerberg, Mark 28
Zynga 5

Continued from Back Cover

> *The [early] introduction of the class concept is clearly presented. A comprehensive overview of control structures and the pitfalls that befall new programmers. I applaud the authors for their topical research and illustrative examples. The [arrays] exercises are sophisticated and interesting. The clearest explanation of pass-by-value and pass-by-reference that I've encountered. A logical progression of inheritance and the rationale for properly implementing encapsulation in a system involving an inheritance hierarchy. The polymorphism and exception handling discussions are the best I've seen. An excellent strings chapter. I like the [recursion] discussions of the 'Lo Fractal' and backtracking (which is useful in computer vision applications). A good segue into a data structures course.*"—**Ric Heishman, George Mason University**

> *Practical top-down, solution approach to teaching programming basics, covering pseudocode, algorithm development and activity diagrams. Of immense value to practitioners and students of the object-oriented approach. Demystifies inheritance and polymorphism, and illustrates their use in getting elegant, simple and maintainable code. The [optional OO design] case study presents the object-oriented approach in a simple manner, from requirements to Java code.*"
> —**Vinod Varma, Astro Infotech Private Limited**

> *Easy-to-follow examples provide great teaching opportunities! I like the [optional] graphics track early in the book—the exercises will be fun for the students. OO design techniques are incorporated throughout. The concept of inheritance is built through the examples and very understandable. Great examples of polymorphism and interfaces. Great comparison of recursion and iteration. The [Searching and Sorting] chapter is just right. A simplified explanation of Big O—the best I've read! I appreciate the coverage of GUI threading issues. Great approach to Java web technologies.*"
> —**Sue McFarland Metzger, Villanova University**

> *The Making a Difference exercises are inspired—they have a real contemporary feeling, both in their topics and in the way they encourage the student to gather data from the Internet and bring it back to the question at hand.*"
> —**Vince O'Brien, Pearson Education (our publisher)**

> *Most major concepts are illustrated by complete, annotated programs. Abundant exercises hone your understanding of the material. JDBC is explained well.*"—**Shyamal Mitra, University of Texas at Austin**

> *The best introductory textbook that I've encountered. I wish I had this book when I was learning how to program! Good introduction to UML and the software engineering process.*"—**Lance Andersen, Oracle**

> *You'll be well on your way to becoming a great Java programmer with this book. The Polymorphism and Generic Collections chapters are excellent for a new student.*"—**Peter Pilgrim, Java Champion, Consultant**

> *A good objects-early introduction to Java. Exceptionally well-written chapter on recursion. Graphics exercises are great. Excellent descriptions of the search and sort algorithms and a gentle introduction to Big-O notation. The examples give the code for the algorithms, and output that creates a picture of how the algorithms work.*"
> —**Diana Franklin, University of California, Santa Barbara**

> *Suitable for new programmers, intermediate-level programmers who want to hone their skills, and expert programmers who need a well-organized reference. Glad to see the mention of Nimbus. Swing components, event handling and layouts are well explained.*"—**Manjeet Rege, Rochester Institute of Technology**

> *Beautiful collections of exercises—a nice illustration of how to use Java libraries to generate impressive and stimulating graphics.*"—**Amr Sabry, Indiana University**

> *Excellent coverage of exception handling. A complete introduction to Java networking. Great coverage of multithreading.*"
> —**José Antonio González Seco, Parliament of Andalusia**

> *Good explanation of static vs. non-static methods and variables. Best introduction to Java 2D I've seen! The collections framework is well explained.*"—**Manfred Riem, Java Champion, Consultant, Robert Half**

> *The [optional] OOD ATM case study puts many concepts from previous chapters together in a plan for a large program, showing the object-oriented design process. The discussion on inheritance and polymorphism is especially good as [the authors] integrate these into the design.*"—**Susan Rodger, Duke University**

> *The transition from design to implementation is explained powerfully—the reader can easily understand the design issues and how to implement them in Java.*"—**S. Sivakumar, Astro Infotech Private Limited**